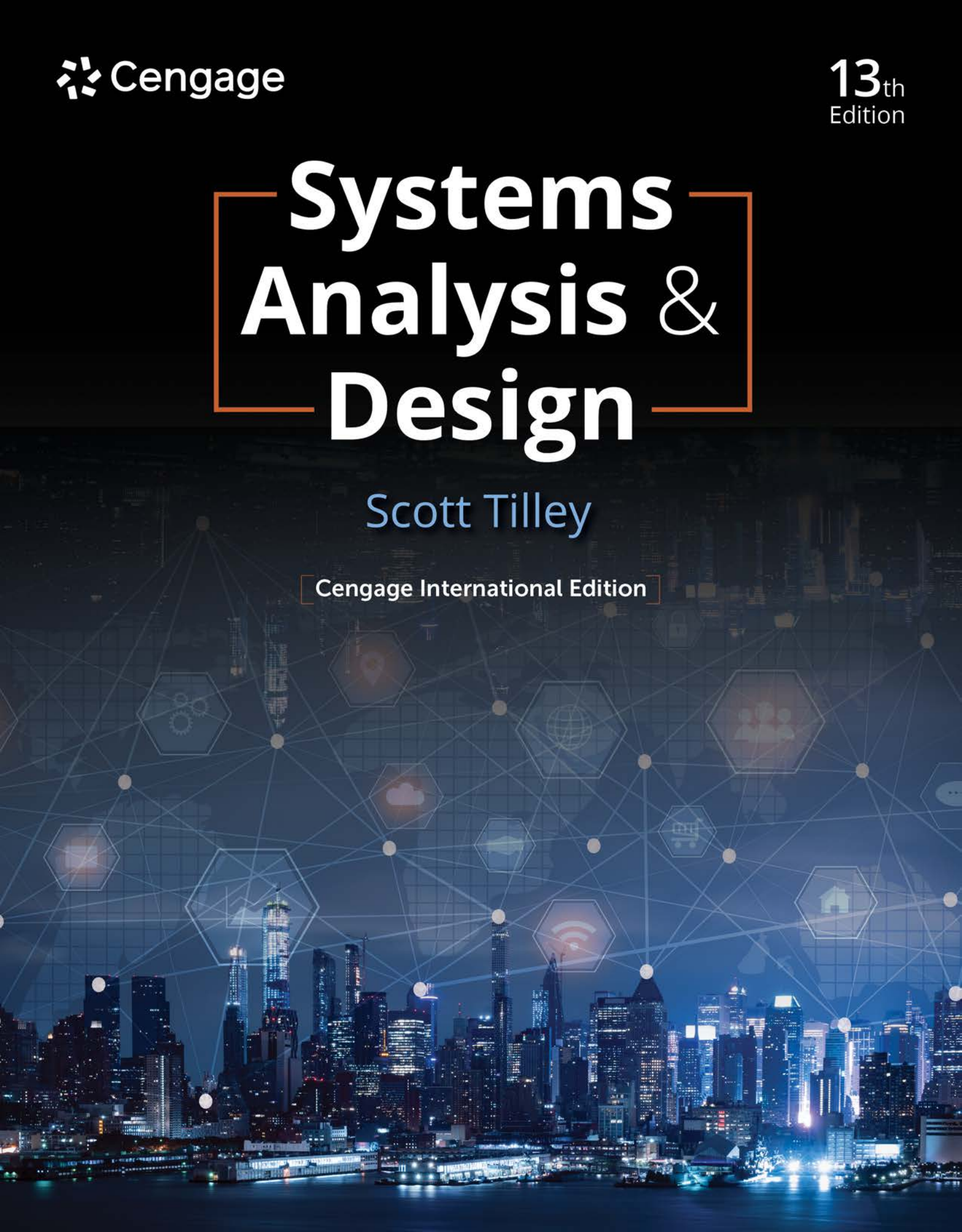
Cengage

13th Edition

Systems Analysis & Design

Scott Tilley

Cengage International Edition



Systems Analysis & Design

Thirteenth Edition

Scott Tilley, PhD

Cengage

Australia • Brazil • Canada • Mexico • Singapore • United Kingdom • United States

This is an electronic version of the print textbook. Due to electronic rights restrictions, some third party content may be suppressed. Editorial review has deemed that any suppressed content does not materially affect the overall learning experience. The publisher reserves the right to remove content from this title at any time if subsequent rights restrictions require it. For valuable information on pricing, previous editions, changes to current editions, and alternate formats, please visit www.cengage.com/highered to search by ISBN#, author, title, or keyword for materials in your areas of interest.

Important Notice: Media content referenced within the product description or the product text may not be available in the eBook version.

Cengage

Systems Analysis & Design,
Thirteenth Edition
Cengage International Edition
Scott Tilley

SVP, Product Management: Cheryl Costantini VP,

Product Management: Mark Santee Portfolio

Product Director: Rita Lombard Portfolio Product

Manager: Natalie Onderdonk Product Assistant:

Anh Nguyen

Learning Designer: Carolyn Mako

Content Manager: Marlena Sullivan

Digital Project Manager: Jim Vaughey

Developmental Editor: Dan Seiter

Senior Director, Product and Platform Marketing: Charlotte McLaren

Senior Manager, Product Marketing: Neena Bali

Senior Product Marketing Managers: Mackenzie Paine and Cybèle Beckham

Content Acquisition Analysts: Callum Panno and Nichole Nalenz

Production Service: Straive

Art Director: Erin Griffin

Cover Image Source: MR.Cole_Photographer/ Moment/Getty Images

Copyright © 2025 Cengage Learning, Inc. ALL RIGHTS RESERVED.

WCN: 02-300-527

No part of this work covered by the copyright herein may be reproduced or distributed in any form or by any means, except as permitted by U.S. copyright law, without the prior written permission of the copyright owner.

Unless otherwise noted, all content is Copyright © Cengage Learning, Inc.

The names of all products mentioned herein are used for identification purposes only and may be trademarks or registered trademarks of their respective owners. Cengage Learning disclaims any affiliation, association, connection with, sponsorship, or endorsement by such owners.

Previous editions: © 2020, © 2017, © 2014

For product information and technology assistance, contact us at **Cengage Customer & Sales Support, 1-800-354-9706 or support.cengage.com**.

For permission to use material from this text or product, submit all requests online at **www.copyright.com**.

Cengage International Edition
ISBN: 979-8-144-0591-9

Cengage
5191 Natorp Boulevard
Mason, OH 45040
USA

Cengage is a leading provider of customized learning solutions. Our employees reside in nearly 40 different countries and serve digital learners in 165 countries around the world. Find your local representative at **www.cengage.com**.

To learn more about Cengage platforms and services, register or access your online learning solution, or purchase materials for your course, visit **www.cengage.com**.

Notice to the Reader
Publisher does not warrant or guarantee any of the products described herein or perform any independent analysis in connection with any of the product information contained herein. Publisher does not assume, and expressly disclaims, any obligation to obtain and include information other than that provided to it by the manufacturer. The reader is expressly warned to consider and adopt all safety precautions that might be indicated by the activities described herein and to avoid all potential hazards. By following the instructions contained herein, the reader willingly assumes all risks in connection with such instructions. The publisher makes no representations or warranties of any kind, including but not limited to, the warranties of fitness for particular purpose or merchantability, nor are any such representations implied with respect to the material set forth herein, and the publisher takes no responsibility with respect to such material. The publisher shall not be liable for any special, consequential, or exemplary damages resulting, in whole or part, from the readers' use of, or reliance upon, this material.

Print Number: 01 Print Year: 2024

Dedication

To my teachers—past and present, Mr. Kuiper, Mr. Louvaris, Professor Grogono, Professor Müller, and Dr. Smith: Each of you has made an indelible mark on my academic journey, contributing in your own unique way. Collectively, you have shaped me into the educator I am today.

Brief Contents

Phase 1: Planning 001

Chapter 1	Introduction to Systems Analysis & Design	002
Chapter 2	Analyzing the Business Case	036
Chapter 3	Managing Systems Projects	063

Phase 2: Analysis 107

Chapter 4	Requirements Engineering	108
Chapter 5	Data & Process Modeling	147
Chapter 6	Object Modeling	178

Phase 3: Design 197

Chapter 7	User Interface Design	198
Chapter 8	Data Design	238
Chapter 9	System Architecture	281

Phase 4: Implementation 313

Chapter 10	Development Strategies	314
Chapter 11	Managing Systems Implementation	354

Phase 5: Operations 407

Chapter 12	Maintenance & Support	408
Chapter 13	Managing Systems Security	463

Appendix A: Communications **516**
Appendix B: Finance **542**
Glossary **568**
Index **595**

Contents

Phase 1: Planning

Chapter 1
Introduction to Systems Analysis & Design

Learning Objectives 2
1.1 Information Technology 3
1.1.1 The Changing Nature of Information Technology 3
1.1.2 Systems Analysis and Design 3
1.1.3 What Does a Systems Analyst Do? 4
1.2 Information Systems 4
1.2.1 Hardware 5
1.2.2 Software 5
1.2.3 Data 6
1.2.4 Processes 7
1.2.5 People 7
Case in Point 1.1: Data Breaches 8
1.3 Internet Business Strategies 8
1.3.1 B2C (Business-to-Consumer) 8
1.3.2 B2B (Business-to-Business) 8
1.3.3 C2C (Consumer-to-Consumer) 9
1.4 Modeling Business Operations 9
1.5 Business Information Systems 11
1.5.1 Enterprise Computing 12
1.5.2 Transaction Processing 12
1.5.3 Business Support 12
1.5.4 Knowledge Management 13
1.5.5 User Productivity 14
1.5.6 Digital Assistants 15
1.5.7 Systems Integration 15
Case in Point 1.2: Autonomous Vehicles 16
1.6 Organizational Information Models 16
1.6.1 Functions and Organizational Levels 16
1.6.2 Top Managers 16
1.6.3 Middle Managers and Knowledge Workers 17
1.6.4 Supervisors and Team Leaders 17
1.6.5 Operational Employees 18
1.7 Systems Development 18
1.7.1 Structured Analysis 18
1.7.2 Object-Oriented Analysis 21
1.7.3 Agile Methods 22
1.7.4 Prototyping 24
1.7.5 Tools 24
1.8 The Information Technology Department 25
1.8.1 Application Development 26
Case in Point 1.3: Global Hotels and Momma's Motels 26
1.8.2 Systems Support and Security 26
1.8.3 User Support 27
1.8.4 Database Administration 27
1.8.5 Network Administration 27
1.8.6 Web Support 27
1.8.7 Quality Assurance (QA) 27
1.9 The Systems Analyst 27
1.9.1 Role 27
1.9.2 Knowledge, Skills, and Education 28
1.9.3 Certification 29
1.9.4 Career Opportunities 30
1.9.5 Trends in Information Technology 31
Summary 32
Key Terms 33
Review Questions 34
Discussion Topics 34
Hands-On Projects 35
Ethical Issues 35

Chapter 2
Analyzing the Business Case

Learning Objectives 36
2.1 Strategic Planning 37
2.1.1 Strategic Planning Overview 37
Case in Point 2.1: Pets for Rent 37
2.1.2 SWOT Analysis 37
2.1.3 The Role of the IT Department 38
2.2 Strategic Planning Tools 39
2.3 The Business Case 40
2.4 Systems Requests 40
2.5 Factors Affecting Systems Projects 42
2.5.1 Internal Factors 42
2.5.2 External Factors 43
2.6 Processing Systems Requests 45
2.6.1 Systems Request Forms 45
2.6.2 Systems Request Tools 46
2.6.3 Systems Review Committee 46

Case in Point 2.2: Attaway Airlines, Part One 47
2.7 Assessing Request Feasibility 47
2.7.1 Feasibility Studies 48
2.7.2 Operational Feasibility 48
2.7.3 Economic Feasibility 49
2.7.4 Technical Feasibility 50
2.7.5 Schedule Feasibility 50
2.8 Setting Priorities 50
2.8.1 Dynamic Priorities 51
2.8.2 Factors That Affect Priority 51
2.8.3 Discretionary and Nondiscretionary Projects 51
Case in Point 2.3: Attaway Airlines, Part Two 52
2.9 The Preliminary Investigation 52
2.9.1 Planning the Preliminary Investigation 52
2.9.2 Performing the Preliminary Investigation 52
2.9.3 Summarizing the Preliminary Investigation 59
Summary 59
Key Terms 60
Review Questions 61
Discussion Topics 61
Hands-On Projects 62
Ethical Issues 62

Chapter 3
Managing Systems Projects

Learning Objectives 63
3.1 Project Management 64
3.1.1 What Shapes a Project? 64
3.1.2 What Is a Project Triangle? 64
3.1.3 What Does a Project Manager Do? 65
3.2 Task Management 65
3.2.1 Work Breakdown Structures 65
Case in Point 3.1: Sunrise Software 69
3.2.2 Task Patterns 71
Case in Point 3.2: Parallel Services 74
3.2.3 The Critical Path 74
3.3 People Management 76
3.3.1 Understanding the Human Element 76
3.3.2 Team Dynamics and Collaboration 76
3.3.3 Nurturing Growth 77
3.3.4 Dealing with Change 77
3.3.5 The Role of Trust 77
3.4 Communications Management 78
3.4.1 Miscommunication of Requirements 78
3.4.2 Lack of Regular Project Updates 78
3.4.3 Ineffective Change Communication 79
3.4.4 Cultural and Language Barriers 80
3.4.5 Dealing with Problems 80
3.5 Technology Management 80
3.5.1 Technology Selection 81
3.5.2 Technology Implementation 81
3.5.3 Technology Review 81
3.6 Financial Management 82
3.6.1 Budgets and Cost Estimation 82
3.6.2 Skills and Technology Investment 82
3.6.3 Return on Investment Analysis 83
3.7 Risk Management 83
3.7.1 The Importance of Risk Management 83
3.7.2 The Origins of Risks 84
3.7.3 Risk Management Tasks 85
3.7.4 The Risk Management Plan 87
3.7.5 Risk Management Issues 88
3.8 Project Management Software 88
3.8.1 Areas of Support 89
3.8.2 Examples of Project Management Software 91
3.8.3 Using Project Management Software 96
3.9 Managing for Success 98
3.9.1 Project Monitoring and Control 98
3.9.2 Issue Resolution 98
Case in Point 3.3: Just-in-Time Software 99
3.9.3 Adopting an Agile Mindset 102
Summary 103
Key Terms 104
Review Questions 104
Discussion Topics 104
Hands-On Projects 105
Ethical Issues 106

Phase 2 : Analysis

Chapter 4
Requirements Engineering

Learning Objectives 108
4.1 System Requirements 109
4.1.1 Types of Requirements 109
4.1.2 Requirements Challenges 110
4.1.3 Additional Considerations 110

4.2 Team-Based Techniques 112
4.2.1 Joint Application Development 112
Case in Point 4.1: North Hills College 115
4.2.2 Rapid Application Development 115
4.2.3 Agile Methods 118
4.3 Requirements Elicitation 120
4.4 Interviews 121
4.4.1 The Interview Process 121
4.5 Other Elicitation Techniques 126
4.5.1 Document Review 126
4.5.2 Observation 127
4.5.3 Questionnaires and Surveys 128
4.5.4 Interviews versus Questionnaires 130
4.5.5 Brainstorming 130
4.5.6 Sampling 130
4.5.7 Research 131
Case in Point 4.2: CyberStuff 132
4.6 Requirements Elicitation in Agile Projects 133
4.6.1 A Variation of Interviews 133
4.6.2 User Story Examples 134
4.6.3 Scenario Example 134
4.7 Representing Requirements 135
4.7.1 Natural Language 135
Case in Point 4.3: Digital Pen Transcription 136
4.7.2 Diagrams 136
4.7.3 Models 138
4.8 Validation and Verification 140
4.9 Tools 140
Summary 143
Key Terms 144
Review Questions 145
Discussion Topics 145
Hands-On Projects 145
Ethical Issues 146

Chapter 5
Data & Process Modeling

Learning Objectives 147
5.1 Logical versus Physical Models 148
5.2 Data Flow Diagrams 148
5.3 Data Flow Diagram Symbols 149
5.3.1 Process Symbols 149
5.3.2 Data Flow Symbols 150
5.3.3 Data Store Symbols 151
5.3.4 Entity Symbols 153
5.3.5 Using DFD Symbols 154
5.4 Drawing Data Flow Diagrams 154
5.5 Drawing a Context Diagram 155
5.5.1 Example Context Diagram for a Grading System 156
5.5.2 Example Context Diagram for an Order System 156
5.6 Drawing a Diagram 0 DFD 157
5.6.1 Example Diagram 0 DFD for a Grading System 157
5.6.2 Example Diagram 0 DFD for an Order System 159
5.7 Drawing Lower-Level DFDs 160
5.7.1 Leveling Examples 160
5.7.2 Balancing Examples 163
Case in Point 5.1: Big Ten University 164
5.8 Data Dictionary 164
5.8.1 Documenting the Data Elements 165
5.8.2 Documenting the Data Flows 166
5.8.3 Documenting the Data Stores 166
5.8.4 Documenting the Processes 166
5.8.5 Documenting the Entities 167
5.8.6 Documenting the Records 167
5.8.7 Data Dictionary Reports 167
5.9 Process Description Tools in Modular Design 168
5.9.1 Process Descriptions in Object-Oriented Development 168
5.9.2 Modular Design 168
5.9.3 Structured English 169
5.9.4 Decision Tables 170
Case in Point 5.2: Rock Solid Outfitters (Part 1) 174
5.9.5 Decision Trees 174
Case in Point 5.3: Rock Solid Outfitters (Part 2) 175
Summary 175
Key Terms 176
Review Questions 176
Discussion Topics 176
Hands-On Projects 177
Ethical Issues 177

Chapter 6
Object Modeling

Learning Objectives 178
6.1 Object-Oriented Analysis 179
Case in Point 6.1: TravelBiz 180
6.2 Objects 180
6.2.1 Examples 180
6.3 Attributes 182
6.4 Methods 182
6.4.1 Example 183
6.5 Messages 183
6.5.1 Examples 184

6.6 Classes 184
6.6.1 Example 185
6.7 Relationships Among Objects and Classes 186
6.8 The Unified Modeling Language 187
6.8.1 Use Case Modeling 187
6.8.2 Use Case Diagrams 189
Case in Point 6.2: Hilltop Motors 190
6.8.3 Class Diagrams 190
Case in Point 6.3: Train the Trainer, Inc. 191
6.8.4 Sequence Diagrams 191
6.8.5 State Transition Diagrams 193
6.8.6 Activity Diagrams 193
6.8.7 Business Process Modeling 194
6.9 Tools 194
Summary 194
Key Terms 195
Review Questions 195
Discussion Topics 196
Hands-On Projects 196
Ethical Issues 196

Phase 3 : Design

Chapter 7
User Interface Design

Learning Objectives 198
7.1 User Interfaces 199
7.2 Human–Computer Interaction 200
Case in Point 7.1: Casual Observer Software 202
7.3 Seven Habits of Successful Interface Designers 202
7.3.1 Understand the Business 202
7.3.2 Maximize Graphical Effectiveness 203
7.3.3 Think Like a User 203
7.3.4 Use Models and Prototypes 204
7.3.5 Focus on Usability 204
7.3.6 Invite Feedback 205
7.3.7 Document Everything 205
7.4 Guidelines for User Interface Design 206
7.4.1 Create an Interface That Is Easy to Learn and Use 207
7.4.2 Enhance User Productivity 208
7.4.3 Provide Flexibility 209
7.4.4 Provide Users with Help and Feedback 209
7.4.5 Create an Attractive Layout and Design 210
7.4.6 Enhance the Interface 211
7.4.7 Focus on Data Entry Screens 214
7.4.8 Use Validation Rules 216
7.4.9 Manage Data Effectively 218
7.4.10 Reduce Input Volume 219
Case in Point 7.2: Boolean Toys 219
7.5 Source Document and Form Design 219
7.6 Printed Output 220
7.6.1 Report Design 221
7.6.2 Report Design Principles 221
7.6.3 Types of Reports 223
Case in Point 7.3: Lazy Eddie 224
7.7 Technology Issues 224
7.7.1 Input Technology 225
7.7.2 Output Technology 226
7.7.3 Input/Output Technology Linkage 228
7.8 Control and Security Issues 228
7.8.1 Input Control and Security 229
7.8.2 Output Control and Security 229
7.8.3 Input and Output Control and Security 230
7.9 Emerging Trends 231
7.9.1 Responsive Web Design 231
7.9.2 Prototyping 231
7.9.3 Wearable Computing 234
Summary 235
Key Terms 235
Review Questions 236
Discussion Topics 236
Hands-On Projects 237
Ethical Issues 237

Chapter 8
Data Design

Learning Objectives 238
8.1 Data Design Concepts 239
8.1.1 Data Structures 239
8.1.2 Mario and Danica: A Data Design Example 239
8.1.3 Database Management Systems 241

8.2 DBMS Components 243
8.2.1 Interfaces for Users, Database Administrators, and Related Systems 243
8.2.2 Schema 244
8.2.3 Physical Data Repository 244
8.3 Web-Based Design 244
8.4 Data Design Terms 246
8.4.1 Definitions 246
8.4.2 Key Fields 246
8.4.3 Referential Integrity 249
8.5 Entity-Relationship Diagrams 250
8.5.1 Drawing an ERD 251
8.5.2 Types of Relationships 251
8.5.3 Cardinality 253
Case in Point 8.1: TopText Publishing 254
8.6 Data Normalization 255
8.6.1 Standard Notation Format 256
8.6.2 First Normal Form 257
8.6.3 Second Normal Form 258
8.6.4 Third Normal Form 260
Case in Point 8.2: CyberToys 261
8.6.5 Example 262
8.7 Codes 267
8.7.1 Overview of Codes 267
8.7.2 Types of Codes 268
8.7.3 Designing Codes 269
Case in Point 8.3: Madera Tools 272
8.8 Data Storage and Access 272
8.8.1 Tools and Techniques 272
8.8.2 Logical versus Physical Storage 274
8.8.3 Data Coding 274
8.9 Data Control 276
Summary 277
Key Terms 278
Review Questions 279
Discussion Topics 279
Hands-On Projects 279
Ethical Issues 280

Chapter 9
System Architecture

Learning Objectives 281
9.1 Architecture Checklist 282
9.1.1 Corporate Organization and Culture 282
9.1.2 Enterprise Resource Planning 282
9.1.3 Scalability 283
9.1.4 Web Integration 283
9.1.5 Legacy Systems 284
9.1.6 Processing Options 284
9.1.7 Security 284
9.1.8 Compliance 285
9.1.9 Corporate Portals 285
Case in Point 9.1: ABC Systems 286
9.2 The Evolution of System Architecture 286
9.2.1 Mainframe Architecture 287
9.2.2 Personal Computers 287
9.2.3 Network Evolution 288
9.3 Client/Server Architecture 289
9.3.1 The Client's Role 290
9.3.2 Client/Server Tiers 290
9.3.3 Middleware 291
9.3.4 Cost–Benefit Issues 292
9.3.5 Performance Issues 292
9.4 The Impact of the Internet 293
9.4.1 Internet-Based Architecture 293
9.4.2 Cloud Computing 293
9.4.3 Web 2.0 295
9.5 E-Commerce Architecture 295
9.5.1 In-House Solutions 295
9.5.2 Packaged Solutions 296
9.5.3 Application Service Providers 297
Case in Point 9.2: Small Potatoes 298
9.6 Processing Methods 298
9.6.1 Online Processing 298
9.6.2 Batch Processing 299
9.6.3 Example 300
9.7 Network Models 301
9.7.1 The OSI Model 301
9.7.2 Topologies 301
9.8 Network Devices 305
9.9 Wireless Networks 306
9.9.1 Standards 306
9.9.2 Topologies 307
9.9.3 Trends 307
Case in Point 9.3: Spider IT Services 309
Summary 309
Key Terms 310
Review Questions 311
Discussion Topics 311
Hands-On Projects 311
Ethical Issues 312

Phase 4 : Implementation

Chapter 10
Development Strategies

Learning Objectives 314

10.1 Traditional versus Web-Based Systems Development 315
- 10.1.1 Traditional Development 315
- 10.1.2 Web-Based Development 315
- 10.1.3 Choosing Between Development Strategies 316

10.2 Evolving Trends 316
- 10.2.1 Artificial Intelligence and Machine Learning Integration 316
- 10.2.2 Cloud Computing 317
- 10.2.3 Internet of Things and Edge Computing 317
- 10.2.4 Mobile Devices 318
- 10.2.5 Web 2.0 and 3.0 319

10.3 In-House Development Options 319
- 10.3.1 Make or Buy Decision 320
- 10.3.2 Developing Software In-House 320
- 10.3.3 Purchasing a Software Package 324
- 10.3.4 Customizing a Software Package 327
- 10.3.5 Creating User Applications 328

Case in Point 10.1: Doug's Sporting Goods 329

10.4 Outsourcing 329
- 10.4.1 The Growth of Outsourcing 330
- 10.4.2 Outsourcing Fees 331
- 10.4.3 Outsourcing Issues and Concerns 332

10.5 Offshoring 333

Case in Point 10.2: Turnkey Services 334

10.6 Software as a Service 334
- 10.6.1 Advantages 335
- 10.6.2 Shortcomings 336
- 10.6.3 Example: Microsoft 365 336

10.7 Data Management and Analytics 337

10.8 Selecting a Development Strategy 338
- 10.8.1 The Systems Analyst's Role 338
- 10.8.2 Analyzing Costs and Benefits 339
- 10.8.3 Cost–Benefit Analysis Checklist 340

Case in Point 10.3: Sterling Associates 341

10.9 Software Acquisition 342
- 10.9.1 Analyzing System Requirements 342
- 10.9.2 Identifying Potential Vendors or Outsourcing Options 346
- 10.9.3 Evaluating the Alternatives 348
- 10.9.4 Performing Cost–Benefit Analysis 349
- 10.9.5 Preparing a Recommendation 350

Summary 350

Key Terms 351

Review Questions 352

Discussion Topics 352

Hands-On Projects 352

Ethical Issues 352

Chapter 11
Managing Systems Implementation

Learning Objectives 354

11.1 Quality Assurance 355
- 11.1.1 Software Engineering 355
- 11.1.2 Systems Engineering 356
- 11.1.3 International Organization for Standardization 358

11.2 Application Development 360
- 11.2.1 Design Translation Strategies 360
- 11.2.2 Application Development Tasks 360
- 11.2.3 Systems Development Tools 362

11.3 Structured Development 365
- 11.3.1 Structure Charts 365
- 11.3.2 Cohesion and Coupling 368
- 11.3.3 Drawing a Structure Chart 369

11.4 Object-Oriented Development 371
- 11.4.1 Characteristics of Object-Oriented Development 371
- 11.4.2 Implementation of Object-Oriented Designs 372
- 11.4.3 Object-Oriented Cohesion and Coupling 373

11.5 Agile Development 374
- 11.5.1 Extreme Programming 375
- 11.5.2 User Stories 375
- 11.5.3 Iterations and Releases 376

11.6 Coding 377
- 11.6.1 Programming Languages 377
- 11.6.2 Integrated Development Environments 378
- 11.6.3 Online Learning Resources 381

11.7 Testing 382
- 11.7.1 Unit Testing 384
- 11.7.2 Integration Testing 386
- 11.7.3 System Testing 386

Case in Point 11.1: Your Move, Inc. 387

11.8 Documentation 388
- 11.8.1 Program Documentation 388
- 11.8.2 System Documentation 388
- 11.8.3 Operations Documentation 389
- 11.8.4 User Documentation 389
- 11.8.5 Online Documentation 390

11.9 Installation 391
- 11.9.1 Operational and Test Environments 391
- 11.9.2 System Changeover 392

Case in Point 11.2: Global Cooling 395
- 11.9.3 Data Conversion 395
- 11.9.4 Training 396
- 11.9.5 Preparing for Deployment 400

Case in Point 11.3: Yorktown Industries 404
Summary 404
Key Terms 405
Review Questions 405
Discussion Topics 406
Hands-On Projects 406
Ethical Issues 406

Phase 5 : Operations

Chapter 12
Maintenance & Support

Learning Objectives 408
12.1 User Support and Training 409
12.1.1 Training Programs 409
12.1.2 Support Services 410
12.1.3 Managing Outsourcing 413
12.2 Maintenance Tasks 414
12.2.1 Types of Maintenance 414
12.2.2 Corrective Maintenance 416
12.2.3 Adaptive Maintenance 417
12.2.4 Perfective Maintenance 417
12.2.5 Preventive Maintenance 418
Case in Point 12.1: NovaTech Solutions 418
12.3 Maintenance Operations Management 419
12.3.1 The Maintenance Team 419
12.3.2 Roles and Responsibilities 421
12.3.3 Handling Maintenance Requests 422
12.3.4 Prioritizing Maintenance Work 424
12.3.5 Maintenance Process Improvement 425
12.4 Configuration Management 426
12.4.1 Maintenance Releases 427
12.4.2 Version Control 428
12.4.3 Baselines 429
12.5 System Performance Management 430
12.5.1 Fault Management 430
12.5.2 Performance and Workload Measurement 431
12.5.3 Capacity Planning 433
12.6 Quality Assurance in System Support 433
12.6.1 QA Strategies 434
12.6.2 Testing Practices 436
12.6.3 Feedback and Improvement 438
12.7 Security Management in Maintenance 440
12.7.1 Security Updates 440
Case in Point 12.2: HealthFirst Clinics 441
12.7.2 Security Audits 442
12.7.3 Data Protection 444
12.8 Documentation and Knowledge Management 446
12.8.1 Comprehensive Documentation Strategy 446
12.8.2 Knowledge Base and Content Accessibility 448
12.8.3 Enhancing Support through Documentation 450
12.9 System Retirement and Transition 452
12.9.1 Planning for Retirement 452
12.9.2 Data Migration 453
Case in Point 12.3: DynamicRelations, LLC 455
12.9.3 Decommissioning Procedures 455
12.9.4 Transition to New Systems 457
12.9.5 Archiving and Documentation 458
Summary 460
Key Terms 461
Review Questions 461
Discussion Topics 462
Hands-On Projects 462
Ethical Issues 462

Chapter 13
Managing Systems Security

Learning Objectives 463
13.1 System Security 464
13.1.1 System Security Concepts 464
13.1.2 Identifying Security Threats and Vulnerabilities 464
13.1.3 Security Policies and Procedures 465
13.1.4 Security Frameworks and Standards 465
13.1.5 Legal Implications and Reputational Damage 466
13.2 Understanding and Mitigating Attacks 467
13.2.1 Attacker Profiles and Motivations 467
13.2.2 Types of Cyberattacks 468
13.2.3 Preventive Measures Against Common Attacks 471
13.2.4 Advanced Persistent Threats 471
13.2.5 Social Engineering and Insider Threats 471
13.3 Risk Management in System Security 472
13.3.1 Risk Identification 472
13.3.2 Risk Assessment Techniques 474
13.3.3 Risk Control Strategies 474
13.3.4 Continuous Risk Monitoring 474
13.3.5 Incident Response Planning 475

13.4 Security Levels 475
13.4.1 Physical Security 476
13.4.2 Network Security 480
13.4.3 Application and Software Security 484
13.4.4 Data and File Security Strategies 487
13.4.5 User Security 488
Case in Point 13.1: Pandemic Virtual University 493
13.5 Procedural and Organizational Security 493
13.5.1 Developing and Implementing Security Policies 494
13.5.2 Role-Based Access Control Systems 494
13.5.3 Security Auditing and Compliance 494
13.5.4 Security Information and Event Management 495
Case in Point 13.2: Chain Link Consulting, Inc. 496
13.5.5 Business Continuity Planning 496
13.6 Backup and Recovery Strategies 497
13.6.1 Designing Effective Backup Policies 497
13.6.2 Backup Media Formats and Storage Solutions 498
13.6.3 Enhancing Resilience Against Ransomware Attacks 499
13.6.4 Disaster Recovery Planning 499
13.6.5 Restarting Operations from Backups 500
13.7 Legal and Ethical Considerations in System Security 500
13.7.1 Cyberethics 501
13.7.2 Compliance with Data Protection Laws 501
Case in Point 13.3: Continental Coders, LLC 501
13.7.3 Ethical Hacking and Penetration Testing 502
13.7.4 Privacy Issues and Solutions 502
13.7.5 Intellectual Property Rights and Security 503
13.8 Emerging Technologies and Security 504
13.8.1 The Impact of Cloud Computing on System Security 504
13.8.2 Securing IoT Devices and Networks 505
13.8.3 Blockchain for Enhancing Security 505
13.8.4 Artificial Intelligence in Cybersecurity 506
13.8.5 Future Technologies and Potential Security Implications 506
13.9 Building a Resilient Cybersecurity Culture 506
13.9.1 The Role of Organizational Culture in Cybersecurity 507
13.9.2 Developing a Security-First Mindset 508
13.9.3 Leadership and Governance in Promoting Cybersecurity 509
13.9.4 Integrating Emerging Technologies into Security Strategies 510
13.9.5 Sustaining and Evolving a Cybersecurity Culture 511
Summary 512
Key Terms 513
Review Questions 514
Discussion Topics 514
Hands-On Projects 514
Ethical Issues 515

Appendix A: Communications 516
Appendix B: Finance 542
Glossary 568
Index 595

Preface

We're delighted that instructors and students have enthusiastically received previous editions of *Systems Analysis & Design*. With the release of the 13th edition, we continue our commitment to delivering innovative, high-quality content that meets the expectations of our dedicated audience. This latest edition underscores our ongoing pledge to provide educational materials that adapt to the shifting needs of those studying systems analysis and design.

In crafting this edition, we have meticulously evaluated our teaching strategies to serve today's students better. The modern learner interacts with content in increasingly dynamic ways, necessitating a focus on effective knowledge retention and the practical application of skills in varied contexts. Our teaching methods have evolved to actively engage students and present consistent challenges, ensuring they are fully immersed in their educational journey. Guided by a commitment to student-centered learning, we strive to offer an engaging and insightful resource.

In today's highly competitive global environment, the importance of a robust information technology (IT) infrastructure is more apparent than ever. Our readers, the future systems analysts, managers, and IT professionals, play a crucial role in defining the technological frameworks of their fields. This textbook equips them for these critical roles, providing a thorough grounding in systems analysis and design. It is enriched with real-world scenarios, critical thinking exercises, and updates on the latest technological trends, such as agile methodologies, artificial intelligence, cloud services, and mobile app development. This approach ensures students are prepared to convert business needs into effective information systems, securing strategic advantages and enhancing operational efficiencies.

Overview

Systems Analysis & Design, *13th edition*, presents a comprehensive journey through the systems development life cycle (SDLC). The book is structured to support instructors and students with a straightforward, systematic approach to understanding and applying the principles and practices of systems development. It covers the essential phases from planning and analysis through design and implementation to operations and security management, enriched with real-world examples and contemporary methodologies.

The book solves the critical problem of bridging theoretical concepts with practical application, providing instructors with a robust framework to teach complex subjects in an accessible manner. Appendices on communications and finance offer additional resources that extend learning beyond the core topics, addressing the broader skills needed in systems projects.

An instructor might adopt this book for its holistic coverage, pedagogical clarity, and mirroring of professional practice, preparing students for the realities of the field. Its comprehensive package, which includes detailed explorations of each phase of system development, practical examples, and supplementary content, makes it a valuable tool for fostering critical thinking, technical skills, and the practical know-how essential for student success.

Textbook Benefits

The 13th edition of *Systems Analysis & Design* offers several compelling advantages:

- **Contemporary Coverage:** The textbook is meticulously updated to include the latest methods and technologies, ensuring instructors and students can access the most current information. This relevance prepares students for the realities of the industry, making the learning experience both timely and practical.
- **Holistic Approach:** The book offers a holistic view of systems development, covering not just the technical aspects but also incorporating critical thinking, practical know-how, and ethical considerations. This comprehensive approach ensures students are well-rounded and ready to face the multifaceted challenges of the professional world.
- **Enhanced Learning Tools:** The textbook provides an enriched learning experience with updated figures, diagrams, exercises, examples, and case studies. These tools are designed to facilitate understanding, encourage

engagement, and enhance retention, making the complex topics of systems analysis and design more accessible and enjoyable to learn.

- **Focus on Real-World Application:** The content bridges the gap between theory and practice, strongly emphasizing real-world application. Including "Case in Point" questions and end-of-chapter ethical issues encourages students to apply what they've learned in practical, often complex, scenarios, fostering a deeper understanding of the material.
- **Customized Supplementary Materials:** The appendices on communications and finance provide targeted supplementary content that extends learning beyond the core curriculum. This material addresses the broader skills essential for success in today's interdisciplinary projects, offering added value to instructors and students.
- **Adaptability and Flexibility:** The textbook's structure and content are designed to adapt to various teaching styles and course structures. Whether for a semester-long course or intensive workshops, instructors can easily tailor the material to fit their specific educational needs and goals.
- **Commitment to Ethical Education:** By concluding each chapter with an ethical issue for consideration, the book underscores the importance of ethics in technology. This commitment enriches students' learning experience and aligns with the growing demand for ethical practitioners in the field.

Systems Analysis & Design is an academically rigorous, up-to-date resource deeply committed to preparing students for successful careers in systems analysis and design. Investing in high-quality education equips learners with the knowledge, skills, and ethical grounding necessary to excel in an ever-evolving technological landscape.

Intended Audience

Systems Analysis & Design is designed with the student in mind, providing a clear, structured path through the SDLC's complexities. This book is appealing and helpful to students because it blends theoretical grounding with practical application. By incorporating real-world examples alongside contemporary methodologies, it demystifies the principles of systems development, making them accessible and engaging.

Students will benefit from the textbook's in-depth coverage of each development phase, from planning and analysis to design, implementation, operations, and security management. Each chapter meticulously bridges the gap between theoretical concepts and their practical application in real-world scenarios. This pedagogical approach is significantly enriched by including end-of-chapter exercises, review questions, discussion topics, and hands-on projects, reinforcing understanding and facilitating knowledge application. Additionally, the three "Case in Point" questions in each chapter offer real-life challenges for students to solve, further immersing them in the practical aspects of systems analysis and design. An ethical issue presented at the end of each chapter encourages students to consider the broader implications of technology in society, preparing them for the professional environment and instilling a sense of responsibility and ethical consideration in their work.

The textbook's appendices on communications and finance extend learning beyond traditional systems development topics, addressing the broader skills crucial for today's interdisciplinary projects. This integration creates a comprehensive framework that enhances learning and ensures students are well-prepared for future careers. By equipping them with technical proficiency, critical thinking, practical expertise, and ethical awareness, the textbook ensures students are ready to succeed in the field. This holistic approach makes the book an indispensable resource for students seeking to master the intricacies of contemporary systems projects.

New to this Edition

Systems Analysis & Design has been significantly updated from the 12th edition to reflect modern systems projects. These fundamental changes and enhancements ensure that the textbook stays abreast of the evolving landscape of systems analysis and design and provides a comprehensive, practical, and ethically grounded educational resource. This edition is more evenly divided into five main phases, each addressing a critical stage in the SDLC.

Here is a summary of the key changes:

- **Moved Discussion of Development Strategies:** The new edition's shift of the coverage of development strategies from Phase 2 to Phase 4 better aligns with their practical application during the implementation phase. This update reflects a more intuitive structuring of content, ensuring that discussions on development methodologies occur at a more relevant stage in the SDLC.

- **Expanded Phase 5:** By dividing the discussion of support and security into two distinct chapters in the new operations phase, the new edition emphasizes each area, acknowledging their critical importance in the digital landscape. This separation allows for a deeper dive into the nuances of maintaining and securing systems, which is paramount given today's sophisticated cyber threats and the need for continuous system support.
- **Appendices versus Online Toolkits:** The transition from providing online toolkits to incorporating appendices dedicated to communications and finance marks a significant shift in the supplementary material's format and focus. This change reflects a strategic enhancement to offer more targeted and valuable resources integral to understanding and applying systems analysis and design in real-world settings.
- **Integration of Modern Methods and Technologies:** The updated edition of our textbook explicitly integrates discussions on state-of-the-art methods and technologies such as artificial intelligence and machine learning, blockchain, cloud computing, cybersecurity, and the Internet of Things. By incorporating these pivotal subjects, we ensure that students are not only exposed to the latest trends and tools in the field but are also thoroughly prepared to engage with the technologies shaping the future of systems development. This inclusion underscores our commitment to maintaining a relevant and forward-looking curriculum, equipping students with the knowledge and skills necessary to excel in their future careers amidst rapid technological advancements.
- **Enhanced Use of Figures and Diagrams:** The new edition extensively uses updated figures and diagrams to aid in visualizing complex concepts, facilitating a better understanding of abstract ideas through graphical representation. This visual approach enhances comprehension and retention of material.
- **Comprehensive Exercises, Examples, and Case Studies:** The textbook includes various exercises, real-world examples, and case studies, providing hands-on experience with the material. These additions are designed to bridge the gap between theory and practice, offering students practical applications of what they learn and preparing them for real-world challenges.
- **Ethical Considerations:** Each chapter concludes with an ethical issue for consideration, prompting students to think critically about the moral implications of technology. This focus on ethics is crucial for developing responsible practitioners mindful of their work's impact on society and the environment.

Organization

Systems Analysis & Design provides a holistic view of systems analysis and design, guiding readers through every stage of the SDLC, from initial planning and analysis through design, implementation, and operations, with a strong emphasis on practical application and current methodologies. Including focused appendices on communications and finance adds valuable context and tools for managing the broader implications of systems projects.

The book has 13 chapters across five phases, each addressing a critical SDLC stage.

Phase 1: Planning

This initial phase sets the foundation for the systems development process. It begins with an introduction to systems analysis and design, explaining fundamental concepts and the importance of a structured approach. It then analyzes the business case to justify and align the project with business needs and objectives. Managing systems projects is also covered, highlighting project management techniques and tools essential for planning, executing, and monitoring system development projects.

- **Chapter 1 – Introduction to Systems Analysis & Design:** Chapter 1 introduces systems analysis and design, detailing the evolution of IT, information system components, and the role of systems analysts. It discusses Internet business models, business operation modeling, and organizational and development methodologies.
- **Chapter 2 – Analyzing the Business Case:** Chapter 2 examines business case analysis within strategic planning, including SWOT analysis and the IT department's role, systems requests and feasibility, project prioritization, and preliminary investigations for informed project startup.
- **Chapter 3 – Managing Systems Projects:** Chapter 3 delves into managing systems projects, highlighting the importance of project management, task and people management, the incorporation of technology and financial strategies, risk management, and the use of project management software. The chapter concludes with essential strategies for project success.

Phase 2: Analysis

The analysis phase delves into gathering and specifying system requirements. Requirements engineering is emphasized as a critical step in understanding what the system is supposed to do. Data and process modeling and object modeling are introduced as techniques for conceptualizing and documenting the system requirements, ensuring clear communication among stakeholders and a solid blueprint for system design.

- **Chapter 4 – Requirements Engineering:** Chapter 4 addresses requirements engineering, detailing system requirements, exploring team-based techniques like joint application development and rapid application development, and discussing requirements elicitation methods. It emphasizes the significance of accurately representing requirements and the necessity of validation, verification, and tools in this process.
- **Chapter 5 – Data & Process Modeling:** Chapter 5 discusses data and process modeling, contrasting logical and physical models. It introduces data flow diagrams, their symbols, and the importance of data dictionaries for documenting system components alongside process description tools in modular design.
- **Chapter 6 – Object Modeling:** Chapter 6 focuses on object modeling and analysis, defining core concepts such as objects, attributes, and classes. It also introduces the Unified Modeling Language (UML) to explore various UML diagrams for modeling systems and the application of tools in object modeling.

Phase 3: Design

Focusing on design aspects, this phase addresses how the system will fulfill the requirements outlined in the analysis phase. User interface design, data design, and system architecture are the main components, each dealing with different aspects of the system's design—how users will interact with the system, how data is organized and managed, and the overall system structure and technology stack.

- **Chapter 7 – User Interface Design:** Chapter 7 highlights the significance of user interface design and human–computer interaction, offering design habits, discussing principles for various outputs, and addressing modern trends like responsive and wearable technology.
- **Chapter 8 – Data Design:** Chapter 8 investigates data design, explaining data structures, database management system components, entity-relationship diagrams, data normalization, coding practices, and methods for managing data storage and access.
- **Chapter 9 – System Architecture:** Chapter 9 explores system architecture, detailing the progression from mainframes to network computing, highlighting the role of the Internet, cloud computing, and e-commerce, and discussing various processing methods and network technologies.

Phase 4: Implementation

The implementation phase is where the system is built. It includes development strategies and discusses various approaches to system development, such as waterfall and agile. Managing systems implementation covers the practical aspects of putting the system into operation, including deployment strategies, testing, and transitioning to the new system.

- **Chapter 10 – Development Strategies:** Chapter 10 analyzes development strategies, spotlighting the contrast between traditional and web-based methods. It also examines current technological trends, exploring options like in-house development, outsourcing, and software as a service, along with the significance of data management and the strategic selection process.
- **Chapter 11 – Managing Systems Implementation:** Chapter 11 emphasizes the challenges of system implementation, covering quality assurance, engineering standards, development tools, and methodologies. It also details coding practices, testing phases, documentation, and installation processes, ensuring readiness for deployment.

Phase 5: Operations

The final phase addresses the system's post-deployment considerations. Maintenance and support are covered, detailing how the system will be kept operational, updated, and user-supported over time. System security management emphasizes protecting the system and its data against threats and ensuring confidentiality, integrity, and availability.

- **Chapter 12 – Maintenance & Support:** Chapter 12 covers facets of maintenance and support, detailing aspects from user training to system retirement. It addresses the management of tasks, operations, quality assurance, security, and knowledge within system support.
- **Chapter 13 – Managing Systems Security:** Chapter 13 concentrates on the breadth of systems security management, from foundational concepts to cultivating a robust cybersecurity culture. It discusses risk management, security protocols, legal issues, and integrating emerging technologies into security measures.

Appendices

The appendices provide additional resources on communications and finance, offering insights and tools relevant to systems development projects that are not fully covered in the main chapters. These sections guide effective communication strategies for project teams and stakeholders and financial analysis techniques for budgeting, cost–benefit analysis, and financial planning of systems projects.

- **Appendix A – Communications:** Appendix A discusses communication tools to help the systems analyst write clearly, speak effectively, and deliver powerful presentations—in person or online.
- **Appendix B – Finance:** Appendix B covers the financial aspects of systems analysis, including cost–benefit analysis, cost management, and financial tools for project evaluation.

Features

To help you fully understand systems analysis and design, this material includes many features designed to enhance your learning experience.

Learning Objectives: Each chapter lists the concepts to be mastered within that chapter. This list is a quick reference to the chapter's contents and a useful study aid.

Figures and Tables: Numerous full-color diagrams illustrating abstract ideas and screenshots of systems analysis and design tools help learners better visualize concepts. Tables provide details and comparisons that can be easily reviewed for future reference.

Case in Point: Each chapter includes brief cases that provide a contextual business example for students focused on the key issues covered in the chapter.

Chapter Summaries: Each chapter reading concludes with a summary of the concepts introduced in that chapter. These summaries revisit the ideas covered in each chapter.

Key Terms: Following the chapter summary, all terms introduced in the chapter are gathered in the Key Terms list. This list encourages a more thorough understanding of the chapter's key concepts and is a valuable reference.

Review Questions: The end-of-chapter assessment begins with a set of review questions that reinforce the main concepts in each chapter. These questions help you evaluate and apply the material you have learned.

Discussion Topics: The end-of-chapter discussion topics reinforce the main concepts in each chapter and offer opportunities to dig deeper and learn more about given content areas.

Hands-On Projects: Projects at the end of each chapter provide the opportunity to apply and practice key concepts covered in the chapter. These projects include detailed instructions for walking through various tasks and processes associated with systems analysis and design.

Ethical Issues: A realistic ethical issue is presented at the end of each chapter. These examples force you to examine your reactions and how you would respond to everyday workplace situations.

Text and Graphic Conventions

This book contains the following text and graphic conventions.

Case in Point

These brief cases provide a contextual business example for students focused on the key issues covered in the chapter.

Instructor's Materials

Instructors, please visit cengage.com and sign in to access instructor-specific resources, which include the Instructor's Manual, Solution and Answer Guide, instructor test banks, and PowerPoint presentations.

Instructor's Manual: The Instructor's Manual accompanying this text provides additional instructional material to assist in class preparation, including suggestions for discussion topics and additional projects.

Solution and Answer Guide: The instructor's resources include solutions to all end-of-chapter material, including review questions, discussion topics, hands-on projects, and ethical issues.

Cengage Testing Powered by Cognero: This flexible, online system allows you to do the following.

- Author, edit, and manage test bank content from multiple Cengage solutions.
- Create multiple test versions in an instant.
- Deliver tests from your learning management system, classroom, or wherever you want.

PowerPoint Presentations: This course comes with Microsoft PowerPoint slides for each chapter. These slides are meant to be used as a teaching aid for classroom presentations, made available to students on the network for chapter review, or printed for classroom distribution. Instructors can add their own slides for additional topics introduced to the class.

MindTap

MindTap for *Systems Analysis & Design, 13th edition*, is an online learning solution designed to help you master the skills needed in today's workforce. Research shows that employers need critical thinkers, troubleshooters, and creative problem solvers to stay relevant in our fast-paced, technology-driven world. MindTap helps you achieve this with assignments and activities that provide hands-on practice, real-life relevance, and proficiency in complex concepts. Students are guided through assignments, progressing from basic knowledge and understanding to more challenging problems. MindTap activities and assignments are tied to learning objectives. MindTap features include the following:

Concept Clips: Concept Clip videos focus learners on a key concept in each chapter and are designed to deepen their understanding of the topic.

Appliance Warehouse Case: We've created an updated design and flow of the running Appliance Warehouse case activities. The case challenges learners to apply key systems analysis and design concepts and skills to a realistic scenario they would encounter in the workplace. The case brings together each chapter's key concepts and skills in an authentic assignment. The look and feel of the case tool has been updated to be an authentic, immersive experience for students.

Information Systems for Life: These assignments encourage you to stay current with developments in systems analysis and design.

Reflection: These activities encourage classroom and online discussion of key topics covered in the chapters.

Pre- and Post-Assessments: Assess your understanding of key concepts at the beginning and end of the course and emulate the text.

About the Author

Scott Tilley is an emeritus professor at the Florida Institute of Technology, president and founder of the Center for Technology & Society, president and co-founder of Big Data Florida, president of Precious Publishing, a member of the Board of Directors of the Florida Authors and Publishers Association, a Space Coast Writers' Guild Fellow, and an ordained minister. In addition to this book, he is the author or editor of numerous other publications, including *Poems of the Moment* (2023), *AFTERMATH* (2022), *PETS* (2021), *Perspectives on Systems Engineering from Florida's Space Coast* (2020), and *Technical Justice* (2019). He previously was a faculty member at the University of California, Riverside, a research scientist at Carnegie Mellon University's Software Engineering Institute, and a software developer at IBM. He holds a PhD in computer science from the University of Victoria.

Acknowledgments

Modern textbooks transcend the conventional boundaries of "books": they are dynamic educational platforms involving various components and multiple stakeholders. Thus, updating a comprehensive and mature volume like *Systems Analysis & Design* is equally challenging and rewarding. The complexity of this endeavor escalates with tight schedules—a common feature of contemporary projects.

I am profoundly grateful for the exceptional efforts of the Cengage production team. I thank Natalie Onderdonk, Marlena Sullivan, Carolyn Mako, and Dan Seiter for their invaluable assistance and guidance. Thanks also to the reviewers who provided valuable criticism that helped shape each chapter: Dr. Utpal Bose, University of Houston–Downtown; Dr. Fitzroy Gordon, Liberty University; and Dr. Carl Rebman, University of San Diego. Any mistakes or oversights in this edition are solely my responsibility.

My gratitude extends to the instructors and students who provide insightful feedback and comments. Responding to every query is not always possible, but rest assured, I consider all input valuable and will continue to listen carefully to your suggestions. Please get in touch with me via email at stilley@fit.edu or connect with me online at https://www.linkedin.com/in/drtilley/.

Phase 1

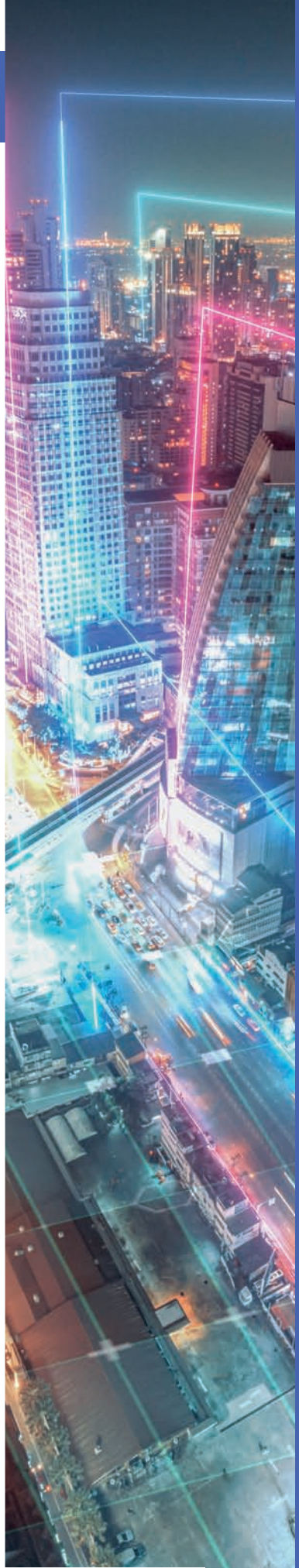
Planning

The planning phase of the systems development life cycle is crucial for aligning technology projects with organizational goals. Strategic planning and project management assess feasibility, set priorities, enhance communication, and manage risks to drive efficiency and innovation. Key activities include conducting a thorough business case analysis and ensuring alignment with objectives. By prioritizing effective planning, fostering clear communication, and engaging stakeholders early on, organizations lay the groundwork for a development process that maximizes efficiency, sparks innovation, and tackles potential challenges.

Chapter 1 introduces systems analysis and design, covering the changing nature of information technology, the components of information systems, and the role of systems analysts. It explores Internet business strategies, modeling business operations, and various types of business information systems. The chapter also discusses organizational information models, systems development methods, the structure of IT departments, and the responsibilities and career prospects of systems analysts.

Chapter 2 examines business case analysis within the context of strategic planning. It covers techniques like SWOT analysis and the role of the IT department in this process. The chapter explores systems requests, factors affecting systems projects, and the process of assessing request feasibility. It also discusses prioritizing projects and conducting preliminary investigations to guide decision making and project initiation.

Chapter 3 delves into managing systems projects, emphasizing project management and the project triangle. It covers task and people management, including work breakdown structures, critical paths, team dynamics, and communication challenges. The chapter discusses technology and financial management, risk management strategies, and project management software. It concludes with key strategies for project success, including monitoring, issue resolution, and adopting an agile mindset.



Chapter

1

Introduction to Systems Analysis & Design

Learning Objectives

When you finish this chapter, you should be able to:

1. Describe the impact of information technology on society.
2. Describe the five main components of an information system.
3. Explain Internet business strategies and relationships, including B2C and B2B.
4. Explain how to use business profiles and models.
5. Identify the seven types of information systems used in business.
6. Describe the types of information the four classes of users need.
7. Distinguish between structured analysis, object-oriented analysis, and agile systems development methods.
8. Explain the seven main functions of the information technology department.
9. Describe the roles and responsibilities of a systems analyst within the enterprise.

Chapter 1 is the first of three chapters in the systems planning phase. This chapter explains the role of information technology in today's dynamic business environment. It also describes information systems development, systems analysis and design concepts, and various systems development methods. Finally, this chapter summarizes the role of the information technology department and its people in the enterprise.

The chapter includes three "Case in Point" discussion questions to help contextualize the concepts described in the text. The scenario described in the "Ethical Issues" section of the end-of-chapter exercises invites examination of the ACM's code of ethics and those of a developing systems analyst.

1.1 Information Technology

Information technology (IT) combines hardware, software, and services people use to manage, communicate, and share information. Companies use the information to increase productivity, deliver high-quality products and services, maintain customer loyalty, and make sound decisions. Information technology can mean the difference between success and failure in a competitive global economy.

More than ever, business success depends on information technology. IT is driving a new digital economy in which hardware, software, and connectivity advances can benefit businesses and individuals enormously. Although economic trends affect IT spending levels, most companies prioritize IT budgets in good times or bad. The reason is simple: during periods of growth, companies must pay attention to the IT curve. Conversely, firms often use IT to reduce operating costs and improve efficiency when the economy slows down.

Information technology also has a profound influence on modern life. Although fictitious, the headlines in Figure 1-1 offer dramatic examples of how IT issues such as data privacy, mobile devices, and social media affect our society. We live in a world where we can be traced, analyzed, and surveilled without our knowledge. This raises many important questions, such as securing personal data while providing useful functionality and business value.

Figure 1-1 These headlines illustrate the enormous impact of information technology on our lives.

The following sections provide a sense of IT history, an overview of systems analysis and design, and a description of the systems analyst's role.

1.1.1 The Changing Nature of Information Technology

The history of IT is a fascinating study of human progress and achievement. We are dazzled by the latest and greatest technology, just as our parents and grandparents were astonished by the arrival of television, space flight, and personal computing. However, it is vital for IT professionals, who live and work in this exciting world, to realize that each technological advance is part of a long-term process that often brings dramatic change but never really ends. The story of IBM is a good example.

As its name suggests, International Business Machines was a major supplier of office equipment and typewriters long before the modern computer era. Herman Hollerith, who invented a card that identified characters by the location of punched holes, founded IBM's predecessor company in 1896. A deck of hundreds or even thousands of these cards could store data that was quickly sorted, queried, and printed by machines. This system sounds archaic now, but punch card technology was a huge advance that revolutionized the business world and was used in the 1960s and beyond.

Today, IBM is a globe-spanning company with several hundred thousand employees. It has succeeded in part by constantly adapting to its changing business environment. For example, while it was once known primarily as a hardware company, IBM now makes significant revenue from software and services. It also invests in its people and tries to hire the best talent available. The result is that IBM has more patents and Nobel Prize winners than any other IT company.

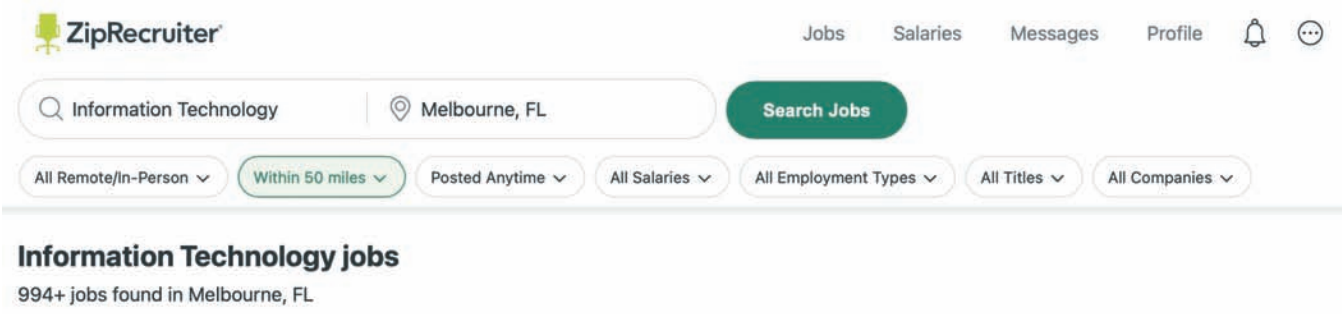
1.1.2 Systems Analysis and Design

Systems analysis and design is a step-by-step process for developing high-quality information systems. An **information system** combines technology, people, and data to support business functions such as order processing, inventory control, human resources, and accounting.

Some information systems handle routine day-to-day tasks, while others can help managers make better decisions, spot marketplace trends, and reveal patterns that might be hidden in stored data.

Talented people, including a mix of managers, users, network administrators, web designers, programmers, and systems analysts, typically develop information systems. Capable IT professionals like these are always in demand, even in a slow economy. For example, notice how many positions related to IT are available in the Melbourne, Florida area, as shown on ZipRecruiter's job search website in Figure 1-2.

Figure 1-2 ZipRecruiter is an example of a job search website that IT professionals can use.
Source: ZipRecruiter, Inc



1.1.3 What Does a Systems Analyst Do?

A **systems analyst** is a valued member of the IT department team who helps plan, develop, and maintain information systems. Therefore, analysts must be excellent communicators with strong analytical and critical thinking skills. In addition, because systems analysts transform business requirements into IT projects, they must be business-savvy and technically competent, and be equally comfortable with managers and programmers, who sometimes have different points of view.

Most companies assign systems analysts to the IT department, but analysts also can report to a specific user area such as marketing, sales, or accounting. As a member of a functional team, an analyst can better understand the group's needs and how IT supports the department's mission. In addition, smaller companies often use consultants to perform systems analysis work as needed.

On any given day, an analyst might be asked to document business processes, test hardware and software packages, design input screens, train users, and plan e-commerce websites. In addition, a systems analyst may occasionally manage IT projects, including tasks, resources, schedules, and costs. The analyst conducts meetings, delivers presentations, and writes memos, reports, and documentation to keep managers and users informed.

Section 1.9 lists typical skills and education requirements, certifications, career opportunities, and the possible impact of future IT trends for systems analysts.

1.2 Information Systems

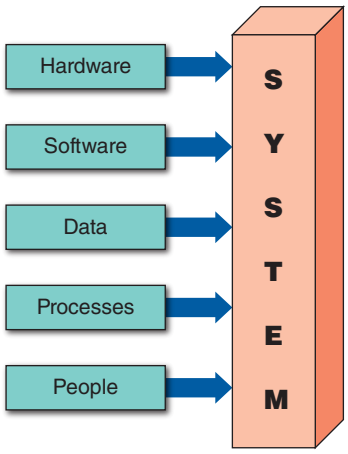
A **system** is a set of related components that produces specific results. For example, specialized systems route Internet traffic, manufacture microchips, and control complex entities like the James Webb Space Telescope, which took the fantastic image shown in Figure 1-3. A **mission-critical system** is vital to a company's operations. An order processing system, for example, is mission-critical because the company cannot do business without it.

Every system requires input data. For example, a computer receives data when a key is pressed or a menu command is selected. In an information system, **data** consists of basic facts that are the system's raw material. **Information** is data that has been transformed into valuable output for users.

Figure 1-3 Consider the amazing technology that enabled the James Webb Space Telescope to capture this image of the star-forming region NGC 3324 in the Carina Nebula.
NASA and the Space Telescope Science Institute (STScI).

An information system has five key components, as shown in Figure 1-4: hardware, software, data, processes, and people.

Figure 1-4 An information system needs these components.



1.2.1 Hardware

Hardware consists of everything in the physical layer of the information system. For example, hardware can include servers, workstations, networks, telecommunications equipment, fiber-optic cables, mobile devices, scanners, digital capture devices, and other technology-based infrastructure. A large concentration of networked computers working together is called a **data center**. As new technologies emerge, manufacturers race to market the innovations and reap the rewards.

Hardware purchasers today face a wide array of technology choices and decisions. In 1965, Gordon Moore, a co-founder of Intel, predicted that the number of transistors on an integrated circuit chip would double approximately every 24 months. His concept, called **Moore's Law**, has remained valid for over 50 years. Fortunately, as hardware became more powerful, it became much less expensive. However, large businesses with thousands or millions of sales transactions require company-wide information systems and powerful servers, often now in the cloud, such as those shown in Figure 1-5.

1.2.2 Software

Software refers to the programs that control the hardware and produce the desired information or results. Software consists of system software and application software.

System software manages the hardware components, which can include a single computer or a global network with many thousands of clients. The hardware manufacturer supplies the system software, or a company purchases it from a vendor. System software includes the operating system, security software that protects the computer from intrusion, device drivers that communicate with hardware such as printers, and utility programs that handle specific

Figure 1-5 Cloud computing provides the enormous storage and speed modern IT systems need.
Oleksiy Mark/Shutterstock.com

tasks such as data backup and disk management. System software also controls data flow, provides data security, and manages network operations. In today's interconnected business world, network software is vitally important.

Application software consists of programs supporting day-to-day business functions and providing users with necessary information. Examples of company-wide applications, called **enterprise applications**, include order processing systems, payroll systems, and company communications networks. On a smaller scale, individual users can boost productivity with tools such as spreadsheets, presentation software, and database management systems.

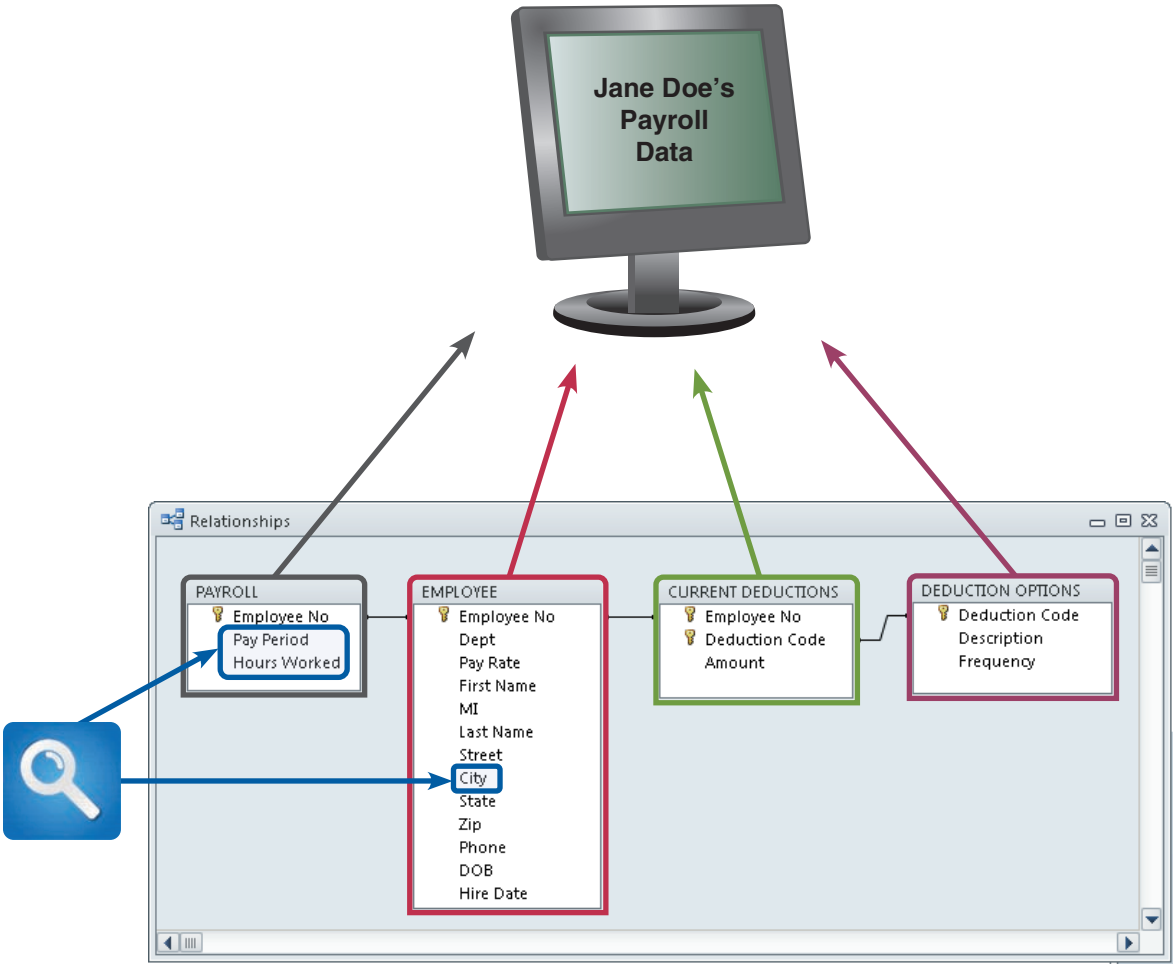
Application software includes horizontal and vertical systems. A **horizontal system** is a system, such as an inventory or payroll application, that can be adapted for use in many different types of companies. A **vertical system** is designed to meet the unique requirements of a specific business or industry, such as an online retailer, a medical practice, or an auto dealership.

Most companies use a mix of software that is acquired at various times. Therefore, when planning an information system, a company must consider how a new system will interface with older systems, which are called **legacy systems**. For example, a new human resources system might need to exchange data with a legacy payroll application.

1.2.3 Data

Data is the raw material that an information system transforms into useful information. For example, an information system using a relational database can store data in various locations, called tables. The system can display the specific information the user needs by linking the tables. For example, Figure 1-6 shows a payroll system that stores data in four separate tables. Notice that the linked tables work together to supply 19 different data items. A user can display any or all data items and filter the data to fit defined limits. In this example, the user requested a list of employees who live in a specific city and worked more than 40 hours in the last pay period. Jane Doe's name was the first to display.

Figure 1-6 In a typical payroll system using a relational model, data is stored in separate tables linked to form an overall database.



The growth of **big data** has given rise to new ways of storing, searching, and managing data. Traditional relational models are still used, but so-called **NoSQL databases** are gaining popularity due to their ability to scale to vast unstructured data sets.

1.2.4 Processes

Processes describe the tasks and business functions that users, managers, and IT staff perform to achieve specific results. Processes are an information system's building blocks because they represent day-to-day business operations. Therefore, analysts must understand business processes and document them carefully to build a successful information system.

1.2.5 People

People who have an interest in an information system are called **stakeholders**. Stakeholders include the management group responsible for the system, the **users** (sometimes called end users) inside and outside the company who will interact with the system, and IT staff members, such as systems analysts, programmers, and network administrators who develop and support the system.

Each stakeholder group has a vital interest in the information system. Still, most experienced IT professionals agree that the success or failure of a system usually depends on whether it meets the needs of its users. Therefore, it is essential to understand user requirements and expectations throughout the development process.

Case in Point 1.1

Data Breaches

A data breach occurs when a hacker gains illegal access to a system and steals personal data, such as credit card numbers or home addresses. Data breaches are becoming increasingly common, with more information stored in the cloud. Research recent news articles about large-scale data breaches, summarize why they occurred, and suggest how they might be prevented.

1.3 Internet Business Strategies

Systems analysts must understand a company's business operations to design successful systems. Each situation is different. For example, a retail store, a medical practice, and a hotel chain all have unique information systems requirements. As the business world changes, systems analysts can expect to work in new companies requiring innovative IT solutions.

Business today is shaped by three significant trends: rapidly increasing globalization, technology integration for seamless information access across various devices such as laptops and smartphones, and the rapid growth of cloud-based computing and software services. The immense power of the Internet is driving these trends.

Internet-based commerce is called **e-commerce (electronic commerce)**. Internet-based systems involve various hardware and software designs, but a typical model is a series of webpages that provides a user interface that communicates with database management software and a web-based data server. The user interacts with the system through an **app** on mobile devices, but the same back-end services are accessed. As Internet-based commerce grows, career opportunities for IT professionals such as web designers, database developers, and systems analysts will significantly expand.

1.3.1 B2C (Business-to-Consumer)

Using the Internet, consumers can purchase an enormous variety of products and services online. This new shopping environment allows customers to research, compare prices and features, check availability, arrange delivery, and choose payment methods in a single, convenient session. Many companies, such as airlines, offer incentives for online transactions because web-based processing costs are lower than those using traditional methods. By making flight information available online to last-minute travelers, some airlines also offer special discounts on seats that might otherwise go unfilled.

B2C (business-to-consumer) is changing traditional business models and creating new ones. For example, a typical business model is a retail store that sells a product to a customer. To conduct that transaction online, the company must develop an online store and deal with different marketing, advertising, and profitability issues.

Some companies have found new ways to use established business models. For example, Airbnb and VRBO have transformed the traditional hospitality service industry into a popular and successful way for individuals to rent their properties. Other retailers seek to enhance the online shopping experience by offering gift advisors, buying guides, how-to clinics, and similar features. However, in the e-commerce battles, the real winners are online consumers, who have more information, better choices, and the convenience of shopping at home.

1.3.2 B2B (Business-to-Business)

Although the B2C sector is more familiar to retail customers, the volume of **B2B (business-to-business)** transactions is many times greater. Moreover, industry observers predict that B2B sales will increase sharply as more firms seek to improve efficiency and reduce costs.

Initially, electronic commerce between two companies used a data-sharing arrangement called **electronic data interchange (EDI)**. EDI enabled computer-to-computer data transfer, usually over private telecommunications lines. Firms used EDI to plan production, adjust inventory levels, or stock up on raw materials using data from another company's information system. As B2B volume soared, company-to-company transactions migrated to the Internet, which offered standard protocols, universal availability, and low communication costs. The main advantage of the web is that it provides seamless communication between different hardware and software environments, anywhere and anytime.

Because it allows companies to reach the global marketplace, B2B is vital to smaller suppliers and customers needing instant price and availability information. In an approach that resembles an open marketplace, some B2B sites invite buyers, sellers, distributors, and manufacturers to offer products, submit specifications, and transact business.

Most large firms and government agencies use **supply chain management (SCM)** software. A **supply chain** refers to all the companies that provide materials, services, and functions needed to deliver products to a customer. For example, a Sherwin-Williams customer who buys a gallon of paint is at the end of a chain that includes the raw material sources, packaging suppliers, manufacturers, transporters, warehouses, and retail stores. Because SCM is complex and dynamic, specialized software helps businesses manage inventory levels, costs, alternate suppliers, and much more.

1.3.3 C2C (Consumer-to-Consumer)

The COVID-19 pandemic forced many consumers to buy products and services online. As a result, an entire industry of new companies quickly blossomed, selling products and services online to match burgeoning consumer demand. The result has been tremendous growth of **C2C (consumer-to-consumer)** e-commerce transactions.

An example of C2C e-commerce is the fashion industry, particularly the popularity of buying and selling used clothing. What was previously seen as the purview of secondhand thrift stores rapidly morphed into an economical way of reusing valuable clothing items in an environmentally friendly marketplace. Companies such as Poshmark (shown in Figure 1-7), Debop, and Mercari are examples of successful C2C Internet businesses.

1.4 Modeling Business Operations

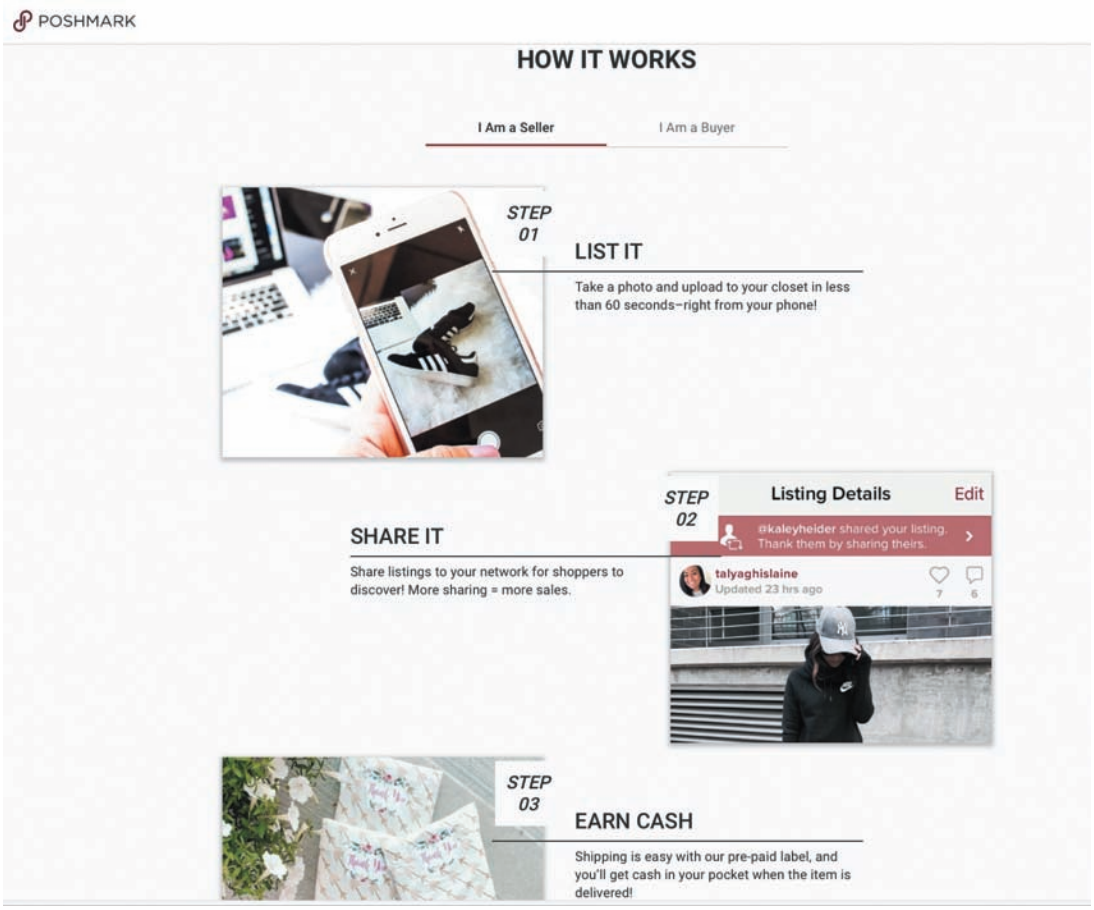
Systems analysts use modeling to represent company operations and information needs. **Modeling** produces a graphical representation of a concept or process that systems developers can analyze, test, and modify. A systems analyst can describe and simplify an information system using business, data, object, network, and process models.

A **business profile** is an overview of a company's mission, functions, organization, products, services, customers, suppliers, competitors, constraints, and future direction. Although much of this information is readily available, a systems analyst may need additional research and fact-finding to fill out missing or incomplete information. A business profile is the starting point for the modeling process. A systems analyst can describe and simplify an information system by using a set of business models and business process models.

A **business model** describes the information that a system must provide. Analysts also create models representing data, objects, networks, and other system components. Although the models might appear to overlap, they work together to describe the same environment from different points of view.

Business process modeling involves a business profile and a set of models that document business operations. **Model-based systems engineering (MBSE)** is one of the leading methods systems analysts use to develop information systems.

Figure 1-7 Poshmark is a leading social marketplace for new and secondhand styles for women, men, kids, pets, home, and more.
Source: Poshmark, Inc.



A **business process** is a specific set of transactions, events, and results that can be described and documented. A **business process model (BPM)** graphically displays one or more business processes, such as handling an airline reservation, filling a product order, or updating a customer account. The sales order example in Figure 1-8 shows a simple model that includes an event, three processes, and a result.

A rough sketch might be sufficient to document a simple business process. However, for complex models, analysts can choose computer-based tools that use **business process modeling notation (BPMN)**. BPMN includes standard shapes and symbols representing events, processes, and workflows. Multipurpose applications such as Microsoft Visio or online diagramming tools such as bpmn.io can be used to create BPMN models. For example, notice that the model in Figure 1-9 uses BPMN symbols to represent the same sales order process shown in Figure 1-8.

Systems developers often use multipurpose charting tools like Lucidchart to display business-related models. Lucidchart is a popular tool that systems analysts can use to create business process diagrams, flowcharts, organizational charts, network diagrams, floor plans, project timelines, and workflow diagrams. For example, Figure 1-10 shows how to drag and drop various symbols from the left pane into the drawing on the right and connect them to illustrate a business process.

Figure 1-8 A simple business model might include an event, three processes, and a result.

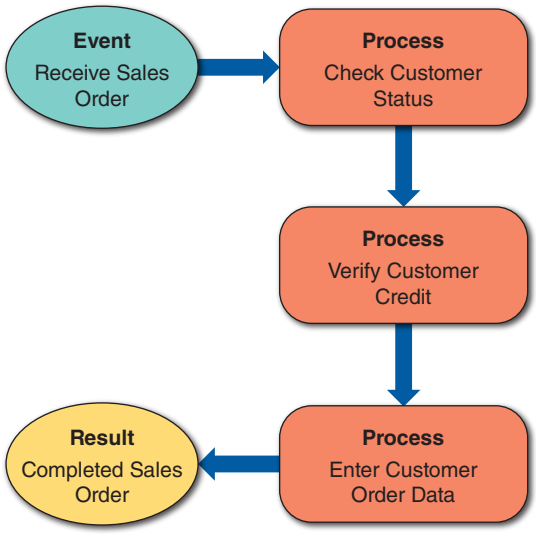


Figure 1-9 This sample uses business process modeling notation (BPMN) to represent the same events, processes, and workflow shown in Figure 1-8.
Source: OMG - Object Management Group

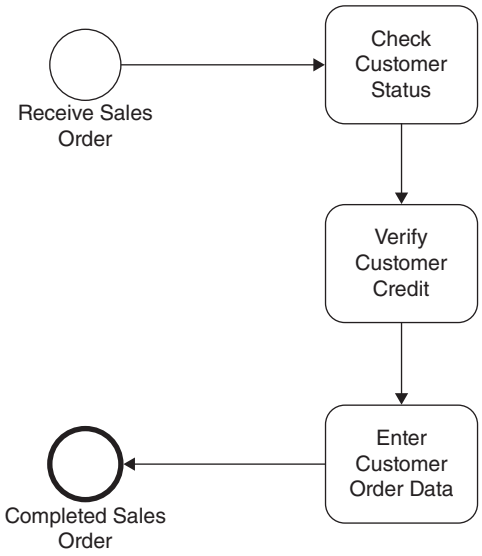
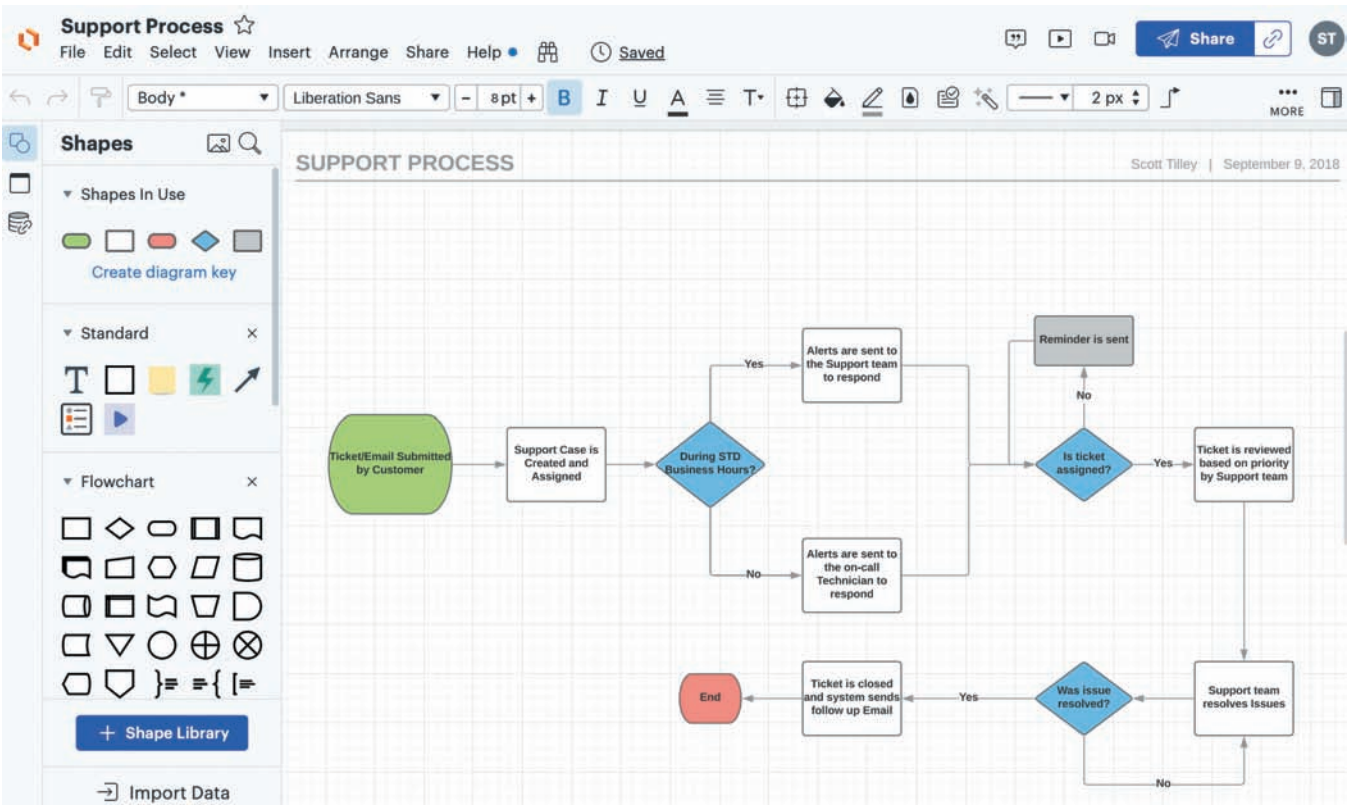


Figure 1-10 Lucidchart allows you to drag and drop various symbols and connect them to model a business process.
Source: LucidChart



1.5 Business Information Systems

In the past, IT managers identified an information system based on its primary users. For example, administrative staff used *office systems*, operational people used *operational systems*, middle managers used *decision support systems*, and top managers used *executive information systems*.

Today, those traditional labels no longer apply. For example, all employees, including top managers, use office productivity systems to do their jobs. Similarly, operational users often require decision support systems to do their jobs. As business changes, information use also changes, and now it makes more sense to identify a system by its functions and features rather than by its users. A new set of system definitions includes enterprise computing systems, transaction processing systems, business support systems, knowledge management systems, user productivity systems, digital assistants, and systems integration.

1.5.1 Enterprise Computing

Enterprise computing refers to information systems that support company-wide operations and data management requirements. Walmart's inventory control system, Boeing's production control system, and Hilton Hotels' reservation system are examples of enterprise computing systems. The main objective of enterprise computing is to integrate a company's primary functions (such as production, sales, services, inventory control, and accounting) to improve efficiency, reduce costs, and help managers make key decisions. Enterprise computing also enhances data security and reliability by imposing a company-wide data access and storage framework.

In many large companies, applications called **enterprise resource planning (ERP)** systems provide cost-effective support for users and managers throughout the company. For example, a car rental company can use ERP to forecast customer demand for rental cars at hundreds of locations. Because of its growth and potential, many hardware and software vendors target the enterprise computing market and offer various products and services.

Many firms have achieved dramatic cost reductions by providing a company-wide computing environment. However, other companies have been disappointed in the time, money, and commitment necessary to implement ERP successfully. In addition, a potential disadvantage is that ERP systems generally impose an overall structure that might not match how a company operates.

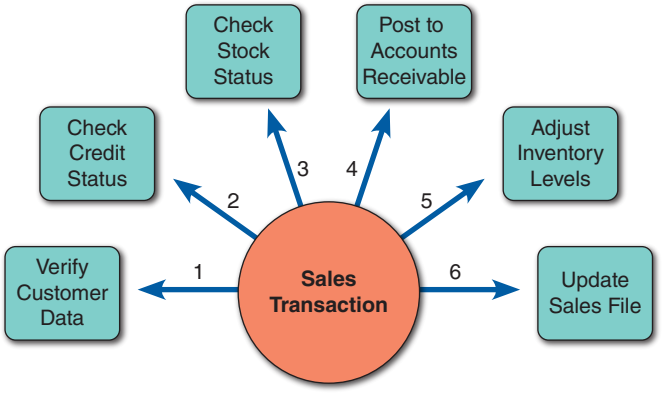
1.5.2 Transaction Processing

Transaction processing (TP) systems process data generated by day-to-day business operations. Examples of TP systems include customer order processing, accounts receivable, and warranty claim processing.

TP systems perform a series of tasks whenever a specific transaction occurs. In the example shown in Figure 1-11, a TP system verifies the customer's data, checks the customer's credit status, checks the stock status, posts to accounts receivable, adjusts the inventory level, and updates the sales file. TP systems typically involve large amounts of data and are mission-critical because the enterprise cannot function without them.

TP systems are efficient because they process transaction-related commands as a group rather than individually. In addition, to protect data integrity, TP systems ensure that if any single transaction element fails, the system does not process the rest of the transaction.

Figure 1-11 A single sales transaction consists of six separate tasks, which the TP system processes as a group.



1.5.3 Business Support

Business support systems provide job-related information support to users at all levels of a company. These systems can analyze transactional data, generate information needed to manage and control business processes, and provide information that leads to better decision making.

The earliest business computer systems replaced manual tasks, such as payroll processing. However, companies soon realized that computers also could produce valuable information. The new systems were

called **management information systems (MIS)** because managers were the primary users. Today, employees at *all* levels need information to perform their jobs and rely on information systems for that support.

A business support system can work hand in hand with a TP system. For example, when a company sells merchandise to a customer, a TP system records the sale, updates the customer's balance, and makes a deduction from inventory. A related business support system highlights slow- or fast-moving items, customers with past-due balances, and inventory levels that need adjustment.

Firms must collect production, sales, and shipping data and immediately update the company-wide business support system to compete effectively. Automated data acquisition is possible using technology such as **radio frequency identification (RFID)**, which uses high-frequency radio waves to track physical objects, such as the shirt shown in Figure 1-12. Major retailers such as Walmart, which requires suppliers to add RFID tags to all items, have fueled RFID's dramatic growth by tracking products throughout the retail process.

Figure 1-12 With an RFID tag, items can be tracked and monitored throughout retail.
Tatchaphol/Shutterstock.com

An essential feature of a business support system is decision support capability. Decision support helps users make decisions by creating a computer model and applying a set of variables. For example, a truck fleet dispatcher might run a series of what-if scenarios to determine the impact of increased shipments or lousy weather. Alternatively, a retailer might use what-if analysis to determine the price it must charge to increase profits by a specific amount while volume and costs remain unchanged.

1.5.4 Knowledge Management

Knowledge management systems use a large database called a **knowledge base** that allows users to find information by entering keywords or questions in common English phrases. In addition, a knowledge management system uses **inference rules**, which are logical rules that identify data patterns and relationships.

The WolframAlpha website, shown in Figure 1-13, describes itself as a "computational knowledge engine." It has a sophisticated natural language front end that understands user queries in several domains. As shown in the figure, these domains include Mathematics,

Figure 1-13 WolframAlpha describes itself as a "computational knowledge engine."
Source: Wolfram Alpha LLC

Science & Technology, Society & Culture, and Everyday Life. WolframAlpha relies on an extensive knowledge base spanning multiple websites and proprietary algorithms to provide users with detailed answers to their questions on many different topics. The results are displayed using a mix of multimedia, including mathematical equations if appropriate.

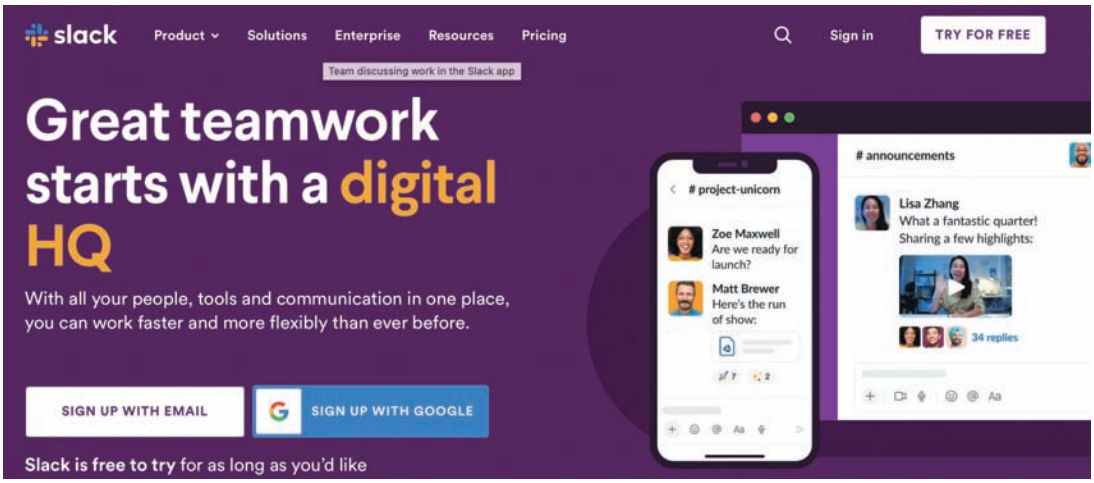
1.5.5 User Productivity

Companies provide employees at all levels with technology that improves productivity. Examples of **user productivity systems** include email, voice mail, video and web conferencing, word processing, automated calendars, database management, spreadsheets, desktop publishing, presentation graphics, company intranets, and integrated mobile computing systems. User productivity systems also include **groupware**, which enables users to share data, collaborate on projects, and work in teams. One popular groupware product is Slack, shown in Figure 1-14. Slack provides common app integration and unified communication channels for distributed teams.

When companies first installed word processing systems, managers expected to reduce the number of employees as office efficiency increased. That did not happen, primarily because the fundamental nature of clerical work changed. With computers performing the repetitive work, office personnel could handle tasks requiring more judgment, decision making, and access to information.

Computer-based office work expanded rapidly as companies assigned more responsibility to lower-level employees. Relatively inexpensive hardware, powerful networks, corporate downsizing, and a move toward employee empowerment also contributed to this trend. Today, administrative assistants and company presidents alike are networked, use computer workstations, and share corporate data to perform their jobs.

Figure 1-14 Slack is a popular groupware application that provides common app integration and unified communication channels for distributed teams.
Source: Slack Technologies, LLC



1.5.6 Digital Assistants

Rapid advances in natural language processing have made a new type of business information system possible: the **personal digital assistant**. These systems are combinations of knowledge management and user productivity systems, enhanced with **artificial intelligence (AI)** and **machine learning** capabilities. They are typically cloud-based and can be embedded in various sizes and types of hardware devices.

Digital assistants are exemplified by Amazon.com's Alexa, Apple's Siri, and Google's Assistant. Users speak to these applications just as they would speak to an actual person. The device replies in a human-sounding voice. These services increase their capabilities over time. They can integrate with other software applications and hardware components, such as devices that control lights at home or the temperature at the office. An image of the Amazon Echo Dot, a smart speaker powered by Alexa, is shown in Figure 1-15.

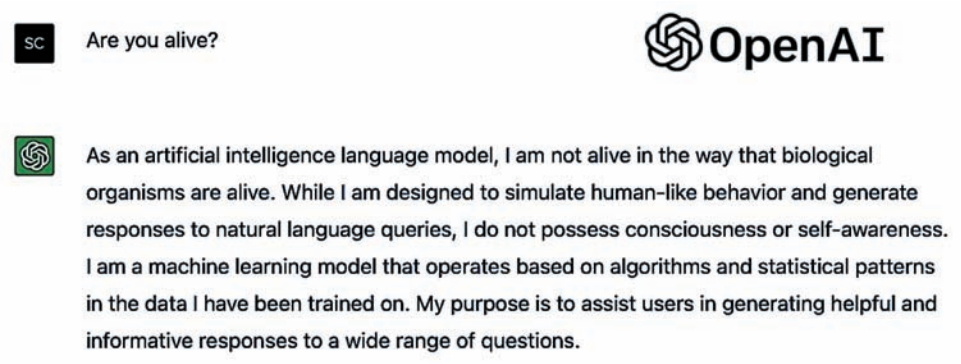
A revolutionary digital assistant has recently stormed onto the scene: generative AI systems based on large language models. ChatGPT, an AI-powered chatbot from OpenAI, best exemplifies this development. "GPT" stands for "Generative Pre-trained Transformer." ChatGPT describes itself as a "natural language processing and machine learning algorithm to understand and respond to a wide range of questions and statements posed to me by users. I have been trained on vast amounts of text data and can generate human-like responses to a variety of prompts." ChatGPT is currently used for various tasks, from code debugging to writing poetry. However, it does hold the promise of supporting the systems analyst in ways that are still hard to predict. Figure 1-16 shows a sample interaction with ChatGPT that is highly philosophical (and somewhat alarming) in nature.

Figure 1-15 Amazon.com's Echo Dot, a digital assistant embedded in a smart speaker powered by Alexa.
ClassyPictures/Shutterstock.com

1.5.7 Systems Integration

Most large companies require systems that combine transaction processing, business support, knowledge management, and user productivity features. For example, suppose an international customer makes a warranty claim. A customer service representative enters the claim into a

Figure 1-16 OpenAI's ChatGPT is at the forefront of AI-powered digital assistants.
Source: OpenAI



TP system, which updates two other systems: a knowledge management system that tracks product problems and warranty activity, and a quality control system with decision support capabilities. A quality control engineer uses what-if analysis to determine if the firm should make product design changes to reduce warranty claims.

Case in Point 1.2

Autonomous Vehicles

Imagine you work for a large automotive company. Your manager asks you to investigate integrating a digital assistant into a new vehicle for the next production year. How would understanding your company's business profile help you complete this task?

1.6 Organizational Information Models

Corporate organizational structure has changed considerably in recent years. To increase productivity, many companies reduced the number of management levels and delegated responsibility to operational personnel. As a result, although modern organizational charts tend to be flatter, most firms still have an organizational hierarchy.

1.6.1 Functions and Organizational Levels

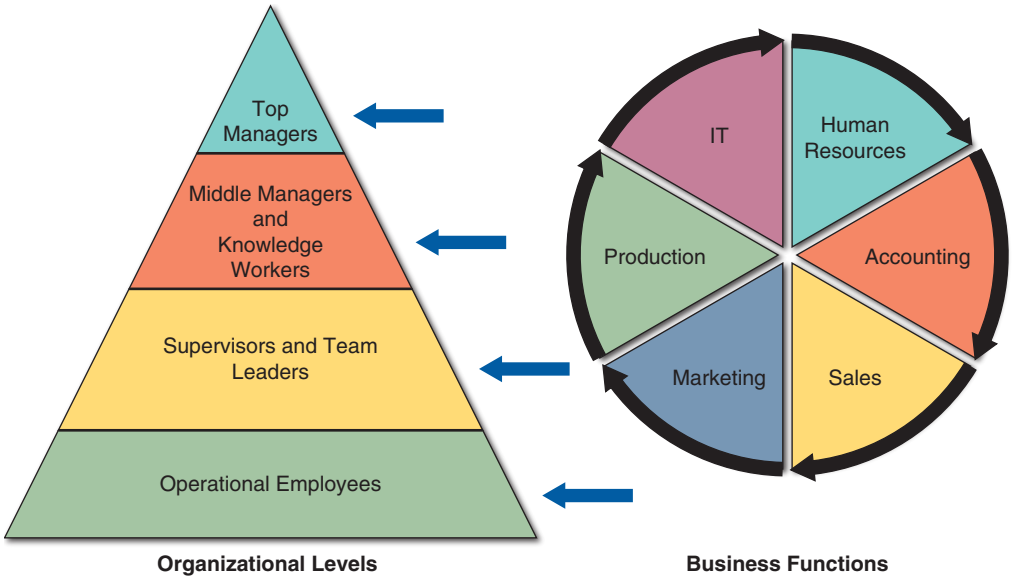
A typical organizational model identifies business functions and organizational levels, as shown in Figure 1-17. Within the functional areas, operational personnel report to supervisors and team leaders. The next level includes middle managers and knowledge workers, who report to top managers. In a corporate structure, the top managers report to a board of directors elected by the company's shareholders.

A systems analyst must understand the company's organizational model to recognize who is responsible for specific processes and decisions and know what information is required by whom.

1.6.2 Top Managers

Top managers develop long-range **strategic plans** defining the company's mission and goals. For example, to plot a future course, top managers ask questions such as "How much should the company invest in information technology?" or "How much will Internet sales grow in the next five years?" or "Should the company build new factories or contract out production functions?"

Figure 1-17 A typical organizational model identifies business functions and organizational levels.



Strategic planning affects the company's future survival and growth, including long-term IT plans. Top managers focus on the overall business enterprise and use IT to set the company's course and direction. To develop a strategic plan, top managers also need information from outside the company, such as economic forecasts, technology trends, competitive threats, and governmental issues.

1.6.3 Middle Managers and Knowledge Workers

Most companies have a layer of middle managers and knowledge workers below the top management level. Middle managers provide direction, necessary resources, and performance feedback to supervisors and team leaders. Because they focus on a shorter time frame, middle managers need more detailed information than top managers but less than supervisors overseeing day-to-day operations. For example, a middle manager might review a weekly sales summary for a three-state area. In contrast, a local sales team leader would need a daily report on customer sales at a single location.

A 2020 article by the Society for Human Resource Management reported that middle managers feel more distressed because they must enforce company policies that employees dislike. Experts imply that managers' dissatisfaction may be attributed to the excessive time devoted to administrative duties, which leaves them with limited time to lead. Adopting technology-based tools can help them shrink their time spent on administrating and free up more time for leading. This insight encourages the development of IT systems better suited to the needs of middle managers.

In addition to middle managers, every company has knowledge workers who support the organization's essential functions. Knowledge workers include systems analysts, programmers, accountants, researchers, trainers, human resource specialists, and other professionals. Knowledge workers also use business support systems, knowledge management systems, and user productivity systems. Just as a military unit requires logistical support, a successful company needs knowledge workers to accomplish its mission.

1.6.4 Supervisors and Team Leaders

Often called team leaders, supervisors oversee operational employees and perform day-to-day functions. They coordinate operational tasks and people, make necessary decisions, and ensure that the right tools, materials, and training are available. Like other managers,

supervisors and team leaders need decision-support information, knowledge management systems, and user productivity systems to carry out their responsibilities.

1.6.5 Operational Employees

Operational employees include users who rely on transaction processing systems to enter and receive data they need to perform their jobs. In many companies, operational users also need information to handle tasks and make decisions previously assigned to supervisors. This trend, called **empowerment**, gives employees more responsibility and accountability. Many companies find that empowerment improves employee motivation and increases customer satisfaction.

1.7 Systems Development

Many options exist for developing information systems, but the most popular alternatives are **structured analysis**, which is a traditional method that still is widely used; **object-oriented (O-O) analysis**, which is a more recent approach that many analysts prefer; and **agile methods**, which include the latest techniques in software development. Table 1-1 provides an overview of the three methods discussed in the following sections.

Although most projects utilize one approach, it is common for systems developers to mix and match methods to gain a better perspective. In addition to these three main development methods, some organizations develop in-house approaches or use techniques offered by software suppliers, tool vendors, or consultants. Many alternatives exist, and IT experts agree that no single development method is best in all cases. An approach that works well for one project might have disadvantages or risks in another situation. The important thing is to understand the various methods and the strengths and weaknesses of each approach.

Regardless of the development strategy, people, tasks, timetables, and costs must be managed effectively. Complex projects can involve dozens of people, hundreds of tasks, and thousands of dollars. **Project management** is the process of planning, scheduling, monitoring, controlling, and reporting on the development of an information system. Project management tools and techniques are described in more detail later in the book.

1.7.1 Structured Analysis

Structured analysis is a traditional systems development technique that is time-tested and easy to understand. Structured analysis uses a series of phases called the **systems development life cycle (SDLC)** to plan, analyze, design, implement, and support an information system. Although structured analysis evolved many years ago, it remains a popular systems development method. Structured analysis is based on an overall plan, similar to a blueprint for constructing a building, so it is called a predictive approach.

Structured analysis uses a set of process models to describe a system graphically. Because it focuses on processes that transform data into useful information, structured analysis is a process-centered technique. In addition to modeling the processes, structured analysis addresses data organization and structure, relational database design, and user interface issues.

A process model shows the data that flows in and out of system processes. Inside each process, input data is transformed by **business rules** that generate the output. Figure 1-18 shows a process model that was created with SmartDraw. The model, which represents an insurance claims system, is called a **data flow diagram (DFD)** because it uses various symbols and shapes to represent data flow, processing, and storage. DFDs are discussed in more detail later in the book.

Structured analysis uses the SDLC to plan and manage the systems development process. The SDLC describes activities and functions that all systems developers perform, regardless

Table 1-1 Comparison of structured, object-oriented, and agile development methods.

	Structured Analysis	Object-Oriented Analysis	Agile Methods
Description	Represents the system in terms of data and the processes that act upon that data. System development is organized into phases, with deliverables and milestones to measure progress. The waterfall model typically consists of five phases: requirements, design, construction, testing, and maintenance & evolution. Iteration is possible among the phases.	Views the system in terms of objects that combine data and processes. The objects represent actual people, things, transactions, and events. Compared to structured analysis, O-O phases tend to be more interactive. Can use the waterfall model or a model that stresses greater iteration.	Stresses intense team-based effort. Breaks development into cycles, or iterations, that add functionality. Each cycle is designed, built, and tested in an ongoing process. Attempts to reduce major risks by incremental steps in short time intervals.
Modeling Tools	Data flow diagrams (DFDs) and process descriptions, which are described in Chapter 5. Also, business process modeling.	Various object-oriented diagrams depict system actors, methods, and messages, which are described in Chapter 6. Also, business process modeling.	Tools that enhance communication, such as collaborative software, brainstorming, and whiteboards. Business process modeling works well with agile methods.
Pros	Traditional method that has been very popular over time. Relies heavily on written documentation. Frequent phase iteration can provide flexibility comparable to other methods. Well-suited to traditional project management tools and techniques.	Integrates easily with object-oriented programming languages. Code is modular and reusable, which can reduce cost and development time. Easy to maintain and expand because new objects can be created using inherited properties.	Very flexible and efficient in dealing with change. Stresses team interaction and reflects a set of community-based values. Frequent deliverables constantly validate the project and reduce risk.
Cons	Changes can be costly, especially in later phases. Requirements are defined early and can change during development. Users might not be able to describe their needs until they can see examples of features and functions.	Somewhat newer method might be less familiar to development team members. Interaction of objects and classes can be complex in larger systems.	Team members need a high level of technical and communications skills. Lack of structure and documentation can introduce risk factors. Overall project might be subject to scope change as user requirements change.

of their approach. In the **waterfall model**, the result of each phase is called a **deliverable**, which flows into the next phase.

Some analysts see a disadvantage in the built-in structure of the SDLC because the waterfall model does not emphasize interactivity among the phases. This criticism can be valid if the SDLC phases are followed too rigidly. However, adjacent phases can and do interact, as shown by the circular arrows in Figure 1-19, and interaction among several phases is not uncommon. Used in this manner, the traditional model is not as different from agile methods as it might appear.

The SDLC model usually includes five steps, which are described in the following sections: systems planning, systems analysis, systems design, systems implementation, and systems operations.

Figure 1-18 This SmartDraw screen shows a process model (DFD) for an insurance claims system.
Source: SmartDraw, LLC

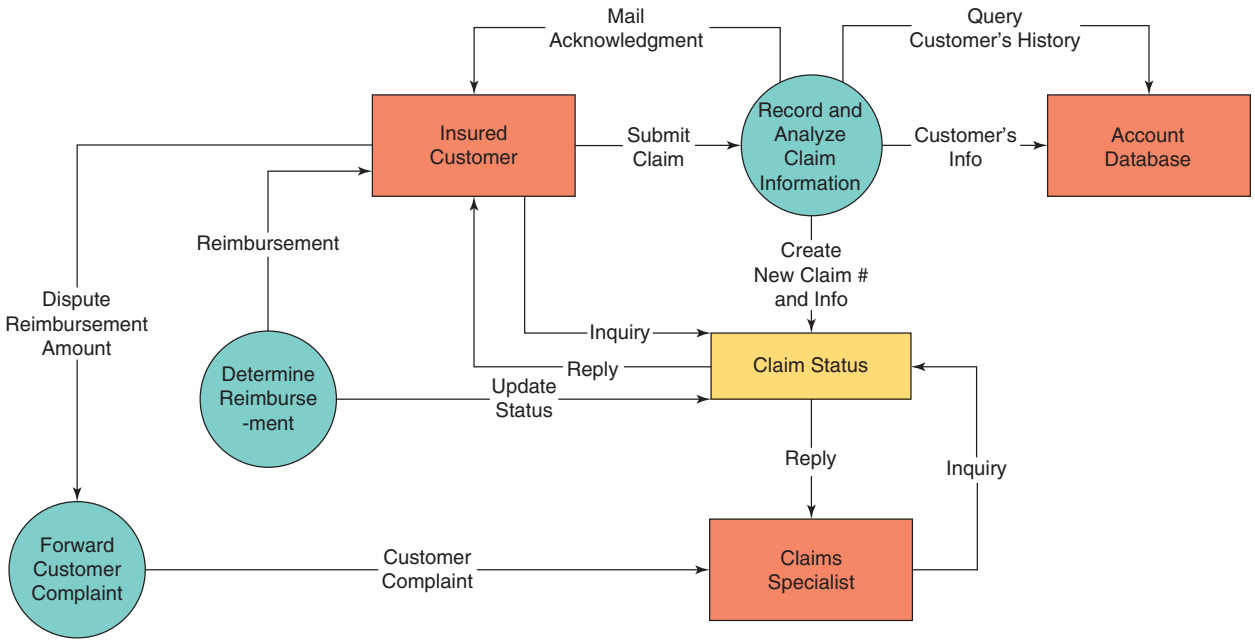
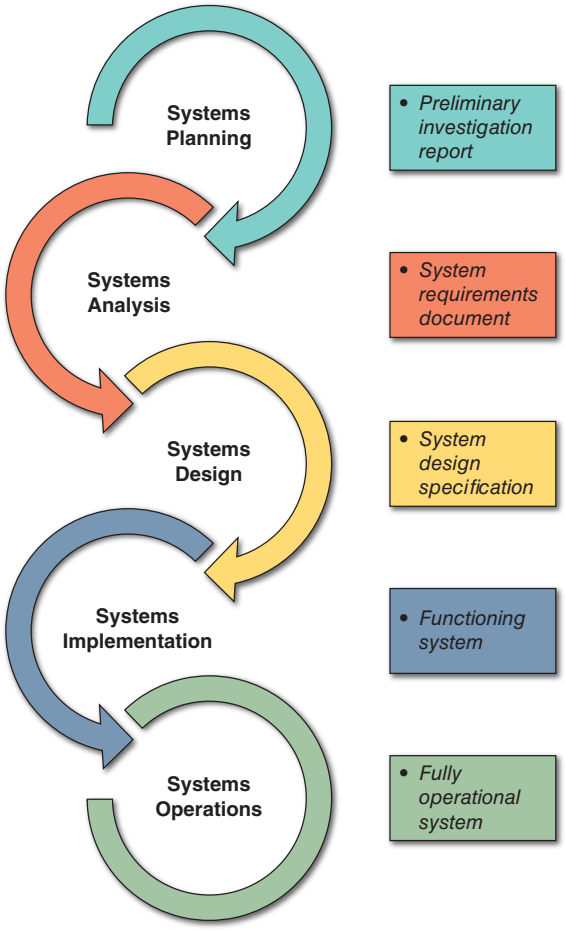


Figure 1-19 A modified waterfall model shows the SDLC phases and deliverables. The circular symbols indicate interaction among the phases.



Systems Planning

The **systems planning phase** usually begins with a formal request to the IT department, called a **systems request**, that describes problems or desired changes in an information system or a business process. In many companies, IT systems planning is integral to overall business planning. When managers and users develop their business plans, they usually include IT requirements that generate systems requests. A systems request can come from a top manager, a planning team, a department head, or the IT department itself. The request can be very significant or relatively minor. For example, a major request might involve a new information system or upgrading an existing system. In contrast, a minor request might ask for a new feature or a change to the user interface.

This phase aims to perform a **preliminary investigation** to evaluate an IT-related business opportunity or problem. The preliminary investigation is critical because the outcome will affect the entire development process. A vital part of the preliminary investigation is a **feasibility study** that reviews anticipated costs and benefits and recommends a course of action based on operational, technical, economic, and time factors.

Suppose a systems analyst receives a request for a system change or improvement. The first step is determining whether it makes sense to launch a preliminary investigation. Before a conclusion can be reached, more information about the business operations may be needed. After an investigation, the systems analyst might determine that the information system functions properly, but users need more training. Sometimes, a business process review rather than an IT solution may be recommended. In other cases, a full-scale systems review may be necessary. If the development process continues, the next step is the systems analysis phase.

Systems Analysis

The **systems analysis phase** aims to build a logical model of the new system. The first step is **requirements modeling**, where the analyst investigates business processes and documents what the new system must do to satisfy users. Requirements modeling continues the investigation that began during the systems planning phase. Next, to understand the system, fact-finding is performed using techniques such as interviews, surveys, document review, observation, and sampling. Finally, the fact-finding results are used to build business, data, process, and object models.

The deliverable for the systems analysis phase is the **system requirements document**. The system requirements document describes management and user requirements, costs, and benefits, and outlines alternative development strategies.

Systems Design

The purpose of the **systems design phase** is to create a physical model that will satisfy all documented requirements for the system. The user interface is designed at this stage, and necessary outputs, inputs, and processes are identified. In addition, internal and external controls are designed, including computer-based and manual features, to guarantee that the system will be reliable, accurate, maintainable, and secure. During the systems design phase, the application architecture is also determined, which programmers will use to transform the logical design into program modules and code.

The deliverable for this phase is the **system design specification**, which is presented to management and users for review and approval. Management and user involvement are critical to understanding what the new system will do, how it will do it, and what it will cost.

Systems Implementation

The new system is constructed during the **systems implementation phase**. Whether the developers use structured analysis or O-O methods, the procedure is the same: programs are written, tested, and documented, and the system is installed. If the system was purchased as a package, systems analysts configure the software and perform any necessary modifications. The systems implementation phase aims to deliver a completely functioning, documented information system. After this phase, the system is ready for use. Final preparations include converting data to the new system's files, training users, and transitioning to the new system.

The systems implementation phase also includes an assessment, called a systems evaluation, to determine whether the system operates correctly and if costs and benefits are within expectations.

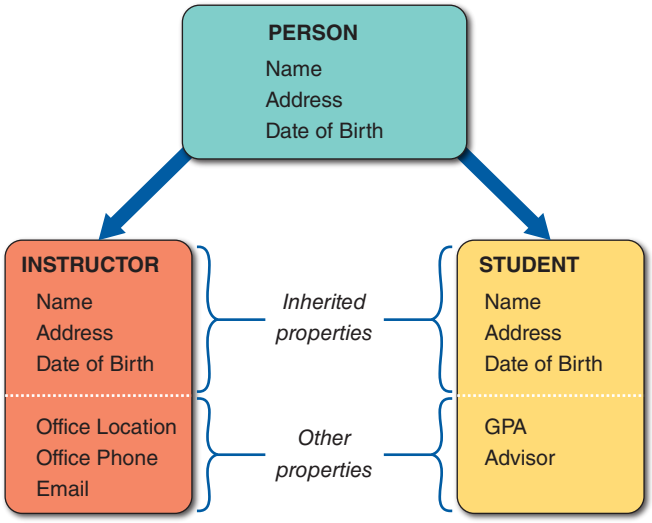
Systems Operations

During the **systems operations phase**, the IT staff transitions the fully functional implemented system to the user and thereafter maintains, enhances, and secures the operational system. Maintenance changes correct errors and adapt to environmental changes, such as new tax rates. Enhancements provide new features and benefits. This phase aims to maximize the return on the IT investment. Security controls safeguard the system from both external and internal threats. A well-designed system must be secure, reliable, maintainable, and scalable. A **scalable** design can expand to meet new business requirements and volumes. Information systems development is always a work in progress. Business processes change rapidly, and most information systems must be updated significantly or replaced after several years of operation. For example, a web-based system may need more servers to cope with the increased workload.

1.7.2 Object-Oriented Analysis

Whereas structured analysis treats processes and data as separate components, object-oriented analysis combines data and the processes that act on the data as **objects**. Systems analysts use O-O to model real-world business processes and operations. The result is software objects

Figure 1-20 The PERSON class includes INSTRUCTOR and STUDENT objects, which have their own properties and inherited properties.



representing actual people, things, transactions, and events. Using an O-O programming language, a programmer then writes the code that creates the objects.

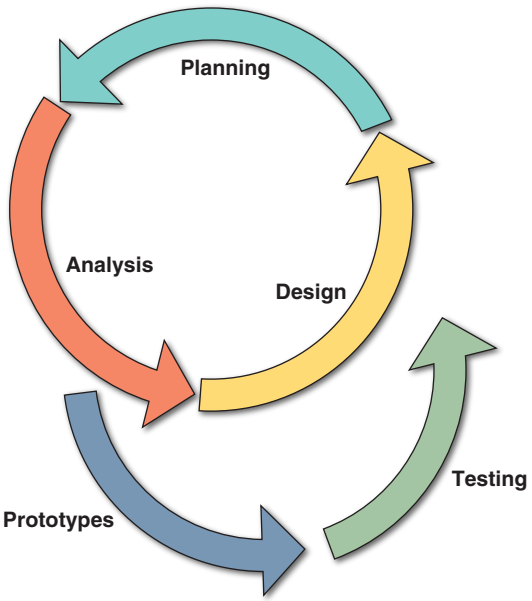
An object is a member of a **class**, which is a collection of similar objects. Objects possess characteristics called **properties**, which the object inherits from its class or possesses on its own. For example, as shown in Figure 1-20, the PERSON class includes INSTRUCTOR and STUDENT. Because the PERSON class has a property called Address, a STUDENT inherits the Address property. A STUDENT also has a property called GPA that other members of the PERSON class do not share.

In O-O design, built-in processes called *methods* can change an object's properties. For example, in an online catalog store, an ORDER object might have a property called STATUS that changes when a CUSTOMER object clicks to place, confirm, or cancel the order.

One object can send information to another object by using a message. A *message* requests specific behavior or information from another object. For example, an ORDER object might send a message to a CUSTOMER object that requests a shipping address. When it receives the message, the CUSTOMER object supplies the information. The ORDER object can send the message, and the CUSTOMER object knows what actions to perform when it receives it. O-O analysis uses object models to represent data and behavior, and to show how objects affect other objects. A systems developer can design reusable components that speed up system implementation and reduce development costs by describing the objects and methods needed to support a business operation.

Object-oriented methods usually follow a series of analysis and design phases similar to those of the SDLC, although there is less agreement on the number of phases and their names. In an O-O model, the phases tend to be more interactive. Figure 1-21 shows an O-O development model where planning, analysis, and design tasks interact to produce prototypes that can be tested and implemented. The result is an interactive model that can accurately depict real-world business processes.

Figure 1-21 In a typical O-O development model, planning, analysis, and design tasks interact continuously to generate prototypes that can be tested.



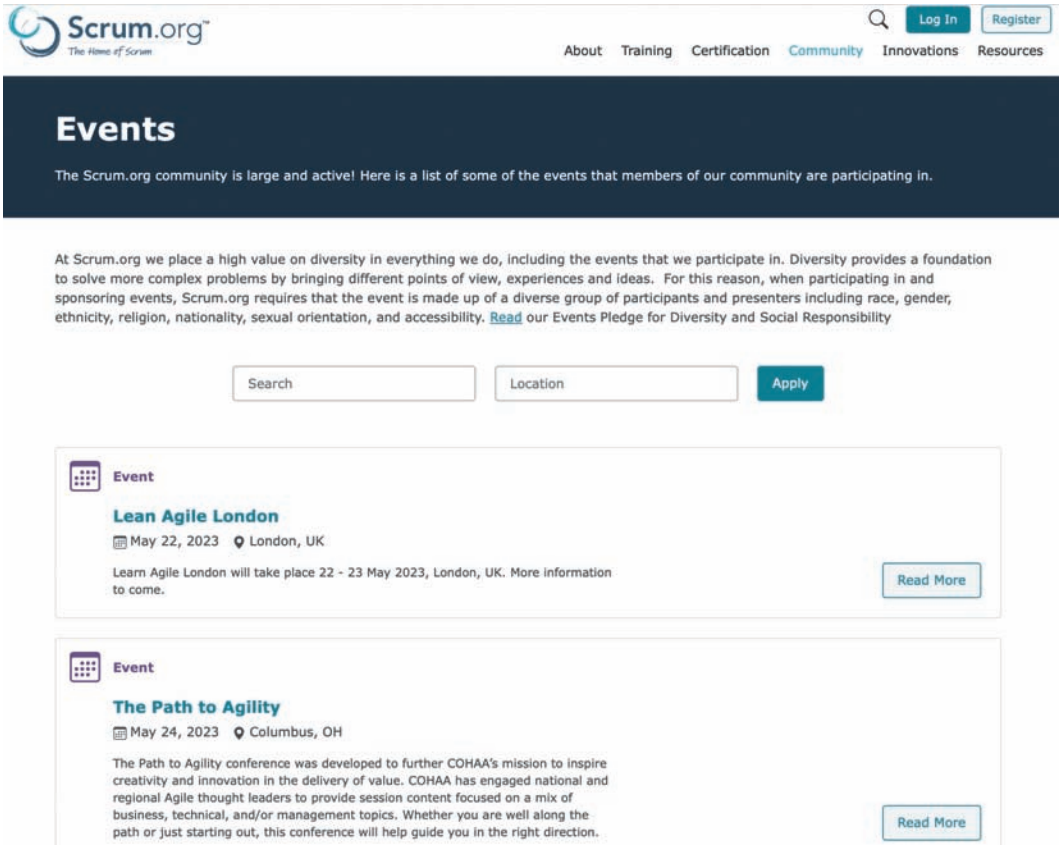
Object-oriented methodology is popular because it provides an easy transition to O-O programming languages such as C++, Java, and Swift. O-O analysis and design, with a detailed description of O-O terms, concepts, tools, and techniques, is covered later in the book.

1.7.3 Agile Methods

Development techniques change over time. For example, structured analysis is a traditional approach, and agile methods are the newest development. Structured analysis builds an overall plan for the information system, just as a contractor might use a blueprint for constructing a building. Agile methods, in contrast, attempt to develop a system incrementally by building a series of prototypes and constantly adjusting them to user requirements. Developers revise, extend, and merge earlier versions into the final product as the agile process continues. An agile approach emphasizes continuous feedback, and each incremental step is affected by what was learned in the previous steps.

Although relatively new to software development, the notion of **iterative** development can be traced back to Japanese auto firms that boosted productivity by using a flexible manufacturing system, where team-based effort and short-term milestones helped keep quality up and costs down. As a result, agile methods have attracted a broad following and an entire community of users, as shown in Figure 1-22.

Figure 1-22 Scrum.org is a popular website supporting the agile community.
Source: Scrum.org



Agile methods typically use a **spiral model**, representing a series of iterations or revisions based on user feedback. As the process continues, the final product gradually evolves. An agile approach requires intense interactivity between developers and individual users and does not begin with an overall objective. Instead, the agile process determines the end result. Proponents of the spiral model believe this approach reduces risks and speeds up software development.

A noted software engineering professor, Barry Boehm, initially suggested spiral models in the 1990s. He stated that each model iteration, or phase, must have a specific goal accepted, rejected, or changed by the user (or client). Thus, each iteration produces feedback and enhancements, which enable the team to reach the overall project goal. Typically, each iteration in a spiral model includes planning, risk analysis, engineering, and evaluation. The repeated iterations produce a series of prototypes, which evolve into the finished system. Notice that these phases resemble SDLC tasks, which also can be iterative.

Numerous other adaptive variations and related methods exist, and most IT developers expect this trend to continue. Two examples are Scrum and Extreme Programming (XP).

Although agile methods are becoming popular, analysts should recognize that these approaches have advantages and disadvantages. By their nature, agile methods can allow developers to be much more flexible and responsive but take more risks than with traditional methods. For example, without a detailed set of system requirements, certain features requested by some users might not be consistent with the company's larger game plan.

Other potential disadvantages of agile methods include weak documentation, blurred lines of accountability, and too little emphasis on the larger business picture. Also, a long series of iterations might add to project cost and development time unless properly implemented. The bottom line is that systems analysts should understand the pros and cons of any approach before selecting a development method for a specific project.

1.7.4 Prototyping

Structured analysis, O-O analysis, and agile methods can all employ prototyping as a supporting systems development method. Prototyping tests system concepts and allows one to examine input, output, and user interfaces before making final decisions. A **prototype** is an early working version of an information system. Just as an aircraft manufacturer tests a new design in a wind tunnel, systems analysts construct and study information system prototypes. For example, wireframes are often used to create mockups of user interfaces.

A prototype can serve as an initial model used as a benchmark to evaluate the finished system, or the prototype itself can be developed into the final version of the system. Either way, prototyping speeds up the development process significantly.

A possible disadvantage of prototyping is that important decisions might be made too early, before business or IT issues are understood thoroughly. However, a prototype based on careful fact-finding and modeling techniques can be an extremely valuable tool.

1.7.5 Tools

All systems development methods must be supported by tools to enable the systems analyst to develop, manage, and maintain large-scale information systems. These tools go by various names, including **application lifecycle management (ALM)**, also called **product lifecycle management (PLM)**; **integrated development environments (IDE)**; and **computer-aided systems engineering (CASE)**, also called **computer-aided software engineering**. **CASE tools** provide a framework for systems development and support various design methodologies, including structured and O-O analyses.

Tools make it easier to build an information system, thereby boosting IT productivity and improving the quality of the finished product. For example, many CASE tools can generate program code after developing a model, which speeds up the implementation process. Figure 1-23 shows the website for Jira, an agile ALM tool from Atlassian. Figure 1-24 shows

Figure 1-23 Jira is an agile application lifecycle management (ALM) tool from Atlassian.
Source: Atlassian

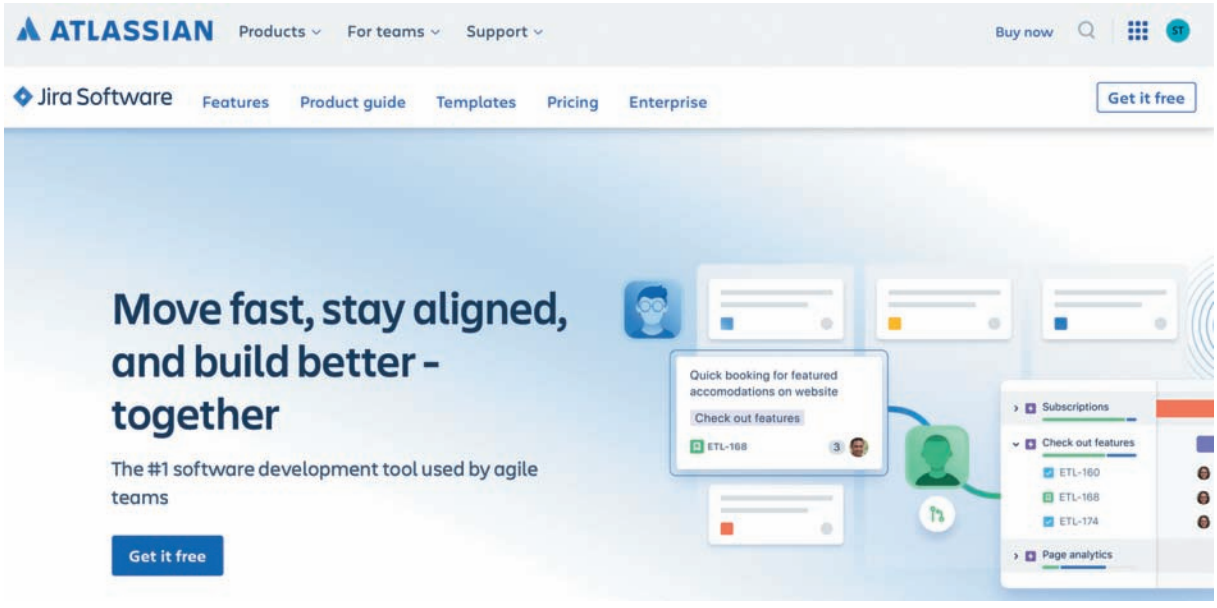
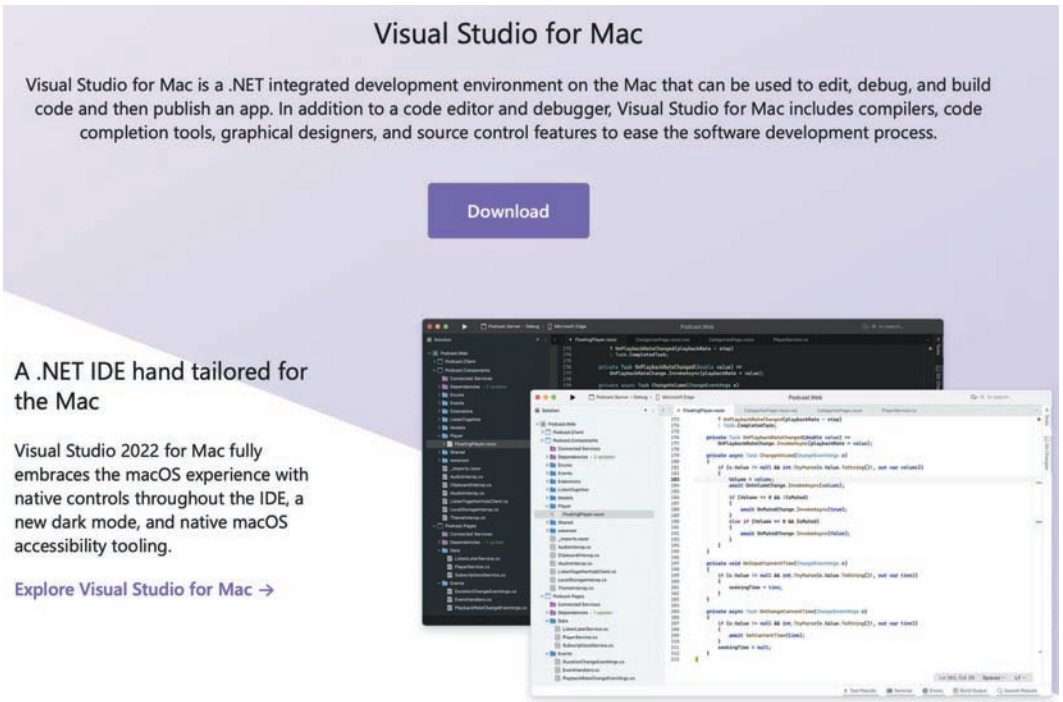


Figure 1-24 Microsoft Visual Studio is a fully featured integrated development environment (IDE).
Source: Microsoft Corporation

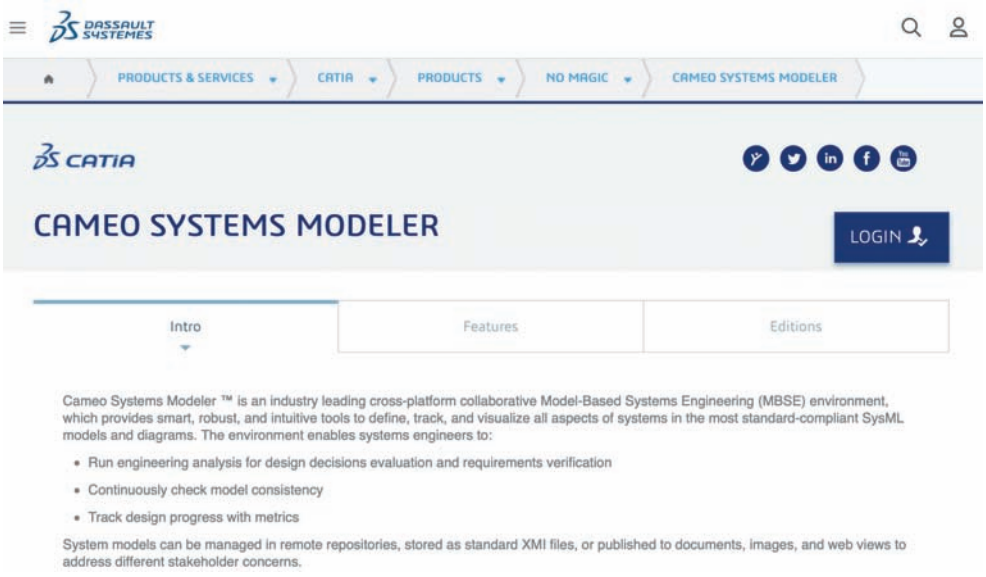


the website for the Mac version of Microsoft Visual Studio, a leading IDE. Figure 1-25 shows the website for Cameo Systems Modeler, a leading MBSE tool.

1.8 The Information Technology Department

The IT department develops and maintains information systems. In addition, the IT group provides **technical support**, which includes seven main functions: application development, systems support and security, user support, database administration, network administration,

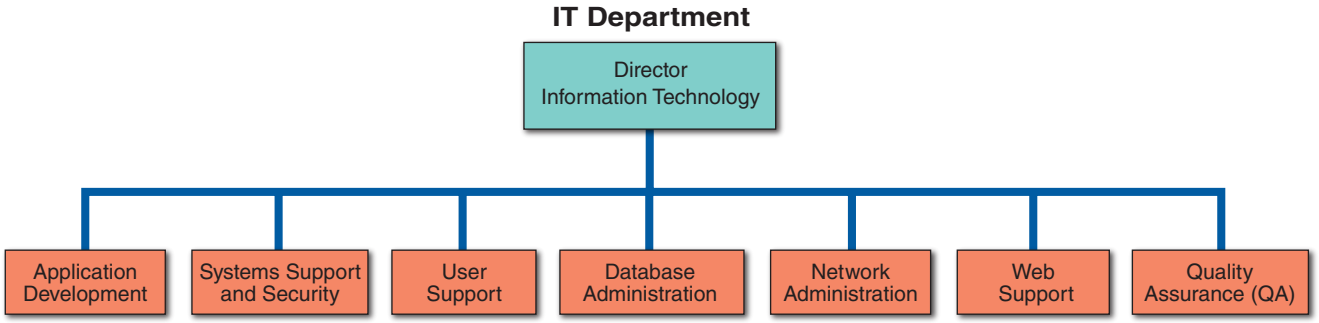
Figure 1-25 Cameo Systems Modeler is a cross-platform, collaborative, model-based systems engineering (MBSE) environment.
Source: Dassault Systèmes



web support, and quality assurance. These functions overlap considerably and often have different names in different companies.

The structure of the IT department varies among companies, as does its name and placement within the organization. For example, one person might handle all computer support activities and services in a small firm. In contrast, a large corporation might require many people with specialized skills to provide information systems support. Figure 1-26 shows a typical IT organization with networked PCs, enterprise-wide databases, centralized processing, and web-based operations.

Figure 1-26 Depending on its size, an IT department might have separate organizational units for these functions, or they might be combined into smaller teams.



1.8.1 Application Development

The IT application development group typically provides leadership and overall guidance, but teams of users, managers, and IT staff members develop the systems themselves. A popular model for information systems development is a project-oriented team with IT professionals providing overall coordination, guidance, and technical support.

Case in Point 1.3

Global Hotels and Momma's Motels

Suppose you work in the IT department of Global Hotels, a multinational hotel chain. Global Hotels runs several specialized business support systems, including a guest reservations system developed in-house to meet a large company's worldwide operations requirements. Guests can make one-stop online reservations by visiting Global's website, which has links to all major travel industry sites.

Global Hotels just acquired Momma's, a regional chain of 20 motels. Momma's uses a vertical reservations package suitable for small- to medium-sized businesses and a generic accounting and finance package. Should Momma's use Global Hotels' information systems or continue with its own? In your answer, consider issues such as business profiles, business processes, system interactivity, and e-commerce. What additional information would be helpful to you in making a recommendation?

1.8.2 Systems Support and Security

Systems support and security provide vital protection and maintenance services for system hardware and software, including enterprise computing systems, networks, transaction processing systems, and corporate IT infrastructure. The systems support and security group implements and monitors physical and electronic security hardware, software, and procedures. This group also installs and supports operating systems, telecommunications software, and centralized database management systems. In addition, systems support and security

technicians provide technical assistance to other groups in the IT department. For example, if a site has many remote clients, the systems support group often includes a deployment team that installs and configures the workstations.

1.8.3 User Support

User support provides users with technical information, training, and productivity support. The user support function usually is called a **help desk**. A help desk's staff trains users and managers on application software such as email, word processing, spreadsheets, and graphics packages. In addition, user support specialists answer questions, troubleshoot problems, and serve as a clearinghouse for user problems and solutions.

1.8.4 Database Administration

Database administration involves data design, management, security, backup, and access. In small- and medium-sized companies, an IT support person performs those roles and other duties. However, regardless of company size, mission-critical database applications require continuous attention and technical support.

1.8.5 Network Administration

Business operations depend on networks that enable company-wide information systems. Network administration includes hardware and software maintenance, support, and security. In addition to controlling user access, network administrators install, configure, manage, monitor, and maintain network applications. Network administration is discussed in more detail later in the book.

1.8.6 Web Support

Web support is a vital technical support function. Web support specialists design and construct webpages, monitor traffic, manage hardware and software, and link web-based applications to the company's information systems. Reliable, high-quality web support is especially critical for companies engaged in e-commerce.

1.8.7 Quality Assurance (QA)

Many large IT departments also use a quality assurance (QA) team that reviews and tests all applications and systems changes to verify specifications and software quality standards. The QA team usually is a separate unit that reports directly to IT management.

1.9 The Systems Analyst

A systems analyst investigates, analyzes, designs, develops, installs, evaluates, and maintains a company's information systems. To perform those tasks, a systems analyst constantly interacts with users and managers within and outside the company. The following sections describe a systems analyst's role, knowledge, skills, education, certifications, and career opportunities.

1.9.1 Role

A systems analyst helps develop IT systems that support business requirements. To succeed, analysts often must act as translators. For example, when describing business processes to programmers, they must speak a language that programmers will understand clearly. Typically, the analyst builds a series of models, diagrams, and decision tables, and uses other descriptive tools and techniques. Similarly, when communicating with managers, the analyst often must translate complex technical issues into words and images that nontechnical people can grasp. Again, the analyst uses various presentation skills, models, and communication methods to do this.

Analysts are often the company's best line of defense against an IT disaster—a system that is technically sound but fails because it does not meet the needs of users and managers. When this occurs, poor communication is usually to blame. For an analyst, the most valuable skill is the ability to listen. An effective analyst will involve users in every step of the development process and listen carefully to what they have to say. Then, as the process continues, the analyst will seek user feedback and comments. This input can provide a valuable early warning system for projects that might otherwise go off track.

1.9.2 Knowledge, Skills, and Education

A successful systems analyst needs technical knowledge, oral and written communication skills, an understanding of business operations, and critical thinking skills. Educational requirements vary widely depending on the company and the position. In a rapidly changing IT marketplace, systems analysts must manage their careers and have a plan for professional development.

Technical Knowledge

State-of-the-art knowledge is vital in a rapidly changing business and technical environment. The Internet offers numerous opportunities to update technical knowledge and skills. Many IT professionals go online to learn about technical developments, exchange experiences, and get answers to questions. For example, the International Council on Systems Engineering (INCOSE), shown in Figure 1-27, is one of the leading organizations offering systems analysts a wealth of information, news, training, support communities, and more. Analysts also maintain their skills by attending onsite and online training courses. Networking with colleagues is another way to keep up with new developments, and membership in professional associations is also important.

Figure 1-27 INCOSE is one of the leading organizations offering systems analysts a wealth of information, news, training, communities, and more.
Source: INCOSE - International Council on Systems Engineering

Communication Skills

A systems analyst needs strong oral and written communication skills, and the ability to interact with people at all levels, from operational staff to senior executives. Often, the analyst must work with people outside the company, such as software and hardware vendors, customers, and government officials. In addition, analysts often coordinate IT project teams, using communication skills to guide and motivate team members.

Business Skills

A systems analyst works closely with managers, supervisors, and operational employees. To be effective, they must understand business operations and processes, communicate clearly, and translate business needs into requirements that can be understood by programmers and systems developers. In addition, to be successful, systems analysts must be business-oriented, curious, comfortable with financial tools, and able to see the big picture.

Critical Thinking Skills

Most educators agree that **critical thinking skills** include the ability to compare, classify, evaluate, recognize patterns, analyze cause-and-effect, and apply logic. Critical thinkers often use a *what-if* approach, and they can assess their thinking and reasoning. Critical thinking skills are valuable in the IT industry, where employers seek job candidates who can demonstrate these skills and bring them to the workplace.

Education

Companies typically require systems analysts to have a college degree in information systems, computer science, or business, and some IT experience usually is needed. For higher-level positions, many companies require an advanced degree. Sometimes, educational requirements can be waived if a candidate has significant experience, skills, or professional certifications.

1.9.3 Certification

Many hardware and software companies offer certification for IT professionals. **Certification** verifies that an individual demonstrated a certain level of knowledge and skill on a standardized test. Certification is an excellent way for IT professionals to learn new skills and gain recognition. Although certification does not guarantee competence or ability, many companies regard certification as essential for hiring or promotion.

In addition to traditional hardware and software certifications, some firms are exploring ways to assess critical thinking skills, such as the Watson–Glaser Critical Thinking Appraisal from Pearson TalentLens, shown in Figure 1-28. These skills include perception, organization, analysis, problem solving, and decision making. Whether or not formal

Figure 1-28 Employers like to hire people who can think logically and effectively.
Source: Pearson

certification is involved, these skills are extremely valuable to IT professionals and the employers who hire them.

1.9.4 Career Opportunities

The demand for systems analysts is expected to remain strong. Companies will need systems analysts to apply new information technology, and the explosion in e-commerce will fuel IT job growth. The systems analyst position is a challenging and rewarding one that can lead to a top management position. With an understanding of technical and business issues, a systems analyst has an unlimited horizon. Many companies have presidents and senior managers who started in IT departments as systems analysts.

The responsibilities of a systems analyst at a small firm differ from those at a large corporation. Working at a small or large company is a matter of personal choice.

Job Titles

First, do not rely on job titles alone. Some positions are called systems analysts but involve only programming or technical support. In other cases, systems analyst responsibilities are found in positions titled computer specialist, programmer, programmer/analyst, systems designer, software engineer, and various others. Be sure the job's duties are clearly stated when considering a position.

Company Organization

Find out everything about the company and where the IT department fits in the organizational chart. Where are IT functions performed, and by whom? A firm might have a central IT group but decentralize the systems development function. This situation sometimes occurs in large conglomerates, where the parent company consolidates information developed and managed at the subsidiary level.

Company Size

A smaller firm might provide more variety. However, a larger company with state-of-the-art systems provides opportunities for specialization. Although there might be more responsibility in a smaller company, the promotional opportunities and financial rewards could be greater in larger companies. Working as an independent consultant is also an option. Many consulting firms have successfully offered their services to smaller business enterprises that do not have the expertise to handle systems development on their own.

Salary, Location, and Future Growth

Consider salary, location, and the company's future growth and success prospects. Initial impressions from employment interviews with the company and its people are important. Most importantly, carefully review short- and long-term goals before deciding on the most suitable position.

Corporate Culture

Besides having goals, methods, and information systems requirements, every firm has an underlying **corporate culture**. Corporate culture is the set of beliefs, rules, traditions, values, and attitudes that define a company and influence its way of doing business. To succeed, a systems analyst must understand the corporate culture and how it affects managing information. Companies sometimes include statements about corporate culture in their mission statements, which are explained later in the book.

A systems analyst must understand the firm's culture and find it comfortable. Work is much more enjoyable if the company encourages personal growth and empowerment. For example, consider the Cisco Systems corporate culture described in Figure 1-29. A company like Cisco Systems will likely attract, retain, and motivate the best and brightest people. As a result, Cisco Systems ranked #1 in *Fortune*'s 2022 list of the country's best places to work.

Figure 1-29 A corporate culture like that of Cisco Systems will likely attract, retain, and motivate the best and brightest people.
Source: Fortune Media IP Limited

1.9.5 Trends in Information Technology

Systems analysts need to track trends in information technology because technological changes affect business operations, career opportunities, and enterprise strategies. Very few areas evolve as fast as information technology. Each year sees evolutionary developments in current technology, such as faster processors, wider network bandwidth, and increased storage capabilities. Occasionally a transformative change occurs, such as the injection of artificial intelligence applications across the enterprise or a revolution in the basic tenets of computation with the nascent introduction of quantum computing.

Key trends disrupting information technology include agile methods, cloud computing, data science, mobile devices, service orientation, and social media networks. These trends can affect education and training needs, give rise to new certifications, and open lucrative career opportunities.

Agile methods have already been discussed in this chapter and are covered in more detail later in the book. The agile movement is a significant trend in information technology that all systems analysts should follow. It started as a response to the heavy-weight process models and was initially used for smaller teams, but interest in agile methods has grown to encompass almost all application areas and business organizations.

Cloud computing is, in many ways, a return to the past: a model of shared computing and data storage resources accessed from remote clients, rather like the mainframe era. However, cloud computing is different because it offers virtualized resources that can grow to accommodate increased requirements as needed. It's also different because buying capacity on demand from a thriving vendor marketplace can reduce the capital costs required for in-house computing infrastructure. Unfortunately, it also presents additional security needs to address.

If cloud computing represents the computational aspect of processing, **data science** represents the data aspect. The world is producing more and more data at an incredible rate. New tools and techniques are being developed to manage these large data sets. Areas such as machine learning and predictive analytics are experiencing tremendous growth as companies try to apply artificial intelligence techniques to the problem of big data. In fact, big data drives many developments related to information technology trends that a systems analyst must follow to stay current.

Mobile devices and the app ecosystem that goes with them are revolutionary developments in information technology. The shift of computing capabilities to "the edge" of the network has put unprecedented power in the hands of end users. Smartphones are the most common computing device on the planet, and systems analysts will need to know about the role of apps and the effect of the **bring your own device (BYOD)** movement on the enterprise.

Traditionally, IT companies were identified as product-oriented or service-oriented. **Product-oriented** firms manufactured computers, routers, or microchips, while **service-oriented** companies included consultants, vendors, software developers, and service providers. Today, those distinctions are gone. Most successful IT companies offer a mix of products, services, and support. Value-added services such as consulting, software, and technical support often are more profitable than hardware sales.

Social media ride on some of the above-mentioned trends, such as cloud computing, data science, mobile devices, and service orientation. However, social media have arguably had the most profound influence on society. Witness the popularity of Facebook, with over 2 billion users worldwide. Companies tap into the power of social media and location-aware apps to create truly innovative solutions that are changing the face of information technology.

Summary

This chapter described how IT combines hardware, software, and services people use to manage, communicate, and share information in today's dynamic business environment. Systems analysis and design is a step-by-step process for developing high-quality information systems. A systems analyst is a valued member of the IT department team who helps plan, develop, and maintain information systems.

A system is a set of related components that produces specific results. An information system has five key components: hardware, software, data, processes, and people. Hardware consists of everything in the physical layer of the information system. Software refers to the programs that control the hardware and produce the desired information or results. Data is the raw material that an information system transforms into useful information. Processes describe the tasks and business functions that users, managers, and IT staff perform to achieve specific results. People who have an interest in an information system are called stakeholders.

Business today is shaped by three significant trends: rapidly increasing globalization, technology integration for seamless information access across various devices such as laptops and smartphones, and the rapid growth of cloud-based computing and software services. Internet-based commerce is called e-commerce, with three prevalent types: B2C, B2B, and C2C.

Systems analysts use modeling to represent business operations and information needs. Modeling produces a graphical representation of a concept or process that systems developers can analyze, test, and modify. MBSE is one of the leading methods systems analysts use to develop information systems.

In the past, IT managers identified a business information system based on its primary users, but now it makes more sense to identify a system by its functions and features. A new set of system definitions includes enterprise computing systems, transaction processing systems, business support systems, knowledge management systems, user productivity systems, digital assistants, and systems integration. Digital assistants, in particular, are experiencing a revolution in capabilities, as exemplified by the AI-powered ChatGPT.

Corporate organizational structure has changed considerably in recent years, resulting in modern organizational charts that are flatter. A typical organizational model identifies business functions and organizational levels. Top managers develop long-range strategic plans defining the company's mission and goals. Middle managers provide direction, necessary resources, and performance feedback to supervisors and team leaders. Often called team leaders, supervisors

oversee operational employees and perform day-to-day functions. Operational employees include users who rely on transaction processing systems to enter and receive data they need to perform their jobs.

The most popular alternatives for developing information systems are structured analysis, object-oriented analysis, and agile methods. Structured analysis is a traditional systems development technique that uses the SDLC to plan, analyze, design, implement, and support an information system. O-O analysis combines data and the processes that act on the data as objects, which systems analysts use to model real-world business processes and operations. Agile methods attempt to develop a system incrementally by building prototypes and constantly adjusting them to user requirements.

The IT department develops and maintains information systems. In addition, the IT group provides technical support, which includes seven main functions: application development, systems support and security, user support, database administration, network administration, web support, and quality assurance. The structure of the IT department varies among companies, as does its name and placement within the organization.

The systems analyst investigates, analyzes, designs, develops, installs, evaluates, and maintains a company's information systems. To perform those tasks, a systems analyst constantly interacts with users and managers within and outside the company. Therefore, a successful systems analyst needs technical knowledge, oral and written communication skills, an understanding of business operations, and critical thinking skills.

Key Terms

agile methods
app
application lifecycle management (ALM)
application software
artificial intelligence (AI)
B2B (business-to-business)
B2C (business-to-consumer)
big data
bring your own device (BYOD)
business model
business process
business process model (BPM)
business process modeling notation (BPMN)
business profile
business rules
business support systems
CASE tools
C2C (consumer-to-consumer)
certification
class
computer-aided software engineering (CASE)
computer-aided systems engineering (CASE)
corporate culture
critical thinking skills
data
data center
data flow diagram (DFD)
data science
deliverable
e-commerce (electronic commerce)
electronic data interchange (EDI)
empowerment
enterprise applications
enterprise computing
enterprise resource planning (ERP)
feasibility study
groupware
hardware
help desk
horizontal system
inference rules
information
information system
information technology (IT)
integrated development environments (IDE)
iterative
knowledge base
legacy system
machine learning
management information system (MIS)
mission-critical system
model-based systems engineering (MBSE)
modeling
Moore's Law
NoSQL databases
object
object-oriented (O-O) analysis
personal digital assistant
preliminary investigation
process
product lifecycle management (PLM)
product-oriented
project management
properties
prototype
radio frequency identification (RFID)
requirements modeling
scalable

service-oriented
software
spiral model
stakeholder
strategic plans
structured analysis
supply chain
supply chain management (SCM)
system
system design specification
system requirements document
system software
systems analysis and design
systems analysis phase
systems analyst
systems design phase
systems development life cycle (SDLC)
systems implementation phase
systems operations phase
systems planning phase
systems request
technical support
transaction processing (TP) systems
user productivity systems
users
vertical system
waterfall model

Review Questions

1. What is information technology, and why is it important to society?
2. What are the five main components of an information system?
3. Explain how ridesharing services such as Uber and Lyft disrupt traditional taxicab business models.
4. Describe the business profile of a home improvement store like Home Depot or Lowe's and how it is used.
5. What are the seven types of information systems used in business?
6. What types of information are needed by the four organizational levels common to many businesses?
7. Compare three systems development methods.
8. Name the tools that enable a systems analyst to develop, manage, and maintain large-scale information systems.
9. Summarize the seven main functions of the IT department.
10. What are the roles and responsibilities of a systems analyst in a modern business?

Discussion Topics

1. Some experts believe that the growth in e-commerce will cause state and local governments to lose tax revenue unless Internet transactions are subject to sales tax. What is one argument that supports this view and one that opposes it?
2. When team members are geographically dispersed, communication becomes more challenging. Explain how groupware can increase user productivity in this context.
3. Under what circumstances should a systems analyst recommend an agile methodology over structured development or object-oriented analysis?
4. Should the IT director report to the company president or someone else? Does it matter?
5. Rapid advancements in areas such as machine learning and predictive analytics in data science are affecting the daily operations of many IT departments. What should a systems analyst do to stay current?

Hands-On Projects

1. Contact three people at your school who use information systems. List their positions, the information they need, the systems they use, and the business functions they perform.
2. Visit three websites to learn more about agile system development. Prepare a list of the sites you visited and a summary of the results.
3. Model-based systems engineering (MBSE) is one of the leading methods systems analysts use to develop information systems. Cameo Systems Modeler is one of the leading tools supporting MBSE. Research magazine articles and the web to learn more about this tool's capabilities. Identify three of its strengths in improving a finished product's quality.
4. Explore the Critical Thinking Community website at criticalthinking.org. Identify three important topics currently being discussed and describe your findings.
5. Compare the corporate culture of three leading IT companies and show how their statements of values could attract (or repel) systems analysts from joining their organizations.

Ethical Issues

You are enjoying your job as a summer intern in the IT department of a local company. At lunch yesterday, several people were discussing ethical issues. You learned that some people belong to IT organizations with ethical codes to guide members and set professional standards. For example, your supervisor belongs to the Association for Computing Machinery (ACM), which has over 100,000 members from more than 100 countries and a website at acm.org. Your supervisor said the ACM code of ethics is important to them and would influence their views. On the other hand, one of the senior programmers believes that their standards would be sufficient to guide them if ethical questions arose.

Because you are excited about your career as an IT professional, you decide to visit ACM's website to examine the code of ethics and make up your own mind. After doing so, would you tend to agree more with your supervisor or the senior programmer?

Chapter

2

Analyzing the Business Case

Learning Objectives

When you finish this chapter, you should be able to:

1. Describe the strategic planning process.
2. Explain how tools can support strategic planning.
3. Explain the concept of a business case.
4. Summarize the six main reasons for systems requests.
5. Describe the two factors affecting systems projects.
6. Explain how systems requests are processed.
7. Explain how systems request feasibility is assessed.
8. Explain how systems requests are prioritized.
9. Conduct a preliminary investigation.

Chapter 2 explains how to analyze a business case. This chapter also explains why it is essential to understand business operations and requirements, how information technology (IT) projects support a company's overall strategic plan, how systems projects get started, and how systems analysts conduct a feasibility study and perform preliminary investigations, which conclude with a management report. Fact-finding techniques that begin at this point and carry over into later development phases are also described.

The chapter includes three "Case in Point" discussion questions to help contextualize the concepts described in the text. The scenario described in the "Ethical Issues" section of the end-of-chapter exercises asks the systems analyst to consider the implications of granting preferential treatment to a system enhancement request when the request originates with a friend of the project's manager.

2.1 Strategic Planning

Strategic planning is identifying long-term organizational goals, strategies, and resources. A strategic plan looks beyond day-to-day activities and focuses on a horizon of three, five, ten, or more years. The IT team must deliver IT resources to support the firm's long-term strategic goals. Therefore, IT managers and systems analysts must understand and participate in strategic planning activities.

IT managers have to prepare for long-range needs, such as a new data warehouse, even as they handle immediate problems, such as a logic bug in the payroll system. In most companies, the IT team reviews each IT-related proposal, project, and systems request to determine if it presents a strong business case or justification.

The following sections provide an overview of strategic planning, a description of SWOT analysis, and a look at the role of the IT department.

2.1.1 Strategic Planning Overview

Strategic planning starts with a **mission statement** that reflects the firm's vision, purpose, and values. Mission statements usually focus on long-term challenges and goals, the importance of the firm's stakeholders, and a commitment to the firm's role as a corporate citizen. For example, Google's mission statement posted on its website is stated succinctly as "to organize the world's information and make it universally accessible and useful."

With the mission statement as a backdrop, a firm develops short-term goals and objectives. For example, the company might establish one-year, three-year, and five-year plans for expanding market share. Next, the company might create a list of shorter-term objectives to achieve those goals. For example, if it wants to increase online orders by 30% next year, a company might set quarterly objectives with monthly milestones. High-priority objectives are called critical success factors. A **critical success factor** must be achieved to fulfill the company's mission.

Objectives also might include tactical plans, such as creating a new website and training a special customer support group to answer email inquiries. Finally, the objectives translate into day-to-day business operations, supported by IT and other corporate resources. The outcome is a set of business results that affect company stakeholders.

Case in Point 2.1

Pets for Rent

Pets for Rent is an innovative company that brings puppies to corporate meetings. The company's CEO says that having puppies in a room with employees in an informal setting helps break down communication barriers and fosters a greater sense of teamwork and comradery. Since this is a new company, the CEO has asked you if a mission statement is necessary. After you review the chapter material, write a brief memo with your views. Be sure to include good (and not-so-good) examples of actual mission statements you find online.

2.1.2 SWOT Analysis

The letters SWOT stand for strengths, weaknesses, opportunities, and threats. A **SWOT analysis** can focus on a specific product or project, an operating division, an entire company, or a mission statement. The aim is to avoid seeking unrealistic, unprofitable, or unachievable goals.

An enterprise SWOT analysis usually begins with these questions:

- What are our strengths, and how can we use them to achieve our business goals?
- What are our weaknesses, and how can we reduce or eliminate them?
- What are our opportunities, and how do we plan to take advantage of them?
- What are our threats, and how can we assess, manage, and respond to the possible risks?

A SWOT analysis examines a firm's technical, human, and financial resources. In Figure 2-1, the bulleted lists show typical strengths, weaknesses, opportunities, and threats for an organization considering expanding its web-based online operations.

As the SWOT process continues, management reviews specific resources, business operations, and valuable assets. For example, suppose that the company owns a vital patent. A SWOT review for the patent might resemble Figure 2-2.

Figure 2-1 A SWOT analysis might produce results similar to those shown here.

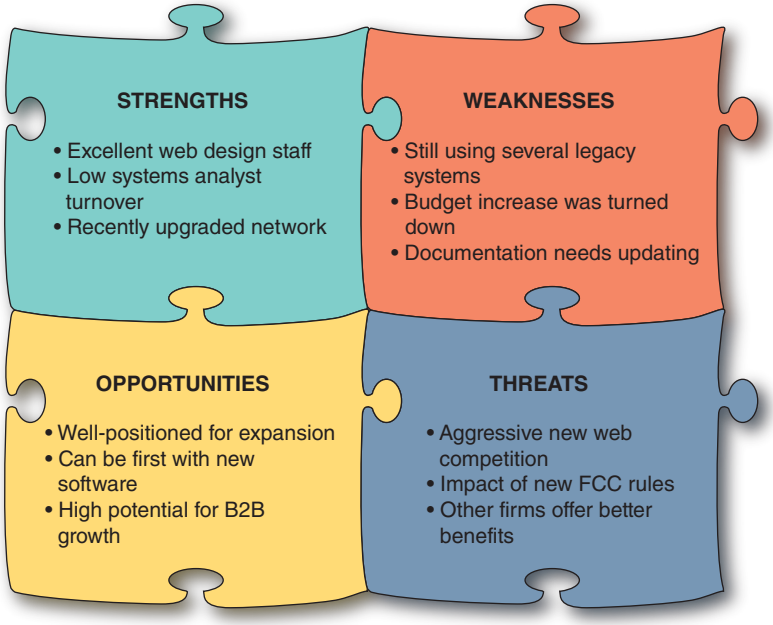
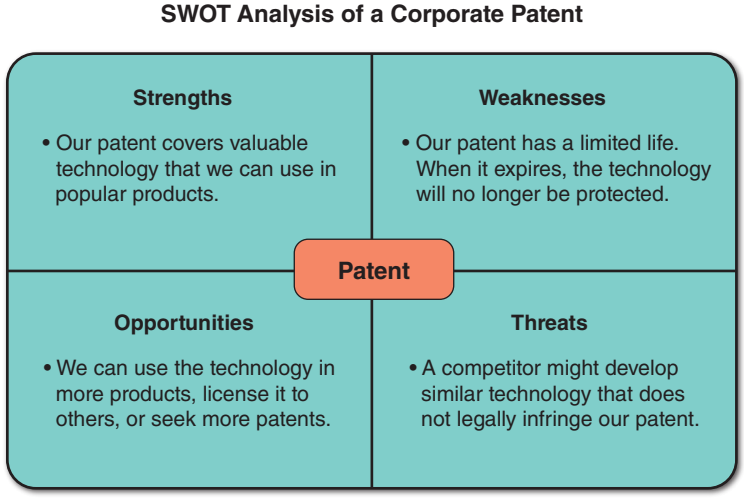


Figure 2-2 This SWOT analysis example focuses on a specific asset: a company patent.



There is no standard approach to strategic planning. Some managers believe a firm's mission statement should be inspirational to its stakeholders. Others feel that unless a firm starts with a realistic SWOT assessment, it might develop an unachievable mission statement. Most companies view the strategic planning process as a dynamic interaction. The company's mission statement reflects a long-term horizon but sets achievable and consistent goals with real-world conditions.

2.1.3 The Role of the IT Department

A systems analyst should be interested in strategic planning because it reflects a higher level of involvement in supporting the project's direction. For example, while working on the same project, one analyst might say, "I am using a CASE tool," while another might say, "I am helping the company succeed in a major new business venture." Systems analysts should focus on the IT department's more significant, strategic role even as they carry out their day-to-day technical tasks.

Experienced analysts know planning is essential for IT project success and must start as early as possible. Careful planning can help ensure that:

- The project supports overall business strategy and operational needs.
- The project scope is well-defined and clearly stated.
- The project goals are realistic, achievable, and tied to specific statements, assumptions, constraints, factors, and other inputs.

During the planning process, management and IT should be closely linked. In the past, a typical IT department handled all aspects of systems development and consulted users only when the department wanted user input. Today, systems development is much more team-oriented. In addition, new approaches to systems development, such as agile methods, typically involve groups of users, managers, and IT staff working together right from the start of the project.

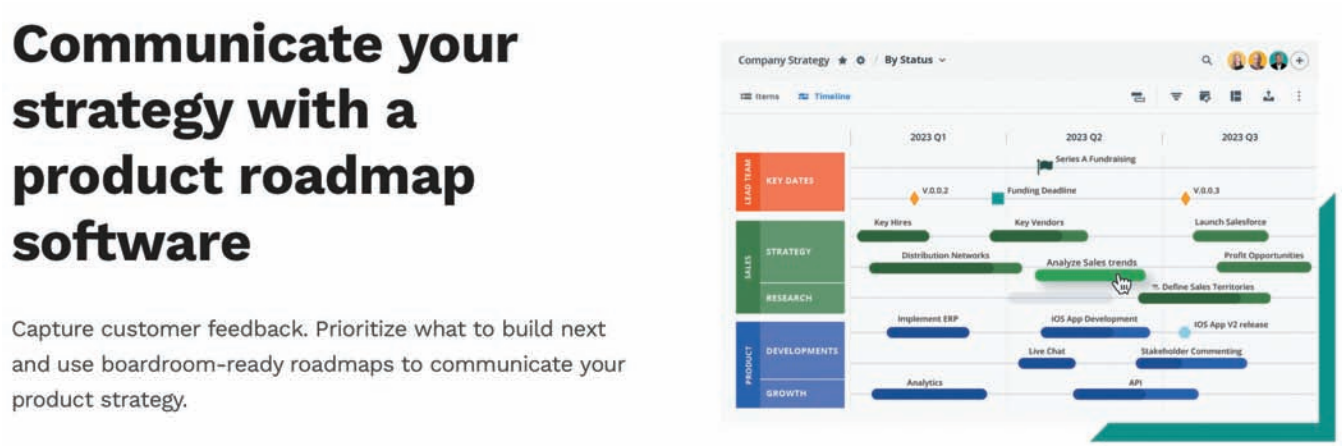
Although team-oriented development is the norm, some companies still see the role of the IT department as a gatekeeper responsible for screening and evaluating systems requests. Should the IT department perform the initial evaluation, or should a cross-functional team do it? The answer probably depends on the company's size, the nature of the system request, and the scale of the project. For example, in smaller companies or firms where only one person has IT skills, that person acts as a coordinator and consults closely with users and managers to evaluate systems requests. Larger firms are likelier to use an evaluation team or a systems review committee.

2.2 Strategic Planning Tools

Irrespective of the development strategy used, many organizations still rely on the IT group to provide guidance when selecting tools to support strategic planning activities. Some analysts stick to traditional text-based methods, using Microsoft Word tables, to provide structure and clarity. Others prefer a spreadsheet, such as Microsoft Excel, because it is easy to display priorities and the relative importance of planning assumptions.

A more sophisticated approach uses a CASE tool to define and document the overall environment. Such devices can integrate various statements, entities, data elements, and graphical models into an overall structure. The result is more consistency, better quality, and much less effort for the systems analyst. Figure 2-3 shows a cloud-based roadmapping software product from Roadmunk that provides integrated support for product strategy visioning, development, and communication.

Figure 2-3 Roadmunk is an end-to-end roadmapping tool that lets you capture customer feedback, prioritize what to build next, and use boardroom-ready roadmaps to communicate your strategy.
Source: Roadmunk



Other strategic planning "tools" are not CASE tools but techniques sometimes supported by software programs. For example, mind maps, balanced scorecards, and gap analysis are all valuable techniques that can be part of strategic planning in an organization. An essential role of the systems analyst is to know when these tools and techniques can best be used in particular project contexts.

2.3 The Business Case

During the systems planning phase, the IT team reviews a request to determine if it presents a strong business case. The term **business case** refers to a proposal's reasons or justification. In addition, the analyst must consider the company's overall mission, objectives, and IT needs to perform the review.

A business case should be comprehensive yet easy to understand. It should clearly describe the project, justify proceeding, and estimate its financial impact. Specifically, the business case should answer questions such as the following:

- Why are we doing this project?
- What is the project about?
- How does this solution address key business issues?
- How much will it cost, and how long will it take?
- Will we suffer a productivity loss during the transition?
- Who will work on the project?
- What is the return on investment and payback period?
- What are the risks of doing the project? What are the risks of not doing the project?
- Which metrics will be used to measure the success of the project?
- What alternatives exist?

Examples of business cases, both good and bad, can be found online. Search for "sample business case" and examine the structure and content of some available samples. Comparing and contrasting business cases for different areas, such as government contracts versus private enterprises, is particularly instructive.

2.4 Systems Requests

The starting point for most information systems projects is called a **systems request**, a formal way of asking for IT support. For example, a systems request might propose enhancements for an existing system, correcting problems, replacing an older system, or developing an entirely new information system to support a company's current and future business needs.

Figure 2-4 Six main reasons for systems requests.

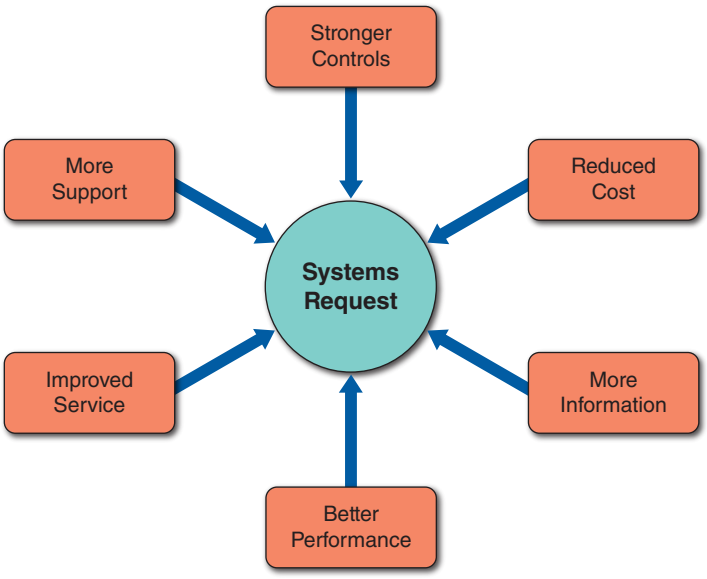


Figure 2-4 shows that the six main reasons for systems requests are stronger controls, reduced cost, more information, better performance, improved customer service, and more support for new products and services.

Stronger Controls

A system must have adequate controls to ensure that data is secure and accurate. This is becoming increasingly important, given the number of data breaches that occur daily. In addition, for specific application areas (e.g., healthcare and financial services), there are standards that the company must follow to ensure compliance with applicable laws.

Some standard security controls include passwords, various levels of user access, and

encryption, or coding data to keep it safe from unauthorized users. In addition, hardware-based security controls include **biometric devices** that can identify a person by a retina scan or by mapping a fingerprint pattern. The technology uses infrared scanners that create images with thousands of measurements of personal physical characteristics, as shown in Figure 2-5, which displays Apple's Face ID security mechanism on the iPhone.

In addition to being secure, data must be accurate. Controls should minimize data entry errors whenever possible. For example, if a user enters an invalid customer number, the order processing system should reject the entry immediately and prompt the user to enter a valid number. Data entry controls must be effective without being excessive. If a system requires users to confirm every item with an "Are you sure? Y/N" message, internal users and customers might complain that the system is not user-friendly.

Figure 2-5 Apple Face ID uses advanced machine learning to recognize users.
Source: Apple Inc.

Reduced Cost

The current system could be expensive to operate or maintain due to technical problems, design weaknesses, or changing business demands. It might be possible to adapt the system to newer technology or upgrade it. On the other hand, a cost–benefit analysis might show that a new system would be more cost-effective and provide better support for long-term objectives.

More Information

The system might produce information that is insufficient, incomplete, or unable to support the company's changing information needs. For example, a system that tracks customer orders might not be capable of analyzing and predicting marketing trends. However, in the face of intense competition and rapid product development cycles, managers need the best information to make significant decisions on planning, designing, and marketing new products and services.

Better Performance

The current system might not meet performance requirements. For example, it might respond slowly to data inquiries at certain times or be unable to support company growth. Performance limitations also occur when a system designed for a specific hardware configuration becomes obsolete when new hardware is introduced.

Improved Service

Systems requests often aim to improve service to customers or users within the company. For instance, allowing mutual fund investors to check their account balances on a website, storing data on rental car customer preferences, or creating an online college registration system provides valuable services and increased customer satisfaction.

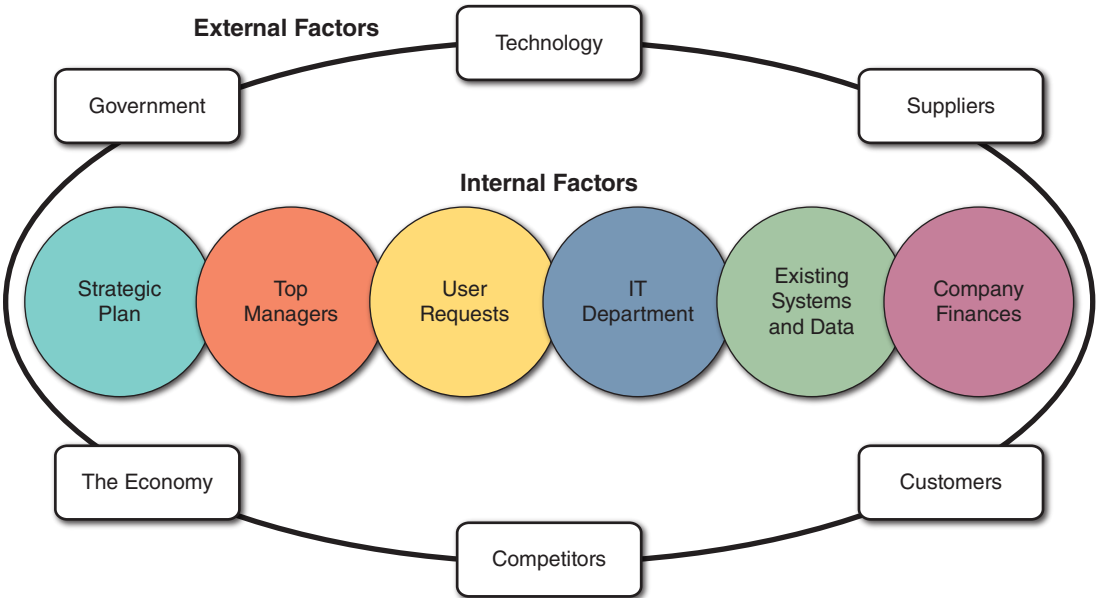
More Support for New Products and Services

New products and services often require new types or levels of IT support. For example, a software vendor might offer an automatic upgrade service for subscribers, or a package delivery company might add a special service for RFID-tagged shipments. In situations like these, additional IT support will likely be required. At the other end of the spectrum, product obsolescence can also be an important factor in IT planning. As new products enter the marketplace, vendors often announce that they will no longer provide support for older versions. A lack of vendor support would be an important consideration in deciding whether or not to upgrade.

2.5 Factors Affecting Systems Projects

Internal and external factors affect every company's business decisions, and IT projects are no exception. Figure 2-6 shows internal and external factors that shape corporate IT choices.

Figure 2-6 Internal and external factors affecting IT projects.



2.5.1 Internal Factors

Internal factors originating from changes in company policies and standards can impact a project. Examples are changes in product standards, quality standards, the company's infrastructure, and the location of resources. Other internal factors include the strategic plan, top managers, user requests, IT department, existing systems and data, and company finances.

Strategic Plan

A company's strategic plan sets its overall direction and impacts IT projects. Company goals and objectives that need IT support will generate systems requests and influence IT priorities. A strategic plan emphasizing technology creates a favorable climate for IT projects throughout the organization.

Top Managers

Because significant resources are required, top management usually initiates large-scale projects. Those decisions often result from strategic business goals that require new IT systems, more information for decision-making processes, or better support for mission-critical information systems.

User Requests

As users rely more heavily on information systems to perform their jobs, they will likely request even more IT services and support. For example, sales reps might ask for improvements to the company's website, a more robust sales analysis report, a network to link all sales locations, or an online system that instantly lets customers obtain their orders' status.

Or, users might not be satisfied with the current system because it is difficult to learn or lacks flexibility. In addition, they might want information systems to support business requirements that did not even exist when the system was first developed.

Information Technology Department

Systems project requests also come from the IT department. IT staff members often make recommendations based on their knowledge of business operations and technology trends. IT proposals might be strictly technical, such as replacing specific network components, or suggestions might be more business oriented, such as proposing a new reporting or data collection system.

Existing Systems and Data

Errors or problems in existing systems can trigger requests for systems projects. Analysts sometimes spend too much time reacting to day-to-day issues with older systems without looking at underlying causes. This approach can turn an information system into a patchwork of corrections and changes that cannot support the company's overall business needs. This problem typically occurs with legacy systems, which are older and less technologically advanced. When migrating to a new system, IT planners must plan the conversion of existing data, as described in detail later in this book.

Company Finances

A company's financial status can affect systems projects. If the company is going through a difficult time, the project may be postponed until more cash is available to finance the effort. On the other hand, if the company is enjoying financial success, the decision to embark on a new project may be easier to make.

2.5.2 External Factors

External factors include technology, suppliers, customers, competitors, the economy, and the government. The **PESTEL framework** can help in assessing these external factors. The framework is based on six environmental types: political, economic, social, technological, environmental, and legal.

Technology

Changing technology is a major force affecting business and society. For example, the rapid growth of telecommunications, increased computing power, and continuous miniaturization of electronic components has created new industries and technologies, including the proliferation of smartphones and the app ecosystem.

Technology also dramatically reshapes existing business operations. For example, the success of scanner technology resulted in universal barcoding that now affects virtually all products. However, some industry experts predict that barcode technology, which is many decades old, will be overshadowed by **electronic product code (EPC)** technology that uses RFID tags to identify and monitor the movement of each product from the factory floor to the retail checkout counter.

Quick response codes (QR codes), as shown in Figure 2-7, are like barcodes but square in shape. They contain more information than traditional barcodes but less than RFID tags. However, they are less expensive than RFID tags and can be printed on almost anything, including online advertisements.

Figure 2-7 QR code.
Source: Amazon.com, Inc.

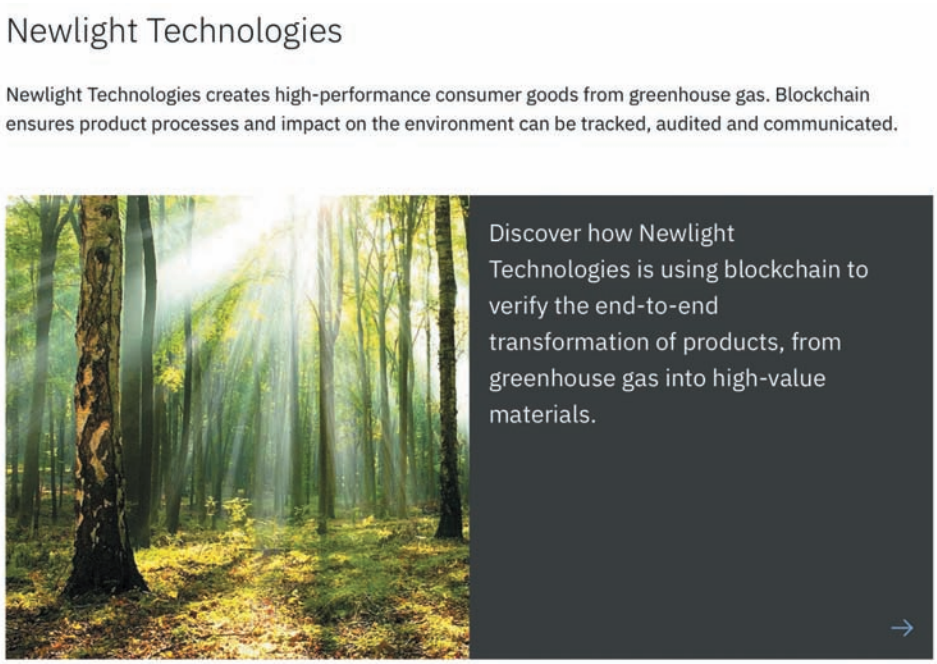
The **Internet of Things (IoT)** is a newer development involving almost all electronic devices communicating over a computer network. The communication can use radio signals, as with RFID tags, digital messages, or other means. IoT devices can be sensors, sending important information to centralized data storage and processing nodes. IoT devices also raise new security and privacy concerns that the systems analyst must consider.

Suppliers

With the growth of **electronic data interchange (EDI)**, relationships with suppliers are critically important. For example, an automobile company might require that suppliers code their parts in a specific manner to match the auto company's inventory control system. EDI also enables **just-in-time (JIT)** inventory systems that rely on computer-to-computer data exchange to minimize unnecessary inventory. The purpose of a JIT system is to provide the right products at the right place and time.

Blockchain technology is a promising mechanism for managing supply chains more powerfully than before. This is because blockchain provides an efficient, secure, and transparent distributed ledger system. Large companies like IBM already use blockchain to improve customer operations, such as Newlight Technologies shown in Figure 2-8. According to the IBM Blockchain "client success stories" website related to sustainability, "Newlight Technologies creates high-performance consumer goods from greenhouse gas. Blockchain ensures product processes and environmental impact can be tracked, audited, and communicated." In other words, IBM's blockchain technology ensures transparency and trust in Newlight's operations.

Figure 2-8 IBM Blockchain used to support sustainability.
Source: IBM



Customers

Customers are vitally important to any business. Information systems that interact with customers usually receive top priority. Many companies implement **customer relationship management (CRM)** systems that integrate all customer-related events and transactions, including marketing, sales, and customer service activities. Vendor-oriented CRM systems often interconnect with supply chain management (SCM) systems. CRM

components can provide automated responses to sales inquiries, online order processing, and inventory tracking. Some suppliers use robots for order fulfillment, such as the Robotic Drive Units from Amazon Robotics that Amazon.com uses in their warehouses.

Another RFID application is called **electronic proof of delivery (EPOD)**. Using EPOD, a supplier uses RFID tags on each crate, case, or shipping unit to create a digital shipping list. The customer receives the list and scans the incoming shipment. If a discrepancy is detected, it is reported and adjusted automatically. Minor shipping inconsistencies might not otherwise be traced because they would be expensive to investigate manually. This is an example of technology-related cost control.

Competitors

Competition drives many information systems decisions. For example, if one cellular telephone provider offers a new digital service, other firms must match the plan to remain competitive. In addition, new product research and development, marketing, sales, and service require IT support.

The Economy

Economic activity has a powerful influence on corporate information management. In a period of economic expansion, firms must be ready with scalable systems that can handle additional volume and growth. However, predicting the business cycle is not an exact science, and careful research and planning are essential.

Government

Federal, state, and local government regulations directly affect the design of corporate information systems. For example, up-to-date IRS reporting requirements must be designed into a payroll package.

2.6 Processing Systems Requests

In most organizations, the IT department receives more systems requests than it can handle. As a result, many organizations assign responsibility for evaluating systems requests to a group of key managers and users. Many companies call this group a **systems review committee** or **computer resources committee**. Regardless of the name, the objective is to use several analysts' combined judgment and experience to evaluate project requests.

2.6.1 Systems Request Forms

Many organizations use a special form for systems requests, similar to the online example for tracking requests shown in Figure 2-9. A properly designed form streamlines the request process and ensures consistency. The form must be easy to understand and include clear instructions. It should have enough space for all required information and indicate what supporting documents are needed. Most companies use online systems request forms that users submit electronically because the form can be processed automatically.

When a systems request form is received, a systems analyst or IT manager examines it to determine what IT resources are required for the preliminary investigation. A designated person or a committee then decides whether to proceed with a preliminary investigation. Sometimes, a situation requires an immediate response. For example, if the problem involves a mission-critical system, an IT maintenance team must restore normal operations immediately. When the system functions correctly, the team reviews and prepares a systems request to document the work performed.

Figure 2-9 Example of an online systems request form.
Source: Florida Institute of Technology

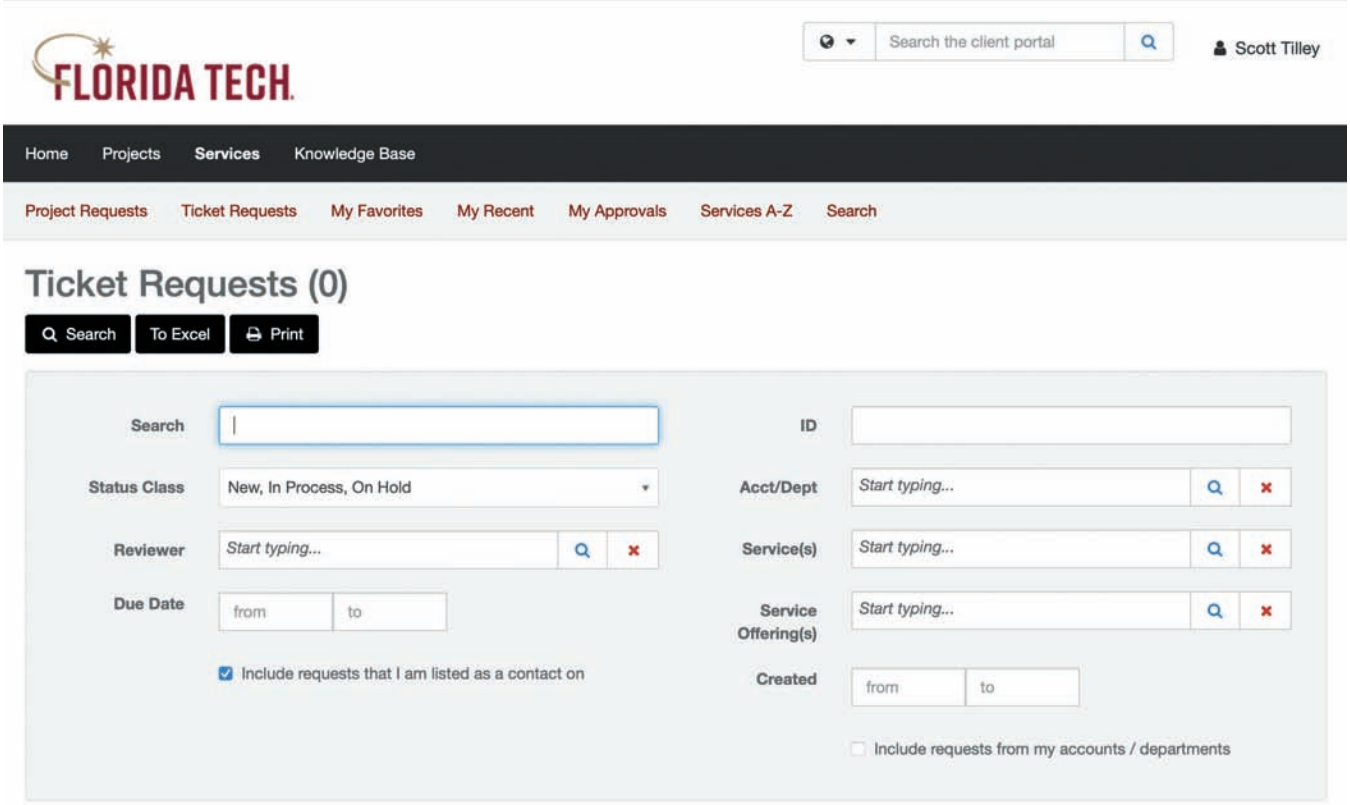
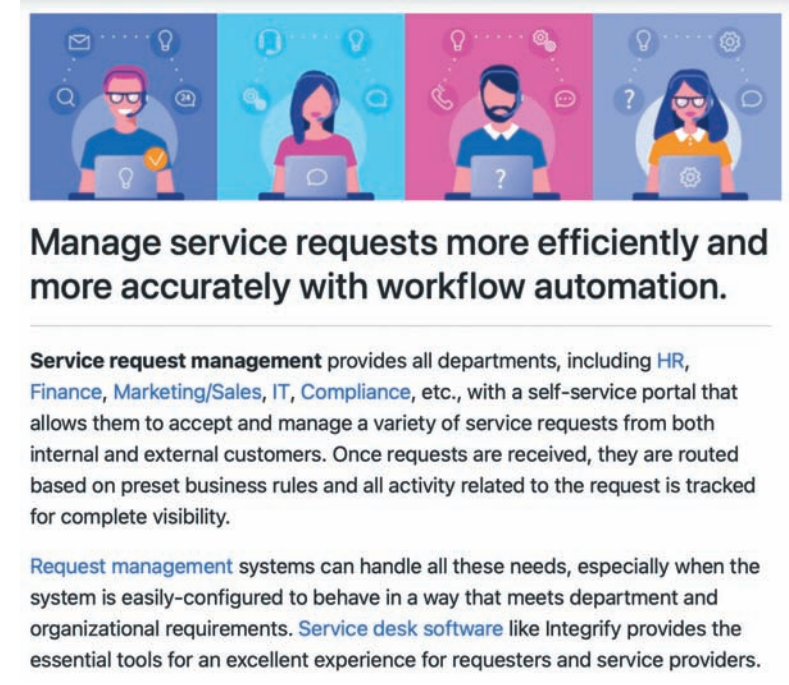


Figure 2-10 A service request management system from Integrify.
Source: Irina Strelnikova/Shutterstock.com and integrify.com



2.6.2 Systems Request Tools

When the number of requests submitted through automated forms becomes significant, or if requests originate from internal sources and external customers, special-purpose systems request tools can help manage the workflow. For example, Figure 2-10 illustrates the service request capabilities from Integrify that capture, manage, and route requests to systems analysts based on definable business rules. In this way, requests are tracked and analyzed for improved performance.

2.6.3 Systems Review Committee

Most large companies use a systems review committee to evaluate systems requests. Instead of relying on a single individual, a committee approach provides a variety of experiences and knowledge. In addition, with a broader viewpoint, a committee can establish priorities more effectively than an individual, and one person's bias is less likely to affect the decisions.

A typical committee comprises the IT director and several managers or representatives from other departments. In addition, the IT director usually serves as a technical consultant to ensure that committee members are aware of crucial issues, problems, and opportunities.

Although a committee offers many advantages, some disadvantages exist. For example, action on requests must wait until the committee meets. Another potential drawback of a committee is that members might favor projects requested by their departments, and internal political differences could delay important decisions.

Many smaller companies rely on one person to evaluate systems requests instead of a committee. However, suppose only one person has the necessary IT skills and experience. In that case, that person must consult closely with users and managers throughout the company to ensure that business and operational needs are considered carefully.

Whether one person or a committee is responsible, the goal is to evaluate the requests and set priorities. For example, suppose four requests must be reviewed:

1. The marketing group wants to analyze customer spending habits and forecast future trends.
2. The technical support group wants a cellular link so service representatives can instantly download technical data.
3. The accounting department wants to redesign customer statements and allow Internet access.
4. The production staff wants an inventory control system to exchange data with major suppliers.

Which projects should the firm pursue? What criteria should be applied? How should priorities be determined? To answer those questions, the individual or the committee must assess the feasibility of each request.

Case in Point 2.2

Attaway Airlines, Part One

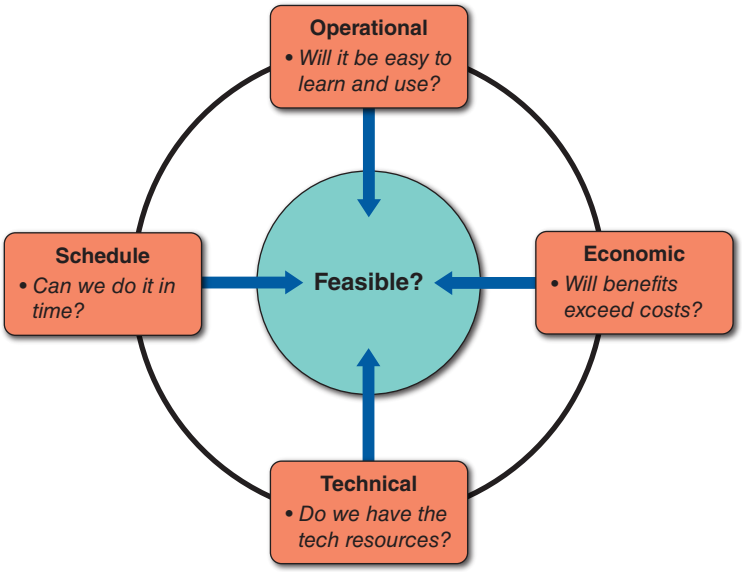
You are the IT director at Attaway Airlines, a small regional air carrier. You chair the company's systems review committee and deal with strong disagreements about two key projects. The marketing manager says it is vital to have a new computerized reservation system that can provide better customer service and reduce operational costs. The vice president of finance is equally adamant that a new accounting system is needed immediately because it will be costly to adjust the current system to new federal reporting requirements. The VP outranks the marketing manager, and the VP is your boss. The next meeting is set for 9:00 a.m. tomorrow. How will you prepare for the vigorous discussions likely to occur at the meeting? What questions and issues should be raised?

2.7 Assessing Request Feasibility

As described earlier, a systems request must pass several tests to see whether it is worthwhile to proceed further. The first step is to identify and eliminate systems requests that are not feasible. For example, a request would not be feasible if it required hardware or software that the company already had rejected.

Even if the request is feasible, it might not be necessary. For example, requesting multiple report versions (e.g., a spreadsheet and a graphics series) could require considerable design and programming effort. A better alternative might be downloading the server data to a personal computer–based software package and showing users how to produce their own reports. In this case, training users would be a better investment than making reports for them.

Figure 2-11 A feasibility study examines operational, technical, economic, and schedule factors.



Sometimes, assessing request feasibility is simple and can be done in a few hours. If the request involves a new system or a significant change, however, extensive fact-finding and investigation in the form of feasibility studies are required.

2.7.1 Feasibility Studies

As shown in Figure 2-11, a feasibility study uses four main yardsticks to measure a proposal: operational feasibility, economic feasibility, technical feasibility, and schedule feasibility.

How much effort should go into a feasibility study depends on the nature of the request. For example, if a department wants an existing report sorted in a different order, the analyst can decide quickly whether the request is feasible. On the other hand, a proposal by the marketing department for a new market research system to predict sales trends would require much more effort. In either case, the systems analyst should ask these important questions:

- Is the proposal desirable in an operational sense? Is it a practical approach that will solve a problem or take advantage of an opportunity to achieve company goals?
- Is the proposal technically feasible? Are the necessary technical resources and people available for the project?
- Is the proposal economically desirable? What are the projected savings and costs? Are other intangible factors involved, such as customer satisfaction or company image? Is the problem worth solving, and will the request result in a sound business investment?
- Can the proposal be accomplished within an acceptable time frame?

To obtain more information about a systems request, initial fact-finding might be accomplished by studying organizational charts, performing interviews, reviewing current documentation, observing operations, and surveying users. Sometimes, developing prototypes can provide additional insight into the feasibility of the request. If the systems request is approved, more intensive fact-finding will continue during the systems analysis phase.

2.7.2 Operational Feasibility

Operational feasibility means that a proposed system will be used effectively after it has been developed. If users struggle with a new system, it will not produce the expected benefits. Organizational culture can also affect operational feasibility. For instance, a system that works well in a highly structured workplace might be unpopular in a more relaxed corporate culture. Operational feasibility is difficult to measure precisely but must be studied very carefully. The following questions would help predict a system's operational feasibility:

- Does management support the project? Do users support the project? Is the current system well-liked and effectively used? Do users see the need for change?
- Will the new system result in a workforce reduction? If so, what will happen to the affected employees?
- Will the new system require training for users? If so, is the company prepared to provide the necessary resources for training current employees?

- Will users be involved in planning the new system right from the start?
- Will the new system place new demands on users or require any operational changes? For example, will any information be less accessible or produced less frequently? Will performance decline in any way? If so, will an overall gain to the organization outweigh individual losses?
- Will customers experience adverse effects in any way, either temporarily or permanently?
- Will any risk to the company's image or goodwill result?
- Does the development schedule conflict with other company priorities?
- Do legal or ethical issues need to be considered? For example, protecting customer data privacy (whether explicit in law or expected by the customer) has become paramount.

2.7.3 Economic Feasibility

Economic feasibility means that the projected benefits of the proposed system outweigh the estimated costs, usually considered the **total cost of ownership (TCO)**, which includes ongoing support and maintenance costs as well as acquisition costs. To determine TCO, the analyst must estimate costs in each of the following areas:

- People, including IT staff and users
- Hardware and equipment
- Software, including in-house development as well as purchases from vendors
- Formal and informal training, including peer-to-peer support
- Licenses and fees
- Consulting expenses
- Facility costs
- The estimated cost of not developing the system or postponing the project

Tangible costs, such as those listed above, can usually be measured in dollars. But **intangible costs** also must be considered. For example, low employee morale might not have an immediate dollar impact but certainly will affect the company's performance.

In addition to costs, tangible and intangible benefits to the company must be assessed. The systems review committee will use those figures and cost estimates to decide whether to pursue the project beyond the preliminary investigation phase.

Tangible benefits are benefits that can be measured in dollars. Tangible benefits result from a decrease in expenses, an increase in revenues, or both. Examples of tangible benefits include the following:

- A new scheduling system that reduces overtime
- An online package tracking system that improves service and decreases the need for clerical staff
- A sophisticated inventory control system that cuts excess inventory and eliminates production delays

Intangible benefits are advantages that are difficult to measure in dollars but are important to the company. Examples of intangible benefits include the following:

- A user-friendly system that improves employee job satisfaction
- A sales tracking system that supplies better information for marketing decisions
- A new website that enhances the company's image

The development timetable must also be considered because some benefits might occur when the system is operational, but others might only arise later.

2.7.4 Technical Feasibility

Technical feasibility refers to the technical resources needed to develop, purchase, install, or operate the system. When assessing technical feasibility, an analyst should consider the following points:

- Does the company have the necessary hardware, software, and network resources? If not, can those resources be acquired without difficulty?
- Does the company have the needed technical expertise? If not, can it be acquired?
- Does the proposed platform have sufficient capacity for future needs? If not, can it be expanded?
- Will a prototype be required?
- Will the hardware and software environment be reliable? Will it integrate with other company information systems now and in the future? Will it interface properly with external systems operated by customers and suppliers?
- Will the combination of hardware and software supply adequate performance? Do clear expectations and performance specifications exist?
- Will the system be able to handle future transaction volume and company growth?

Remember that systems requests not technically feasible can be resubmitted as new hardware, software, or expertise becomes available. Development costs might decrease, or the value of benefits might increase enough that a systems request eventually becomes feasible. Conversely, an initially feasible project can be rejected later.

2.7.5 Schedule Feasibility

Schedule feasibility means a project can be implemented in an acceptable time frame. A systems analyst must consider the interaction between time and costs when assessing schedule feasibility. For example, speeding up a project schedule might make a project feasible but much more expensive.

Other issues that relate to schedule feasibility include the following:

- Can the company or the IT team control the factors that affect schedule feasibility?
- Has management established a firm timetable for the project?
- What conditions must be satisfied during the development of the system?
- Will an accelerated schedule pose any risks? If so, are the risks acceptable?
- Will project management techniques be available to coordinate and control the project?
- Will a project manager be appointed?

Various project management tools (e.g., Microsoft Project) and techniques (e.g., Gantt charts and network diagrams) are described later in the book.

2.8 Setting Priorities

After rejecting systems requests that are not feasible, the systems review committee must establish priorities for the remaining items. If tools are used as part of the review process, the requests may already be in a partially or fully sorted order. The highest priority goes to project requests that provide the most significant benefit, at the lowest cost, in the shortest time. However, many factors influence project evaluation. More details on project tasks and priorities are discussed later in the book.

2.8.1 Dynamic Priorities

It's important to note that many projects are dynamic in nature. For example, projects that have adopted an agile methodology are prone to rapid changes throughout the SDLC. These changes can cause request priorities to change as well.

For example, acquisition costs might increase over time, making the project more expensive than anticipated. This can affect the economic feasibility of several requests. In addition, managers and users sometimes lose confidence in a project. For all those reasons, feasibility analysis and priority setting are ongoing tasks that must be performed throughout the systems development process.

2.8.2 Factors That Affect Priority

When assessing a project's priority, a systems analyst should consider the following:

- Will the proposed system reduce costs? Where? When? How? By how much?
- Will the system increase revenue for the company? Where? When? How? By how much?
- Will the systems project result in more information or produce better results? How? Are the results measurable?
- Will the system serve customers better?
- Will the system serve the organization better?
- Can the project be implemented in a reasonable time frame? How long will the results last?
- Are the necessary financial, human, and technical resources available?

Few projects will score high in all areas. Some proposals might not reduce costs but will provide important new features. Other systems might reduce operating costs substantially but require the purchase or lease of additional hardware. Finally, some systems might be desirable but require several years of development before producing significant benefits.

The analyst should use tangible costs and benefits that can be measured in dollars whenever possible. However, the proposal might involve intangible benefits, such as enhancing the organization's image, raising employee morale, or improving customer service. These examples are harder to measure but should also be considered.

2.8.3 Discretionary and Nondiscretionary Projects

Projects where management has a choice in implementing them are called **discretionary projects**. Projects where no choice exists are called **nondiscretionary projects**. For example, creating a new report for a user is an example of a discretionary project; adding a report required by a new federal law is an example of a nondiscretionary project.

If a project is not discretionary, the systems analyst should ask if the systems review committee must evaluate it. Some people believe that waiting for committee approval delays critical nondiscretionary projects unnecessarily. Others believe submitting all requests to the systems review committee makes it aware of all projects competing for IT resources. As a result, the committee can review priorities and create realistic schedules.

Many nondiscretionary projects are predictable. Examples include annual updates to payroll, tax percentages, or quarterly changes in reporting requirements for an insurance processing system. By planning for predictable projects, the IT department manages its resources better and keeps the systems review committee fully informed without needing prior approval in every case.

Case in Point 2.3

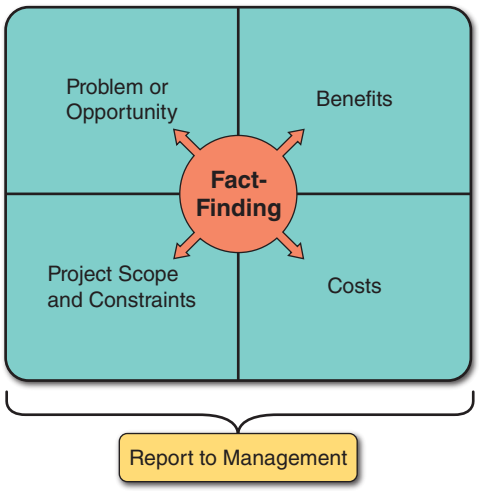
Attaway Airlines, Part Two

Back at Attaway Airlines, the morning meeting ended with no agreement between the VP of finance and the marketing manager. In fact, a new issue arose. The VP now says the new accounting system is entitled to the highest priority because the federal government will soon require reporting certain types of company-paid health insurance premiums. Because the current system will not handle this report, the VP insists that the entire accounting system is a nondiscretionary project. As you might expect, the marketing manager is upset. Can part of a project be nondiscretionary? What issues need to be discussed? The committee meets again tomorrow, and the members will look to you, as the IT director, for guidance.

2.9 The Preliminary Investigation

A systems analyst conducts a **preliminary investigation** to study the systems request and recommend specific action. After obtaining authorization to proceed, the analyst interacts with managers, users, and other stakeholders, as shown in the model in Figure 2-12. The analyst gathers facts about the problem or opportunity, project scope and constraints, project benefits, and estimated development time and costs. The end product of the preliminary investigation is a management report.

Figure 2-12 Model of a preliminary investigation. Note the importance of fact-finding in each of the four areas.



2.9.1 Planning the Preliminary Investigation

Before starting a preliminary investigation, it is important to inform people about the investigation and explain the role of the systems analyst. Meetings with key managers, users, and other stakeholders, such as the IT staff, should be scheduled to describe the project, explain roles and responsibilities, answer questions, and invite comments. Interactive communication with users starts at this point and continues throughout development.

A systems project often produces significant changes in company operations. Employees may be curious, concerned, or even opposed to those changes. Therefore, it is common to encounter some user resistance during a preliminary investigation. Employee attitudes and reactions are essential and must be considered.

When interacting with users, use the word *problem* carefully because it has a negative meaning. When users are asked about *problems*, some emphasize current system limitations rather than desirable new features or enhancements. Instead of focusing on difficulties, question users about additional capabilities they would like. This approach highlights ways to improve the user's job, provides a better understanding of operations, and builds better, more positive relationships with users.

2.9.2 Performing the Preliminary Investigation

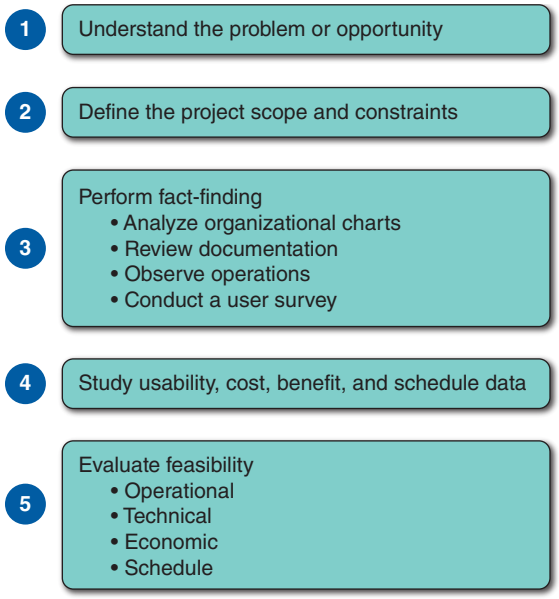
A systems analyst typically follows a series of steps during a preliminary investigation, as shown in Figure 2-13. The exact procedure depends on the nature of the request, the size of the project, and the degree of urgency.

Step 1: Understand the Problem or Opportunity

Suppose the systems request involves a new information system or a substantial change in an existing system. In that case, systems analysts might need to develop a business profile that describes current business processes and functions. Even where the request involves minor changes or enhancements, how those modifications affect business operations and other information systems must be understood. Often a change in one system has an unexpected ripple effect on another. Therefore, when a systems request is analyzed, which departments, users, and business processes are involved must be determined.

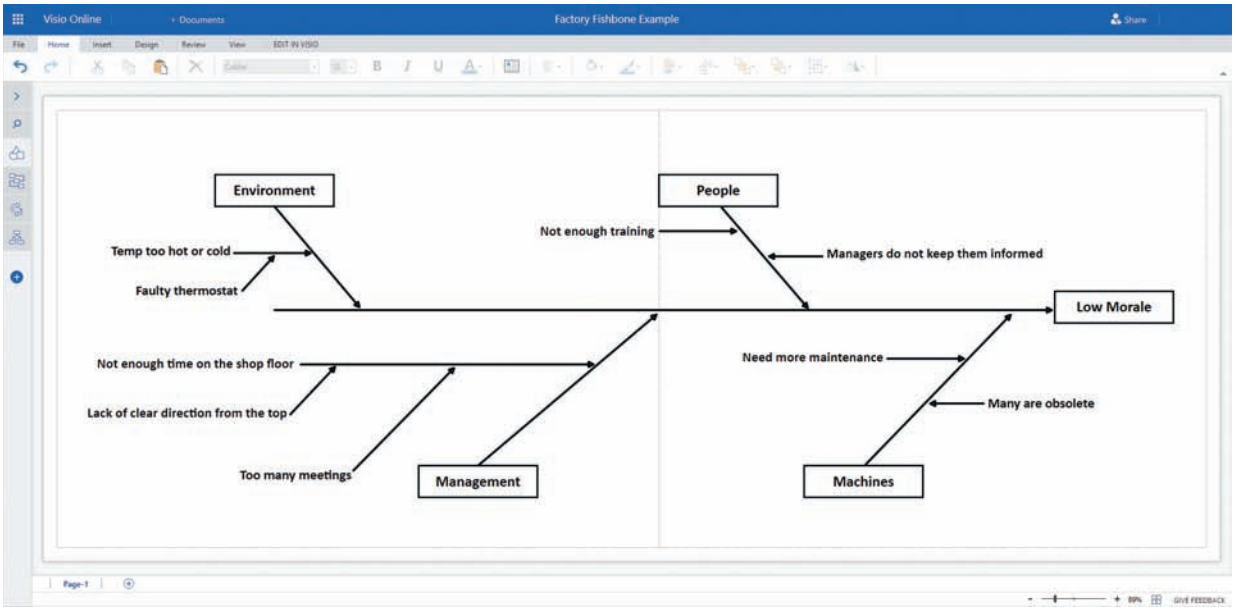
In many cases, the systems request does not reveal the underlying problem but only a symptom. For example, a request to investigate centralized processing delays might indicate improper scheduling practices rather than hardware problems. Similarly, a request for analysis of customer complaints might disclose a lack of sales representative training rather than issues with the product.

Figure 2-13 Five main steps in a typical preliminary investigation.



A popular technique for investigating causes and effects is a **fishbone diagram**, as shown in Figure 2-14. A fishbone diagram is an analysis tool that represents the possible causes of a problem as a graphical outline. When using a fishbone diagram, an analyst first states the problem and draws a main bone with sub-bones that represent possible causes of the problem. In the example shown in Figure 2-14, the problem is *Low Morale*, and the analyst has identified four areas to investigate: *Environment*, *People*, *Management*, and *Machines*. The analyst identifies possible causes in each area and draws them as horizontal sub-bones. For example, *Temp too hot or cold* is a probable cause in the *Environment* bone. For each reason, the analyst must ask: What could be causing this symptom? For example, *why* is it too hot? If the answer is *Faulty thermostat*, the analyst indicates this as a sub-bone to the *Temp too hot or cold* cause. In this manner, the analyst adds sub-bones to the diagram until they uncover the root causes of a problem rather than just the symptoms.

Figure 2-14 A fishbone diagram displays the causes of a problem. Typically, you must dig deeper to identify actual causes rather than just symptoms.



Step 2: Define the Project Scope and Constraints

Determining the **project scope** means defining the specific boundaries, or extent, of the project. For example, a statement that *payroll needs to be produced accurately* is very general, compared with the statement that *overtime pay needs to be calculated correctly for production workers on the second shift at the Cape Canaveral plant*. Similarly, the statement that *the project scope is to modify the accounts receivable system* is less specific than the statement that *the project scope allows customers to inquire online about account balances and recent transactions*.

Some analysts find it helpful to define project scope by creating a list with sections called *Must Do*, *Should Do*, *Could Do,* and *Won't Do*. This list can be reviewed later, during the systems analysis phase, when the systems requirements document is developed.

Projects with very general scope definitions risk expanding gradually, without specific authorization, in a process called **project creep**. The project scope should be clearly defined to avoid this problem. A graphical model that shows the systems, people, and business processes that will be affected is sometimes helpful. The project's scope also establishes the boundaries of the preliminary investigation itself. A systems analyst should limit the focus to the problem at hand and avoid unnecessary expenditure of time and money.

Along with defining the project's scope, any constraints on the system must be identified. A **constraint** is a requirement or condition the system must satisfy or an outcome the system must achieve. A constraint can involve hardware, software, time, policy, law, or cost. System constraints also define project scope. For example, if the system must operate with existing hardware, that constraint affects potential solutions. Other examples of constraints are:

- The order entry system must accept input from 15 remote sites.
- The human resources information system must produce statistics on hiring practices.
- The new website must be operational by March 1.

When examining constraints, their characteristics should be identified as follows.

Present versus Future. Is the constraint something that must be met as soon as the system is developed or modified, or is the constraint necessary at some future time?

Internal versus External. Is the constraint due to a requirement within the organization, or does some external force impose it, such as government regulation?

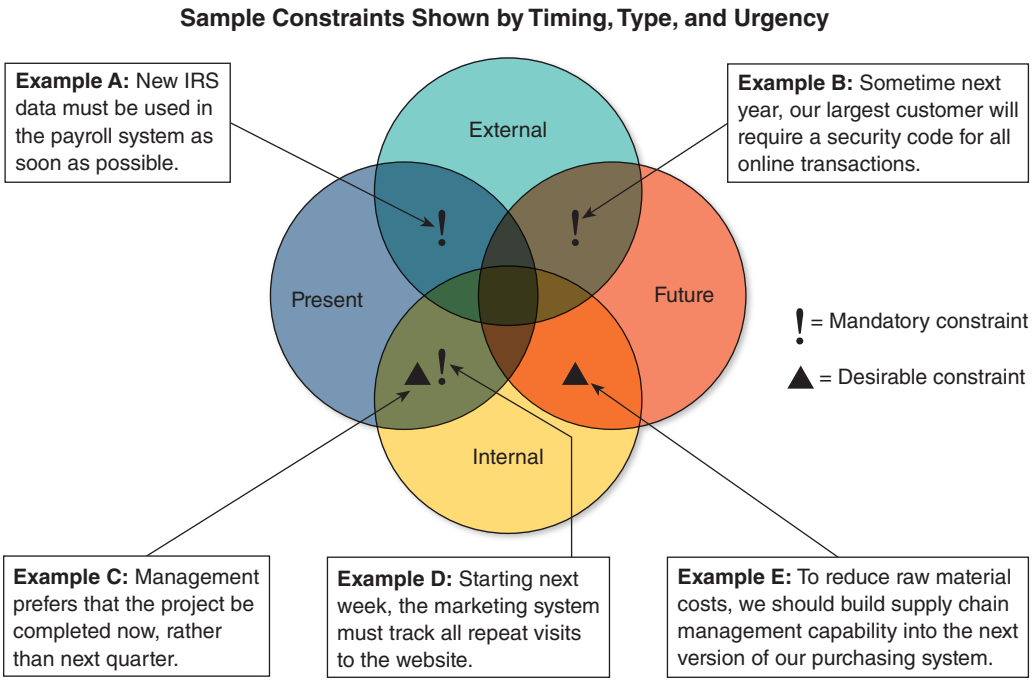
Mandatory versus Desirable. Is the constraint mandatory? Is meeting the constraint essential, or is it merely desirable?

Figure 2-15 shows five examples of constraints. Notice that each constraint has three characteristics, which are indicated by its position in the figure and by the symbol that represents the constraint, as follows:

- The constraint in Example A is present, external, and mandatory.
- The constraint in Example B is future, external, and mandatory.
- The constraint in Example C is present, internal, and desirable.
- The constraint in Example D is present, internal, and mandatory.
- The constraint in Example E is future, internal, and desirable.

All constraints should be identified as early as possible, regardless of the type, to avoid future problems and surprises. A clear definition of project scope and constraints avoids misunderstandings when managers assume that the system will have a particular feature or support for a project but later find that the feature will not be included.

Figure 2-15 Examples of various types of constraints.



Step 3: Perform Fact-Finding

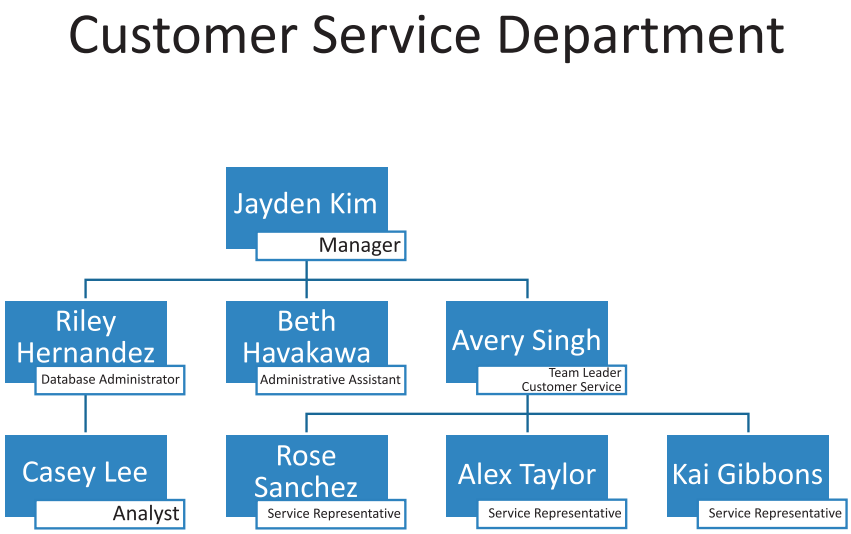
Fact-finding aims to gather data about project usability, costs, benefits, and schedules. Fact-finding involves various techniques, which are described below. Fact-finding might consume several hours, days, or weeks depending on the information needed to investigate the systems request. For example, a change in a report format or data entry screen might require a single telephone call or email message to a user. In contrast, a new inventory system would involve a series of interviews. During fact-finding, the analyst might analyze organizational charts, conduct interviews, review current documentation, observe operations, and conduct a user survey.

Analyze Organizational Charts. An analyst will only sometimes know the organizational structure of the departments involved in the study. Therefore, organizational charts should be obtained to understand the functions and identify people to interview.

If organizational charts are unavailable or outdated, the necessary information should be obtained from department personnel to construct the charts, as shown in Figure 2-16.

Even when charts are available, their accuracy should be verified. Recall that an organizational chart shows formal reporting relationships but not a group's informal alignment, which is also important.

Figure 2-16 Specialized tools such as Microsoft Visio can be used to draw organizational charts, but general-purpose presentation tools such as Microsoft PowerPoint can also be used.



Conduct Interviews. The primary method of obtaining information during the preliminary investigation is the interview. The interviewing process involves a series of steps:

1. Determine the people to interview.
2. Establish objectives for the interview.
3. Develop interview questions.
4. Prepare for the interview.
5. Conduct the interview.
6. Document the interview.
7. Evaluate the interview.

These seven steps are discussed in detail later in the book. The steps describe fact-finding techniques that occur during the systems analysis phase of the SDLC.

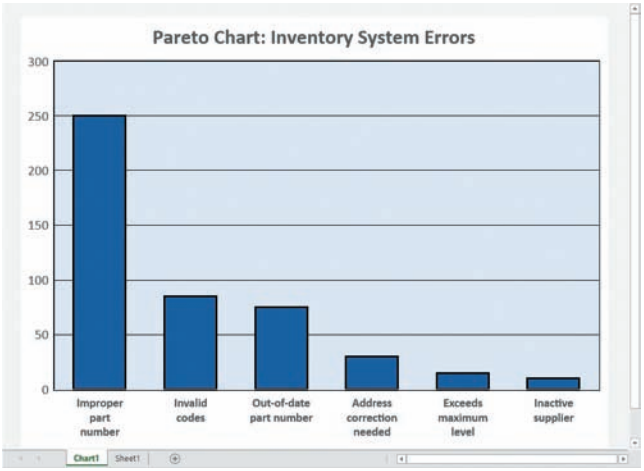
Remember that the interview's purpose and the preliminary investigation's purpose are to uncover facts, not to convince others that the project is justified. Therefore, the analyst's primary role in an interview is to ask effective questions and listen carefully. If several people are asked about the same topic, a standard set of questions for all the interviews should be prepared. Be sure to include open-ended questions, such as "What else do you think I should know about the system?" or "Is there any other relevant information we have not discussed?"

When conducting interviews during the preliminary investigation, interview managers and supervisors who have a broad knowledge of the system and can provide an overview of the business processes. Depending on the situation, talking to operational personnel to learn how the system functions daily may also be helpful.

Review Documentation. Although interviews are an essential method of obtaining information, investigating the current system documentation is also useful. The documentation might need to be updated, so check with users to confirm that the information is accurate and complete.

Observe Operations. Another fact-finding method is to observe the current system in operation. Observe how workers carry out typical tasks. Trace or follow the paths taken by input source documents or output reports. In addition to observing operations, consider sampling the inputs or outputs of the system. Using sampling techniques described later in the book, valuable information about the nature and frequency of the problem can be obtained.

Figure 2-17 A Pareto chart displays the causes of a problem in priority order so that an analyst can tackle the most important cases first. The part number issue would be the obvious starting point in this example.

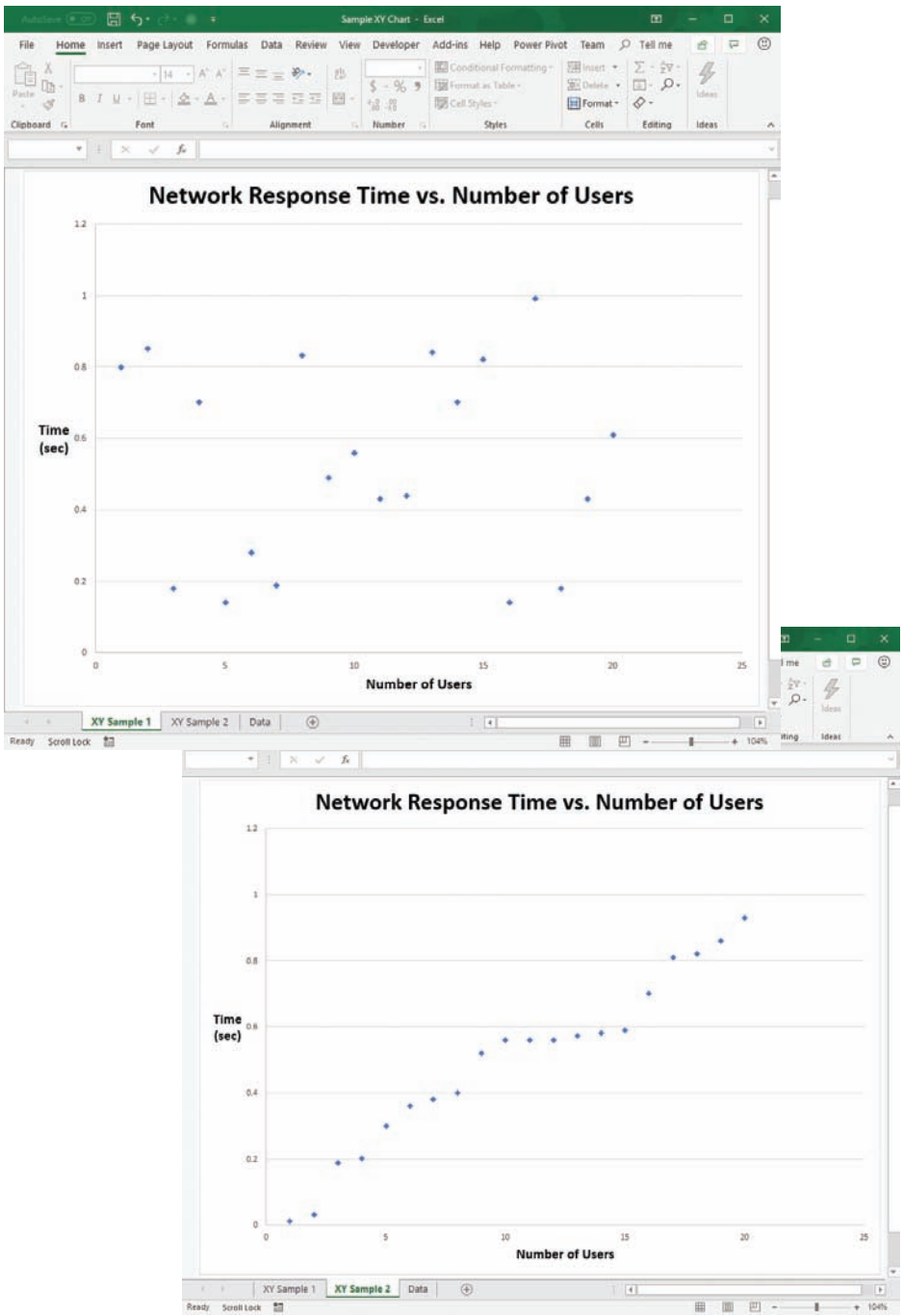


Conduct a User Survey. Interviews can be time-consuming. Sometimes information from a larger group can be obtained by conducting a user survey. In this case, design a form that users complete and return for tabulation. A survey is not as flexible as a series of interviews, but it is less expensive, generally takes less time, and can involve a broad cross-section of people.

Analyze the Data. Systems analysts use many techniques to locate the source of a problem. For example, the **Pareto chart** is widely used for visualizing issues that need attention. Named for a 19th-century economist, a Pareto chart is drawn as a vertical bar graph, as shown in Figure 2-17. The bars, which represent various causes of a problem, are arranged in descending order so the team can focus on the most important reasons. In the example shown, a systems analyst might use a Pareto chart to learn more about the causes of inventory system problems so that necessary improvements can be made.

The **XY chart**, sometimes called a **scatter diagram**, is another problem-solving tool. Often, an analyst looks for a correlation between two variables. For example, suppose complaints about network response time are received and the cause needs to be determined. The analyst would try to identify variables, such as the number of users, to see whether there is a correlation or pattern. For example, Figure 2-18 shows two XY charts with data samples. The first chart sample would suggest no correlation between the delays and the number of users, so the analyst would look elsewhere for the source of the problem. However, if the data resembles the second XY sample, it indicates a strong relationship between the number of users and the longer response times. That information would be extremely valuable in the problem-solving process.

Figure 2-18 An XY chart shows a correlation between variables, which is very important in problem solving. Conversely, a lack of correlation suggests that the variables are independent and that you should look elsewhere for the cause.



Step 4: Analyze Project Usability, Cost, Benefit, and Schedule Data

Data about the project's predicted costs, anticipated benefits, and schedule issues that could affect implementation is gathered during fact-finding. Before feasibility can be evaluated, this data must be analyzed carefully. For example, if interviews or surveys were conducted, the data should be tabulated to make it easier to understand. If current operations were observed, the results should be reviewed, and key facts useful in the feasibility analysis should be highlighted. If cost and benefit data were gathered, financial analysis and impact statements could be prepared using spreadsheets and other decision-support tools.

Also, time and cost estimates should be developed for the requirements modeling tasks for the next SDLC phase, systems analysis. Specifically, the following should be considered:

- What information must be obtained, and how will it be gathered and analyzed?
- Who will conduct the interviews? How many people will be interviewed, and how much time will be needed to meet with the people and summarize their responses?
- Will a survey be conducted? Who will be involved? How much time will it take people to complete it? How much time will it take to tabulate the results?
- How much will it cost to analyze the information and prepare a report with findings and recommendations?

Step 5: Evaluate Feasibility

At this point, the problem or opportunity has been analyzed, the project scope and constraints have been defined, and fact-finding has been performed to evaluate project usability, costs, benefits, and time constraints. The next step is assessing the project's feasibility, beginning with reviewing the answers to the questions in Section 2.7. The following guidelines should also be considered:

Operational Feasibility. Fact-finding should have included a review of user needs, requirements, and expectations. When this data is analyzed, look for areas that might present problems for system users and how they might be resolved. This is a vital concern because operational feasibility means a system will be used effectively.

Economic Feasibility. Using fact-finding data, financial analysis tools can be applied to assess feasibility. The cost–benefit data will be an important factor for management to consider. Also, a cost estimate for the project development team will be built into the project management plan.

Technical Feasibility. The fact-finding data should identify the hardware, software, and network resources needed to develop, install, and operate the system. With this data, a checklist can be developed to highlight technical costs and concerns.

Schedule Feasibility. The fact-finding data should include stakeholder expectations regarding acceptable timing and completion dates. As mentioned, a trade-off often exists between a project's schedule and costs. For example, compressing a project schedule might be possible, but only if the budget is increased accordingly to add extra personnel. The schedule data will be incorporated into the project plan through task durations and milestones.

At this stage of the preliminary investigation, there are several alternatives. No action is necessary, or some other strategy, such as additional training, is needed. A simple solution might solve a minor problem without further analysis. In different situations, it will be recommended that the project proceeds to the next phase, systems analysis.

2.9.3 Summarizing the Preliminary Investigation

The final task in the preliminary investigation is to summarize the results and recommendations, which can be conveyed to management in a report and a presentation. The written report and the oral presentation are examples of the need for systems analysts to develop strong communication skills. The report includes an evaluation of the systems request, an estimate of costs and benefits, and a **case for action**, which is a summary of the project request and a specific recommendation.

The specific format of a preliminary investigation report varies. However, a typical report might consist of the following sections:

- *Introduction*—The first section is an overview of the report. The introduction contains a brief description of the system, the name of the person or group who performed the investigation, and the name of the person or group who initiated the investigation.
- *Systems Request Summary*—The summary describes the basis of the systems request.
- *Findings*—The findings section contains the preliminary investigation results, including a description of the project's scope, constraints, and feasibility.
- *Recommendations*—A summary of the project request and a specific recommendation. Management will decide, but the IT department's input is important.
- *Project Roles*—This section lists the people who will participate in the project and describes each person's role.
- *Time and Cost Estimates*—This section describes the cost of acquiring and installing the system and the total cost of ownership during the system's useful life. Intangible costs also should be noted.
- *Expected Benefits*—This section includes anticipated tangible and intangible benefits and a timetable showing when they will occur.
- *Appendix*—An appendix is included in the report if supporting information must be attached. Supporting information consists of the interviews conducted, the documentation reviewed, and other sources for the information obtained. Detailed interview reports do not need to be included, but those supporting the report's findings should be retained for future reference.

Summary

This chapter explained how to analyze a business case. This chapter also explained why it is essential to understand business operations and requirements, how IT projects support a company's overall strategic plan, how systems projects get started, and how systems analysts conduct a feasibility study and perform preliminary investigations, which conclude with a management report. Fact-finding techniques that begin at this point and carry over into later development phases were also described.

Strategic planning is identifying long-term organizational goals, strategies, and resources. A strategic plan looks beyond day-to-day activities and focuses on a horizon of three, five, ten, or more years. The IT team must deliver IT resources to support the firm's long-term strategic goals. Therefore, IT managers and systems analysts must understand and participate in strategic planning activities.

Many organizations still rely on the IT group for guidance when selecting tools to support strategic planning activities. Some analysts stick to traditional text-based methods, using Microsoft Word tables, to provide structure and clarity. Others prefer a spreadsheet, such as Microsoft Excel, because it is easy to display priorities and the relative importance of planning assumptions. A more sophisticated approach uses a CASE tool to define and document the overall

environment. Such devices can integrate various statements, entities, data elements, and graphical models into an overall structure. The result is more consistency, better quality, and much less effort for the systems analyst.

During the systems planning phase, the IT team reviews a request to determine if it presents a strong business case. The term "business case" refers to a proposal's reasons or justification. A business case should be comprehensive yet easy to understand. It should clearly describe the project, justify proceeding, and estimate its financial impact. In addition, the analyst must consider the company's overall mission, objectives, and IT needs to perform the review.

The starting point for most information systems projects is called a systems request, a formal way of asking for IT support. For example, a systems request might propose enhancements for an existing system, correcting problems, replacing an older system, or developing an entirely new information system to support a company's current and future business needs. The six main reasons for systems requests are stronger controls, reduced cost, more information, better performance, improved customer service, and more support for new products and services.

Internal and external factors affect every company's business decisions, and IT projects are no exception. Internal factors originating from changes in company policies and standards can impact a project. Examples are changes in product standards, quality standards, the company's infrastructure, and the location of resources. External factors include technology, suppliers, customers, competitors, the economy, and the government. The PESTEL framework can help in assessing these external factors. The framework is based on six environmental types: political, economic, social, technological, environmental, and legal.

In most organizations, the IT department receives more systems requests than it can handle. As a result, many organizations assign responsibility for evaluating systems requests to a group of key managers and users. Many companies call this group a systems review committee or computer resources committee. Regardless of the name, the objective is to use several analysts' combined judgment and experience to evaluate project requests.

A systems request must pass several tests to see whether it is worthwhile to proceed further. The first step is to identify and eliminate systems requests that are not feasible. For example, a request would not be feasible if it required hardware or software that the company already had rejected. Even if the request is feasible, it might not be necessary. Sometimes, assessing request feasibility is simple and can be done in a few hours. If the request involves a new system or a significant change, however, extensive fact-finding and investigation in the form of feasibility studies are required.

After rejecting systems requests that are not feasible, the systems review committee must establish priorities for the remaining items. If tools are used as part of the review process, the requests may already be in a partially or fully sorted order. The highest priority goes to project requests that provide the most significant benefit, at the lowest cost, in the shortest time. However, many factors influence project evaluation, including dynamic priorities in an agile setting where project parameters change quickly.

A systems analyst conducts a preliminary investigation to study the systems request and recommend specific action. After obtaining authorization to proceed, the analyst interacts with managers, users, and other stakeholders. The analyst gathers facts about the problem or opportunity, project scope and constraints, project benefits, and estimated development time and costs. The end product of the preliminary investigation is a management report.

Key Terms

biometric devices
blockchain
business case
case for action
computer resources committee
constraint
critical success factor
customer relationship management (CRM)
discretionary projects
economic feasibility
electronic data interchange (EDI)
electronic product code (EPC)
electronic proof of delivery (EPOD)
encryption

fishbone diagram
intangible benefits
intangible costs
Internet of Things (IoT)
just-in-time (JIT)
mission statement
nondiscretionary projects
operational feasibility
Pareto chart
PESTEL framework
preliminary investigation
project creep
project scope
scatter diagram
schedule feasibility
strategic planning
SWOT analysis
systems request
systems review committee
tangible benefits
tangible costs
technical feasibility
total cost of ownership (TCO)
XY chart

Review Questions

1. Why should a systems analyst be interested in strategic planning?
2. List the four factors involved in a SWOT analysis.
3. Describe how CASE tools can support strategic planning.
4. List five questions the business case should answer.
5. What are the six main reasons for systems requests?
6. Explain the two main factors affecting systems requests.
7. Describe the role of the systems review committee in processing systems requests.
8. Define operational, economic, technical, and schedule feasibility.
9. List seven questions the systems analyst should consider when assessing project priorities.
10. What are the five steps of a preliminary investigation?

Discussion Topics

1. One of your co-workers says, "Mission statements are nice, but they don't change things down here where the work gets done." How would you reply?
2. Discuss how a company's financial status can affect systems projects.
3. The vice president of accounting says to you, the IT director, "This request procedure takes too long. My people know what they are doing, and their systems requests are necessary and important." The vice president of accounting suggests that the IT department bypass the initial steps and immediately get to work on their requests. What would you say in response?
4. When setting priorities for systems requests, the highest priority goes to projects that provide the greatest benefit, at the lowest cost, in the shortest time. How would you reconcile projects that can produce good results in the short term versus projects that can produce excellent results in the long term?
5. The final task in the preliminary investigation is to summarize the results and recommendations in a report and a presentation. Which form of communication, written or oral, is the most effective for conveying your findings to management?

Hands-On Projects

1. Use the Internet to find three examples of corporate mission statements.
2. Prepare a SWOT analysis of your school or your employer.
3. A mind map is a diagram used to organize information visually. Identify a tool that supports the creation of mind maps and explain how they can be a valuable part of strategic planning.
4. Visit the website for an IT magazine and find an article that discusses business cases. Then, summarize the article and what you learned from it.
5. Think of a technical problem you have experienced at school or work and draw a sample fishbone diagram with at least two levels.

Ethical Issues

As a new systems analyst at Premier Financial Services, you are getting quite an education. You report to the IT manager, who also chairs the systems review committee. Several months ago, the committee rejected a request from the finance director for an expensive new accounts payable system because the benefits did not appear to outweigh the costs.

Yesterday, the IT manager's boss asked the IT manager to reconsider the finance director's request and to persuade the other members to approve it. The IT manager wanted to discuss the merits of the request, but the discussion was cut off rather abruptly. It turns out the IT manager's boss and the finance director are longtime friends.

The IT manager is now very uncomfortable meeting with their boss. The IT manager believes the directive to reconsider the finance director's request would undermine the integrity of the systems review process and feels it would be unethical to grant preferred treatment just because a friendship is involved. The IT manager is considering submitting a request to step down as review committee chair, even though that might harm their career at the company.

Is this an ethical question, or just a matter of office politics? What would you do if you were the IT manager?

Chapter

3

Managing Systems Projects

Learning Objectives

When you finish this chapter, you should be able to:

1. Explain project planning, scheduling, monitoring and controlling, and reporting.
2. Demonstrate the three activities of task management.
3. Summarize the five key elements of people management.
4. Discuss the five key issues in communications management.
5. Describe the importance of technology management in systems projects.
6. Summarize the three areas of concern in financial management.
7. Explain why risk management is crucial in systems projects.
8. State the key benefits of using project management software.
9. Describe three areas requiring attention to manage systems projects successfully.

Chapter 3 is the final chapter in the systems planning phase of the systems development life cycle (SDLC). This chapter describes managing projects, tasks, people, communications, technology, finances, and risks associated with complex systems projects. In addition, the judicious use of project management software is discussed. Finally, the keys to successfully managing systems projects are also examined.

The chapter includes three "Case in Point" discussion questions to help contextualize the concepts described in the text. The scenario described in the "Ethical Issues" section of the end-of-chapter exercises considers the implications of raising awareness of a project going astray even when the project manager is reluctant to do so.

3.1 Project Management

Many professionals manage business and personal projects every day but only sometimes give them much thought. The management process for developing an information system or working on a construction project is much the same. The only difference is the nature of the project. To manage a large-scale information technology (IT) project, specific tools and techniques are needed. A project manager responsible for overseeing all relevant tasks is also required. **Project management** for IT professionals includes planning, scheduling, monitoring and controlling, and reporting on information system development.

A project manager will break the project into individual tasks, determine the order in which the tasks must be performed, and determine how long each task will take. With this information, Gantt charts or PERT/CPM charts can be used to schedule and manage the work. (Both types of charts are discussed in this chapter.) Microsoft Project is a popular project management tool that can help create and monitor the project plan, report progress, and use risk management to make the process easier for everyone.

3.1.1 What Shapes a Project?

A successful project must be completed on time, within budget, and deliver a high-quality product that satisfies users and meets requirements. Project management techniques can be used throughout the SDLC. System developers can initiate a formal project as early as the preliminary investigation stage or later as analysis, design, and implementation activities occur.

Systems development projects tend to be dynamic and challenging. However, as discussed earlier, there is always a balance between constraints and interactive elements such as project cost, scope, and time.

3.1.2 What Is a Project Triangle?

Figure 3-1 shows a straightforward example of a project triangle. Each project team must decide what is most important because the work cannot be good *and* fast *and* cheap.

When it comes to project management, things are more complex. Decisions do not need to be all-or-nothing but recognize that any change in one leg of the triangle will affect the other two legs. Figure 3-2 represents a common view of a **project triangle**, where the three legs are cost, scope, and time. The challenge is to find the optimal balance among these factors. Most successful project managers rely on personal experience, communication ability, and resourcefulness. For example, suppose an extremely time-critical project starts to slip. In that case, the project manager might have to trim some features, seek approval for a budget increase, add new personnel, or combine all three actions.

Figure 3-1 You can't get everything you want; you must make choices.

Figure 3-2 A typical project triangle includes cost, scope, and time.

On its website, Microsoft offers an interesting suggestion for project managers who have a project at risk: find the "stuck side" of the triangle. Microsoft states that in most projects, at least one side of the triangle is fixed and unlikely to change. It might be a budget cast in stone, an inflexible scope, or a schedule driven by factors beyond the firm's control. Whichever side is fixed is probably critical to the project's success. The leg where the problem resides must also be identified: cost, scope, or time.

3.1.3 What Does a Project Manager Do?

Good leadership is essential whether a project involves a new office building or an information system. In a systems project, the **project manager**, or **project leader**, usually is a senior systems analyst or an IT department manager if the project is significant. An analyst or a programmer/analyst might manage smaller projects.

In addition to the project manager, most large projects have a project coordinator. A **project coordinator** handles administrative responsibilities for the team and negotiates with users who might have conflicting requirements or want changes requiring additional time or expense.

Project managers typically perform several activities (or functions), including planning, scheduling, monitoring, and reporting:

- **Project planning** includes identifying all project tasks and estimating the completion time and cost.
- **Project scheduling** involves the creation of a specific timetable, usually in the form of charts that show tasks, task dependencies, and critical tasks that might delay the project. Scheduling also involves selecting and staffing the project team and assigning specific tasks to team members. Project scheduling uses Gantt charts and PERT/CPM charts, which are explained in the following sections.
- **Project monitoring** requires guiding, supervising, and coordinating the team's workload. The project manager must monitor the progress, evaluate the results, and take corrective action when necessary to control the project and stay on target.
- **Project reporting** includes regular progress reports to management, users, and the project team. Effective reporting requires strong communication skills and a sense of what others want and need to know about the project.

Project managers spend most of their time tracking project tasks along the critical path because delays in those tasks have the greatest potential to delay or jeopardize the project. The following sections describe the project planning and scheduling steps.

3.2 Task Management

Task management is integral to project management because most projects are composed of discrete tasks that must be managed individually and collectively. Task management involves creating a **work breakdown structure (WBS)**, modeling **task patterns**, and identifying the **critical path**.

3.2.1 Work Breakdown Structures

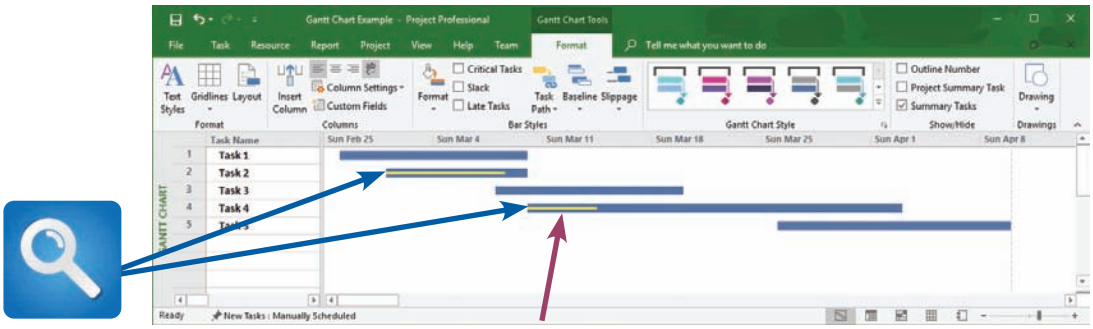
A work breakdown structure involves breaking a project down into a series of smaller tasks. Before creating work breakdown structures, two primary chart types should be understood: Gantt charts and PERT/CPM charts.

Gantt Charts

Henry L. Gantt, a mechanical engineer and management consultant, developed Gantt charts almost 100 years ago. His goal was to design a chart showing planned and actual progress on a project. A **Gantt chart** is a horizontal bar chart that represents a set of tasks. For example,

the Gantt chart from Microsoft Project in Figure 3-3 displays five tasks in a vertical array, with time shown on the horizontal axis. The bar's position shows the planned starting and ending time of each task, and the length of the bar indicates its duration. On the horizontal axis, time can be shown as elapsed from a fixed starting point or as actual calendar dates. A Gantt chart also can simplify a complex project by combining several activities into a **task group** that contains subsidiary tasks. This allows a complex project to be viewed as integrated modules.

Figure 3-3 In this Gantt chart, notice the yellow bars that show the percentage of task completion.



A Gantt chart can show task status by adding a contrasting color to the horizontal bars. For example, a vertical red arrow marks the current date in Figure 3-3. With a fixed reference point, it is easy to see that Task 1 is way behind schedule, Task 2 is only about 80 percent done and is running behind schedule, Task 3 should have started but no work has been done, Task 4 is running ahead of schedule, and Task 5 will begin in several weeks.

Gantt charts can present an overview of the project's status, but they do not provide enough detailed information, which is necessary when managing a complex project. Therefore, some project managers may find that PERT/CPM charts, discussed in the following section, are better tools for managing large projects.

PERT/CPM Charts

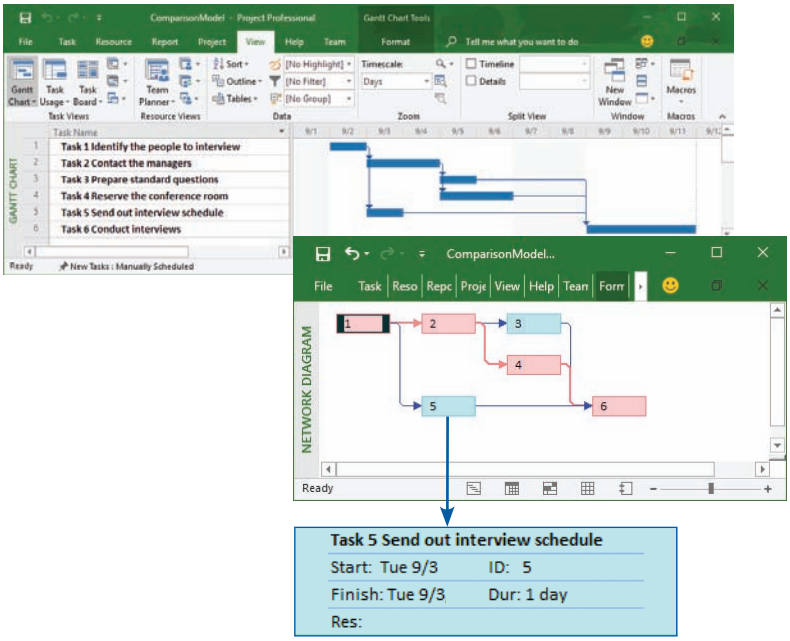
The U.S. Navy developed the **Program Evaluation Review Technique (PERT)** to manage complex projects, such as constructing nuclear submarines. At approximately the same time, the **Critical Path Method (CPM)** was developed by private industry to meet similar project management needs. The distinction between the two methods has disappeared over time, and today the technique is called either PERT, CPM, or **PERT/CPM**. This textbook uses the term "PERT chart."

PERT is a **bottom-up technique** because it analyzes a large, complex project as a series of individual tasks, just as a pyramid is built from the bottom up using individual blocks. To create a PERT chart, identify all the project tasks and estimate how long each task will take. Next, determine the logical order in which the tasks must be performed. For example, some tasks cannot start until other tasks have been completed. In other situations, several tasks can be performed at the same time.

Once the tasks are known, including their durations and the order in which they must be performed, calculate the time needed to complete the project. The specific tasks critical to the project's on-time completion can also be identified. An example of a PERT chart, which Microsoft calls a **network diagram**, is shown on the lower screen in Figure 3-4.

Although a Gantt chart offers a valuable snapshot view of the project, PERT charts are more useful for scheduling, monitoring, and controlling the actual work. With a PERT chart, a project manager can convert task start and finish times to exact dates by laying out the entire project on a calendar. Then, on any given day, the manager can compare what should be happening with what is taking place and react accordingly. Also, a PERT chart displays

Figure 3-4 The top screen shows a Gantt chart with six tasks. The PERT chart on the bottom screen displays an easy-to-follow task pattern for the same project. The details become visible when the user mouses over the summary box for Task 5.



complex task patterns and relationships. This information is valuable to a manager trying to address high-priority issues. PERT and Gantt charts are not mutually exclusive techniques, and project managers often use both methods.

Figure 3-4 shows both chart types. The top screen is a Gantt chart with six tasks. The PERT chart below shows the same project, using a separate box for each task instead of a horizontal bar. Although they both show the task patterns and flow, the PERT chart boxes can provide more information, such as task duration, start date, finish date, and the names of resources assigned to the task. The PERT chart in Figure 3-4 would be too small to view the exact details shown in the expanded text box at the bottom of the figure. Creating PERT charts is explained in a later section.

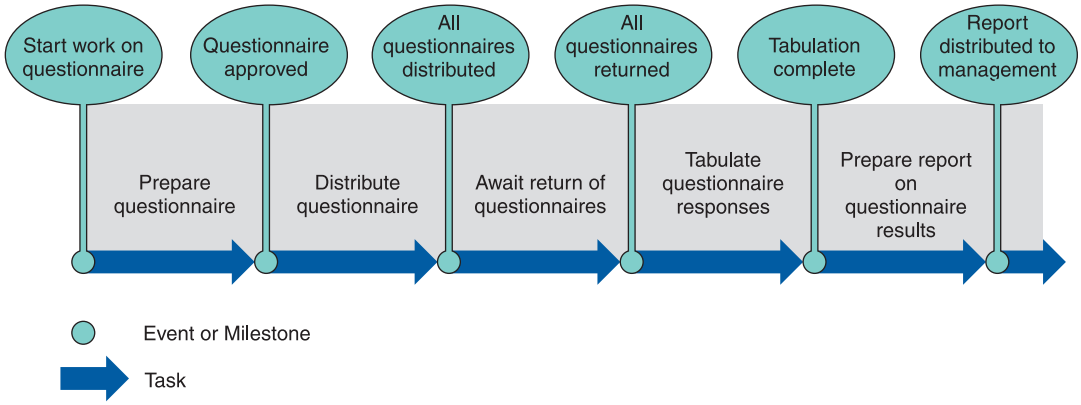
Identifying Tasks in a Work Breakdown Structure

A work breakdown structure must clearly identify each task and include an estimated duration. A **task**, or **activity**, is any work with a beginning and an end that requires company resources such as people, time, or money. Examples of tasks include conducting interviews, designing a report, selecting software, waiting for the delivery of equipment, and training users. Tasks are basic work units that the project manager plans, schedules, and monitors—so they should be relatively small and manageable.

In addition to tasks, every project has **events**, or **milestones**. An event is a recognizable reference point that can be used to monitor progress. For example, an event might be the start of user training, the conversion of system data, or the completion of interviews. On the other hand, a milestone such as "complete 50% of program testing" would not be helpful unless it could be determined precisely when that event will occur.

Figure 3-5 shows tasks and events that might be involved in creating, distributing, and tabulating a questionnaire. Notice that a recognizable event marks the beginning and end of each task. It would be virtually impossible to manage a project as one large task. Instead, the project is divided into smaller tasks, creating a WBS. The first step in creating a WBS is to list all the tasks.

Figure 3-5 Using a questionnaire requires a series of tasks and events to track progress. The illustration shows the relationship between the tasks and the events, or milestones, that mark the beginning and end of each task.



Listing the Tasks. While this step sounds simple, it can be challenging because the tasks might be embedded in a document, such as the one shown in the first version of Figure 3-6. One trick is to start by highlighting the individual tasks, as shown in the second version. Then, as shown in the third version, adding bullets makes the tasks stand out more clearly. The next step is to number the tasks and create a table like the one shown in Table 3-1, with columns for task number, description, duration, and **predecessor tasks**, which must be completed before another task can start.

Figure 3-6 The three versions show how to transform a task statement into a list of specific tasks for a WBS.

First version

First, reserve the meeting room. Then order the marketing materials and brief the managers. After the briefings, send out customer emails and upload the program to the app store. When the emails are sent and the program is available at the app store, load the new software. When the marketing materials have arrived and the software is ready, do a dress rehearsal.

Second version

First, *reserve the meeting room*. Then *order the marketing materials* and *brief the managers*. After the briefings, *send out customer emails* and *upload the program to the app store*. When the emails are sent and the program is available at the app store, *load the new software*. When the marketing materials have arrived and the software is ready, *do a dress rehearsal*.

Third version

- First, *reserve the meeting room*.
- Then *order the marketing materials* and *brief the managers*.
- After the briefings, *send out customer emails* and *upload the program to the app store*.
- When the emails are sent and the program is available at the app store, *load the new software*.
- When the marketing materials have arrived and the software is ready, *do a dress rehearsal*.

Estimating Task Duration. Task duration can be hours, days, or weeks, depending on the project. Because the following example uses days, the units of measurement are called person-days. A **person-day** represents the work that one person can complete in one day. This approach, however, can present some problems. For example, if it takes one person 20 days to perform a particular task, it might not be true that two people could complete the same task in 10 days or that 10 people could perform it in two days. Some tasks can be

Table 3-1 In this table, columns have been added for task number, description, duration, and predecessor tasks, which must be completed before another task can start.

Task No.	Description	Duration (Days)	Predecessor Tasks
1	Reserve the meeting room		
2	Order the marketing materials		
3	Brief the managers		
4	Send out customer emails		
5	Upload the program to the app store		
6	Load the new software		
7	Do a dress rehearsal		

divided evenly, so it is possible to use different combinations of time and people—up to a point—but not all. In most systems analysis tasks, time and people are not interchangeable. If one analyst needs two hours to interview a user, two analysts also will need two hours to do the same interview.

Project managers often use a weighted formula to estimate each task's duration. The project manager first makes three time estimates for each task: an optimistic or **best-case estimate** (B), a **probable-case estimate** (P), and a pessimistic or **worst-case estimate** (W). The manager then assigns a **weight**, which is a significant value, to each estimate. The weight can vary, but a common approach is to use a ratio of B = 1, P = 4, and W = 1. The expected task duration is calculated as follows:

$$\frac{(B + 4P + W)}{6}$$

For example, a project manager might estimate that a file-conversion task could be completed in as few as 20 days or could take as many as 34 days, but most likely will require 24 days. Using the formula, the expected task duration is 25 days, calculated as follows:

$$\frac{(20 + (4 * 24) + 34)}{6} = 25$$

Case in Point 3.1

Sunrise Software

A lively discussion is underway at Sunrise Software, where you are a project manager. The main question is whether the person-days concept has limitations. In other words, if a task will require 100 person-days, does it matter whether two people in 50 days, five people in 20 days, 10 people in 10 days, or some other combination that adds up to 100 performs the work?

Two programmers on the project seem to think it doesn't matter. On the other hand, one of the project's systems analysts says it is ridiculous to believe that any combination would work. This extreme example supported the analyst's point: Could 100 people accomplish a task estimated at 100 person-days in one day?

Is the systems analyst correct? If so, what are the limits of the "people versus days" equation? Taking the concept further, is there an optimum number of people to be assigned to a task? If so, how would that number be determined? You need to offer some guidance at the next project team meeting. What will you say?

Factors Affecting Task Duration

When developing task duration estimates, project managers consider four factors:

1. Project size
2. Human resources
3. Experience with similar projects
4. Constraints

Project Size. As described in an earlier chapter, information systems have various characteristics that affect their complexity and cost. In addition to considering those factors, a project manager must estimate the time required to complete each project phase. To develop accurate estimates, a project manager must identify all project tasks, from initial fact-finding to system implementation. Regardless of the systems development methodology used, the project manager must determine how much time will be needed to perform each task. In developing an estimate, the project manager must allow time for meetings, project reviews, training, and other factors (e.g., scheduled vacations or unscheduled medical leave) that could affect the development team's productivity.

Human Resources. Companies must invest heavily in cutting-edge technology to remain competitive in a connected world. In many areas, skilled IT professionals are in great demand, and firms must work hard to attract and retain the talent they need. A project manager must assemble and guide a development team with the skill and experience to handle the project. If necessary, additional systems analysts or programmers must be hired or trained, and this must be accomplished within a specific time frame. After a project gets underway, the project manager must deal with turnover, job vacancies, and escalating salaries in the technology sector—all of which can affect whether the project can be completed on time and within budget. The project manager must also accommodate official holidays, family emergencies, and other events that may affect the schedule.

Experience with Similar Projects. A project manager can develop time and cost estimates based on the resources for similar, previously developed information systems. The experience method works best for small or medium-sized projects where the two systems are similar in size, primary content, and operating environment. In large systems with more variables, the estimates are less reliable.

Constraints. You have learned that constraints are defined during the preliminary investigation. A constraint is a condition, restriction, or requirement that the system must satisfy. For example, a constraint might involve maximums for one or more resources, such as time, dollars, or people. A project manager must define system requirements that can be achieved realistically within the required constraints. In the absence of constraints, the project manager calculates the resources needed. However, the project manager must adjust other resources or change the project's scope if constraints exist. This approach is similar to the what-if analysis described later in this book.

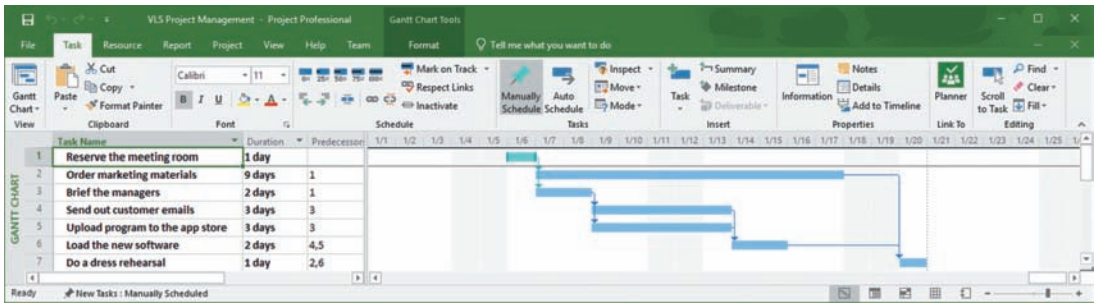
Displaying the Work Breakdown Structure

After the task durations are entered, the WBS will look like Table 3-2. To simplify the list, task groups can be used to manage a complex project with many tasks, just as with a Gantt chart. Note that the WBS shown in Table 3-2 is still incomplete: it does not show fields such as Start Date, End Date, and Task Name—fields that can be key for project managers. As a result, the WBS (including some of these missing fields) might resemble Figure 3-7 with Microsoft Project.

Table 3-2 Task durations have been added, and the WBS is complete except for predecessor task information. The predecessor tasks will determine task patterns and sequence of performance.

Task No.	Description	Duration (Days)	Predecessor Tasks
1	Reserve the meeting room	1	
2	Order the marketing materials	9	
3	Brief the managers	2	
4	Send out customer emails	3	
5	Upload the program to the app store	3	
6	Load the new software	2	
7	Do a dress rehearsal	1	

Figure 3-7 This Microsoft Project screen displays the same WBS as Table 3-2, including task number, task name, duration, and predecessor tasks.



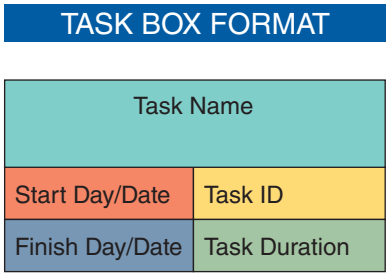
3.2.2 Task Patterns

Tasks in a WBS must be arranged in a logical sequence called a task pattern. In any project, large or small, tasks depend on each other and must be performed in a sequence, not unlike the commands in a software program. Task patterns can involve dependent tasks, multiple successor tasks, and multiple predecessor tasks. These patterns can be very complex in larger projects, and an analyst must study the logical flow carefully. This section explains how to understand and create graphical models of these patterns.

Using Task Boxes to Create a Model

In a PERT/CPM chart, project tasks are shown as rectangular boxes arranged in the sequence they must be performed. Each rectangular box, called a **task box**, has five sections, as shown in Figure 3-8. Each section of the task box contains important information about the task, including the Task Name, Task ID, Task Duration, Start Day/Date, and Finish Day/Date.

Figure 3-8 Each section of the task box contains important information about the task, including the Task Name, Task ID, Task Duration, Start Day/Date, and Finish Day/Date.



Task Name. The **task name** should be brief and descriptive, but it does not have to be unique to the project. For example, a task named Conduct Interviews might occur in several phases of the project.

Task ID. The **task ID** can be a number or code that provides unique identification.

Task Duration. The **duration** is the time it will take to complete a task, which is not necessarily the same as the elapsed time. For example, a task that takes

8 hours of effort to complete would be done in one day by a person dedicated 100%, but if the person assigned this task is only working 50% on this project, the task will take two days elapsed time to complete. All tasks must use the same time units, which can be hours, days, weeks, or months, depending on the project. An actual project starts on a specific date but can also be measured from a point in time, such as Day 1.

Start Day/Date. The **start day/date** is when a task is scheduled to begin. For example, suppose that a simple project has two tasks: Task 1 and Task 2. Also, suppose that Task 2 cannot start until Task 1 is finished. An analogy might be that a program cannot run until the computer is turned on. If Task 1 begins on Day 1 and lasts three days, it will finish on Day 3. Because Task 2 cannot begin until Task 1 is completed, the start time for Task 2 is Day 4, the day after Task 1 is finished.

Finish Day/Date. The **finish day/date** is when a task is scheduled for completion. Add the duration to the start day or date to calculate the finish day or date. When doing this, be very careful to add only a few days. For example, if a task starts on Day 10 and lasts five days, the finish would be on Day 14—not Day 15.

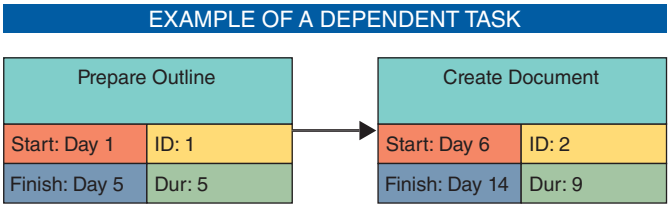
Task Pattern Types

A project is based on a pattern of tasks. In a large project, the overall pattern is complex. Still, it can be broken down into three basic types of patterns: dependent tasks, multiple successor tasks, and multiple predecessor tasks.

Dependent Tasks. When tasks must be completed one after another, like the relay race shown in Figure 3-9, they are called **dependent tasks** because one depends on the other. For example, Figure 3-10 shows that Task 2 depends on Task 1 because Task 2 cannot start until Task 1 is completed. In this example, the finish time of Task 1, Day 5, controls the start date of Task 2, which is Day 6.

Figure 3-9 In a relay race, each runner depends on the preceding runner and cannot start until the earlier runner finishes.
William Perugini/Shutterstock.com

Figure 3-10 This example of a dependent task shows that the finish time of Task 1, Day 5 controls the start date of Task 2, which is Day 6.



Multiple Successor Tasks. When several tasks can start simultaneously, each is called a **concurrent task**. Often, two or more concurrent tasks depend on a single prior task, which is called a predecessor task. In this situation, each concurrent task is called a **successor task**. In the example shown in Figure 3-11, successor Tasks 2 and 3 can begin as soon as Task 1 is finished. Notice that the finish time for Task 1 determines the start time for Tasks 2 and 3. In other words, the earliest that Task 1 can finish is Day 30, so Day 31 is the earliest that Tasks 2 and 3 can start.

Multiple Predecessor Tasks. Suppose a task requires two or more prior tasks to be completed before starting. For example, Figure 3-12 shows that Task 3 cannot begin until Tasks 1 and 2 are completed. Since the two tasks might not finish simultaneously, the longer (later) predecessor task becomes the controlling factor. Notice that the start for Task 3 is Day 16, not Day 6. Why is this so? Because Task 3 depends on two predecessor tasks, Tasks 1 and 2, Task 3 can only begin once the later of those tasks is complete. Therefore, the start time for a successor task must be the latest (largest) finish time for any preceding tasks. In the example shown, Task 1 ends on Day 15, while Task 2 ends on Day 5, so Task 1 controls the start time for Task 3.

Figure 3-11 This example of multiple successor tasks shows that the finish time for Task 1 determines the start time for Tasks 2 and 3.

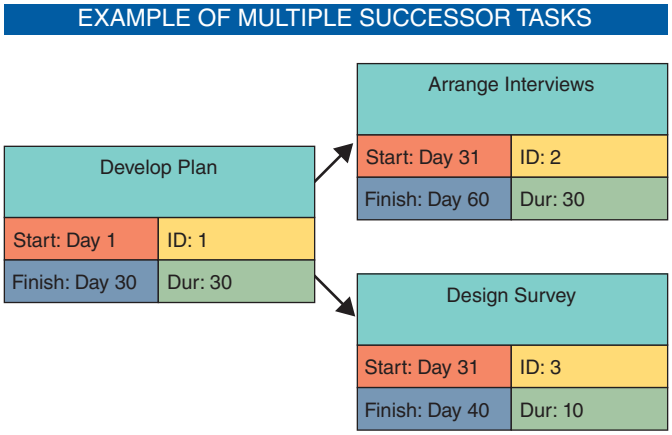
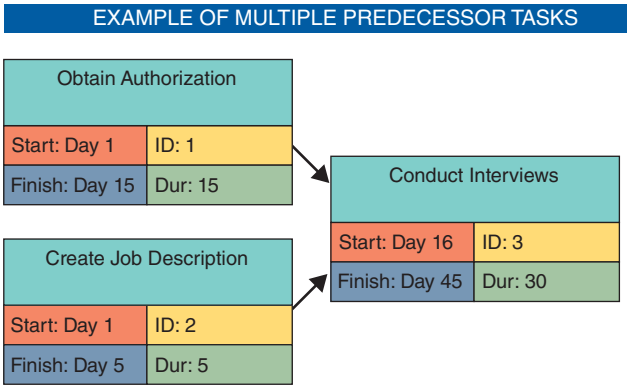


Figure 3-12 This example of multiple predecessor tasks shows that the start time for a successor task must be the latest (largest) finish time for any preceding tasks. In the example shown, Task 1 ends on Day 15, while Task 2 ends on Day 5, so Task 1 controls the start time for Task 3.



Task pattern types are identified by looking carefully at the wording of the task statement. Words like *then*, *when*, or *and* are action words that signal a sequence of events. Here are three simple examples:

- Do Task 1, ***then*** Task 2 describes dependent tasks that must be completed one after the other.
- ***When*** Task 2 is finished, start two tasks: Task 3 ***and*** Task 4 describe multiple successor tasks that can both start as soon as Task 2 is completed.
- ***When*** Tasks 5 ***and*** 6 are done, start Task 7. It is a multiple predecessor task because it can't start until two or more previous tasks are completed.

Working with Complex Task Patterns

When several task patterns combine, the facts must be studied very carefully to understand the logic and sequence. A project schedule is only accurate if the underlying task pattern is correct. For example, consider the following three fact statements and the task patterns they represent. Examples of task patterns are shown in Figures 3-13, 3-14, and 3-15.

Figure 3-13 Dependent tasks.

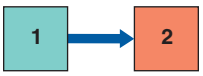


Figure 3-14 Dependent tasks and multiple successor tasks.

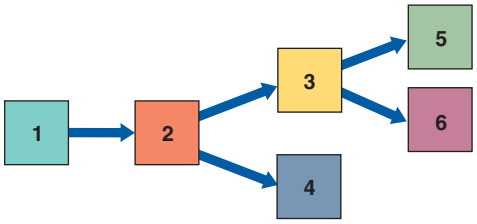
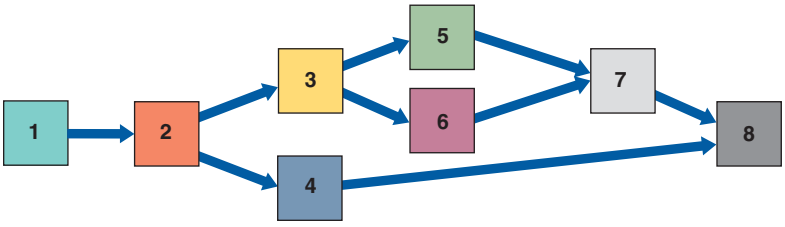


Figure 3-15 Dependent tasks, multiple successor tasks, and multiple predecessor tasks.



1. **Dependent tasks:** Perform Task 1. When Task 1 is complete, perform Task 2.
2. **Dependent tasks and multiple successor tasks:** Perform Task 1. When Task 1 is complete, perform Task 2. When Task 2 is finished, start Task 3 and Task 4. When Task 3 is complete, start two more tasks: Task 5 and Task 6.
3. **Dependent tasks, multiple successor tasks, and multiple predecessor tasks:** Perform Task 1. When Task 1 is complete, perform Task 2. When Task 2 is finished, start Task 3 and Task 4. When Task 3 is complete, start two more tasks: Task 5 and Task 6. When Tasks 5 and 6 are done, start Task 7. Then, when Tasks 4 and 7 are finished, perform Task 8.

Case in Point 3.2

Parallel Services

The project management team at Parallel Services is debating how to define tasks in the WBS. The project manager wants to break tasks down into the smallest possible units. For example, they objected to a broad task statement called "Develop a training schedule." They suggested three subtasks instead: (1) "Determine availability of training room," (2) "Determine availability of attendees," and (3) "Select specific dates and training times."

Another project team member disagrees, saying the broader task statement is better because it allows more flexibility and produces the same result. They say that if you break tasks into pieces that are too small, you risk overmanaging the work and spending more time on monitoring than actually performing the tasks. As a fellow team member, which approach do you agree with more? What are the pros and cons of each?

3.2.3 The Critical Path

Task patterns determine the order in which the tasks are performed. Once the task sequence has been defined, a project manager can schedule the tasks and calculate the critical path. A critical path is a series of tasks that, if delayed, would affect the completion date of the

overall project. If any task on the critical path falls behind schedule, the entire project will be delayed.

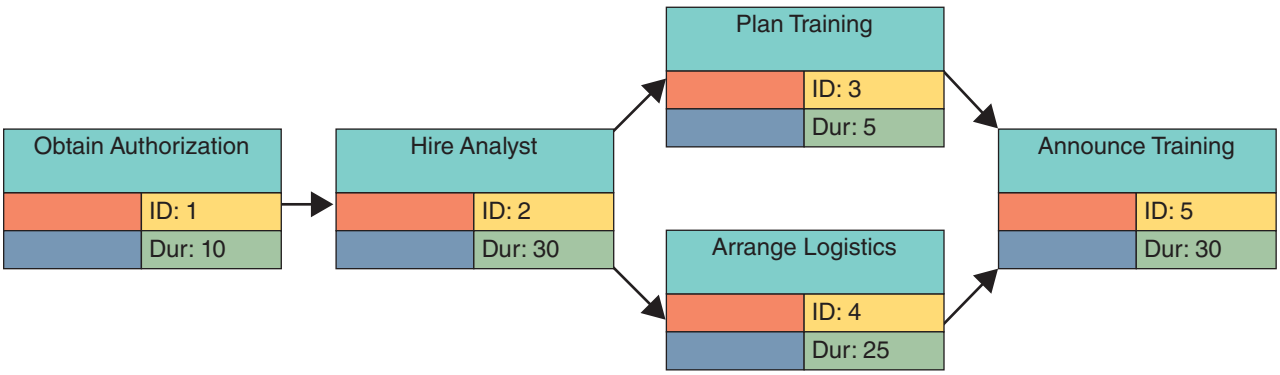
For example, suppose that Joan and Jim are invited to someone's home for dinner. Joan arrives on time, but Jim arrives 30 minutes late. Jim's arrival is part of the critical path because the host does not want to start without him, so the meal will be served 30 minutes later than initially planned.

Project managers must always be aware of the critical path so they can respond quickly to keep the project on track. Microsoft Project and other project management software can highlight the series of tasks that form the critical path.

Calculating the Critical Path

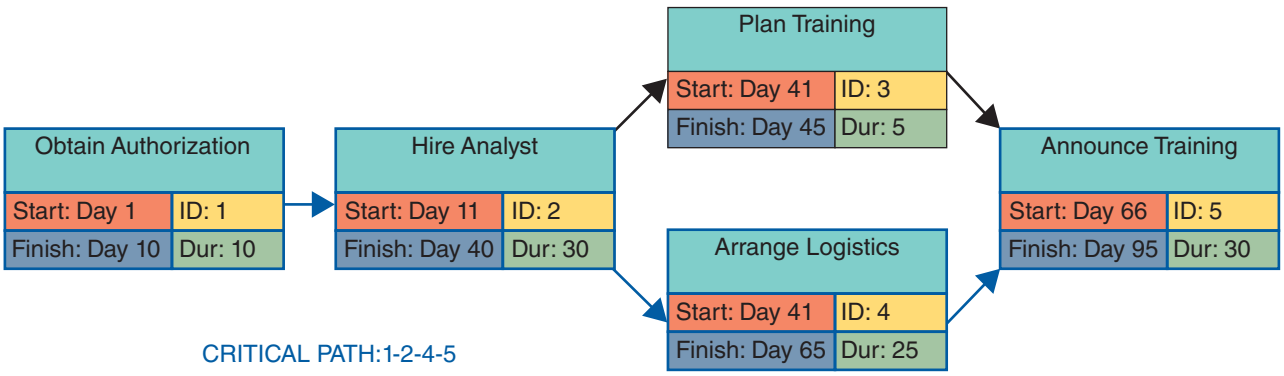
Figure 3-16 shows a training project with five tasks. Notice that the analyst has arranged the tasks and entered task names, IDs, and durations. The task patterns should be reviewed first. In this example, Task 1 is followed by Task 2, a dependent task. Task 2 has two successor tasks: Task 3 and Task 4. Tasks 3 and 4 are predecessor tasks for Task 5.

Figure 3-16 Example of a PERT/CPM chart with five tasks. Task 2 is a dependent task that has multiple successor tasks. Task 5 has multiple predecessor tasks. In this figure, the analyst has arranged the tasks and entered task names, IDs, and durations.



The next step is to establish start and finish dates, which will determine the critical path for the project. The following explanation outlines a step-by-step process. The result is shown in Figure 3-17.

Figure 3-17 The analyst has entered the start and finish times using the rules explained in this section. Notice that the overall project has a duration of 95 days.



- Task 1 starts on Day 1 and lasts 10 days, so the finish date is Day 10.
- Task 2, which is dependent on Task 1, can start on Day 11—the day after Task 1 ends. With a duration of 30 days, Task 2 will end on Day 40.

- Tasks 3 and 4 are multiple successor tasks that can start after Task 2 is done. Task 2 ends on Day 40, so Tasks 3 and 4 can start on Day 41. Task 3 lasts five days and will end on Day 45. Task 4 lasts 25 days and will end on Day 65.
- Task 5 depends on Tasks 3 and 4, which are multiple predecessors. Because Task 5 depends on both tasks, it cannot start until the later of the two tasks is complete. In this example, Task 3 ends earlier, but Task 4 will not be completed until Day 65, so Task 5 cannot start until Day 66.

Recall that the critical path is a series of tasks that, if delayed, would affect the completion date of the overall project. In this example, Tasks 1 and 2 are the first tasks on the critical path. Now look at Task 5, which cannot start until Tasks 3 and 4 are done. In this case, Task 4 is the controlling factor because Task 4 finishes on Day 65, 20 days later than when Task 3 finishes on Day 45. Therefore, the start date for Task 5 is determined by the finish date for Task 4.

In contrast, Task 3 has slack time and could be delayed up to 20 days without affecting Task 5. **Slack time** is the amount of time that the task could be late without pushing back the completion date of the entire project. Tasks 1, 2, 4, and 5 represent the critical path, highlighted with red arrows in Figure 3-17.

3.3 People Management

Technical prowess is highly prized in the dynamic world of systems analysis and design. However, success in this field is more than just about understanding and applying complex methodologies or navigating sophisticated software. For example, one of the most critical skills involves something other than code or systems: people management. Indeed, it is a complementary skill vital for a project's success.

People are the most valuable asset in any industry, and systems projects are no exception. Effective people management can make or break a project, whether managing software developers, data analysts, user experience (UX) designers, or stakeholders. The ability to empathize with users, manage team dynamics, foster growth, handle change, and build trust can significantly impact the success of a project. It enhances team morale, improves user satisfaction, and ultimately leads to better, more effective systems.

For students aspiring to enter this field, developing people management skills alongside their technical abilities can help them become well-rounded professionals capable of leading successful systems projects in the future.

3.3.1 Understanding the Human Element

Every project begins and ends with people. Before the first line of code is written, people identify a problem and conceptualize a solution. Then, once the system is operational, it is used by people who benefit from its functionalities and face its limitations.

For instance, an analyst designing an online banking system should understand the users' proficiency level with digital platforms, their concerns regarding security, and their expectations of convenience and efficiency. This human-focused approach can mean the difference between a well-received system and one that frustrates users.

This fundamental human element underscores the importance of effective communication (described in more detail below). A proficient systems analyst must communicate complex ideas clearly and concisely, translate technical jargon into lay terms for stakeholders, and accurately grasp and address users' needs. Empathy, active listening, and patience become as vital as technical expertise.

3.3.2 Team Dynamics and Collaboration

People management in systems analysis and design also involves navigating team dynamics. Diverse teams can provide a wealth of ideas and perspectives, leading to innovative solutions. Analysts with solid people management skills can harness this diversity for innovation and

problem solving. However, it can also lead to conflicts. A good team leader must ensure that all voices are heard and that disagreements are handled constructively.

For example, in a project aiming to develop an e-commerce platform, a systems analyst might need to facilitate discussions between UX/UI designers prioritizing user experience and database administrators concerned about data integrity and security. Balancing these different perspectives and fostering a spirit of collaboration can lead to a more effective, comprehensive design.

Moreover, fostering effective collaboration is critical given the multidisciplinary nature of systems projects. This involves building a conducive environment that encourages open communication, shared responsibility, and mutual respect among team members.

3.3.3 Nurturing Growth

Part of people management also involves fostering growth. Nurturing growth in a team is about creating opportunities for learning and development. The systems analysis and design field constantly evolves, and keeping a team's skills current is crucial. This might involve providing training opportunities, encouraging attendance at industry events, or allocating time for self-learning and exploration.

For instance, if a company transitions from a relational database management system to a NoSQL database, a manager might organize training sessions for the team, allocate time to familiarize themselves with the new system, or even invite industry experts for a workshop.

Another aspect of nurturing growth is fostering a culture of feedback and continuous improvement. Regularly reviewing performance, acknowledging good work, and providing constructive criticism can help team members refine their skills and grow as professionals.

3.3.4 Dealing with Change

Change is the only constant in a technical industry like systems analysis and design. Systems projects often bring about significant changes in processes, technology, and job roles. Such changes can be stressful for the people involved. Managing these changes effectively requires empathy, flexibility, and good communication skills.

For example, when a company implements a new enterprise resource planning (ERP) system, it's not just about setting up the system but also ensuring the employees are prepared for the change. This could involve organizing training sessions, being open to employees' concerns and questions, addressing any resistance or fear, and reassuring them of their role and importance within the changed scenario.

Effective people management means recognizing the disruptive nature of change and providing support during transitions. This can involve clearly communicating what the change entails and why it's necessary, addressing concerns, and providing training or resources to help people adapt.

3.3.5 The Role of Trust

Trust is a fundamental aspect of effective people management. When people feel trusted, they tend to be more committed, more willing to take on challenges, and more likely to feel a sense of ownership over their work. But building trust within a team isn't an overnight job. It takes time, consistency, and fairness. As a systems analyst or manager, your team needs to trust your decisions and your ability to guide the project to successful completion.

In the context of systems projects, trust can take several forms—for example, trust in a team member's technical skills, decision-making abilities, or understanding of user needs. Cultivating this trust requires consistency, reliability, and open, honest communication.

For example, consider implementing a complex project, like developing an artificial intelligence (AI)–driven customer service system. The project would require the team to work with unfamiliar technology and might involve a higher risk of failure. Trust in the project leader's competence and decision-making ability can ensure the team's commitment and cooperation, even in the face of challenges and uncertainty.

3.4 Communications Management

Effective communication is a critical aspect of systems projects. However, it can often pose significant challenges. Communication-related issues in this context include miscommunication of requirements, lack of regular project updates, ineffective change communication, cultural and language barriers, and dealing with problems.

More details on tools and techniques that support the communication aspects of managing systems projects are provided in Appendix A.

3.4.1 Miscommunication of Requirements

Miscommunication or misinterpretation of system requirements is a common issue that can significantly impact the effectiveness of systems projects. Clear, accurate communication of requirements from the stakeholders to the project team is the foundation for the entire project. However, when requirements are miscommunicated, it often leads to a product that fails to meet user needs or expectations, causing dissatisfaction and necessitating costly and time-consuming rework.

As detailed later in the book, one of the main reasons for the miscommunication of requirements is the difficulty in translating business needs into technical requirements. Stakeholders might need a clearer understanding of what they need or struggle to articulate their needs in a way the project team can understand. Similarly, the project team, especially those with a more technical focus, may not fully grasp the business implications of specific requirements, leading to misunderstandings.

To address this issue, it's vital to facilitate open, clear, and regular communication between all parties. This can be done through workshops, interviews, or regular meetings where stakeholders and project teams can discuss requirements in detail. Case scenarios, user stories, or prototype demonstrations can help clarify requirements. Furthermore, employing a business analyst with technical and business acumen can help bridge the gap between stakeholders and the project team.

3.4.2 Lack of Regular Project Updates

Regular communication about the project's status is essential to keep all team members and stakeholders informed. It ensures all stakeholders are aligned, fosters a sense of involvement, and prevents surprises or misunderstandings about the project's progress. Project updates can be communicated via project reports, status meetings, and status reports.

When project updates are irregular or nonexistent, it can lead to confusion and misaligned expectations, often resulting in delays or conflicts. This could occur if project managers do not communicate consistently, if there are too many communication channels causing information to get lost, or if team members are not proactive in sharing updates. Team members might assume that everyone knows what they're working on or might be too engrossed in their tasks to provide updates. This can lead to a lack of understanding of the bigger picture and the project's overall progress.

Addressing this challenge requires establishing explicit communication norms and expectations. This might involve setting up regular team meetings, creating project dashboards for real-time updates, or using project management tools to track tasks and progress. It's also essential to foster a culture where communication is valued and encouraged, and team members feel comfortable sharing updates, asking questions, and discussing issues.

Project Reporting

Project team members regularly report their progress to the project manager, who in turn reports to management and users. The project manager collects, verifies, organizes, and evaluates the information received from the team. Then the manager decides which information needs to be passed along, prepares a summary that can be understood easily, adds comments and explanations if required, and submits it to management and users.

Project Status Meetings

Project managers, like the one shown in Figure 3-18, schedule regular meetings to update the team and discuss project status, issues, problems, and opportunities. Although meetings can be time-consuming, most project managers believe they are worthwhile. The sessions allow team members to share information, discuss common problems, and explain new techniques. The meetings also enable the project manager to seek input and conduct brainstorming sessions.

Figure 3-18 Project managers schedule regular meetings to update the project team and discuss project status, issues, problems, and opportunities.
Hero Images/Getty Images

Project Status Reports

A project manager must report regularly to an immediate supervisor, upper management, and users. Although a progress report might be given verbally to a direct supervisor, reports to management and users are usually written. Gantt charts often are included in progress reports to show project status graphically.

3.4.3 Ineffective Change Communication

Systems projects often involve changes—changes in requirements, design, project scope, and so on. If these changes are not effectively communicated to all concerned parties, it can lead to confusion, errors, and delays. For example, if a change in system design is not adequately communicated to the development team, they may continue working based on the old design, wasting time and resources.

Ineffective change communication often stems from a need for clearer change management procedures. For example, changes might be discussed in an ad-hoc manner, or information might only reach some team members, leading to inconsistencies. Additionally, the impact of changes might not be adequately communicated, leading to a lack of understanding of the change's significance.

Addressing these communication issues involves creating clear communication protocols, implementing robust change management procedures, utilizing effective communication tools, and fostering an open and transparent communication culture within the project team. In addition, all changes should be documented, and relevant parties should be informed

promptly. It may also be helpful to hold meetings or workshops to discuss significant changes, ensuring everyone understands them and their implications. Lastly, fostering a culture that embraces change (see below), rather than fearing it, can encourage team members to stay adaptable and responsive to changes, leading to smoother project execution.

3.4.4 Cultural and Language Barriers

In today's globalized world, systems projects often involve diverse or geographically distributed teams. While this diversity can bring a wide range of perspectives and skills to the project, it can also lead to communication issues due to cultural and language barriers. For example, team members from different cultural backgrounds may have different communication styles or interpretations of messages. Similarly, team members speaking other primary languages can lead to misunderstandings or misinterpretations.

For example, direct communication may be the norm in one culture, while a more indirect, nuanced approach is preferred in another. Misunderstandings can arise if these differences are not understood and respected. Similarly, if the project's working language differs from a team member's first language, they might need help expressing their thoughts accurately or might need clarification on information.

Addressing this challenge requires building a culture of understanding, respect, and patience within the team. Team members should be mindful of potential cultural and language barriers and seek clarification when in doubt. Tools like translation apps or multilingual team members can assist in overcoming language barriers. Additionally, training in intercultural communication can help team members understand and navigate cultural differences more effectively.

3.4.5 Dealing with Problems

Deciding how to communicate potential problems can take time and effort. At what point should management be informed about the possibility of cost overruns, schedule delays, or technical problems? At one extreme is the overly cautious project manager who alerts management to every potential snag and slight delay. The danger here is that the manager loses credibility over time, and management might ignore potentially serious situations. At the other extreme is the project manager who tries to handle all situations single-handedly and only alerts management once a problem is serious. By the time management learns of the problem, little time might remain to react or devise a solution.

A project manager's best course of action lies between the two extremes but is probably closer to the first. If the consequences are unclear, the analyst should err on the side of caution and warn management about the possibility of a problem.

When the situation is reported, what is being done to handle and monitor the problem should be explained. If the situation is beyond the analyst's control, possible actions management can take to resolve the situation should be suggested. Most managers recognize that problems occur on most projects; it is better to alert management sooner rather than later.

3.5 Technology Management

Technology management for systems projects refers to the strategic selection, implementation, and review of the technology resources used throughout the SDLC. It's a crucial aspect of ensuring that the systems developed are efficient, reliable, and capable of meeting the organization's current and future needs.

Effective technology management starts with choosing the right technology tools. In systems projects, this can include software for modeling and diagramming, project management tools (see the next section), platforms for collaboration and communication, testing tools, and more. The selection process should consider factors such as the functionality and scalability of the tools, their compatibility with the existing technology infrastructure, the learning curve for the team, and the cost.

However, technology management continues beyond selection and acquisition. Once the tools are in place, they must be properly maintained to ensure their ongoing performance and reliability. This involves regular software updates, troubleshooting, security checks, and system backups. Moreover, as business needs and technology landscapes evolve, the technology strategy must be regularly reviewed and updated. This could mean phasing out outdated tools, adopting new ones, or upgrading existing systems. This continuous attention to technology management is key to ensuring that the systems analysis and design process remains efficient and effective, delivering high-quality results that drive the organization's success.

3.5.1 Technology Selection

Selecting the right technology tools is a critical part of effective technology management. Several software tools are available in systems projects, from modeling to testing. The choice depends on various factors, including the scope and nature of the project, the team's familiarity with the tools, and the tools' alignment with the project's goals.

Functionality is a key criterion, meaning the tools must possess the features required to carry out the intended tasks. Scalability is another important consideration, particularly for large projects or those expected to grow over time. In addition, the tools must handle increased data volume or complexity without performance degradation. Also, compatibility with existing technology infrastructure is essential to ensure seamless integration and avoid disruptions.

Finally, cost-effectiveness should be evaluated. This includes the purchase price or subscription fee and factors like training costs, maintenance expenses, and potential costs associated with downtime or tool-related issues.

3.5.2 Technology Implementation

Once suitable tools are selected, the implementation phase begins. This involves acquiring the software (e.g., development, purchase, or outsourcing), installing it, integrating it with existing systems, and configuring it to meet project requirements. It also includes training the project team to use the tools effectively.

Effective implementation is not just about technical setup. It also involves managing change within the team and the broader organization. This can mean addressing resistance, promoting buy-in, and ensuring that everyone understands the new tools' benefits and how to use them.

Moreover, it's important to manage expectations during the implementation phase. Not all benefits will be immediate, and challenges or "teething" problems may exist. Therefore, patience, regular communication, and responsiveness to feedback and issues are essential during this transitional period.

Technology implementation related to product development and systems support strategies is covered in more detail later in the book.

3.5.3 Technology Review

Even the best tools require ongoing maintenance to perform optimally. Regular software updates must be installed to benefit from new features, improvements, or bug fixes. In addition, regular checks should be made for any issues or irregularities, and prompt troubleshooting should be carried out to minimize downtime.

Security is another critical aspect of technology maintenance, particularly given the sensitive data often involved in systems analysis and design. Therefore, regular security checks and updates, strong access controls, and proactive measures against potential threats are essential.

Lastly, continuous review of technology tools is necessary as the business environment and technology landscapes evolve. Once-effective tools might become outdated or less efficient, or better options might become available. Regularly reassessing the technology strategy and

adjusting as needed helps ensure that the systems analysis and design process remains effective and agile, ready to deliver high-quality results in a constantly changing world.

Technology review related to systems security and support strategies is covered in more detail later in the book.

3.6 Financial Management

Sound financial management is a crucial, often overlooked aspect of a successful systems project. Understanding the economic implications of systems projects can profoundly impact project outcomes and the overall efficiency of the business operation. The most critical areas of concern include budgets and cost estimation, skills and technology investment, and return on investment analysis.

More details on financial analysis tools and techniques that support the financial aspects of managing systems projects are provided in Appendix B.

3.6.1 Budgets and Cost Estimation

Every project begins with a vision and a budget. Leaders and practitioners need to assess the financial viability of a project before embarking on it. Detailed cost estimation and budgeting are pivotal to this assessment. Factors to consider may include labor costs, software and hardware expenses, training requirements, maintenance costs, and potential cost overruns. Proper cost estimation at the beginning can set the stage for a project's financial health and prevent unexpected monetary issues later in the SDLC.

Once a budget is set, the challenge lies in efficiently allocating resources. This process requires balancing the dual goals of meeting project needs and staying within budget constraints. The resource allocation plan should consider the most critical aspects of the project and assign resources accordingly. It should also factor in potential risks and have contingency provisions.

Financial management is not a one-time event but an ongoing process throughout the project life cycle. Regular monitoring of expenses helps identify and address financial deviations promptly. In addition, this proactive approach allows for timely corrective actions, ensuring the project remains financially viable.

3.6.2 Skills and Technology Investment

Investing in skills and technology is critical in the financial management aspect of systems projects. With technology evolving at a breakneck speed, organizations must ensure their teams are equipped with the latest tools and possess the required skills to use them effectively. While these investments may initially seem like additional costs, they often lead to long-term financial benefits by increasing efficiency and reducing time to market. As such, investing in the right technology and nurturing the talent to maximize its potential is not just an expenditure but an investment in the organization's future.

Investing in technology includes procuring the latest software for design and analysis, upgrading hardware to support new applications, and maintaining a robust technological infrastructure to facilitate seamless work processes. For instance, investing in tools that enable agile methodologies or adopting advanced technologies like AI or **machine learning (ML)** could provide a significant competitive edge. However, the choice of technology investments must align with the organization's needs, capacity, and strategic goals to ensure it brings real value and doesn't become a cost burden.

In parallel, investment in skills is a cornerstone of maximizing the benefits of technology investments. It entails continuous learning opportunities for team members to acquire new skills, update existing ones, and keep pace with industry trends. This could involve structured training programs, workshops, seminars, or even encouraging self-paced online learning. Equally important is cultivating a culture that values and rewards knowledge sharing. As teams gain proficiency in the latest methodologies, tools, and best practices, they enhance

their competencies and boost the organization's overall capability to deliver high-quality, efficient system designs.

3.6.3 Return on Investment Analysis

Leaders should weigh the projected benefits of the proposed system against the projected costs. A practical return on investment (ROI) analysis clarifies the new system's value to the organization. Benefits could be tangible, like cost savings from increased efficiency, or intangible, like improved customer satisfaction or employee morale. Leaders should prioritize projects with a high ROI, indicating that the benefits outweigh the costs.

In systems projects, financial management is not just about tracking dollars and cents but understanding their strategic implications. Therefore, leaders and practitioners must consider the financial dimensions of their work, from initial cost estimation and budgeting to ongoing monitoring and controlling expenditure, resource allocation, and risk management. By doing so, they can ensure that their projects are technically sound and financially viable, maximizing value for the organization.

Every project carries potential financial risks, such as cost overruns, unexpected expenses, or the new system failing to deliver the expected ROI. Regular financial risk assessments enable leaders to identify and mitigate these risks before they become unmanageable. This could involve having contingency funds, diversifying investments, or securing comprehensive insurance. Managing financial risk is part of a project's overall risk management and mitigation strategy.

3.7 Risk Management

Every systems project involves risks that systems analysts and project managers must address. Therefore, risk management is crucial in systems projects, as it helps identify, assess, and manage potential threats that could hinder project success. A **risk** is the possibility of an adverse event that could negatively affect the project. **Risk management** is a fundamental aspect of managing systems projects, as it helps identify, assess, and prioritize uncertainties that could impact the achievement of project objectives.

3.7.1 The Importance of Risk Management

The importance of risk management lies in its potential to mitigate the adverse impact of uncertainties across the SDLC. Risk management is essential for reducing uncertainty, improving project planning and control, enhancing stakeholder confidence, mitigating potential losses, and facilitating decision making.

Fortunately, a wide variety of risk management software is available to help a project manager with these tasks. For example, most packages allow a project manager to assign specific dates as constraints, align task dependencies, note external factors that might affect a task, track progress, and display tasks that are behind schedule. Armed with this information, the IT team can quantify the project's risks, just as they use financial analysis tools to quantify costs and benefits.

Ultimately, effective risk management enhances the likelihood of project success. By proactively managing potential threats, the project is more likely to be delivered on time, within budget, and to the required quality standards. Thus, risk management is not just about preventing negative outcomes but enabling positive ones.

Reducing Uncertainty

Systems projects often involve using advanced or emerging technologies, complex integration with existing systems, and meeting evolving user requirements. These factors introduce considerable uncertainty to the project. Effective risk management allows these uncertainties to be identified, analyzed, and managed proactively, reducing the potential for unpleasant surprises during the project.

Improving Project Planning and Control

By identifying potential risks early in the project life cycle, risk management helps inform project planning decisions. It can guide the allocation of resources, the scheduling of tasks, and the setting of project milestones. Furthermore, the continual monitoring and controlling of risks help keep the project on track, detecting deviations from the plan early and facilitating timely corrective actions.

Enhancing Stakeholder Confidence

Stakeholders, including clients, team members, and upper management, must have confidence in the project's ability to achieve its objectives. A commitment to due diligence and proactive problem solving is demonstrated by systematically managing risks, thereby boosting stakeholder confidence.

Mitigating Potential Losses

Some risks, if realized, can lead to substantial financial losses, delays, or even the total failure of the project. Risk management helps to prevent these outcomes by identifying potential threats and putting measures in place to avoid or mitigate them.

Facilitating Decision Making

Risk management provides valuable information that aids in decision making. By understanding the potential risks and their impacts, project managers can make more informed decisions, balancing the potential benefits of an action against its associated risks.

3.7.2 The Origins of Risks

Risks could arise from a myriad of sources, such as technological complexities, evolving stakeholder requirements, team dynamics, external environmental factors, and inherent system complexity. The importance of risk management lies in its potential to mitigate the adverse impact of these uncertainties, thereby improving the chances of project success.

It's crucial to note that these sources of risk can interact with each other, with one risk potentially triggering risks in other areas. For instance, a change in requirements (stakeholder needs) could lead to technical challenges (technological complexities), which in turn could lead to project delays (project management). Understanding the origins of these risks can significantly aid in their early identification and successful management.

Technological Complexities

One significant source of risk stems from technological complexities. With the rapid pace of technological innovation, system projects frequently involve using new or unproven technologies. As a result, there may be uncertainties about the technical feasibility of the project or the performance and reliability of these technologies. Additionally, risks could arise from software or hardware failures, security vulnerabilities, or data management issues.

Evolving Requirements

Changes in requirements are another significant origin of risks in system projects. Stakeholders may alter their needs and expectations during the project, leading to project scope, timeline, or budget changes. This can introduce significant uncertainties into the project and may require rework, leading to increased costs and delays. Furthermore, discrepancies or misunderstandings in interpreting the requirements may lead to misalignment between the designed system and stakeholder needs.

Team Dynamics

The project team and management practices can also be a source of risk. This can include risks related to team dynamics, such as conflicts, lack of cohesion, or inadequate communication. In addition, there may be risks associated with project management, such as inadequate planning, poor risk management, or ineffective leadership. Human errors during requirements elicitation, system design, coding, testing, or deployment can also lead to significant issues.

External Factors

The external environment can also give rise to risks. This includes changes in market trends, regulatory changes, or shifts in economic conditions. For instance, new regulations may require changes to the system's data management practices, or an economic downturn may affect the project's budget.

Size and Complexity

Lastly, the inherent complexity and size of the system can be a risk factor. Larger and more complex projects have more elements that could go wrong, leading to increased uncertainties. Additionally, the interdependencies between different parts of the system mean that an issue in one area can have a ripple effect, leading to problems in other areas.

3.7.3 Risk Management Tasks

Although project management experts might have differing views about the number of steps (or phases) needed, a basic list of risk management tasks would include risk identification, analysis, and response planning. These tasks form the core of risk management in systems projects, serving as essential components for the successful completion of any project. Their importance is underscored by their role in reducing project uncertainties, promoting informed decision making, and maximizing project success.

Risk Identification

Risk identification is the first and arguably the most crucial step in the risk management process. It involves systematically identifying possible threats or uncertainties that could hamper the success of the systems analysis and design project. A detailed understanding of the project—its context, requirements, and the technologies involved—is imperative to identify potential risks. This stage forms the foundation of the risk management process because identifying potential risks is necessary to ensure further steps can be taken toward managing them.

The process of risk identification can be conducted using various techniques, some of which include brainstorming sessions with project stakeholders; the **Delphi technique**, in which experts anonymously contribute their opinions; SWOT analysis to examine strengths, weaknesses, opportunities, and threats; and risk breakdown structures, which offer a hierarchical decomposition of risks. Other techniques can also be used, such as interviewing stakeholders, root-cause analysis, or reviewing historical data and lessons from similar past projects. The goal is to compile a comprehensive list of possible risks that may arise during the project's life cycle.

The result of risk identification is usually a risk register, which is a document detailing all the identified risks along with their characteristics. This register forms a key input to the subsequent risk analysis stage. However, it's essential to remember that risk identification is not a one-time process but a continual activity throughout the project. New risks may emerge, and old ones may become irrelevant as the project progresses, hence the need for regular reassessment and updating of the risk register.

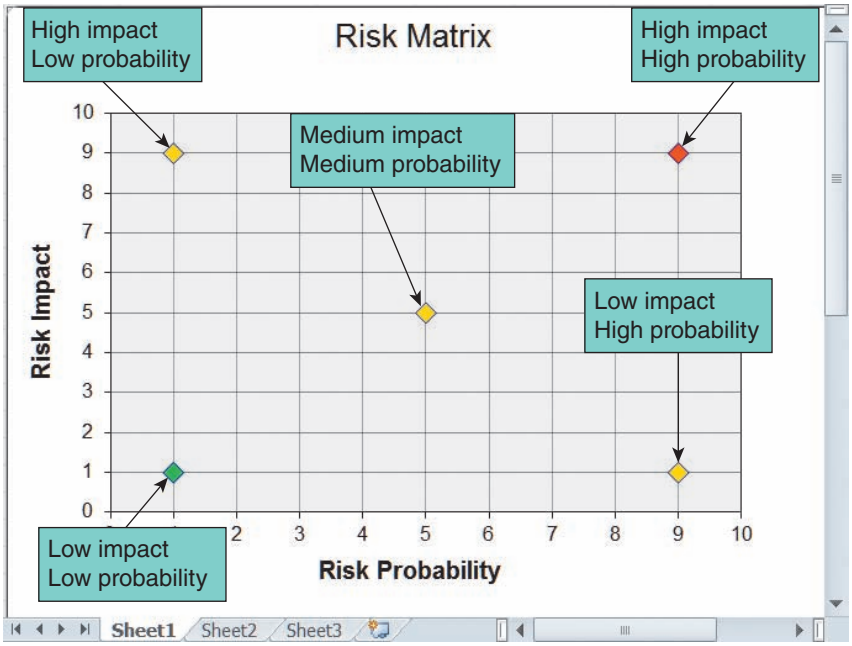
Risk Analysis

Risk analysis is the second stage in the risk management process, following the identification of potential risks. This stage involves assessing the identified risks based on their likelihood of occurrence and the severity of their impact should they materialize. Risk analysis aims to prioritize risks and help project managers understand the risks' potential influence on the project. This helps in decision making regarding the allocation of resources and the focus of risk management efforts.

Two types of risk analysis are commonly employed in systems analysis and design projects: qualitative and quantitative. **Qualitative risk analysis** involves assessing and prioritizing risks based on their probability and impact. Project managers can use a formula to weigh risk and impact values or display the results in a two-axis grid known as a risk matrix, where risks are ranked based on their potential impact and likelihood.

For example, a Microsoft Excel XY chart can display the risk matrix, as shown in Figure 3-19. The chart shows the various combinations of risk and impact ratings for the five sample values. This tool can help a project manager focus on the most critical areas with high risk probability and potential impact.

Figure 3-19 You can use a Microsoft Excel XY Chart type to display a risk matrix that shows risk probability and potential impact.



Quantitative risk analysis seeks to assign numerical values to risks, estimating their potential impact on the project objectives in terms of dollars, time, project scope, or quality. This could involve techniques such as sensitivity analysis, decision tree analysis, or Monte Carlo simulations. It can also involve a modeling process called what-if analysis, which allows a project manager to vary one or more elements in a model to measure the effect on other elements. This topic is discussed in more detail later in the book.

The output of the risk analysis activity forms a crucial input to the risk response planning stage. As with risk identification, risk analysis is not a one-time activity but should be periodically revisited as the project progresses. Changes in project parameters, environment, or stakeholder inputs can change the risk landscape, hence the need for regular updates and reassessments.

Risk Response

Creating a **risk response plan** is the next stage in risk management. After identifying and analyzing risks, the focus turns toward deciding how to address each. Each identified and analyzed risk requires a suitable response strategy, chosen based on the nature and severity of the risk. This stage aims to minimize the potential impact of risks on the project's objectives.

There are typically four risk response strategies: avoidance, transfer, mitigation, and acceptance. Risk avoidance involves changing the project plan to eliminate the risk or protect the project objectives from its impact. Risk transfer shifts the effects of the risk to a third party, typically through insurance or outsourcing. Risk mitigation seeks to reduce the probability or impact of the risk to an acceptable level. Lastly, risk acceptance involves acknowledging the risk and making a deliberate decision to accept it without engaging in special efforts to control it.

Regardless of the chosen strategy, each response must be assigned to a responsible individual, the **risk owner**, who monitors and manages the risk. Additionally, the chosen

responses form the basis for the risk management plan, a critical component of the overall project management plan. Like the previous steps, risk response planning is an ongoing process that should be revisited regularly as the project progresses.

3.7.4 The Risk Management Plan

The **risk management plan** is a summary of the risk identification, risk analysis, and risk response activities. It can include a review of the project's scope, stakeholders, budget, schedule, and any other internal or external factors that might affect the project. The risk management plan should address three additional areas: monitoring and control, roles and responsibilities, and risk communication.

The risk management plan should be reviewed and updated regularly, particularly when significant project changes occur. It serves as a roadmap for managing risks and helps ensure that everyone involved in the project understands how risks are to be dealt with.

Monitoring and Control

This component of the risk management plan is crucial, as it outlines how the project team will keep track of the identified risks, monitor residual risks, identify new risks that may arise, and ensure that risk responses are implemented effectively throughout the SDLC. These procedures aim to ensure that risks are continually reassessed and managed throughout the project life cycle, thereby minimizing their impact on the project's objectives.

This process is iterative and dynamic, as risks and their impacts and likelihood can change throughout the project. There could be triggers or signs that a risk event is about to occur, or new risks may be introduced due to changes in project scope, timeline, technology, or other factors. Hence, having a well-defined procedure for monitoring and controlling risks is vital.

The plan could outline specific risk indicators to watch for, regular risk review meetings, or specific risk monitoring tools to be used. It could include updating and revising the risk register, tracking metrics, reassessing existing risks, conducting audits, and keeping stakeholders informed about the status of risks.

On the control side, it involves executing the planned risk response strategies when necessary, reassessing the effectiveness of the strategies, and modifying them if they are not working as expected. It could also involve taking corrective action if the project veers off track due to realized risks.

This section of the plan also defines the frequency of review and update cycles. For some projects, this could be a weekly activity, while for others, it might be less frequent. Again, this largely depends on the nature and complexity of the project and the volatility of the project environment.

Roles and Responsibilities

This part of the risk management plan delineates the specific duties of each team member concerning risk management. It defines who will identify, analyze, and manage risks, and ensures a clear chain of responsibility. This could range from the person in charge of the overall risk management process, often the project manager, to the individuals responsible for specific tasks such as risk identification, analysis, and response implementation.

Some projects may designate risk owners who are responsible for specific identified risks. By explicitly stating who is responsible for what, the plan ensures that risk management tasks are performed efficiently and that there's clear accountability for each risk. This eliminates confusion about who should act when a risk event occurs and ensures that risks are not overlooked because everyone thought someone else was handling them.

Risk Communication

Effective risk communication (as with communications management in general, as described in Section 3.4) is crucial to successful risk management. This part of the plan describes how risks and risk management activities will be communicated to stakeholders. This might include the methods of communication (e.g., email updates, project meetings, risk reports),

the frequency of communication (e.g., daily, weekly, as needed), and the stakeholders who should receive the communication.

It also indicates what kind of information should be communicated, such as changes in project risks, results of risk analyses, and progress in implementing risk responses. The goal of the risk communication plan is to ensure that everyone involved in the project has the necessary information to make informed decisions and that there are no surprises when a risk event occurs.

3.7.5 Risk Management Issues

Risk management in systems projects is often challenging due to their inherent complexities and uncertainties. Some common risk management issues in such projects include inadequate risk identification, poor risk assessments, and ineffective risk responses. Addressing these issues requires a systematic, proactive approach to risk management, with a commitment from all stakeholders, effective communication, continuous monitoring and updating of risks, and a culture that values and understands the importance of risk management.

Inadequate Risk Identification

One of the biggest challenges of risk management is identifying all potential risks. Due to the complexity of systems projects, unknown risks may only become apparent as the project progresses. Some project teams might not invest sufficient time identifying potential risks or lack the necessary expertise to foresee potential threats. This could result in critical risks going unnoticed until they become actual issues. Inadequate risk identification can lead to unpleasant surprises and unfortunate disruptions in the project.

Poor Risk Assessments

Even when risks are identified, assessing their potential impact and likelihood can be challenging. If risks are not adequately evaluated, the team might underestimate the potential damage or overcommit resources to less significant threats. In addition, there may be a lack of objective criteria for evaluating risks, leading to subjective or inaccurate risk assessments. Similarly, a risk assessment based on incorrect or outdated information could lead to misguided decisions and strategies.

Ineffective Risk Responses

Once risks are identified and assessed, developing mitigation strategies is next. However, these strategies can often be ineffective or misaligned with the risk's nature without clear procedures or guidelines. This could be due to a lack of resources, a lack of commitment, or inadequate planning. Or, the risk management process might be reactive rather than proactive, addressing risks only after they've occurred. For instance, a team might rely on a generic response strategy for all identified risks, rather than tailoring their approach based on the specific risk characteristics. Ineffective risk responses can increase project costs and delays.

3.8 Project Management Software

Project management software plays a crucial role in the execution and control of systems projects. These projects often involve many tasks, diverse teams, numerous deadlines, and resources requiring careful orchestration. The software provides a comprehensive platform for managing these different components, helping to plan, execute, monitor, and control all project activities. Additionally, these tools promote real-time collaboration, document sharing, and communication among diverse stakeholders, including systems analysts, developers, testers, and end users, facilitating an effective and cohesive project environment.

One of the key benefits of project management software is the enhanced efficiency it brings to the project. The software streamlines operations by automating several project management tasks like scheduling, resource allocation, and progress tracking, saving time and effort. This

lets project managers concentrate on higher-value activities. Furthermore, these tools often incorporate risk management features, which are vital for identifying, analyzing, and controlling potential project risks, thus decreasing the likelihood of project disruptions or failures.

Project management software also promotes transparency and accountability by providing visibility into project progress, resource utilization, and task completion. This clarity fosters a sense of responsibility, as each team member can see who is accountable for each task and whether it has been completed as scheduled. In addition, the software supports informed decision making by offering up-to-date, accurate data on project status and performance. Lastly, the software serves as a repository for project documentation, historical data, and lessons learned, which is beneficial for knowledge management and future project planning. Integrating with other tools used in systems projects further enhances the utility of project management software, allowing for seamless workflows and reliable data across different tools.

3.8.1 Areas of Support

Project managers use software applications providing an array of features that assist in areas such as planning and scheduling, resource management, communication and collaboration, documentation and knowledge management, risk management, monitoring and control, and tool integration. By providing these capabilities, project management software can significantly enhance the efficiency and effectiveness of systems projects. It can help reduce project risks, improve team collaboration, ensure project objectives are met on time and within budget, and ultimately increase the likelihood of project success.

It's important to note that many modern project management tools are quite versatile and combine features from multiple categories. For instance, tools like Jira, monday.com, and Zoho Projects provide comprehensive functionalities that span task management, resource management, and portfolio management.

Planning and Scheduling

Complex systems projects often involve numerous interdependent tasks and milestones that must be carefully scheduled. Project management software provides tools like Gantt charts and critical path analysis that help create detailed project schedules, identify dependencies between tasks, and visualize the project timeline. This helps in setting realistic timelines and monitoring progress against the plan.

Resource Management

Resource management in systems projects, facilitated by project management software, involves effectively allocating and tracking organizational resources, including human resources and tangible assets. The software ensures optimal utilization of resources, balancing workloads, monitoring costs, and maintaining the project within its budget. Effective resource management helps to ensure that projects are completed on time and within predefined objectives.

Communication and Collaboration

Communication and collaboration are key in systems projects, and project management software enhances these aspects by providing platforms for real-time interaction, file sharing, and cooperative work. Such tools break down silos and allow diverse project stakeholders, including systems analysts, developers, and testers, to communicate and collaborate effectively. The result is a well-coordinated project team working seamlessly toward shared project goals.

Documentation and Knowledge Management

Documentation in systems projects is crucial, and project management software acts as a centralized repository for all project-related documents, from system specifications to user manuals. This centralized storage allows team members easy access to the information they need, reducing time spent searching for documents and enhancing overall project efficiency.

Furthermore, this documentation is a historical record, invaluable for future project planning and knowledge management.

Risk Management

Many project management software solutions include tools for identifying, tracking, and managing risks. These could include risk registers, risk analysis tools, and dashboards that visually overview the project's risk status. These tools can support the risk management process by making it easier to record, update, and communicate about risks.

Monitoring and Control

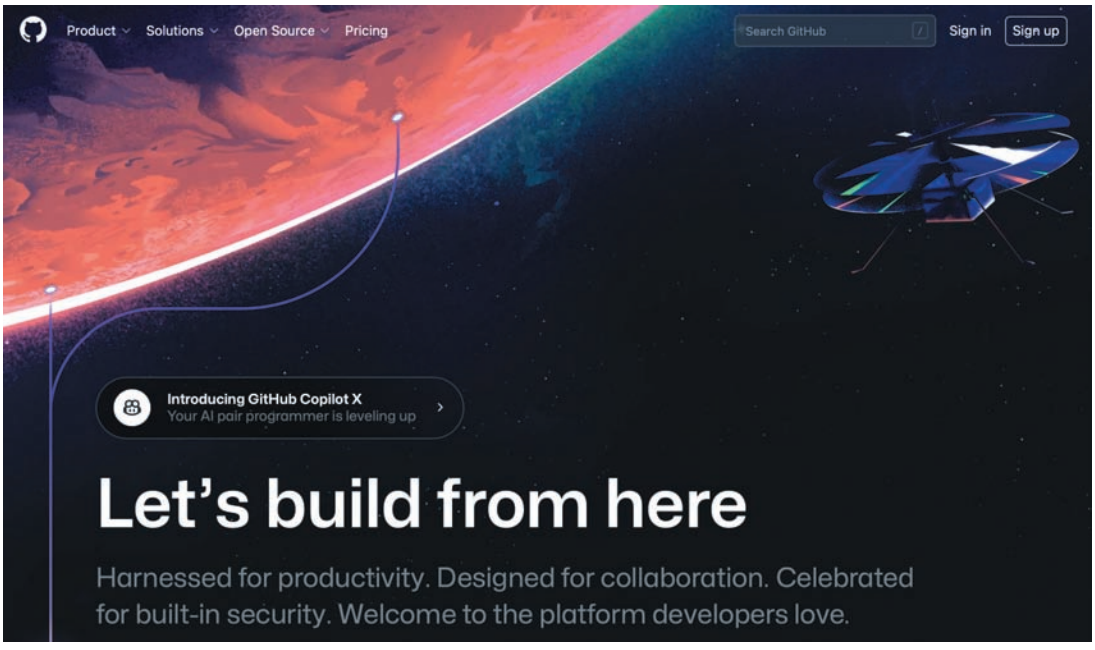
Project management software provides various tools for monitoring project progress and controlling deviations from the plan. These could include dashboards that display key performance indicators, reporting tools that generate detailed progress reports, and alert systems that notify the project manager when tasks are running late or resources are overallocated.

Tool Integration

A key advantage of project management software in systems projects is its ability to integrate with other software tools the team might use. This can include integration with coding and design tools, version control systems, testing tools, customer relationship management systems, and more. This interoperability helps streamline workflows, minimize manual data transfer, and ensure consistency and accuracy of information across different tools.

For instance, some project management software can integrate with tools like Microsoft's GitHub (see Figure 3-20), allowing you to track code commits and link them directly to specific tasks in your project plan. Similarly, integration with a tool like Jira can help track and manage bugs and issues directly within the project management software.

Figure 3-20 Microsoft's GitHub is a web-based hosting service for version control using Git, enabling developers to collaboratively work on projects, manage changes to code, and host open-source projects.
Source: GitHub, Inc.



This high level of integration increases efficiency and enhances visibility across the project, enabling team members and stakeholders to have a comprehensive and unified view of the project status. In addition, it ensures everyone can access the most recent and accurate project information, leading to better decision making and improved project outcomes.

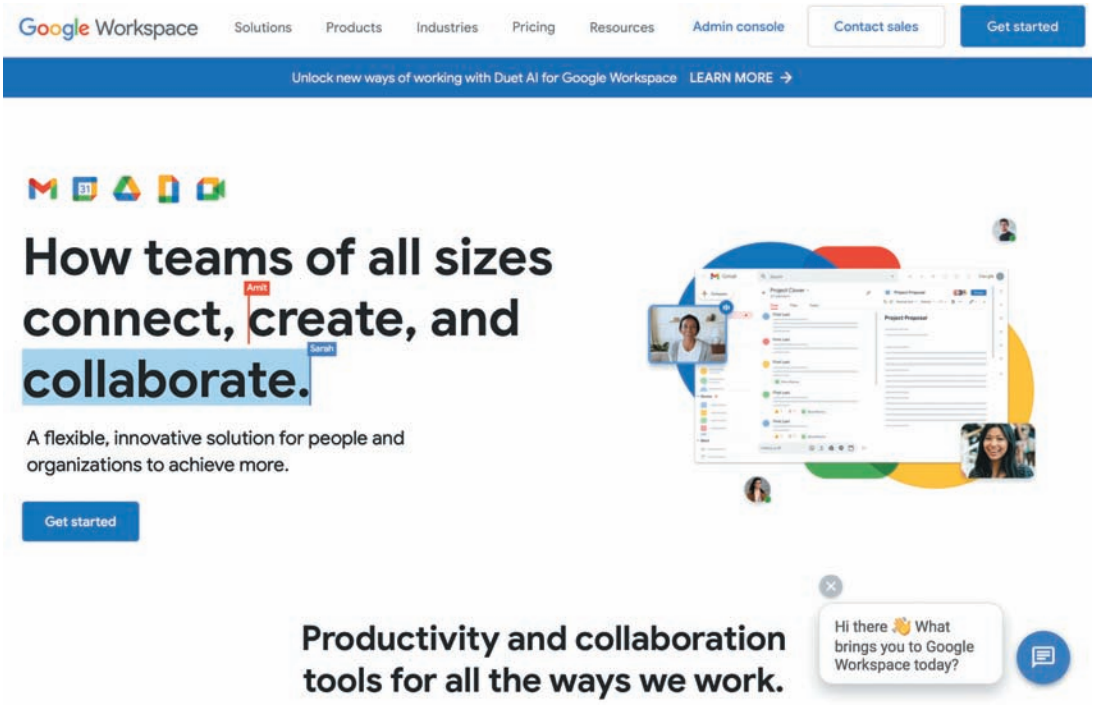
3.8.2 Examples of Project Management Software

Project management software is pivotal in planning, executing, and controlling various projects. They enable project leaders and team members to track tasks, collaborate, manage resources, and analyze project performance. Project management software includes tools and apps for collaboration, document management, issue and bug tracking, project portfolio management, resource management, task management, and time tracking. The websites for these tools have more information about their capabilities, including demos, trial versions (where applicable), and training material.

Collaboration

Collaboration software helps teams communicate and collaborate effectively, especially when team members work remotely or across different locations. These tools provide features like file sharing, real-time editing, comment threads, and integrated messaging systems. As a result, they aid in reducing communication bottlenecks and facilitate efficient collaboration among team members. Examples include Slack, Microsoft Teams, and Google Workspace (Figure 3-21).

Figure 3-21 Google Workspace lets teams of all sizes connect, create, and collaborate.
Source: Google Workspace



Document Management

These tools help teams organize, manage, and track documents related to the project. Features typically include version control, document sharing and collaboration, search functionality, and access control. They can be especially valuable in projects with a large volume of documentation or where documents must be regularly shared and collaborated on. Examples include Google Docs, Microsoft SharePoint, and Atlassian's Confluence (see Figure 3-22).

Issue and Bug Tracking

This type of software is crucial in software development and IT project management. It helps teams record, track, and manage issues and bugs that arise during the development process. These tools generally provide features like issue logging, tracking, assignment, and resolution status updates, making it easier for teams to collaborate on fixing issues and improving product quality. Examples include Jira, Bugzilla (see Figure 3-23), and GitHub Issues.

Figure 3-22 Atlassian's Confluence is a company-wide collaboration tool for creating and managing documentation for the entire team.
Source: Atlassian

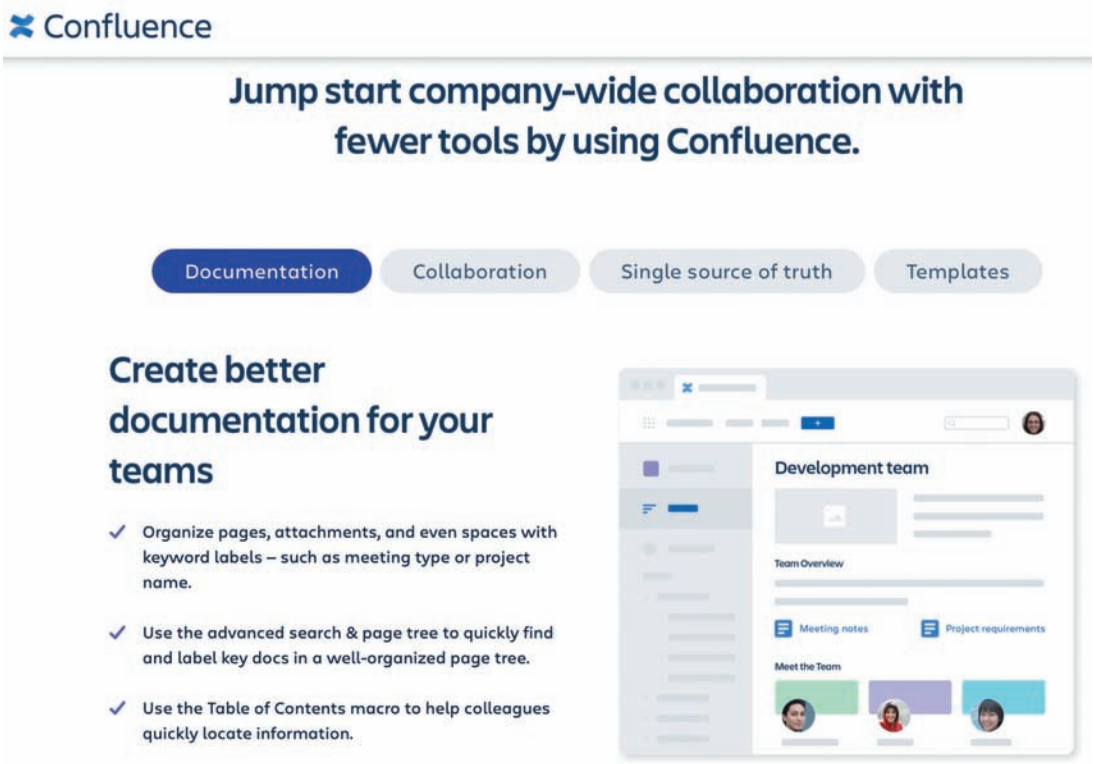
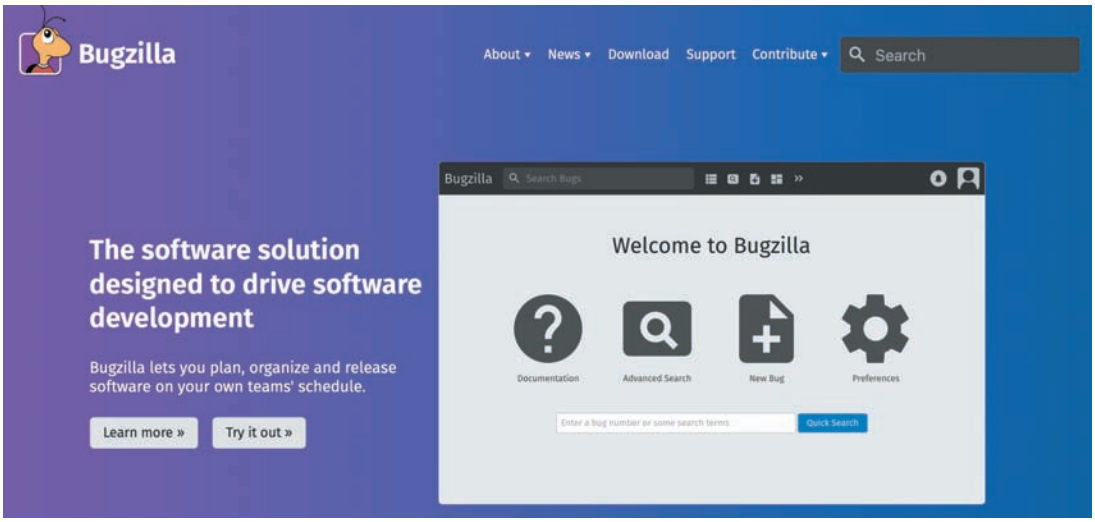


Figure 3-23 Bugzilla is a free, robust, and mature defect-tracking (bug-tracking) system.
Source: Bugzilla.org contributors



Project Portfolio Management

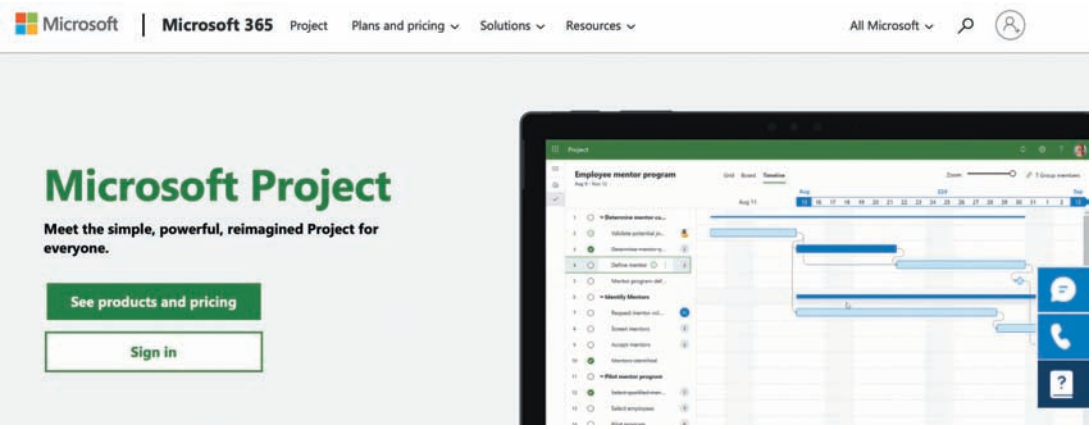
Project portfolio management (PPM) software provides a high-level perspective of all the projects in an organization's portfolio. It's a tool commonly used by project managers and executives to make strategic decisions, offering features to align projects with business objectives, track their progress, and manage associated risks.

PPM software provides a comprehensive view of all ongoing projects. It helps leaders understand how resources are allocated across the portfolio, how individual projects perform,

and how they align with the organization's broader strategic goals. These insights can inform decision making, enabling leaders to prioritize projects, allocate resources more effectively, and identify and mitigate potential risks.

As shown in Figure 3-24, Microsoft Project is a full-featured, traditional project management tool with a significant market share. It offers PERT/CPM and Gantt charts, resource scheduling, and project calendars. Microsoft Project is available as a software product for Windows and as an add-on online service as part of selected Office 365 plans.

Figure 3-24 Microsoft Project.
Source: Microsoft Corporation



In addition to Microsoft Project, several other project management tools are available. For example, GanttProject is a cloud-based project management tool that can automatically produce Gantt charts and PERT/CPM charts, calculate the critical path, and read/write Microsoft Project files. Gantter is a free, cloud-based project management tool. Apptivo Projects and Smartsheet are other project management tools offering similar capabilities.

As shown in Figure 3-25, the "monday work management" tool is the web-based project management component of the monday.com Work OS (operating system). Trello, shown in Figure 3-26, is a product management tool from Atlassian tailored toward agile development.

Resource Management

Resource management software is another crucial project management tool useful in systems projects. These tools help project managers effectively manage and allocate resources across different tasks or projects, such as personnel, hardware, software, and budgets. In addition, they provide an overview of the available resources, their current allocation, and their future availability.

One key benefit of resource management software is the ability to forecast resource needs and potential bottlenecks. By visualizing resource allocation and tracking resource utilization, project managers can identify overutilization or underutilization of resources and adjust allocations accordingly to ensure more efficient use of resources.

Examples of resource management software include Resource Guru (shown in Figure 3-27), known for its intuitive resource scheduling and time tracking features; Float, a popular tool for planning resources in agencies and studios; and Smartsheet, a versatile tool that combines project and resource management capabilities.

Task Management

Task management software is an essential tool in the world of project management, particularly when it comes to systems projects. These types of software are primarily used to manage, track, and coordinate individual tasks within a larger project. They often provide features such as to-do lists, deadlines, and task assignments, which help ensure that every team member knows their responsibilities and can monitor the progress of their tasks.

Figure 3-25 The "monday work management" tool is the project management component of the monday.com Work OS.
Source: Monday.com

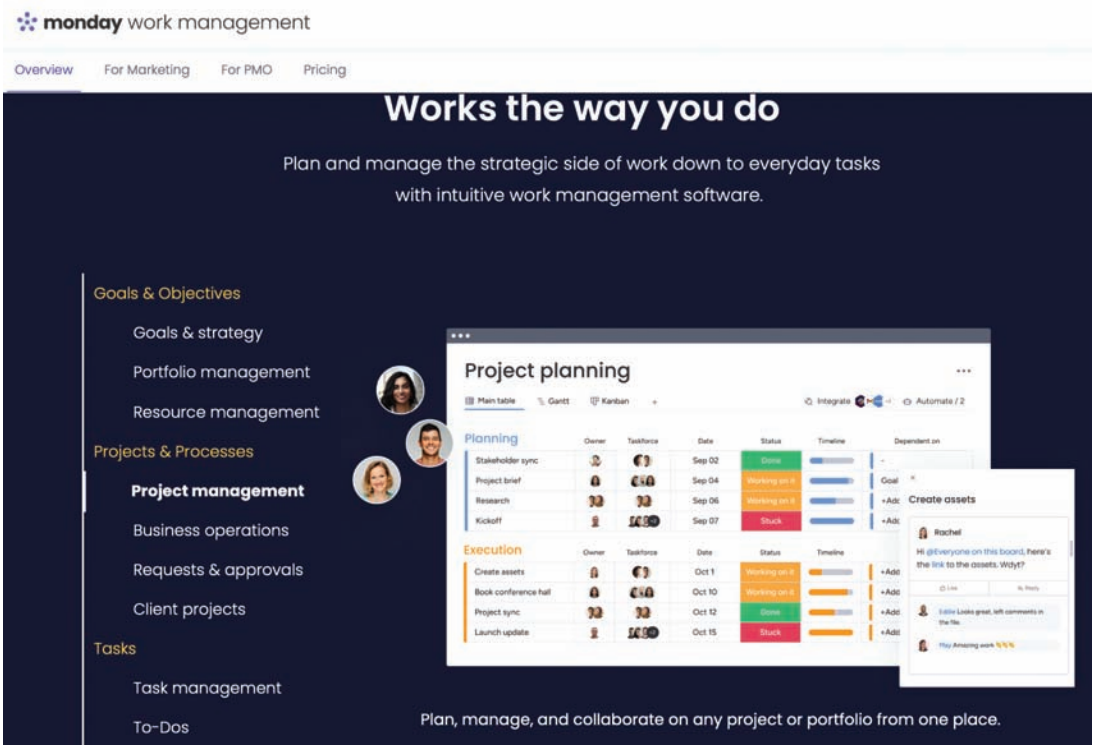


Figure 3-26 Trello is a product management tool tailored toward agile development.
Source: Atlassian

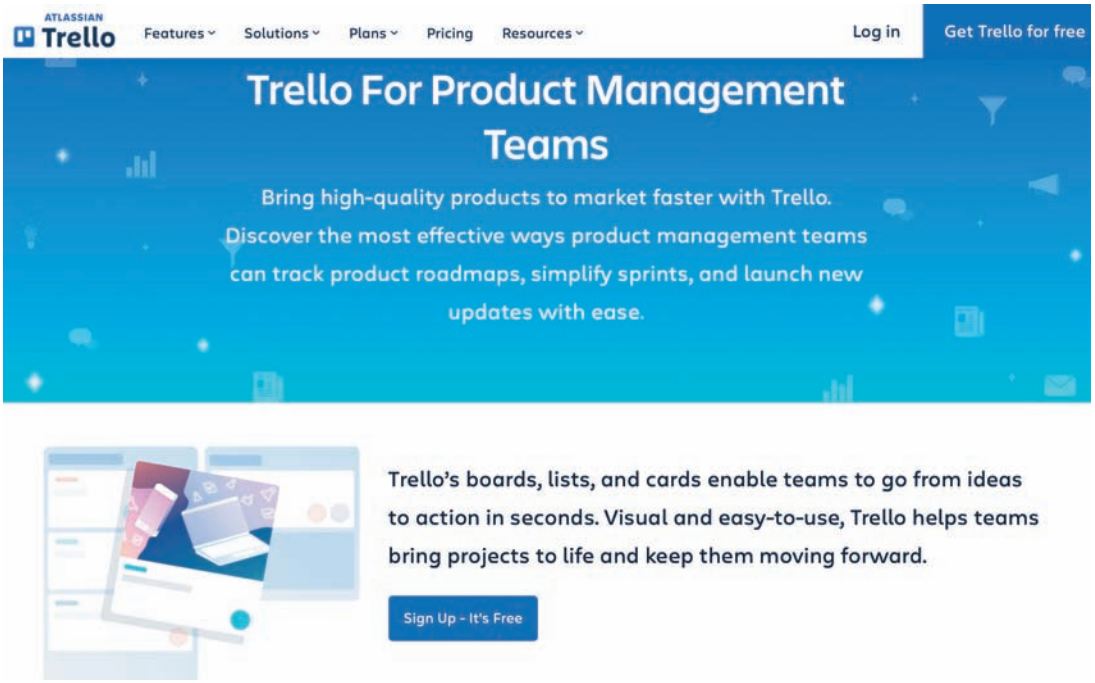
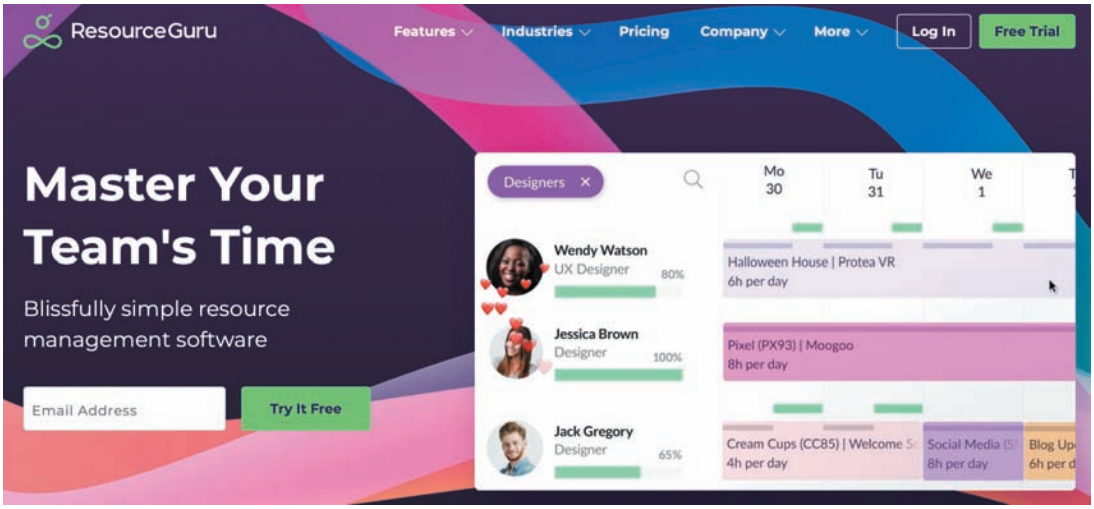


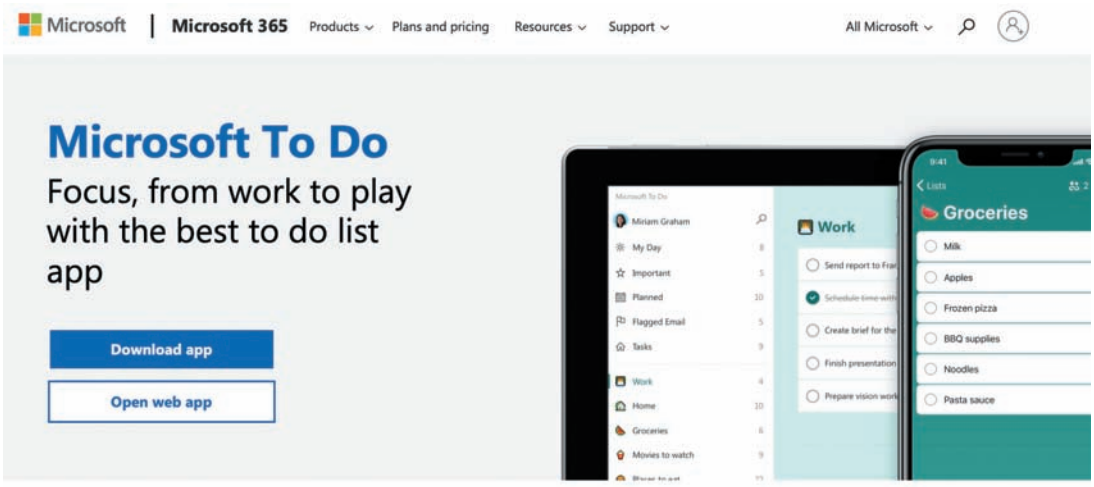
Figure 3-27 Resource Guru is an online resource management solution that uses a centralized resource pool and employee directory.
Source: Resource Guru



Task management software can also provide real-time updates and notifications to keep the team on the same page. This can significantly enhance communication within the group, as team members can discuss tasks, share updates, and resolve issues more efficiently. By enhancing visibility and communication, task management software can improve productivity and ensure tasks are completed promptly.

Examples of task management software include Asana, monday.com, and Trello. For Office 365 users, Microsoft To Do (shown in Figure 3-28) enables seamless task management within familiar software environments for personal and work use. It allows users to manage tasks from smartphones, tablets, browsers, and computers.

Figure 3-28 Microsoft To Do is a simple, cross-platform task management application.
Source: Microsoft Corporation



Time Tracking

This type of software allows teams to track the time spent on individual tasks or projects, helping to ensure efficiency and productivity. In addition, time-tracking tools can provide insights into how much time is spent on various tasks, which can help with project planning,

resource allocation, and performance evaluation. They can also facilitate accurate client billing for businesses that charge by the hour. Examples of time-tracking software include Harvest, Toggl, and TimeCamp.

3.8.3 Using Project Management Software

The following sections explain how Microsoft Project could be used to handle the sample task summary for a preliminary investigation shown in Figure 3-29. This example illustrates that project management is dynamic and challenging. Irrespective of which project management software is used, a step-by-step process is followed to develop a WBS, work with task patterns, and analyze the critical path. The main difference is that the software does most of the work automatically. One significant advantage of this approach is that it allows the project manager to rapidly adjust schedules, estimates, and resource assignments to respond to real-world events.

Figure 3-29 A sample task summary for a preliminary investigation.

Please study the following task summary:

- First, we need to spend one day studying potential problems or opportunities.
- Then, we will define the project scope and constraints. That will take two days.
- Next, we will analyze the organizational charts. That will take one day.
- After we analyze the charts, four fact-finding tasks can start at once:
  - Observe operations (two days)
  - Conduct a user survey (three days)
  - Conduct interviews (two days)
  - Review documentation (one day)
- When all four fact-finding tasks are finished, we will spend one day evaluating feasibility.
- Then we will spend one day presenting the results and recommendations to management.

Work Breakdown Structures

Creating a WBS using Microsoft Project is much the same as creating it manually. The tasks, durations, and task patterns must still be identified. This information might have to be developed, or a task summary like the one in Figure 3-29 might be used. The goal is to document all tasks, dependencies, dates, and total project duration. The first step is to create a Gantt chart showing the necessary information. As the information for each task is entered into Microsoft Project, the duration and the predecessor tasks, if any, should also be noted.

Gantt Charts

As tasks are entered, the program automatically performs the calculations, detects the task patterns, and creates a Gantt chart. The chart consists of horizontal bars connected with arrows that indicate task dependencies. For example, tasks will not be scheduled on Saturdays and Sundays if a typical workweek is selected. However, for a mission-critical project, a 24/7 calendar might be created. Whatever is specified, the program will handle the tasks accordingly. Microsoft Project offers numerous display settings, formats, and calculation methods.

Network Diagrams

After the Gantt chart is completed, the data can be viewed as a Microsoft Project network diagram, which is similar to a PERT chart. When the Network Diagram option is selected, the project tasks, dependencies, and a start and finish date for each task are shown. A network

diagram displays the same information as the Gantt chart, including task dependencies, but uses task boxes to include much more detail. Using Microsoft Project, each task can be assigned to one or more people, budgets assigned targets, progress reports produced, and schedules and deadlines readjusted as necessary.

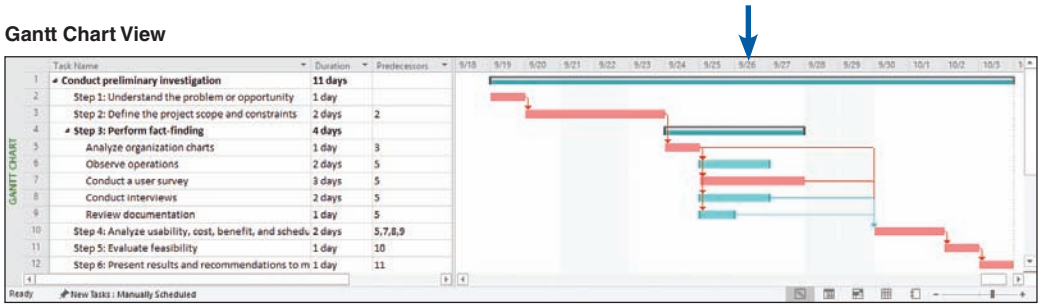
Calendar View

A calendar view is an excellent way to manage day-to-day activity. This view shows the tasks, similar to a PERT chart, as an overlay on the actual calendar. Because the critical path is highlighted in red, it is easy for a project manager to determine priorities at any point in time.

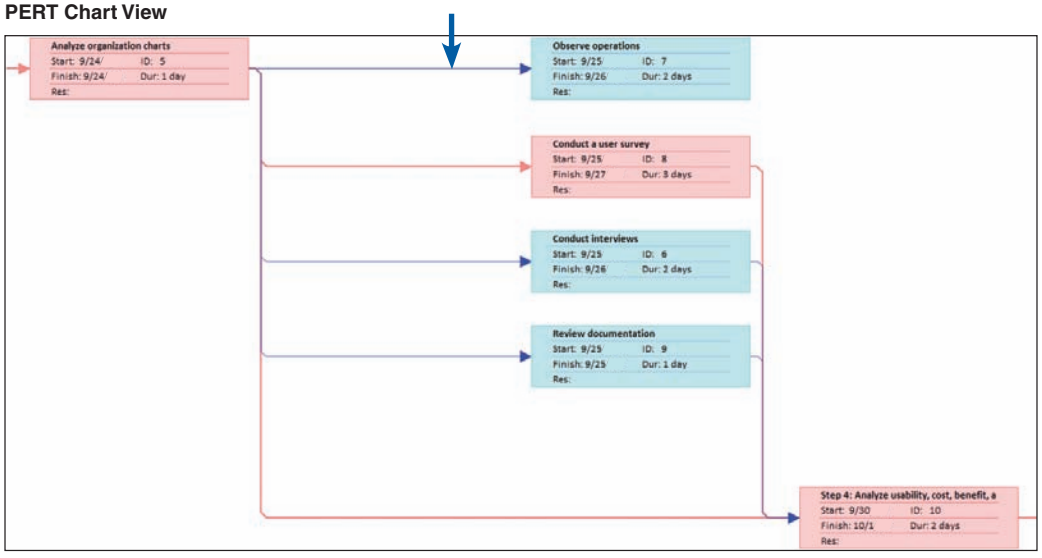
Suppose the project manager wants to view the preliminary investigation in Figure 3-29 as a Gantt chart, a PERT chart, and a day-to-day calendar. All three views are shown in Figure 3-30. Each view shows the tasks, the timing, the dependencies, and the critical path. Notice that of the four tasks scheduled for September 25, only the user survey is on the critical path. Therefore, that should be the project manager's primary concern.

Figure 3-30 Notice how each view displays the project and highlights the critical path. If you were the project manager on September 25, your primary concern should be conducting the user survey.

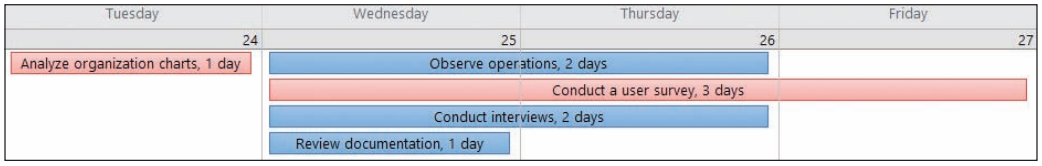
Gantt Chart View



PERT Chart View



Calendar View



3.9 Managing for Success

Managing a systems project is a challenging task. It requires a balanced approach that addresses technical requirements while focusing on people and processes. Project managers must be alert, technically competent, and highly resourceful. Successfully managing systems projects requires attention to several areas, including project monitoring and control, managing issues as they arise, and adopting an agile mindset that acknowledges the inevitable changes that will occur as the project proceeds. Project managers can be proud when they handle a successful project that helps the company achieve its business objectives.

3.9.1 Project Monitoring and Control

Project monitoring and control is a vital aspect of managing systems projects regardless of whether the project was planned and scheduled with project management software or in some other manner. It involves the processes, tools, and techniques required to keep the project on track and meet its goals and objectives. It comprises tracking, reviewing, and regulating progress to meet the performance objectives defined in the project management plan.

Furthermore, communication (see Section 3.4) plays a vital role in project monitoring and control, so project managers also must be good communicators with strong human resource skills. They should regularly update stakeholders to ensure transparency, manage expectations, and facilitate informed decision making.

In addition, a crucial part of project monitoring and control is managing risks (see Section 3.7). This involves identifying potential issues that could disrupt the project, estimating their impact, developing mitigation strategies, and monitoring risks throughout the SDLC.

Tracking

Tracking involves regularly collecting and analyzing project progress, budget, schedule, and scope data. This could include tracking milestones, deliverables, resources used, and costs incurred. Tools such as project management software can assist in automating data collection and providing real-time insight.

Reviewing

Periodic reviews compare the project's actual progress with the planned progress. In the context of a systems project, this could involve reviewing the results of each phase of the SDLC, such as requirements analysis, design, implementation, and testing.

Regulating

Corrective actions are taken if deviations from the project plan or guiding quality standards are identified during the review process. For project planning issues, this could involve adjusting schedules, reallocating resources, or modifying the project scope. For example, this might include revising design plans, reallocating developer time, or reassessing project priorities in a systems project.

To help ensure that quality standards are met, many project managers institute structured walk-throughs. A **structured walk-through** is a review of a project team member's work by other team members. Generally, systems analysts review the work of other systems analysts, and programmers review the work of other programmers as a form of peer review. Structured walk-throughs occur throughout the SDLC and are called **design reviews**, **code reviews**, or **testing reviews**, depending on the phase in which they appear.

3.9.2 Issue Resolution

A multitude of issues will inevitably arise across the SDLC, leading to projects derailing for various reasons. When problems occur, the project manager's ability to resolve the situation becomes critical. When a project manager first recognizes that a project is in trouble, what options are available? Alternatives can include trimming the project requirements, adding to

the project resources, delaying the project deadline, and improving management controls and procedures. Sometimes, when a project experiences delays or cost overruns, the system can still be delivered on time and within budget if several less critical requirements are trimmed. The system can be delivered to satisfy the most necessary requirements, and additional features can be added later as a part of a maintenance or enhancement project.

If a project is in trouble because of a lack of resources or organizational support, management might be willing to give the project more commitment and higher priority. For example, management might agree to add more people to a project that is behind schedule. Adding staff, however, will reduce the project's completion time only if the additional people can be integrated effectively into the development team. If team members lack experience with certain aspects of the required technology, temporary help might be obtained from IT consultants or part-time staff. Adding staff can mean training and orienting the new people, however. In some situations, adding more people to a project might actually increase the time necessary to complete the project because of a principle called **Brooks' Law**. Frederick Brooks Jr., then an IBM engineer, observed that adding staffing to a late software project only makes it later. Brooks reached this conclusion when he saw that new workers on a project first had to be educated and instructed by existing employees, whose productivity was reduced accordingly.

To succeed, an information system must satisfy business requirements, stay within budget, be completed on time, and—most importantly—be managed effectively. As stated above and detailed below, when a project develops problems, the reasons typically involve business, people, technology, budget, or schedule issues. Therefore, in addition to planning and managing the project, a project manager must be able to recognize these issues and deal with them effectively.

Case in Point 3.3

Just-in-Time Software

You are a systems analyst at Just-in-Time Software, a company that specializes in short delivery cycles for its products. One of your projects is running behind schedule, and the manager wants to bring in a few extra programmers to help with the work.

You are familiar with Brooks' Law. How can you explain to the project manager that adding more people to the project at this late stage may worsen things? You don't want to be seen as a negative team player, but you're convinced that the project's schedule will slip even more if you don't speak up.

Business Issues

The primary objective of every system is to provide a solution to a business problem or opportunity. If the system does not do this, then it is a failure—regardless of positive reaction from users, acceptable budget performance, or timely delivery. When the information system does not meet business requirements, causes can include unidentified or unclear requirements, inadequately defined scope, imprecise targets, shortcuts or sloppy work during systems analysis, poor design choices, insufficient testing or inadequate testing procedures, and lack of change control procedures.

As explained elsewhere in the book, projects without clear scope definitions are risky because they tend to expand gradually, without specific authorization, in a process called *project creep*. However, even when a project is clearly described, it must be managed constantly.

Various business-related challenges can arise from the intersection of technology and business operations, and how changes in one affect the other. Understanding and addressing these business issues is crucial for the project's success. It requires a blend of

technical expertise, business acumen, and strong communication and change management skills to navigate the complexities of business transformations. Three common business-related problems that may arise are misalignment with business objectives, business process reengineering, and ROI justification.

Misalignment with Business Objectives. One of the primary challenges in systems projects is ensuring alignment with business objectives. The designed system should support the company's strategic goals, whether increasing efficiency, improving customer service, or driving innovation. If the system does not align with these goals or the business objectives change during the project, it can lead to conflicts, wasted resources, and a system that doesn't deliver the expected value.

Business Process Reengineering. Implementing a new system often requires changing existing business processes, which can be a significant challenge. Employees may resist changes to their established routines, and there can be logistical challenges in reorganizing workflows. If these process changes aren't managed effectively, it can lead to disruption, decreased productivity, and potential failure of the new system.

ROI Justification. The cost of systems projects can be high, and businesses often need to justify this investment by demonstrating a positive ROI. Calculating ROI for such projects can be complex, as benefits may be indirect or long-term. If the system does not deliver the expected financial return or if costs are higher than anticipated, it can lead to dissatisfaction among stakeholders and jeopardize future projects.

People Issues

As described in Section 3.3, managing a systems project involves technical challenges and people issues. Three common people-related challenges are communication breakdown, resistance to change, and skill gaps.

Communication Breakdown. Effective communication is crucial for successfully completing a project. When there's a communication breakdown, misunderstandings can arise, causing confusion about project requirements, roles and responsibilities, or project status. This can lead to errors, delays, and conflicts. For instance, if a systems analyst fails to communicate design specifications clearly to the development team, the final system may not meet the intended user requirements.

Resistance to Change. Introducing a new system often means changes to an organization's workflows, processes, or roles. People often resist change, especially if they feel it threatens their jobs or if they're comfortable with the existing system. Such resistance can slow project implementation and potentially cause the new system to fail once deployed.

Skill Gaps. Systems projects often involve using new technology or implementing innovative solutions. The project can suffer if team members lack the necessary skills or knowledge. This could lead to errors, inefficiencies, or delays. It could also lead to frustration among team members, lower morale, and increased resistance to the project.

Technology Issues

Systems projects, by their nature, are heavily dependent on technology. While this reliance on technology offers many benefits, it can also introduce potential challenges. Managing these technology issues requires staying abreast of the latest technologies and trends, robust planning, and ongoing review throughout the project. A proactive approach to technology management can help anticipate and mitigate these challenges, leading to more successful systems projects. Three common issues might arise: technological compatibility, rapid technological change, and security and privacy concerns (described in more detail later in the book).

Technological Compatibility. One of the frequent issues encountered in systems projects is technological compatibility. The new system may need to integrate with existing systems, platforms, or software, and compatibility issues can arise during this process. For instance, there might be data format discrepancies, communication protocol mismatches, or different software requirements. These issues can delay the project and increase costs.

Rapid Technological Change. Technological change is fast, and what is considered state-of-the-art technology today may become outdated tomorrow. A tool or technology chosen at the beginning of the project may become obsolete by the time the project is nearing completion. This can lead to inefficiencies, higher costs, and even the need to redo some parts of the project.

Security and Privacy. Data security and privacy are paramount in the digital age. When designing a new system, it's necessary to include robust security measures to protect sensitive data and ensure compliance with relevant laws and regulations. However, designing for security is often complex and requires specialized knowledge. If not handled properly, security vulnerabilities can lead to data breaches, legal issues, and a loss of trust among users.

Budget Issues

Systems projects involve significant financial planning and management. Despite the best efforts of project managers, budget-related issues can still arise. Effective financial management—including accurate budget planning, regular financial review, and proactive management of risks and changes—can help mitigate these budget-related challenges and keep the project on track.

As described in Section 3.4, it's also essential to maintain clear and regular communication with all stakeholders about the project's financial status and any issues that arise, to manage expectations and ensure informed decision making. Three common budget issues that may be encountered are cost overruns, unexpected expenses, and inaccurate financial forecasting.

Cost Overruns. One of the most common budget-related problems is cost overruns, where the project's actual cost exceeds the initial budget. This can occur due to unforeseen complexities, changes in project scope (also known as scope creep), or underestimation of the expenses during the budgeting process. For example, if the price of hardware or software licenses increases during the project, or if additional personnel are needed, this can lead to increased costs that were not originally budgeted.

Unexpected Expenses. Unexpected expenses can arise at any stage of the project, throwing the project budget off track. These could be related to technical challenges, such as the need for additional software tools, hardware upgrades, or system reworks due to design flaws. Alternatively, they could stem from nontechnical factors, such as changes in regulation, market fluctuations affecting the price of resources, or personnel-related costs.

Inaccurate Financial Forecasting. Accurate financial forecasting is crucial in budget planning for systems analysis and design projects. However, errors or oversights in the initial financial forecasting can lead to an unrealistic or unsustainable budget. This could involve underestimating costs, overestimating available funds, or failing to account for potential risks and contingencies.

Schedule Issues

Proper scheduling is crucial in systems projects to ensure that deliverables are completed on time and within budget. However, maintaining a project schedule can be challenging, and most projects run into at least some problems, including project delays, overlapping tasks, and underestimation of task duration. By monitoring and controlling the work, the project manager tries to anticipate problems, avoid them or minimize their impact, identify potential solutions, and select the best solution.

Effective project management, including detailed planning, regular progress tracking, and proactive problem solving, can help to mitigate these scheduling issues. Using project management tools can also assist in visualizing the project schedule, identifying potential issues before they arise, and making adjustments as needed.

The better the original plan, the easier it will be to control the project. If clear, verifiable milestones exist, it will be simple to determine if and when those targets are achieved. If enough milestones and frequent checkpoints exist, problems will be detected rapidly. A project planned and scheduled with PERT/CPM or in a WBS with Gantt charts can be tracked and controlled using these same techniques. As work continues, the project manager revises the plan to record actual times for completed tasks and adjusts times for tasks that still need to be finished.

Project Delays. Delays are one of the most common scheduling issues. They can occur due to various reasons, such as unexpected technical challenges, changes in project scope, resource unavailability, or external factors like changes in market conditions or regulatory requirements. Such delays can push back the project timeline, increasing costs and potentially causing stakeholder conflict.

Overlapping Tasks. Problems with timetables and project milestones can indicate a failure to recognize task dependencies, confusion between effort and progress, poor monitoring and control methods, personality conflicts among team members, or turnover of project personnel. If task dependencies are not managed correctly, you might end up with overlapping tasks, which can cause bottlenecks in the workflow. For example, suppose the design phase of a system is scheduled to begin before the analysis phase is complete. In that case, it can lead to delays and inefficiencies, as the design team may need more information to start their work.

Underestimation of Task Duration. Estimating how long each task within the project will take is an essential part of scheduling. If task durations are underestimated, this can disrupt the project schedule. It's common to underestimate the time needed for tasks, especially in complex projects where certain tasks may be unfamiliar or unexpected challenges arise. There is an old saying that people overestimate what can be done in the short term and underestimate what can be done in the long term, and systems projects are no different in this regard.

3.9.3 Adopting an Agile Mindset

Systems projects often involve complex, dynamic requirements. In such cases, an agile mindset can be highly effective. Unlike traditional project management methodologies that try to define all requirements upfront, the agile approach allows for flexibility and iterative development where appropriate. An agile *mindset* can be adopted even when using a traditional project management approach.

As described elsewhere in this book, in an agile project, work is broken down into small, manageable chunks (or "sprints"), and each sprint ends with a deliverable that can be reviewed and tested. This allows for regular feedback and adjustments, ensuring the final system is closely aligned with user needs.

Moreover, agile methodologies emphasize regular reflection and continuous improvement in the product being developed and the team's working processes. An agile mindset also fosters an environment that encourages collaboration and teamwork, respects diverse perspectives, and constructively resolves conflict. This can lead to higher-quality outcomes and a more engaged, effective project team.

Summary

Successfully managing systems projects is about getting the job done and delivering a system that meets user needs and adds value to the organization while creating a positive experience for the project team. It is a complex activity requiring a specialized skill set for managing projects, tasks, people, communications, technology, finances, and risks. Fortunately, project management software is available to make the challenge more manageable.

Project management for systems professionals includes planning, scheduling, monitoring and controlling, and reporting on information system development. A project manager will break the project into individual tasks, determine the order in which the tasks must be performed, and determine how long each task will take. With this information, Gantt charts or PERT/CPM charts can be used to schedule and manage the work.

Task management is integral to project management because most projects are composed of discrete tasks that must be managed individually and collectively. Task management involves creating a WBS, modeling task patterns, and identifying the critical path. A WBS involves breaking a project down into a series of smaller tasks. Task patterns determine the order in which the tasks are performed. A critical path is a series of tasks that, if delayed, would affect the completion date of the overall project. If any task on the critical path falls behind schedule, the entire project will be delayed.

People management is a skill vital for a project's success. A project manager must understand the human element by empathizing with users, managing team dynamics, fostering growth, handling change, and building trust. These aspects of project management can significantly impact the project's success. It also enhances team morale, improves user satisfaction, and ultimately leads to better, more effective systems.

Effective communication is a critical aspect of systems projects. However, it can often pose significant challenges. Communication-related issues that a project manager must address include miscommunication of requirements, lack of regular project updates, ineffective change communication, cultural and language barriers, and dealing with problems.

Technology management for systems projects refers to the strategic selection, implementation, and review of the technology resources used throughout the SDLC. It's a crucial aspect of ensuring that the systems developed are efficient, reliable, and capable of meeting the organization's current and future needs. Technology implementation and review related to product development and systems support strategies are covered in more detail later in the book.

Sound financial management is a crucial, often overlooked aspect of a successful systems project. Understanding the economic implications of systems projects can profoundly impact project outcomes and the overall efficiency of the business operation. The most critical areas of concern include budgets and cost estimation, skills and technology investment, and ROI analysis.

Every systems project involves risks that systems analysts and project managers must address. A risk is the possibility of an adverse event that could negatively affect the project. Therefore, risk management is a fundamental aspect of managing systems projects, as it helps identify, assess, and prioritize uncertainties that could impact the achievement of project objectives.

Project managers use software applications to help plan, schedule, monitor, and report on a project. Most programs offer features such as PERT/CPM, Gantt charts, resource scheduling, project calendars, and cost tracking. Microsoft Project is a full-featured program with a significant market share, but several other options are available. Irrespective of which project management tool is used, a step-by-step process is followed to develop a WBS, work with task patterns, and analyze the critical path. The main difference is that the software does most of the work automatically, which enables much more effective project management.

Successfully managing a systems project requires a balanced approach that addresses technical requirements while focusing on people and processes. Project managers must be alert, technically competent, and highly resourceful. In addition, they must pay attention to several areas, including project monitoring and control, managing issues as they arise, and adopting an agile mindset that acknowledges the inevitable changes that will occur as the project proceeds.

Key Terms

activity
best-case estimate
bottom-up technique
Brooks' Law
code review
concurrent task
critical path
Critical Path Method (CPM)
Delphi technique
dependent task
design review
duration
event
finish day/date
Gantt chart
machine learning (ML)
milestone
network diagram
person-day
PERT/CPM
predecessor task
probable-case estimate
Program Evaluation Review Technique (PERT)
project coordinator
project leader
project management
project manager
project monitoring
project planning
project reporting
project scheduling
project triangle
qualitative risk analysis
quantitative risk analysis
risk
risk analysis
risk identification
risk management
risk management plan
risk owner
risk response plan
slack time
start day/date
structured walk-through
successor task
task
task box
task group
task ID
task name
task pattern
testing review
weight
work breakdown structure (WBS)
worst-case estimate

Review Questions

1. Write a brief explanation of basic project management concepts.
2. Explain the differences between a Gantt chart and a PERT/CPM chart.
3. What are the three main task pattern types? Provide an example of each.
4. Explain why trust is a fundamental aspect of effective people management.
5. What happens when project updates are irregular or nonexistent?
6. Why is functionality a key criterion in technology selection?
7. Summarize why investing in skills and technology is a critical factor in the financial management aspect of systems projects.
8. Why are requirements changes a significant source of risks in system projects?
9. What is a significant advantage of using project management software?
10. Explain Brooks' Law.

Discussion Topics

1. When using a project triangle to illustrate conflicting priorities, Microsoft suggests that if the problem is in the fixed leg, work on the other two legs. For example, if the project must be within the budget and is starting to run over, adjust the schedule, the scope, or both. However, the adjustment might be in the remaining portion if the problem is unrelated to the fixed leg. So, when faced with a limited budget (fixed leg) and the schedule is slipping (problem leg), the project's scope (remaining leg) might have to be adjusted. Why is explaining this situation to management sometimes difficult for the systems analyst?

2. Consider a scenario where a task depends on another task being started but not necessarily completed. For example, a project may depend on a task being started and one-fourth completed before the group can start their portion. Do you think this situation occurs frequently in systems projects? Why or why not?
3. If you manage a large project, would you prefer Gantt charts or PERT/CPM charts to communicate project status? Explain why.
4. Some project management applications can be quite expensive. As a manager, how would you justify the purchase of this software?
5. Risk analysis is typically a two-step process: qualitative risk analysis and quantitative risk analysis. As a systems analyst, for which sorts of project management decisions would you use the results from qualitative risk analysis? From quantitative risk analysis?

Hands-On Projects

1. Think of all the tasks that you perform when you purchase a car. Include any research, decisions, or financial issues related to the purchase. Create a WBS that shows all the tasks, their estimated duration, and any predecessor tasks.
2. Table 3-3 shows a WBS with 11 tasks. Notice that each task has an ID, a description, duration, and a reference to predecessor tasks, if any, which must be completed before the task can begin. Also, notice that dependent tasks can have one predecessor task or several. Construct a PERT/CPM chart from these tasks. Recall that this is done as a two-step process: (1) display the tasks and task patterns, and (2) enter start and finish times.

Table 3-3 Example of a WBS listing 11 tasks, together with their descriptions, durations, and predecessor tasks.

Task No.	Description	Duration (Days)	Predecessor Tasks
1	Develop Plan	1	-
2	Assign Tasks	4	1
3	Obtain Hardware	17	1
4	Programming	70	2
5	Install Hardware	10	3
6	Program Test	30	4
7	Write User Manual	25	5
8	Convert Files	20	5
9	System Test	25	6
10	User Training	20	7, 8
11	User Test	25	9, 10

3. Many of today's projects involve team members scattered across different time zones and physical locations. Moreover, the projects may have adopted an agile methodology, dramatically reducing cycle time. Write a brief report summarizing some of the key differences a manager would face with this type of project as opposed to a traditional one.
4. Perform an Internet search to learn more about project risk management and write a summary of the results. Be sure to search for the classic book *Waltzing with Bears: Managing Risk on Software Projects* by Tom DeMarco and Timothy Lister.

5. Go to the websites for project management tools besides Microsoft Project, such as Apptivo (www.apptivo.com), GanttProject (www.ganttproject.biz), Gantter (www.gantter.com), Smartsheet (www.smartsheet.com/product-tour/gantt-charts), Monday (www.monday.com), and Trello (www.trello.com). Explore each program's features and describe what you like and don't like.

Ethical Issues

"Better blow the whistle," says your friend and project teammate. "The project is out of control, and you know it!" "Maybe so," you respond, "but that's not my call—I'm not the project manager." What you don't say is that the project manager feels like their career is on the line and is reluctant to bring bad news to management. They believe the project can catch up and say an alarming report on a major project could result in bad publicity for the firm and frighten potential customers.

To be fair, the next management progress report is due in three weeks. It is possible that the team could catch up, but you doubt it. You wonder if there is an ethical question here: Even though the report isn't due yet, should a significant problem be reported to management as soon as possible? You are concerned about the issue and decide to discuss it with the project manager. What will you say to them?

Phase 2

Analysis

Analysis, the second SDLC phase, lays the groundwork for accurately defining system requirements and ensuring that all aspects of the system are thoroughly understood and documented. This stage utilizes methodologies like data and process modeling to outline and examine system operations, facilitating the early identification of potential issues. Object modeling through the Unified Modeling Language (UML) enables a precise depiction of system components and their interactions, promoting effective communication among stakeholders and guiding the progression toward efficient and effective system solutions.

Chapter 4 addresses requirements engineering, focusing on system requirements, their types, and associated challenges. It explores team-based techniques like joint application development, rapid application development, agile methods, and various requirements elicitation techniques such as interviews, document reviews, observations, and brainstorming. The chapter also discusses representing requirements using natural language, diagrams, and models, emphasizing the importance of validation and verification. It concludes with the role of tools in requirements engineering.

Chapter 5 discusses data and process modeling, comparing logical and physical models. It introduces data flow diagrams (DFDs) and their symbols, covering the creation of DFDs at different levels, from context diagrams to lower-level DFDs, with examples for grading and order systems. The chapter highlights the significance of data dictionaries in documenting system components and examines process description tools used in modular design.

Chapter 6 focuses on object modeling and object-oriented analysis. It defines objects, attributes, methods, messages, and classes and provides examples. The chapter explores relationships among objects and classes and introduces UML. It covers various UML diagrams, including use case, class, sequence, state transition, activity, and business process modeling diagrams. The chapter also addresses the role of tools in object modeling.

Chapter

4

Requirements Engineering

Learning Objectives

When you finish this chapter, you should be able to:

1. Explain system requirements and the challenges associated with the requirements engineering process.
2. Apply team-based requirements engineering techniques, including joint application development (JAD), rapid application development (RAD), and agile methods.
3. Develop a fact-finding plan for gathering requirements.
4. Conduct an interview to gather system requirements.
5. Use other requirements gathering techniques, including document review, observation, questionnaires and surveys, brainstorming, sampling, and research.
6. Explain how requirements are gathered in agile projects.
7. Utilize different requirements representation techniques, including natural language, diagrams, and models.
8. Explain how to validate and verify requirements.
9. Explain how tools can help with requirements engineering activities.

Chapter 4 is the first of three chapters in the systems analysis phase of the systems development life cycle (SDLC). This chapter describes the requirements engineering process: gathering facts about a systems project, creating models that will be used to design and develop the system, and verifying and validating that the models are correct before proceeding to the next phase.

The chapter includes three "Case in Point" discussion questions to help contextualize the concepts described in the text. The scenario described in the "Ethical Issues" section raises concern over a request by a supervisor to identify departments that reported the lowest ratings in an anonymous survey.

4.1 System Requirements

During the SDLC's first part of the systems analysis phase, systems analysts must identify and describe all system requirements. A **system requirement** is a characteristic or feature that must be included in an information system to satisfy business requirements and be acceptable to users. System requirements serve as benchmarks to measure the overall acceptability of the finished system.

Because system requirements are the focus of the start of the systems analysis phase, it's essential to get them correct at the beginning. Any problems with the requirements will have a ripple effect that could negatively affect subsequent phases of the SDLC. Poor **requirements engineering** is a leading cause of failed projects.

Requirements engineering is composed of three main activities:

1. Gathering requirements: *understanding* the problem
2. Representing requirements: *describing* the problem
3. Validating and verifying requirements: *agreeing* on the problem

Each of these activities is described in more detail in this chapter. The output of requirements engineering is requirements documents that capture the essence of what the system should do. These documents are the input to the next step in the SDLC, data and process modeling, described later in the book.

4.1.1 Types of Requirements

Requirements can be classified according to various characteristics. For example, requirements may be primarily for the system user, in which case they are referred to as **requirements definitions**. Requirements may also be primarily for the engineering team, in which case they are called **requirements specifications**.

System requirements can also be classified as functional and non-functional. A **functional requirement** is a statement of the services a system provides. Examples of functional requirements include the following:

- The website shall report online volume statistics every four hours and hourly during peak periods.
- The inventory system shall produce a daily report showing the part number, description, quantity on hand, quantity allocated, quantity available, and unit cost of all products sorted by part number.
- The contact management system shall generate a daily reminder list for all sales representatives.
- Each input form must include the date, time, product code, customer number, and quantity.
- The system must provide login security at the operating system and application levels.

A **non-functional requirement** is a statement of operational system constraints. Non-functional requirements are also known as **quality attributes**. Examples include the following:

- Data entry screens must be uniform, except for the background color, which the user can change.
- The system must support 25 users online simultaneously.
- Response time must be four seconds at most.
- The system must be operational seven days a week, 365 days a year.
- The system should work on Windows and Mac platforms.

Non-functional requirements may be more critical than functional requirements; the system is only useful if the former is satisfied. Conflicts between different non-functional requirements are common in complex systems. For example, a user may request that a system be 100% secure but very usable—two requirements that are hard to reconcile.

4.1.2 Requirements Challenges

Requirements present numerous challenges to the systems analyst. Three of the most important are imprecision, agreement, and creep.

Imprecision

Requirements are often imprecise because they are usually represented using natural language, as shown in the previous section. Natural language is expressive, but it is prone to misinterpretation. It is not uncommon for stakeholders to completely disagree about the meaning of a simple requirement. For this reason, other techniques are used to represent requirements, as described in Section 4.7.

A requirement may be a high-level abstract statement of a service or a system constraint, but it can also be a detailed mathematical specification. This is because requirements often serve two functions: a basis for a bid for a contract and the basis for the contract itself.

If the requirements are the basis for the contract bid, it must be open to interpretation. If they are the basis for the contract itself, they must be defined in detail. These two constraints are difficult to satisfy.

Agreement

One of the main problems with requirements is getting everyone to agree on the exact meaning of the requirements statements. In other words, we want to develop requirements so that, upon completion of the system, both the systems analyst and the client can agree on whether a specific requirement has been met.

In theory, requirements should be both complete and consistent. A requirement is complete if it includes descriptions of all facilities the system needs. In practice, it's impossible to describe the requirements for a complex system completely.

A requirement is consistent if there are no conflicts or contradictions in the description of the system facilities. Ensuring no conflicts among thousands of system requirements is quite challenging.

Creep

Many social and organizational factors influence system requirements. For example, business changes inevitably lead to changing requirements—usually, more of them as the project progresses. This is particularly true for long-lived projects in which personnel change over time. This phenomenon is known as "feature creep."

Rapidly changing requirements can cause problems for systems analysts and other team members. This is particularly true for projects that follow a traditional waterfall model of the SDLC. This is partly why agile methods are popular: They explicitly address changing requirements as part of the project's management structure.

4.1.3 Additional Considerations

In addition to the three challenges outlined above, systems analysts must consider three important supplementary factors: (1) scalability, which determines how a system will handle future growth and demands; (2) security, which is all-important for today's networked systems; and (3) the total cost of ownership, which includes all future operational and support costs.

Scalability

Scalability refers to a system's ability to handle increased business volume and transactions in the future. Because it will have a longer useful life, a scalable system offers a better return on the initial investment.

Information about projected future volume for all outputs, inputs, and processes is needed to evaluate scalability. For example, for a web-based order processing system, one needs to know the maximum projected number of concurrent users, the periods of peak online activity, the number and types of data items required for each transaction, and the method of accessing and updating customer files.

Even to print customer statements, the analyst needs to know the number of active accounts and forecast for one, two, or five years because that information affects future hardware decisions. In addition, with realistic volume projections, reliable cost estimates can be provided for related expenses, such as postage and online charges.

Similarly, the analyst would need to project activity levels for several years of operation to ensure that a web-based hotel reservation system is sufficiently scalable. For example, one might forecast the frequency of online queries about room availability and estimate the time required for each question and the average response time. With that information, server transaction volume and network requirements could be estimated.

Transaction volume has a significant impact on operating costs. When the volume exceeds a system's limitations, maintenance costs increase sharply. Volume can change dramatically if a company expands or enters a new line of business. For example, a new Internet-based marketing effort might require an additional server and 24-hour technical support.

Data storage also is an important scalability issue. The analyst must determine the required data storage and predict future needs based on system activity and growth. Those requirements affect the hardware, software, and network bandwidth needed to maintain system performance. Data retention requirements must also be considered to determine whether data can be deleted or archived on a specific timetable.

Security

In the past, security was considered an add-on to a system's design. This is particularly true for legacy systems deployed for some time. Nowadays, security is so crucial for networked systems that it has changed from a non-functional requirement to a functional requirement. In other words, security is an essential consideration for all systems development.

Incorporating security as a first-class requirement is paramount in light of the seemingly endless news reports of massive data breaches. These hacks release personal information from online systems at a scale previously unseen. If the systems had been made with security in mind from the beginning, they would be much harder to infiltrate.

The challenge with security as a system requirement is that it often conflicts with other user requirements. For example, requiring the system to be accessible online using a web interface immediately makes securing the system much more challenging. The systems analyst must attempt to reconcile and prioritize the conflicting requirements during the requirements engineering process.

Total Cost of Ownership

In addition to direct costs, systems developers must identify and document indirect expenses contributing to the **total cost of ownership (TCO)**. TCO is vital if the development team is assessing several alternatives. After considering the indirect costs, which are not always apparent, an inexpensive system might be the costliest choice. One problem is that cost estimates tend to understate indirect costs such as user support and downtime productivity losses. Even if accurate figures are unavailable, systems analysts should try to identify indirect costs and include them in TCO estimates.

Because cost control is so important, vendors often claim that their products or services will reduce TCO significantly. For example, one of the most common reasons to migrate a legacy system to the cloud is reduced TCO. Cloud computing offers lower operational costs due to outsourcing expenses such as capital investment in exchange for a pay-as-you-go pricing model.

4.2 Team-Based Techniques

The IT department aims to deliver the best possible information system, at the lowest possible cost, in the shortest possible time. System developers view users as partners in the development process to achieve the best results. Greater user involvement usually results in better communication, faster development times, and more satisfied users.

The traditional model for systems development was an IT department that used structured analysis and consulted users only when their input or approval was needed. Although the IT staff still has a central role, and structured analysis remains a popular method of systems development, most IT managers invite system users to participate actively in various development tasks.

As described earlier in the book, team-based approaches have existed for some time. For example, **joint application development (JAD)** is a user-oriented technique for fact-finding and requirements engineering. Because it is not linked to a specific development methodology, systems developers use JAD whenever group input and interaction are desired.

Another popular user-oriented method is **rapid application development (RAD)**. RAD resembles a condensed version of the entire SDLC, with users involved every step of the way. While JAD typically focuses only on fact-finding and requirements determination, RAD provides a fast-track approach to a full spectrum of system development tasks, including planning, design, construction, and implementation.

Finally, as described earlier in the book, agile methods represent a recent trend emphasizing intense interaction between system developers and users. JAD, RAD, and agile methods are discussed in the following sections.

4.2.1 Joint Application Development

JAD is a collaborative and interactive approach used in systems analysis and design to gather requirements and facilitate the development of information systems. It involves bringing stakeholders, end users, and development teams together in structured workshops or meetings to identify, define, and refine system requirements.

The key objective of JAD is to create a shared understanding among all participants and ensure that the system being developed meets the needs of the users and stakeholders. By involving key individuals from different backgrounds, such as business analysts, subject matter experts, system developers, and end users, JAD aims to capture a comprehensive and accurate representation of the system requirements.

Planning and preparation. The JAD sessions are planned, including selecting participants, defining the scope, and setting goals and objectives for the sessions. Relevant materials and documentation, such as existing system documentation, process flows, and business rules, are gathered and distributed to the participants before the JAD sessions. This helps in providing context and background information.

Conducting JAD sessions. The JAD sessions are facilitated by a trained moderator or facilitator who guides the participants through a structured agenda. The participants engage in collaborative activities such as brainstorming, group discussions, and requirements elicitation exercises to identify and document system requirements. Visual aids, such as whiteboards or electronic tools, are often used to capture and document information.

Requirements documentation. The information gathered during the JAD sessions is organized and documented clearly and concisely. This documentation includes functional requirements, non-functional requirements, use cases, process models, data models, and other relevant artifacts.

Review and validation. The documented requirements are reviewed and validated by the participants to ensure accuracy and completeness. This step helps identify gaps or inconsistencies requiring further clarification or refinement.

Iterative process. JAD sessions are often conducted iteratively, allowing for continuous refinement of requirements. The sessions may be repeated until a consensus is reached among the participants and the requirements are considered finalized.

User Involvement

Users have a vital stake in an information system and should participate fully in the development process. Many years ago, the IT department usually had sole responsibility for systems development, and users had a relatively passive role. During development, the IT staff would collect user information, define system requirements, and construct the new system. At various stages of the process, the IT staff might ask users to review the design, offer comments, and submit changes.

Today, users typically have a much more active role in systems development. IT professionals now recognize that successful systems must be user oriented, and users must be involved formally or informally at every stage of system development.

One popular strategy for user involvement is a JAD team approach, which involves a task force of users, managers, and IT professionals who gather information, discuss business needs, and define the new system requirements.

JAD Participants and Roles

A JAD team usually meets for days or weeks in a special conference room or off-site location. Either way, JAD participants should be insulated from the distraction of day-to-day operations. The objective is to analyze the existing system, obtain user input and expectations, and document user requirements for the new system.

The JAD group usually has a project leader who needs strong interpersonal and organizational skills, and one or more members who document and record the results and decisions. Table 4-1 describes typical JAD participants and their roles. IT staff members often serve as JAD project leaders, but that is not always true. Systems analysts on the JAD team participate in discussions, ask questions, take notes, and support the team. Analysts can

Table 4-1 Typical JAD participants and roles.

JAD Participant	Role
JAD project leader	Develops an agenda, acts as a facilitator, and leads the JAD session
Top management	Provides enterprise-level authorization and support for the project
Managers	Provide department-level support for the project and understanding of how the project must support business functions and requirements
Users	Provide operational-level input on current operations, desired changes, input and output requirements, user interface issues, and how the project will support day-to-day tasks
Systems analysts and other IT staff members	Provide technical assistance and resources for JAD team members on issues such as security, backup, hardware, software, and network capability
Recorder	Documents results of JAD sessions and works with systems analysts to build system models and develop CASE tool documentation

develop models and enter documentation from the JAD session directly into the CASE tool if the tools are available.

A typical JAD session agenda is shown in Table 4-2. The JAD process involves an intensive effort by all team members. Because of the wide input range and constant participant interaction, many companies believe that a JAD group produces the best possible definition of the new system.

Table 4-2 Typical agenda for a JAD session.

Topic Leader	Activities
Project leader	• Introduce all JAD team members • Discuss ground rules, goals, and objectives for the JAD sessions • Explain methods of documentation and use of CASE tools, if any
Top management (sometimes called the project owner or sponsor)	• Explain the reason for the project and express top management authorization and support
Project leader	• Provide overview of the current system and proposed project scope and constraints • Present outline of specific topics and issues to be investigated
Open discussion session, moderated by project leader	• Review the main business processes, tasks, user roles, input, and output • Identify specific areas of agreement or disagreement • Break team into smaller groups to study specific issues and assign group leaders
JAD team members working in smaller group sessions, supported by IT staff	• Discuss and document all system requirements • Develop models and prototypes
Group leaders	• Report on results and assigned tasks and topics • Present issues that should be addressed by the overall JAD team
Open discussion session, moderated by project leader	• Review reports from small group sessions • Reach consensus on main issues • Document all topics
Project leader	• Present overall recap of JAD session • Prepare report that will be sent to JAD team members

JAD Benefits

JAD provides a structured and collaborative approach to gathering requirements, ensuring that the developed system aligns with the needs and expectations of the stakeholders. Compared with traditional methods, JAD can be more expensive and cumbersome if the group is too large relative to the project size. However, many companies find numerous benefits to using JAD in systems projects.

Stakeholder Involvement. JAD encourages active participation from all relevant stakeholders, ensuring their perspectives and requirements are considered in the system development process.

Improved Communication. By bringing stakeholders together in a collaborative environment, JAD promotes effective communication and shared understanding, reducing the chances of miscommunication and misunderstandings.

Time and Cost Efficiency. JAD sessions help streamline the requirements gathering process by condensing multiple meetings into focused workshops. This can save time and resources compared to traditional one-on-one interviews or lengthy feedback cycles.

Accelerated Decision Making. JAD sessions facilitate real-time decision making by allowing participants to discuss and resolve issues on the spot rather than relying on back-and-forth communication or delayed feedback.

Reduced Rework. By involving stakeholders early on and obtaining their feedback iteratively, JAD helps identify and address requirements gaps or inconsistencies, reducing the need for major rework during the later stages of system development.

Case in Point 4.1

North Hills College

North Hills College has decided to implement a new registration system allowing students to register online and in person. As an IT manager, you decide to set up a JAD session to help define the requirements for the new system. The North Hills organization is fairly typical, with an administrative staff including a registrar, student support and services team, business office, IT group, and several academic departments. Using this information, you start planning to carry out the JAD session. Whom would you invite to the session, and why? What would be your agenda for the session, and what would take place at each stage?

4.2.2 Rapid Application Development

RAD is an iterative and incremental team-based approach to systems analysis and design that focuses on quickly developing high-quality software applications. It emphasizes active user involvement, prototyping, and iterative development cycles to accelerate the software development process. Like JAD, RAD uses a group approach but goes much further. While the end product of JAD is a requirements model, the end product of RAD is the new information system.

Iterative and incremental development. RAD follows an iterative and incremental development process, breaking down the project into multiple smaller iterations or phases. Each iteration typically lasts for a short period, ranging from a few weeks to a couple of months, and involves the completion of a subset of system functionality.

Active user involvement. RAD emphasizes continuous and active user involvement throughout development. Users actively participate in requirements gathering, design reviews, and testing, providing feedback and guiding the development team.

Prototyping. RAD extensively uses prototyping techniques to build working models of the software application quickly. These prototypes are used to gather user feedback, validate requirements, and make design decisions. The prototypes evolve iteratively, allowing for progressive refinement and improvement.

Timeboxing. RAD projects are timeboxed, meaning a fixed time frame is allocated for each iteration or phase. This helps maintain a sense of urgency and discipline, ensuring the project progresses rapidly.

Collaborative development. RAD encourages close collaboration among the development team members, including analysts, designers, developers, and testers. The team works closely, often in the same physical space, to facilitate communication, knowledge sharing, and rapid decision making.

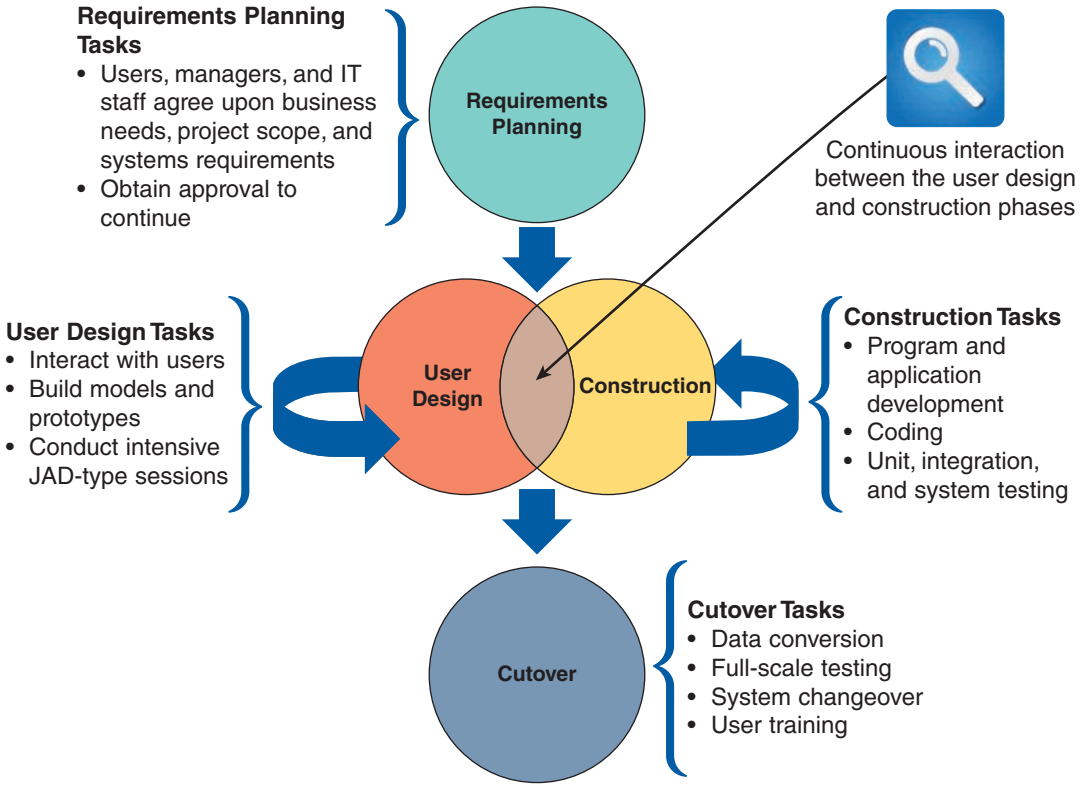
Reusability. RAD promotes the reuse of existing components, frameworks, and libraries to accelerate development. By leveraging prebuilt components, developers can focus on building the unique and critical aspects of the application, reducing the overall development time.

Continuous integration and testing. RAD emphasizes continuous integration and testing throughout development. Frequent integration of developed components and continuous testing help identify and address issues early on, ensuring the overall quality and stability of the application.

RAD Phases and Activities

The RAD model consists of four phases: requirements planning, user design, construction, and cutover, as shown in Figure 4-1. Notice the continuous interaction between the user design and construction phases.

Figure 4-1 The four phases of the RAD model are requirements planning, user design, construction, and cutover. Notice the continuous interaction between the user design and construction phases.



Requirements Planning. The **requirements planning phase** combines elements of the SDLC's systems planning and systems analysis phases. Users, managers, and IT staff members discuss and agree on business needs, project scope, constraints, and system requirements. The requirements planning phase ends when the team agrees on the key issues and obtains management authorization to continue.

User Design. During the **user design phase**, users interact with systems analysts and develop models and prototypes representing all system processes, outputs, and inputs. The RAD group or subgroups typically combine JAD techniques and CASE tools to translate user needs into working models. User design is a continuous, interactive process that allows users to understand, modify, and eventually approve a working model of the system that meets their needs.

Construction. The **construction phase** focuses on program and application development tasks similar to the SDLC. In RAD, however, users continue to participate and can suggest changes or improvements as screens or reports are developed.

Cutover. The **cutover phase** resembles the final tasks in the SDLC implementation phase, including data conversion, testing, the changeover to the new system, and user training. Compared with traditional methods, the entire process is compressed. As a result, the new system is built, delivered, and placed in operation much sooner.

RAD Objectives

The main objective of all RAD approaches is to cut development time and expense by involving users in every phase of systems development. Because it is a continuous process, RAD allows the development team to make necessary modifications quickly as the design evolves. With tight corporate budgets, limiting the cost of changes typically occurring in a long, drawn-out development schedule is especially important.

In addition to user involvement, a successful RAD team must have IT resources, skills, and management support. Because it is a dynamic, user-driven process, RAD is especially valuable when a company needs an information system to support a new business function. By obtaining user input from the beginning, RAD also helps a development team design a system that requires a highly interactive or complex user interface.

RAD Benefits

RAD offers a streamlined and flexible approach to systems analysis and design, allowing for the quick and efficient development of software applications while maintaining a strong focus on user satisfaction. The primary advantage of RAD is that systems can be developed more quickly with significant cost savings. Care must be taken to ensure the accelerated time cycle does not result in lower-quality products. However, RAD can be an attractive alternative if an organization understands the possible risks.

Faster Time to Market. RAD's iterative and incremental nature allows for rapid development and deployment of software applications, enabling faster value delivery to end users.

Improved User Satisfaction. Active user involvement and prototyping in RAD ensure that the system meets users' needs and expectations. User feedback is incorporated early and frequently, resulting in higher user satisfaction.

Increased Flexibility and Adaptability. RAD's iterative approach enables flexibility and adaptability to changing requirements. As user feedback is continuously incorporated, modifications and adjustments can be made easily throughout the development process.

Risk Mitigation. RAD's incremental and iterative approach helps mitigate risks associated with software development. Issues and challenges are identified and addressed early, reducing the likelihood of major problems later in the project.

Enhanced Collaboration. RAD promotes close collaboration among team members, facilitating communication and knowledge sharing. This fosters a sense of ownership and teamwork, leading to improved collaboration and productivity.

4.2.3 Agile Methods

As mentioned earlier in the book, **agile methods** attempt to develop a system incrementally by building prototypes and constantly adjusting them to user requirements. Developers revise, extend, and merge earlier versions into the final product as the agile process continues. An agile approach emphasizes continuous feedback, and each incremental step is affected by what was learned in the previous steps.

A large community of agile-related software and services has evolved as agile methods become more popular. Many agile developers prefer not to use CASE tools at all; as shown in Figure 4-2, they rely instead on whiteboard displays and arrangements of movable sticky notes. This approach, they believe, reinforces the agile strategy: simple, rapid, flexible, and user oriented.

Figure 4-2 Reinforcing the agile strategy: simple, rapid, flexible, and user oriented.
Sam Edwards/OJO Images/Getty Images

Scrum is another agile approach. The name comes from the rugby term, where players lunge at each other to achieve their objectives, as shown in Figure 4-3. The systems development version of Scrum involves the same intense interaction, though it is more mental than physical. Agile team members play specific roles in a Scrum session, including colorful designations like "pigs" or "chickens." These roles are based on the old joke about the pig and chicken, who discuss a restaurant where ham and eggs would be served. However, the pig declines because that role would require a total commitment, while it would only be a contribution for the chicken.

Figure 4-3 In a rugby scrum, players prepare to lunge at each other to achieve their objectives.
Getnikov/Shutterstock.com

In the agile world, the pigs include the product owner, the facilitator, and the development team, while the chickens include users, other stakeholders, and managers. Scrum sessions have specific guidelines that emphasize time blocks, interaction, and team-based activities that result in deliverable software. An agile team uses a series of scrums to pause the action and allow the players to reset the game plan, which remains in effect until the next scrum.

Agile Benefits

Using agile methods in requirements engineering for systems projects offers several benefits. These benefits collectively contribute to a more collaborative, responsive, and customer-centric approach to requirements engineering in systems analysis and design projects. Agile methods enable teams to adapt to changing needs, actively involve stakeholders, deliver value incrementally, provide transparency, and maintain a strong focus on quality.

Adaptability to Changing Requirements. Agile methods embrace change and provide a flexible approach to requirements engineering. As customer needs evolve or new information emerges, agile teams can easily accommodate changes and adjust the requirements accordingly. This adaptability ensures that the developed system meets the changing needs of stakeholders.

Continuous Customer Involvement. Agile methods emphasize close collaboration and continuous customer involvement throughout development. Customers, end users, and other stakeholders actively participate in requirements discussions, feedback sessions, and reviews. This high level of engagement ensures that the delivered system aligns with customer expectations and minimizes the risk of misinterpretation or misunderstanding.

Early and Frequent Delivery of Value. Agile methods promote iterative and incremental development, allowing for the early and frequent delivery of functional increments or minimum viable products (MVPs). This enables stakeholders to experience working software and provide feedback early in the project. By delivering value iteratively, agile teams can validate requirements, identify issues, and make necessary adjustments promptly.

Increased Transparency and Visibility. Agile methods promote transparency by providing clear visibility into the development process and progress. Techniques like user stories, Kanban boards, and burndown charts enable stakeholders to track the status of requirements, understand development priorities, and monitor progress. This transparency helps build trust, manage expectations, and foster effective communication among project participants.

Continuous Improvement and Quality Focus. Agile methods prioritize continuous improvement and a strong focus on quality. Through regular retrospectives, the team reflects on its processes, identifies areas for improvement, and implements changes. Quality assurance practices, such as continuous integration, automated testing, and frequent reviews, ensure that the system meets the desired standards and reduces the risk of defects or inconsistencies.

4.3 Requirements Elicitation

Requirements elicitation (gathering) or **fact-finding** (collecting information) is the first step in the requirements engineering process. During requirements gathering, the systems analyst will use various techniques, including interviews, document review, observation, surveys and questionnaires, sampling, and research, whether working solo or as a team member.

Although software can help gather and analyze requirements, no program collects them automatically. First, the information needed must be identified. Typically, this activity begins by asking a series of questions, such as:

- What business functions are supported by the current system?
- What strategic objectives and business requirements must be supported by the new system?
- What are the benefits and TCO of the proposed system?
- What transactions will the system process?
- What information do users and managers need from the system?
- Must the new system interface with legacy systems?
- What procedures could be eliminated by business process reengineering?
- What security issues exist?
- What risks are acceptable?
- What budget and timetable constraints will affect system development?

To obtain answers to these questions, the analyst develops a fact-finding plan, which involves answers to five familiar questions: who, what, where, when, and how. For each of those questions, one also must ask another essential question: why. Some examples of these questions are:

1. *Who*? Who performs each of the procedures within the system? Why? Are the correct people performing the activity? Could other people perform the tasks more effectively?
2. *What*? What is being done? What procedures are being followed? Why is the process necessary? Often, procedures are followed for many years, and no one knows why. Question why a procedure is being followed at all.
3. *Where*? Where are operations being performed? Why? Could they be performed more efficiently elsewhere?
4. *When*? When is a procedure performed? Why is it being performed at this time? Is this the best time?
5. *How*? How is a procedure performed? Why is it performed in that manner? Could it be performed better, more efficiently, or less expensively in some other way?

There is a difference between asking what is being done and what could or should be done. The systems analyst must first understand the current situation. Only then can the analyst answer the question of what should be done. Table 4-3 lists the basic questions and when they should be asked. Notice that the first two columns relate to the current system, but the third column focuses on the proposed system.

Table 4-3 Sample questions during requirements elicitation as the focus shifts from the current system to the proposed system.

Current System		Proposed System
Who does it?	Why does this person do it?	Who should do it?
What is done?	Why is it done?	What should be done?
Where is it done?	Why is it done there?	Where should it be done?
When is it done?	Why is it done then?	When should it be done?
How is it done?	Why is it done this way?	How should it be done?

4.4 Interviews

Interviewing is an important requirements elicitation technique during the systems analysis phase. An **interview** is a planned meeting during which the analyst obtains information from another person. The skills needed to plan, conduct, document, and evaluate interviews successfully must be understood.

4.4.1 The Interview Process

The interviewing process commences after identifying the information needed, as described earlier in the chapter. The process consists of seven steps for each interview:

1. Determine the people to interview.
2. Establish objectives for the interview.
3. Develop interview questions.
4. Prepare for the interview.
5. Conduct the interview.
6. Document the interview.
7. Evaluate the interview.

Step 1: Determine the People to Interview

To get an accurate picture, the analyst must select the right people to interview and ask them the right questions. The preliminary investigation involved mainly middle managers or department heads. Now, during the systems analysis phase, people from all levels of the organization should be interviewed. In some situations, it might be prudent to interview stakeholders who are not members of the organization because their opinion is valuable.

Although interview candidates can be selected from the formal organizational charts reviewed earlier, one must also consider any informal structures in the organization. **Informal structures** usually are based on interpersonal relationships and can develop from previous work assignments, physical proximity, unofficial procedures, or personal relationships such as the informal gathering shown in Figure 4-4. In an informal structure, some people have more influence or knowledge than appears on an organizational chart. The analyst's understanding of the company's formal and informal structures helps determine the people to interview during the systems analysis phase.

Figure 4-4 When setting up interviews, an analyst should look outside a formal organizational chart to identify people who might provide valuable information.
GaudiLab/Shutterstock.com

Should several people be interviewed at the same time? Group interviews can save time and allow observing interactions among the participants. Group interviews also can present problems. One person might dominate the conversation, even when questions are explicitly addressed to others. The organization level can also present a problem because senior managers in an interview might prevent lower-level employees from expressing themselves candidly.

Step 2: Establish Objectives for the Interview

After deciding on the people to interview, objectives for the session must be established. First, the general areas to be discussed should be determined, and then the facts to be gathered should be listed. Soliciting ideas, suggestions, and opinions during the interview is also good.

The objectives of an interview depend on the role of the person being interviewed. Upper-level managers can provide the big picture to help explain the system as a whole. Specific details about operations and business processes are best learned from people who work with the system daily.

In the early stages of systems analysis, interviews usually are general. However, as the fact-finding process continues, the discussions focus more on specific topics. Interview objectives also vary at different stages of the investigation. Interviews should be as brief as possible (though as long as needed) because time is valuable to employees, especially managers. Setting specific objectives creates a framework that helps the analyst decide what questions to ask and how to phrase them.

Step 3: Develop Interview Questions

Creating a standard list of interview questions helps keep the session on track and avoid unnecessary tangents. Also, if several people who perform the same job are interviewed, a common question list permits a comparison of their answers. Although there may be a list of specific questions, the interviewer might decide to depart from it because an answer to one question leads to another topic that warrants pursuing. That question or topic should be included in a revised set of questions for future interviews. If the question proves extremely important, returning to a previous interviewee may be necessary to query them on the topic.

The interview should consist of several questions: open-ended, closed-ended, or questions with a range of responses. Avoid **leading questions** that suggest or favor a particular reply with their phrasing. For example, instead of asking, “What advantages do you see in the proposed system?” ask, “Do you see any advantages in the proposed system?”

Open-ended Questions. **Open-ended questions** encourage spontaneous and unstructured responses. Such questions help interviewers understand a larger process or draw out the interviewee's opinions, attitudes, or suggestions. Here are some examples of open-ended questions:

- What are users saying about the new system?
- How is this task performed?
- Why do you perform the task that way?
- How are the checks reconciled?
- What added features would you like to have in the new billing system?

Also, an open-ended question can be used to probe further: Is there anything else you can tell me about this topic?

Closed-ended Questions. **Closed-ended questions** limit or restrict the response. Closed-ended questions are used when more specific information is needed or when facts must be verified. Examples of closed-ended questions include the following:

- How many personal computers do you have in this department?
- Do you review the reports before they are sent out?
- How many hours of training does a clerk receive?
- Is the calculation procedure described in the manual?
- How many customers ordered products from the website last month?

Range-of-response Questions. **Range-of-response questions** are closed-ended questions that ask the person to evaluate something by providing limited answers to specific responses or on a numeric scale. This method makes it easier to tabulate the answers and interpret the results. Range-of-response questions might include these:

- How effective was your training on a scale of 1 to 10, with 1 being the lowest score and 10 the highest?
- How would you rate the severity of the problem: low, medium, or high?
- Is the system shutdown occurring sometimes, often, always, or never?

Step 4: Prepare for the Interview

Preparing for the interview is next after setting the objectives and developing the questions. Careful preparation is essential because an interview is an important meeting, not just a casual chat. When the interview is scheduled, suggest a specific day and time and let the interviewee know how long the meeting will last. It is also a good idea to send an email or place a reminder call the day before the interview.

Remember that the interview interrupts the other person's routine, so the discussion should be limited to no more than one hour. If business pressures force a postponement of the meeting, schedule another appointment as soon as it is convenient. Remember to keep department managers informed of meetings with their staff members. Sending a message to each department manager listing planned appointments is a good way to keep them informed. Figure 4-5 is an example of such a message.

A list of topics should be sent to an interviewee several days before the meeting, especially when detailed information is needed, so the person can prepare for the interview and minimize the need for a follow-up meeting. Figure 4-6 shows a sample message that lists specific questions and confirms the interview's date, time, location, purpose, and anticipated duration.

Figure 4-5 Sample message to a department head about interviews.

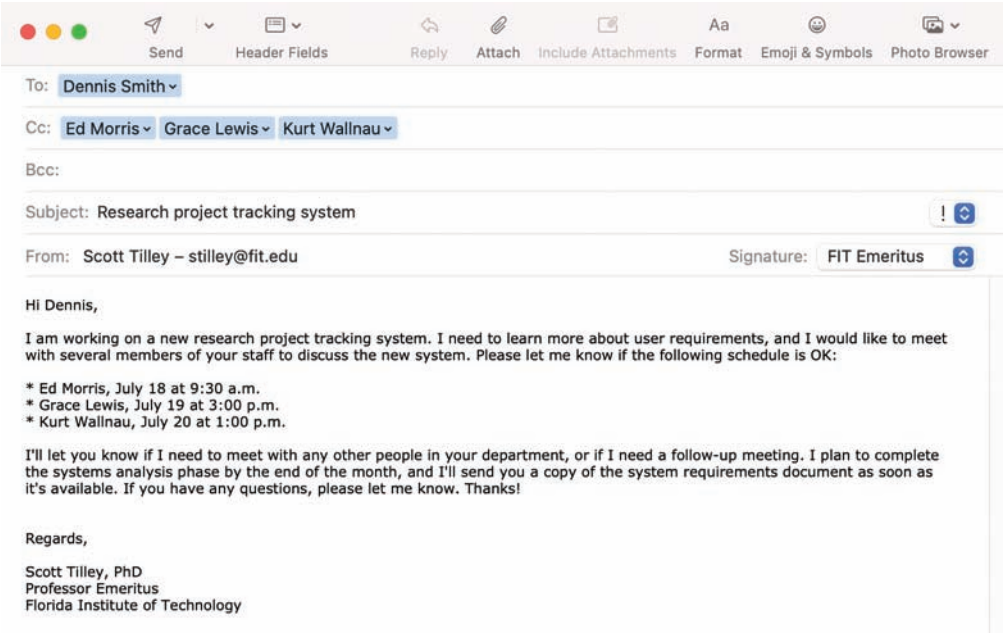
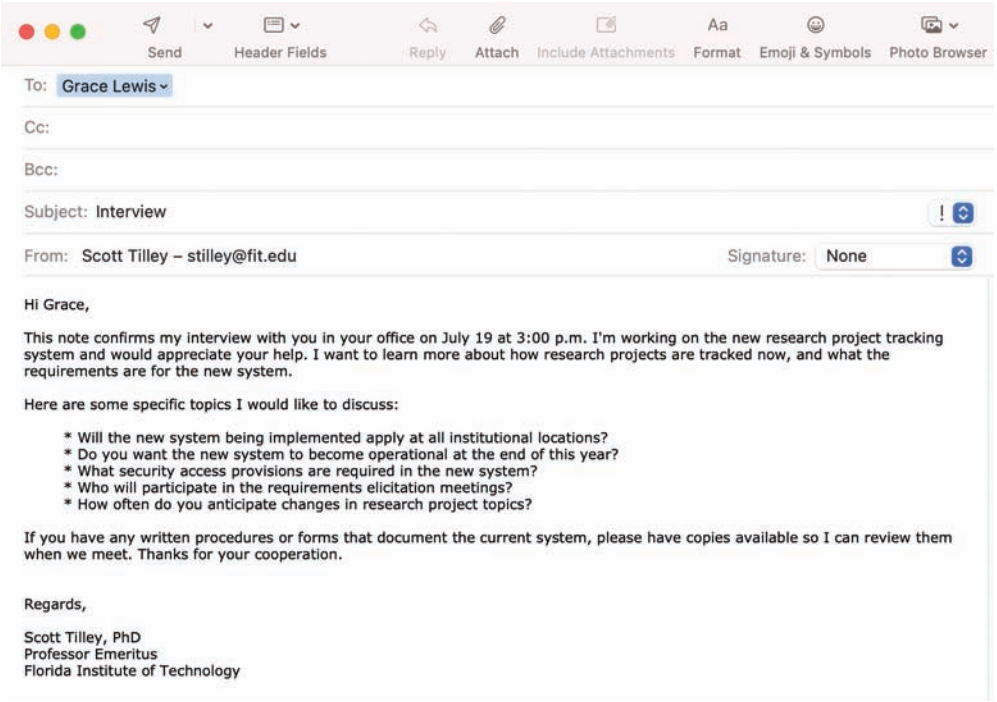


Figure 4-6 Sample message to confirm an interview.



If there are questions about documents, ask the interviewee to have samples available at the meeting. The advance memo should include a list of the documents to discuss (if it is known what they are). Also, a general request for documents can be made, as the analyst did in the email shown in Figure 4-6.

Two schools of thought exist about the best location for an interview. Some analysts believe that interviews should occur in the interviewee's office, whereas others feel a neutral location, such as a conference room, is better. The COVID-19 pandemic popularized virtual meetings, making the location choice moot.

Supporters of interviews in the interviewee's office believe that is the best location because it makes the interviewee feel comfortable during the meeting. A second argument in favor of the interviewee's office is that they have the easiest access to supporting material that might be needed during the discussion. However, if a complete list of topics is provided in advance, the interviewee can bring the necessary items to a conference room or other location.

Supporters of neutral locations emphasize the importance of reducing interruptions so both people can concentrate fully. In addition, an interview that is free of interruptions takes less time. If the meeting occurs in the interviewee's office, tactfully suggest that all calls be held until the interview's conclusion.

Step 5: Conduct the Interview

A specific plan for the meeting should be developed after determining the people to interview, setting the objectives, and preparing the questions. When conducting an interview, begin with introductions, describe the project, and explain the interview objectives.

During the interview, ask questions in the order they were prepared, and give the interviewee sufficient time to provide thoughtful answers. Some answers will lead to additional questions, which should be asked in a logical order. Establishing a good rapport with the interviewee is essential, especially if this is the first meeting. If the other person feels comfortable, they will probably provide more complete and candid answers. The analyst's primary responsibility during an interview is to listen carefully to the answers. Analysts sometimes hear only what they expect to hear. Concentrate on what is said and notice any nonverbal communication that takes place. This process is called **engaged listening**.

After asking a question, allow the person enough time to think about the question and arrive at an answer. Studies have shown that the maximum pause during a conversation is usually three to five seconds. After that interval, one person will begin talking. Analysts must be patient and practice their skills in many interview situations to succeed.

When all the questions have been asked, summarize the main points covered in the interview and explain the next course of action. For example, mention that a follow-up memo will be sent or that the interviewee should send certain requested information after the meeting. When the interview has concluded, thank the person and encourage them to reach out with any questions or additional comments. Also, when the interview ends, asking the interviewee whether they can suggest any additional topics to discuss is a good idea.

After an interview, summarize the session and seek confirmation from the other person. By hearing the interviewer's understanding of the discussion, the interviewee can respond and provide corrections if necessary. One approach is to rephrase the interviewee's answers. For example, the analyst could say, "If I understand you correctly, you are saying that ..." and then reiterate the information given by the interviewee.

Step 6: Document the Interview

Although taking notes during an interview has advantages, it should be kept to a minimum. It is a good idea to write down a few notes to remember key points after the interview but avoid writing down everything said. Too much writing distracts the other person and makes it harder to establish a good rapport.

After conducting the interview, record the information quickly. Set aside time right after the meeting to record the facts and evaluate the information. For that reason, try to avoid scheduling back-to-back interviews. Studies have shown that 50 percent of conversations are forgotten within 30 minutes. Therefore, use the notes to record the facts immediately so they will be remembered. Summarize the points by preparing a narrative describing what took place or by recording the answers received next to each question on the prepared list.

Small, portable recorders are practical interview tools, but some people are uncomfortable using them. Before using a recorder, discuss its use with the interviewee. Assure them that the recording will be erased after its contents are transcribed into note form and that the interview

can be stopped at any time at the interviewee's request. If sensitive questions are asked, or the interviewee wants to answer a question without being recorded, explain that the recorder can be turned off during the interview.

Instead of a traditional recorder that calls attention to its presence, an interviewer can use built-in audio (or even video) recording features on a notebook or mobile device. Also, as pointed out in Section 4.9, an interviewer can use powerful information management software, such as Microsoft OneNote, to record the meeting, store the results, and create a searchable file for easy access. Irrespective of the mechanism used to record the meeting, all participants should know what they say is being recorded.

Whether or not the meeting is recorded, listen carefully to the interviewee's responses so good follow-up questions can be asked. Otherwise, a second visit might be needed to ask the questions missed the first time. Also, remember that each recorded interview takes twice the amount of time because the analyst must listen to or view the recorded meeting again after conducting it.

After the meeting, a memo should be sent to the interviewee expressing appreciation for their time and cooperation. In the message, note the date, time, location, purpose of the interview, and the main points discussed so the interviewee has a written summary and can offer additions or corrections.

Step 7: Evaluate the Interview

In addition to recording the facts obtained in an interview, try to identify any possible biases. For example, an interviewee trying to protect their area or function might give incomplete answers or refrain from volunteering information. Or, an interviewee with strong opinions about the current or future system might distort the facts. Some interviewees might answer questions in an attempt to be helpful even though they do not have the necessary experience to provide accurate information.

Some interviews are unsuccessful irrespective of the amount of preparation. One of the main reasons could be that the interviewer and the interviewee did not get along well. Several factors can cause such a situation. For example, a misunderstanding or personality conflict could affect the interview negatively, or the interviewee might be afraid that the new system will eliminate or change their job.

In other cases, the interviewee might give only short or incomplete responses to the open-ended questions. If so, switching to closed-ended questions or questions with a range of replies may elicit more favorable responses. If that still does not help, find a tactful way to conclude the meeting.

Continuing an unproductive interview is tricky. The interviewee could be more cooperative later, or the analyst might find the information required elsewhere. If failure to obtain specific details will jeopardize the project's success, the supervisor should be informed; they can help decide what action to take. The supervisor might contact the interviewee's supervisor, ask another systems analyst to interview the person, or find another way to get the needed information.

4.5 Other Elicitation Techniques

In addition to interviewing, systems analysts use other requirements elicitation techniques, including document review, observation, questionnaires and surveys, sampling, and research. Such methods are used before interviewing begins to obtain a good overview and to help develop better interview questions.

4.5.1 Document Review

Document review can help the analyst understand how the current system is supposed to work. Remember that system documentation sometimes is out of date. Forms can change or be discontinued, and documented procedures often are modified or eliminated. It is prudent

to obtain copies of actual forms and operating documents currently in use to review blank copies of the forms and samples of completed forms. Document samples can be obtained during interviews with those who perform a relevant procedure. If the system uses a software package, review the documentation for that software.

4.5.2 Observation

The **observation** of current operating procedures is another fact-finding technique. Seeing the system in action provides additional perspective and a better understanding of system procedures. Personal observation also allows the analyst to verify statements made in interviews and determine whether procedures operate as described. Through observation, it might be discovered that neither the system documentation nor the interview statements are accurate.

Personal observation also can provide significant advantages as the development process continues. For example, recommendations are often better accepted when based on personal statements of actual operations. Observation can also provide the knowledge needed to test or install future changes and help build relationships with the users who will work with the new system.

Plan observations in advance by preparing a checklist of specific tasks to observe and questions to ask. Consider the following issues when preparing the list:

- Ask enough questions to ensure a complete understanding of the present system operation. A primary goal is to identify the methods of handling situations not covered by standard operating procedures. For example, what happens in a payroll system if an employee loses a time card? What is the procedure if an employee starts a shift 10 minutes late but then works 20 minutes overtime? Often, the rules for exceptions such as these are not written or formalized. Therefore, try to document any procedures for handling exceptions.
- Observe all the steps in a transaction and note the documents, inputs, outputs, and processes involved.
- Examine each form, record, and report. Determine the purpose each item of information serves.
- Consider each user who works with the system and the following questions: What information does that person receive from others? What information does this person generate? How is the information communicated? How often do interruptions occur? How much downtime occurs? How much support does the user require, and who provides it?
- Talk to the people who receive current reports to see whether the reports are complete, timely, accurate, and valuable. Ask whether information can be eliminated or improved and whether people would like to receive additional information.

As people are observed at work, as shown in Figure 4-7, consider a factor called the **Hawthorne Effect**. The name comes from a well-known study performed in the Hawthorne plant of the Western Electric Company in the 1920s. The study aimed to determine how various changes in the work environment would affect employee productivity. The surprising result was that productivity improved during observation, whether the conditions were better or worse. Researchers concluded that productivity improved whenever the workers knew they were being observed.

Although some recent studies have raised questions about the original findings, observation can and does affect normal operations. With this in mind, always give advance notice to the supervisor in the area that will be observed. In some situations, explaining the purpose of the visit to the people being observed might be helpful.

Figure 4-7 The Hawthorne study suggested that worker productivity improves during observation. Always consider the Hawthorne Effect when observing the operation of an existing system.
Monkey Business Images/Shutterstock.com

4.5.3 Questionnaires and Surveys

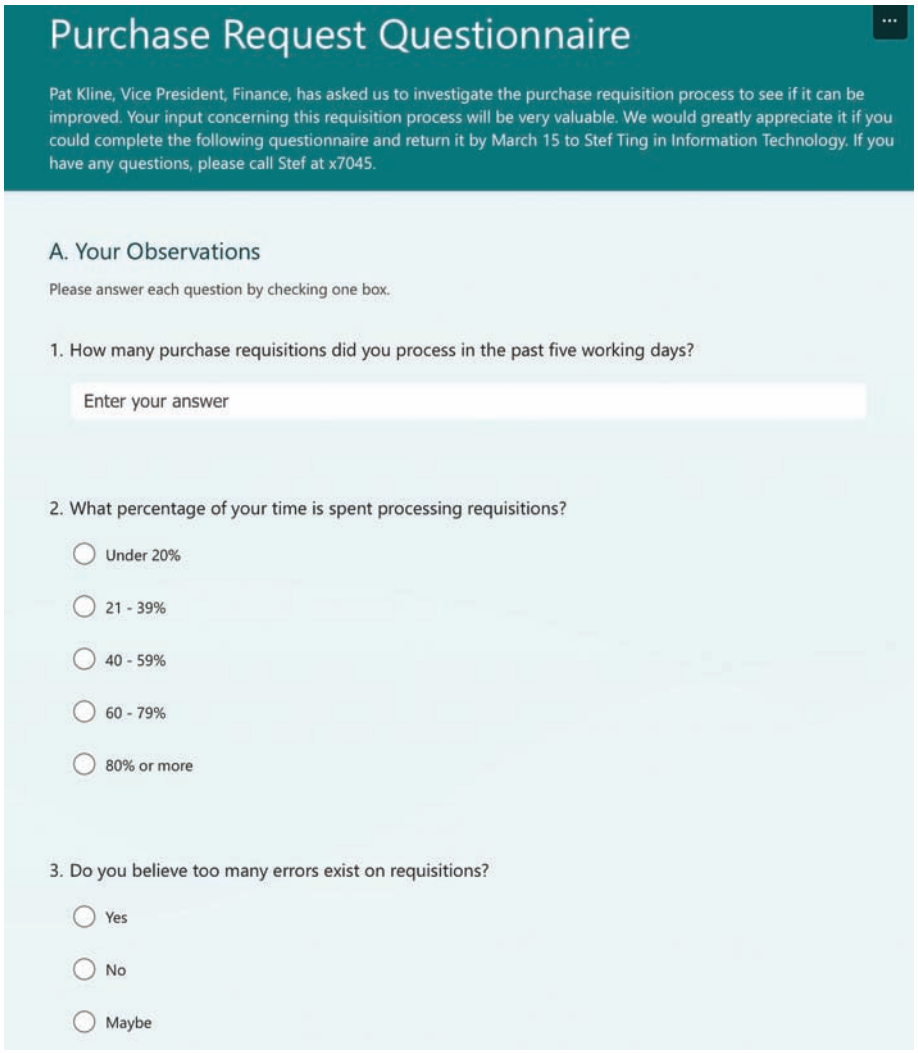
A questionnaire can be a valuable tool in projects where it is desirable to obtain input from many people. A **questionnaire (survey)** contains several standard questions that can be sent to many individuals.

Questionnaires can be used to obtain information about a wide range of topics, including workloads, reports received, volumes of transactions handled, job duties, difficulties, and opinions of how the job could be performed better or more efficiently. Figure 4-8 shows parts of a sample questionnaire created with Microsoft Forms with several question and response formats.

A typical questionnaire starts with a heading, which includes a title, a brief statement of purpose, the name and phone number of the contact person, the deadline date for completion, and how and where to return the form. The heading is usually followed by general instructions that provide clear guidance on answering the questions. Headings also are used to introduce each main section or portion of the survey and include instructions when the type of question or response changes. A long questionnaire might end with a conclusion that thanks the participants and reminds them how to return the form.

What about the issue of anonymity? Should people be asked to sign the questionnaire, or is it better to allow anonymous responses? The answer depends on two questions. First, must an analyst know who the respondents are to match or correlate information? For example, it might be important to know what percentage of users need a particular software feature, but specific names of users might not be relevant. Second, does the questionnaire include any sensitive or controversial topics? Many people do not want to be identified when answering such questions as "How well has your supervisor explained the system to you?" In such cases, anonymous responses might provide better information.

Figure 4-8 Part of an online questionnaire.
Source: Created by the author using Microsoft Forms



When designing a questionnaire, the most important rule is to ensure that the questions collect the correct data in a form that can further the fact-finding effort. Here are some additional ideas to keep in mind when designing the questionnaire:

- Keep the questionnaire brief and user-friendly.
- Provide clear instructions that will answer all anticipated questions.
- Arrange the questions logically, going from simple to more complex topics.
- Phrase questions to avoid misunderstandings; use simple terms and wording.
- Avoid leading the response or using questions that give clues to expected answers.
- Limit the use of open-ended questions that are difficult to tabulate.
- Limit questions that can raise concerns about job security or other negative issues.
- Include a section at the end of the questionnaire for general comments.
- Test the questionnaire whenever possible on a small test group before finalizing and distributing it to a large group.

A questionnaire can be a traditional paper form or it can be created as a **fill-in form** to collect data on the Internet or a company intranet. Before publishing the form, protect it so users can fill it in but cannot change the layout or design. Online survey websites like Survey Monkey and Google Forms can also be used to create and manage questionnaires.

4.5.4 Interviews versus Questionnaires

A questionnaire is a handy tool when seeking input from a large group. On the other hand, if detailed information is required from only a few people, then each person should be interviewed individually. Is it better to interview or use a questionnaire? Each situation is different; consider the type of information, time constraints, and expense factors.

The interview is more familiar and personal than a questionnaire. People unwilling to put critical or controversial comments in writing might talk more freely in person. Moreover, during a face-to-face interview, the interviewer can react immediately to anything the interviewee says. If surprising or confusing statements are made, the topic can be pursued with additional questions. In addition, during a personal interview, the analyst can watch for clues to help determine if responses are knowledgeable and unbiased. Participation in discussions can also affect user attitudes because people asked for their opinions often view the project more favorably.

Interviewing, however, is a costly and time-consuming process. In addition to the meeting, both people must prepare, and the interviewer must do follow-up work. When several interviews are planned, the total cost can be quite substantial. The personal interview usually is the most expensive fact-finding technique.

In contrast, a questionnaire allows many people to provide input and suggestions. Questionnaire recipients can answer the questions at their convenience and do not have to set aside a block of time for an interview. People might offer more candid responses than in an interview if the questionnaire allows anonymous responses.

Preparing a good questionnaire, however, like a good interview, requires skill and time. If a question is misinterpreted, its meaning cannot be clarified as quickly as in a face-to-face interview. Furthermore, recipients might view questionnaires as intrusive, time-consuming, and impersonal unless designed well. The analyst should select the technique that will work best in a particular situation.

4.5.5 Brainstorming

Another popular method of obtaining input is **brainstorming**, which refers to a small group discussion of a specific problem, opportunity, or issue. This technique encourages new ideas, allows team participation, and enables participants to build on each other's inputs and thoughts. Brainstorming can be structured or unstructured. In **structured brainstorming**, each participant speaks when it is their turn or passes. In **unstructured brainstorming**, anyone can speak at any time. At some point, the results are recorded and made part of the fact-finding documentation process.

4.5.6 Sampling

When studying an information system, examples of actual documents should be collected using **sampling**. The samples might include records, reports, operational logs, data entry documents, complaint summaries, work requests, and various forms. Sampling techniques include systematic sampling, stratified sampling, and random sampling.

Suppose there is a list of 200 customers complaining about errors in their statements, and a representative sample of 20 customers will be reviewed. A **systematic sampling** would select every tenth customer for review. However, a **stratified sample** could select five customers from each of four postal codes to ensure the model is geographically balanced. Another example of stratified sampling is choosing a certain percentage of transactions from each postal code rather than a fixed number. Finally, a **random sample** selects any 20 customers.

The main objective of a sample is to ensure that it represents the overall population accurately. For example, if inventory transactions are being analyzed, select a sample of transactions typical of actual inventory operations and do not include unusual or unrelated examples. For instance, if a company performs special processing on the last business day of the month, that day is not a good time to sample typical daily operations. A sample must be large enough to represent the overall data fairly to be valid.

Sampling should also be considered when using interviews or questionnaires. Rather than interviewing everyone or sending a questionnaire to the entire group, a sample of participants can be used. Sound sampling techniques must be used to reflect the overall population and obtain an accurate picture.

4.5.7 Research

Research is another important fact-finding technique. Research can include the Internet, IT magazines, and books to obtain background information, technical material, and news about industry trends and developments. In addition, attending professional meetings, seminars, and discussions with other IT professionals can be very helpful in problem solving.

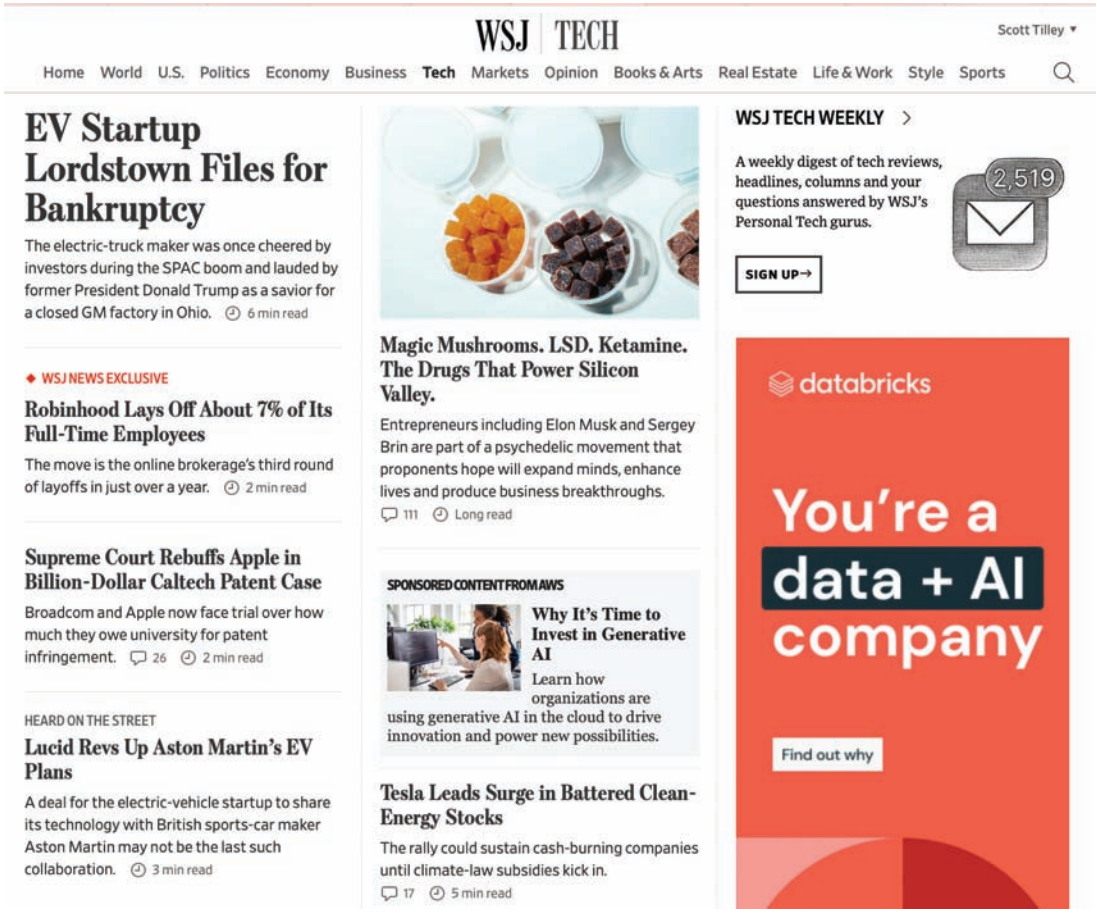
The Internet is an extremely valuable resource. Using the Internet, the analyst can also access information from federal and state governments and from publishers, universities, and libraries worldwide. Online forums and newsgroups are good resources for exchanging information with other professionals, seeking answers to questions, and monitoring discussions of mutual interest.

All major hardware and software vendors maintain websites with information about products and services offered by the company. Websites maintained by publishers and independent firms provide links to hundreds of hardware and software vendors. Examples of popular websites for IT professionals include CNET, InfoWorld (Figure 4-9), TechCrunch, The Verge, and the *Wall Street Journal*'s Technology pages (Figure 4-10).

Figure 4-9 InfoWorld covers software development, cloud computing, data analytics, and machine learning tools and technologies for an audience of IT professionals.
Source: InfoWorld

Research also can involve a visit to a physical location, called a **site visit**, where the objective is to observe a system used at another location. For example, if a firm's human resources information system is the subject of study, it might be beneficial to see how another company's system works. Site visits also are important when considering the purchase of a software package. If the software vendor suggests possible sites, be aware that such places might constitute a biased sample. A single site visit seldom provides accurate pictures, so try to visit more than one installation.

Figure 4-10 The *Wall Street Journal*'s Technology website contains valuable information for IT professionals.
Source: The Wall Street Journal



Before a site visit, prepare for it like an interview. Contact the appropriate manager and explain the purpose of the visit. Decide what questions will be asked and what processes will be observed. During the visit, observe the system and note any problems or limitations. Also, learn about the support provided by the vendor, the quality of the system documentation, and so on.

Case in Point 4.2

CyberStuff

CyberStuff is a large company that sells computer hardware and software by phone and online. It processes several thousand weekly transactions on a three-shift operation and employs 50 full-time and 125 part-time employees. Lately, the billing department has experienced increased customer complaints about incorrect bills. You have been tasked with finding out why this is happening.

During your preliminary investigation, you discovered that some CyberStuff representatives did not follow established order entry procedures. You feel that with more information you might find a pattern and identify a solution for the problem, but you are still trying to figure out how to proceed.

Is a questionnaire the best approach, or would interviews be better? Whether you use interviews, a questionnaire, or both techniques, should you randomly select the participants, include an equal number of people from each shift, or use another approach? How do you proceed?

4.6 Requirements Elicitation in Agile Projects

Requirements elicitation in agile projects is a continuous, collaborative, and adaptive process that involves frequent interaction with stakeholders and incremental refinement of the system's requirements. The focus is on satisfying the customer through early and continuous delivery of valuable software. In this context, requirements elicitation takes on a few characteristics that differentiate it from traditional approaches.

Iterative and incremental progress. Agile methodologies focus on delivering software in small, workable increments, which means that requirements are elicited iteratively, in tandem with the project's development. This differs from traditional waterfall methods, where requirements are gathered at the outset and may not change substantially during the project life cycle.

Collaboration. Agile places a significant emphasis on collaboration. In requirements elicitation, this means there's often direct and frequent communication between the development team and stakeholders. This can involve regular meetings, such as daily stand-ups, sprint planning, and sprint reviews, where requirements are discussed, refined, and prioritized.

Adaptive process. Agile methodologies are designed to embrace change, so the requirements elicitation process in agile is flexible and adaptive. As the project progresses, it's expected that the understanding of the system will improve. New requirements may emerge, or old ones may change or become obsolete.

Backlog management. The collected user stories (see below) are maintained in a backlog, which is continuously updated. The backlog items are prioritized based on factors like business value, risk, and dependencies. During each sprint, the most important items are selected for development in the next sprint.

Validation and verification. Requirements are constantly validated against the needs of the stakeholders and verified during each sprint's review or demo. This ensures that what is being developed aligns with what is actually needed, and any course correction can be made promptly.

4.6.1 A Variation of Interviews

Agile methods are particularly well suited to projects in which the requirements are expected to change frequently. Because agile methods are iterative, requirements are gathered and successively refined. Requirements elicitation is performed using a variation of interviews. Requirements begin as **features**, which are then split into smaller **user stories**, which in turn are distilled into **scenarios** where **storyboards** are used.

Features. A feature (sometimes called an **epic**) is a simple, high-level statement of a requirement. A feature has a descriptive name, an estimate of its size regarding derived requirements or user stories, and a priority. The stakeholders typically provide features through initial interviews with the systems analyst.

User stories. User stories represent more fine-grained requirements. Taken together, a set of user stories forms a feature. A user story also has a descriptive name, along with a simple sentence of the form, "As a [user role], I want [action] so that [goal]." User roles, actions, and goals represent a category of stakeholder, a particular effect, and an outcome, respectively. User stories also include an optional condition of satisfaction, which can be used as a guide to determine whether the product satisfies the requirement. A user story is meant to be succinct and is often drawn on a 3 × 5-inch index card (or the software equivalent).

Scenarios. A scenario is a real-world example of how users interact with the system. A scenario describes a particular set of steps taken or events that will occur while the system is used for a specific function. Scenarios are used to refine the system requirements to reflect the actual usage of the system better.

Storyboards. A storyboard is a simple graphic organizer that helps systems analysts visualize the status of a project. A storyboard can be as rudimentary as a wall with sticky notes. Many software tools enhance storyboards while maintaining their essential feel.

4.6.2 User Story Examples

The following examples of user stories are for a hypothetical system focused on borrowing a book from the library.

Title: Borrow a Book

As a registered library member,

I want to be able to borrow a book from the library,

So that I can read it at home.

This is a very general user story. It could be broken down into smaller, more specific user stories, depending on the system's complexity:

Title: Search for a Book

As a registered library member,

I want to be able to search for a book using the online catalog,

So that I can see if it's available for borrowing.

Title: Reserve a Book

As a registered library member,

I want to be able to reserve a book online,

So that I can pick it up later from the library.

Title: Check Out a Book

As a registered library member,

I want to be able to use a self-service kiosk to check out my book,

So that I don't have to wait in line at the service desk.

Title: Receive Borrowing Confirmation

As a registered library member,

I want to receive a confirmation email when I successfully borrow a book,

So that I can keep track of my borrowings and their due dates.

Each user story provides a specific requirement from the user's perspective and can be developed and tested independently.

4.6.3 Scenario Example

A scenario, also known as a use case, describes a specific situation in which a system interacts with an end user (or another system). It typically includes the user's goal, the interaction flow, and the system's behavior. Here is an example of a scenario for the same hypothetical system focused on borrowing a book from the library:

Title: Borrow a Book from the Library

Actor: Registered Library Member

Preconditions: The user is a registered library member and has logged into the system.

Main Flow:

1. The user uses the search function to look for a specific book by entering the book title, author, or ISBN in the search bar.
2. The system displays a list of matching results.
3. The user selects the desired book from the list.
4. The system displays the book's details, including availability.
5. The user selects the "Reserve Book" option if the book is available.
6. The system reserves the book for the user and displays a confirmation message, including when and where the book can be picked up.
7. The user goes to the library, finds the book, and brings it to the self-service kiosk.
8. The user scans their library card and the book at the self-service kiosk.
9. The system checks out the book to the user's account and prints a receipt with the due date.

Postconditions: The book is checked out to the user's account. The system sends a confirmation email to the user with the details of the borrowed book and the return date.

Exceptions:

- If the book is unavailable, the system informs the user and offers to add the user to a waiting list.
- If the user has outstanding fines or has reached the borrowing limit, the system prevents the user from reserving or checking out books.

4.7 Representing Requirements

Once requirements have been gathered, they must be recorded. Keeping accurate records of interviews, facts, ideas, and observations is essential to successful systems development. As information is collected, the importance of a single item can be overlooked or complex system details can be forgotten. The ability to manage information is the mark of a successful systems analyst and an essential skill for all IT professionals. The basic rule is to write it down. An analyst should document their work according to the following principles:

- Record information as soon as it is obtained.
- Use the simplest recording method possible.
- Record findings in such a way that someone else can understand them.
- Organize documentation so related material is located easily.

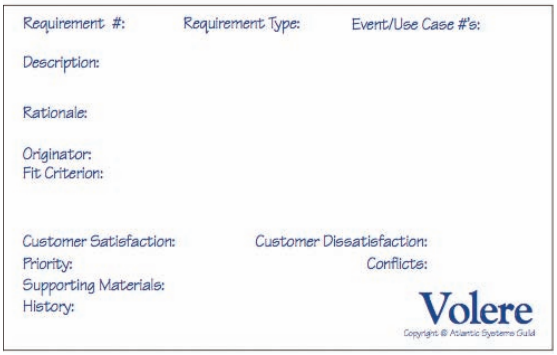
Several techniques are used to help with this documentation, ranging from an unordered collection of sentences to a structured database of formal models. Whatever representation is chosen, properly managing requirements over the lifetime of a project is vital to its success.

4.7.1 Natural Language

Most requirements are represented using unstructured natural language—plain English. Examples of these requirements are shown in Section 4.1.1. This representation is easy to create but is prone to problems such as imprecision and a lack of shared understanding.

Requirements represented as unstructured natural language can be stored in a simple file, a database that may facilitate searching its contents, or an Excel spreadsheet. The latter choice is popular because most systems analysts are familiar with the tool and are likely to have access to it already.

Figure 4-11 The Volere shell.
© Atlantic Systems Guild



An improvement upon unstructured natural language is structured natural language. This representation tags part of the requirement, like an XML document. This facilitates automated processing of the requirement statements, but it's not user-friendly.

Requirements represented as structured natural language can be stored on a simple index card. The Volere shell from the Atlantic Systems Guild, shown in Figure 4-11, is a mature method of representing single requirements on a 3 × 5-inch card. Several tools mimic these features, providing automation while maintaining the simple representational format.

There are also formal techniques based on mathematics that can be used to represent complex requirements. Languages such as VDM and Z are used for this purpose. They have the advantage of mathematical consistency. However, a significant disadvantage of this technique is that many systems analysts are unfamiliar with the language's constructs, which can affect the requirements' comprehensibility.

Irrespective of the natural language technique chosen, individual requirements must be collected into a requirements document. This document is the official statement of what the users require of the system. The Volere template is a good template for a comprehensive requirements document. It is populated by atomic requirements from the Volere shells.

Case in Point 4.3

Digital Pen Transcription

You are the lead systems analyst on a large project. A big part of your responsibilities is to handle the system requirements. Extensive interviews have taken place, and you have written down your notes using a digital pen system. This system recognizes letters and words while you write on paper and stores them in digital format on your computer. In this way, the requirements are written in unstructured natural language but stored digitally, which means they can be searched and processed. Would it be worthwhile to edit the transcription and insert tags to delineate key terms in the requirements so that a CASE tool could better analyze them? Extensive effort is required to do the manual tagging, but the rewards could be better requirements. How would you perform the trade-off analysis?

4.7.2 Diagrams

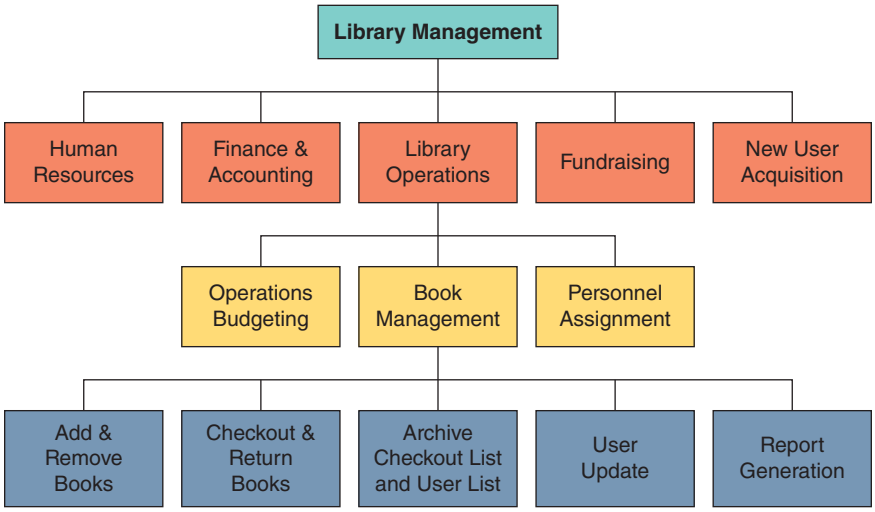
Many people are more visual than textual. For them, diagrams are an excellent choice to represent system requirements. Diagrams help users, managers, and IT professionals understand the design of a system. Diagramming involves graphical methods and nontechnical language representing the system at various stages of development. During requirements engineering, the analyst can use multiple tools to describe business processes, requirements, and user interaction with the system.

Systems analysts use diagramming and fact-finding interactively. They build fact-finding results into diagrams. Then they study the diagrams to determine whether additional fact-finding is needed. To help them understand system requirements, analysts use functional decomposition diagrams, business process diagrams, and data flow diagrams. These diagrams can be created with CASE tools or stand-alone drawing tools if desired.

Functional Decomposition Diagrams

A **functional decomposition diagram (FDD)** is a top-down representation of a function or process. Using an FDD, an analyst can show business functions and break them down into lower-level functions and processes. Creating an FDD is similar to drawing an organizational chart: start at the top and work downward. Figure 4-12 shows an FDD of a library system. FDDs can be used at several stages of systems development. During requirements engineering, analysts use FDDs to model business functions and show how they are organized into lower-level processes. Those processes translate into program modules during application development.

Figure 4-12 This FDD shows a library system with five top-level functions. The Library Operations function includes two additional levels of processes and subprocesses.
Source: https://lucid.app/lucidchart/



Business Process Diagrams

As described earlier in the book, a business process model (BPM) represents one or more business processes, such as handling an airline reservation, filling a product order, or updating a customer account. During requirements engineering, analysts often create diagrams that use a standard syntax called business process modeling notation (BPMN). BPMN includes various shapes and symbols representing events, processes, and workflows.

When creating a business process diagram using a CASE tool, the diagram automatically becomes part of the overall model. In the example shown in Figure 4-13, using BPMN terminology, the overall diagram is called a **pool** and the designated customer areas are called **swim lanes**. Integrating BPM into the CASE development process leads to faster results, fewer errors, and reduced costs.

Figure 4-13 This business process diagram is called a pool, and the two separate customer areas are called swim lanes.
Source: http://www.bpmn.io

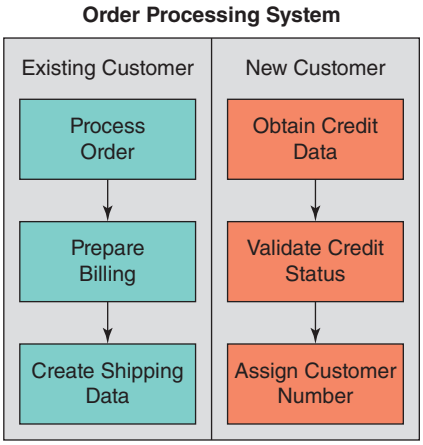
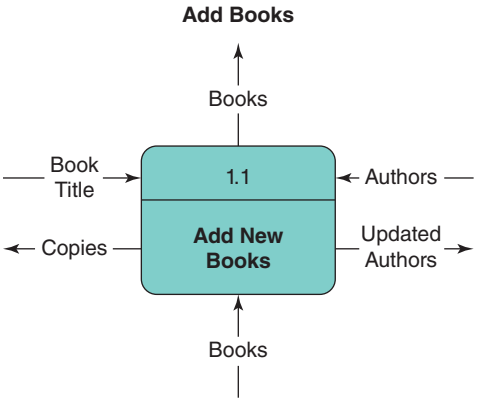


Figure 4-14 This DFD shows how new books are added to a library system.
Source: https://lucid.app/lucidchart/



Data Flow Diagrams

Analysts can create data flow diagrams (DFDs) from a functional decomposition diagram to show how the system stores, processes, and transforms data. The DFD in Figure 4-14 describes adding new books, which is part of the "Add & Remove Books" function shown in the Library Management FDD in Figure 4-12. Notice that the box in the DFD represents a process with various inputs and outputs. Additional levels of information and detail are depicted in related DFDs. Data and process modeling are described later in the book.

4.7.3 Models

Models provide a more formal representation of system requirements. They are often depicted as graphical in nature, so they share some of the characteristics of the techniques described in Section 4.7.2. But models have an additional feature: the underlying language has semantics, which means the diagram has a significance that can be automatically analyzed by a CASE tool.

The **Unified Modeling Language (UML)** is the most widely used modeling technique for visualizing and documenting software systems design. UML uses object-oriented design concepts, but it is independent of any specific programming language and can be used to describe business processes and requirements generally. **SysML** is a dialect of UML and has become the standard for Model-Based Systems Engineering (MBSE) applications.

UML provides various graphical tools, such as use case diagrams and sequence diagrams. During requirements engineering, a systems analyst can utilize UML to represent the information system from a user's viewpoint. Use case diagrams, sequence diagrams, and other UML concepts are discussed in more detail later in the book, along with other object-oriented analysis concepts. A brief description of each technique follows.

Use Case Diagrams

During requirements engineering, systems analysts and users work together to document requirements and model system functions. A **use case diagram** visually represents the interaction between users and the information system. In a use case diagram, the user becomes an **actor** with a specific role that describes how they interact with the system. Systems analysts can draw use case diagrams freehand or use CASE tools that integrate the use cases into the overall system design.

Figure 4-15 shows a simple use case diagram for a sales system in which the actor is a customer, and the use case involves a credit card validation performed by the system. Because use cases depict the system through a user's eyes, common business language can be used to describe the transactions. For example, Table 4-4 documents the credit card validation use case, and Figure 4-16 shows a student records system with several use cases and actors.

Figure 4-15 This use case diagram shows a sales system where the actor is a customer and the use case is a credit card validation.
Source: https://lucid.app/lucidchart/

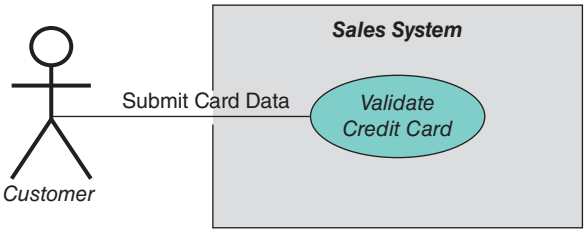
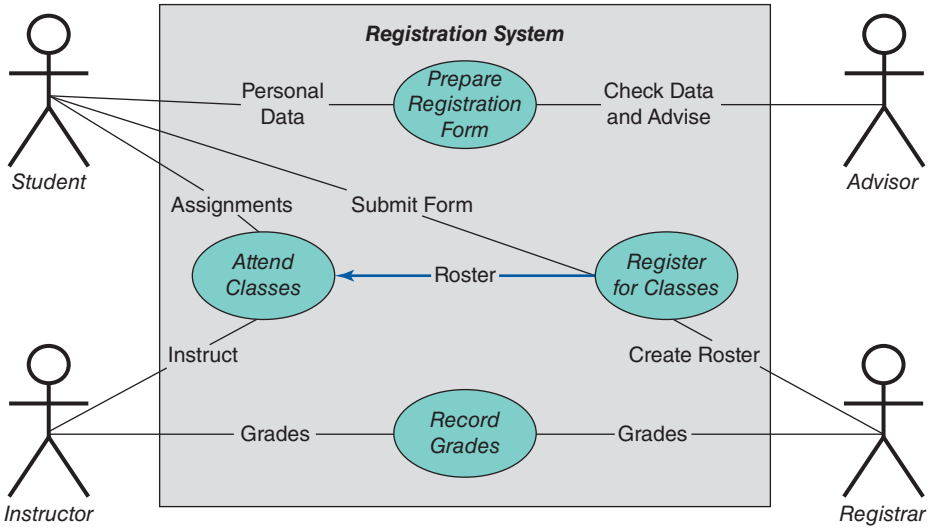


Table 4-4 This table documents the credit card validation use case shown in Figure 4-15.

Name of Use Case:	Credit card validation process
Actor:	Customer
Description:	Describes the credit card validation process
Successful Completion:	1. Customer clicks the input selector and enters credit card number and expiration date 2. System verifies card 3. System sends authorization message
Alternative:	1. Customer clicks the input selector and enters credit card number and expiration date 2. System rejects card 3. System sends rejection message
Precondition:	Customer has selected at least one item and has proceeded to checkout area
Postcondition:	Credit card information has been validated Customer can continue with order
Assumptions:	None

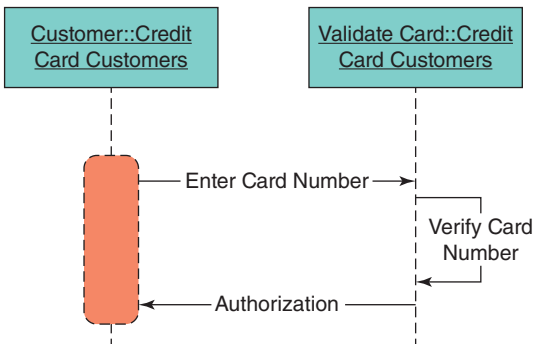
Figure 4-16 This use case diagram shows a student records system.
Source: https://lucid.app/lucidchart/



Sequence Diagrams

A **sequence diagram** shows the timing of interactions between objects as they occur. A systems analyst might use a sequence diagram to show all possible outcomes or focus on a single scenario. Figure 4-17 shows a simple sequence diagram of successful credit card validation. The interaction proceeds from top to bottom along a vertical timeline, while the horizontal arrows represent messages from one object to another.

Figure 4-17 This sequence diagram shows a credit card validation process.
Source: https://lucid.app/lucidchart/



4.8 Validation and Verification

Requirements **validation and verification (V&V)** is concerned with demonstrating that the requirements define the system that the customer wants. Because requirements error costs are high, V&V is very important; fixing a system later in the SDLC is many times more expensive than repairing it during requirements engineering.

Requirements V&V focuses on answering two essential questions:

- Validation: Are the correct requirements stated?
- Verification: Are the requirements stated correctly?

To answer these questions, the following requirements attributes should be checked:

- Validity: Does the system provide the functions that best support the customer's needs?
- Consistency: Are there conflicting requirements?
- Completeness: Are all functions required by the customer included?
- Realism: Can the requirements be implemented given the available budget and technology?
- Verifiability: Can the requirements be checked?
- Comprehensibility: Is the requirement adequately understood?
- Traceability: Is the origin of the requirement clearly stated?
- Adaptability: Can the requirement be changed without significantly impacting other requirements?

To check these, the following techniques can be used:

- Requirements review: Systematic manual analysis of the requirements.
- Prototyping: Using an executable model of the system to check the requirements.
- Test-case generation: Developing tests for requirements to check testability.
- Automated consistency analysis: Checking the consistency of structured or formal requirements descriptions.

Consider having requirements reviews. Regular reviews can be held while the requirements are initially being formulated. Ideally, the systems analyst and the customer (and other key stakeholders) should be involved in the reviews. The reviews can be formal (with complete documentation) or informal. The key is that good communication between analysts, customers, and others can resolve problems early—which is better for everyone.

4.9 Tools

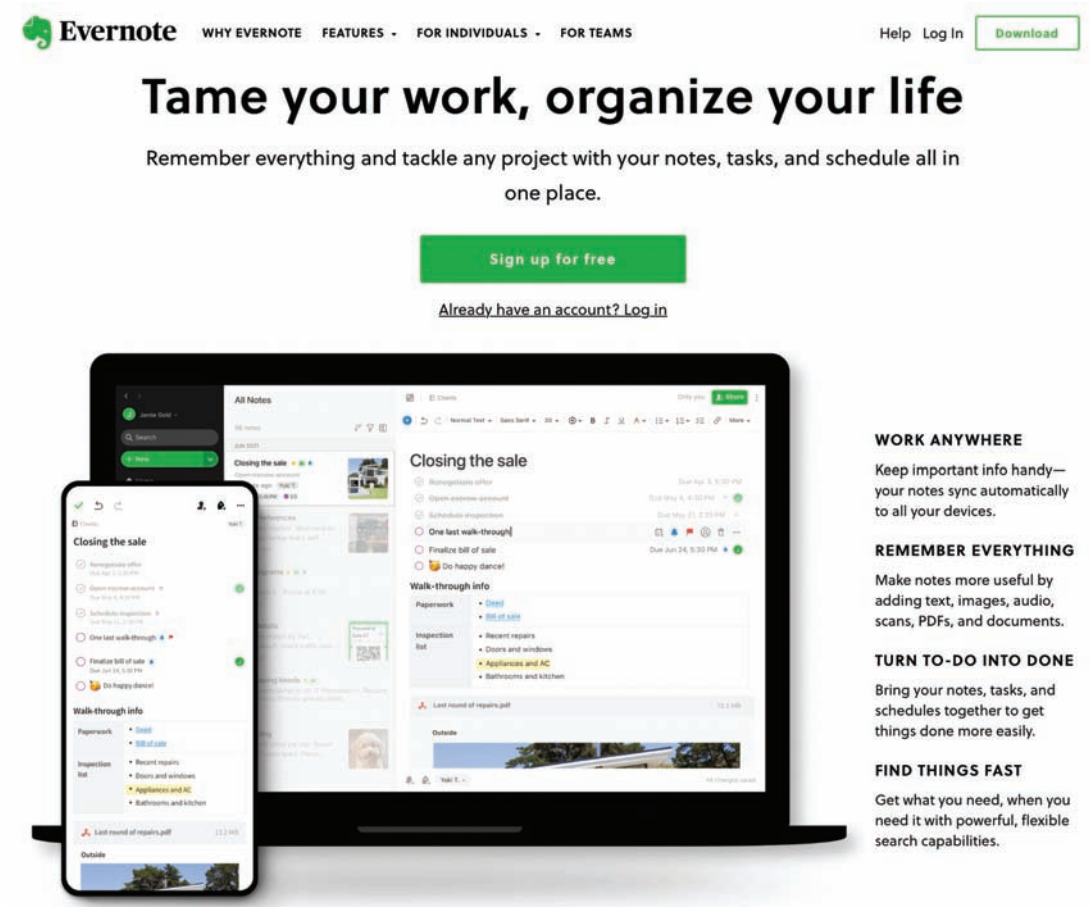
All requirements engineering activities can be helped through the judicious use of tools. For example, many software programs are available to help record and document information elicited during the requirements gathering process. This type of **productivity software** includes automation, word processing, spreadsheet, database management, presentation graphics, and collaboration software programs. Although Microsoft Office is the best-known set of productivity software programs, other vendors offer products in each category.

A **personal information manager (PIM)**, such as Microsoft Outlook, includes a personal calendar, a to-do list with priorities, the capability to check off completed items, and powerful contact management features. Outlook can manage email and appointments, and it supports collaboration and team projects.

Although a PIM such as Microsoft Outlook can handle day-to-day activities, tasks, and schedules, there are better ways to capture and organize large amounts of information. Instead, analysts use information management software such as Microsoft OneNote, a powerful, flexible tool that can handle many input types, including text, handwritten notes, images, audio and video recording, and web links. OneNote is included in several versions of the Office suite.

Figure 4-18 shows another popular personal information manager called Evernote. It is free on most computing platforms, including smartphones and the web. There are also premium versions available on a monthly subscription model. Evernote does a great job of handling multimedia content, adding free-form notes, and providing templates for organizing projects. It also syncs files across all devices.

Figure 4-18 Evernote offers a free version of its popular information management software for most computing platforms, including smartphones and the web.
Source: Evernote Corporation

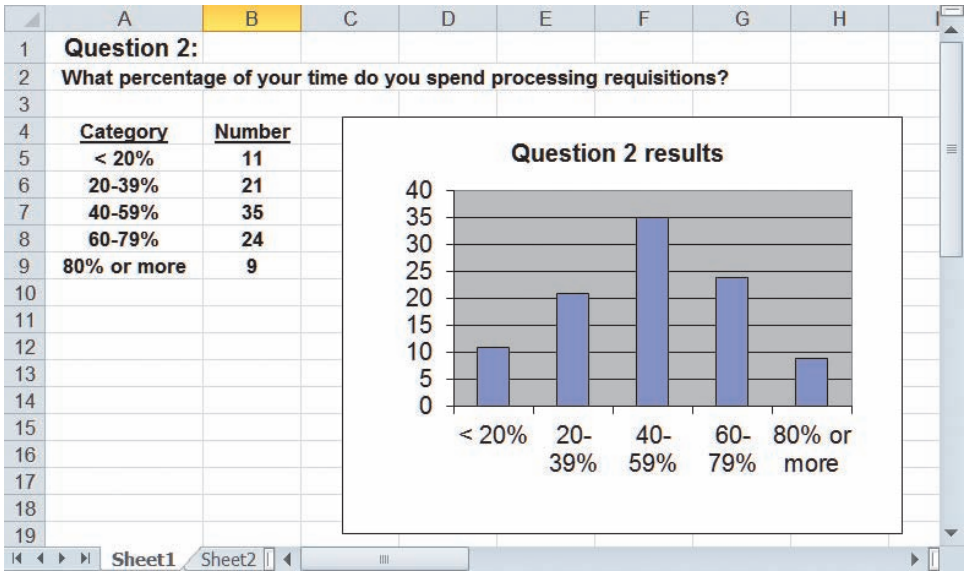


The analyst can create reports, summaries, tables, and forms using word processing software such as Microsoft Word. In addition to standard document preparation, the program can help organize a presentation with templates, bookmarks, annotations, revision control, and an index. As described earlier in this chapter, fill-in forms can also be created to conduct surveys and questionnaires.

Spreadsheet software like Microsoft Excel can help track and manage numeric data or financial information. Excel is one of the most popular ways of representing requirements informally. Graphs and charts can also be generated that display the data and show possible patterns. The statistical functions in a spreadsheet can be used to tabulate and analyze questionnaire data. A graphical format is often used in quality control analysis because it

highlights problems and their possible causes and is effective when presenting results to management. A common tool for distributing questionnaires or sampling results is a vertical bar chart called a **histogram**. Most spreadsheet programs can create histograms and graphs that display the collected data. Figure 4-19 depicts a typical histogram that might have resulted from the questionnaire shown in Figure 4-8.

Figure 4-19 This histogram displays the results from Question 2 in the questionnaire shown in Figure 4-8.



Database management software allows the analyst to document and organize fact-finding results such as events, observations, and data samples. A database program such as Microsoft Access can be used to manage the details of a complex project, create queries to retrieve specific information, and generate custom reports.

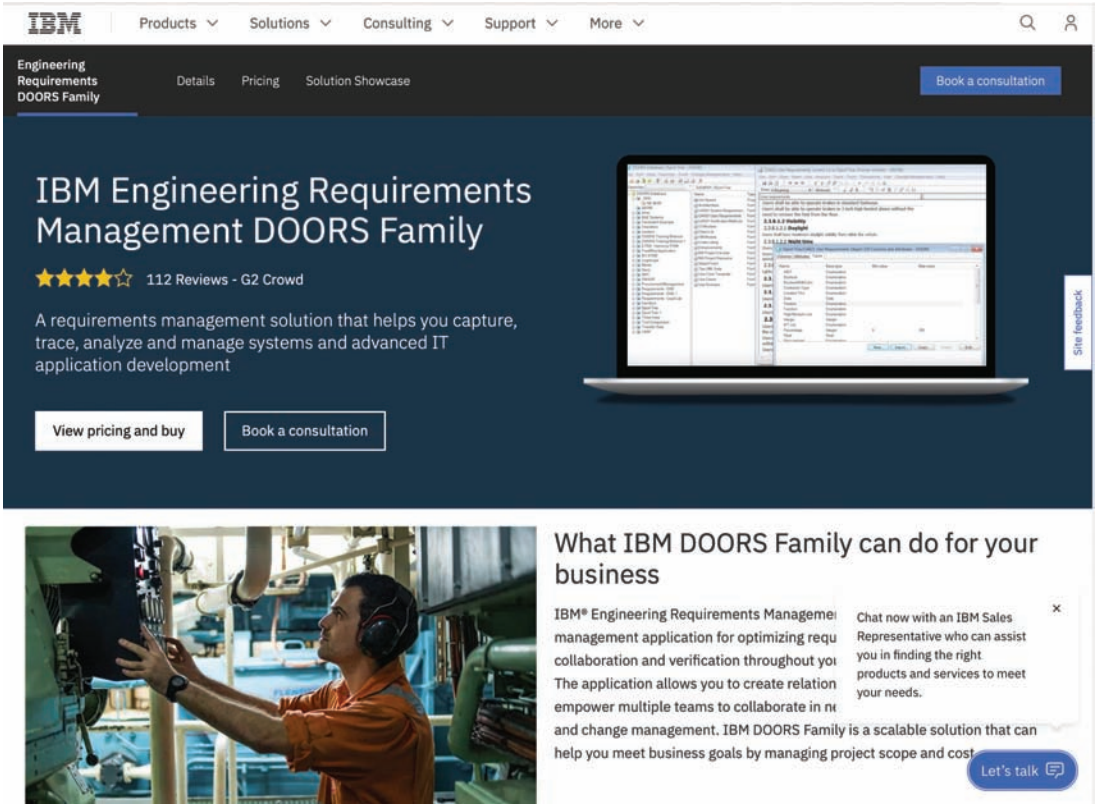
Presentation graphics software, such as Microsoft PowerPoint, is a powerful tool for organizing and developing formal presentations. Presentation graphics programs enable the creation of organizational charts that can be used in a preliminary investigation and later during requirements engineering. These high-quality charts also can be included in written reports and management presentations.

Collaboration software is the latest weapon in the struggle to boost productivity. People work in teams and use web-based software such as Google Docs and Microsoft Office 365 to access data and share files. Google and others are betting that cloud computing will create a virtual workplace where people can interact in real time, with all the benefits of a traditional face-to-face workplace but none of the limitations.

When creating diagrams representing requirements, Microsoft Visio is a popular graphic modeling tool that can produce a wide range of charts and diagrams. Visio includes a library of templates, stencils, and shapes. An analyst can use Visio to create many visual models, including business process diagrams, flowcharts, network diagrams, and organizational charts.

Special-purpose tools such as IBM Engineering Requirements Management DOORS Family (Figure 4-20) are used for more formal requirements models. These tools facilitate the use of UML (and SysML) to model system requirements in a way that enables desirable characteristics, such as **traceability**, where the origin of a requirement is connected back to the requirement itself, which in turn is linked forward to design artifacts, code fragments, and even test cases in the SDLC.

Figure 4-20 IBM® Engineering Requirements Management DOORS® Family is a requirements management application for optimizing requirements communication, collaboration, and verification throughout your organization and supply chain.
Source: IBM Corporation



Summary

The systems analysis phase includes three activities: requirements engineering, data and process modeling, and consideration of development strategies. The main objective is to understand the proposed project, ensure it will support business requirements, and build a solid foundation for the systems design phase. Requirements engineering itself is composed of three main parts: gathering requirements, representing requirements, and validating and verifying requirements.

During requirements engineering, the business-related requirements for the new information system are identified. Scalability is considered to ensure that the system can support future growth and expansion. Security is an essential requirement of all modern connected systems. The total cost of ownership (TCO) is also estimated to identify all costs, including indirect costs.

Popular team-based approaches include JAD, RAD, and agile methods. Joint application development (JAD) is a popular, team-based approach to fact-finding and requirements engineering. JAD involves an interactive group of users, managers, and IT professionals who participate in requirements engineering and develop a greater commitment to the project and their common goals.

Rapid application development (RAD) is a team-based technique that speeds up information systems development and produces a functioning information system. RAD is a complete methodology with a four-phase life cycle that parallels the traditional SDLC phases.

Agile methods attempt to develop a system incrementally by building prototypes and constantly adjusting them to user requirements. Tools are often avoided and replaced with simpler aids, such as whiteboards and sticky notes, to facilitate communication.

The requirements gathering process includes interviewing, document review, observation, questionnaires, sampling, and research. Successful interviewing requires good planning and strong interpersonal and communication skills. The systems analyst must decide on the people to interview, set interview objectives, and prepare for, conduct, and analyze interviews. The analyst also might find it helpful to use one or more software tools during fact-finding.

Systems analysts use various tools and techniques to represent system requirements. Natural language, structured or unstructured, is still the default. Requirements can be stored as simple text in a plain file or an Excel spreadsheet. They can also be stored online for automated search and analysis.

Diagrams are another way of representing requirements. They are suitable for analysts who are more visually oriented. They can also capture complementary aspects of the system requirements. Sample diagram types include functional decomposition diagrams (FDD), business process diagrams, and data flow diagrams (DFD).

Models provide a more formal representation of system requirements. The Unified Modeling Language (UML) is a widely used method of visualizing and documenting software design through the eyes of the business user. UML tools include use case diagrams and sequence diagrams to represent actors, their roles, and the sequence of transactions.

Systems analysts should carefully record and document factual information as it is collected, and various software tools can help an analyst visualize and describe an information system.

Requirements validation and verification (V&V) is concerned with demonstrating that the requirements define the system that the customer wants. Validation asks if the correct requirements are stated, while verification asks if the requirements are stated correctly.

All requirements engineering activities can be helped through the judicious use of tools. They provide automated support for requirements attributes such as traceability. CASE tool assistance is necessary for large systems with thousands of requirements to manage.

Key Terms

actor
agile methods
brainstorming
closed-ended questions
construction phase
cutover phase
document review
engaged listening
epic
fact-finding
feature
fill-in form
functional decomposition diagram (FDD)
functional requirement
Hawthorne Effect
histogram
informal structure
interview
joint application development (JAD)
leading questions
non-functional requirements
observation
open-ended questions
personal information manager (PIM)
pool
productivity software
quality attributes
questionnaire
random sample
range-of-response questions
rapid application development (RAD)
requirements definitions
requirements elicitation
requirements engineering
requirements planning phase
requirements specifications
research
sampling
scalability
scenarios
Scrum
sequence diagram
site visit
storyboard
stratified sample
structured brainstorming
survey
swim lanes
SysML
system requirement
systematic sample
total cost of ownership (TCO)
traceability
Unified Modeling Language (UML)
unstructured brainstorming
use case diagram
user design phase
user stories
validation and verification (V&V)

Review Questions

1. What is a system requirement, and what are three challenges it presents to the systems analyst?
2. Is the requirement "The system shall respond within 2 seconds" a functional or non-functional requirement?
3. What is Scrum?
4. What five questions typically are used in fact-finding?
5. Provide three examples each of closed-ended, open-ended, and range-of-response questions.
6. Explain how the observation fact-finding technique works, including the Hawthorne Effect.
7. What is the relationship between user stories and features in agile projects?
8. What is a functional decomposition diagram, and why would you use one?
9. What is the difference between validation and verification of system requirements?
10. Why is traceability important in tool support for requirements engineering?

Discussion Topics

1. JAD requires strong interpersonal and communication skills from the systems analyst. Are those skills different from the ones that an analyst needs when conducting one-to-one interviews? Explain your answer.
2. Agile methods use rapid development cycles to iteratively produce running system versions. How would these shorter cycles affect the analyst's ability to manage system requirements?
3. A group meeting sometimes is suggested as a practical compromise between interviews and questionnaires. In a group meeting, a systems analyst meets with several users simultaneously. Discuss the advantages and disadvantages of group meetings.
4. Research the Internet, magazines, or textbooks to find examples of visual aids, including bar charts, pie charts, line charts, tables, diagrams, and bulleted lists. How effective was each example? Find at least one example that you could improve. Explain your choice.
5. Traceability is an essential requirement attribute. It's one of the things that should be checked when performing validation and verification of the system requirements. Describe how you would manually check traceability for an existing system and list a few features of a CASE tool that you think would help you with the task.

Hands-On Projects

1. Use the requirements elicitation techniques of document review, observation, brainstorming, sampling, and research to capture the requirements of a typical elevator control system. Use natural language and the UML to represent the situation in which people on different floors simultaneously push the "Up" button.
2. Prepare a presentation summarizing JAD and RAD. Explain how they differ from traditional fact-finding methods. What are the main advantages of team-based approaches?

3. Design a questionnaire to learn what students think of the registration process at your school. Apply the guidelines you learned in this chapter.
4. Create an FDD like Figure 4-12, showing your school instead of the library example.
5. A desirable requirement characteristic is that it is consistent and complete. Examine several requirements engineering CASE tools and document how (if at all) they support automated consistency analysis for validating and verifying requirements.

Ethical Issues

Your supervisor manages the corporate office where you work as a systems analyst. Several weeks ago, after hearing rumors of employee dissatisfaction, they asked you to create a survey for all IT employees. After the responses were returned and tabulated, your supervisor was disappointed that many employees assigned low ratings to morale and management policies.

This morning your supervisor called you into their office and asked whether you could identify the departments that submitted the lowest ratings. No names were used on the individual survey forms. However, you could identify the departments with some analysis because several questions were related.

Now you need to figure out how to respond. The expectation was that the survey would be anonymous. Even though no individuals would be identified, would it be ethical to reveal which departments sent in the low ratings? Would your supervisor's motives for wanting this information matter?

Chapter

5

Data & Process Modeling

Learning Objectives

When you finish this chapter, you should be able to:

1. Describe the relationship between logical and physical models.
2. Explain data flow diagrams.
3. Draw the four basic data flow diagram symbols.
4. Explain the six guidelines used when drawing data flow diagrams.
5. Draw context diagrams.
6. Draw diagram 0 data flow diagrams.
7. Draw lower-level data flow diagrams.
8. Create a data dictionary.
9. Apply process description tools in modular design.

Chapter 5 is the second of three chapters in the systems analysis phase of the SDLC. This chapter discusses data and process modeling techniques that analysts use to show how the system transforms data into useful information. Data and process modeling involves three main items: data flow diagrams, a data dictionary, and process descriptions. The deliverable is a logical model that will satisfy user requirements and support business operations.

The chapter includes three "Case in Point" discussion questions to help contextualize the concepts described in the text. The scenario described in the "Ethical Issues" section explores how a systems analyst should handle their new IT manager's persistent requests for proprietary information from the analyst's previous employer (a competitor).

5.1 Logical versus Physical Models

During the requirements engineering process described in a previous chapter, fact-finding techniques were used to investigate the current system and identify user requirements. This chapter and the next explain how that information is used to develop a logical model of the proposed system and document the system requirements. A **logical model** shows *what* the system must do, regardless of how it will be implemented physically. Later, in the systems design phase, a **physical model** is built that describes *how* the system will be constructed.

While structured analysis tools are used to develop a logical model for a new information system, such tools also can be used to create physical models of an information system. A physical model shows how the system's requirements are implemented. During the systems design phase, a physical model of the new information system follows the logical model and involves operational tasks and techniques.

To understand the relationship between logical and physical models, consider the beginning of the systems analysis phase. The physical operations of the existing system were studied before the logical model to understand how the current tasks are carried out in the existing system. Many systems analysts create a physical model of the current system and then develop a logical model before tackling a logical model of the new system. Performing that extra step allows them to understand the current system better.

Many analysts follow a **four-model approach**, which means that they develop a physical model of the current system, a logical model of the current system, a logical model of the new system, and a physical model of the new system. The primary benefit of the four-model approach is that it provides a clear picture of current system functions before any modifications or improvements are made. That is important because mistakes made early in systems development will affect later SDLC phases and can result in unhappy users and additional costs. Taking extra steps to avoid these potentially costly mistakes can prove worthwhile. Another advantage is that a new information system's requirements are often quite similar to those of the current information system, especially where the proposal is based on new computer technology rather than many new requirements. Adapting the current system's logical model to the new one in these cases is a straightforward process.

The only disadvantage of the four-model approach is the added time and cost needed to develop a logical and physical model of the current system. Most projects have very tight schedules that might need more time to create the current system models. Additionally, users and managers want to see progress on the new system—they are much less concerned about documenting the current system. The systems analyst must emphasize the importance of careful documentation and resist the pressure to hurry the development process at the risk of creating serious problems later.

5.2 Data Flow Diagrams

Systems analysts use many graphical techniques to describe an information system. One popular method is to draw a set of data flow diagrams (DFDs). A DFD uses various symbols to show how the system transforms input data into useful information. Other graphical tools include object models, as explained in the chapter on object modeling, and entity-relationship diagrams, which are described in the chapter on data design.

During the systems analysis phase of the SDLC, the systems analyst creates a visual model of the information system using a set of DFDs. In his seminal book *The Visual Display of Quantitative Information,* author and renowned academic Edward Tufte provides guidance on creating effective diagrams to convey complex information concisely. A DFD is an example of this visual explanation of system behavior.

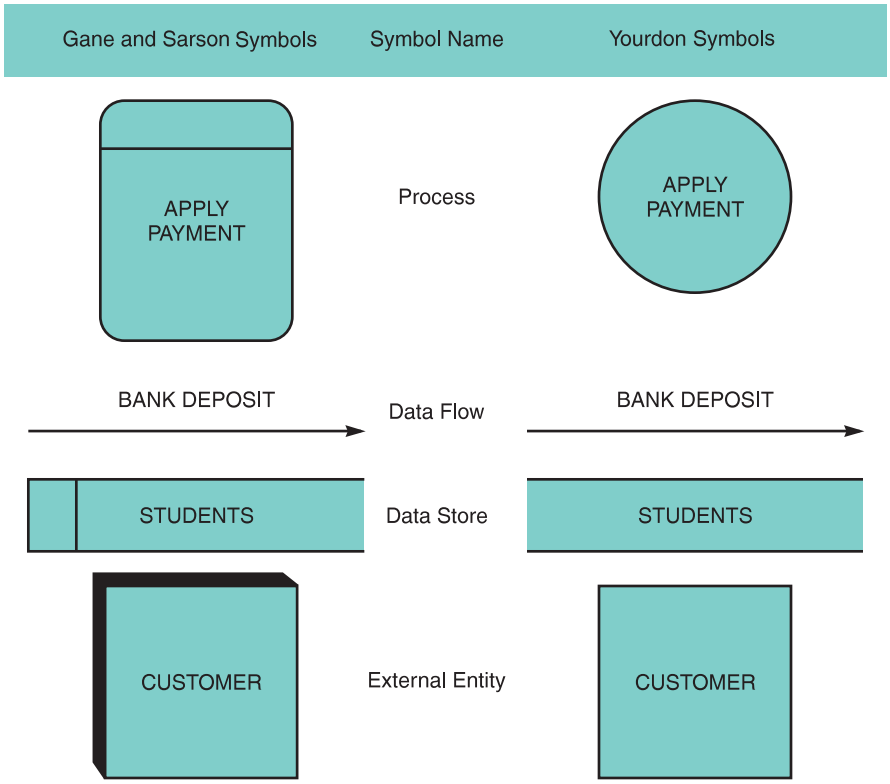
A DFD shows how data moves through an information system but does not show program logic or processing steps. A set of DFDs provides a logical model that shows *what* the system

does, not *how* it does it. That distinction is important because focusing on implementation issues at this point would restrict the search for the most effective system design.

5.3 Data Flow Diagram Symbols

Data flow diagrams use four basic symbols representing processes, data flows, data stores, and entities. Several versions of DFD symbols exist, but they all serve the same purpose. DFD examples in this textbook use **Gane and Sarson symbols**. **Yourdon symbols** are another popular symbol set. Figure 5-1 shows examples of both versions. In this text, symbols are referenced using all capital letters for the symbol name.

Figure 5-1 Data flow diagram symbols and symbol names. The examples use Gane and Sarson symbols and Yourdon symbols.



5.3.1 Process Symbols

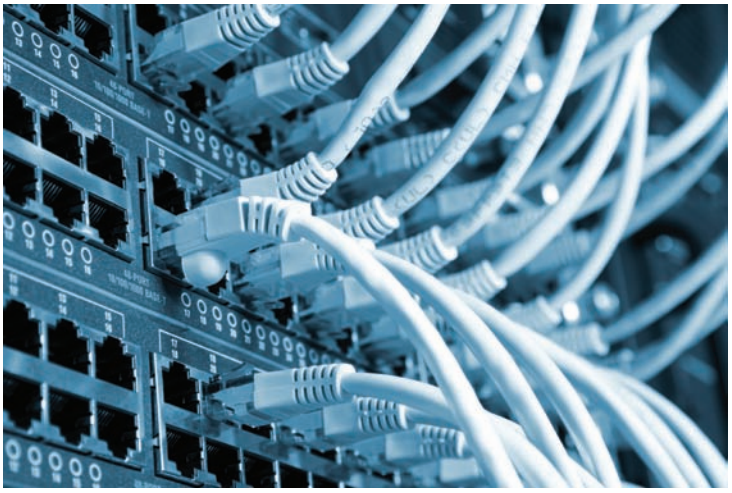
A process receives input data and produces output with a different content, form, or both. For instance, calculating pay uses two inputs (pay rate and hours worked) to produce one output (total pay). Processes can be very simple or quite complex. In a typical company, processes might include calculating sales trends, filing online insurance claims, ordering inventory from a supplier's system, or verifying email addresses for web customers. Processes contain **business logic**, also called business rules, which transform the data and produce the required results.

The Gane and Sarson symbol for a process is a rectangle with rounded corners. The name of the process appears inside the rectangle. The process name identifies a specific function and consists of a verb (and an adjective, if necessary) followed by a singular noun. Process names include APPLY RENT PAYMENT, CALCULATE COMMISSION, ASSIGN FINAL GRADE, VERIFY ORDER, and FILL ORDER.

Processing details are not shown in a DFD. For example, there might be a process named DEPOSIT PAYMENT. The process symbol does not reveal the business logic for the DEPOSIT PAYMENT process. A process description is created to document the logic, as explained later in this chapter.

In DFDs, a process symbol can be called a **black box** because the process's inputs, outputs, and general functions are known, but the underlying details and logic are hidden. By showing processes as black boxes, an analyst can create DFDs that show how the system functions but avoid unnecessary detail and clutter. When analysts want to show additional levels of detail, they can zoom in on a process symbol and create a more in-depth DFD that illustrates the process's internal workings—which might reveal even more processes, data flows, and data stores. In this manner, the information system can be modeled as a series of increasingly detailed pictures.

Figure 5-2 A router acts like a black box for network data. Cables carry data in and out, but internal operations are hidden inside the case.
Source: macka/Shutterstock.com.



The network router shown in Figure 5-2 is an example of a black box. An observer can see cables that carry data into and out of the router, but the router's internal operations are not revealed—only the results are apparent.

5.3.2 Data Flow Symbols

A **data flow** is a path for data to move from one part of the information system to another. A data flow in a DFD represents one or more data items. For example, a data flow could consist of a single data item (such as a student ID number) or a set of data (such as a class roster with student ID numbers, names, and registration dates for a specific class). Although the DFD does not show the detailed contents of a data flow, that information is included in the data dictionary, which is described later in this chapter.

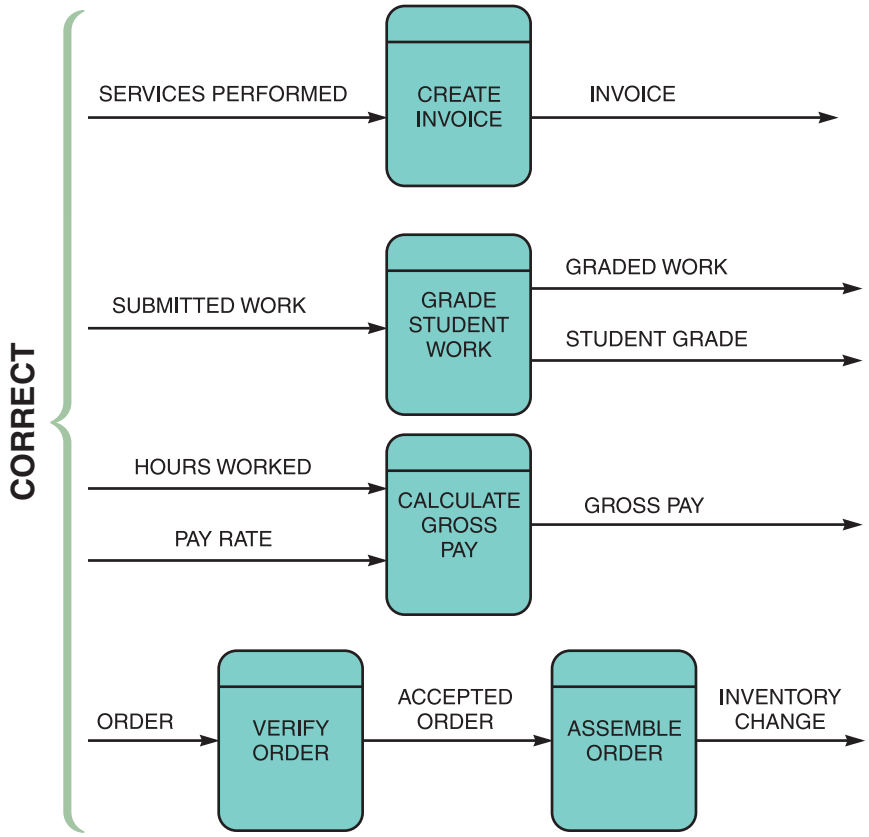
The symbol for a data flow is a line with a single or double arrowhead. The data flow name appears above, below, or alongside the line. A data flow name includes a singular noun and an adjective (if needed). Examples of data flow names are DEPOSIT, INVOICE PAYMENT, STUDENT GRADE, ORDER, and COMMISSION. Exceptions to the singular name rule are data flow names, such as GRADING PARAMETERS, where a singular name could mislead the analyst into thinking a single parameter or data item exists.

Figure 5-3 shows correct examples of data flow and process symbol connections. Because a process changes the data's content or form, at least one data flow must enter and one data flow must exit each process symbol, as in the CREATE INVOICE process. A process symbol can have more than one outgoing data flow, as shown in the GRADE STUDENT WORK process, or more than one incoming data flow, as shown in the CALCULATE GROSS PAY process. A process also can connect to any other symbol, including another process symbol, as shown by the connection between VERIFY ORDER and ASSEMBLE ORDER in Figure 5-3. A data flow, therefore, *must* have a process symbol on at least one end.

Figure 5-4 shows three data flow and process combinations that must be avoided:

- **Spontaneous generation**. For instance, the APPLY INSURANCE PREMIUM process produces output without input data flow. Because it has no input, the process is called a spontaneous generation process.
- **Black hole**. CALCULATE GROSS PAY is called a black hole process, which is a process that has input but produces no output.

Figure 5-3 Examples of correct data flow and process symbol combinations.



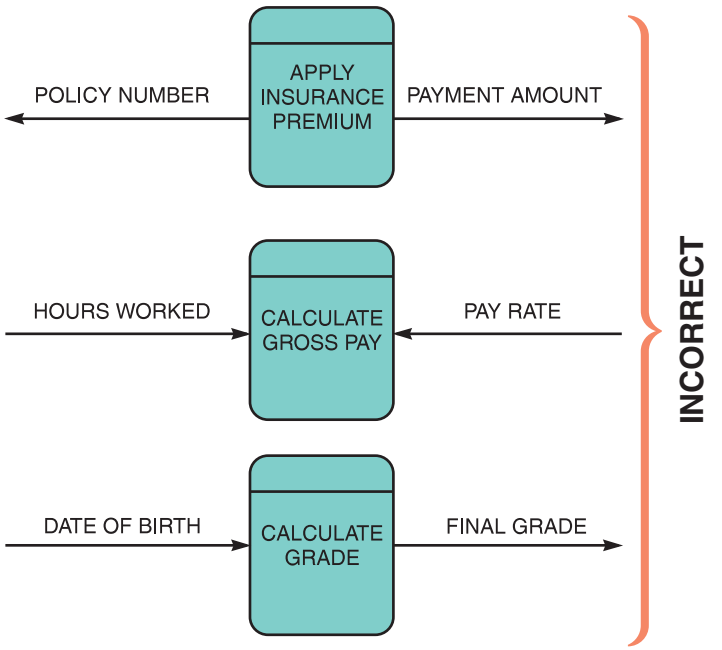
- **Gray hole**. A gray hole is a process with at least one input and one output, but the input is insufficient to generate the output shown. For example, a date of birth input cannot produce a final grade output in the CALCULATE GRADE process.

Spontaneous generation, black holes, and gray holes are logically impossible in a DFD because a process must act on input, shown by an incoming data flow, and produce output, represented by an outgoing data flow.

5.3.3 Data Store Symbols

A **data store** is used in a DFD to represent data that the system stores because one or more processes need to use the data later. For instance, instructors need to store student scores on tests and assignments during the semester to assign final grades at the end of the term. Similarly, a company stores employee salary and deduction data during the year to print W-2 forms with total earnings and deductions at the end of the year. A DFD does not show the detailed contents of a data store—the specific structure and data elements are defined in the data dictionary, which is discussed later in this chapter.

Figure 5-4 Examples of incorrect data flow and process symbol combinations. APPLY INSURANCE PREMIUM has no input and is called a spontaneous generation process. CALCULATE GROSS PAY has no outputs and is called a black hole process. CALCULATE GRADE has an input that is obviously unable to produce the output. This process is called a gray hole.

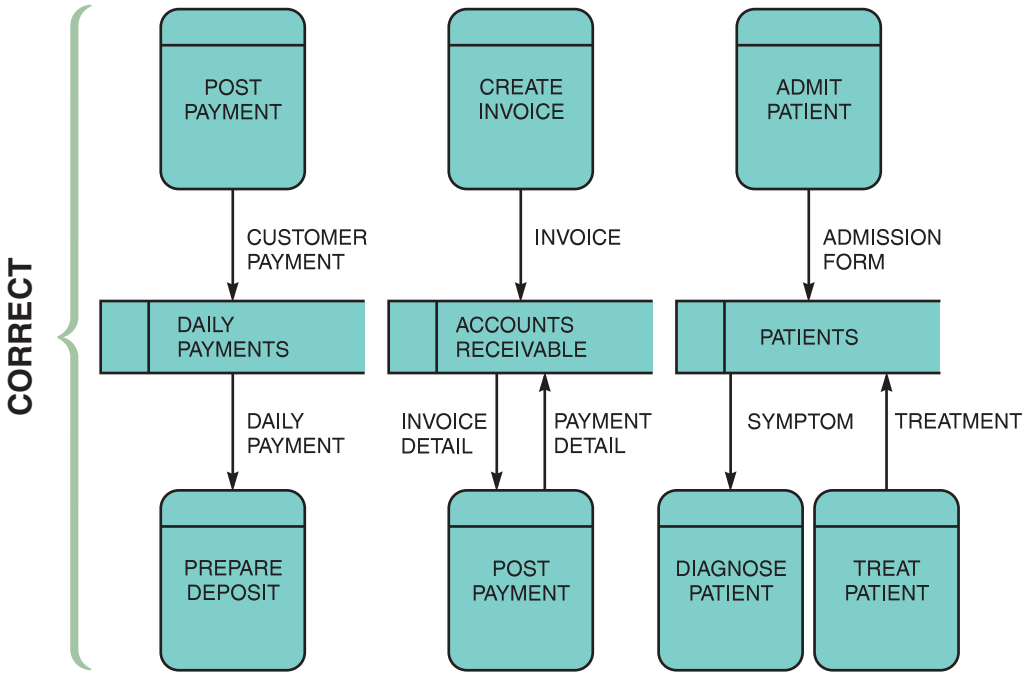


The physical characteristics of a data store are unimportant because the logical model is the only concern at this point. Also, the time that the data is stored is unimportant—it can be a matter of seconds while a transaction is processed or a period of months while data is accumulated for year-end processing. What is important is that a process needs access to the data at some later time.

In a DFD, the Gane and Sarson symbol for a data store is a flat rectangle that is open on the right side and closed on the left side. The name of the data store appears between the lines and identifies the data it contains. A data store name is a plural name consisting of a noun and adjectives (if needed). Examples of data store names are STUDENTS, ACCOUNTS RECEIVABLE, PRODUCTS, DAILY PAYMENTS, PURCHASE ORDERS, OUTSTANDING CHECKS, INSURANCE POLICIES, and EMPLOYEES. Exceptions to the plural name rule are collective nouns that represent multiple occurrences of objects. For example, GRADEBOOK represents a group of students and their scores.

A data store must be connected to a process with a data flow. Figure 5-5 illustrates typical examples of data stores. Because data stores represent data storage for use by another process in the future, each data store has at least one incoming and one outgoing data flow. It is connected to a process symbol with a data flow.

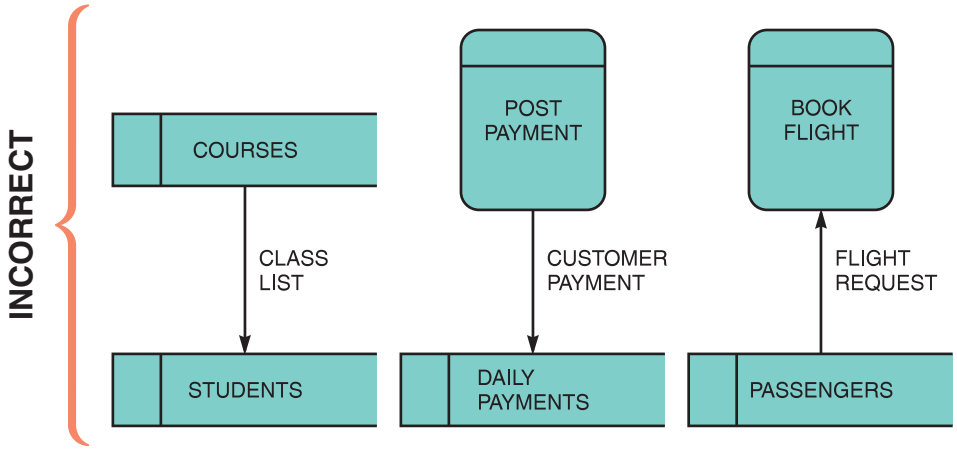
Figure 5-5 Examples of correct use of data store symbols in a DFD.



Violations of the rule that a data store must have at least one incoming and one outgoing data flow are shown in Figure 5-6. In the first example, two data stores are misconnected because no process is between them. Also, COURSES has no incoming data flow, and STUDENTS has no outgoing data flow. In the second and third examples, the data stores lack either an outgoing or incoming data flow.

There is an exception to the requirement that a data store must have at least one incoming and one outgoing data flow. In some situations, a data store has no input data flow because it contains fixed reference data not updated by the system. For example, consider a data store called TAX TABLE, which contains withholding tax data that a company downloads from the Internal Revenue Service. When the company runs its payroll, the CALCULATE WITHHOLDING process accesses data from this data store. This would be represented

Figure 5-6 Examples of incorrect use of data store symbols in a DFD. Two data stores cannot be connected by a data flow without an intervening process, and each data store should have an outgoing and incoming data flow.



as a one-way outgoing data flow from the TAX TABLE data store into the CALCULATE WITHHOLDING process on a DFD.

5.3.4 Entity Symbols

The symbol for an **entity** is a rectangle, which may be shaded to make it look three-dimensional. The name of the entity appears inside the symbol.

A DFD shows only external entities that provide data to the system or receive output from the system. A DFD shows the boundaries of the system and how the system interfaces with the outside world. For example, a customer entity submits an order to an order processing system. Other examples of entities include a patient who supplies data to a medical records system, a homeowner who receives a bill from a city property tax system, or an accounts payable system that receives data from the company's purchasing system.

DFD entities also are called **terminators** because they are data origins or final destinations. An entity that supplies data to the system is called a **source**, and an entity that receives data from the system is a **sink**. An entity name is the singular form of a department, outside organization, other information system, or person. An external entity can be a source, sink, or both, but each entity must be connected to a process by a data flow. Figures 5-7 and 5-8 show correct and incorrect examples of this rule, respectively.

Figure 5-7 Examples of correct uses of external entities in a DFD.

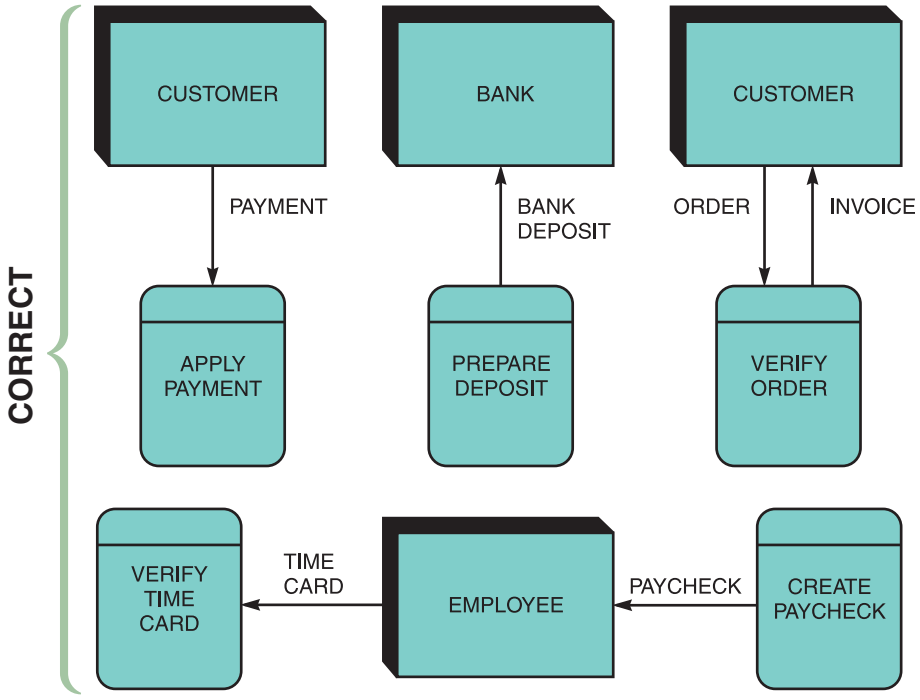
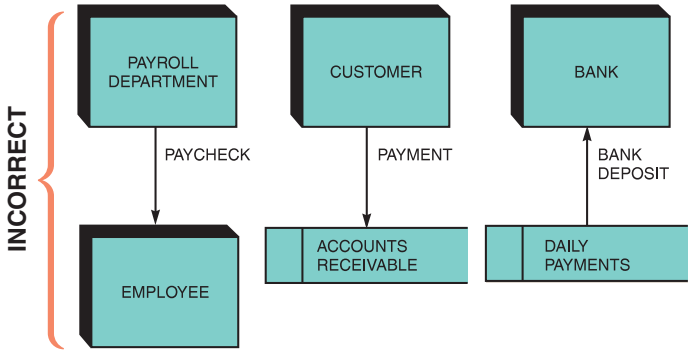


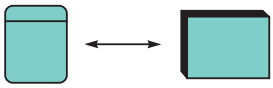
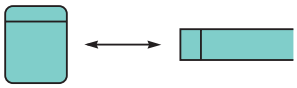
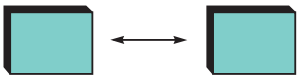
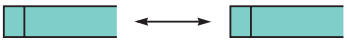
Figure 5-8 Examples of incorrect uses of external entities in a DFD. A data flow to a process must connect to an external entity and not directly to a data store or another external entity.



5.3.5 Using DFD Symbols

With an understanding of the proper use of DFD symbols, the next step is to construct diagrams that use these symbols. Figure 5-9 shows a summary of the rules for using DFD symbols.

Figure 5-9 Examples of correct and incorrect uses of data flows.

Correct and Incorrect Examples of Data Flows		
	Process to Process	✓
	Process to External Entity	✓
	Process to Data Store	✓
	External Entity to External Entity	✗
	External Entity to Data Store	✗
	Data Store to Data Store	✗

5.4 Drawing Data Flow Diagrams

During requirements engineering, interviews, questionnaires, and other techniques were used to gather facts about the system, and it was explained how the various people, departments, data, and processes fit together to support business operations. Now a graphical information system model is created based on the fact-finding results.

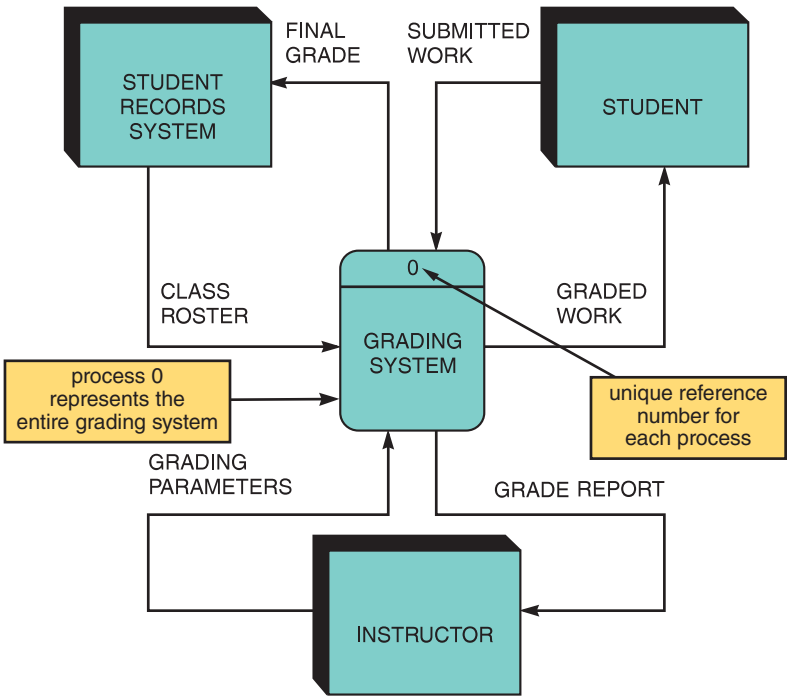
To explain how to draw DFDs, examples of two information systems will be used. The first example is a grading system that instructors use to assign final grades based on students'

scores during the term. The second example is an order system a company uses to enter orders and apply payments against a customer's balance.

When drawing a context diagram and other DFDs, these guidelines should be followed:

- Draw the context diagram so it fits on one page. See the next section for more details about context diagrams.
- Use the name of the information system as the process name in the context diagram. For example, the process name in Figure 5-10 is GRADING SYSTEM. Notice that the process name is the same as the system name. This is because the context diagram shows the entire information system as a single process. For processes in lower-level DFDs, use a verb followed by a descriptive noun, such as ESTABLISH GRADEBOOK, ASSIGN FINAL GRADE, or PRODUCE GRADE REPORT.

Figure 5-10 Context diagram DFD for a grading system.



- Use unique names within each set of symbols. For instance, the diagram in Figure 5-10 shows only one entity named STUDENT and one data flow called FINAL GRADE. Whenever the entity STUDENT appears on any other DFD in the grading system, it is the same entity. Whenever the FINAL GRADE data flow appears, it is the same data flow. The naming convention also applies to data stores.
- Do not cross lines. One way to achieve that goal is to restrict the number of symbols in any DFD. At most, nine process symbols should be included on lower-level diagrams with multiple processes. Including more than nine symbols usually signals that the diagram is too complex and that the analysis should be reconsidered. Another way to avoid crossing lines is to duplicate an entity or data store. When duplicating a symbol on a diagram, document the duplication to avoid possible confusion. A special notation, such as an asterisk, next to the symbol name and inside the duplicated symbols signifies that they are duplicated on the diagram.
- Provide a unique name and reference number for each process. Because it is the highest-level DFD, the context diagram contains process 0, representing the entire information system but not the internal workings. To describe the next level of detail inside process 0, create a DFD named diagram 0, revealing additional processes that must be named and numbered. As lower-level DFDs are created, assign unique names and reference numbers to all processes until the logical model is completed.
- Obtain as much user input and feedback as possible. The main objective is to ensure that the model is accurate, easy to understand, and meets the needs of its users.

5.5 Drawing a Context Diagram

The first step in constructing a set of DFDs is to draw a context diagram. A **context diagram** is a top-level view of an information system that shows the system's boundaries and scope. To draw a context diagram, start by placing a single process symbol in the center of the page.

The symbol represents the entire information system and is identified as **process 0** (the numeral zero, not the letter O). Then place the system entities around the perimeter of the page and use data flows to connect the entities to the central process. Data stores are not shown in the context diagram because they are contained within the system and remain hidden until more detailed diagrams are created.

To determine which entities and data flows to place in the context diagram, begin by reviewing the system requirements to identify all external data sources and destinations. During that process, identify the entities, the name and content of the data flows, and the direction of the data flows. If that is done carefully, and the job of fact-finding was done well in the previous stage, drawing the context diagram should be relatively easy. Now review the following context diagram examples.

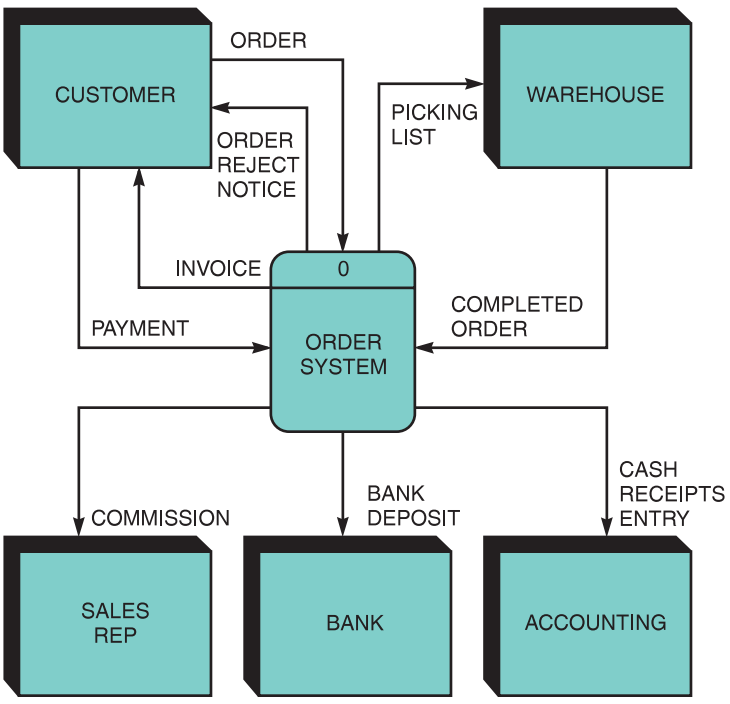
5.5.1 Example Context Diagram for a Grading System

The context diagram for a grading system is shown in Figure 5-10. The GRADING SYSTEM process is at the center of the diagram. The three entities (STUDENT RECORDS SYSTEM, STUDENT, and INSTRUCTOR) are placed around the central process. Interaction among the main process and the entities involves six different data flows. The STUDENT RECORDS SYSTEM entity supplies data through the CLASS ROSTER data flow and receives data through the FINAL GRADE data flow. The STUDENT entity delivers data through the SUBMITTED WORK data flow and receives data through the GRADED WORK data flow. Finally, the INSTRUCTOR entity provides data through the GRADING PARAMETERS data flow and receives data through the GRADE REPORT data flow.

5.5.2 Example Context Diagram for an Order System

The context diagram for an order system is shown in Figure 5-11. Notice that the ORDER SYSTEM process is at the center of the diagram, and five entities surround the process. Three entities, SALES REP, BANK, and ACCOUNTING, have single incoming data flows for COMMISSION, BANK DEPOSIT, and CASH RECEIPTS ENTRY, respectively. The WAREHOUSE entity has one incoming data flow—PICKING LIST—a report showing the

Figure 5-11 Context diagram DFD for an order system.



items ordered and their quantity, location, and sequence to pick from the warehouse. The WAREHOUSE entity has one outgoing data flow: COMPLETED ORDER. Finally, the CUSTOMER entity has two outgoing data flows, ORDER and PAYMENT, and two incoming data flows, ORDER REJECT NOTICE and INVOICE.

The context diagram for the order system appears more complex than the grading system because it has two more entities and three more data flows. What makes one system more complex than another is the number of components, the number of levels, and the degree of interaction among its processes, entities, data stores, and data flows.

5.6 Drawing a Diagram 0 DFD

The previous section explained how a context diagram provides the most general view of an information system and contains a single process symbol, which is like a black box. To show the detail inside the black box, a DFD diagram 0 is created. **Diagram 0** (the numeral zero, not the letter *O*) provides an overview of all the components that interact to form the overall system. It zooms in on the system and shows major internal processes, data flows, and data stores. Diagram 0 also repeats the entities and data flows in the context diagram. When the context diagram is expanded into DFD diagram 0, all the connections that flow into and out of process 0 must be retained.

5.6.1 Example Diagram 0 DFD for a Grading System

Figure 5-12 shows a context diagram at the top and diagram 0 DFD beneath. Notice that diagram 0 is an expansion of process 0. Also, notice that the same three entities (STUDENT RECORDS SYSTEM, STUDENT, and INSTRUCTOR) and the same six data flows (FINAL GRADE, CLASS ROSTER, SUBMITTED WORK, GRADED WORK, GRADING PARAMETERS, and GRADE REPORT) appear in both diagrams. In addition, diagram 0 expands process 0 to reveal four internal processes, one data store, and five additional data flows.

Notice that each process in diagram 0 has a reference number: ESTABLISH GRADEBOOK is 1, ASSIGN FINAL GRADE is 2, GRADE STUDENT WORK is 3, and PRODUCE GRADE REPORT is 4. These reference numbers are significant because they identify a series of DFDs. If more detail were needed for ESTABLISH GRADEBOOK, for example, a diagram 1 would be drawn because ESTABLISH GRADEBOOK is process 1.

The process numbers do not suggest that the processes are accomplished in sequential order. Each process is always considered available, active, and awaiting processing data. If processes must be performed in a specific sequence, the information should be documented in the process descriptions (discussed later in this chapter), not in the DFD.

The FINAL GRADE data flow output from the ASSIGN FINAL GRADE process is a diverging data flow that becomes an input to the STUDENT RECORDS SYSTEM entity and the GRADEBOOK data store. A **diverging data flow** is a data flow in which the same data travels to two or more locations. In that situation, a diverging data flow is the best way to show the flow rather than two identical data flows, which could be misleading.

A double-headed arrow can connect the symbols if the same data flows in both directions. To identify specific data flows into and out of a symbol, separate data flow symbols with single arrowheads should be used. For example, in Figure 5-12, the separate data flows (SUBMITTED WORK and GRADED WORK) go into and out of the GRADE STUDENT WORK process.

Because diagram 0 is an exploded version of process 0, it shows considerably more detail than the context diagram. Diagram 0 can also be called a partitioned or decomposed view of process 0. When a DFD is exploded, the higher-level diagram is called the **parent diagram** and the lower-level diagram is referred to as the **child diagram**. The grading system is simple enough that no additional DFDs are needed to model the system. At that point, the four processes, the one data store, and the 10 data flows can be documented in the data dictionary.

Figure 5-12 Context diagram and diagram 0 for the grading system.

Context Diagram for Grading System

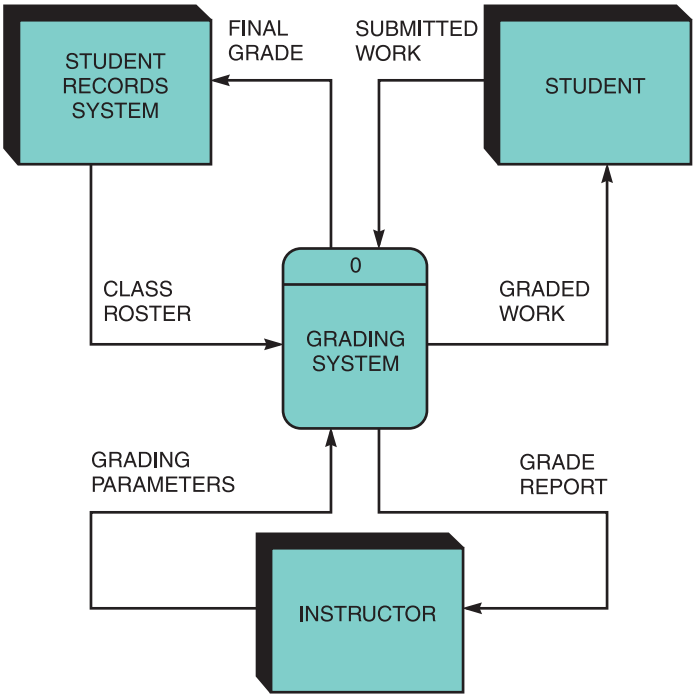
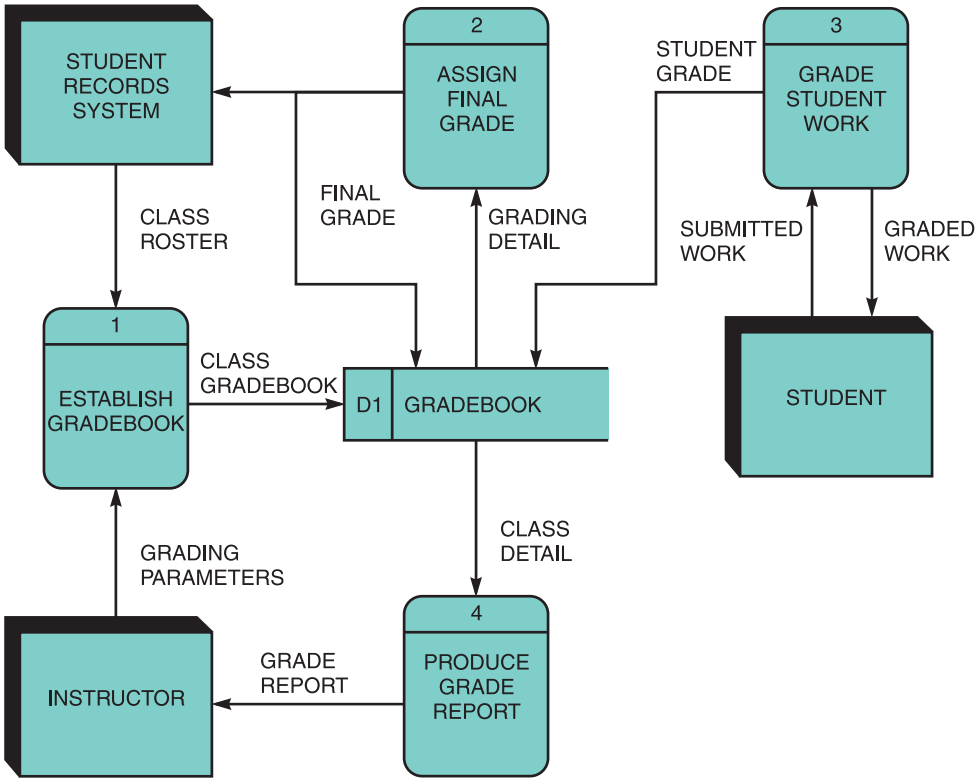


Diagram 0 for Grading System



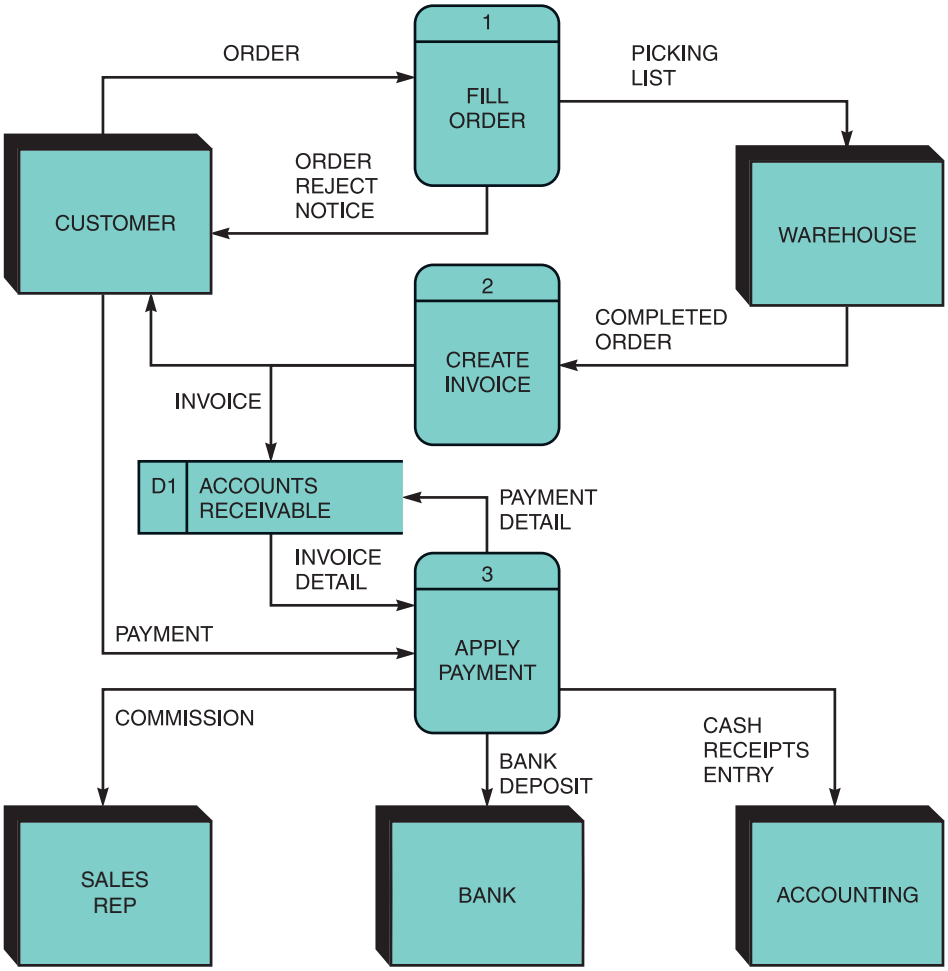
When a set of DFDs is created for a system, the processing logic is broken down into smaller units, called functional primitives, which programmers will use to develop code. A **functional primitive** is a process that consists of a single function that is not exploded further. For example, each of the four processes shown in the lower portion of Figure 5-12 is a functional primitive. The logic for a functional primitive is documented by writing a process description in the data dictionary. Later, when the logical design is implemented as a physical system, programmers will transform each functional primitive into program code and modules that carry out the required steps. Deciding whether to explode a process further or determine that it is a functional primitive is a matter of experience, judgment, and interaction with programmers who must translate the logical design into code.

5.6.2 Example Diagram 0 DFD for an Order System

Figure 5-13 shows a diagram 0 DFD for an order system. Process 0 on the order system's context diagram is exploded to reveal three processes (FILL ORDER, CREATE INVOICE, and APPLY PAYMENT), one data store (ACCOUNTS RECEIVABLE), two additional data flows (INVOICE DETAIL and PAYMENT DETAIL), and one diverging data flow (INVOICE).

The following walkthrough explains the DFD shown in Figure 5-13.

Figure 5-13 Diagram 0 DFD for the order system.



1. A CUSTOMER submits an ORDER. Depending on the processing logic, the FILL ORDER process either sends an ORDER REJECT NOTICE back to the customer or sends a PICKING LIST to the WAREHOUSE.

2. A COMPLETED ORDER from the WAREHOUSE is input to the CREATE INVOICE process, which outputs an INVOICE to both the CUSTOMER process and the ACCOUNTS RECEIVABLE data store.
3. A CUSTOMER makes a PAYMENT that is processed by APPLY PAYMENT. APPLY PAYMENT requires INVOICE DETAIL input from the ACCOUNTS RECEIVABLE data store along with the PAYMENT. APPLY PAYMENT also outputs PAYMENT DETAIL back to the ACCOUNTS RECEIVABLE data store and outputs COMMISSION to the SALES REP, BANK DEPOSIT to the BANK, and CASH RECEIPTS ENTRY to ACCOUNTING.

The walkthrough of diagram 0 illustrates the basic requirements of the order system. Examine the detailed description of each separate process to learn more.

5.7 Drawing Lower-Level DFDs

This section's examples of lower-level DFDs are based on the example order system. To create lower-level diagrams, leveling and balancing techniques must be used. **Leveling** draws increasingly detailed diagrams until all functional primitives are identified. **Balancing** maintains consistency among DFDs by ensuring that input and output data flows align properly. Leveling and balancing are described in more detail in the following sections.

5.7.1 Leveling Examples

Leveling uses a series of increasingly detailed DFDs to describe an information system. For example, a system might consist of dozens or hundreds of separate processes. Using leveling, an analyst starts with an overall view, which is a context diagram with a single process symbol. Next, the analyst creates diagram 0, which shows more detail. The analyst continues to create lower-level DFDs until all processes are identified as functional primitives representing single processing functions. More complex systems have more processes; analysts must work through many levels to determine the functional primitives. Leveling also is called **exploding**, **partitioning**, or **decomposing**.

Figures 5-13 and 5-14 provide an example of leveling. Figure 5-13 shows diagram 0 for an order system, with the FILL ORDER process labeled as process 1. Now consider Figure 5-14, which provides an exploded view of the FILL ORDER process. Notice that FILL ORDER (process 1) consists of three processes: VERIFY ORDER (process 1.1), PREPARE REJECT NOTICE (process 1.2), and ASSEMBLE ORDER (process 1.3).

As Figure 5-14 shows, all processes are numbered using a decimal notation consisting of the parent's reference number, a decimal point, and a sequence number within the new diagram. In Figure 5-14, the parent process of diagram 1 is process 1, so the processes in diagram 1 have reference numbers 1.1, 1.2, and 1.3. If process 1.3, ASSEMBLE ORDER, is decomposed further, then it would appear in diagram 1.3, and the processes in diagram 1.3 would be numbered as 1.3.1, 1.3.2, 1.3.3, and so on. This numbering technique makes it easy to integrate and identify all DFDs.

When Figures 5-13 and 5-14 are compared, it is apparent that Figure 5-14 (the exploded FILL ORDER process) shows two data stores (CUSTOMERS and PRODUCTS) that do not appear in Figure 5-13, which is the parent DFD. Why not? The answer is based on a simple rule: When drawing DFDs, a data store is shown only when two or more processes use that data store. The CUSTOMERS and PRODUCTS data stores are internal to the FILL ORDER process, so the analyst did not show them on diagram 0, which is the parent. However, when the FILL ORDER process is exploded into a diagram 1 DFD, three processes (1.1, 1.2, and 1.3) interacting with the two data stores are now shown.

Now compare Figures 5-14 and 5-15. Figure 5-15 shows the same data flows but not the CUSTOMER and WAREHOUSE entities. Analysts often use this technique to simplify a DFD and reduce unnecessary clutter. Because the missing symbols appear on the parent DFD, that diagram can be used to identify the source or destination of the data flows.

Figure 5-14 Diagram 1 DFD details the FILL ORDER process in the order system.

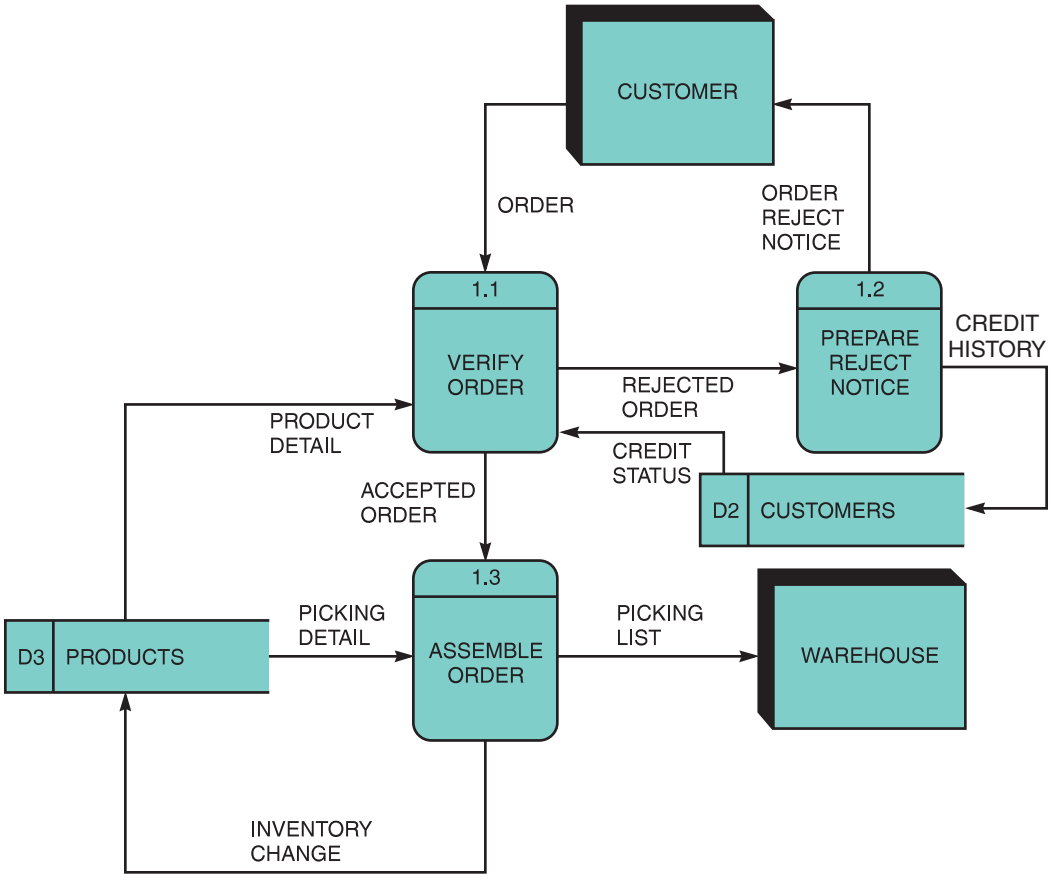


Figure 5-15 This diagram does not show the symbols that connect to data flows entering or leaving FILL ORDER on the context diagram.

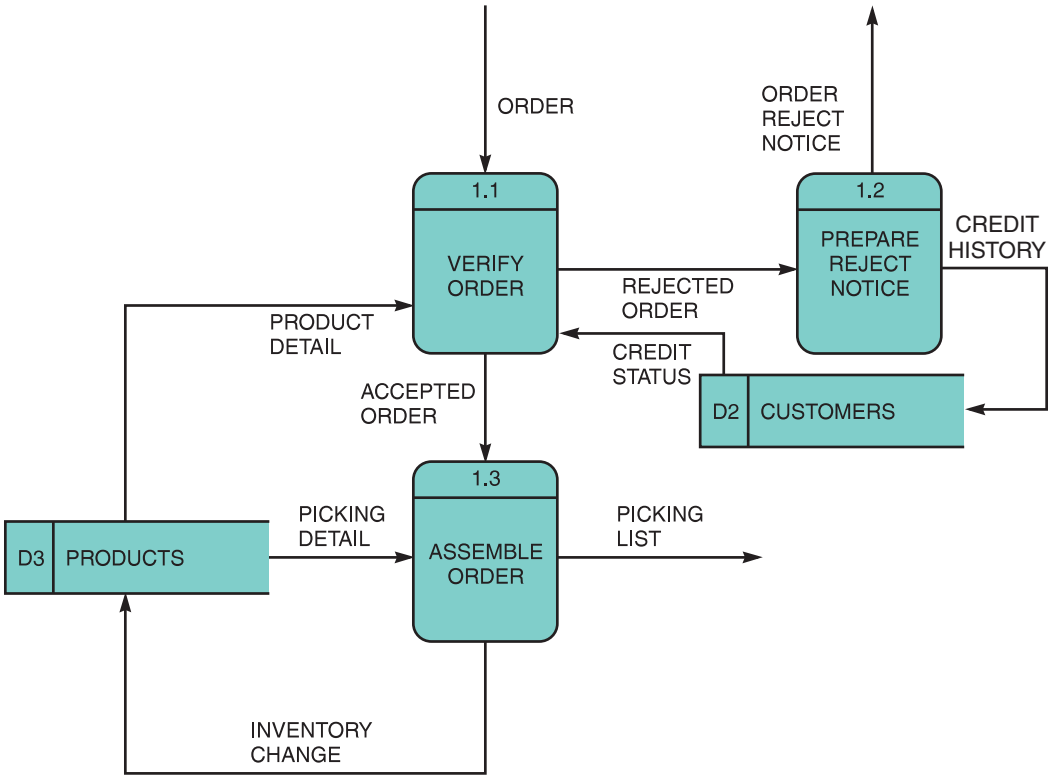
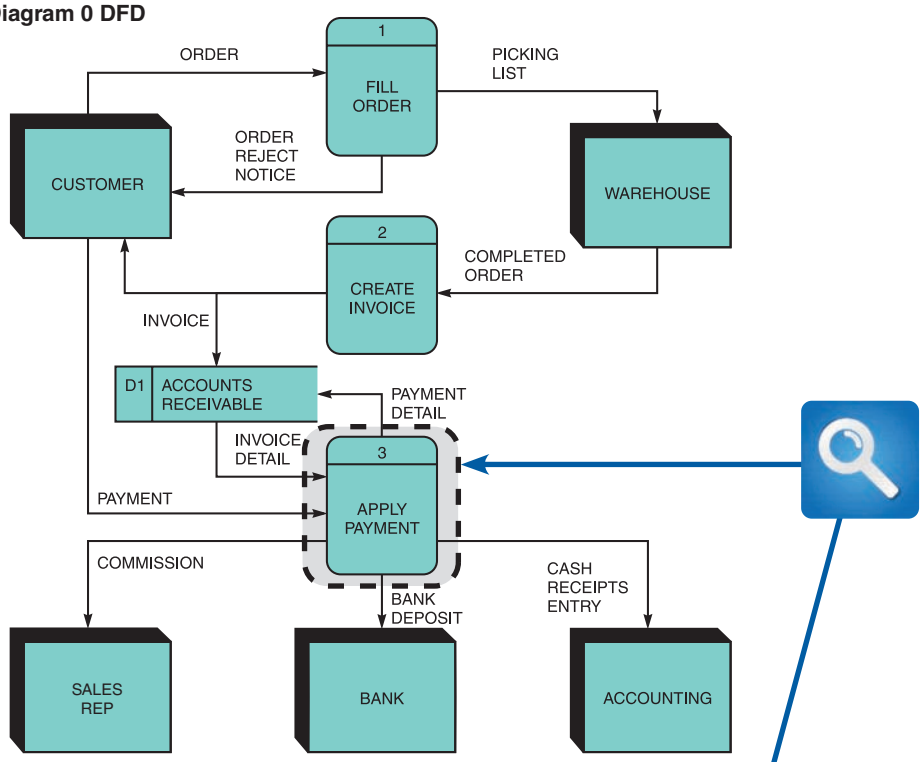
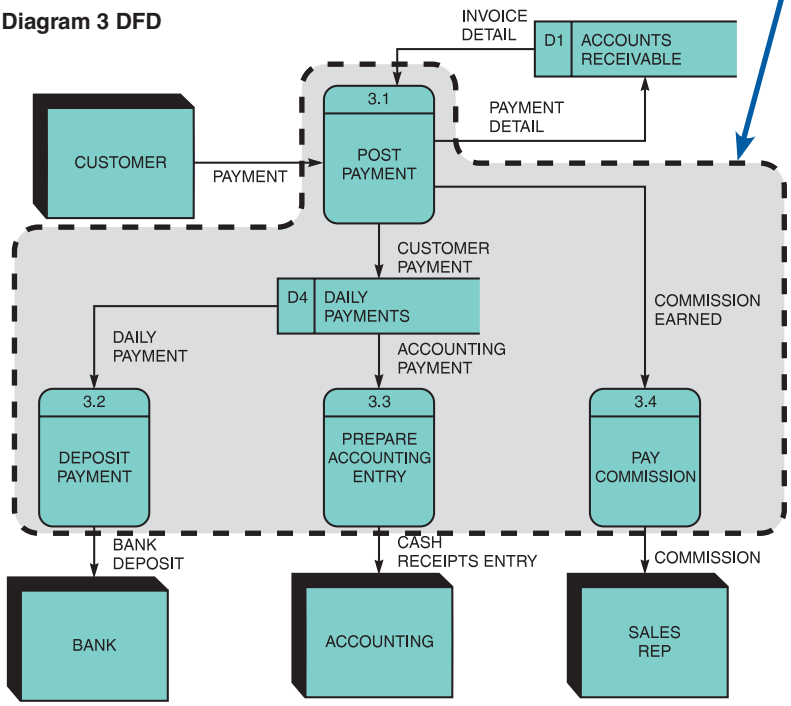


Figure 5-16 The order system diagram 0 is shown at the top of the figure, and the exploded diagram 3 DFD (for the APPLY PAYMENT process) is shown at the bottom. The two DFDs are balanced because the child diagram at the bottom has the same input and output flows as the parent process 3 shown at the top.

Order System Diagram 0 DFD



Order System Diagram 3 DFD



5.7.2 Balancing Examples

Balancing ensures that the input and output data flows of the parent DFD are maintained on the child DFD. For example, Figure 5-16 shows two DFDs: the order system diagram 0 is at the top of the figure and the exploded diagram 3 DFD is at the bottom.

The two DFDs are balanced because the child diagram at the bottom has the same input and output flows as the parent process 3 shown at the top. To verify the balancing, notice that the parent process 3, APPLY PAYMENT, has one incoming data flow from an external entity and three outgoing data flows to external entities. Examine the child DFD, which is diagram 3. Ignore the internal data flows and count the data flows to and from external entities. From the diagram, it is evident that the three processes maintain the same one incoming and three outgoing data flows as the parent process.

Another example of balancing is shown in Figures 5-17 and 5-18. Figure 5-17 shows a sample context diagram. The process 0 symbol has two input flows and two output flows. Process 0 can be considered a black box, with no internal detail shown. In Figure 5-18, process 0 (the parent DFD) is exploded into the next level of detail. Now three processes, two data stores, and four internal data flows are visible. Notice that the details of process 0 are shown inside a dashed line, just as if the inside of the process was visible.

Figure 5-17 Example of a parent DFD, showing process 0 as a black box.
Source: Author-generated content (Lucidchart).

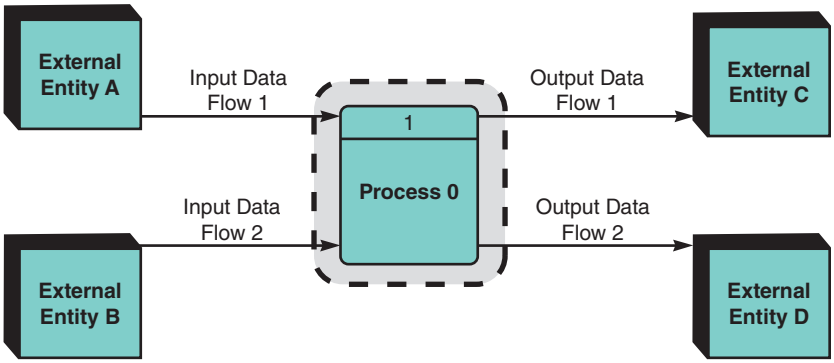
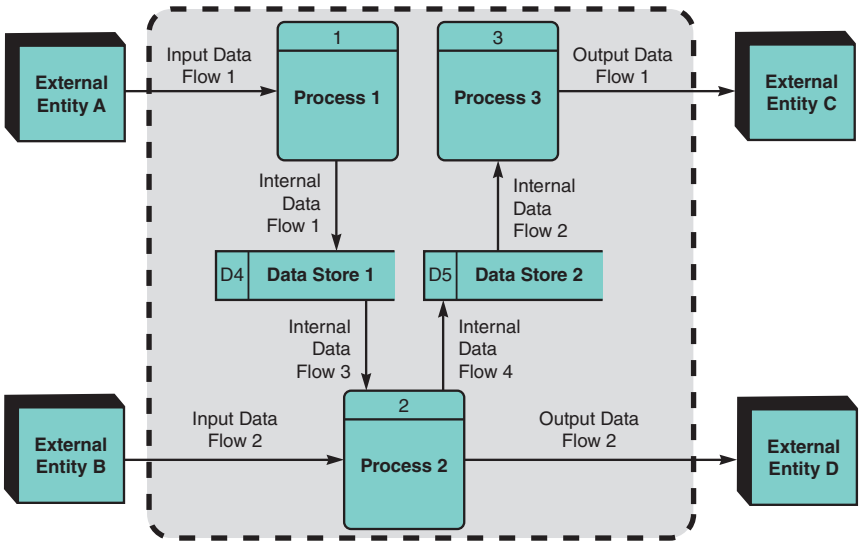


Figure 5-18 In the next level of detail, the process 0 black box reveals three processes, two data stores, and four internal data flows—all shown inside the dashed line.
Source: Author-generated content (Lucidchart).



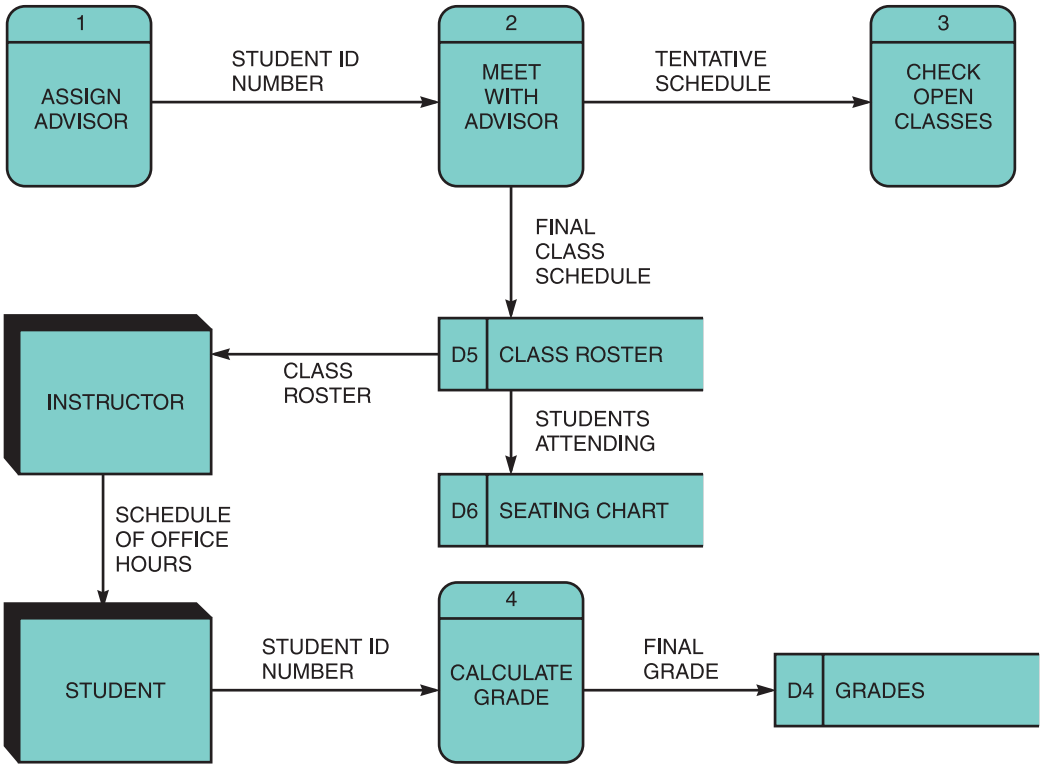
The DFDs in Figures 5-17 and 5-18 are balanced because the four data flows into and out of process 0 are maintained on the child DFD. The DFDs also are leveled because each internal process is numbered to show that it is a child of the parent process.

Case in Point 5.1

Big Ten University

You are the IT director at Big Ten University. As part of a training program, you draw a DFD that includes some obvious mistakes to see whether your newly hired junior analysts can find them. You came up with the diagram 0 DFD shown in Figure 5-19. Based on the rules explained in this chapter, how many problems should the analysts find?

Figure 5-19 What are the mistakes in this diagram 0 DFD?



5.8 Data Dictionary

A set of DFDs produces a logical system model. Still, the details within those DFDs are documented separately in a **data dictionary**, which contains information about data, such as its meaning, relationships to other data, origin, usage, and format. A data dictionary typically includes the following:

- **Data element descriptions.** Also called a **data item** or **field**, a **data element** is the smallest piece of data that has meaning within an information system. This includes the names of all data elements in a system, the meaning of these data elements, and the abbreviations or codes used to represent them. Data elements are combined into **records**, also called **data structures**. A record is a meaningful combination of related data elements included in a data flow or retained in a data store.
- **Attribute/property definitions.** This includes the characteristics of the data elements, like the type of data (e.g., integer, real number, character, date), length of the data field, the allowable range of values, the default value, and whether the field is a key or part of a key.

- **Relationship descriptions.** Describes relationships between data elements or groups of data elements.
- **Rules and constraints.** Business rules or integrity constraints associated with the data. This can include conditions that must be true before a transaction can occur or after it has occurred.
- **Other metadata.** Other important information about the data, including ownership, security, or privacy information, data origin (source), data usage, and so on.

Significant relationships exist among the items in a data dictionary. For example, data stores and data flows are based on data structures composed of data elements. Data flows are connected to data stores, entities, and processes. Accurately documenting these relationships is essential so the data dictionary is consistent with the DFDs. The more complex the system, the more difficult it is to maintain full and accurate documentation. Fortunately, modern tools simplify the task by flowing documentation automatically from the modeling diagrams into the central repository, along with information entered by the user.

A **data repository** ensures data consistency, especially when multiple systems require the same data. In a large company, for example, the sales, accounting, and shipping systems all might use a data element called CUSTOMER NUMBER. Once the CUSTOMER NUMBER element has been defined in the repository, other processes can access it, data flows, and data stores. The result is that all systems across the enterprise can share up-to-date and consistent data.

5.8.1 Documenting the Data Elements

Every data element in the data dictionary must be documented. Some analysts like to record their notes on online or manual forms. Others prefer to enter the information directly into a repository. Irrespective of the specific tool used, the objective is to provide clear, comprehensive information about the data and processes that make up the system.

The following data element attributes are usually documented in the data dictionary:

- **Data element name** or **label.** The data element's standard name, which should be meaningful to users.
- **Alias.** Any name(s) other than the standard data element name; this alternate name is called an **alias**. For example, suppose there is a data element named CURRENT BALANCE. Users might refer to it by alternate names such as OUTSTANDING BALANCE, CUSTOMER BALANCE, RECEIVABLE BALANCE, or AMOUNT OWED.
- **Type** and **length.** Type refers to whether the data element contains numeric, alphabetic, or character values. Length is the maximum number of characters for an alphabetic or character data element or the maximum number of digits and decimal positions for a numeric data element. In addition to text and numeric data, sounds and images can be stored digitally. In some systems, these binary data objects are managed and processed as traditional data elements are. For example, an employee record might include a digitized photo image of the person.
- **Default value.** The default value is the value for the data element if a value is not entered for it. For example, all new customers might have a default value of $500 for the CREDIT LIMIT data element.
- **Acceptable values.** Specification of the data element's **domain**, which is the set of values permitted for the data element. These values can be specifically listed or referenced in a table or selected from a specified range of values. Also, indicate if a value for the data element is optional. Some data elements have additional **validity rules**. For example, an employee's salary must be within the range defined for the employee's job classification.

- **Source.** The origin of the data element's values. The source could be a specific form, a department or outside organization, another information system, or the result of a calculation.
- **Security.** Identification of the individual or department with access or update privileges for each data element. For example, only a credit manager can change a credit limit, while sales reps are authorized to access data in a read-only mode.
- **Responsible user(s).** Identification of the user(s) responsible for entering and changing values for the data element.
- **Description** and **comments.** This part of the documentation allows the entry of additional notes.

5.8.2 Documenting the Data Flows

In addition to documenting each data element, all data flows in the data dictionary must be documented. Although terms can vary, the typical attributes are as follows:

- **Data flow name** or **label.** The data flow name as it appears on the DFDs.
- **Description.** Describes the data flow and its purpose.
- **Alternate name(s).** Aliases for the DFD data flow name(s).
- **Origin.** The DFD beginning, or source, for the data flow; the origin can be a process, a data store, or an entity.
- **Destination.** The DFD ending point(s) for the data flow; the destination can be a process, a data store, or an entity.
- **Record.** Each data flow represents a group of related elements called a record or data structure. In most data dictionaries, records are defined separately from the data flows and data stores. When records are defined, more than one data flow or data store can use the same record if necessary.
- **Volume** and **frequency.** Describes the expected number of occurrences for the data flow per unit of time. For example, if a company has 300 employees, a TIME CARD data flow would involve 300 transactions and records each week as employees submit their work-hour data.

5.8.3 Documenting the Data Stores

Every DFD data store in the data dictionary must be documented. Typical characteristics of a data store are as follows:

- **Data store name** or **label.** The data store name as it appears on the DFDs.
- **Description.** Describes the data store and its purpose.
- **Alternate name(s).** Aliases for the DFD data store name.
- **Attributes.** Standard DFD names that enter or leave the data store.
- **Volume** and **frequency.** Describes the estimated number of records in the data store and how frequently they are updated.

5.8.4 Documenting the Processes

Every DFD process in the data dictionary must be documented. The documentation includes a description of the process's characteristics and, for functional primitives, a process description, which is a model that documents the processing steps and business logic.

The following are typical characteristics of a process:

- **Process name** or **label.** The process name as it appears on the DFDs.
- **Description.** A brief statement of the process's purpose.
- **Process number.** A reference number that identifies the process and indicates relationships among various levels in the system.
- **Process description.** This section includes the input and output data flows. For functional primitives, the process description also documents the processing steps and business logic. Section 5.9 explains how to write process descriptions.

5.8.5 Documenting the Entities

The data dictionary can describe all external entities interacting with the system by documenting all entities. Typical characteristics of an entity include the following:

- **Entity name.** The entity name as it appears on the DFDs.
- **Description.** A brief explanation of the entity and its purpose.
- **Alternate name(s).** Any aliases for the entity name.
- **Input data flows.** The standard DFD names for the input data flows to the entity.
- **Output data flows.** The standard DFD names for the data flows leaving the entity.

5.8.6 Documenting the Records

A record is a data structure containing related data elements stored and processed together. Data flows and data stores consist of records that must be documented in the data dictionary. Typical characteristics of a record include the following:

- **Record** or **data structure name.** The record name as it appears in the related data flow and data store entries in the data dictionary.
- **Definition** or **description.** A brief explanation of the record.
- **Alternate name(s).** Any aliases for the record name.
- **Attributes.** A list of all the data elements, or fields, included in the record. The data element names must match exactly those entered in the data dictionary.

5.8.7 Data Dictionary Reports

The data dictionary serves as an information system's central storehouse of documentation. A data dictionary is created when the system is developed and is updated constantly as the system is implemented, operated, and maintained. In addition to describing each data element, data flow, data store, record, entity, and process, the data dictionary documents the relationships among these components.

Many valuable reports can be obtained from a data dictionary, including the following:

- An alphabetized list of all data elements by name.
- A report describing each data element and indicating the user or department responsible for data entry, updating, or deletion.
- A report of all data flows and data stores using a particular data element.
- Detailed reports showing all characteristics of data elements, records, data flows, processes, or any other selected item stored in the data dictionary.

5.9 Process Description Tools in Modular Design

A **process description** documents the details of a functional primitive and represents a specific set of processing steps and business logic. When a functional primitive is analyzed, the processing steps are broken down into smaller units in a modular design process. Using a set of process description tools, a model is created that is accurate, complete, and concise. Typical process description tools include structured English, decision tables, and decision trees.

5.9.1 Process Descriptions in Object-Oriented Development

This chapter deals with structured analysis, but process descriptions can also be used in object-oriented development, as described later in the book. Recall that O-O analysis combines data and the processes that act on the data into things called objects. Similar objects can be grouped into classes, and O-O processes are called methods. Although O-O programmers use different terminology, they create the same kind of modular coding structures, except that the processes (methods) are stored inside the objects rather than as separate components.

5.9.2 Modular Design

Modular design is based on combinations of three **logical structures**, sometimes called **control structures**, which serve as building blocks for the process. Each logical structure must have a single entry and exit point. The three structures are called sequence, selection, and iteration. A rectangle represents a step or process, a diamond shape represents a condition or decision, and the logic follows the lines in the direction indicated by the arrows.

1. **Sequence**. The completion of steps in sequential order, one after another, as shown in Figure 5-20. One or more steps might represent a subprocess containing additional logical structures.
2. **Selection**. The completion of one of two or more process steps based on the results of a test or condition. In the example shown in Figure 5-21, the system tests the input; if the hours are greater than 40, it performs the CALCULATE OVERTIME PAY process.
3. **Iteration**. The completion of a process step is repeated until a specific condition changes, as shown in Figure 5-22. An example of iteration is a process that prints paychecks until the payroll file ends. Iteration also is called **looping**.

Figure 5-20 Sequence structure.

Figure 5-21 Selection structure.

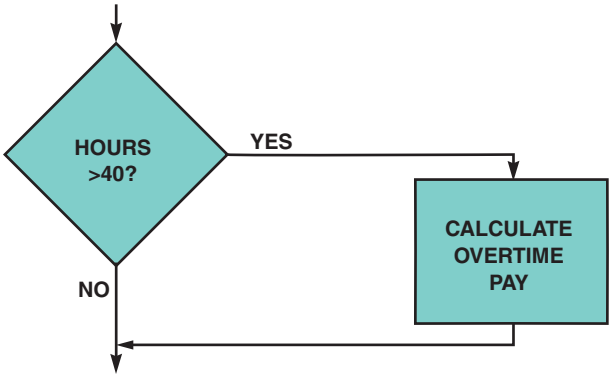
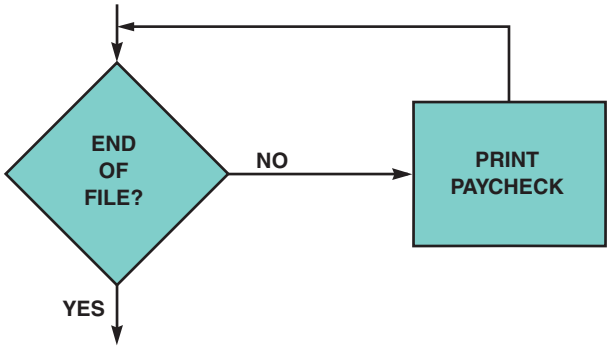


Figure 5-22 Iteration structure.



Sequence, selection, and iteration structures can be combined in various ways to describe processing logic.

5.9.3 Structured English

Structured English is a subset of standard English that describes logical processes clearly and accurately. When using structured English, be mindful of the following guidelines:

- Use only the three building blocks of sequence, selection, and iteration.
- Use indentation for readability.
- Use a limited vocabulary, including standard terms in the data dictionary and specific words that describe the processing rules.

An example of structured English appears in Figure 5-23, which shows the VERIFY ORDER process that was illustrated earlier. Notice that the structured English version documents the logic that will be coded into the system. Structured English can help make process descriptions accurate and understandable to users and system developers.

Figure 5-23 The VERIFY ORDER process description includes logical rules and a structured English version of the policy. Notice the alignment and indentation of the logic statements.
Source: Author-generated content (MS Word).

Label:	VERIFY ORDER
Entry Type:	Process
Description:	Accept or reject customer order based on credit status and product availability.
Process #:	
Process Description:	Input data flows: ORDER, CREDIT STATUS, PRODUCT DETAIL Output data flows: ACCEPTED ORDER, REJECTED ORDER For each ORDER If CREDIT STATUS = Y and PRODUCT DETAIL = OK Output ACCEPTED ORDER Else Output REJECTED ORDER } Structured English Statements
Notes:	

Structured English might look familiar to programming students because it resembles **pseudocode** used in program design. Although the techniques are similar, the primary purpose of structured English is to describe the underlying business logic. At the same time, programmers, who are concerned with coding, mainly use pseudocode as a shorthand notation for the actual code.

Following structured English rules ensures that process descriptions are understandable to users who must confirm the process is correct and to other analysts and programmers who must design the information system from the descriptions.

5.9.4 Decision Tables

A **decision table** is a logical structure that shows every combination of conditions and outcomes. Analysts often use decision tables to describe a process and ensure they have considered all possible situations. Decision tables can be created using Microsoft PowerPoint, Word, or Excel.

Tables with one condition. If a process has a single condition, there are only two possibilities—*yes* or *no*. Either the condition is present or not, so there are only two rules. For example, to trigger an overtime calculation, the process condition might be: *Are the hours greater than 40?* If so, the calculation is made. Otherwise, it is not.

Tables with two conditions. Suppose there is a need to create a decision table based on the VERIFY ORDER business process shown in Table 5-1. When documenting a process, it is important to list every possibility. In this example, the process description contains two conditions: product stock status and customer credit status. If *both* conditions are met, the order is accepted. Otherwise, the order is rejected.

Table 5-1 The VERIFY ORDER business process has two conditions. For an order to be accepted, the product must be in stock and the customer must have an acceptable credit status.

VERIFY ORDER Business Process with Two Conditions
• An order will be accepted only if the product is in stock and the customer's credit status is OK.
• All other orders will be rejected.

After all the conditions and outcomes have been identified, the next step is to create a decision table similar to the one shown in Figure 5-24. Notice that each condition has two possible values, so the number of rules doubles when another condition is added. For example, one condition creates two rules, two conditions create four rules, three conditions create eight rules, and so on. Four possibilities exist in the two-condition example in Figure 5-24, but Rule 1 is the *only* combination that will accept an order.

1. Place the name of the process in a heading at the top left.
2. Enter the conditions under the heading, with one condition per line, to represent the customer status and availability of products.
3. Enter all potential combinations of Y/N for the conditions. Each column represents a numbered possibility called a rule.
4. Place an X in the action entries area for each rule to indicate whether to accept or reject the order.

Figure 5-24 Example of a simple decision table showing the processing logic of the VERIFY ORDER process. To create the table, follow the four steps shown.

VERIFY ORDER Process				
	1	2	3	4
Credit status is OK	Y	Y	N	N
Product is in stock	Y	N	Y	N
Accept order	X			
Reject order		X	X	X

1 → VERIFY ORDER Process
2 → Credit status is OK; Product is in stock
3 → Y/N combinations
4 → Accept order; Reject order

Tables with three conditions. Suppose the company now decides that the credit manager can waive the customer credit requirement, as shown in Table 5-2. That creates a third condition, so there will be eight possible rules. The new decision table might resemble the one shown in Table 5-3.

Table 5-2 A third condition has been added to the VERIFY ORDER business process. For an order to be accepted, the product must be in stock and the customer must have an acceptable credit status. However, the credit manager now has the authority to waive the credit status requirements.

VERIFY ORDER Business Process with Three Conditions
• An order will be accepted only if the product is in stock and the customer's credit status is OK.
• The credit manager can waive the credit status requirement.
• All other orders will be rejected.

Table 5-3 This decision table is based on the VERIFY ORDER conditions shown in Table 5-2. With three conditions, there are eight possible combinations or rules.

VERIFY ORDER Process with Credit Waiver (initial version)								
	1	2	3	4	5	6	7	8
Credit status is OK	Y	Y	Y	Y	N	N	N	N
Product is in stock	Y	Y	N	N	Y	Y	N	N
Waiver from credit manager	Y	N	Y	N	Y	N	Y	N
Accept order	X	X			X			
Reject order			X	X		X	X	X

First, the Y-N patterns must be filled in, as shown in Table 5-3. Using patterns like these is the best way to ensure all combinations appear. The first condition uses a pattern of Y-Y-Y-Y followed by N-N-N-N; the second condition uses a repeating Y-Y-N-N pattern; and the pattern in the third condition is a series of Y-Ns.

The next step is crucial, regardless of the number of conditions. Each numbered column, or rule, represents a different set of conditions. The logic must be carefully analyzed, and the outcome for each rule must be shown. Before going further, study Table 5-3 to be sure the logical outcome for each of the eight rules is understood.

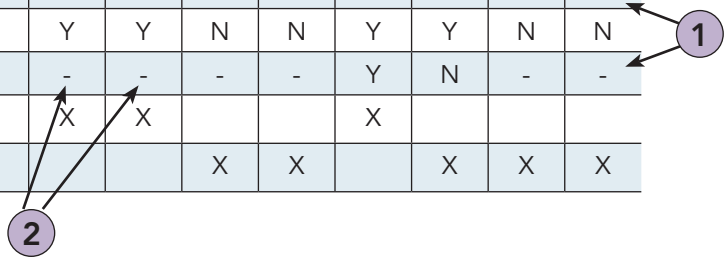
When all the outcomes have been determined, the next step is simplifying the table. Some rules might be duplicates, redundant, or unrealistic in a multicondition table. To simplify the table, follow these steps:

1. Study each combination of conditions and outcomes. When there are rules with three conditions, only one or two of them may control the outcome, and the other conditions do not matter.
2. If conditions do not affect the outcome, mark them with dashes (-), as shown in the first table in Figure 5-25.
3. Now combine and renumber the rules, as shown in the second table in Figure 5-25.

After following these steps, Rules 1 and 2 can be combined because credit status is okay in both rules, so the waiver would not matter. Rules 3, 4, 7, and 8 also can be combined because the product is not in stock, so other conditions do not matter. The result is that instead of eight possibilities, only four logical rules control the Verify Order process.

Figure 5-25 In the first decision table, dashes have been added to indicate that a condition is not relevant. In the second version of the decision table, rules are combined following the steps below. Notice that in the final version, only four rules remain. These rules document the logic and will be transformed into program code when the system is developed.

VERIFY ORDER Process with Credit Waiver (with rules marked for combination)								
	1	2	3	4	5	6	7	8
Credit status is OK	Y	Y	-	-	N	N	-	-
Product is in stock	Y	Y	N	N	Y	Y	N	N
Waiver from credit manager	-	-	-	-	Y	N	-	-
Accept order	X	X			X			
Reject order			X	X		X	X	X



VERIFY ORDER Process with Credit Waiver (after rule combination and simplification)				
	1 (Combines previous 1, 2)	2 (Previous 5)	3 (Previous 6)	4 (Combines previous 3, 4, 7, 8)
Credit status is OK	Y	N	N	-
Product is in stock	Y	Y	Y	N
Waiver from credit manager	-	Y	N	-
Accept order	X	X		
Reject order			X	X

1. Because the product is not in stock, the other conditions do not matter.
2. Because the other conditions are met, the waiver does not matter.

Multiple outcomes. Besides multiple conditions, decision tables can have more than two possible outcomes. For example, the sales promotion policy shown in Table 5-4 includes three conditions: Was the customer a preferred customer, did the customer order $1,000 or more, and did the customer use the company charge card? Based on these conditions, four possible actions can occur, as shown in the decision table in Table 5-5.

Table 5-4 A sales promotion policy with three conditions. Notice that the first statement contains two *separate* conditions: one for the 5% discount and another for the additional discount.

SALES PROMOTION POLICY – Holiday Season
• Preferred customers who order $1,000 or more are entitled to a 5% discount and an additional 5% discount if they use our charge card.
• Preferred customers who do not order $1,000 or more will receive a $25 bonus coupon.
• All other customers will receive a $5 bonus coupon.

Table 5-5 This decision table is based on the sales promotion policy in Table 5-4. This is the initial version of the table before simplification.

Sales Promotion Policy (initial version)								
	1	2	3	4	5	6	7	8
Preferred customer	Y	Y	Y	Y	N	N	N	N
Ordered $1,000 or more	Y	Y	N	N	Y	Y	N	N
Used our charge card	Y	N	Y	N	Y	N	Y	N
5% discount	X	X						
Additional 5% discount	X							
$25 bonus coupon			X	X				
$5 bonus coupon					X	X	X	X

As explained in the preceding section, most tables can be simplified, and this one is no exception. After studying the conditions and outcomes, the following becomes apparent:

- In Rule 1, all three conditions are met, so *both* 5% discounts apply.
- In Rule 2, a preferred customer orders $1,000 or more but does not use the charge card, so only *one* 5% discount applies.
- Rules 3 and 4 can be combined into a single rule. Why? If preferred customers do not order $1,000 or more, it does not matter whether they use the charge card—either way, they earn only a $25 bonus coupon. Therefore, Rules 3 and 4 are a single rule.
- Rules 5, 6, 7, and 8 can also be combined into a single rule: If the person is not a preferred customer, they can only receive a $5 bonus coupon, and the other conditions do not matter. A dash is inserted if a condition is irrelevant, as shown in Table 5-6.

If dashes are added for rules that are not relevant, the table should resemble the one shown in Table 5-6. When the results are combined and simplified, only four rules remain: Rule 1, Rule 2, Rule 3 (a combination of initial Rules 3 and 4), and Rule 4 (a combination of initial Rules 5, 6, 7, and 8).

Table 5-6 In this version of the decision table, dashes have been added to indicate that a condition is irrelevant. At this point, several rules can be combined.

Sales Promotion Policy (final version)								
	1	2	3	4	5	6	7	8
Preferred customer	Y	Y	Y	Y	N	N	N	N
Ordered $1,000 or more	Y	Y	N	N	-	-	-	-
Used our charge card	Y	N	-	-	-	-	-	-
5% discount	X	X						
Additional 5% discount	X							
$25 bonus coupon			X	X				
$5 bonus coupon					X	X	X	X

Decision tables are often the best way to describe complex conditions. Many analysts use decision tables because they are easy to construct and understand, and programmers find it easy to work from a decision table when developing code.

Case in Point 5.2

Rock Solid Outfitters (Part 1)

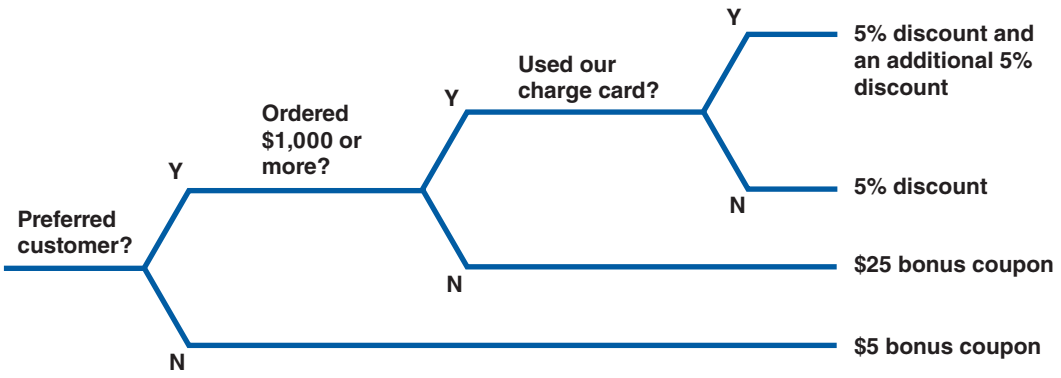
The marketing director at Rock Solid Outfitters, a medium-sized outdoor climbing and camping gear supplier, has asked the IT manager to develop a special web-based promotion. Rock Solid will provide free shipping for any customer who either completes an online survey form or signs up for the Rock Solid online newsletter. Additionally, if a customer completes the survey and signs up for the newsletter, Rock Solid will provide a $10 merchandise credit for orders of $100 or more. The IT manager has asked you to develop a decision table that will reflect the promotional rules that a programmer will use. The manager wants you to show all possibilities and then simplify the results to eliminate any combinations that would be unrealistic or redundant. How will you proceed?

5.9.5 Decision Trees

A **decision tree** is a graphical representation of a decision table's conditions, actions, and rules. Decision trees show the logic structure in a horizontal form that resembles a tree with the roots at the left and the branches to the right. Like flowcharts, decision trees are useful ways to present the system to management. Decision trees and decision tables provide the same results but in different forms. In many situations, a graphic is the most effective means of communication.

Figure 5-26 is based on the sales promotion policy shown in Table 5-4. A decision tree is read from left to right, with the conditions along the various branches and the actions at the far right. Because the example has two conditions with four resulting sets of actions, the example has four terminating branches at the right side of the tree.

Figure 5-26 This decision tree example is based on the same sales promotion policy shown in the decision tables in Tables 5-5 and 5-6. Like a decision table, a decision tree shows all combinations of conditions and outcomes. The main difference is the graphical format, which some viewers may find easier to interpret.



Whether to use a decision table or a decision tree often is a matter of personal preference. A decision table might be a better way to handle complex combinations of conditions. On the other hand, a decision tree is an effective way to describe a relatively simple process.

Case in Point 5.3

Rock Solid Outfitters (Part 2)

The IT manager at Rock Solid Outfitters thinks you did well on the assigned decision table task. Now they want you to use the same data to develop a decision tree showing all the possibilities for the web-based promotion described in Part 1 of the case. They also want you to discuss the pros and cons of decision tables versus decision trees. How shall you proceed this time?

Summary

Structured analysis tools can be used to develop a logical model during one systems analysis phase and a physical model during the systems design phase. Many analysts use a four-model approach, which involves a physical model of the current system, a logical model of the current system, a logical model of the new system, and a physical model of the new system.

During data and process modeling, a systems analyst develops graphical models to show how the system transforms data into useful information. The end product of data and process modeling is a logical model that will support business operations and meet user needs. Data and process modeling involves three main tools: data flow diagrams, a data dictionary, and process descriptions.

Data flow diagrams (DFDs) graphically show the movement and transformation of data in the information system. DFDs use four symbols: The process symbol transforms data, the data flow symbol shows data movement, the data store symbol shows data at rest, and the external entity symbol represents someone or something connected to the information system. Various rules and techniques are used to name, number, arrange, and annotate the set of DFDs to make them consistent and understandable.

A set of DFDs is like a pyramid with a context diagram at the top. The context diagram represents the information system's scope and external connections, not internal workings. Diagram 0 displays the information system's major processes, data stores, and data flows and is the exploded version of the context diagram's process symbol, representing the entire information system. Lower-level DFDs show additional detail of the information system through the leveling technique of numbering and partitioning. Leveling continues until the functional primitive processes are reached; they are not decomposed further and are documented with process descriptions. All diagrams must be balanced to ensure their consistency and accuracy.

The data dictionary is the central documentation tool for structured analysis. All data elements, data flows, data stores, processes, entities, and records are documented in the data dictionary. Consolidating documentation in one location allows the analyst to verify the information system's accuracy and consistency more easily and generate useful reports.

Each functional primitive process is documented using structured English, decision tables, and decision trees. Structured English uses a subset of standard English that defines each process with combinations of the basic building blocks of sequence, selection, and iteration. Using decision tables or decision trees can also document the logic.

Key Terms

alias
balancing
black box
black hole
business logic
child diagram
context diagram
control structure
data dictionary
data element
data flow
data item
data repository
data store
data structure
decision table
decision tree
decomposing
diagram 0
diverging data flow
domain
entity
exploding
field
four-model approach
functional primitive
Gane and Sarson symbols
gray hole
iteration
leveling
logical model
logical structure
looping
modular design
parent diagram
partitioning
physical model
process 0
process description
pseudocode
record
selection
sequence
sink
source
spontaneous generation
structured English
terminator
validity rules
Yourdon symbols

Review Questions

1. What is the relationship between logical and physical models?
2. What is the function of a data flow diagram in the SDLC?
3. Draw examples of the four basic data flow diagram symbols.
4. What are the six guidelines to follow when drawing data flow diagrams?
5. What is the difference between a context diagram and diagram 0?
6. Which symbol is *not* used in a context diagram?
7. How would you level a data flow diagram?
8. How would you balance a data flow diagram?
9. What is a data element?
10. What is the purpose of a decision table?

Discussion Topics

1. How would you convince management that following a four-model approach is wise?
2. When might it be appropriate to violate the "no crossed lines" guideline in data flow diagrams?
3. What is the relationship between system requirements and context diagrams?
4. How might tools be used to document the design of a data dictionary?
5. Some systems analysts find it better to start with a decision table and then construct a decision tree. Others believe it is easier to use the reverse order. Which do you prefer? Why?

Hands-On Projects

1. The data flow symbols shown in Figure 5-1 were designed by Ed Yourdon, a well-known IT author, lecturer, and consultant. Many IT professionals consider him among the most influential people in the software field. Learn more about Yourdon by visiting online resources and writing a brief review of his work.
2. Explore three tools to draw the four basic data flow diagram symbols and describe what you liked and disliked about each tool.
3. Draw a context diagram and a diagram 0 DFD representing a typical library's information system.
4. Create a decision table with three conditions. You can make one up or use a scenario from everyday life. Either way, be sure to show all possible outcomes.
5. Explore using structured English to describe processes in fields other than systems analysis.

Ethical Issues

This is your first week in your new job at Safety Zone, a leading producer of IT modeling software. Your prior experience with a smaller competitor gave you an edge in landing the job, and you are excited about joining a larger company in the same field.

So far, all is going well, and you are getting used to the new routine. However, you are concerned about one issue. In your initial meeting, the IT manager seemed very interested in the details of your prior position, and some of the questions made you a little uncomfortable. The manager did not actually ask you to reveal any proprietary information but made it clear that Safety Zone likes to know as much as possible about its competitors.

Thinking about it, you try to draw a line between information that is okay to discuss and topics such as software specifics or strategy that should be considered private. This is your first time in a situation like this. How will you handle it?

Chapter

6

Object Modeling

Learning Objectives

When you finish this chapter, you should be able to:

1. Demonstrate how object-oriented analysis can describe an information system.
2. Explain what an object represents in an information system.
3. Explain object attributes.
4. Explain object methods.
5. Explain object messages.
6. Explain classes.
7. Explain relationships among objects and classes.
8. Describe an object-oriented system using UML.
9. Explain how tools can support object modeling.

Chapter 6 is the third of three chapters in the systems analysis phase of the SDLC. This chapter discusses object modeling techniques analysts use to create a logical model of the information system. In addition to structured analysis, object-oriented analysis is another way to represent and design an information system.

The chapter includes three "Case in Point" discussion questions to help contextualize the concepts described in the text. The scenario in the "Ethical Issues" section considers a situation where an employee wants to skip a training course and take the certification exam. To skip the course, the employee requests a copy of the training materials from someone else who took it.

6.1 Object-Oriented Analysis

As stated previously, the most popular systems development options are structured analysis, **object-oriented (O-O) analysis**, and agile methods. Table 1-1, shown earlier, highlights the three alternatives and describes some pros and cons for each approach.

O-O analysis describes an information system by identifying things called objects. An **object** represents a real person, place, event, or transaction. For example, when a patient makes an appointment to see a doctor, the patient is an object, the doctor is an object, and the appointment itself is an object. The end product of O-O analysis is an **object model** representing the information system in terms of objects and O-O concepts.

O-O analysis has gained substantial popularity in systems analysis and design for several reasons, each contributing to this approach's effectiveness, efficiency, and adaptability in developing complex software systems. Some of the key reasons for its popularity include:

Real-world representation. O-O analysis is popular for representing real-world entities, behaviors, and interactions. By defining objects that represent real-world entities, O-O analysis allows analysts to intuitively model and understand complex systems, making the conceptualization and design process more natural and straightforward.

Modularity and encapsulation. O-O analysis promotes the development of modular and encapsulated components, where each object is a self-contained unit with its attributes and behaviors. This modularity facilitates easier maintenance, modification, and extension of systems, as changes to one object do not necessarily impact others, reducing the risk of unintended consequences when models and related artifacts are modified.

Reusability. Objects created through O-O analysis are often reusable across different projects. This reusability reduces development time and effort and promotes consistency and reliability across different systems, contributing to more efficient and cohesive software development.

Integration with O-O programming languages. O-O analysis integrates seamlessly with O-O programming languages like Java, C++, and Python. This integration ensures coherence among systems analysis, design, and implementation phases, allowing developers to transition smoothly from conceptual models to actual code, thereby reducing the likelihood of discrepancies between design and implementation.

Scalability. The structure provided by O-O analysis supports the development of scalable systems. The modularity and reusability inherent to O-O approaches allow systems to grow and evolve efficiently, accommodating new requirements and functionality with less effort and disruption.

Enhanced communication. O-O analysis utilizes the Unified Modeling Language (UML), a popular method of visualizing and documenting information systems. As discussed earlier in the book, UML provides a standardized way of representing system components and their interactions. These visual models are effective communication tools, enabling clearer understanding and discussion among developers, analysts, and nontechnical stakeholders. In this chapter, the UML is used to develop object models. The first step is understanding basic O-O terms, including objects, attributes, methods, messages, and classes. These terms are explained in the following sections.

Support for design patterns. O-O analysis provides a foundation for implementing **design patterns**, which are proven solutions to common design problems. The use of design patterns promotes best practices, improves code reliability and maintainability, and reduces the occurrence of bugs and errors.

Flexibility and maintenance. The O-O approach allows for more straightforward modifications and enhancements in the system. If a particular object needs to be altered, it can be done without affecting other parts of the system, making maintenance and updating of systems more manageable.

Enhanced security. O-O analysis offers an added layer of security through encapsulation, where the internal state of every object is protected from outside access. By restricting access to the internal components and exposing only what is necessary, encapsulation provides a precise mechanism for controlling interactions with objects, preventing unintended interference and misuse. This enhanced security is crucial in developing robust and secure applications, particularly in environments where data sensitivity and integrity are paramount.

Efficient problem solving. O-O analysis aids in efficient problem solving by allowing analysts and developers to focus on high-level abstraction and decomposition of the system into manageable, logical components (objects). This approach facilitates a clearer understanding of the system's complexities and a more organized, systematic way of addressing each component's functionalities and interactions. Enabling analysts to tackle problems in segments rather than dealing with the system as a whole fosters more effective and accurate solutions, leading to the development of more reliable and high-quality systems.

Case in Point 6.1

TravelBiz

TravelBiz is a nationwide travel agency that specializes in business travel. It has decided to expand into the vacation travel market by launching a new business division called TravelFun. The information technology (IT) director assigned two systems analysts to create a flexible and efficient information system for the new division. One analyst wants to use traditional analysis and modeling techniques for the project, while the other wants to use an O-O methodology. Which approach would you suggest and why?

6.2 Objects

In O-O analysis, objects are crucial foundational elements representing the instances of classes, and they are at the core of understanding and defining the system's behavior and structure. In systems analysis and design, objects are the functional units that interact and collaborate to realize the system's intended functionalities.

An object represents a person, place, event, or transaction that is significant to the information system. In an earlier chapter, data flow diagrams (DFDs) were created that treated data and processes separately. An object, however, includes data *and* the processes that affect that data. For example, a customer object has a name, an address, an account number, and a current balance. Customer objects also can perform specific tasks, such as placing an order, paying a bill, and changing their address.

The accurate representation and interaction of objects are essential for developing robust, reliable systems that reflect real-world entities and their interactions. Analysts can create secure, maintainable, and adaptable systems capable of evolving to meet changing needs and requirements through proper object definition, encapsulation, and interaction.

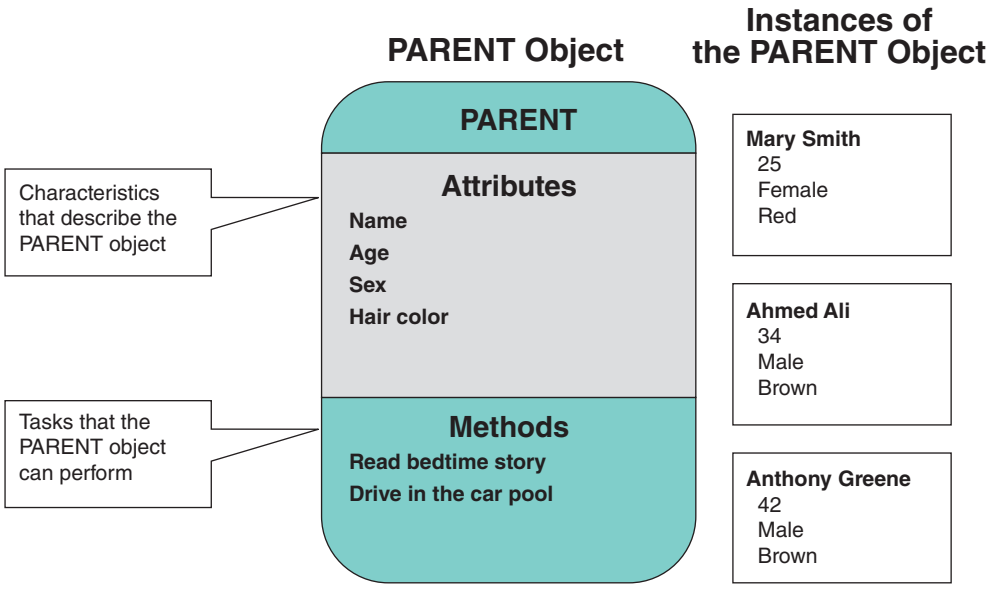
6.2.1 Examples

Consider a simplistic example of how the UML might describe a family with parents and children. UML represents an object as a rounded rectangle with the object name at the top, followed by the object's attributes and methods.

Figure 6-1 shows a PARENT object with attributes such as name, age, sex, and hair color. If there are two parents, there are two instances of the PARENT object. The PARENT object can perform methods like reading a bedtime story, driving the carpool van, or preparing a school lunch. When a PARENT object receives a message, it performs an action, or method.

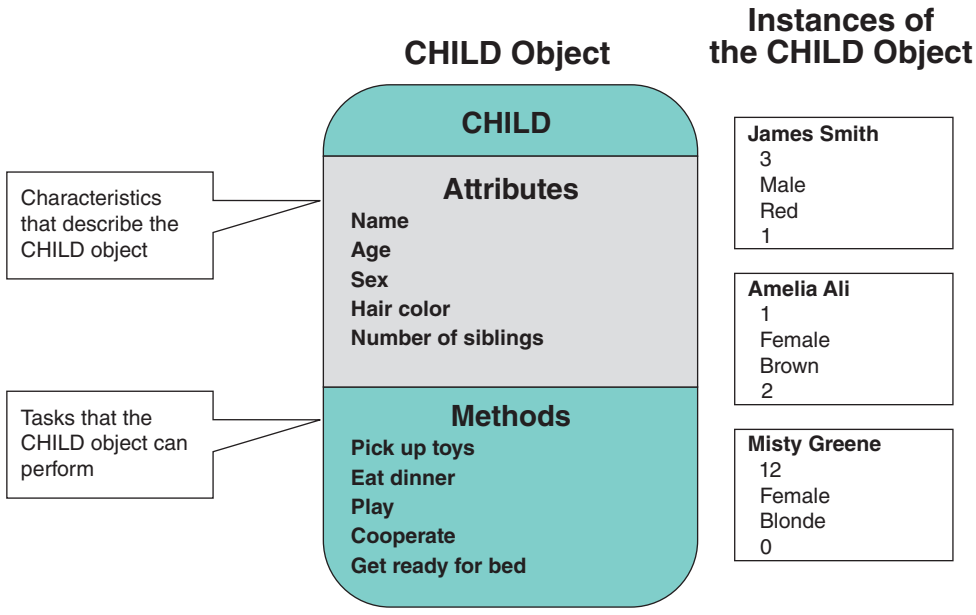
For example, the message GOOD NIGHT from a child might tell the PARENT object to read a bedtime story, while DRIVE from another parent signals that it is the PARENT object's turn to drive in the carpool.

Figure 6-1 The PARENT object has four attributes and two methods. Mary Smith, Ahmed Ali, and Anthony Greene are instances of the PARENT object.



Continuing with the family example, the CHILD object in Figure 6-2 possesses the same attributes as the PARENT object and an additional attribute that shows the number of siblings. A CHILD object performs certain methods, such as picking up toys, eating dinner, playing, cooperating, and getting ready for bed. A PARENT can send specific messages that the CHILD object will understand and signal it to perform appropriate tasks. For example, the DINNER'S READY message tells a CHILD object to come to the table, while the SHARE WITH YOUR SIBLING message tells a CHILD object to cooperate with other CHILD objects.

Figure 6-2 The CHILD object has five attributes and five methods. James Smith, Amelia Ali, and Misty Greene are instances of the CHILD object.



6.3 Attributes

In O-O analysis, objects are the fundamental building blocks used to represent real-world entities within a system. An object has certain **attributes**, which are characteristics that describe the object. If objects are like nouns, attributes are like adjectives that describe the characteristics of an object.

For example, in a system modeling a library, a BOOK object might have attributes such as title, author, ISBN, publication date, and genre. These attributes hold the data or information pertinent to the object and differentiate one object from another within the same class. Some objects might have a few attributes; others might have dozens. In an O-O system, objects can inherit, or acquire, specific attributes from other objects.

Attributes are usually represented as variables within the class definition in O-O programming languages, and they can have different data types, such as string, integer, or date, depending on the nature of the attribute. For instance, a birthdate attribute would typically be of the date data type, while a name attribute would be of the string data type.

Objects can have a specific attribute called a **state**. The state of an object is an adjective that describes the object's current status. For example, depending on the state, a student can be a future, present, or past student. Similarly, a bank account can be active, inactive, closed, or frozen.

In O-O analysis, identifying the correct attributes for each object is crucial, as it determines how accurately the real-world entity is represented within the system. By ensuring that each object has the appropriate attributes, analysts can create more realistic and functional models of the entities within the system.

6.4 Methods

In O-O analysis, a system is conceived as a collection of interacting objects representing entities from the system's real-world domain. In addition to attributes, an object has **methods**, which are tasks or functions that the object performs when it receives a **message**, or command, to do so.

Methods perform operations, manipulate object attributes, and implement the object's behavior within a system. They define how the object can interact with other objects and the outside world. In systems analysis and design, understanding object methods is essential in defining the interactions and functionalities within the system.

Some of the significant characteristics of object methods include:

Interaction and collaboration. Object methods facilitate interaction and collaboration between objects within a system, enabling the realization of complex behaviors and processes by coordinating simpler, well-defined operations.

Encapsulation and security. Methods allow for encapsulation, a core principle of O-O design, where the internal state of an object is protected and access to it is controlled through well-defined interfaces, enhancing security and integrity.

Flexibility and maintainability. Defining behaviors as methods allows for more flexible and maintainable models. If an object needs to change its behavior, only the methods related to that behavior must be modified, reducing the impact on the rest of the system.

Clarity and understandability. Methods provide a clear and structured way to represent the functionalities of objects, making it easier for developers to understand and work with the system.

Enhanced modularity and reusability. Object methods contribute to an O-O design's enhanced modularity and reusability. Objects can be designed as modular components by encapsulating specific behaviors within methods, where each method has a single responsibility or function. This modularity allows objects and their methods to be reused across different system parts or even in other projects, promoting reusability and reducing development time and effort.

6.4.1 Example

Consider a server who prepares fries in a fast-food restaurant. A systems analyst might describe the operation as MORE FRIES, as shown in Figure 6-3. The MORE FRIES method includes the steps required to heat the oil, fill the fry basket with frozen potato strips, lower it into the hot oil, check for readiness, remove the basket when ready, drain the oil, pour the fries into a warming tray, and add salt.

Figure 6-3 The MORE FRIES method requires the server to perform seven specific steps.

Method:	Steps:
MORE FRIES	1. Heat oil 2. Fill fry basket with frozen potato strips 3. Lower basket into hot oil 4. Check for readiness 5. When ready, raise basket and let drain 6. Pour fries into warming tray 7. Add salt

6.5 Messages

Messages are how objects communicate and interact with each other within a system, serving as the conduits for invoking behaviors and exchanging information. In systems analysis and design, a message is a call from one object to another, requesting a certain operation to be performed or aiming to access information. When an object receives a message, it responds by executing a method associated with the message, effectively acting on the request or providing the needed information.

Polymorphism allows objects of different classes to respond differently to the same message (method call). It enables objects to be treated as instances of their parent class, leading to simpler code and fewer errors. It enables one interface to be used for different data types, allowing for the implementation of elegant software designs that can handle new, unforeseen circumstances and data types. In practical terms, polymorphism allows for overriding and overloading methods, providing flexibility and extensibility in designing robust and versatile software applications.

An object can be viewed as a "black box" because a message to the object triggers changes within it without specifying how the changes must be carried out. A gasoline pump is an example of a black box. When economy-grade gas is selected at a pump, it is unnecessary to consider how the pump determines the correct price and picks the proper fuel.

The black box concept is an example of **encapsulation**, meaning all data and methods are self-contained. A black box does not want or need outside interference. By limiting access to internal processes, an object prevents its internal code from being altered by another object or process. Encapsulation allows objects to be used as modular components anywhere in the system because objects send and receive messages but do not alter the internal methods of other objects.

Object messages are pivotal in O-O analysis and design, acting as the mechanism through which objects invoke behaviors and share information. They play a critical role in defining and designing the interactions and behaviors within a system, contributing to its modularity, clarity, and reliability. Some of the significant features of object messages include:

Defining interactions. In systems analysis, understanding and defining the messages that objects can send and receive is crucial for outlining the interactions within the system. It helps create clear and concise models of how different components will cooperate to fulfill the system's requirements and achieve desired outcomes.

Designing behaviors. In the design phase, messages help detail objects' behaviors and responsibilities, aiding developers in structuring the code to ensure that objects interact seamlessly and reliably. Properly designed messages contribute to the robustness and reliability of the system, preventing errors and inconsistencies.

Enhancing modularity and decoupling. Messages contribute to the modularity and decoupling of objects within a system. Decoupling refers to reducing the dependencies between different system parts to improve modularity and facilitate manageability, changeability, and scalability. Because objects interact through well-defined messages, changes in one object's internal structure or methods do not necessarily affect others as long as the message interfaces remain consistent.

Enabling parallel development. Because messages act as defined interfaces between objects, other teams can design and develop different parts of a system in parallel if the agreed-upon messages are adhered to. This parallel development can significantly reduce development time and allow more flexible project management.

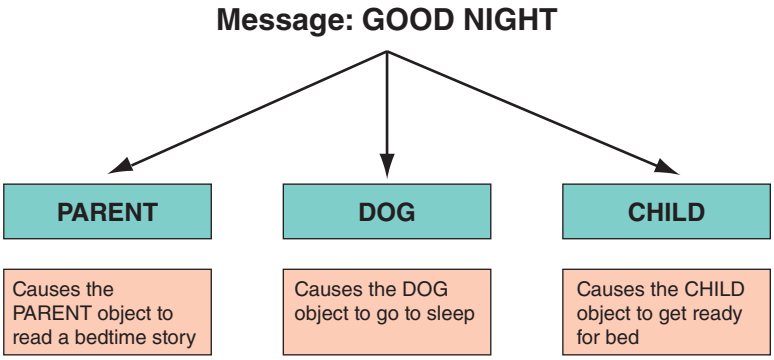
Enhancing clarity and understandability. By elucidating the interactions between objects, messages aid in creating more understandable and manageable system representations. This clarity is crucial for analysts and stakeholders to comprehend the system's structure and behavior, facilitating effective communication, troubleshooting, and enhancements.

6.5.1 Examples

A message is a command that tells an object to perform a particular method. For example, the message PICK UP TOYS directs the CHILD class to perform all the necessary steps to pick up the toys. The CHILD class understands the message and executes the method.

In Figure 6-4, the message GOOD NIGHT signals the PARENT object to read a bedtime story, but the same message to the CHILD object signals it to get ready for bed. The same message would tell the dog to sleep if the family also had a DOG object.

Figure 6-4 In an example of polymorphism, the message GOOD NIGHT produces different results depending on which object receives it.



O-O designs are typically implemented with O-O programming languages. A significant advantage of O-O designs is that systems analysts can save time and avoid errors by using modular objects. Programmers can translate the designs into code, working with reusable program modules that have been tested and verified.

Figure 6-5 In a school information system, an INSTRUCTOR object sends an ENTER GRADE message to an instance of the STUDENT RECORD class.

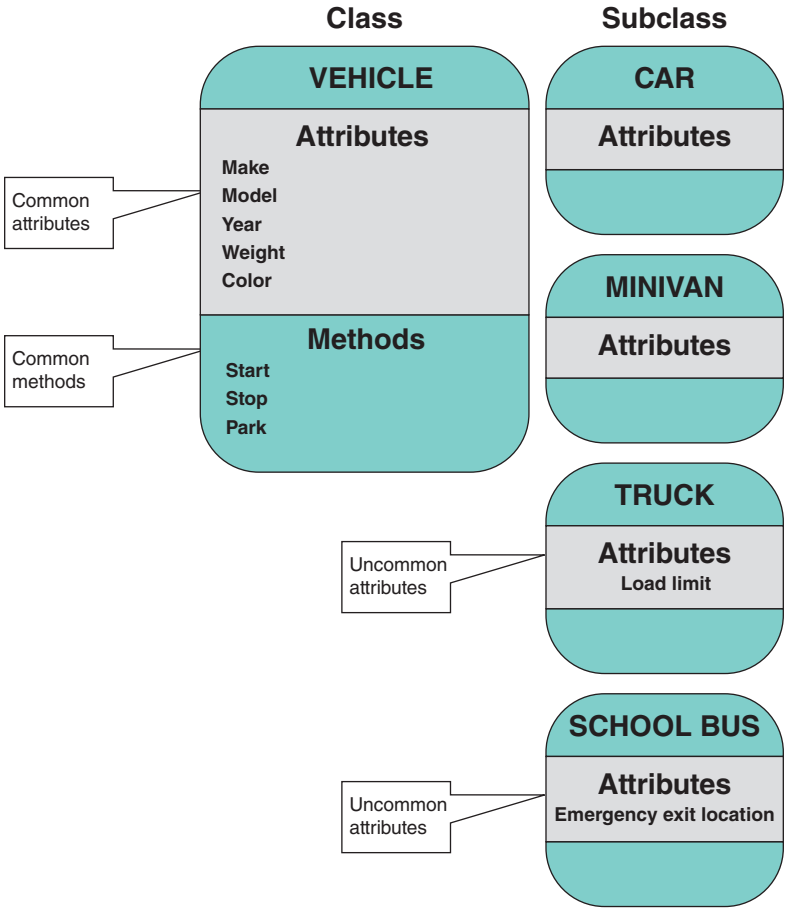
For example, in Figure 6-5, an INSTRUCTOR object sends an ENTER GRADE message to an instance of the STUDENT RECORD class. Notice that the INSTRUCTOR object and STUDENT RECORD class could be reused, with minor modifications, in other school information systems where many of the attributes and methods would be similar.

6.6 Classes

An object belongs to a group or category called a **class**. Classes are foundational elements, providing a blueprint or template from which objects are created. Classes serve as the structural basis defining the properties (attributes) and actions (methods) that instances of the class (objects) will possess and can perform. Classes play a pivotal role in the organization, structure, and functionality of O-O systems.

For example, a Honda Civic belongs to a class called CAR. An **instance** is a specific member of a class. A Toyota Camry is another instance of the CAR class. At an auto dealership, numerous cases of the CAR class may be observed: the TRUCK class, the MINIVAN class, and the SPORT UTILITY VEHICLE class, among others.

Figure 6-6 The VEHICLE class includes common attributes and methods. CAR, TRUCK, MINIVAN, and SCHOOL BUS are instances of the VEHICLE class.



Objects within a class can be grouped into **subclasses**, which are more specific categories within a class. For example, TRUCK objects represent a subclass within the VEHICLE class; other subclasses are called CAR, MINIVAN, and SCHOOL BUS, as shown in Figure 6-6. Notice that all four subclasses share common traits of the VEHICLE class, such as make, model, year, weight, and color. Each subclass also can possess uncommon characteristics, such as a load limit for the TRUCK or an emergency exit location for the SCHOOL BUS.

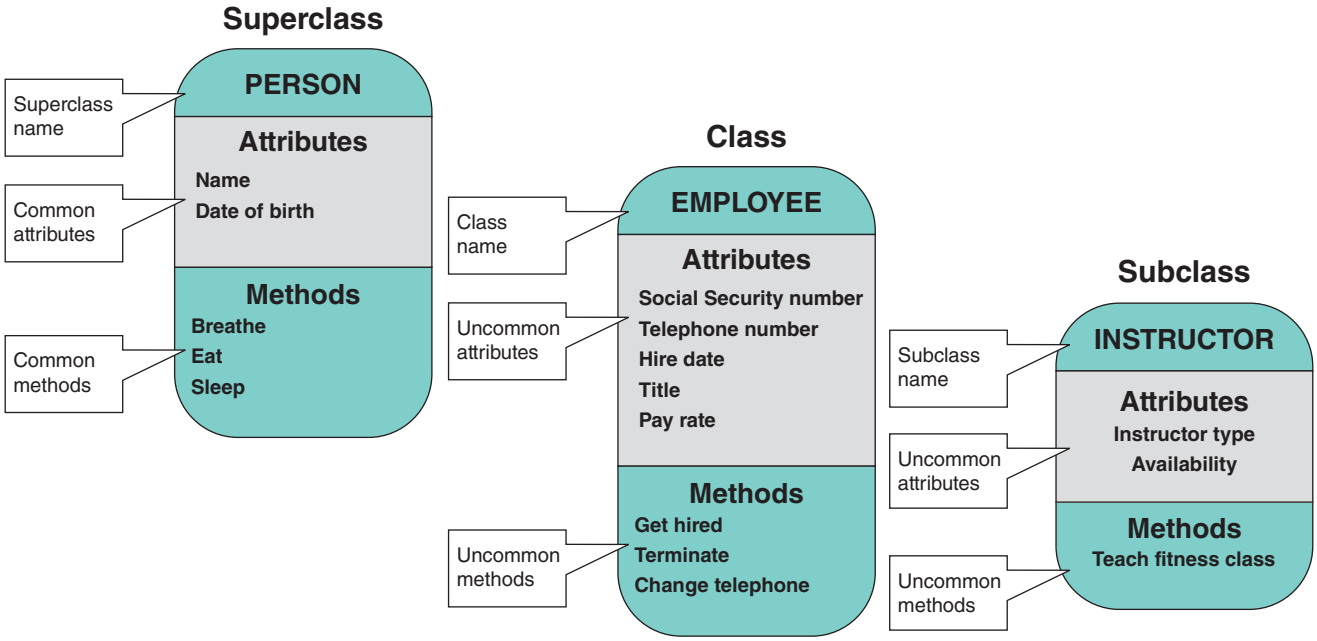
A class can belong to a more general category called a **superclass**. For example, a NOVEL class belongs to a superclass called BOOK because all novels are books in this example. The NOVEL class can have subclasses called HARDCOVER, PAPERBACK, and DIGITAL.

Figure 6-7 A typical fitness center might have students, instructors, class schedules, and a registration process.
Source: StockLite/Shutterstock.com

6.6.1 Example

Consider a fitness center, illustrated in Figure 6-7, with students, instructors, class schedules, and a registration process. As shown in Figure 6-8, the EMPLOYEE class belongs to the PERSON superclass because every employee is a person, and the INSTRUCTOR class is a subclass of EMPLOYEE.

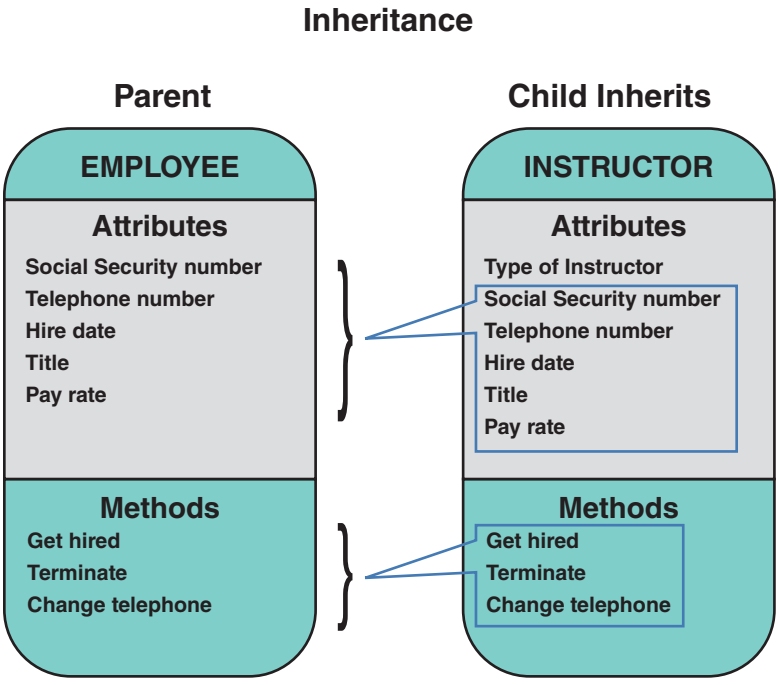
Figure 6-8 At the fitness center, the PERSON superclass includes common attributes and methods. EMPLOYEE is a class within the PERSON superclass. INSTRUCTOR is a subclass within the EMPLOYEE class.



6.7 Relationships Among Objects and Classes

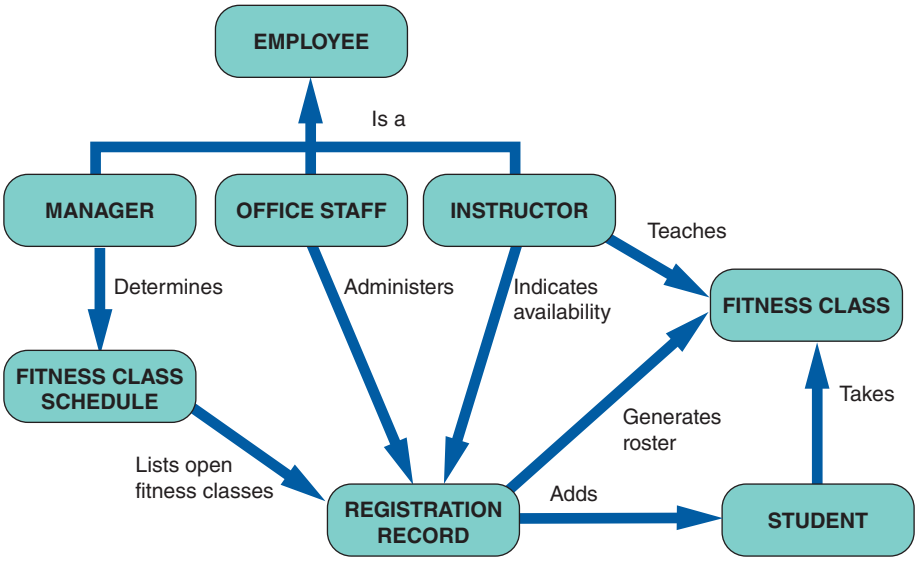
Relationships enable objects to communicate and interact as they perform business functions and transactions required by the system. Relationships describe what objects need to know about each other, how objects respond to changes in other objects, and the effects of membership in classes, superclasses, and subclasses. Some relationships are more robust than others (just as a relationship between family members is stronger than between casual acquaintances). The strongest relationship is called **inheritance**. Inheritance enables an object, called a **child**, to derive one or more of its attributes from another object, called a **parent**. In the example in Figure 6-9, the INSTRUCTOR object (child) inherits many traits from the EMPLOYEE object (parent), including Social Security number, phone number, and hire date. The INSTRUCTOR object can also possess additional attributes, such as the type of instructor. Because all employees share certain attributes, those attributes are assumed through inheritance and do not need to be repeated in the INSTRUCTOR object.

Figure 6-9 The INSTRUCTOR and EMPLOYEE objects have an inheritance relationship. The INSTRUCTOR (child) object inherits characteristics from the EMPLOYEE (parent) class and can have additional attributes.



After identifying objects, classes, and relationships, an object relationship diagram can be prepared to provide an overview of the system. That model is used as a guide to continue to develop additional diagrams and documentation. Figure 6-10 shows an object relationship diagram for a fitness center. Notice that the model shows the objects and their interactions to perform business functions and transactions.

Figure 6-10 An object relationship diagram for the fitness center.



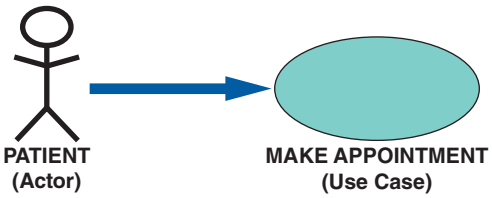
6.8 The Unified Modeling Language

Just as structured analysis uses DFDs to model data and processes, systems analysts use UML to describe O-O systems. An earlier chapter explained that UML is a popular technique for documenting and modeling a system. UML uses a set of symbols to graphically represent the various components and relationships within a system. Although UML can be used for business process and requirements modeling, it is mainly used to support O-O system analysis and develop object models.

6.8.1 Use Case Modeling

A **use case** represents the steps in a specific business function or process. An external entity, called an actor, initiates a use case by asking the system to perform a function or process. For example, a PATIENT (actor) can MAKE APPOINTMENT (use case) in a medical office system, as shown in Figure 6-11.

Figure 6-11 A PATIENT (actor) can MAKE APPOINTMENT (use case) in a medical office system. The UML symbol for a use case is an oval. The actor is shown as a stick figure.

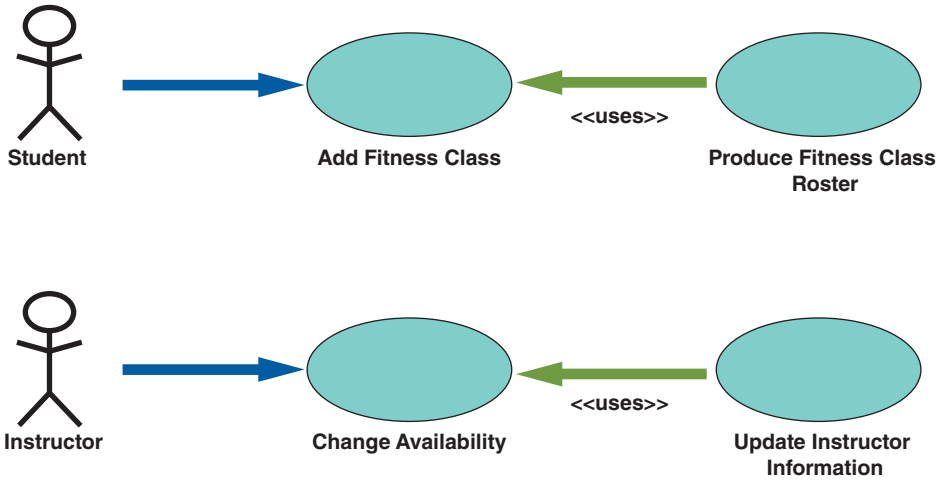


Notice that the UML symbol for a use case is an oval with a label that describes the action or event. The actor is shown as a stick figure with a label identifying the actor's role. The line from the actor to the use case is called an association because it links a particular actor to a use case.

Use cases also can interact with other use cases. When a use case incorporates the outcome of another use case, we say the second case *uses* the first case. UML indicates the relationship with an arrow pointing at the use case. Figure 6-12 shows an example in which a student adds a fitness class and PRODUCE FITNESS CLASS ROSTER *uses* the results of ADD FITNESS CLASS to generate a new fitness class roster. Similarly, if an instructor changes their availability, UPDATE INSTRUCTOR INFORMATION *uses* the CHANGE AVAILABILITY use case to update the instructor's information.

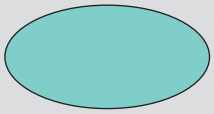
Review the information gathered during the requirements modeling phase to create use cases. The objective is to identify the actors and the functions or transactions they initiate. For each use case, develop a **use case description** as a table. A use case description documents

Figure 6-12 When a student adds a class, PRODUCE FITNESS CLASS ROSTER uses the results of ADD FITNESS CLASS to generate a new roster. When an instructor changes their availability, UPDATE INSTRUCTOR INFORMATION uses the CHANGE AVAILABILITY use case to update the instructor's information.



the name of the use case, the actor, the use case, a step-by-step list of the tasks and actions required for successful completion, and a description of alternative courses of action, preconditions, postconditions, and assumptions. Figure 6-13 shows an example of the ADD NEW STUDENT use case for the fitness center.

Figure 6-13 The ADD NEW STUDENT use case description documents the process used to add a current student to an existing fitness center class.



ADD NEW STUDENT Use Case

Name:	Add New Student
Actor:	Student/Manager
Description:	Describes the process used to add a student to a fitness class
Successful completion:	1. Manager checks FITNESS CLASS SCHEDULE object for availability 2. Manager notifies student 3. Fitness class is open and student pays fee 4. Manager registers student
Alternative:	1. Manager checks FITNESS CLASS SCHEDULE object for availability 2. Fitness class is full 3. Manager notifies student
Precondition:	Student requests fitness class
Postcondition:	Student is enrolled in fitness class and fees have been paid
Assumptions:	None

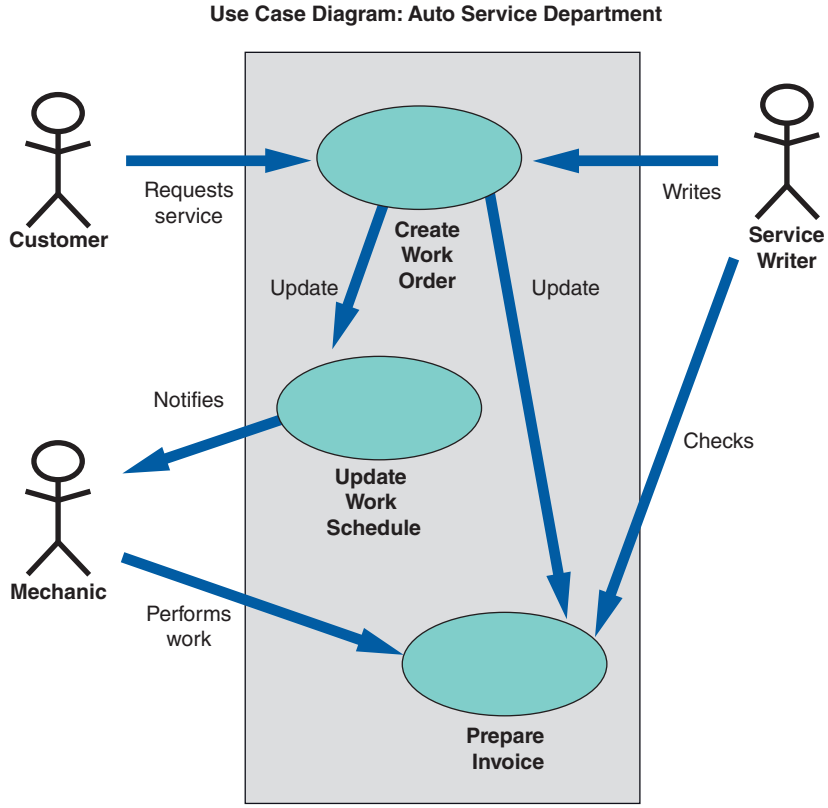
All related transactions should be grouped into a single use case when use cases are identified. For example, when a hotel customer reserves a room, the reservation system blocks out a room, updates the occupancy forecast, and sends the customer a confirmation. Those events are all part of a single use case called RESERVE ROOM, and the specific actions are step-by-step tasks within the use case.

6.8.2 Use Case Diagrams

A use case diagram is a visual summary of several related use cases within a system or subsystem. Consider a typical auto service department, as shown in Figure 6-14. The service department involves customers, service writers who prepare work orders and invoices, and mechanics who perform the work. Figure 6-15 shows the auto service department's possible use case diagram.

Figure 6-14 A typical auto service department might involve customers, service writers who prepare work orders, and mechanics who perform the work.
Source: michaeljung/Shutterstock.com

Figure 6-15 A use case diagram to handle work at an auto service department.



When creating a use case diagram, the first step is identifying the system boundary, represented by a rectangle. The **system boundary** shows what is included in the system (inside the rectangle) and what is not included in the system (outside the rectangle). After the system boundary is identified, use cases are placed on the diagram, the actors are added, and the relationships are shown.

Case in Point 6.2

Hilltop Motors

You were hired by Hilltop Motors as a consultant to help the company plan a new information system. Hilltop is an old-line dealership whose prior owner was slow to change. A new management team has taken over, eager to develop a first-class system. You are reviewing the service department, which is undergoing a significant expansion. You create a model of the service department in the form of a use case diagram. The main actors in the service operation are customers, service writers who prepare work orders and invoices, and mechanics who perform the work. You are meeting with the management team tomorrow morning. How would you create a draft of the diagram to present to them?

6.8.3 Class Diagrams

A **class diagram** shows a use case's object classes and relationships. Like a DFD, a class diagram is a logical model that evolves into a physical model and becomes a functioning information system. Entities, data stores, and processes are transformed into data structures and program code in structured analysis. Similarly, class diagrams evolve into code modules, data objects, and other system components.

In a class diagram, each class appears as a rectangle, with the class name at the top, followed by the class's attributes and methods. Lines show relationships between classes and have labels identifying the action that relates the two classes. To create a class diagram, review the use case and determine the classes participating in the underlying business process.

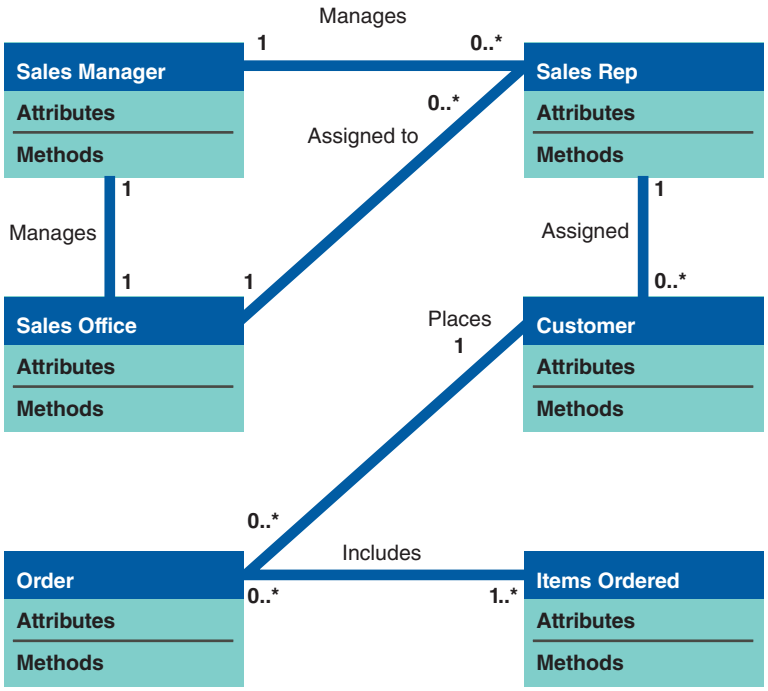
The class diagram also includes **cardinality**, which describes how instances of one class relate to another. For example, an employee might have earned no vacation days, one vacation day, or many. Similarly, an employee might have no spouse or one spouse. Figure 6-16 shows various UML notations and cardinality examples. Notice that in the figure, the first column shows a UML notation symbol that identifies the relationship in the second column. The third column provides a typical example of the relationship described in the last column. In the first row of the figure, the UML notation *0..** identifies *zero or many* relations. An example is an employee with no payroll deductions or many deductions.

Figure 6-16 Examples of UML notations that indicate the nature of the relationship between instances of one class and instances of another class.

UML Notation	Nature of the Relationship	Example	Description
0..*	Zero or many	Employee — Payroll Deduction (1 — 0..*)	An employee can have no payroll deductions or many deductions.
0..1	Zero or one	Employee — Spouse (1 — 0..1)	An employee can have no spouse or one spouse.
1	One and only one	Office Manager — Sales Office (1 — 1)	An office manager manages one and only one office.
1..*	One or many	Order — Item Ordered (1 — 1..*)	One order can include one or many items ordered.

Figure 6-17 shows a class diagram for a sales order use case. Notice that the sales office has one sales manager with anywhere from zero to many sales reps. Each sales rep can have anywhere from zero to many customers, but each customer has only one sales rep.

Figure 6-17 Class diagram for a sales order use case (attributes and methods omitted for clarity).



Case in Point 6.3

Train the Trainer, Inc.

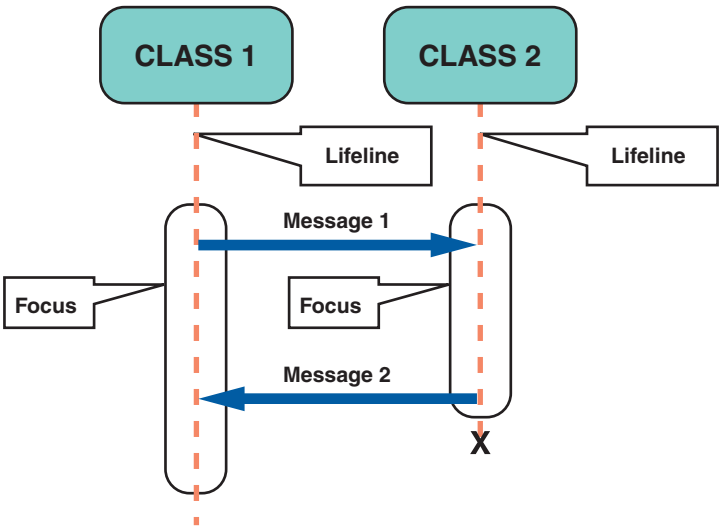
Train the Trainer develops seminars and workshops for corporate training managers who train their employees. Your job at Train the Trainer is to assemble the training materials. Right now, you are up against a deadline. The new object modeling seminar has a chapter on cardinality, and the client wants you to come up with at least three more examples for each of the four cardinality categories listed in Figure 6-16. The four categories are *zero or many*, *zero or one*, *one and only one*, and *one or many*. Even though you are under pressure, you are determined to use realistic and familiar examples for the students. What examples will you submit?

6.8.4 Sequence Diagrams

A sequence diagram is a dynamic model of a use case, showing the interaction among classes during a specified period. A sequence diagram graphically documents the use case by showing the classes, the messages, and the timing of the messages. Sequence diagrams include symbols that represent classes, lifelines, messages, and focuses. These symbols are shown in Figure 6-18.

Classes. A class is identified by a rounded rectangle with the name inside. Classes that send or receive messages are shown at the top of the sequence diagram.

Figure 6-18 A sequence diagram with two classes. Notice the X that indicates the end of the CLASS 2 lifeline. Also, notice that each message is represented by a line with a label that describes the message and that each class has a focus that shows the period when messages are sent or received.



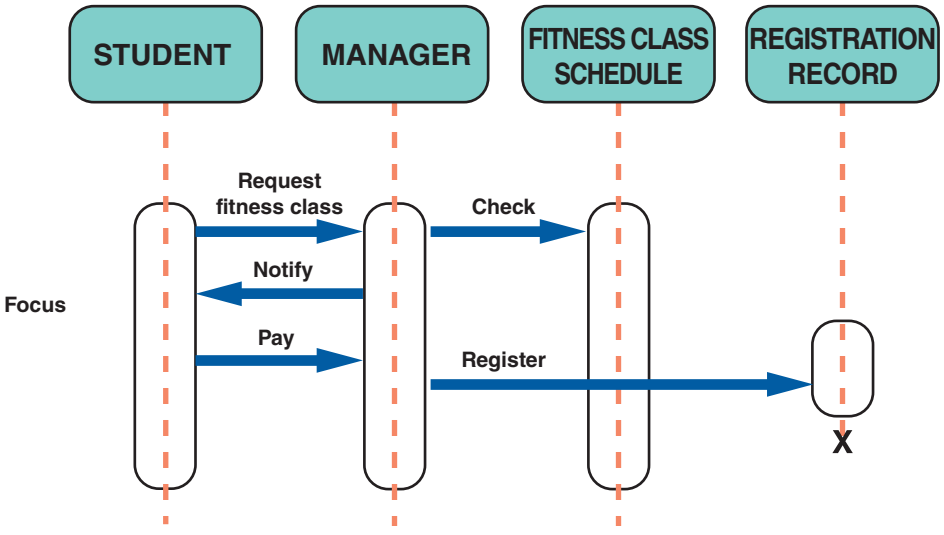
Lifelines. A lifeline is identified by a dashed line. The **lifeline** represents the time the object above it can interact with the other objects in the use case. An *X* marks the end of the lifeline.

Messages. A message is identified by a line showing the direction between two objects. The label shows the name of the message and can include additional information about the contents.

Focuses. A narrow vertical shape that covers the lifeline identifies a **focus**. The focus indicates when an object sends or receives a message.

Figure 6-19 shows a sequence diagram for the fitness center example's ADD NEW STUDENT use case. Notice that the vertical position of each message indicates the timing of the message.

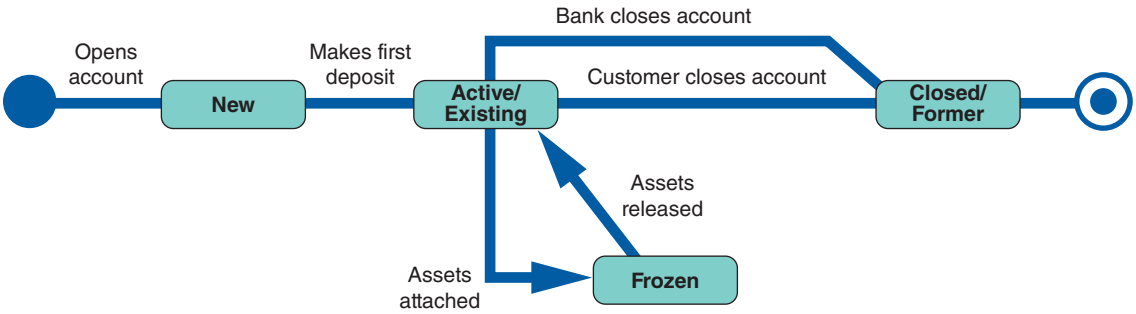
Figure 6-19 The ADD NEW STUDENT sequence diagram. The use case description for ADD NEW STUDENT is shown in Figure 6-13.



6.8.5 State Transition Diagrams

Earlier in this chapter, it was explained that state refers to an object's current status. A **state transition diagram** shows how an object changes from one state to another depending on events that affect the object. All possible states must be documented in the state transition diagram, as shown in Figure 6-20. A bank account, for example, could be opened as a NEW account, changed to an ACTIVE or EXISTING account, and eventually become a CLOSED or FORMER account. Another possible state for a bank account could be FROZEN if the account's assets are legally attached.

Figure 6-20 An example of a state transition diagram for a bank account.

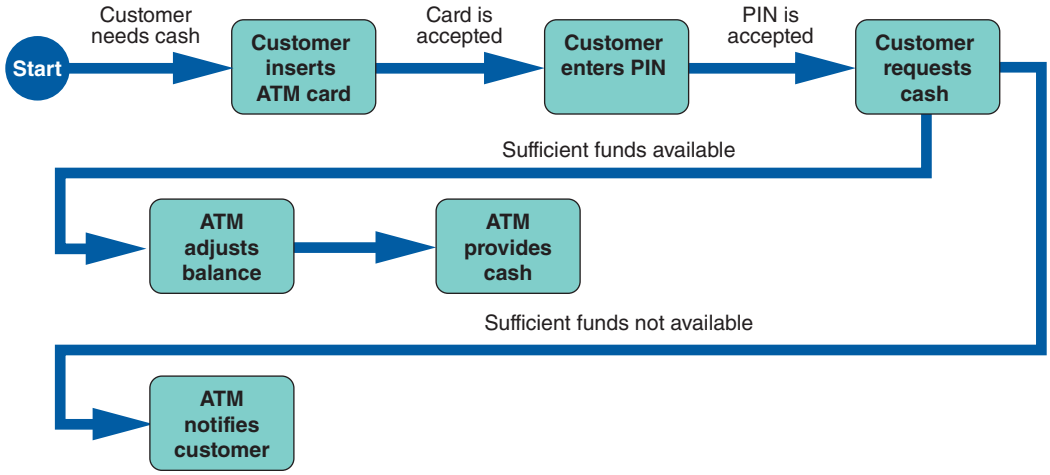


In a state transition diagram, the states appear as rounded rectangles with the state names inside. The small circle to the left is the initial state, or the point where the object first interacts with the system. Reading from left to right, the lines show direction and describe the action or event that causes a transition from one state to another. The circle at the right with a hollow border is the final state.

6.8.6 Activity Diagrams

An **activity diagram** resembles a horizontal flowchart that shows actions and events as they occur. Activity diagrams show the order in which the actions occur and identify the outcomes. Figure 6-21 shows an activity diagram for a cash withdrawal at an ATM. Notice that the customer initiates the activity by inserting an ATM card and requesting cash. Activity diagrams can also display multiple use cases in a grid, where classes are shown as vertical bars and actions appear as horizontal arrows.

Figure 6-21 An activity diagram showing the actions and events involved in withdrawing cash from an ATM.



Sequence, state transition, and activity diagrams are dynamic modeling tools that can help a systems analyst understand how objects behave and interact with the system.

6.8.7 Business Process Modeling

In addition to sequence and activity diagrams, business process models (BPMs) can represent a system's people, events, and interactions. BPMs were previously discussed as a requirement diagramming tool, but they can be used anytime during systems development. A BPM works well with object modeling because both methods focus on the actors and their behavior.

6.9 Tools

Object modeling requires many types of diagrams to represent the proposed system. Creating the diagrams by hand is time-consuming and tedious, so systems analysts rely on tools to speed up the process and provide an overall framework for documenting the system components. In addition, tools ensure consistency and provide standard links so that once objects are described and used in one part of the design, they can be reused multiple times without further effort.

Many computer-aided software engineering (CASE) tools are tailored to UML. Tools such as Visio are popular for drawing UML diagrams. Still, they lack semantics: they help create visually pleasing figures but do not know the meaning of the underlying artifacts. This means a systems analyst can create diagrams that appear correct but need to be revised according to the rules of UML.

Proper systems modeling tools, such as Cameo Systems Modeler or IBM's Rational product family, are typically used to overcome this shortcoming. These tools understand the meaning of the diagrams they help create, which supports creating syntactically and semantically correct UML diagrams. These tools also provide traceability, an essential feature in the SDLC that links design artifacts backward to requirements and forward to development and testing.

Summary

This chapter introduced object modeling, a popular technique that describes a system in terms of objects. Objects represent real people, places, events, and transactions. Unlike structured analysis, which treats data and processes separately, objects include data and processes that can affect the data. During implementation, systems analysts and programmers transform objects into program code modules that can be optimized, tested, and reused as often as necessary.

O-O terms include attributes, methods, messages, and classes. Attributes are characteristics that describe the object. Methods are tasks or functions the object performs when receiving a command. Objects can send messages, or commands, which require other objects to perform specific methods, or tasks. Polymorphism is the concept that a message gives different meanings to different objects. An object resembles a black box with encapsulated, or self-contained, data and methods.

Classes include objects that have similar attributes, or characteristics. Individual members of a class are called object instances. Objects within a class can be grouped into subclasses, which are more specific categories within the class. A class also can belong to a more general category called a superclass.

After identifying objects, classes, and relationships, an object relationship diagram can be prepared that shows the objects and how they interact to perform business functions and transactions. The most robust relationship between objects is inheritance.

The Unified Modeling Language (UML) is a widely used method of visualizing and documenting an information system. UML techniques include use cases, use case diagrams, class diagrams, sequence diagrams, state transition diagrams, and activity diagrams.

A use case describes a business situation initiated by an actor interacting with the information system. Each use case represents a specific transaction, or scenario. A use case diagram is a visual summary of related use cases within a system or subsystem. A class diagram illustrates a detailed view of a single use case, showing the classes that participate in the underlying business transaction and the relationship among class instances, called cardinality. A sequence diagram is a dynamic model of a use case, showing the interaction among classes during a specified period. Sequence diagrams include lifelines, messages, and focuses. A state transition diagram shows how an object changes from one state to another depending on events that affect the object. An activity diagram resembles a horizontal flowchart that shows actions and events as they occur in a system.

In addition to object models, business process modeling can represent a system's people, events, and interactions.

CASE tools provide an overall framework for system documentation. These tools can speed development, ensure consistency, and provide standard links that enable reuse during O-O analysis.

Key Terms

activity diagram
attribute
cardinality
child
class
class diagram
design patterns
encapsulation
focus
inheritance
instance
lifeline
message
method
object
object model
object-oriented (O-O) analysis
parent
polymorphism
relationships
state
state transition diagram
subclass
superclass
system boundary
use case
use case description

Review Questions

1. What is object-oriented analysis?
2. Define an object in an information system and provide three examples.
3. Define an attribute and provide three examples.
4. Define a method and provide three examples.
5. Define encapsulation and explain how it is used in O-O analysis.
6. Define a class, subclass, and superclass, and provide three examples of each.
7. Explain the concept of inheritance in object relationships.
8. Draw an object relationship diagram for a typical library system.
9. Define a use case and a use case diagram and prepare a sample of each.
10. Why is it important to use a modeling tool, not just a diagramming tool, during O-O analysis?

Discussion Topics

1. You are an IT consultant asked to create a new system for a small real estate brokerage firm. You need to gain experience with an O-O approach and decide to try it. How will you begin? How will the tasks differ from structured analysis?
2. Some professionals believe it is harder for experienced analysts to learn object-modeling techniques because they are accustomed to thinking about data and processes as separate entities. Others believe that solid analytical skills are easily transferable to the newer approach. What do you think, and why?
3. Polymorphism is the concept that the same message gives different meanings to different objects. Can you think of examples where this behavior may provide unexpected results?
4. You are creating a system for a bowling alley to manage information about its leagues. During the modeling process, you create a state transition diagram for an object called *League Bowlers*. What are the possible states of a league bowler, and what happens to a bowler who quits the league and rejoins the following season?
5. UML is a large and complex modeling language. How can an IT professional tell when a UML diagram is correct and not just visually pleasing?

Hands-On Projects

1. Contact your school's or local business's IT staff to learn if they use O-O programming languages. If so, determine what languages and versions are used, how long they have been used, and why they were selected.
2. Create a presentation explaining O-O analysis, including definitions of basic terms such as objects, attributes, methods, messages, and classes.
3. Draw an activity diagram showing the actions and events involved in depositing a check to a bank account using a mobile app.
4. Investigate business process modeling languages, such as BPEL.
5. Prepare a report on at least three tools that provide UML support. Use the tools' capabilities to create class and sequence diagrams as a key requirement.

Ethical Issues

Your company sent several staff members for UML training by an outside vendor. Everyone who attended the training received a copy of the instructor's materials, which included study guides and sample exam questions and solutions.

After completing the training course, you can take a certification exam. If you pass the exam, you will be credentialed as a "UML Expert" by an independent agency. You can parlay these credentials into a higher salary and boost your career.

A co-worker who did not attend the training asked for a copy of the training materials and wants to take the exam without "wasting time in class." Should you provide a copy of the training materials? If you do, how might this diminish your accomplishments? If you don't, would you hurt the team by not helping another member become more knowledgeable about UML?

Phase 3

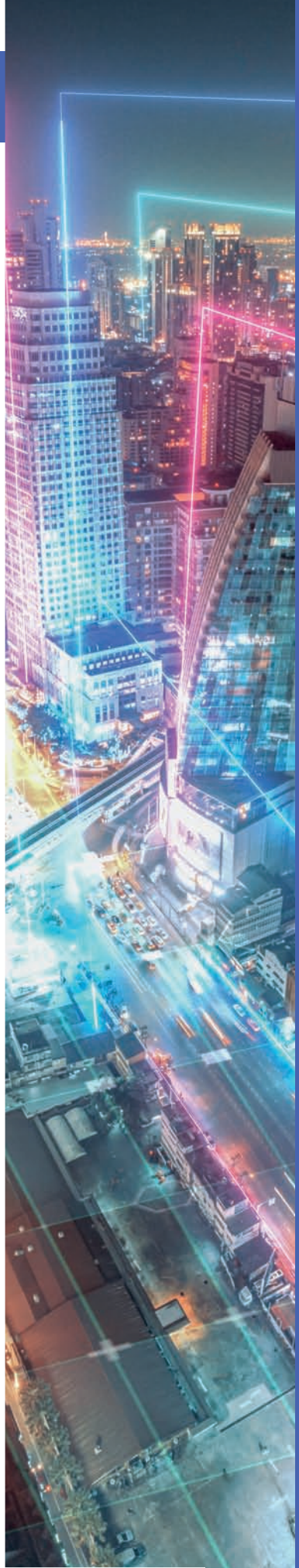
Design

The design phase of the SDLC focuses on creating user interfaces that prioritize human–computer interaction, usability, and flexibility. It emphasizes practices for understanding user needs and ensuring technology supports intuitive, productive interactions. This phase also covers organizing data effectively, highlighting the importance of database management systems and structured data design for efficient storage and retrieval. Furthermore, it examines the evolution of system architecture, emphasizing the significance of security, scalability, and adapting to Internet technologies. Design is crucial in developing functional, user-friendly, and secure systems.

Chapter 7 highlights the importance of user interface design and human–computer interaction. It presents seven habits of successful interface designers and guidelines for creating compelling, user-friendly interfaces. The chapter covers source document and form design, printed output, and report design principles. It also discusses technology issues, control and security considerations, and emerging trends such as responsive web design, prototyping, and wearable computing.

Chapter 8 investigates data design concepts, including data structures, database management systems, and their components. It covers web-based design, key data design terms, and entity-relationship diagrams. The chapter explains data normalization, standard notation format, and the first, second, and third normal forms. It also discusses codes, their types, and their design. Finally, it explores data storage, access, and control techniques.

Chapter 9 explores system architecture, providing a checklist of corporate culture, scalability, web integration, security, and compliance considerations. It traces the evolution of system architecture from mainframes to personal computers and networks. The chapter explores client/server architecture, the impact of the Internet, cloud computing, and e-commerce solutions. It also covers processing methods, network models, devices, and wireless networks, including standards, topologies, and trends.



Chapter

7

User Interface Design

Learning Objectives

When you finish this chapter, you should be able to:

1. Explain user interfaces.
2. Explain the concept of human–computer interaction.
3. Summarize the seven habits of successful interface designers.
4. Summarize the 10 guidelines for user interface design.
5. Design effective source documents and forms.
6. Explain report design techniques for printed output.
7. Discuss output and input technology issues.
8. Describe output and input security and control issues.
9. Explain emerging user interface trends.

Chapter 7 is the first of three chapters in the systems design phase of the SDLC. User interface design is the first task in this phase. Designing the interface is extremely important because everyone wants a system that is easy to learn and use. This chapter explains how to design an effective user interface and how to handle data security and control issues. The chapter emphasizes the importance of user feedback and involvement in all design decisions.

The chapter includes three "Case in Point" discussion questions to help contextualize the concepts described in the text. The scenario described in the "Ethical Issues" section considers the ethical (and possibly legal) constraints on how far the creative work of others can be used without crediting the source—for example, in the design of a company's website.

7.1 User Interfaces

A **user interface (UI)** describes how users interact with a computer system and consists of all the hardware, software, screens, menus, functions, output, and features that affect two-way communications between the user and the computer. The UI is the key to **usability**, including user satisfaction, business function support, and system effectiveness.

Traditionally, a chapter on UI design started with a discussion of output because output is what users touch, view, and need to do their jobs. Today, the situation is different for several important reasons:

- Users can design their output. System designers are more aware of user needs and desires. A system can maintain data integrity and allow users to view, sort, filter, and examine data in any way that helps them do their jobs. There was a time when the MIS department made those choices, and users had little or no say in the matter. Today, successful applications are designed quite differently—the system developer identifies user needs and then creates a design that will satisfy users *and* meet corporate requirements.
- Centralized IT departments no longer produce reams of printed reports. Those reports often gathered dust while sitting on top of file cabinets. While a few examples might persist, customer-designed output is the overwhelming trend. The customer might be an individual user or a community of users, such as a department. As discussed earlier in the book, the IT team must understand user requirements before creating a solution.
- The UI has evolved into a two-way channel with powerful output capability, and most user information needs can be met with screen-generated data, which a user can print, view, or save. Well into the 1980s and beyond, a UI was a blank character-based screen, which might or might not offer menu choices. If a user entered a command improperly, the system responded with an error message, frustrating and stifling productivity. Many hardware-centric vendors did not understand the importance of the UI and its implications.

Apple pioneered UI development, introducing the **graphical user interface (GUI)**, complete with mouse and screen icons, in the early 1980s. At that point, only a few companies were ready for this concept. When Microsoft jumped on the GUI bandwagon with its Windows operating system (OS), the corporate doors swung open, and everyone from managers on down said, "How did we ever do without this?"

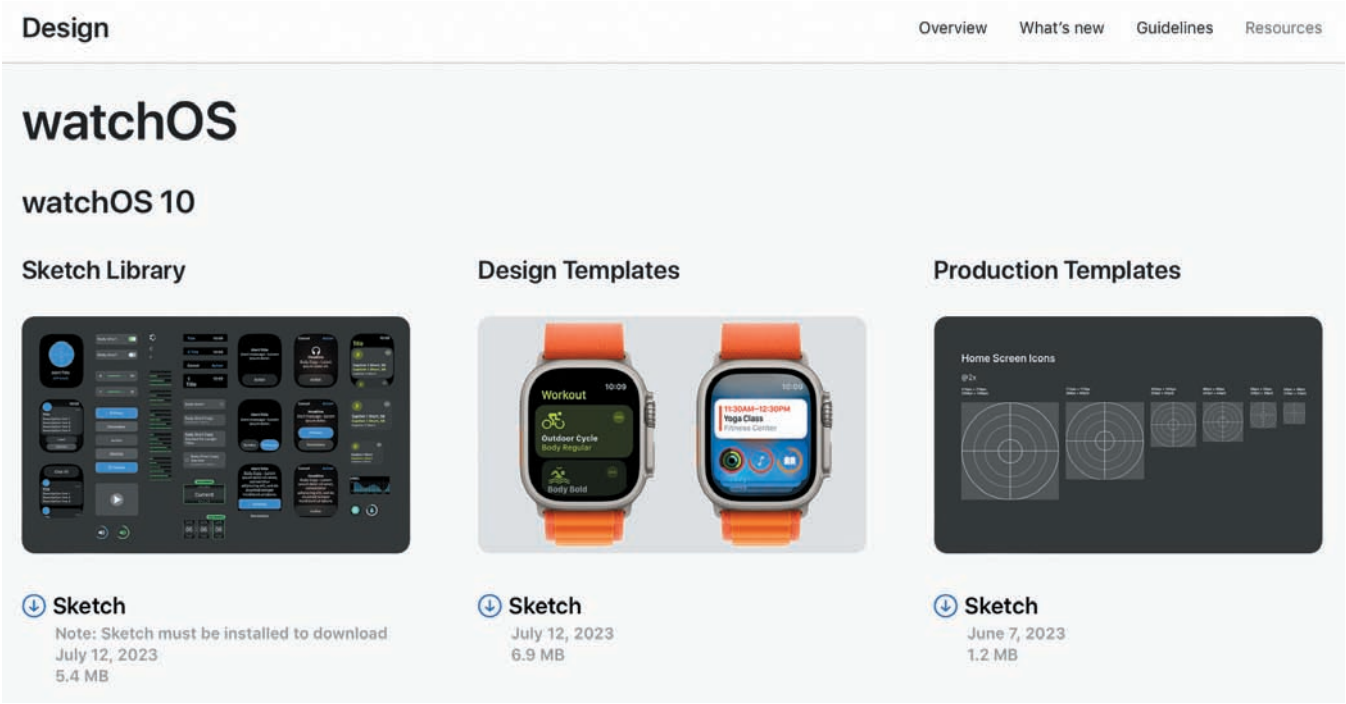
Many industry leaders believe that the best interfaces are the ones that users do not even notice—they make sense because they do what users expect. For example, as shown in Figure 7-1, Apple believes designing an exceptional UI is essential to a successful product. Apple has long distinguished itself from its competitors by the intuitiveness of its products. Apple's command of the market suggests that consumers are willing to pay a premium for products that "just work."

When developing older systems, analysts typically designed all the printed and screen output first, then worked on the inputs necessary to produce the results. Often, the UI mainly consisted of process-control screens that allowed the user to send commands to the system. That approach worked well with traditional systems that transformed input data into structured output.

As information management evolved from centralized data processing to dynamic, enterprise-wide systems, the primary focus shifted from the IT department to the users. The IT group became a supplier of information technology rather than a supplier of information. Today, the focus is on users within and outside the company, how they communicate with the information system, and how the system supports the firm's business operations.

In a **user-centered** system, the distinction blurs between input, output, and the interface itself. Most users work with a varied mix of input, screen output, and data queries as they perform their day-to-day job functions. Because all those tasks require interaction with the computer system, the UI is vital in the systems design phase.

Figure 7-1 Apple has long been a leader in creating elegant product user interfaces.
Source: Apple Inc.



7.2 Human–Computer Interaction

A UI is based on fundamental principles of human–computer interaction. **Human–computer interaction (HCI)** describes the relationship between computers and those who use them to perform their jobs, like the worker shown in Figure 7-2. HCI concepts apply to everything from smartphones to global networks. In its broadest sense, HCI includes all the communications and instructions necessary to enter input into the system and obtain output through screen displays or printed reports.

HCI encompasses a variety of methods that facilitate how humans interact with computers, adapting to technological advances and user needs. GUIs offer intuitive, visual-based interaction using windows, icons, and menus, primarily navigated via pointing

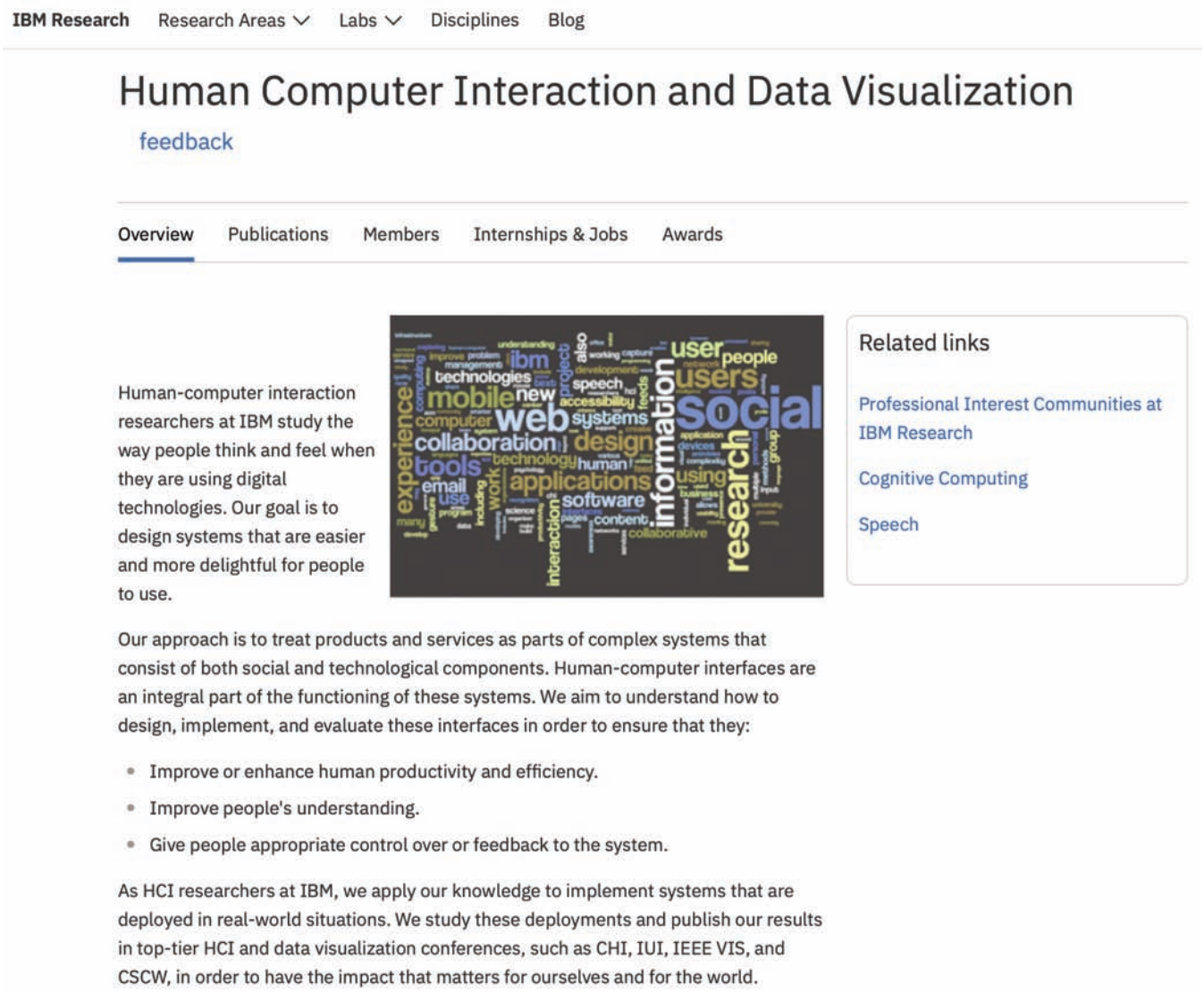
Figure 7-2 HCI is essential to employee productivity, whether the work is done in a traditional office setting or on a construction site.
Source: Gorodenkoff/Shutterstock.com

devices like a mouse. Command-line interfaces (CLIs) allow interaction through text-based commands, offering power and efficiency for skilled users. In smartphones and tablets, touch interfaces enable direct manipulation through gestures like tapping and swiping. Voice user interfaces (VUIs) employ voice commands for interaction, exemplified by virtual assistants. Gesture-based interfaces recognize and respond to physical movements and are increasingly used in gaming and virtual environments. Augmented reality (AR) and virtual reality (VR) offer immersive experiences, blending digital and physical worlds. Each type serves different user requirements and application contexts, marking the diverse evolution of HCI.

A systems analyst designs UIs for software developed in-house and customizes interfaces for various commercial packages and user productivity applications. The main objective is to create a user-friendly, easy-to-learn design, blending different types of HCI as needed. Designers strive to translate user behavior, needs, and desires into interfaces users don't notice. IBM has stated that the best UIs are "almost transparent—you can see right through the interface to your work." In other words, a **transparent interface** does not distract the user and calls no attention to itself.

Industry leaders Microsoft and IBM both devote considerable resources to UI research. Figure 7-3 describes IBM Research's work on HCI. They aim to "design systems that are easier and more delightful for people to use."

Figure 7-3 IBM's research division is a leader in exploring HCI.
Source: IBM Corporation



Because HCI significantly impacts user productivity, it gets lots of attention—especially where multimillion-dollar issues are concerned. For example, software usability substantially impacts the medical profession, particularly physicians, who often struggle with poorly designed **electronic health record (EHR)** systems. Physicians often multitask, answering a question about one patient while prescribing medication for another. EHR software was not designed around that type of workflow.

Case in Point 7.1

Casual Observer Software

Casual Observer Software's main product is a program that monitors and analyzes user keystrokes and mouse clicks to learn more about how employees use their computer systems. The problem is that some users feel this is an unwarranted intrusion into their privacy and prefer not to be observed. Some even fear the data could be used for other reasons, including performance appraisal. You are a consultant hired by a client firm trying to decide whether to use this software.

Before you advise the client, remember the Hawthorne Effect, which suggests that employees might behave differently when they know they are being observed. Finally, consider the ethical issues that might be involved in this situation. What will you advise your client, and why?

7.3 Seven Habits of Successful Interface Designers

Although IT professionals have different views about interface design, most agree that good design depends on seven fundamental principles. Successful interface designers use these basic principles as a matter of course—they become habits. These desirable habits are described in the following sections.

7.3.1 Understand the Business

The interface designer must understand the underlying business functions and how the system supports individual, departmental, and enterprise goals. This comprehensive understanding involves grasping the fundamental business operations and recognizing how the proposed system aligns with the goals at various levels, from individual employees to entire departments and the enterprise. Such knowledge is crucial because the primary purpose of any system is to facilitate and enhance the efficiency of business operations. Therefore, an interface designer must focus on the aesthetic and usability aspects of the design and ensure that the interface serves the broader business objectives effectively.

A good starting point is analyzing a functional decomposition diagram (FDD) to design an effective and user-centric interface. As described earlier in the book, an FDD is a tool that visually represents the business's functions. It starts by outlining the major functions and then delves into more detailed levels. This breakdown helps in understanding not just the high-level operations of the business but also the intricate, day-to-day tasks that employees engage in. By using an FDD, an interface designer can identify the critical functions that users need to perform and ensure that these tasks are supported and streamlined through the interface. This process helps in bridging the gap between system capabilities and user requirements.

Using an FDD in interface design is particularly valuable; it provides a structured approach to understanding user tasks. It also acts as a checklist, guiding the designer to include all necessary elements in the interface. This ensures the final design is aesthetically pleasing, intuitive, and functionally comprehensive, covering all aspects of user tasks as the business functions dictate. Such an approach guarantees that the interface looks good and aligns perfectly with the business processes and user needs, ultimately leading to a more effective and successful system implementation.

7.3.2 Maximize Graphical Effectiveness

Maximizing graphical effectiveness in UI design is crucial for creating user-friendly and efficient systems. Numerous studies have demonstrated that people generally learn and comprehend better visually. This principle is a key reason behind the widespread success and popularity of operating systems like Apple's iOS and Microsoft Windows, renowned for their GUIs. With their intuitive design and visual appeal, these interfaces facilitate easy learning and use, even for those new to technology. A well-crafted interface that leverages these visual strengths can significantly enhance a user's ability to quickly learn and adapt to a new system, thereby increasing productivity and user satisfaction.

Graphical interfaces offer several practical advantages in a systems environment. One notable feature is the ability to display and interact with multiple windows on a single screen, a functionality that is indispensable in today's multitasking work culture. This feature allows users to efficiently navigate between different tasks or applications without losing context, enhancing workflow and productivity. A well-designed GUI also enables seamless data transfer between programs, simplifying many complex tasks that would otherwise require cumbersome steps. This integration and ease of use are fundamental in designing systems that cater to the evolving needs of users in a dynamic technological landscape.

Regarding data entry, the importance of adhering to specific design guidelines becomes even more pronounced. In later sections of this chapter, detailed guidelines on data entry screen design are discussed, underscoring the need for interfaces that are not only visually appealing but also functionally efficient. These guidelines ensure that data entry, a fundamental component of many systems, is streamlined and error-free. Adherence to these principles ensures that the interface supports users in their tasks effectively, reducing the likelihood of errors and enhancing overall data integrity. In sum, maximizing graphical effectiveness in interface design is not just about aesthetics; it's about creating an intuitive environment that aligns with user needs and system functionalities.

7.3.3 Think Like a User

Adopting a user-centric mindset is fundamental for a systems analyst aiming to create effective and intuitive interfaces. Understanding potential users' varied experience, knowledge, and skill levels is crucial. This understanding becomes even more important when dealing with a diverse user base, where capabilities range from novice to highly experienced. An effective interface should cater to this spectrum, offering flexibility and adaptability to accommodate all users. This approach ensures that the system is accessible and user-friendly for everyone, irrespective of their prior experience or technical proficiency. The goal is to create a system that is not only functional but also inclusive, ensuring that no user feels alienated or overwhelmed by the technology.

To develop a genuinely user-centered interface, designers must immerse themselves in the mindset of the users, seeing and experiencing the system as they would. This involves using language, terms, and metaphors that are familiar and relatable to the users. For instance, users often have real-world experience with interactive systems like automobiles, automated teller machines (ATMs), or microwave ovens, which provide immediate and understandable feedback. Drawing parallels from these everyday experiences, users naturally expect similar levels of clarity and responsiveness from computer systems. Therefore, a system that leverages familiar concepts and feedback mechanisms can significantly enhance user comfort and efficiency. It's about creating an environment where users can intuitively navigate and interact with the system without a steep learning curve, thus making technology more approachable and user-friendly.

In addition to conceptually understanding the user's perspective, practical considerations are equally important. This includes meticulous attention to the details of input and output processes. Inputs should be designed to be intuitive, straightforward, and forgiving of errors, reducing the likelihood of user frustration and mistakes. This aspect is particularly critical in scenarios where incorrect input can lead to significant problems or inefficiencies. On the

output side, the information presented to users must be attractive, easy to comprehend, and appropriately detailed. It should provide users with the necessary information clearly and concisely, avoiding unnecessary complexity or ambiguity. By carefully examining and refining these interaction points, designers can create a system that offers a more pleasant and efficient experience, ultimately leading to higher user satisfaction and better system adoption.

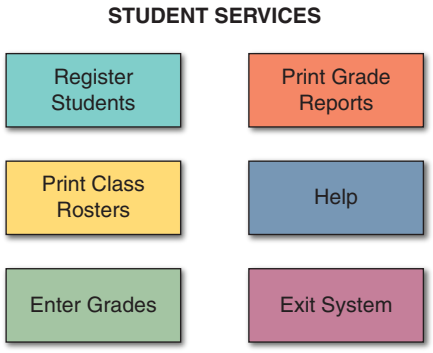
7.3.4 Use Models and Prototypes

The interface holds a place of paramount importance from a user's perspective. It's the primary medium users have for interacting with the system, often for extended periods. This high level of interaction makes the interface not just a component of the system but the face of the system itself. For this reason, the interface must be designed with the user's needs and preferences in mind. Constructing models and prototypes is essential in this process, as they serve as a preliminary interface version for user testing and approval. These prototypes provide a tangible interface representation, allowing users to interact, give feedback, and ensure that the final product meets their expectations and requirements. Therefore, the role of an interface designer transcends mere aesthetics; it involves understanding and implementing the functional needs and usability aspects that define the user experience.

The process of interface design benefits greatly from early and extensive user feedback. One effective method for initiating this feedback loop is to use storyboards. A storyboard presents initial screen designs as a series of sketches, displaying the general layout and design of the interface. This visual representation is instrumental in conveying the proposed design to users, enabling them to envision how the interface will look and function. Storyboards can be created using specialized software or even through freehand drawing, offering flexibility in the design process. The key advantage of storyboards is that they facilitate early user involvement, allowing for adjustments and improvements based on user reactions before the design is fully implemented. This early engagement with users ensures the user-centered design process focuses on real-world applicability and usability.

Finally, comprehensive user testing of all aspects of the interface design is critical. The interface is scrutinized during this testing phase, with users providing feedback on usability, functionality, and overall user experience. Various methods can be employed to gather user input, such as conducting interviews, administering questionnaires, and observing user interactions with the interface. Additionally, using software to record and measure user interactions provides valuable **usability metrics** data. This data can offer insights into how users navigate the interface, identify potential areas for improvement, and assess overall usability. By incorporating this user feedback and data into the design process, interface designers can refine and optimize the interface, ensuring it effectively meets users' needs. This iterative design, testing, and refinement process is crucial in creating a functional, efficient, intuitive, and user-friendly interface.

Figure 7-4 The opening screen displays the main options for a student registration system. A user can click an option to see lower-level actions and menu choices.



7.3.5 Focus on Usability

The design of the UI is a critical aspect that directly impacts the usability and effectiveness of the information system. A well-designed UI should encompass all the tasks, commands, and communications between the users and the system. The primary goal is to create an interface that is intuitive and easy to navigate, ensuring users can interact with the system efficiently to complete their tasks. The opening screen of the interface plays a pivotal role, as it sets the tone for the user's interaction with the system. It should present the main options clearly and concisely, as demonstrated in the example in Figure 7-4. This initial presentation helps users quickly grasp the system's capabilities and how to navigate it, providing a solid foundation for their subsequent interactions.

Each screen within the interface should logically lead to subsequent screens, offering more specific options and functionalities.

This hierarchical design is crucial in managing the flow of information and user actions within the system. The challenge lies in striking the right balance in the number of choices presented on each screen. Too many options can overwhelm and confuse users, making the interface cumbersome and challenging to navigate. On the other hand, offering too few options can result in excessive submenu levels, complicating the navigation process and potentially frustrating users. The objective is to offer a reasonable number of choices on each screen, enough to provide necessary functionality but not so many that it becomes overwhelming. This balance ensures that users can find what they need quickly and efficiently, enhancing the overall user experience.

An effective strategy in interface design is to prioritize the presentation of options based on their frequency of use or importance. Presenting the most-used choice as the default option is practical, as it aligns with most users' needs and expectations. However, it's equally important to provide users the flexibility to select other alternatives if their needs differ from the default. This flexibility allows for a more personalized and user-centric experience, catering to a broader range of user requirements. By carefully considering these design elements, interface designers can create a system that is not only functional but also adaptable to the diverse needs of its users, ultimately leading to a more successful and user-friendly information system.

7.3.6 Invite Feedback

Launching a system is not the end of the design process; it is a critical phase in the system's ongoing development and refinement. Once a system becomes operational, continuous monitoring and evaluation of its usage are essential. This post-launch phase is crucial for understanding how the system is utilized in real-world scenarios and identifying any unforeseen issues or areas for improvement. Systems analysts play a pivotal role here, observing and surveying users to gauge how well the system features are being used. This can include tracking which features are most frequently used, which are ignored, and any difficulties users encounter. Such continuous assessment is vital for ensuring that the system not only functions as intended but also meets the evolving needs of its users.

During full-scale operations, specific problems or inefficiencies in the system that were not noticeable during prototype testing may become apparent. This is common, as real-world usage often presents scenarios and challenges that are difficult to simulate in a controlled testing environment. The feedback from users during this phase is invaluable, providing insights into practical issues and user experiences that may have been overlooked earlier. For example, users might find certain features confusing or inadequate for their needs. This user feedback is a gold mine of information that can guide further refinements and enhancements to the system.

Based on the feedback and observations gathered, there might be a need for revisions in various aspects of the system, such as the help screens or other UI elements. Help screens are crucial; they guide users through the system and help resolve common queries or problems. If these screens are unhelpful or challenging to understand, they can significantly hinder users' ability to use the system effectively. Therefore, redesigning these screens based on user input can dramatically enhance the system's usability. Similarly, other design changes may be required to allow the system to reach its full potential. These modifications, driven by real user experiences and feedback, ensure that the system continuously evolves and improves, ultimately leading to a more effective, user-friendly, and successful system.

7.3.7 Document Everything

Documentation plays a pivotal role, especially when it comes to the design of UIs. Detailed documentation of all screen designs is not just a formality—it's a crucial resource for future development and maintenance. For programmers and other team members who will work on or modify the system in the future, having a comprehensive, well-documented set of designs is invaluable. This documentation should include every aspect of the screen designs, including layout, functionality, and user interactions. If CASE tools or screen generators are employed in the design process, organizing and saving these designs in a structured format, akin to

a menu tree, is beneficial. This hierarchical organization helps maintain clarity and ease of navigation through the various elements of the interface, facilitating easier understanding and implementation by the programming team.

Moreover, the documentation process extends beyond technical schematics and includes user-approved sketches and storyboards. These user-centric elements are key in documenting the evolution of the UI design, showcasing how user feedback and testing have shaped the final product. These sketches and storyboards serve as a tangible record of the decisions made and provide valuable context for why certain design choices were preferred. They can also be an essential reference when making future changes or updates to the system, ensuring that any modifications align with the original user requirements and design intent. This comprehensive documentation bridges the initial user-centered design principles and the technical implementation, ensuring coherence and consistency throughout the system's life cycle.

Applying basic, user-centered design principles is at the heart of creating a successful UI, where a systems analyst's expertise becomes crucial. These principles guide the design process to ensure the interface is intuitive, efficient, and tailored to the users' needs. From the initial planning stages to the final delivery, every step should be informed by a deep understanding of the end users—their preferences, behaviors, and challenges. By prioritizing the user experience in the design process, a systems analyst can develop an interface that meets the system's technical requirements and provides a seamless and satisfying user experience. The meticulous documentation of each step in this process further solidifies the effectiveness of the design, providing a clear blueprint for current and future development efforts. This approach underscores the importance of a user-centered perspective in systems analysis and design, ensuring the creation of functional and user-friendly interfaces.

7.4 Guidelines for User Interface Design

The effectiveness of a system is primarily determined by its UI. User satisfaction and usability accurately measure success, no matter how advanced the underlying technology. This underscores the importance of UI design, which should be approached with the user's needs and preferences at the forefront. Over the years, industry best practices have distilled a set of general guidelines for successful UI design. These guidelines emphasize the importance of the design's simplicity, clarity, consistency, and responsiveness. They are based on the premise that an intuitive and user-friendly interface enhances user engagement and productivity. However, it's crucial to note that these guidelines often overlap, as many core principles are interrelated and contribute to a cohesive user experience.

For example, Ben Shneiderman, an emeritus professor at the University of Maryland and noted expert in HCI, crafted "Eight Golden Rules of Interface Design." These principles are aimed at creating user-friendly and efficient interfaces. They emphasize the importance of consistency in design elements and actions across the interface, enabling expert users to utilize shortcuts for quicker navigation. The rules advocate for informative feedback responding to user actions and structured dialogs that provide a sense of completion. They emphasize the importance of error prevention and easy error handling, allowing for reversible actions to reduce user anxiety and encourage exploration. Additionally, these guidelines stress the need for interfaces that support users' sense of control and minimize their short-term memory load, making interactions more intuitive and reducing cognitive strain. These rules enhance the user experience by making interfaces more accessible, efficient, and engaging.

While these guidelines offer a solid foundation for UI design, it's important to recognize that there is no one-size-fits-all approach. The guidelines are particularly tailored for traditional systems development, serving as a starting point from which designers can adapt and innovate according to the specific needs of their project. However, UI development for the web, mobile apps, or wearable devices presents additional challenges and considerations. These platforms have unique constraints and opportunities, such as varying screen sizes, touch interfaces, and contextual usage (like on-the-go use for mobile apps). As such, while the general guidelines provide a helpful framework, designers working on web or mobile

interfaces must also consider platform-specific factors to ensure their designs are effective in those environments.

Moreover, HCI usability varies significantly between devices such as keyboards, touchscreens, and voice control, offering distinct advantages for different contexts. Keyboards are highly efficient for text input and complex command execution, and favored for their precision and speed, especially in professional or typing-intensive environments. Touchscreens offer intuitive and direct interaction, excelling in ease of use for navigating graphical interfaces, and are ideal for mobile or tablet use where quick, straightforward access is required. Voice control provides a hands-free experience and is excellent for multitasking or accessibility, allowing natural language use for commands and searches. However, it can sometimes struggle with accuracy and complex commands. Each method caters to different user needs and scenarios, making their effectiveness context-dependent.

The most crucial aspect to remember in UI design is that adherence to guidelines should not be dogmatic. The primary goal is to create an interface that resonates with its users, fulfilling their needs and preferences. This means that not all guideline recommendations will apply universally to every project. The best UI is tailored to its specific user base, considering their behaviors, preferences, and context. Designers must be willing to adapt, experiment, and sometimes even disregard certain guidelines to achieve a more user-centric design. In essence, the effectiveness of an interface is determined by its ability to connect with its users and provide them with a seamless, intuitive, and satisfying experience. This user-centered approach is at the heart of successful interface design in systems analysis and design.

7.4.1 Create an Interface That Is Easy to Learn and Use

Creating an interface that is easy to learn and use is crucial because it significantly enhances user adoption and satisfaction, ensuring that users can quickly become proficient without extensive training or frustration. An intuitive interface improves efficiency and productivity, as users spend less time figuring out how to use the system and more time accomplishing their tasks effectively. Furthermore, it broadens the accessibility of the technology, making it usable for a diverse range of people with varying levels of technical proficiency, thus expanding its potential user base.

The following UI design guidelines support these goals:

- Focus on system design objectives rather than calling attention to the interface.
- Create a design that is easy to understand and remember.
- Maintain a standard design in all interface **modules**, including color, screen placements, fonts, and the overall "look and feel."
- Provide commands, actions, and system responses that are consistent and predictable.
- Allow users to correct errors quickly.
- Clearly label all controls, buttons, and icons.
- Make it easy to navigate or return to any level in the menu structure.

It's also helpful to select familiar images that users can understand and to provide logical, concise, and clear instructions on the screen. For example, the top screen in Figure 7-5 shows four control buttons, but none has an obvious meaning. In the bottom screen, the first five messages provide little or no information. The last message is the only one that is easy to understand.

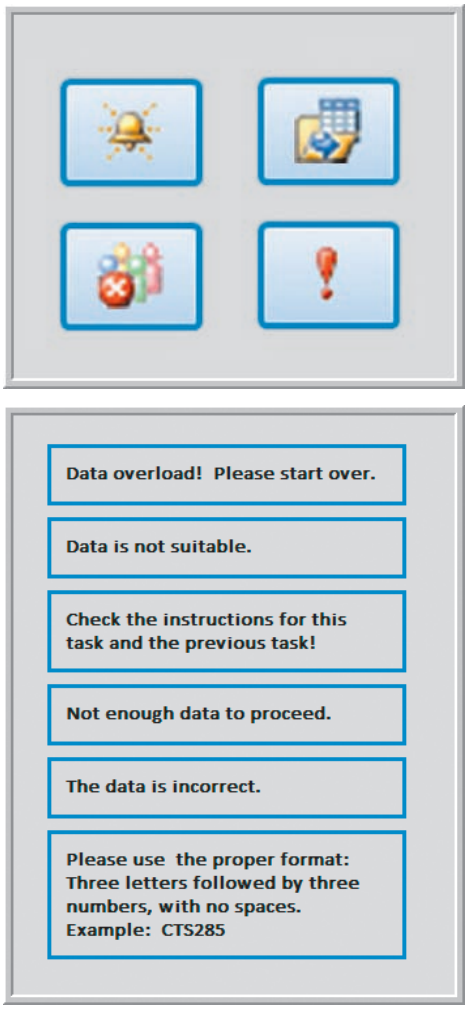


Figure 7-5 In the example at the top, the icons do not have a clear message. Only one message is understandable in the text examples at the bottom. The other messages would frustrate and annoy most users.

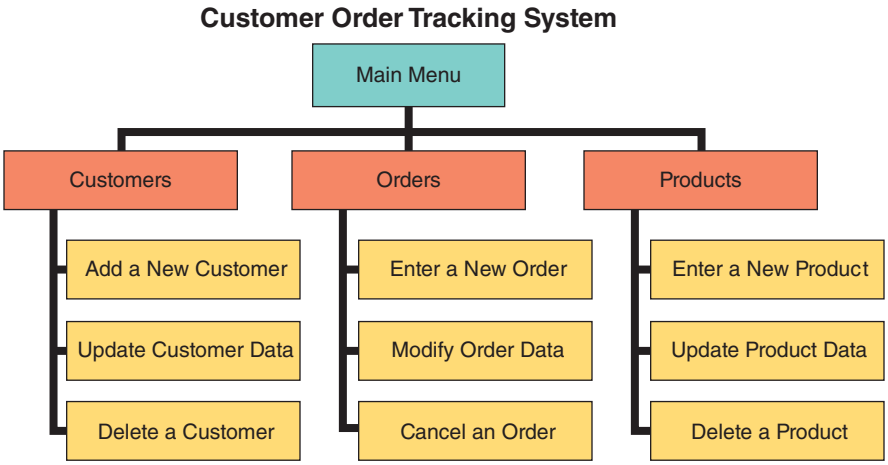
7.4.2 Enhance User Productivity

The interface is where a user interacts with the system, so it can dramatically affect productivity. The user becomes more productive if the interface empowers them and enables them to handle more complex tasks. Conversely, if the interface is difficult to work with, productivity declines.

The following UI design guidelines support productivity enhancements:

- Organize tasks, commands, and functions in groups resembling business operations. Group functions and submenu items in a multilevel menu hierarchy, or tree, that is logical and reflects how users typically perform the tasks. Figure 7-6 shows an example of a menu hierarchy for an order tracking system.

Figure 7-6 This menu hierarchy shows tasks, commands, and functions organized into logical groups and sequences. The structure resembles an FDD.



- Create alphabetical menu lists or place the selections used frequently at the top of the menu list. No universally accepted approach to menu item placement exists. The best strategy is to design a prototype and obtain feedback from users. Some applications even allow menus to show recently used commands first. Some users like that feature, but others find it distracting. The best approach is to offer a choice and let users decide.
- Provide shortcuts for experienced users so they can avoid multiple menu levels. Shortcuts can be created using hotkeys that allow a user to press the Alt key plus the underlined letter of a command.
- Use default values if most values in a field are the same. For example, if 90% of a firm's customers live in Albuquerque, use it as the default value in the City field.
- Use a duplicate value function that enables users to insert the value from the same field in the previous record, but allow them to turn this feature on or off.
- Provide a fast-find feature that displays a list of possible values when users enter the first few letters.
- Consider a **natural language** feature that allows users to type commands or requests in customary text phrases, if available. For example, many applications allow users to request help by typing a question into a dialog box. The software then uses natural language technology to retrieve a list of topics that match the request. Natural language technology is used in speech recognition systems, text-to-speech synthesizers, automated voice response systems, web search engines, text editors, and language instruction materials.

A productivity-optimized interface enhances user satisfaction, as users are more likely to have a positive experience with a system that enables them to work seamlessly and intuitively. Furthermore, in a competitive market, systems that boost user productivity can stand out, offering a distinct advantage by attracting and retaining users who value an efficient and streamlined workflow.

7.4.3 Provide Flexibility

Flexibility within UIs is crucial for accommodating diverse user needs and preferences. Consider a scenario in which a user requires a screen display of all customer balances exceeding $5,000 in an accounts receivable system. One way to approach this is by coding the program to automatically check customer balances against a fixed value of $5,000. This method is straightforward for programmers and users, requiring no additional input or interaction from the user to generate the desired display. It offers simplicity and ease of use, which can be beneficial when user requirements are consistent and unlikely to change. However, this simple approach lacks flexibility and may not cater to the varying needs of different users or changing business requirements.

A more flexible approach would allow users to input the amount themselves. Or, start with a **default value** that displays automatically. This method introduces an element of user control and customization, enabling users to adjust parameters based on their specific needs at any given time. For instance, the system could present a default value of $5,000 and allow users to accept it by pressing ENTER, or they could enter a different amount per their requirement. This design strategy caters to the initial scenario and accommodates various other use cases, making the system more versatile and user-friendly. It empowers users by allowing them to define their parameters, leading to a more personalized and effective user experience.

Incorporating several alternatives and choices within the system design often represents the best strategy for balancing simplicity and flexibility. By offering multiple options, the system can cater to a broader range of user preferences and scenarios, allowing users to select the best approach. This could mean providing default values for quick tasks while also allowing customization for more specific requirements. Such a design philosophy ensures the system is functional and adaptable to its users' diverse and evolving needs. In essence, flexibility in system design is about understanding and accommodating the unique ways in which different users interact with the system, thereby enhancing overall usability and user satisfaction.

7.4.4 Provide Users with Help and Feedback

Designing a sound help system UI is important for several reasons. First, it directly impacts user support and satisfaction, providing users with easy access to assistance and information when they encounter difficulties, which can reduce frustration and improve their overall experience. Second, a well-designed help system empowers users to find solutions independently, potentially reducing the reliance on direct customer support and lowering support costs. Finally, it contributes to the usability of the overall system; a good help system can act as a learning tool, helping users understand and utilize the full range of functionalities offered, thereby enhancing their efficiency and effectiveness in using the product or service.

Help systems should not slow users down because they are intended to aid, not hinder, the user experience. When users seek help, it's often because they've encountered a barrier in their workflow, so the help system needs to provide efficient and effective assistance to minimize disruption and frustration. If the help system is intrusive or slows down the user, it can compound their frustration, negatively impacting their overall experience and productivity. Ideally, help should be easily accessible and available in the background, hidden from those who don't need it. This design approach ensures that users who require assistance can quickly and effortlessly access it without interrupting their workflow.

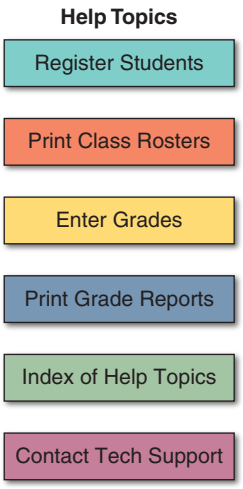


Figure 7-7 The main help screen for a student registration system.

Consider the following when designing a help system UI:

- Ensure that help is always available on demand. Help screens should provide information about menu choices, procedures, shortcuts, and errors.
- Provide user-selected help and context-sensitive help. User-selected help displays information when the user requests it. The user eventually reaches a screen with the desired information by making appropriate choices through the menus and submenus. Figure 7-7 shows the main help screen for the student registration system referenced earlier. **Context-sensitive** help offers assistance for the task in progress.
- Provide a direct route for users to return to where help was requested. Title every help screen to identify the topic and keep the text simple and concise. Insert blank lines between paragraphs to make help text easier to read and provide examples where appropriate.
- Include contact information, such as a phone extension or email address, if a department or help desk is responsible for assisting users.
- Require user confirmation before data deletion ("Are you sure?") and provide a method of recovering data that is deleted inadvertently. Build safeguards that prevent critical data from being changed or erased.
- Provide an "Undo" key or a menu choice allowing the user to undo the most recent command or action results.
- When a user-entered command contains an error, highlight the erroneous part, and allow the user to correct it without retyping the entire command.
- Use hypertext links to assist users as they navigate help topics.
- Display messages at a logical place on the screen and be consistent.
- Alert users to lengthy processing times or delays. Give users an on-screen progress report, particularly if the delay is prolonged.
- Allow messages to remain on the screen long enough for users to read them. In some cases, the screen should display messages until the user takes some action.
- Let the user know whether the task or operation was successful. For example, use messages such as "Update completed," "All transactions have been posted," or "The ID number was not found."
- Provide a text explanation if an icon or image is used on a control button. This can be accomplished using a "mouse hover" to display a pop-up box explaining the icon or image when the mouse is moved over the button.
- Use messages that are specific, understandable, and professional. Avoid cute, cryptic, or vague messages, such as "ERROR—You have entered an unacceptable value" or "Error DE-4-16." Better examples are:
  - Enter a number from 1 (low) to 5 (high)
  - Customer number must be numeric
  - Please reenter a numeric value
  - Call the Accounting Department, ext. 239, for assistance

7.4.5 Create an Attractive Layout and Design

This is subjective because people differ on what is attractive. The analyst should consider color, layout, and ease of use. Screen mock-ups and menu trees can be created and tried on users to get their input. If in doubt, err on the side of doing a bit less. For example, blinking messages initially may seem like a good idea, but there might be better choices for the interface design. Also, avoid too many fonts, styles, and sizes, which can be distracting.

Each separate style should communicate something—a different level of detail, another topic, mandatory versus optional actions, and so on.

Consider the following:

- Use appropriate colors to highlight different screen areas; avoid gaudy and bright colors.
- Use special effects sparingly. For example, animation and sound might be effective in some situations, but too many special effects can be distracting and annoying to a user, especially if they must view the impact repeatedly.
- Use hyperlinks that allow users to navigate to related topics.
- Group related objects and information. Visualize the screen the way a user will see it and simulate the tasks that the user will perform.
- Keep screen displays uncluttered, with enough white space to create an attractive, readable design.
- Display titles, messages, and instructions on all screens consistently and in the same general locations.
- Use consistent terminology. For example, do not use "delete," "cancel," and "erase" to indicate the same action. Similarly, the same sound should always signal the same event.
- Ensure that commands always will have the same effect. For example, if the BACK control button returns a user to the prior screen, the BACK command should always perform that function throughout the application.
- When the user enters data that fills a field, do not move automatically to the next field. Instead, the user must confirm the entry by pressing the Enter or Tab key at the end of every fill-in field.
- Remember that users are accustomed to a pattern of red meaning "stop," yellow indicating caution, and green equaling "go." Stick to that pattern and use it when appropriate to reinforce on-screen instructions.
- Provide a keystroke alternative for each menu command with easy-to-remember letters, such as F for File, X for Exit, and H for Help.
- Use familiar commands such as Cut, Copy, and Paste, if possible.
- Provide a Windows look and feel in the interface design if users are familiar with Windows-based applications.
- Avoid complex terms and technical jargon; instead, select words from everyday business processes and the vocabulary of a typical user.

Creating an attractive layout and design is crucial for several reasons. First, it significantly impacts user engagement and retention; a visually appealing interface draws users in and encourages longer and more frequent interactions with the system. Second, a well-designed layout aids usability, as it helps organize information in an intuitive and accessible manner, making it easier for users to navigate and find what they need. Finally, an attractive design contributes to the overall user experience, creating a positive impression and enhancing the perceived credibility and professionalism of the product or service, which is especially important in competitive markets.

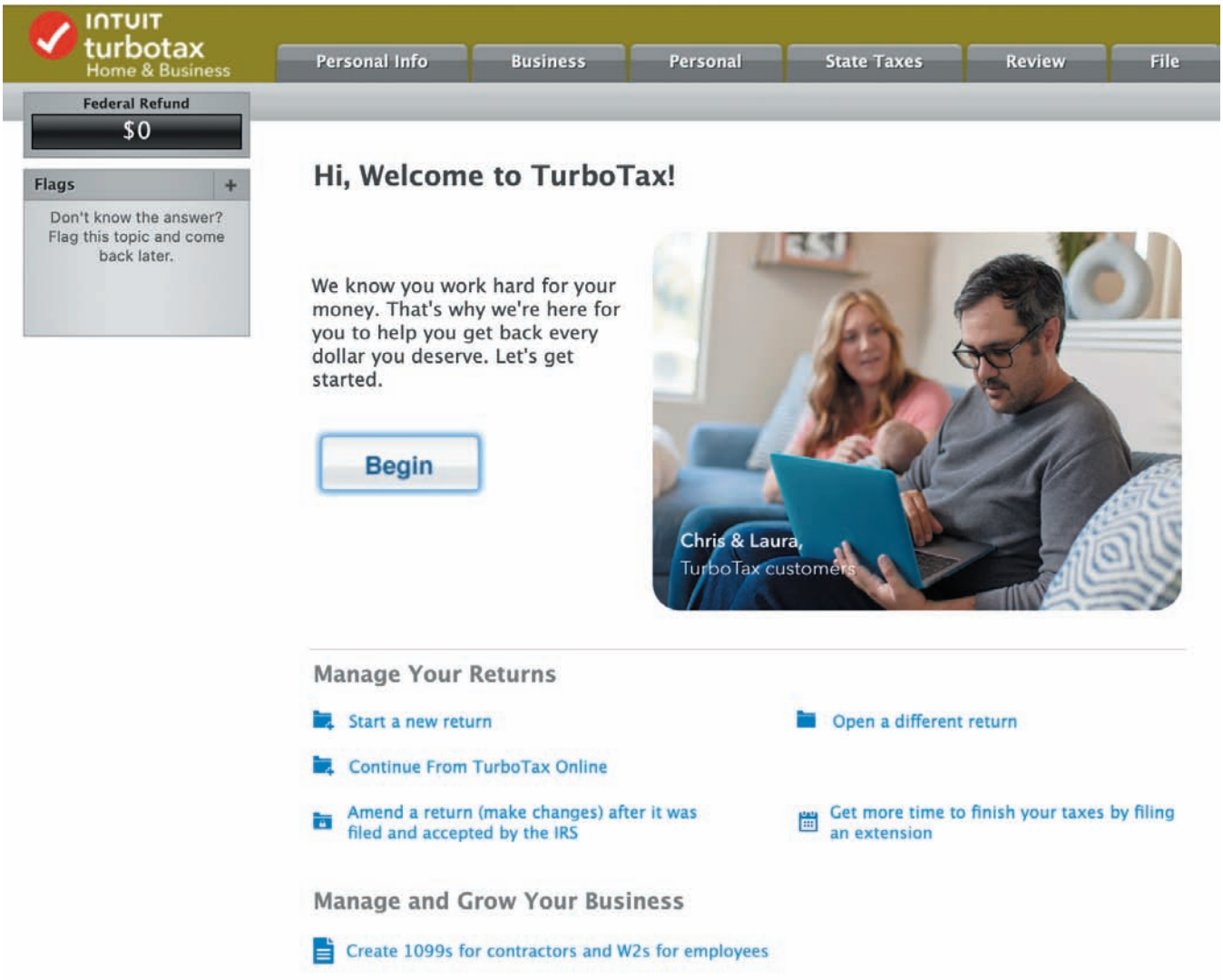
7.4.6 Enhance the Interface

A designer can include many features, such as menu bars, toolbars, dialog boxes, text boxes, toggle buttons, list boxes, scroll bars, drop-down list boxes, option buttons, check boxes, command buttons, and calendar controls. Screen design requires a sense of aesthetics as well as technical skills. User feedback should be obtained early and often as the design process continues.

The UI can be enhanced by keeping the following guidelines in mind during design:

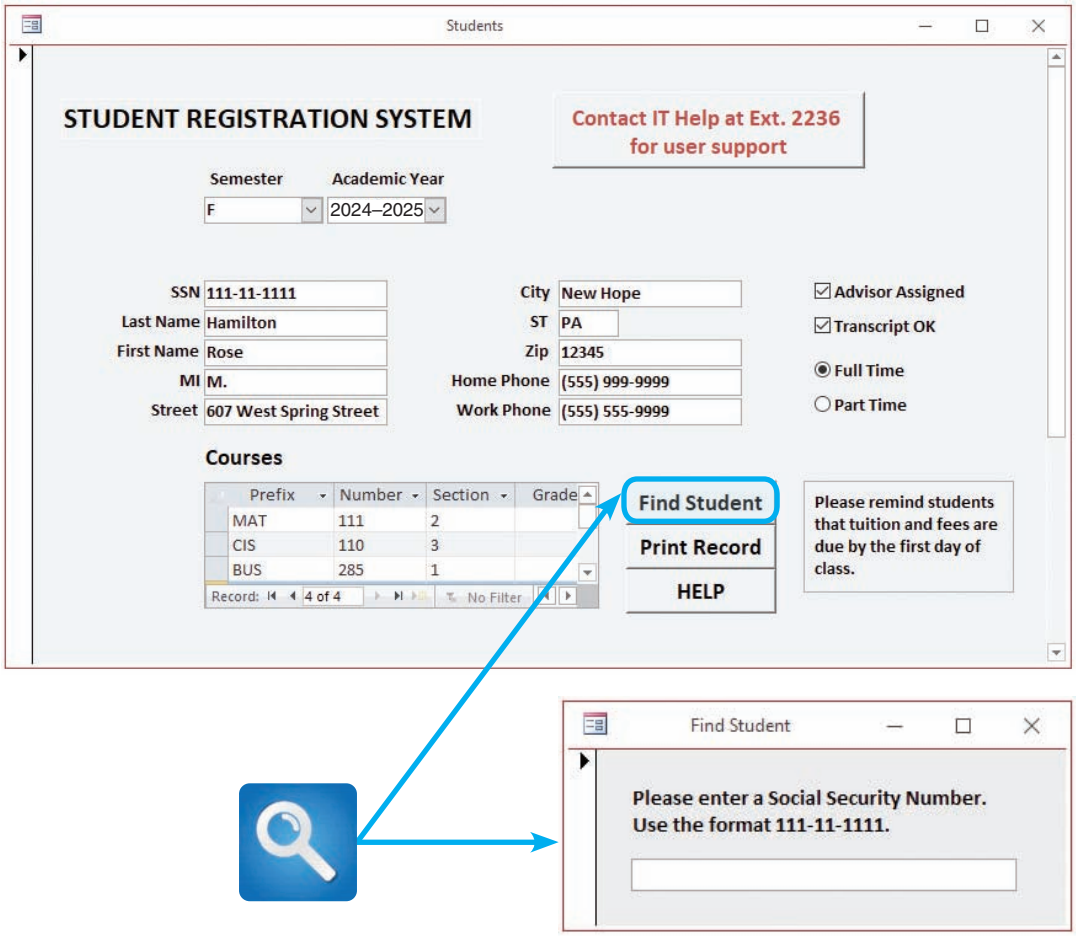
- The opening screen is significant because it introduces the application and lets users view the primary options. The starting point can be a **switchboard** with well-placed command buttons that allow users to navigate the system. Figure 7-8 shows the switchboard of TurboTax, a tax preparation program. The main options are displayed on an uncluttered screen. Adding the picture showing a typical couple preparing their tax returns provides the user a sense of calm and confidence—something vital for users who are likely to be confused and nervous when beginning the tax preparation process.

Figure 7-8 An example of a switchboard introducing TurboTax. The main options are displayed on an uncluttered screen. This is particularly important for users who are likely to be confused and nervous when beginning the tax preparation process.
Source: Intuit TurboTax Home & Business 2023



- Use a **command button** to initiate an action, such as printing a form or requesting help. For example, when a user clicks the Find Student command button in Figure 7-9, a dialog box opens with instructions.
- If a software package is being used, check to see if it allows the creation of customized **menu bars** and toolbars. If so, consider these options.

Figure 7-9 A data entry screen for a student registration system. This screen uses several design features that are described in the text. When a user clicks the Find Student command button, a dialog box is displayed with instructions.



- Add a shortcut feature that lets users select a menu command by clicking the desired choice or pressing the Alt key plus the underlined letter. Some forms also use a **toolbar** that contains icons or buttons representing shortcuts for executing common commands.
- If variable input data is needed, provide a **dialog box** that explains what is required.
- A **toggle button** makes it easy to show on or off status—clicking the toggle button switches to the other state.
- Use **list boxes** that display the available choices. If the list does not fit in the box, a **scroll bar** allows the user to move through the available options. Also, provide another way to enter data that does not align with a specific list choice.
- Use an **option button**, or **radio button**, to control user choices. For example, if the user can select only one option, display a suitable message ("Choose one item"), but if there is no restriction, display a different message ("Choose all that apply"). Use a black dot to show selected options.
- If **check boxes** are used to select one or more choices from a group, show the choices with a checkmark or an X.
- When dates must be entered, use a **calendar control** that allows the user to select a date that the system will use as a field value.

Note that a UI becomes too enhanced and cluttered when it includes excessive features, controls, or information that overwhelm rather than aid the user, leading to a confusing and distracting experience. Overloading a UI with too many elements can make it difficult for users to locate the needed functions, reducing efficiency and increasing the likelihood of errors. The key to effective UI design is balance: offering sufficient functionality and customization options while maintaining simplicity, clarity, and ease of navigation.

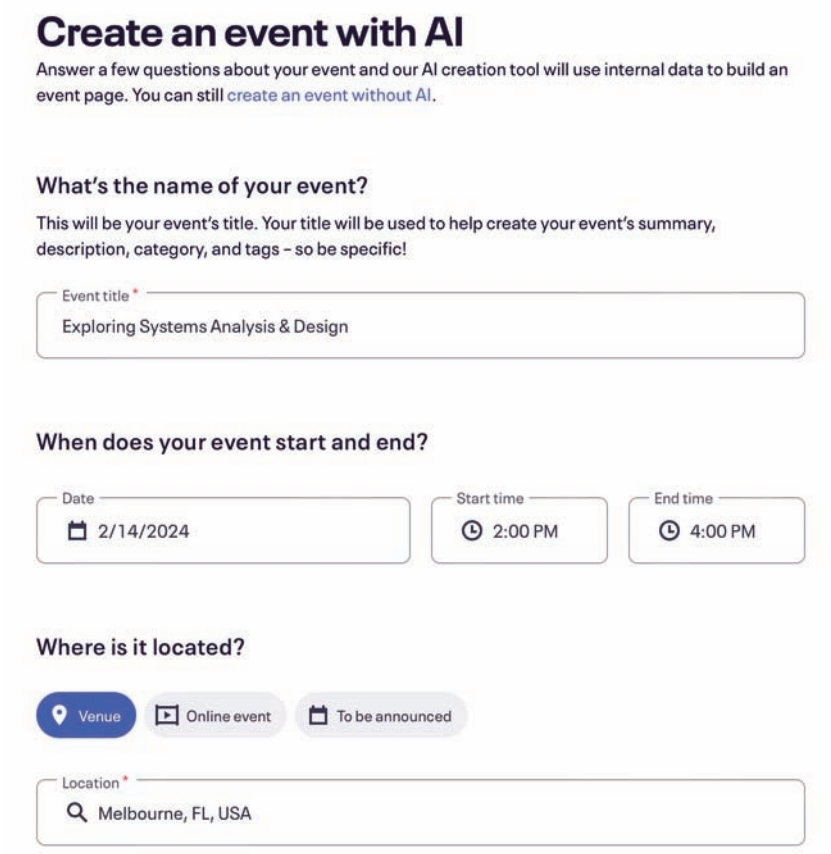
7.4.7 Focus on Data Entry Screens

Data entry is especially important because it is in the job description of so many users. Focusing on data entry screens is vital, as it directly impacts the accuracy and efficiency of data input, which are crucial for maintaining data integrity in critical applications. Focusing on data entry screens is also vital for reducing cognitive load on users, as well-organized and intuitive screens simplify inputting information, allowing users to complete tasks with less mental effort and more focus.

The following guidelines should be kept in mind:

- Whenever possible, use a data entry method called **form filling**, where a blank form that duplicates or resembles the source document is completed on the screen.
- Restrict user access to screen locations where data is entered. For example, in the Eventbrite event management screen shown in Figure 7-10, the system should initially position the insertion point in the first data entry location. After the user enters an event title, the insertion point should move automatically to the entry location for the next field (Date). A user should be able to position the insertion point only in places where data is entered on the form.

Figure 7-10 Eventbrite's artificial intelligence (AI) creation tool uses internal data to build an event page. A red asterisk beside "Event title" indicates a required field. Certain fields are constrained by data type (e.g., "Date").
Source: Eventbrite



- Provide a Cancel button or another way to leave the data entry screen without entering the current record. Because the application shown in Figure 7-10 is web-based, users can exit by selecting the "Back" button of the browser (not shown).
- Provide a descriptive caption for every field and show the user where to enter the data and the required field size. Typically, white boxes show the location and length of each field. Other methods to indicate field locations are video highlighting, underscores, special symbols, or a combination of these features.
- Provide a means for users to move among fields on the form in a standard order or in any order they choose. Users can override the regular field order in a GUI and select field locations using the mouse or arrow keys.
- Allow users to add, change, delete, and view records. Messages such as "Apply these changes? (Y/N)" or "Delete this record? (Y/N)" should require users to confirm their actions. Highlighting the letter *N* as a default response will avoid problems if the user accidentally presses the Enter key.
- Design the screen form layout to match the layout of the source document. If the source document fields start at the top of the form and run down in a column, the input screen should use the same design.
- Display a sample format if a user must enter values in a field in a specific format. For example, provide an on-screen instruction to let users know that the date format is MMDDYY, and give an example if the user must enter separators, such as slashes.
- In addition to the sample format in the preceding rule, it might be better to use an **input mask**, a template or a pattern that restricts data entry and prevents errors. Microsoft Access provides standard input masks for fields such as dates, phone numbers, postal codes, and Social Security numbers, as shown in Table 7-1. In addition, custom input masks can be created, as shown in Table 7-2. A mask can have a specific format. For example, if a user enters text in lowercase letters, the input mask >L<??????????? will capitalize the first letter automatically.
- Do not require users to type leading zeroes for numeric fields. For example, if a three-digit project number is 045, the user should be able to type 45 instead of 045 before pressing the Enter key. An exception to that rule might occur when entering a date, where a leading zero is needed to identify single-digit months or days, such as 06-04-2013.

Table 7-1 Microsoft Access provides input masks for phone numbers, digits, and mixed data.

Source: Microsoft Corporation

This Input Mask	Provides This Type of Value	Notes
(000) 000-0000	(206) 555-0199	In this case, you must enter an area code because that section of the mask (000, enclosed in parentheses) uses the 0 placeholder.
(999) 000-0000!	(206) 555-0199 ( ) 555-0199	In this case, the area code section uses the 9 placeholder, so area codes are optional. Also, the exclamation point (!) causes the mask to fill in from left to right.
(000) AAA-AAAA	(206) 555-TELE	Allows you to substitute the last four digits of a U.S.-style phone number with letters. Note the use of the 0 placeholder in the area code section, which makes the area code mandatory.
#999	-20 2000	Any positive or negative number, no more than four characters, and with no thousands separator or decimal places.
>L????L?000L0	GREENGR339M3 MAY R 452B7	A combination of mandatory (L) and optional (?) letters and mandatory numbers (0). The greater-than sign forces users to enter all letters in uppercase. To use an input mask of this type, you must set the data type for the table field to **Text** or **Memo**.

Table 7-2 Custom character input masks can be created in Microsoft Access.

Source: Microsoft Corporation

Character	Explanation
0	User must enter a digit (0 to 9).
9	User can enter a digit (0 to 9).
#	User can enter a digit, space, or plus or minus sign. If skipped, Access enters a blank space.
L	User must enter a letter.
?	User can enter a letter.
A	User must enter a letter or a digit.
a	User can enter a letter or a digit.
&	User must enter either a character or a space.
C	User can enter characters or spaces.

- Do not require users to type trailing zeroes for numbers that include decimals. For example, when a user types a 98, the system should interpret the value as 98.00 if the field has been formatted to contain numbers with two decimal places. The decimal point is needed only to indicate nonzero decimal places, such as 98.76.
- Display default values so users can press the Enter key to accept the suggested value. If the default value is not appropriate, the user can change it.
- Use a default value when a field value will be constant for successive records or throughout the data entry session. For example, if records are input in order by date, the date used in the first transaction should be the default date until a new date is entered; then, the new date becomes the default value.
- Display a list of acceptable field values and provide meaningful error messages if the user enters an unacceptable value. An even better method is to provide a drop-down list box containing acceptable values that allow the user to select a value by clicking it.
- Provide users with an opportunity to confirm the accuracy of input data before entering it by displaying a message such as "Add this record? (Y/N)." A positive response (Y) adds the record, clears the entry fields, and positions the insertion point in the first field so the user can input another record. If the response is negative (N), the current record is not added, and the user can correct the errors.

Data entry screens should also anticipate future needs. Consider a parts inventory database containing a one-character category field, such as E for electrical, M for mechanical, or H for hydraulic. The design works well, but what if the company breaks these overall groups into more specific segments? A better design would anticipate possible expansion to two or more characters. For example, in 1999, there was widespread concern about the Y2K issue because many older programs used only two characters to store the year and would not recognize the start of the new century (and millennium).

7.4.8 Use Validation Rules

Reducing input errors improves data quality. One way to reduce input errors is to eliminate unnecessary data entry. For example, a user cannot misspell a customer name if it is entered automatically based on the user entering the customer ID. Similarly, an outdated item price cannot be used if the item price is retrieved from a master file instead of being entered manually. The best defense against incorrect data is identifying and correcting errors using data validation rules before entry. Table 7-3 shows a **data validation rule** that improves input

quality by testing the data and rejecting any entry that fails to meet specified conditions. The design can include at least eight types of data validation rules. Microsoft Access also provides validation rule operators for common tests, as shown in Table 7-4.

Table 7-3 Microsoft Access provides validation rules that can improve data quality by requiring the input to meet specific requirements or conditions.

Source: Microsoft Corporation

Validation Rule	Validation Text
<>0	Enter a nonzero value.
>=0	Value must be zero or greater. or You must enter a positive number.
0 or >100	Value must be either 0 or greater than 100.
BETWEEN 0 AND 1	Enter a value with a percent sign (for use with a field that stores number values as percentages).
<#01/01/2007#	Enter a date before 2007.
>=#01/01/2007# AND <#01/01/2008#	Date must occur in 2007.
<Date()	Birth date cannot be in the future.

Table 7-4 Microsoft Access provides validation rule operators for common tests.

Source: Microsoft Corporation

Operator	Function	Example
NOT	Tests for converse values. Use before any comparison operator except IS NOT NULL.	**NOT > 10** (the same as **<=10**)
IN	Tests for values equal to existing members in a list. Comparison value must be a comma-separated list enclosed in parentheses.	**IN ("Tokyo","Paris","Moscow")**
BETWEEN	Tests for a range of values. You must use two comparison values—low and high—and you must separate those values with the AND separator.	**BETWEEN 100 AND 1000** (the same as **>=100 AND <=1000**)
LIKE	Matches pattern strings in Text and Memo fields.	LIKE "Geo*"

The following data validation checks are beneficial to overall UI design:

- A **sequence check** can be used when the data must be in some predetermined sequence. Suppose the user must enter work orders in a numerical sequence; in that case, an out-of-sequence order number indicates an error. If the user must enter transactions chronologically, then a transaction with an out-of-sequence date means a mistake.
- An **existence check** can apply to mandatory data items. For example, if an employee record requires a Social Security number, an existence check would only allow users to save it once they enter a suitable value in the Social Security number field.
- A **data type check** can test to ensure that a data item fits the required data type. For example, a numeric field must have only numbers or numeric symbols, and an alphabetic field can contain only the characters A through Z (or a through z).

- A **range check** can be used to verify that data items fall between a specified minimum and maximum value. The daily hours worked by an employee, for example, must fall within the range of 0 to 24. This test is called a **limit check** when the validation check involves a minimum or a maximum value but not both. Checking that a payment amount exceeds zero but not specifying a maximum value is an example of a limit check.
- A **reasonableness check** identifies values that are questionable but not necessarily wrong. For example, input payment values of $.05 and $5,000,000.00 pass a simple limit check for a payment value greater than zero, yet both values could be errors. Similarly, a daily-hours-worked value of 24 passes a 0 to 24 range check; however, the value seems unusual, and the system should verify it using a reasonableness check.
- A **validity check** can be used for data items with specific values. For example, if an inventory system has 20 valid item classes, then any input item that does not match one of the valid classes will fail the check. Verifying that a customer number on an order matches a customer number in the customer file is another validity check. Because the value entered must refer to another value, that type of check is also called referential integrity, which is explained later in the book. Another validity check might verify that a new customer number does not match a number already stored in the customer master file.
- A **combination check** is performed on two or more fields to ensure they are consistent or reasonable. Even though all the fields involved in a combination check might pass their validation checks, the combination of the field values might be inconsistent or unreasonable. For example, if an order input for 30 units of a particular item has an input discount rate applicable only for purchases of 100 or more units, then the combination is invalid; either the input order quantity or the input discount rate is incorrect.

Batch controls are totals used to verify batch input. They can check data items such as record counts and numeric field totals. For example, before entering a batch of orders, a user might calculate the total number of orders and the sum of all the order quantities. When the batch of orders is entered, the order system calculates the same two totals. A data entry error occurs if the system totals do not match the input totals. Unlike the other validation checks, batch controls do not identify specific errors. For example, if the sum of all the order quantities does not match the batch control total, the only thing known is that one or more orders in that batch were entered incorrectly or not input. The batch control totals are often called **hash totals** because they are not meaningful numbers but are helpful for comparison purposes.

7.4.9 Manage Data Effectively

In addition to its effect on users, data management impacts company efficiency, productivity, and security. To reduce input errors, data should be entered and verified as soon as possible, and each data item should have a specific type, such as alphabetic, numeric, or alphanumeric, and a range of acceptable values.

It is important to collect input data as close to its source as possible—for instance, using barcode scanners rather than manual forms on a warehouse freight dock or having salespeople use tablets to record orders rather than filling in source documents. Automated data capture is the easiest, most accurate, and least expensive data input strategy.

In an efficient design, data is entered only once. For example, if input data for a payroll system is also needed for a human resources system, the analyst could design an interface to transfer data automatically, or a central data storage area could be created that both systems can access. Normalization (described later in the book) is a set of rules that can help avoid data design problems. A secure system also includes **audit trails** that can log every instance of data entry and changes. For example, the system should record when a customer's credit limit was set, by whom, and any other information necessary to construct the history of a transaction.

7.4.10 Reduce Input Volume

Reducing input volume is crucial because it streamlines the data entry process, making it faster and less cumbersome for users. This efficiency saves time and minimizes the likelihood of errors, which are more common with extensive or repetitive data entry tasks. Therefore, the analyst should reduce the required data items for each transaction.

Consider the following guidelines to accomplish this task:

- Only input a data item if the system needs it. A completed order form, for example, might contain the name of the clerk who took the order. If the system does not require that data, the user should not enter it.
- Do not input data the user can retrieve from system files or calculate from other data. This reduces input errors and data inconsistencies.
- Automate repetitive data entry where possible. Automation can significantly reduce the volume of manual input for fields that often contain the same value or a predictable pattern of values. For instance, if a user enters multiple records in which a particular field (like a department name or a location) remains constant, the system can auto-fill that field based on the first entry or a set pattern. This speeds up data entry and minimizes the risk of errors that can occur with repetitive manual entry.
- Do not input constant data. If orders are in batches with the same date, then a user should enter the order date only once for the first order in the batch. If orders are entered online, the user can retrieve the order date automatically using the current system date.
- Use codes. Codes are shorter than the data they represent, and coded input can reduce data entry time. Codes are discussed in more detail later in the book.

This is the final guideline, but it affects all the rest. Reducing input volume in UI design enhances simplicity and clarity, aligning with guidelines that emphasize creating user-friendly and intuitive interfaces. This streamlines tasks and minimizes users' time on data entry, allowing them to focus on more critical aspects of their work.

Case in Point 7.2

Boolean Toys

Suppose you are a systems analyst studying the order processing system at Boolean Toys, a fast-growing software developer for preschool children. You know that many data entry users have complained about the input screens. Some users would prefer to rearrange the order of the fields, others would like to change the background color on their screens, and others want shortcuts that would allow them to avoid a series of introductory screens.

What if Boolean users could customize their data entry screens without assistance from the IT staff by using a menu-driven utility program? What would be the pros and cons of such an approach? When should a systems analyst decide on a design issue, and when should users be allowed to select what works best for them?

7.5 Source Document and Form Design

No matter how data enters an information system, the output quality is only as good as the quality of the input. The term **garbage in, garbage out (GIGO)** is familiar to IT professionals, who know that the best time to avoid problems is when the data is entered. The main objective is to ensure input data quality, accuracy, and timeliness. Unfortunately, the "paperless office" dream has never been realized. Even with RFID technology and automated data capture, we still enter data on source documents and forms. Instead of a human–computer interface, systems analysts must deal with the challenge of a human–paper interface.

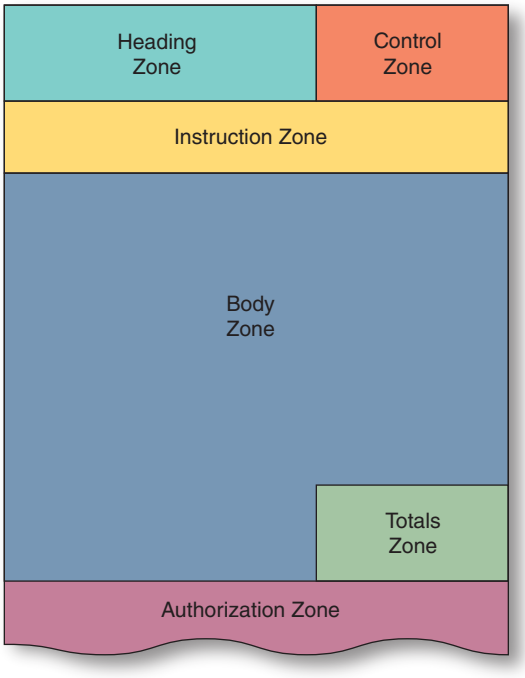
A **source document** collects input data, triggers or authorizes an input action, and provides a record of the original transaction. During the input design stage, the analyst develops source documents that are easy to complete and use for data entry. Source documents are generally paper-based but also can be provided online. Either way, the design considerations are the same.

Consider a time when completing a poorly designed form was a struggle. There might have been insufficient space, confusing instructions, or poor organization—all symptoms of incorrect **form layout**.

A good form layout makes the form easy to complete and provides enough vertical and horizontal space for users to enter the data. A form should indicate data entry positions using blank lines or boxes and descriptive captions. Also, consider using check boxes whenever possible so users can select choices easily. However, be sure to include an option for any input that does not match a specific check box.

The placement of information on a form is also important. Source documents typically include most of the zones shown in Figure 7-11. The heading zone usually contains the company name or logo and the title and number of the form. The control zone contains codes, identification information, numbers, and dates for storing completed forms. The instruction zone includes instructions for completing the form. The central part of the form, called the body zone, usually takes up at least half of the space on the form and contains captions and areas for entering variable data. If totals are included on the form, they appear in the **totals zone**. Finally, the **authorization zone** has any required signatures.

Figure 7-11 Source document zones.



Information should flow on a form from left to right and top to bottom to match the way users read documents naturally. That layout makes the form easy for the person who completes the form and for users who enter data into the system using the completed form.

The same user-friendly design principles apply to printed forms such as invoices and monthly statements, except that heading information is usually preprinted. Column headings should be short but descriptive, avoiding nonstandard abbreviations, with reasonable spacing between columns for better readability.

The order and placement of printed fields should be logical, and totals should be identified clearly. When designing a preprinted form, contact the vendor for advice on paper sizes, type styles and sizes, paper and ink colors, field placement, and other essential form details. The goal is to design an attractive, readable, and practical form.

Layout and design are also important in web-based forms. Many resources can help with designing efficient, user-friendly forms. These include websites that must conform to the U.S. Government's accessibility guidelines, available at http://www.section508.gov.

7.6 Printed Output

Before designing printed output, there are several questions to consider:

- Why is this delivered as printed output rather than screen-based information, allowing users to view, print, or save it as needed?
- Who wants the information, why is it needed, and how will it be used?
- What specific information will be included?
- Will the printed output be designed for a specific device?

- When and how will the information be delivered, and how often must it be updated?
- Do security or confidentiality issues exist? How will they be managed?

The design process should begin once these questions have been answered. Some information was gathered during the systems analysis phase. To gain a better understanding, the analyst should meet with users to determine their desired output. Prototypes and mock-ups can be used to obtain feedback throughout the design process.

7.6.1 Report Design

Although many organizations strive to reduce the flow of paper and printed reports, only some firms have been able to eliminate printed output. Because they are portable, printed reports are convenient and even necessary in some situations. Many users find it handy to view screen output and print the information they need for a discussion or business meeting. The printed output is also used in **turnaround documents** in the same or another information system. In some places, the phone or utility bill, for example, might be a turnaround document printed by the company's billing system. When the required portion of the bill is returned with payment, the bill is scanned into the company's accounts receivable system to record the payment accurately.

Designers use a variety of styles, fonts, and images to produce attractive and user-friendly reports. Whether printed or viewed on-screen, reports must be easy to read and well organized. Rightly or wrongly, some managers judge an entire project by the quality of the reports they receive.

Database programs like Microsoft Access include various report design tools, including a Report Wizard, a menu-driven feature designers can use to create reports quickly and easily. Many online web-based database systems also provide similar report design guidelines.

Although most reports are designed graphically, some systems still produce one or more **character-based reports** that use a character set with fixed spacing. Printing character-based reports on high-speed impact printers is a fast, inexpensive method for producing large-scale reports, such as payroll, inventory reports, or registration rosters at a school. This is especially true if multiple copies are required.

Users should approve all report designs in advance. The best approach is to prepare a sample report, a **mock-up** or prototype, for users to review. The sample should include typical field values and contain enough records to show all the design features. Depending on the type of printed output needed, a Microsoft Word document can be created, or a report generator can be used to generate mock-up reports.

7.6.2 Report Design Principles

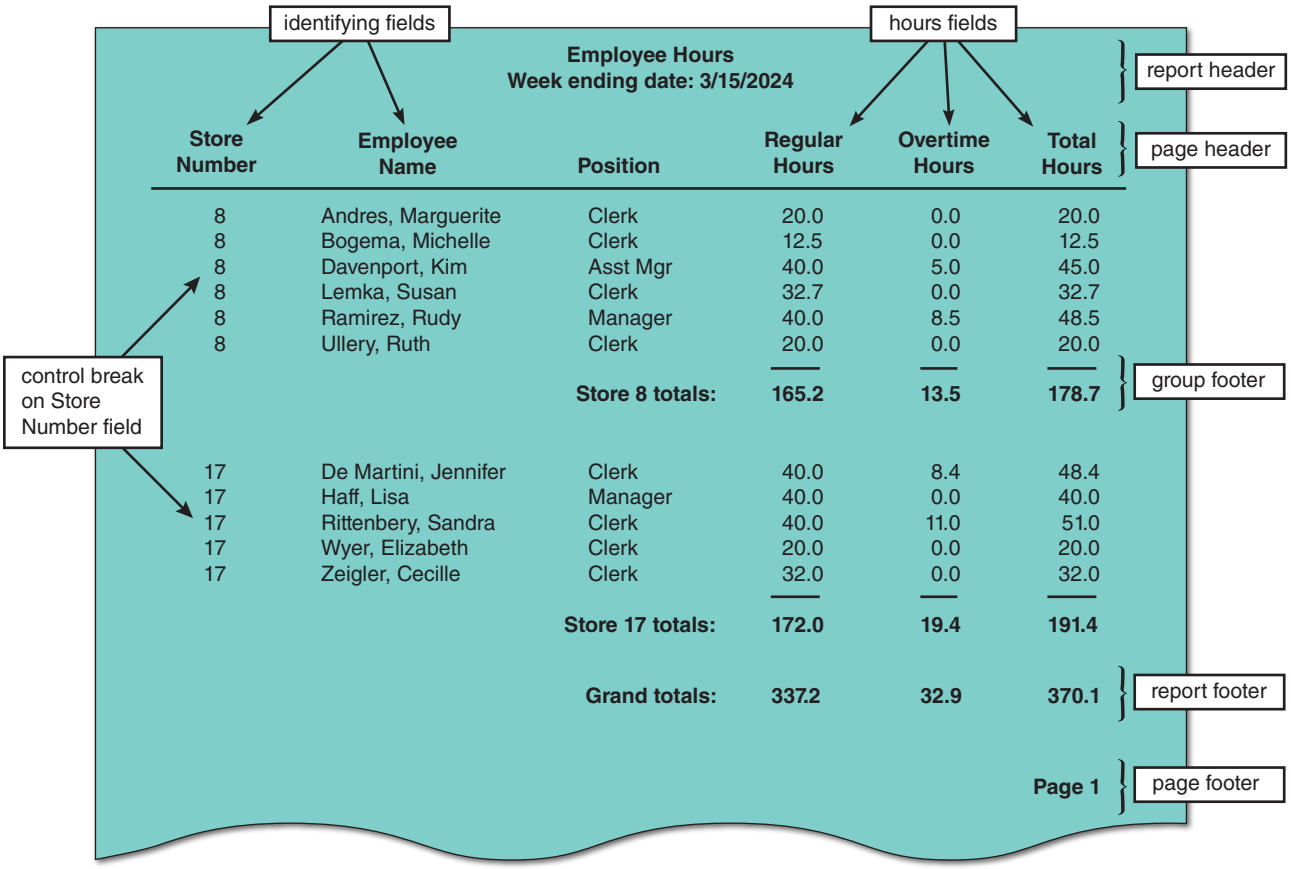
Printed reports must be attractive, professional, and easy to read. For example, a well-designed report should provide totals and subtotals for numeric fields. In the report shown in Figure 7-12, notice that when the value of a control field such as Store Number changes, a **control break** occurs. A control break usually causes specific actions, such as printing subtotals for a group of records. That type of detail report is called a **control break report**. To produce a control break report, the records must be arranged, or sorted, in **control field order**.

Good report design requires effort and attention to detail. To produce a well-designed report, the analyst must consider design features such as report headers and footers, page headers and footers, column headings and alignment, column spacing, field order, and grouping of detail lines.

Report Headers and Footers

Every report should have a report header and footer. The **report header**, which appears at the beginning of the report, identifies the report and contains the report title, date, and other necessary information. The **report footer**, which appears at the end of the report, can include grand totals for numeric fields and additional end-of-report details, as shown in Figure 7-12.

Figure 7-12 The Employee Hours report is detailed with control breaks, subtotals, and grand totals. A report header identifies the report, a page header contains column headings, a group footer includes subtotals for each store, and a report footer contains grand totals. A page footer identifies the page number.



Page Headers and Footers

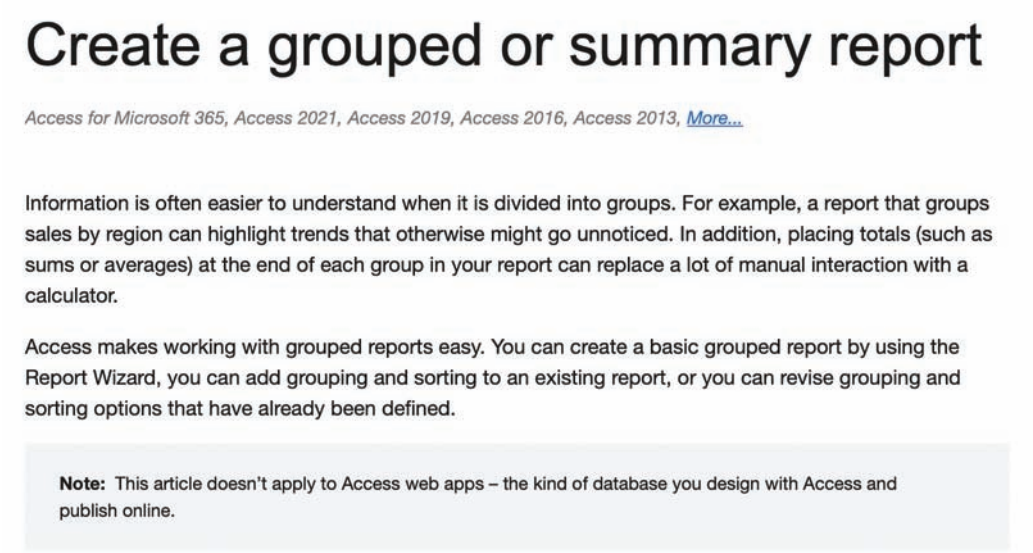
Every page should include a **page header**, which appears at the top and consists of the column headings identifying the data. Titles should be short but descriptive. Avoid abbreviations unless the users will understand them clearly. A page header or a **page footer**, which appears at the bottom of the page, is used to display the report title and the page number.

Database programs like Microsoft Access make it easy to create groups and subgroups based on particular fields. The report can also calculate and display totals, averages, record counts, and other data for any group or subgroup. For example, a large company might want to see total sales and number of sales broken down by product within each of the 50 states. The information shown in Figure 7-13 is part of Access's online help, which refers to a step-by-step process for creating multilevel grouping.

Repeating Fields

Report design is an art, not a science. User involvement is essential, but users often don't know what they want without seeing samples. For example, consider the issue of repeating fields. The sample report in Figure 7-12 repeats the store number on every row. Is that a good thing? The best advice is to ask users what they think and be guided accordingly. A similar issue exists for the overtime hours column. Is it better to print the zero-overtime data or only print actual hours so the data stands out clearly? Again, the preferred answer is usually the one that works best for users.

Figure 7-13 Microsoft Access includes an easy-to-use tool for grouping data.
Source: Microsoft Corporation



Consistent Design

Look and feel are important to users, so reports should be uniform and consistent. When a system produces multiple reports, each should share common design elements. For example, the date and page numbers should be printed in the same place on each report page. Abbreviations used in reports also should be consistent. For example, when indicating a numeric value, it is confusing for one report to use #, another *NO*, and a third *NUM*. Items in a report also should be consistent. If one report displays the inventory location as a shelf number column followed by a bin number column, that same layout should be used on all inventory location reports.

7.6.3 Types of Reports

To be helpful, a report must include the information a user needs. From a user's point of view, a report with too little information is of no value. Too much information, however, can make a report confusing and challenging to understand. When designing reports, the essential goal is to match the report to the user's specific information needs. Depending on their job functions, users might need one or more of the reports described in the following sections.

Detail Reports

A **detail report** produces one or more lines of output for each record processed. A detail report can be lengthy because it contains one or more lines for each record. Consider, for example, a large auto parts business. If the firm stocks 3,000 parts, the detail report would include 3,000 detail lines on approximately 50 printed pages. A user who wants to locate any part in short supply has to examine 3,000 detail lines to find the critical items. A better alternative might be an exception report.

Exception Reports

An **exception report** displays only those records that meet specific conditions. Exception reports are helpful when the user wants information only on records that might require action but does not need to know the details. For example, a credit manager might use an exception report to identify only those customers with past-due accounts, or a customer service manager might want a report on all packages not delivered within a specified period.

Summary Reports

Upper-level managers often want to see total figures and do not need supporting details. A sales manager, for example, might want to know the total sales for each sales representative but not want a detailed report listing every sale they made. In that case, a **summary report** is appropriate. Similarly, a personnel manager might need to know the total regular and overtime hours worked by employees in each store but might not be interested in the number of hours worked by each employee.

Case in Point 7.3

Lazy Eddie

Lazy Eddie is a furniture chain specializing in recliners. Their management has asked you to review the large number of printed reports distributed to Lazy Eddie's 35 store managers. Management is unsure that the store managers read or use the reports, even though they say they want them. Store visits have shown many of the reports end up stacked on top of filing cabinets, seemingly untouched.

To determine if store managers use printed reports, management has asked you to create a procedure requiring users to review and justify their information needs. You could design a form asking if the information is still required and why. You could get users to decide if a report is worth the cost of producing it. How do you proceed?

7.7 Technology Issues

Unlike early innovations such as the mouse and the inkjet printer, most technology advances today affect both input and output. This shift reflects a growing interdependence between input and output in the context of UIs. Today, it's challenging to consider changes in either input or output without acknowledging their effect on the other. This interplay is evident in various technological advancements where the evolution of one aspect directly influences the other. For instance, developments in touch-screen technology necessitate corresponding changes in the output, such as UI design and display characteristics. This is particularly crucial for devices of varying sizes and functionalities ranging from smartphones to large desktop monitors. Each device demands a tailored approach to input and output to ensure optimal usability and performance.

The subsequent sections delve into input and output technologies separately for analytical clarity. However, interface designers must know how closely linked these two aspects are. Input/output linkage is vital when designing a UI. Every decision in the design process can have implications for both input and output, affecting how users interact with and experience the system. For instance, the choice of a particular input method, like touch or voice commands, directly impacts the design of the output, such as how information is displayed or how interactive elements are arranged on the screen.

This interdependence underscores the need for a holistic approach in interface design, where designers must constantly look for opportunities and challenges arising from technology's evolving nature. Being alert to the potential impacts of input/output linkage allows designers to innovate and adapt, ensuring that the interfaces they create are technologically advanced but also user-friendly and effective. This approach is crucial in a landscape where technology is rapidly evolving, and the line between input and output is increasingly blurred. Understanding and leveraging this relationship is critical to designing interfaces that meet the needs of modern users and stand the test of time.

7.7.1 Input Technology

Input technology has changed dramatically in recent years. In addition to traditional devices and methods, there has been a rapid expansion of new hardware and ways to capture and enter data into a system, some of which are shown in Table 7-5. Businesses use the latest technology to speed up the input process, reduce costs, and capture data in new forms, such as digital signatures.

Table 7-5 Input devices can be traditional or based on the latest technology.

INPUT TECHNOLOGY		
Traditional	**Evolving**	**Emerging**
Keyboard	Body motion detection	BCI (brain–computer interface)
Mouse	Voice recognition	Neural networks
Pointing devices	Biological feedback	AI
Microphone	Embedded magnetic data	Advanced motion sensors
OCR (optical character recognition)	RFID (radio frequency identification)	Two-way satellite interface
MICR (magnetic ink character recognition)	Optical recognition	AR/VR (augmented reality and virtual reality) environments
Graphic input devices	Physical adaptation devices	3D technology

Input methods should be cost-efficient, timely, and as simple as possible. Systems analysts study transactions and business operations to determine how and when data should enter the system. Usually, the first decision is whether to use batch or online input methods. Each method has advantages and disadvantages, and the systems analyst must consider the following factors.

Batch Input

Using **batch input**, data entry is usually performed on a specified schedule, such as daily, weekly, monthly, or longer. For example, batch input occurs when a payroll department collects timecards at the end of the week and enters the data as a **batch**. Another example is a school that enters all grades for the academic term in a set.

Online Input

Although batch input is used in specific situations, most business activity requires **online data entry**. The online method offers significant advantages, including immediate validation and data availability. A popular online input method is **source data automation**, which combines online data entry and automated data capture using input devices such as **RFID (radio frequency identification) tags**, magnetic data strips, or even smartphones. Source data automation is fast and accurate and minimizes human involvement in translation.

Many large companies use a combination of source data automation and a powerful communication network to manage global operations instantly. Some common examples of source data automation are:

- Businesses that use point-of-sale (POS) terminals equipped with barcode scanners and magnetic swipe scanners to input credit card data
- ATMs that read data strips on bank cards

- Factory employees who use magnetic ID cards to clock on and off specific jobs so the company can track production costs accurately
- Hospitals that imprint barcodes on patient identification bracelets and use portable scanners when gathering data on patient treatment and medication
- Retail stores that use portable barcode scanners to log new shipments and update inventory data
- Libraries that use handheld scanners to read optical strips on books

Trade-Offs

Although online input offers many advantages, it has disadvantages. For example, manual data entry is slower and more expensive than batch input unless source data automation is used because manual entry is performed when the transaction occurs and is often done when computer demand is at its highest.

The decision to use batch or online input depends on business requirements. For example, hotel reservations must be entered and processed immediately, but hotels can enter their monthly performance figures in a batch. Some input occurs naturally in batches. A cable TV provider, for example, receives customer payments in batches when the mail arrives.

7.7.2 Output Technology

Although business information systems still provide most output as screen displays and printed matter, technology enormously impacts how people communicate and obtain information. This trend is significant to firms using IT to lower costs, improve employee productivity, and communicate effectively with customers.

In addition to screen output and printed matter, output can be delivered in many ways. The system requirements document probably identifies a company's user output needs. Now, in the systems design phase, the analyst creates the actual forms, reports, documents, and other types of output that might be accessed from workstations, notebooks, tablets, smartphones, and other devices. How the information will be used, stored, and retrieved must also be considered. The following subsections explain various output types and technologies.

Internet-Based Information Delivery

Millions of firms use the Internet to reach new customers and markets worldwide. To support the explosive growth in e-commerce, web designers must provide user-friendly screen interfaces that display output and accept customer input. For example, a business can link its inventory system to its website so the output from the inventory system is shown as an online catalog. Customers visiting the site can review the items, obtain current prices, and check product availability.

Another example of web-based output is a system that provides customized responses to product or technical questions. When a user enters a product inquiry or requests technical support, the system responds with appropriate information from an on-site knowledge base. Web-based delivery allows users to download a universe of files and documents to support their information needs. For example, the web provides consumers instant access to brochures, product manuals, and parts lists; prospective homebuyers can obtain instant quotes on mortgages, insurance, and other financial services.

Companies also use a live or prerecorded **webcast**, an audio or video media file distributed over the Internet, to reach prospective customers and investors. Radio and TV stations also use this technique to broadcast program material to their audiences.

Email

Email is an essential means of internal and external business communication. Employees send and receive email on local or wide area networks, including the Internet. Companies send new product information to customers via email, and financial services companies use email messages to confirm online stock trades. Employees use email to exchange documents, data,

and schedules and to share business-related information they need to perform their jobs. In many firms, email has virtually replaced traditional memos and printed correspondence.

Blogs

Web-based logs, called **blogs**, are another form of web-based output. Because blogs are journals written from a particular point of view, they not only deliver facts to web readers but also provide opinions. Blogs help post news, review current events, and promote products.

Instant Messaging

This widespread form of communication is another way for individuals and companies to communicate effectively over the Internet. Although some users feel it can be a distraction, others like the constant flow of communication, especially as team members in a collaborative situation.

Wireless Devices

Messages and data can be transmitted to various mobile devices, including tablet computers, smartphones, and similar wireless products that combine portable computing power, multimedia capability, and Internet access.

Digital Audio, Images, and Video

Sounds, images, and video clips can be captured, stored in digital format, and transmitted as output to users who can reproduce the content. Audio or video output can be attached to an email message or inserted as a clip in a Microsoft Word document. Businesses also use automated systems to handle voice transactions and provide customer information. For example, using a phone keypad, a customer can confirm an airline seat assignment, check a credit card balance, or determine the current price of a mutual fund.

If a picture is worth a thousand words, digital images and video clips are high-value output types that offer a whole new dimension. For example, an insurance adjuster with a digital camera phone can take a picture, submit the image via a wireless device, and receive immediate authorization to pay a claim on the spot. Images are a valuable output form, but video clips are even better in some situations. For example, video clips provide online virtual tours that allow real estate agents to show off the best features of the homes they are marketing. The user can zoom in or out and rotate an image in any direction.

Automated Fax Systems

An **automated fax** or **faxback** system allows customers to request a fax via email, company website, or phone. The response is transmitted back to the user's fax machine in seconds. Although most users prefer to download documents from the web, many established organizations still offer an automated faxback service to provide immediate response 24 hours a day to a certain set of customers. Some industries, such as drug stores, doctors' offices, insurance companies, and real estate brokers, still rely on fax machines to communicate.

Podcasts

A **podcast** is a specially formatted digital audio file that Internet users can download from various content providers. Many firms use podcasts as sales and marketing tools to communicate with their employees. Using software such as iTunes, users can receive a podcast, launch the file on their computer, and store it on their portable player. Podcasts can include images, sounds, and video.

Computer Output to Digital Media

This process is used when many paper documents must be scanned, stored in digital format, and retrieved quickly. For example, suppose an insurance company holds thousands of paper application forms. In that case, special software can treat the documents as data and extract information from a particular column or area on the form. Modern digital storage media include solid-state drives (SSDs), USB flash drives, Blu-ray discs, network-attached storage (NAS) systems, and cloud storage services.

Specialized Forms of Output

An incredibly diverse marketplace requires many forms of specialized output and devices. For example:

- Portable, web-connected devices that can run multiple apps, handle multimedia output, and provide robust, multipurpose communication for users
- Retail point-of-sale terminals that handle computer-based credit card transactions, print receipts, and update inventory records
- ATMs that can process bank transactions and print deposit and withdrawal slips
- Special-purpose printers that can produce labels, employee ID cards, driver's licenses, gasoline pump receipts, and, in some states, lottery tickets
- Plotters that can produce high-quality images such as blueprints, maps, and electronic circuit diagrams
- Electronic detection of data embedded in credit cards, bank cards, and employee ID cards

7.7.3 Input/Output Technology Linkage

The concept of linkage between input and output in UI design is a critical factor in creating a cohesive and intuitive user experience. This linkage is the harmonious integration of how users input data or commands into a system and how the system subsequently presents information or feedback to the users. In modern technology, where there are diverse modes of interaction, understanding and effectively designing this linkage is paramount. For instance, in a touch-based interface, how users interact with the screen (input) directly influences the design of visual elements such as buttons, menus, and other interactive components (output). The output must be responsive and tailored to the input method, ensuring elements are visually appealing and functionally suitable for touch interactions.

Moreover, this linkage is not static; it evolves with technological advancements and changing user behaviors. As new input methods emerge, such as voice control or gesture recognition, the output design must adapt accordingly. Voice-controlled systems, for example, rely heavily on auditory output rather than visual output, requiring a different design approach. The interface must provide clear and concise auditory feedback that guides the user through voice commands, making the interaction as natural and efficient as possible. Similarly, for gesture-based inputs, the output needs to be designed to intuitively reflect the user's actions, providing real-time, relevant feedback that aligns with the gestures made. This dynamic nature of input/output linkage demands continual adaptation and innovation in interface design, ensuring that the system remains user-friendly and effective as interaction paradigms shift.

In essence, the linkage between input and output forms the backbone of user interaction with technology. A well-designed interface considers this linkage at every step, ensuring that input methods and output responses are seamlessly integrated. This integration enhances the system's usability and enriches the overall user experience. It requires a deep understanding of the technology's capabilities and the users' needs and preferences. By focusing on creating a harmonious input/output linkage, designers can craft interfaces that are functional, intuitive, and engaging, meeting the diverse needs of users in an ever-evolving technological landscape.

7.8 Control and Security Issues

A company must do everything in its power to protect its data. This includes the firm's information and that of its customers, employees, and suppliers. Most assets have value, but corporate data is priceless—without safe, secure, accurate data, a company cannot function. The following sections discuss input and output data control and security.

7.8.1 Input Control and Security

Input control includes the necessary measures to ensure input data is correct, complete, and secure. Input control must be the focus during every phase of input design, starting with source documents that promote data accuracy and quality. When a batch input method is used, the computer can produce an input log file that identifies and documents the data entered.

All information should be traceable back to the input data that produced it. That means the analyst must provide an audit trail that records the source of each data item and when it entered the system. In addition to recording the source, an audit trail must show how and when data is accessed or changed and by whom. All those actions must be logged in an audit trail file and monitored carefully.

A company must have procedures for handling source documents to ensure that data is not lost before it enters the system. All source documents that originate from outside the organization should be logged when they are received. Whenever source documents pass between departments, the transfer should be recorded.

Data security policies and procedures protect data from loss or damage, a vital goal in every organization. If the safeguards are not 100% effective, data recovery utilities should be able to restore lost or damaged data. Once data is entered, the company should store source documents in a safe location for some time. The company should have a **records retention policy** that meets all legal requirements and business needs.

Audit trail files and reports should be stored and saved. Then, if a data file is damaged, the information can be used to reconstruct the lost data. Data security also involves protecting data from unauthorized access. System sign-on procedures should prevent unauthorized individuals from entering the system, and users should change their passwords regularly. Having several levels of access is also advisable. For example, a data entry person might be allowed to view a credit limit but not change it.

7.8.2 Output Control and Security

Output must be accurate, complete, current, and secure. Companies use various **output control** methods to maintain output integrity and security. For example, every report should include an appropriate title, report number or code, printing date, and period covered. Reports should have pages numbered consecutively, identified as "Page *nn* of *nn*," and the end of the report should be labeled clearly. Control totals and record counts should be reconciled against input totals and counts. Reports should be randomly selected for a thorough check of correctness and completeness. All processing errors or interruptions must be logged so they can be analyzed.

Output security protects privacy rights and shields the organization's proprietary data from theft or unauthorized access. Several vital tasks must be performed to ensure output security. First, limit the number of printed copies and use a tracking procedure to account for each copy. When the printed output is distributed from a central location, specific procedures should ensure that the output is delivered to authorized recipients only—especially when reports such as payroll data contain sensitive information. All sensitive reports should be stored in secure areas. All pages of confidential reports should be labeled appropriately.

As shown in Figure 7-14, it is important to shred sensitive reports, out-of-date reports, and output from aborted print runs. Blank check forms must be stored securely and inventoried regularly to verify that no forms are missing. If signature stamps are used, they must be stored securely away from the form's storage location.

In most organizations, the IT department coordinates output control and security measures. Systems analysts must be concerned with security issues as they design, implement, and support information systems. Security should be designed into the system whenever possible by using passwords, shielding sensitive data, and controlling user access. Physical security will always be necessary, especially for tangible printed output that can be viewed and handled easily.

Figure 7-14 It is important to shred sensitive material to maintain output security.
Source: Yankittaphak Phoyalo/Shutterstock.com

Enterprise-wide data access creates a whole new set of security and control issues. Many firms have responded to those concerns by installing diskless workstations. A **diskless workstation** is a network terminal that supports a full-featured UI but limits data printing or copying, except for specific network resources that can be monitored and controlled. This concept typically precludes using portable storage devices like USB thumb drives.

7.8.3 Input and Output Control and Security

Input and output data control and security are measures and practices implemented to protect data as it enters a system (input) and leaves it (output). For input security, this involves ensuring that the data being received is from a legitimate source and is free from malicious content, often through validation and authentication processes. For output security, it's about safeguarding the data being transmitted or displayed, ensuring that it is accessible only to authorized individuals or systems. This is typically achieved through encryption, secure transmission protocols, and access controls. Both aspects are crucial for maintaining data confidentiality, integrity, and availability, protecting it from unauthorized access, manipulation, or breaches, and creating a robust information security strategy.

Data Encryption Techniques

Data encryption is a fundamental aspect of both input and output data security. It involves transforming readable data (plaintext) into a coded form (ciphertext) that can only be decoded by those with the correct decryption key. This is crucial for protecting sensitive information when transmitting data (output) and receiving it (input). Encryption techniques vary in complexity and application, ranging from symmetric encryption, where the same key is used for encryption and decryption, to asymmetric encryption, which uses different keys for each process. The topic of encryption also covers using secure protocols like SSL/TLS for secure data transmission and maintaining robust encryption standards to protect against evolving cyber threats.

Access Control and Authentication Mechanisms

Robust access control and authentication mechanisms ensure only authorized users can input or access output data. This security topic delves into various authentication methods, such as passwords, biometrics, two-factor authentication (2FA), and role-based access control

(RBAC) systems. This topic also explores how such mechanisms can effectively control access to sensitive data, thereby preventing unauthorized data entry, modification, or retrieval. It also examines the importance of maintaining up-to-date access control policies and regularly auditing access logs to detect and mitigate unauthorized access attempts.

Data Integrity and Validation

Ensuring the integrity and validation of input and output data is critical for maintaining data control and security. This area focuses on methods and protocols to verify that data has not been altered or corrupted during transmission or storage. Techniques such as checksums, digital signatures, and input validation protocols are key discussion topics. Input validation is particularly important, as it involves checking the accuracy and quality of incoming data before it's processed or stored. This topic also covers the importance of secure backup and recovery processes to protect data integrity during system failures, cyberattacks, or other disruptions.

7.9 Emerging Trends

The UI continues to evolve. Advanced technology will support the evolution, but the real driving force will be user empowerment, which results in customer satisfaction, increased productivity, and bottom-line savings. The widespread use of mobile devices such as the iPad and other tablets, coupled with the ubiquity of the iPhone and other smartphones, has dramatically influenced UI design. Indeed, key user experience lessons from these devices have been reflected in the UI for mainstream operating systems such as macOS and Microsoft Windows.

7.9.1 Responsive Web Design

Today's content is viewed by users on multiple devices: computers, tablets, and smartphones. Each device has its form factors that can limit the user experience. For example, the physical dimensions of a smartphone preclude the use of wide menus—users must scroll horizontally.

Responsive web design is an emerging trend that renders web content correctly, independently of the device in use. This means the developer only has to focus on essential UI issues; how the GUI artifacts are presented on the device is handled automatically by the underlying framework. Responsive web design addresses various non-functional attributes, including usability, performance, and maintainability.

Responsive web design relies on several underlying technologies, including CSS3, flexible images, and fluid grids. Page elements are expressed in relative terms (e.g., percentages) rather than absolute (e.g., pixels). This allows the content to "flow" correctly in the UI, irrespective of the display device.

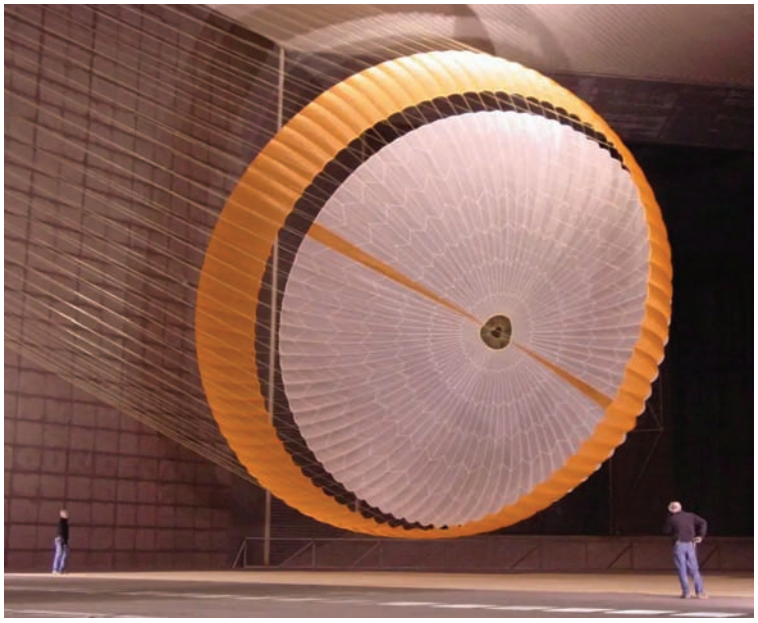
7.9.2 Prototyping

Prototyping produces a prototype, an early, rapidly constructed working version of the proposed information system. Prototyping involves a repetitive sequence of analysis, design, modeling, and testing. For example, in Figure 7-15, NASA engineers use a prototype to evaluate a parachute design at the Mars Science Laboratory landing system.

User input and feedback are essential at every stage of the systems development process. Prototyping allows users to examine a model that accurately represents system outputs, inputs, interfaces, and processes. Users can "test-drive" the model in a risk-free environment and approve it or request changes. In some situations, the prototype evolves into the final version of the information system. In other cases, the prototype is intended only to validate user requirements and is discarded afterward.

The most intense form of prototyping occurs when agile methods are used. As described earlier in the book, agile methods build a system by creating a series of prototypes and

Figure 7-15 Wind tunnel testing of an early parachute design for the Mars Science Laboratory landing system at the National Full-Scale Aerodynamics Complex.
Source: NASA/JPL/Pioneer Aerospace



constantly adjusting them to user requirements. Developers revise, extend, and merge earlier versions into the final product as the agile process continues. An agile approach emphasizes continuous feedback, and each incremental step is affected by what was learned in the prior steps.

Low-Fidelity Prototypes

Low-fidelity prototypes are basic and often rudimentary versions of the UI, focusing more on functionality and layout than detailed design. They are typically created using simple tools like paper and pencil or basic digital tools like PowerPoint or simple drawing software. Low-fidelity prototypes are quick and inexpensive, making them ideal for initial brainstorming and concept development stages. They allow designers and stakeholders to focus on the basic structure and flow of the interface without getting bogged down by aesthetic details.

Low-fidelity prototypes are built using **design prototyping**, or **throwaway prototyping**. This type of prototyping is beneficial for facilitating early discussions, gathering initial feedback, and verifying user requirements. The end product is a user-approved model that documents and benchmarks the features of the finished system. Design prototyping makes it possible to capture user input and approval while continuing to develop the system within the framework of the SDLC. Systems analysts typically use design prototyping to construct outputs, inputs, and UIs.

High-Fidelity Prototypes

High-fidelity prototypes are more polished and interactive representations of the final product. These prototypes are often created using advanced software tools like Adobe XD, Sketch, or Figma, and they closely mimic the final interface's look, feel, and functionality. High-fidelity prototypes provide a realistic experience of how the interface will function in the real world. They are helpful for user testing and presenting the product vision to stakeholders. They are particularly beneficial for assessing the user experience and conducting more detailed usability testing.

High-fidelity prototypes are built using **system prototyping**, which produces a full-featured information system model by adding advanced interactivity, animations, and transitions that

mimic the software application's or website's actual behavior. Tools like Axure RP or InVision are often used to create these types of prototypes. Interactive prototypes are useful for testing complex workflows, user interactions, and the system's overall usability. They allow designers and testers to experience and evaluate the dynamic aspects of the UI, such as response to user inputs, transitions between screens, and the functionality of interactive elements.

Trade-Offs

Prototypes are a crucial tool in the SDLC, offering practical insights and facilitating a more efficient, effective, and user-centered approach to UI design.

Some of the benefits of prototyping include the following:

- **Enhanced user involvement and feedback.** Prototypes provide a tangible representation of the final product, which helps engage users early in the design process. By interacting with the prototype, users can give more precise and practical feedback on the system's design, functionality, and usability. This early involvement helps identify potential issues and preferences, leading to a more user-centric final product.
- **Improved communication among stakeholders.** Prototypes are visual and interactive tools that facilitate better communication among team members, stakeholders, and clients. They help convey complex design concepts and functionalities more effectively than verbal descriptions or written specifications alone. This shared understanding reduces ambiguities and aligns everyone's vision for the project.
- **Early identification of design and usability issues.** Prototyping allows designers to identify and rectify design flaws and usability issues early in development. Changing a prototype is easier and less costly than changing a fully developed system. By catching issues early, the team can avoid costly rework and ensure a smoother development process.
- **Rapid iteration and experimentation.** Prototypes enable rapid iteration and experiment with different design ideas and approaches. Designers can quickly create, test, modify, and retest different interface versions. This flexibility allows for a more creative and thorough exploration of design possibilities, leading to a more refined and effective final product.
- **Improved planning and resource allocation.** By providing a clearer picture of how the final product will look and function, prototypes help in better planning and resource allocation. They can reveal the need for specific skills, technologies, or resources early in the process, allowing project managers to make informed decisions and plan more effectively for the development phase.

Although most systems analysts believe that the advantages of prototyping far outweigh any disadvantages, the following potential shortcomings should also be considered:

- **Lack of realism in low-fidelity prototypes.** Low-fidelity prototypes, such as paper sketches or basic wireframes, often fail to convey the actual look and feel of the final product. They lack the interactive elements, colors, and detailed graphics that high-fidelity prototypes or the final product will have. This can make it difficult for some stakeholders or users to fully grasp the final design's user experience or visual appeal.
- **Time and resource constraints for high-fidelity prototypes.** Creating high-fidelity or interactive prototypes can be time-consuming and resource-intensive. These prototypes require a significant investment in the design effort and the use of advanced tools, which may not be feasible for all projects, especially those with tight timelines or limited budgets.
- **Potential misalignment with final implementation.** Prototypes, even interactive ones, are often not built using the same code or technology that will be used in the final product. This can lead to a disconnect between the prototype and the actual implementation, where certain design elements or interactions envisioned

in the prototype may not be feasible or may behave differently in the actual application environment.

- **User testing limitations.** While prototypes are great for user testing and gathering feedback, they may not always provide accurate data about user behavior or real-life interaction. This is especially true for less interactive or non-functional prototypes, where users might interact with the prototype differently than a fully functional system.
- **Overemphasis on design aesthetics.** In some cases, particularly with high-fidelity prototypes, there can be an overemphasis on the visual and interactive aspects of the design. This focus might divert attention from other important factors, such as the system's back-end functionality, performance, and scalability. It's crucial to balance the aesthetic aspects with the technical and functional requirements of the final product.

7.9.3 Wearable Computing

The emergence of wearable computing, epitomized by devices like the Apple Watch, has significantly influenced the landscape of UI design. This impact is multifaceted, reflecting wearable devices' unique characteristics and use.

Compactness and Screen Size

One of the most evident impacts is the challenge posed by the small screen sizes of wearables. UI designers must work within these constraints to deliver clear, readable, and interactable content. This necessitates focusing on simplicity and minimalism, prioritizing essential information and functionality. The design must be intuitive and efficient, with limited space to display content or instructions. The Apple Watch, for instance, utilizes a combination of simple icons, voice input, and haptic feedback to maximize its small interface.

Context-Aware and Glanceable Interface

Wearable devices are often used on the go, requiring interfaces that deliver information quickly and efficiently. This has led to the development of "glanceable" interfaces that allow users to swiftly absorb necessary information without delving into detailed interactions. Context awareness is also crucial; the device should present relevant information based on the user's current activity or location, such as workout statistics during a run or meeting reminders when it's almost time for a scheduled appointment.

Integration with Other Devices and Services

Wearables often function as part of a larger ecosystem of devices and services. This requires UI designs that facilitate seamless interaction and data synchronization across different platforms, like smartphones, tablets, and even smart home devices. For example, the Apple Watch's UI design allows for smooth integration with the iPhone, enabling users to manage notifications, calls, and messages from their wrists.

New Modes of Interaction

Wearables have also introduced new interaction modalities. For example, the Apple Watch's Digital Crown is a hardware feature that offers a new way to interact with technology beyond traditional touch or voice commands. This requires UI designers to think innovatively about navigation and control, ensuring that these new interaction methods are intuitive and enhance the user experience.

Privacy and Data Security

Because wearables often collect sensitive personal data, such as health metrics, UI design must consider privacy and security. This data is exchanged via radio signals that may not be obvious to the user. Designing interfaces that inform users about data collection and provide easy access to privacy settings is becoming increasingly important.

Summary

This chapter discussed user interface (UI) design. A UI describes how users interact with a computer system and consists of all the hardware, software, screens, menus, functions, output, and features that affect two-way communications between the user and the computer. The UI is the key to usability, including user satisfaction, business function support, and system effectiveness.

A UI is based on fundamental principles of human–computer interaction (HCI). HCI describes the relationship between computers and those who use them to perform their jobs. HCI concepts apply to everything from smartphones to global networks. In its broadest sense, HCI includes all the communications and instructions necessary to enter input into the system and obtain output through screen displays or printed reports.

Most IT professionals agree that good design depends on seven fundamental principles. These desirable habits include understanding the business, maximizing graphical effectiveness, thinking like a user, using models and prototypes, focusing on usability, inviting feedback, and documenting everything. Successful UI designers adopt these principles in their work.

The effectiveness of a system is highly dependent on the quality of its UI, with user satisfaction and usability being critical indicators of success, regardless of the system's underlying technology or features. Industry best practices have established guidelines for UI design. Designers need to tailor these guidelines to specific project needs, especially for web and mobile applications with unique constraints, and sometimes even deviate from these guidelines to create a more user-centric and effective interface.

No matter how data enters an information system, the output quality is only as good as the quality of the input. The best time to avoid problems is when the data is entered. The main objective is to ensure input data quality, accuracy, and timeliness. Input design includes a description of source documents and the various zones in a document. Input masks and validation rules are used to reduce data errors.

Printed reports must be attractive, professional, and easy to read. To produce a well-designed printed report, the analyst must consider design features such as report headers and footers, page headers and footers, column headings and alignment, column spacing, field order, and grouping of detail lines. The three types of printed reports are detail reports, exception reports, and summary reports.

Most technology advances today affect both input and output. In addition to traditional input devices and methods, new hardware and ways to capture and enter data into a system have rapidly expanded. Most business information systems still provide output as screen displays and printed matter. However, alternate output mechanisms are also used. Input/output technology linkage is the harmonious integration of how users input data or commands into a system and how the system subsequently presents information or feedback to the users.

A company must do everything in its power to protect its data. Consequently, input and output data control and security must be considered part of the UI design. This refers to the measures and practices implemented to protect data entering (input) and leaving (output) a system, including data encryption techniques, access control and authentication mechanisms, and data integrity and validation.

Widespread use of mobile devices such as the iPad and other tablets, coupled with the ubiquity of the iPhone and other smartphones, has dramatically influenced UI design. Emerging trends include responsive web design, prototyping, and wearable computing.

Key Terms

audit trail
authorization zone
automated fax
batch
batch control
batch input
blog
calendar control
character-based report
check box
combination check
command button

context-sensitive
control break
control break report
control field order
data security
data type check
data validation rule
default value
design prototyping
detail report
dialog box
diskless workstation
electronic health record (EHR)
exception report
existence check
faxback
form filling
form layout
garbage in, garbage out (GIGO)
graphical user interface (GUI)
hash totals
human–computer interaction (HCI)
input control
input mask
limit check
list box
menu bar
mock-up
modules
natural language
online data entry
option button
output control
output security
page footer
page header
podcast
prototyping
radio button
range check
reasonableness check
records retention policy
report footer
report header
RFID (radio frequency identification) tag
scroll bar
sequence check
source data automation
source document
summary report
switchboard
system prototyping
throwaway prototyping
toggle button
toolbar
totals zone
transparent interface
turnaround document
usability
usability metrics
user-centered
user interface (UI)
validity check
webcast

Review Questions

1. Explain Apple's view of UI design, especially for apps.
2. What is human–computer interaction?
3. Why is a transparent interface desirable?
4. What are the seven habits of successful interface designers?
5. How would you rank the 10 guidelines for UI design in order of importance? Explain your answer.
6. What are the main principles of source document design?
7. What is the difference between a detail report, a summary report, and an exception report?
8. How has input technology changed in recent years?
9. What is output security?
10. What are three emerging trends in UI design?

Discussion Topics

1. Some systems analysts argue, "Give users what they ask for. If they want lots of reports and reams of data, then that is what you should provide. Otherwise, they will feel you are trying to tell them how to do their jobs." Others say, "Systems analysts should let users know what information can be obtained from the system. If you listen to users, you'll never get anywhere because they don't know what they want and don't understand information systems." What do you think of these arguments?

2. Some systems analysts maintain that source documents are unnecessary. They say that all input can be entered directly into the system without wasting time in an intermediate step. Do you agree? Can you think of any situations where source documents are essential?
3. Suppose your network support company employs 75 technicians who travel constantly and work at customer sites. Your task is to design an information system that provides technical data and information to the field team. What types of output and information delivery would you suggest for the system?
4. A UI can be quite restrictive. For example, the interface design might not allow a user to exit to a Windows desktop or to log on to the Internet. Should a UI include such restrictions? Why or why not?
5. How does the increased use of smartphones and tablets, with their smaller screen size, affect UI design practices?

Hands-On Projects

1. Visit the administrative office at your school or a local company. Ask to see examples of input screens. Analyze the design and appearance of each screen and try to identify at least one possible improvement.
2. Search the web for a good example of a UI that incorporates guidelines in this chapter. Document your research and discuss it with your class.
3. Review Section 7.2 and the comments about EHR usability. Research the status of EHR usability and describe all noteworthy developments.
4. Suggest at least two good examples and two bad examples of source document design.
5. Explore the emerging area of wearable computing, such as the Apple Watch, and comment on the impact of these devices on UI design.

Ethical Issues

One of the systems analysts on the project team thought they did a good job designing the company's tech support webpage, but the supervisor isn't so sure. The supervisor is concerned that the design is very similar to a page used by the company's primary competitor and asked the analyst whether they had used any HTML code from that site in the design. Although the analyst didn't copy any of the code, they did examine it in a web browser to see how it handled some design issues.

The supervisor asked the analyst to investigate webpage copyright issues and report back. The analyst learned that outright copying would be a copyright violation, but merely viewing other sites to get design ideas would be permissible. What needs to be clarified is the gray area in the middle. The analyst asked you, as a friend, for your opinion on this question: Even if no actual copying is involved, are there ethical constraints on how far you should go in using the creative work of others? How would you answer?

Chapter

8

Data Design

Learning Objectives

When you finish this chapter, you should be able to:

1. Explain basic data design concepts, including data structures, database management systems, and the evolution of the relational database model.
2. Explain the main components of a database management system.
3. Define the significant characteristics of web-based design.
4. Define data design terminology.
5. Draw entity-relationship diagrams.
6. Apply data normalization.
7. Utilize codes to simplify output, input, and data formats.
8. Explain data storage tools and techniques.
9. Explain data control measures.

Chapter 8 is the second of three chapters in the systems design phase of the SDLC. The chapter focuses on how data is organized, stored, and managed. These are essential issues that affect data quality and consistency. Mastering data design is necessary for a systems analyst to construct the physical model of the information system.

The chapter includes three "Case in Point" discussion questions to help contextualize the concepts described in the text. The scenario in the "Ethical Issues" section raises the timely issue of sharing customer data without explicit consent.

8.1 Data Design Concepts

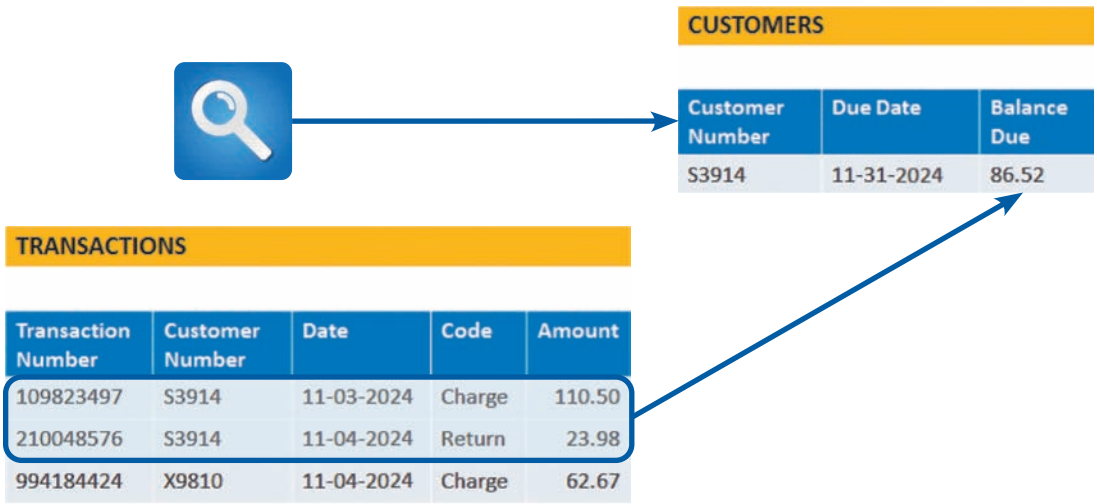
Systems analysts must understand basic data design concepts. This includes the pros and cons of file-oriented and table-oriented data structures. Understanding the evolution of relational database management systems (DBMSs) in the context of the SDLC is also essential.

8.1.1 Data Structures

A data structure is a framework for organizing, storing, and managing data. Data structures consist of files or tables that interact in various ways. Each file or table contains data about people, places, things, or events. For example, one file or table might have customer data; other files or tables might store data about products, orders, suppliers, or employees.

Many older legacy systems utilized file processing because it worked well with mainframe hardware and batch input. Some companies use this method to handle large volumes of structured data regularly because it can be cost-effective in certain situations. For example, consider a credit card company that posts thousands of daily transactions from a TRANSACTIONS file to account balances stored in a CUSTOMERS file, as shown in Figure 8-1. For that relatively straightforward process, file processing might be an option.

Figure 8-1 A credit card company that posts thousands of daily transactions might consider a file processing option.



Over time, the modern relational database became a standard model for systems developers. The following example of an auto service shop will compare the two concepts.

8.1.2 Mario and Danica: A Data Design Example

Figure 8-2 shows an auto shop mechanic at work. Consider two similar shops using different information system designs: Mario's Auto Shop and Danica's Auto Shop. Mario uses two file-oriented systems, while Danica uses a DBMS.

Figure 8-2 In an auto shop, data about the mechanic, the customer, and the work performed might be stored in a file-oriented system or a database system.
Source: Lisa F. Young/Shutterstock.com

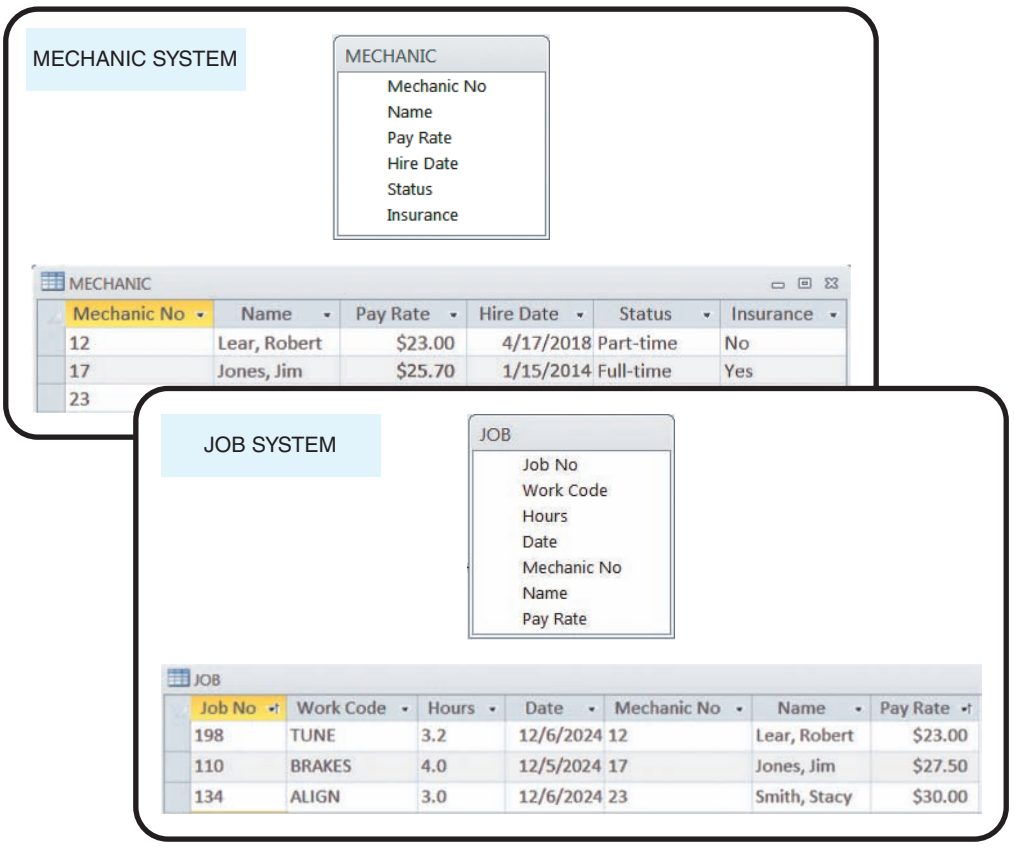
Mario's Auto Shop

Mario relies on two **file-oriented systems**, sometimes called file processing systems, to manage the business. The two systems store data in separate files that are not connected or linked. Figure 8-3 shows Mario's file-oriented systems:

- The MECHANIC SYSTEM uses the MECHANIC file to store data about shop employees.
- The JOB SYSTEM uses the JOB file to store data about work performed at the shop.

Unfortunately, using two separate systems means that some data is stored in two different places, which might or might not be consistent. For example, three data items (Mechanic No, Name, and Pay Rate) are stored in both files. This redundancy is a significant disadvantage of file-oriented systems because it threatens data quality and integrity. Figure 8-3 includes a typical discrepancy: Jim Jones' pay rate is $25.70 in the MECHANIC SYSTEM file and $27.50 in the JOB SYSTEM file.

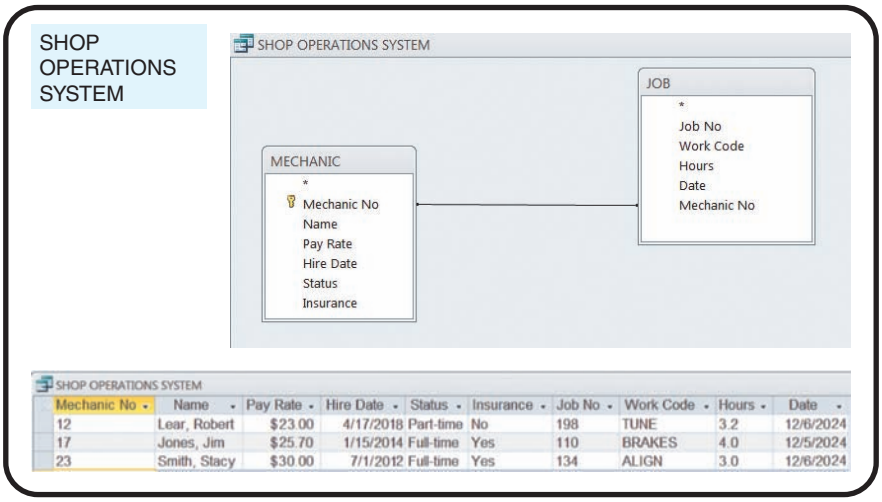
Figure 8-3 Mario's auto shop uses two separate systems, so certain data must be entered twice. This redundancy is inefficient and can produce errors.



Danica's Auto Shop

Danica uses a DBMS with two joined tables acting like one large table, as shown in Figure 8-4. In Danica's SHOP OPERATIONS SYSTEM, the tables are linked by the Mechanic No field, a *common field* because it connects the tables. Notice that no other data items are duplicated except for the common field. The DBMS design, also called a **relational database** or **relational model**, was introduced in the 1970s and continues to be the dominant approach for organizing, storing, and managing business data.

Figure 8-4 Danica's auto shop uses a database design, which avoids duplication. The data can be viewed as one large table, regardless of where it is physically stored.



Mechanic No	Name	Pay Rate	Hire Date	Status	Insurance	Job No	Work Code	Hours	Date
12	Lear, Robert	$23.00	4/17/2018	Part-time	No	198	TUNE	3.2	12/6/2024
17	Jones, Jim	$25.70	1/15/2014	Full-time	Yes	110	BRAKES	4.0	12/5/2024
23	Smith, Stacy	$30.00	7/1/2012	Full-time	Yes	134	ALIGN	3.0	12/6/2024

Mario's file-oriented systems show two different pay rates for Jim Jones, most likely because of a data entry error in one of them. That error could not occur in Danica's relational database because an employee's pay rate is stored in only one place. However, DBMSs are not immune to data entry problems, which are discussed in detail later in this chapter.

8.1.3 Database Management Systems

A database provides a framework that avoids data redundancy and supports a real-time, dynamic environment. Figure 8-5 shows a company-wide database that supports four separate information systems.

A **database management system (DBMS)** is a collection of tools, features, and interfaces that enable users to add, update, manage, access, and analyze data. From a user's point of view, the main advantage of a DBMS is that it offers timely, interactive, and flexible data access. Specific DBMS advantages include the following:

- **Scalability.** Scalability means that a system can be expanded, modified, or downsized easily to meet the rapidly changing needs of a business enterprise. For example, if a company decides to add data about secondary suppliers of material it uses, a new table can be added to the relational database and linked with a common field.

Figure 8-5 A sales database can support four separate business systems in this example.

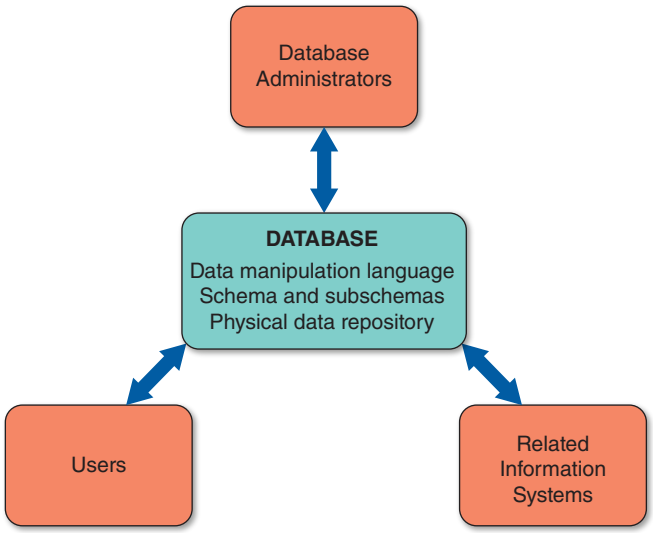
- **Economy of scale.** Database design allows better utilization of hardware. If a company maintains an enterprise-wide database, processing is less expensive using powerful servers and communication networks. The inherent efficiency of high-volume processing on larger computers is called **economy of scale**.
- **Enterprise-wide application.** A DBMS is typically managed by a **database administrator (DBA)**. The DBA assesses overall requirements and maintains the database for the benefit of the entire organization rather than a single department or user. Database systems can support enterprise-wide applications more effectively than file-processing systems. Effective database administration also helps ensure that data names, formats, and documentation standards are followed uniformly throughout the organization.
- **Improved security.** The DBA can define authorization procedures to ensure that only legitimate users can access the database and allow different users to have different levels of access. Most DBMSs provide sophisticated security support.
- **Data independence.** Systems that interact with a DBMS are relatively independent of how the physical data is maintained. That design allows the DBA to alter data structures without modifying information systems that use the data.
- **Data redundancy.** One of the primary advantages of a DBMS is its ability to reduce data redundancy, which is the unnecessary duplication of data. A DBMS manages data so that each piece of data is stored only once or as few times as necessary, thereby saving storage space and improving data consistency.
- **Data concurrency.** DBMSs are designed to allow multiple users simultaneous access without interfering with each other. This is known as concurrency. Concurrency control mechanisms ensure that transactions are processed reliably and that users can access data in a consistent state, even when multiple users are working with the same data at the same time.
- **Data sharing.** A DBMS allows different users and applications to access and share data in a controlled and systematic manner. It facilitates a collaborative environment where data can be accessed across various departments or user groups, enhancing data communication and decision making.
- **Data integrity.** A DBMS ensures data integrity (the accuracy and consistency of data over its life cycle) by enforcing data validation rules and integrity constraints. These constraints, like primary keys, foreign keys, and check constraints, prevent invalid data entry and maintain the relationships between data tables, ensuring that the data stored in the database remains accurate and reliable.
- **Data searching.** A DBMS provides sophisticated tools and query languages (such as SQL) for searching and retrieving data. Users can perform complex queries to search for specific data without knowing where the data is stored. This capability makes extracting meaningful information from large and complex data sets easier.

Although the trend is toward enterprise-wide database design, many companies still use a combination of centralized DBMSs and smaller, department-level database systems. This is because most large businesses view data as a company-wide resource that must be accessible to users throughout the company. At the same time, other factors encourage a decentralized design, including network expense, a reluctance to move away from smaller, more flexible systems, and a realization that enterprise-wide DBMSs can be highly complex and expensive to maintain. The compromise is often a client/server design, where processing is shared among several computers. Client/server systems are described in detail later in the book. As with many design decisions, the best solution depends on the organization's needs and circumstances.

8.2 DBMS Components

A DBMS provides an interface between a database and users who need to access the data. Although users are concerned primarily with an easy-to-use interface and support for their business requirements, a systems analyst must understand all of the components of a DBMS. In addition to interfaces for users, database administrators, and related systems, a DBMS has a data manipulation language, a schema and subschemas, and a physical data repository, as shown in Figure 8-6.

Figure 8-6 In addition to interfaces for users, database administrators, and related information systems, a DBMS has a data manipulation language, a schema and subschemas, and a physical data repository.



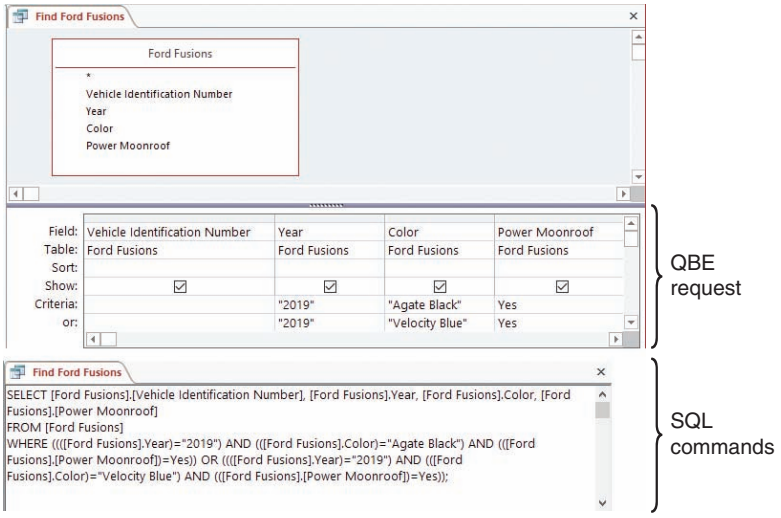
8.2.1 Interfaces for Users, Database Administrators, and Related Systems

When users, database administrators, and related information systems request data and services, the DBMS processes the request, manipulates the data, and provides a response. A **data manipulation language (DML)** controls database operations, including storing, retrieving, updating, and deleting data. Most commercial DBMSs, such as Oracle and IBM's DB2, use a DML. Some database products, such as Microsoft Access, also provide an easy-to-use graphical environment that enables users to control operations with menu-driven commands.

Users

Users typically work with predefined queries and switchboard commands (described previously in the book) and use query languages to access stored data. A **query language** allows a user to specify a task without specifying how the task will be accomplished. Some query languages use natural language commands that resemble ordinary English sentences. With a **query by example (QBE)** language, the user provides an example of the data requested. Many database programs also use **SQL (Structured Query Language)**, which is a language that allows client workstations to communicate with servers and mainframe computers. Figure 8-7 shows a QBE request for all Agate Black or Velocity Blue 2019 Ford Fusions with a power moonroof. The QBE request generates the SQL commands shown at the bottom of Figure 8-7.

Figure 8-7 Using QBE, a user can display all 2019 Ford Fusions with a power moonroof and either an Agate Black or Velocity Blue color.



Database Administrators

A DBA is responsible for DBMS management and support. DBAs are concerned with data security and integrity, preventing unauthorized access, providing backup and recovery, audit trails, maintaining the database, and supporting user needs. Most DBMSs provide utility programs to assist the DBA in creating and updating data structures, collecting and reporting database usage patterns, and detecting and reporting database irregularities.

Related Information Systems

A DBMS can support several related information systems that provide input to and require specific data from the DBMS. Unlike a user interface, no human intervention is necessary for two-way communication between the DBMS and the related systems.

8.2.2 Schema

The complete definition of a database, including descriptions of all fields, tables, and relationships, is called a **schema**. It is a structural blueprint that defines how data is organized, stored, and managed within a database. Essentially, a schema acts as a template for the database, detailing the structure of the database in a formal language supported by the DBMS. A schema provides a logical view of the entire database, including constraints and indexes, and guides administrators and developers in managing and querying it.

A **subschema** is a view of the database used by one or more systems or users. A subschema defines only those portions of the database that a particular system or user can access. More than one subschema can be defined.

For example, to protect individual privacy, a company's project management system should not be permitted to retrieve employee pay rates. The project management system subschema would not include the pay rate field. Database designers also use subschemas to restrict the level of access permitted. For example, specific users, systems, or locations might be allowed to create, retrieve, update, or delete data, depending on their needs and the company's security policies.

8.2.3 Physical Data Repository

As discussed earlier in the book, a data dictionary describes all data elements in the logical design. The data dictionary is transformed into a physical data repository containing the schema and subschemas at this stage of the systems development process. The physical repository might be centralized, or it might be distributed at several locations. In addition, the stored data might be managed by a single DBMS or several systems.

To resolve potential database connectivity and access problems, companies use ODBC-compliant software that enables communication among various systems and DBMSs. **ODBC**, which stands for **open database connectivity**, is an industry-standard protocol that allows software from different vendors to interact and exchange data. ODBC uses SQL statements that the DBMS understands and can execute, similar to the ones shown in Figure 8-7.

Another common standard is called **JDBC**, or **Java database connectivity**. JDBC enables Java applications to exchange data with any database that uses SQL statements and is JDBC-compliant. Physical design issues are described in more detail later in the book.

8.3 Web-Based Design

Table 8-1 lists some significant characteristics of web-based design. In a web-based design, the Internet serves as the front end, or interface, for the DBMS. Internet technology provides enormous power and flexibility because the related information system is not tied to any specific combination of hardware and software. Access to the database requires only a web browser and an Internet connection. Web-based systems are popular because they offer ease of access, cost-effectiveness, and worldwide connectivity—all vital to companies competing in a global economy.

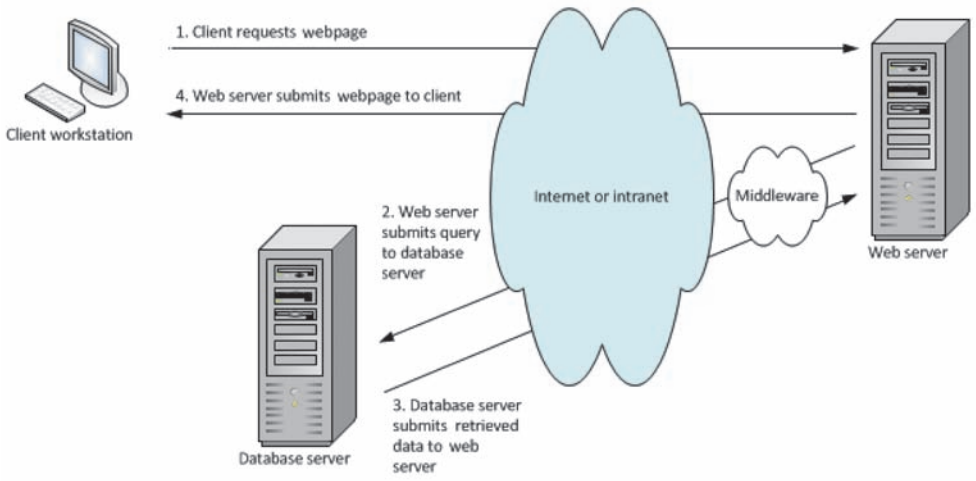
The database must be connected to the Internet or intranet to enable data access in a web-based system. The database and the Internet speak two different languages, however. Databases are created and managed using various languages and commands that have nothing to do with HTML. The objective is to connect the database to the web and enable data to be viewed and updated.

To bridge the gap, it is necessary to use **middleware**, which is software that integrates different applications and allows them to exchange data. Middleware can interpret client requests in HTML form and translate the requests into commands that the database can execute. When the database responds to the commands, middleware translates the results into HTML pages that can be displayed by the user's browser, as shown in Figure 8-8. Notice that the four steps in the process can occur using the Internet or a company intranet as the communications channel. Middleware is discussed in more detail later in the book.

Table 8-1 Web-based design characteristics include global access, ease of use, multiple platforms, cost-effectiveness, security, and adaptability issues. In a web-based design, the Internet serves as the front end, or interface, to the database management system. Access to the database requires only a web browser and an Internet connection.

Characteristic	Explanation
Global access	The Internet enables worldwide access, using existing infrastructure and standard telecommunications protocols.
Ease of use	Web browsers provide a familiar interface that is user-friendly and easily learned.
Multiple platforms	Web-based design is not dependent on a specific combination of hardware or software. All that is required is a browser and an Internet connection.
Cost-effectiveness	Initial investment is relatively low because the Internet serves as the communication network. Users require only a browser, and web-based systems do not require powerful workstations. Flexibility is high because numerous outsourcing options exist for development, hosting, maintenance, and system support.
Security issues	Security is a universal issue, but Internet connectivity raises special concerns. These can be addressed with a combination of good design, software that can protect the system and detect intrusion, stringent rules for passwords and user identification, and vigilant users and managers.
Adaptability issues	The Internet offers many advantages in terms of access, connectivity, and flexibility. Migrating a traditional database design to the web, however, can require design modification, additional software, and some added expense.

Figure 8-8 When a client workstation requests a webpage (1), the web server uses middleware to generate a data query to the database server (2). The database server responds (3), and the middleware translates the retrieved data into an HTML page that can be sent by the web server and displayed by the user's browser (4).



Web-based data must be secure yet easily accessible to authorized users. To achieve this goal, well-designed systems provide security at three levels: the database, the web server, and the telecommunication links connecting the system components. Data security is discussed in more detail later in the book.

8.4 Data Design Terms

A systems analyst can select a design approach and construct the system using the concepts discussed in the previous sections. The first step is to understand data design terminology.

8.4.1 Definitions

Data design terms include entity, table, file, field, record, and tuple. These terms are explained in the following sections.

Entity

An entity is a person, place, thing, or event for which data is collected and maintained. For example, an online sales system may include CUSTOMER, ORDER, PRODUCT, and SUPPLIER entities. Various entities and data stores were identified when data flow diagrams (DFDs) were prepared during the systems analysis phase. Now, the relationships among the entities will be considered.

Table or File

Data is organized into tables or files. A **table**, or **file**, contains related records that store data about a specific entity. Tables and files are shown as two-dimensional structures that consist of vertical columns and horizontal rows. Each column represents a field or characteristic of the entity, and each row represents a record, an individual instance, or an occurrence of the entity. For example, if a company has 10,000 customers, the CUSTOMER table will include 10,000 records, each representing a specific customer.

Although they have different meanings in a specific context, *table* and *file* can often be used interchangeably.

Field

A field, also called an attribute, is a single characteristic or fact about an entity. For example, a CUSTOMER entity might include the Customer ID, First Name, Last Name, Address, City, State, Postal Code, and Email Address.

A **common field** is an attribute that appears in more than one entity. Common fields can be used to link entities in various types of relationships.

Record or Tuple

A record, also called a **tuple** (rhymes with "couple"), is a set of related fields that describes one instance or occurrence of an entity, such as one customer, one order, or one product. Depending on the needed information, a record might have one or dozens of fields.

8.4.2 Key Fields

In database design, **key fields**, also known as keys, are critical for ensuring data integrity and optimizing database performance. They are used to organize, access, and maintain data structures. Various types of key fields are used for different purposes.

Primary Key

A **primary key** is a field (or combination of fields) that uniquely identifies each record in a table. Each table can have only one primary key and cannot contain null values. Primary keys ensure that each record is unique and easily identifiable.

For example, the customer number is unique because no two customers can have the same customer number. That key is also minimal because it contains no information beyond what is needed to identify the customer. A Customer ID might be a unique primary key in a CUSTOMER table. Customer ID is an example of a primary key based on a single field.

A primary key also can be composed of two or more fields. For example, if a student registers for three courses, their student number will appear in three records in the registration system. If one of those courses has 20 students, 20 separate records will exist for that course number—one record for each registered student.

In the registration file, neither the student number nor the course ID is unique, so neither field can be a primary key. To identify a specific student in a particular course, the primary key must be a combination of the student number and course ID. In that case, the primary key is called a **combination key**. A combination key also can be called a **composite key**, a **concatenated key**, or a **multivalued key**.

Figure 8-9 shows four tables: STUDENT, ADVISOR, COURSE, and GRADE. Three of these tables have single-field primary keys. Notice that in the GRADE table, the primary key combines two fields: STUDENT NUMBER and COURSE NUMBER.

Surrogate Key

A **surrogate key** is an artificial key introduced in a table to identify a row uniquely. It is not derived from application data and is often used as a primary key. These are typically sequential numbers (like an ID number) used when no natural primary key is present.

Unique Key

A **unique key** is similar to a primary key in that it must contain unique values but can contain null values (unless explicitly declared as NOT NULL). A table can have multiple unique keys, which are used to prevent duplicate entries in a column.

Some people consider a primary key to be a unique key, but they are slightly different. For example, a unique key can contain a NULL value. In addition, a table can only have one primary key, whereas a table can have multiple unique keys.

Super Key

A **super key** is a set of one or more columns that can uniquely identify a record in a table. A super key may include additional columns that are not necessary for unique identification. Every table can have multiple super keys, and each super key can contain other attributes beyond what is required to identify a record uniquely.

Candidate Key

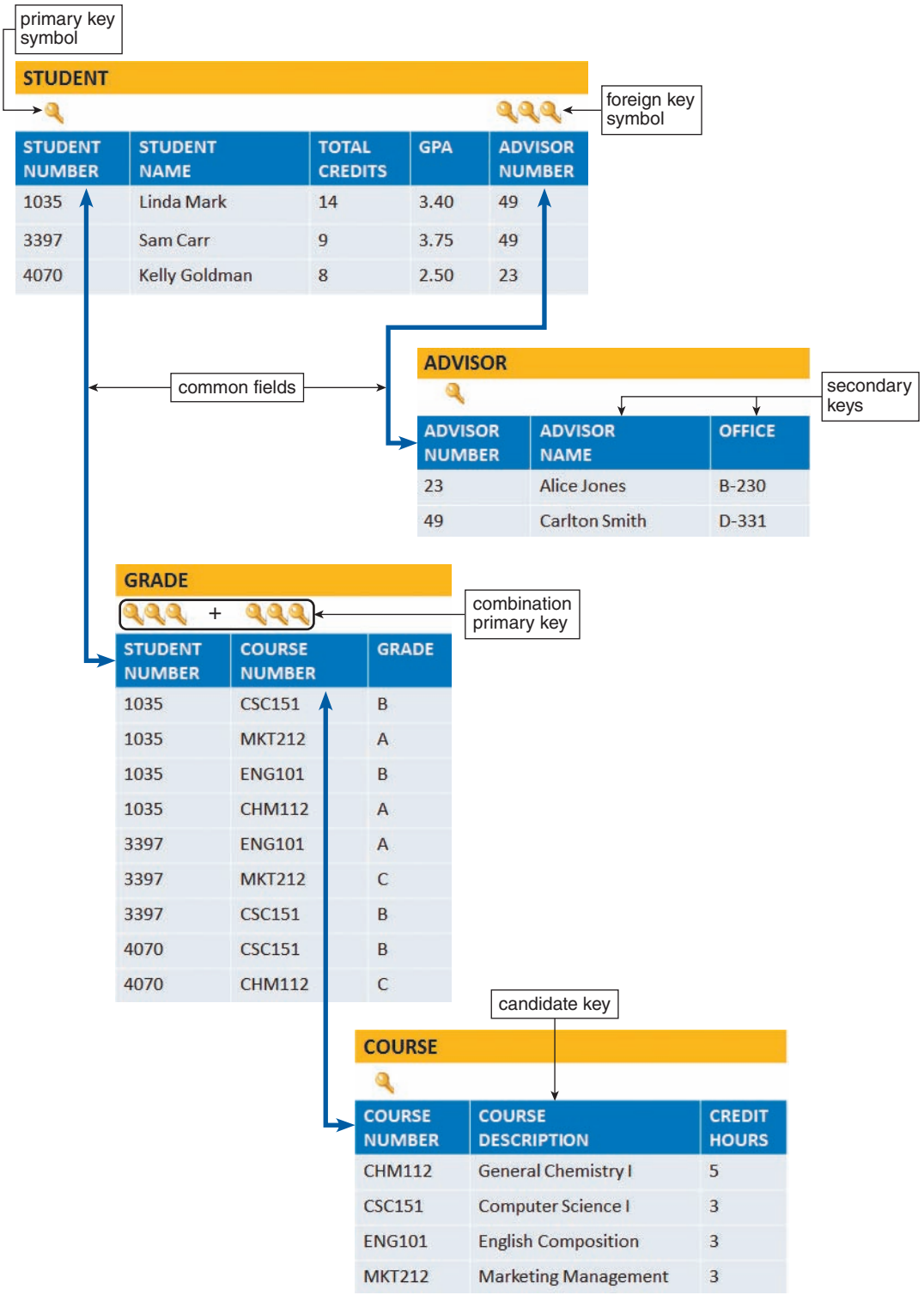
A **candidate key** is a field in a table that qualifies as the primary key. A candidate key can uniquely identify each row in a table and must not contain null values. A table can have multiple candidate keys, but only one is chosen as the primary key. Candidate keys are selected from the set of super keys without redundant attributes. For this reason, they are also referred to as minimal super keys.

For example, if every employee has a unique employee number, it could be used as a primary key. Note that an employee's Social Security number would not be a good choice for a candidate key; contrary to popular belief, Social Security numbers are not unique. Because only one field can be designated as a primary key, the field that contains the least amount of data and is the easiest to use should be selected. Any field that is not a primary or candidate key is called a **non-key field**.

The primary keys shown in Figure 8-9 also are candidate keys. Another candidate key is the COURSE DESCRIPTION field in the COURSE table. What about the OFFICE field in the ADVISOR table? It could not be a candidate key because more than one advisor might share the same office.

An **alternate key** is any candidate key not chosen as the primary key. These keys are alternatives for the primary key and are unique and non-null.

Figure 8-9 Examples of common fields, primary keys, candidate keys, foreign keys, and secondary keys.



Foreign Key

A **foreign key** is a field (or collection of fields) in one table that uniquely identifies a row of another table. It establishes and enforces a link between the data in two tables. The foreign key in the referencing table points to a primary key in the referenced table.

Recall that a common field exists in multiple tables and can be used to form a relationship or link between the tables. For example, in Figure 8-9, the ADVISOR NUMBER field appears in the STUDENT table and the ADVISOR table and joins the tables together. Notice that ADVISOR NUMBER is a primary key in the ADVISOR table, where it uniquely identifies each advisor, and is a foreign key in the STUDENT table.

Figure 8-9 also shows how two foreign keys can be a composite primary key in another table. Consider the GRADE table near the bottom of the figure. The two fields that form the primary key for the GRADE table are both foreign keys: the STUDENT NUMBER field, which must match a student number in the STUDENT table, and the COURSE NUMBER field, which must match one of the course IDs in the COURSE table.

How can these two foreign keys serve as a primary key in the GRADE table? Notice that student numbers and course IDs can appear any number of times in the table, but the combination of a specific student and a specific course occurs only once. For example, student 1035 appears four times, and course CSC151 appears three times—but there is only one combined instance of student 1035 and course CSC151. Because the combination of the specific student (1035) and the particular course (CSC151) is unique, it ensures that the grade (B) will be assigned to the proper student in the appropriate class.

Unlike a primary key, a foreign one must not be unique. For example, Carlton Smith has advisor number 49. The value 49 must be unique in the ADVISOR table because it is the primary key, but 49 can appear any number of times in the STUDENT table, where the advisor number serves as a foreign key.

Secondary Key

A **secondary key** is a field or combination of fields that can be used to access or retrieve records. Secondary key values are not unique. For example, the postal code field would be used as a secondary key to access records only for customers in a specific postal code. Secondary keys also can be used to sort or display records in a particular order. For example, the GPA field in a STUDENT file could be used to display records for all students in order of grade point average.

A secondary key arises because a table can have only one primary key. The CUSTOMER NUMBER is the primary key in a CUSTOMER file, so it must be unique. The customer's name might be known, but not the customer's number. For example, the table can be searched using the CUSTOMER NAME field as a secondary key to access Alex Mbeki without knowing the customer number. All records for Alex Mbeki are retrieved, and then the correct record is selected.

In Figure 8-9, student and advisor names are identified as secondary keys, but other fields could also be used. For example, the ADVISOR NUMBER field in the STUDENT table could be used as a secondary key to find all students with a particular advisor.

8.4.3 Referential Integrity

Validity checks can help avoid data input errors. One type of validity check, **referential integrity**, is a set of rules that avoids data inconsistency and quality problems. In a relational database, referential integrity means that a foreign key value cannot be entered in one table unless it matches an existing primary key in another table. For example, referential integrity would prevent a customer order from being entered in an order table unless that customer already exists in the customer table. Without referential integrity, an order might be called an **orphan** because it had no related customer.

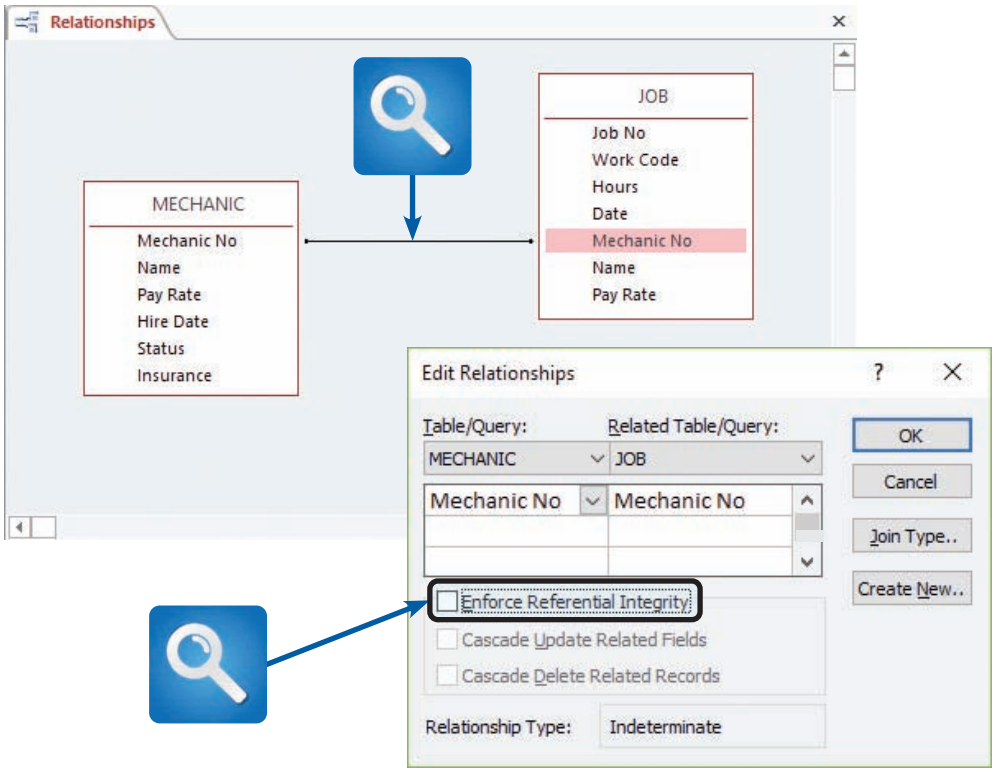
In the example shown in Figure 8-9, referential integrity will not allow a user to enter an advisor number (foreign key value) in the STUDENT table unless a valid advisor number (primary key value) already exists in the ADVISOR table.

Referential integrity can also prevent the deletion of a record if the record has a primary key that matches foreign keys in another table. For example, suppose an advisor resigns to accept a position at another school. The advisor cannot be deleted from the ADVISOR table while records in the STUDENT table still refer to that advisor number. Otherwise, the

STUDENT records would be orphans. To avoid the problem, students must be reassigned to other advisors by changing the value in the ADVISOR NUMBER field; then, the advisor record can be deleted.

Referential integrity can be built into the design when creating a relational database. Figure 8-10 shows a Microsoft Access screen that identifies a common field and allows users to enforce referential integrity rules.

Figure 8-10 Microsoft Access allows users to specify that referential integrity rules will be enforced in a relational database design.



Referential integrity is a fundamental aspect of relational database design and is critical for ensuring data accuracy and reliability in database systems. It helps maintain logical relationships between data entities, which is essential for the integrity and usability of the database.

8.5 Entity-Relationship Diagrams

An **entity-relationship diagram (ERD)** is a visual tool used in database design to represent the relationships between different data entities in a system. An ERD illustrates how data is structured and interrelated. An ERD consists of entities, attributes, and relationships:

- **Entities.** In an ERD, an entity represents a real-world object or concept with stored data. Entities are typically depicted as rectangles and might represent tangible objects like "Employee" or "Product" or more abstract concepts like "Transaction" or "Order."
- **Attributes.** Attributes are properties or characteristics of an entity. For example, an "Employee" entity might have EmployeeID, Name, and Address attributes. In ERDs, attributes are often shown as ovals connected to their respective entity.

- **Relationships.** These are associations between entities. Relationships are represented by lines connecting entities and often have a descriptor to clarify the relationship's nature. For instance, a relationship between "Employee" and "Department" might be labeled "works in" or "manages."

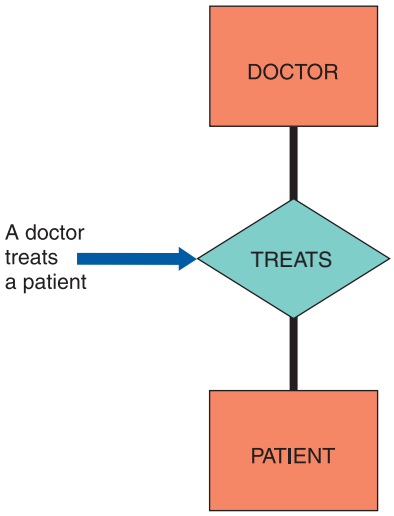
ERDs are a powerful tool for visualizing and designing the structure of a database. They help technical and nontechnical stakeholders understand a system's data relationships and requirements, making them essential in developing effective and efficient databases.

8.5.1 Drawing an ERD

The first step is to list the entities identified during the systems analysis phase and consider the nature of the relationships that link them. At this stage, a simplified method can show the relationships between entities.

Although there are different ways to draw ERDs, a popular method is representing entities as rectangles and relationships as diamond shapes. The entity rectangles are labeled with singular nouns, and the relationship diamonds are labeled with verbs, usually in a top-to-bottom and left-to-right fashion. For example, in Figure 8-11, a DOCTOR entity treats a PATIENT entity. Unlike DFDs, ERDs depict relationships, not data or information flows.

Figure 8-11 In an ERD, entities are labeled with singular nouns and relationships are labeled with verbs. The relationship is interpreted as a simple English sentence.



8.5.2 Types of Relationships

Three relationships can exist between entities: one-to-one, one-to-many, and many-to-many.

One-to-One

A **one-to-one relationship**, abbreviated **1:1**, exists when precisely one of the second entities occurs for each instance of the first entity. Figure 8-12 shows examples of several 1:1 relationships. A number 1 is placed alongside the two connecting lines to indicate the 1:1 relationship.

One-to-Many

A **one-to-many relationship**, abbreviated **1:M**, exists when one occurrence of the first entity can relate to many instances of the second entity. Still, each instance of the second entity can be associated with only one instance of the first entity. For example, the relationship between DEPARTMENT and EMPLOYEE is one-to-many: One department can have many employees, but each employee works in only one department at a time. Figure 8-13 shows several 1:M relationships. The line connecting the *many* entities is labeled with the letter M, and the number 1 labels the other connecting line. How many is *many*? The second 1:M relationship in Figure 8-13 shows the entities INDIVIDUAL and AUTOMOBILE. One individual might own five automobiles, or one, or none. Thus, *many* can mean any number, including zero.

Figure 8-12 Examples of one-to-one (1:1) relationships.

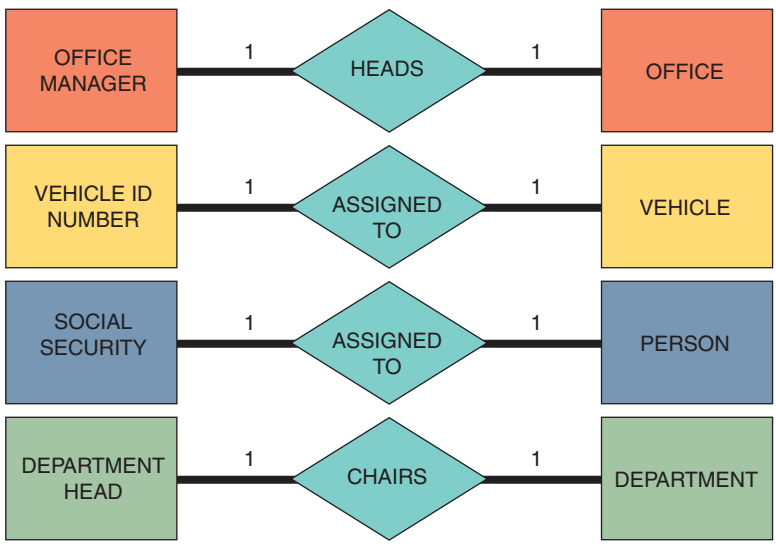
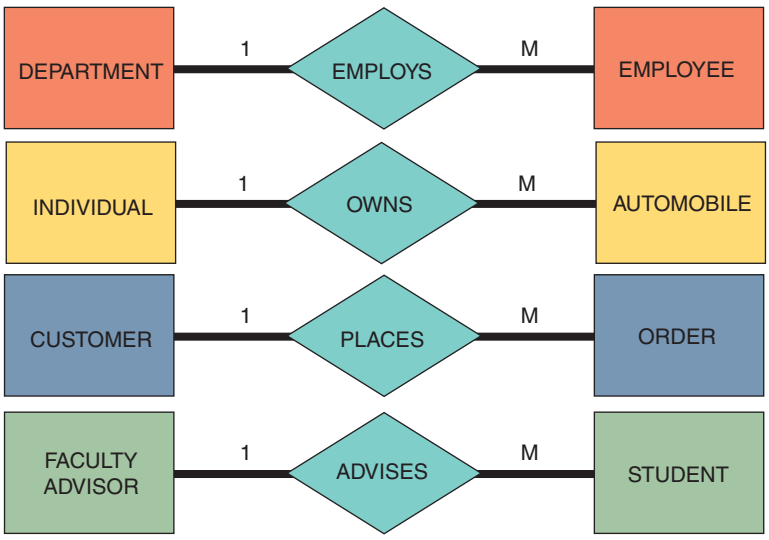


Figure 8-13 Examples of one-to-many (1:M) relationships.

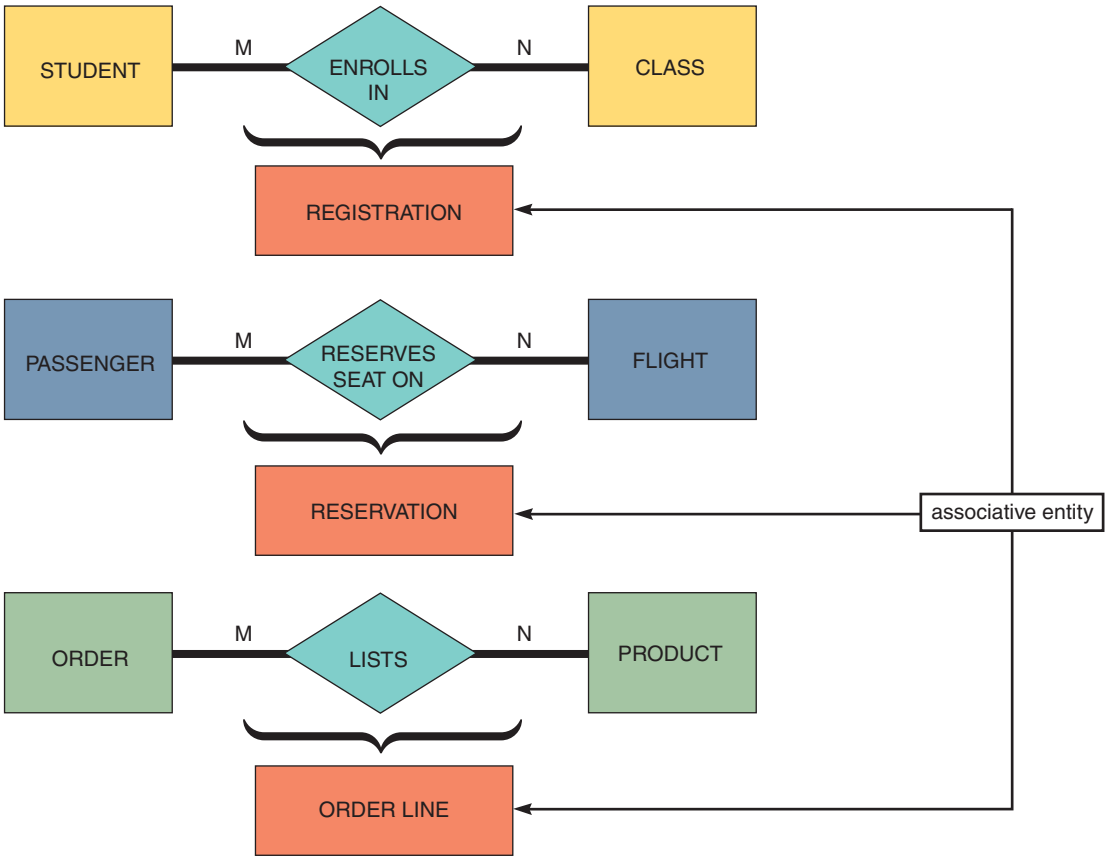


Many-to-Many

A **many-to-many relationship**, abbreviated **M:N**, exists when one instance of the first entity can relate to many instances of the second entity, and one instance of the second can relate to many instances of the first entity. For example, the relationship between STUDENT and CLASS is many-to-many—one student can take many classes, and one class can have many students enrolled. Figure 8-14 shows several M:N entity relationships. One of the connecting lines is labeled with the letter M, and the letter N labels the other connection.

Notice that an M:N relationship differs from 1:1 or 1:M relationships because the event or transaction that links the two entities is a third entity, called an **associative entity**, with its own characteristics. In the

Figure 8-14 Examples of many-to-many (M:N) relationships. Notice that the event or transaction that links the two entities is an associative entity with its own attributes and characteristics.



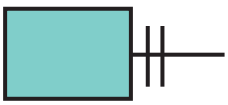
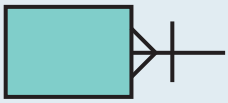
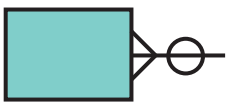
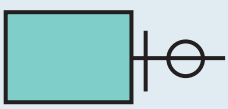
first example in Figure 8-14, the ENROLLS IN symbol represents a REGISTRATION entity that records each instance of a specific student enrolling in a particular course. Similarly, the RESERVES SEAT ON symbol represents a RESERVATION entity that records each instance of an individual passenger reserving a seat on a specific flight. In the third example, the LISTS symbol represents an ORDER LINE entity that records each instance of a specific product listed in a particular customer order.

8.5.3 Cardinality

After an analyst draws an initial ERD, they must define the relationships in more detail using cardinality. Cardinality describes the numeric relationship between two entities and shows how instances of one entity relate to another. For example, consider the relationship between two entities: CUSTOMER and ORDER. One customer can have one order, many orders, or none, but each order must have one and only one customer. An analyst can model this interaction by adding **cardinality notation**, which uses special symbols to represent the relationship.

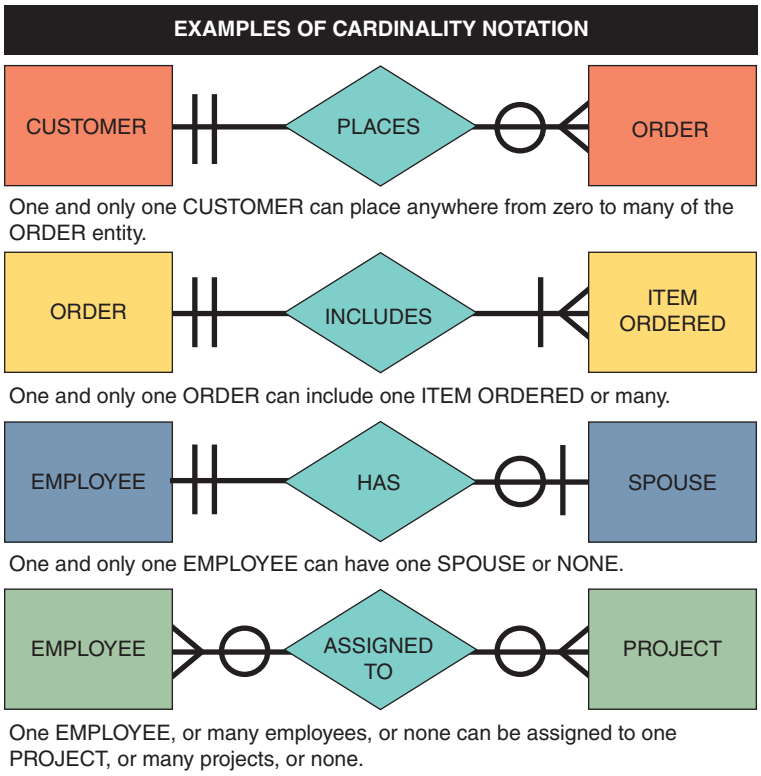
A common method of cardinality notation is called **crow's foot notation** because of a set of shapes, which include circles, bars, and symbols, that indicate various possibilities. A single bar indicates one possibility, a double bar indicates one and only one, a circle indicates zero, and a crow's foot indicates many. Figure 8-15 shows various cardinality symbols, their meanings, and the UML representations of the relationships. As described earlier in this book, the Unified Modeling Language (UML) is a widely used method of visualizing and documenting software systems design.

Figure 8-15 Crow's foot notation is a common method of indicating cardinality. The four examples show how various symbols can describe the relationships between entities.

Symbol	Meaning	UML Representation
	One and only one	1
	One or many	1..*
	Zero, or one, or many	0..*
	Zero, or one	0..1

In Figure 8-16, four examples of cardinality notation are shown. In the first example, one and only one CUSTOMER can place anywhere from zero to many of the ORDER entity. In the second example, one and only one ORDER can include one ITEM ORDERED or many. In the third example, one and only one EMPLOYEE can have one SPOUSE or none. In the fourth example, one EMPLOYEE, many employees, or none can be assigned to one PROJECT, many projects, or none.

Figure 8-16 Examples of cardinality notation.



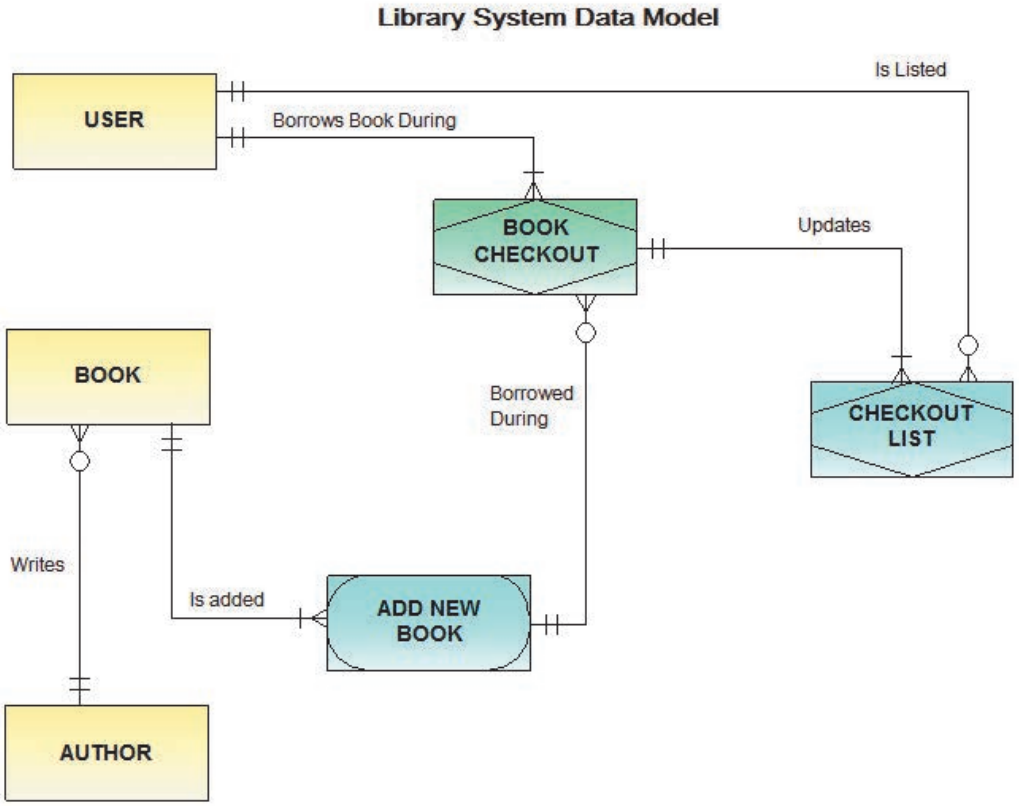
Most CASE products support the drawing of ERDs from entities in the data repository. Figure 8-17 shows part of a library system ERD drawn using the Visible Analyst CASE tool. Notice that the crow's foot notation shows the nature of the relationships, which are described in both directions.

Case in Point 8.1

TopText Publishing

TopText Publishing is a textbook publishing company with a headquarters location, a warehouse, and three sales offices, each with a sales manager and sales reps. TopText sells to schools, colleges, and individual customers. Many authors write more than one book for TopText, and some books are written by more than one author. TopText maintains an active list of more than 100 books, each identified by a universal code called an ISBN. How would you draw an ERD for the TopText information system, including cardinality notation?

Figure 8-17 An ERD for a library system drawn with Visible Analyst. Notice that crow's foot notation has been used, and relationships are described in both directions.



8.6 Data Normalization

Data **normalization** is a process in database design used to organize data to reduce redundancy and dependency, thereby increasing data integrity. It involves structuring a relational database so that each table represents a single concept or entity, and relationships between tables are established clearly and efficiently. This process typically involves dividing large tables into smaller, more manageable ones and establishing relationships using foreign keys. The aim is to minimize data duplication across the database, ensure data consistency, and simplify the database structure, making it easier to maintain and update.

A **table design** specifies the fields and identifies the primary key in a particular table or file. Working with a set of initial table designs, normalization is used to develop an overall database design that is simple, flexible, and free of data redundancy. Normalization involves applying rules to help identify and correct inherent problems and complexities in table designs. The concept of normalization is based on the work of Edgar Codd, a British computer scientist who formulated the basic principles of relational database design.

The normalization process typically involves four stages: unnormalized design, first normal form, second normal form, and third normal form (3NF). The three normal forms constitute a progression in which 3NF represents the best design. Most business-related databases must be designed in 3NF. Note that normal forms beyond 3NF exist but are rarely used in business-oriented systems.

8.6.1 Standard Notation Format

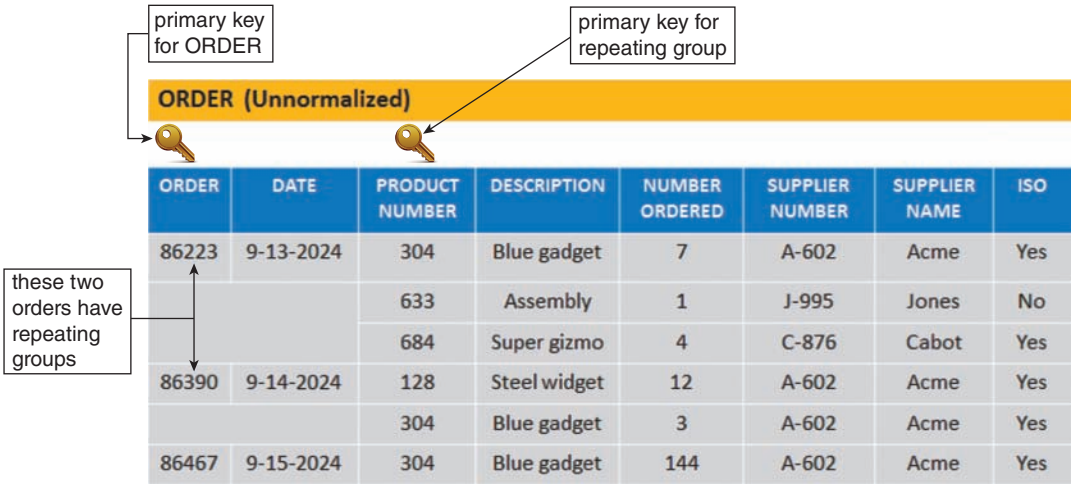
Designing tables is easier if a **standard notation format** shows a table's structure, fields, and primary key. The standard notation format in the following examples of an ORDER system starts with the table's name, followed by a parenthetical expression containing the field names separated by commas. The primary key field(s) is underlined, like this:

NAME (FIELD 1, FIELD 2, FIELD 3)

During data design, the analyst must recognize a repeating group of fields. A **repeating group** is a set of one or more fields that can occur any number of times in a single record, with each occurrence having different values.

A typical example of a repeating group is shown in Figure 8-18. A company that uses written source documents to record orders might have data that looks like this. Figure 8-18 shows two orders containing multiple items, constituting repeating groups within the same order number. Notice that in addition to the order number and date, the records with multiple products may have repetitions of the product number, description, number ordered, supplier number, supplier name, and International Organization for Standardization (ISO) status. A repeating group can be considered a set of child (subsidiary) records contained within the parent (main) record.

Figure 8-18 In the ORDER table design, two orders have repeating groups that contain several products. ORDER is the primary key for the ORDER table, and PRODUCT NUMBER is the primary key for the repeating group. Because it includes repeating groups, the ORDER table is unnormalized.



ORDER (Unnormalized)

ORDER	DATE	PRODUCT NUMBER	DESCRIPTION	NUMBER ORDERED	SUPPLIER NUMBER	SUPPLIER NAME	ISO
86223	9-13-2024	304	Blue gadget	7	A-602	Acme	Yes
		633	Assembly	1	J-995	Jones	No
		684	Super gizmo	4	C-876	Cabot	Yes
86390	9-14-2024	128	Steel widget	12	A-602	Acme	Yes
		304	Blue gadget	3	A-602	Acme	Yes
86467	9-15-2024	304	Blue gadget	144	A-602	Acme	Yes

A table design that contains a repeating group is called **unnormalized**. The standard notation method for representing an unnormalized design is to enclose the repeating group of fields within a second set of parentheses. An example of an unnormalized table looks like this:

NAME (FIELD 1, FIELD 2, FIELD 3, (REPEATING FIELD 1, REPEATING FIELD 2))

Now review the unnormalized ORDER table design shown in Figure 8-18. Following the notation guidelines, the design can be described as follows:

ORDER (ORDER, DATE, (PRODUCT NUMBER, DESCRIPTION, NUMBER ORDERED, SUPPLIER NUMBER, SUPPLIER NAME, ISO))

The notation indicates that the ORDER table design contains eight fields in the outer parentheses. The ORDER field is underlined to show that it is the primary key. The PRODUCT NUMBER, DESCRIPTION, NUMBER ORDERED, SUPPLIER NUMBER, SUPPLIER NAME, and ISO fields are enclosed within an inner set of parentheses to indicate that they are fields

within a repeating group. Notice that PRODUCT NUMBER is also underlined because it acts as the primary key of the repeating group. If a customer orders three different products in one order, six fields must be repeated for each product, as shown in Figure 8-18.

8.6.2 First Normal Form

A table is in **first normal form (1NF)** if it does not contain a repeating group. To convert an unnormalized design to 1NF, the table's primary key must be expanded to include the primary key of the repeating group.

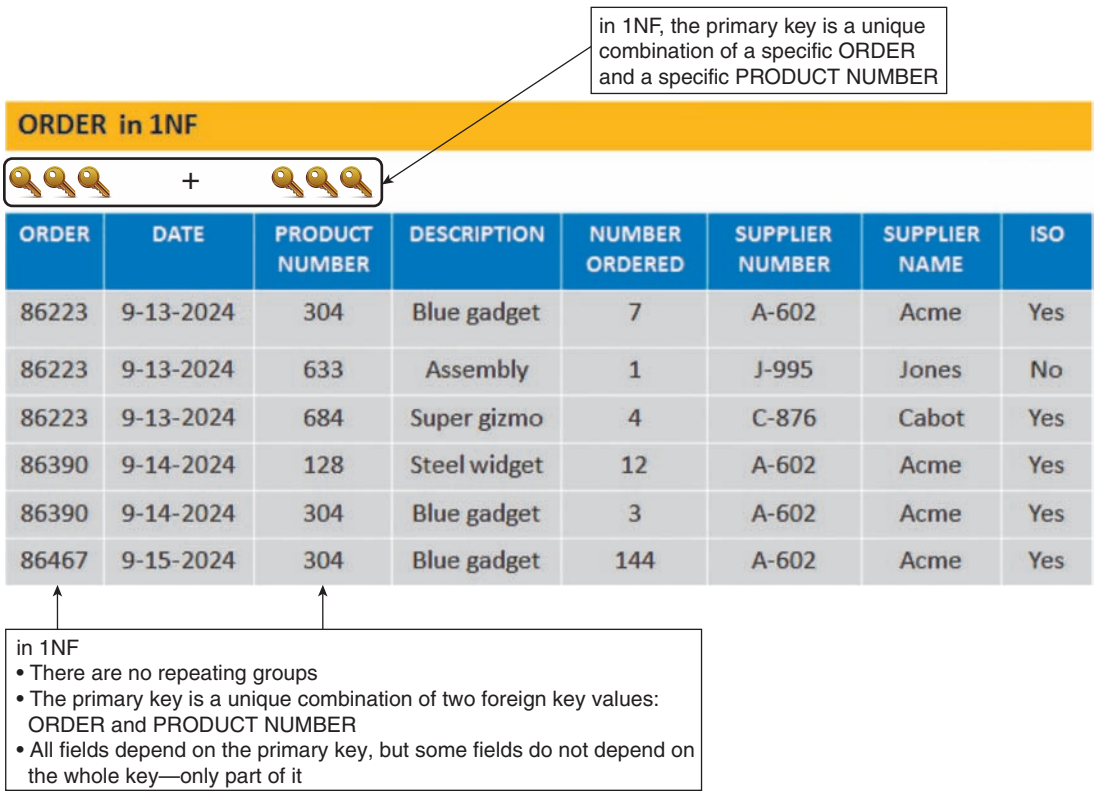
For example, in the ORDER table shown in Figure 8-18, the repeating group consists of six fields: PRODUCT NUMBER, DESCRIPTION, NUMBER ORDERED, SUPPLIER NUMBER, SUPPLIER NAME, and ISO. Of the three fields, only PRODUCT NUMBER can be a primary key because it uniquely identifies each instance of the repeating group. The DESCRIPTION cannot be a primary key because it might or might not be unique. For example, a company might sell many parts with the same descriptive name, such as a washer, relying on a coded part number to identify each washer size uniquely.

When the primary key of the ORDER table is expanded to include PRODUCT NUMBER, the repeating group is eliminated, and the ORDER table is now in 1NF, as shown:

ORDER (ORDER, DATE, PRODUCT NUMBER, DESCRIPTION, NUMBER ORDERED, SUPPLIER NUMBER, SUPPLIER NAME, ISO)

Figure 8-19 shows the ORDER table in 1NF. Notice that when a repeating group is eliminated, additional records emerge—one for each combination of a specific order and a specific product. The result is more records but a greatly simplified design. In the new version, the repeating group for order number 86223 has become three separate records, and the

Figure 8-19 The ORDER table as it appears in 1NF. The repeating groups have been eliminated. Notice that the repeating group for order 86223 has become three separate records, and the repeating group for order 86390 has become two separate records. The 1NF primary key is a combination of ORDER and PRODUCT NUMBER, which uniquely identifies each record.



ORDER	DATE	PRODUCT NUMBER	DESCRIPTION	NUMBER ORDERED	SUPPLIER NUMBER	SUPPLIER NAME	ISO
86223	9-13-2024	304	Blue gadget	7	A-602	Acme	Yes
86223	9-13-2024	633	Assembly	1	J-995	Jones	No
86223	9-13-2024	684	Super gizmo	4	C-876	Cabot	Yes
86390	9-14-2024	128	Steel widget	12	A-602	Acme	Yes
86390	9-14-2024	304	Blue gadget	3	A-602	Acme	Yes
86467	9-15-2024	304	Blue gadget	144	A-602	Acme	Yes

repeating group for order number 86390 has become two separate records. Therefore, when a table is in 1NF, each record stores data about a single instance of a specific order and a specific product.

Also, the 1NF design shown in Figure 8-19 has a combination primary key. The primary key of the 1NF design cannot be the ORDER field alone because the order number does not uniquely identify each product in a multiple-item order. Similarly, PRODUCT NUMBER cannot be the primary key because it appears more than once if several orders include the same product. Because each record must reflect a specific product in a specific order, both fields, ORDER and PRODUCT NUMBER, are needed to identify a single record uniquely. Therefore, the primary key is the *combination* of two fields: ORDER and PRODUCT NUMBER.

8.6.3 Second Normal Form

The concept of functional dependence must be understood to understand **second normal form (2NF)**. For example, Field A is **functionally dependent** on Field B if the value of Field A depends on Field B. In Figure 8-19, the DATE value is functionally dependent on the ORDER because there can be only one date for a specific order number. In contrast, a product description is *not* dependent on the order number. There might be several product descriptions for a particular order number—one for each item ordered.

A table design is in 2NF if it is in 1NF *and* if all fields that are not part of the primary key functionally depend on the *entire* primary key. If any field in a 1NF table depends on only one of the fields in a combination primary key, then the table is not in 2NF.

Notice that if a 1NF design has a primary key that consists of only one field, the problem of partial dependence does not arise—because the entire primary key is a single field. Therefore, a 1NF table with a single-field primary key is automatically in 2NF.

Now reexamine the 1NF design for the ORDER table shown in Figure 8-19:

ORDER (ORDER, DATE, PRODUCT NUMBER, DESCRIPTION, NUMBER ORDERED, SUPPLIER NUMBER, SUPPLIER NAME, ISO)

Recall that the primary key combines the order number and the product number. The NUMBER ORDERED field depends on the *entire* primary key because NUMBER ORDERED refers to a specific product number *and* a specific order number. In contrast, the DATE field depends on the order number, which is only a part of the primary key. Similarly, the DESCRIPTION field depends on the product number, which is only a part of the primary key. Because some fields are not dependent on the *entire* primary key, the design is not in 2NF.

A standard process exists for converting a table from 1NF to 2NF. The objective is to break the original table into two or more new tables and reassign the fields so that each non-key field will depend on the entire primary key in its table. To accomplish this, the following steps should be followed:

1. Name a separate table for each field in the existing primary key. For example, in Figure 8-19, the ORDER table's primary key has two fields, ORDER and PRODUCT NUMBER, so two tables must be created. The ellipsis (. . .) indicates that fields will be assigned later. The result is:

   ORDER (ORDER, . . .)
   PRODUCT (PRODUCT NUMBER, . . .)

2. Create a new table for each possible combination of the original primary key fields. In the example in Figure 8-19, a new table would be created with a combination of the primary keys ORDER and PRODUCT NUMBER. This table describes individual lines in an order, so it is named ORDER LINE, as shown:

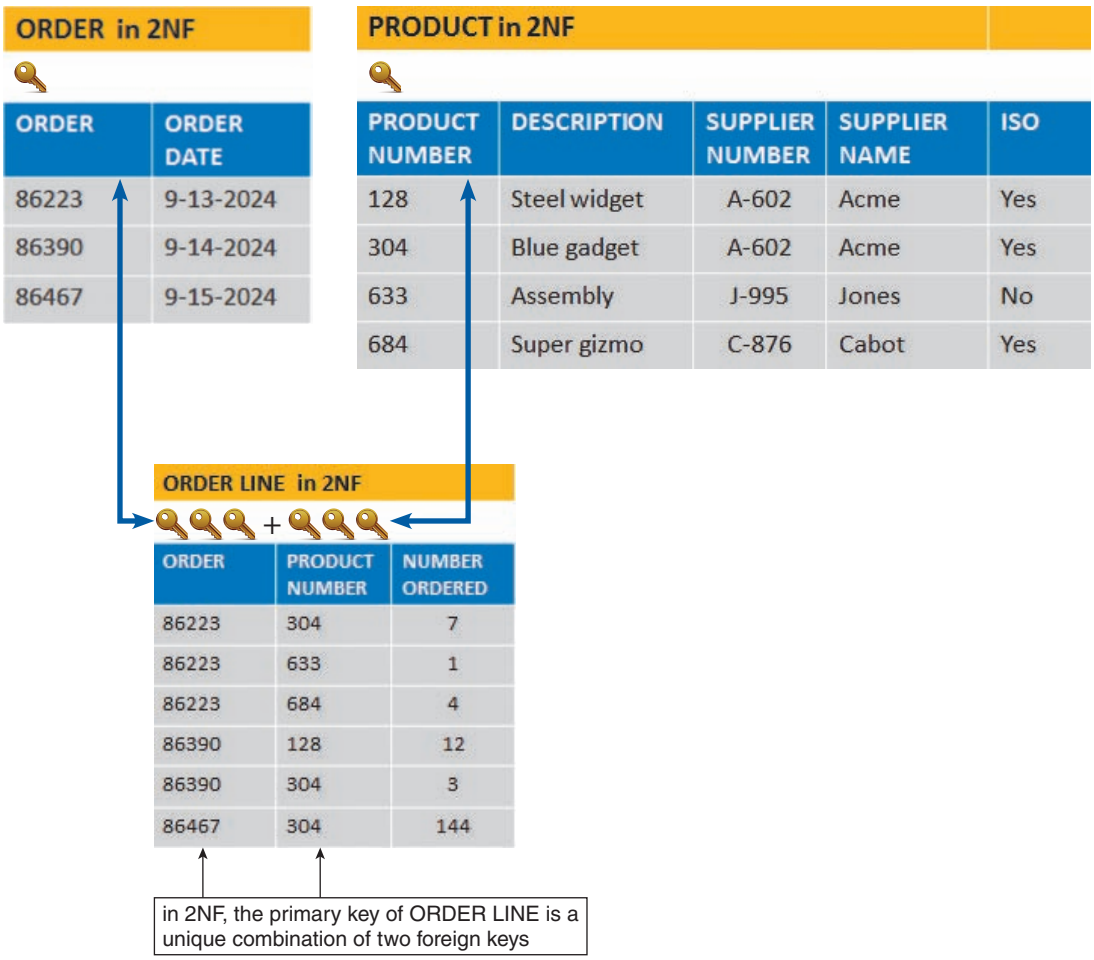
   ORDER LINE (ORDER, PRODUCT NUMBER)

3. Study the three tables and place each field with its appropriate primary key, which is the minimal key on which it functionally depends. When all the fields have been placed, remove any table with no additional fields assigned to it. The remaining tables are the 2NF version of the original table. The three tables can be shown as follows:

   ORDER (ORDER, ORDER DATE)
   PRODUCT (PRODUCT NUMBER, DESCRIPTION, SUPPLIER NUMBER, SUPPLIER NAME, ISO)
   ORDER LINE (ORDER, PRODUCT NUMBER, NUMBER ORDERED)

Figure 8-20 shows the 2NF table designs. Following these steps, the original 1NF table has been converted into three 2NF tables.

Figure 8-20 ORDER, PRODUCT, and ORDER LINE tables in 2NF. All fields are functionally dependent on the primary key.



It is vital to move from 1NF to 2NF due to four kinds of problems found with 1NF designs that do not exist in 2NF:

1. Consider the work necessary to change a particular product's description. Suppose 500 current orders exist for product number 304. Changing the product description involves modifying 500 records for product number 304. Updating all 500 records would be cumbersome and expensive.

2. 1NF tables can contain inconsistent data. Because someone must enter the product description in each record, nothing prevents product number 304 from having different product descriptions in different records. If product number 304 appears in many order records, some matching product descriptions might be inaccurate or improperly spelled. Even the presence or absence of a hyphen in the orders for "All-purpose gadget" would create consistency problems. Some inconsistency may result if a data entry person must enter a term such as "IO1 Queue Controller" numerous times.
3. Adding a new product is a problem. Because the primary key must include an order number and a product number, values are needed for both fields to add a record. What value should be used for the order number when no customer has ordered the product? A placeholder order number could be replaced with an actual order number when the product is ordered, but that solution also creates difficulties.
4. Deleting a product is a problem. If all the related records are deleted once an order is filled and paid for, what happens if the only record that contains product number 633 is deleted? The information about that product number and its description is lost.

Has the 2NF design eliminated all potential problems? Just one PRODUCT record must be changed to change a product description. Multiple, inconsistent values for the product description are impossible because the description appears in only one location. A new PRODUCT record is created to add a new product instead of creating a placeholder order record. When the last ORDER LINE record for a particular product number is removed, that product number and description are not lost because the PRODUCT record still exists. The four potential problems are eliminated, and the three 2NF designs are superior to the original unnormalized table and the 1NF design.

8.6.4 Third Normal Form

A popular rule of thumb is that a design is in **third normal form (3NF)** if every non-key field depends on "the key, the whole key, and nothing but the key." A 3NF design avoids redundancy and data integrity problems that still exist in 2NF designs.

Continuing the ORDER example, now review the PRODUCT table design in Figure 8-21:

PRODUCT (PRODUCT NUMBER, DESCRIPTION, SUPPLIER NUMBER, SUPPLIER NAME, ISO)

The PRODUCT table is in 1NF because it has no repeating groups. The table is also in 2NF because the primary key is a single field. But the table still has four potential problems:

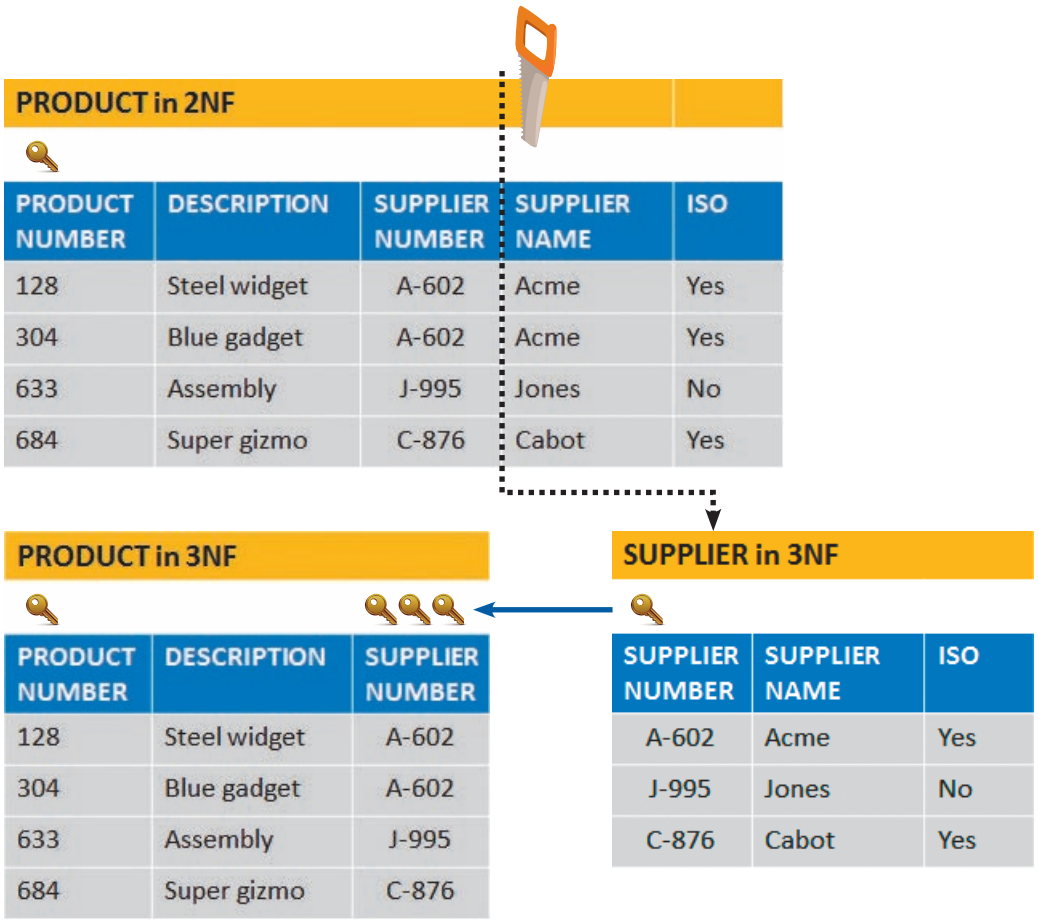
1. Every record in which the supplier name appears must be changed to completely change a supplier name in the table. With hundreds or even thousands of records, the process would be slow, expensive, and subject to input errors.
2. The 2NF design allows suppliers to have different names or ISO statuses in other records.
3. Because the supplier name is included in the ORDER table, a placeholder ORDER record must be created to add a new supplier who has not received any orders.
4. If all the orders for a supplier are deleted, that supplier's number and name will be lost.

Those potential problems are caused because the design is not in 3NF. A table design is in 3NF if it is in 2NF and no non-key field is dependent on another non-key field. Remember that a non-key field is a field that is not a candidate key for the primary key.

The PRODUCT table at the top of Figure 8-21 is not in 3NF because two non-key fields, SUPPLIER NAME and ISO, depend on another non-key field, SUPPLIER NUMBER.

To convert the table to 3NF, all fields from the 2NF table that depend on another non-key field must be removed and placed in a new table that uses the non-key field as a primary

Figure 8-21 When the PRODUCT table is transformed from 2NF to 3NF, the result is two separate tables: PRODUCT and SUPPLIER. Note that in 3NF, all fields depend on the key alone.



key. In the PRODUCT example, SUPPLIER NAME and ISO must be removed and put into a new table that uses SUPPLIER NUMBER as the primary key. As shown in Figure 8-21, 3NF divides the 2NF version into two separate 3NF tables:

PRODUCT (PRODUCT NUMBER, DESCRIPTION, SUPPLIER NUMBER)

SUPPLIER (SUPPLIER NUMBER, SUPPLIER NAME, ISO)

Case in Point 8.2

CyberToys

You handle administrative support for CyberToys, a small chain that sells computer hardware and software and specializes in personal service. The company has four stores located in malls and is planning more. Each store has a manager, a technician, and between one and four sales reps.

The owners want to create a personnel records database and have asked you to review a table they designed. They suggested fields for store number, location, store phone, manager name, and manager home phone. They also want fields for the technician's name and home phone number and fields for up to four sales rep names and their home phone numbers.

Draw and analyze their suggested design using the normalization concepts you learned in the chapter. What do you think of their design and why? What would you propose?

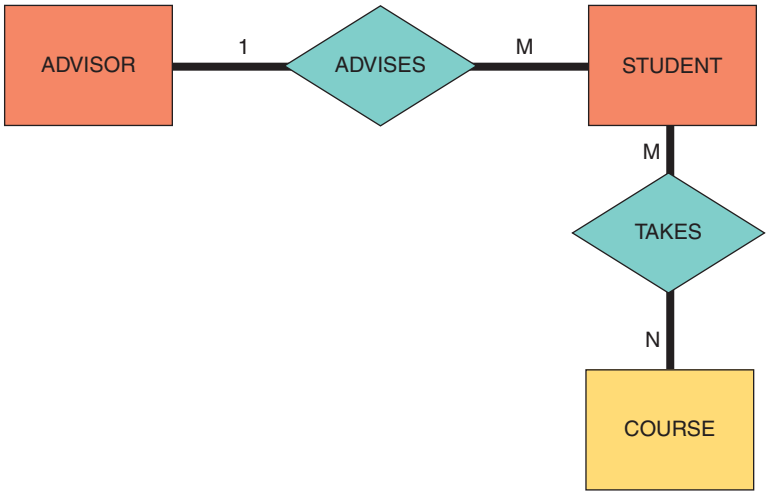
8.6.5 Example

Applying the rules to a representative situation is an excellent way to learn about normalization. If a step-by-step process is followed, data designs that are efficient, maintainable, and error-resistant can be created. This section presents an example of a hypothetical school, Crossroads College.

Consider the familiar situation in Figure 8-22, which depicts several entities in the Crossroads College advising system: ADVISOR, COURSE, and STUDENT. The relationships among the three entities are shown in the ERD in Figure 8-23. The following sections discuss normalization rules for these three entities.

Figure 8-22 A faculty advisor, who represents an entity, can advise many students, each of whom can register for one or many courses.
Source: Monkey Business Images/Shutterstock.com

Figure 8-23 An initial ERD for ADVISOR, STUDENT, and COURSE.



Before the normalization process is started, it is noted that the STUDENT table contains fields that relate to the ADVISOR and COURSE entities, so a decision is made to begin with the initial design for the STUDENT table, which is shown in Figure 8-24. Notice that the table

Figure 8-24 The STUDENT table is unnormalized because it contains a repeating group representing each student's courses.

STUDENT (Unnormalized)

STUDENT NUMBER	STUDENT NAME	TOTAL CREDITS	GPA	ADVISOR NUMBER	ADVISOR NAME	OFFICE	COURSE NUMBER	CREDIT HOURS	GRADE
1035	Linda	47	3.60	49	Smith	B212	CSC151	4	B
							MKT212	3	A
							ENG101	3	B
							CHM112	4	A
							BUS105	2	A
3397	Sam	29	3.00	49	Smith	B212	ENG101	3	A
							MKT212	3	C
							CSC151	4	B
4070	Kelly	14	2.90	23	Jones	C333	CSC151	4	B
							CHM112	4	C
							ENG101	3	C
							BUS105	2	C

design includes the student number, student name, total credits taken, grade point average (GPA), advisor number, advisor name, advisor office number, and, for every course the student has taken, the course number, number of credits, and grade received.

The STUDENT table in Figure 8-24 is unnormalized because it has a repeating group. The STUDENT table design can be written as:

STUDENT (STUDENT NUMBER, STUDENT NAME, TOTAL CREDITS, GPA, ADVISOR NUMBER, ADVISOR NAME, OFFICE, (COURSE NUMBER, CREDIT HOURS, GRADE))

To convert the STUDENT record to 1NF, the primary key must be expanded to include the key of the repeating group, producing:

STUDENT (STUDENT NUMBER, STUDENT NAME, TOTAL CREDITS, GPA, ADVISOR NUMBER, ADVISOR NAME, OFFICE, COURSE NUMBER, CREDIT HOURS, GRADE)

Figure 8-25 shows the 1NF version of the sample STUDENT data. Do any fields in the 1NF STUDENT table depend on only a portion of the primary key? The student name, total credits, GPA, advisor number, and advisor name relate only to the student number and have no relationship to the course number. The course description depends on the course number rather than the student number. Only the GRADE field depends on the entire primary key.

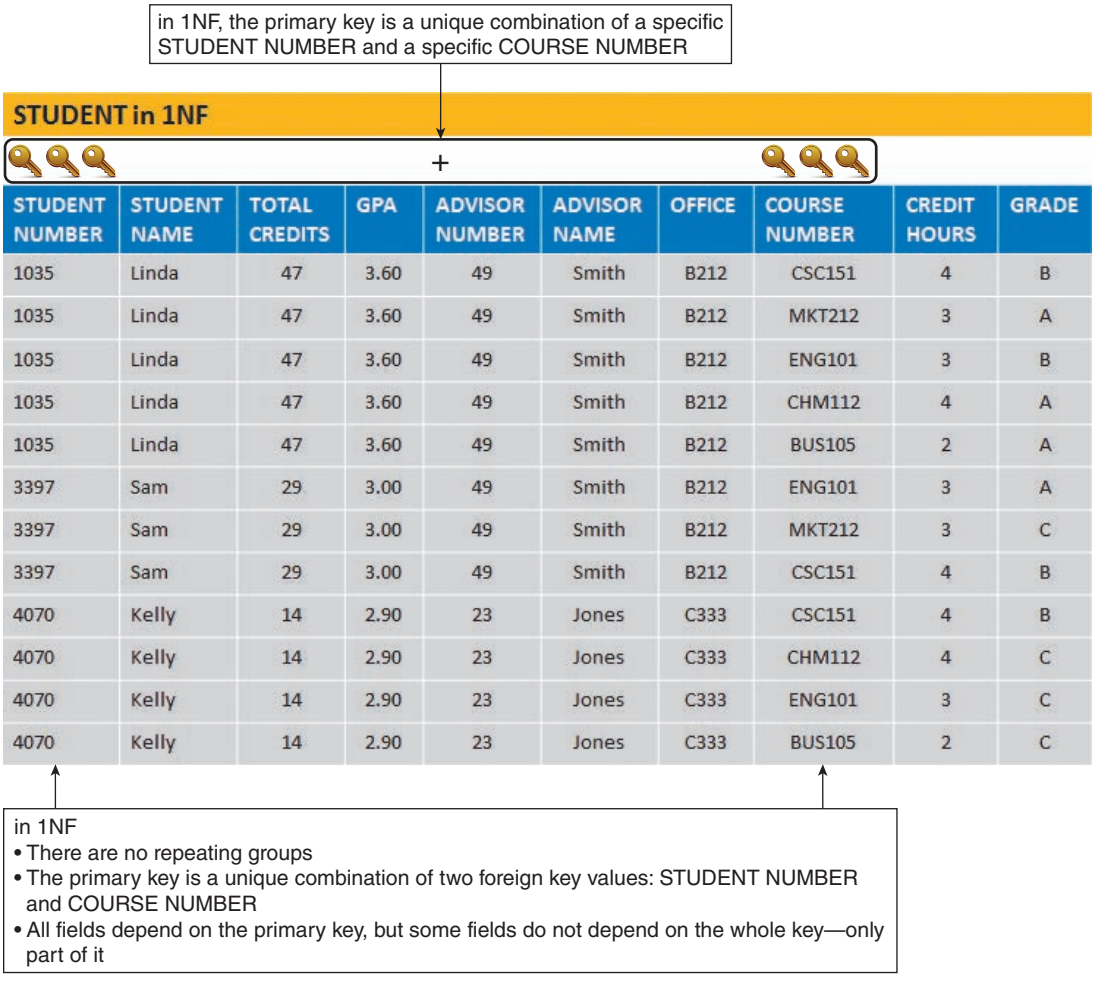
Following the 1NF–2NF conversion process described earlier, a new table would be created for each field and combination of fields in the primary key, and the other fields would be placed with their appropriate key. The result is:

STUDENT (STUDENT NUMBER, STUDENT NAME, TOTAL CREDITS, GPA, ADVISOR NUMBER, ADVISOR NAME, OFFICE)

COURSE (COURSE NUMBER, CREDIT HOURS)

GRADE (STUDENT NUMBER, COURSE NUMBER, GRADE)

Figure 8-25 The student table in 1NF. Notice that the primary key has been expanded to include STUDENT NUMBER and COURSE NUMBER.



STUDENT NUMBER	STUDENT NAME	TOTAL CREDITS	GPA	ADVISOR NUMBER	ADVISOR NAME	OFFICE	COURSE NUMBER	CREDIT HOURS	GRADE
1035	Linda	47	3.60	49	Smith	B212	CSC151	4	B
1035	Linda	47	3.60	49	Smith	B212	MKT212	3	A
1035	Linda	47	3.60	49	Smith	B212	ENG101	3	B
1035	Linda	47	3.60	49	Smith	B212	CHM112	4	A
1035	Linda	47	3.60	49	Smith	B212	BUS105	2	A
3397	Sam	29	3.00	49	Smith	B212	ENG101	3	A
3397	Sam	29	3.00	49	Smith	B212	MKT212	3	C
3397	Sam	29	3.00	49	Smith	B212	CSC151	4	B
4070	Kelly	14	2.90	23	Jones	C333	CSC151	4	B
4070	Kelly	14	2.90	23	Jones	C333	CHM112	4	C
4070	Kelly	14	2.90	23	Jones	C333	ENG101	3	C
4070	Kelly	14	2.90	23	Jones	C333	BUS105	2	C

The original 1NF STUDENT table has now been converted into three tables in 2NF. In each table, every non-key field depends on the entire primary key.

Figure 8-26 shows the 2NF STUDENT, COURSE, and GRADE designs and sample data. Are all three tables in 3NF? The COURSE and GRADE tables are in 3NF. STUDENT is not in 3NF because the ADVISOR NAME and OFFICE fields depend on the ADVISOR NUMBER field, separate from the STUDENT primary key. To convert STUDENT to 3NF, the ADVISOR NAME and OFFICE fields are removed from the STUDENT table and placed into a table with ADVISOR NUMBER as the primary key.

Figure 8-27 shows the 3NF versions of the sample data for STUDENT and ADVISOR. The final 3NF design is:

STUDENT (STUDENT NUMBER, STUDENT NAME, TOTAL CREDITS, GPA, ADVISOR NUMBER)

ADVISOR (ADVISOR NUMBER, ADVISOR NAME, OFFICE)

COURSE (COURSE NUMBER, CREDIT HOURS)

GRADE (STUDENT NUMBER, COURSE NUMBER, GRADE)

Figure 8-28 shows the complete ERD after normalization. There are four entities: STUDENT, ADVISOR, COURSE, and GRADE (an associative entity). Note that

Figure 8-26 The STUDENT, COURSE, and GRADE tables in 2NF. Notice that all fields functionally depend on their respective tables' primary keys.

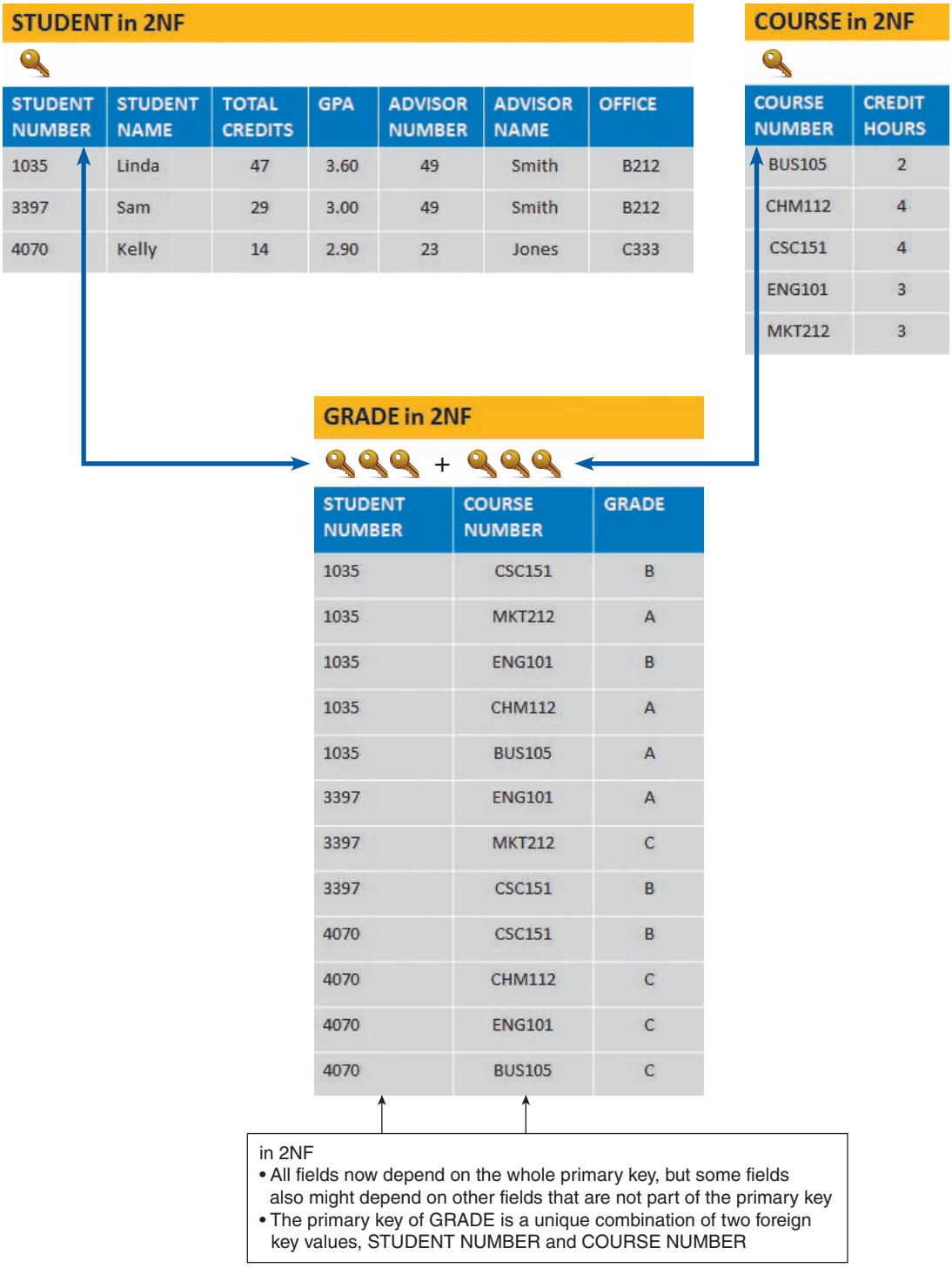
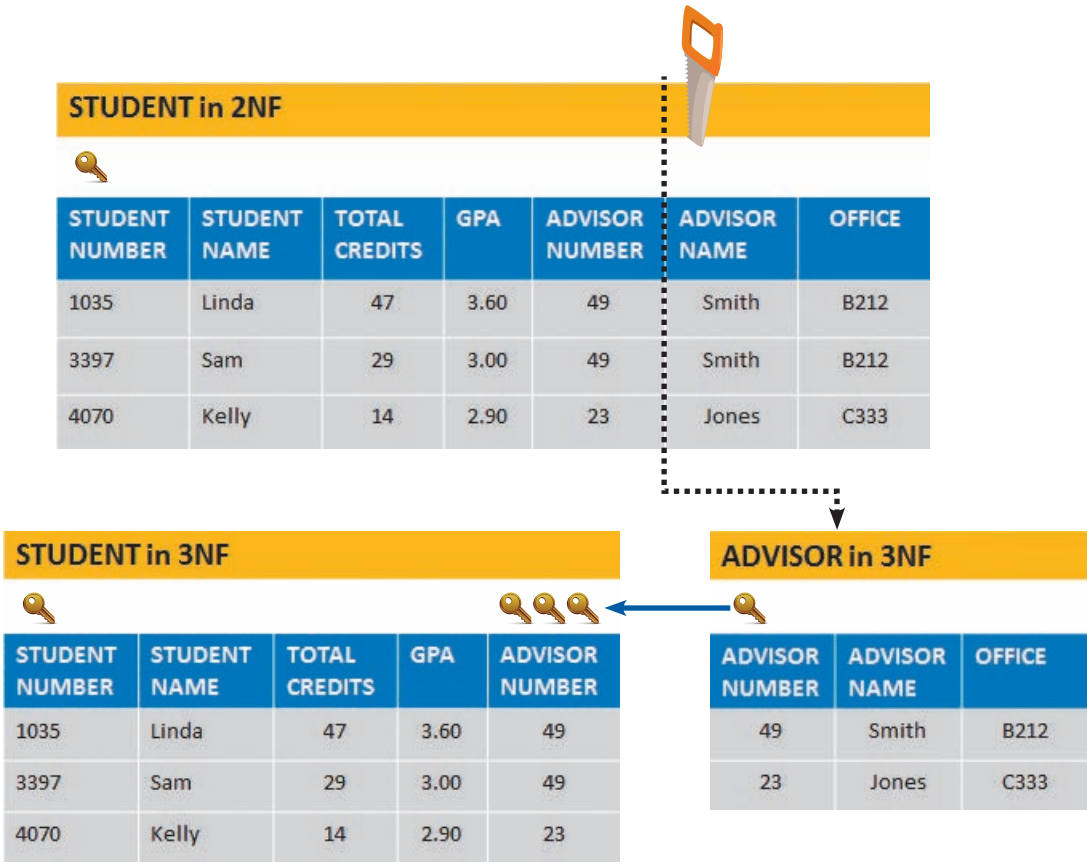


Figure 8-23 was drawn before GRADE was identified as an entity, but Figure 8-28 shows that the M:N relationship between STUDENT and COURSE has been converted into two 1:M relationships: one relationship between STUDENT and GRADE and the other relationship between COURSE and GRADE.

Figure 8-27 STUDENT and ADVISOR tables in 3NF. When the STUDENT table is transformed from 2NF to 3NF, the result is two tables: STUDENT and ADVISOR.



STUDENT in 2NF

STUDENT NUMBER	STUDENT NAME	TOTAL CREDITS	GPA	ADVISOR NUMBER	ADVISOR NAME	OFFICE
1035	Linda	47	3.60	49	Smith	B212
3397	Sam	29	3.00	49	Smith	B212
4070	Kelly	14	2.90	23	Jones	C333

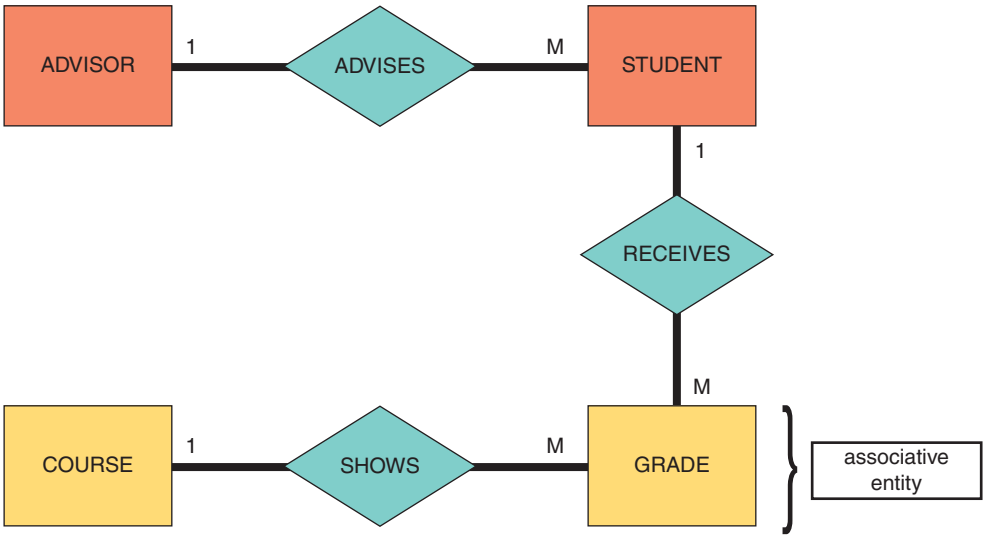
STUDENT in 3NF

STUDENT NUMBER	STUDENT NAME	TOTAL CREDITS	GPA	ADVISOR NUMBER
1035	Linda	47	3.60	49
3397	Sam	29	3.00	49
4070	Kelly	14	2.90	23

ADVISOR in 3NF

ADVISOR NUMBER	ADVISOR NAME	OFFICE
49	Smith	B212
23	Jones	C333

Figure 8-28 The ERD for STUDENT, ADVISOR, and COURSE after normalization. The GRADE entry was identified during the normalization process. GRADE is an associative entity that links the STUDENT and COURSE tables.



To create 3NF designs, the nature of first, second, and third normal forms must be understood. A systems analyst will encounter designs that are much more complex than the examples in this chapter.

8.7 Codes

The term "code" is commonly associated with application development in an information system. However, "coded data" in databases relates to specific ways of representing data, and "coding" in this context often means writing SQL stored procedures to manage and manipulate this data. In data design, a **code** is a set of letters and numbers encapsulating specific information in a standardized format. Codes can be used to simplify input, output, and data formats. Codes are crucial in organizing, managing, and utilizing data efficiently.

8.7.1 Overview of Codes

Because codes can represent data, they are encountered constantly in everyday life. Student numbers, for example, are unique codes to identify students in a school registration system. Three students named Jordan Nguyen might be enrolled at the same school, but only one is student number 268960.

A postal code is another typical example. A nine-digit postal code contains a lot of information. For example, the first digit identifies one of the 10 main geographical areas in the United States. Combining the next three numbers identifies a major city or distribution point. The fifth digit identifies an individual post office, an area within a city, or a specific delivery unit. The last four digits identify a post office box or street address.

For example, consider the zip code 32901-6975, shown in Table 8-2. This is called the "5+4" zip code format. The first digit, 3, indicates a broad geographical area in the southeastern United States. The following two digits, 29, indicate the area east of Orlando in Florida. The next two digits, 01, represent the city of Melbourne, Florida. The last four digits represent the Florida Institute of Technology location: 150 West University Boulevard.

Table 8-2 A zip code is an example of a significant digit code that uses groups and subgroups to store data. This example is for the zip code 32901-6975, the Florida Institute of Technology location in Melbourne, Florida.

Broad Geographical Location in Southeastern U.S.	FL Orlando East	Melbourne	Florida Tech 150 W. University Blvd.
3	29	01	6975

Codes can be used in many ways. Because codes are shorter than the data they represent, they save storage space and costs, reduce data transmission time, and decrease data entry time. Codes also can be used to reveal or conceal information. For example, the last two digits of a seven-digit part number might represent the supplier number or the maximum discount a salesperson can offer.

Finally, codes can reduce data input errors when the coded data is easier to remember and enter than the source data, when only specific valid codes are allowed, and when something within the code can provide immediate verification that the entry is correct.

8.7.2 Types of Codes

Companies use many different coding methods. Because information system users must work with coded data, the codes should be easy to learn and apply. User comments and feedback should be obtained if a company plans to create or change new codes.

Sequence Codes

Sequence codes are numbers or letters assigned in a specific order. Sequence codes contain no additional information other than an indication of the order of entry into the system. For example, a human resource system issues consecutive employee numbers to identify employees. Because the codes are assigned in the order in which employees are hired, the code can be used to see that employee number 584 was hired after employee number 433. The code, however, does not indicate the starting date of either person's employment.

Block Sequence Codes

Block sequence codes use blocks of numbers for different classifications. For example, college course numbers are usually assigned using a block sequence code. The 100-level courses, such as Chemistry 110 and Mathematics 125, are freshman-level, whereas course numbers in the 200s indicate sophomore-level courses. Within a particular block, the sequence of numbers can have some additional meaning, such as when English 151 is the prerequisite for English 152.

Alphabetic Codes

Alphabetic codes use letters to distinguish one item from another based on a category, an abbreviation, or an easy-to-remember value, called a mnemonic code. Many classification codes fit more than one of the following definitions:

- **Category codes** identify a group of related items. For example, a local department store uses a two-character category code to identify the department where a product is sold: GN for gardening supplies, HW for hardware, and EL for electronics.
- **Abbreviation codes** are alphabetic abbreviations. For example, standard state codes include NY for New York, ME for Maine, and MN for Minnesota. Some abbreviation codes are called **mnemonic codes** because they use a specific combination of letters that are easy to remember. Many three-character airport codes, such as those pictured in Figure 8-29, are mnemonic codes, such as ATL for Atlanta and MIA for Miami. However, some airport codes, such as YUL (Montréal) or MCO (Orlando), are not mnemonic.

Figure 8-29 Abbreviations for some of the world's busiest airports.
Source: BLANKartist/Shutterstock.com

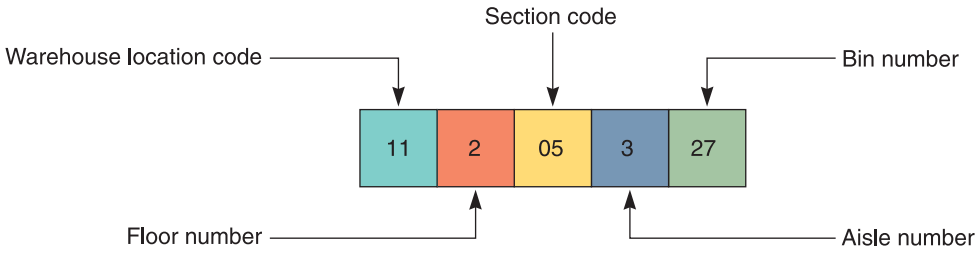
Significant Digit Codes

Significant digit codes distinguish items by using a series of subgroups of digits. Postal codes, for example, are significant digit codes. Other such codes include inventory location codes that consist of a two-digit warehouse code, followed by a one-digit floor number code, a two-digit section code, a one-digit aisle number, and a two-digit bin number code. Figure 8-30 illustrates the inventory location code 11205327. What looks like a sizeable eight-digit number is five separate numbers, each with significance.

Derivation Codes

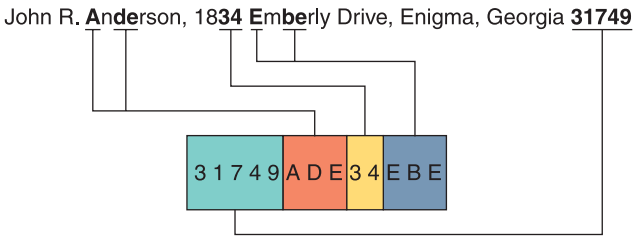
Derivation codes combine data from different item attributes, or characteristics. Most magazine subscription codes are derivation codes.

Figure 8-30 Sample of a code that uses significant digits to pinpoint the location of an inventory item.



For example, one popular magazine uses a subscriber's five-digit postal code, followed by the first, third, and fourth letters of the subscriber's last name, the last two digits of the subscriber's house number, and the first, third, and fourth letters of the subscriber's street name. A sample is shown in Figure 8-31.

Figure 8-31 A magazine subscriber code is derived from various parts of the name and address.



Cipher Codes

Cipher codes use a keyword to encode a number. A retail store, for example, might use a 10-letter word, such as CAMPGROUND, to code wholesale prices, where the letter C represents 1, A represents 2, and so on. Thus, the code GRAND indicates that the store paid $562.90 for the item.

Action Codes

Action codes refer to specific codes used to represent or trigger certain actions or processes. These codes can be part of a system, software, or protocol designed to initiate predefined activities or responses when used. For example, a student records program might prompt a user to enter or click an action code such as D (to display a record), A (to add a record), and X (to exit the program).

8.7.3 Designing Codes

Designing effective codes, whether they are for data categorization, identification, or geographic representation, requires careful planning and consideration of various factors. Well-designed codes contribute significantly to data organization, integrity, and usability.

Clarity and Meaningfulness

Codes should be designed to be as intuitive and meaningful as possible. They should convey information about the item or category they represent, making them easy to understand, remember, and use. For example, using SW as a code for the southwest sales region has far more meaning than code 14. Similarly, using ENG as the code for the English department is easier to interpret and remember than XVA or 132.

Consistency

Maintain a consistent structure across all codes. For example, if alphanumeric codes are used, keep the format consistent (like starting with letters followed by numbers) throughout all your coding systems. Similarly, if the payroll system already uses two-digit codes for departments, do not create a new, different coding scheme for the personnel system. Establish a consistent coding scheme if the two systems use different coding schemes.

Uniqueness

Each code should uniquely identify an element or category. There should be no ambiguity or overlap between codes to prevent confusion and errors in data handling. For example, if the code HW can indicate hardware or houseware, the code is not very useful.

Do not use a code to classify unrelated attributes. For example, suppose a single code is used to identify the combination of an employee's department and the employee's insurance plan type. In that case, users will have difficulty identifying all the subscribers of a particular plan, the workers in a specific department, or both. A separate code for each distinct characteristic makes much more sense.

Stability

Changes in regulations can cause consistency problems and require data updates. During the changeover period, all the stored occurrences of a particular code and all documents containing the old code will have to change as users switch to the new code. Usually, the old and new codes are used for an interim period, and special procedures are required to handle the two codes. For example, when phone area codes change, either area code (old or new) can be used for a certain period.

Scalability and Flexibility

At the same time, codes should be designed with future expansion in mind. Ensure the coding system can accommodate new categories or changes without requiring a complete overhaul. For example, if the company has eight warehouses, do not use a one-digit code for the warehouse number. If two more warehouses are added, the code must be increased to two digits or changed to a character code to identify each location.

The rule also applies to using a single letter as a character code because more than 26 data items might be needed. Of course, more characters can be added, which is just what the airline industry has done to accommodate new airports. Most airlines now use six-character codes that allow millions of combinations.

Simplicity and Brevity

Codes should be as simple and brief as possible while retaining their meaningfulness. Overly complex or long codes can lead to errors in data entry and difficulty in recall. For example, a six-digit code is not required to identify each of the 250 customers in a database.

Do not code some part numbers with two letters, a hyphen, and one digit and others with one letter, a hyphen, and two digits. Avoid allowing letters and numbers to occupy the same positions within a code because some are easily confused. This situation might be a good place to use an input mask to ensure the correct data type is entered.

Standardization

Standardization in designing codes refers to establishing and adhering to consistent rules and formats across the coding process. This is crucial for ensuring that codes are universally understandable, interoperable, and effective in their intended application. Standardization is essential for multinational corporations or systems used in many countries and cultures.

Government, industry, and professional standards should be followed where applicable. Regulatory bodies mandate standardized codes in many fields for compliance and reporting purposes. Adhering to these standards is not just best practice but a legal requirement.

Standardized codes are easier to teach and learn. New employees or system users can quickly become proficient in a system if they only have to learn a standard set of rules

and formats. This can also reduce the chances of errors in data entry, processing, and interpretation. A uniform set of rules dramatically diminishes the likelihood of misinterpreting a code or entering one incorrectly.

Avoidance of Ambiguous Characters

Avoiding ambiguous characters in code design is an essential guideline to ensure clarity and reduce errors in data interpretation. Ambiguous characters can be easily misread or mistaken for others, particularly when written, printed, or displayed on screens. Common examples include the number 0 (zero) and the letter O, or the number 1 (one), the lowercase letter l, and the uppercase letter I.

Such ambiguity can lead to significant data processing and interpretation errors in many coding systems, especially those involving manual data entry or visual scanning. For example, the five-character code 5Z081 can easily be misread as 5ZO8I or 52081. Therefore, avoiding these characters in codes is a key step in enhancing the accuracy and reliability of data handling.

In practice, this guideline means deliberately designing codes without characters that can be easily confused. For instance, a serial number or product code might exclude the number 0 and instead use only the letter O. Similarly, the number 1 might be avoided in favor of the uppercase letter I or lowercase letter l, depending on which set of characters is less prone to misinterpretation in the given context. This approach is essential in industries where accuracy is critical, such as healthcare for patient IDs.

Error Detection Capability

Error detection in code design ensures the accuracy and integrity of data in various systems. This process involves implementing specific mechanisms within codes that allow for the identification and correction of errors during data entry, transmission, or processing. Such mechanisms are particularly crucial in systems where precision is essential, such as financial transactions, healthcare data management, and critical communication systems. The goal is to minimize the impact of errors, ensuring that the data remains reliable and trustworthy.

One standard method employed in error detection is the use of check digits. These are additional digits appended to a code, calculated based on the other digits within the code. This method allows systems to verify the validity of the code during data entry or scanning. Check digits are widely used in various applications, including those that use bank account numbers, credit card numbers, and other identification numbers. They provide a simple yet effective way to catch common data entry errors, such as transposition or substitution of digits.

For more complex systems, advanced error detection algorithms are used. An example is the cyclic redundancy check (CRC), which is particularly effective for detecting alterations in large data sets. Additionally, parity checks are employed in binary coding systems; an extra bit is added to ensure that the total number of 1s in a string is even or odd, enabling the detection of single-bit errors. These sophisticated methods enhance data security and integrity, ensuring that the data transmitted across networks or stored in databases is accurate and consistent.

Accommodation of Sorting and Grouping

Design codes in a way that they can be easily sorted and grouped. This is particularly important for systems that require frequent data retrieval and analysis. For example, if products with three-digit codes in the 100s or the 300s are of one type, while products with codes in the 200s are a different type, a simple sort will not group all the products of one type. In addition, be careful that single-digit character codes will sort properly with double-digit codes. Sometimes, a leading zero must be added (01, 02, 03, and so on) to ensure that codes are sorted correctly.

Case in Point 8.3

Madera Tools

Madera Tools is a small business that specializes in hard-to-find woodworking tools. The firm advertises in various woodworking magazines and accepts mail and phone orders. Madera is planning a website for the firm's primary sales channel. The site will feature an online catalog, powerful search capabilities, and links to woodworking information and resources.

Madera has asked you, an IT consultant, whether a set of codes would be advantageous. What codes would you suggest? Provide at least two choices for a customer code and at least two choices for a product code. Be sure to describe your choices and provide specific examples. Explain why you selected these codes and what advantages they might offer.

8.8 Data Storage and Access

Data storage and access involve strategic business tools and methods, such as data warehousing and data mining software, logical and physical storage, selection of data storage formats, and special considerations for storing date fields.

8.8.1 Tools and Techniques

Companies use data warehousing and data mining as strategic tools to help manage the huge quantities of data they need for business operations and decisions. Many software vendors compete for business in this fast-growing IT sector.

Data Warehousing

Large firms maintain many databases, which might or might not be linked to an overall structure. To provide rapid access to this information, companies use software packages that organize and store data in special configurations called data warehouses. A **data warehouse** is an integrated collection of data that can include seemingly unrelated information, no matter where it is stored in the company. Because it can link various information systems and databases, a data warehouse provides an enterprise-wide view to support management analysis and decision making.

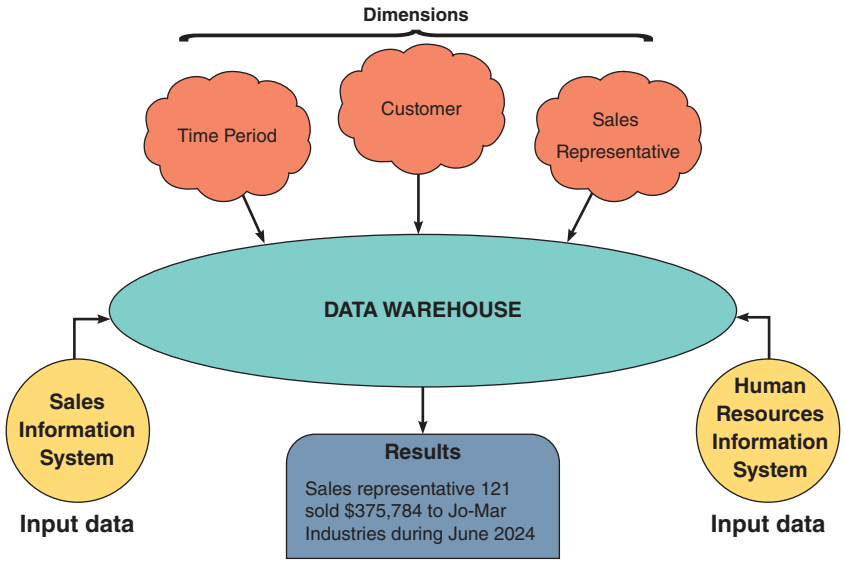
A data warehouse allows users to specify dimensions, or characteristics of data. Users can obtain multidimensional information from the stored data by selecting values for each characteristic. For example, in a typical company, most data is generated by transaction-based systems, such as order processing, inventory, and payroll systems. If a user wants to identify the customer on sales order 34071, they can retrieve the data quickly from the order processing system by entering an order number.

On the other hand, suppose that a user wants to see June 2024 sales results for the sales rep assigned to Jo-Mar Industries. The data is stored in two different systems with different databases: the sales information system and the human resources information system, as shown in Figure 8-32. Without a data warehouse, it would be difficult for a user to extract data that spans several information systems and time frames. Rather than accessing separate systems, a data warehouse stores transaction data in a format that allows users to retrieve and analyze the data quickly.

While a data warehouse typically spans the entire enterprise, many firms prefer a **data mart** designed to serve a specific department's needs, such as sales, marketing, or finance. Each data mart includes only the data that users in that department require to perform their jobs. Both approaches have pros and cons; the best solution usually depends on the situation.

Regardless of the overall approach, storing large quantities of data is like building a house—it doesn't just happen. A well-constructed data warehouse needs an architecture that includes detailed planning and specifications.

Figure 8-32 A data warehouse stores data from several systems. By selecting data dimensions, a user can retrieve specific information without knowing how or where the data is stored.



Data Mining

Data mining as a marketing tool has become increasingly significant in the era of big data and advanced analytics. It allows businesses to derive actionable insights from their data. It involves analyzing large data sets to uncover patterns, correlations, and insights that can help companies make informed decisions, personalize marketing efforts, predict customer behavior, and ultimately drive more efficient and effective marketing strategies.

In the current data-driven business landscape, the ability to effectively mine and analyze data can provide a significant competitive advantage, as follows:

- **Customer segmentation.** Data mining allows marketers to segment customers into distinct groups based on attributes like purchasing behavior, demographics, and preferences. This segmentation enables personalized marketing, ensuring that customers receive messages and offers that are most relevant to them, thus increasing the chances of conversion.
- **Predictive analytics.** Data mining helps predict future trends and customer actions by analyzing past customer behavior. Marketers use this information to anticipate customer needs and preferences, leading to more effective targeting strategies—for instance, predicting which customers are most likely to respond to specific promotions or which products they might be interested in next.
- **Market basket analysis.** This involves analyzing purchase patterns to understand the relationships between products that customers buy. Market basket analysis helps identify cross-selling and upselling opportunities by revealing which products are frequently bought together. This information is crucial for strategic product placement, promotions, and inventory management decisions.
- **Customer lifetime value (CLV) prediction.** Data mining enables businesses to predict the lifetime value of customers, helping them identify and focus on the most profitable segments. By understanding CLV, marketers can tailor their strategies to retain high-value customers and increase long-term profitability.
- **Sentiment analysis.** Marketers can gauge public sentiment toward their brand or products by mining data from social media, reviews, and customer feedback. This is invaluable for managing brand reputation, addressing customer concerns, and understanding the impact of marketing campaigns.
- **Churn analysis.** Data mining helps identify the warning signs that a customer is likely to stop using a company's products or services. Businesses can develop targeted retention strategies by understanding the factors contributing to customer churn.
- **Optimization of marketing campaigns.** Data mining provides insights that help fine-tune marketing campaigns for maximum effectiveness. This includes determining the

proper channels, the best time to launch campaigns, and the most appealing messages for different customer segments. It can also reduce **clicks to close**, the average number of page views needed to accomplish a purchase or obtain desired information.

Gathering information about customer behavior is sometimes called **clickstream storage**. Armed with this information, a skillful analyst could build a profile of typical new customers, returning customers, and customers who browse but do not buy. Although this information would be very valuable to the retailer, clickstream storage could raise serious legal and privacy issues if an unscrupulous firm sought to link a customer's web behavior to a specific name or email address and then sell or otherwise misuse the information.

8.8.2 Logical versus Physical Storage

It is essential to understand the difference between logical storage and physical storage. **Logical storage** refers to data a user can view, understand, and access, regardless of how or where the information is organized or stored. In contrast, **physical storage** is strictly hardware-related because it involves the process of reading and writing binary data to physical media such as a hard drive, a USB thumb drive, or a network-based storage device. For example, portions of a document might be stored in different physical locations on a hard drive, but the user sees the document as a single logical entity on the computer screen.

Logical storage consists of alphabetic and numeric **characters**, such as the letter A or the number 9. As described earlier in this chapter, a set of related characters forms a field representing a single characteristic, or attribute, of a person, place, thing, or event. A field is also called a data element or a data item.

When designing fields, space should be provided for the largest values that can be anticipated without allocating unnecessarily large storage capacities that will not be used. For example, suppose a customer order entry system is being designed for a firm with 800 customers. Limiting the customer number field to three or even four characters would be a mistake. Instead, a five-character field with leading zeros that could store customer numbers from 00001 to 99999 should be considered.

A mix of alphabetic and numeric characters can also be considered, as many people find it easier to view and use. Alphabetic characters expand the storage capacity because there are 26 possible values for each character position. Most airlines now use six alphabetic characters as a record locator with over 300 million possible values.

A **logical record** is a set of field values describing a person, place, thing, or event. For example, a logical customer record contains specific field values for a single customer, including the customer number, name, address, phone number, credit limit, and so on. Application programs see a logical record as a group of related fields, regardless of how or where the data is stored physically.

The term *record* usually refers to a logical record. Whenever an application program issues a read or write command, the operating system supplies one logical record to the program or accepts one logical record from the program. The physical data might be stored on one or more servers, in the same building or thousands of miles away, but all the application program sees is the logical record—the physical storage location is irrelevant.

8.8.3 Data Coding

Computers represent data as **bits** (**b**inary dig**its**) with only two possible values: 1 and 0. A computer understands a group of bits as a digital code that can be transmitted, received, and stored. Computers use data coding and storage schemes like EBCDIC, ASCII, and binary. A more recent coding standard called Unicode also is popular. In addition, the storage of dates raises some design issues that must be considered.

ASCII, EBCDIC, and Binary

ASCII (pronounced "ASK-ee"), an abbreviation of American Standard Code for Information Interchange, is a coding method used on most personal computers. **EBCDIC** (pronounced

"EB-see-dik"), an acronym for Extended Binary Coded Decimal Interchange Code, is a coding method used on mainframe computers and high-capacity servers. ASCII and EBCDIC require 8 bits, or 1 **byte**, for each character. For example, the name "Ann" requires 3 bytes of storage, 12,345 requires 5, and 1,234,567,890 requires 10.

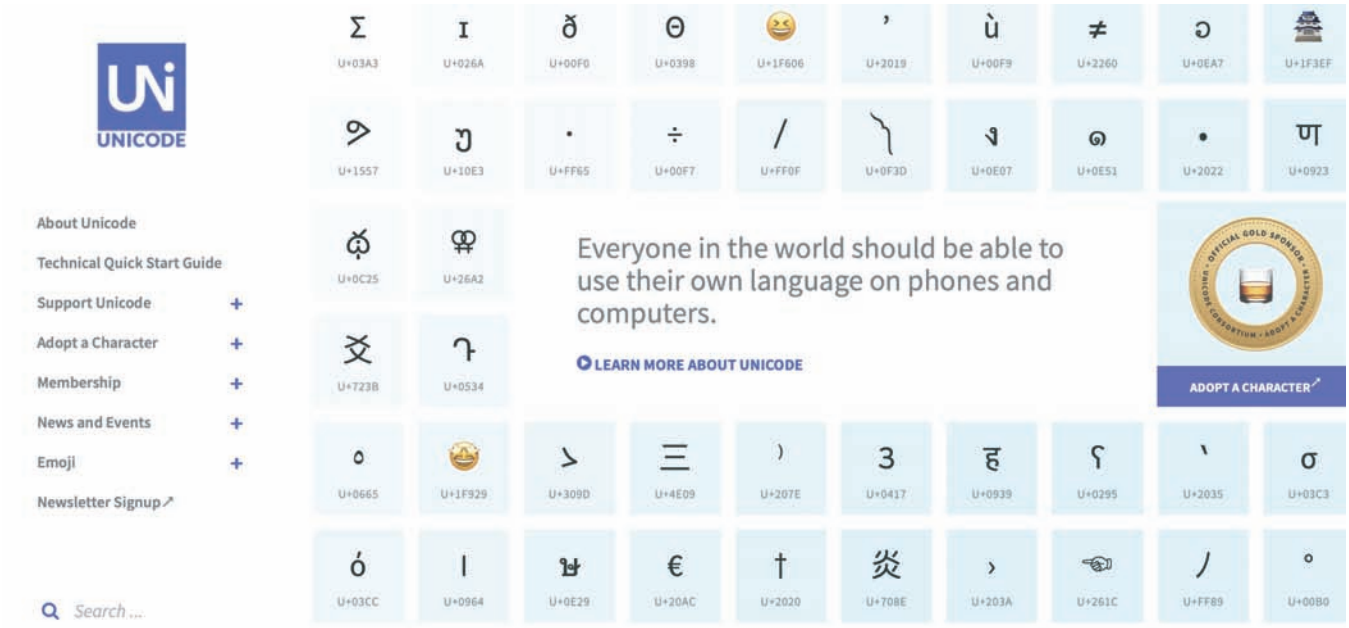
Compared with character-based formats, a **binary storage format** offers a more efficient storage method because it represents numbers as binary values rather than coded numeric digits. For example, an integer format uses only 16 bits, or 2 bytes, to represent 12,345 in binary form. A long integer format uses 32 bits, or 4 bytes, to represent 1,234,567,890 in binary form.

Unicode

Unicode is a more recent coding standard that uses 2 bytes per character rather than one. This expanded scheme enables Unicode to represent more than 65,000 unique, multilingual characters. Why is this important? Consider the challenge of running a multinational information system or developing a program that will be sold in Asia, Europe, and North America. Because it supports virtually all languages, Unicode has become a global standard.

Traditionally, domestic software firms developed a product in English and then translated the program into one or more languages. This process was expensive, slow, and error-prone. In contrast, Unicode creates translatable content right from the start. Today, most popular operating systems support Unicode, and the Unicode Consortium maintains standards and support, as shown in Figure 8-33.

Figure 8-33 Unicode is an international coding format representing characters as integers, using 16 bits (2 bytes) per character. The Unicode Consortium maintains standards and support for Unicode.
Source: Unicode, Inc.

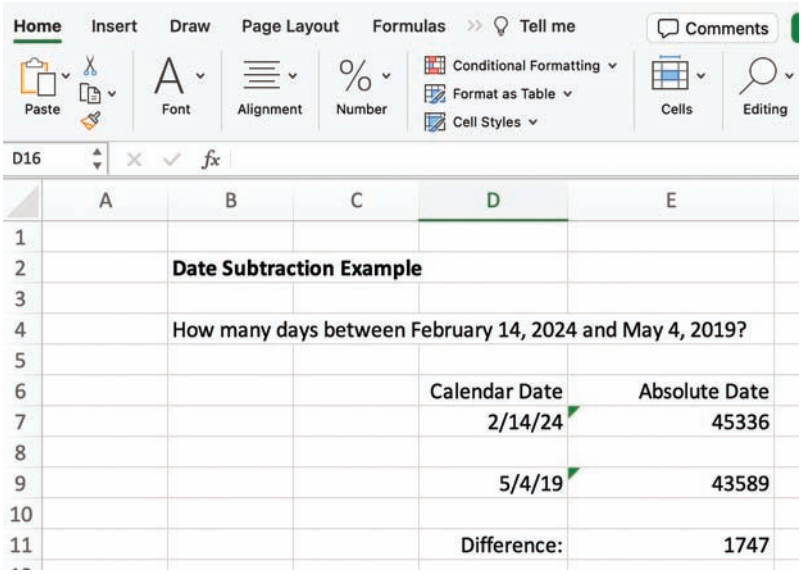


Storing Dates

The best way to store dates depends on how they will be displayed and whether they will be used in calculations.

At the beginning of the 21st century, many firms that used only two digits to represent the year faced a major problem called the **Y2K issue**. Based on that experience, most date formats are based on the model established by the **International Organization for Standardization (ISO)**, which requires four digits for the year, two for the month, and two for the day (YYYYMMDD). A date stored in that format can be sorted quickly and used in comparisons. If a date in ISO form is "larger" than another date in the same form, the first date is later. For example, 20150504 (May 4, 2015) is later than 20130927 (September 27, 2013).

Figure 8-34 Microsoft Excel uses absolute dates in calculations. In this example, the absolute date of February 14, 2024, is 45336, and May 4, 2019, is 43589. The difference between the dates is 1747 days.
Source: Microsoft Corporation



What if dates must be used in calculations? For example, if a manufacturing order placed on June 23 takes three weeks to complete, when will the order be ready? If a payment due on August 13 is paid on April 27 of the following year, how late is the payment, and how much interest is owed? In these situations, it is easier to use absolute dates.

An **absolute date** is the total number of days from some specific base date. One date is subtracted from the other to calculate the days between two absolute dates. For example, if the base date is January 1, 1900, then May 4, 2015, has an absolute date of 42128. Similarly, September 27, 2013, has an absolute date value of 41544. If the earlier date value is subtracted from the later one, the result is 584 days. A spreadsheet can quickly determine and display absolute dates, as shown in Figure 8-34.

8.9 Data Control

Just as it is essential to secure the physical part of the system, as shown in Figure 8-35, file and database control must include all measures necessary to ensure that data storage is correct, complete, and secure. File and database control also relate to input and output techniques discussed earlier.

Figure 8-35 Besides network monitoring, system security includes access codes, data encryption, passwords, and audit trails.
Source: Gorodenkoff/Shutterstock.com

A well-designed DBMS must provide built-in control and security features, including subschemas, passwords, encryption, audit trail files, and backup and recovery procedures to maintain data. The analyst's primary responsibility is to ensure that the DBMS features are correctly used.

Earlier in this chapter, it was explained that a subschema can provide a limited view of the database to a specific user or level of users. Limiting access to files and databases is the most common way of protecting stored data. Users must furnish a proper user ID and password to access a file or database. Different privileges, also called **permissions**, can be associated with different users so that some employees can be limited to read-only access. In contrast, other users might be allowed to update or delete data. Additional access codes can be established for sensitive data, restricting specific records or fields within records. Stored data also can be encrypted to prevent unauthorized access. Encryption is converting readable data into unreadable characters to prevent unauthorized access to the data.

All system files and databases must be backed up regularly, and **backup** copies must be retained for a specified period. In the event of a file catastrophe, **recovery procedures** can be used to restore the file or database to its current state at the time of the last backup. **Audit log files**, which record details of all accesses and changes to the file or database, can be used to recover changes made since the last backup. **Audit fields**, which are special fields within data records to provide additional control or security information, can also be included. Typical audit fields include the date the record was created or modified, the name of the user who acted, and the number of times the record has been accessed.

Summary

This chapter continued the study of the systems design phase of the SDLC. The chapter focused on how data is organized, stored, and managed. The data represents people, places, things, or events that affect the information system.

A database consists of linked tables that form an overall data structure. A database management system (DBMS) is a collection of tools, features, and interfaces that enable users to add, update, manage, access, and analyze data in a database. DBMS designs are more powerful and flexible than traditional file-oriented systems.

DBMS components include interfaces for users, database administrators, and related systems, a data manipulation language, a schema, and a physical data repository. The schema includes descriptions of all fields, tables, and relationships in the database. The data dictionary is transformed into a physical data repository during data design.

In a web-based design, the Internet serves as the front end, or interface, for the DBMS. Access to the database requires only a web browser client and an Internet connection. Middleware can interpret client requests and translate them into commands that the database can execute. For a web-based system to be secure, it must provide security at three levels: the database, the web server, and the telecommunication links connecting the system components.

In an information system, an entity is a person, place, thing, or event for which data is collected and maintained. A field, or attribute, is a single characteristic of an entity. A record, or tuple, is a set of related fields describing one entity instance. Data is stored in files (in a file-oriented system) and tables (in a database environment).

A primary key is the field or field combination that uniquely and minimally identifies a specific record; a candidate key is any field that could serve as a primary key. A foreign key is a field or field combination that must match the primary key of another file or table. A secondary key is a field or field combination used as the basis for sorting or retrieving records.

An entity-relationship diagram (ERD) is a graphic representation of all system entities and the relationships among them. The ERD is based on entities and data stores in DFDs prepared during the systems analysis phase. The three fundamental relationships represented in an ERD are one-to-one (1:1), one-to-many (1:M), and many-to-many (M:N). In a M:N relationship, the two entities are linked by an associative entity.

The relationship between two entities is also referred to as cardinality. A common form of cardinality notation is called crow's foot notation, which uses various symbols to describe the characteristics of the relationship.

Normalization is a process for avoiding problems in data design. A first normal form (1NF) record has no repeating groups. A record is in second normal form (2NF) if it is in 1NF and all non-key fields depend on the entire primary key. A record is in third normal form (3NF) if it is in 2NF and no field depends on a non-key field.

A code is a set of letters or numbers used to represent data in a system. Using codes can speed up data entry, reduce data storage space, and reduce transmission time. Codes can also be used to reveal or conceal information. The main types of codes are sequence codes, block sequence codes, classification codes, alphabetic codes, significant digit codes, derivation codes, cipher codes, and action codes.

Logical storage is information seen through a user's eyes, regardless of how or where that information is organized or stored. Physical storage is hardware-related and involves reading and writing binary data to physical media. A logical record is a related set of field values describing a person, place, thing, or event. Data storage formats include ASCII, EBCDIC, binary, and Unicode. Dates can be stored in several formats, including ISO and absolute format.

File and database control measures include limiting access to the data, data encryption, backup/recovery procedures, audit trail files, and internal audit fields.

Key Terms

1:1
1:M
abbreviation code
absolute date
action code
alphabetic code
alternate key
ASCII
associative entity
audit fields
audit log files
backup
binary storage format
bit
block sequence code
byte
candidate key
cardinality notation
category codes
character
churn analysis
cipher codes
clicks to close
clickstream storage
code
combination key
common field
composite key
concatenated key
crow's foot notation
customer lifetime value (CLV) prediction
customer segmentation
data manipulation language (DML)
data mart
data mining
data warehouse
database administrator (DBA)
database management system (DBMS)
derivation code
EBCDIC
economy of scale
entity-relationship diagram (ERD)
file
file-oriented system
first normal form (1NF)
foreign key
functionally dependent
International Organization for Standardization (ISO)
JDBC (Java database connectivity)
key fields
logical record
logical storage
M:N
many-to-many relationship
market basket analysis
middleware
mnemonic code
multivalued key
non-key field
normalization
ODBC (open database connectivity)
one-to-many relationship
one-to-one relationship
orphan
permissions
physical storage
predictive analytics
primary key
query by example (QBE)
query language
recovery procedure
referential integrity
relational database
relational model
repeating group
schema
second normal form (2NF)
secondary key
sentiment analysis
sequence code
significant digit code

SQL (Structured Query Language)
standard notation format
subschema
super key
surrogate key
table
table design
third normal form (3NF)
tuple
Unicode
unique key
unnormalized
Y2K issue

Review Questions

1. What is a data structure?
2. Briefly describe the components of a DBMS.
3. List the major characteristics of web-based design.
4. Explain the primary, super, candidate, foreign, and secondary keys.
5. What are entity-relationship diagrams, and how are they used?
6. How do you convert an unnormalized design to 1NF? In your answer, refer to specific pages and figures in this chapter.
7. How are codes used in data design?
8. What is data warehousing and data mining?
9. How would a specific date be represented as an absolute date?
10. How are permissions used to control access to data?

Discussion Topics

1. In the auto shop examples in Section 8.1.2, what problems might arise in Mario's system? Why won't Danica run into the same problems? Provide specific examples in your answer.
2. Many large organizations have had their database system hacked and customer data stolen. How should the database's security differ from the rest of the system? Does it make a difference for web-based data designs? If so, how?
3. Suggest three typical business situations in which referential integrity avoids data problems.
4. Name some of the codes used in our personal and business lives.
5. Are there ethical issues to consider when planning a database? For example, should sensitive personal data (such as medical information) be stored in the same DBMS that manages employee salary and benefits data? Why or why not?

Hands-On Projects

1. Consider an automobile dealership with three locations. Data fields exist for stock number, vehicle identification number, make, model, year, color, and invoice cost. Identify the possible candidate keys, the likely primary key, a probable foreign key, and potential secondary keys.
2. Visit the bookstore at your school or in your area. Interview the manager or store employees to learn more about the business and the entities involved in bookstore operations. Remember that an entity is a person, place, thing, or event that affects an information system. Draw an ERD, including cardinality, that describes the bookstore's operations.

3. Cludadwy Chairs sells a patented seat that spectators can take to youth soccer games. The seat folds so it is small enough to fit in the glove box of most vehicles. The company operates a Kansas factory and contracts its manufacturing projects to small firms in Canada and Mexico.

   An unusual problem has occurred for this small multinational company: people need clarification about dates in internal memos, purchase orders, and emails. When the company's database was initially designed, the designer was unaware that the format for dates in Canada and Mexico differed from those used in the United States. For example, in Canada and Mexico, the notation 7/1/24 indicates January 7, 2024, whereas the same notation indicates July 1, 2024, in the United States. Although it seems minor, the date confusion has resulted in several order cancellations.

   Cludadwy Chairs has asked for your advice. You could suggest writing a simple program to convert the dates automatically or design a switchboard command allowing users to select a date format as data is entered. You realize that Cludadwy Chairs might want to do business in other countries. What would be the best course of action? Should the company adapt to each country's standard or maintain a single international format? What are the arguments for each option?
4. Use Microsoft Access or similar database software to create a DBMS for the imaginary company TopText Publishing described in Case in Point 8.1. Add several sample records to each table and report to the class on your progress.
5. Consider a database system in an organization where employees have varying access levels based on their job roles. Your task is to design a user privilege structure that categorizes users into different groups and assigns appropriate permissions to each group. Specifically, ensure that some employees are limited to read-only access while others have more extensive privileges, such as read-write or administrative access. Describe the steps you would take to implement this structure, including defining user roles, assigning privileges to each role, and ensuring that these privileges are enforced in the database system. Additionally, provide an example scenario illustrating how different employees would interact with the database based on their assigned privileges.

Ethical Issues

Tip Top Toys is a relatively small division of Worldwide Enterprises. Worldwide has nine other divisions, which include insurance, healthcare products, and financial planning services, to name a few.

The corporate marketing director for Worldwide has requested Tip Top's customer shopping data to target people who might be likely to purchase items or services from other Worldwide divisions. The database manager is uncomfortable with this and pointed out that Tip Top's web privacy policy states, "Tip Top Toys, a division of Worldwide Enterprises, will not share personal data with other companies without a customer's consent."

The marketing director replied that the statement only applies to outside companies—not other Worldwide divisions. The marketing director checked with the corporate legal department, and they agreed. The database manager replied, "Even if it is legally OK, it's not the right thing to do. Many believe our statement means their data does not leave Tip Top. At the very least, we should give customers a choice and share the data only with their consent."

Do you agree with the marketing director? Why or why not?

Chapter

9

System Architecture

Learning Objectives

When you finish this chapter, you should be able to:

1. Provide a checklist of issues to consider when selecting a system architecture.
2. Trace the evolution of system architecture from mainframes to current designs.
3. Explain client/server architecture.
4. Describe the impact of the Internet on system architecture.
5. Compare in-house e-commerce development with packaged solutions and service providers.
6. Explain online and batch processing.
7. Describe network models and topologies.
8. Explain network devices, including routers, gateways, and proxy servers.
9. Describe wireless networking, including standards, topologies, and trends.

Chapter 9 is the final chapter in the systems design phase of the SDLC. This chapter describes system architecture, which translates the logical design of an information system into a physical blueprint. Designing the system architecture requires consideration of servers, clients, processing methods, networks, and related issues.

The chapter includes three "Case in Point" discussion questions to help contextualize the concepts described in the text. The scenario described in the "Ethical Issues" section explores the trade-offs between the potential benefits of installing tracking software on users' computers to improve the system's response to typical usage patterns versus the ethical issues related to invasion of privacy by constant monitoring of employees' online activity.

9.1 Architecture Checklist

At this point in the SDLC, the objective is to determine an overall architecture to implement the information system. **System architecture** translates the logical design of an information system into a physical structure that includes hardware, software, data, network support, processing methods, and security. System architecture is an organized framework that conceptualizes system elements, their relationships, and the principles guiding its design and evolution. It is a vital aspect of any complex system development. It acts as a roadmap for the project, guides the team through the intricacies of design and implementation, and ensures that the end product is effective, efficient, and aligned with the intended purpose.

Hardware architecture involves the physical components of a system, like servers, computers, networks, and data centers. Software architecture includes the design decisions for software applications, encompassing the frameworks, programming languages, and interfaces selected. Data architecture focuses on how the system stores, manages, and processes data. Network architecture relates to the design of communication networks, including the layout of network devices and connections.

The end product of the systems design phase is the system design specification. If this document is approved, the next step is systems implementation. Just as an architect begins a project with a list of the owner's requirements, a systems analyst must consider several issues affecting the architecture choice. This is done with an overall architecture checklist:

- Corporate organization and culture
- Enterprise resource planning
- Scalability
- Web integration
- Legacy systems
- Processing options
- Security
- Compliance
- Corporate portals

9.1.1 Corporate Organization and Culture

To be successful, an information system must perform well in a company's organization and culture. For example, consider two large bicycle brands, Green Bikes and Blue Bikes. Each firm has three operating divisions: an Asian subsidiary that manufactures the bicycles, a factory in Los Angeles that produces bike accessories and clothing, and a Canada-based plant that makes bike carriers, racks, and custom trailers.

The two firms are similar but have very different organizations and corporate cultures. Green Bikes is highly centralized and oversees day-to-day operations from its Los Angeles office. Blue Bikes also has a Los Angeles executive office but allows its three business units to operate separately, with minimal corporate oversight. Both firms are successful, and their managerial styles will likely remain the same.

Suppose both firms asked a consultant to suggest an IT architecture to boost productivity and reduce costs. How might corporate organization and culture issues affect the consultant's recommendation? There is no easy answer to that question. The best approach would probably be to study day-to-day business functions, talk to users at all levels, and focus on operational feasibility issues, as was done earlier in the development process.

9.1.2 Enterprise Resource Planning

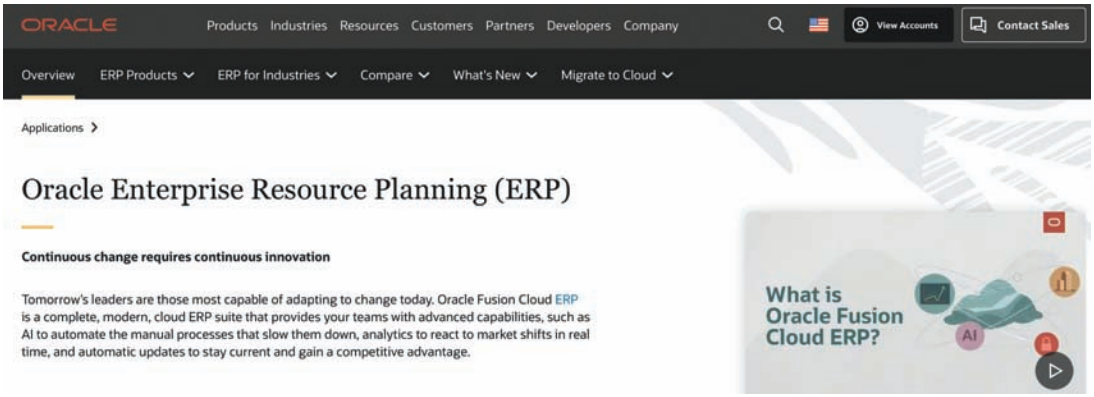
Many companies use enterprise resource planning (ERP) software to establish a company-wide strategy for using IT that includes a specific architecture and standards for data, processing, networks, and user interface design. A primary advantage of ERP is that it

describes a particular hardware and software environment, also called a **platform**, which ensures connectivity and easy integration of future systems, including in-house software and commercial packages.

Many companies are extending internal ERP systems to their suppliers and customers using a supply chain management (SCM) concept. For example, in an integrated supply chain system, a customer order could cause a manufacturing system to schedule a work order, which triggers a call for more parts from one or more suppliers. In a dynamic, highly competitive economy, SCM can help companies achieve faster response, better customer service, and lower operating costs.

Oracle is an example of a company offering ERP solutions. As shown in Figure 9-1, Oracle's ERP products are cloud-based services that support employee collaboration, provide access to information from mobile devices, and deliver data analytics capabilities to gain insight into business processes.

Figure 9-1 Oracle offers ERP solutions as a cloud-based service.
Source: Oracle



9.1.3 Scalability

A network is composed of individual nodes. A **node** represents a physical device, wired or wireless, that can send, receive, or manage network data. For example, nodes can be servers, computers, shared printers, mass storage devices, wireless access points, or tablets.

Scalability, sometimes also called **extensibility**, refers to a system's ability to expand, change, or downsize to meet the changing needs of a business enterprise. Scalability is crucial in implementing volume-related systems, such as transaction processing systems. A scalable system is necessary to support a dynamic, growing business.

For example, a scalable network could handle anywhere from a few dozen to thousands of nodes, and a scalable DBMS could support the acquisition of an entire new sales division. When investing large amounts of money in a project, management is especially concerned about scalability issues affecting the system's life expectancy.

9.1.4 Web Integration

An information system includes **applications**, which are programs that handle input, manage processing logic, and provide required output. The systems analyst must know if a new application will be part of an e-commerce strategy and the degree of integration with other web-based components.

A **web-centric** architecture is a system design with core functionalities and services built around web technologies and Internet protocols. This enables a company to integrate the new application into its e-commerce strategy. In this architecture, web servers, browsers, and Internet-based technologies are central to the system's operation, facilitating communication, data exchange, and application delivery. Even where e-commerce is not involved, a web-centric application can run on the Internet or a company intranet or extranet.

A web-based application avoids many connectivity and compatibility problems from different hardware environments. A firm's external business partners can import and export data in a web-based environment using standard web browsers. This approach is particularly prevalent in modern system development, emphasizing scalability, accessibility, and cross-platform compatibility through web-based interfaces and services.

9.1.5 Legacy Systems

A new system might have to interface with one or more legacy systems, which are older systems that use outdated technology but still are functional. These systems are often critical to the daily operations of an organization but may pose challenges due to their age, incompatibility with modern systems, and difficulty in maintenance and integration. Legacy systems typically persist due to the high costs, risks, and complexities involved in replacing or upgrading them and the potential disruption to business operations.

Interfacing a new system with a legacy system involves analysis of data formats and compatibility. Sometimes, a company will need to convert legacy file data, which can be expensive and time-consuming. For example, a new marketing information system might need to report sales data to a server-based accounting system and obtain product cost data from a legacy manufacturing system. Middleware, discussed later in this chapter, might be required to pass data between new and legacy systems.

To select the best architecture, the analyst must know if the new application will eventually replace or coexist with the legacy system. Coexisting with a legacy system presents significant challenges, including integration difficulties, as these outdated systems often struggle to communicate effectively with modern technologies, leading to fragmented and complex IT architectures. Maintenance and support become increasingly challenging with the need for specialized skills and the high costs of keeping older systems operational. Additionally, legacy systems pose security risks due to their incompatibility with current security standards and protocols, necessitating extra protective measures and potentially exposing the organization to cyber threats.

9.1.6 Processing Options

In planning the architecture, designers also must consider how the system will process data: online or in batches. Online and batch processing are two fundamentally different data processing approaches, and each significantly impacts system architecture. The choice between these methods depends on the task's nature, the data volume, and the required processing speed.

For example, a high-capacity transaction processing system, such as an order entry system, requires more network, processing, and data storage resources than a monthly billing system that handles data in batches. Also, if the system must operate online, 24 hours a day and seven days a week (24/7), provision must be made for backup and speedy recovery in case of system failure.

In many cases, systems incorporate both online and batch processing for different aspects of their operations. This hybrid approach can offer the benefits of real-time processing for critical tasks while leveraging batch processing for less time-sensitive functions. The system architecture in such cases becomes more complex, as it must support the characteristics of both processing types, often leading to a multilayered architecture where different layers or components are optimized for either online or batch processing. These characteristics are described in Section 9.6, with examples of each type.

9.1.7 Security

The role of architecture in system security is pivotal, as it lays the foundational framework for implementing robust security measures throughout the system's life cycle. A well-designed architectural framework integrates security considerations from the onset, embedding them into every system layer, from hardware and network infrastructure to software applications

and data management. This approach ensures that security is not an afterthought but a core component of the system's design, facilitating identifying and mitigating vulnerabilities, enforcing data integrity, and maintaining confidentiality and availability of services. By prioritizing security in the architectural design, organizations can effectively safeguard against potential threats and cyberattacks, ensuring a resilient and secure environment for their operations and data.

Web-based systems introduce additional security concerns, as critical data must be protected in the Internet environment. From the simple password protection shown in Figure 9-2 to complex intrusion detection systems, security threats and defenses are a significant concern in this context. Security is critical when data or processing is performed at remote locations rather than a centralized facility.

Figure 9-2 User IDs and passwords are traditional system security elements.
Source: JMiks/Shutterstock.com

Firms that use e-commerce applications must assure customers that their data is safe and secure. The lamentable number of high-profile security breaches in large corporations in recent years suggests that security issues should play an even more prominent role in system architecture considerations. System security concepts and strategies are discussed in detail later in the book.

9.1.8 Compliance

Ensuring compliance with legal and regulatory standards is essential to system architecture. This process thoroughly examines the system to ensure it aligns with all relevant laws and industry-specific regulations. These standards can vary widely, depending on factors such as the geographical location of the organization and the specific industry it operates in. For instance, systems that handle personal data need to adhere to privacy laws like the General Data Protection Regulation (GDPR) in the European Union or the Health Insurance Portability and Accountability Act (HIPAA) in the United States healthcare sector.

Noncompliance with these legal and regulatory requirements can seriously affect an organization. It can lead to hefty legal penalties, including fines and sanctions, which can be financially damaging. Beyond the direct financial impact, noncompliance can harm an organization's reputation and erode customer trust. This loss of trust can have long-term implications for customer relationships and business sustainability.

Given these stakes, organizations must conduct regular and thorough systems reviews, and the chosen architecture must support these reviews. These audits should assess compliance with all applicable laws and regulations. Maintaining compliance is an ongoing process as legal and regulatory environments continually evolve. Keeping up to date with these changes and adapting the system architecture accordingly are vital to ensure continuous compliance and protect the organization from potential risks associated with noncompliance.

9.1.9 Corporate Portals

Depending on the system, the planned architecture might include a corporate portal. A **portal** is an entrance to a multifunction website. After entering a portal, a user can navigate to a destination using various tools and features provided by the portal designer. A **corporate portal** provides access to customers, employees, suppliers, and the public. A well-designed portal can integrate with various other systems and provide a consistent look and feel across multiple organizational divisions.

Incorporating a corporate portal into an organization's system architecture demands careful planning and consideration of various factors. It requires a balance among integration

capabilities, scalability, security, user experience, and efficient data management. While the ultimate goal is to create a seamless, secure, and effective environment for users to access and interact with corporate resources and services, several issues must be considered.

Implementing a corporate portal significantly impacts system architecture by necessitating robust integration and interoperability with various existing systems. This includes legacy systems, third-party applications, and modern cloud-based services. To achieve a smooth integration, the architecture often requires the incorporation of middleware or extensive use of APIs. This integration layer ensures that disparate systems can communicate effectively, but it also adds complexity to the overall system architecture, requiring careful design and management to maintain system efficiency and reliability.

Corporate portals also influence system architecture through their scalability and performance demands. The portal must be capable of handling varying levels of user traffic and data load, necessitating an architecture that can dynamically adjust and allocate resources as needed. This includes implementing caching and load-balancing techniques to ensure the portal remains responsive and available, even under high demand. Such considerations require a scalable and flexible architectural approach, often leading to the adoption of cloud-based solutions or distributed architectures that can efficiently manage the load and maintain optimal performance.

Introducing a corporate portal places heightened emphasis on security within the system architecture. A portal often provides access to sensitive business data and personal information, so robust security mechanisms are crucial. This includes secure authentication and authorization processes to control access and measures to protect data in transit and at rest. The architecture must comply with various data protection standards and regulations, requiring a comprehensive security strategy encompassing network security, data encryption, and regular security audits to safeguard against potential breaches and vulnerabilities.

Case in Point 9.1

ABC Systems

You are a systems analyst at ABC Systems, a fast-growing IT consulting firm that provides a wide range of services to companies that want to establish e-commerce operations. During the last 18 months, ABC acquired two smaller firms and set up a new division specializing in supply chain management. Aligning ABC's internal systems was challenging, and top management was unhappy with the integration cost and the timetable. To avoid future problems, you suggest an ERP strategy and plan to present your views at tomorrow's staff meeting. ABC's management team is very informal and prefers a loose, flexible management style. How will you persuade them that ERP is the way to go?

9.2 The Evolution of System Architecture

Every business information system must carry out three main functions:

1. Manage applications that perform the processing logic.
2. Handle data storage and access.
3. Provide an interface that allows users to interact with the system.

Depending on the architecture, the three functions are performed on a server or a client or are divided between the server and client. During system design, the analyst must determine where the functions will be carried out and ascertain the advantages and disadvantages of each design approach.

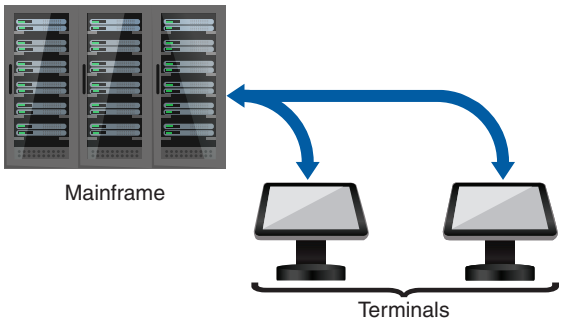
9.2.1 Mainframe Architecture

A **server** is a computer that supplies data, processing services, or other support to one or more computers, called **clients**. The earliest servers were mainframe computers, and a system design in which the server performs all the processing is sometimes described as **mainframe architecture**. Although the server does not have to be a mainframe, the term "mainframe architecture" typically describes a multiuser environment in which the server is significantly more powerful than the clients. A systems analyst should know the history of mainframe architecture to understand the server's role in modern system design.

In the 1960s, mainframe architecture was the only choice. In addition to centralized data processing, the earliest systems performed all data input and output at a central location, often called a **data processing center**. Data was delivered or transmitted in some manner to the data processing center, where it was entered into the system. Users in the organization had only input or output capability for printed reports that a corporate IT department distributed.

As network technology advanced, companies installed terminals at remote locations so that users could enter and access data from anywhere in the organization, regardless of where the centralized computer was located. A terminal included a keyboard and display screen to handle input and output but lacked independent processing capability. In a centralized design, as shown in Figure 9-3, the remote user's keystrokes are transmitted from their terminal to the mainframe, which responds by sending screen output back to the user's screen.

Figure 9-3 In a centralized design, the remote user's keystrokes are transmitted to the mainframe, which responds by sending screen output back to the user's screen.



Mainframe computers, known for their robustness, offer high reliability and availability, exceptional processing power for handling large volumes of transactions, and superior scalability, making them ideal for large organizations with intensive data processing needs. They also provide advanced security features and a centralized computing environment, simplifying data management and IT operations. For these reasons, mainframe architecture is still used in industries that require large amounts of processing that can be done at a central location. For example, a bank might update customer balances each night using a mainframe.

However, these advantages come with significant downsides, including high initial and ongoing maintenance costs, the complexity of the systems requiring specialized skills, limited flexibility for specific applications, and potential challenges in integrating newer technologies and cloud services. This combination of factors makes mainframes a powerful but complex and expensive solution suited for specific organizational needs and contexts. In a blend of old and new technology, organizations are moving some of their data processing off their mainframes and into the cloud, as discussed in Section 9.4.2. Indeed, a mainframe can be used to implement part of a cloud computing infrastructure.

9.2.2 Personal Computers

When PC technology exploded in the 1990s, it fundamentally transformed system architecture, shifting the paradigm from centralized to distributed computing. This shift enabled the widespread distribution of processing power and data storage across individual workstations, reducing reliance on large, centralized mainframes or minicomputers. It led to the development of networked environments and client/server architectures, where PCs (clients) could access shared resources and services hosted on servers.

The PC revolution also spurred the development of user-friendly graphical interfaces and software applications tailored for individual productivity, changing how users interact with computers and data. This democratization of computing power not only made technology

more accessible to a broader audience but also fostered innovation in software development, networking, and data management, laying the groundwork for today's diverse and interconnected IT ecosystems.

Although **stand-alone** PCs improved employee productivity and allowed users to perform tasks that previously required IT department assistance, stand-alone computing could be inefficient and expensive. Maintaining data on individual workstations raised significant concerns about data security, integrity, and consistency. Protecting and backing up valuable business data was impossible without a central storage location, and companies were exposed to enormous risks. In some cases, users frustrated by a lack of support and services from the IT department created and managed their databases. In addition to security concerns, this led to data inconsistency and unreliability.

9.2.3 Network Evolution

As technology became available, companies resolved the problems of stand-alone computing by joining clients into a **local area network (LAN)** that allows the sharing of data and hardware resources, as shown in Figure 9-4. One or more LANs, in turn, can connect to a centralized server. Further technological advances have made it possible to create powerful networks that use satellite links, high-speed fiber-optic lines, or the Internet to share data.

A **wide area network (WAN)** spans long distances and can connect LANs that are continents apart, as shown in Figure 9-5. When a user accesses data on a LAN or WAN, the network is **transparent** because users see the data as stored on their workstation. Company-wide systems that connect one or more LANs or WANs are called **distributed systems**. The capabilities of a distributed system depend on the power and capacity of the underlying data communication network.

Distributed systems spread computational tasks across multiple interconnected computers that are either physically close or geographically dispersed. This decentralization enhances system resilience and scalability, as workload distribution can increase fault tolerance and allow for more flexible resource allocation. While mainframes focus on centralized control and robust processing capabilities, distributed systems emphasize networked collaboration, scalability, and redundancy—an evolution from monolithic processing to collaborative, network-based computing.

Figure 9-4 A LAN allows sharing data and hardware, such as printers and scanners.

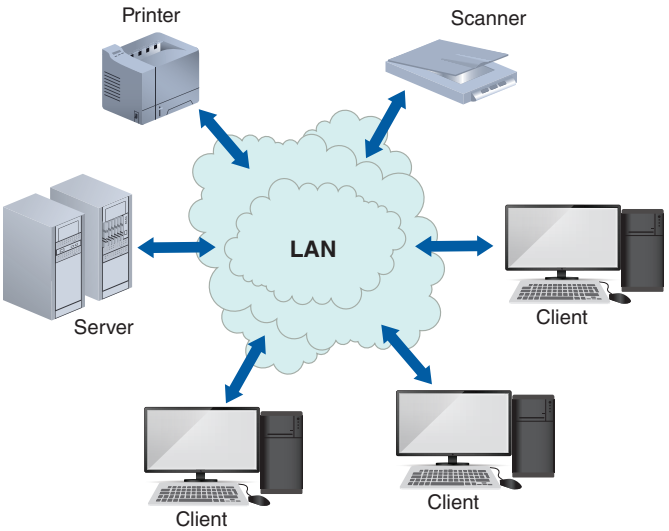
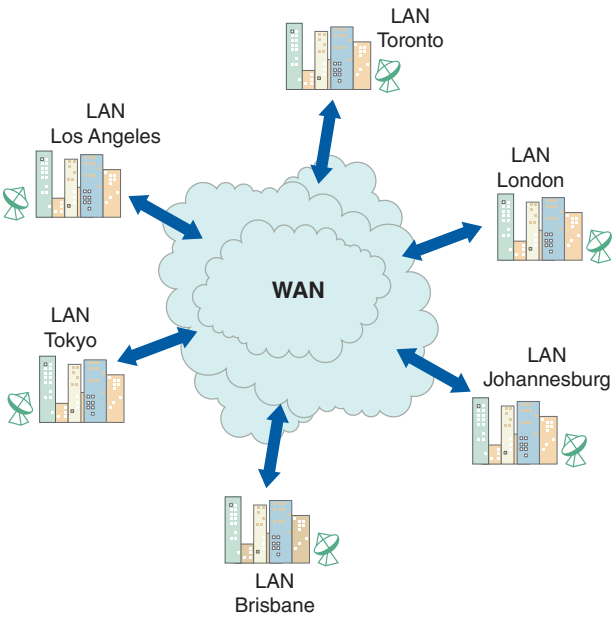


Figure 9-5 A WAN can connect many LANs and link users who are continents apart.



9.3 Client/Server Architecture

Today's interconnected world requires an information architecture that spans the entire enterprise. Whether in a departmental network or a multinational corporation, a systems analyst uses a distributed computing strategy called **client/server architecture**. "Client/server" refers to systems that divide processing between one or more networked clients and a central server. The client handles the entire user interface in a typical client/server system, including data entry, queries, and screen presentation logic. The server stores the data and provides data access and database management functions. Application logic is divided in some manner between the server and the clients.

In a client/server interaction, the client submits a request for information from the server, which carries out the operation and responds to the client. As shown in Figure 9-6, the data file is not transferred from the server to the client—only the request and the result are transmitted across the network. To fulfill a request from a client, the server might contact other servers for data or processing support, but that process is transparent to the client.

Figure 9-6 In a client/server design, data is stored and usually processed on the server.

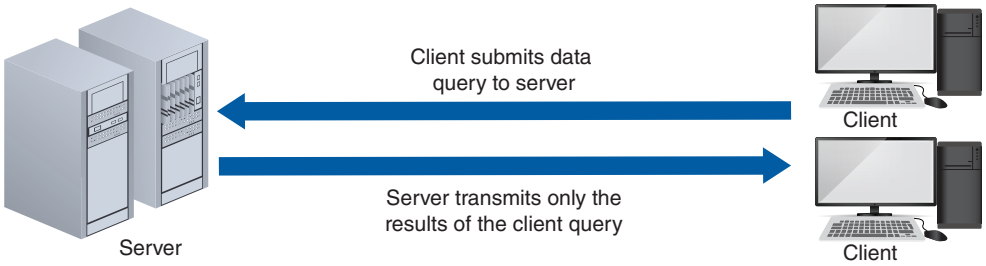


Table 9-1 lists some significant differences between client/server and traditional mainframe systems. Many early client/server systems did not produce expected savings because few clear standards existed, and development costs often were higher than anticipated. Implementation was expensive because clients needed powerful hardware and software to handle shared processing tasks. In addition, many companies had an installed base of data, called **legacy data**, which was difficult to access and transport to a client/server environment.

As large-scale networks grew more powerful, client/server systems became more cost-effective. Many companies have invested in client/server systems to achieve a unique combination of computing power, flexibility, and support for changing business operations. Today, client/server architecture remains a popular form of systems design, using Internet protocols and network models such as the ones described later in this chapter. As businesses form new alliances with customers and suppliers, the client/server concept continues to expand to include clients and servers outside the organization. Service-oriented architecture (SOA) is a networked system in which a service can simultaneously be a client and a server and exist outside corporate boundaries.

Some observers see cloud computing as an entirely new concept. Others see it as the ultimate form of client/server architecture, where Internet-based computing becomes the server part of the client/server model and handles processing tasks. In contrast, the Internet itself becomes the platform that replaces traditional networks. The bottom line is that it doesn't matter whether cloud computing is part of a client/server evolution or a new way of thinking about computing. Either way, successful systems must support business requirements, and system architecture is an essential step in the systems development process.

Table 9-1 Comparison of the characteristics of client/server and mainframe systems.

Comparison of Client/Server and Mainframe Systems		
Characteristics	**Client/Server**	**Mainframe**
Basic architecture	Very flexible	Very rigid
Application development	Flexible Fast Object-oriented	Highly structured Slow Traditional
User environment	PC-based Graphical user interface Empowers the user Improves productivity	Uses terminals Text interface Constrains the user Limited options
Security and control features	Decentralized Difficult to control	Centralized Easier to control
Processing options	Can be shared and configured in any form desired	Extensive and programmable
Data storage options	Can be distributed to place data closer to users	All data is stored centrally
Hardware/software integration	Very flexible Multivendor model	Very rigid Single proprietary vendor

9.3.1 The Client's Role

The client/server relationship must specify how the processing will be divided between the client and the server. A **fat client**, also called a **thick client**, is designed to locate all or most of the application processing logic at the client. A **thin client** design finds all or most of the processing logic on the server.

In the late 1990s, Sun Microsystems was a strong advocate of thin-client computing, which was also referred to as **net-centric computing**. The thin client was a Java-powered terminal, which communicated using standard Internet protocols with powerful servers. Thin clients were expected to provide a lower total cost of ownership (TCO) because maintenance was centralized. However, many users rebelled at the limited functionality provided by thin clients (e.g., no Microsoft Office) and the latency problems inherent in network access to remote applications and data. In the end, fat clients (e.g., regular PCs) remained popular despite all their management issues and higher TCO.

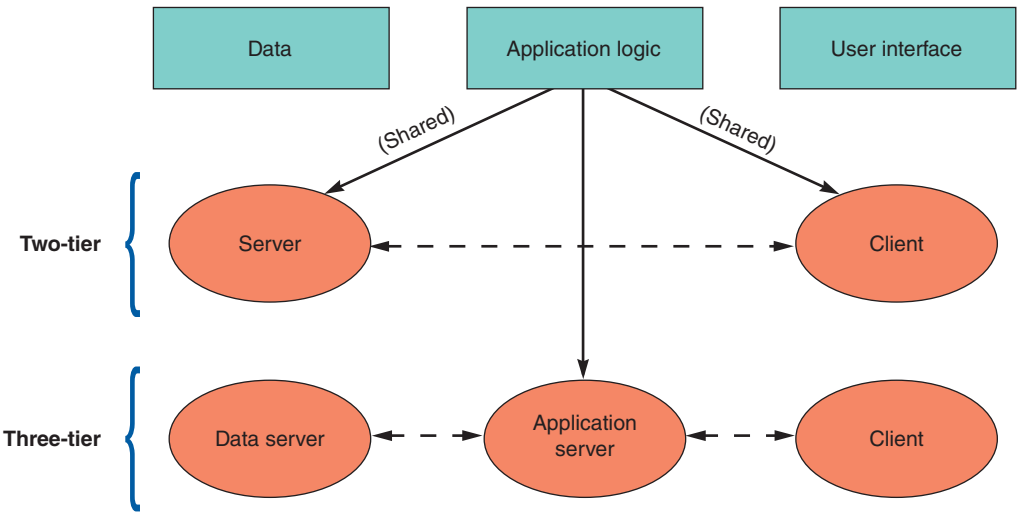
Today's laptop computers, tablets, and smartphones are so powerful that the allure of thin clients has mostly passed, except for access to the cloud for large data sets and specialized processing needs. The app ecosystem has also changed the TCO equation in favor of powerful computing for clients at the edge of the system architecture.

9.3.2 Client/Server Tiers

Early client/server designs were called two-tier designs. In a **two-tier design**, the user interface resides on the client, all data resides on the server, and the application logic can run either on the server or the client or be divided between them.

Another client/server design, called three-tier, has become more popular. In a **three-tier design**, the user interface runs on the client, and the data is stored on the server, just as in a two-tier design. A three-tier design also has a middle layer between the client and server that processes the client requests and translates them into data access commands that can be understood and carried out by the server, as shown in Figure 9-7. The middle layer can be considered an **application server** because it provides the **application logic**, or business logic, required by the system. Three-tier designs, called **n-tier designs**, indicate that some designs use more than one intermediate layer.

Figure 9-7 Two-tier versus three-tier client/server design characteristics.



The advantage of the application logic layer is that a three-tier design may enhance overall performance by reducing the data server's workload. The separate application logic layer also relieves clients of complex processing tasks. The middle layer is more efficient and cost-effective in large-scale systems because it can run on a server that is much more powerful than the typical client workstations. Table 9-2 shows where the data, the application logic, and the user interface are located on various architectures. The tiers in a client/server system communicate using middleware software, as described in the following section.

Table 9-2 The location of the data, the application logic, and the user interface depend on the type of architecture.

Architecture		Data	Application Logic	User Interface
Central data processing center	Server	X	X	X
	Client			
Central server with remote terminals	Server	X	X	
	Client			X
Stand-alone client	Server			
	Client	X	X	X
Two-tier client/server	Server	X	X	
	Client		X	X
Three-tier client/server	Data server	X		
	Application server		X	
	Client			X

9.3.3 Middleware

Middleware is a crucial software layer that acts as an intermediary between an operating system and its applications, facilitating communication and data management in multitier distributed systems. It serves as a bridge, allowing different applications, often on diverse platforms and networks, to interact seamlessly. Middleware connects two or more software

components in a federated system architecture. Middleware is vital in integrating legacy systems and web- and cloud-based applications. Middleware can also represent the slash in the term "client/server."

There are various types of middleware, each designed to address specific needs in a distributed system. For instance, database middleware enables applications to access database systems, message-oriented middleware (MOM) manages the sending and receiving of messages between distributed systems, and web middleware handles web-based interactions. Transaction-processing monitors (TPMs) are another type, ensuring reliable transaction processing across multiple networked resources. These different types of middleware enhance the system's capabilities, such as load balancing, transaction management, and application scalability, thereby facilitating efficient and robust distributed computing.

The role of middleware extends beyond just easing communication challenges; it significantly enhances development efficiency and scalability. Providing standardized programming abstractions reduces the complexity and time required to develop distributed applications. Additionally, middleware supports various operating systems and programming languages, enabling interoperability and integration among applications written in different languages and running on multiple platforms. This flexibility and support for multiplatform integration make middleware indispensable in modern system architecture, especially in enterprise applications and cloud computing environments.

9.3.4 Cost–Benefit Issues

Information systems must be scalable, robust, and flexible to support business requirements. For most companies, client/server systems offer the best combination of features to meet those needs. Whether a business is expanding or downsizing, client/server systems enable the firm to scale the system in a rapidly changing environment. As the size of the business changes, it is easier to adjust the number of clients and the processing functions they perform than to alter the capability of a large-scale central server.

Client/server computing also allows companies to transfer applications from expensive mainframes to less expensive client platforms, sometimes moving heavyweight processing needs to the cloud. In addition, clients and servers can communicate across multiple platforms using standard languages such as SQL. That difference is significant because many businesses invest substantially in various hardware and software environments.

Client/server systems can influence network load and improve response times. For example, consider a user at a company headquarters who wants information about total sales figures. In a client/server system, the server locates the data, performs the necessary processing, and responds immediately to the client's request. The data retrieval and processing functions are transparent to the client because they are done on the server, not the client.

9.3.5 Performance Issues

While it provides many advantages, client/server architecture does involve performance issues that relate to the separation of server-based data and networked clients that must access the data. Consider the difference between client/server design and a centralized environment, where a server-based program issues a command executed by the server's CPU. Processing speed is enhanced because program instructions and data travel on an internal system bus, which moves data more efficiently than an external network.

In contrast to the centralized system, a client/server design separates applications and data. Networked clients submit data requests to the server, which responds by sending data back to the clients. Network capacity becomes a constraint when the number of clients and demand for services increase beyond a certain level and system performance declines dramatically.

According to IBM, a client/server system's performance characteristics differ from those of a centralized processing environment. Client/server response times increase gradually as more requests are made but then rise dramatically when the system nears its capacity. This point is called the **knee of the curve** because it marks a sharp decline in the system's speed and

efficiency. To deliver and maintain acceptable performance, system developers must anticipate the number of users, network traffic, and server size and location to design a client/server architecture that can support current and future business needs.

To enhance performance, client/server systems must be designed so the client contacts the server only when necessary and makes as few trips as possible. This is one of the goals of the **HTTP/2** protocol used between a server and a web browser. Key features of HTTP/2 include multiplexing, allowing multiple requests and responses over a single connection to reduce latency; stream prioritization, which facilitates faster rendering of webpages; header compression via HPACK to reduce overhead; server push, where servers preemptively send resources to clients; and a shift to a binary protocol, enhancing parsing efficiency and robustness. While HTTP/2 does not inherently require encryption, its widespread implementation over HTTPS has further bolstered web security. These enhancements make HTTP/2 especially beneficial in optimizing webpage loading times and handling higher traffic volumes, thereby significantly enhancing the user experience, particularly on mobile devices and in areas with slower Internet connections.

Another issue that affects client/server performance is data storage. Just as processing can be done at various places, data can be stored in more than one location using a **distributed database management system (DDBMS)**. Using a DDBMS offers several advantages: data stored closer to users can reduce network traffic; the system is scalable, so new data sites can be added without reworking the system design; and with data stored in various locations, the system is less likely to experience a catastrophic failure. A potential disadvantage of distributed data storage involves data security: Maintaining controls and standards can be more difficult when data is stored in various locations. In addition, the architecture of a DDBMS is more complex and difficult to manage. From a system design standpoint, the challenge is that companies often want it both ways—they want the control that comes with centralization and the flexibility associated with decentralization.

9.4 The Impact of the Internet

The Internet has had an enormous impact on system architecture. It has become more than a communication channel—many IT observers see it as a fundamentally different environment for system development.

9.4.1 Internet-Based Architecture

Recall that the client handles the user interface in a traditional client/server system, as shown in Table 9-2, and the server (or servers in a multitier system) handles the data and application logic. In a sense, part of the system runs on the client and another on the server.

In contrast, Internet-based system architecture refers to the design and structure of systems that operate over the Internet, leveraging its global connectivity to deliver services and functionalities. This architecture is inherently distributed, with components and services often spread across multiple locations and connected via the Internet. Shifting the responsibility for the interface from the client to the server simplifies data transmission and results in lower hardware cost and complexity.

The advantages of Internet-based architecture have changed fundamental ideas about how computer systems should be designed for an online environment. At the same time, millions of people are using web-based collaboration and social networking applications to accomplish tasks that used to be done in person, over the phone, or by more traditional Internet channels.

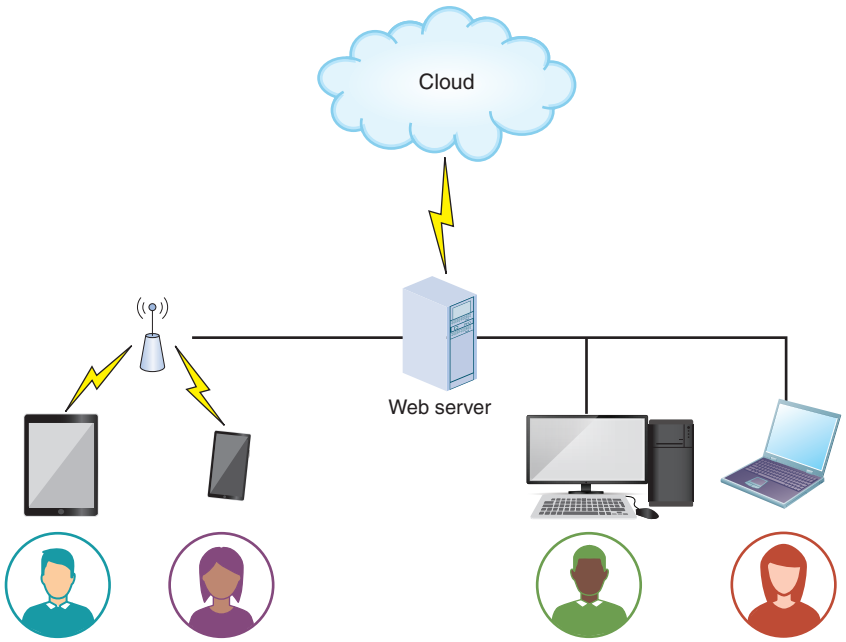
9.4.2 Cloud Computing

Cloud computing refers to the cloud symbol often used to represent the Internet. The cloud computing concept envisions remote computers providing an online software and data environment hosted by third parties. For example, a user's computer does not perform all the

processing or computing tasks—the cloud does some or all of it. This concept contrasts with today's more common computing model, which is based on networks strategically distributing processing and data across the enterprise. In a sense, the cloud of computers acts as one giant computer that performs user tasks.

Figure 9-8 shows users connected to the cloud, which performs the computing work. Instead of requiring specific hardware and software on the user's computer, cloud computing spreads the workload to powerful remote systems that are part of the cloud. The user appears to be working on a local system, but all computing is performed in the cloud. No software updates or system maintenance is required of the user.

Figure 9-8 Cloud computing.



Cloud computing effectively eliminates compatibility issues because the Internet itself is the platform. This architecture also provides **scaling on demand**, which matches resources to needs at any given time. For example, additional cloud servers might come online automatically during peak loads to support the workload.

Cloud computing is ideal for powerful software-as-a-service (SaaS) applications. As described later in the book, SaaS is a popular deployment method in which software is not purchased but is paid for as a service, much like one pays for electricity or cable TV each month. In this architecture, service providers can easily make updates and changes without involving the users.

Even though cloud computing has tremendous advantages, some concerns exist. First, cloud computing may require more **bandwidth** (the amount of data that can be transferred in a fixed period) than traditional client/server networks. Second, because cloud computing is Internet-based, if a user's Internet connection becomes unavailable, the user will be unable to access any cloud-based services. In addition, security concerns are associated with sending large amounts of data over the Internet and storing it securely. Finally, there is the issue of control. Because a service provider hosts the resources and manages data storage and access, the provider has complete control of the system. Many firms are wary of handing over control of mission-critical data and systems to a third-party provider. This is particularly true when the cloud provider's servers are physically located in another jurisdiction or country. Nevertheless, cloud computing has become a cornerstone of enterprise system architecture and will continue for the foreseeable future.

9.4.3 Web 2.0

The shift to Internet-based collaboration has been so compelling that it has been named Web 2.0. Web 2.0 differs from a more technically advanced version of the current web. Instead, Web 2.0 envisions a second generation of the web, enabling people to collaborate, interact, and share information more dynamically. Some view Web 2.0 as a stepping stone toward the **semantic web**, sometimes called Web 3.0, where the documents shared on the Internet have semantics (meaning) and not just syntax (HTML markup).

Social networking sites like Facebook, X (formerly known as Twitter), and LinkedIn are seeing explosive growth in the Web 2.0 environment. Another form of social collaboration is called a wiki. A **wiki** is a web-based repository of information that anyone can access, contribute to, or modify. In a sense, a wiki represents the collective knowledge of a group of people. One of the best-known wikis is Wikipedia.org, but smaller-scale wikis proliferate at businesses, schools, and other organizations that want to compile and share information.

One of the goals of Web 2.0 is to enhance creativity, interaction, and shared ideas. In this regard, the Web 2.0 concept resembles the agile development process and the open-source software movement. Web 2.0 communities and services are based on a body of data created by users. As users collaborate, new layers of information are added in an overall environment known as the **Internet operating system**. These layers can contain text, audio, images, and video clips shared with the user community.

9.5 E-Commerce Architecture

The massive expansion of online commerce is reshaping the IT landscape. Internet business solutions must be efficient, reliable, and cost-effective. When planning an e-commerce architecture, analysts can examine in-house development, packaged solutions, and service providers. Note that whether a firm uses an in-house or a packaged design, the decision about web hosting is a separate issue.

9.5.1 In-House Solutions

If a decision is made to proceed with an in-house solution, there must be an overall plan to help achieve the project's goals. Table 9-3 offers guidelines for companies developing e-commerce strategies. An in-house solution usually requires a more significant initial investment but provides more flexibility for a company that adapts quickly to a dynamic

Table 9-3 Guidelines for companies developing e-commerce strategies.

Guidelines for In-House E-Commerce Site Development
Analyze the company's business needs and develop a clear statement of your goals. Consider the experience of other companies with similar projects.
Obtain input from users who understand the business and technology issues involved in the project. Plan for future growth but aim for ease of use.
Determine whether the IT staff has the necessary skills and experience to implement the project. Consider training, additional resources, and the use of consultants if necessary.
Consider integration requirements for existing legacy systems or enterprise resource planning. Select a physical infrastructure carefully so it will support the application now and later.
Develop the project in modular form so users can test and approve the functional elements as you go along.
Connect the application to existing in-house systems and verify interactivity.
Test every aspect of the site exhaustively. Consider a preliminary rollout to a pilot group to obtain feedback before a full launch.

e-commerce environment. By working in-house, a company has more freedom to integrate with customers and suppliers and is less dependent on vendor-specific solutions.

The decision about in-house web development is even more critical for smaller companies. This approach will require financial resources and management attention that many small companies might be unable or unwilling to commit. An in-house strategy, however, can provide valuable benefits, including customization and control, improved integration with internal systems, and enhanced security and data privacy.

In-house web development offers unparalleled customization and control over the development process, enabling businesses to tailor their web applications precisely to their needs. This level of customization extends to every aspect of the project, from the user interface design to specific functionalities and features. Having complete control over the development process means that the final product can be closely aligned with the company's strategic goals and user requirements. This tailored approach ensures that the web application meets current needs and is adaptable to future changes and expansions, providing a valuable, long-term digital asset.

Developing web applications in-house greatly facilitates their integration with existing internal systems and processes, a crucial aspect for maintaining organizational efficiency and coherence. The in-house team's deep understanding of the company's infrastructure and operational needs allows for seamless and effective integration of new web applications with legacy systems, databases, and workflows. This integration is vital for ensuring consistent data management, avoiding silos, and enhancing the overall functionality and user experience of the system, leading to more streamlined and efficient business operations.

In-house web development provides enhanced control over security and data privacy, which is especially critical for businesses handling sensitive information. An internal development team can implement specific security measures and protocols tailored to the company's requirements and compliance needs. This approach allows for a more proactive and focused strategy in safeguarding data, ensuring that the web applications adhere to industry standards and regulatory requirements. Moreover, having direct oversight of the development process reduces the risk of security vulnerabilities arising from external dependencies or third-party development practices.

9.5.2 Packaged Solutions

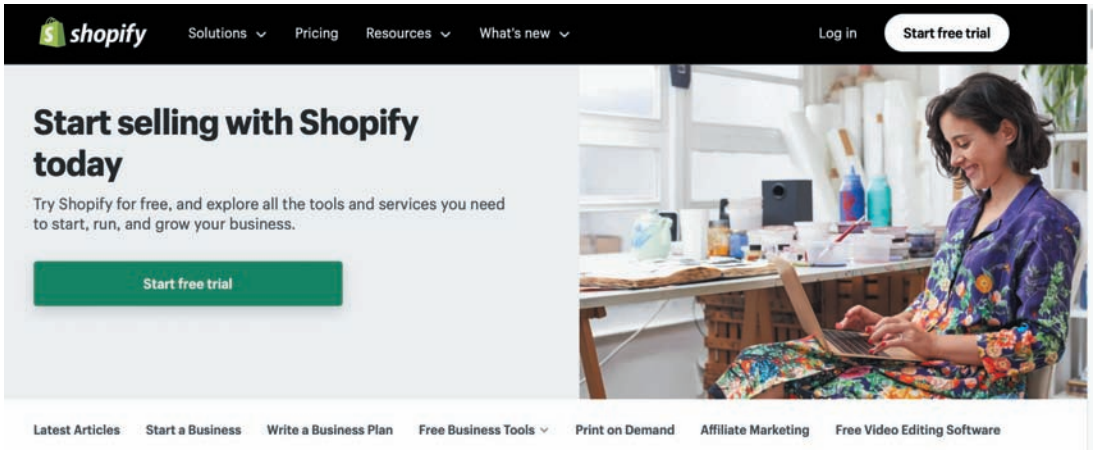
A packaged solution for Internet commerce, often called an e-commerce platform or software, is a comprehensive software package that provides businesses with all the necessary tools and features to conduct and manage online sales and transactions. These solutions are designed to simplify the process of setting up and operating an online store, and they typically include a range of integrated functionalities. Some key features of packaged e-commerce solutions include:

- **Shopping cart functionality.** Enables customers to select and store products for purchase, calculating prices, taxes, and shipping costs.
- **Payment processing.** Integration with various payment gateways to accept payments through credit cards, PayPal, and other methods.
- **Product management.** Tools for managing inventory, adding or removing products, adjusting prices, and handling stock levels.
- **Customer management.** Features for managing customer information, tracking customer behavior, and supporting customer service.
- **Order and shipping management.** Systems for processing orders, managing order fulfillment, and handling returns and exchanges.
- **Website design and customization.** Templates and tools for creating a user-friendly and visually appealing online store.

- **Mobile compatibility.** Ensuring the online store is accessible and functional on mobile devices.
- **Marketing tools.** Features for email marketing, search engine optimization (SEO), and social media integration.
- **Analytics and reporting.** Tools for tracking sales, understanding customer behavior, and gaining insights into business performance.
- **Security features.** Ensuring secure transactions and protecting customer data with encryption, SSL certificates, and compliance with regulations like PCI DSS.

Popular examples of packaged e-commerce solutions include Shopify (shown in Figure 9-9), Magento, WooCommerce (for WordPress), BigCommerce, and Salesforce Commerce Cloud. Each platform offers different features and pricing models, catering to businesses of various sizes and needs. The choice of a packaged solution depends on factors such as the business size, budget, technical expertise, and specific e-commerce requirements.

Figure 9-9 Shopify is a leading provider of e-commerce software solutions.
Source: Shopify



9.5.3 Application Service Providers

Another alternative is to use an application service provider (ASP). As explained later in the book, an ASP provides applications, or access to applications, by charging a usage or subscription fee. Today, many ASPs offer full-scale Internet business services for companies that decide to outsource those functions.

ASPs offer significant advantages for e-commerce architecture, especially in terms of cost-effectiveness and ease of use. They typically operate on a subscription model, making them more accessible for small to medium-sized businesses due to lower upfront costs. ASPs provide ready-to-use, scalable e-commerce platforms requiring minimal technical expertise to set up and manage, allowing enterprises to launch and operate their online stores quickly. They handle maintenance, updates, and security, ensuring the platform remains current with the latest features and protections. Furthermore, many ASPs come with integrated features such as payment processing, inventory management, and customer relationship management, streamlining various e-commerce operations and potentially reducing the need for additional software or services.

However, using an ASP for e-commerce architecture has its drawbacks. Customization options can be limited, posing a challenge for businesses seeking a unique online presence or specific functionalities. Reliance on an ASP means dependence on their reliability and performance, with any service disruptions directly impacting the business. Over time, the ongoing subscription fees may accumulate, potentially making ASPs more expensive than

in-house solutions. Security concerns arise as businesses must trust the provider to safeguard sensitive data effectively. Additionally, transitioning away from an ASP can be challenging due to issues with data portability and integration with other systems, which might hinder future changes or scalability plans.

Case in Point 9.2

Small Potatoes

Small Potatoes is a family-operated seed business that has grown rapidly. It specializes in supplying home gardeners with the finest seeds and gardening supplies. Until now, the firm has done all its business by placing ads in gardening and health magazines and taking orders using a toll-free telephone number.

The family has decided to establish a website and sell online, but there is disagreement about the best way to proceed. Some say it would be better to develop the site in-house, and one of the employees, a recent computer science graduate, believes they can handle the task. Others feel outsourcing the site and focusing on the business would be better. Suppose the family asked for your opinion. What would you say? What additional questions would you ask?

9.6 Processing Methods

In selecting an architecture, the systems analyst must determine which transactions will be handled online and what functions can be carried out using batch processing.

9.6.1 Online Processing

Early computer systems were designed to handle data records as a group, or batch. Fewer systems use that model today. However, even the most advanced online systems must perform maintenance, post large quantities of data during off-hours when network traffic is low, and perform housekeeping tasks just as their legacy computer ancestors did. This section discusses the online processing capability at the core of powerful, modern systems, and the following section describes the evolution of batch processing.

An **online system** handles transactions when and where they occur and provides output directly to users. Because it is interactive, online processing avoids delays and allows a constant dialogue between the user and the system. These systems are commonly used in environments where timely data processing and quick response times are critical. For this reason, online processing systems are also known as real-time systems.

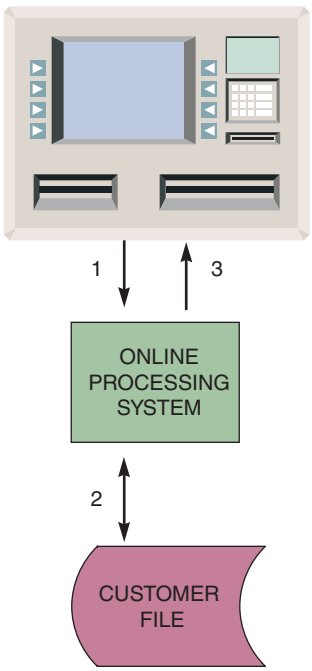
An airline reservations system is a familiar example of online processing. When online customers visit the airline's website, they can enter their desired flight's origin, destination, travel dates, and travel times. The system searches a database and responds by displaying available flights, times, and prices. The customer can make a reservation and enter a name, address, credit card information, and other required data. The system creates reservations, assigns seats, and updates the flight database immediately.

Online processing systems have five typical characteristics:

- **Real-time data processing.** Online processing systems can process data in real time as soon as the data is entered or an event occurs. This feature is essential in scenarios like online banking transactions, flight reservation systems, or e-commerce websites, where immediate processing and feedback are necessary.
- **High availability and reliability.** These systems are designed to be highly available and reliable. Because they are often used for critical business operations, they need to operate continuously without failure. This requires robust infrastructure with redundancy and failover mechanisms to ensure the system remains operational despite partial failures.

- **Concurrent user support.** Online processing systems can handle multiple users or transactions simultaneously. They are built to manage many concurrent sessions efficiently, ensuring that all users receive prompt and accurate responses.
- **Quick response time.** Speed is a crucial attribute of online processing systems. They are optimized for fast response times to ensure users do not experience delays. This involves efficient algorithm design, high-performance hardware, and optimization of network and database queries.
- **Dynamic interaction.** These systems often support dynamic user interaction. Instead of processing data in batches, they interact with users in real time, allowing immediate input and output. This makes them suitable for applications where user interaction is continuously changing or where decisions need to be made instantaneously based on the latest data.

Figure 9-10 When a customer requests a balance, the ATM system verifies the account number, submits the query, retrieves the current balance, and displays the balance on the ATM screen.



ATM QUERY PROCESS

Step 1: Customer enters account number and requests an account balance

Step 2: Retrieves current account balance

Step 3: Verifies bank account number and displays balance on ATM screen

Online processing can also be used with file-oriented systems. Figure 9-10 shows what happens when a customer uses an ATM to inquire about an account balance. After the ATM verifies the customer's card and password, the customer enters the request (Step 1). Then, the system accesses the account master file using the account number as the primary key and retrieves the customer's record (Step 2). The system verifies the account number and displays the balance (Step 3). Data is retrieved, and the system transmits the current balance to the ATM, which prints it for the customer.

9.6.2 Batch Processing

Batch processing means that data is managed in groups, or batches. It was acceptable in the 1960s and the only choice for most firms. Today, all businesses need real-time information, and batch processing is only sometimes desirable. However, batch methods can be efficient and convenient in some situations.

Batch processing systems have five typical characteristics:

- **Scheduled processing.** Batch processing systems typically operate on a schedule, processing data at predetermined times, often during off-peak hours. This scheduling is crucial for optimizing system resources and ensuring the processing doesn't interfere with other real-time operations.
- **Handling large data sets.** These systems are ideal for processing large data sets that do not require immediate action. For example, they are often used for end-of-day transactions in banking, payroll processing, or generating reports. The ability to efficiently process large volumes of data makes batch systems ideal for comprehensive data analysis and complex computational tasks.
- **Efficiency in resource utilization.** Batch processing can be more efficient regarding resource utilization than real-time processing. Because tasks are accumulated and processed together, the system can optimize processing power and memory, potentially lowering operational costs.

- **Delayed output.** One of the key characteristics of batch processing is the delay between input and output. Data is collected over a period, processed in a batch, and then the results are produced. This delay means that batch processing is unsuitable for tasks requiring immediate results or real-time data analysis.
- **Error handling and stability.** Batch processing allows for errors and inconsistencies in the data to be handled before processing, contributing to the system's stability. Additionally, because the entire batch is often tested and validated before processing, there is a lower risk of operational errors during the processing. However, if errors do occur, they might affect the entire batch, which can be time-consuming to correct.

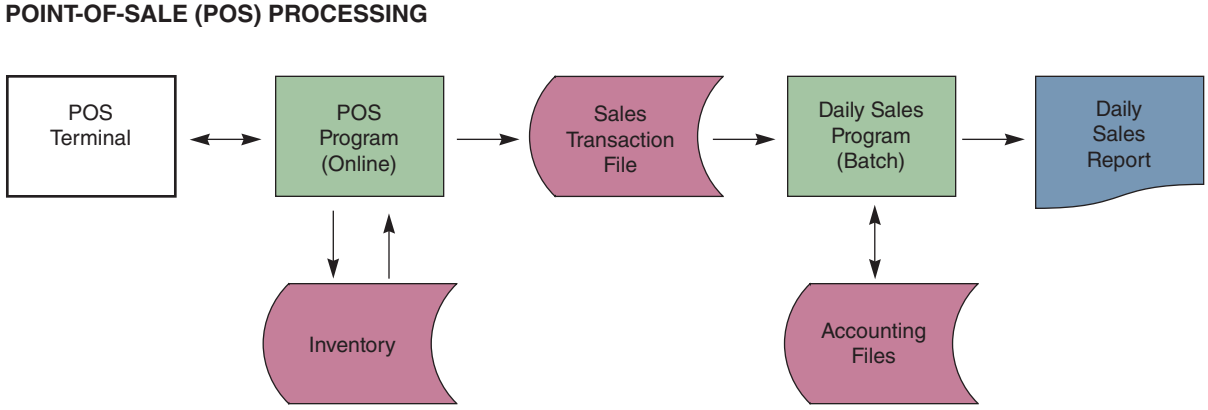
Batch processing offers several advantages, particularly when large volumes of data need to be processed without immediate time constraints. It is highly efficient for routine, repetitive tasks, as it allows for data accumulation and processing collectively, often during off-peak hours, optimizing system resources and reducing operational costs. Batch processing can also be more stable and error-resistant, as it allows for thorough testing of the batch before execution. This method is less demanding on system resources at any given moment, as it doesn't require the system to be constantly ready for immediate input and output, making it ideal for data-intensive tasks like payroll processing, end-of-day transactions, and complex computational tasks such as data analysis or report generation.

However, batch processing has its drawbacks. The most significant is the lack of immediacy; because tasks are processed in batches, there can be a delay between data collection and results availability, which may not be suitable for time-sensitive operations. This method can also lead to less frequent but more substantial system loads, which might require powerful processing capabilities to handle large batches of data efficiently. Another potential issue is the complexity of managing and scheduling batch jobs, which can become cumbersome, especially in systems with high frequency and volumes of batch processing.

9.6.3 Example

The diagram in Figure 9-11 shows how a **point-of-sale (POS)** terminal, such as that used in a supermarket, might trigger a series of online and batch-processing events. The system uses online processing to handle data entry and inventory updates, while reports and accounting entries are performed in a batch. A company would choose a mix of online and batch processing when it makes good business sense.

Figure 9-11 Many retailers use a combination of online and batch processing. When a salesperson enters the sale on the POS terminal, the online system retrieves data from the item file, updates the quantity in stock, and produces a sales transaction record. A batch processing program has a daily sales report and updates the accounting system.



Consider the following scenario in a typical retail store:

- During business hours, a salesperson enters a sale on a POS terminal, part of an online system that handles daily sales transactions and maintains an up-to-date inventory file.
- When the salesperson enters the transaction, online processing occurs. The system performs calculations, updates the inventory file, and produces output on the POS terminal through a screen display and a printed receipt. At the same time, each sales transaction creates input data for day-end batch processing.
- When the store closes, the system uses the sales transactions to produce the daily sales report, perform the related accounting entries, and analyze the data to identify slow- or fast-moving items, sales trends, and related issues, such as store discounts for the next day.

Online and batch processing are different but can work well together. In this scenario, an online system handles POS processing, which must be done as it occurs, while a batch method provides routine, overnight processing and marketing analysis. Online processing allows the data to be entered and validated immediately, so the information is always current. However, a heavy volume of online transactions can be expensive for smaller firms, and data backup and recovery also add to IT costs. In contrast, when appropriately used, batch processing can be cost-effective and less vulnerable to system disruption.

9.7 Network Models

A network allows the sharing of hardware, software, and data resources to reduce expenses and provide more capability to users. When planning a network design, the systems analyst must consider network terms and concepts, including the OSI model, network modeling tools, network topology, network protocols, and wireless networks covered in this section. Other important issues, such as network performance and security, are covered later in the book.

9.7.1 The OSI Model

The discussion of system architecture earlier in this chapter introduced basic network terms such as client, server, LAN, WAN, client/server architecture, tiers, middleware, and cloud computing. The **OSI (Open Systems Interconnection) model** describes how data moves from an application on one computer to an application on another networked computer. The OSI model consists of seven layers, each performing a specific function. This model provides physical design standards that ensure seamless network connectivity, regardless of the particular hardware environment.

9.7.2 Topologies

The way a network is configured is called the **network topology**. Topology can refer to a physical or logical view of the network. For example, **physical topology** describes the network cabling and connections, while **logical topology** describes how the components interact. It is essential to understand the distinction because a specific physical topology can support more than one logical topology. For example, it is not uncommon to run cabling in a particular pattern because of physical installation and cost issues but to use a different pattern for the logical topology.

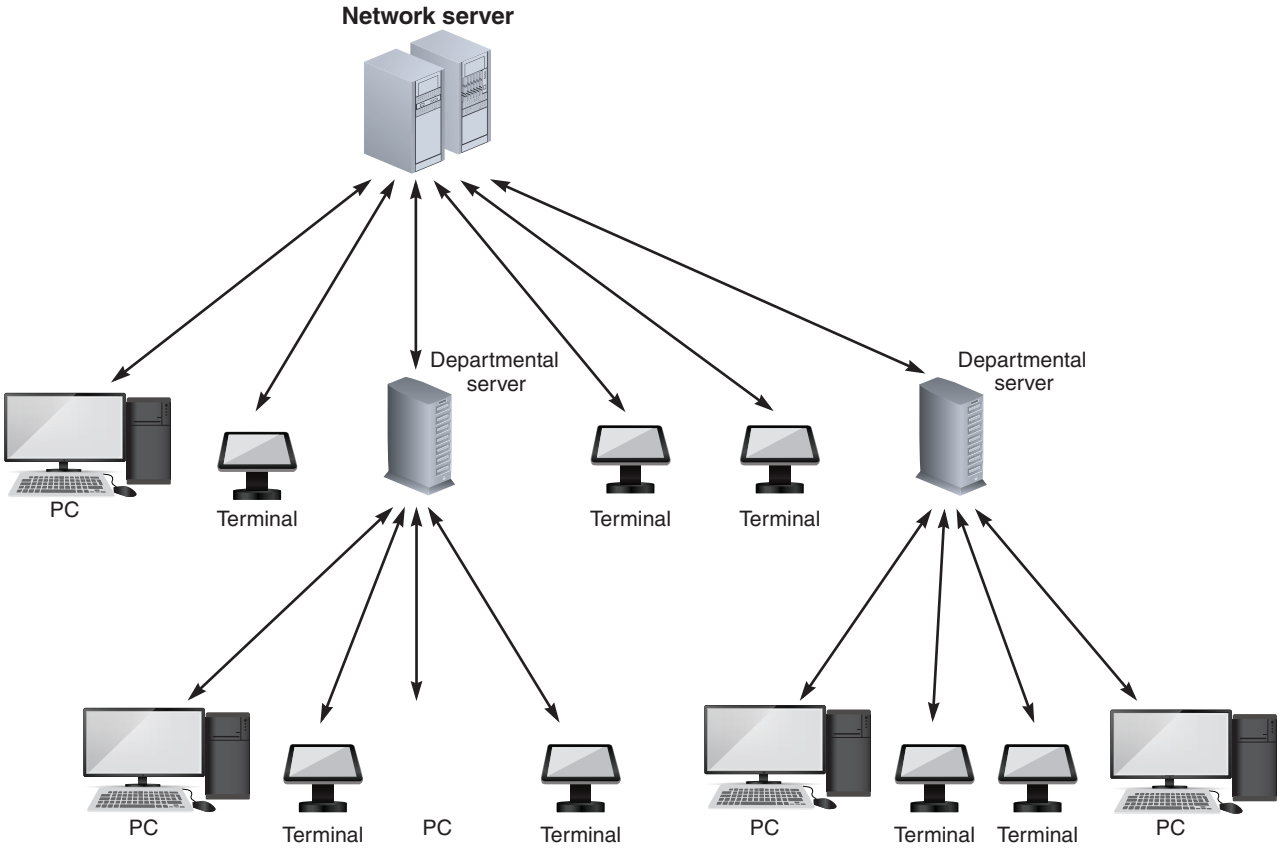
Computers may be physically arranged in a circular shape, but that might not reflect the network topology. The examples shown in Figures 9-12 to 9-16 represent a logical topology seen by network users, who may not know or care about the physical cabling pattern.

LANs and WANs typically are arranged in four patterns: hierarchical, bus, ring, and star. The concepts are the same regardless of the network size, but the physical implementation is different for a large-scale WAN that spans an entire business enterprise compared with a small LAN in a single department. The four common topologies, as well as mesh networks, are described in the following sections.

Hierarchical Network

In a **hierarchical network**, as shown in Figure 9-12, one or more powerful servers control the entire network. Departmental servers control lower levels of processing and network devices. Hierarchical networks were often used in traditional mainframe-based systems but are less common today.

Figure 9-12 A hierarchical network with a server that controls the network.



An example of a hierarchical network might be a retail clothing chain with a central computer that stores sales activity and inventory-level data and local computers that handle store-level operations. The stores transmit data to the central computer, which analyzes sales trends, determines optimum stock levels, and coordinates a supply chain management system. In this situation, a hierarchical network might be used because it mirrors the operational flow in the organization.

Hierarchical network architectures offer several key advantages, including enhanced scalability and manageability, improved security and control, and optimized network performance. The tiered structure, typically comprising core, distribution, and access layers, simplifies network expansion and integration, making it easier to manage and troubleshoot, especially in large-scale environments. This layered approach also enhances security, allowing for more effective implementation of security policies and isolation of sensitive areas, and provides the ability to apply tailored security measures at each layer. Furthermore, hierarchical networks optimize overall performance; the clear segregation of duties across different layers ensures efficient routing and switching, reducing unnecessary traffic and minimizing latency, thereby facilitating smooth and efficient network operations even in high-traffic scenarios.

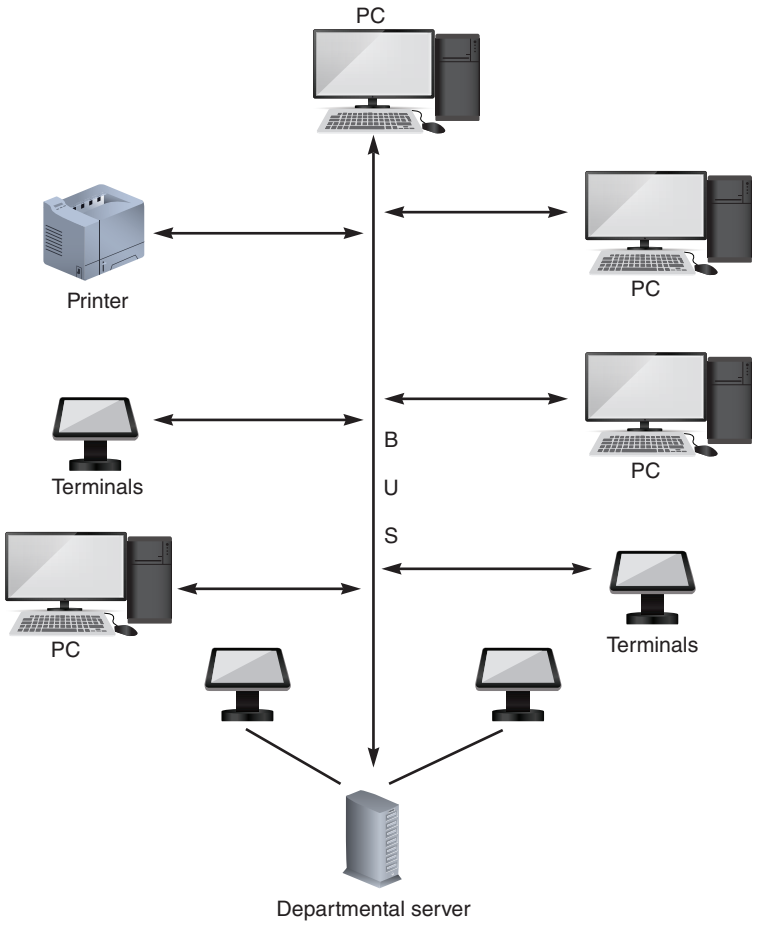
While beneficial in many aspects, hierarchical networks also present certain disadvantages, including complexity in design and implementation, increased infrastructure and maintenance

costs, and potential inefficiency in smaller networks. The necessity for careful planning and specialized equipment across multiple layers adds to the complexity and initial setup costs, making setup a more demanding task, particularly for large and intricate networks. This complexity also translates into higher ongoing maintenance and operational costs due to the need for skilled personnel to manage the layered architecture. Furthermore, in smaller network environments, the structured and segmented hierarchy approach might introduce unnecessary complexity and equipment, leading to inefficiencies and increased costs without corresponding benefits in performance or scalability.

Bus Network

As shown in Figure 9-13, a single communication path connects the central server, departmental servers, workstations, and peripheral devices in a **bus network**. Information is transmitted in either direction between networked devices, and all messages travel over the same central bus. Bus networks require less cabling than other topologies because only a single cable is used. Devices can also be attached or detached from the network at any point without disturbing the rest of the network. In addition, a failure in one workstation on the network does not necessarily affect other workstations.

Figure 9-13 A bus network with all the devices connected to a single communication path.



Bus networks offer several advantages. First, they are cost-effective and easy to implement, especially in small networks, as they require less cabling and networking hardware than more complex topologies. This simplicity also makes bus networks relatively easy to understand and manage, which benefits smaller organizations or setups with limited technical resources. Additionally, bus networks are flexible in terms of expansion; new devices can be easily added to the network by simply connecting them to the bus. However, this may impact overall network performance as the number of devices increases.

Despite their simplicity, bus networks have notable disadvantages, particularly regarding scalability, reliability, and performance. As more devices are added, the network can become congested, leading to decreased performance and slower data transmission rates. The entire network also relies on the single bus (central cable), making it vulnerable to failures; if the bus goes down, the whole network can be rendered inoperative. Additionally, bus networks are prone to security and privacy issues because all network devices share the same transmission medium, making it easier for data to be intercepted or for unauthorized access. These limitations make bus networks less ideal for larger, more complex, or highly sensitive environments.

The bus network is one of the oldest LAN topologies and is a simple way to connect multiple workstations. Before the proliferation of star networks, bus networks were ubiquitous. They share characteristics of hardware bus networks. Today, the bus design is less popular, but some firms have retained bus networks to avoid the expense of new wiring and hardware.

Figure 9-14 A ring network with a set of computers that sends and receives data flowing in one direction.

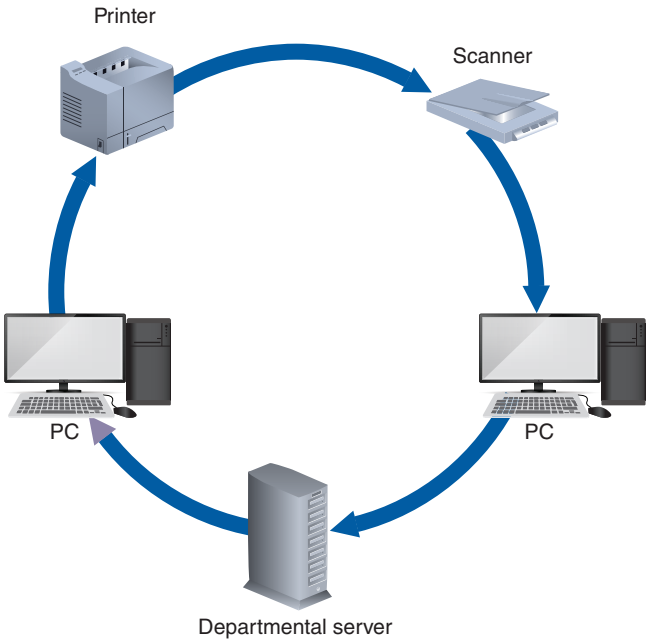
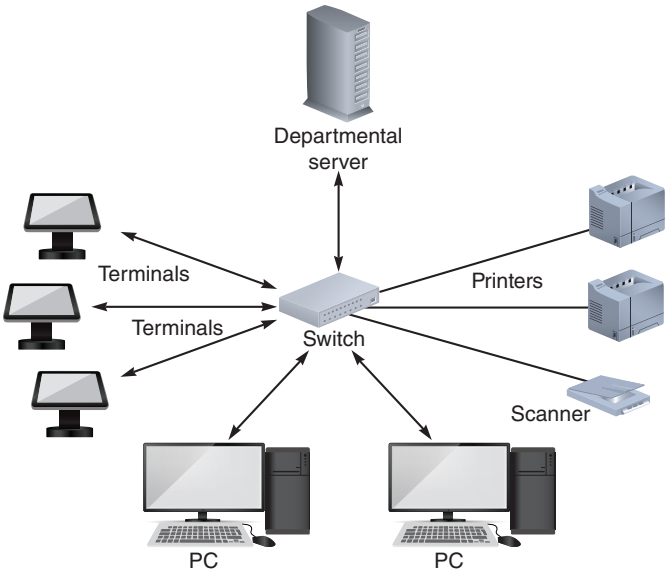


Figure 9-15 A star network with a switch, departmental server, and connected computers and devices.



Ring Network

The **ring network** topology, characterized by its circular layout in which each node connects to exactly two other nodes to form a ring (see Figure 9-14), dates to the early days of computing. A ring network can be considered a bus network with the ends connected. One disadvantage of a ring network is that if a network device (such as a PC or a server) fails, the devices downstream from the failed device cannot communicate with the network.

The ring network was developed as an efficient and structured way to manage data transmission between multiple devices. One of the most notable implementations of ring topology was the development of the IBM Token Ring network in the mid-1980s. This technology was designed to reduce the chance of data collision by passing a token around the network to control which computer could send data at a given time.

The ring network offers a more predictable and orderly method of data transmission compared to the bus topology. Its development was driven by the need for reliable, orderly, and collision-free data transmission in environments with numerous networked computers, and it became a popular network architecture for LANs before the advent of more advanced technologies like Ethernet.

Star Network

The star network is a popular LAN topology because of its speed and versatility. As shown in Figure 9-15, a **star network** has a central networking device called a **switch**, which manages the network and acts as a communications conduit for all network traffic. A device known as a **hub** was used to connect star networks in the past, but a switch offers advanced technology and much better performance. A hub or switch functions like a familiar multisocket power strip but with network devices such as servers, workstations, and printers plugged in rather than electrical appliances. The hub broadcasts network traffic (**data frames**) to all connected devices. In contrast, a switch enhances network performance by sending traffic only to specific network devices that need to receive the data.

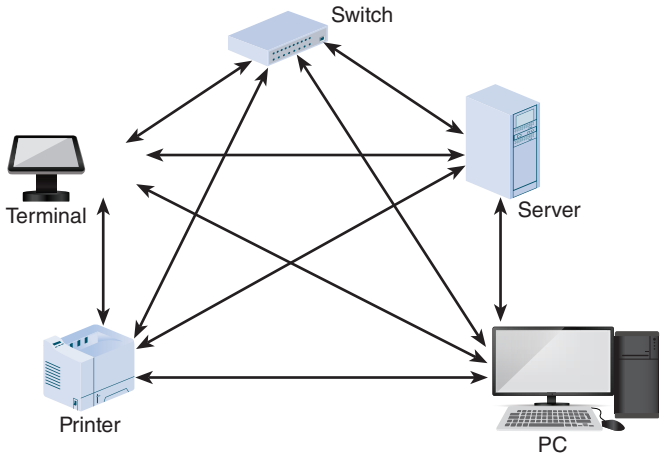
Star networks offer several distinct advantages. First, they provide enhanced reliability; if one connection fails, it does not impact the others, ensuring that the rest of the network remains operational. This individual connection setup facilitates problem identification and isolation, making troubleshooting more straightforward than in other topologies. Furthermore, star networks typically exhibit better performance, as the central hub efficiently manages and routes network traffic, preventing data collisions and ensuring smoother data flow, especially in networks with a high volume of communication. Additionally, the star configuration allows for easy scalability; new nodes can be added without disrupting the existing network simply by connecting them to the central hub.

However, star networks do present certain disadvantages. For example, the dependency on a central hub makes the network vulnerable to a single point of failure; if the central hub fails, the entire network goes down. This can be a significant issue in critical systems where continuous network availability is essential. However, in most large star networks, backup switches are available immediately in case of hardware failure. Additionally, the cost of setting up and maintaining a star network can be higher than that of other topologies like bus or ring, as it requires more cabling and potentially more expensive network hardware (hubs or switches). Finally, the scalability of a star network can be limited by the capacity of the central hub. As more devices are added, the hub can become a bottleneck, affecting overall network performance, especially if the hub cannot handle high traffic volumes or many connections.

Mesh Network

Initially developed for military applications, a **mesh network** is a type of network topology in which each node is interconnected with multiple other nodes, allowing data to be transmitted via multiple paths, as shown in Figure 9-16.

Figure 9-16 A mesh network, such as in military applications, is used when a high degree of redundancy is needed. The redundant design provides alternate data paths but is expensive to install and maintain.



Unlike other network topologies that rely on a single path between nodes, mesh networks resemble the Internet by creating a highly redundant and resilient system. This direct node-to-node communication also improves data transmission speeds and reduces latency, as data can travel along the shortest and most efficient paths. Mesh networks are beneficial in large-scale wireless networks, like Wi-Fi and IoT (Internet of Things) applications, as they provide robust coverage over large areas and can dynamically self-organize and self-heal, adjusting connections as nodes are added or removed. Their decentralized nature also contributes to increased security and scalability, making them well-suited for complex and dynamic network environments.

Mesh networks also present certain disadvantages. The setup and maintenance complexity is greater than with simpler topologies like star or bus networks. Each node must be configured to communicate with multiple other nodes, which can become challenging as the network grows. The cost of implementation can be higher due to the need for more nodes, each of which must be capable of acting as both a transmitter and a receiver. This requirement can lead to higher initial investment and operational costs. In densely populated mesh networks, there can be an issue of network congestion, as the numerous paths available for data transmission can lead to increased network traffic. This congestion can potentially reduce the network's overall efficiency, especially if the nodes are not effectively managing the data routing.

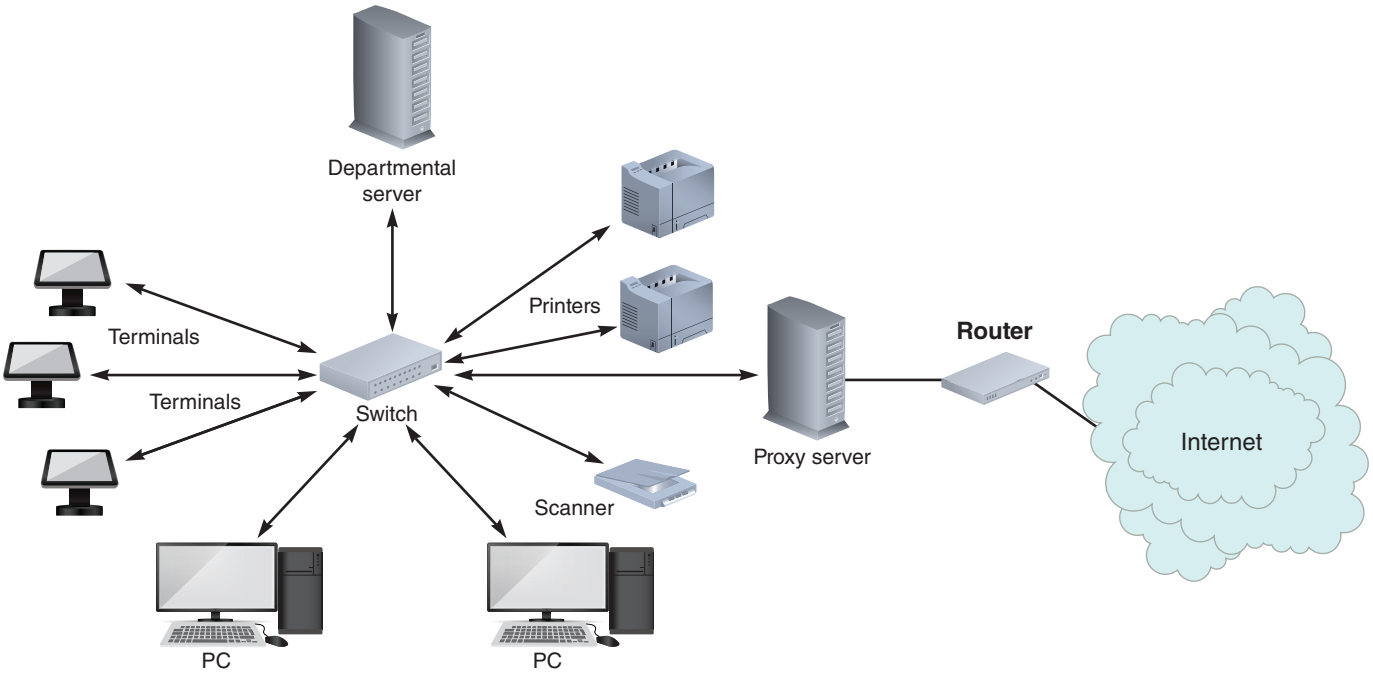
9.8 Network Devices

Network devices are integral hardware components of system architecture that facilitate communication and data exchange within and between networks, forming the backbone of modern interconnected systems. These devices include routers, switches, hubs, bridges, proxy servers, and modems, each serving specific roles. Switches connect devices within a single network. Hubs function as basic connection points. Bridges link separate networks. Modems connect networks to Internet service providers.

A **router** is a device that connects network segments, determines the most efficient data path, and guides data flow. Modern routers often include additional functionalities like quality of service (QoS) settings to prioritize certain types of traffic, virtual private network

(VPN) support for secure remote access, and parental control options. Using a router, any network topology can connect to a larger, dissimilar network like the Internet. This connection is called a **gateway**. The example in Figure 9-17 shows a star topology, in which a switch connects nodes in the LAN, and the router links the network to the Internet. A device called a **proxy server** provides Internet connectivity for internal LAN users. Most business networks use routers to integrate the overall network architecture.

Figure 9-17 Routers can create gateways between different network topologies and large, dissimilar networks such as the Internet.



The placement and configuration of these devices are crucial in determining the overall network architecture, affecting aspects such as network speed, capacity, resilience to failures, security, and the ability to handle different types of traffic. Network devices are pivotal in shaping system architecture, influencing how information is routed, managed, and secured across diverse and complex networking environments.

9.9 Wireless Networks

Although a wired LAN provides enormous flexibility, the cabling cost can be substantial, as well as the inevitable wiring changes in a dynamic organization. Many companies find wireless technology to be an attractive alternative. A **wireless local area network**, or **WLAN**, is inexpensive to install and well-suited to workgroups and users not anchored to a specific desk or location. Most notebook computers and other mobile devices are equipped with built-in wireless capability, and it is relatively simple to add this feature to existing desktop computers and workstations to set up a wireless network. Like their wired counterparts, wireless networks have specific standards and topologies, as discussed in the following sections.

9.9.1 Standards

Wireless networks are based on various standards and protocols that are still evolving. The most popular is IEEE **802.11**, a family of standards developed by the **Institute of Electrical and Electronics Engineers (IEEE)** for wireless LANs.

Current wireless networks are based on variations of the original 802.11 standard. Several versions, or amendments, were intended to improve bandwidth, range, and security. The IEEE 802.11 set of standards changes rapidly due to pressure from consumer groups and industry leaders moving toward ever-faster wireless networks. Wireless network speed is measured in **Mbps (megabits per second)** or **Gbps (gigabits per second)**.

For example, when the **802.11b** standard was introduced in 1999, the average speed was 11 Mbps. Later versions, such as **802.11g** and **802.11n**, increased bandwidth to 54 and 450 Mbps, respectively, and were widely accepted by the IT industry.

Standards such as **802.11ac** (Wi-Fi 5) can theoretically reach speeds of 7 Gbps. The increased speed is accomplished using **multiple input/multiple output (MIMO)** technology to boost performance. MIMO relies on multiple data paths, called **multipath design**, to increase bandwidth and range.

Wi-Fi 6, also known as **802.11ax**, was introduced in 2019 as the next standard for wireless networking (using the 2.4 and 5 GHz bands), replacing the Wi-Fi 5 standard. Wi-Fi 6, also known as High-Efficiency Wi-Fi, is used to improve wireless clients in dense urban environments. Wi-Fi 6E was released in 2020 and uses the 6 GHz band.

802.11be, dubbed Extremely High Throughput, is the latest amendment of the IEEE 802.11 standard, which is designated Wi-Fi 7. Introduced in 2024, it builds on 802.11ax, focusing on WLAN indoor and outdoor operation with stationary and pedestrian speeds in the 2.4, 5, and 6 GHz frequencies.

WLANs are replacing wired networks in many situations as wireless capacity expands and security issues are addressed. Wireless security is discussed in detail later in the book.

9.9.2 Topologies

Like wired networks, wireless networks also can be arranged in different topologies. The two most common network topologies for IEEE 802.11 WLANs are the basic and extended service sets. Figure 9-18 shows simplified models of these topologies.

The **basic service set (BSS)**, the **infrastructure mode**, is shown at the top of Figure 9-18. In this configuration, a central wireless device, an **access point** or **wireless access point (WAP)**, serves all wireless clients. The access point is similar to a hub in the LAN star topology, except it provides network services to wireless clients instead of wired clients. Because access points use a single communications medium—the air—they broadcast all traffic to all clients, just as a hub would do in a wired network. The access point is typically connected to a wired network so wireless clients can access it.

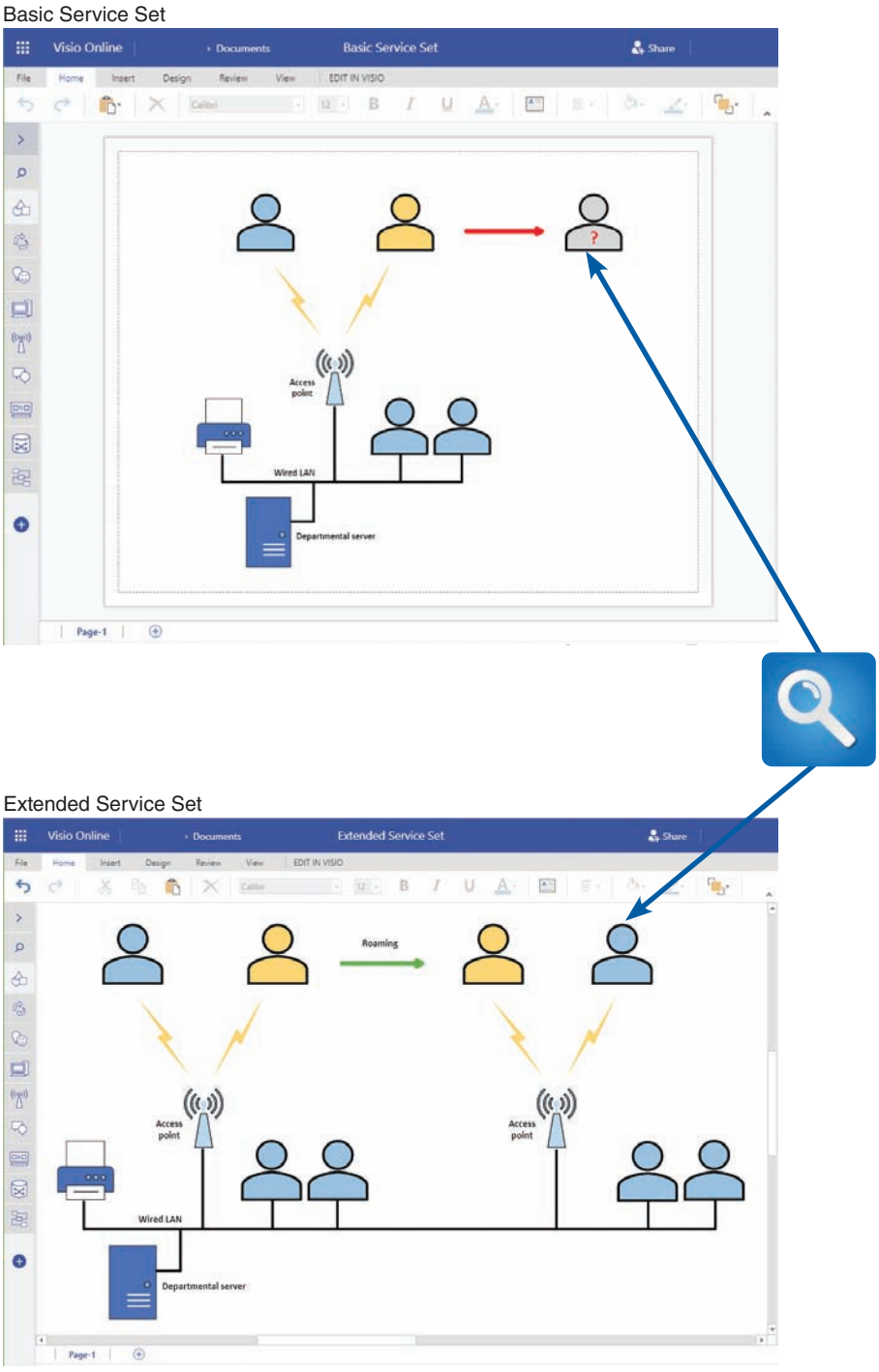
The second wireless topology is the **extended service set (ESS)**, as shown at the bottom of Figure 9-18. An ESS comprises two or more BSS networks. Thus, wireless access can be expanded over a larger area using an ESS topology. Each access point provides wireless services over a limited range. As a client moves away from one access point and closer to another, **roaming** automatically allows the client to associate with the stronger access point, allowing for undisrupted service.

9.9.3 Trends

Wireless technology has brought explosive change to the IT industry and will continue to affect businesses, individuals, and society. Even in the ever-changing world of IT, it would be difficult to find a more dynamic area than wireless technology.

With the growing popularity of 802.11, many firms offer networking products, services, and information. One of the most significant groups is the **Wi-Fi Alliance**, which maintains a website at www.wi-fi.org. According to the site, the Alliance is a nonprofit international association formed in 1999 to certify interoperability of wireless network products based on IEEE 802.11 specifications. Products that meet the requirements are certified **Wi-Fi (wireless fidelity)** compatible. The stated goal of the Wi-Fi Alliance is to enhance the user experience through product interoperability.

Figure 9-18 The user in the upper screen has moved out of the BSS coverage area and cannot communicate. In the lower screen, the user roams into another ESS coverage area, and the transition is seamless.



Even though they have many advantages, wireless networks have limitations and disadvantages. For example, devices that use the 2.4 GHz band can pick up interference from appliances such as microwave ovens and cordless telephones that use the same band. Wireless networks pose major security concerns because wireless transmissions are much more susceptible to interception and intrusion than wired networks. These issues are discussed in detail later in the book.

In addition to Wi-Fi, another form of wireless transmission called **Bluetooth** is very popular for short-distance wireless communication that does not require high power. Bluetooth devices include wireless keyboards, mice, printers, cell phone headsets, and digital cameras. People with Bluetooth-equipped phones or tablets can even beam information to each other and exchange digital notes.

Near-field communication (NFC) allows devices within a few centimeters of each other to exchange information wirelessly. It is used in smartphones and wearable devices. For example, the Apple Watch has an integrated NFC chip to support Apple Pay and Apple Wallet.

Case in Point 9.3

Spider IT Services

Spider IT Services specializes in custom network design and installation. Firms hire Spider to do an overall analysis of their network needs, including a detailed cost–benefit study. Recently, a problem arose. One of Spider's clients complained that the relatively new network was too slow and lacked sufficient capacity. Reviewing the case, Spider's top management realized that the rapidly growing client had outgrown the network much earlier than anticipated. How could this problem have been avoided?

Summary

System architecture translates the logical design of an information system into a physical blueprint. Designing the system architecture requires consideration of servers, clients, processing methods, networks, and related issues.

A comprehensive checklist for evaluating system architecture focuses on the influence of corporate culture, the integration of enterprise resource planning (ERP) systems, and the importance of scalability. It highlights the challenges of incorporating legacy systems and the need for robust security and compliance measures. Additionally, it discusses the role of corporate portals as centralized hubs for information and services.

The evolution of system architecture can be traced from mainframe architectures, emphasizing centralized processing, to the rise of personal computers that brought computing to individual users. This evolution includes the transformative impact of networks on data communication and system design. This historical perspective sets the context for understanding modern networked environments and the integration of various computing paradigms.

In a client/server architecture, responsibilities and resources between clients and servers are distributed in a networked environment. This architecture uses different design tiers and employs middleware to facilitate communication. Cost–benefit analysis and performance issues must also be examined to weigh this architecture's advantages and potential drawbacks.

The Internet has revolutionized system architecture, mainly through Internet-based architecture and the advent of cloud computing. The emergence of Web 2.0 is characterized by user-generated content and increased interactivity. These developments have significantly influenced the design and use of web services and applications.

E-commerce architecture options include in-house solutions, packaged solutions, and the use of application service providers (ASPs). Each approach must be analyzed to address issues of control, costs, and scalability.

Online processing handles real-time data transactions, while batch processing is suited for scheduled, large-volume data processing. Online processing allows data to be entered and validated immediately, but a heavy volume of online transactions can be expensive for smaller firms, and data backup and recovery add to IT costs. When appropriately used, batch processing can be cost-effective and less vulnerable to system disruption.

Network models include the OSI model and various network topologies like bus, star, and ring. Systems analysts need a foundational understanding of different network designs and their suitability for specific networking needs. This serves as a guide to understanding how network structures influence system functionality.

Network devices are crucial for any network infrastructure. The functions and roles of these devices facilitate communication and data exchange within and between networks, forming the backbone of modern interconnected systems. These devices include routers, switches, hubs, bridges, proxy servers, and modems.

Wireless networks are based on various standards, topologies, and protocols that are still evolving. The evolution of wireless technology, the governing standards, and different network topologies provide a glimpse into the future of wireless networking. Advancements in wireless network technologies are growing in importance.

Key Terms

802.11
802.11ac
802.11ax
802.11b
802.11be
802.11g
802.11n
access point
application
application logic
application server
bandwidth
basic service set (BSS)
Bluetooth
bus network
client
client/server architecture
corporate portal
data frames
data processing center
distributed database management system (DDBMS)
distributed system
extended service set (ESS)
extensibility
fat client
gateway
Gbps (gigabits per second)
hierarchical network
HTTP/2
hub
infrastructure mode
Institute of Electrical and Electronics Engineers (IEEE)
Internet operating system
knee of the curve
legacy data
local area network (LAN)
logical topology
mainframe architecture
Mbps (megabits per second)
mesh network
multipath design
multiple input/multiple output (MIMO)
n-tier design
near-field communication (NFC)
net-centric computing
network topology
node
online system
OSI (Open Systems Interconnection) model
physical topology
platform
point-of-sale (POS)
portal
proxy server
ring network
roaming
router
scaling on demand
semantic web
server
stand-alone
star network
switch
system architecture
thick client
thin client
three-tier design
transparent
two-tier design
web-centric
Wi-Fi (wireless fidelity)
Wi-Fi Alliance
wide area network (WAN)
wiki
wireless access point (WAP)
wireless local area network (WLAN)

Review Questions

1. What would your list look like if you had to rank the items in the architecture checklist from most important to least important?
2. What three functions must every business information system carry out, irrespective of system architecture?
3. What is client/server architecture?
4. What has been the impact of the Internet on system architecture?
5. What are the differences between in-house e-commerce development with packaged solutions and service providers?
6. What are the advantages of online and batch processing, respectively?
7. Explain the five main network models.
8. What functions do routers, gateways, and proxy servers serve in a network?
9. What role do standards play in wireless networking?
10. Describe two technologies for short-distance wireless communication.

Discussion Topics

1. How important are security and compliance in system architecture?
2. An increasing number of mobile devices are locally powerful, use apps instead of full-fledged applications, and rely on wireless network connectivity. How does this proliferation change system architecture design considerations?
3. E-commerce has seen explosive growth in recent years. What are the most important reasons for this trend? Will it continue? Why or why not?
4. Is batch processing still relevant? Why or why not?
5. What are the main differences between the BSS and ESS wireless topologies?

Hands-On Projects

1. Visit the IT department at your school or a local company to learn about the network they use. Describe the network and sketch the configuration.
2. Prepare a 10-minute talk explaining Web 2.0 and cloud computing to a college class. Using the text and your Internet research, briefly describe the five most important points you will include in your presentation.
3. Perform research on the Internet to identify a service provider that offers web-based business solutions, and write a report describing the firm and its services.
4. Perform research on the Internet to learn about emerging trends in wireless networking and typical costs involved in installing a wireless LAN.
5. Examine the role wireless networks are playing in the developing world. Why are some places bypassing LANs and physical cabling and moving to a wireless system architecture? What are the advantages and disadvantages of this?

Ethical Issues

The new accounting system is operational, but user feedback has been negative. The most common complaint is that the system could be more user-friendly. Some people in the IT department think that more user training would solve the problem. However, the IT manager is opposed to a fresh round of training. "Let's just set up the network to monitor the users' keystrokes and mouse clicks and see the patterns," the manager suggested. "We can analyze the data and develop tips and suggestions to make the system easier to use."

Your initial reaction is that the IT manager is wrong for two reasons. First, you believe that monitoring would not be an effective method to learn what users want. In your view, that should have been done in the system requirements phase. Second, you are bothered by an ethical question: Even though the proposed monitoring would involve company business, the company network, and company time, you feel that many users would resent the unannounced monitoring and might think that their performance or other computing activities were being appraised without their knowledge.

The IT manager has asked you to write up a recommendation. What will you say about the ethical question that troubles you?

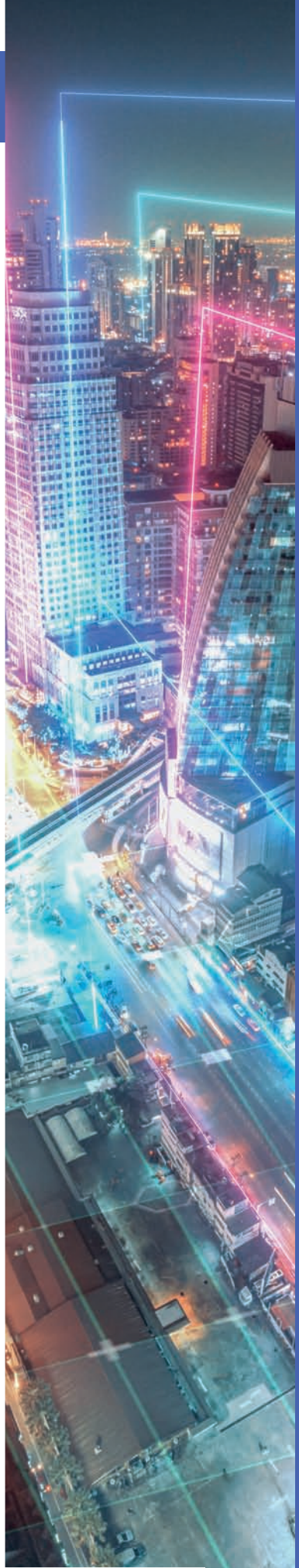
Phase 4

Implementation

The fourth SDLC phase, implementation, focuses on contrasting development strategies, from traditional to web-based, emphasizing the emerging role of artificial intelligence and cloud computing. It evaluates in-house development and outsourcing, discusses the advantages and challenges of software as a service (SaaS), and advocates for strategic choices through cost–benefit analysis. This phase also addresses implementation management, quality assurance, application development, and agile and object-oriented methodologies. The goal is to ensure systems are robust, well-tested, comprehensively documented, and ready for seamless deployment and use.

Chapter 10 analyzes development strategies, comparing traditional and web-based approaches. It discusses evolving trends such as artificial intelligence, cloud computing, the Internet of Things, mobile devices, and Web 2.0 and 3.0. The chapter covers in-house development options, outsourcing, offshoring, and software as a service. It also addresses data management, analytics, and selecting a development strategy, including cost–benefit analysis and the systems analyst's role in software acquisition.

Chapter 11 emphasizes the complexities of managing systems implementation, including quality assurance, software and systems engineering, ISO standards, and application development strategies and tools. It compares structured, object-oriented, and agile methodologies and discusses their characteristics and implementation. The chapter also addresses coding, programming languages, integrated development environments, testing, documentation, and installation, including operational and test environments, system changeover, data conversion, training, and deployment preparation.



Chapter

10

Development Strategies

Learning Objectives

When you finish this chapter, you should be able to:

1. Compare and contrast traditional and web-based systems development.
2. Explain the possible impact of evolving IT trends on systems development.
3. Outline the process of selecting an in-house software development option.
4. Explain outsourcing.
5. Summarize the main advantages and the unique concerns of offshoring.
6. Describe the concept of software as a service.
7. Explain why data management and analytics are integral to selecting a development strategy.
8. Outline how a systems analyst helps in choosing a development strategy.
9. Execute the main steps in the software acquisition process.

Chapter 10 is the first chapter in the systems implementation phase of the SDLC. The first step in this phase includes evaluating alternative development strategies, assessing various development options, and considering data management and analytics issues. The analyst must also learn about managing the software acquisition process.

The chapter includes three "Case in Point" discussion questions to help contextualize the concepts described in the text. The scenario described in the "Ethical Issues" section concerns how truthful an answer should be when submitting a response to a request for proposal. Is a slight exaggeration acceptable if it means the company will win the contract?

10.1 Traditional versus Web-Based Systems Development

A few years ago, a typical company either developed software, purchased a software package (which might require customization), or hired consultants or outside resources to perform the work. Today, a company has many more choices for software acquisition, including application service providers, web-hosted software options, and firms that offer a variety of enterprise-wide software solutions. This proliferation of choices is partly due to the enormous changes in business methods and operations made possible by the Internet.

10.1.1 Traditional Development

Several factors heavily influence the design process in a traditional systems development environment. Compatibility is a crucial consideration, encompassing existing hardware and software platforms as well as the requirements of legacy systems. These systems are generally designed to operate effectively on local and wide-area company networks. Although they often utilize Internet links and resources, web-based features are typically regarded as supplementary enhancements rather than fundamental components of the system design. This approach focuses on integrating with and maximizing the existing technological infrastructure.

The development of these traditional systems follows one of three primary paths. The first involves in-house development, where the organization's IT team takes full responsibility for the system's creation and implementation. The second option is to purchase an off-the-shelf software package, which may be modified to suit the organization's specific needs better. The third path involves engaging external consultants who bring specialized expertise to the development process. Each approach has unique advantages and challenges, and the choice depends on factors such as the organization's internal capabilities, budget, and specific requirements.

Scalability and security are also critical considerations in traditional systems development. Network limitations and constraints can significantly impact the scalability of a system, affecting its ability to grow and adapt to increasing demands. Many applications developed in this context require substantial desktop computing power and resources to function effectively. As for security, these systems often present less complexity than web-based systems because they operate within a private company network instead of the more open and inherently risky Internet environment. This distinction in the operating environment necessitates different security approaches and considerations.

10.1.2 Web-Based Development

In a web-based systems development environment, the approach to systems design and delivery fundamentally differs from traditional methods. Systems are developed within an Internet-based framework, such as .NET, where the web is treated as the primary platform rather than just a means of communication. This shift places a greater emphasis on web-based technologies and strategies. Web-based systems are inherently scalable and capable of running across various hardware environments. This flexibility makes them particularly appealing to large firms, which often deploy these systems as enterprise-wide solutions for critical business functions like customer relationship management, order processing, and materials management.

The nature of web-based software also changes how applications are perceived and utilized. In this context, software is often treated more as a service than a product, reducing dependence on desktop computing power and resources. This model allows companies to acquire web-based software as a service (SaaS), limiting the need for extensive in-house involvement. Instead, the vendor is responsible for installing, configuring, and maintaining the system, usually in exchange for agreed-upon fees. This service-oriented approach simplifies the deployment and management of complex systems for businesses.

However, web-based solutions introduce their own challenges, particularly for integration and security. They often require additional software layers, known as middleware, to facilitate communication with existing software and legacy systems. This integration ensures seamless operation and data consistency across different platforms. Moreover, the open nature of the Internet brings more complex security issues. These concerns must be thoroughly addressed to protect sensitive data and maintain the integrity of the systems. Web-based development thus demands a careful balance among accessibility, functionality, and security to meet business needs effectively.

10.1.3 Choosing Between Development Strategies

A systems analyst must consider whether development will occur in a traditional or web-centric environment. There are similarities and differences between both approaches. For example, in an Internet-based system, the web becomes an integral part of the application rather than just a communication channel, and systems analysts need new application development tools and solutions to handle the new systems.

Microsoft's .NET and the open-source MERN stack represent web-based development environments. Microsoft describes .NET as a developer platform for building and running various application types written in C# and Visual Basic, including web-based, mobile, and traditional desktop applications. The acronym MERN stands for MongoDB, Express, React, and Node. MERN is used to develop universal applications in JavaScript. MongoDB is a database, and Express, React, and Node are libraries or frameworks used for full-stack web development.

Although there is a significant trend toward web-based architecture, many firms rely on traditional systems because they use legacy applications that are not easily replaced or do not require a web component to satisfy user needs. To choose between traditional and web-based development, consider some key differences between them. Building the application in a web-based environment can offer more significant benefits (and sometimes greater risks) than in a traditional environment.

10.2 Evolving Trends

In the constantly changing world of IT, no area is more dynamic than Internet technology. Evolving trends include artificial intelligence (AI) and machine learning (ML) integration, cloud computing, IoT and edge computing, mobile devices, and Web 2.0 and 3.0. These trends reflect the dynamic nature of systems analysis and design, emphasizing agility, efficiency, intelligence, security, and the integration of emerging technologies. Systems analysts should be aware of these trends and consider them as they plan large-scale systems.

10.2.1 Artificial Intelligence and Machine Learning Integration

AI and ML are revolutionizing systems operations, paving the way for more automated decision making, predictive analysis, and enriched user experiences. These technologies are increasingly being integrated into various applications, from customer service chatbots that can handle inquiries and provide instant support to complex data analytics tools capable of processing vast amounts of information for insightful business intelligence. Organizations can create innovative and adaptive solutions by embedding AI and ML into systems. These systems are designed to respond to and learn from user inputs, continuously improving performance and accuracy. This ability to learn and adapt makes AI and ML integral to developing innovative and efficient modern systems.

The impact of AI and ML integration extends beyond mere automation to fundamentally transforming how systems interact with users and data. Personalization, an essential aspect of modern software, is greatly enhanced through AI and ML. These technologies enable systems to analyze user behavior, preferences, and patterns, leading to tailored experiences for each user.

For instance, recommendation algorithms in e-commerce platforms or content streaming services use ML to provide personalized suggestions, significantly improving user engagement and satisfaction. Moreover, in sectors like healthcare and finance, AI-driven predictive analytics assist in forecasting trends and making informed decisions, contributing to better outcomes and more efficient operations.

The role of AI and ML in system development is not just about enhancing current capabilities but also about driving innovation and opening new possibilities. As these technologies continue to evolve, they enable the creation of previously unimaginable systems. Autonomous vehicles, smart city infrastructures, and advanced health diagnostics are just a few examples of how AI and ML are pushing the boundaries of technology. Furthermore, integrating AI and ML makes systems more responsive and adaptive, capable of handling complex tasks and providing deep insights that significantly boost efficiency and effectiveness. AI and ML are not just add-ons to modern systems but fundamental components that redefine what is possible in technology and software development.

10.2.2 Cloud Computing

As defined by Google, **cloud computing** represents a transformative approach to accessing and managing computing resources. It essentially provides various computing services, including servers, storage, databases, networking, and software, over the Internet ("the cloud"). This model shifts from traditional, locally hosted hardware and software to a more flexible, on-demand service. Users can access a wide range of computing resources as needed without the burden of owning and maintaining physical infrastructure. This on-demand availability simplifies IT management for businesses and individuals and provides scalability to handle fluctuating workloads effortlessly.

The essence of cloud computing lies in its similarity to an online SaaS and data environment supported by powerful remote servers. This setup enables users to run applications and store data on remote servers, accessing them online. This means that complex applications and large amounts of data can be managed more efficiently as cloud providers' robust computing infrastructures handle the heavy lifting of data processing and storage. The cloud environment also facilitates collaboration and data sharing, as information can be accessed from anywhere, anytime, provided Internet connectivity exists. This aspect mainly benefits businesses with distributed teams or those requiring remote access capabilities.

Google's paraphrased definition highlights a key advantage of cloud computing: eliminating the need for individuals and businesses to manage physical computing resources. Instead, users pay only for the resources they consume, similar to how utilities like electricity are billed. This model reduces the capital expenditure of purchasing and maintaining hardware and allows for flexible scaling of resources to match usage needs. The financial and operational flexibility cloud computing offers is a significant driver of its widespread adoption. Visually represented by a cloud symbol, this concept is now synonymous with modern computing's global, ubiquitous, and accessible nature.

10.2.3 Internet of Things and Edge Computing

The Internet of Things (IoT) represents a significant shift in systems analysis and design, extending its reach beyond conventional computing devices to encompass an extensive network of interconnected "things." This network includes various devices, such as home appliances, industrial machinery, and wearable technology, all embedded with sensors, software, and connectivity capabilities. This expansion has dramatically broadened the scope and complexity of systems design, requiring new approaches to ensure seamless integration and communication between various devices. IoT's integration into everyday objects and industrial equipment transforms them into intelligent systems capable of collecting, transmitting, and processing data, enhancing efficiency, functionality, and user experience.

Complementing IoT is the concept of edge computing, which addresses some of the inherent challenges of centralized data processing. In edge computing, data processing occurs closer to where it is generated (i.e., at the "edge" of the network) rather than being transmitted to distant data centers. This proximity to data sources significantly reduces latency, leading to faster response times, and decreases the bandwidth needed for data transmission. This is particularly crucial in real-time applications and limited connectivity scenarios. By processing data locally, edge computing enhances privacy and security, as sensitive information can be analyzed and acted upon locally without sending it across the network. This shift to decentralized computing is a natural evolution in IoT, enabling smarter and more responsive systems capable of operating independently and more efficiently.

The synergy of IoT and edge computing catalyzes the development of increasingly distributed and context-aware systems. These systems are designed to be more aware of their environment and to respond dynamically to changes, offering tailored services and experiences. For instance, in smart homes, IoT devices can detect user preferences and behaviors, adjusting settings like lighting and temperature in real time for optimal comfort. In industrial settings, IoT-enabled machinery with edge computing capabilities can monitor production processes, predict maintenance needs, and optimize operations autonomously. As these technologies continue to evolve and mature, the potential applications are vast, heralding a future where systems are interconnected, intelligent, and adaptive to their environments, revolutionizing how we interact with technology.

10.2.4 Mobile Devices

Mobile devices, including smartphones and tablets, have become ubiquitous in modern life. Their proliferation is not just limited to personal use; in the corporate world, these devices are integral tools for business operations. With their advanced computing power, mobile devices can now perform tasks that were once the domain of desktop computers. This shift has brought about a significant change in the way we interact with technology, making computing power more accessible and portable. Edge computing (discussed in the previous section) has become relevant in this context, where processing occurs directly on mobile devices at the network's edge rather than in a centralized data center. This approach enhances the speed and efficiency of data processing and reduces latency, making mobile devices compelling tools for personal and business use.

The development of applications for mobile devices has, as a result, become an essential area for systems analysts. Unlike traditional software development, mobile app development requires consideration of various factors such as screen sizes, operating system diversity, and different hardware specifications. This has given rise to numerous new platforms and frameworks specifically designed for mobile development. However, many of today's development tools and environments have adapted to these changes, offering web-based and mobile application development support. This dual capability allows developers to create applications that operate seamlessly across multiple platforms, providing a consistent user experience whether accessed via a web browser or a mobile app. This cross-platform development approach is crucial in today's interconnected world, where users expect to access services and information from any device.

Furthermore, the ubiquity of mobile devices has significant implications for businesses and developers alike. For businesses, it means rethinking how to engage with customers and employees, as mobile devices offer new channels for communication, marketing, and service delivery. For developers, it presents both a challenge and an opportunity to innovate and create applications that are not only functional but also engaging and user-friendly. The emphasis is on designing intuitive interfaces, ensuring optimal performance, and considering mobile devices' unique constraints and capabilities. Additionally, with the increasing concern over mobile security and data privacy, developers are tasked with incorporating robust security measures into their applications. Integrating mobile devices into various aspects of daily life and business operations signifies a shift in the technological landscape, making mobile app development a key area of focus in systems analysis and design.

10.2.5 Web 2.0 and 3.0

Many IT professionals use **Web 2.0** to describe a second generation of the web that enables people to collaborate, interact, and share information much more effectively. This era is characterized by the rise of social media platforms, blogs, wikis, and other user-generated content, allowing users to consume, create, and share information. Unlike the earlier version of the web, which was more about disseminating information, Web 2.0 fosters collaboration and communication, building online communities and social networks. Central to this phase was the development of web applications that are continuously available and evolving, accommodating an unlimited number of users who interact with data in diverse ways. This interactive experience, enhanced by Web 2.0, transformed the Internet into a real-time collaboration and sharing platform, making it an integral part of daily personal and professional life.

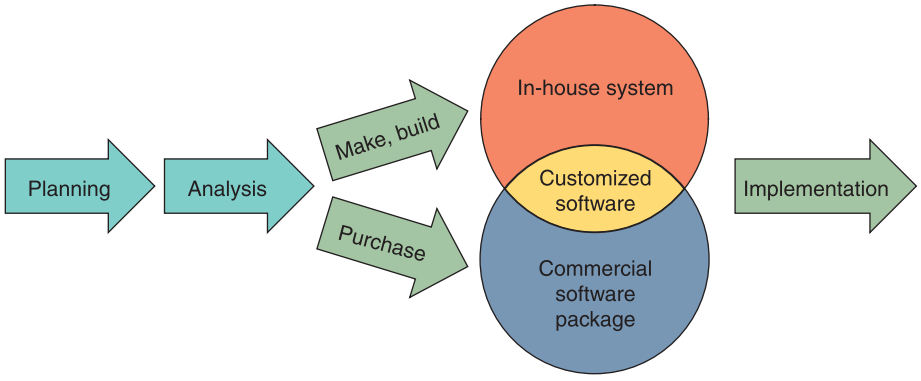
Building on the interactive foundation of Web 2.0, **Web 3.0**, or the semantic web, is poised to redefine our online experiences further. This evolution focuses on making the web more intelligent and efficient, with data being interconnected in a way that is easily accessible and understandable by both humans and machines. A key feature of Web 3.0 is the enhanced capability for data to be contextualized and processed by computers, using technologies that add metadata to information, thus making the Internet more intuitive and capable of delivering accurate and relevant results. Additionally, the advent of Web 3.0 brings a significant shift toward a decentralized web. Leveraging blockchain technology, this new phase of the web aims to distribute data across multiple nodes, enhancing security, privacy, and control over personal data, countering the centralized control seen in Web 2.0.

Integrating AI and ML in Web 3.0 promises a more personalized web experience. Computers can understand human language more effectively through natural language processing, enabling more natural interactions. IoT also plays a crucial role in Web 3.0, creating a network of interconnected devices that seamlessly communicate, further merging the physical and digital worlds. In essence, Web 3.0 aims to create an immersive, responsive web where the boundaries between individual services blur and the Internet becomes more tailored to individual needs.

10.3 In-House Development Options

A company can develop its systems or purchase (and possibly customize) a software package. These development alternatives are shown in Figure 10-1. Although many factors influence this decision, the most crucial consideration is the total cost of ownership (TCO), as explained previously in the book. In addition to these options, companies develop user applications designed around commercial software packages, such as Microsoft Office, to improve user productivity and efficiency.

Figure 10-1 Instead of outsourcing, a company can develop a system in-house or purchase and customize a commercial package.



10.3.1 Make or Buy Decision

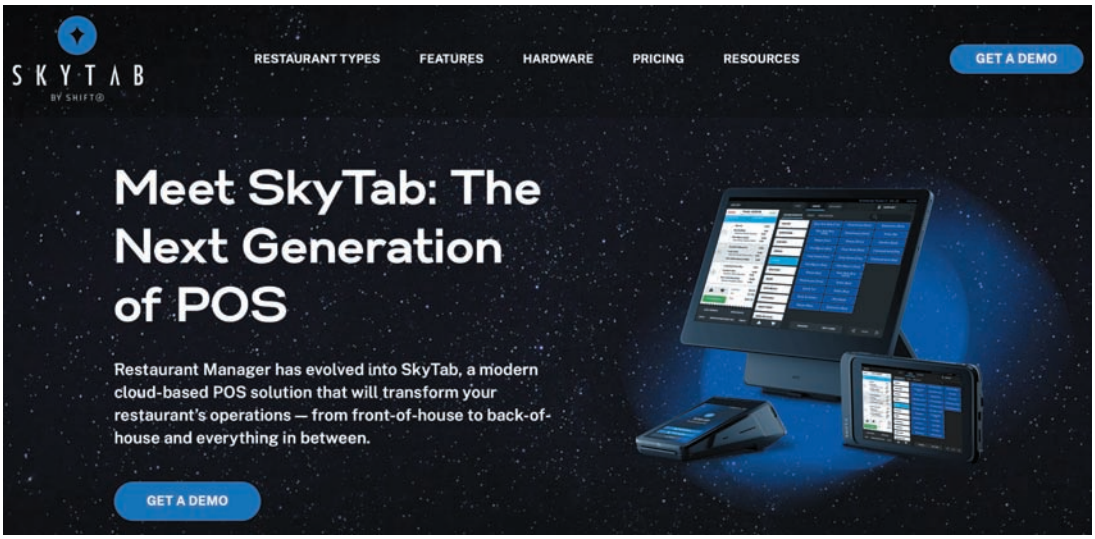
The choice between developing versus purchasing software often is called a **make or buy**, or **build or buy**, decision. The company's IT department makes, builds, and develops **in-house software**. A **software package** is obtained from a vendor or application service provider.

The package might be a standard commercial application or a customized package for purchasers. Companies that develop software for sale are called **software vendors**. A firm that enhances a commercial package by adding custom features and configuring it for a particular industry is a **value-added reseller (VAR)**.

Software packages are available for every type of business activity. A software package that many organizations can use is called a **horizontal application**. An accounting package is an excellent example of a horizontal application because many other businesses or separate divisions in large, diversified companies can utilize it.

In contrast, a software package developed to handle information requirements for a specific type of business is called a **vertical application**. For example, organizations with special system requirements include colleges, banks, hospitals, insurance companies, construction companies, real estate firms, and airlines. Figure 10-2 shows a typical restaurant point-of-sale (POS) system running on various devices. Organizations may need vertical applications to handle their unique business requirements but often use horizontal applications for basic business needs, such as payroll processing and accounts payable.

Figure 10-2 Restaurants use vertical applications like point-of-sale (POS) systems to support their unique business requirements.
Source: Shift4



Each in-house software acquisition option—developing a system, buying a software package, or customizing a software package—has advantages, disadvantages, and cost considerations, as shown in Table 10-1. These software acquisition options are described in detail in the following sections.

10.3.2 Developing Software In-House

In today's technology-driven business environment, an extensive range of software packages that cater to horizontal and vertical business operations are available. Despite this abundance, firms often opt for in-house development of their software. The primary reason for this choice is the unique nature of specific business requirements that off-the-shelf software packages may not adequately meet. Custom-developed software allows companies to tailor

Table 10-1 Companies consider various factors when comparing in-house development with purchasing a software package.

Reasons for In-House Development	Reasons for Purchasing a Software Package
Satisfies unique business requirements	Lower costs
Minimizes changes in business procedures and policies	Requires less time to implement
Meets constraints of existing systems	Proven reliability and performance benchmarks
Meets constraints of existing technology	Requires less technical development staff
Develops internal resources and capabilities	Future upgrades provided by vendor

the functionality to their needs, ensuring that the software aligns perfectly with their business processes, strategies, and goals. Additionally, developing software in-house can minimize the need to alter existing business procedures and policies to fit the constraints of prebuilt software. It also allows companies to build solutions compatible with their existing systems and technology infrastructure, thereby avoiding the complexities and costs of integrating disparate systems.

Developing software in-house also helps an organization cultivate internal resources and capabilities. Two variations on custom in-house development can aid this: open-source and reuse-oriented development strategies. Open-source software, with its publicly accessible source code, provides a flexible foundation for customization. Companies can use and modify open-source software to create tailored solutions without starting from scratch. This approach combines the benefits of custom development with the efficiency and cost-effectiveness of leveraging existing, community-vetted code. Furthermore, open-source development fosters innovation and collaboration, as businesses can benefit from the collective intelligence and advancements made by the global developer community. By participating in open-source projects, companies can contribute to and influence the development of software critical to their operations, ensuring that the software continues evolving in ways that align with their needs.

Software reuse is integral to efficient software development, whether in-house or open source. Reusing existing code, libraries, or software modules can accelerate development, reduce costs, and improve reliability. When companies develop custom software, they can design reusable components that can be leveraged in future projects, thereby maximizing the return on their investment in software development. In open-source development, the principle of reuse is even more pronounced, as developers have access to a vast repository of existing components and modules that they can adapt to their specific requirements. This not only speeds up the development process but also enhances the quality of the software, as reused components have been tested often and refined in diverse scenarios. Reuse is a strategic approach in software development that promotes efficiency, fosters standardization, and drives innovation by building upon proven solutions.

Satisfy Unique Business Requirements

Companies frequently opt for in-house software development when commercially available software packages fail to meet their unique and specific business requirements. This approach is particularly prevalent in industries or sectors with highly specialized operations. For instance, a college might require a tailored course scheduling system that intricately balances and aligns many factors, such as curriculum requirements, student demand, classroom space availability, and instructor schedules. The complexity and uniqueness of such a system are often beyond the scope of generic software solutions, necessitating a custom-developed application that can accurately address these specific needs. Similarly, a

package delivery company faces unique logistical challenges, needing a system capable of optimizing routes and loading patterns for its delivery fleet. Such a system must consider variables like delivery routes, vehicle capacity, traffic patterns, and delivery time frames, which are specific to the company's operational model. In cases like these, where the operational demands are highly tailored and nuanced, off-the-shelf software packages typically lack the necessary flexibility and customization, making in-house development preferable and often the only viable option.

This necessity for in-house development arises because generic software packages are usually designed to cater to a broad market, with features and functionalities that address standard business processes and practices. While versatile and cost-effective for many businesses, these off-the-shelf solutions might not be able to handle the unique challenges and complexities specific to certain organizations. For instance, the college's course scheduling system and the package delivery company's logistics application require a level of customization and specificity that standard software packages are not built to provide. Developing in-house software allows these organizations to tailor the application to their requirements, ensuring that the software adequately supports every aspect of their operations. Moreover, in-house development offers the flexibility to modify and upgrade the software as the organization grows and its needs evolve, which is often challenging with rigid, prebuilt software solutions. Therefore, for businesses with particular operational needs, in-house software development is not just a choice but a strategic necessity to ensure efficiency, competitiveness, and success.

Minimize Changes in Business Procedures and Policies

Opting for in-house software development can be a strategic choice for a company, especially when implementing commercially available software packages necessitates substantial changes in existing business operations or processes. This situation often arises when the workflows and functionalities embedded in off-the-shelf software do not align with the company's established practices and procedures. While adapting to new software typically involves a learning curve and some operational adjustments, the extent of change required can sometimes be too disruptive or impractical. For instance, a software package might demand a complete overhaul of current processes, necessitate significant staff training, or even alter how a business interacts with its customers or manages its internal affairs. In such scenarios, the cost of adapting to a new system, both in terms of operational disruption and the potential impact on company culture, can outweigh the benefits of the software. Consequently, a company may develop software tailored to dovetail seamlessly with its existing operations, thereby minimizing disruption and preserving established workflows.

The decision to develop in-house software under these circumstances also allows businesses to align their technological tools closely with their strategic objectives. By building custom software, companies can ensure that every feature and functionality directly contributes to their operational efficiency, competitive advantage, and overall business goals. This customized approach enables a more precise fit for the company's needs without the compromises that often come with generic software solutions. Moreover, in-house development can be more agile and responsive to business or market environment changes. As the company grows or shifts direction, the software can be adapted and scaled accordingly, which might not be as feasible with a rigid, purchased package. This flexibility and adaptability are crucial in dynamic business landscapes where staying responsive and efficient is vital to success. Therefore, when introducing a commercial software package poses significant challenges to a company's existing business, developing custom software becomes a valuable alternative, offering operational continuity and strategic alignment.

Meet Existing Systems Constraints

Integrating new software into an organization's technological framework is critical to any system implementation. The new software must be compatible and function seamlessly with the current systems. For example, in the case of a company looking to implement a new

budgeting system, this system must interface effectively with the existing accounting system. This integration is crucial to maintain data consistency, ensure smooth operations, and avoid disruption in financial management. However, finding a commercially available software package that perfectly aligns with and complements existing systems can be challenging. Off-the-shelf software often has predefined specifications and limitations, which might not align with the unique configurations and customizations of a company's current systems. As a result, achieving seamless integration without extensive modifications can be a complex and sometimes unfeasible task.

Companies may develop their own software solutions when suitable off-the-shelf software is unavailable or the integration challenges are too significant. They can tailor the new software to interface with their existing systems by choosing this route. This custom development ensures that the new software meets the functional requirements and aligns with the technical architecture of the current system. It allows for customization and flexibility that is typically impossible with standard software packages. Additionally, in-house development gives the company complete control over the software's features and capabilities, enabling them to create a solution that integrates seamlessly with their existing systems and evolves with them. Developing custom software becomes a strategic decision to ensure technological coherence, operational efficiency, and the long-term scalability of the company's IT infrastructure.

Meet Existing Technology Constraints

Developing software in-house is often a deliberate choice for organizations when the new system needs to integrate seamlessly with existing hardware and legacy systems. This requirement can pose significant challenges, as off-the-shelf software may not always be compatible with the specific configurations and limitations of an organization's current technological infrastructure. Custom software design becomes essential in such cases, either to ensure that the new software can operate effectively within the constraints of the existing systems or to facilitate necessary upgrades to the environment. This approach allows organizations to maintain their investment in legacy systems while still benefiting from new, tailored functionalities. In-house development provides the flexibility to create software designed to work with the organization's unique hardware and system setups, ensuring compatibility and preventing potential disruptions that could arise from integration issues.

The role of a systems analyst is crucial in determining the technical feasibility and appropriateness of in-house software development. During the preliminary investigation phase, the analyst evaluates the technical feasibility of various software solutions, including compatibility with existing systems and hardware. This assessment is vital to the decision-making process, as it determines whether the organization can realistically develop and implement the software in-house or should consider alternative solutions. In the systems analysis phase, the analyst delves deeper, considering factors such as the organization's technical capabilities, resource availability, time constraints, and the potential return on investment. This comprehensive analysis is critical in determining whether developing software in-house is the most viable and cost-effective solution for meeting the organization's specific needs. The analyst's findings and recommendations are pivotal in ensuring that the chosen approach aligns with the current technological environment and the organization's strategic objectives.

Develop Internal Resources and Capabilities

Designing and developing an in-house system offers companies the advantage of building and nurturing an IT staff intimately familiar with the organization's unique business functions and information support requirements. This deep understanding of the business enables the IT team to tailor systems precisely to the company's needs, ensuring that the technology aligns closely with business objectives and processes. Additionally, having an in-house team means the company possesses IT resources and capabilities finely attuned to its operations, which can be a substantial competitive advantage. An in-house IT team can react swiftly and

effectively to business challenges and opportunities, implementing immediately relevant and practical solutions. For instance, in scenarios requiring rapid response to address emerging market trends or operational issues, an internal team can mobilize quickly, applying their understanding of the company's systems and strategies to develop timely and effective technological responses.

On the other hand, relying solely on external resources for IT support can place a company in a dependent position, particularly in critical situations where quick and informed decisions are needed. While outsourcing IT functions can be attractive, especially for short-term projects or specific technical needs, it doesn't always lead to a lower TCO in the long run. Outsourced solutions often involve dealing with multiple vendors and contracts, which can add complexity and costs over time. Moreover, top management generally feels more confident having an internal IT team that provides consistent, long-term guidance and stability for the company's technological direction. This internal team understands the company's current technology landscape and is instrumental in shaping its future trajectory. In-house development allows the firm to capitalize on the skills and expertise of the existing IT staff, who are already integrated into the company's culture and operations and are being compensated as part of the workforce. This approach fosters a more cohesive and aligned IT strategy, which is crucial for long-term success in a technology-driven business environment.

10.3.3 Purchasing a Software Package

When a company decides to acquire new software, purchasing a commercially available software package is one of the primary alternatives to in-house development or outsourcing. This approach offers several advantages. First, it tends to be more cost-effective than developing software from scratch, as it eliminates extensive expenses associated with the development process, such as hiring specialized personnel and significant research and development investment. Additionally, commercially available software can often be implemented more quickly than a system developed in-house, allowing businesses to benefit from new functionalities sooner. These packages generally have proven reliability and performance, giving businesses confidence in their stability and efficiency. Another significant advantage is the reduced need for an extensive technical development staff, as the vendor typically manages future updates and system maintenance. Moreover, there is an opportunity to learn from other companies that have already implemented the software, which can be invaluable in informing the integration and deployment process.

However, purchasing a software package versus developing it in-house involves carefully considering multiple factors. The cost is crucial; while buying a package might seem less expensive upfront, it's essential to consider TCO, including ongoing licensing fees, costs for customization, and system integration. Functionality is another vital consideration; the software should meet specific business requirements, and off-the-shelf solutions might necessitate significant modifications to achieve this. Vendor support also plays a key role; dependable and responsive vendor support is essential for the timely resolution of issues and ensures the long-term effectiveness of the software. To determine the most suitable approach, businesses must evaluate these and other factors, such as compatibility with existing systems, scalability for future growth, and adherence to industry standards.

Choosing whether to purchase a software package or develop software in-house is a significant decision that impacts various aspects of a business. While buying a package presents benefits such as cost savings, rapid deployment, and established reliability, it is crucial for companies to thoroughly assess how these solutions fit their specific needs and future objectives. Decision making should comprehensively evaluate cost, functionality, vendor support, and system compatibility. The chosen route should address not only the immediate requirements of the business but also offer the necessary flexibility and scalability to accommodate future business challenges and growth.

Lower Costs

Companies' widespread use of software packages allows software vendors to distribute the development costs across a broad customer base. This economic model significantly reduces software costs for each customer, making commercially available software packages a cost-effective choice, particularly in terms of the initial investment. Purchasing a software package is generally more economical than the expenses associated with in-house software development, which includes hiring or training specialized development staff, investing in research and development, and dedicating resources to ongoing maintenance and updates. This lower cost is a significant factor driving businesses toward opting for off-the-shelf software solutions, as it allows them to access sophisticated technology without incurring the substantial costs of developing it internally.

However, the financial consideration of purchasing software extends beyond just the initial cost. Adopting a new software package can entail additional expenses that stem from the need to modify or disrupt existing business processes to accommodate the new system. This disruption often necessitates significant changes in how employees perform their tasks, leading to reduced productivity as staff adjust to the new system. Moreover, retraining employees to use the latest software effectively can incur considerable costs, both in terms of the training itself and the potential loss of productivity during the training period. These hidden costs sometimes offset the initial savings and should be carefully considered when deciding between purchasing a software package and developing a solution in-house. While the upfront investment in a software package might be lower, the long-term financial implications, including the cost of integrating the software into the existing business ecosystem and training employees, can make it a less economical choice in some scenarios.

Requires Less Time to Implement

Purchasing a software package offers the distinct advantage of acquiring a solution that has already gone through the critical stages of design, programming, testing, and documentation. This aspect of prebuilt software significantly reduces the time and resources a company would otherwise have to invest if it were developing a solution in-house. When software is developed internally, substantial efforts and resources are required for these stages, often involving lengthy development cycles and extensive testing to ensure that the software meets the required standards and functions as intended. A company bypasses these time-consuming and resource-intensive processes by opting for a prepackaged solution. However, while the core software is ready for use, it still requires installation and integration into the company's existing systems environment. Though generally less time-consuming than full-scale development, this integration process can still be complex, especially if the new software needs to interface with various existing applications and systems.

Despite the quicker implementation of purchasing a software package, the TCO can sometimes be higher than initially expected. This cost increase can be attributed to additional factors arising after the purchase, such as training expenses and the need for software modifications. Training employees to effectively use new software can be a significant investment in direct training costs and indirect costs associated with employees' time to become proficient with the new system. Additionally, off-the-shelf software may not perfectly align with a company's specific processes and workflows, necessitating customizations or modifications to the software. While essential for ensuring that the software meets the business's unique needs, these modifications can add to the overall cost of the technical work required and potential disruptions to business operations during the integration process. While purchasing a software package can offer time savings and immediate access to advanced functionalities, these benefits must be weighed against the potential long-term costs of training and customization.

Proven Reliability and Performance Benchmarks

When a software package has been available on the market for a while, it offers the advantage of having undergone extensive real-world usage, during which any significant issues are likely identified and addressed by the vendor. Over time, continuous user feedback

and experience contribute to the refinement of the software, leading to the resolution of bugs and enhancement of features. This process of ongoing improvement means that later versions of the software are generally more stable and reliable, having benefited from the iterative process of troubleshooting and updates. The vendor's commitment to rectifying problems and upgrading the software enhances its performance and reassures customers of the product's quality and reliability. This aspect is vital for businesses, as it reduces the risk of implementing new software and ensures a smoother integration into their operational environment.

In addition to the improvements made by vendors, popular software products often undergo independent evaluations and reviews, providing potential buyers with unbiased insights into the software's performance and suitability. These reviews, conducted by industry experts or technical publications, can be invaluable resources for businesses considering the purchase of a software package. They offer an external perspective on the software's functionality, user-friendliness, compatibility with various systems, and overall value. This independent assessment helps businesses make informed decisions by highlighting the strengths and weaknesses of the software based on real-world testing and analysis. These reviews can provide assurance and guidance for a company investing in a software solution, supplementing the vendor's claims with objective evaluations. Such independent assessments are especially beneficial when comparing multiple software options, as they provide a standardized benchmark for assessing the quality and effectiveness of different products in the market.

Requires Less Technical Development Staff

Commercial software packages allow companies to streamline their IT staff, particularly in reducing the number of programmers and systems analysts required. This reduction is possible because commercial software typically comes predesigned, developed, and tested, thus eliminating the need for extensive in-house development and troubleshooting teams. Companies can avoid the often resource-intensive process of building applications from scratch by relying on externally developed software solutions. This cuts down on the overhead costs associated with maintaining a large IT staff and allows for reallocating resources to other critical areas of the business. In addition, using commercial software can often bring about efficiencies in deployment and maintenance, as these tasks are generally more straightforward than those for custom-built software. Consequently, companies can maintain a leaner, more focused IT team optimized for managing and supporting the implemented software systems.

Furthermore, by incorporating commercial software into their operations, companies enable IT staff to shift their focus toward more specialized systems that require custom solutions. These are typically systems with unique requirements that off-the-shelf software packages cannot adequately address. By freeing up IT resources from the routine tasks of developing and maintaining standard applications, companies can channel their technical expertise into areas where specially made solutions are necessary. This approach allows the IT team to concentrate on developing systems that provide a competitive edge or address specific operational challenges unique to the business. It also encourages more strategic use of the IT department's skills and expertise, aligning their efforts with the company's core business objectives and fostering innovation where it is most needed. This strategic alignment not only enhances the effectiveness of the IT staff but also ensures that the company's technological capabilities are fully leveraged to support its overall goals and strategies.

Future Upgrades Provided by the Vendor

Software vendors routinely update their software packages, adding enhancements and improvements to develop new versions or releases. These upgrades are integral to maintaining the software's relevance and usability in an ever-evolving technological landscape. A typical example of such upgrades can be the inclusion of new drivers in a software package to support emerging hardware technologies, like a recently launched laser printer or a novel

data storage technology. These additions ensure that the software remains compatible with the latest hardware and continues to meet the changing needs of its users. Additionally, vendors often incorporate features that enhance the software's performance, user interface, and security, improving the overall user experience. Regular updates keep the software current and help address any bugs or security vulnerabilities that may have been identified in previous versions. This continual improvement is crucial for maintaining the software's market competitiveness and user satisfaction.

In planning these upgrades, vendors frequently rely on feedback and suggestions from their current user base. This user input is invaluable, as it provides insights into how the software is being used in real-world scenarios and highlights areas that need improvement or additional features. Users, the direct beneficiaries of the software, can offer practical suggestions on functionality, usability, and integration needs based on their experiences. By incorporating this feedback into future upgrades, vendors can ensure that the software evolves in a way that aligns with the actual requirements and preferences of its users. This collaboration between vendors and users fosters a user-centric development approach, leading to software that is not only technologically advanced but also highly attuned to the needs of its market. Such a strategy enhances the utility and appeal of the software and builds a loyal user community that feels valued and heard.

10.3.4 Customizing a Software Package

When a standard version of a software product doesn't fully meet a company's specific requirements, one viable option is to adapt or customize the software package. One way to achieve this is by purchasing a basic package version, which vendors can customize to suit the company's needs. Many software vendors offer their products in a modular format, providing a standard core package with the option of adding individual components or features. This modular approach allows for customization, enabling companies to select and incorporate the most relevant functionalities to their operations. For example, in the case of a human resources information system, customization is often necessary, as companies have unique ways of handling employee compensation and benefits. By choosing a basic package with customizable options, a firm can tailor the software to its specific processes, ensuring that the software aligns with its unique business practices.

Another approach to customizing a software package involves direct negotiation with the software vendor to make specific enhancements that meet the project's unique requirements. This method typically involves an additional cost, as the company pays the vendor to develop and implement the necessary changes. The advantage of this approach is that it leverages the vendor's expertise in their product, ensuring that the modifications are seamlessly integrated and function as intended. This direct collaboration with the vendor can be particularly effective for complex or specialized changes, where the vendor's in-depth knowledge of the software's architecture and capabilities can be invaluable. However, the company must clearly communicate its needs and work closely with the vendor to ensure that the final product effectively meets the desired specifications.

The third way to customize a software package is for the company to purchase the software and make project-specific modifications in-house, provided the software's licensing terms permit such changes. This approach gives the company direct control over the customization process, tailoring the software closely to its specific needs. However, it does come with challenges. For instance, the company's systems analysts and programmers must be sufficiently familiar with the software to make effective modifications. This often requires time for the staff to learn the intricacies of the software package, understand its underlying architecture, and develop the skills necessary to make the required changes accurately. While this approach offers a high degree of flexibility, it also demands significant internal expertise and resources, and there's the risk of encountering difficulties in maintaining the software over time, especially when dealing with complex systems or extensive modifications.

Unfortunately, the benefits of purchasing a standard software package can diminish when significant product customization is required. Customization, mainly when performed by the software vendor, often leads to additional costs, as tailoring the software to specific business needs is typically billed separately from the standard package price. Furthermore, customized solutions usually take longer to deploy than standard off-the-shelf software because of the extra time needed for the vendor to implement the specific changes requested by the client. This delay in implementation can be a critical factor for businesses that must deploy the new system quickly to meet operational demands or gain a competitive advantage. Thus, while customization allows for a more tailored fit to the company's requirements, it can erode some of the primary benefits of opting for a standard software package.

Future support and maintenance present another significant issue with customized software packages. Vendors regularly update their standard software offerings, enhancing features, improving security, and fixing bugs. However, these updates may not always apply to customized software versions. If a company has had its software package customized by the vendor, future standard upgrades might not integrate seamlessly with the modified system, potentially leading to compatibility issues or functional gaps. The situation becomes even more complex if the company has made the modifications in-house. In such cases, when a new version of the software is released, the company is often responsible for updating and modifying it to incorporate their custom changes, as vendors typically do not support modifications made by the customer. This adds another layer of complexity and resource allocation to the upgrade process and can lead to significant challenges in maintaining the integrity and functionality of the customized software over time.

10.3.5 Creating User Applications

A **user application** can often be a suitable alternative to formal information systems or commercial software packages for fulfilling specific business requirements. User productivity systems are discussed earlier in the book. Unlike traditional software solutions, user applications leverage standard business software like Microsoft Word or Excel, configured to maximize user productivity. For instance, an IT support person might set up a form letter in Word, linked to an Excel spreadsheet that calculates incentives and discounts, to assist a sales representative in quickly responding to customer pricing inquiries. The IT staff can further enhance these applications by creating user-friendly interfaces, which include screens, commands, controls, and features that facilitate more efficient interaction with the application.

In certain scenarios, user applications offer a simple, cost-effective solution to meet business needs. This is particularly relevant when IT departments face project backlogs and solutions tailored for individuals or small groups don't take priority. Modern application software has become more powerful, flexible, and user-friendly, making it an ideal tool for creating efficient user applications. Companies like Apple and Microsoft provide software suites and integrated applications to exchange data across various programs. These suites often include tutorials, wizards, and help features, guiding users who understand their tasks but require assistance with the software.

Many companies are moving toward empowering lower-level employees by providing them with greater data access and potent data management tools. The primary goal is to enable these employees to access the data needed for their jobs independently, without relying on the IT department. This empowerment can be achieved through effective user interfaces for company-wide applications, like accounting or sales systems, or by customizing standard productivity software to create specific user applications. Such empowerment not only enhances the productivity of the IT department by reducing the time spent on daily user queries but also allows the IT team to focus on more strategic, high-impact system development projects.

However, while empowerment through user applications reduces costs and makes good business sense, companies must provide necessary technical support for the users. In most medium- to large-sized companies, this support is offered through a **service desk** within the IT department. The service desk provides hotline assistance, training, and guidance, helping users tackle technical challenges. This support structure ensures that employees can effectively utilize these tools for their information needs.

When deploying user applications, especially those accessing corporate data, it is imperative to implement appropriate controls to maintain data security and integrity. Specific files might need to be completely hidden from user view, while others might be set to read-only mode, allowing users to view data without the ability to modify it. This careful management of data access rights ensures that while users are empowered with the tools they need, the integrity and security of the company's data are not compromised. This balance is essential for maintaining operational efficiency and data safety within the organization.

Case in Point 10.1

Doug's Sporting Goods

Doug's Sporting Goods sells hiking and camping supplies. The company has grown considerably in the last two years. They want to develop a customer order entry system and hired your IT consulting firm to advise them about development strategies. They lean toward in-house development because they do not want to depend on outside vendors and suppliers for technical support and upgrades. They also say they are not interested in selling online, but that could change. They want to meet with you tomorrow to make a decision. What will you say to them at the meeting?

10.4 Outsourcing

Outsourcing in IT is a strategic decision by companies to transfer various aspects of their information systems—including development, operation, or maintenance—to an external service provider. This transfer is conducted under an agreement where the external firm provides these services for a specified fee, which can be structured for either short-term tasks or long-term engagements. The scope of outsourcing can vary widely, ranging from relatively minor and specific tasks to more significant and comprehensive functions. For instance, a company might outsource small-scale programming jobs to specialized firms to leverage niche expertise or to handle overflow work. In other cases, businesses might rent software from a service provider, which is particularly common with **software as a service (SaaS)** models, as explained in Section 10.6. This approach allows companies to use high-quality, up-to-date software without the substantial investment typically required for purchasing and maintaining such systems in-house.

Outsourcing extends to more substantial business functions through **business process outsourcing (BPO)**. BPO involves handing over entire business processes to an outside firm, such as customer service operations, human resources management, or accounting. Outsourcing allows businesses to focus on their core competencies while entrusting non-core functions to external specialists, often resulting in increased efficiency and cost savings. By outsourcing these processes, companies can benefit from the expertise and advanced technologies offered by service providers, leading to improved process execution and innovation. Furthermore, outsourcing arrangements can be highly flexible, allowing businesses to scale services up or down based on their changing needs. This flexibility is particularly valuable in dynamic market conditions.

Sometimes, companies may outsource their entire IT function, representing a significant shift in strategy and operational structure. This comprehensive form of outsourcing involves entrusting all aspects of a company's IT operations—from software development and system maintenance to IT infrastructure management and technical support—to an external provider. This approach can offer several benefits, including access to a broader range of expertise, state-of-the-art technologies, and potentially significant cost reductions. Outsourcing the entire IT function can also free up internal resources, allowing companies to allocate more focus and capital to their primary business activities. However, this level of outsourcing also requires careful consideration and management of risks, such as loss of control over certain aspects of the IT operations and reliance on the external provider's ability to deliver consistent, high-quality services. Therefore, while complete IT outsourcing can be transformative for businesses, it demands a strategic approach, thorough planning, and robust partnership management to ensure success.

10.4.1 The Growth of Outsourcing

Traditionally, firms have turned to outsourcing IT tasks primarily to control costs and manage the challenges associated with rapid technological advancements. According to data cited by Oracle, businesses often allocate up to 80% of their IT budgets to maintaining existing software and systems. This financial burden forces IT managers to focus their attention and resources on managing upgrades and routine maintenance, diverting them from potentially revenue-generating IT projects. While cost management and technological agility remain valid reasons for outsourcing, it has become a key component of broader IT strategies for many organizations. This shift reflects a deeper integration of outsourcing into the core operational frameworks of businesses.

The global outsourcing industry has witnessed substantial growth in recent years, driven by several key factors. The increasing adoption of digital technologies across various sectors and the strategic need for cost efficiency have prompted businesses to focus on their core competencies and outsource nonessential activities. This trend has led to the expansion of the industry beyond traditional IT services to include diverse areas such as BPO, **knowledge process outsourcing (KPO)**, and **legal process outsourcing (LPO)**.

The rise of remote work, significantly accelerated by the COVID-19 pandemic, has played a crucial role in reshaping the landscape of businesses worldwide, thereby contributing to the expansion of the outsourcing sector. The shift to **work from home (WFH)** has highlighted the need for more flexible and scalable solutions that adapt to the changing work environment. As businesses scrambled to adjust to the new reality of a distributed workforce, the demand for outsourcing services that could provide remote support, cloud-based solutions, and digital transformation expertise grew exponentially. Outsourcing companies, with their ability to offer diverse and adaptable services, became key partners for businesses looking to navigate the challenges posed by this sudden transition. This need for agility and scalability in operations, driven by remote work, has increased the reliance on outsourcing and prompted these service providers to innovate and expand their offerings. Consequently, the outsourcing sector has experienced significant growth, as it plays an instrumental role in enabling businesses to maintain continuity, efficiency, and competitiveness in a rapidly evolving work environment.

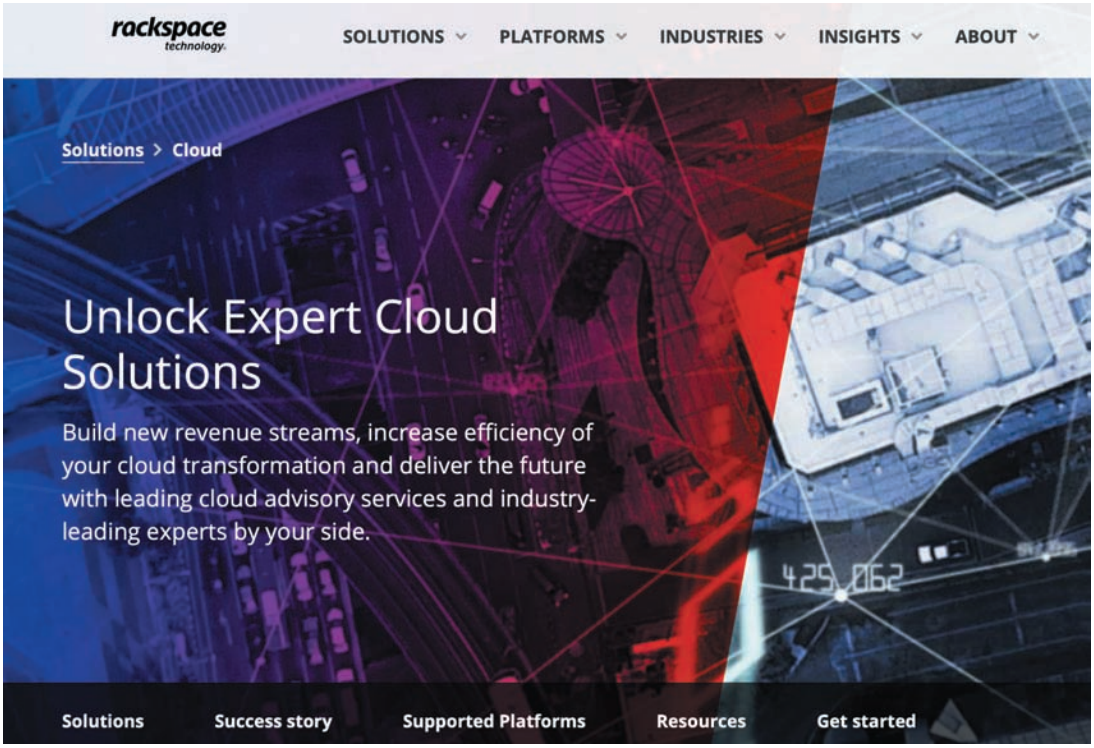
Global market expansion has also been a significant driver behind the rise of outsourcing. Companies can gain valuable local insights and expertise crucial for successful market entry by engaging with outsourced teams, especially in target international markets. These teams often bring a deep understanding of local cultures, legal requirements, and business practices, which can be instrumental in navigating new and unfamiliar territories. Establishing a presence through outsourcing is a strategic foothold, allowing companies to test markets with minimal investment compared to setting up full-scale local offices. This approach mitigates the risks associated with global expansion and offers the flexibility to adapt strategies based on market responses.

Within the outsourcing industry, firms known as **service providers** offer a range of solutions. Some providers specialize in certain software applications, while others deliver business services such as order processing and customer billing. Service providers also offer comprehensive, enterprise-wide software solutions, integrating and managing functions like accounting, manufacturing, and inventory control. This variety of services reflects the diverse needs of businesses seeking outsourcing solutions, ranging from specific software functionality to broader business process management.

Another significant aspect of the outsourcing industry is the role of the **application service provider (ASP)**. These firms deliver software applications or provide access to applications by charging a usage or subscription fee, offering more than just a license to use the software. ASPs rent out software packages to customers, typically including commercially available software such as databases and accounting packages. For instance, when a company uses an ASP for a data management package, it eliminates the need for designing, developing, implementing, or maintaining the software in-house.

In addition to ASPs, some firms offer **Internet business services (IBS)**, which provide robust web-based support for critical business transactions like order processing and customer relationship management. IBS, also known as **managed hosting**, allows companies to benefit from online data center support, mainframe computing power for essential functions, and universal Internet access. This model has become increasingly attractive to businesses seeking cloud-based solutions. Many firms, such as Rackspace, compete in the managed cloud services market, as shown in Figure 10-3.

Figure 10-3 Rackspace managed cloud services.
Source: Rackspace Technology



10.4.2 Outsourcing Fees

Firms offering SaaS have transitioned from selling software as a product to providing it as a service, leading to diverse fee structures. These structures are primarily based on how customers intend to use the application over a specific period. The essence of these pricing

models is to offer flexibility and scalability, both for the service providers and their users. This approach allows SaaS providers to cater to a wide range of customers, from small to large enterprises, with varying software needs and usage patterns. The pricing models are carefully designed to align with the services offered, the specific needs of the target customer base, and the overall value proposition of the software. This customer-centric approach ensures that users pay for what they need and use, making SaaS an attractive and cost-effective software solution for various businesses.

One prevalent fee structure in SaaS is the **fixed fee model**. Under this arrangement, customers pay a set fee, which is determined based on a predefined level of service and user support. This model suits businesses with stable and predictable software usage needs. It offers the simplicity of a consistent monthly or annual cost, making budgeting and financial planning more straightforward for companies. Organizations that prefer a clear and predictable expenditure pattern for their software services often favor the fixed fee model without the variability of usage-based models.

Another popular pricing model in the SaaS industry is the **subscription model**. In this framework, the fee varies based on the number of users or workstations that access the application. This highly scalable and flexible model suits businesses with fluctuating software usage needs. It allows companies to add or remove users as required, ensuring they only pay for the capacity they need at any time. The subscription model particularly appeals to growing businesses anticipating changes in their software requirements over time.

The **usage model** or **transaction model** is another flexible pricing structure employed in SaaS offerings. Here, the fee charged to the customer varies according to the volume of transactions or operations the application processes. This model aligns the cost directly with the level of activity or usage of the software, making it an equitable and usage-centric approach. It is especially suitable for businesses whose software needs fluctuate significantly, as it ensures that they only pay for the resources they consume. This model can lead to significant cost savings for companies with highly variable or seasonal software usage patterns.

When a company considers outsourcing its software needs to a SaaS provider, it is crucial to carefully assess its usage characteristics to determine the most suitable fee structure. Estimating the expected level of software use, whether in terms of the number of users, the volume of transactions, or a fixed service level, can guide the company in selecting the most cost-effective pricing model. Once the desired model is identified, the company can negotiate a service contract with the SaaS provider based on that structure. This careful planning and negotiation process ensures that the company secures a software solution that meets its needs and aligns with its financial and strategic objectives.

10.4.3 Outsourcing Issues and Concerns

When a company opts to outsource its IT functions, it takes a crucial step that can significantly impact its resources, operations, and profitability. The decision to outsource mission-critical IT systems must be carefully evaluated to ensure that it results in a cost-effective, reliable business solution that aligns with the company's long-term strategy. Outsourcing should only be considered if it involves an acceptable level of risk and contributes positively to the organization's overall objectives. Additionally, when moving IT work overseas, companies face other challenges that include concerns about maintaining control, managing cultural and communication differences, and ensuring robust security measures are in place to protect the company's assets and data.

Beyond the strategic implications, outsourcing also raises several operational concerns. One of the primary concerns is the handling of sensitive data. When outsourcing, a company entrusts its critical data to an external service provider, relying on them to uphold stringent standards of security, confidentiality, and quality. Furthermore, companies must thoroughly examine legal and contractual issues before outsourcing. These include matters related to

insurance, potential liability, licensing, information ownership, warranties, and plans for disaster recovery. Such due diligence is crucial to mitigate risks and safeguard the company's interests.

Outsourcing also involves complexities related to data transfer jurisdictions, mainly when dealing with international service providers. Under regulations such as the **General Data Protection Regulation (GDPR)**, there are stringent rules governing the transfer of personal data outside the European Union (EU) or the European Economic Area (EEA). Companies must ensure that any outsourcing arrangement complies with these regulations and that data subjects' rights, such as access, rectification, and erasure of personal data, are fully respected and maintained by the third-party service provider. This adherence to legal requirements is essential to avoid potential penalties and maintain the trust of customers and stakeholders.

The reliability and stability of the outsourcing firm are also critical factors to consider. In a dynamic economic environment, the viability of business partners can change, impacting the quality and continuity of services provided. Companies must conduct thorough due diligence on potential outsourcing partners, reviewing their operational history, financial health, and track record. This scrutiny helps assess the outsourcing firm's ability to deliver consistent, high-quality services and mitigate risks associated with business uncertainties like economic downturns or service disruptions.

Finally, the implications of outsourcing on internal staffing and corporate culture must be considered. Outsourcing can be particularly beneficial for companies with fluctuating work volumes, such as defense contractors, as it provides flexibility in scaling IT operations without needing to adjust in-house staffing levels continually. However, a significant downside of outsourcing is the potential impact on employee morale and job security, especially within the IT department. Talented IT professionals often prefer stable positions that offer long-term prospects and opportunities for growth, and they might be inclined to seek employment elsewhere, including with the service provider, if they perceive a lack of commitment to in-house development. This aspect underscores the need for careful planning and communication when implementing an outsourcing strategy to maintain a motivated and skilled workforce.

10.5 Offshoring

Offshoring, also commonly referred to as **offshore outsourcing** or **global outsourcing**, involves relocating IT development, support, and operational tasks to countries other than where the primary business operations are located. This practice has become increasingly prevalent among firms looking to leverage the global talent pool and cost efficiencies. Much like the trend observed with manufacturing jobs over the past several decades, IT work is being sent overseas at a growing rate. The primary motivation behind this shift is the search for more cost-effective IT development and maintenance solutions as businesses strive to optimize their operations and reduce expenses. Offshoring offers access to diverse skills and technologies that might not be readily available or affordable domestically.

The transition of IT work to offshore locations tends to be more rapid and seamless than in manufacturing due to the nature of the work and the ease of transfer. IT tasks can be shipped across networks, and consultants can be flown in as needed, bypassing the logistical complexities associated with the physical movement of raw materials, factory setup, and the intricacies of tariffs and transportation. This ease of transfer has been a significant factor in the growth of offshore outsourcing. Several years ago, IT consulting firm Gartner accurately forecasted a steady increase in offshore outsourcing. Their predictions highlighted an evolution like outsourced work, shifting from labor-intensive maintenance and support to more sophisticated tasks like high-level systems development and software design. This evolution signifies a move toward outsourcing more complex and critical IT functions, underscoring the increasing trust and reliance on offshore solutions.

The primary driving force behind the decision to offshore IT functions is the same as with domestic outsourcing—a reduction in costs, leading to an improved bottom line. By offshoring, companies can take advantage of lower labor costs in other countries, significantly reducing expenses. This approach allows businesses to focus on their core competencies by delegating IT tasks to specialized offshore teams. The cost savings achieved through offshoring can be substantial, making it an attractive option for companies looking to enhance their competitiveness and profitability.

However, offshore outsourcing has its unique set of risks and concerns. One of the primary issues is the potential backlash from workers, customers, and shareholders, particularly in the home country. These groups often raise concerns about the economic impact of sending jobs overseas, including the loss of local employment opportunities and its broader financial implications. Public protests and heightened awareness about these issues can pose significant challenges for companies considering offshoring. Additionally, offshoring involves complex matters related to project control, as managing and overseeing tasks across different time zones and geographical locations can be challenging. Ensuring the security of data and intellectual property is another critical concern, given the diverse legal and regulatory environments in different countries.

Moreover, successful offshore outsourcing requires navigating cultural differences and maintaining effective communication across borders. These challenges are particularly pronounced when critical business functions are handled by teams located halfway around the globe. Misunderstandings arising from cultural disparities can lead to inefficiencies and errors, while communication barriers can hinder collaboration and project progress. Companies must invest in establishing robust communication channels and cultural training to ensure smooth interactions and efficient management of offshore teams. Addressing these challenges is crucial for leveraging the full benefits of offshore outsourcing while minimizing potential risks and disruptions to the business. In response to these challenges, some companies are considering or undertaking **reshoring**, also known as **onshoring**, a process of bringing back previously offshored operations to their home country to regain more direct control and reduce the complexities associated with offshore outsourcing.

Case in Point 10.2

Turnkey Services

Turnkey Services is an application service provider that offers payroll and tax preparation services for hundreds of businesses in the Midwest. The firm is considering a significant expansion into accounting and financial services and is looking into possibly supporting this move by hiring IT subcontractors in several foreign countries. Turnkey's management has asked you to help them reach a decision. Specifically, they want you to cite the pros and cons of offshoring. You will present your views at a meeting of Turnkey managers next week. How will you proceed?

10.6 Software as a Service

The traditional software development and deployment model follows a straightforward approach in which software vendors develop application packages sold to customers. In this model, customers purchase licenses that give them the right to use the software under the specified terms of the license agreement. This model has been prevalent for many years and places the onus of installation, maintenance, and upgrades squarely on the shoulders of the customer. Once purchased, the software must be installed on the customer's servers

or computers. The responsibility for ensuring the software's smooth operation, including managing updates and security patches and addressing any technical issues, rests with the customer. This approach requires the customer to have specific IT expertise and resources to manage and maintain the software effectively. This can be a significant undertaking, especially for smaller businesses or those with limited IT capabilities.

In contrast, the SaaS model is increasingly supplementing and, in some cases, replacing the traditional software model. SaaS signifies a paradigm shift in the delivery and management of software. Unlike the traditional model, where the software is installed and runs on the customer's hardware, SaaS applications are hosted on the vendor's servers and accessed by the customer over the Internet. This remote hosting means the software doesn't reside on the customer's local machines but is accessed through a web browser or a dedicated application interface. This model effectively outsources much of the IT burden associated with software management, such as maintenance, updates, and security, to the vendor. It provides a more streamlined and hassle-free approach to using software, reducing the need for extensive in-house IT infrastructure and expertise.

The SaaS model aligns closely with the current **digital process management (DPM)** demands in business operations. DPM involves optimizing and automating business processes through digital technology, and SaaS applications provide the necessary tools and flexibility to achieve this. With features like cloud-based data storage, real-time data analytics, and integration capabilities with other digital platforms, SaaS applications enable businesses to streamline their processes, improve decision making, and enhance overall efficiency. The ability to access these applications from anywhere broadens the scope for remote collaboration and management, which are key components in effective digital process management. Therefore, the SaaS model not only simplifies software management but also actively supports the digital transformation goals of businesses, making it a strategic asset in today's fast-paced and increasingly digital business landscape.

10.6.1 Advantages

The SaaS model has emerged as a highly advantageous approach in modern software deployment, offering myriad benefits, particularly in reducing the burden of software maintenance and operations for customers. In the SaaS model, the software is hosted on the vendor's servers, which shifts the responsibility for maintenance, updates, and security to the vendor. This aspect is a significant relief for customers, as it eliminates the need for them to handle complex software installations and continual updates. Instead, the vendor takes care of these tasks, ensuring the software remains up to date with the latest features and security enhancements. This ongoing management by the vendor ensures that customers have continuous access to the most current version of the software and enhances the overall security and reliability of the application. The reduction in the need for in-house technical expertise and infrastructure for software maintenance frees up customer resources, allowing them to focus more on their core business activities.

Another key advantage of the SaaS model is the increased flexibility and accessibility it provides. Because the software is accessible over the Internet, users can use the service from virtually anywhere if they have an Internet connection. This accessibility is particularly beneficial in today's increasingly mobile and remote work environments, as it allows users to stay connected and productive regardless of their physical location. This feature enables businesses to support a more flexible work model, accommodating remote employees and facilitating collaboration across different regions and time zones.

The SaaS model typically operates on a subscription-based pricing structure and offers significant scalability and financial flexibility, making it a versatile and cost-effective solution for businesses of various sizes. This model allows firms to adjust their software usage up or down based on current needs, a feature particularly beneficial for organizations with fluctuating workloads or those undergoing rapid growth. Customers usually pay a recurring fee, either monthly or annually, which can be tailored to match their requirements and

budget. This flexibility in scaling and pricing is especially advantageous for small to medium-sized businesses that might not have the capital for sizeable upfront software investments or the resources to manage complex installations and maintenance. Moreover, the ability of SaaS providers to offer different service tiers enables customers to select a level of functionality that aligns with their business needs and to modify it as their business evolves, thereby enhancing operational efficiency and agility in a dynamic business environment.

10.6.2 Shortcomings

While the SaaS model offers numerous benefits, it also comes with certain shortcomings that businesses must consider. As discussed in Section 10.4.3, one of the primary concerns is the issue of data security and privacy. Because SaaS applications store data on external servers managed by the service provider, companies have less control over their data security. This external data hosting raises concerns about vulnerability to data breaches, unauthorized access, and potential misuse of sensitive information. Moreover, compliance with various data protection regulations can be more complex, as the data is stored offsite and often across multiple jurisdictions. This can be a significant deterrent to adopting a SaaS model, despite its other advantages, for businesses that deal with highly sensitive or confidential information.

Another limitation of SaaS is related to Internet dependency and potential performance issues. Because SaaS applications are accessed and operated over the Internet, they require a stable, high-speed Internet connection to function effectively. This dependency can be problematic in areas with unreliable Internet connectivity or during Internet outages, potentially leading to disruptions in business operations and productivity losses. Furthermore, performance can be affected by bandwidth limitations and server response times, which can vary depending on the service provider and current usage loads. These performance issues can be particularly challenging for businesses that require real-time data processing or have high transaction volumes.

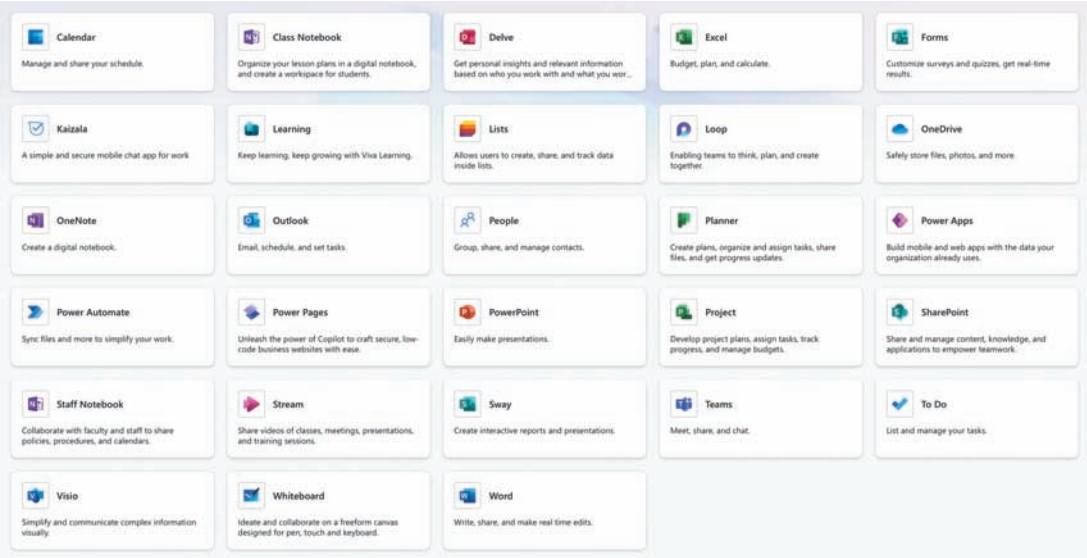
Customization and integration challenges also pose significant drawbacks in the SaaS model. While SaaS applications are generally user-friendly and designed for a broad user base, they may offer a different customization level than in-house software. This lack of customization can limit the ability of businesses to tailor the software to their specific needs and processes. Additionally, integrating SaaS applications with existing in-house systems or third-party applications can be complex and sometimes limited by the software's API capabilities or vendor restrictions. These integration challenges can hinder data flow across different business functions, potentially impacting operational efficiency and data consistency. As a result, businesses considering SaaS solutions must carefully evaluate these factors against their specific operating needs and IT infrastructure.

10.6.3 Example: Microsoft 365

In the fiercely competitive software solutions landscape, major vendors continuously push the envelope to deliver new and better offerings. This competitive drive is particularly evident in SaaS, where leading companies make bold claims about the superiority and business value of their platforms. Microsoft, for instance, positions its SaaS platform as offering the best solution in terms of functionality, integration capabilities, and overall business value. This claim is not just a marketing strategy but a reflection of the significant investments and advancements the company has made in its SaaS offerings.

One of Microsoft's more popular consumer-focused SaaS offerings is Microsoft 365, formerly Office 365, as shown in Figure 10-4. This service exemplifies the modern approach to software delivery, where traditional, locally installed office suites are transformed into cloud-based services. Microsoft 365 runs entirely in a browser window, offering a full-fledged version of the Microsoft Office suite, including well-known applications like Word, Excel, PowerPoint, and Outlook. This approach allows users to access and work with the applications from virtually anywhere, as long as they have an Internet connection.

Figure 10-4 Microsoft 365 provides web-based access to the complete Office suite.
Source: Microsoft Corporation



The convenience and flexibility offered by Microsoft 365 illustrate how SaaS models reshape the way individuals and businesses interact with software. By providing these applications through a browser, Microsoft ensures ease of access and use and guarantees that users always have the latest software versions, complete with up-to-date features and security enhancements. This transition from traditional software models to SaaS indicates how the software industry moves, driven by consumer demand for more accessible, flexible, and scalable software solutions.

Adding AI capabilities to Microsoft 365 through features like Co-Pilot represents a significant advancement in the SaaS context, pushing the boundaries of what cloud-based software can achieve. Co-Pilot transforms Microsoft 365 into an even more intelligent and interactive platform. This integration enhances the user experience by providing advanced features such as intelligent suggestions, automated data analysis, and predictive text, thereby increasing productivity and efficiency. In the context of SaaS, these AI-driven capabilities are particularly advantageous because they are seamlessly updated and improved upon in the cloud, ensuring that users always have access to the most advanced tools without needing manual upgrades or installations.

The continuous evolution of AI features in SaaS applications like Microsoft 365 demonstrates the potential for these technologies to significantly enhance the functionality and value of cloud-based software solutions, making them more adaptive, intuitive, and capable of efficiently handling complex tasks. This integration of AI into SaaS offerings not only caters to the growing demand for smarter, more efficient software solutions but also sets a new standard in the industry for what cloud-based SaaS applications can offer.

10.7 Data Management and Analytics

In addition to the system's software, data management and analytics considerations are integral to selecting a development strategy (see Section 10.8). As traditional and web-based systems evolve, they generate vast amounts of data that must be effectively managed and analyzed. Whether in a traditional setup where data might be more structured and stored within internal networks or in web-based systems that often handle diverse, large-scale data, robust data management and analytics are paramount. These systems require architectures that support the efficient gathering, storage, and processing of data, aligning with the varied development strategies chosen based on the system's requirements.

Integrating the latest trends in technology, such as AI and cloud computing, directly impacts data management and analytics strategies. ML algorithms rely heavily on large data sets to learn and make predictions, necessitating sophisticated data management solutions. IoT devices generate a continuous stream of data that must be captured and analyzed in real time or near real time, highlighting the need for advanced analytics and data processing capabilities. Cloud computing offers scalable and flexible data storage and analytics services, aligning with the cloud-based development and deployment strategies discussed earlier.

When organizations develop in-house software or customize software packages, they must consider how these systems will handle data. This includes designing databases, ensuring data integrity, and integrating application analytics capabilities. Even when purchasing off-the-shelf software, it's crucial to assess how these solutions will align with the organization's data strategy, especially for data storage, accessibility, and analytical capabilities.

Outsourcing and offshoring bring additional considerations. When outsourcing software development or related services, data handling and analytics capabilities often become critical factors in vendor selection. Organizations must ensure that vendors have the expertise to manage data securely and efficiently and provide the necessary analytics tools and services. In the context of offshoring, data privacy laws and regulations in different jurisdictions become crucial, especially when managing and analyzing data across borders.

SaaS solutions often come with built-in analytics tools and capabilities, providing organizations with powerful insights without requiring extensive in-house analytics infrastructure. The scalability of SaaS solutions also means that data storage and analytics capabilities can grow with the organization's needs. In this context, the systems analyst's role in selecting the right development strategy and performing cost–benefit analysis must consider the chosen solutions' data management and analytics capabilities, ensuring they align with the organization's overall data strategy.

10.8 Selecting a Development Strategy

Selecting the best development strategy is an important decision that requires companies to consider multiple factors. The systems analyst has a vital role to play in this decision-making process. Analyzing the costs and benefits of each development alternative is essential in providing objective data to management. A cost–benefit checklist can help guide this analysis.

10.8.1 The Systems Analyst's Role

During the systems development process, a company's pivotal decision is choosing the appropriate approach for software development, including acquisition (see Section 10.9). This decision can involve various options, such as outsourcing IT functions, developing software in-house, acquiring a commercial software package, creating user-specific applications, or a blend of these methods. The choice critically depends on the company's immediate and projected future needs, impacting subsequent stages of the SDLC and determining the extent of the systems analyst's involvement in the project. For instance, in-house software development typically requires substantial input and oversight from the systems analyst, whereas opting for outsourcing or a commercial package might necessitate less intensive involvement from them. This decision-making process usually involves company management, which considers detailed recommendations and presentations from the IT department, weighing the advantages and disadvantages of each option.

It is common for a single system within an organization to utilize a combination of software solutions to meet its diverse requirements. For example, a company may opt for a standard software package to manage functions like payroll while concurrently developing

custom software for specific needs, such as integrating the payroll system with an in-house manufacturing cost analysis system. This hybrid strategy allows the organization to take advantage of the reliability and support offered by commercial software while also addressing unique business requirements through custom-developed applications. Such an approach ensures that the company benefits from the best of both worlds—the efficiency and support of off-the-shelf software and the flexibility and specificity of tailored software solutions.

The evaluation and selection of these various software alternatives are complex and require careful consideration. One of the main objectives in this phase is to identify the option that offers the lowest TCO. However, forecasting each alternative's actual costs and performance can be challenging. Systems analysts typically work as an **evaluation and selection team** to navigate this process. This collaborative approach ensures that all critical aspects are thoroughly considered, leading to an informed and balanced decision. Including users in this team is also crucial, as their involvement ensures that the system meets their needs and helps foster a sense of ownership and acceptance of the final choice.

The fundamental goal of the evaluation and selection team is to methodically weed out any software alternatives that do not align with the organization's requirements. The process involves an in-depth assessment of each feasible option, ranking them in terms of suitability to the company's needs, and finally presenting the most promising options to management for the final decision. This begins with a detailed analysis of each alternative's costs and benefits. The evaluation is not limited to financial factors alone but also includes considerations like compatibility with existing systems, scalability, ease of use, and the level of support vendors provide.

By meticulously evaluating these factors, the team can narrow the options to those that best align with the company's operational requirements and strategic goals. This process is essential in ensuring that the selected software solution addresses the company's immediate needs and is viable and sustainable in the long term. It helps strike the right balance among cost, functionality, and future scalability, setting the foundation for a successful software implementation and supporting the company's growth and evolution.

10.8.2 Analyzing Costs and Benefits

Financial analysis tools have been pivotal in business and technology for centuries, evolving from devices like the abacus to modern digital calculators. These tools have always been integral to simplifying the process of working with numbers, a necessity that remains paramount in today's technology-driven world. In systems analysis, these tools are essential for conducting cost and benefit analyses of IT projects. They provide systems analysts with the means to objectively assess the financial viability of various technology initiatives, ensuring that investments are both prudent and aligned with business goals. This section delves into the specifics of cost and benefit analysis, highlighting popular financial tools that aid systems analysts in evaluating the potential and feasibility of IT projects.

The concept of economic feasibility, one of the four critical assessments conducted during the preliminary investigation of a systems request, heavily relies on financial analysis tools and techniques. These tools are crucial for evaluating different development strategies and making informed decisions about the progression of a project. Appendix B outlines three widely used financial analysis tools: **payback analysis**, **return on investment (ROI)**, and **net present value (NPV)**. Payback analysis is a method used to determine how long it takes for an information system to recoup its costs through reduced expenses and increased benefits. ROI, on the other hand, is a metric that calculates the percentage rate of return from a project by comparing its total net benefits to its total costs. NPV provides a comprehensive measure of a project's value, calculating the total worth of its benefits minus the costs, with both factors adjusted for their occurrence.

These tools, among others, are instrumental in determining the TCO of an IT project, a concept explored earlier in the book. At this analysis stage, the systems analyst identifies specific development strategies and decides on the most suitable course of action. For instance,

a company may discover that its TCO is higher for in-house system development than outsourcing the project or using an ASP. This financial analysis helps make strategic decisions that align with the company's financial constraints and goals, ensuring that the chosen development path offers the best value and ROI.

Gartner emphasizes the importance of accurate TCO forecasting. According to their research, nearly 80% of the costs associated with an IT project are incurred after the initial purchase of hardware and software. This insight highlights the need for IT departments to develop comprehensive TCO estimates that take into account all the costs that will be incurred throughout the life cycle of the project. IT departments can either develop these estimates internally or utilize TCO calculation tools provided by vendors. Either way, a thorough understanding of TCO is crucial for making informed decisions beyond the upfront costs of hardware and software, encompassing all related expenses incurred during the implementation and operation of the IT project.

Financial analysis tools such as payback analysis, ROI, and NPV play a crucial role in the systems development process, especially in determining the TCO of IT projects. These tools enable systems analysts to conduct comprehensive economic feasibility studies, ensuring that all financial aspects of a project are thoroughly evaluated. This rigorous financial scrutiny is essential for making informed decisions about development strategies and choosing the most cost-effective and beneficial course of action. The insights gained from these economic analyses are invaluable in guiding strategic decisions, helping companies to invest wisely in IT initiatives that deliver the greatest value and align with their long-term business objectives.

10.8.3 Cost–Benefit Analysis Checklist

Appendix B explains how to use the payback analysis tool during the preliminary investigation to help determine whether a project is economically feasible. All the financial analysis tools will be used to evaluate various development strategies. The best way to apply the tools is to develop a cost–benefit checklist.

List Each Development Strategy under Consideration

The first step in evaluating development strategies is to list each option comprehensively. This involves detailing all the potential paths for system development, such as in-house development, outsourcing, purchasing commercial off-the-shelf software, or utilizing a SaaS model. This step requires a broad overview of each strategy, considering the company's technical expertise, resource availability, and the project's specific requirements. Creating a detailed and inclusive list of all possible strategies ensures no viable option is overlooked and sets the stage for a thorough evaluation of each alternative.

Identify All Costs and Benefits

Once each development strategy has been listed, the next step is to identify all associated costs and benefits for each alternative. This task involves a detailed analysis of each option's immediate and long-term financial implications. Consider development expenses, implementation costs, ongoing maintenance, and potential training requirements. Benefits, on the other hand, might encompass efficiency gains, productivity improvements, and enhanced capabilities. Analysts must quantify these factors, determine when the costs will be incurred, and determine when the benefits are expected to be realized. This temporal aspect is vital to understanding the financial impact of each strategy over time.

Consider Future Growth

Considering future growth and the need for scalability is an essential aspect of evaluating development strategies. This step involves assessing how well each alternative can accommodate potential changes to a company's size, market conditions, and technological advancements. Scalability is critical, especially for businesses anticipating rapid growth or fluctuating demands. The chosen strategy should offer the flexibility to scale up or down as

needed without incurring prohibitive costs or operational disruptions. This forward-looking approach ensures that the selected development strategy remains viable and effective in the long term, even as the company evolves.

Include Support Costs for Hardware and Software

Including support costs for hardware and software is another crucial step in the evaluation process. This entails a comprehensive assessment of the expenses of maintaining and supporting the IT infrastructure required for each development strategy. These costs can vary significantly between different strategies and substantially impact a project's financial feasibility. Hardware support costs might include repairs, upgrades, and replacements, while software support costs could involve troubleshooting, updates, and technical assistance. Accurately estimating these costs is essential for understanding each development option's TCO.

Analyze Licensing Options

Analyzing various software licensing options forms an integral part of the evaluation process. This analysis should encompass different pricing models, such as fixed fees or variable costs, based on the number of users or transactions. Each licensing model has its advantages and disadvantages, and the choice depends on factors like the size of the user base, the frequency and volume of transactions, and the company's budget constraints. Understanding the financial implications of these different licensing options is crucial for determining which software solution aligns best with the company's operational needs and financial capabilities.

Apply Financial Tools

Applying financial analysis tools to each alternative is a critical step. Tools such as ROI, NPV, and TCO provide a quantitative assessment of each development strategy. These tools help compare each option's long-term value and cost-effectiveness, allowing for a more objective and data-driven evaluation. By applying these financial metrics, companies can gain a clearer understanding of the potential financial impact of each strategy, guiding them toward the most economically viable choice.

Prepare a Report for Management

The final step involves studying the results of the financial analyses and preparing a comprehensive report for management. This report should synthesize the findings from the previous steps, presenting a clear and concise comparison of the different development strategies. It should highlight the key costs and benefits, scalability considerations, support expenses, licensing options, and the outcomes of the financial analyses. The goal is to provide management with a detailed and informed overview of each option, enabling them to make a well-considered decision based on each development strategy's strategic and financial implications. This report is crucial for guiding the company's decision-making process, ensuring that the chosen strategy aligns with its current needs and future growth objectives.

Case in Point 10.3

Sterling Associates

Sterling Associates specializes in advising clients on IT projects and information systems development. Marketing is creating a brochure for prospective new clients, and they want you to develop a section that describes payback analysis, ROI, and NPV in simple terms and mentions the pros and cons of each financial analysis tool. How do you proceed?

10.9 Software Acquisition

Software acquisition refers to the process by which organizations obtain software to meet their specific operational needs and strategic objectives. As described earlier in this chapter, the process can take various forms, including in-house software development, where a company uses its resources to create custom software tailored to its unique requirements; outsourcing, which involves hiring external firms to develop software; offshoring, a type of outsourcing that utilizes overseas companies to take advantage of lower costs; and adopting SaaS, where businesses subscribe to cloud-based software provided by external vendors. Software acquisition is so important to large organizations that Carnegie Mellon University's Software Engineering Institute (CMU/SEI) developed the Software Acquisition Capability Maturity Model (SA-CMM) to provide a structured framework for improving and evaluating software acquisition processes.

Each software acquisition method offers unique benefits and poses specific challenges, including costs, the degree of control over the development process, the level of expertise required, and the time it takes to bring the software to market. For instance, in-house development provides greater control but often at a higher cost and longer development time. In comparison, outsourcing can be cost-effective and quicker but may offer less control and require rigorous vendor management. In addition, the compatibility of the chosen software approach with the organization's existing data management and analytics capabilities is crucial. The selected software should not only meet the immediate functional requirements of the business but also be scalable and adaptable to support its evolving data needs and long-term strategic objectives. This alignment ensures that the software remains a valuable asset to the organization, capable of adapting to future changes in the business environment and technological advancements.

While unique to each organization, the software acquisition process typically involves a series of fundamental steps that guide decision makers through the complexities of selecting the right software solution. These steps start with a thorough analysis of system requirements, where the organization defines what it needs regarding functionality, scalability, and compatibility with existing systems. The next stage is identifying potential vendors or outsourcing options, which involves researching and shortlisting providers who meet these requirements. The process then moves to evaluating these alternatives, where each option is assessed for its merits and drawbacks, often through a detailed cost–benefit analysis. This analysis not only considers the financial implications but also weighs the operational impact of each option. Finally, all these insights culminate in preparing a well-structured recommendation for management, providing a clear and comprehensive understanding of the available options and guiding them toward a decision that aligns with the organization's strategic goals and operational needs.

10.9.1 Analyzing System Requirements

Analyzing system requirements is foundational in identifying key features necessary for a system's functionality and effectiveness. The requirements are derived from data gathered during the fact-finding stage and were compiled into the system requirements document in the SDLC's analysis phase. This document serves as a comprehensive guide for the development team, whether internal or external, ensuring that the developed system meets all identified needs.

This step thoroughly examines network and web-related issues to ensure the system integrates seamlessly with the existing IT infrastructure, addressing aspects such as compatibility and interoperability. Additionally, network and web-related considerations play a significant role in this phase. The analyst must determine the system's operating environment, whether a network, the Internet, or a company intranet, and incorporate these requirements into the system design.

It's also crucial to consider the system's capacity to exchange data with external systems, such as those of vendors or customers, ensuring compatibility and seamless data flow. Any existing hardware, software, or personnel constraints must be clearly defined to align the proposed system with the organization's capabilities and limitations. For example, Table 10-2 shows volume estimates for an order processing system. In addition to current levels, the table displays two forecasts: one is based on the existing order processing procedures, and another assumes a new website is operational.

Table 10-2 Volume estimates for an order processing system showing current activity levels and two forecasts: one based on the existing order processing procedures and another that assumes a new website is operational.

Online Order Processing System Estimated Activity During Next 12-Month Period			
	Current Level	Future Growth (based on existing procedures)	Future Growth (assuming new website is operational)
Customers	36,500	40,150	63,875
Daily Orders	1,435	1,579	2,811
Daily Order Lines	7,715	7,893	12,556
Sales Reps	29	32	12
Order Processing Support Staff	2	4	3
Products	600	650	900

A comparison of the two forecasts shows that the website will generate more new customers, process almost 80% more orders, and substantially reduce the need for sales reps and support staff. If in-house development is being considered, ensure that the software and hardware can handle future transaction volumes and data storage requirements. Conversely, volume and usage data are essential to analyze ASP fee structures and develop cost estimates for outsourcing options if outsourcing is being considered.

This comprehensive analysis then forms the basis for preparing a detailed **request for proposal (RFP)** or **request for quotation (RFQ)**, a formal invitation for vendors to submit their proposals, encompassing all the factors discussed in this section. The two documents are similar but used in different situations, depending on whether a specific software product has been selected. Evaluation models, often developed in spreadsheet programs, can then be used to compare vendor responses using a common yardstick to measure and assess each vendor's offerings, ultimately aiding in a well-informed selection decision.

Using RFPs

An RFP is an essential document in the procurement process for IT services and products. It provides a detailed description of the company and a clear statement of its specific requirements, including a list of the IT services or products required and the particular features each must possess. To aid in this process, RFPs often utilize standardized templates found through various online sources; they offer a structured and organized format that simplifies the proposal evaluation process by allowing for easier comparison of different vendors' responses. This standardization ensures that potential vendors effectively communicate and meet the organization's business needs.

The RFP outlines necessary services and support levels, giving vendors a comprehensive understanding of what the organization seeks and enabling them to assess their ability to meet these needs. Consequently, using an RFP ensures a transparent, competitive procurement process, leading to selecting the most suitable vendor that aligns with the organization's specific IT needs. This strategic use of an RFP in the acquisition process sets the stage for a focused and effective bidding process. It is fundamental in facilitating the selection of appropriate vendors, increasing the likelihood of a successful match between the organization's requirements and the vendor's offerings.

The structure and content of an RFP can vary greatly, reflecting the diversity and complexity of the systems it seeks to procure. An RFP may encompass numerous pages detailing various requirements and features for large and intricate systems. Within the RFP, certain features are designated as essential, signifying their critical role in the system's functionality. In contrast, others may be identified as desirable, providing vendors with insights into potential value-adding features. Additionally, the RFP stipulates specific, detailed pricing and payment terms, setting clear financial expectations for the proposed engagement. This comprehensive approach in the RFP ensures that all aspects of the procurement are thoroughly addressed, paving the way for a well-informed and prudent selection decision.

Using RFQs

An RFQ is distinct from an RFP in its level of specificity and focus. In the case of an RFQ, the organization already clearly understands the specific product or service it requires; the primary goal is to obtain price quotations or bids from potential suppliers. This approach is beneficial when the requirements are straightforward and well-defined, allowing for direct comparison of vendor offerings based solely on price and the terms of service. RFQs are commonly used for outright purchases and various leasing options, giving organizations the flexibility to choose the best financial arrangement for their needs. They can also specify maintenance or technical support requirements, ensuring these essential services are included in the vendors' proposals. In addition, many vendors provide preformatted RFQ (and RFP) templates on their websites, simplifying the process for organizations and ensuring that all necessary information is captured. The primary aim of an RFQ is to gather precise, comparable, and responsive bids from vendors, enabling organizations to make well-informed decisions based on cost and service terms.

An RFQ is particularly advantageous when the organization seeks to streamline the procurement process by focusing exclusively on pricing and service terms. By eliminating the need for detailed descriptions of functionalities and features, which are already known and specified, the RFQ process tends to be faster and more straightforward than the RFP process. This efficiency is beneficial when time is critical or when the products or services needed are standard in the industry and do not require extensive customization or explanation. The clarity and directness of RFQs make them ideal tools for organizations looking to quickly and effectively compare vendors' pricing and service offers, leading to a more efficient and focused selection process.

Despite their differences, RFPs and RFQs share a common objective: to solicit clear, comparable, and responsive vendor information, facilitating well-informed decision making. While RFPs are comprehensive and focus on the broader scope of a project, including detailed requirements and specifications, RFQs are more concise and centered primarily on pricing and straightforward service terms. Whether an organization chooses to use an RFP or an RFQ depends on the complexity of its needs and the nature of the product or service being procured. In either case, these documents are vital in ensuring that the procurement process is transparent, competitive, and results in the best possible outcome for the organization.

Using Evaluation Models

When an organization receives multiple responses to an RFP or RFQ, employing an **evaluation model** becomes critical in decision making. This model serves as a standardized framework or "common yardstick" to assess and compare the proposals or bids received from various

vendors. Using a consistent set of criteria for evaluation, the model ensures that each vendor's offer is assessed fairly and objectively. This process involves rating each vendor on various aspects such as cost, technical capabilities, experience, service terms, and compliance with the requirements specified in the RFP or RFQ.

Using an evaluation model is particularly important in complex projects or procurements, where proposals may vary significantly in content, scope, and pricing. This model standardizes the assessment process by establishing a uniform set of criteria to evaluate each proposal, ensuring a fair and objective comparison. It enables decision makers to analyze factors such as cost-effectiveness, technical feasibility, vendor reputation, and compliance with project specifications. This structured approach simplifies identifying the proposal that offers the best value and aligns closely with the organization's needs. Implementing such a model streamlines the selection process, which is critical in high-stakes projects where choosing the right vendor is essential to success.

Evaluation models are not just limited to the procurement phase; they are valuable tools that can be utilized throughout the SDLC. Building these models is often done using spreadsheet programs, which offer the capability to create comprehensive evaluation criteria and apply different weighting factors to these criteria. This flexibility is crucial, as it allows decision makers to prioritize certain aspects over others based on the project's specific needs or strategic objectives. For example, cost may be weighted more heavily in some cases, while technical capability or vendor reliability might be of greater importance in others.

Using spreadsheet programs also enables the visual representation of evaluation results, such as through graphs and charts. This visual aspect can be particularly helpful in presenting the findings to stakeholders or team discussions, as it provides a clear and concise overview of how each vendor scored against the evaluation criteria. Graphical representations can make it easier to identify strengths and weaknesses in each proposal at a glance, facilitating more informed and transparent discussions and decision making. Furthermore, the ability to experiment with different weighting factors and instantly see the impact on vendor ratings adds a dynamic aspect to the evaluation process. It allows decision makers to explore various scenarios and understand how changes in priorities or criteria weightings could affect the final decision. This level of analysis is essential in ensuring that the chosen vendor meets the current requirements and is the best fit for the organization's long-term goals and strategies.

For example, Table 10-3 shows an unweighted evaluation model for a network project. The evaluation lists the key elements of the project and each vendor's score. In the example, Vendor A has the highest total points. The model in Table 10-4 adds a weight factor. In this model, Vendor C is the winner. No standard method exists for assigning the weight factors. Each firm will have its own approach, which might be tailored to fit a specific situation. An analyst usually obtains as much input as possible and then circulates proposed values for further comment to achieve a consensus.

Table 10-3 Vendor A has the most total points in this unweighted model. Each vendor was rated on a scale from 1 (lowest) to 10 in several categories, and then the scores were added to calculate total points.

Unweighted Evaluation Model for a Network Project			
	Vendor A	**Vendor B**	**Vendor C**
Price	6	5	9
Completion Date	2	5	8
Layout/Design	8	8	5
References	10	6	3
TOTAL POINTS	26	24	25

Table 10-4 Vendor C has the most total points in this weighted model. Each vendor was rated on a scale from 1 (lowest) to 10 in several categories; scores were then multiplied by a weight factor and added to calculate total points.

Weighted Evaluation Model for a Network Project				
	Weight Factor	Vendor A	Vendor B	Vendor C
Price	25	6 * 25 = 150	5 * 25 = 125	9 * 25 = 225
Completion Date	25	2 * 25 = 50	5 * 25 = 125	8 * 25 = 200
Layout/Design	35	8 * 35 = 280	8 * 35 = 280	5 * 35 = 175
References	15	10 * 15 = 150	6 * 15 = 90	3 * 15 = 45
TOTAL POINTS	100	630	620	645

10.9.2 Identifying Potential Vendors or Outsourcing Options

Identifying potential vendors or outsourcing providers is a crucial step in software acquisition. The Internet is a primary marketplace for all IT products and services, and descriptive information can be found on the web about all significant products and acquisition alternatives. Businesses can access detailed descriptions and reviews of major products and acquisition alternatives online, providing a comprehensive overview of available options. This online research is particularly useful for gathering initial data and forming a list of potential vendors.

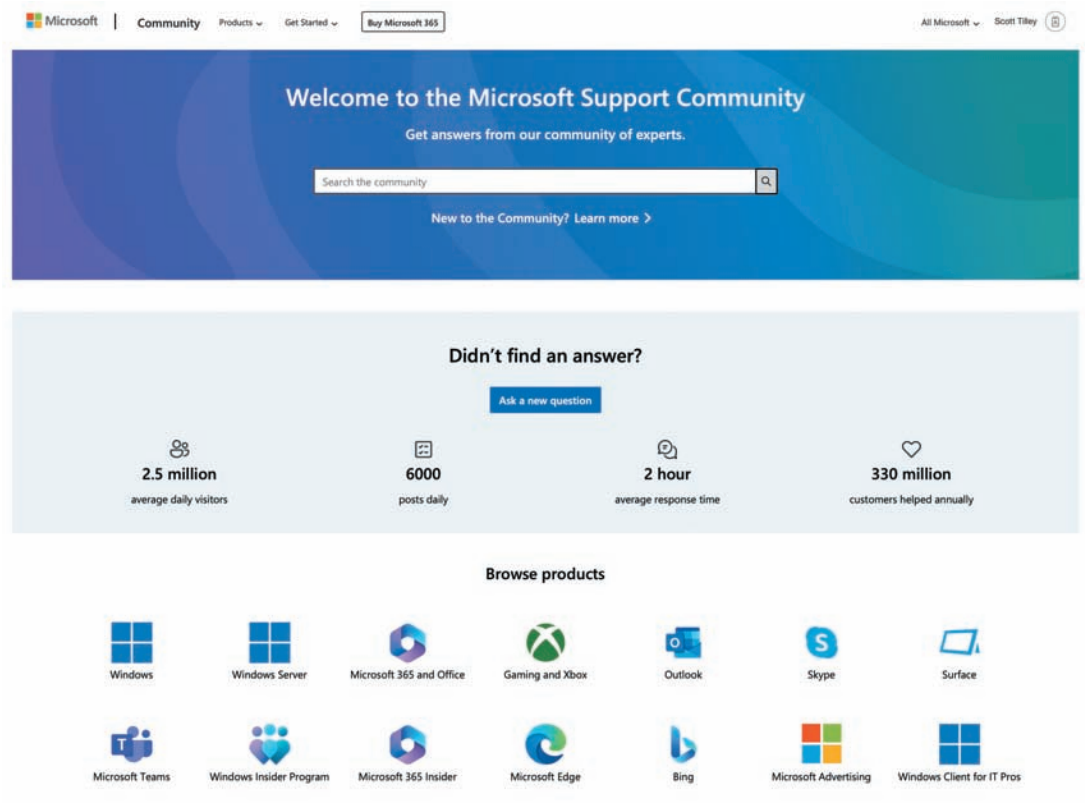
Resources like industry trade journals or websites are invaluable for specialized needs, such as vertical applications tailored to specific industries. These platforms often feature reviews and insights into specific software solutions, giving businesses a clearer understanding of the options that align with their unique industry requirements. Additionally, industry trade groups can be a source of referrals, pointing businesses toward companies that offer the specialized software solutions they need.

Online **forums** are invaluable for anyone seeking information, support, or a community of like-minded individuals, regardless of the topic of interest. These forums exist in various forms, from private platforms hosted by specific entities to public spaces within larger online communities like Google Groups or Reddit. In these expansive digital environments, users can join groups aligned with their interests or create new ones to gather a niche audience. To find relevant forums, a simple web search can be effective. It can lead not only to specific forums but also to the websites of companies like Microsoft, which often host dedicated discussion boards.

For IT professionals, forums offer a treasure trove of information and networking opportunities. These platforms are not limited to text-based discussions; they often encompass various resources such as blogs, webcasts, and other multimedia content. This variety ensures that professionals find information in the format that best suits their learning style and needs. Engaging in these forums allows IT professionals to stay updated with the latest industry trends, gain insights from peers and experts, and even seek solutions to specific technical challenges. As illustrated in Figure 10-5, these online spaces are dynamic, interactive communities that foster learning, collaboration, and professional growth.

Another practical approach in the vendor selection process is to collaborate with a consulting firm. Many IT consultants specialize in assisting companies with software package selection, offering expert advice and insights. The significant advantage of working with a consultant lies in their broad experience and expertise, which can be difficult for a single company to match internally. These consultants bring knowledge about software solutions, industry trends, and vendor performance, providing a more

Figure 10-5 Microsoft Communities is a valuable resource for IT professionals.
Source: Microsoft Corporation



nuanced and informed perspective. Consultants can be located through professional organizations, industry contacts, or online searches. Although engaging a consultant incurs additional expenses, their expertise can be instrumental in avoiding costly mistakes, guiding businesses to make well-informed decisions that align with their strategic objectives and operational needs.

In addition to these methods, it's important to incorporate **vendor risk assessments (VRAs)** and **third-party risk management (TPRM)** into the selection process. VRAs and TPRM ensure a defined, systematic approach for evaluating potential vendors, focusing on security, compliance, financial stability, and reputation. This risk assessment is crucial in today's environment, where businesses increasingly rely on external providers for critical IT services and solutions. By thoroughly assessing potential risks associated with each vendor, companies can mitigate potential issues down the line, ensuring that the chosen vendor meets their current requirements and is a reliable and secure partner for the future.

This comprehensive approach to vendor selection integrates various crucial elements to form a robust framework, ensuring that organizations can effectively identify and choose the most suitable vendors or outsourcing providers for their IT needs. By combining online research, organizations can gather extensive information on potential vendors, including their product offerings, customer reviews, and market reputation. Industry resources, such as trade journals and professional networks, offer insights into specialized vendors and emerging trends. At the same time, consultants bring a wealth of expertise and experience to guide the selection process. Furthermore, incorporating risk assessments, including VRAs and TPRM, adds an essential layer of due diligence, ensuring that selected vendors meet technical and operational requirements and adhere to security, compliance, and reliability standards. This holistic approach maximizes the likelihood of a successful partnership with vendors, aligning with the organization's strategic goals and IT requirements.

10.9.3 Evaluating the Alternatives

After identifying the alternatives, the analyst must select the one that best fits the company's needs. Information about the options should be obtained from as many sources as possible, including vendor presentations and literature, product documentation, trade publications, and companies that perform software testing and evaluation. Analysts can search the Internet using keywords that describe the application to learn more about software packages. Websites maintained by consultants and software publishers often include product references and vendor links. As part of the evaluation process, information from existing users should be obtained, the application should be tested, and the package should be benchmarked.

Existing Users

Reaching out to users of existing software packages, ASPs, or vendor services is a valuable step in the evaluation process, as it provides real-world insights and feedback about their experiences. For larger software packages, ASPs and vendors often offer a list of user references, which can be a rich source of information about the software's performance and suitability. These user references are particularly crucial, as they offer firsthand accounts of how well the software has functioned in environments like that of the prospective buyer's company.

However, it's important to approach this feedback with a discerning mind, as vendors tend to curate their reference lists to include predominantly satisfied clients, leading to a bias toward positive feedback. Therefore, while user references can be extremely informative, they should be considered part of a broader evaluation process, supplemented by independent reviews and thorough analysis to ensure a well-rounded and unbiased understanding of the software's capabilities and performance in various settings.

Application Testing

If considering a software package as part of the development options, it's beneficial to determine whether users within the organization can test the product firsthand. Access to a demo or trial version of the software allows for a practical evaluation of its features and suitability for the organization's needs. This hands-on experience is particularly useful for horizontal applications or smaller systems, where entering a few sample transactions can provide a quick but insightful look into the software's functionality, user interface, and performance. Testing a demo version helps users and decision makers gauge how well the software aligns with their workflows and whether it meets their expectations for ease of use and efficiency. This initial testing phase can be a decisive factor in whether to proceed with a particular software package, as it offers a glimpse into the potential benefits and limitations of the software in a real-world context.

A comprehensive testing approach is required for specialized vertical applications or larger, more complex systems. In such cases, assembling a team of IT staff and end users for an in-depth evaluation is essential. This team may need several days or weeks to thoroughly test the software, explore its various features, assess its scalability, and ensure its compatibility with existing systems and processes. This extended testing period allows the team to simulate real operational scenarios and workflows, providing a deeper understanding of how the software will perform under actual working conditions. This thorough evaluation is crucial for large systems, as it helps uncover potential issues or limitations that might not be apparent during a brief demo. It also enables the team to provide detailed feedback and recommendations, ensuring that the final decision on the software package is well-informed and considers the diverse needs and perspectives of all users within the organization.

Benchmarking

Benchmarking is a vital evaluation tool in the software selection process, offering a quantifiable measure of a software package's performance by timing its processing of a specific number of transactions. For instance, a **benchmark** test might determine the time it takes for a software package to post 1,000 sales transactions. This quantitative approach provides a clear metric to assess the efficiency and speed of the software under controlled test

conditions. However, it's essential to recognize that these tests are conducted in controlled environments that may not fully replicate the everyday operational conditions of the company undertaking the project. Therefore, while benchmarking offers valuable insights into the relative performance of different software packages, it may not entirely predict real-world, project-specific outcomes. The results of benchmark tests should be interpreted with an understanding of their limitations and the context in which they were conducted.

Numerous IT publications play a significant role in the software evaluation process by regularly publishing reviews of individual software packages, which often include the results of benchmark tests. These reviews are widely available in traditional print formats and increasingly through online and mobile platforms. The digital versions of these publications typically offer enhanced features, including search capabilities and links to relevant IT resources, making it easier for readers to access and navigate the information. These reviews, particularly annual surveys covering various software categories, are valuable for organizations seeking detailed insights into different software options' performance, features, and reliability.

In addition to these publications, some firms specialize in benchmarking various software packages and analyzing industry trends. These firms provide in-depth reports and analyses that can be crucial for organizations in selecting software. Their expertise in evaluating and comparing software packages across different parameters offers a more comprehensive understanding of how these packages perform in test conditions and practical, real-world scenarios. By combining the insights from benchmark tests, expert reviews, and industry analyses, businesses are equipped with a thorough perspective, enabling them to make more informed and effective decisions for software acquisition.

10.9.4 Performing Cost–Benefit Analysis

To effectively assess the TCO for each development option under consideration, utilize the guidelines and tools provided in this chapter and Appendix B to develop a comprehensive spreadsheet. This spreadsheet should meticulously account for all associated costs, incorporating the volume forecasts previously prepared. For those considering outsourcing, it's crucial to delve into the fee structure models mentioned earlier and understand how they might influence the overall cost.

Enhance this analytical tool by incorporating graphical charts that visually represent the results. Additionally, build in a "what-if" analysis feature, which allows you to modify variables and instantly gauge the potential impact on the TCO. This capability is invaluable for exploring various scenarios and understanding how changes in factors such as volume, costs, or service requirements might affect the overall investment.

When evaluating the acquisition of a software package, it's important to thoroughly explore the various options available. When software is purchased, a **software license** specifies how and where the software can be used, with terms varying from single-computer use to network-wide access. Understanding the nuances of these licensing agreements is crucial, as they often include restrictions on software distribution and modifications. While license terms for desktop applications are usually non-negotiable, there may be more room for negotiation in licenses for large-scale systems. This flexibility can be important for organizations requiring specific terms to meet their operational needs.

In addition to the upfront costs of software acquisition, it's vital to consider user support issues, as they can constitute a significant portion of the TCO. The arrangement typically includes technical support and maintenance services if an outsourcing solution is chosen. However, if in-house development is recommended, the internal costs of providing these support services must be factored in.

A supplemental **maintenance agreement** might also be considered when purchasing a software package. Such contracts can offer additional support and assistance from the vendor, sometimes starting with comprehensive backing and transitioning to specific charges for certain services. It's also common for software packages to include a period of complimentary

technical support, after which support becomes a paid service based on usage. Some vendors proactively notify registered users about new releases, offering these updates at a reduced price. Understanding and planning for these ongoing support and maintenance costs are essential for accurately estimating the TCO and ensuring the long-term sustainability of the chosen development strategy.

10.9.5 Preparing a Recommendation

As outlined in Section 10.8.3, when analyzing the cost–benefit aspects of different development strategies, the systems analyst should prepare a comprehensive acquisition recommendation report. This report should meticulously evaluate and describe each acquisition alternative under consideration. The report needs to delve into the details of each option, presenting a precise analysis of their respective advantages and disadvantages, facilitating a balanced view for decision makers. Preparing a well-structured presentation may also be required, particularly when the findings and recommendations must be communicated to stakeholders or decision-making authorities. These documents and presentations aim to provide a clear, comprehensive, and well-reasoned basis for selecting the most appropriate software acquisition path.

When preparing these recommendations, it's crucial to consult the guidelines provided in Appendix A, especially those for the presentation of written and oral proposals. A well-crafted written report should be clear, concise, and well-organized, making it easy for readers to understand the key points and follow the logic behind each recommendation. It should effectively use visuals like charts or graphs to illustrate complex data or comparisons between different options. For the oral presentation, the analyst should focus on clear communication, emphasizing the most critical aspects of the report and engaging the audience to ensure the message is conveyed effectively. This might involve rehearsing the presentation, anticipating potential questions from stakeholders, and being prepared to provide additional information or clarification. The ability to effectively present written and oral information is crucial in persuading and influencing decision makers.

These recommendations guide the organization toward the most beneficial and cost-effective software acquisition decision. By thoroughly evaluating each option and presenting the findings clearly and persuasively, the analyst can steer the organization toward a solution that aligns with its strategic goals, budget constraints, and operational requirements. These recommendations help make an informed choice and ensure all stakeholders are aligned and clearly understand the rationale behind the selected option. This process, therefore, is not just about the technical aspects of software acquisition and development strategy but also involves effective communication and stakeholder management, which are critical for successfully implementing the chosen solution.

Summary

Traditional development is characterized by compatibility with existing hardware and software, with systems designed for local and wide area networks without a primary focus on web-based features. In contrast, web-based development treats the Internet as the leading platform, emphasizing scalability, cross-hardware functionality, and integrating web-based features as core elements. The choice between these development strategies depends on organizational needs, system requirements, and the intended user base.

The evolving trends in systems development include the integration of AI and ML for more intelligent, adaptive systems; the widespread adoption of cloud computing for scalable and flexible infrastructure; IoT and edge computing to enhance device connectivity and data processing; the increasing relevance of mobile devices in accessing and interacting with systems; and the emergence of Web 2.0 and 3.0, focusing on user-generated content and social interaction.

In-house development involves decisions on whether to make or buy software, develop software in-house, purchase off-the-shelf software packages, customize existing software packages, or enable end users to create applications. These choices hinge on cost, expertise availability, time constraints, and specific business requirements. Each option offers different levels of customization, control, and investment, influencing the organization's approach to systems development.

Outsourcing, the practice of hiring third-party vendors for software development, grew due to its cost-effectiveness, access to global talent, and focus on core business functions. Various fee structures are possible for outsourcing, including fixed price, time and materials, and outcome-based models. Potential issues and concerns include quality control, data security, and dependency on external entities.

Offshoring is a specific type of outsourcing in which services are relocated to another country, often for cost savings. Offshoring has numerous advantages and challenges, such as lower labor costs, time zone differences, cultural barriers, and communication challenges.

SaaS is a modern approach to software distribution in which applications are hosted remotely and accessed over the Internet. This model emphasizes subscription-based pricing and cloud-based delivery, offering scalability, ease of use, and reduced dependency on in-house IT infrastructure.

Data management and analytics are vital in a software system's development strategy. They ensure efficient handling, storage, and interpretation of vast amounts of data, which are crucial for informed decision making and optimizing system performance. Incorporating these elements from the outset allows for the design of scalable, adaptable systems that can leverage data-driven insights to enhance user experience and operational efficiency. Furthermore, robust data management and analytics enable compliance with data governance standards and provide a competitive edge by transforming raw data into actionable business intelligence.

The systems analyst's role in selecting an appropriate development strategy is critical. It emphasizes the importance of analyzing costs and benefits, including conducting cost–benefit analyses to ascertain the feasibility and profitability of different development approaches. This analysis helps companies make informed decisions aligned with organizational goals and resources.

The software acquisition process involves evaluating system requirements, identifying potential vendors or outsourcing options, evaluating alternatives, performing a cost–benefit analysis, and preparing a recommendation. This systematic approach ensures that the selected software aligns with organizational needs and offers the best value for the investment.

Key Terms

application service provider (ASP)
benchmark
build or buy
business process outsourcing (BPO)
cloud computing
digital process management (DPM)
evaluation and selection team
evaluation model
fixed fee model
forum
General Data Protection Regulation (GDPR)
global outsourcing
horizontal application
in-house software
Internet business services (IBS)
knowledge process outsourcing (KPO)
legal process outsourcing (LPO)
maintenance agreement
make or buy
managed hosting
mobile devices
net present value (NPV)
offshore outsourcing
offshoring
onshoring
outsourcing
payback analysis
request for proposal (RFP)
request for quotation (RFQ)
reshoring
return on investment (ROI)
service desk
service provider
software as a service (SaaS)
software license
software package
software vendor
subscription model
third-party risk management (TPRM)
transaction model
usage model
user application
value-added reseller (VAR)
vendor risk assessments (VRAs)
vertical application
Web 2.0
Web 3.0
work from home (WFH)

Review Questions

1. List three characteristics of traditional and web-based development.
2. How does cloud computing support Web 2.0 applications?
3. Why would a company choose in-house software development?
4. What is outsourcing?
5. List two reasons offshoring may be risky.
6. What is software as a service?
7. Why are data management and analytics integral to selecting a development strategy?
8. What is the primary objective of the evaluation and selection team in selecting a development strategy?
9. What are the steps of the software acquisition process?
10. What is an RFP, and how does it differ from an RFQ?

Discussion Topics

1. How has the proliferation of mobile devices affected IT professionals?
2. As more companies outsource systems development, will there be less need for in-house systems analysts? Why or why not?
3. Select a financial software-as-a-service application and describe its main features.
4. Suppose you tried to explain the concept of weighted evaluation models to a manager who responded, "So, how do you set the weight factors? Is it just a subjective guess?" How would you reply?
5. How does an organization transition from traditional in-house software development to SaaS?

Hands-On Projects

1. Investigate the ROI of cloud-based software development environments.
2. Many financial tools are developed using Microsoft Excel. Identify three applications built using Excel and describe how they are used in an organization.
3. Visit your school's IT department or a local company and determine whether the systems were developed in-house or purchased. If packages were acquired, find out what customizing was done, if any. Write a brief memo describing the results.
4. Profile three companies that provide offshore software development services.
5. Various firms and organizations offer IT benchmarking. Locate an example on the Internet and describe its services.

Ethical Issues

The department manager has asked a junior analyst at a medium-sized IT consulting firm to draft a response to an RFP from a large company seeking IT consulting services in connection with a new accounting system.

The analyst working on the RFP noticed a specific question about the firm's recent experience with this type of system. To the best of the analyst's knowledge, the firm has only worked on one other accounting project in the last three years. After reading the analyst's draft response, the manager was upset about the reply to the question. "You don't have to be quite that candid. Even though we only had one formal project, we have several people who worked on accounting systems before they came here."

"Yes," the analyst replied, "but that isn't what the question asks." Still unhappy, the manager said, "If we want that job, we'll have to come up with a better answer." Thinking about it, the analyst isn't comfortable providing anything but a straight answer. Is this an ethical question? What are the analyst's options?

Chapter

11

Managing Systems Implementation

Learning Objectives

When you finish this chapter, you should be able to:

1. Define the concepts of quality assurance in software and systems engineering.
2. Describe the differences between traditional and agile methods in application development tasks.
3. Understand the principles of cohesion and coupling in structured development.
4. Recall the characteristics of object-oriented development.
5. Apply agile development.
6. Explain coding.
7. Explain unit, integration, and system testing.
8. Differentiate among program, system, operations, and user documentation.
9. Describe the five tasks involved in system installation.

Chapter 11 is the second chapter in the systems implementation phase of the SDLC. The first step in this phase focused on evaluating alternative development strategies, assessing various development options, and considering data management and analytics issues. The second step concerns managing the implementation of the system. This involves paying constant attention to quality assurance throughout application development, coding, testing, documentation, and installation.

The chapter includes three "Case in Point" discussion questions to help contextualize the concepts described in the text. The scenario described in the "Ethical Issues" section concerns an issue that all testers must deal with: when to stop testing. An ethical question is raised in the example about whether a 90% pass rate is sufficient to ship a product, assuming the remaining bugs will be fixed once the system is operational.

11.1 Quality Assurance

In the modern, highly competitive business landscape, the emphasis on product and service quality has become paramount for companies striving for success. This focus extends beyond the traditional realms of manufacturing or service delivery, profoundly impacting the domain of information systems. For an organization to thrive, its information technology resources must meet the highest quality standards. This necessitates a comprehensive approach to quality management, encompassing everything from software functionality and user interface design to data security and system reliability. The role of top management in this process is indispensable. They must advocate for high-quality IT resources and actively foster a culture that prioritizes and values quality. This involves providing necessary resources, setting clear quality benchmarks, and encouraging continuous improvement in all facets of the IT infrastructure.

However, achieving such high standards of quality has its challenges. No matter the level of diligence in the design and implementation of systems, the potential for problems always exists. These issues can be wide-ranging, from minor glitches to major system failures. Rigorous testing is critical in identifying and addressing these issues during implementation.

An even more effective strategy is identifying and correcting mistakes early in development. This proactive approach is more cost-effective, as rectifying errors after deployment can be significantly more expensive, but it also ensures a smoother and more reliable system operation after the launch. **Quality assurance** is, therefore, an ongoing process that starts from the initial stages of systems development and continues throughout its life cycle.

The primary objective of quality assurance in IT is to prevent problems from occurring in the first place or to identify them as early as possible when they do occur. This goal is achieved through a series of meticulously planned and executed steps. These steps include thorough requirements analysis to ensure the system meets its designed needs, careful design to prevent architectural and interface problems, and diligent **coding** practices to avoid errors. Additionally, quality assurance involves creating clear and accurate documentation, essential for system implementation and maintenance, and conducting effective testing to uncover and rectify any faults before the system goes live. Each step is critical, and their collective execution ensures the final product is of the highest quality.

Quality assurance is not solely about technical accuracy and system functionality. Poor quality can often be traced back to more fundamental issues, such as inaccurate or incomplete requirements, design flaws, coding mistakes, inadequate documentation, or insufficient testing procedures. These problems can impact the system's performance, user satisfaction, and operational efficiency. Organizations must adopt a holistic view of quality encompassing all systems development and operation aspects. This approach should involve all stakeholders, including developers, managers, and end users, to ensure the system functions as intended and aligns with the broader business objectives and user needs. By doing so, organizations can ensure that their IT resources are technically sound, strategically aligned, and user-centric, contributing to overall business success.

Toward this end, systems analysts should consider best practices in software engineering, systems engineering, and internationally recognized quality standards.

11.1.1 Software Engineering

Software engineering is a structured and systematic approach to developing software, employing engineering principles to construct complex and enduring applications. This discipline is not limited to coding; it encompasses a comprehensive process that involves careful planning, design, and maintenance. At its core, software engineering integrates three key elements: people, processes, and technology. The human aspect includes the collaboration of diverse

teams, each bringing their expertise and perspective to the project. Processes involve methodologies and practices that guide the SDLC, ensuring efficiency and quality. Finally, technology refers to the tools, languages, and platforms used to build the software. This blend of elements ensures that software engineering is a holistic and adaptive field capable of responding to evolving technological challenges and user needs.

The technical aspects of software engineering are divided into five primary activity areas, each playing a crucial role in the software's life cycle. These areas are:

- Requirements, where the functions of the software are identified and documented
- Design, which involves planning the software's architecture and user interfaces
- Construction, the actual coding and building of the software
- Testing, where the software is rigorously evaluated to ensure it meets quality standards and functions as intended
- Maintenance and evolution, which covers the ongoing updates and improvements after deployment to keep the software relevant and efficient

This structured approach ensures that software is built to meet immediate needs and is sustainable and adaptable over time, accommodating changes in technology and user requirements.

In addition to these technical activities, software engineering is supported by crucial nontechnical activities for a project's successful completion. These include cost and effort estimation, which allows for effective budgeting and resource allocation; project management, which involves overseeing the project's progress to ensure it meets deadlines and objectives; and **process improvement**, which focuses on continually refining and enhancing the development process for greater efficiency and quality. These activities provide a framework that supports the technical side of software engineering, ensuring that projects are technically sound and managed efficiently and economically. The technical and nontechnical facets of software engineering contribute to creating robust, reliable, and high-quality software systems.

The website for the Software Engineering Institute (SEI) at Carnegie Mellon University is shown in Figure 11-1. SEI is a leader in software engineering and provides quality standards and suggested procedures for software developers and systems analysts. SEI's primary objective is to find better, faster, and less expensive software development methods. To achieve that goal, SEI designed an influential set of software development standards called the **Capability Maturity Model (CMM)**, which has been used successfully by thousands of organizations around the globe. The model aims to improve software quality, reduce development time, and cut costs. The five maturity levels of the software CMM are shown in Figure 11-2.

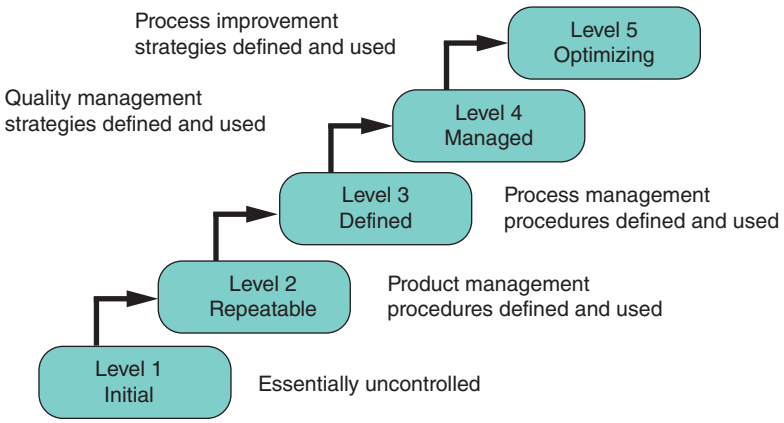
After the original software CMM was released and updated, other capability maturity models were introduced. Eventually, the SEI established a new model, **Capability Maturity Model Integration (CMMI)**, that combines software and systems development into a much larger framework. CMMI tracks an organization's processes using five maturity levels, from Level 1, which is unpredictable, poorly controlled, and reactive, to Level 5, in which the optimal result is process improvement.

11.1.2 Systems Engineering

Systems engineering is an interdisciplinary field that extends beyond the scope of software engineering to encompass various components of a comprehensive system, including hardware, networks, and user interfaces. This approach is crucial in ensuring that all aspects of a system work harmoniously to achieve the desired outcome. By integrating these diverse elements, systems engineering addresses the complexities and interdependencies inherent in modern technological systems. It focuses on the software and other critical components like physical hardware and network infrastructure, which are essential for the system's overall functionality and performance.

Figure 11-1 The Software Engineering Institute at Carnegie Mellon University has profoundly influenced software engineering research and practice.
Source: Carnegie Mellon University

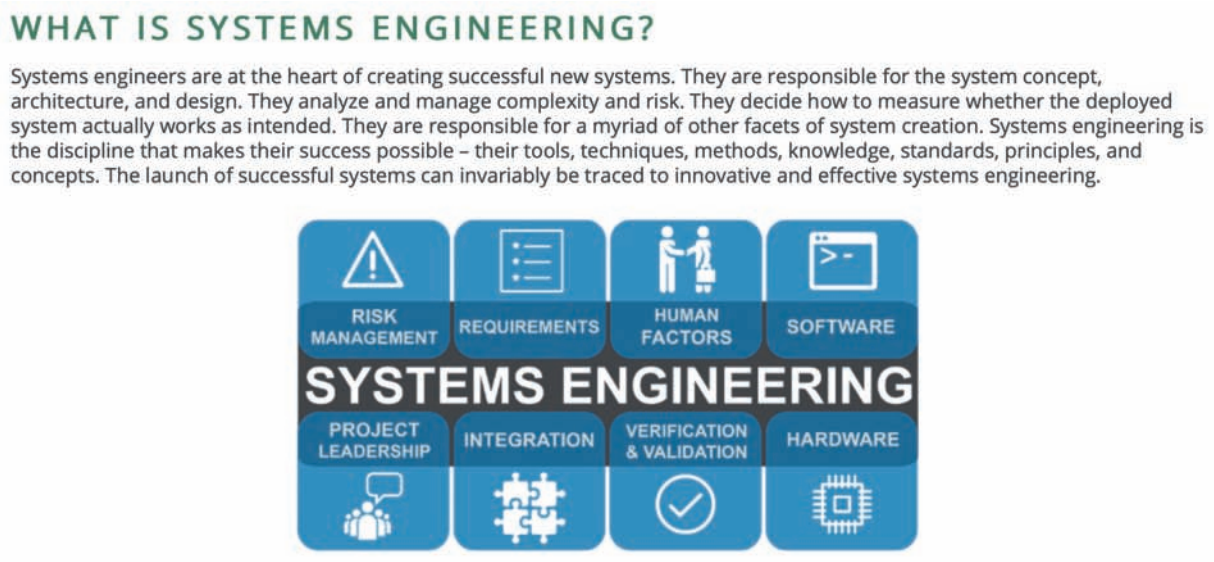
Figure 11-2 The CMM has five maturity levels, from Level 1 (Initial), which is essentially uncontrolled development, to Level 5 (Optimizing), in which process improvement strategies are defined and used.



Trade organizations such as the International Council on Systems Engineering (INCOSE) and the IEEE Systems Council play a pivotal role in shaping the field of systems engineering. These organizations are instrumental in setting best practices, providing guidance on ethical considerations, and fostering the development of new methodologies and technologies. They offer a platform for professionals to share knowledge, collaborate on research, and stay abreast of emerging trends in systems engineering. Their contributions are vital in ensuring that the field evolves in response to the rapid technological advancements and changing requirements of modern systems.

As illustrated in Figure 11-3, systems analysts benefit significantly from adopting a holistic problem-solving approach, especially in large-scale projects. This approach involves considering all facets of a system, from technical specifications to user experience, and requires a broad perspective beyond traditional software development. By embracing a systems engineering mindset, analysts can better understand the interplay between various system components and their impact on the project. This comprehensive perspective is essential for identifying potential issues, optimizing system performance, and ensuring that the final product meets both technical and end-user needs. It highlights the importance of a well-rounded approach in tackling complex systems analysis and design challenges.

Figure 11-3 INCOSE's view of systems engineering.
Source: INCOSE - International Council on Systems Engineering



11.1.3 International Organization for Standardization

Automobiles, water, and software might seem unrelated at first glance, but they share a common thread: all are subject to standards set by the International Organization for Standardization (ISO). This connection highlights the vast range of products and services governed by ISO standards to ensure quality, safety, and efficiency. These standards are not limited to physical products but extend to management practices and processes that underpin the delivery of high-quality goods and services across various sectors, including the technology industry. ISO standards serve as a global benchmark for excellence, providing universally recognized symbols (see Figure 11-4), the ISBN numbering system that identifies this textbook, and many other specifications to facilitate unified understanding and recognition across international borders.

The importance of software in the success of companies cannot be overstated, leading many organizations to seek assurances that their software systems adhere to rigorous quality standards. In response to this need, ISO introduced **ISO/IEC/IEEE 90003:2018**, guidelines that apply the principles of **ISO 9001:2015** to computer software. This adaptation of ISO 9001:2015, a global standard for quality management systems, sets forth criteria to ensure the

Figure 11-4 ISO symbols include internationally recognized symbols.
Source: bytedust/Shutterstock.com

consistent quality of products and services. Emphasizing customer satisfaction and regulatory compliance fosters a culture of continuous improvement within organizations, ensuring that software development and management processes meet high-quality standards.

For companies procuring software or developing it in-house, adherence to ISO standards represents a strategic approach to guaranteeing software quality and reliability. By incorporating ISO standards into procurement specifications, organizations establish clear quality benchmarks suppliers must meet, ensuring that software products align with international quality norms. This approach enhances the trust and reliability between suppliers and customers and instills a culture of quality and precision within teams responsible for in-house software development. ISO guidelines offer a structured development framework from software conception to deployment, ensuring the creation of robust, efficient, and purpose-fit software.

ISO standards are known for their meticulous and exhaustive nature, especially in software development. A fundamental requirement of these standards is formulating a detailed development plan that maps out the conversion of user requirements into a tangible software product. Such a plan is pivotal, as it delineates the various phases of development, guaranteeing that every aspect of the software is carefully considered and addressed. Additionally, ISO standards emphasize the importance of thorough documentation and rigorous software testing. Suppliers must document all testing activities and maintain records of outcomes, a practice critical for troubleshooting and quality assurance. This documentation ensures a transparent and accountable resolution process for any identified issues, requiring that faults be promptly corrected and retested to validate the resolution.

ISO standards play a crucial role in elevating the quality of software development and procurement. ISO standards facilitate high transparency and control over software development by establishing specific development plans, rigorous documentation, and testing

requirements. This structured approach ensures the production of high-quality software and aligns with the overarching goal of ISO to promote superior management practices and product quality across diverse industries. By adhering to these standards, companies can achieve consistent software quality, enhance customer satisfaction, and meet regulatory demands through continuous improvement.

11.2 Application Development

Application development, the process of building programs and code modules for information systems, incorporates structured analysis, object-oriented (O-O) analysis, and agile methods as popular development strategies. These strategies aim to convert designs into functional programs and code modules. This task becomes increasingly complex with the addition of hundreds or thousands of modules in even modest-sized projects. Managing this complexity and controlling schedules and budgets are critical, necessitating effective project management. As users and managers anticipate deploying the new system, they must meet schedules, control costs, and ensure quality. Thus, as previously discussed, employing project management tools and techniques is essential for the systems analyst or project manager to successfully monitor and steer the development process toward achieving project goals efficiently.

11.2.1 Design Translation Strategies

Design translation strategies serve as a critical bridge between the conceptual design of an information system and its practical implementation. This process involves converting the abstract representations and models created during the analysis phase—whether through structured analysis, O-O analysis, or agile methodologies—into tangible program and code modules. The focus is identifying the most effective strategies to ensure the theoretical designs align with the system's functional requirements. These strategies encompass a range of practices, from determining the appropriate programming languages and frameworks that best realize the design's intent to mapping out design patterns that ensure code modularity and reusability. By carefully selecting and applying these strategies, developers can translate complex designs into efficient, maintainable, scalable software architectures.

Emphasis is placed on maintaining the integrity of the design throughout the development process. This involves rigorous adherence to the design specifications detailed in the initial phases, ensuring that the final software product accurately reflects the planned architecture and functionalities. Techniques such as code reviews, pair programming, and continuous integration play significant roles in this context, helping to identify deviations from the design early and facilitating corrective actions. Moreover, these strategies advocate for a collaborative approach to development, where designers and developers work closely to navigate the challenges of translating design concepts into code, ensuring that the system's vision and practicality are preserved.

The design translation strategy must be adaptable and responsive to change. In the dynamic landscape of software development, requirements can evolve and new constraints may emerge. Agile methodologies in particular provide a framework for incorporating changes while keeping sight of the original design goals. Through iterative development cycles and constant feedback loops, these strategies support a flexible approach to design translation, allowing teams to refine and adjust their code in response to new information or shifting project objectives. This adaptability ensures that the development process remains aligned with stakeholders' needs and the technological landscape, ultimately creating robust and effective information systems.

11.2.2 Application Development Tasks

Translating system designs into functional applications is effectively achieved through traditional structured or O-O methodologies. These approaches provide developers with a systematic framework for turning detailed design diagrams and specifications into executable

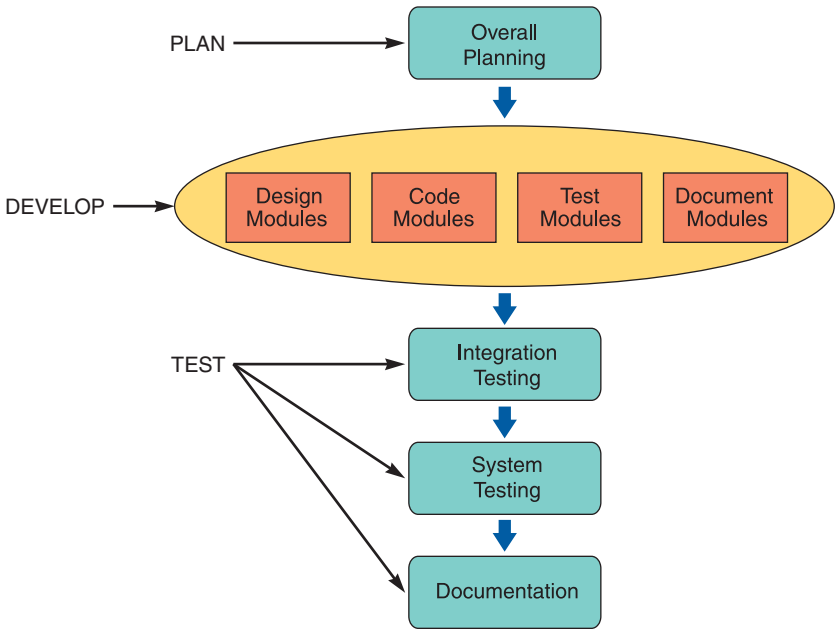
code. As a result, the final application closely mirrors the intended architecture and functionalities outlined during the design phase, ensuring that the system meets its original objectives and requirements.

In contrast, opting for an agile development methodology shifts the focus toward project planning, foundational groundwork, and team assembly, emphasizing the importance of continuous customer feedback and engagement. This adaptability is central to the agile approach, fostering an environment where project goals and customer needs can evolve and ensuring the final product remains relevant and directly aligned with user expectations. Through this iterative process, agile methodologies enable a dynamic development cycle that is more flexible and responsive, allowing for continuous refinement of project scope and deliverables.

Traditional Methods

Constructing a new system is a process that demands meticulous planning and a strategic approach. Initially, an overarching strategy must be established to guide the development process. This strategy sets the stage for the subsequent design, coding, testing, and documentation of individual modules within the system. Modules of related program code are organized into small, manageable units to facilitate ease of understanding and maintenance. This modular approach simplifies the development process and enhances the system's maintainability. Once these modules have been individually developed and tested, further testing is conducted on the system. As shown in Figure 11-5, this comprehensive testing phase is complemented by the thorough documentation of the entire system, ensuring that every aspect of the system's operation and maintenance is clearly understood and well-recorded.

Figure 11-5 The main steps in traditional application development.



The creation of program modules, whether through structured or O-O methodologies, commences with a review of the requirements documentation generated in earlier phases of the SDLC. This review process leads to the formulation of a detailed set of program designs that serve as a blueprint for development. The existence of a documentation file, initiated early in the development cycle and meticulously updated, becomes an invaluable asset at this stage. Central to this repository of information is the system design specification, bolstered by various supporting materials such as diagrams, source documents, screen layouts, report designs, data dictionary entries, and user feedback. Utilizing a CASE tool throughout the

systems analysis and design phases significantly eases the analyst's workload, providing a comprehensive and integrated view of the system's architecture and requirements.

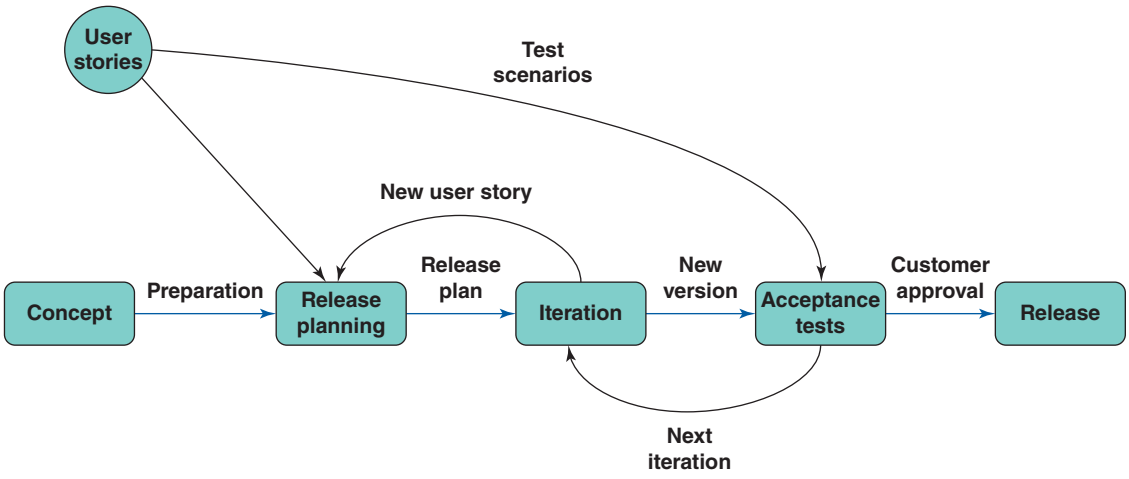
With the foundational program designs in place, the focus shifts to coding and testing tasks. While programmers are primarily responsible for writing the code, the collaborative nature of modern development projects often sees IT managers assigning systems analysts to work alongside programmers and form a cohesive team. This collaborative approach ensures a seamless transition from design to implementation, facilitating clear communication and mutual understanding among team members. The analysts bring a broader system perspective, identifying potential integration issues and ensuring the code aligns with the system requirements. This phase is crucial for transforming the theoretical designs into a functional, error-free system ready for deployment and use.

Agile Methods

Adopting an agile approach significantly enhances interaction and cooperation between the IT team and end users or customers, establishing a dynamic environment where continuous feedback is encouraged and fundamental to the development process. This methodological shift aims to construct the system through a repetitive cycle of planning, designing, coding, and testing, allowing for regular refinements based on user input and evolving project requirements.

Agile projects thrive on iterative and incremental models, such as Extreme Programming (XP), as shown in Figure 11-6, which facilitates rapid development cycles and frequent delivery of functional software components. This approach ensures the development process is highly responsive to changes, enabling the team to adapt and integrate new requirements or feedback efficiently. The concepts of agile development and XP are pivotal to understanding this adaptive approach to system creation; they are explored in further detail in Section 11.5.

Figure 11-6 Simplified model of an Extreme Programming project. Note the emphasis on iteration and testing.



11.2.3 Systems Development Tools

Different systems development methodologies leverage distinct tools that align with their specific processes and objectives, illustrating the diverse approaches to software creation. Structured development, for instance, utilizes DFDs and structure charts to visually represent data processes and the architecture of systems, providing a clear, top-down view of system interaction. In contrast, O-O methods employ a range of UML diagrams, such as use case diagrams to define system interactions, class diagrams to model system classes and their relationships, sequence diagrams for depicting object interactions over time, and state transition diagrams to describe state changes in response to events. Agile methodologies, noted for their flexibility and responsiveness to change, often incorporate spiral or other

iterative models that emphasize continuous feedback and incremental development, enabling rapid adaptation to new requirements or changes in project scope.

Beyond the specific tools associated with each development methodology, system developers have access to a suite of multipurpose tools designed to abstract and clarify system logic across various stages of development. These tools are indispensable for translating complex system designs into functional program modules. Entity-relationship diagrams (ERDs) are widely used to model the data and relationships within a system, flowcharts provide a straightforward method of outlining process flows and decision points, pseudocode offers a high-level description of an algorithm in a format close to that of human-readable languages, decision tables present a concise way to represent complex decision logic, and decision trees visualize the branching paths and outcomes of different decision scenarios. These generic tools serve as fundamental building blocks in the developer's toolkit and are applicable across different methodologies for their ability to simplify and clarify the development process.

Using various tools tailored to specific methodologies or serving more general purposes is crucial for effectively managing the SDLC. Developers can create well-designed and effective systems by leveraging the strengths of these tools. Structured and O-O methodologies emphasize thorough planning and modeling to predict system behavior and structure. On the other hand, agile methods focus on adaptability and continuous feedback from stakeholders to improve the system gradually. Regardless of the methodology, effective tool use helps turn conceptual designs into software that is robust, efficient, and user-focused.

Entity-Relationship Diagrams

During the data design phase of system development, ERDs play a pivotal role in illustrating the interactions between entities and objects within the system. These diagrams are invaluable across various development methodologies, providing a visual representation that facilitates a deeper understanding of the system's data structure. ERDs clearly outline the different types of relationships—one-to-one, one-to-many, and many-to-many—crucial for accurately modeling how data elements interrelate. This visual tool is essential for developers and analysts to build a database structure that effectively aligns with the system's functional requirements and supports its operations.

The proper use and integration of ERDs in the SDLC enable developers to anticipate and strategically plan for data interactions and dependencies. This foresight contributes to creating a more robust and coherent system architecture, ensuring the final application is efficient and scalable. ERDs help lay a solid foundation for application development by meticulously mapping the relationships and interactions between data points. As a result, the system developed is more aligned with user needs and business goals, facilitating smoother operations and enhanced performance.

Flowcharts

As highlighted earlier, flowcharts are essential in software development, visually mapping out program logic to simplify understanding a system's structure. A **flowchart** uses a combination of symbols and arrows to represent logical rules and interactions, breaking down complex processes into smaller, more digestible pieces. This ability to segment large systems into understandable subsystems and modules aids programmers in the coding process, making the task less daunting and more efficient. Flowcharts are particularly useful for delineating process flows and critical decision points, which help developers identify and address potential challenges early on.

The utility of flowcharts extends beyond the programming and development team. They are crucial during the initial design stages, offering a clear and understandable representation of system logic that is accessible to both technical and nontechnical stakeholders. This clarity benefits planning and development, troubleshooting issues, and facilitating discussions about system functionality and logic. By providing a common visual language, flowcharts ensure that everyone involved in the project, regardless of their technical expertise, can understand and contribute to the development process, promoting transparency and collaboration throughout the project's life cycle.

Pseudocode

Pseudocode is a powerful tool for outlining program logic in a format that mirrors structured English, making it accessible and easy to understand, as previously discussed in this text. By not being bound to the syntax of any specific programming language, pseudocode allows for the description of software modules in straightforward English. This approach eliminates the complexities associated with the stringent syntax rules of programming languages, providing a flexible means for systems analysts and programmers to convey the functionalities and logic of a program. It enables the drafting of algorithms and process flows in an understandable manner across the board, simplifying the transition to actual code in any programming language selected for the project.

Using pseudocode significantly improves communication within development teams, effectively bridging the gap between individuals with varying levels of technical expertise. This ensures that all team members understand the system's intended functionalities. As a fundamental component of the software development process, pseudocode lays the foundation for effective coding and system implementation, streamlining the development workflow. It plays a pivotal role in making the initial stages of development more efficient, ultimately contributing to the successful realization of the system as envisioned. Figure 11-7 details a sales promotion policy, exemplifying pseudocode's practical application and utility in documenting and explaining system processes.

Figure 11-7 Sample of a sales promotion policy with logical rules (top) and a pseudocode version of the same policy (bottom). Notice the alignment and indentation of the logic statements in the pseudocode.

Sample of a Sales Promotion Policy

- Preferred customers who order more than $1,000 are entitled to a 5% discount, and an additional 5% discount if they used our charge card.
- Preferred customers who do not order more than $1,000 receive a $25 bonus coupon.
- All other customers receive a $5 bonus coupon.

Pseudocode Version of the Sales Promotion Policy

```
IF customer is a preferred customer, and
    IF customer orders more than $1,000 then
        Apply a 5% discount, and
        IF customer uses our charge card, then
            Apply an additional 5% discount
    ELSE
            Award a $25 bonus coupon
ELSE
    Award a $5 bonus coupon
```

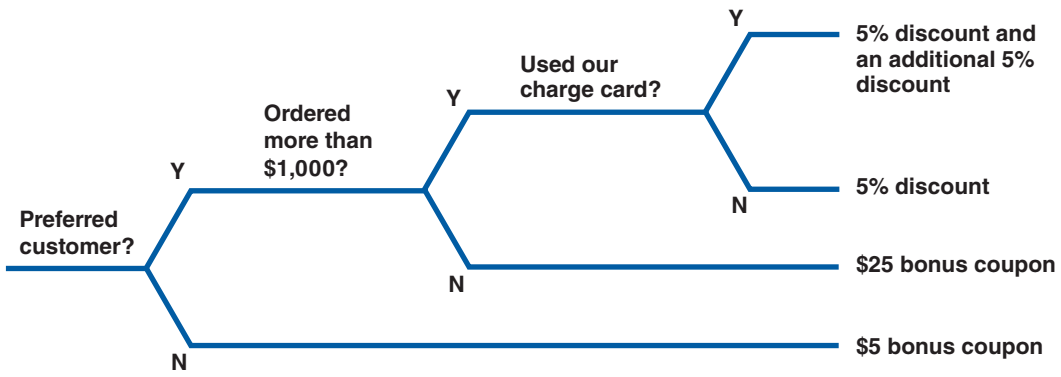
Decision Tables and Decision Trees

Earlier in the book, decision tables and decision trees were described as effective tools for modeling an information system's business logic. These tools provide a structured approach for outlining complex decision-making processes, making them invaluable for the initial design and development phases. By breaking down business logic into precise, manageable components, decision tables and decision trees help ensure all potential scenarios are considered and appropriately addressed.

During the systems development phase, analysts and programmers can further utilize decision tables and decision trees to guide the creation of code modules that embody these logical rules. This application ensures that the software accurately reflects the intended business logic, facilitating a more direct translation of business requirements into technical solutions. By incorporating these tools into the workflow, developers can create more reliable information systems that closely align with organizational objectives and user needs.

Figure 11-8 shows an example of a decision tree that documents the sales promotion policy shown in Figure 11-7. Notice that the decision tree accurately reflects the policy, which has three conditions and four possible outcomes.

Figure 11-8 Sample decision tree that reflects the sales promotion policy in Figure 11-7. Like a decision table, a decision tree shows the action based on specific properties.



11.3 Structured Development

Structured application development involves a **top-down approach**, transitioning from a broad design concept to a granular, detailed structure. This process begins once the system's requirements are thoroughly documented, enabling the systems analyst to decompose these requirements into smaller, manageable subsystems and modules through a technique known as partitioning. Comparable to constructing a hierarchical set of DFDs, this modular design strategy facilitates the division of the development workload, allowing multiple programming teams to work on different project areas concurrently.

Project management software plays a crucial role in enhancing the efficiency and oversight of the software development process, as previously discussed. Equipped with sophisticated features, this software enables project managers to monitor progress across individual modules closely, guaranteeing alignment with the project timeline. It also provides tools for accurately predicting development timelines, assessing the required human and technical resources, and identifying the project's critical path. Such capabilities help ensure that development efforts remain on schedule and resources are allocated most effectively.

To ensure the cohesive functioning of all modules within the final system, systems analysts must engage in meticulous planning and continuous collaboration with programmers and IT management. This collaborative effort is essential for crafting a coherent and well-integrated system architecture. Analysts are tasked with incorporating integration capabilities into each module's design and conducting rigorous testing to verify these integration points. This careful attention to detail and integration ensures the system operates seamlessly as a unified whole, meeting the project's objectives and requirements.

11.3.1 Structure Charts

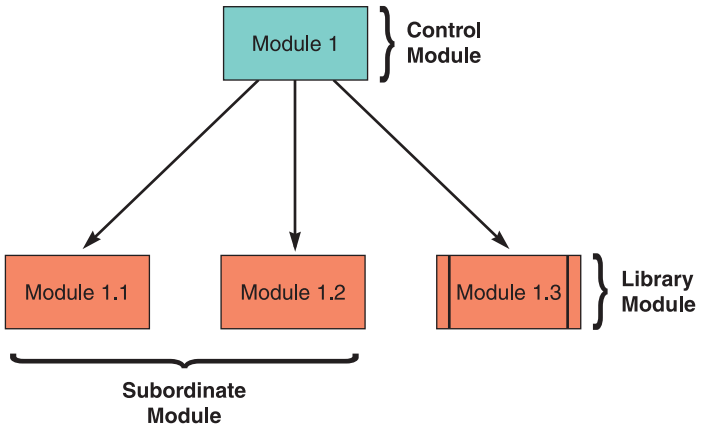
Structure charts show the program modules and the relationships among them. A **structure chart** consists of rectangles representing the program modules, with arrows and other symbols providing additional information. Typically, a higher-level module, called a **control module**,

directs lower-level modules, called **subordinate modules**. In a structure chart, symbols represent various actions or conditions. Structure chart symbols represent modules, data couples, control couples, conditions, and loops.

Modules

As shown in Figure 11-9, a rectangle represents a module. In the lower-right part of the figure, vertical lines at the edges of the rectangle indicate that module 1.3 is a library module. A **library module** is reusable code and can be invoked from more than one point in the chart.

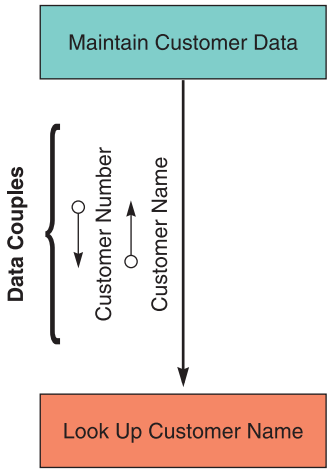
Figure 11-9 An example of structure chart modules.



Data Couples

An arrow with an empty circle represents a data couple. A **data couple** shows data that one module passes to another. In the data couple example shown in Figure 11-10, the Look Up Customer Name module exchanges data with the Maintain Customer Data module.

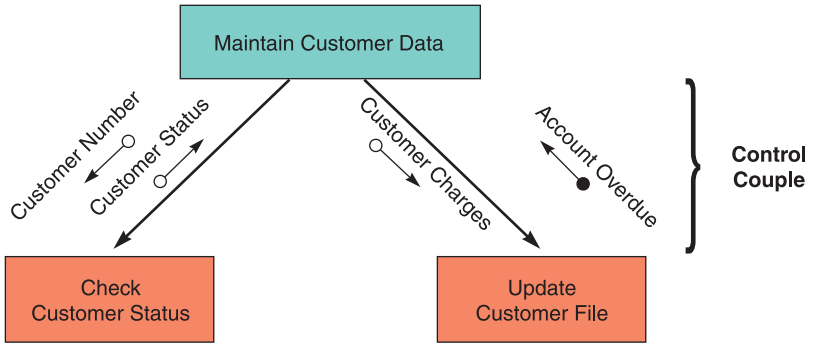
Figure 11-10 An example of a structure chart data couple.



Control Couples

An arrow with a filled circle represents a control couple. A **control couple** shows a **status flag** message, which one module sends to another. In the example shown in Figure 11-11, the Update Customer File module sends an Account Overdue flag back to the Maintain Customer Data module. A module uses a flag to signal a specific condition or action to another module.

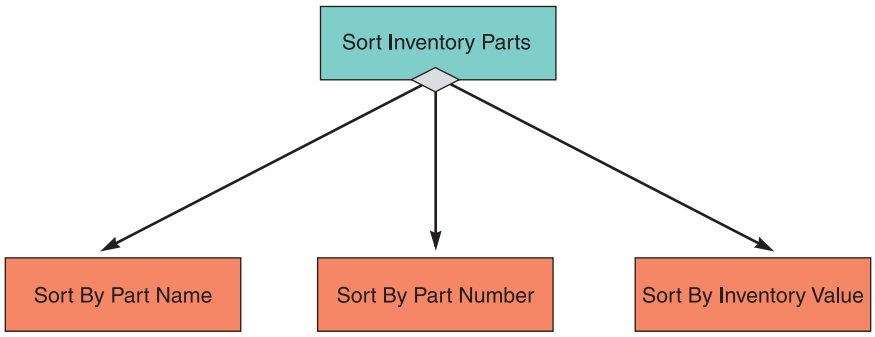
Figure 11-11 An example of a structure chart control couple.



Conditions

A line with a diamond on one end represents a condition. A **condition** line indicates that a control module determines which subordinate modules will be invoked depending on a specific condition. In the example shown in Figure 11-12, Sort Inventory Parts is a control module with a condition line that triggers one of the three subordinate modules.

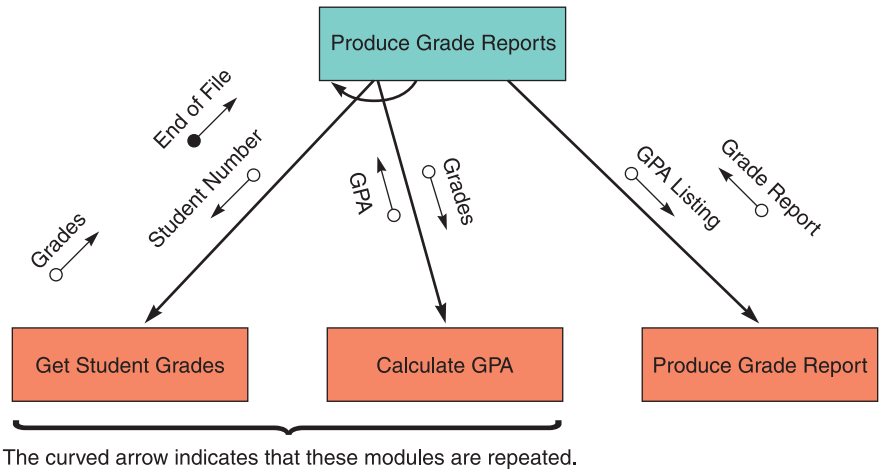
Figure 11-12 The diagram shows a control module that triggers three subordinate modules.



Loops

A curved arrow represents a loop. A **loop** indicates that one or more modules are repeated. In the example shown in Figure 11-13, the Get Student Grades and Calculate GPA modules are repeated.

Figure 11-13 The diagram shows a structure chart loop with two repeating modules.



11.3.2 Cohesion and Coupling

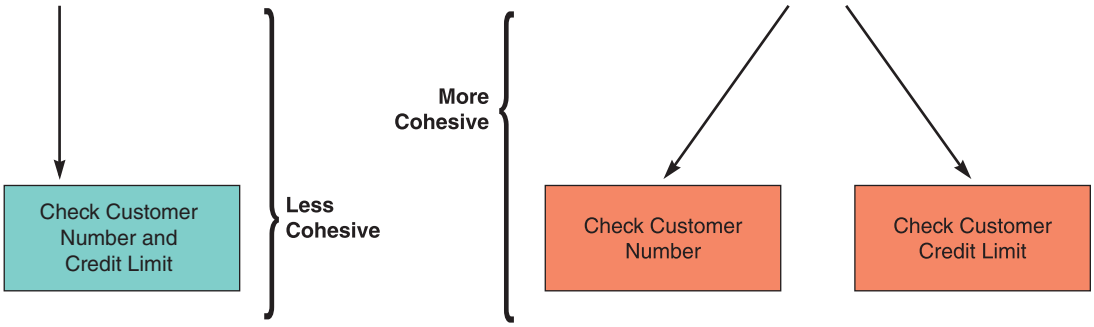
Cohesion and **coupling** are crucial metrics for assessing the quality of a system's design. The goal is to achieve highly cohesive and loosely coupled modules. High cohesion within a module signifies that its functions are closely related and focused, enhancing understandability and reducing complexity. In contrast, loose coupling between modules indicates minimal dependencies, promoting flexibility and ease of modification. When these conditions are met, the system becomes easier and less costly to maintain, as changes to one part of the system have minimal impact on others, significantly reducing the effort and risk associated with modifying the system's structure.

Cohesion

Cohesion measures a module's scope and processing characteristics. A module that performs a single function or task has a high degree of cohesion, which is desirable. Because it focuses on a single task, a cohesive module is much easier to code and reuse. For example, a Verify Customer Number module sounds more cohesive than a Calculate and Print Statements module. If the word *and* is found in a module name, it implies that more than one task is involved.

If a module must perform multiple tasks, more complex coding is required, and the module will be more challenging to create and maintain. To make a module more cohesive, split it into separate units, each with a single function. For example, cohesion is greatly improved by splitting the Check Customer Number and Credit Limit module in Figure 11-14 into two separate modules, Check Customer Number and Check Customer Credit Limit.

Figure 11-14 Two examples of cohesion. Notice that the single module on the left is less cohesive than the two on the right.

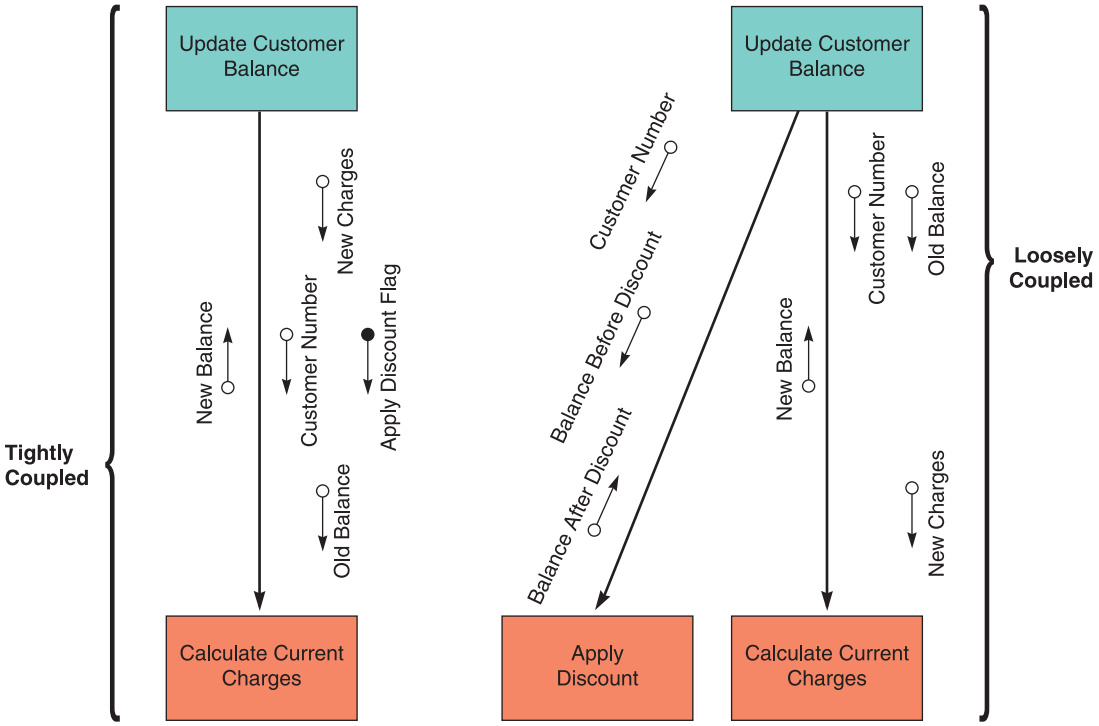


Coupling

Coupling describes the degree of interdependence among modules. Independent modules are **loosely coupled**, which is desirable. Loosely coupled modules are easier to maintain and modify because the logic in one module does not affect other modules. If a programmer needs to update a loosely coupled module, they can accomplish the task in a single location. If modules are **tightly coupled**, one module is linked to internal logic contained in another module. For example, Module A might refer to an internal variable in Module B. In that case, a logic error in Module B will affect the processing in Module A. For that reason, passing a status flag down as a message from a control module is generally regarded as poor design. It is better to have subordinate modules handle processing tasks as independently as possible to avoid a cascade effect of logic errors in the control module.

In Figure 11-15, the tightly coupled example on the left shows that the subordinate module Calculate Current Charges depends on a status flag sent down from the control module Update Customer Balance. It would be preferable to have the modules loosely coupled and logically independent. A status flag on the right in the example is unnecessary because the subordinate module Apply Discount handles discount processing independently. Any logic errors are confined to a single location: the Apply Discount module.

Figure 11-15 An example of tightly coupled and loosely coupled structure charts.



11.3.3 Drawing a Structure Chart

When employing a structured analysis method during system design, structure charts are derived from the DFDs developed in the data and process modeling stages. Creating a structure chart involves three key steps: reviewing the DFDs to pinpoint processes and methods, identifying program modules and determining their control and subordinate relationships, and incorporating symbols to represent data flows, control flows, and loops. After their development, these structure charts undergo a thorough analysis to verify their alignment with the system documentation. This ensures that the visual representations accurately reflect the intended system architecture and functionality.

Review the DFDs

The initial phase in advancing system design involves thoroughly reviewing all DFDs to ensure their accuracy and completeness, especially considering that modifications might have been made since the systems analysis phase. This scrutiny is crucial for verifying that the DFDs accurately represent the current understanding of the system's processes and data flows, laying a solid foundation for subsequent design work. Additionally, suppose object models were developed alongside or after DFDs. In that case, these models require a detailed examination to pinpoint the specific objects within the system, the functions or methods each object is responsible for, and how these objects interact, ensuring a comprehensive understanding of the system's object-oriented aspects.

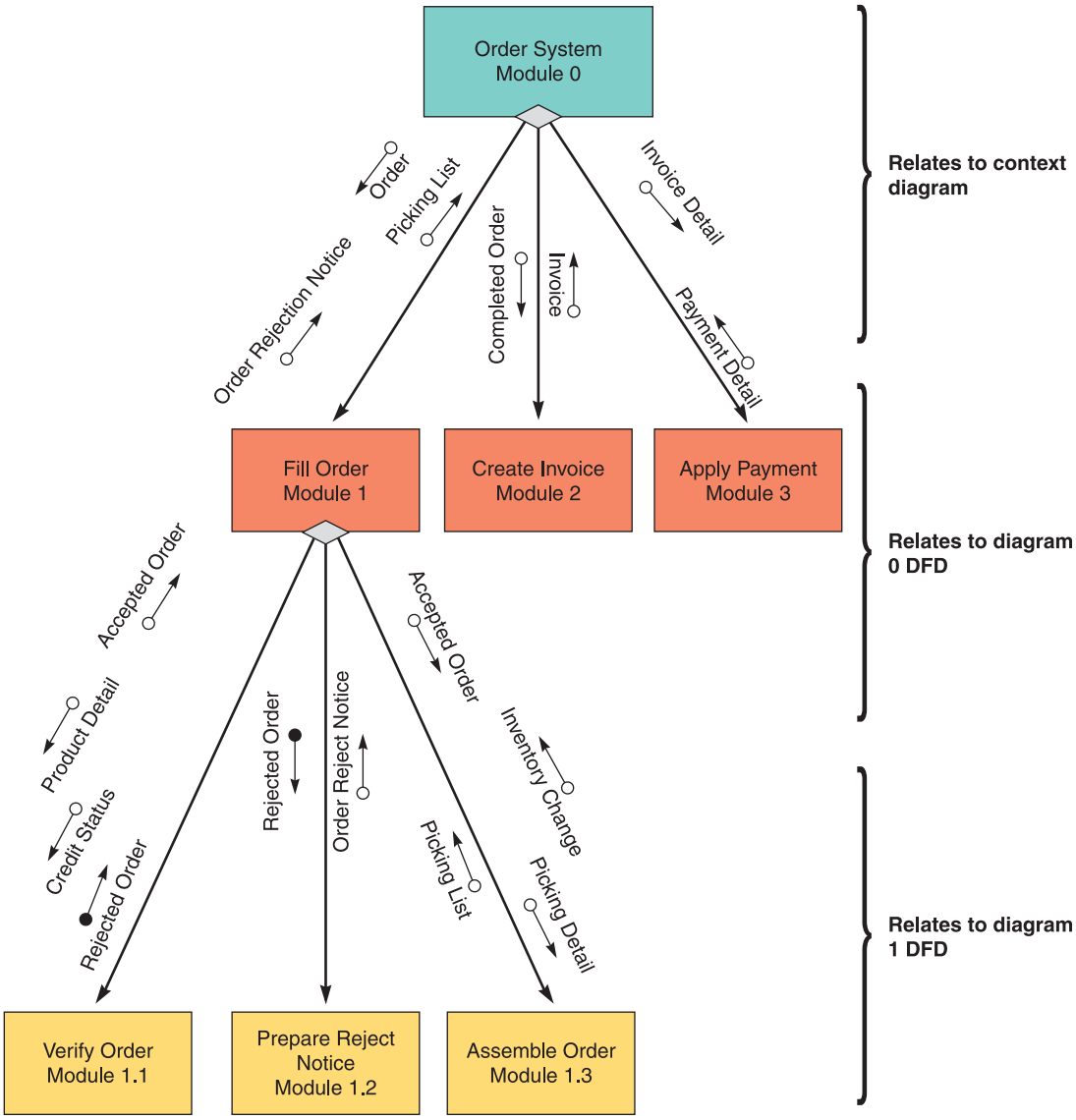
Understanding methods in the context of object models is pivotal; each method, akin to a functional primitive in structured analysis, encapsulates a sequence of actions or operations that an object can perform, necessitating precise coding to implement the necessary actions. This step highlights the functionalities that must be implemented and emphasizes the importance of delineating clear relationships and interactions among objects to facilitate a coherent and functional system architecture. This detailed analysis aids in bridging the conceptual gap between system analysis artifacts, such as DFDs and object models, and the practical requirements of system design, setting the stage for effectively translating these abstract models into executable code and tangible system components.

Identify Modules and Relationships

Starting with the logical model, the transition from functional primitives or object methods to program modules forms the crux of system design. This process involves a detailed analysis of DFDs, where each level of a DFD signifies a distinct processing level within the system. The progression from the overarching context diagram down to more detailed, lower-level diagrams facilitates identifying control and subordinate modules, guiding developers until they pinpoint the functional primitives. This hierarchical approach ensures a systematic and comprehensive transition from abstract system processes to concrete program modules. Additionally, to enhance module cohesion, processes identified in the DFDs can be further decomposed into smaller, more focused modules that manage a singular task, simplifying the overall system structure and improving manageability.

The relationship between DFD levels and the structure of program modules is visually depicted in structure charts, such as the one illustrated in Figure 11-16, which is based on the order system discussed earlier in the book. This three-level structure chart mirrors the three DFD levels, offering a clear and organized representation of how system processes are compartmentalized into program modules. Such structure charts help developers visualize

Figure 11-16 A structure chart based on the order system DFDs from a previous chapter in the book. The three-level structure chart relates to the three DFD levels.



the system's architecture and understand how each module fits the overall design. By clearly delineating the connections between different levels of DFDs and the corresponding structure chart, developers can ensure that the software architecture is logical and aligned with the system's intended functionalities, facilitating a smoother development process.

Add Couples, Loops, and Conditions

Next, couples, loops, and conditions are added to the structure chart. If DFDs are used, the data flows and dictionary can be reviewed to identify the data elements that pass from one module to another. In addition to the data couples, control couples are added where a module sends a control parameter, or flag, to another module. Loops and condition lines that indicate repetitive or alternative processing steps are also added to the diagram as needed. If an object model is developed, the class and object relationship diagrams can be reviewed to ensure the interaction among the objects is fully understood.

At this point, the structure chart is ready for careful analysis. Each process, data element, or object method should be checked to ensure that the chart reflects all previous documentation and that the logic is correct. All modules should be strongly cohesive and loosely coupled. Often, several versions of the chart must be drawn. Some CASE tools can help analyze the chart and identify problem areas.

11.4 Object-Oriented Development

O-O methods were introduced earlier in the book. O-O analysis simplifies the direct translation of an object model into an O-O programming language, a process known as **object-oriented development (OOD)**. The earlier discussion underscored the efficiency of O-O analysis in bridging the gap between conceptual models and their practical implementation in software development. By adopting O-O methods, developers can utilize the conceptual framework provided by object models to directly inform the structure and logic of their code, significantly simplifying the development process.

Though OOD and structured design share foundational principles, such as modularity and abstraction, OOD distinguishes itself through its unique approach to system development. These differences highlight the distinct advantages of using O-O methodology, including improved system flexibility and the ability to model complex, real-world scenarios. The emphasis on objects as the cornerstone of analysis, design, and implementation introduces a paradigm that mirrors real-world interactions and relationships more closely than traditional structured design approaches.

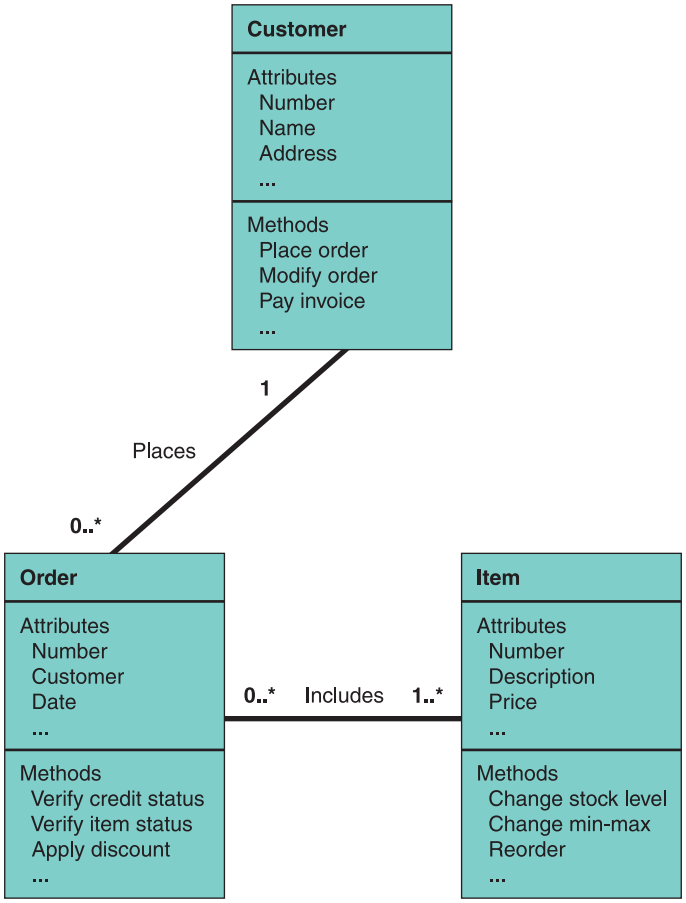
The transition from system analysis to implementation is streamlined with OOD, leveraging the inherent alignment between O-O analysis models and the programming constructs of O-O languages. This congruence allows for an intuitive development workflow, where the design decisions made during the analysis phase directly apply to coding. By leveraging the natural alignment between analysis models and programming constructs, OOD enables a more efficient and effective path from concept to functional software, underscoring its value in modern software development.

11.4.1 Characteristics of Object-Oriented Development

When implementing a structured design, a structure chart is used to describe the interaction between program modules, as explained earlier. In OOD, the emphasis is on defining the interactions between objects early in the analysis process, which then informs the overall structure of the application. The object model represents the application's structure. This departs from structured design, where structure charts are used after analysis to outline program module interactions.

Objects in OOD encapsulate data and behaviors (methods) organized into classes based on shared characteristics, facilitating a modular and reusable codebase. Class diagrams like the one shown in Figure 11-17 play a crucial role in illustrating the relationships and interactions

Figure 11-17 A class diagram for a simplified customer order processing system.



among these classes, serving as a blueprint for the system's architecture. For instance, a Customer class with attributes like Number, Name, and Address as well as methods for managing orders and payments demonstrates how objects can interact within the system, such as exchanging messages with an Order class. These diagrams, alongside object relationship diagrams, provide a visual representation of the system, aiding developers in understanding and implementing complex relationships and functionalities. Figure 11-18 shows an object relationship diagram for a fitness center, illustrating the objects and their interactions to perform business functions and transactions.

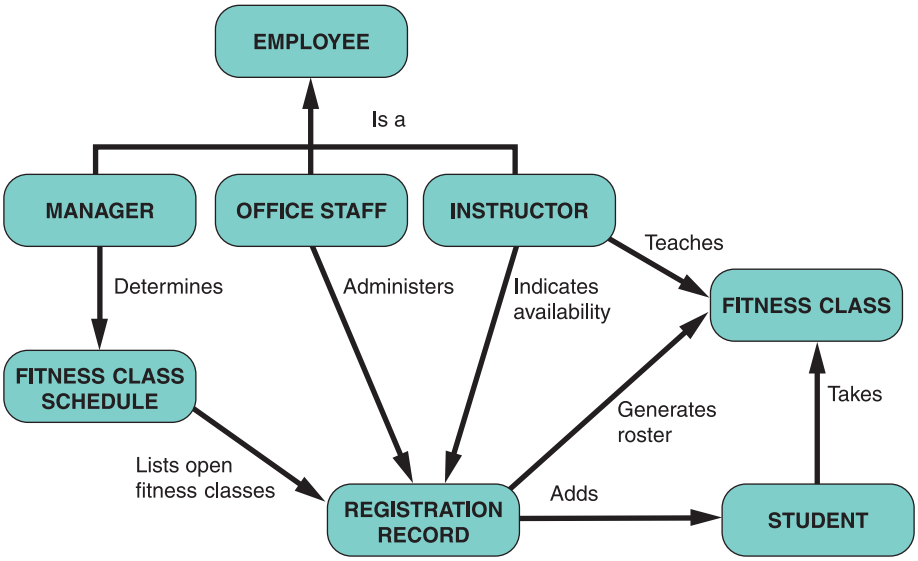
However, the transition to OOD is challenging. Organizations often have unrealistic expectations and need more time to learn about, prepare for, and implement the OOD process. For example, no one would build a bridge without an analysis of needs, supporting data, and a detailed blueprint—and the bridge would not be opened for traffic until it had been carefully inspected and checked to ensure that all specifications were met. O-O software developers sometimes forget that the basic rules of architecture also apply to their projects.

11.4.2 Implementation of Object-Oriented Designs

When programmers translate an O-O design into an application, they analyze the classes, attributes, methods, and messages documented in the object model. During this process, the programmer makes necessary revisions and updates to class, sequence, state transition, and activity diagrams. These diagrams were discussed in a previous chapter.

The role of a programmer in O-O software development is multifaceted and crucial. Their primary goal is to transform the methods defined for each object in the system into executable program code modules. This process involves thoroughly understanding how each module

Figure 11-18 An object relationship diagram for a fitness center.



will be activated or triggered, typically in response to specific events or messages. To achieve this, programmers closely analyze sequence diagrams and state transition diagrams. These diagrams are instrumental, as they visually represent the sequence of events and messages that cause changes in an object's state. By studying these diagrams, programmers can accurately determine the interactions and dependencies between different modules, which is essential for writing effective and efficient code. This understanding is critical in ensuring that each part of the program responds appropriately to the events or messages it receives.

The programming of O-O applications is inherently event-driven, a paradigm where the flow of the program is determined by events such as user actions, sensor outputs, or message passing. Each event, transaction, or message in the system triggers a corresponding action within the software, creating a dynamic and responsive application. To facilitate this process, programmers often start by drafting the steps of the program in pseudocode. Alternatively, or in conjunction, they might employ CASE tools and code generators. These advanced tools can generate O-O code directly from the object model, streamlining the development process. By using these methodologies and tools, programmers can efficiently translate the theoretical models and diagrams into functional, robust code that adheres to the principles of O-O design and effectively respond to the various events and interactions within the application.

11.4.3 Object-Oriented Cohesion and Coupling

The concepts of cohesion and coupling hold significant importance in O-O application development. Classes should be as loosely coupled as possible for effective design, meaning they should operate independently. Furthermore, the methods within an object are expected to be highly cohesive, ensuring that they execute closely related actions. Adhering to these principles not only simplifies the understanding of classes and objects but also facilitates their modification, enhancing the overall maintainability of the system.

O-O programmers must recognize the importance of cohesion and coupling in their code structure. Ignoring these critical design concepts can create a tangled web of code, making the system increasingly difficult to manage. A lack of attention to maintaining loose coupling and high cohesion can result in an intricately interwoven codebase that is challenging to decipher, posing significant hurdles for future development and maintenance efforts.

The consequences of neglecting cohesion and coupling principles in O-O development are far-reaching. Code that lacks these characteristics tends to be scattered across various parts of the application, complicating the process of making edits or updates. Such disorganization can make the codebase challenging to navigate and expensive to modify, as identifying and

implementing changes becomes a cumbersome and time-consuming task. Ensuring that classes and methods adhere to the ideals of being loosely coupled and highly cohesive is therefore essential for creating a maintainable, efficient, and scalable O-O application.

11.5 Agile Development

As stated at the beginning of the book, agile development is a distinctly different systems development method. It shares many steps with traditional development but uses a highly iterative process. The development team constantly communicates with the primary user, who is called the **customer**, shaping and forming the system to match the customer's specifications. Agile development is aptly named because it is based on a quick and nimble development process that easily adapts to change. Agile development focuses on small teams, intense communication, and rapid development iterations. The four key values of agile software development are shown in Figure 11-19.

Figure 11-19 The original manifesto for agile software development.
Source: Ward Cunningham

Programmers can use popular agile-friendly languages like Python, Ruby, and Perl. However, agile methods do not require a specific programming language; programmers also use O-O languages such as Java, C++, and C#.

As with traditional methodologies, agile development has both positive and negative characteristics. Today, agile methodology is very popular for software projects. Its supporters boast that it speeds up software development and delivers precisely what the customer wants, when the customer wants it, while fostering teamwork and empowering employees. However, there are drawbacks to this adaptive (rather than predictive) method. Critics of agile development often claim that because it focuses on quick iterations and fast releases, it lacks discipline and produces systems of questionable quality. In addition, agile methodology generally does not work as well for larger projects because of their complexity and the lack of focus on a well-defined product.

Before implementing agile development, the proposed system and development methods should be scrutinized. As experienced IT professionals know, a one-size-fits-all solution does not exist. For more information on agile methods, refer to the discussion of systems development methods earlier in the book.

11.5.1 Extreme Programming

XP is an agile development method. It is an iterative approach, as shown in Figure 11-20, where a team of users and developers immerse themselves in systems development. XP supporters emphasize simplicity, communication, feedback, respect, and courage. Success requires a solid commitment to the process, corporate support, and dedicated team members.

Figure 11-20 The five core values of Extreme Programming.
Source: Don Wells

The Values of Extreme Programming

Extreme Programming (XP) is based on values. The rules we just examined are the natural extension and consequence of maximizing our values. XP isn't really a set of rules but rather a way to work in harmony with your personal and corporate values. Start with XP's values listed here then add your own by reflecting them in the changes you make to the rules.

Simplicity: We will do what is needed and asked for, but no more. This will maximize the value created for the investment made to date. We will take small simple steps to our goal and mitigate failures as they happen. We will create something we are proud of and maintain it long term for reasonable costs.

Communication: Everyone is part of the team and we communicate face to face daily. We will work together on everything from requirements to code. We will create the best solution to our problem that we can together.

Feedback: We will take every iteration commitment seriously by delivering working software. We demonstrate our software early and often then listen carefully and make any changes needed. We will talk about the project and adapt our process to it, not the other way around.

Respect: Everyone gives and feels the respect they deserve as a valued team member. Everyone contributes value even if it's simply enthusiasm. Developers respect the expertise of the customers and vice versa. Management respects our right to accept responsibility and receive authority over our own work.

Courage: We will tell the truth about progress and estimates. We don't document excuses for failure because we plan to succeed. We don't fear anything because no one ever works alone. We will adapt to changes when ever they happen.

XP uses a concept called **pair programming**. In pair programming, two programmers work on the same task on the same computer; one drives (programs) while the other navigates (watches). The onlooker examines the code strategically to see the *forest*, while the driver is concerned with the individual *trees* immediately in front of them. The two discuss their ideas continuously throughout the process.

Another important concept in XP is that unit tests are designed before code is written. This **test-driven development (TDD)** focuses on initial results and prevents programmers from straying from their goals. Because of the magnitude and intensity of the multicycle process, agile testing relies heavily on automated testing methods.

11.5.2 User Stories

Suppose that a customer has requested a sales tracking system. Like any other development method, the first step in an agile process would be to define the system requirements. The customer begins by meeting with programmers and providing user stories. A **user story** is a short, simple requirements definition. Programmers review user stories to determine the project's requirements, priorities, and scope. Here are three user story examples:

- As the sales manager, I want to identify fast- or slow-moving items to manage our inventory more effectively.
- As a store manager, I need enough lead time to replenish my stock so I don't run out of hot items.
- As a sales representative, I want to offer the best selection of fast-selling items and clear out the old stock that is not moving.

User stories do not deal with technical details and are so short that they are often written on index cards. The customer prioritizes each user story so the requirements can be ranked. In addition, programmers assign a score to each user story that indicates the estimated difficulty of implementation. This information helps the team form a plan and allocate its resources. Projects are often composed of many user stories, which taken together form *epics*, from which programmers can estimate the scope, time requirements, and project difficulty. In addition to user stories, frequent face-to-face meetings with customers provide more detail as the project progresses.

11.5.3 Iterations and Releases

In XP, software development unfolds through short, planned increments, with a clear **release plan** dictating user story implementation and the timing of regular releases. Acting as iterative prototypes, these releases undergo continuous testing and refinement based on user feedback, ensuring the software adapts and improves over time. Development is segmented into focused **iteration cycles**, promoting swift feature development, testing, and integration. The **planning game** enhances this approach by enabling developers and stakeholders to prioritize tasks collaboratively and fine-tune the development plan, ensuring it efficiently meets project objectives and user requirements.

Release Plan

The team must formulate a detailed release plan that clearly defines the sequence in which user stories will be addressed and establishes the schedule for subsequent releases. This strategic planning ensures a structured approach to deploying new features and functionalities. Releases in agile environments are designed to be relatively frequent, facilitating a continuous feedback loop with stakeholders. Each release functions like a prototype, providing a tangible version of the system that can be evaluated, tested, and refined based on user feedback and performance metrics. This iterative release strategy accelerates the delivery of value to users. It allows the development team to adapt and make informed real-time adjustments to the project, enhancing the software product's overall quality and relevance.

Iteration Cycles

User stories are implemented in a series of iteration cycles. An iteration cycle includes planning, designing, coding, and testing one or more features based on user stories. At the beginning of each iteration cycle, which is often two weeks long, the team holds an **iteration planning meeting** to break down the user stories into specific tasks that are assigned to team members. As new user stories or features are added, the team reviews and modifies the release plan.

The Planning Game

Release and iteration planning are part of the planning game, which aims to prioritize and schedule the development of features (or user stories) for the upcoming release. It is divided into two main parts: release planning, where long-term goals are set and the release schedule is determined, and iteration planning, where more immediate tasks for the next iteration (usually spanning a few weeks) are identified and assigned. This approach ensures that planning is flexible, adaptive, and closely aligned with customer needs, allowing for frequent reassessment and adjustment of the project's direction based on feedback and changing requirements.

The planning game can also be considered as consisting of three distinct phases: exploration, commitment, and steering. Exploration encompasses comprehending the project's requirements, exploring potential avenues, gathering information, discussing user stories, and evaluating various approaches' technical and business implications. Following exploration, the team commits to achievable goals and tasks for the upcoming iteration or release; this process entails detailed planning, work prioritization, and decisions on deliverables. Steering is the adaptive phase, when the team reviews progress, incorporates feedback, and adjusts the plan as needed, guiding the project based on real-world feedback and development realities.

11.6 Coding

This section delves into the critical aspects of coding, a fundamental development process that transforms conceptual ideas into functioning software. It begins by exploring programming languages, the building blocks that allow developers to convert abstract logic into concrete, executable scripts. These languages are diverse, each with unique syntax and capabilities tailored to different types of development projects, from web applications to system software. The discussion then shifts to integrated development environments (IDEs), powerful platforms that consolidate various development tools into a single application. IDEs simplify the coding process by providing developers with efficient writing, testing, and code debugging features to enhance productivity and reduce errors. The section also highlights the transformative role of online learning resources in democratizing education in coding and related disciplines. Through platforms offering interactive tutorials and comprehensive courses, individuals worldwide can now easily access the knowledge and skills needed for programming, significantly broadening the pool of skilled developers and contributing to the continuous evolution of the tech industry.

11.6.1 Programming Languages

Programming languages are the foundation of coding, enabling developers to convert intricate program logic into actionable code that computers can interpret and execute. Each developer prefers specific languages and development environments that best align with their workflow and project requirements. Popular choices among these languages include Visual Basic, Java, and Python, each renowned for flexibility and robust support across various application types. These languages provide a versatile toolkit for developers, enabling the creation of everything from simple scripts to complex, integrated systems.

Legacy applications and programming languages such as COBOL and FORTRAN remain fundamental in many large organizations, underpinning crucial financial, administrative, and scientific systems. Similarly, the C and C++ programming languages remain pivotal for developing systems programs, owing to their efficiency, portability, and close-to-hardware operation capabilities. Despite the advent of newer programming languages, these older technologies are indispensable due to their proven reliability, stability, and specific functionality tailored for legacy systems and core business processes.

However, integrating legacy programming languages with contemporary software infrastructures presents a formidable challenge for IT departments striving to keep pace with rapid technological advancements. While these older languages are still critical for maintaining and updating existing systems, they often operate under paradigms vastly different from today's development frameworks, leading to compatibility issues and increased complexity in system integration. Additionally, a significant hurdle emerges from the dwindling pool of developers proficient in these venerable languages; the lack of knowledge and expertise in older languages among newer generations of programmers complicates efforts to bridge the gap between old and new systems. This gap affects the maintenance of legacy systems. It hinders the seamless integration of these systems with cutting-edge technologies, posing a multifaceted challenge to organizations aiming to modernize their IT infrastructure without disrupting ongoing operations.

The increasing focus on web, mobile, and Internet applications has led to the adoption of languages specifically designed for these platforms, indicating a shift in user interaction patterns and the software industry's ability to adapt. Rust, known for its safety features and performance, enhances the capabilities of web-centric languages like JavaScript and Swift, aiding in creating advanced digital solutions. This movement toward specialized programming languages reflects a broader trend in software development, emphasizing the selection of languages that best fit the unique requirements of the current digital environment.

Choosing the correct programming language is a crucial step in systems implementation, directly influencing functionality, user experience, and team productivity. The decision should

align with the project's goals and the developers' expertise to navigate the complexities of modern software development successfully. This decision underlines the vital role of programming languages in evolving conceptual designs into practical, user-centric applications that meet today's digital demands.

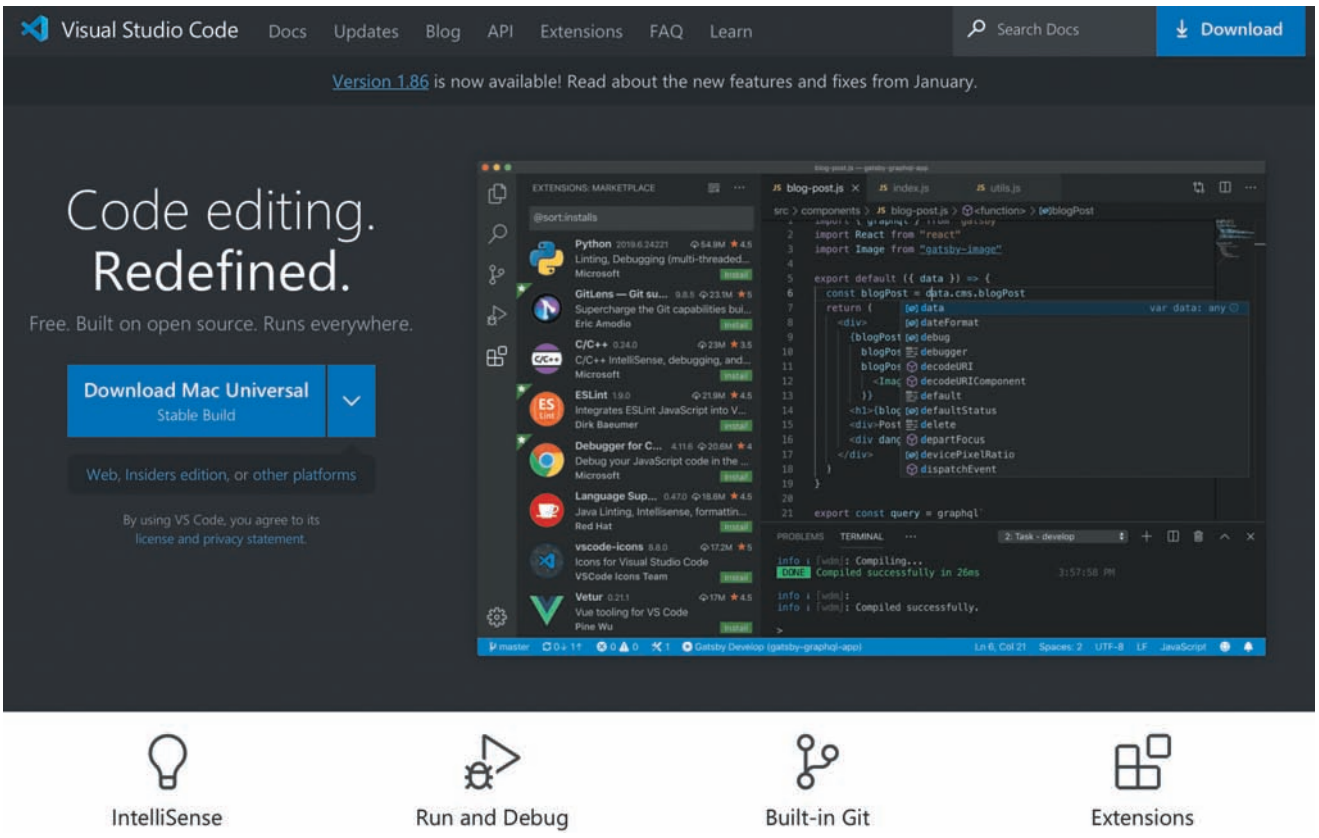
11.6.2 Integrated Development Environments

Numerous programmers turn to IDEs to streamline the process of integrating various system components and minimize the time spent on code development. These environments enhance the programming experience, especially for complex, interactive software products, by offering built-in tools and sophisticated features. Among these are real-time error detection, which immediately identifies issues as they arise, as well as syntax hints that assist in writing correct code more efficiently, code highlighting for easier readability, class browsers for navigating through code, and version control systems to manage changes and collaboration effectively.

Visual Studio Code

Microsoft Visual Studio, as previously mentioned in this book, is a popular IDE for .NET and C++ developers on Windows. Microsoft offers a related product, Visual Studio Code (VS Code), a lightweight desktop source code editor. As shown in Figure 11-21, it runs on Windows, macOS, and Linux, comes with built-in support for JavaScript, TypeScript, and Node.js, and has extensions for other languages and runtimes. It offers syntax highlighting, intelligent code completion, snippets, code refactoring, and integrated Git control, making it a powerful tool for developers working on front-end and back-end projects. VS Code is designed to be highly customizable, allowing users to personalize their coding environment

Figure 11-21 Microsoft's Visual Studio Code is a stand-alone source code editor that runs on Windows, macOS, and Linux.
Source: Microsoft Corporation

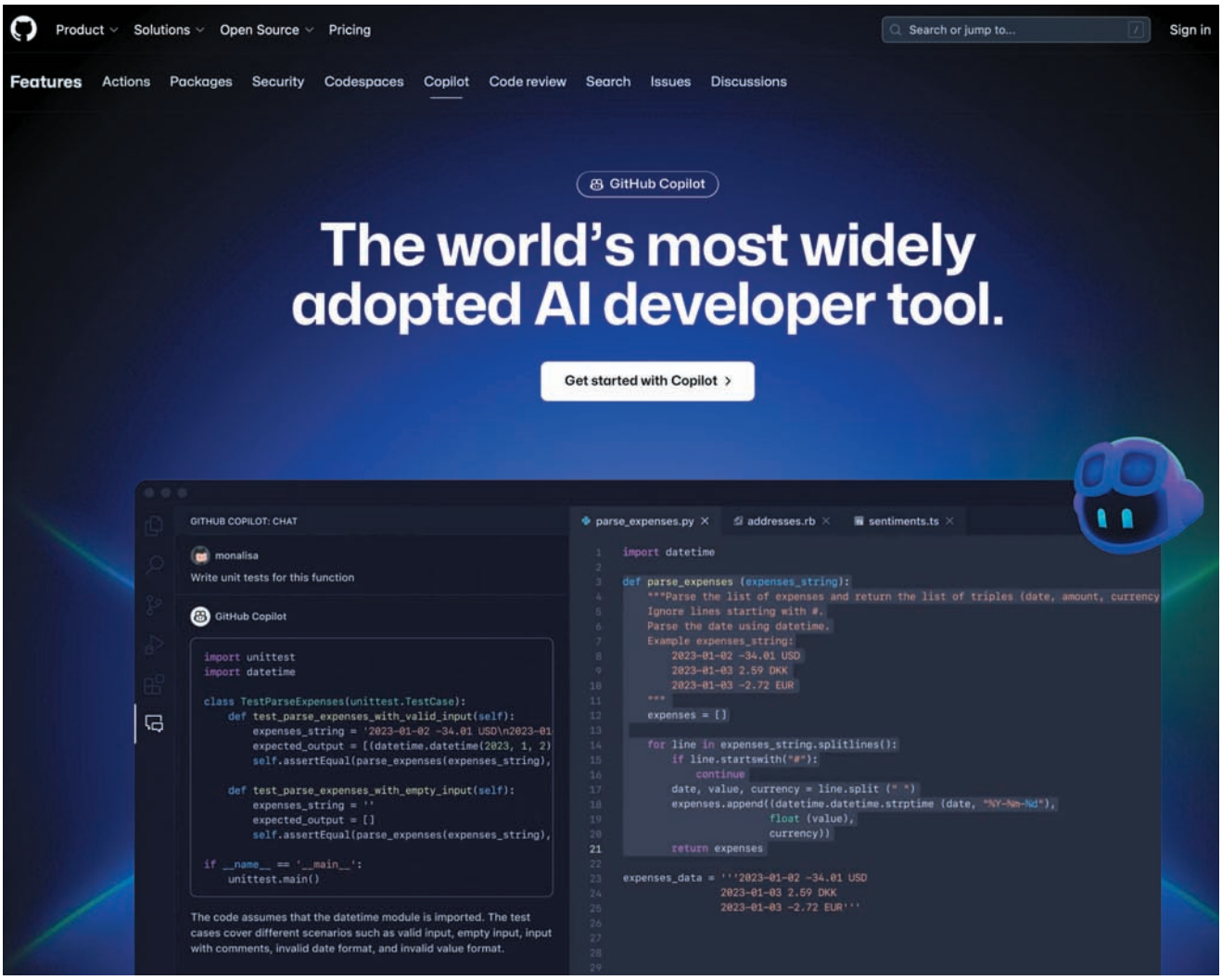


with themes, keyboard shortcuts, and other preferences. Its lightweight design and powerful development tools make it popular among developers for web and traditional application development.

GitHub Copilot

GitHub Copilot is an AI-powered code completion tool developed by GitHub and designed to work seamlessly within the Microsoft Visual Studio IDE. It leverages the OpenAI Codex to suggest entire lines or blocks of code as developers type, as shown in Figure 11-22, essentially functioning as a pair programmer that helps to improve productivity and streamline the coding process. GitHub Copilot can generate code for various languages and frameworks, understand context from comments and code, and provide recommendations for code refactoring, bug fixes, and even complex algorithmic solutions. It acts as an AI-driven "navigator" in the XP sense described in Section 11.5.1, offering real-time code suggestions and improvements and thereby enhancing the pair programming process with its vast knowledge base and instant feedback capabilities. This innovative tool is part of GitHub's broader effort to integrate AI and machine learning into the SDLC, offering developers an advanced resource to accelerate development tasks, reduce potential errors, and explore new coding solutions within Visual Studio.

Figure 11-22 GitHub Copilot is an AI-powered code completion tool developed by GitHub and OpenAI that assists developers by suggesting entire lines or blocks of code in real time within the IDE.
Source: GitHub, Inc.



Xcode

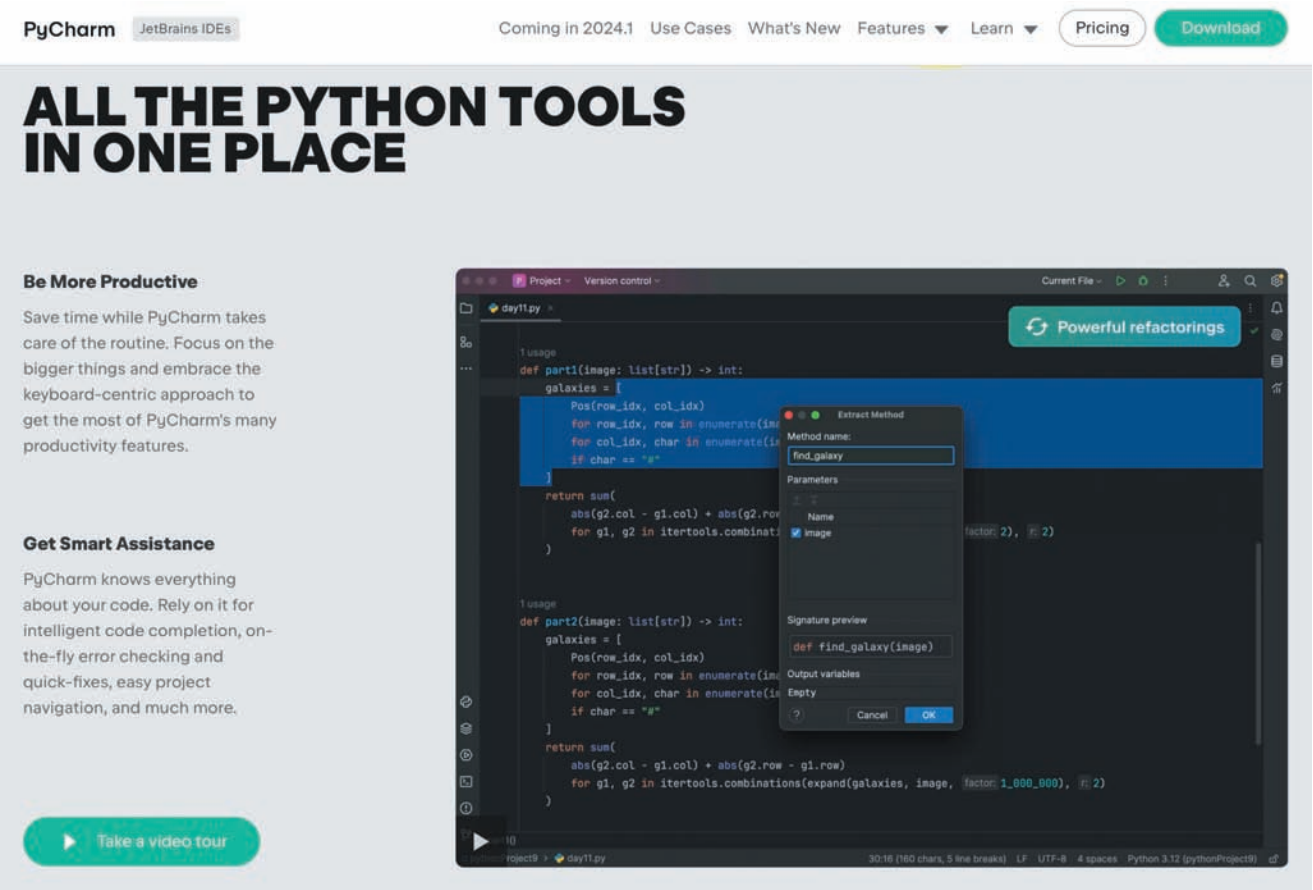
Xcode is a comprehensive IDE developed by Apple for macOS, facilitating software development for macOS, iOS, watchOS, and tvOS. It provides developers with powerful tools to create, test, and debug applications for Apple devices. As shown in Figure 11-23, Xcode includes a user-friendly interface for coding, a sophisticated compiler, and a graphical user interface editor. It also incorporates Interface Builder, an application that allows developers to design and test user interfaces without writing any code, and Simulator, which enables developers to run and test their applications on a virtual iOS device. With its support for Swift and Objective-C, Xcode is essential for developers looking to build applications within the Apple ecosystem. It offers a seamless and integrated workflow from the initial coding phase to final deployment.

Figure 11-23 Xcode is an IDE from Apple that enables engineers to develop, test, and distribute apps for all Apple platforms.
Source: Apple Inc.

PyCharm

JetBrains' PyCharm, a leading IDE for Python, is equipped with features that boost coding efficiency and simplify project workflows. As shown in Figure 11-24, it offers intelligent code completion, real-time error detection, automated fixes, and a graphical debugger for in-depth code analysis, breakpoint management, and evaluations. PyCharm also includes a unit testing framework for test-driven development and has built-in support for Django web development, complete with specific project templates and database tools. Django is a Python framework designed for fast and pragmatic web application development. PyCharm's comprehensive features make it a crucial tool for Python developers aiming to improve their productivity and code quality.

Figure 11-24 PyCharm is JetBrains' leading IDE for Python development.
Source: JetBrains s.r.o.



Eclipse

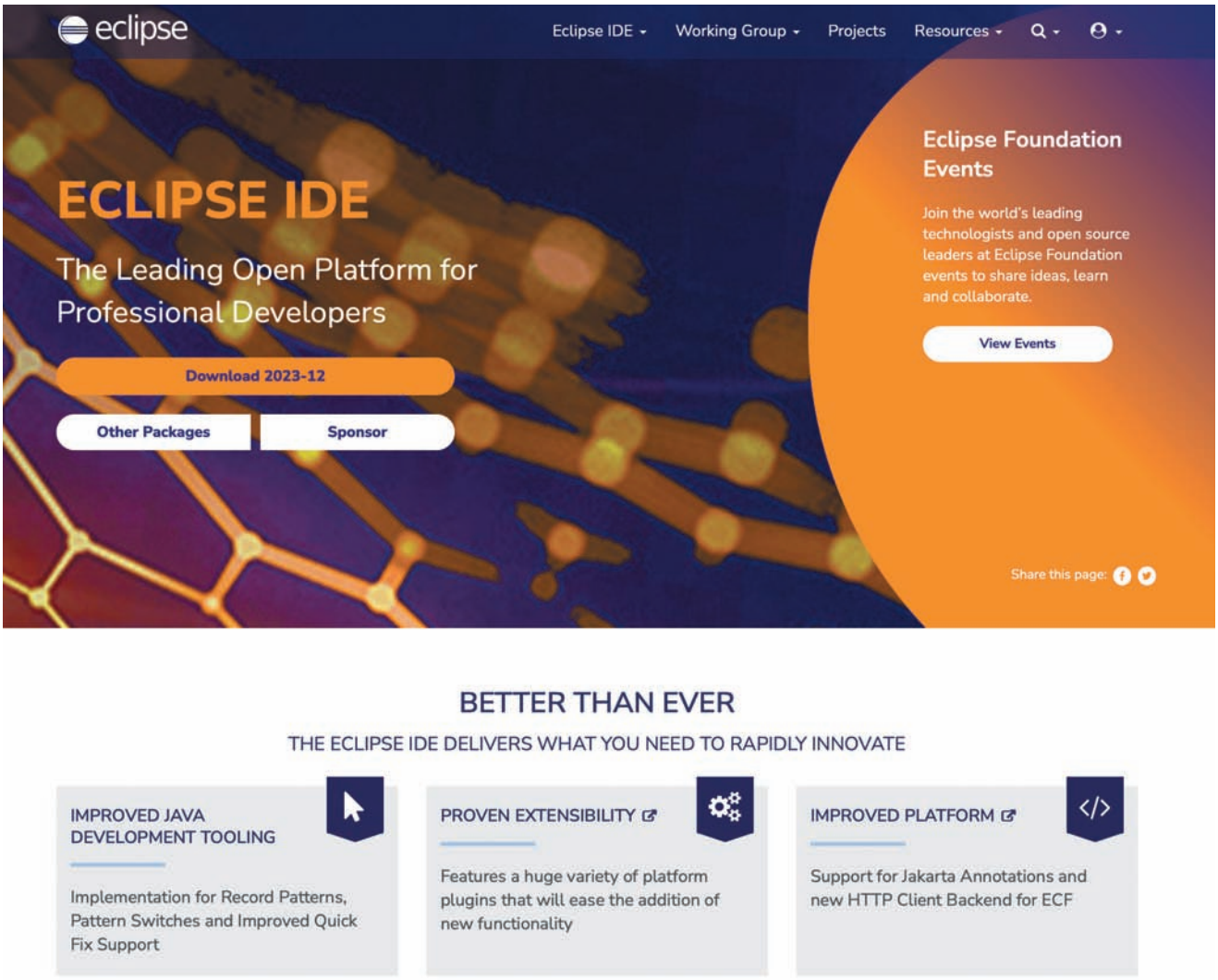
In addition to commercial packages, programmers can use open-source IDEs such as Eclipse, a free and extensible IDE primarily used for Java application development. As illustrated in Figure 11-25, Eclipse can also support various programming languages such as C, C++, PHP, and Python. Initially developed by IBM and now managed by the Eclipse Foundation, Eclipse offers comprehensive tools for software development, including a source code editor, build automation tools, a debugger, and a graphical user interface designer for creating complex user interfaces. Eclipse's plugin-based architecture allows it to be adapted to various software development and testing tasks, making it a versatile tool for developers. It is particularly favored for Java development, offering robust support for Java project management, debugging, and code refactoring.

11.6.3 Online Learning Resources

The rise of free online coding resources has dramatically changed how people worldwide learn programming. These platforms offer a range of learning materials, from interactive tutorials for beginners to advanced courses for professionals looking to improve their skills. By integrating these online resources with traditional college education, learners can pursue their interests at their own pace while enhancing their formal studies with real-world applications. This blend of academic and self-directed learning creates a comprehensive educational experience, preparing individuals more effectively for the dynamic needs of the tech industry.

Platforms like Codecademy (shown in Figure 11-26) and FreeCodeCamp (shown in Figure 11-27) offer interactive courses that cover a broad spectrum of programming languages. These online resources allow learners to engage with material actively, ensuring

Figure 11-25 Eclipse is a free, open-source IDE known for its Java support.
Source: Eclipse Foundation AISBL



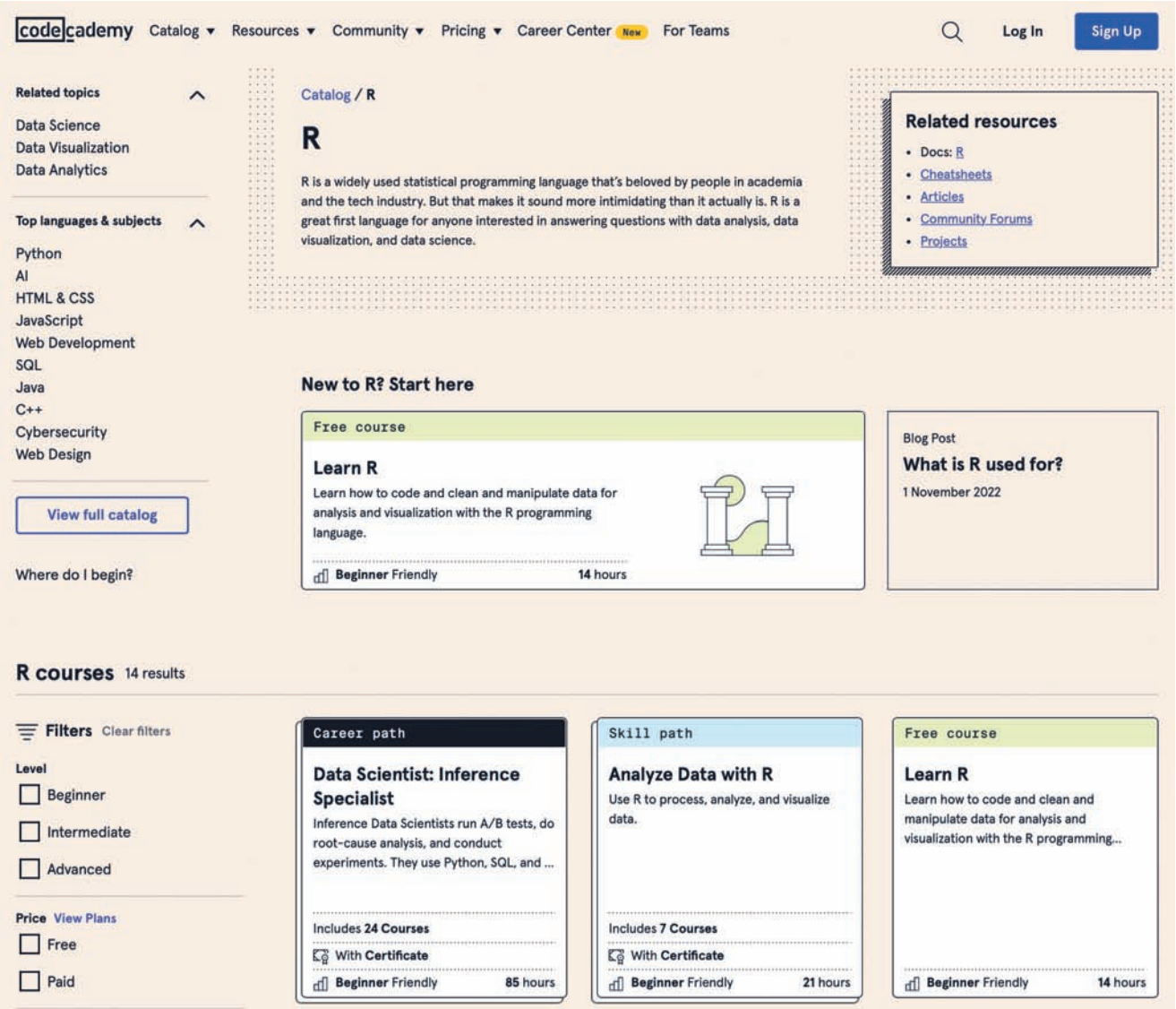
a practical understanding of concepts. By offering lessons in languages like R, they cater specifically to the growing field of data science. Users benefit from the flexibility to learn at their own pace and balance learning with other commitments. This approach to education emphasizes hands-on practice, reinforcing learning through the direct application of skills.

These platforms distinguish themselves from websites like Coursera (Figure 11-28) and edX (Figure 11-29) by focusing primarily on interactive coding exercises and projects, catering mainly to beginners and those seeking a practical approach to learning programming and web development. For example, FreeCodeCamp offers a free, comprehensive curriculum in full-stack web development, while Codecademy provides a variety of coding courses, blending free content with premium options that include more in-depth material. Conversely, edX and Coursera collaborate with universities to deliver a broad array of courses not limited to computer science, featuring structured formats akin to traditional online college courses with video lectures, assignments, and forums.

11.7 Testing

After coding, a programmer must test each program to ensure it functions correctly. Later, programs are tested in groups, and the development team must finally test the entire system. The first step is to compile the program using a CASE tool or a language compiler.

Figure 11-26 Codecademy offers interactive courses across various programming languages.
Source: Codecademy LLC



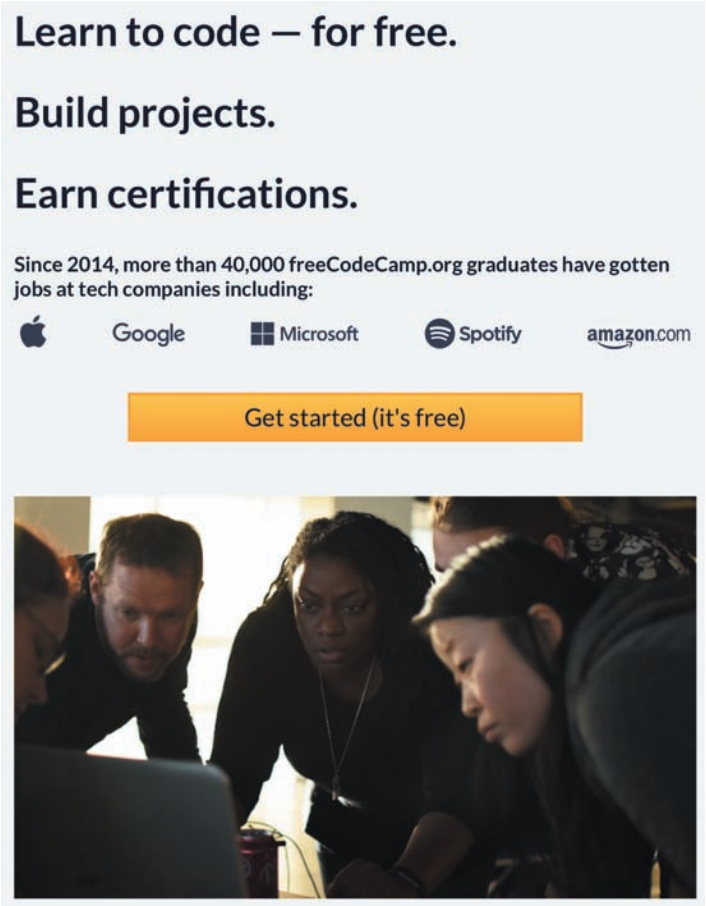
This process detects **syntax errors**, which are language grammar errors. The programmer corrects the errors until the program executes properly.

Next, the programmer desk-checks the program. **Desk checking** reviews the program code to spot **logic errors** that produce incorrect results. This process can be performed by the person who wrote the program or by other programmers. Many organizations require a more formal type of desk checking called a structured walkthrough, or code review.

Typically, three to five IT staff members participate in code review. The group usually consists of project team members, including other programmers and analysts who did not work on the project. The objective is to have a peer group identify errors, apply quality standards, and verify that the program meets the requirements of the system design specification. Errors found during a structured walkthrough are easier to fix while the code is being developed.

In addition to analyzing logic and program code, the project team usually holds a session with users called a **design walkthrough** to review the interface with a cross-section of people working with the new system and ensure that all necessary features have been included. This continues the modeling and prototyping effort that began early in the systems development process.

Figure 11-27 FreeCodeCamp is a community of people worldwide learning to code together. They're a 501(c)(3) public charity.
Source: Free Code Camp, Inc.



The next step in application development is initiating a unit, integration, and system testing sequence, as shown in Figure 11-30.

11.7.1 Unit Testing

The testing of an individual program or module is called **unit testing**. The objective is to identify and eliminate execution errors that could cause the program to terminate abnormally and logic errors that could have been missed during desk checking.

Test data should contain both correct data and erroneous data and should test all possible situations that could occur. For example, for a field that allows a range of numeric values, the test data should contain minimum values, maximum values, values outside the acceptable range, and alphanumeric characters. During testing, programmers can use software tools to determine the location and potential causes of program errors.

During unit testing, programmers must test programs that interact with other programs and files individually before integrating them into the system. This requires a technique called stub testing. In **stub testing**, the programmer simulates each program outcome or result and displays a message indicating whether the program executed successfully. Each stub represents an entry or exit point that will be linked later to another program or data file.

To obtain an independent analysis, someone other than the programmer who wrote the program usually creates the test data and reviews the results. Systems analysts frequently create test data during the systems design phase as part of an overall test plan. A **test plan** consists of detailed procedures that specify how and when the testing will be performed, who will participate, and what test data will be used. A comprehensive test plan should include scenarios for every situation the program could encounter.

Figure 11-28 Coursera is an online learning platform that offers various courses, specializations, and degrees from universities and educational institutions worldwide, covering multiple subjects, including technology, business, and the arts.
Source: Coursera Inc.

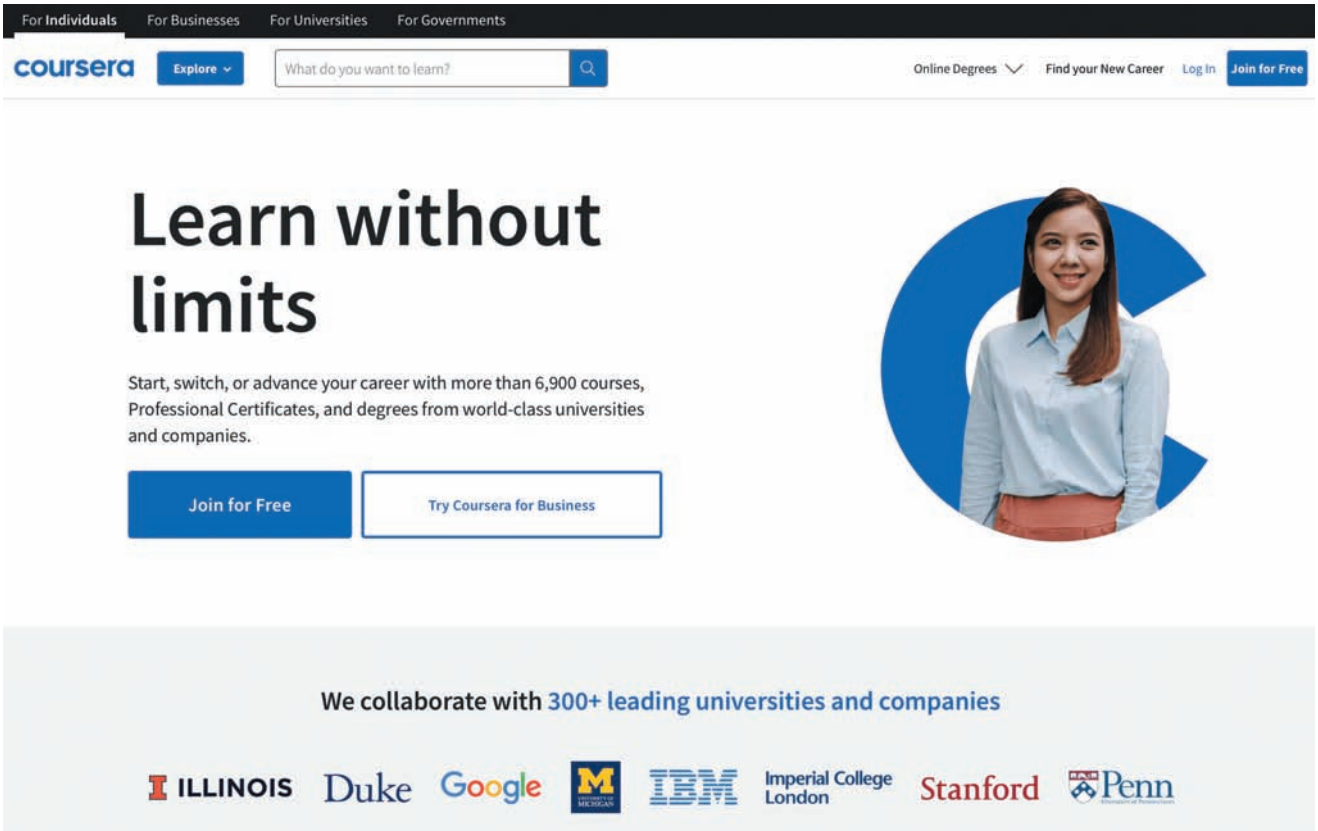


Figure 11-29 The edX platform was founded by Harvard University and the Massachusetts Institute of Technology as an experiment to make the world's best education available to everyone. It is now an online learning platform from the digital education company 2U, Inc.
Source: edX LLC

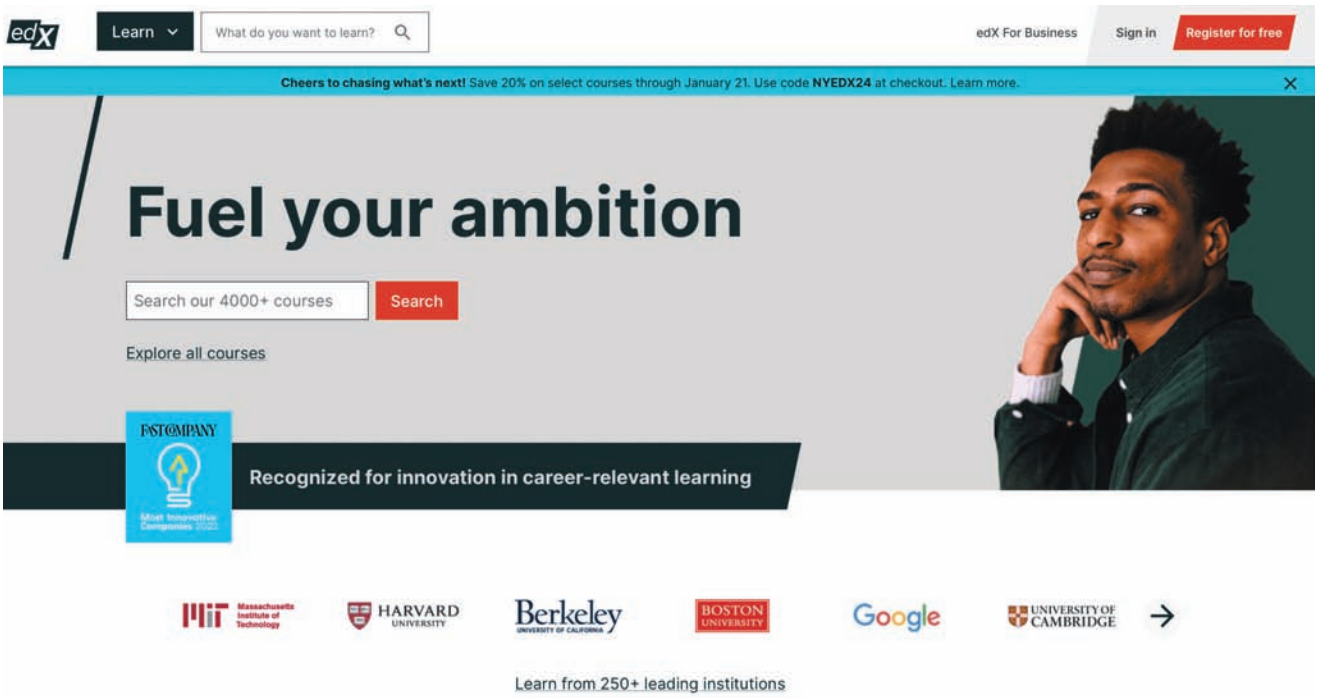
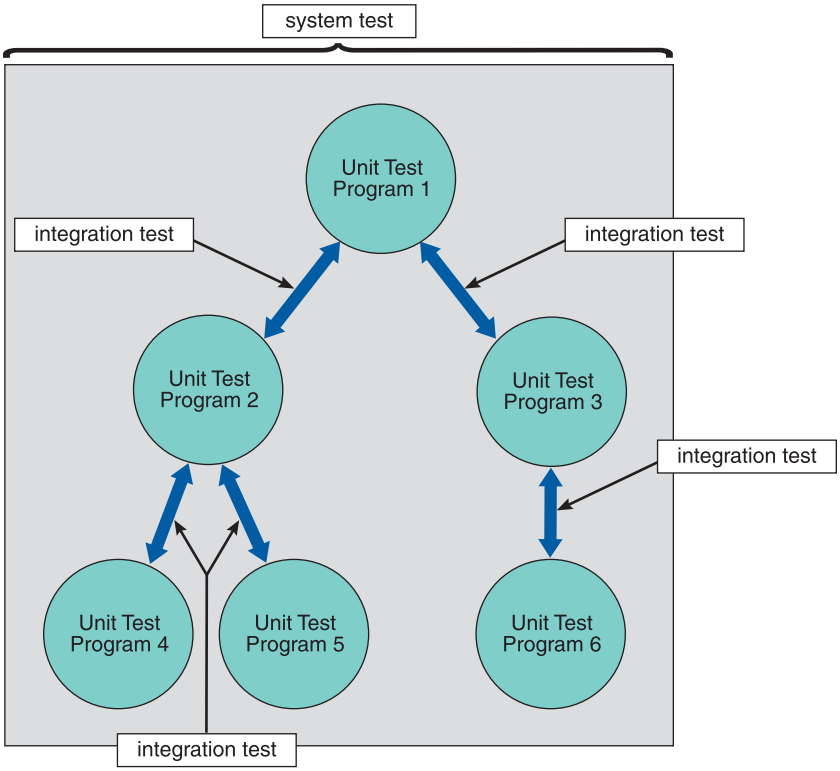


Figure 11-30 The first step in testing is unit testing, followed by integration and system testing.



Regardless of who creates the test plan, the project manager or a designated analyst also reviews the final test results. Some organizations also require users to approve final unit test results.

11.7.2 Integration Testing

Integration testing is testing two or more programs that depend on each other. For example, consider an information system that checks and validates customer credit status and a separate program that updates data in the customer master file. The output from the validation program becomes input to the master file update program. Testing the programs independently does not guarantee that the data passed between them is correct. Only by performing integration testing for this pair of programs can one ensure the programs work together correctly. Figure 11-30 shows integration testing for several groups of programs. Notice that a program can have membership in two or more groups.

Systems analysts usually develop the data they use in integration testing. As is the case with all forms of testing, integration test data must consider both usual and unusual situations. For example, integration testing might pass typical records between two programs, followed by blank records, to simulate an extraordinary event or an operational problem. Test data that simulates actual conditions should be used because the interface that links the programs is being tested. A testing sequence should only move to the integration test stage if it performed properly in all unit tests.

11.7.3 System Testing

After completing integration testing, **system testing** is performed on the entire information system, as shown in Figure 11-30. A system test includes all likely processing situations. It is intended to assure users, developers, and managers that the program meets all specifications and that all necessary features have been included.

During a system test, users enter data, including samples of actual (or live) data, perform queries, and produce reports to simulate actual operating conditions. Users and the IT project development team verify all processing options and outputs to ensure the system functions correctly. Commercial software packages must undergo system testing similar to that for in-house developed systems, although unit and integration testing usually are not performed.

Regardless of how the system was developed, system testing has the following primary objectives:

- Perform a final test of all programs.
- Verify that the system will handle all input data, both valid and invalid.
- Ensure that the IT staff has the documentation and instructions needed to operate the system correctly and that the backup and restart capabilities of the system are adequate. (The details of creating this documentation are discussed later in this chapter.)
- Demonstrate that users can interact with the system successfully.
- Verify that all system components are appropriately integrated and that actual processing situations will be handled correctly.
- Confirm that the information system can handle predicted volumes of data in a timely and efficient manner.

Successful completion of system testing is the key to user and management approval, which is why system tests are sometimes called **acceptance tests**. Final acceptance tests, however, are performed during systems installation and evaluation with actual user data, as described later in this chapter.

How much testing is necessary depends on the situation and requires good judgment and input from other IT staff members, users, and management. Unfortunately, IT project managers often are pressured to finish testing quickly and hand the system over to users. Common reasons for premature or rushed testing are user demands, tight systems development budgets, and demands from top management to finish projects early. Those pressures hinder the testing process and harm the final product's quality.

Testing can be a cost-effective means of providing a high-quality product. Every error caught during testing eliminates potential expenses and operational problems. No system, however, is 100% error-free. Often, errors go undetected until the system becomes operational. Errors that affect the integrity or accuracy of data must be corrected immediately. Minor errors, such as typographical errors in the user interface, can be rectified later.

Some users want a finished system, while others realize that minor changes can be treated as maintenance items after the system is operational. In the final analysis, a decision must be made whether to postpone system installation if problems are discovered. If conflicting views exist, management will decide whether to install the system after discussing the options.

Case in Point 11.1

Your Move, Inc.

You work at Your Move, Inc., a large retailer specializing in games of all kinds. The company is in the final stages of developing a new inventory management system, and the IT manager wants you to handle the testing.

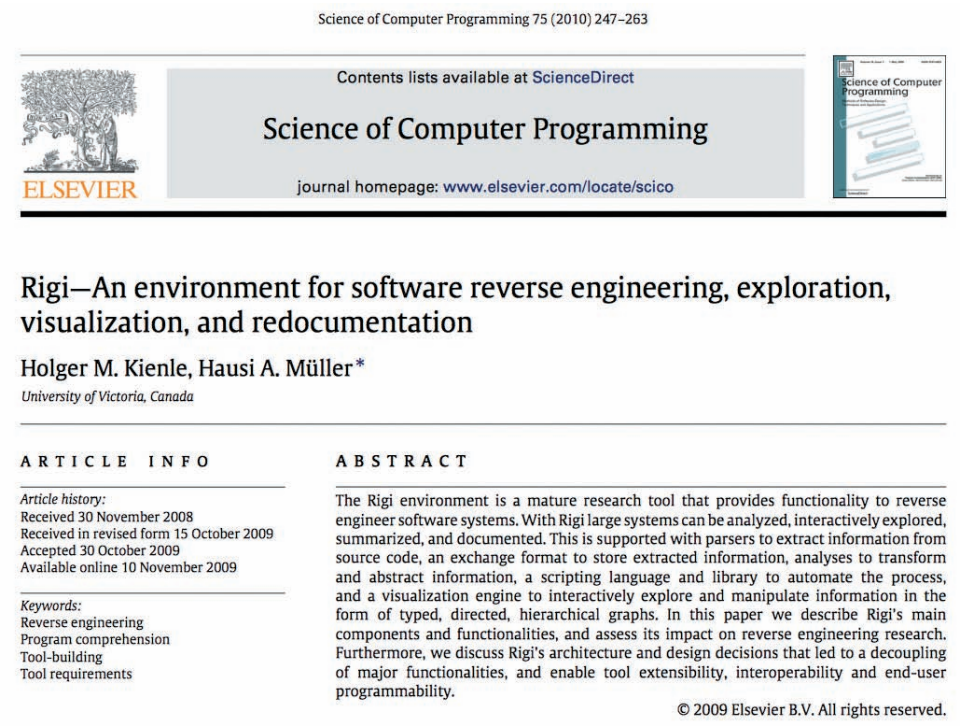
"Be sure you put many errors into the test data," the IT manager said. "Users are bound to make mistakes, and we need to design built-in safeguards that will catch the mistakes and either fix them automatically or alert the user to the problem."

Of course, this comment makes a lot of sense, but you've never done this before and wonder how to proceed. Should you try to invent every possible data error? How will you know that you've thought of every situation that could occur? Consider the problem, develop an approach, and write your plan in a brief memo.

11.8 Documentation

Documentation describes an information system and helps the users, managers, and IT staff who must interact with it. Accurate documentation can reduce system downtime, cut costs, and speed up maintenance tasks. Figure 11-31 shows an example of the Rigi research environment that can automate the documentation process and help software developers generate accurate, comprehensive reference material through detailed source code analysis.

Figure 11-31 Rigi is a research environment that uses reverse engineering technology to provide software redocumentation capabilities that support program understanding.
Source: Elsevier © 2009



Science of Computer Programming 75 (2010) 247–263

Contents lists available at ScienceDirect

Science of Computer Programming

journal homepage: www.elsevier.com/locate/scico

ELSEVIER

Rigi—An environment for software reverse engineering, exploration, visualization, and redocumentation

Holger M. Kienle, Hausi A. Müller*

University of Victoria, Canada

ARTICLE INFO

Article history:
Received 30 November 2008
Received in revised form 15 October 2009
Accepted 30 October 2009
Available online 10 November 2009

Keywords:
Reverse engineering
Program comprehension
Tool-building
Tool requirements

ABSTRACT

The Rigi environment is a mature research tool that provides functionality to reverse engineer software systems. With Rigi large systems can be analyzed, interactively explored, summarized, and documented. This is supported with parsers to extract information from source code, an exchange format to store extracted information, analyses to transform and abstract information, a scripting language and library to automate the process, and a visualization engine to interactively explore and manipulate information in the form of typed, directed, hierarchical graphs. In this paper we describe Rigi's main components and functionalities, and assess its impact on reverse engineering research. Furthermore, we discuss Rigi's architecture and design decisions that led to a decoupling of major functionalities, and enable tool extensibility, interoperability and end-user programmability.

© 2009 Elsevier B.V. All rights reserved.

Documentation is essential for successful system operation and maintenance. In addition to supporting a system's users, accurate documentation is critical for IT staff members who must modify the system, add a new feature, or perform maintenance. Documentation includes program, system, operations, and user documentation.

11.8.1 Program Documentation

Program documentation describes all program modules' inputs, outputs, and processing logic. The program documentation process starts in the systems analysis phase and continues during implementation. Analysts prepare overall documentation, such as process descriptions and report layouts, early in the SDLC. This documentation guides programmers in constructing modules supported by internal and external comments and descriptions that can be easily understood and maintained. A systems analyst usually verifies that program documentation is complete and accurate.

System developers also use **defect tracking software**, sometimes called **bug tracking software**, to document and track program defects, code changes, and replacement code, called **patches**.

11.8.2 System Documentation

System documentation describes the system's functions and how they are implemented. System documentation includes data dictionary entries, DFDs, object models, screen layouts, source documents, and the system request that initiated the project. System documentation

is necessary reference material for the programmers and analysts who must support and maintain the system.

Most system documentation is prepared during the systems analysis and design phases. During the systems implementation phase, an analyst must review prior documentation to verify that it is complete, accurate, and up to date, including any changes made during the implementation process. For example, the analyst must update the documentation if a screen or report has been modified. Updates to the system documentation should be made promptly to prevent oversights.

11.8.3 Operations Documentation

The analyst must prepare documentation for the IT group that supports centralized operations if the information system environment involves a mainframe or centralized servers. A mainframe installation might require the scheduling of batch jobs and the distribution of printed reports. In this environment, the IT operations staff serves as the first point of contact when users experience problems with the system.

Operations documentation contains all the information needed for processing and distributing online and printed output. Typical operations documentation includes the following information:

- Program, systems analyst, programmer, and system identification
- Scheduling information for printed output, such as report run frequency and deadlines
- Input files and where they originate; output files and destinations
- Email and report distribution lists
- Special forms that are required, including online forms
- Error and informational messages to operators and restart procedures
- Special instructions, such as security requirements

Operations documentation should be clear, concise, and available online. If the IT department has an operations group, the documentation should be reviewed with them to identify any problems. Keeping the operations group informed at every phase of the SDLC allows operations documentation to be developed as the project progresses.

11.8.4 User Documentation

User documentation consists of instructions and information for users interacting with the system and includes user manuals, help screens, and online tutorials. Programmers or systems analysts usually create program documentation and system documentation. To produce effective and clear user documentation—and hence have a successful project—someone with expert skills in this area is needed. The skill set required to create documentation usually differs from that needed to develop a system. This is particularly true in online documentation, which needs to coordinate with print documentation and intranet and Internet information. Technical writing requires specialized skills, and competent technical writers are valuable IT team members.

Just as a system cannot be thrown together in several days, documentation cannot be added at the end of the project. While this has always been true of traditional user documentation, it is an even more critical issue now that online help and context-sensitive help are often used. Context-sensitive help is part of the program and must be tested, too.

Systems analysts usually prepare documentation to help users learn the system. In larger companies, a technical support team that includes technical writers might assist in preparing user documentation and training materials. Regardless of the delivery method, user documentation must be clear, understandable, and readily accessible to users at all levels.

User documentation includes the following:

- A system overview that clearly describes all major system features, capabilities, and limitations
- Descriptions of source document content, preparation, processing, and samples

- An overview of menu and data entry screen options, contents, and processing instructions
- Examples of reports that are produced regularly or available at the user's request, including samples
- Security and audit trail information
- Explanations of responsibility for specific input, output, or processing requirements
- Procedures for requesting changes and reporting problems
- Examples of exceptions and error situations
- Frequently asked questions (FAQs)
- An explanation of how to get help and procedures for updating the user manual

11.8.5 Online Documentation

Most users now prefer **online documentation**, which provides immediate help when they have questions or encounter problems. Many users are accustomed to context-sensitive help screens, hints and tips, hypertext, on-screen demos, and other user-friendly features commonly found in popular software packages; they expect the same kind of support for in-house developed software.

If the system will include online documentation, that fact needs to be identified as one of the system requirements. If someone other than the systems analysts will create the documentation, that person or group needs to be involved as early as possible to become familiar with the software and begin developing the required documentation and support material. In addition, system developers must determine whether the documentation will be available from within the program or as a separate entity in the form of a tutorial, slide presentation, reference manual, or website. If necessary, links within the program should take the user to the appropriate documentation.

Effective online documentation is an essential productivity tool because it empowers users and reduces the time IT staff must spend providing telephone, email, or face-to-face assistance. Interactive tutorials are especially popular with users who like to learn by doing, and visual impact is significant. The use of YouTube as a host for tutorial videos has become commonplace.

In addition to program-based assistance, the Internet offers an entirely new level of comprehensive, immediate support. Many programs include links to websites, intranet sites, and Internet-based technical support. For example, the Cisco Systems website shown in Figure 11-32 offers a wide range of support services and social media links that allow Cisco users to collaborate and share their knowledge.

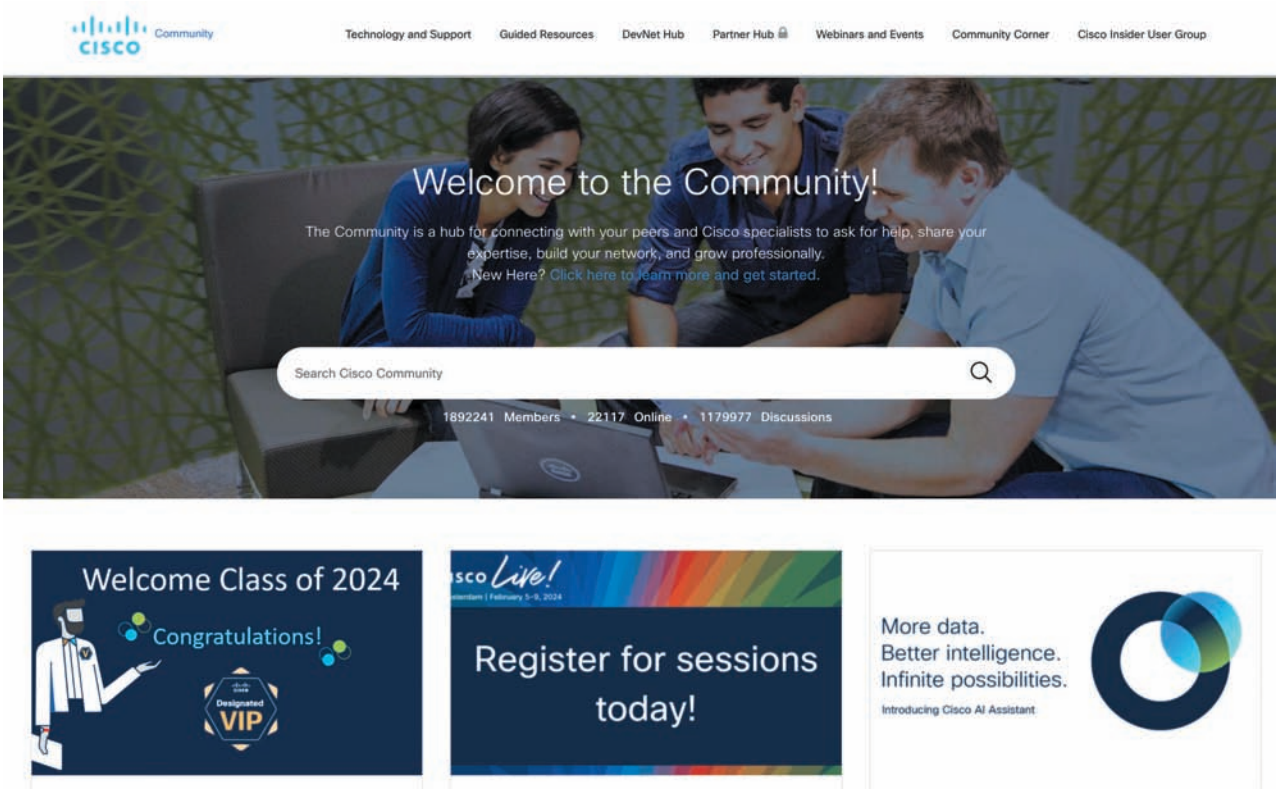
Although online documentation is essential, written documentation material is also valuable, especially in training users and for reference purposes. Systems analysts or technical writers usually prepare the manual, but many companies invite users to review the material and participate in the development process.

No matter what form of user documentation the system requires, remember that it can take a lot of time to develop. The time between finishing software coding and when a complete package—including documentation—can be released to users depends on how well the documentation is planned. If the completion of the project includes providing user documentation, this issue needs to be addressed from the very beginning of the project. Determining what the user documentation requirements are and ascertaining who will complete the documents is critical to a timely release of the project.

Neglecting user documentation issues until after the program is complete often leads to one of two things: (1) the documentation will be thrown together quickly to get it out the door on time, and it more than likely will be inadequate, or (2) it will be done correctly, and the product release will be delayed considerably.

User training is typically scheduled when the system is installed. The training sessions offer an ideal opportunity to distribute the user manual and explain the procedures for updating it in the future. Training for users, managers, and IT staff is described later in this chapter.

Figure 11-32 The Cisco Support Community invites users to contribute valuable experience and documentation using social media.
Source: Cisco Systems Inc.



11.9 Installation

After system testing is complete, the results are presented to management. The test results should be described, the status of all required documentation updated, and input from users who participated in system testing summarized. Detailed time schedules, cost estimates, and staffing requirements for making the system fully operational should also be provided. If system testing produces no technical, economic, or operational problems, management determines a schedule for system installation.

The following system installation tasks are performed for every information systems project, whether the application is developed in-house or purchased as a commercial package:

- Prepare a separate operational and test environment.
- Perform system changeover.
- Perform data conversion.
- Provide training for users, managers, and IT staff.
- Carry out post-implementation tasks.

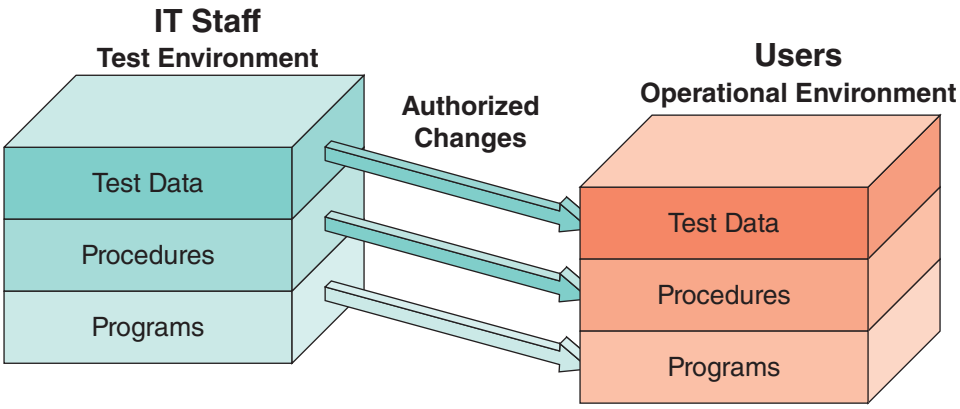
11.9.1 Operational and Test Environments

Recall that an environment, or platform, is a specific combination of hardware and software. The environment for the actual system operation is called the **operational environment** or **production environment**. The environment analysts and programmers use to develop and maintain programs is called the **test environment**. A separate test environment is necessary to maintain system security and integrity and protect the operational environment. Typically, the test environment resides on a limited-access workstation or server located in the IT department.

Access to the operational environment is limited to users and must be strictly controlled. Systems analysts and programmers should not have access to it except to correct a system problem or to make authorized modifications or enhancements. Otherwise, IT department members have no reason to access the day-to-day operational system.

The test environment for an information system contains copies of all programs, procedures, and test data files. Before making any changes to an operational system, the analyst must verify them in the test environment and obtain user approval. Figure 11-33 shows the differences between test environments and operational environments.

Figure 11-33 The test environment versus the operational environment. Notice that access to the test environment is limited to IT staff, while the operational environment is restricted to users.



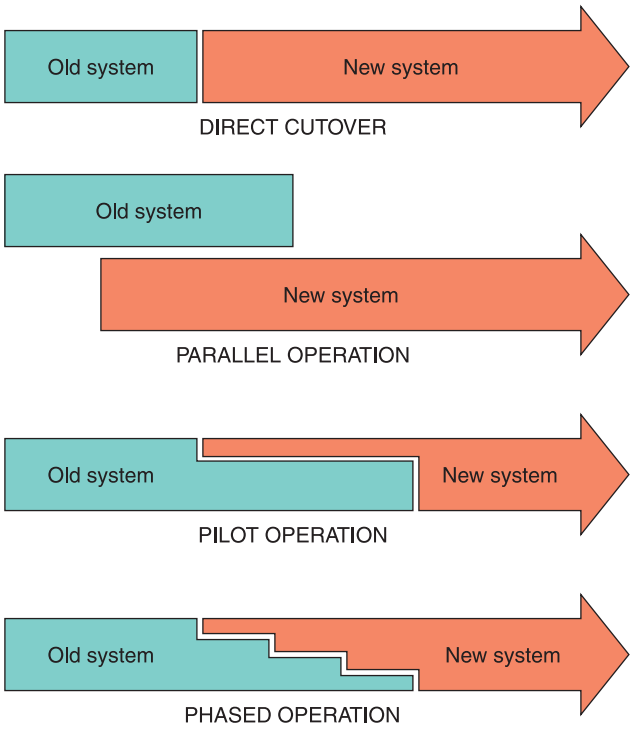
An effective testing process is essential to ensure product quality. Every experienced systems analyst can tell a story about an innocent program change that was introduced without being appropriately tested. In those stories, the harmless change invariably ends up causing some unexpected and unwanted changes to the program. After any modification, the same acceptance tests run when the system was developed should be repeated. By restricting access to the operational area and performing all tests in a separate environment, the system can be protected, and problems that could damage data or interrupt operations can be avoided.

The operational environment includes hardware and software configurations and settings, system utilities, telecommunications resources, and other components affecting system performance. Because network capability is critically important in a client/server environment, connectivity, specifications, and performance must be verified before installing applications. All communication features in the test environment should be checked carefully and again after loading the applications into the operational environment. The documentation should identify all network specifications and settings, including technical and operational requirements for communications hardware and software. If network resources must be built or upgraded to support the new system, the platform must be tested rigorously before system installation begins.

11.9.2 System Changeover

System changeover is putting the new information system online and retiring the old system. Changeover can be rapid or slow, depending on the method. The four changeover methods are direct cutover, parallel, pilot, and phased operation. Direct cutover is like throwing a switch that instantly changes from the old system to the new one. Parallel operation, the slowest method, requires that both systems run simultaneously for a specified period. The other methods, pilot and phased operation, fall between direct cutover and parallel operation. Figure 11-34 illustrates the four system changeover methods.

Figure 11-34 The four system changeover methods.



Direct Cutover

The **direct cutover** approach causes the changeover from the old system to the new system to occur immediately when the new system becomes operational. Direct cutover is usually the least expensive changeover method because the IT group must operate and maintain only one system at a time.

Direct cutover, however, involves more risk than other changeover methods. Regardless of how thoroughly and carefully testing and training are conducted, some difficulties can arise when the system goes into operation. Problems can result from data situations that were not tested or anticipated or errors caused by users or operators. A system also can encounter difficulties because live data typically occurs in much larger volumes than test data.

Although initial implementation problems can occur with all four changeover methods, they are most significant when using the direct cutover approach. Detecting minor errors is also more difficult with direct cutover because users cannot verify current output by comparing it to output from the old system. Major errors can cause a system process to terminate abnormally, and with the direct cutover method, reverting to the old system as a backup option is impossible.

Companies often choose the direct cutover method for implementing commercial software packages because they feel the packages involve less risk of total system failure. Commercial software is certainly not risk-free, but the software vendor usually maintains an extensive knowledge base and can supply reliable, prompt fixes for most problems.

For systems developed in-house, most organizations use direct cutover only for noncritical situations. However, direct cutover might be the only choice if the operating environment cannot support the old and new systems or if the old and new systems are incompatible.

Timing is crucial when using a direct cutover strategy. Most systems operate on weekly, monthly, quarterly, and yearly cycles. For example, consider a payroll system that produces output every week. However, some employees are paid twice a month, so the system operates semimonthly. Monthly, quarterly, and annual reports also require the system to produce output at the end of every month, quarter, and year. When a cyclical information system is implemented in the middle of any cycle, complete processing for the entire cycle requires information from both the old and the new systems. Cyclical information systems are usually

converted using the direct cutover method at the beginning of a quarter, calendar year, or fiscal year to minimize the information required from two systems.

Parallel Operation

The **parallel operation** changeover method requires the old and new information systems to operate fully for a specified period. Data is input into both systems, and the output generated by the new system is compared with the equivalent output from the old system. When users, management, and the IT group are satisfied that the new system operates correctly, the old system is terminated.

The most apparent advantage of parallel operation is lower risk. If the new system does not work correctly, the company can use the old system as a backup until appropriate changes are made. It is much easier to verify that the new system is working correctly under parallel operation than under direct cutover because the output from both systems is compared and verified during similar operations.

Parallel operation, however, does have some disadvantages. First, it is the costliest changeover method. The company pays for both systems during the parallel period because the old and new systems are in full operation. Users must work in both systems, and the company might need temporary employees to handle the extra workload. In addition, running both systems might place a burden on the operating environment and cause processing delays.

Parallel operation is impractical if the old and new systems are technically incompatible or the operating environment cannot support both systems. Parallel operation is also inappropriate when the two systems perform different functions or if the new system involves a new method of business operations.

Pilot Operation

The **pilot operation** changeover method involves implementing the new system at a selected company location. For instance, a new sales reporting system might be implemented in only one branch office or a new payroll system in only one department. The **pilot site** is the first group using the new system in these examples. During pilot operation, the old system continues to operate for the entire organization, including the pilot site. After the system proves successful at the pilot site, it is implemented in the rest of the organization, usually using the direct cutover method. Therefore, a pilot operation combines parallel operation and direct cutover methods.

Restricting the implementation to a pilot site reduces the risk of system failure compared with a direct cutover method. Operating both systems for only the pilot site is less expensive than a parallel operation for the entire company. In addition, if a parallel approach to complete the implementation is used later, the changeover period can be much shorter if the system proves successful at the pilot site.

Phased Operation

The **phased operation** changeover method allows the new system to be implemented in stages, or modules. For example, instead of implementing a new manufacturing system all at once, the materials management subsystem is installed first, then the production control subsystem, then the job cost subsystem, and so on. Each subsystem can be implemented using the other three changeover methods.

Analysts sometimes need clarification on phased and pilot operation methods. Both methods combine direct cutover and parallel operation to reduce risks and costs. With the phased operation, however, only a part of the system is given to all users, while pilot operation provides the entire system to only some users.

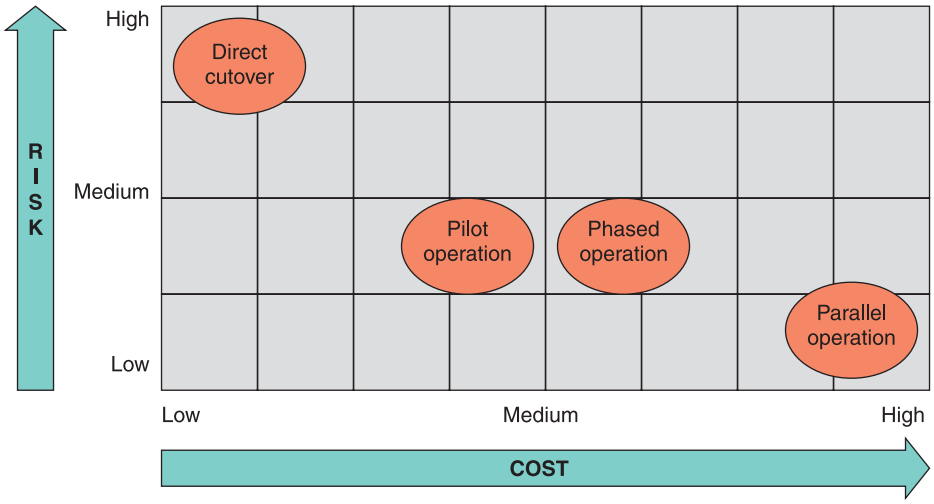
One advantage of a phased approach is that the risk of errors or failures is limited to the implemented module only. For instance, if a new production control subsystem fails to operate correctly, that failure might not affect the new purchasing or shop floor control subsystem.

Phased operation is less expensive than full parallel operation because the analyst must work with only one part of the system at a time. However, a phased approach is only possible

if the system can be separated easily into logical modules or segments. In addition, if the system involves many separate phases, phased operation can cost more than a pilot approach.

Figure 11-35 shows that each changeover method has risk and cost factors. A systems analyst must weigh the advantages and disadvantages of each technique and recommend the best choice in each situation. The final changeover decision will be based on input from the IT staff, users, and management—and the choice must reflect the nature of the business and the degree of acceptable risk.

Figure 11-35 Relative risk and cost characteristics of the four changeover methods.



Case in Point 11.2

Global Cooling

You are a systems analyst at Global Cooling, a leading manufacturer of air conditioning units. You are leading a team that is developing a new production scheduling system. The project is now in the application development stage, and you are in the final stages of integration testing. Your supervisor is eager to implement the new application ahead of schedule and asked if you could trim system testing from two weeks to three days and use a direct cutover method instead of the parallel changeover method that was initially planned. Write a brief memo expressing your views.

11.9.3 Data Conversion

Data conversion is an integral part of the system installation process. During **data conversion**, existing data is loaded into the new system. Depending on the system, data conversion can be done before, during, or after the operational environment is complete. A data conversion plan should be developed as early as possible, and the conversion process should be tested when the test environment is created.

When a new system replaces an existing system, the data conversion process should be automated. The old system might be capable of exporting data in an acceptable format for the new system or a standard format, such as ASCII or ODBC. ODBC (Open Database Connectivity) is an industry-standard protocol allowing various vendor DBMSs to interact and exchange data. Most database vendors provide ODBC drivers, which are a form of middleware. As discussed elsewhere in the book, middleware connects dissimilar applications and enables them to communicate.

A program must be developed to extract and convert the data to an acceptable format if a standard format is unavailable. Data conversion is more difficult when the new system replaces a manual system because all data must be entered manually unless scanned. Even when the data conversion is automated, a new system often requires additional data items, which might need manual entry.

Strict input controls should be maintained when data is highly vulnerable during conversion. System control measures should be in place and operational to protect data from unauthorized access and to help prevent erroneous input.

Even with careful data conversion and input controls, some errors will occur. For example, duplicate customer records or inconsistent part numbers might have been tolerated by the old system but will cause the new system to crash. Most organizations require that users verify all data, correct all errors, and supply every missing item during conversion. Although the process can be time-consuming and expensive, the new system must have accurate, error-free data.

11.9.4 Training

A system can only succeed with proper software, hardware, or manufacturing training. A successful information system requires user, manager, and IT staff member training. The success of the entire systems development effort can depend on whether people understand the system and know how to use it effectively.

Training Plan

A **training plan** should be considered early in the systems development process. Consider how to use the material in future training sessions as documentation is created. When the system is implemented, providing the proper training for the right people at the right time is essential. The first step is identifying who should receive training and what training is needed. The organization should be carefully examined to determine how the system will support business operations and who will be involved or affected. Figure 11-36 shows specific training topics for users, managers, and IT staff. Notice that each group needs a mix of general background and detailed information to understand and use the system.

As shown in Figure 11-36, the three main groups for training are users, managers, and IT staff. Managers do not need to understand every submenu or feature. Still, they require a system overview to ensure that users are being trained properly and using the system correctly. Similarly, users need to know how to perform their day-to-day job functions. Still, they do not need to see how the company allocates system operational charges among user departments. IT staff people need the most information. To support the new system, they must clearly understand how it functions, how it supports business requirements, and what skills users need to operate the system and perform their tasks.

After identifying the objectives, the company must determine how to provide training. The main choices are to obtain training from vendors or outside training firms or use IT staff and other in-house resources.

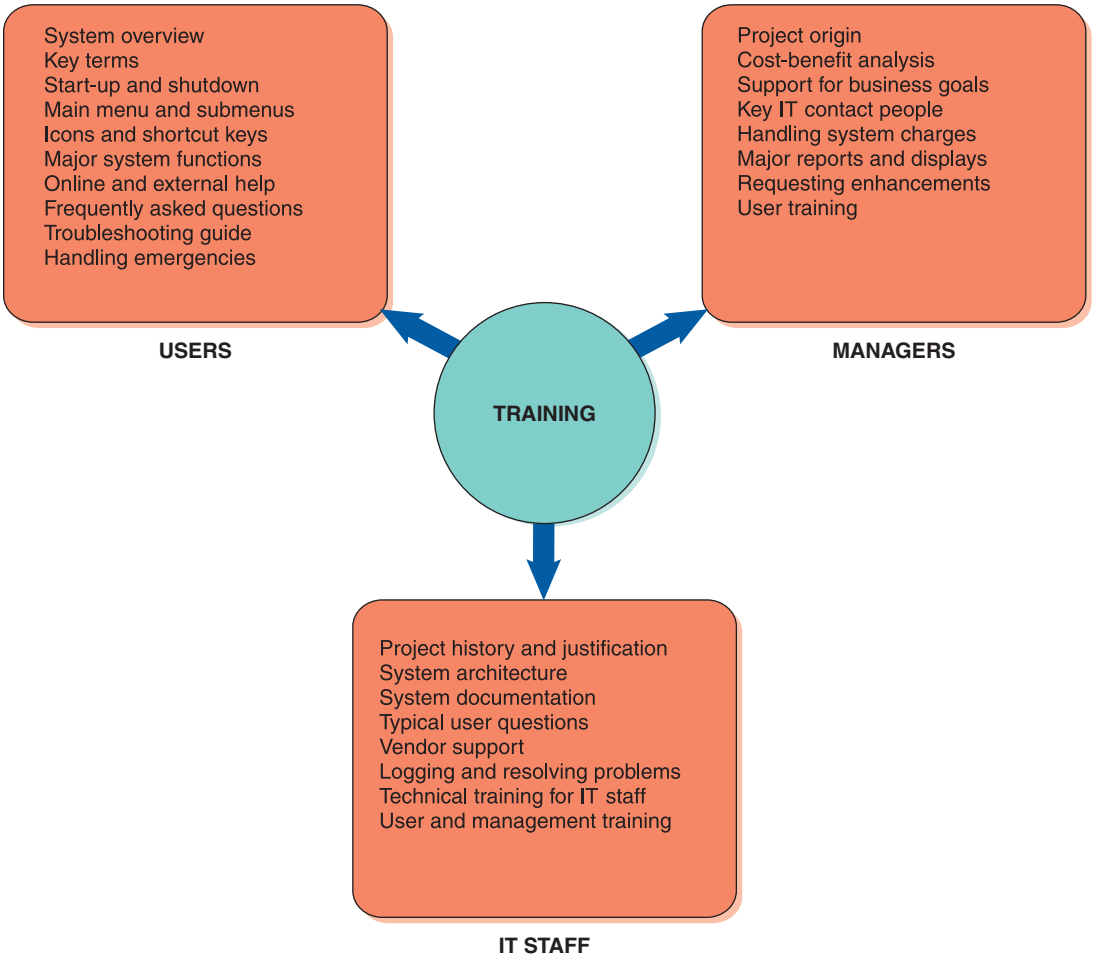
Vendor Training

If system requirements include purchasing software or hardware, vendor-supplied training should be in the RFPs and RFQs sent to potential vendors.

Many hardware and software vendors offer training programs free or at a nominal cost for the products they sell. In other cases, the company might negotiate the price for training, depending on their relationship with the vendor and the prospect of future purchases. The training usually is conducted at the vendor's site by experienced trainers who provide valuable hands-on experience. If many people need training, classes may be held at the company's location.

Vendor training often gives the best return on training dollars because it is focused on products that the vendor developed. However, vendor training is usually limited to a

Figure 11-36 Examples of training topics for three groups. Users, managers, and IT staff members have different training needs.



standard version of the vendor's software or hardware. The vendor's training may have to be supplemented with in-house training, especially if the IT staff customized the vendor's package.

Webinars, Podcasts, and Tutorials

Many vendors offer web-based training options, including webinars, podcasts, and tutorials. A **webinar** (web and seminar) is an Internet-based training session that provides an interactive experience. Most webinars are scheduled events with a group of preregistered users and an online presenter or instructor. A prerecorded webinar session can also be delivered as a webcast, which is a one-way transmission, whenever a user wants or needs training support.

A podcast is a web-based broadcast allowing users to download multimedia files to a PC or portable device. Podcasts can be prescheduled, made available on demand, or delivered as automatic updates, depending on a user's preference. An advantage of a podcast is that subscribers can access the recorded material anywhere, anytime.

A **tutorial** is a series of online interactive lessons that present material and provide a dialogue with users. Software vendors can develop tutorials, or a company's IT team can develop them. Third parties can also create them for popular software packages and sell them separately online.

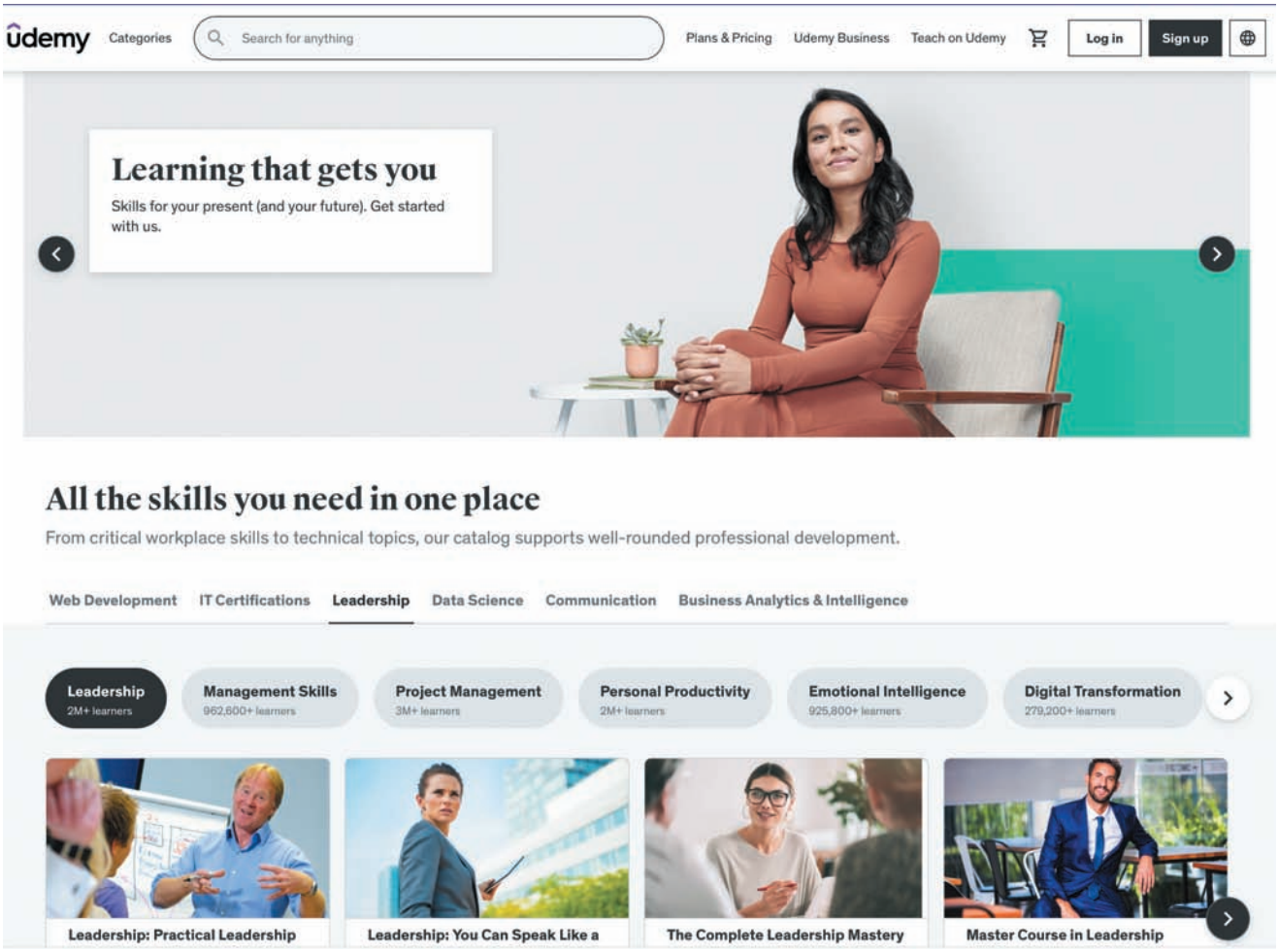
Outside Training Resources

Independent training firms can also provide in-house hardware or software training. If vendor training is not practical and the organization does not have the internal resources to perform the training, outside training consultants can be a viable alternative.

The rapid expansion of IT has produced tremendous growth in the computer training field. Many training consultants, institutes, and firms that provide standardized or customized training packages are available. Assistance can also be sought from nonprofit sources interested in training, including universities, industry associations, and information management organizations.

Many people now use one of the many sources of third-party training, such as courses on leadership from Udemy shown in Figure 11-37, to satisfy their training needs. Assistance can also be sought from nonprofit sources of training, including universities, industry associations, and information management organizations.

Figure 11-37 Udemy is a global online learning marketplace offering various courses from software development and business to personal development and hobbies, taught by industry experts and instructors.
Source: Udemy, Inc.



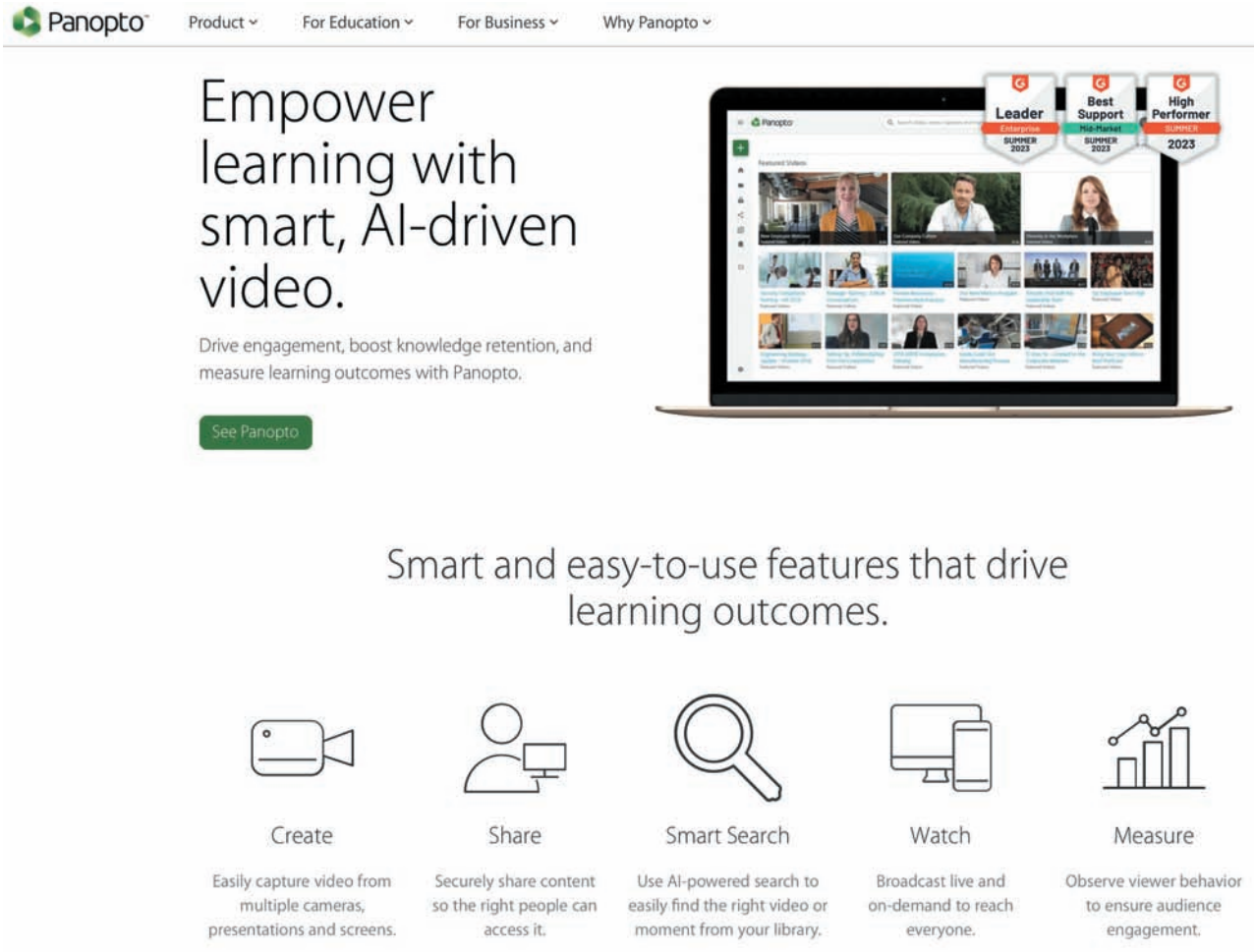
Training Tips

The IT staff and user support departments often share responsibility for developing and conducting training programs for internally developed software. If the organization has a service desk, its staff can handle user training.

Multimedia is an effective training method. Presentation software, such as Microsoft PowerPoint or Apple Keynote, can design training sessions that combine slides, animation,

and sound. Programs that capture actual keystrokes and mouse actions, such as Camtasia and Panopto (shown in Figure 11-38), can be used to replay the screens as a demonstration for users. If a media or graphic arts group is available, they can help prepare training aids such as videos, charts, and other instructional materials.

Figure 11-38 Panopto is a leading video platform for businesses and universities.
Source: Panopto.com



The following guidelines should be kept in mind when developing a training program:

- **Train people in groups, with separate training programs for distinct groups.** Group training makes the most efficient use of time and training facilities. In addition, if the group is small, trainees can learn from the questions and problems of others. A training program must address the job interests and skills of a wide range of participants. For example, IT staff personnel and users require very different information. Problems often arise when some participants have technical backgrounds and others do not. A single program will not meet everyone's needs.
- **Select the most effective place to conduct the training.** Training employees at the company's location offers several advantages. Employees incur no travel expense, they can respond to local emergencies requiring immediate attention, and training can occur in the actual environment where the system will operate. There can be some disadvantages, however. Employees distracted by telephone calls and other duties will not benefit from the training. In addition, using the organization's computer facilities for training can disrupt normal operations and limit the amount of actual hands-on training.

- **Provide for learning by hearing, seeing, and doing.** Some people learn best from lectures, discussions, and question-and-answer sessions. Others learn best from viewing demonstrations or from reading documentation and other material. Most people learn best from hands-on experience. Training that supports each type of learning should be provided.
- **Rely on previous trainees.** After one group of users has been trained, they can assist others. Users learn more quickly from co-workers with everyday experience and similar job responsibilities. Using a **train-the-trainer** strategy, knowledgeable users can be selected to conduct sessions for others. When utilizing train-the-trainer techniques, the initial training must include the application or system's use and some instruction on effectively presenting the materials.

Interactive Training

Usually, a relationship exists between training methods and costs. Training an airline pilot in a state-of-the-art simulator differs from helping corporate users learn a new inventory system. Training budgets are business decisions, and IT staff sometimes have to work with available resources rather than the resources they wish they had. Most people prefer hands-on training. However, other less expensive methods, including training manuals, printed handouts, and online materials, can be used.

If a new system is being launched and the resources to develop formal training materials are lacking, a series of dialog boxes that respond with help information and suggestions whenever users select various menu topics can still be designed. As discussed earlier in the book, an exemplary user interface includes helpful error messages and hints. However, the most effective training is interactive, self-paced, and multimedia-based.

Online Training

Regardless of the instructional method, training lessons should include step-by-step instructions for using the features of the information system. Training materials should resemble actual screens, and tasks should be typical of a user's daily work—the more realistic, the better. Video lessons are popular online, particularly on YouTube.

Sophisticated online training systems may offer interactive sessions where users can perform practice tasks and view feedback. Online training materials should also include a reference section that summarizes all options and commands, lists all possible error messages, and explains the user's actions when a problem arises.

When training is complete, many organizations conduct a full-scale test, or **simulation**, which is a dress rehearsal for users and IT support staff. In the simulation, organizations include all procedures, such as those executed only at the end of a month, quarter, or year. As questions or problems arise, the participants consult the system documentation, help screens, or each other to determine appropriate answers or actions. This full-scale test provides valuable experience and builds confidence for everyone involved with the new system.

11.9.5 Preparing for Deployment

Before deploying a new system, three steps must be meticulously undertaken to ensure its readiness and the organization's preparedness for the transition. The initial step in preparing the system for deployment entails creating a comprehensive post-implementation evaluation plan. This plan specifies the criteria and methodologies to assess the system's performance against the project's objectives once it becomes operational. Such an evaluation is crucial, as it helps pinpoint the system's strengths and areas for improvement, providing valuable insights for future projects and ongoing system optimization.

The second step involves establishing a precise schedule for the system's deployment and outlining specific dates and times for its rollout. It must also detail any last-minute checks or preparations required before the launch, ensuring everything is in place for a smooth

transition. Additionally, a well-defined plan must be in place for shifting operations from the existing system to the new one, minimizing disruptions and ensuring continuity.

The third step involves submitting a detailed final report to management that encapsulates the entire development journey and highlights significant milestones, challenges faced, and the system's readiness for deployment. As an essential communication tool, this report offers stakeholders clear visibility into the project's progress and outcomes. It ensures transparency and furnishes decision makers with the necessary insights to gauge the project's success and prepare for the system's introduction.

Post-Implementation Evaluation

A **post-implementation evaluation** assesses the overall quality of the information system. The review verifies that the new system meets specified requirements, complies with user objectives, and produces the anticipated benefits. In addition, by providing feedback to the development team, the evaluation helps improve IT development practices for future projects.

A post-implementation evaluation should examine all aspects of the development effort and the product—the developed information system. A typical evaluation includes feedback for the following areas:

- Accuracy, completeness, and timeliness of information system output
- User satisfaction
- System reliability and maintainability
- Adequacy of system controls and security measures
- Hardware efficiency and platform performance
- Effectiveness of database implementation
- Performance of the IT team
- Completeness and quality of documentation
- Quality and effectiveness of training
- Accuracy of cost–benefit estimates and development schedules

The same fact-finding techniques used to determine system requirements during the systems analysis phase can be applied in a post-implementation evaluation. When evaluating a system, the following should be done:

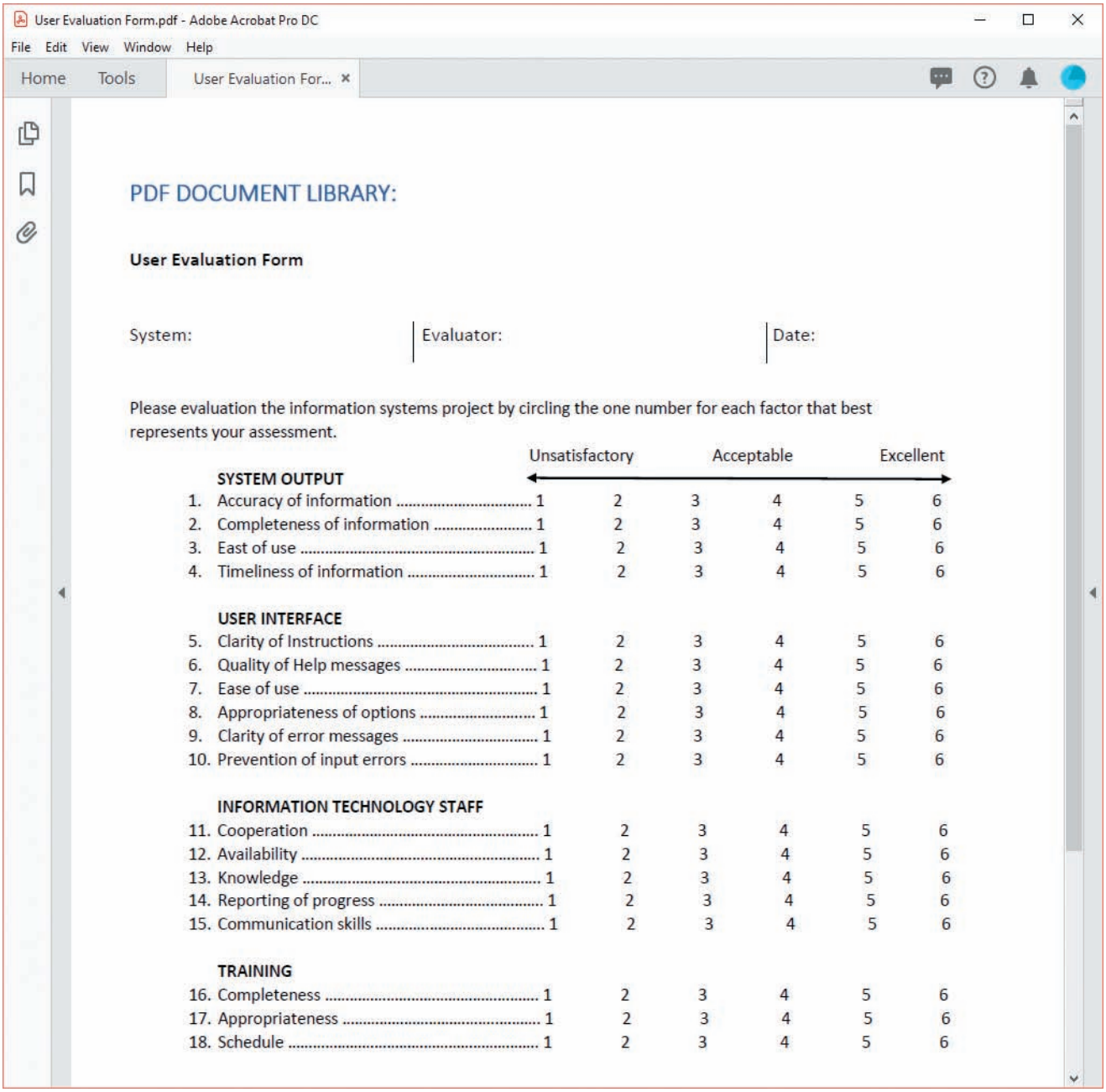
- Interview members of management and key users.
- Observe users and computer operations personnel working with the new information system.
- Read all documentation and training materials.
- Examine all source documents, output reports, and screen displays.
- Use questionnaires to gather information and opinions from many users.
- Analyze maintenance and help desk logs.

Figure 11-39 shows the first page of a sample user evaluation form for a new information system, where users evaluate 18 separate elements on a numerical scale so the results can be tabulated easily. Following that section, the form provides space for open-ended comments and suggestions.

People not directly involved in developing the system should conduct the post-implementation evaluation whenever possible. IT staff and users usually perform the evaluation, although some firms use an internal audit group or independent auditors to ensure the accuracy and completeness of the evaluation.

When to perform a post-implementation evaluation for a new system is not always clear. Users forget details of the developmental effort if too much time elapses before the evaluation.

Figure 11-39 Sample user evaluation form. The numerical scale allows easy tabulation of results. Following this section, the form provides space for open-ended comments and suggestions.
Source: Adobe


User Evaluation Form.pdf - Adobe Acrobat Pro DC

File Edit View Window Help

Home Tools User Evaluation For...

PDF DOCUMENT LIBRARY:

User Evaluation Form

System: | Evaluator: | Date:

Please evaluation the information systems project by circling the one number for each factor that best represents your assessment.

	Unsatisfactory			Acceptable		Excellent
SYSTEM OUTPUT						
1. Accuracy of information	1	2	3	4	5	6
2. Completeness of information	1	2	3	4	5	6
3. East of use	1	2	3	4	5	6
4. Timeliness of information	1	2	3	4	5	6
USER INTERFACE						
5. Clarity of Instructions	1	2	3	4	5	6
6. Quality of Help messages	1	2	3	4	5	6
7. Ease of use	1	2	3	4	5	6
8. Appropriateness of options	1	2	3	4	5	6
9. Clarity of error messages	1	2	3	4	5	6
10. Prevention of input errors	1	2	3	4	5	6
INFORMATION TECHNOLOGY STAFF						
11. Cooperation	1	2	3	4	5	6
12. Availability	1	2	3	4	5	6
13. Knowledge	1	2	3	4	5	6
14. Reporting of progress	1	2	3	4	5	6
15. Communication skills	1	2	3	4	5	6
TRAINING						
16. Completeness	1	2	3	4	5	6
17. Appropriateness	1	2	3	4	5	6
18. Schedule	1	2	3	4	5	6

For instance, after several months or a year, users might not remember whether they learned a procedure through training, from user documentation, or by experimenting with the system independently.

Users might also forget their impressions of IT team members over time. An important purpose of the post-implementation evaluation is to improve the quality of IT department functions, including interaction with users, training, and documentation. Consequently, the evaluation team should assess while users recall specific incidents, successes, and problems to suggest improvement.

On the other hand, if the evaluation is done too soon, the users may be unable to provide sufficient feedback due to a lack of experience with the new system. A post-implementation

evaluation is primarily concerned with assessing the quality of the new system. If the team performs the evaluation too soon after implementation, users will need more time to learn the new system and appreciate its strengths and weaknesses. Although many IT professionals recommend evaluating after at least six months of system operation, pressure to finish the project sooner usually results in an earlier evaluation to allow the IT department to move on to other tasks.

Conducting a post-implementation evaluation should be standard practice for all information systems projects. Sometimes, evaluations are skipped because users are eager to work with the new system or IT staff members have more pressing priorities. In some organizations, management might not recognize the importance and benefits of a post-implementation evaluation. The evaluations are critical because they enable the development team and the IT department to learn what worked and what did not. Otherwise, developers might commit the same errors in another system.

System Deployment Schedule

Establishing a detailed schedule for the system's deployment is an essential component of the deployment strategy, requiring the formulation of an exhaustive timeline that delineates each stage of the rollout process. This timeline meticulously defines crucial dates and times for the new system's introduction, encompassing all requisite final adjustments, designated testing phases, and the carefully planned shift from the legacy system to the newly implemented solution. Such a comprehensive schedule ensures that all stakeholders are aligned, resources are allocated efficiently, and the transition is executed smoothly, minimizing potential disruptions and facilitating a seamless integration of the new system into the organization's operational framework.

In this phase of system deployment, the systems analyst emerges as a key figure, working hand in hand with management and other stakeholders to construct a schedule that is both all-encompassing and grounded. Their collaboration is essential in identifying potential challenges and integrating diverse perspectives to create a timeline that accurately reflects the project's scope and complexity. This concerted effort ensures that the deployment plan is meticulously crafted and adaptable, laying a solid foundation for a transition that is smooth, efficient, and aligned with the organization's strategic objectives.

Final Report to Management

As the system implementation phase of the SDLC concludes, a crucial final step is creating and submitting a comprehensive final report to management. This report chronicles the development journey from start to finish, showcasing key milestones, obstacles encountered, and the system's preparedness for deployment. It acts as a critical tool for communication, providing stakeholders with a transparent overview of the project's progress and outcomes.

The report should include the following:

- A detailed account of the project, including key milestones and challenges encountered
- The system's readiness for deployment, showcasing its preparedness for use
- Final versions of all system documentation, providing a comprehensive overview of the system's design and functionality
- Planned modifications and enhancements to the system, outlining future development paths
- A recap of all systems development costs and schedules, offering a transparent view of the project's financial and temporal management
- A comparison of actual costs and schedules to the original estimates, highlighting efficiency and accuracy in planning and execution
- A post-implementation evaluation, if it has been performed, to assess the system's impact and effectiveness after deployment

Case in Point 11.3

Yorktown Industries

You like your new job as lead systems analyst at Yorktown Industries. You were pleased that your development team completed the new human resources system on schedule and under budget. You looked forward to receiving the post-implementation evaluation because you were confident that the system and the development team would receive high marks from users and managers.

After the system operated for one month, you received a call from your supervisor, who told you that you would have to handle the evaluation, even though you headed the development effort. You told your supervisor you did not feel comfortable evaluating your team's work. You explained that someone not involved in its development should do an independent evaluation. Your supervisor responded by expressing complete confidence in your ability to be objective. Your supervisor also explained that no one else was available and the evaluation was needed quickly to move forward with the next stage in the corporate development plan.

You are troubled about the situation. What should you do, and why?

Summary

Managing systems implementation is a complex set of activities, from quality assurance and application development to structured and object-oriented development, agile methodologies, coding practices, testing, documentation, and installation. This chapter offers a detailed exploration of each phase within the SDLC, presenting methods, tools, and strategies to ensure effective and efficient system implementation.

Quality assurance emphasizes the importance of software and systems engineering alongside adherence to international standards like those set by ISO, highlighting the role of rigorous engineering practices and global standards in achieving high-quality software systems that meet or exceed stakeholder expectations.

Application development includes strategies for translating design into functional applications, tasks associated with development, and the tools that facilitate this process. It covers the transition from conceptual designs to executable programs, underlining the significance of design translation strategies, the execution of development tasks, and the use of systems development tools to streamline the creation of software applications.

Structured development focuses on the principles of cohesion and coupling and the methodologies for creating detailed structure charts. This approach outlines the systematic arrangement of software components and their interactions, ensuring clarity and maintainability of system architecture.

By adopting O-O methods, developers can utilize the conceptual framework provided by object models to directly inform the structure and logic of their code, significantly simplifying the development process. This form of development emphasizes the importance of cohesion and coupling within object-oriented paradigms, showcasing how object-oriented principles support the development of modular, scalable, and maintainable software systems.

Agile development is explored through Extreme Programming (XP), user stories, quick iterations, and fast releases, illustrating the dynamic and iterative nature of agile methodologies in responding to changing requirements and fostering continuous improvement.

Coding is a fundamental process in software development that transforms conceptual ideas into functioning software. It involves the selection of programming languages, the benefits of an IDE, and the value of online learning resources.

Testing underscores the necessity of unit, integration, and system testing in identifying and resolving issues throughout the development life cycle, ensuring the reliability and functionality of the system before deployment.

Various forms of documentation are necessary to support system development, operation, and use. These forms include program, system, operations, user, and online documentation.

Installation of the system includes setting up operational and test environments, managing system changeover, converting data, conducting training, and executing post-implementation tasks to ensure a smooth transition to the new system.

Key Terms

acceptance test
application development
bug tracking software
Capability Maturity Model (CMM)
Capability Maturity Model Integration (CMMI)
coding
cohesion
condition
control couple
control module
coupling
customer
data conversion
data couple
defect tracking software
design walkthrough
desk checking
direct cutover
documentation
flowchart
integration testing
ISO 9001:2015
ISO/IEC/IEEE 90003:2018
iteration cycle
iteration planning meeting
library module
logic error
loop
loosely coupled
object-oriented development (OOD)
online documentation
operational environment
operations documentation
pair programming
parallel operation
patch
phased operation
pilot operation
pilot site
planning game
post-implementation evaluation
process improvement
production environment
program documentation
quality assurance (QA)
release plan
simulation
software engineering
status flag
structure chart
stub testing
subordinate module
syntax error
system changeover
system documentation
system testing
test data
test-driven development (TDD)
test environment
test plan
tightly coupled
top-down approach
training plan
train-the-trainer
tutorial
unit testing
user documentation
user story
webinar

Review Questions

1. What is quality assurance?
2. What is application development?
3. Explain how structure charts are used in application development.
4. Should classes be tightly coupled or loosely coupled in O-O development?
5. What is pair programming?
6. What role do IDEs play in coding?
7. Describe three main types of testing and the order in which they are performed.

8. What are the differences among program, system, operations, and user documentation?
9. What is the role of online documentation?
10. What is the difference between an operational environment and a test environment?

Discussion Topics

1. Discuss the three techniques used to improve quality assurance.
2. What are the most significant differences among structured, O-O, and agile methods? What do they have in common?
3. Experienced programmers sometimes eschew IDEs for hand-crafted tools, often connected in a pipeline to facilitate complex workflows. What advantages and disadvantages would this approach have over sophisticated IDEs?
4. Your supervisor said, "Integration testing is a waste of time. If each program is tested adequately, integration testing isn't needed. Instead, we should move on to system testing as soon as possible. If modules don't interact properly, we'll handle it then." Do you agree or disagree with this comment?
5. Suppose you designed a tutorial to train someone to use specific hardware or software, such as a web browser. What specific information would you want to know about the recipient of the training? How would that information affect the design of the training material?

Hands-On Projects

1. Using the material in this chapter and your Internet research, prepare a presentation on the pros and cons of agile development methods.
2. Design a generic test plan that describes the testing for an imaginary system.
3. Which system changeover method would you recommend for an air traffic control system upgrade?
4. Prepare an online training module for an AI-powered IDE like Amazon CodeWhisperer.
5. Design a generic post-implementation evaluation form. The form should consist of questions that you could use to evaluate any information system. The form should evaluate the training received and any problems associated with the program.

Ethical Issues

A team member is handling the testing for the new accounting system and is very upset about the most recent results. "Every time we fix one problem, another one pops up! After 10 days of testing and adjusting, we meet over 90% of the goals and benchmarks. If we're looking for perfection, we'll never make the implementation deadline for the new system, and the users will be all over us. Not to mention top management's reaction to a delay. I'm sure we can resolve some of these issues after the system becomes operational."

How would you respond to this statement? Are ethical issues involved? What are your responsibilities as an employee, as an IT professional, and as a co-worker?

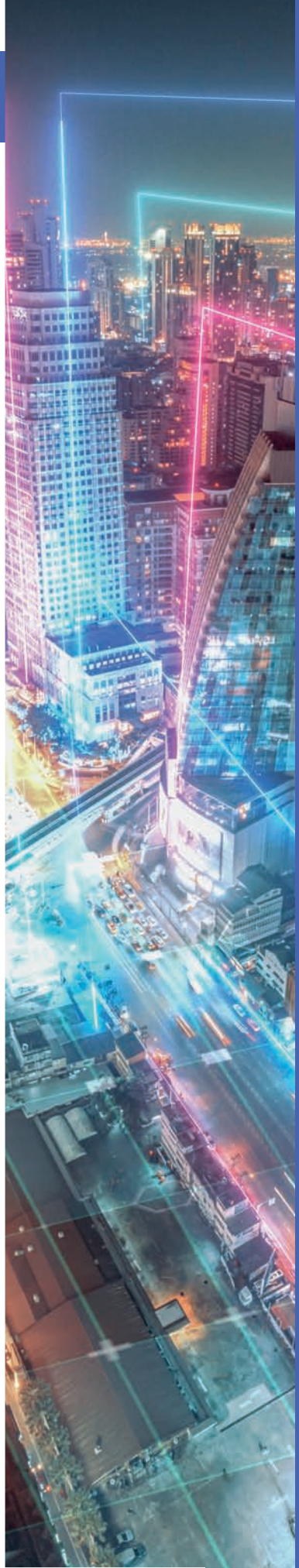
Phase 5

Operations

The operations phase concludes the SDLC, focusing on maintenance, support, and security. It covers user training, maintenance task categorization, and operations management, including configuration and quality assurance. This phase emphasizes the importance of comprehensive security strategies, from threat mitigation and policy implementation to risk management, advocating a multilayered approach. It also addresses security policy creation, backup and recovery plans, legal and ethical considerations, and the impact of new technologies on security, promoting a strong cybersecurity culture to ensure system protection.

Chapter 12 covers maintenance and support, including user training, support services, and outsourcing management. It discusses maintenance tasks, operations management, configuration management, and system performance management. The chapter also addresses quality assurance, security management, documentation, and knowledge management in system support. Finally, it explores system retirement and transition, including planning, data migration, decommissioning procedures, and transitioning to new systems.

Chapter 13 concentrates on managing systems security, covering concepts, threats, policies, and frameworks. It addresses attack mitigation, risk management, security levels, and procedural and organizational security. The chapter explores backup and recovery strategies, legal and ethical considerations, and the impact of new technologies. It emphasizes building a resilient cybersecurity culture through leadership, governance, and incorporating cutting-edge solutions into security strategies.



Chapter

12

Maintenance & Support

Learning Objectives

When you finish this chapter, you should be able to:

1. Describe the role of user support and training in system usability and user satisfaction.
2. Define the purpose of the different maintenance tasks in IT system upkeep.
3. Outline organizing maintenance operations: forming teams and setting task priorities.
4. Explain how configuration management ensures IT system stability during updates.
5. Describe how fault management and capacity planning enhance IT system efficiency.
6. List quality assurance components in system support and their service impact.
7. Explain how regular security updates and audits secure IT systems.
8. Identify the benefits of updated documentation and knowledge bases for users.
9. Outline the steps for planning and executing IT system retirement and transition.

Chapter 12 is the first chapter in the operations phase of the SDLC. The first step in this phase is maintenance and support, which emphasizes the essential practices and strategies for ensuring IT systems operate efficiently, securely, and in alignment with user needs. It covers a broad spectrum of activities, from user training and technical support to regular maintenance tasks, system performance management, the careful planning required for system updates, security management, and the eventual retirement and transition to new technologies.

The chapter includes three "Case in Point" discussion questions to help contextualize the concepts described in the text. The "Ethical Issues" scenario concerns a directive to skip regression testing for a critical security patch due to tight deadlines. The ethical dilemma revolves around balancing the urgency of security improvements against the potential risk of introducing new system issues.

12.1 User Support and Training

A systems analyst functions similarly to an in-house consultant, offering users essential guidance, support, and training. Interestingly, the most successful systems often require the greatest level of support. This is because users are eager to explore and master the system's features, fully leverage its capabilities, and understand how it can enhance their work performance.

In a typical organization, over half of the IT department's efforts are dedicated to supporting existing systems, underscoring the critical nature of this support. This significant investment highlights the importance of maintaining and enhancing system functionality to meet evolving user needs and organizational goals. Companies implement various user support mechanisms to fulfill these needs, including comprehensive user training programs and help desks that provide technical support and assistance.

This support infrastructure can be managed internally or through outsourcing, depending on the organization's resources and strategic priorities. Leveraging external expertise through outsourcing can often complement internal efforts, providing scalability and specialized skills that enhance overall support quality. Such arrangements ensure that users receive the help they need to utilize the systems effectively, thereby maximizing the investment in technology.

12.1.1 Training Programs

Training programs are pivotal in empowering users to harness the full capabilities of a system, ensuring they can navigate its complexities and utilize its features to enhance their work efficiency. Starting with user training, these programs lay the groundwork for all users, acquainting them with the system's basic functionalities and interface. The curriculum then proceeds to advanced training for power users, where more sophisticated features and customization options are explored, catering to users who seek to leverage the system for complex tasks and optimization. Finally, continuous learning and development keep users abreast of new updates and best practices, adapting to the evolving nature of technology and business needs. By covering these key areas, training programs aim to build a proficient user base capable of maximizing the system's potential to drive organizational success.

User Training

Within the broader framework of user training, it's crucial to recognize that the learning process is both an initial and an ongoing endeavor. The importance of foundational training during the rollout of a new system was underscored earlier in the book, setting the stage for users to understand and embrace the technology from day one. However, the IT department's responsibility extends beyond this initial phase. As new employees join the organization, they too require comprehensive training to familiarize themselves with the company's information systems.

Moreover, the evolution of technology and business processes necessitates periodic system updates. Significant modifications or the release of a new system version prompt the development of a specialized **user training package**. Tailored to the specific nature of the updates, these packages may encompass a variety of support tools, including online assistance via email, dedicated websites for resource access, updated user guides, supplemental training manuals, or even formal training sessions designed to bring users up to speed. The essence of training users on system changes mirrors that of the initial training efforts. The primary aim remains to demonstrate how the system **enhancements** or new features can streamline and improve job performance, ensuring users can leverage these updates to contribute more effectively to their roles and the organization's success.

Advanced Training for Power Users

Advanced training for power users takes the concept of system mastery a step further, focusing on those users who seek to extend their use of the system beyond routine tasks. This level of training is designed to uncover the system's advanced features, customization options, and integration capabilities. By doing so, it targets users eager to explore the system's full potential and apply its more complex functions to their work processes.

This advanced training aims to enhance individual proficiency and cultivate a group of in-house experts. These power users are equipped to support their peers, troubleshoot complex issues, and contribute significantly to the system's ongoing development. Their in-depth knowledge and hands-on experience with the system make them invaluable assets for providing feedback on system use and suggesting practical improvements.

Continuous Learning and Development

Continuous learning and development is a critical aspect of training programs, acknowledging the ever-changing nature of the technological landscape and the dynamic needs of organizations. As systems evolve and new functionalities are introduced, users must stay abreast of these changes to utilize the system effectively. This component of the training strategy ensures that all users, from novices to power users, have access to the latest information and tools. Through regular workshops, update briefings, and an extensive online resource library, users are continuously educated on system enhancements, advanced features, and industry best practices. This ongoing education helps maintain high organizational competency, enabling users to adapt to new technologies and methodologies swiftly.

Moreover, continuous learning and development initiatives foster a culture of knowledge sharing and innovation within the organization. By providing users with regular opportunities to enhance their skills and knowledge, organizations empower their workforce to keep pace with technological advancements and contribute to the system's improvement and optimization. Access to a well-curated online resource library allows users to learn at their own pace and according to their specific needs, promoting a self-driven approach to professional development. This continual investment in user education is essential for maximizing the return on technology investments and ensuring that the organization can effectively leverage its systems to achieve strategic objectives.

12.1.2 Support Services

Support services are an essential pillar of user satisfaction and system efficiency, a bridge between the technical complexities of a system and its everyday users. By offering a robust framework of assistance through help desks, online support portals, and community forums, organizations can significantly enhance the user experience, ensuring that help is readily available whenever needed. A help desk is the first place users turn when they need information or assistance; it provides a direct line to expert assistance, resolving technical issues, and answering queries to minimize downtime. Online support portals offer a self-service option, allowing users to find solutions independently through a comprehensive database of FAQs, guides, and troubleshooting tips. Meanwhile, community forums encourage a collaborative problem-solving approach where users can share insights, solutions, and best practices, enriching the support ecosystem. Together, these support services ensure that users can access the information and assistance they need to effectively navigate and utilize the system, fostering a culture of efficiency and continuous improvement.

Help Desks

A help desk, also called a service desk, serves as the front line of user support, providing an accessible and efficient means for users to seek assistance with technical issues, system queries, and troubleshooting guidance. Staffed by knowledgeable technicians, help desks are essential for resolving problems quickly, ensuring that users experience minimal disruption to

their work. This direct support channel fixes immediate issues and enhances the overall user experience by offering personalized assistance. Through various modes of communication, including phone calls, emails, and live chats, help desks cater to users' diverse preferences, ensuring that help is just a few clicks or a call away.

Because they interact with users in many departments, help desk representatives need strong interpersonal and technical skills and a solid understanding of the business. A help desk should carefully document all inquiries, support tasks, and activity levels. This information can identify trends and common problems and help build a technical support knowledge base.

Help desks are crucial in identifying common user challenges and trends in system issues. By logging and analyzing support requests, IT departments can gain valuable insights into potential areas for system improvement, user training needs, and developing more effective solutions. This data-driven approach to user support helps refine the system and tailor training programs to address frequently encountered issues, ultimately leading to a more intuitive and user-friendly system.

By integrating **remote control software** into its operations, a help desk can significantly enhance its productivity, allowing IT staff to directly access a user's workstation for immediate support and troubleshooting. GoToMyPC by GoTo, as illustrated in Figure 12-1, exemplifies such a software application that enables efficient problem solving by bypassing the need for lengthy verbal explanations. Video chat functions, complemented by the capability to view the customer's screen, leverages the effectiveness of visual communication. This combination substantially increases the likelihood of swiftly resolving customer issues, facilitating a more intuitive understanding of complex information and direct interaction with the problem.

During a typical day, help desk staff members might have to perform the following tasks:

- Show users how to create a data query or report that displays specific business information.
- Resolve network access or password problems.

Figure 12-1 Remote access software, such as GoToMyPC, lets users access their PCs from anywhere—even with their smartphones or tablets.
Source: GoToMyPC

- Demonstrate an advanced feature of a system or a commercial package.
- Help a user recover damaged data.
- Offer tips for better operation.
- Explain an undocumented software feature.
- Show a user how to use web conferencing.
- Explain how to access the company's intranet or the Internet.
- Assist a user in developing a simple database to track time spent on various projects.
- Answer questions about software licensing and upgrades.
- Provide information about system specifications and the cost of new hardware or software.
- Recommend a system solution that integrates data from different locations to solve a business problem.
- Provide hardware support by installing or reconfiguring devices such as scanners, printers, network cards, wireless devices, optical drives, backup devices, and multimedia systems.
- Show users how to maintain data consistency and integrity among desktop computers, notebook computers, and handheld computers or smartphones.
- Troubleshoot software issues via remote control utilities.

The help desk serves as a nexus between IT staff and users. It is a communication bridge and the hub for all IT **maintenance activities**, including reporting system problems, requesting maintenance, and submitting new system requests. Given the pivotal role of the help desk, it is imperative to staff it with highly knowledgeable and skilled personnel, as the frustration customers experience when they perceive themselves to be more informed about an issue than the support team can be detrimental to user satisfaction. Therefore, the recruitment and training of support staff demand significant investment to ensure they possess technical knowledge and excellent customer service skills. They should also have the authority to make immediate, problem-solving decisions without awaiting supervisor approval, thereby streamlining the resolution process and enhancing efficiency.

Online Support Portals

Online support portals offer users a comprehensive self-service solution, providing around-the-clock access to a wealth of resources designed to help them navigate and troubleshoot system issues independently. These portals typically include FAQs, instructional guides, video tutorials, and troubleshooting tools that cover a wide range of topics, from basic operations to complex problem solving. By empowering users to find answers and solutions independently, these portals can significantly reduce the requests directed to help desks, allowing support staff to focus on more complex queries.

In addition to providing immediate assistance, online support portals are an invaluable resource for continuous learning and self-improvement. Users can explore new features, learn advanced functionalities, and stay updated with the latest system enhancements at their own pace. Accessing this information anytime from anywhere fosters a more confident and competent user base. Furthermore, these portals often include feedback mechanisms, enabling users to contribute suggestions for new content and report on the usefulness of existing materials, thus playing an active role in the evolution of the support ecosystem.

Community Forums

Community forums create a dynamic space for users to engage with peers, share experiences, and collaborate on solving problems. This peer-to-peer support model leverages the collective knowledge and expertise of the user community, offering diverse perspectives and solutions

that might not be covered in formal support channels. Forums facilitate discussions on various topics, from troubleshooting and best practices to innovative ways of leveraging system features, thus fostering a collaborative learning environment.

The value of community forums extends beyond immediate problem solving. They serve as a repository of knowledge and experiences, where users can search for past discussions, learn from others' challenges and solutions, and contribute their insights. Moderated by experts or experienced users, these forums maintain a productive and respectful space for dialogue, encouraging users to participate and share. This enhances the support network and builds a community among users, encouraging engagement and loyalty to the system. Through this collaborative platform, organizations can tap into user feedback and suggestions, driving continuous improvement and innovation in system development and support services.

12.1.3 Managing Outsourcing

Managing outsourced services is a strategic approach that organizations employ to ensure the quality and efficiency of user support and training services provided by external vendors. This approach focuses on the challenges and considerations of relying on third parties for critical support functions; vendor management, which entails the careful selection, oversight, and collaboration with service providers to align their services with organizational goals; and quality assurance in outsourced services, ensuring that the services rendered meet the high standards expected by the organization and its users. Through effective management of outsourced services, organizations can leverage external expertise and resources to enhance their support infrastructure while maintaining control over service quality and aligning outsourced services with their strategic priorities. This careful balance enables companies to optimize their support services, ensuring that users receive timely, practical assistance, contributing to overall satisfaction and productivity.

Outsourcing Issues

Outsourcing IT support and help desks is a strategic move embraced by many firms alongside the outsourcing of various aspects of application development. This trend is primarily motivated by the potential for significant cost reductions, allowing companies to allocate resources more efficiently, such as channeling funds saved from offshore call centers toward product development. However, this approach has its challenges. An organization must ensure its external service providers align with its quality standards, operational needs, and cultural values. The benefits of accessing specialized expertise and cost savings are balanced against the risks of losing control over service quality and the potential for misalignment with user expectations. Thus, companies must navigate these waters with caution, establishing robust communication channels, **service level agreements (SLAs)**, and performance metrics to uphold the integrity of their user support functions.

Moreover, as discussed earlier in the book, the decision to outsource brings concerns about confidentiality and data security to the forefront. Therefore, entrusting sensitive information to external entities requires stringent **security** measures and adherence to data protection regulations. Organizations need to conduct thorough due diligence in selecting outsourcing partners, ensuring compliance with industry best practices for data security and privacy. Regular audits and compliance checks are vital to maintaining trust and preventing potential security breaches. Ultimately, the challenge for companies lies in achieving the desired cost efficiencies without compromising their reputation, customer base, or information security. Active management and monitoring of support quality and consistency are crucial in limiting the risks associated with outsourcing, ensuring that firms can enjoy the benefits while safeguarding their critical assets and customer relationships.

Vendor Management

Effective vendor management is crucial for the success of outsourced user support and training services. It begins with a meticulous selection process, where potential vendors are evaluated based on their expertise, reliability, and ability to meet the organization's specific requirements. Establishing a robust and collaborative relationship with selected vendors is essential, involving regular communication, joint planning sessions, and feedback mechanisms to ensure services are continuously aligned with the company's goals. Well-defined contracts and SLAs specify the expectations and responsibilities of both parties, providing a framework for accountability and performance measurement.

Once a vendor is selected, ongoing management becomes vital to sustaining a productive partnership. This includes monitoring the vendor's performance against agreed-upon metrics, addressing any issues promptly, and working collaboratively to identify areas for improvement. Regular SLA reviews and updates may be necessary to adapt to changing needs and technologies. Effective vendor management ensures the quality of outsourced services and fosters a partnership that can adapt and grow with the organization's evolving needs, contributing to long-term success.

Quality Assurance in Outsourced Services

Quality assurance (QA) in outsourced services is pivotal to ensuring that the support and training meet the organization's standards and user expectations. As firms have learned, a decline in technical support quality—manifested through extended phone wait times, subpar performance of support staff, and inadequate online support tools—can significantly impact customer satisfaction and loyalty. Therefore, implementing a comprehensive QA program is necessary. This involves setting clear performance indicators, conducting regular audits, and gathering user feedback to assess the effectiveness of the outsourced services. This proactive approach allows organizations to identify areas of improvement and work closely with vendors to implement necessary changes, ensuring that services retain high quality and are responsive to users' needs.

In addition to regular performance evaluations, fostering an environment of continuous improvement is essential. This can be achieved through ongoing training for the vendor's staff, regular updates to training materials and support protocols, and the incorporation of innovative support solutions. By focusing on QA, organizations can ensure that outsourcing decisions enhance rather than compromise the support experience for users, contributing to higher satisfaction levels and more effective use of the system.

12.2 Maintenance Tasks

The operations phase is essential to TCO because ongoing **maintenance expenses** can determine a system's economic life. Figure 12-2 shows a typical pattern of operational and maintenance expenses during a system's useful life. **Operational costs** include supplies, equipment rental, and software leases. Notice that the lower area shown in Figure 12-2 represents fixed operational expenses, while the upper area represents maintenance expenses.

Maintenance expenses vary significantly during the system's operational life, including spending to support maintenance activities. Maintenance activities include changing programs, procedures, or documentation to ensure correct system performance, adapting the system to the evolving requirements, and making the system operate more efficiently. Different types of maintenance meet those needs.

12.2.1 Types of Maintenance

Although some overlap exists, four types of maintenance tasks can be identified, as shown by the examples in Table 12-1. **Corrective maintenance** is performed to fix errors, **adaptive maintenance** adds new capabilities and enhancements, **perfective maintenance** improves efficiency, and **preventive maintenance** reduces the possibility of future system failure. Some

Figure 12-2 The total cost of operating an information system includes operational and maintenance costs. Operational costs (blue) are relatively constant, while maintenance costs (green) vary over time.

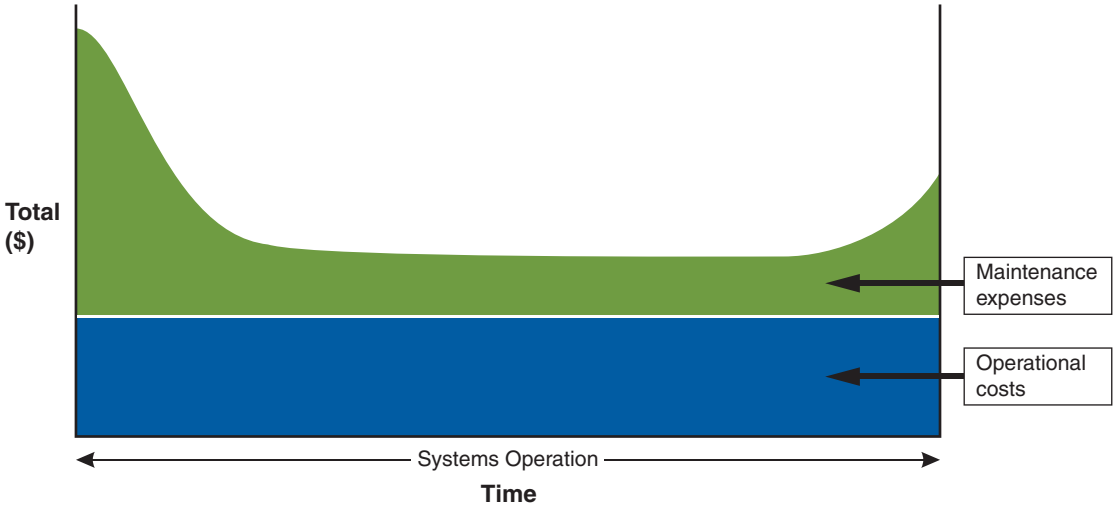


Table 12-1 Corrective maintenance fixes errors and problems. Adaptive maintenance enhances a system. Perfective maintenance improves a system's efficiency, reliability, or maintainability. Preventive maintenance avoids future problems.

Examples of Maintenance Tasks
Corrective Maintenance
• Diagnose and fix logic errors
• Replace defective network cabling
• Restore proper configuration settings
• Debug program code
• Update drivers
• Utilize remote control software for problem diagnosis and resolution
Adaptive Maintenance
• Add online capability
• Add support for mobile devices
• Add new data entry field to input screen
• Install links to website
• Create employee portal
Perfective Maintenance
• Upgrade or replace outdated hardware
• Write macros to handle repetitive tasks
• Compress system files
• Optimize user desktop settings
• Upgrade wireless network capability
• Install more powerful network server
Preventive Maintenance
• Install new antivirus software
• Develop standard backup schedule, including off-site and cloud-based strategies
• Implement regular defragmentation process
• Analyze problem report for patterns
• Tighten all cable connections
• Develop user guide covering confidentiality rules and unauthorized use of company IT resources

analysts use the term *maintenance* to describe only corrective maintenance that fixes problems. It is helpful, however, to view the maintenance concept more broadly and identify the different types of tasks.

Maintenance expenses are usually high when a system is implemented because corrective maintenance must detect, investigate, and resolve problems. Costs remain low once the system becomes stable and minor adaptive maintenance is involved. Eventually, both adaptive and perfective maintenance activities increase in a dynamic business environment.

Near the end of a system's useful life, adaptive and corrective maintenance expenses increase rapidly, but perfective maintenance typically decreases when it becomes clear that the company plans to replace the system. Table 12-2 shows the typical patterns for each of the four classifications of maintenance activities over a system's lifespan.

12.2.2 Corrective Maintenance

Corrective maintenance diagnoses and corrects errors in an operational system. To avoid introducing new problems, all maintenance work requires careful analysis before making changes. The best maintenance approach is a scaled-down version of the SDLC, where investigation, analysis, design, and testing are performed before implementing any solution. Recall from earlier in the book the differences between a test environment and an operational environment. Any maintenance work that could affect the system must first be performed in the test environment and then migrated to the operational system.

IT support staff respond to errors in various ways, depending on the nature and severity of the problem. Most organizations have standard procedures for handling minor mistakes, such as an incorrect report title or an improper format for a data element. In a typical procedure, a user submits a system request that is evaluated, prioritized, and scheduled by the system administrator or the systems review committee. If the request is approved, the maintenance team designs, tests, documents, and implements a solution.

Many organizations use a standard online form for systems requests. In smaller firms, the process might be an informal email message. A user submits a system request with supporting evidence for more severe situations, such as incorrect report totals or inconsistent data. Those requests are prioritized, and a maintenance team begins work on the problem immediately.

The worst-case situation is a system failure. If an emergency occurs, the maintenance team bypasses the initial steps and tries to correct the problem immediately. This often requires a patch, a specially written software module that provides temporary repairs to resume

Table 12-2 Information systems maintenance costs depend on the type of maintenance and the age of the system.

	Immediately After Implementation	Early Operational Life	Middle Operational Life	Later Operational Life
Corrective Maintenance	High	Low	Low	High
Adaptive Maintenance (Minor Enhancements)	None	Medium	Medium	Medium
Adaptive Maintenance (Major Enhancements)	None	None	Medium to High	Medium to High
Perfective Maintenance	Low	Low to Medium	Medium	Low
Preventive Maintenance	Low	Medium	Medium	Low

Table 12-3 This three-level ranking framework for IT support considers potential impact and response urgency.

Priority	Impact	Time Frame
Level 1	Significant impact on IT operations, security, or business activity that requires immediate attention.	Implement patch as soon as possible.
Level 2	Some impact on IT operations, security, or business activity. Requires prompt attention, but operations can continue.	Patch as necessary and begin implementation prior to next release.
Level 3	Little or no impact on current IT operations, security, or business activity.	Implement in the next release.

operations. Meanwhile, a written systems request is prepared by a user or a member of the IT department and added to the maintenance log. When the system is operational again, the maintenance team determines the cause, analyzes the problem, and designs a permanent solution. The IT response team updates the test data files, thoroughly tests the system, and prepares complete documentation. Regardless of the priorities, a standard ranking method can be helpful. For example, Table 12-3 shows a three-level framework for IT support, given the potential impact on operations.

Section 12.3 describes the process of managing system support in more detail, including an overview of maintenance tasks and a procedural flowchart.

12.2.3 Adaptive Maintenance

Adaptive maintenance enhances an operational system and makes it easier to use. An enhancement is a new feature or capability. The need for adaptive maintenance usually arises from business environment changes such as new products or services, new manufacturing technology, or support for a new web-based operation.

The procedure for minor adaptive maintenance is similar to that for routine corrective maintenance. A user submits a system request that is evaluated and prioritized by the systems review committee. A maintenance team then analyzes, designs, tests, and implements the enhancement. Although the procedures for the two types of maintenance are alike, adaptive maintenance requires more IT department resources than minor corrective maintenance.

A major adaptive maintenance project is like a small-scale SDLC project because the development procedure is similar. Adaptive maintenance can be more complicated than new systems development because the enhancements must work within the constraints of an existing system.

12.2.4 Perfective Maintenance

Perfective maintenance involves changing an operational system to make it more efficient, reliable, or maintainable. Requests for corrective and adaptive maintenance typically come from users, while the IT department usually initiates perfective maintenance.

Changes in user activity or data patterns during system operation can cause efficiency to decline, and perfective maintenance might be needed to restore performance. When users are concerned about performance, it should be determined if a perfective maintenance project could improve response time and system efficiency.

Perfective maintenance can also improve system reliability. For example, input problems might cause a program to terminate abnormally. By modifying the data entry process, errors can be highlighted and users can be notified that they must enter proper data. Support is less costly and less risky when a system is easier to maintain. In many cases, a complex program can be simplified to improve maintainability.

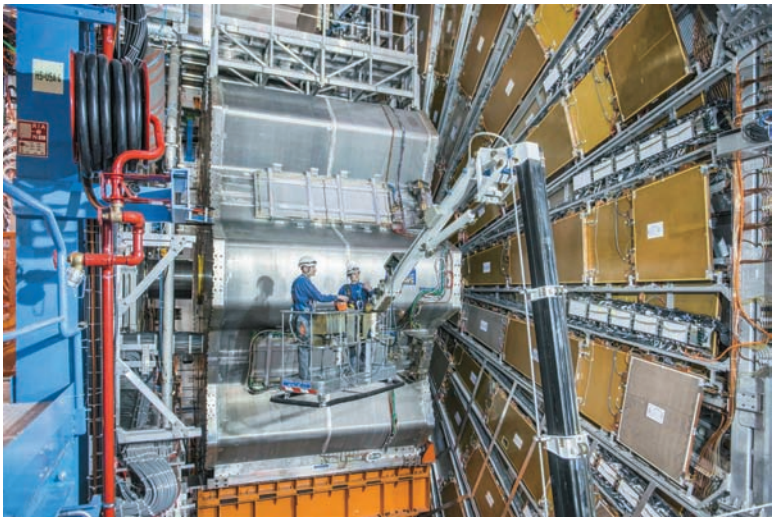
In many organizations, perfective maintenance needs to be performed more frequently. Companies with limited resources often consider new systems development, adaptive maintenance, and corrective maintenance more critical than perfective maintenance. Managers and users constantly request new projects, so few resources are available for perfective maintenance work. As a practical matter, perfective maintenance can be performed as part of another project. For example, if a new function must be added to a program, perfective maintenance can be included in the adaptive maintenance project.

Perfective maintenance is usually cost-effective during the middle of the system's operational life. Early in systems operation, perfective maintenance is usually not needed. Later, perfective maintenance might be necessary but have a high cost. Perfective maintenance is less important if the company plans to discontinue the system.

When performing perfective maintenance, analysts often use a technique called software reengineering. **Software reengineering** uses analytical techniques to identify an information system's potential quality and performance improvements. Software reengineering is similar to business process reengineering, which seeks to simplify operations, reduce costs, and improve quality.

Programs that need many maintenance changes are usually good candidates for reengineering. The more a program changes, the more likely it becomes inefficient and difficult to maintain. Detailed records of maintenance work can identify systems with a history of frequent corrective, adaptive, or perfective maintenance.

Figure 12-3 Technicians use a motorized lift to get into position by the ATLAS particle detector at CERN. Regardless of the type of system, trained professionals must perform high-quality maintenance.
Source: Anna Pantelia/CERN



12.2.5 Preventive Maintenance

Preventive maintenance requires analyzing areas where trouble is likely to occur to avoid problems. Like perfective maintenance, the IT department typically initiates preventive maintenance. Preventive maintenance often increases user satisfaction, decreases downtime, and reduces TCO. However, it competes for IT resources with other projects and sometimes does not receive the high priority it deserves.

Regardless of the type of maintenance, trained professionals must support computer systems, just as skilled technicians must service the particle detector at CERN, as shown in Figure 12-3. In both cases, the quality of the maintenance will directly affect the organization's success.

Case in Point 12.1

NovaTech Solutions

You are a systems analyst for NovaTech Solutions, a company that has recently embraced cloud computing and is revolutionizing its IT infrastructure. This transition to the cloud has introduced new dynamics and dependencies that challenge traditional maintenance routines. How would you adapt your existing maintenance tasks to ensure seamless operation and security in this cloud-centric environment, primarily focusing on adaptive and preventive maintenance strategies?

12.3 Maintenance Operations Management

Maintenance operations management is a comprehensive approach encompassing the strategic oversight and execution of maintenance activities to ensure information systems operate at peak efficiency and align with business goals. This section delves into the critical components of effective maintenance management, including the structuring and roles of the maintenance team, the process for handling maintenance requests, and the methodologies for establishing priorities among various maintenance tasks. By focusing on these areas, organizations can optimize their maintenance efforts, reduce downtime, and continuously ensure that their information systems meet user and business needs.

12.3.1 The Maintenance Team

The **maintenance team** is the backbone of any successful maintenance strategy. It comprises individuals with diverse skills and roles dedicated to the upkeep of information systems. The team typically includes a **system administrator** and one or more systems analysts and programmers. The system administrator should have solid technical expertise and experience in troubleshooting and configuring operating systems and hardware. Successful analysts need a strong IT background, solid analytical abilities, good communication skills, and an overall understanding of business operations.

In many IT departments, managers traditionally segregate teams into those focusing on new system development and those dedicated to maintenance tasks, creating distinct organizational roles. However, a growing number of organizations are adopting a more dynamic approach by integrating these functions, allowing the same team that develops a system also to be responsible for its maintenance. This integrated model promotes a deeper understanding of the system, reduces the need for additional training or expenses, and fosters a strong sense of ownership among team members from the beginning. The developers' familiarity with the system's intricacies simplifies troubleshooting and enhances the maintenance process, making it a more efficient and cohesive operation.

Despite these benefits, there exists a perception among some analysts that maintenance work lacks the creativity and excitement found in developing new systems, often due to the challenges associated with deciphering and supporting systems that need to be better documented and organized by others. To combat this and enhance skill sets, some organizations practice rotating their staff between maintenance and development roles, ensuring a versatile workforce capable of adapting to the organization's evolving needs. While analysts gain valuable insights into the importance of creating maintainable systems and appreciating the nuances of system development through this rotation, it is not without drawbacks—primarily the loss of productivity as individuals transition between roles, hindering the opportunity to specialize deeply in one area. Moreover, assigning new or less experienced IT staff to maintenance projects, though seen as a valuable learning experience, can be daunting compared to the collaborative and supportive environment of a development team, highlighting the need for a balanced approach in task assignments to nurture development and maintain effectiveness.

System Administrator

A system administrator plays a crucial role in organizations, managing computer and network systems to ensure smooth and efficient operation. This position demands high proficiency and flexibility, as the administrator must effectively work under pressure, demonstrating exceptional organizational and communication skills. The ability to quickly understand and resolve complex technical issues within tight deadlines is fundamental to the role, ensuring that system disruptions are minimized and productivity remains high. In many organizations, the system administrator is primarily responsible for the IT infrastructure's operational integrity, configuration, and security. This encompasses various tasks, from overseeing server performance to ensuring network security protocols are up to date and effective against potential threats.

Beyond these responsibilities, the system administrator plays a vital role in the routine maintenance of systems, proactively identifying and addressing potential issues before they escalate into critical problems. This often includes the authority to implement preventive measures, such as updating software to patch known vulnerabilities, upgrading hardware components to improve system performance, or restructuring network configurations to enhance security and efficiency. The system administrator's ability to anticipate and avert crises, such as server crashes, network outages, security breaches, or hardware failures, is invaluable to maintaining organizational continuity. Their actions prevent operational downtime and safeguard the organization's data and technology assets, highlighting system administrators' pivotal role in the modern digital workplace.

Systems Analysts

Systems analysts specializing in maintenance teams embody the role of astute detectives, delving deep into complex IT environments to pinpoint and rectify issues that disrupt system functionality. This task demands a high degree of analytical and synthetic thinking; analysts dissect the system to understand its constituent elements (analysis) and then piece together these insights to grasp the system's holistic operation (synthesis). Their work is crucial in ensuring the continuous, efficient, and secure operation of IT systems that businesses rely on. Beyond mere technical prowess, these analysts must comprehensively understand the business landscape in which these systems operate. This includes knowing how different processes interlink and how technology supports these processes, ensuring that solutions are technically viable and align with business goals and strategies.

The role of a systems analyst in maintenance goes beyond technical and business acumen to encompass strong interpersonal and communication skills. These professionals often serve as the bridge between the IT department and business units, translating complex technical issues into understandable terms for nontechnical stakeholders. Creativity, energy, and an insatiable appetite for learning are essential as technology and business environments constantly evolve. Analysts must stay abreast of the latest technological advancements, industry trends, and business practices to recommend and implement solutions that keep systems current and competitive. Their ability to innovate, communicate effectively, and understand the broader business context makes them invaluable in maintaining the vitality and resilience of IT systems within any organization.

Programmers

In smaller organizations, the scope of a programmer's responsibilities often spans multiple domains due to the compact size of the IT team. This multifaceted role requires programmers to juggle various tasks ranging from software development and system maintenance to troubleshooting and user support. The nature of these organizations necessitates a versatile skill set, allowing programmers to adapt to the diverse needs of the business, whether that involves writing code, configuring systems, or even participating in strategic planning sessions. However, as organizations grow in size and complexity, the demand for specialization within the programming domain increases. Larger firms typically have the resources and scale to segment programming roles according to specific functions or technologies, allowing for a deeper focus and expertise in particular areas of the IT landscape.

For instance, in these larger settings, job titles reflect the specialization and concentrated focus areas of programming roles. An **applications programmer** might dedicate their expertise to developing and maintaining tailored software solutions that address specific business needs. In contrast, a **systems programmer** could focus on the foundational operating system software and utilities, ensuring the infrastructure's efficiency and reliability. Similarly, a **database programmer** specializes in developing and supporting large-scale database systems, which is crucial for managing the vast amounts of data modern businesses accumulate. Additionally, the role of a **programmer/analyst** emerges in environments that value a hybrid skill set, combining the analytical prowess to assess and design system solutions with the technical capability to implement those solutions. This blend of analysis and programming is particularly valuable

in dynamic IT departments aiming for agile response capabilities and integrated system development practices. It highlights the evolving nature of programming roles in response to organizational needs and technological advancements.

12.3.2 Roles and Responsibilities

Clarifying the roles and responsibilities within the maintenance team is essential for delineating clear lines of accountability and facilitating the efficient distribution of tasks. This delineation becomes crucial in the team structure where the organization is optimized, including roles such as systems analysts, developers, testers, and support personnel. Each member is assigned distinct tasks, from identifying problems to executing solutions and liaising with end users. Defining these responsibilities enhances the maintenance workflow through collaboration and coordination, promoting seamless interaction among team members. This clarity accelerates the maintenance process and ensures that each individual understands their role in the system's upkeep. Additionally, this structure supports skill development by identifying areas for individual growth and learning, ensuring team members have the necessary skills to meet their responsibilities and contribute effectively to the team's success.

Team Structure

The effectiveness of a maintenance team is heavily dependent on its structural organization. A well-structured team with clearly defined roles and efficient communication pathways is more likely to excel in coordination and expedite the decision-making process. The organization's size and the intricacy of its systems often dictate the team's configuration, ranging from a centralized approach—where a single team manages all maintenance activities—to a decentralized structure that allocates specific tasks to various specialized teams. The optimal structure is determined by balancing the organization's unique requirements and the characteristics of its information systems, ensuring that the maintenance team can operate effectively and respond promptly to issues.

Choosing between a centralized or decentralized model involves weighing each option's benefits against its potential drawbacks. Centralized models can streamline processes and simplify oversight, making them ideal for smaller organizations with less complex systems. In contrast, decentralized structures offer greater flexibility and allow for a more specialized focus on distinct maintenance areas, which is suited for larger organizations with diverse and complex systems. This flexibility can enhance problem-solving efficiency and adaptability to changing technological landscapes. Ultimately, the decision on the team structure should align with the organization's operational goals, the nature of its IT infrastructure, and its capacity to support effective maintenance workflows, ensuring the longevity and reliability of its systems.

Collaboration and Coordination

Effective collaboration and coordination within the maintenance team and across different organizational departments are crucial for the seamless management and resolution of maintenance issues. The success of maintenance operations hinges not just on the cohesiveness of the maintenance team but also on their ability to engage constructively with other departments, stakeholders, and external partners. By instituting regular channels for communication, setting up collaborative problem-solving sessions, and forming cross-functional teams, the organization can significantly improve its capacity to tackle complex system issues holistically. Utilizing collaboration tools and platforms further streamlines this process, promoting knowledge sharing and aligning efforts across various parts of the organization. This concerted effort fosters a culture of cooperation and enhances the efficiency and effectiveness of maintenance activities.

Adopting an integrated approach to maintenance tasks allows the team to draw on a wide range of experts, reduces overlap in efforts, and ensures that all maintenance actions align with the organization's strategic objectives. Leveraging insights from different departments

and specialists contributes to a more comprehensive understanding of system issues, leading to more effective and sustainable solutions. Moreover, this collaborative model ensures that maintenance efforts are not siloed but are instead part of a broader organizational framework, optimizing resource use and contributing to the overall health and performance of the IT infrastructure. Fostering an environment that values collaboration and coordination improves the outcomes of maintenance activities and supports the organization's long-term success and adaptability.

Skill Development

For the maintenance team to effectively navigate the rapidly evolving landscape of technology and maintenance methodologies, ongoing skill development is essential. Committing resources to training and professional growth guarantees the team's up-to-date knowledge of the newest tools and techniques, boosting their competency in swiftly identifying and fixing problems. This investment in skill enhancement equips the team with the necessary capabilities to maintain system efficiency and ensures they can leverage cutting-edge solutions to address maintenance tasks.

Cultivating an environment that prioritizes learning and exchanging ideas among team members can significantly impact the team's innovation ability. Such a culture encourages the exploration of creative approaches to overcome maintenance hurdles, leading to the development of more effective and efficient solutions. This emphasis on continuous learning and collaboration elevates the team's standard of system support. It contributes to a more dynamic and adaptable maintenance operation that is prepared to meet the demands of current and future technological challenges.

12.3.3 Handling Maintenance Requests

As shown in Figure 12-4, handling maintenance requests necessitates the creation of a well-defined process that encompasses the receipt, evaluation, and resolution of issues, whether they are flagged by users or detected through systematic system monitoring. This process typically involves a ticketing system where problems are logged, categorized according to severity and impact, and then assigned to the appropriate maintenance personnel for action. Organizations can ensure timely and effective responses to system issues by standardizing the approach to managing these requests, minimizing downtime, and maintaining optimal system performance.

Request Intake Process

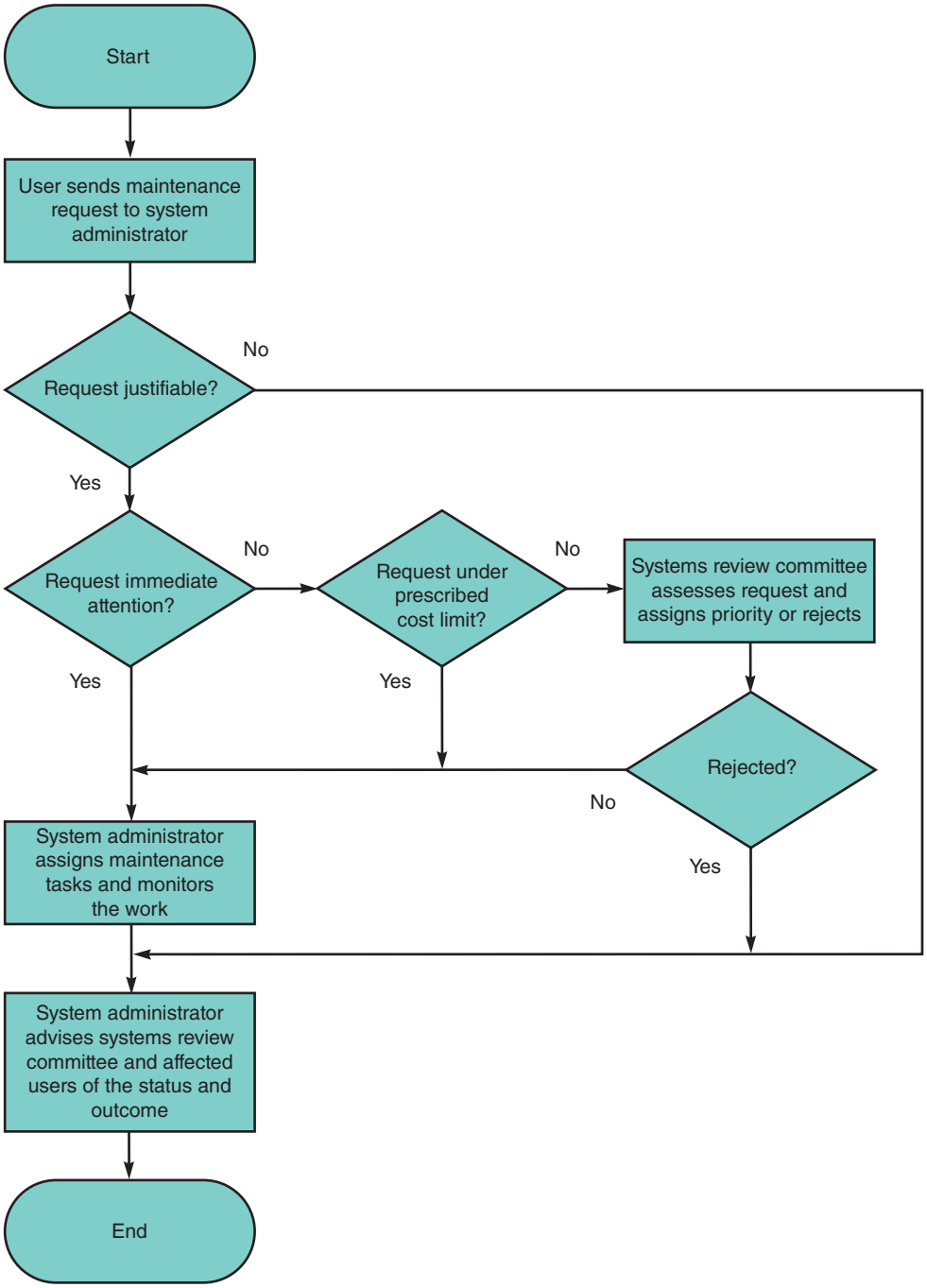
Developing an efficient request intake process is essential for managing the flow of maintenance requests effectively. This process often involves using a ticketing system in which users can report issues, which are categorized, prioritized, and assigned to the appropriate team members. An effective intake process ensures that all requests are documented, tracked, and resolved promptly, minimizing disruptions to business operations.

Users submit most requests for corrective and adaptive maintenance when the system is not performing correctly or if they want new features. IT staff members usually initiate requests for perfective and preventive maintenance. To keep a complete maintenance log, all work must be covered by a specific request that users submit in writing or by email.

Prioritization Criteria

Establishing clear prioritization criteria for maintenance requests ensures that critical issues are addressed promptly while effectively managing the team's workload. Factors such as the impact on business operations, the severity of the issue, and the resources required for resolution play a key role in determining the priority of each request. By adhering to these criteria, the maintenance team can ensure a balanced approach to handling requests, focusing on urgent issues without neglecting long-term system improvements.

Figure 12-4 Although the procedure varies from company to company, this flowchart shows a typical process for handling maintenance requests.



When a user submits a maintenance request, the system administrator conducts an initial assessment to determine the severity and justifiability of the issue. If the request involves a critical problem that demands immediate attention, action is taken promptly. The administrator evaluates whether the issue falls within a preauthorized cost limit for non-emergency requests. Requests exceeding this cost threshold are forwarded to a systems review committee, which decides whether to approve (assigning a priority) or reject the request based on its merits and cost implications. Following this evaluation, the system administrator communicates the decision to the affected users, ensuring transparency in the maintenance process. For approved requests within the cost limit, the administrator coordinates the necessary maintenance tasks and oversees their execution to resolution.

Tracking and Reporting

Maintaining a detailed record of maintenance requests and their resolution status ensures transparency and accountability throughout the maintenance process. This practice facilitates the effective management of maintenance tasks and builds trust between the IT department and the system users. By systematically tracking each request from inception to resolution, organizations can provide real-time updates and maintain a clear line of communication with all stakeholders. This level of oversight allows for immediate identification of bottlenecks or delays in the maintenance workflow, enabling timely interventions to address any issues.

Regularly compiling and analyzing data on key performance metrics such as resolution times, the volume of pending requests, and levels of user satisfaction offers valuable insights into the efficiency and effectiveness of the maintenance team. This analysis can highlight strengths and pinpoint improvement areas, guiding strategic decisions regarding process optimizations and resource allocations. Furthermore, sharing these insights through periodic reports with stakeholders reinforces the maintenance team's commitment to continuous improvement and ensures that all parties are informed of progress. This constant feedback loop is essential for refining maintenance strategies over time, ensuring that the team can adapt to changing needs and maintain high levels of service quality.

12.3.4 Prioritizing Maintenance Work

Prioritizing maintenance involves a strategic assessment that directs resources toward the most pressing issues, ensuring that the most critical tasks are addressed first to maintain system stability and performance. This prioritization process is essential for achieving an optimal balance between addressing immediate technical concerns and implementing enhancements that contribute to the system's long-term resilience and efficiency. By carefully evaluating the urgency and impact of each maintenance task, organizations can allocate their technical and human resources effectively, ensuring that short-term fixes and strategic system improvements are managed effectively to support overall business objectives.

The systems review committee in many companies separates maintenance and new development requests when setting priorities. In other organizations, all requests are considered together, and the most critical project gets top priority, whether it is maintenance or new development. The most important objective is a procedure that best balances new development and necessary maintenance work to support business requirements and priorities.

Some IT managers believe evaluating all projects together leads to the best possible decisions because maintenance and new development require similar IT department resources. It might be better to assess requests separately in IT departments where maintenance and new development are separate. Another advantage of an individual approach is that maintenance is more likely to receive a proportional share of IT department resources.

Establishing Priorities

Setting priorities for maintenance work is a critical component of effective systems management, necessitating a thorough evaluation of each task's urgency and potential impact on the organization's operations and strategic goals. This evaluative process allows for a nuanced understanding of immediate needs versus longer-term improvements. It guides decision makers in identifying which maintenance activities demand prompt action due to their critical nature and which can be deferred with minimal risk to business continuity. By doing so, organizations ensure that resources are strategically deployed to address the most pressing issues first, mitigating potential disruptions and maintaining essential services without compromising operational efficiency.

Moreover, a systematic approach to this prioritization ensures that the maintenance activities are reactive and proactively aligned with the organization's overarching business objectives. This alignment is crucial for optimizing system performance and reliability over time, enabling the organization to leverage its IT infrastructure as a strategic **asset** rather than a mere operational necessity. Through such a structured prioritization process, maintenance efforts contribute directly to the organization's ability to adapt to changing market demands,

support strategic initiatives, and ultimately achieve a competitive edge by ensuring that their systems are robust, efficient, and capable of supporting business growth and innovation.

Impact Analysis

Conducting an impact analysis for each maintenance task is pivotal in prioritization, serving as a foundational activity that informs the maintenance strategy. By carefully assessing how each maintenance activity could affect system functionality, user experience, and the broader scope of business operations, organizations can gauge the severity and implications of issues within their IT infrastructure. This thorough analysis helps categorize tasks based on urgency and impact, enabling maintenance teams to allocate resources effectively. It ensures that tasks critical to system stability and business continuity are addressed promptly, safeguarding against system failures that could disrupt operations and impact the bottom line.

Furthermore, understanding the broader implications of maintenance tasks through impact analysis facilitates a more strategic approach to system management. It allows maintenance teams to address immediate technical concerns and consider the long-term effects of their actions on the user experience and operational efficiency. For instance, prioritizing tasks that improve system performance or enhance security can significantly improve productivity and data protection. By integrating impact analysis into maintenance prioritization, organizations can ensure a balanced maintenance schedule that supports immediate needs and strategic objectives, ultimately leading to a more resilient and efficient IT ecosystem.

Stakeholder Communication

Effective communication with stakeholders is pivotal in maintenance prioritization, serving as the cornerstone of a successful maintenance strategy. By maintaining open lines of communication, the maintenance team can ensure that stakeholders are fully aware of the rationale behind maintenance schedules, the anticipated impacts on system functionality, and any expected downtime or service interruptions. This level of transparency is crucial for managing expectations, allowing stakeholders to plan accordingly and minimize the operational impact. Moreover, clear and consistent communication helps in preempting any concerns or questions stakeholders may have, providing reassurance about the maintenance team's capability to manage the system's health effectively and efficiently.

Moreover, transparent communication builds trust and collaboration between the maintenance team and system users, which is invaluable for securing buy-in and support for necessary maintenance activities. When stakeholders understand the importance of each maintenance task and how it contributes to the overall reliability and performance of the system, they are more likely to be accommodating and cooperative during periods of disruption. This mutual understanding and respect facilitate a more supportive environment where maintenance activities are not seen as inconveniences but as essential steps toward maintaining a high-performing and dependable system. Through effective stakeholder communication, the maintenance team can foster a positive perception of maintenance work, emphasizing its role in enhancing the system's value and ensuring its alignment with the organization's strategic goals.

12.3.5 Maintenance Process Improvement

Maintenance process improvement is a critical component of a comprehensive maintenance management strategy. It focuses on enhancing maintenance practices to ensure they remain effective, efficient, and aligned with the organization's evolving needs. This process unfolds through a harmonious blend of evaluating existing activities for optimization, integrating cutting-edge technology, and fostering an environment of perpetual learning and innovation. At the heart of this endeavor lies the analytical review and feedback, where maintenance metrics and **key process indicators (KPIs)**, along with input from system users and stakeholders, guide the identification of areas within the maintenance workflow primed for enhancement. By weaving together the threads of technological integration and innovation with a culture that champions continuous improvement, the strategy embarks on a multifaceted approach to elevate maintenance practices. This unified methodology seeks to streamline operations, reduce costs,

and enhance overall system performance by leveraging the latest technological advancements and nurturing a proactive, solution-oriented mindset within the maintenance team.

Analytical Review and Feedback

The foundation of maintenance process improvement is the analytical review and feedback, which comprehensively examines current maintenance practices. This review harnesses data from various sources, including performance metrics, user feedback, and direct observations, to identify inefficiencies and areas ripe for enhancement. By systematically analyzing this information, maintenance managers can understand the workflow's strengths and weaknesses, allowing targeted interventions to streamline processes, enhance system reliability, and elevate user satisfaction.

Building on this analysis, the feedback mechanism is crucial in driving improvements. Encouraging open and ongoing communication among maintenance staff, system users, and other stakeholders ensures a broad perspective is considered when evaluating maintenance activities. This feedback informs immediate improvement actions and fosters a culture of transparency and collaboration, which is essential for sustaining long-term enhancements in maintenance practices.

Technological Integration and Innovation

The adoption of advanced tools and methodologies significantly propels maintenance process improvement. Automation technologies, predictive analytics, and state-of-the-art maintenance management systems offer new avenues for reducing manual labor, anticipating system issues before they manifest, and optimizing resource allocation. The strategic incorporation of these technologies into maintenance operations boosts efficiency and enhances the team's ability to respond to and prevent potential system failures.

Continuous education on industry trends and exploring new technological solutions are vital for keeping the maintenance team at the forefront of innovation. Investing in training and development ensures that team members are well-equipped to leverage these advanced tools and integrate best practices into their routines. This commitment to technological advancement and skill enhancement underscores the importance of staying adaptive and competitive in an ever-evolving technological landscape, ensuring maintenance strategies remain robust and forward-thinking.

Cultivating Continuous Improvement

Cultivating continuous improvement within the maintenance team and the broader organization is essential for nurturing an environment where innovation and proactive problem solving thrive. Encouraging team members to voice improvement suggestions and engaging stakeholders in the refinement process guarantees that maintenance strategies are comprehensive and garner widespread support. Regularly scheduled review sessions to evaluate the effectiveness of implemented changes, celebrate achievements, and pinpoint new areas for improvement are crucial for keeping the momentum of process enhancement.

This culture of continuous improvement emphasizes the value of iterative learning and adaptation, ensuring that maintenance practices evolve in concert with organizational needs and technological advancements. It promotes a proactive stance toward maintenance management in which every team member is empowered to contribute to the system's betterment. Through this ongoing evaluation cycle, organizations can achieve superior maintenance outcomes, bolstering operational excellence and ensuring resilience in changing business dynamics.

12.4 Configuration Management

Configuration management (CM), sometimes called **change control (CC)**, is a process for controlling changes in system requirements during software development. Configuration management is also essential for managing system changes and costs after a system becomes operational.

Most companies establish a specific process that describes how system changes must be requested and documented.

As enterprise-wide information systems grow more complex, configuration management becomes critical. Industry standards such as the IEEE's Standard 828-2012 for configuration management in systems and software engineering have emerged, as shown in Figure 12-5.

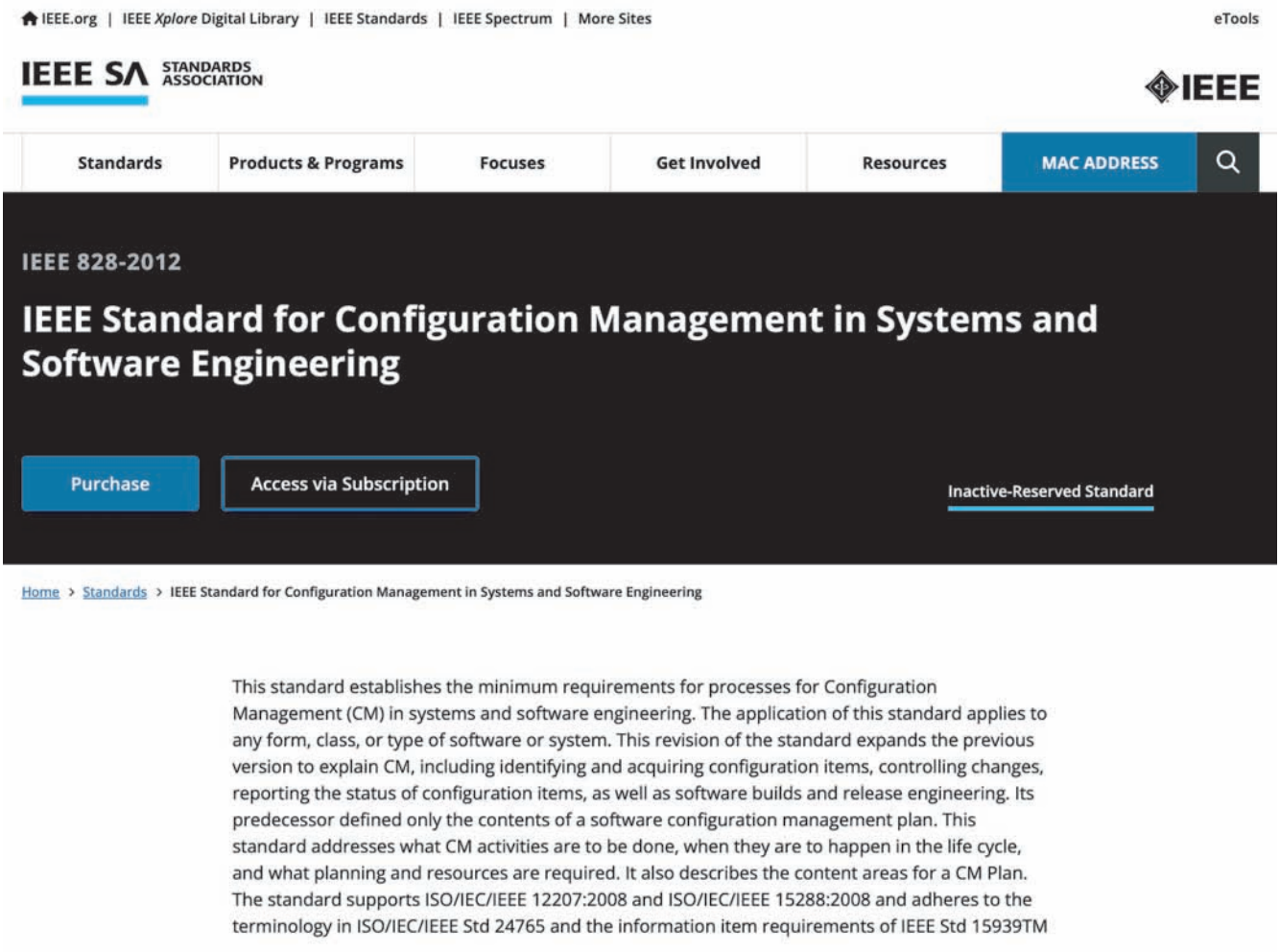
CM is essential if a system has multiple versions that run in different hardware and software environments. Configuration management also helps to organize and handle documentation. An operational system has extensive documentation that covers development, modification, and maintenance for all versions of the installed system. Most documentation material is stored in the IT department, including the initial systems request, project management data, end-of-phase reports, data dictionary, and the IT operations and user manuals.

Keeping track of all documentation and ensuring that updates are correctly distributed are essential aspects of configuration management.

12.4.1 Maintenance Releases

Keeping track of maintenance changes and updates can take time and effort, especially for a complex system. When a **maintenance release methodology** is used, all noncritical changes are held until they can be implemented simultaneously. Each change is documented and installed as a new version of the system called a **maintenance release**.

Figure 12-5 IEEE Standard 828-2012 for configuration management in systems and software engineering.
Source: IEEE



The time between releases for a system developed in-house usually depends on the level of maintenance activity. However, a new release to correct a critical error might be implemented immediately rather than saved for the next scheduled release.

A numbering pattern distinguishes the different releases when a release method is used. In a typical system, the initial version of the system is 1.0, and the release that includes the first set of maintenance changes is version 1.1. A change, for example, from version 1.4 to 1.5 indicates relatively minor enhancements, while whole-number changes, such as from version 1.0 to 2.0 or from version 3.4 to 4.0, indicate a significant upgrade.

The release methodology offers several advantages, especially if two teams perform maintenance on the same system. When a release methodology is used, all changes are tested together before a new system version is released. This approach results in fewer versions, less expense, and less user interruption. Using a release methodology also reduces the documentation burden because all changes are coordinated and become effective simultaneously.

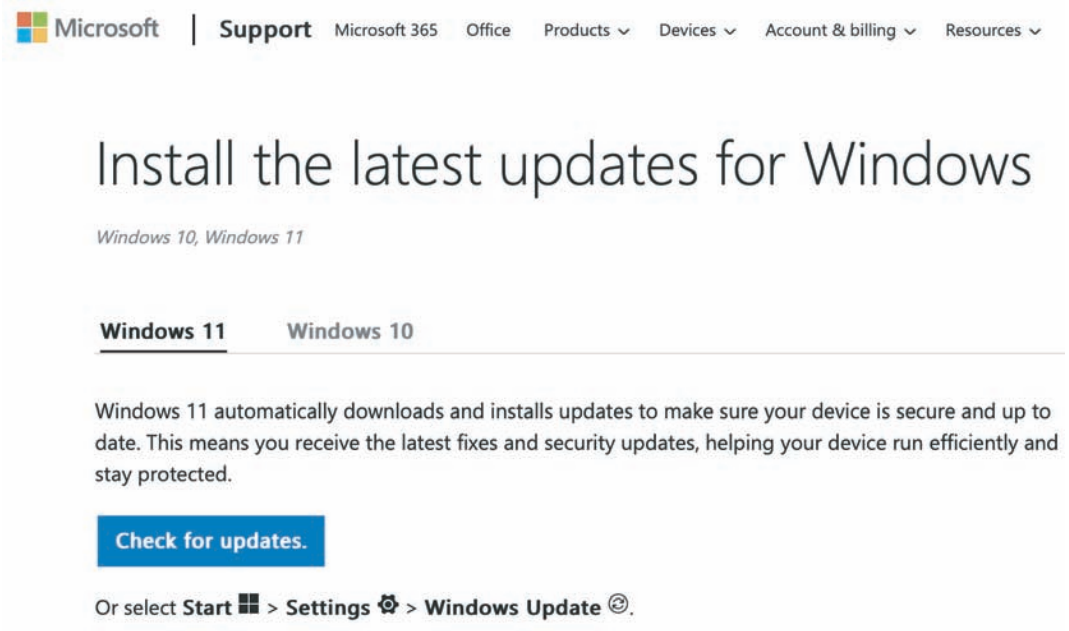
A release methodology also has some potential disadvantages. Users expect a rapid response to their problems and requests, but with a release methodology, new features or upgrades are available less often. Even when changes would improve system efficiency or user productivity, the potential savings must wait until the next release, which might increase operational costs.

Commercial software suppliers also provide maintenance releases, often called **service packs**, as shown in Figure 12-6. As Microsoft explains, a service pack contains all the fixes and enhancements made available since the last program version or service pack.

12.4.2 Version Control

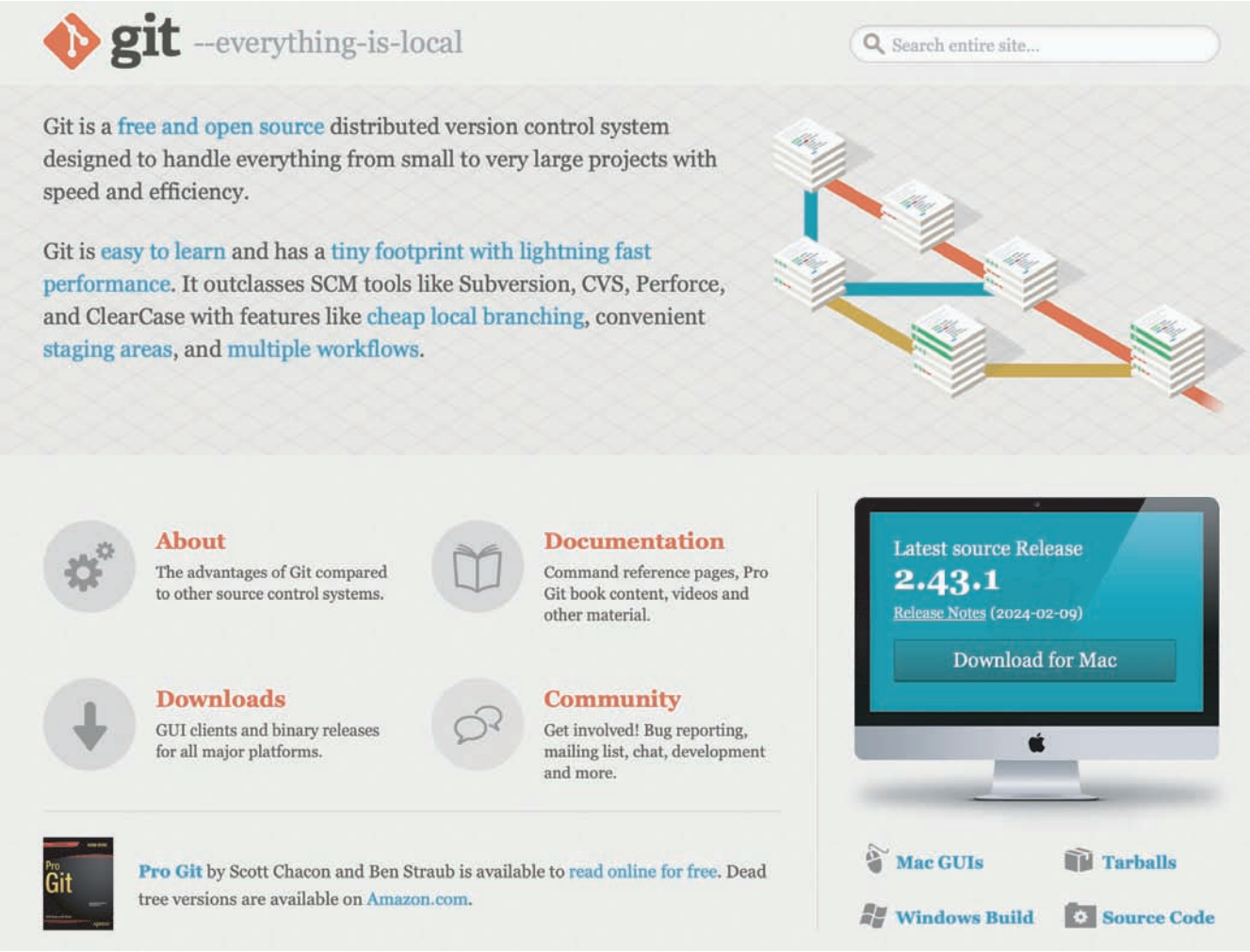
Version control is the process of tracking system releases, or versions. When a new system version is installed, the prior release is **archived**, or stored. If a new version causes a system to fail, a company can reinstall the previous version to restore operations. In addition to tracking system versions, the IT staff is responsible for configuring systems with several modules at various release stages. For example, an accounting system might have a one-year-old accounts receivable module that must interface with a brand-new payroll module.

Figure 12-6 A Microsoft service pack provides access to updated drivers, tools, security patches, and customer-requested product changes.
Source: Microsoft Corporation



Many firms use commercial applications to manage version control for complex systems. However, there are also numerous free and open-source alternatives. For example, one of the most popular version control systems is git, shown in Figure 12-7. Git is a free, open-source program designed for distributed systems. It is relatively easy to use, available across most major platforms, and supported by the developer community.

Figure 12-7 Git is a popular, free version control system.
Source: Git



12.4.3 Baselines

A **baseline** is a formal reference point that measures system characteristics at a specific time. Systems analysts use baselines as yardsticks to document features and performance during the systems development process. The three types of baselines are functional, allocated, and product.

The **functional baseline** is the system configuration documented at the beginning of the project. It consists of all the necessary system requirements and design constraints.

The **allocated baseline** documents the system at the end of the design phase and identifies any changes since the functional baseline. It includes testing and verification of all system requirements and features.

The **product baseline** describes the system at its beginning of operation. It incorporates any changes made since the allocated baseline and includes the results of performance and acceptance tests for the operational system.

12.5 System Performance Management

System performance management has evolved into a critical component for ensuring an organization's IT infrastructure satisfies current operational requirements and is equipped to handle future growth and emerging challenges. In the past, with organizations typically relying on a single central computer for data processing, monitoring and optimizing system efficiency was a more straightforward task. However, the modern business landscape has shifted toward utilizing intricate networks, client/server architectures, and cloud computing solutions to fulfill complex business operations. This shift means that a user at a client workstation is often engaging with an information system that relies on a diverse array of clients, servers, networks, and data repositories spread across various locations. Therefore, today's system performance is no longer determined by the capacity of a single machine but by the seamless integration and collective efficiency of all these interconnected components.

This intricate ecosystem requires a sophisticated approach to performance management that can address the complexities of modern IT environments and ensure that they operate at peak efficiency while remaining agile and scalable in the face of technological advancements and changing business demands. Organizations can identify potential bottlenecks, anticipate system requirements, and implement strategic improvements by focusing on key areas such as **fault management**, performance and workload measurement, and capacity planning. Through continuous assessment and adjustment, organizations can maintain optimal system performance, adapt to evolving technological landscapes, and meet the ever-changing needs of their business environment.

To ensure satisfactory support for business operations, the IT department must manage system faults and interruptions, measure system performance and workload, and anticipate future needs. IT managers use automated software and CASE tools to help with these tasks in many situations. Automated tools can provide valuable assistance during the system operation and support phases. Many CASE tools include system evaluation and maintenance features, such as:

- Performance monitors that provide data on program execution times
- Program analyzers that scan source code, provide data element cross-reference information, and help evaluate the impact of a program change
- Interactive debugging analyzers that locate the source of a programming error
- Reengineering tools
- Automated documentation capabilities
- Network activity monitors
- Workload forecasting tools

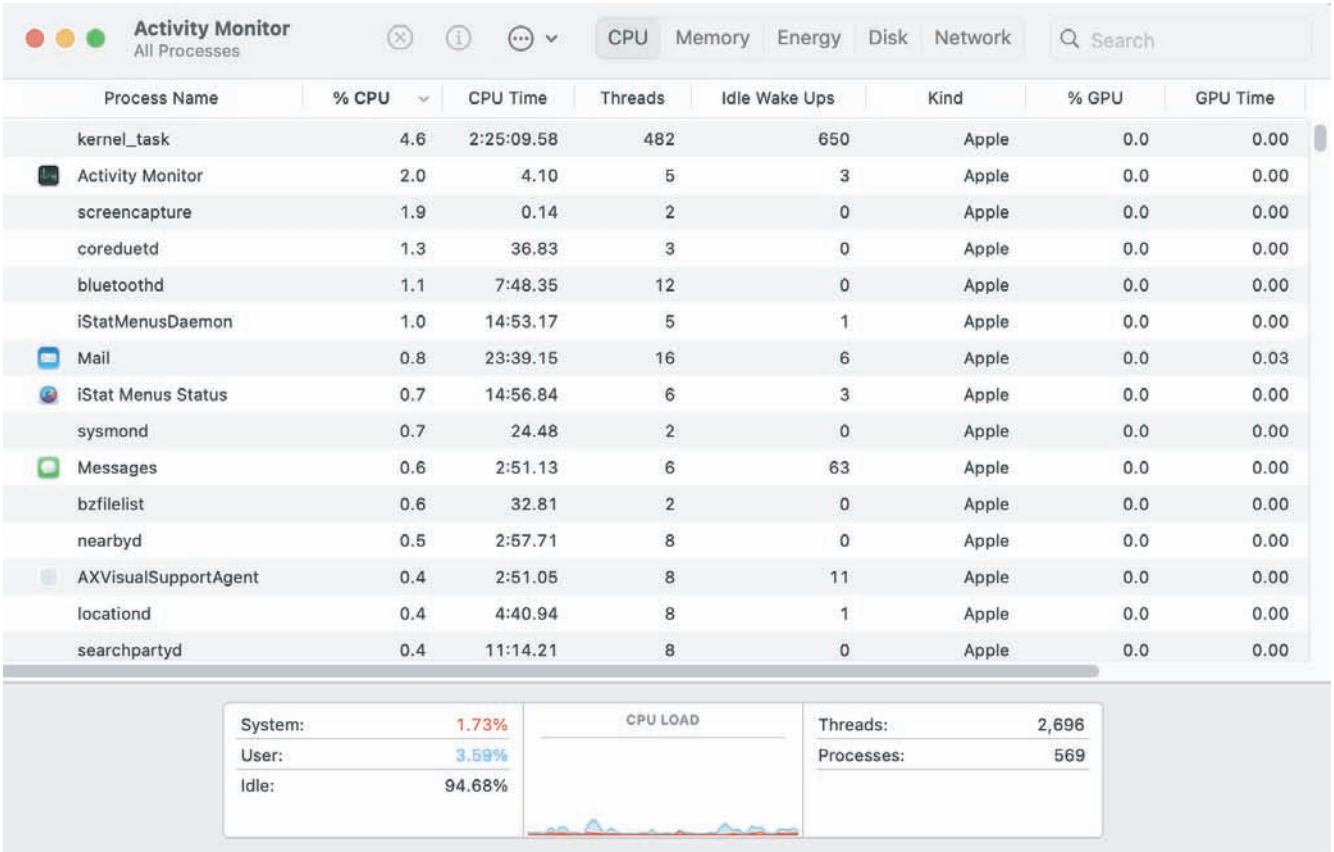
In addition to CASE tools, spreadsheet and presentation software can calculate trends, perform what-if analyses, and create attractive charts and graphs to display the results. IT planning is an essential part of the business planning process and often part of the presentations made to management.

12.5.1 Fault Management

No matter how well it is designed, every system will experience problems, such as hardware failures, software errors, user mistakes, and power outages. A system administrator must detect and resolve operational issues as quickly as possible. That task, often called fault management, includes monitoring the system for signs of trouble, logging all system failures, diagnosing the problem, and applying corrective action.

The more complex the system, the more difficult it can be to analyze symptoms and isolate a cause. In addition to addressing the immediate problem, evaluating performance patterns and trends is essential. For example, the Activity Monitor application shown in Figure 12-8 runs on Apple's macOS to display all running applications' CPU, memory, energy, disk, and

Figure 12-8 The Activity Monitor application on Apple's macOS displays the CPU, memory, energy, disk, and network activity of all running applications in real time.
Source: Apple Inc.



network activity in real time. Similar programs, such as Resource Monitor, are available on Windows. Fault management software can help identify underlying causes, speed up response time, and reduce service outages.

Although system administrators must deal with system faults and interruptions as they arise, the best strategy is to prevent problems by monitoring system performance and workload.

12.5.2 Performance and Workload Measurement

In e-business, slow performance can be as devastating as no performance. Network delays and application bottlenecks affect customer satisfaction, user productivity, and business results. Many IT managers believe network delays do more damage than actual stoppages because they occur more frequently and are difficult to predict, detect, and prevent. Customers expect reliable, fast response 24 hours a day, seven days a week. To support that level of service, companies use performance management software.

Many firms use **benchmark testing** to measure system performance, which utilizes a set of standard tests to evaluate system performance and capacity. In addition to benchmark testing, performance measurements called **metrics** can monitor the number of transactions processed in a given period, the number of records accessed, and the volume of online data. Network performance metrics include response time, bandwidth, throughput, and turnaround time.

Response Time

Response time is the overall time between a request for system activity and the delivery of the response. In the typical online environment, response time is measured from when the user presses the Enter key or clicks a mouse button until the requested screen display appears

or printed output is ready. Response time is affected by the system design, capabilities, and processing methods. If the request involves network or Internet access, response time is affected by data communication factors.

Online users expect an immediate response and are frustrated by any apparent lag or delay. Of all performance measurements, response time is the one that users notice and complain about most.

Bandwidth and Throughput

Bandwidth and **throughput** are closely related terms, and many analysts use them interchangeably. Bandwidth describes the quantity of data the system can transfer in a fixed period. Bandwidth requirements are expressed in bits per second. Depending on the system, bandwidth can be measured in **Kbps (kilobits per second)**, Mbps (megabits per second), or Gbps (gigabits per second). Analyzing bandwidth is similar to forecasting the hourly number of vehicles using a highway to determine the number of lanes required.

Throughput measures system performance under specific circumstances and is affected by network loads and hardware efficiency. Like bandwidth, throughput is expressed as a data transfer rate, such as Kbps, Mbps, or Gbps. As traffic jams delay highway traffic, throughput limitations can slow system performance and response time. That is especially true with graphics-intensive and web-based systems subject to Internet-related conditions.

In addition to the performance metrics explained above, system administrators measure many other performance characteristics. Although no standard set of metrics exists, several typical examples are:

- Arrivals—The number of items that appear on a device during a given observation time
- Busy—The amount of time that a given resource is unavailable
- Completions—The number of arrivals processed during a given observation period
- Queue length—The number of requests pending for a service
- Service time—The time it takes to process a given task once it reaches the front of the queue
- Think time—The time it takes an application user to issue another request
- Utilization—How much of a given resource was required to complete a task
- Wait time—The amount of time requests must wait for a resource to become available

Turnaround Time

Turnaround time is a crucial metric in centralized batch processing operations, which are common in tasks such as customer billing or processing credit card statements. It specifically refers to the duration from when a request for information is submitted to when it is completely fulfilled. Additionally, turnaround time is a key indicator of IT support or service quality. This measurement is vital for assessing the efficiency and effectiveness of IT services, providing a clear benchmark for the timeliness of response to user needs.

Within the IT department, several performance metrics, such as response time, bandwidth, throughput, and turnaround time, are meticulously tracked to assess system performance. This evaluation is critical before and after implementing system changes or adjusting to new business information requirements. The gathered performance data is significant in conducting cost–benefit analyses for proposed maintenance projects. Furthermore, it is instrumental in assessing the viability and remaining economic value of systems approaching the end of their useful life. This analysis helps organizations make informed decisions about system upgrades, replacements, or enhancements to efficiently meet current and future needs.

Finally, current performance and workload data are indispensable for **capacity planning**. Management relies on this data to strategically scale IT infrastructure and meet anticipated

business demands. By analyzing current system performance metrics, organizations can forecast future needs and identify potential bottlenecks before they impact operations. This proactive approach to capacity planning ensures that the IT infrastructure supports current workloads and is prepared to accommodate future growth, aligning IT capabilities with business objectives and ensuring sustained operational efficiency.

12.5.3 Capacity Planning

Capacity planning is a process that monitors current activity and performance levels, anticipates future activity, and forecasts the resources needed to provide desired levels of service.

The first step in capacity planning is to develop a current model based on the system's present workload and performance specifications. Future demand and user requirements are projected over one to three years. The model is analyzed to determine what is needed to maintain satisfactory performance and meet requirements. A technique called **what-if analysis** can assist in the process.

What-if analysis varies one or more elements in a model to measure the effect on other elements. For example, what-if analysis might be used to answer questions such as: How will response time be affected if more client workstations are added to the network? Will the client/server system be able to handle the growth in sales from the new website? What will be the effect on server throughput if more memory is added?

Powerful spreadsheet tools can also assist in performing what-if analyses. For example, Microsoft Excel contains a Goal Seek feature that determines what changes are necessary for one value to produce a specific result for another. In the example shown in Figure 12-9, a capacity planning worksheet indicates that the system can handle 3,840 web-based orders per day at 22.5 seconds each. Excel calculates this value automatically using the formula =86400/B4 for cell B5. (There are 86,400 seconds in a 24-hour day.) The user wants to know the effect on processing time if the number of transactions increases to 9,000. As the Goal Seek solution in the bottom screen shows, order processing must be performed in 9.6 seconds to achieve that goal.

During capacity planning, detailed information is needed about the number of transactions, the daily, weekly, or monthly transaction patterns, queries, and the number, type, and size of all generated reports. If the system involves a LAN, network traffic levels must be estimated to determine whether the existing hardware and software can handle the load. If the system uses a client/server design, performance and connectivity specifications must be examined for each platform.

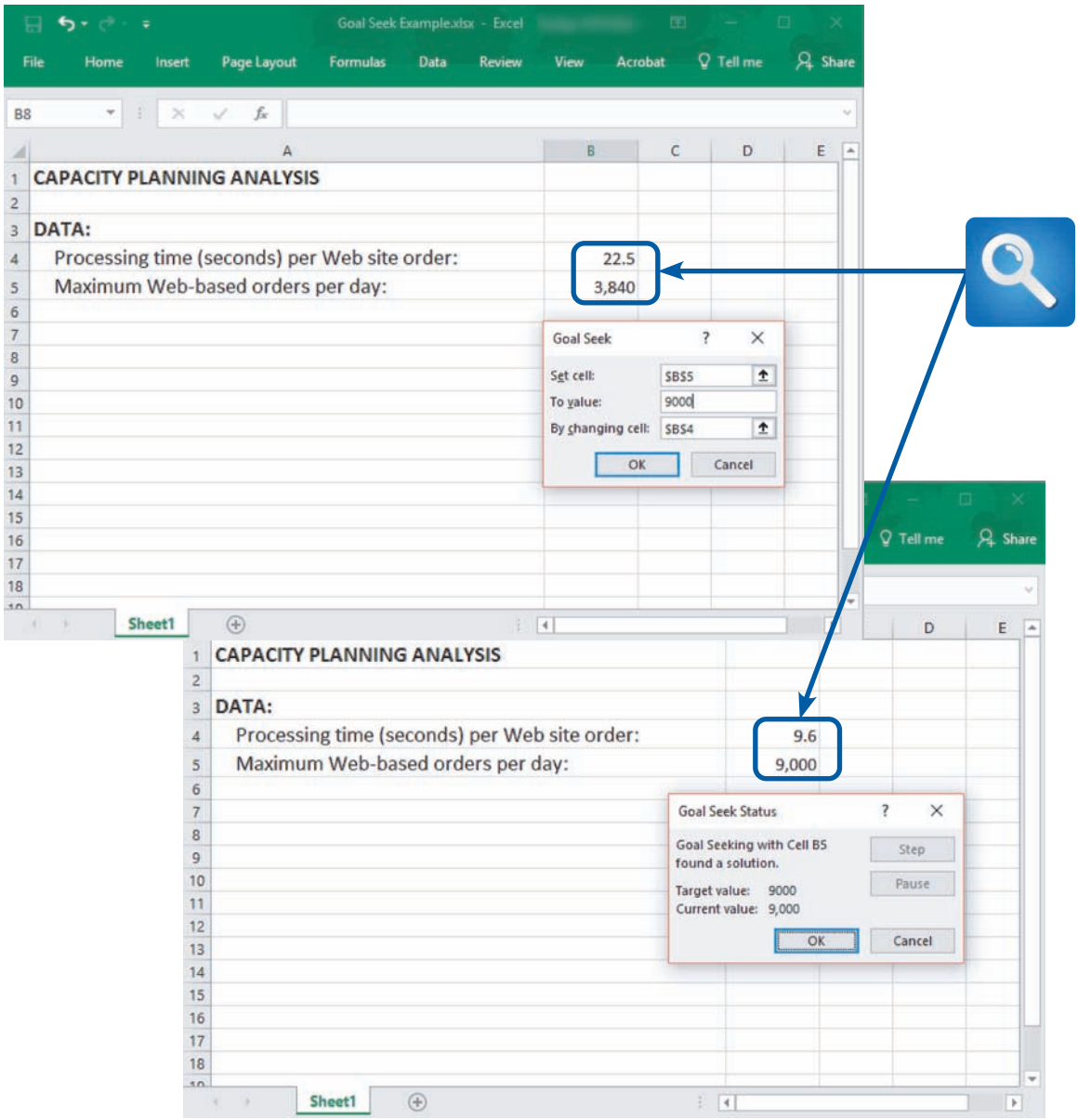
Most importantly, an accurate forecast of future business activities is needed. If new business functions or requirements are predicted, contingency plans should be developed based on input from users and management. The main objective is ensuring the system meets all future demands and effectively supports business operations. Some firms handle capacity planning themselves, while others purchase software such as Idera's Uptime Infrastructure Monitor, shown in Figure 12-10.

12.6 Quality Assurance in System Support

QA in system support is a multifaceted approach designed to ensure that IT systems within an organization operate reliably, efficiently, and in alignment with user expectations and business objectives. It involves a rigorous framework integrating QA strategies for establishing standards and protocols, testing practices for identifying and resolving issues, and mechanisms for incorporating user feedback for continuous system improvement. This structured approach ensures that every facet of IT support, from initial design and development phases to ongoing maintenance and user interaction, adheres to high-quality standards, aligning IT operations with current user needs and long-term business goals.

Figure 12-9 In this Goal Seek example, the user wants to know the effect on processing time if the number of daily transactions increases from 3,840 to 9,000.

Source: Microsoft Corporation



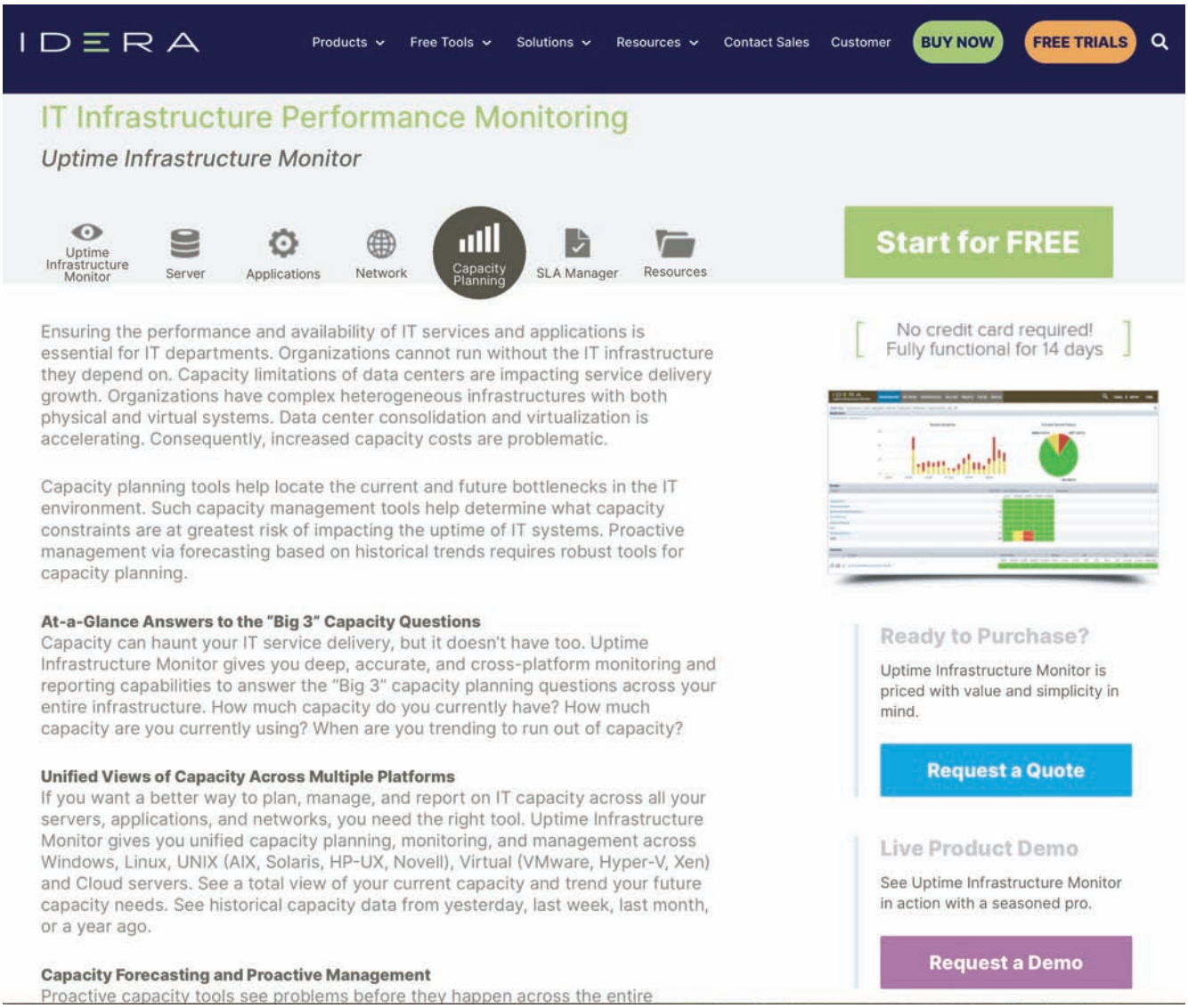
12.6.1 QA Strategies

QA strategies emphasize developing and implementing overarching strategies that guide QA activities across the entire life cycle of IT systems. These strategies establish a foundation for maintaining high-quality standards by applying best practices, policies, and procedures. QA strategies focus on embedding quality into every aspect of system support, from initial design and development to ongoing maintenance and user support. By setting clear quality objectives and methodologies, organizations can proactively address potential issues, ensuring systems are robust, secure, and capable of meeting evolving needs.

QA in Maintenance

QA in maintenance is an essential activity that focuses on the ongoing upkeep and enhancement of IT systems to ensure they function optimally and securely over time. This involves regular checks, updates, and the application of patches to address vulnerabilities,

Figure 12-10 Idera's Uptime Infrastructure Monitor is a capacity planning application.
Source: Idera



alongside the systematic identification and rectification of any defects or performance issues that may arise.

By incorporating QA practices into maintenance routines, organizations can prevent system performance deterioration, mitigate the risk of system failures, and ensure that IT infrastructure remains robust and reliable. This proactive approach to maintenance extends the lifespan of IT systems. It ensures they can adapt to evolving business needs and technological advancements, maintaining their relevance and effectiveness in supporting organizational objectives.

QA in User Support

QA in user support emphasizes the importance of delivering high-quality support services to users, ensuring their interactions with IT systems are seamless, efficient, and satisfactory. This dimension of QA involves establishing clear standards for response times, the accuracy of information provided, and the overall user experience during support interactions. By implementing QA measures in user support, organizations can enhance the effectiveness of their help desks, support portals, and troubleshooting services, increasing user satisfaction and confidence in the IT infrastructure.

Effective QA in user support is a pivotal mechanism for swiftly addressing and resolving user issues and systematically gathering feedback that informs future improvements. This feedback loop is integral to the process, enabling continuous enhancement of the support experience by identifying trends, pinpointing areas for improvement, and implementing targeted solutions based on actual user experiences. Over time, this iterative process of collecting feedback, analyzing it for insights, and making informed adjustments ensures that the quality of user support evolves to meet and exceed user expectations, fostering a culture of excellence and responsiveness within the organization's IT support services.

Continuous Improvement Processes

Continuous improvement is at the heart of a dynamic QA system, fostering an environment where feedback, performance data, and user experiences are continuously analyzed to identify system and process enhancement opportunities. This ongoing evaluation, learning, and improvement cycle ensures that IT systems and support services remain aligned with current user needs and business goals while being flexible enough to adapt to future challenges.

Through continuous improvement, organizations can incrementally enhance system functionality, efficiency, and user satisfaction, ensuring that their IT infrastructure evolves with technological advancements and changing market demands. This proactive approach optimizes IT performance and supports strategic business objectives by ensuring the organization's technology resources are always at the forefront of innovation and quality.

12.6.2 Testing Practices

Testing practices are an essential component of QA in system support, serving as the cornerstone for ensuring that IT systems are robust, secure, and capable of meeting the demands of users and the business environment. By implementing a comprehensive testing strategy supported by tools such as TestRail, shown in Figure 12-11, organizations can identify potential issues before they impact users, optimize system performance, and ensure that changes or updates do not introduce new problems. Effective testing practices help minimize system errors and downtime, enhance user satisfaction, and ensure system updates and enhancements do not introduce new issues.

Maintenance Testing

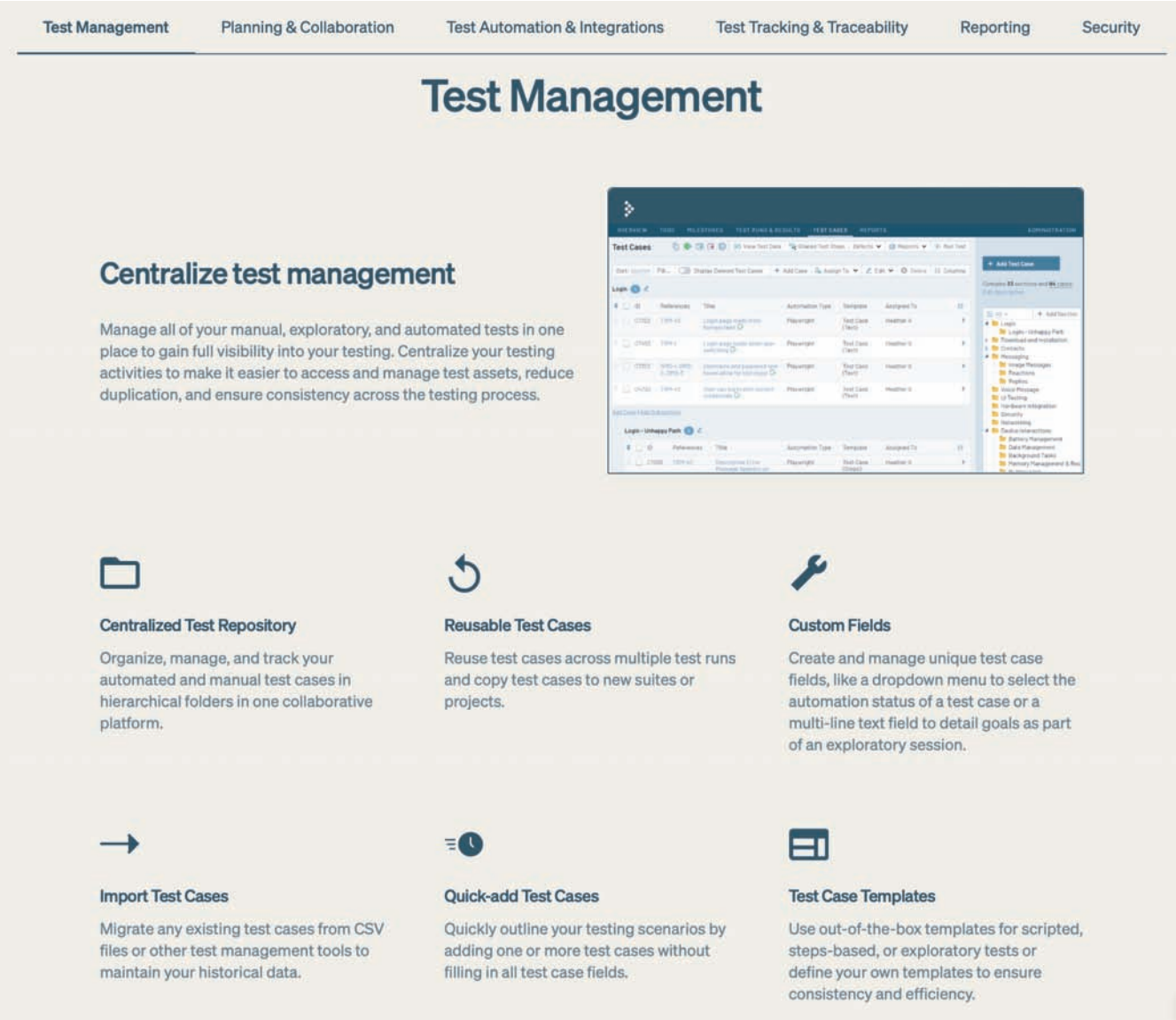
Maintenance testing ensures systems remain functional and efficient after routine updates or modifications. This type of testing is focused on verifying that maintenance activities, such as patch installations, configuration changes, or software updates, do not adversely affect system performance or introduce new issues. By thoroughly testing systems after maintenance, organizations can confirm that all components continue to operate as expected, maintaining the integrity and reliability of the IT infrastructure. This reassurance is vital for minimizing downtime and ensuring the system supports continuous business operations.

Furthermore, maintenance testing plays a pivotal role in detecting problems arising from routine maintenance tasks. Identifying and addressing these issues promptly prevents them from escalating into more significant problems that could disrupt business processes and degrade user satisfaction. As such, maintenance testing is an integral part of a proactive maintenance strategy, ensuring that systems are up to date and continue to meet the operational and performance standards required by the organization.

Regression Testing

Regression testing is an indispensable testing practice designed to ensure that new code changes, updates, or enhancements do not negatively impact the system's existing functionality. This type of testing is crucial whenever modifications are made to the IT system, as it helps to confirm that introducing new features or bug fixes does not introduce unintended consequences elsewhere. By systematically retesting the unchanged parts of the system, organizations can ensure that updates enhance the system without compromising its existing capabilities, maintaining a consistent and reliable user experience.

Figure 12-11 TestRail helps orchestrate a comprehensive QA strategy on a centralized platform, including test management activities.
Source: TestRail

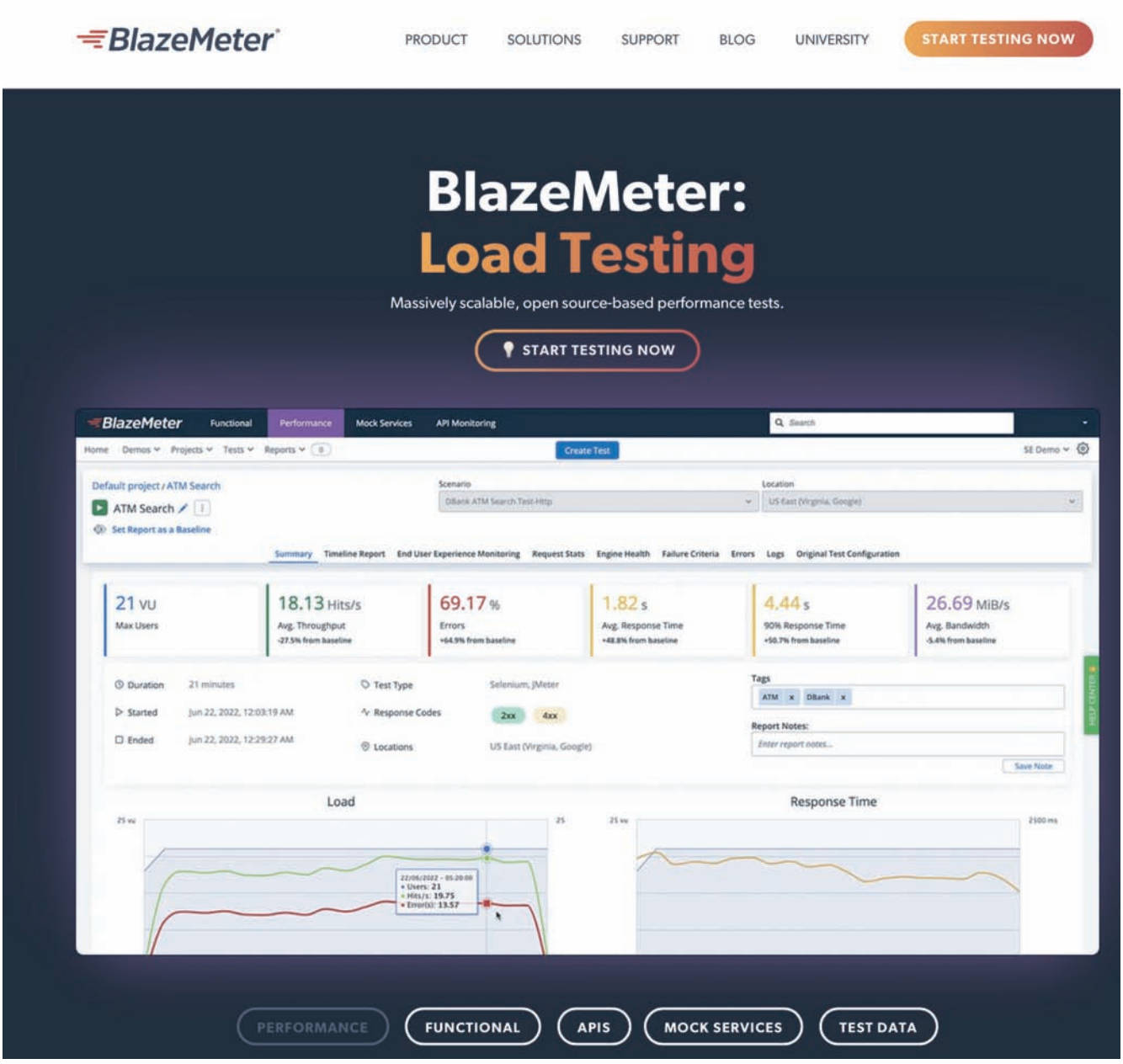


The importance of regression testing extends beyond maintaining system functionality; it also reinforces user confidence in the IT system. When users are assured that system updates will not disrupt their workflows or introduce new issues, their trust in the IT infrastructure and the team managing it increases. This trust is crucial for fostering a positive relationship between IT departments and end users, encouraging open communication and feedback that can lead to further system improvements. Thus, regression testing safeguards system integrity and plays a crucial role in building a collaborative and user-centric IT culture.

Performance Testing

Performance testing is essential for assessing the efficiency and scalability of IT systems under various conditions. It evaluates the system's speed, responsiveness, and stability under a specific workload, providing valuable insights into system capacity limits and overall performance. Through performance testing, aided by tools such as BlazeMeter (see Figure 12-12), organizations can identify bottlenecks and areas that require optimization, ensuring that the system can handle expected user loads without degradation in performance. This testing is crucial for maintaining a seamless user experience, especially in applications where performance is vital to business operations.

Figure 12-12 Perforce's BlazeMeter lets you run scalable, open-source-based performance tests.
Source: Perforce Software, Inc.



Additionally, performance testing helps organizations plan for future growth and scalability. By understanding their IT systems' current performance capabilities and limitations, decision makers can make informed investments in infrastructure upgrades or optimizations supporting increased demand. This foresight is invaluable for strategic planning, allowing organizations to proactively address potential performance issues before they impact users or hinder business growth. In this way, performance testing ensures systems meet current demands and prepares organizations to adapt to future challenges, supporting long-term operational and strategic goals.

12.6.3 Feedback and Improvement

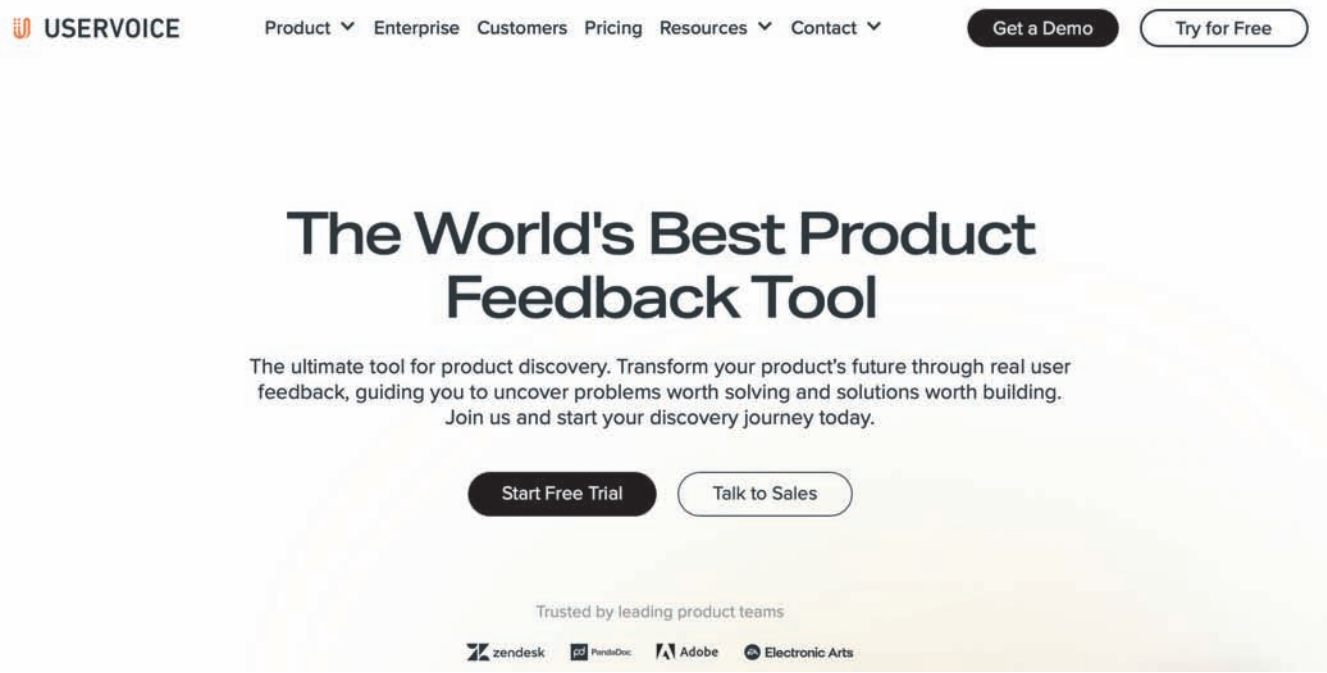
This component underscores the critical role of user feedback in the continuous evolution and enhancement of IT systems. It ensures that systems meet current operational and user needs and are refined and adapted over time to address emerging requirements and challenges.

By carefully gathering, analyzing, and acting on feedback, organizations can create a dynamic IT environment that aligns with user expectations and business objectives, fostering a culture of continuous improvement.

Collecting User Feedback

Collecting user feedback is the foundational step in the feedback and improvement cycle and is essential for understanding how end users interact with the IT system. This phase involves a systematic approach to gathering insights about user experiences, their challenges, their suggestions for system enhancements, and their overall satisfaction levels. Organizations employ various methods, including surveys, interviews, help desk reports, participation in user forums, and tools such as UserVoice, shown in Figure 12-13. Such active solicitation of feedback is crucial for acquiring a comprehensive, user-centric perspective on the system's performance and usability.

Figure 12-13 UserVoice is a versatile tool for collecting and analyzing product feedback.
Source: UserVoice, Inc.



The direct input obtained from users through these feedback mechanisms is invaluable. It enables organizations to pinpoint areas within the IT system that require improvement, aligning the system more closely with user needs and expectations. This feedback highlights immediate issues that may affect user satisfaction and offers suggestions for enhancing the system's functionality and overall user experience. By leveraging this wealth of information, organizations can make informed decisions about where to focus their improvement efforts, ensuring that the IT system evolves to continually meet and exceed user requirements.

Feedback Analysis

Feedback analysis is a pivotal step that entails meticulously examining the collected feedback, identifying prevailing trends, pinpointing specific issues, and discovering opportunities for enhancing the system. This phase is instrumental in converting raw feedback into actionable insights to guide improvement. By applying quantitative and qualitative analysis techniques, analysts can sift through the feedback, prioritizing it based on the frequency of comments, the impact of reported issues, and the practicality of implementing proposed changes. This rigorous analysis is essential for distilling the vast array of user feedback into a focused set of priorities that can drive meaningful improvements.

Analyzing feedback allows organizations to deeply understand the needs and expectations of their users, ensuring that every piece of feedback is given due consideration. By systematically

evaluating user input, organizations can identify the most critical areas for enhancement, aligning their efforts with the aspects of the system that users feel most strongly about. This structured approach to feedback analysis ensures that improvement initiatives are directed toward making changes that will significantly enhance user satisfaction and overall system performance. Consequently, this helps organizations make informed decisions about where to allocate resources for system improvements and fosters a responsive and user-centered IT environment.

Implementing Improvements

Implementing improvements marks the crucial phase in which insights derived from user feedback and thorough analysis are implemented through tangible actions. This stage might encompass a range of activities, such as modifying existing systems, bolstering support services, or rolling out new features specifically designed to meet users' expressed needs. Such implementation is inherently collaborative, necessitating seamless coordination among various teams within the organization, including those responsible for IT, development, and user support. This cooperative effort is key to effectively translating user feedback into meaningful system enhancements that address user concerns and suggestions.

This phase is pivotal in completing the feedback loop, signaling to users that their contributions are acknowledged and impactful. By openly communicating the changes implemented in response to feedback, organizations underline their value on user input and foster an environment that encourages ongoing participation. This feedback collection, analysis, and implementation cycle allows IT systems to adapt and improve dynamically, ensuring they consistently meet user requirements and drive the organization's success. Through this process, organizations enhance their IT systems' functionality and user experience and build stronger relationships with their users based on trust and mutual respect.

12.7 Security Management in Maintenance

Security management in maintenance integrates three essential components—security updates, security audits, and data protection—into a cohesive strategy to safeguard IT systems during maintenance activities. By regularly applying security updates, organizations proactively close vulnerabilities to prevent cyber threats, ensuring systems remain robust against attacks. Concurrently, security audits offer a thorough examination of the IT infrastructure, identifying potential weaknesses and evaluating the efficacy of current security measures, thereby guiding the enhancement of security protocols. The emphasis on data protection ensures sensitive information is shielded from unauthorized access and breaches, maintaining data integrity and compliance with regulatory standards. This comprehensive approach fortifies systems against immediate threats. It establishes a proactive security posture that is essential for the long-term resilience and trustworthiness of the organization's IT infrastructure amidst an ever-evolving cybersecurity landscape, as discussed in more detail in the next chapter.

12.7.1 Security Updates

Security updates are integral to maintaining the integrity and resilience of IT systems within an organization. They involve the systematic identification, assessment, and application of patches to software and hardware vulnerabilities that, if left unaddressed, could serve as potential gateways for cyber threats and attacks. This process is essential for safeguarding sensitive data and system operations and ensuring the organization's IT infrastructure complies with evolving security standards and regulations. By prioritizing security updates, organizations can significantly reduce their exposure to cyber risks, enhance their defense mechanisms against potential security breaches, and maintain the trust of stakeholders and customers.

Identifying Security Risks

The first step in the security update process involves proactively identifying security risks that threaten the organization's IT systems. This entails a comprehensive assessment of the IT environment to pinpoint vulnerabilities, from outdated software to flaws in system

configurations. IT security teams can detect existing and emerging threats using various tools and techniques, such as vulnerability scanners and threat intelligence feeds. This ongoing vigilance is crucial for staying ahead of potential attackers by understanding the landscape of vulnerabilities that could be exploited.

Once risks are identified, the critical task of prioritizing them based on their potential impact on the organization's operations and data integrity begins. This prioritization helps allocate resources effectively, focusing efforts on mitigating risks with the most severe consequences in the event of a breach. By systematically addressing the most critical vulnerabilities first, organizations can better manage their cybersecurity posture and reduce the likelihood of significant disruptions.

Patch Management

Patch management is a core component of implementing security updates and systematically deploying patches to fix vulnerabilities within software and systems. This process requires a well-structured approach to apply patches promptly and efficiently without disrupting system operations. Effective patch management strategies include testing patches in a controlled environment to verify compatibility and functionality before widespread deployment, minimizing the risk of unintended system issues. Tools such as SolarWinds Patch Manager, shown in Figure 12-14, can aid the process.

Maintaining an organized schedule for patch updates and closely monitoring the patching process are essential to ensure no vulnerability is overlooked. Communication between IT teams and system users is also crucial to prepare for potential downtime or changes in system performance during the patch application. Through diligent patch management, organizations can safeguard their systems against known vulnerabilities, keeping their IT infrastructure secure and operational.

Compliance and Standards

Adhering to compliance and standards is a significant aspect of the security update process, ensuring that organizations meet legal, regulatory, and industry-specific security requirements. Regular updates and patches often include enhancements that address new compliance mandates, helping organizations avoid potential fines and legal issues. Keeping systems aligned with these standards is a matter of legal obligation and a best practice for maintaining high security and data protection levels.

Staying informed about the latest security standards and compliance requirements necessitates a collaborative effort among compliance officers, IT security teams, and organizational leadership. Implementing a framework for continuous compliance monitoring and improvement enables organizations to integrate new standards into their security practices seamlessly. By prioritizing compliance within the security update process, organizations can bolster their defenses against cyber threats while ensuring their operations remain in good standing with regulatory bodies.

Case in Point 12.2

HealthFirst Clinics

You are a systems analyst at HealthFirst Clinics, and you've been alerted to a zero-day vulnerability impacting a critical component of your organization's IT infrastructure. You know that a zero-day exploit is an attack that targets previously unknown vulnerabilities in software or hardware. It is executed before developers have had the opportunity to create and distribute a fix, which means the unforeseen security flaw poses an immediate risk to the system's integrity and the confidentiality of sensitive data. Faced with the urgency of this issue, your task is to develop and implement a comprehensive response strategy to protect your infrastructure against potential exploits, even before a patch becomes available.

Figure 12-14 SolarWinds Patch Manager adds functionality to Microsoft Windows Server Update Services (WSUS) and Microsoft System Center Configuration Manager (SCCM).
Source: SolarWinds Worldwide, LLC

12.7.2 Security Audits

Security audits are a critical component of an organization's cybersecurity strategy, offering a comprehensive evaluation of its information systems to ensure they are protected against potential threats and vulnerabilities. These audits involve systematically reviewing security policies, procedures, and technical controls to identify weaknesses that cyber attackers could exploit. By conducting regular security audits, organizations can gain valuable insights into their security posture, uncovering gaps in their defenses and areas that require improvement. This proactive approach helps safeguard sensitive data and systems and ensures compliance with industry regulations and standards, strengthening the organization's security framework.

Audit Procedures

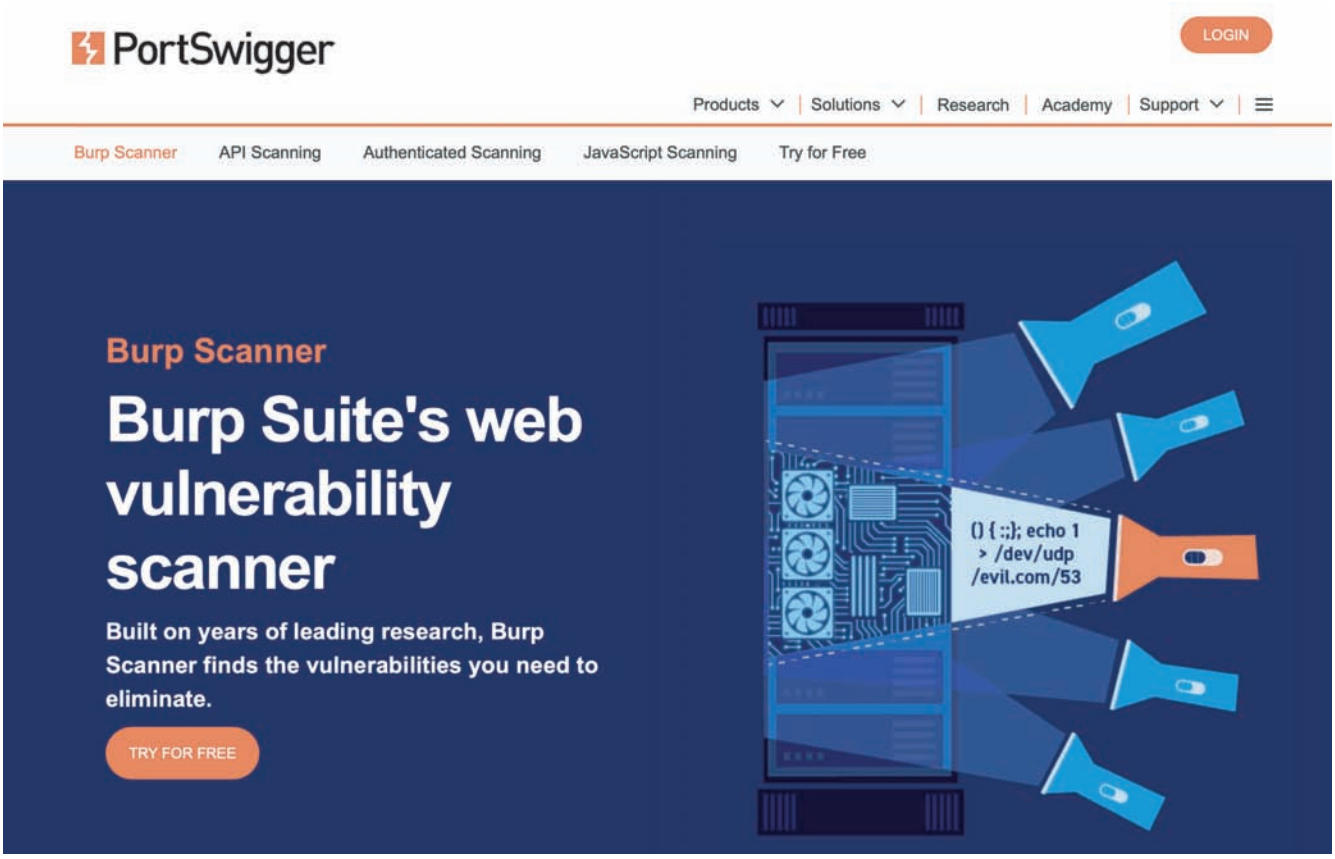
Audit procedures form the backbone of the security audit process, providing a structured approach to evaluating the effectiveness of an organization's cybersecurity measures. These procedures thoroughly examine security policies, access controls, network configurations, and other critical components of the IT infrastructure. By systematically assessing these areas, auditors can identify inconsistencies, unnecessary exposures, and practices that do not align with best security practices or compliance requirements. This comprehensive review allows organizations to understand the robustness of their security measures and identify areas for enhancement.

The execution of audit procedures requires meticulous planning and coordination, ensuring that all aspects of the organization's IT environment are scrutinized. Auditors often employ various tools and methodologies to gather evidence, from interviews with staff to examining system logs and configurations. This detailed investigation helps paint a complete picture of the organization's security practices, facilitating the identification of vulnerabilities that must be addressed to fortify its defenses against cyber threats.

Vulnerability Scanning

Vulnerability scanning is an essential technique used in security audits to automatically identify and report potential vulnerabilities within an organization's network and systems. This process involves using specialized software tools, such as PortSwigger's Burp Scanner shown in Figure 12-15, that scan the IT infrastructure to detect known vulnerabilities such as outdated software, misconfigurations, and weak passwords. By providing a snapshot of the organization's vulnerability landscape, these scans enable IT security teams to prioritize risks based on their severity and potential impact on the organization.

Figure 12-15 PortSwigger's Burp Scanner finds vulnerabilities in web applications.
Source: PortSwigger Ltd.



Regular vulnerability scanning is crucial for maintaining an up-to-date understanding of the organization's security posture. As new vulnerabilities are continually being discovered, frequent scans ensure that emerging threats are identified promptly, allowing for timely remediation. Additionally, vulnerability scanning results can guide the development of mitigation strategies, focusing efforts on areas that pose the greatest risk to the organization's security and compliance status.

Mitigation Strategies

Developing and implementing effective mitigation strategies is fundamental to the security audit process. Once vulnerabilities and security gaps have been identified through audit procedures and vulnerability scanning, organizations must take decisive action to address these issues. Mitigation strategies may include deploying patches, changes in access controls, enhancements to network security measures, or updates to security policies and training programs. These strategies are tailored to address the identified vulnerabilities, reducing the risk of exploitation and enhancing the organization's security posture.

Formulating mitigation strategies requires a collaborative effort across various departments within the organization, ensuring that remediation efforts are comprehensive and aligned with business objectives. Prioritization is vital, with resources allocated to address the most critical vulnerabilities first. Moreover, the effectiveness of these strategies must be continually assessed through follow-up audits and reviews, ensuring that the measures implemented effectively reduce the organization's exposure to cyber risks. This iterative assessment, mitigation, and review process is essential for maintaining a robust and responsive security framework that adapts to the evolving cyber threat landscape.

12.7.3 Data Protection

Data protection is crucial to cybersecurity and IT management, focusing on safeguarding sensitive information from loss, corruption, and unauthorized access. It encompasses a range of practices and strategies designed to ensure data integrity, confidentiality, and availability. As organizations increasingly rely on digital information to conduct their operations, implementing robust data protection measures has never been more critical. These measures help organizations comply with legal and regulatory requirements, maintain customer trust, and protect the organization's reputation. Effective data protection strategies involve comprehensive backup and **recovery** processes and meticulous attention to privacy considerations. These processes ensure that data is secure and recoverable in the event of an incident and used in compliance with privacy laws and expectations.

Backup Strategies

Backup strategies are essential for data protection, providing a safety net against data loss due to system failures, cyberattacks, or human error. A well-designed backup strategy involves regularly creating data copies stored securely in separate locations from the primary data. This redundancy allows organizations to restore data quickly and efficiently, minimizing downtime and the potential impact on operations. The choice of backup frequency, storage media, and the scope of data to be backed up should be tailored to the organization's specific needs and risk profile, ensuring that critical information is prioritized and backup procedures are aligned with business continuity objectives.

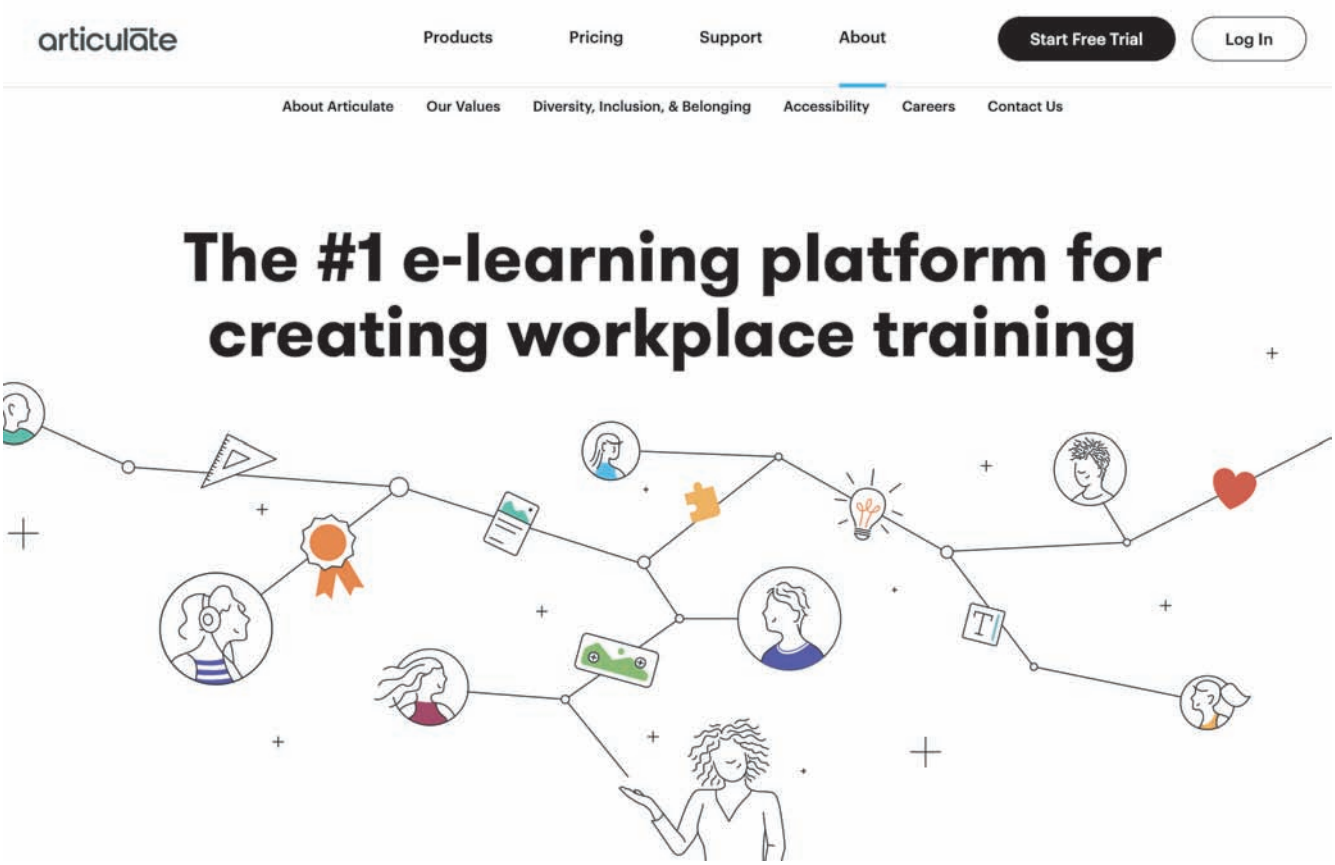
Testing backup systems regularly is crucial to ensure they work as expected when needed. This involves verifying the integrity of the backup data and practicing the restoration process to ensure that data can be quickly and accurately recovered. By establishing and maintaining rigorous backup strategies, organizations can safeguard against data loss, ensuring they can continue operations despite unforeseen challenges. This proactive approach to data protection underscores the organization's commitment to data integrity and operational resilience.

Data Recovery Processes

Data recovery processes are a critical component of data protection, detailing the steps to restore data from backups in the event of loss or corruption. Effective recovery processes are characterized by clearly defined procedures, roles, and responsibilities, ensuring that data can be restored swiftly and with minimal errors. These processes must be designed to meet the organization's **recovery time objectives (RTOs)** and **recovery point objectives (RPOs)**, which dictate maximum tolerable downtime and data loss. Organizations can ensure that data recovery efforts align with business continuity requirements by adhering to these objectives.

The success of data recovery processes often depends on regular training and drills for the IT staff responsible for executing them. Simulated data loss scenarios, such as those created with training tools like Articulate Storyline 360 shown in Figure 12-16, can help identify potential bottlenecks in the recovery process and provide valuable practice to IT personnel, reducing the likelihood of mistakes during an actual data recovery operation. Continuous improvement of data recovery processes, based on lessons learned from drills and actual incidents, is essential for maintaining an effective data protection strategy. This iterative approach ensures that organizations are always prepared to respond to data loss incidents, minimizing their impact on operations and maintaining the trust of stakeholders.

Figure 12-16 Articulate Storyline 360 is a leading e-learning authoring tool for custom course creation.
Source: Articulate Global Inc.



Privacy Considerations

Privacy considerations are increasingly at the forefront of data protection strategies, driven by a complex landscape of privacy laws and regulations, such as GDPR in the European Union and CCPA in California. Organizations must ensure that their data protection practices comply with these regulations, which often require measures to protect personal data from unauthorized access and to ensure that data is processed lawfully, transparently, and for legitimate purposes. This includes implementing strong access controls, data encryption, and

privacy by design principles, where privacy considerations are integrated into the development and operation of IT systems.

Beyond legal compliance, addressing privacy considerations in data protection strategies also involves fostering a culture of privacy awareness among employees. Regular training and awareness programs can help staff understand their data privacy responsibilities and the importance of adhering to organizational policies and procedures. By prioritizing privacy alongside security in data protection efforts, organizations can better protect sensitive information, reduce the risk of data breaches, and build stronger relationships with customers based on trust and respect for their privacy.

12.8 Documentation and Knowledge Management

Documentation and knowledge management within an IT framework are essential for capturing, maintaining, and disseminating information crucial to system support and user engagement. This broad domain is restructured into three refined subsections to streamline the approach to effectively creating, managing, and utilizing documentation and knowledge resources.

12.8.1 Comprehensive Documentation Strategy

Creating effective documentation encompasses setting standards that ensure consistency and clarity across all materials. This strategy includes the development of user manuals and guides that provide end users with easy-to-understand instructions and insights into the system's functionalities. Moreover, technical documentation for maintenance is crafted to help IT professionals maintain, troubleshoot, and enhance the system. This comprehensive approach ensures that all stakeholders have access to the necessary information tailored to their specific needs and roles, facilitating smoother operations and system mastery.

Setting Documentation Standards

Establishing clear and consistent documentation standards is vital for creating a unified approach to managing and disseminating information within an organization. These standards act as a blueprint for all documentation efforts, ensuring a cohesive format, structure, and style regardless of the document type—user manuals, technical guides, or maintenance protocols. This uniformity is essential not only for the aesthetic and professional presentation of documents but also for enhancing their functionality and accessibility. Standards facilitate ease of navigation and comprehension, enabling users to quickly locate and understand the information they require. For IT staff, these guidelines simplify creating, updating, and managing documentation, ensuring that all materials meet organizational expectations and user needs.

Adherence to documentation standards significantly enhances the overall usability of the documentation produced by an organization. When users encounter a familiar structure and presentation style across different documents, their ability to process and apply the information improves. This consistency reduces cognitive load and the learning curve, making users more likely to engage with the documentation and successfully find the answers to their queries. Furthermore, for IT staff, having a set of predefined standards streamlines the documentation process. It eliminates guesswork about formatting and content organization, allowing for more efficient document creation and maintenance. This systematic approach to documentation ensures that all information, whether intended for end-user guidance or technical support, is accessible, understandable, and directly valuable for its intended audience, thereby enhancing the overall effectiveness of the organization's documentation strategy.

Developing User Manuals and Guides

User manuals and guides facilitate the effective use of IT systems by providing end users with comprehensive instructions and information. Crafted with the end user in mind, these documents aim to bridge the gap between complex IT functions and user proficiency. To

achieve this, they are meticulously written in clear, nontechnical language that demystifies IT jargon, making the content accessible to individuals with varying technical expertise. The structure of these manuals and guides is carefully considered to ensure ease of navigation, enabling users to quickly locate specific information or step-by-step instructions relevant to their needs. Visuals, such as diagrams, flowcharts, and screenshots, are crucial in enhancing comprehension, offering users a visual representation of the content that can simplify complex processes or clarify step-by-step instructions. Sources such as the Association for Computing Machinery (ACM) Special Interest Group on Design of Communication (SIGDOC), shown in Figure 12-17, previously known as the ACM Special Interest Group on Documentation, provide guidance on creating effective system documentation.

Figure 12-17 The mission of ACM SIGDOC is to "advance the state of knowledge, encourage the research, and support the interdisciplinary practice of the design of communication."
Source: ACM SIGDOC

The ultimate goal of user manuals and guides is to empower users to handle common issues and tasks independently without the immediate need to contact help desk support. This empowerment not only boosts user confidence in navigating the IT system but also significantly reduces the workload on help desk personnel, allowing them to focus on more complex queries and technical issues. As users become more self-reliant, there is a notable improvement in overall user satisfaction, stemming from their ability to solve problems swiftly and efficiently. Furthermore, well-designed manuals and guides contribute to a positive user experience, fostering a sense of autonomy and competence among users. By investing in high-quality, user-friendly documentation, organizations can enhance the effectiveness of their IT support, improve user engagement with IT systems, and ultimately achieve higher levels of user satisfaction and system usability.

Technical Documentation for Maintenance

Technical documentation is a critical resource for IT staff, offering a deep dive into the inner workings of an organization's IT systems. Aimed specifically at those tasked with maintenance and troubleshooting, this type of documentation provides exhaustive details on system architecture, the underlying codebase, application programming interfaces (APIs), and the various points of integration that enable different components to work seamlessly together. Additionally, it outlines the standard operating procedures for routine maintenance tasks, system updates, and the resolution of any issues that may arise. Such detailed documentation is indispensable for IT professionals, as it gives them the knowledge to pinpoint and address system vulnerabilities or malfunctions swiftly. Moreover, it lays the groundwork for the successful implementation of system upgrades and the introduction of new features, ensuring that any modifications are made with a comprehensive understanding of the system's foundational structure.

The importance of maintaining adequate technical documentation cannot be overstated. It facilitates the prompt identification and rectification of system issues and is pivotal in minimizing potential downtime. This, in turn, contributes to the overall robustness and security of the IT infrastructure. By ensuring that technical documentation is comprehensive and regularly updated, organizations can equip their IT teams with the resources necessary to adapt to and overcome the challenges of an ever-evolving technological landscape. This proactive approach to documentation management supports IT systems' continuous improvement and scalability, safeguarding against obsolescence and ensuring that the organization's technological assets remain aligned with its long-term strategic goals. Through the diligent upkeep of technical documentation, organizations can foster a culture of efficiency and resilience within their IT departments, ultimately enhancing system reliability and performance.

12.8.2 Knowledge Base and Content Accessibility

Knowledge base development goes beyond merely compiling information; it involves creating a structured and searchable repository that houses a wealth of knowledge, from troubleshooting articles to best practices. Content management strategies are employed to organize this information effectively, ensuring that users can find what they need quickly and efficiently. Access control and security measures are paramount in protecting sensitive information while making the knowledge base accessible to authorized users. The goal is to create a robust knowledge base and make sure that the content within is easily accessible, secure, and up to date, serving as a vital resource for both self-service support and IT staff training purposes.

Building a Knowledge Base

The construction of a knowledge base represents a strategic initiative by organizations to centralize and systematize essential information, thereby streamlining access for users and IT staff. This digital repository, encompassing extensive content from FAQs and troubleshooting guides to comprehensive explanations of system functions, becomes a pivotal tool in the IT support ecosystem. By meticulously assembling and categorizing this wealth of information, organizations facilitate a self-service approach for users seeking answers. This autonomy empowers users by enabling them to resolve issues independently and significantly lightens the load on help desk teams, allowing them to concentrate on more complex inquiries. The ability for users to independently navigate and utilize the knowledge base for solutions enhances their overall experience and efficiency in problem resolution.

A well-curated knowledge base transcends the traditional notion of a static collection of documents, embodying a living, evolving entity that grows in tandem with the IT systems it describes. It is regularly enriched with fresh content reflecting the latest updates, fixes, and best practices, ensuring the information remains current and relevant. This ongoing process of renewal and expansion requires a commitment to continuous improvement and adaptation,

responding to new technological developments and user feedback. As the knowledge base evolves, it becomes more comprehensive and valuable, fostering a culture of learning and knowledge sharing within the organization. Through this dynamic resource, organizations can effectively address today's challenges while preparing for tomorrow's technological demands, ensuring that both users and IT staff have the knowledge they need at their fingertips.

Content Management Strategies

Implementing effective content management strategies is vital for a knowledge base's relevance and usability. Such strategies ensure that the repository remains valuable to users and IT staff by keeping the content organized, easily navigable, and current. Categorizing content in a user-friendly manner allows individuals to quickly locate the specific information they need without sifting through irrelevant data. This organization is complemented by routine reviews and updates that identify and remove obsolete information while integrating recent developments and insights. This cycle of continuous content refinement ensures that the knowledge base accurately reflects the current state of the system it supports, making it a reliable resource for solving problems and learning about system functionalities.

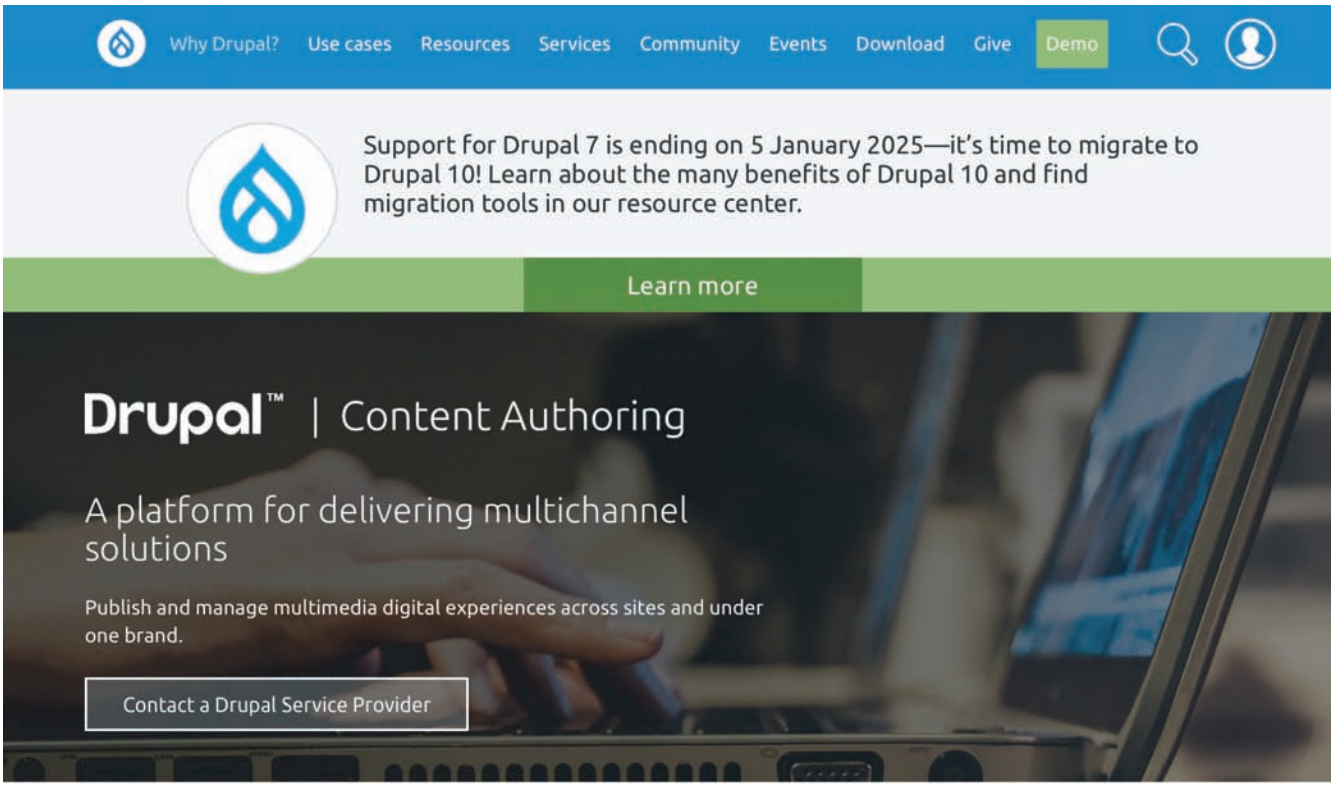
A truly effective content management strategy recognizes and accommodates its user base's varied preferences and needs. By presenting information in multiple formats—such as written text for those who prefer detailed explanations, diagrams for visual learners, and video tutorials for users who benefit from step-by-step guidance—organizations can significantly enhance the accessibility and effectiveness of their knowledge base. This approach ensures that users can engage with content in the way that best suits their learning style, thereby improving the overall user experience. The ultimate aim is to craft a knowledge base that is not only exhaustive and informative but also user-centric, facilitating effortless information retrieval. Through thoughtful content management, organizations can create a resource that empowers users with the knowledge they need in the most helpful format, thereby maximizing the utility and impact of the knowledge base. As shown in Figure 12-18, content management tools such as Drupal support the creation of a knowledge base using various content types and media formats.

Access Control and Security

Access control and security are critical pillars in effectively managing a knowledge base, ensuring that the repository remains a trusted and secure resource for information. In today's digital age, where data breaches are increasingly common, the importance of protecting sensitive information cannot be overstated. Implementing stringent access control mechanisms is essential for preventing unauthorized access to the knowledge base and safeguarding confidential information. This process involves a detailed configuration of user roles and permissions, determining who can view, edit, or manage specific content within the knowledge base. Such granular control not only helps maintain the security of sensitive data but also ensures that users have access to the information pertinent to their roles and needs, thereby enhancing the overall functionality and effectiveness of the knowledge base.

Moreover, the security of a knowledge base is not a one-time effort but a continuous process that requires regular monitoring and updating to adapt to new threats and vulnerabilities. Regular security audits allow organizations to assess the effectiveness of their access control mechanisms and identify any potential security gaps. These audits, coupled with timely updates to access protocols and security measures, are vital for upholding the integrity and trustworthiness of the knowledge base. By demonstrating a commitment to rigorous security practices, organizations can significantly bolster user confidence, ensuring users that their data is handled with the utmost care and that the knowledge base is a secure and reliable source of information. Prioritizing access control and security protects the organization from potential data breaches and compliance issues and reinforces its reputation as a responsible steward of user information.

Figure 12-18 Drupal is content management software popular with various industries.
Source: Drupal



As those in content creation are discovering, Drupal was designed from the ground up for Web publishing. Integrate a diverse ecosystem of digital marketing and related tools and technology. Effortlessly connect to marketing automation, CRM, email marketing, analytics, optimization and other solutions. Build and manage sites, multichannel digital experiences, shopping experiences, tailored mobile Web browsers and native mobile apps. With Web services support, Drupal functions as a content repository that's accessible through an API.

Drupal's authoring experience is synonymous with in-line editing and more WYSIWYG, thus streamlining the content creation experience.

This results in a Drupal-based content flow:

- Create content — even from numerous users — in one place
- Deploy content across channels
- Connect with all of your marketing tools
- Users access your content on any device
- Monitor site content and campaigns in one place, ensuring brand consistency
- Tailor, retarget, personalize content

Discover why our strategic platform is popular among enterprises, media, healthcare, governments and others who deploy content.

12.8.3 Enhancing Support through Documentation

Support can be enhanced by using the knowledge base and existing documentation to empower users with self-service capabilities, reducing the reliance on direct help desk interactions. Integrating documentation with help desk operations enhances the support quality provided, allowing for quicker resolution of user issues. Feedback loops are crucial in this process, enabling continuous documentation improvement based on user experiences and needs.

Self-Service Support through Documentation

Enhancing support and learning through documentation is a strategic approach that capitalizes on structured and easily accessible documentation to boost the efficiency of support services and the quality of teaching resources across an organization. By weaving comprehensive documentation into the fabric of self-service support systems and aligning it closely with help desk operations, organizations grant users the power to troubleshoot issues autonomously. This integration facilitates a seamless support experience in which users can swiftly find solutions to common problems, reducing reliance on direct help desk intervention. Furthermore, establishing feedback loops for continuous documentation improvement ensures that the materials evolve in response to user feedback and changing system functionalities, keeping the content relevant and current.

This methodology alleviates the burden on support teams by streamlining issue resolution and cultivates an environment where users are encouraged to learn and solve problems independently. Thus, documentation transforms into a living, breathing educational tool that addresses users' immediate questions and supports their ongoing learning. Through proactive engagement with documentation, users enhance their understanding and proficiency with the IT systems, increasing satisfaction and a positive overall experience. In this way, effective documentation practices significantly enhance user experience and satisfaction, demonstrating the critical role of well-managed documentation in modern organizations' support and educational ecosystems.

Integrating Documentation with Help Desks

Integrating documentation with help desk operations enhances the efficiency of the support process and ensures a unified experience for users seeking assistance. Support teams can provide more accurate and consistent guidance by granting help desk personnel direct access to the extensive documentation users consult. This alignment allows for a seamless flow of information between users and support staff, reducing misunderstandings and ensuring that users receive the help they need promptly. Moreover, such integration facilitates a synergistic relationship between the help desk and documentation teams, enabling support staff to pinpoint and communicate any deficiencies within the documentation based on user interactions and queries.

This collaborative effort between the documentation and help desk teams fosters a dynamic environment where the knowledge base is continually refined to address and preempt user concerns. By systematically reviewing user inquiries and feedback, the help desk can identify patterns and common issues that may not be adequately covered in the existing documentation. This insight enables the documentation team to tailor updates and create new content that directly responds to user needs, ensuring the knowledge base remains a relevant and valuable resource. Over time, this feedback loop cultivates a knowledge base that supports users in resolving their issues independently and enriches their overall experience with the IT system, contributing to a more effective and user-centric support framework.

Feedback Loops for Continuous Improvement

Incorporating feedback loops into the documentation process is essential for fostering an environment of continuous improvement and adaptation. Actively soliciting and meticulously analyzing user feedback about the utility, clarity, and comprehensiveness of documentation and the knowledge base enables organizations to pinpoint precisely where enhancements are needed. This could involve updating content that has become outdated, expanding on topics where information is sparse, and rectifying inaccuracies. Such an iterative approach to refining documentation ensures that it remains relevant, reliable, and tailored to meet the evolving needs of users. This process maintains the integrity and usefulness of the documentation over time and ensures that it is an effective tool for user support and education.

Organizations underscore their value on user input by transparently communicating changes and updates in response to user feedback, nurturing a collaborative relationship with their audience. This responsiveness to feedback reinforces the users' perception that their

contributions are acknowledged and impactful, fostering a stronger sense of community and engagement with the platform. Consequently, feedback loops play a pivotal role not just in the enhancement of documentation quality but also in ensuring that the knowledge base evolves in harmony with the users' actual experiences and needs. This alignment significantly boosts the efficacy of the documentation as a support and learning resource, making it a more powerful asset in improving user satisfaction and the overall user experience with the IT system.

12.9 System Retirement and Transition

System retirement and transition is a nuanced process requiring detailed planning and execution across five key stages to ensure a smooth shift from outdated systems to advanced technologies while preserving data integrity and minimizing disruptions. The process begins with planning for retirement, where criteria for system decommissioning are established, a retirement plan is developed, and stakeholders are kept informed, ensuring alignment and readiness for the transition. This phase seamlessly leads into data migration, a crucial step where data migration needs are assessed, the migration is meticulously planned and executed, and data integrity is validated to guarantee accuracy and accessibility in the new system. Following this, decommissioning procedures address the careful shutdown of the old system, addressing environmental considerations for hardware disposal and navigating legal and compliance issues related to the decommissioning process. Concurrently, the transition to new systems focuses on minimizing disruptions through strategic planning, training users on new systems to ensure proficiency, and providing support to adapt to the new operational environment. Finally, archiving and documentation are critical in preserving essential data and documentation for future reference, ensuring ongoing access to historical information, and completing the retirement process with thorough final reporting. Together, these stages encompass a comprehensive approach to system retirement and transition, safeguarding the organization's assets and facilitating a seamless move toward more efficient, modern IT solutions.

12.9.1 Planning for Retirement

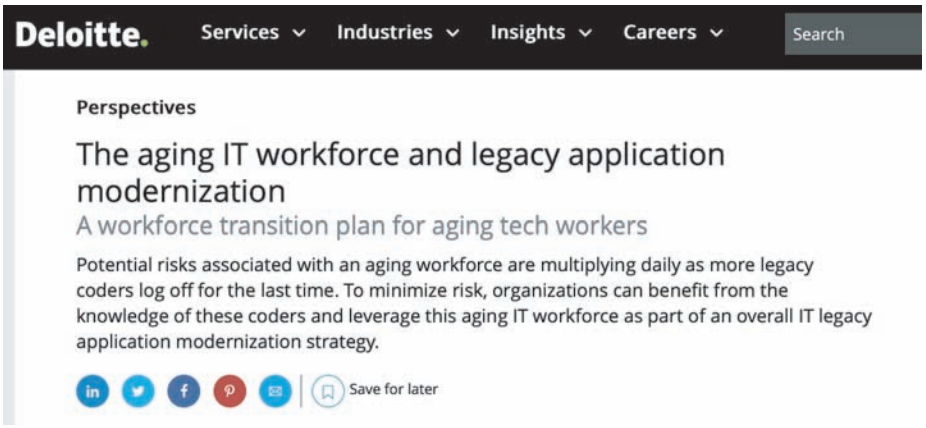
Planning for retirement is a critical phase in the life cycle management of IT systems, marking the transition from active use to decommissioning. This process requires careful consideration and strategic planning to ensure that system retirement aligns with organizational goals, minimizes disruptions to business operations, and maintains data integrity and security. Effective retirement planning involves several key activities: identifying retirement criteria to determine when a system should be phased out, establishing a detailed retirement planning process to guide the transition, and engaging in stakeholder communication to ensure all parties are informed and prepared for the changes. Organizations can create a clear roadmap for retiring outdated systems by addressing these components, facilitating a smooth transition to newer, more efficient technologies.

Identifying Retirement Criteria

Identifying retirement criteria is the first step in planning an IT system's retirement. This involves setting clear, measurable benchmarks that signal when a system is no longer viable or beneficial to the organization. Criteria may include increased maintenance costs, decreased performance and reliability, and the availability of more advanced technologies that offer enhanced capabilities. There is also an aging workforce to consider, as shown in Figure 12-19. By establishing these criteria upfront, organizations can make objective decisions about when to retire a system, avoiding the pitfalls of prolonged use of outdated technologies that can lead to security vulnerabilities and inefficiencies.

Identifying retirement criteria helps organizations proactively plan and budget system replacement or upgrades. Organizations can allocate resources more effectively, ensuring

Figure 12-19 The retirement of legacy coders presents significant risks and workforce management challenges for large organizations dependent on outdated applications.
Source: Deloitte



a minimal gap between the retirement of the old system and the deployment of the new solution. This foresight reduces the risk of disruptions and allows for a smoother transition, ensuring business processes run efficiently throughout the change.

Retirement Planning Process

The retirement planning process outlines the steps and considerations involved in transitioning from an outdated system. It encompasses assessing the current system's architecture, data, and dependencies to understand the scope of the retirement project. This assessment helps identify the resources required, potential risks, and the timeline for retirement. A well-defined process ensures that all technical and logistical aspects of system retirement are addressed, from data migration to the decommissioning of hardware and software.

This planning process also involves developing contingency plans to address potential challenges that may arise during the transition. By anticipating these challenges, organizations can implement measures to mitigate risks, such as data loss or system downtime. Effective retirement planning ensures that the organization remains agile and responsive, minimizing the impact on business operations and maintaining continuity of service for users and customers.

Stakeholder Communication

Stakeholder communication is vital to the success of the system retirement process. It involves informing and engaging all parties affected by the retirement, including IT staff, end users, and business leaders. Clear, consistent communication helps manage expectations, reduce uncertainty, and garner support for the transition. By providing regular updates and involving stakeholders in the planning process, organizations can ensure a collaborative approach to system retirement, addressing concerns and leveraging insights from different perspectives.

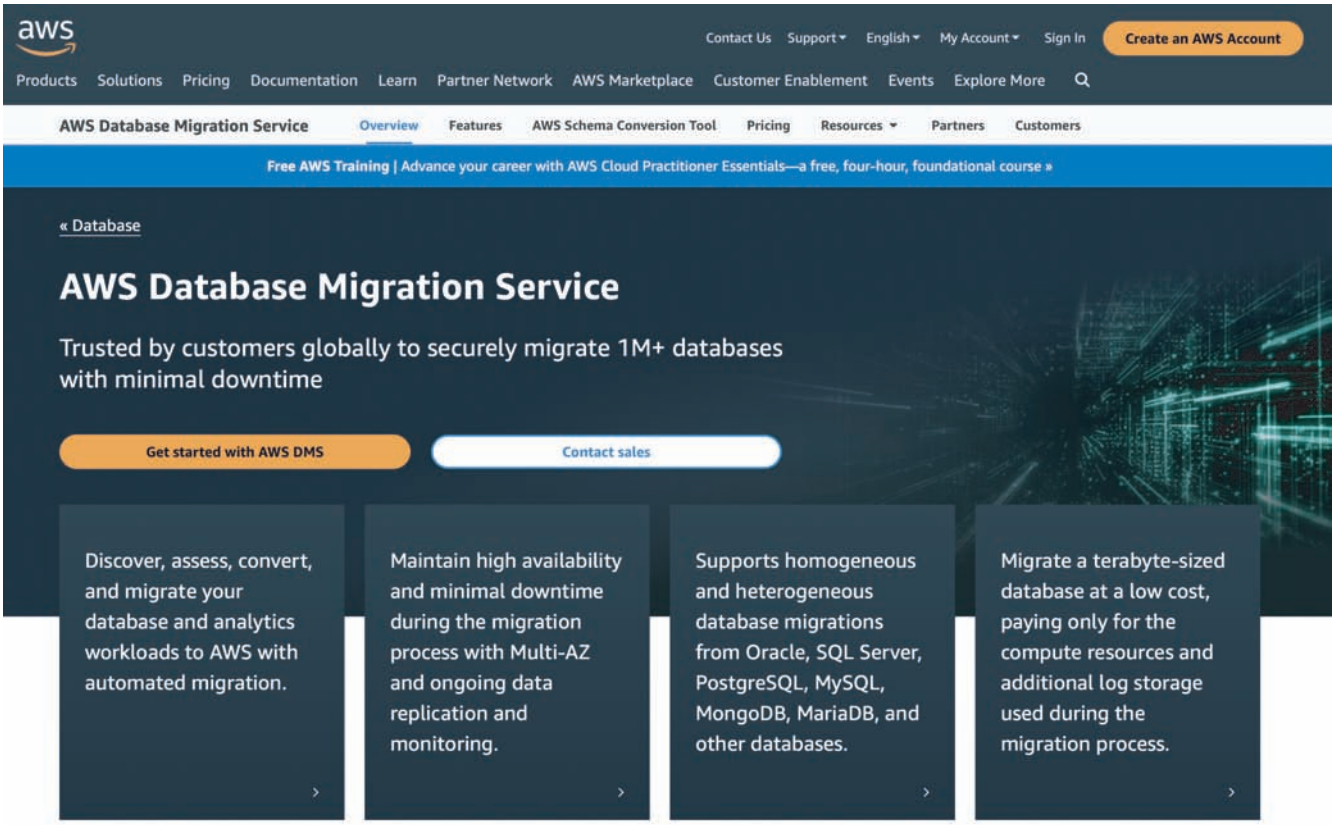
Effective communication facilitates feedback, allowing stakeholders to voice their needs and preferences for transitioning to new systems. This feedback can be invaluable in tailoring the retirement process to suit the needs of the organization and its users, enhancing the project's overall outcome. Through proactive and open communication, organizations can foster a sense of ownership and buy-in among stakeholders, paving the way for a smooth and successful system retirement.

12.9.2 Data Migration

Data migration is a pivotal phase in the system retirement process, involving carefully transferring data from the old system to a new environment. This operation ensures that valuable information is not lost during the transition, allowing businesses to maintain

continuity and access to historical data. Effective data migration requires meticulous planning, execution, and validation to safeguard data integrity and minimize disruptions to operations. The process comprises several key steps: assessing data migration needs to determine what data should be moved, archived, or discarded; planning and executing the migration to ensure a smooth transition; and validating the migrated data to confirm its integrity and completeness. Tools such as AWS Database Migration Service, shown in Figure 12-20, support this process. Through these structured steps, organizations can ensure that data remains a robust, accessible asset, supporting ongoing business processes and decision making in the new system environment.

Figure 12-20 AWS Database Migration Service helps your organization move its database and analytics workloads to the AWS platform.
Source: Amazon Web Services, Inc.



Assessing Data Migration Needs

Assessing data migration needs is the foundational step in the data migration process, where organizations determine the scope and requirements of the migration project. This assessment involves identifying which data sets are critical to business operations and need to be migrated while recognizing data that may be obsolete or no longer necessary. By evaluating the relevance and sensitivity of the data, organizations can prioritize their migration efforts, focusing on valuable and legally required data and deciding on the appropriate treatment for less critical information. This strategic approach helps optimize the migration process, ensuring resources are allocated efficiently and reducing the risk of data overload in the new system.

This assessment phase allows organizations to identify potential challenges and risks of migrating specific data types or volumes. Understanding these challenges early in the process enables planning appropriate mitigation strategies, such as employing specialized migration tools or techniques for high-volume or sensitive data. By thoroughly assessing data migration needs, organizations can develop a clear, targeted migration strategy that aligns with their requirements and strategic objectives, setting the stage for a successful transition.

Planning and Executing Data Migration

Planning and executing data migration involves developing a detailed plan that outlines the methods, tools, and timelines for moving data to the new system. This plan should account for the technical complexities of the migration, including the compatibility of data formats and the need for data transformation or cleansing. Effective planning ensures that all aspects of the migration are considered, from the technical execution to the potential impact on users and business operations. Assigning clear roles and responsibilities and establishing robust communication channels is essential for coordinating efforts and addressing issues promptly during migration.

Executing the data migration plan requires rigorous attention to detail and adherence to established timelines to minimize downtime and disruptions. Employing automated migration tools can enhance efficiency and reduce the likelihood of errors, but human oversight remains crucial for managing unexpected challenges and making real-time decisions. By carefully executing the migration plan, organizations can ensure a seamless data transition to the new system, maintaining continuity and the availability of critical information.

Data Integrity and Validation

Data integrity and validation are critical to the data migration process, ensuring that migrated data accurately reflects the source data without loss or corruption. This validation process involves conducting thorough checks and comparisons between the source and target systems, using automated tools and manual reviews to identify discrepancies or errors. Data integrity is crucial for maintaining the reliability of business operations and stakeholders' trust in the migrated data. Validation activities should be comprehensive and cover all aspects of the data, including its structure, content, and relationships.

Post-migration validation also provides an opportunity to confirm that data security and privacy controls have been effectively implemented in the new system, protecting sensitive information and complying with regulatory requirements. Organizations can confidently decommission the old system by meticulously validating migrated data, knowing that their valuable data assets have been successfully preserved and are fully operational in the new environment. This final step in the data migration process solidifies the foundation for future business activities and decision making, ensuring the organization thrives on a robust and reliable data infrastructure.

Case in Point 12.3

DynamicRelations, LLC

DynamicRelations, LLC, is a small company facing the challenge of retiring its outdated customer relationship management system and transitioning to a cutting-edge, modern solution. The task is complicated by the need to migrate decades' worth of legacy customer data, which is integral to the company's operations and future growth strategies. Ensuring this data's integrity, security, and seamless transfer without disrupting business continuity is your primary objective, highlighting the critical nature of meticulous planning and execution in this data migration project. As a systems analyst, how would you approach the project to ensure data integrity and minimal service disruption?

12.9.3 Decommissioning Procedures

Decommissioning involves the systematic shutdown and disposal of outdated IT systems, a critical phase that ensures the secure, responsible, and compliant conclusion of a system's life cycle. This process entails the technical steps necessary to decommission hardware and software safely and addresses the environmental and legal considerations associated with disposing of IT assets. Effective decommissioning protects sensitive data, adheres to environmental standards, and ensures all actions comply with relevant laws and regulations. By carefully planning and executing these procedures, organizations can mitigate risks,

such as data breaches and environmental harm, while upholding their legal and ethical responsibilities. Decommissioning is a multifaceted process requiring coordination across several departments, including IT, legal, and environmental compliance teams, to ensure that every procedure is conducted thoroughly and thoughtfully.

Decommissioning Steps

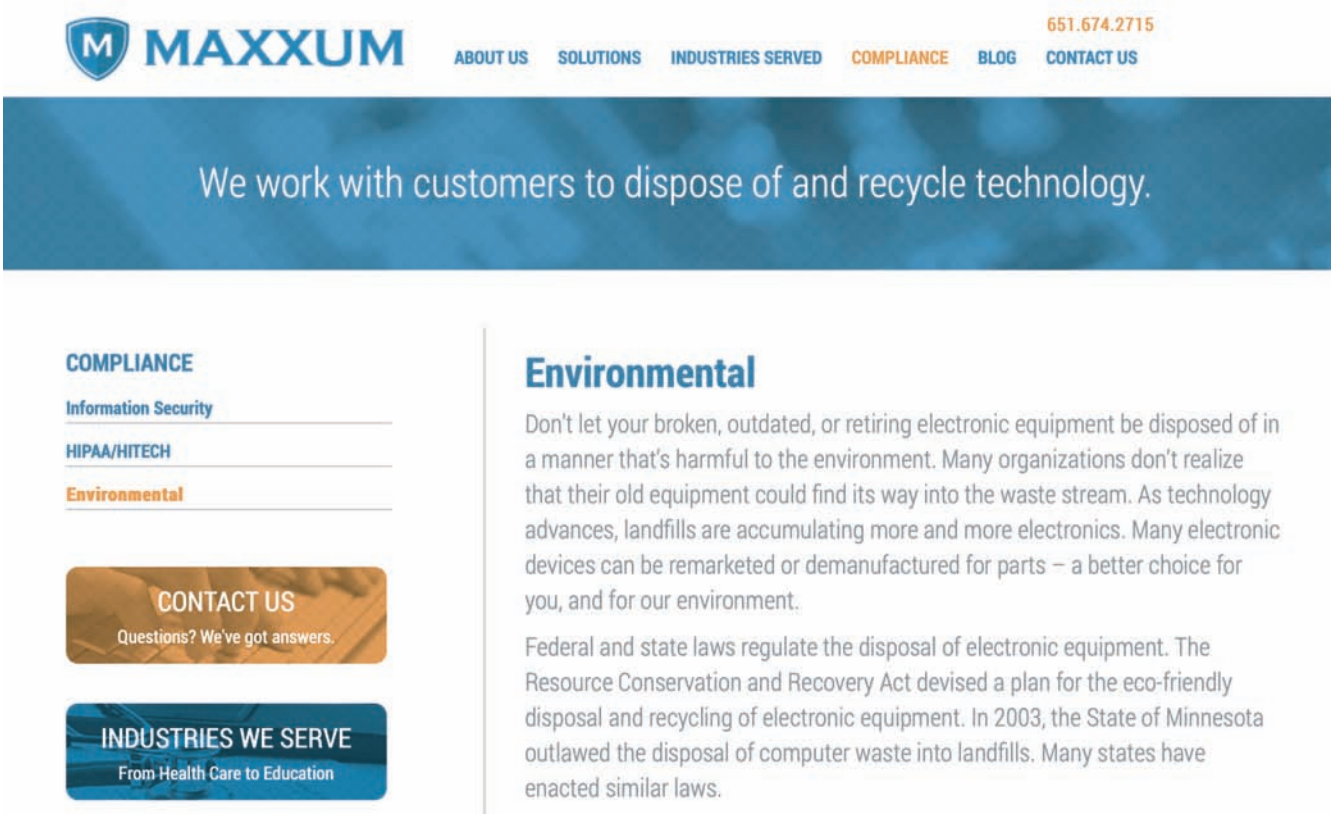
The decommissioning steps outline a structured approach to retire IT systems safely and efficiently. Initially, this involves a detailed inventory of the assets to be decommissioned, identifying all hardware and software components and their respective data contents. Following this, organizations must ensure that all sensitive data stored on these assets is securely erased or migrated, employing data destruction methods that meet industry standards for data sanitization. This crucial step prevents unauthorized access to confidential information after decommissioning.

Once data security is assured, the organization can proceed with the physical decommissioning of hardware, including the systems' safe shutdown, disassembly of hardware components, and preparation for disposal or recycling. Throughout these steps, detailed records should be maintained to document the decommissioning process, providing an audit trail that can verify compliance with data protection and environmental regulations. This approach ensures that decommissioning is performed in a manner that secures data, recycles materials where possible, and prepares the organization for the transition to new systems.

Environmental Considerations for Hardware Disposal

The disposal or recycling of electronic waste is a growing concern. As IT equipment contains potentially harmful substances, organizations are tasked with ensuring that decommissioned hardware is handled responsibly to minimize environmental impact. This involves selecting certified e-waste recycling partners who comply with local and international environmental standards and ensure that hazardous materials are properly contained and recycled. Maxxum, shown in Figure 12-21, is an example of such a company.

Figure 12-21 Maxxum helps organizations recycle their electronic equipment responsibly.
Source: Maxxum



Beyond compliance with environmental laws, adopting sustainable disposal practices demonstrates an organization's commitment to social responsibility and environmental stewardship. By prioritizing eco-friendly disposal methods, organizations can reduce their ecological footprint, contribute to the circular economy, and enhance their reputation among stakeholders who value sustainability. Effective management of hardware disposal requires upfront planning to identify the most environmentally responsible and cost-effective disposal routes, reinforcing the organization's dedication to environmental sustainability throughout the decommissioning process.

Legal and Compliance Issues

Navigating legal and compliance issues is a critical component of the decommissioning process, requiring organizations to be well-informed about the legal requirements and compliance standards relevant to their operations. This includes data protection, privacy laws, and environmental regulations governing electronic waste disposal. Ensuring compliance helps avoid potential legal penalties, data breaches, and reputational damage that could arise from improper decommissioning practices. Organizations must stay updated on changing regulations and implement decommissioning practices that adhere to the highest legal standards.

To effectively address these legal and compliance issues, organizations should collaborate with legal experts and compliance officers during the planning and execution of decommissioning procedures. This collaborative approach ensures that all aspects of decommissioning, from data destruction to hardware disposal, are reviewed for compliance risks and managed accordingly. Documentation of compliance efforts, including certificates of data destruction and records of environmentally compliant disposal, is essential for demonstrating due diligence and safeguarding the organization against potential legal challenges. By prioritizing legal and compliance considerations, organizations can ensure that their decommissioning procedures protect sensitive information and uphold the organization's legal and ethical obligations.

12.9.4 Transition to New Systems

The transition to a new system is a crucial phase in the life cycle of IT infrastructure, involving the strategic shift from outdated systems to modern, more efficient technologies. This transition is not merely a technical upgrade but a comprehensive process that impacts various aspects of an organization, including operational workflows, user experiences, and data management practices. Effective transition planning requires meticulous coordination, a clear understanding of the new system's capabilities, and a focus on minimizing disruptions to ensure business continuity. Additionally, it encompasses the development and execution of training programs to equip users with the necessary skills and knowledge to utilize the new systems effectively. By carefully managing this transition, organizations can maximize the benefits of new technology investments, enhance operational efficiency, and foster a culture of continuous improvement and adaptation.

Transition Planning

Transition planning is the foundational step in ensuring a smooth shift to new systems, requiring detailed analysis and preparation to address the technical and operational challenges of the transition. This process begins with a thorough assessment of the current and new systems to identify gaps, compatibility issues, and opportunities for process optimization. Establishing a clear roadmap that outlines key milestones, timelines, and responsibilities is crucial for coordinating efforts across different teams and ensuring that all aspects of the transition are aligned with the organization's strategic objectives. Effective transition planning also involves risk assessment to anticipate potential obstacles and develop contingency plans, minimizing the likelihood of unforeseen complications that could delay the transition or impact system performance.

Engaging stakeholders early in the planning process is essential for gathering input, setting expectations, and building consensus around the transition strategy. This engagement helps identify specific user needs and concerns, which can be addressed in the planning and

implementation phases to ensure the new system meets or exceeds operational requirements. By prioritizing communication and collaboration, organizations can foster a sense of ownership and enthusiasm for the new system among users and stakeholders, laying the groundwork for a successful transition supported and championed by the entire organization.

Minimizing Disruption to Operations

Minimizing disruption to operations during the transition to new systems is critical for maintaining business continuity and ensuring that core functions remain unaffected. This involves strategically scheduling the transition activities to avoid peak operational periods and implementing the new system in phases to reduce the impact on day-to-day activities. By taking a phased approach, organizations can monitor the performance and stability of the new system in a controlled environment, addressing any issues before a full-scale rollout. This careful management of the transition process helps mitigate downtime risks and ensures that any disruptions are brief and manageable.

Communication is pivotal in minimizing disruptions, with regular updates provided to all affected parties about the transition's progress, potential impacts, and expected outcomes. Preparing users and stakeholders for the transition through clear and consistent communication helps manage expectations and reduce resistance to change. Additionally, establishing support mechanisms, such as help desks or dedicated support teams, ensures that users can access assistance when encountering difficulties during the transition. Through proactive planning and communication, organizations can navigate the complexities of introducing new systems with minimal disruption, safeguarding efficiency and productivity.

Training Users on New Systems

Training users on the new system is an indispensable part of the transition process, ensuring that users can effectively utilize the latest technology and leverage its capabilities to enhance their work processes. A comprehensive training program should be tailored to the new system's features and user skill levels. This program can include a mix of training methods, such as workshops, online tutorials, and hands-on sessions, providing users with the knowledge and confidence to navigate the new system efficiently. Effective training facilitates a smoother transition and maximizes the return on investment in new technology by enabling users to exploit its features and benefits thoroughly.

Beyond initial training sessions, ongoing support and learning opportunities should be provided to address users' evolving needs and questions as they become more familiar with the new system. This can include follow-up training, creating user groups for knowledge sharing, and accessing online resources for self-learning. By investing in comprehensive and continuous user training, organizations can accelerate the adoption of new systems, reduce frustration and productivity losses, and empower users to contribute to process improvements and innovations. This focus on training and support underscores the organization's commitment to its workforce's development and the successful integration of new technologies.

12.9.5 Archiving and Documentation

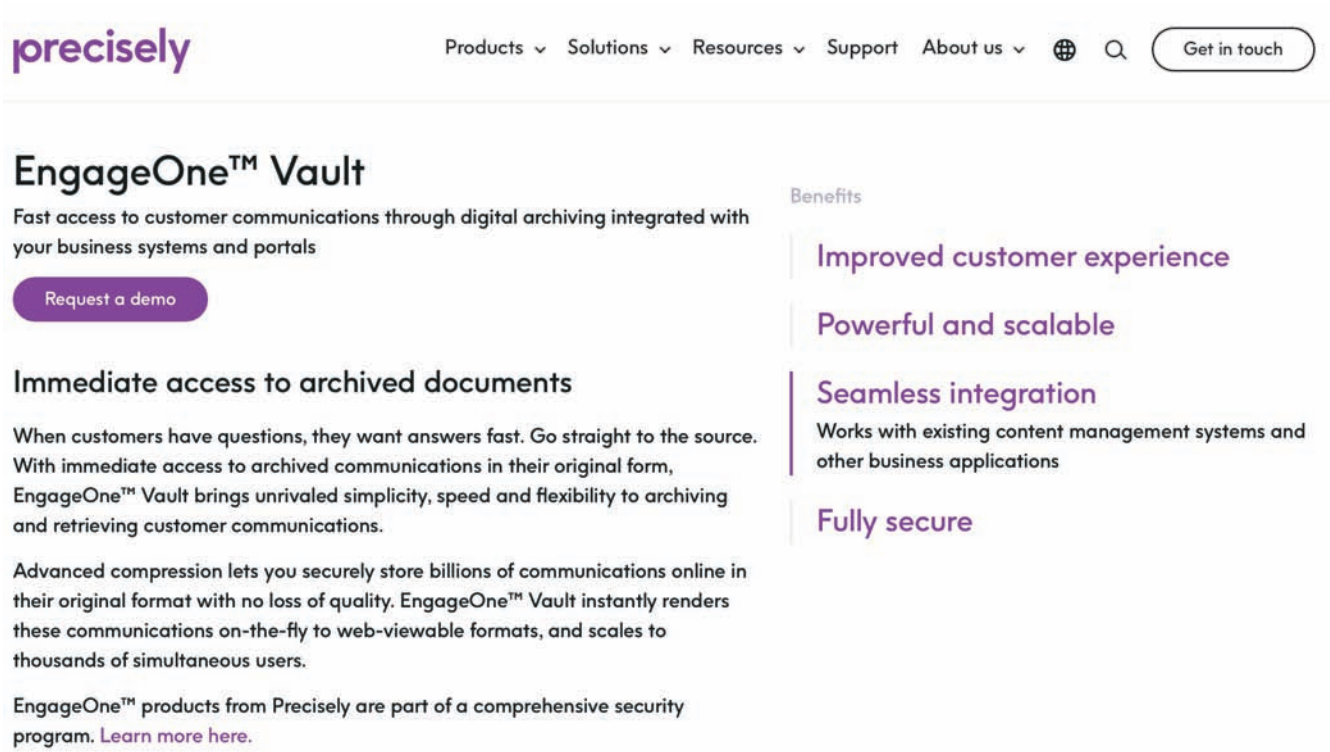
Archiving and documentation after a system's life cycle are crucial steps for preserving the integrity and accessibility of historical data and ensuring that all transition activities are thoroughly documented. This phase ensures that valuable information is not lost but is securely stored and can be accessed for future reference, legal compliance, or historical analysis. Effective archiving and documentation practices involve carefully selecting which data and system documentation need to be preserved, determining the best methods and formats for archiving, and establishing protocols for maintaining and accessing these archives. This phase also includes preparing final reports that summarize the transition process, lessons learned, and recommendations for future initiatives. By meticulously managing this process, organizations can safeguard their intellectual property, comply with regulatory requirements, and maintain a comprehensive record of their IT evolution, contributing to informed decision making and strategic planning.

Archiving Important Data and Documentation

Archiving important data and documentation is a selective process that determines which materials are essential for long-term preservation. This includes data that may be critical for future business operations or compliance with legal obligations and documentation that provides insights into the system's design, development, and operational history. Decisions on what to archive should consider factors such as regulatory requirements, potential future use, and the historical significance of the information. Once identified, the selected data and documentation must be prepared for archiving, which may involve data cleansing, format standardization, and removing sensitive information to ensure privacy and security.

The next step involves choosing appropriate storage solutions that balance accessibility, security, and cost-effectiveness. Digital archiving methods, such as cloud storage or specialized archival systems, offer advantages in terms of scalability and ease of access while also providing robust data protection. The chosen archiving solution must comply with relevant data retention policies and standards, ensuring that archived materials remain intact, unaltered, and retrievable for the duration required. Precisely's EngageOne Vault, shown in Figure 12-22, is an example of a digital archive used by companies worldwide. By strategically archiving essential data and documentation, organizations can create a valuable resource for future reference while mitigating data loss or degradation risks.

Figure 12-22 Precisely's EngageOne Vault is a digital archive used by many of the world's most prominent brands.
Source: Precisely



Maintaining Access to Archived Information

Maintaining access to archived information requires implementing systems and procedures that ensure authorized users can retrieve archived data and documentation when needed. This process involves the technical aspects of data storage and retrieval and establishing clear policies governing access permissions and usage. Accessibility must be balanced with security to protect sensitive information from unauthorized access, necessitating robust access control mechanisms and regular audits of access logs and security measures. Ensuring the usability of archived information also means maintaining detailed indexes or metadata that facilitate efficient search and retrieval, allowing users to locate specific items without extensive manual searching.

In addition to technical and procedural measures, training and awareness programs for staff on how to access and utilize the archives are crucial for maximizing the value of stored information. Users should be familiar with the archiving system's capabilities, understand the policies related to archive access, and know whom to contact for assistance with archive-related queries. Regular reviews and updates of the archiving system and access policies are also necessary to adapt to changing technological landscapes and organizational needs. By effectively maintaining access to archived information, organizations can ensure that their historical data and documentation remain a dynamic and valuable asset rather than an inaccessible trove of knowledge.

Closing Documentation and Final Reporting

Closing documentation and final reporting mark the culmination of the system transition process, providing a comprehensive overview of the activities undertaken, outcomes achieved, and insights gained. This documentation serves multiple purposes, including validating the completion of transition activities, evaluating the project's success against its objectives, and capturing lessons learned that can inform future transitions. Preparing closing documentation involves compiling data from various stages of the transition, including planning documents, progress reports, and post-transition evaluations. This consolidation of information creates a valuable resource for stakeholders to review the project's overall performance and identify areas for improvement.

Final reporting also plays a critical role in organizational learning, allowing one to reflect on the challenges encountered and the strategies contributing to the project's success. These reports should be accessible to relevant stakeholders, including project teams, management, and future project planners, to provide insights that can guide decision making and strategic planning. Additionally, closing documentation and final reporting ensure accountability and transparency, documenting the organization's efforts to manage the transition responsibly and efficiently. By thoroughly documenting the transition process and its outcomes, organizations can enhance their capacity for continuous improvement, laying a solid foundation for future system retirements and transitions.

Summary

A comprehensive framework for maintenance and support is essential for ensuring that IT systems are reliable, efficient, and aligned with organizational objectives. This framework encompasses various activities, from user support and training to the meticulous planning for system retirement and transition. Each component is crucial in maintaining system performance, managing security, and ensuring users have the necessary knowledge and tools. By systematically addressing these areas, organizations can enhance their IT infrastructure's resilience, adaptability, and effectiveness, supporting continuous operational improvement and strategic growth.

User support and training empower users through comprehensive training programs, robust support services, and efficient management of outsourcing relationships. Training programs are designed to enhance user competence and confidence, support services aim to resolve user issues promptly, and managing outsourcing ensures that external services align with organizational standards and user needs.

Maintenance tasks are critical for the ongoing upkeep of IT systems. They encompass corrective, adaptive, perfective, and preventive maintenance. These tasks ensure systems are fixed when issues arise and continuously improved and optimized to meet evolving business requirements and technological advancements.

Maintenance operations management involves overseeing the maintenance team and activities, defining roles and responsibilities, efficiently handling maintenance requests, prioritizing work based on urgency and impact, and continuously improving maintenance processes to enhance system reliability and performance.

Configuration management ensures that all system changes are recorded and managed systematically. It includes maintenance releases, version control, and the establishment of baselines, providing a structured approach to managing system modifications and maintaining system integrity over time.

System performance management entails monitoring and optimizing system performance through fault management, performance and workload measurement, and capacity planning. This ensures the IT infrastructure can support current and future business needs without compromising service quality or user experience.

QA in system support emphasizes the importance of QA strategies, testing practices, and feedback mechanisms to improve system support and maintenance activities. Organizations can reduce errors, enhance system stability, and ensure user satisfaction by focusing on quality.

Security management in maintenance addresses the critical need for security updates, audits, and data protection measures to safeguard systems against cyber threats and ensure compliance with data privacy laws, maintaining the trust of users and stakeholders.

Documentation and knowledge management highlight the creation of a comprehensive documentation strategy, the development of an accessible knowledge base, and the use of documentation to enhance support. This ensures that valuable information is effectively captured, maintained, and utilized.

System retirement and transition involve careful planning, data migration, decommissioning procedures, transitioning to new systems, and archiving and documentation. This structured approach ensures that system transitions are smooth, data is preserved, and the organization is positioned for future technological advancements.

Key Terms

adaptive maintenance
allocated baseline
applications programmer
archived
asset
baseline
benchmark testing
capacity planning
change control (CC)
configuration management (CM)
corrective maintenance
database programmer
enhancement
fault management
functional baseline
Kbps (kilobits per second)
key process indicator (KPI)
maintenance activities
maintenance expenses
maintenance release
maintenance release methodology
maintenance team
metrics
operational costs
perfective maintenance
preventive maintenance
product baseline
programmer/analyst
recovery
recovery point objectives (RPOs)
recovery time objectives (RTOs)
remote control software
response time
security
service level agreement (SLA)
service pack
software reengineering
system administrator
systems programmer
throughput
turnaround time
user training package
version control
what-if analysis

Review Questions

1. What is the purpose of user support and training in IT systems?
2. What are the different types of maintenance of IT systems?
3. What role does a maintenance team play in an organization's IT infrastructure?
4. What role does version control play in configuration management?

5. What is the goal of system performance management?
6. How does capacity planning benefit an organization's IT infrastructure?
7. Why is QA important in system support?
8. Why are security updates an essential part of security management in maintenance?
9. How does effective documentation support IT system users and maintenance teams?
10. Why is planning for system retirement and transition critical?

Discussion Topics

1. How do user support and training enhance system usability and user satisfaction?
2. Discuss the impact of a comprehensive maintenance strategy on the longevity and performance of IT systems.
3. How does configuration management contribute to maintaining system integrity during updates and changes?
4. Why are regular security audits necessary for maintaining IT system integrity?
5. Why is planning for system retirement and transition as critical as planning for system implementation?

Hands-On Projects

1. Design a comprehensive training workshop for introducing new software to your school or a local organization. The project will involve researching the software, identifying key features and common challenges, developing training materials, and facilitating a mock session with your peers.
2. Develop a preventive maintenance schedule for your school computer lab's hardware and software. This includes creating a checklist of maintenance tasks, determining the frequency for each task, and identifying tools or software that can assist in the maintenance process.
3. Design and implement a simple configuration management database (CMDB) using a tool like Microsoft Access. Track the configuration items (CIs) within a small business or school network. This project involves identifying CIs (hardware, software, and network devices), defining attributes for each CI, and establishing relationships.
4. Conduct a mock security audit on a hypothetical company's network infrastructure. This project involves identifying potential security vulnerabilities using tools or checklists, recommending mitigation strategies, and presenting findings in a report.
5. Simulate the data migration process from an old database system to a new one. This involves mapping data fields between the old and new systems, creating scripts or using migration tools to transfer data, and conducting tests to ensure data integrity.

Ethical Issues

A critical security patch has just been developed to address a significant vulnerability in your product. Usually, your team conducts thorough regression testing on updates to ensure they do not introduce new bugs or issues. However, due to pressures to meet tight deployment deadlines, your manager instructs you to skip the regression testing phase and proceed directly to shipping the updated product to customers.

Considering the potential risks to product integrity and customer trust, how should you respond to your manager's request to skip regression testing for the security patch?

Chapter

13

Managing Systems Security

Learning Objectives

When you finish this chapter, you should be able to:

1. Define key system security concepts.
2. Identify different attacker profiles and their motivations.
3. List the steps involved in risk identification.
4. Recall the different levels of systems security.
5. Name key components of developing and implementing security policies.
6. Identify the essential elements of effective backup policies.
7. List major cyberethics principles and data protection laws.
8. Describe the impact of emerging technologies on system security.
9. Identify the components of a resilient cybersecurity culture.

Chapter 13 is the second chapter in the operations phase of the SDLC. The first step in this phase was maintenance and support, which emphasized the essential methods and practices for ensuring IT systems operate efficiently, securely, and in alignment with user needs. Managing systems projects delves into the comprehensive strategies and practices for safeguarding information systems against evolving cybersecurity threats. This chapter also covers the significance of organizational culture, the development of a security-first mindset, the crucial role of leadership and governance, and the integration of emerging technologies into robust security frameworks.

The chapter includes three "Case in Point" discussion questions to help contextualize the concepts described in the text. The "Ethical Issues" scenario concerns a systems analyst who discovers a significant data breach and faces a moral dilemma when the company leadership opts to delay disclosure due to reputational concerns, challenging the analyst's loyalty and ethical duty to customer privacy.

13.1 System Security

System security focuses on five key areas to protect online spaces. It starts with a strong foundation in understanding how to keep data safe, private, and accessible. This basic knowledge is essential for preventing unauthorized access and protecting information.

Vigilant monitoring for threats and vulnerabilities is essential in the rapidly evolving digital landscape, requiring adaptations to security measures based on thorough risk assessments. In parallel, creating clear security policies and procedures provides a structured approach to digital safety, dictating the steps and standards necessary for protection. Integrating security frameworks and standards further elevates these practices, aligning them with industry benchmarks and best practices to achieve a comprehensive system security posture.

An awareness of the legal and reputational risks associated with security breaches is also vital, pushing the scope beyond technical solutions to encompass the broader implications of security practices. This includes considerations for legal compliance and the cultivation of trust within the industry and among stakeholders. A robust strategy emerges by addressing these elements collectively, fortifying defenses against cyber threats while upholding legal standards and fostering industry trust.

13.1.1 System Security Concepts

System security is the bedrock of protecting an organization's digital assets against an ever-evolving cybersecurity landscape. In today's interconnected world, the significance of safeguarding information cannot be overstated, with threats ranging from data breaches to sophisticated cyberattacks aiming to **exploit** system vulnerabilities. System security encompasses a broad spectrum of practices and technologies to protect data, **networks**, and devices from unauthorized access, damage, or theft. It is an essential discipline that requires a deep understanding of potential threats, vulnerability management, and implementing practical security measures.

Central to system security is the principle of the **CIA triangle** shown in Figure 13-1: confidentiality, integrity, and availability. **Confidentiality** ensures that sensitive information is accessible only to those authorized to view it. **Integrity** ensures that the information is accurate and unaltered. **Availability** guarantees that information and resources are accessible to authorized users when needed. Implementing strong encryption techniques, enforcing stringent access control measures, and utilizing secure authentication protocols exemplify the practical application of core security principles in formulating a **security policy**. Once put into practice, these policies can enhance system security.

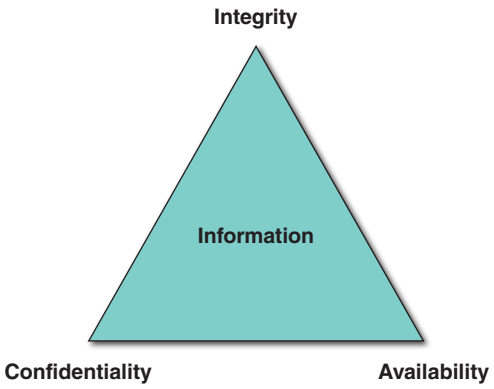


Figure 13-1 System security must provide information confidentiality, integrity, and availability (CIA).

The rapid pace of technological advancements further compounds the complexity of system security. New technologies often introduce new vulnerabilities, making it crucial for security measures to evolve continuously. This dynamic nature of system security demands constant vigilance, a proactive stance toward potential threats, and a broad understanding of the technological landscape. By grounding their policies in fundamental security concepts, organizations can develop resilient defenses that mitigate current threats and adapt to future challenges.

13.1.2 Identifying Security Threats and Vulnerabilities

Identifying security threats and vulnerabilities is a critical first step in developing effective system security strategies. Threats can emanate from various sources, including external attackers aiming to exploit system weaknesses. Internal users may inadvertently or maliciously compromise security, and natural disasters can pose physical and digital infrastructure risks. Threat identification involves thoroughly analyzing the system to uncover potential

vulnerabilities that could be exploited. These vulnerabilities range from software bugs to weaknesses in network configurations.

Vulnerability assessment tools and techniques play a pivotal role in this process, enabling organizations to scan their systems for known vulnerabilities and assess the potential impact of identified threats. This examination helps prioritize security efforts, focusing on mitigating vulnerabilities that pose the most significant risk. Moreover, staying informed about emerging threats through cybersecurity intelligence feeds and industry reports is essential for updating security measures against new attacks.

Equally important is establishing a vulnerability management program that includes regular assessments, timely patching of software vulnerabilities, and continuous monitoring for suspicious activities. Such a program not only aids in the early detection of potential threats but also enhances the organization's ability to respond swiftly to mitigate risks. By systematically identifying and addressing vulnerabilities, organizations can reduce their attack surface and strengthen their security posture.

13.1.3 Security Policies and Procedures

Formulating and implementing security policies and procedures are the underpinnings of maintaining system security. These policies articulate an organization's security philosophy, set forth guidelines for behavior, define roles and responsibilities, and establish procedures for practices such as access control, data encryption, and incident response. They serve as a blueprint for security governance, guiding the organization in protecting its assets and compliance with regulatory requirements.

Developing potent security policies requires understanding the organization's operational environment, including the types of data handled, the architecture of IT systems, and the potential impact of security incidents on business operations. Policies should be tailored to address specific security needs, promote best practices, and promote a culture of security awareness among employees. Moreover, these policies must be regularly reviewed and updated to reflect changes in the organization's risk profile, technological advancements, and evolving legal and regulatory landscapes.

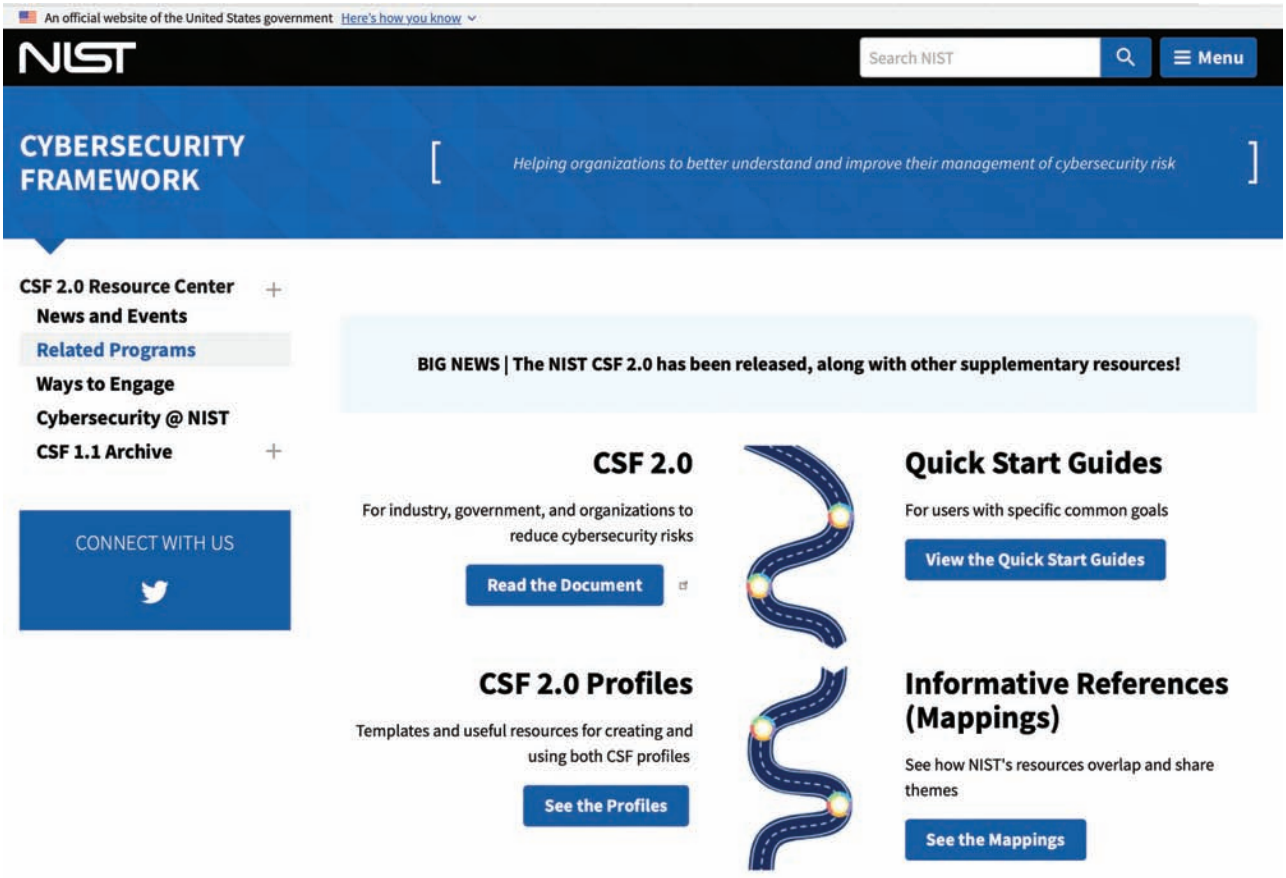
Procedures translate these policies into actionable steps, providing detailed instructions for implementing security controls, conducting risk assessments, and responding to security incidents. Well-defined procedures ensure consistency in security practices, enable swift action in the face of threats, and facilitate audit processes. Training and awareness programs are also crucial, ensuring all employees understand their role in upholding security policies and procedures. Together, these elements form the cornerstone of a system security framework, ensuring that organizations can defend against threats while enabling secure and resilient business operations.

13.1.4 Security Frameworks and Standards

Security frameworks and standards are indispensable in shaping an organization's security posture by offering a structured approach to identifying, managing, and mitigating security risks. These guidelines, such as the **ISO/IEC 27000** series, **NIST Cybersecurity Framework** (Figure 13-2), and **PCI DSS**, are blueprints that organizations can adopt and adapt to fit their specific security needs and objectives. By adhering to these established frameworks and standards, organizations can ensure that their system security is aligned with best practices and industry benchmarks, facilitating a more systematic and successful approach to security management. These frameworks often provide a common language for discussing and assessing security, enabling clearer communication among stakeholders and facilitating compliance with regulatory requirements.

Implementing these frameworks and standards requires a thorough understanding of the organization's security posture and specific risk environment. This process typically involves conducting a gap analysis to identify areas where the organization's current practices fall short of the standards' recommendations. Based on this analysis, organizations can prioritize

Figure 13-2 The NIST Cybersecurity Framework helps organizations better understand and manage cybersecurity risk.
Source: National Institute of Standards and Technology



improvements and adequately allocate resources, ensuring that efforts are focused on areas that will yield the greatest impact on their security posture. Over time, adherence to these frameworks and standards enhances an organization's security. It builds trust with customers, partners, and regulatory bodies by demonstrating a commitment to maintaining high data protection and risk management standards.

The dynamic nature of cybersecurity threats means that security frameworks and standards are continually evolving to address new challenges and vulnerabilities. As such, organizations must stay informed about updates and revisions to the frameworks and standards they follow. Regularly reviewing and updating security policies and practices in line with these changes is crucial for maintaining compliance and ensuring security measures remain applicable against current threats. This ongoing commitment to following security frameworks and standards underscores an organization's dedication to active security management and continuous improvement in its security practices.

13.1.5 Legal Implications and Reputational Damage

The legal implications of system security are vast, encompassing a range of obligations that organizations must fulfill to protect sensitive data and ensure privacy. Regulations such as GDPR in the European Union, the Health Insurance Portability and Accountability Act (**HIPAA**) in the United States, and various other data protection laws mandate strict controls over handling personal and sensitive information. Noncompliance with these legal requirements can result in hefty fines, legal actions, and other sanctions that can impact an organization's financial health and viability. Beyond the immediate legal consequences,

organizations must also consider their broader duty toward clients, employees, and stakeholders to safeguard their data against breaches and unauthorized access.

Reputational damage is another critical consideration that organizations face in system security. In the digital age, news of security breaches spreads rapidly, and public perception can shift dramatically after such incidents. Losing customer trust and confidence can be devastating, often more so than any direct financial penalties. Customers and partners may hesitate to engage with an organization perceived as unable to protect its data, leading to lost business opportunities and a long-term impact on profitability. Therefore, a strong business case exists beyond the legal mandate for system security to preserve an organization's reputation and ensure its continued success.

Addressing legal implications and reputational damage requires a rounded approach to system security that integrates technical measures with a convincing ethical framework and transparent communication. Organizations must implement security measures and respond swiftly when breaches occur, demonstrating accountability and a commitment to rectifying issues. This includes clear communication with affected parties and regulatory bodies, taking corrective actions to prevent future breaches, and being transparent about the steps taken to improve security. By doing so, organizations can mitigate the legal and reputational impacts of security incidents and reinforce their standing as trustworthy and responsible custodians of their stakeholders' data.

13.2 Understanding and Mitigating Attacks

Grasping the full spectrum of cyber threats along with the underlying motivations of attackers is crucial for crafting hardy defense mechanisms. Such comprehension allows organizations to pinpoint potential security gaps and deploy specific countermeasures to minimize exposure. The journey toward achieving a fortified security posture begins by thoroughly analyzing the diverse attacker profiles and their driving forces. It extends through the meticulous identification of the myriad forms of cyberattacks. This sets the stage for developing and implementing defenses tailored to ward off these digital threats.

Securing organizational assets extends into the specialized realms of advanced persistent threats (APTs) and the subtle complexities of social engineering and insider threats. With their stealthy and continuous nature, APTs demand a nuanced approach to security beyond conventional measures, including constant monitoring and sophisticated detection techniques. Similarly, the human element of security, often exploited through social engineering and compromised by insider threats, underscores the need for an all-inclusive approach. This must encompass technological safeguards and strongly emphasize fostering an organizational culture of awareness and vigilance.

Navigating the intricacies of system security necessitates an inclusive approach that embraces both the technological and human facets of defense. By undertaking a detailed exploration of attacker methodologies, the variety of cyberattacks, and the implementation of preemptive measures, organizations can bolster their defenses against a landscape of threats. Central to this endeavor is recognizing the critical role played by continuous education and cultivating a security-conscious workforce equipped to identify and mitigate the sophisticated tactics employed by modern adversaries.

13.2.1 Attacker Profiles and Motivations

Attackers break into a system to cause damage, steal information, or gain recognition, among other reasons. Understanding attacker profiles and motivations is crucial for anticipating the cyberattack's nature and potential targets. As illustrated in Table 13-1, attackers range from individual hackers seeking notoriety to organized crime groups aiming for financial gain to state-sponsored actors pursuing espionage objectives. Their motivations vary widely, including financial profit, political activism (hacktivism), espionage, or simply the challenge and recognition within the hacker community.

Table 13-1 IT security professionals have names for various attacker types, characteristics, and skill set levels.

Attacker Profiles		
Type	**Characteristics**	**Skill Set**
Cyberterrorist	Attacker who attempts to advance political, social, or ideological goals	High
Employee	Attacker who uses unauthorized information or privileges to harm or steal from their own organization	Varies
Hacker	Attacker who employs advanced skills for malicious purposes (black hat) or to identify and improve security flaws (white hat)	High
Hacktivist	Attacker who engages in hacking to promote a social or political cause, often by disrupting or defacing websites	Varies
Script kiddie	Inexperienced attacker using premade software to cause disruption or create notoriety	Low
Spy	Non-employee who infiltrates systems to exfiltrate information, often for financial gain	High
Insider threat	Someone with legitimate access who intentionally or accidentally harms the organization	Varies
Nation-state actor	Government-sponsored attacker targeting other nations for espionage or sabotage	Very high
Organized crime group	Criminal organization engaging in cybercrime for financial gain	High

By analyzing these profiles and motivations, security professionals can better predict the methods and targets of potential attackers. For instance, state-sponsored actors may focus on espionage and target government and defense-related systems. In contrast, cybercriminals motivated by financial gain are more likely to target financial institutions and e-commerce sites. This understanding aids in prioritizing security efforts and resources toward the most likely threats.

Tailoring defense mechanisms to address specific threats requires a holistic security strategy encompassing technical and human elements. Awareness and training programs can mitigate the risk posed by social engineering attacks, while sophisticated detection and response systems can help defend against state-sponsored espionage. The goal is to create a dynamic and resilient security posture capable of adapting to attackers' evolving tactics and motivations.

13.2.2 Types of Cyberattacks

Cyberattacks can take many forms, each with unique characteristics and implications for system security. An **attack** is a hostile act that targets the system or the company itself. Thus, a disgruntled local employee or a hacker 6,000 miles away might launch an attack. Table 13-2 shows common types of cyberattacks, including **malware**, phishing, denial-of-service (DoS) attacks, and SQL (Structured Query Language) injection. Malware encompasses various kinds of malicious software designed to damage or gain unauthorized access to systems, including viruses, worms, and ransomware. Phishing attacks deceive users into disclosing sensitive information, while DoS attacks aim to make a service unavailable to its intended users.

Table 13-2 Attacks can take many forms. IT security managers must be able to detect these attacks and respond with suitable countermeasures.

Types of Attacks with Descriptions and Examples	
Type of Attack	**Description and Example**
Back door	Exploiting vulnerabilities in software to gain unauthorized access Example: Accessing a system using unpatched software vulnerabilities
Credential stuffing	Using automated tools to use stolen username/password pairs to gain unauthorized access to user accounts Example: Breaching a website by trying combinations from previous data breaches
Cross-site request forgery (CSRF)	Tricking a web browser into executing an unwanted action on a site to which a user is logged in Example: Forcing a user's browser to send a request to a banking application to transfer funds without the user's consent
Cross-site scripting (XSS)	Injecting malicious scripts into content viewed by other users of a web application Example: Stealing cookies or session tokens via injected scripts in webpages
Deepfake and AI-powered attacks	Utilizing AI to create fake audio or video recordings to manipulate or mislead Example: Generating a video of a public figure saying something they never did
Denial of service (DoS)/distributed denial of service (DDoS)	Flooding a server with excessive requests to overload and incapacitate it Example: Overwhelming a website's server with traffic from multiple sources
Drive-by download attacks	Downloading malware by simply visiting a compromised website without any user action Example: Unknowingly downloading a keylogger from a compromised blog
Dumpster diving	Searching through trash for sensitive information Example: Finding discarded documents containing passwords or financial information
Mail bombing	Sending massive volumes of email to overwhelm a recipient Example: Targeting a company's email server with thousands of emails
Malicious code	Distributing harmful software like viruses or worms Example: Sending an email with a virus attachment that encrypts files for ransom
Man-in-the-middle (MitM)	Intercepting communications between two parties without their knowledge Example: Capturing bank transaction details

Table 13-2 (continued)

Types of Attacks with Descriptions and Examples	
Type of Attack	**Description and Example**
Password cracking	Attempting to guess or decrypt passwords Example: Using a brute-force attack to access an account
Phishing	Mimicking legitimate entities to deceive users into providing sensitive information Example: Emails appearing to be from a bank asking for login details
Privilege escalation	Gaining higher access levels by exploiting vulnerabilities Example: Elevating a user account to administrator level without authorization
Ransomware attacks	Encrypting files and demanding ransom for decryption keys Example: Locking access to a company's files and demanding payment in cryptocurrency
Rootkit	Gaining unauthorized access and hiding its existence Example: Installing a rootkit to control a computer covertly
Sniffing	Capturing network traffic to gather information Example: Using packet sniffers to capture unsecured passwords
Social engineering	Manipulating individuals into divulging confidential information Example: Posing as IT support to get passwords from employees
Spam	Sending unsolicited bulk messages Example: Flooding inboxes with irrelevant emails
Spoofing	Impersonating another entity on a network Example: Using a forged IP address to gain access to a network
SQL injection	Inserting malicious SQL queries into inputs Example: Manipulating a database query to dump user information
Supply chain attack	Compromising software or hardware in the supply chain Example: Inserting malware into software before it's delivered to the end user
Zero-day exploit	Attacking using unknown vulnerabilities Example: Exploiting a software flaw before a patch is available

Understanding these types enables organizations to implement specific countermeasures. For example, anti-malware software and intrusion detection systems can protect against malware, while employee training and email filtering can reduce the risk of phishing. Regularly updating and patching software can also prevent exploitation through known vulnerabilities, a common vector for many attacks.

Beyond these common types, organizations must also be aware of more sophisticated techniques such as **zero-day exploits**, which exploit undisclosed vulnerabilities. Staying informed about emerging threats and continuously updating security protocols are essential for maintaining defense mechanisms against an ever-changing threat landscape.

13.2.3 Preventive Measures Against Common Attacks

Preventive measures are essential for defending against cyberattacks. Implementing layered security, including firewalls, antivirus software, and encryption, can protect various aspects of an organization's digital infrastructure. Regular security audits and vulnerability assessments help identify and remediate potential weaknesses before they can be exploited.

Employee education and awareness programs are also vital, as many attacks target human vulnerabilities through social engineering tactics. Training staff to recognize and respond appropriately to phishing attempts and suspicious activity can reduce the risk of successful attacks.

Adopting security best practices, such as the principle of least privilege, where users are given only the access necessary to perform their job functions, can limit the potential damage of an attack. Regularly updating and patching software to address known vulnerabilities is another critical preventive measure, as it closes off avenues that attackers could exploit.

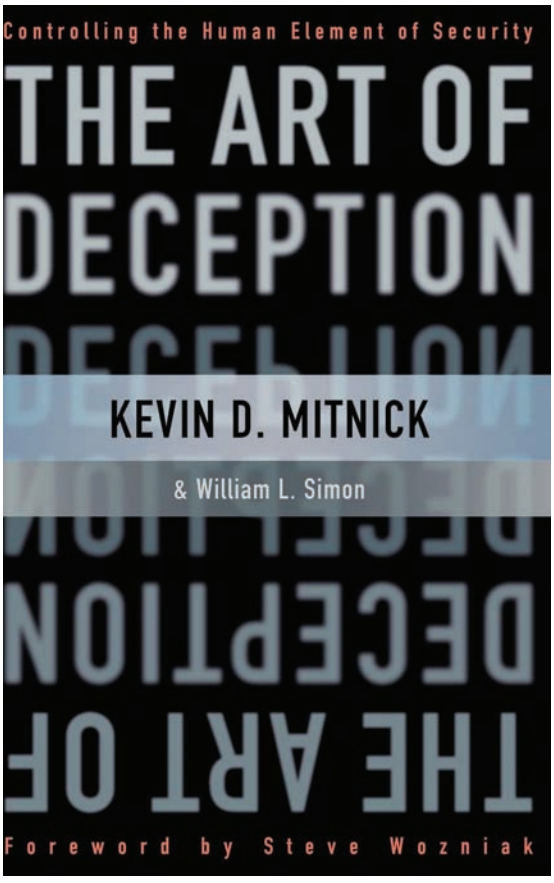
13.2.4 Advanced Persistent Threats

Advanced persistent threats (APTs) represent a sophisticated category of cyberattacks characterized by their long-term nature, significant resources, and high stealth levels. These threats are typically launched by well-funded and organized groups, often with state sponsorship, aiming to infiltrate a target network and remain undetected for extended periods.

The primary goal of APTs is to steal data or monitor activity rather than cause immediate damage. This is achieved through continuous, stealthy, and sophisticated hacking techniques. Unlike conventional threats, APTs often involve custom malware and social engineering to gain initial access, followed by lateral movement within the network to reach valuable data.

Defending against APTs requires advanced security mechanisms, including continuous monitoring, behavioral analysis to detect anomalous activities, and an incident response plan tailored to sophisticated threats. Organizations must also focus on threat intelligence to understand APT groups' tactics, techniques, and procedures (TTPs), enabling them to better anticipate and defend against these attacks.

Figure 13-3 The world's most infamous hacker offers an insider's view of the low-tech threats to high-tech security.
Source: John Wiley & Sons



13.2.5 Social Engineering and Insider Threats

Social engineering attacks leverage human psychology, not technical weaknesses, to bypass security measures and access confidential information or systems. Techniques like phishing, pretexting, and baiting are designed to deceive people into violating security protocols. Kevin Mitnick, a renowned figure in the cybersecurity world, famously demonstrated the power of **social engineering** by manipulating individuals to gain access to networks and data without ever needing to exploit software vulnerabilities. He published his views in the book *The Art of Deception: Controlling the Human Element of Security* (Figure 13-3). Likewise, insider threats pose a significant risk, as they stem from individuals within the organization who misuse their access to compromise systems and data, either

maliciously or inadvertently. Both social engineering and insider threats underscore the critical need for security awareness and training to mitigate human vulnerabilities.

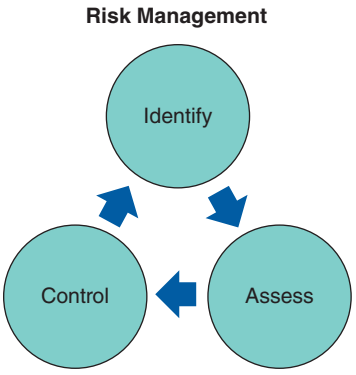
Mitigating these risks requires a focus on both technological solutions and human factors. Security awareness training is crucial for educating employees about the dangers of social engineering and best security practices. Additionally, implementing strict access controls and monitoring systems can help detect and prevent unauthorized access or suspicious activities by insiders.

Regular audits and behavioral analytics can also identify behavior patterns indicative of insider threats. By nurturing a security culture and implementing strong controls and training, organizations can reduce their vulnerability to social engineering and insider threats, safeguarding their critical assets from external and internal dangers.

13.3 Risk Management in System Security

In the real world, *absolute* security is not a realistic goal. Instead, managers must balance the value of protected assets, potential risks to the organization, and security costs. An asset might include company hardware, software, data, networks, people, or procedures. To achieve the best results, most firms use a risk management approach that involves constant attention to three interactive tasks: risk identification, **risk assessment**, and **risk control**, as shown in Figure 13-4. This process aims to prevent security threats from compromising data confidentiality, integrity, or availability. By diligently applying risk management principles, organizations can secure their digital assets against various cyber threats and align them with regulatory standards and industry best practices, ensuring an effective defense mechanism.

Figure 13-4 Risk management requires continuous risk identification, assessment, and control.



The efficacy of risk management lies in its structured approach to dealing with potential **vulnerabilities** and **threats**. It starts with thoroughly identifying risks and a detailed assessment to understand their impact and likelihood. Based on these insights, targeted mitigation techniques are developed and implemented to reduce the identified risks to acceptable levels. Continuous monitoring and incident response planning are integral to this process, ensuring the organization's security posture can adapt to new threats and changes in the digital landscape.

Collectively, these stages form the backbone of preemptive security, enabling organizations to safeguard their information assets. By embracing a risk management approach that includes identification, assessment, mitigation, continuous monitoring, and incident response planning, organizations can bolster their defenses against cyber threats. This reduces the likelihood of security incidents and minimizes their potential impact, ensuring that the organization remains resilient in the face of evolving cyber challenges.

13.3.1 Risk Identification

Risk identification is the first step in the risk management process, and it is dedicated to the methodical pinpointing of potential security threats and vulnerabilities. A threat constitutes any internal or external force that poses a risk to an asset, while a vulnerability represents a security flaw or weakness that an attack could exploit. Exploits are specific attacks that leverage these vulnerabilities to compromise security. Potential threat categories encompass natural disasters, software attacks, theft, and more, as shown in Table 13-3.

After identifying potential risks, it's crucial to systematically document and classify them based on their nature and the impact they could wield. Such classification aids in prioritizing risks and aligning mitigation efforts with the severity and likelihood of these risks materializing. Organizations should keep a current record of all assets and their corresponding vulnerabilities for practical risk identification.

Incorporating insights from stakeholders across different organizational sectors can enrich the risk identification process. This garners a variety of perspectives on potential threats, culminating in a more rounded formulation of risk management that is well-tailored to address the organization's complex risk landscape.

Table 13-3 System threats can be grouped into several broad categories.

Threat Categories and Examples	
Threat Category	**Example**
AI-powered attacks	Cybercriminals use AI algorithms to analyze network traffic and learn patterns, enabling them to craft highly targeted phishing campaigns that bypass traditional detection mechanisms.
Cloud misconfigurations	Incorrectly configured cloud services expose databases to the Internet without proper authentication.
Credential stuffing	Stolen user credentials from one service are used to gain unauthorized access to other services.
Deepfake technology and disinformation	Attackers use deepfake audio or video to impersonate a CEO, issuing fake instructions to transfer funds or disclose sensitive information.
Extortion	A hacker demands ransom after encrypting proprietary company data with ransomware.
Hardware and software failures	Critical server hardware malfunctions or a bug in system software causes data loss.
Human error or failure	An employee mistakenly configures a public cloud storage container, exposing sensitive data.
Insider threats	A disgruntled employee deliberately leaks sensitive customer information to competitors.
IoT vulnerabilities	Unsecured IoT devices are exploited to create a botnet that launches a DDoS attack.
Mobile device threats	A lost smartphone with access to corporate email allows sensitive information to be accessed.
Natural disasters	An earthquake damages a data center, leading to loss of physical servers and data.
Phishing attacks	Employees receive emails that appear to be from trusted sources, tricking them into providing login credentials.
Service failure	A cloud service provider experiences a prolonged outage, causing loss of access to critical business applications.
Social media exploitation	Social engineering via fake corporate social media profiles leads to brand damage and data leaks.
Software attacks	Cybercriminals exploit a zero-day vulnerability to deploy malware across the network.
Supply chain attacks	Compromise of a third-party vendor's systems leads to a breach of the organization's network.
Technical obsolescence	A legacy system lacks support and updates, leading to exploitation by newer attack vectors.
Theft of physical or intellectual property	A laptop containing confidential research data is stolen from a vehicle.
Trespass and espionage	Unauthorized personnel gain access to secure areas, potentially implanting surveillance devices.
Vandalism	A hacktivist group defaces the corporate website to display a political message.

13.3.2 Risk Assessment Techniques

After identifying potential risks, the subsequent step involves assessing their impact and probability. Employing risk assessment methodologies such as qualitative and quantitative analyses alongside risk matrices facilitates a thorough evaluation of the gravity of each identified risk. This critical evaluation aids in grasping the potential repercussions of security threats, thereby enabling their categorization based on urgency and severity for targeted mitigation efforts.

Qualitative analysis leverages descriptive metrics to gauge the severity and probability of risks, yielding an intuitive, albeit subjective, perspective on potential dangers. In contrast, quantitative analysis seeks to quantify risks with numerical values, grounding the assessment in statistical evidence and mathematical models for a more factual, objective outlook. For example, comparing risks by their impact values and vulnerability scores demonstrates how using numbers helps rank risks by their **criticality**. This method works, even though deciding on the exact numbers can be somewhat subjective.

Incorporating these assessments into a risk matrix amalgamates the impact and probability of risks into a cohesive model, enabling organizations to visualize and systematically prioritize risks. This is instrumental in directing resources judiciously toward mitigating the most pressing threats, ensuring a proactive and calculated approach to risk management.

13.3.3 Risk Control Strategies

Once risks have been assessed, the next step involves developing and implementing risk control strategies to reduce the likelihood or impact of potential threats. Control measures might include the following examples:

- Placing a firewall on the proxy server
- Assigning permissions to sensitive files
- Installing biometric devices to guard the computer room

Typically, management chooses one of four risk control strategies: **avoidance**, **mitigation**, **transference**, or **acceptance**. Avoidance eliminates the risk by adding protective safeguards. For example, a secure firewall might be installed to prevent unauthorized access to LAN computers. Mitigation reduces the impact of a risk through careful planning and preparation. For example, a company can prepare a disaster recovery plan for a natural disaster. Transference shifts the risk to another asset or party, such as an insurance company. Acceptance means that nothing is done. Companies usually accept a risk only when the protection is not worth the expense.

Risk management is a cyclical and ongoing process, necessitating continuous risk identification, evaluation, and mitigation. To succeed, those in charge of risk management must possess a well-rounded expertise encompassing business acumen, proficiency in information technology, and a solid understanding of security methodologies and resources. The impact of risk management lies in its adaptability: Constantly refining and adjusting mitigation strategies in response to the emergence of new threats and changes within the organization ensures that the approach to managing risks stays relevant over the long term.

13.3.4 Continuous Risk Monitoring

Continuous risk monitoring is vital for maintaining an up-to-date view of an organization's security posture. This involves regularly reviewing and assessing the usefulness of implemented security measures and identifying new threats as they arise. Monitoring tools and services can automate the detection of security incidents, facilitating timely responses.

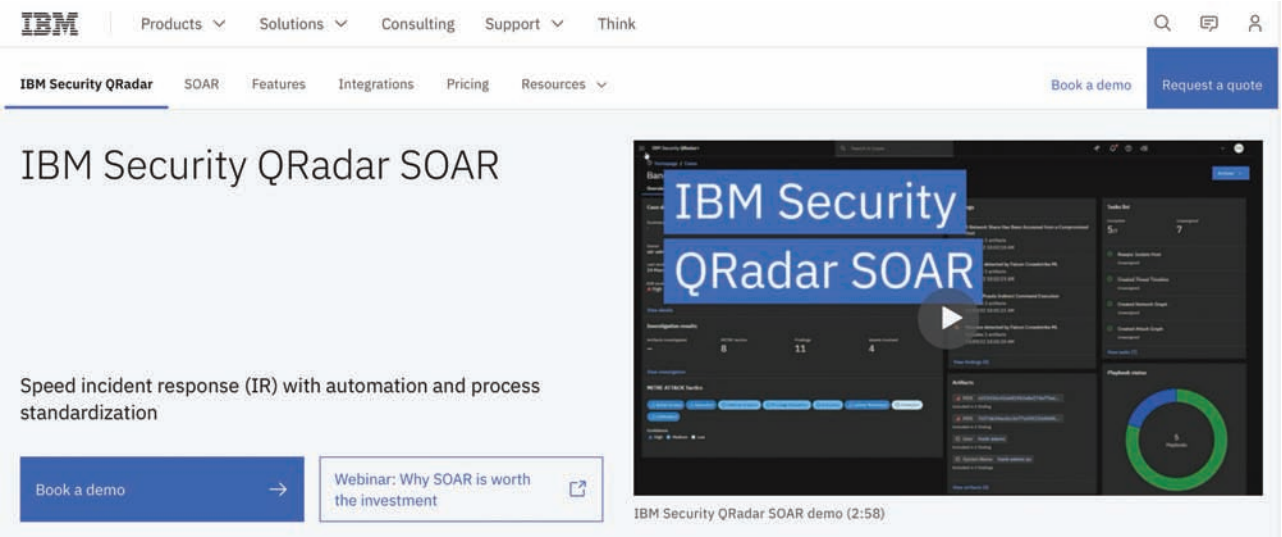
Integrating continuous monitoring with incident response plans ensures organizations can quickly react to potential security breaches. Regularly scheduled audits and assessments also contribute to a healthy monitoring operation, preemptively identifying improvement areas.

Feedback from continuous monitoring activities should inform the risk management process, leading to the refinement of risk assessment and mitigation. This cyclical process ensures that risk management efforts align with the organization's evolving security needs.

13.3.5 Incident Response Planning

Incident response planning is crucial to managing security risks, ensuring organizations are prepared to address and mitigate the impacts of security breaches. This process entails the creation of a plan that details the actions to be taken in the event of a security incident, including the assignment of responsibilities, communication plans, and steps for incident containment and resolution. Tools like IBM Security's QRadar SOAR (Figure 13-5) offer advanced capabilities for orchestrating and automating the response to incidents, showcasing the application of technology in enhancing incident management efforts.

Figure 13-5 IBM Security's QRadar SOAR helps cut incident response time with dynamic playbooks, customizable and automated workflows, and recommended responses.
Source: IBM



Developing an incident response plan requires thoroughly analyzing potential security incidents and crafting tailored responses for various incidents. This allows for rapid and coordinated action when a security breach occurs; organizations aim to limit damage and restore normal operations as quickly as possible. The success of such plans hinges on regular drills and simulations, which help refine the procedures and ensure that the response team is well-versed in their execution.

Ongoing evaluation and updates to the incident response plan are vital. They incorporate insights from practice drills and actual incidents to strengthen the organization's defensive posture. This iterative process contributes to building a resilient infrastructure capable of withstanding and recovering from cyberattacks, emphasizing the importance of preparedness and adaptability in the face of evolving cybersecurity challenges.

13.4 Security Levels

Protecting an organization's assets requires an exhaustive strategy that covers all operational and information areas. This approach spans the physical infrastructure safeguarding the hardware through the intricate digital networks facilitating communication and data exchange to the applications processing critical information. It further extends to preserving the essence of

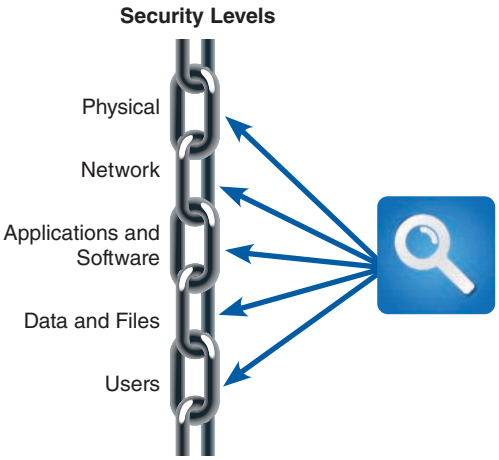


Figure 13-6 Each security link has a specific focus, and the overall chain is only as strong as the weakest link.

organizational data and the individuals who use and manage these systems daily. By adopting a layered security framework (see Figure 13-6), organizations position themselves to shield against various threats, markedly reducing vulnerabilities at each potential entry point.

This security architecture begins with physical security measures to ensure the tangible components of the IT infrastructure are well-protected against intrusion, theft, or damage. Building on this, network security protocols come into play, fortifying the pathways of data transmission against interceptions and unauthorized access. The security mantle then extends to applications and software, where coding and deployment practices are scrutinized and fortified to prevent exploitation. This protection is critical for safeguarding the systems and preserving the integrity and confidentiality of the data they handle.

Beyond the technical defenses, the human element of cybersecurity underscores the importance of cultivating a security-aware culture within the organization. Through continuous training and awareness initiatives, employees become active in defense and equipped to recognize and mitigate potential security threats. This human approach to security complements the technical safeguards, creating a resilient and adaptive security posture that can navigate the complexities of the modern cyber threat landscape. These layers form a cohesive shield, guarding the organization's assets and ensuring a forceful security stance against cyber risks.

13.4.1 Physical Security

Physical security measures are the first line of defense in protecting an organization's assets, ensuring that critical hardware and infrastructure are safeguarded against unauthorized access, theft, and damage. This encompasses a broad spectrum of protective measures, from access control systems that regulate entry into secure zones to surveillance mechanisms that offer real-time monitoring of organizational premises. Additionally, environmental safeguards play a pivotal role, providing a bulwark against natural disasters such as fires and floods and ensuring the continuous operation of critical systems. The layout of physical spaces is also paramount, designed to minimize potential vulnerabilities, coupled with stringent protocols for disposing of sensitive materials and advanced visitor management systems to oversee and control non-employees' access to critical areas.

Focusing on the core areas of operations center security, the approach extends to the meticulous protection of servers, network hardware, and other pivotal equipment. These centers, whether housed within expansive dedicated facilities in larger corporations or multipurpose spaces in smaller entities, demand tailored security measures to thwart unauthorized access. The safeguarding of computing devices, encompassing both stationary and mobile units, further underscores the necessity of a rounded security perspective. This includes securing network infrastructure, which is vital for the uninterrupted flow of information and the overall integrity of data communications within the organization. The protection afforded to storage devices, whether internal or external, ensures data safeguarding from physical threats, embodying a critical aspect of data integrity and confidentiality.

The Internet of Things (IoT) introduces a new dimension to physical security challenges, necessitating inventive ways to protect a vast array of connected devices. These IoT devices, which often extend beyond the traditional confines of secure spaces, require a specialized focus to mitigate the risks posed by their widespread deployment and inherent vulnerabilities. Employee training on physical security practices, the recognition of tailgating risks, the enforcement of secure badge protocols, and the encouragement of vigilance in reporting suspicious activities are indispensable components of a physical security framework. These measures and policies collectively ensure a fortified defense against potential threats,

preserving the organization's operational integrity and safeguarding its technological backbone across all facets of physical security.

Operations Center Security

Ensuring the security of environments that house critical infrastructure, such as data centers, server rooms, and communication hubs, is paramount for protecting essential operational functions. These locations require stringent security measures to prevent unauthorized access and mitigate environmental hazards. Implementing advanced security strategies, including biometric access controls, surveillance systems, and environmental monitoring tools, is crucial for maintaining the integrity of these vital assets. Additionally, developing disaster recovery plans and installing reliable backup power systems like an **uninterruptible power supply (UPS)** are essential for maintaining continuity in the face of unforeseen disruptions, enhancing the resilience of these critical centers against various threats ranging from human error to natural disasters.

To further fortify these essential areas, state-of-the-art security measures bolster perimeter defenses, secure entryways, and restricted access zones. Doors with internal hinges and electromagnetic locks, supported by UPSs, ensure controlled access under all circumstances, with safe exit options in emergencies. Adopting advanced biometric scanning systems enhances traditional security methods, providing more secure and personalized access control by analyzing unique physical characteristics, as shown in Figure 13-7 and exemplified by Apple's Face ID, as described earlier in the book. This is complemented by the deployment of video cameras and motion sensors for comprehensive surveillance, offering real-time monitoring and the ability to respond promptly to unauthorized activities or environmental changes. Technological advancements and rigorous procedural protocols establish a layered security approach to defend against potential threats and ensure critical systems' continuous and secure operation.

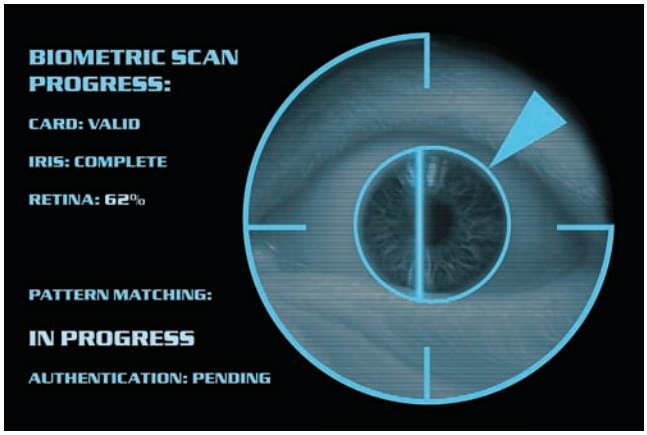


Figure 13-7 Companies use biometric scanning to analyze the features of the eye's iris, which has more than 200 points that can be measured and used for comparison.
Source: Andy Piatt/Shutterstock.com

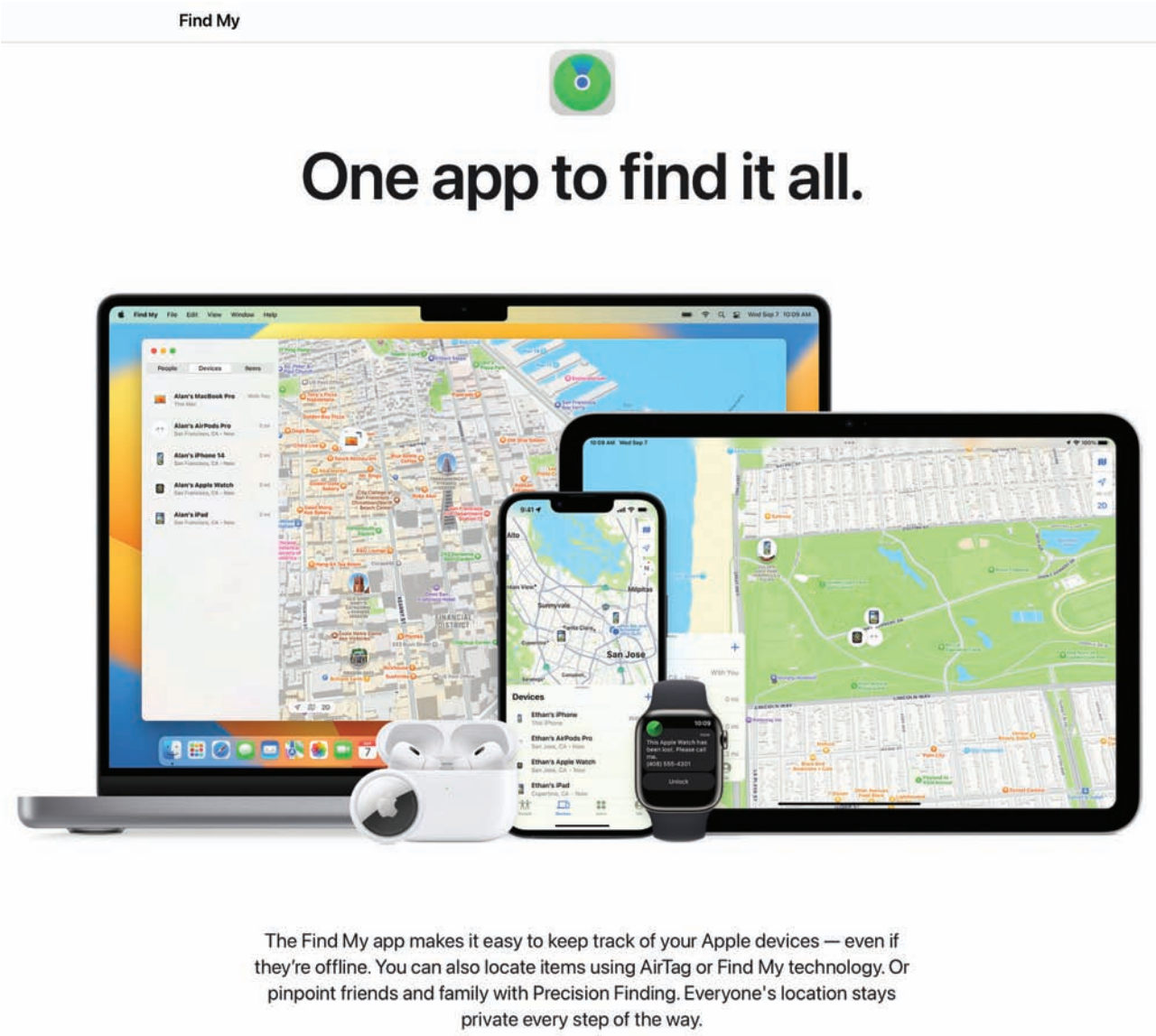
Computing Devices Security

Securing fixed computing devices such as servers and desktops against unauthorized access and tampering is imperative, encompassing a range of physical security measures from locking mechanisms for server racks and secure mounting for desktops to stringent access controls for critical computing areas. Environmental controls complement these measures to shield devices from overheating, power surges, and other hazards that threaten data integrity and availability. Equipping computing cases with locks, securing server racks to deter the unauthorized use of **keystroke loggers**, and employing software solutions to combat keystroke logging programs are crucial. Often hidden from users, these programs can capture every keystroke, including sensitive passwords, posing a serious security risk. They can be removed by antivirus and antispyware software, as discussed in Section 13.4.3. Furthermore, the adoption of monitor screen savers and a **BIOS-level password**, also called a **boot-level password** or a **power-on password**, enhances security for unattended servers and workstations. These measures prevent unauthorized use and ensure that physical security measures extend beyond access restrictions to include protection against hardware tampering and software breaches.

Mobile computing devices, such as smartphones and tablets, pose distinct security challenges due to their portability, making encryption, biometric authentication, and remote wipe capabilities essential for safeguarding data if these devices are lost or stolen. Techniques for secure usage, alongside physical tracking, are vital in minimizing risks,

while user education on secure practices—like not leaving devices unattended and using secure storage—enhances device protection. Additionally, for laptops and notebooks, selecting operating systems with secure logins, BIOS-level passwords, and strong firewall protection is crucial. Engraving devices with identifiable information, utilizing tamper-proof asset ID tags, and opting for nondescript carrying cases can deter theft. Built-in security features such as fingerprint readers, facial recognition, and cable locks or laptop alarms fastened to a **universal security slot (USS)** further secure these devices. Backing up vital data, employing tracking software for recovery, and adhering to rigorous password protection policies complement technical measures, ensuring security across mobile devices. Awareness of high-risk situations and adopting encryption for sensitive files are also vital in maintaining the integrity and privacy of mobile data. For example, services like Apple's Find My app, shown in Figure 13-8, provide tools for location tracking and remote data management, illustrating the multidimensional approach required for mobile device security.

Figure 13-8 Apple's Find My app helps locate your iPhone, iPad, Mac, Apple Watch, AirPods, or Beats. It can also help locate Family Sharing devices, such as AirTags on luggage.
Source: Apple Inc.



Network Infrastructure Security

Securing the physical components of network infrastructure—routers, switches, and firewalls—is paramount to safeguarding the backbone of organizational communication and data flow from physical threats. This complements network security, as discussed in Section 13.4.2, which focuses on data fidelity and availability as it traverses networks. Physical access controls to network closets, data centers, and other critical network points prevent unauthorized personnel from tampering with or accessing these devices. Additionally, measures like locking devices in secure cabinets, using tamper-evident seals, and implementing environmental controls protect against physical damage and ensure the longevity and reliability of network infrastructure.

Beyond these measures, regularly auditing the physical security of network infrastructure helps identify and rectify potential vulnerabilities, ensuring continuous protection. Training network personnel in security best practices and maintaining an inventory of all network devices enhance the ability to respond swiftly to security incidents. By prioritizing the physical security of network infrastructure, organizations can prevent disruptions to network services, ensuring stable and secure operations.

Storage Device Security

Safeguarding storage devices, including internal and external media, is crucial for preventing theft, damage, and unauthorized access. Internal storage devices, such as hard drives in servers or desktops, benefit from physical access controls and secure enclosures that deter tampering and data theft. Adopting stringent usage policies, encryption, and secured storage for USB drives and portable hard drives is imperative to counter the risks their portability introduces. Additionally, the application of **tamper-evident cases**, which indicate unauthorized access attempts through mechanisms such as an indicator light that remains lit until reset by a password, adds a layer of security. While not impervious to intrusion, these cases increase the likelihood of detecting security breaches; many servers incorporate tamper-evident technology as a standard feature to enhance protection.

A solid storage security strategy extends beyond physical safeguards to include user education on the potential risks associated with storage devices and the importance of secure data disposal or destruction methods for media no longer in use. Implementing **data loss prevention (DLP)** that monitors and controls the flow of sensitive information onto external devices amplifies this protection, ensuring that data breaches and losses are minimized. This approach to securing internal and external storage devices is vital for preserving the confidentiality and integrity of critical data, reinforcing an organization's defense against the myriad of threats targeting storage media.

IoT Security

IoT security includes the expanding perimeter of physical security in the digital age. IoT devices often blend into physical environments more seamlessly than traditional computing devices, presenting unique security challenges that merit specific attention. Securing IoT devices involves protecting them from physical tampering and ensuring the security of the environments in which they are deployed. This includes implementing strong authentication measures to prevent unauthorized access and using secure, tamper-evident enclosures for devices located in public or vulnerable areas. Additionally, regular security assessments of IoT devices can identify potential physical vulnerabilities that attackers could exploit.

Given the scale and variety of IoT deployments, adopting a security-by-design approach from the inception of device development is essential. This involves integrating physical security considerations into the design and operations stages, ensuring IoT devices are resilient to physical attacks. Educating users and administrators about the unique security challenges posed by IoT devices and promoting the adoption of standardized security protocols across IoT ecosystems are vital steps in mitigating the risks associated with these increasingly ubiquitous devices.

13.4.2 Network Security

Network security is paramount in ensuring information's integrity, confidentiality, and availability as it traverses through networks. This encompasses a range of critical components designed to protect against unauthorized access and potential security breaches. Firewalls serve as a fundamental barrier, controlling access to the network by blocking unauthorized entries while permitting legitimate communications. **Intrusion detection systems (IDSs)** are also crucial in monitoring network traffic for suspicious activities and promptly identifying potential threats. Moreover, implementing virtual private networks enables secure remote access, extending the network's security measures to users regardless of location, safeguarding data in transit through strong encryption, and adopting secure network protocols.

The network architecture, which connects two or more devices to facilitate the sending, receiving, and sharing of data, underpins the necessity for network security measures. Each computer within the network relies on a **network interface**—a blend of hardware and software—that enables interaction with the network. To protect the data that flows across these connections, encryption is employed, encoding information so that only authorized parties can decode and access it. This process is vital for maintaining the confidentiality and security of network traffic, underscoring the importance of encryption in the broader network security strategy.

Beyond technical safeguards, the human element of network security is equally critical. Educating users on safe Internet practices and the dangers of phishing and other social engineering tactics enhances the overall security posture by equipping users with the knowledge to identify and avoid threats. Regular network security assessments through penetration testing and vulnerability scanning are essential for identifying and addressing vulnerabilities. Additionally, keeping network devices and software up to date with regular patches and updates ensures that security measures evolve in tandem with emerging threats, maintaining the resilience of network security defenses in the face of constantly evolving cyber challenges.

Firewalls

A **firewall** serves as the essential boundary safeguarding a local network or intranet from the external digital world of the Internet. To function correctly, a firewall has at least one network interface connected to the Internet and another linked to the local network, overseeing data flow between these distinct zones. The core of a firewall's operation is its ability to scrutinize network traffic through each interface, applying predefined rules to assess whether the incoming or outgoing data should be permitted passage. These rules reflect the organization's security policies, enabling the firewall to make instantaneous decisions to allow, reject, or silently drop data packets based on compliance with established criteria. Senders are typically notified about rejections, whereas dropped packets are silently discarded, providing no indication of their fate to the potential intruder.

Consider a company firewall configured to manage service access, as shown in Table 13-4. Specific rules are applied to ensure secure communication. For instance, access to the company's public web server is restricted to ports 80 (HTTP) and 443 (HTTPS), while the mail server communication is confined to port 25 (SMTP). This selective traffic permitting ensures that only legitimate requests are processed, safeguarding the servers from unauthorized access and potential cyber threats. Additionally, the firewall allows unrestricted internal traffic within the private LAN, facilitating seamless operation and communication among the internal network's devices. This configuration exemplifies how firewalls protect against external threats and maintain the necessary flexibility for internal network functionality, striking a balance between security and operational efficiency.

Encrypting Network Traffic

The vulnerability of network traffic to interception and potential alteration or theft underscores the critical need for encryption as a protective measure. Sensitive information such as passwords or credit card numbers, when transmitted in **unencrypted** form—known as **plaintext**—can be compromised. Encryption acts as a safeguard, ensuring that while the data remains visible within the network, its contents are rendered unintelligible to unauthorized observers.

Table 13-4 Examples of rules determining whether the firewall will allow traffic to pass.

Firewall Rules						
Rule	**Interface**	**Source**	**Destination**	**Port**	**Action**	**Notes**
1	Public	Any	Web server	80	Accept	Allow HTTP traffic to the web server.
2	Public	Any	Web server	443	Accept	Allow HTTPS traffic to the web server.
3	Public	Any	Web server	Any	Reject	Block all other traffic to the web server.
4	Public	Any	Mail server	25	Accept	Allow SMTP traffic to the mail server.
5	Public	Any	Mail server	Any	Reject	Block all other traffic to the mail server.
6	Public	Any	Any	Any	Drop	Drop all other unspecified public traffic.
7	Private	LAN	Any	Any	Accept	Allow all traffic from LAN.
8	Private	Specific IPs	Sensitive server	Specified	Accept	Specific rules for sensitive or critical services.
9	Public/private	Any	Network devices	Specified	Accept	Allow necessary traffic to network infrastructure.
10	Any	Any	Any	Any	Log	Log denied traffic for review.

This security measure conceals the data's content and purpose, thereby mitigating the risks associated with data transmission over networks and protecting against the unauthorized access and exploitation of personal and financial information.

Encryption employs two primary techniques to secure data: **private key encryption** and **public key encryption (PKE)**. Private key encryption is symmetric because it utilizes a single key for encryption and decryption, offering a straightforward and efficient approach. However, this method introduces the challenge of securely sharing the key between communication parties without interception, a fundamental vulnerability that can compromise the security of the transmitted data. In contrast, PKE is asymmetric, employing a dual-key system wherein a public key encrypts the data and a private key unique to the recipient decrypts it. This system allows for the public key to be shared openly without jeopardizing the security of the encrypted data, ensuring that only the intended recipient can access the information. PKE is especially prevalent in secure online transactions, highlighting its usefulness in maintaining the confidentiality and integrity of digital communications.

Ports and Services

Understanding the role of ports and services within a network is essential to managing its security, allowing for an in-depth analysis of the mechanisms through which network traffic is directed and safeguarded. Network administrators can achieve a detailed oversight of how data is processed and protected by understanding the specific functions of various ports and the services that operate on them. This knowledge is instrumental in managing network

traffic, ensuring that each packet of data is accurately routed to its intended destination within the system, much like how mail is sorted to individual mailboxes in a large apartment complex based on box numbers.

Ports serve as gateways for incoming network traffic, and each is identified by a unique number directing traffic to the appropriate application on a receiving computer. In TCP/IP networks, which form the backbone of the Internet, every piece of data is tagged with a destination port number, guiding the system in traffic distribution. This process is vital for maintaining order and efficiency in data handling and is analogous to distinguishing between multiple mailboxes at the same address using unique identifiers.

Similarly, **services** are applications that "listen" on designated ports, processing incoming data directed to those ports. For instance, a standard email service might monitor port 25, handling all traffic. The relationship between ports and services is crucial for network security. Attacks like port scanning and denial of service can exploit open ports and vulnerabilities.

Understanding how malicious activities can target ports and services is crucial for safeguarding a network's integrity. For example, **port scans** are reconnaissance methods attackers use to identify active services on a computer by attempting connections across various ports. This technique records which ports respond to the connection attempts, revealing the services running on the device. For instance, a response from port 25 would suggest the presence of an email server. Such scans are instrumental for attackers aiming to construct a detailed layout of a network's services, highlighting potential vulnerabilities for further exploitation.

A **denial-of-service (DoS)** attack represents a critical threat to network stability. It functions by inundating services with excessive requests. This deluge of demands can overload the targeted systems, making them unable to process legitimate traffic and denying service to actual users. The mechanism behind these attacks is relatively straightforward but can cause significant disruption, affecting the availability of crucial network services and resources.

A more severe form of these attacks is the **distributed denial-of-service (DDoS)** attack. DDoS attacks utilize a compromised computer network to launch a coordinated attack, as shown in Figure 13-9, overwhelming servers or entire network segments with a flood of malicious traffic. This type of cyberattack can bring about extensive damage and downtime. Entities such as the United States Cybersecurity and Infrastructure Security Agency (CISA), shown in Figure 13-10, are specifically tasked with addressing and mitigating the impacts of such high-level cyber threats.

Figure 13-9 In a DoS attack, a server is overwhelmed by numerous fake authentication requests, leading to its incapacitation. DDoS attacks, a more advanced form, utilize a network of malware-infected botnets to amplify the assault, rendering the target server inoperative.

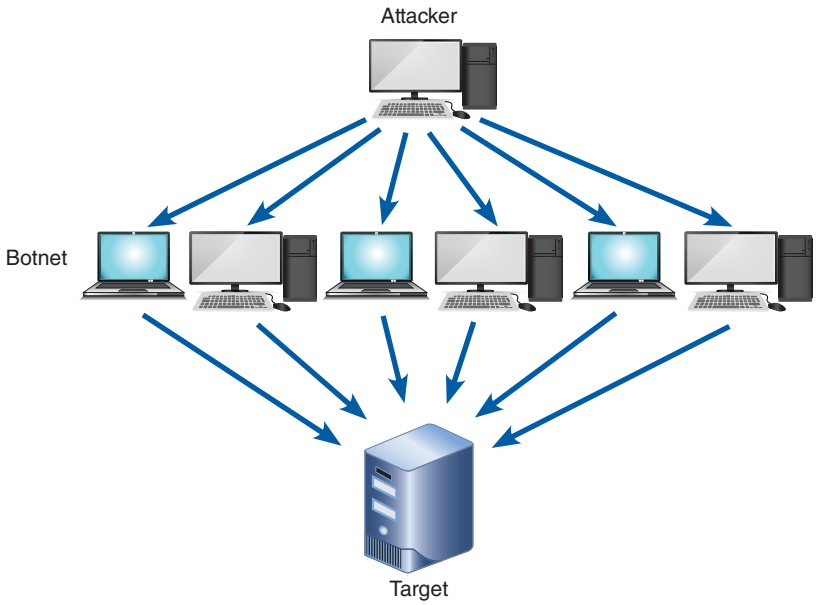


Figure 13-10 CISA is the operational lead for federal cybersecurity and the national coordinator for critical infrastructure security and resilience.
Source: U.S. Cybersecurity and Infrastructure Security Agency

Network Intrusion Detection

After establishing a fundamental security infrastructure encompassing firewalls, encryption, and meticulous management of ports and services, the integration of a **network intrusion detection system (NIDS)** becomes an essential progression. These systems scrutinize network traffic vigilantly, searching for any indicators of suspicious behavior that deviates from the norm. As a critical element of an overarching network security strategy, a NIDS is the technological equivalent of a watchful guard, continuously monitoring digital comings and goings to identify and respond to potential security threats. Its role is to alert system administrators to anomalies, enabling a swift response to mitigate any potential cybersecurity incidents. The success of a NIDS in protecting network integrity relies heavily on its ability to differentiate between legitimate network operations and possible threats. This process requires careful tuning to minimize false alarms and ensure that alerts are both timely and pertinent.

The deployment of a NIDS within a network must be calculated, ensuring these systems are positioned to monitor all network traffic. Broad surveillance of data movement is achieved by pairing a NIDS with network infrastructure components like switches, enabling these systems to offer a bird's-eye view of network activity. Although the implementation and ongoing management of a NIDS demand a dedicated administrative effort, the payoff in enhanced security is meaningful. A NIDS not only aids in identifying and neutralizing attempts at unauthorized access but also plays a vital role in analyzing network traffic patterns. This analysis can reveal insights into network performance and potential vulnerabilities, fortifying the network's defenses and contributing to a more secure and resilient digital environment.

Wireless Networks

As discussed earlier in the book, wireless network security is a vital concern because wireless transmission is much more vulnerable than traffic on a wired network. However, if wireless traffic is encrypted, any data an unintended recipient or intruder intercepts will be useless.

Wired Equivalent Privacy (WEP), the earliest form of wireless security, required each wireless client to use a unique, preshared key. Although many home and small office networks used this method, it provided relatively weak protection.

Wi-Fi Protected Access (WPA) was developed in response to WEP's shortcomings, introducing substantial improvements through protocols established by the Wi-Fi Alliance. The subsequent release of **WPA2** represented a further enhancement in wireless security, adopting the **IEEE 802.11i** standard and mandating WPA2 certification for all Wi-Fi-certified devices from 2006 onward. **WPA3** represents the newest evolution of the WPA protocol, introducing advanced security enhancements and rectifying weaknesses identified in WPA2. This update offers improved methods for user authentication and utilizes stronger encryption techniques, positioning WPA3 as the preferred standard for safeguarding Wi-Fi networks.

Private Networks

Private networks are crafted specifically for use within an organization, creating a secure environment shielded from external access. This exclusivity paves the way for more advanced network security practices, including the segmentation of networks to enhance further protection against unauthorized intrusions. By limiting access to organization members, private networks provide a controlled space where sensitive information can be shared securely, laying the groundwork for deeper network security. This approach to network design is integral in establishing a secure infrastructure, offering a foundation upon which additional security measures can be implemented to safeguard data and resources.

However, the goal of securing all network traffic can sometimes conflict with the need for optimal network performance. Encryption, while pivotal for protecting data in transit, can overload network resources, potentially slowing down critical operations. Organizations may opt for a private network solution in scenarios where maintaining high-speed network communication is paramount, such as the connection between a web server and a database server. This setup involves a dedicated connection that isolates internal traffic from external visibility, allowing the safe transmission of unencrypted data within the network. Organizations can balance security and performance by ensuring that each computer within the private network has a dedicated interface with no external connections, enabling efficient and secure data handling.

Virtual Private Networks

Virtual private networks (VPNs) offer a powerful solution for secure remote access, leveraging the public Internet to connect users to private networks without compromising security. VPNs act as a bridge, extending the protective measures of a private network to include remote users. This capability is particularly valuable for organizations that operate on a global scale or have a workforce distributed across various locations. By employing VPN technology, these organizations can ensure that remote employees have secure and reliable access to the company's internal resources, just as if they were physically in the office.

VPNs are essential for companies supporting a larger remote workforce or connecting multiple office locations securely. VPNs use the Internet or a company intranet as a conduit to create secure connections by initiating a particular key exchange that requires authentication. Once authenticated, a secure communication channel, or **tunnel**, is established, allowing data to be transmitted securely between the remote user and the company's network. This encrypted tunnel ensures that all data exchanged remains confidential and protected from potential interception or eavesdropping. As the trend toward remote work grows, VPNs have become increasingly critical in providing security and reliability, enabling seamless access to corporate resources from any location.

13.4.3 Application and Software Security

Securing applications and software is a critical aspect of an organization's cybersecurity framework, aimed at safeguarding against unauthorized access and data breaches. Secure coding practices are essential to prevent common vulnerabilities such as SQL injection and

cross-site scripting. In addition to coding securely, implementing application firewalls is necessary to oversee and regulate traffic flow to and from applications, adding an extra layer of defense.

Ongoing security evaluations, including thorough code reviews and application testing, are indispensable for uncovering and remedying security weaknesses. Integrating security within the SDLC, a practice known as **DevSecOps**, embeds security as a fundamental component of software creation. This shift tightens security from the initial stages of development and supports collective ownership of cybersecurity responsibilities within the development team.

Educating users on the secure utilization of applications is paramount. Users should know the dangers of downloading and employing nonverified software and the significance of adhering to software update schedules. Training end users strengthens the overall security posture by enabling them to contribute to the organization's security initiatives. Meanwhile, analysts must diligently safeguard server-based applications on the back end by analyzing their functions, pinpointing potential security risks, and meticulously reviewing all pertinent documentation. Application security is built upon a thorough understanding of service configurations, secure coding, system hardening, permission settings, input validation, diligent maintenance of software patches and updates, and logs.

Services

Network security fundamentally involves managing the services applications use to communicate over the network. Each service operates by listening for requests on designated ports; understanding which services are active is crucial for maintaining a secure system. Utilizing port scan utilities, network administrators can survey open ports to identify running services. If a service associated with a specific application is deemed nonessential, disabling it is a standard security practice. This tightens security and enhances the system's performance and reliability by reducing unnecessary load and potential attack vectors. Eliminating superfluous services minimizes the risks of what are known as **security holes**—vulnerabilities that threat actors can exploit.

For instance, a File Transfer Protocol (FTP) service that is not securely configured presents an opportunity for an attacker to compromise the server. In such a scenario, a hacker could exploit the FTP service to upload malicious code, corrupt data, or create a back door for further attacks. This example underscores the importance of stringent service configuration and the principle of least privilege—only necessary services should be enabled and exposed to network traffic. A rigorous approach to managing services includes regular audits, updates, and patches to services and the applications they support, ensuring that the network's integrity is maintained against potential security breaches.

Secure Coding Practices

Secure coding practices are essential for developing software that is inherently resistant to vulnerabilities and security threats. These practices encompass many principles and methodologies to identify and mitigate common security flaws in code, such as buffer overflows and injection flaws. Developers can ensure that their code meets high security benchmarks by adhering to established coding standards emphasizing security, such as those recommended by the **Open Web Application Security Project (OWASP)**. Integrating these security measures from the beginning of the SDLC is crucial, as it allows for identifying and resolving potential security issues before they become problematic.

Including secure coding practices within the software development process is not just a technical necessity; it's an investment in future security and integrity. Emphasizing the importance of security at the code level ensures that applications are built to prioritize safeguarding data and functionality against potential threats. This approach enhances the overall security posture of software systems, making them more durable against attacks and reducing the likelihood of security breaches. Developers can contribute to creating safer, more reliable software solutions by diligently applying secure coding practices.

Hardening

System **hardening** enhances security by stripping down a system to its essential components, eliminating superfluous accounts, services, and functionalities that could serve as potential entry points for attackers. This process is critical in fortifying a system's defenses, especially because many software packages come with default configurations that may not prioritize security, potentially exposing the system to risks. Common examples include overly permissive account settings and lax file-sharing controls. Hardening can be streamlined through standardized configuration templates for efficiency, especially within larger organizations, allowing for consistent and rapid application across multiple systems.

The scope of hardening extends to implementing protective software, such as antivirus and antispyware programs. These tools are indispensable in the detection and elimination of malware, a term that encompasses various forms of harmful software like worms, Trojan horses, keystroke loggers, and spyware—all designed to infiltrate and disrupt the normal functions of a computer system. Through the active monitoring and removal of such threats, these security applications play a vital role in maintaining the integrity and availability of computer systems, serving as a critical component of the hardening process.

Application Permissions

Applications are usually set up to operate under specific user rights to ensure that access and capabilities within the software are appropriately controlled. For instance, accounts with **administrative privileges** or **superuser** privileges typically have unfettered access, allowing these accounts to make necessary changes. In contrast, regular users may be granted the ability to input data while being restricted from altering or removing existing information. This tiered approach to permissions is a fundamental security measure designed to safeguard against unauthorized modifications or potentially harmful actions within the application.

Configuring applications to restrict the ability of nonprivileged users to alter core functionalities or settings is crucial in preventing unauthorized changes. Users should have enough access to perform their roles efficiently but not so much that they can impact the application's integrity or security. The concept of **user rights**, or permissions, is an integral part of system security and is explored in the context of file security later in this chapter. This ensures that every interaction with the application is governed by clearly defined and enforced permissions, minimizing the risk of accidental or malicious disruptions to the system.

Input Validation

Input validation is crucial in maintaining data integrity and system security, as emphasized earlier in this book. When designing user interfaces for applications, it's essential to consider how the system handles unexpected or incorrect inputs. For instance, if a user mistakenly enters an alphabetic character or an out-of-range number like 31 into a field that only accepts the numbers 1 to 10, the application should be programmed to provide a clear and informative error message. Such safeguards are integral to the application's resilience, ensuring that only valid data is processed and stored.

As previously covered, neglecting to implement proper data entry and validation checks can have far-reaching consequences. Systems produce erroneous outputs without these checks, leading to costly maintenance and unpredictable system behavior. Input validation mechanisms are not just a feature of good user interface design; they are a critical component of a system's overall reliability and user experience, helping to prevent the ripple effects of bad data through the system's operations.

Patches and Updates

As highlighted in the chapter on maintenance in this book, security vulnerabilities may emerge at any given moment within a system's operational life cycle. Utilizing patches is critical for addressing these vulnerabilities, enhancing the system's security, and ensuring its components are current. However, introducing new software, including patches, necessitates rigorous testing to avoid unintended consequences. Before implementation, it's essential to assess the

potential risks associated with not applying the patch against the likelihood of it inadvertently impacting other system functionalities.

Many organizations rely on **third-party software** for their operational needs, and while patches from these vendors are generally reliable, each update requires thorough scrutiny. Given the complexity and cost, researching and integrating patches prompts some software providers to offer an **automatic update service**. These services allow applications to verify and incorporate necessary patches from the vendor's server autonomously. Though this method is undeniably efficient, permitting automatic updates without manual oversight introduces risks. Therefore, such practices should be approached with caution; organizations must ensure there is a mechanism to reverse updates should they lead to adverse outcomes or necessitate further adjustments by IT professionals.

Software Logs

Operating systems and applications are designed to meticulously document every event through **logs**, capturing critical details such as dates, times, and specific actions. These logs are invaluable for analyzing past security incidents and fortifying defenses against future threats. For instance, a series of unusual login failures recorded in the logs might shed light on an attempted security breach, enabling a deeper investigation into the incident. Beyond intrusion attempts, logs also chronicle system errors, user login activities, file interactions, and other behaviors that could indicate unauthorized or improper use of system resources.

Software logs must be actively and continuously scrutinized to safeguard against misuse and identify potential security vulnerabilities. Monitoring these logs allows for the timely detection of irregular patterns or activities that may signify security threats. In line with strategies discussed in the network security section earlier, employing a NIDS enhances this monitoring capability. A NIDS acts as an early warning system, promptly notifying system administrators of anomalous events that could indicate a security compromise. This plays a crucial role in system security management.

13.4.4 Data and File Security Strategies

Data and file security measures are crucial in safeguarding data's confidentiality, integrity, and availability across all stages—whether stored, utilized, or transferred. Encryption is a cornerstone technique in this protective arsenal, rendering data unintelligible to unauthorized parties without the corresponding decryption key. This ensures that, in scenarios where data might be intercepted or improperly accessed, it remains secure and indecipherable to those without the necessary permissions.

In addition to encryption, rigorous access control protocols are essential for maintaining data security. User authentication and authorization are pivotal in defining and restricting access to sensitive information, ensuring that only those with explicit permission can view or manipulate critical data. DLP tools augment these protections by overseeing and regulating data movement within an organization, preventing the unauthorized dissemination of sensitive information and guarding against potential leaks or breaches.

Data security resilience heavily relies on establishing hardy backup and redundancy mechanisms. These practices are essential to swiftly restoring data following cyberattacks, equipment malfunctions, or environmental catastrophes. Moreover, educating users about secure data management practices and the potential consequences of data breaches is vital for advancing an organization's security awareness culture. For systems analysts, the focus extends to the meticulous protection of computer configurations, personal information, and other sensitive files through encryption and the assignment of access permissions.

Encryption

As explained in Section 13.4.2, encryption is a vital mechanism to obscure the contents of files or documents, safeguarding them from unauthorized access. This process is essential for all types of corporate data to ensure its confidentiality and security. However, encryption

becomes particularly critical when dealing with sensitive information, such as personnel or financial records. The protective encryption layer helps prevent potential breaches and ensures that sensitive data remains accessible only to those with the requisite authority.

Modern operating systems come equipped with built-in features for data encryption, offering a straightforward means for securing user data. This accessibility allows organizations to implement encryption protocols quickly, shielding sensitive material. By leveraging these built-in encryption tools, companies can enhance their overall security posture, protecting critical data from the ever-present threat of unauthorized access and ensuring that their confidential information remains secure.

Permissions

File security hinges on creating and managing permissions that dictate the level of access a user has to specific files or directories within a server environment. These permissions, typically categorized as read, write, and execute, establish safeguards for data integrity and access control. For example, "read" permission allows a user to view a file's contents, "write" permission enables the modification of a file, and "execute" permission grants the ability to run a program. Additionally, permissions extend to directories, where "read" allows listing the directory contents, "write" enables adding or deleting files within that directory, and "execute" permits visiting or traversing the directory in the file system.

When setting up these permissions, system administrators must adopt a principle of least privilege, ensuring users possess only the permissions necessary for their roles. This approach minimizes potential security risks by restricting access to sensitive information. The process for assigning these permissions may vary across organizations, with some requiring management approval for permissions beyond basic user access. In contrast, others grant system administrators considerable autonomy in this regard. Implementing a detailed, rigorously enforced permissions policy is essential regardless of the method. Such a policy bolsters file security and diminishes the system's susceptibility to unauthorized access and potential security breaches.

User Groups

When individual users require the ability to collaborate and modify shared files, they often seek elevated permissions that allow them to alter file contents. However, a better approach from a system administrator's perspective is establishing a user group specifically for this purpose, incorporating the relevant users into this group, and assigning the necessary file permissions collectively to the group rather than individually. Many organizations favor this method, as it aligns users' access rights more closely with their work responsibilities and tasks rather than their job title or position within the company hierarchy. This approach also counters a **privilege escalation attack**, which is an unauthorized attempt to increase permission levels.

A group-based approach to permissions management also offers flexibility in handling personnel roles or position changes. Should an individual transition to a different department or assume new duties, their group memberships can be adjusted accordingly. This ensures that access rights remain appropriate and secure, minimizing the risk of unnecessary permissions that could lead to security vulnerabilities.

13.4.5 User Security

Safeguarding individual users is a critical cybersecurity pillar, underscoring the necessity of protecting against many threats and vulnerabilities in the digital landscape. User security begins with the meticulous management of identities, where accurate authentication and authorization processes form the bedrock of a secure digital domain. This ensures access is granted judiciously, fortifying the system's defenses against unauthorized breaches. Implementing stringent password protection measures further shields user accounts from potential intrusions, showcasing the importance of cultivating strong password practices among users.

Beyond the technical safeguards, the essence of user security is measurably enhanced through training and awareness programs. Educating users on the spectrum of cybersecurity threats, particularly the sophisticated tactics employed in social engineering attacks, empowers them with the necessary knowledge to navigate the digital sphere cautiously. This education raises awareness and equips users with the tools to detect and thwart malicious attempts to undermine security. Within this context, the nuanced tactics to counteract social engineering attacks gain prominence, emphasizing practical defenses users can employ to protect their digital assets.

As the technological landscape evolves, it introduces challenges and opportunities in user security. Staying abreast of the implications of emerging technologies on user safety is indispensable for preempting new avenues of risk. Concurrently, addressing user resistance toward implemented security measures and technologies is crucial. Organizations can cultivate a culture of vigilance and resilience by nurturing an environment that prioritizes security education and encourages active engagement with cybersecurity practices. This approach to user security, which interweaves technical measures with user-centric education and empowerment, lays the groundwork for a more secure and informed user base.

Identity Management

Identity management is the cornerstone of user security, establishing how users and system components are accurately recognized and authenticated. This process underpins the security measures that protect against unauthorized access and potential breaches. By clearly identifying legitimate users and other elements within the network, organizations can create a secure digital landscape where access is meticulously controlled. The concept extends beyond mere user identification to include machines, devices, applications, and services, each assigned a unique identity that enhances system integrity and security. Authentication processes often rely on multiple factors, such as passwords, biometric data, or physical tokens, to form a defense against unauthorized entry.

The importance of well-structured identity management cannot be overstated. It requires a careful balance among technological solutions, security imperatives, privacy concerns, cost considerations, and the impact on user productivity. As technology evolves, so too does the approach to identity management, with continuous advancements being made to address the challenges posed by new threats and changing user behaviors. Recognizing the critical role of identity management in safeguarding digital assets, IT managers and organizations are urged to prioritize developing and implementing identity management solutions. This emphasis on identity management is essential for mitigating the risk of cyber intrusions and ensuring information systems' ongoing security and resilience.

Password Protection

After establishing user identities, safeguarding them becomes the next critical focus, with password protection as the primary defense mechanism. The efficiency of this protection hinges on the implementation of strict password policies, which dictate the required strength and complexity of passwords. Such policies typically mandate passwords to be a minimum length and include a mix of uppercase and lowercase letters, numbers, and possibly symbols. Despite their importance, passwords are often the weakest link due to user preferences for simplicity, leading to the selection of easily guessable passwords like "123456"—a trend that undermines security efforts. To combat this, IT managers enforce rules for password creation and mandate regular updates. However, the challenge remains to educate users on the importance of these practices without causing undue frustration or resistance.

However, reliance on passwords introduces inherent vulnerabilities, as exemplified by incidents where attackers have exploited weak or predictable passwords to gain

unauthorized access. A notable example involved a hacker accessing a political figure's email by resetting the password through security questions, an attack that breached privacy and violated federal laws. This incident underscores the limitations of traditional password security and the necessity for additional measures, such as multifactor authentication, to enhance user account security. It highlights the ongoing battle between maintaining user convenience and ensuring system security, emphasizing the need for continuous education on safe password practices and implementing more sophisticated security measures to protect digital identities.

Training and Awareness

Establishing a solid foundation in training and awareness is essential before delving into specific security threats like social engineering and the challenges posed by emerging technologies. Highlighting the critical role of continuous education, organizations must adapt to the ever-changing cybersecurity landscape, ensuring that employees are well-versed in current threats and the practices necessary for safeguarding against them. A well-structured security awareness program is vital, encompassing essential topics such as password management, recognition of phishing attempts, and safe Internet usage guidelines. Through such educational initiatives, employees become empowered, gaining the knowledge and skills to navigate the complexities of modern cybersecurity challenges.

To further enhance the practical application of this knowledge, simulation exercises like mock phishing campaigns serve as valuable tools for hands-on learning, enabling employees to experience firsthand the tactics used by cyber attackers. These simulations, targeted feedback, and subsequent training sessions are instrumental in cementing a culture of security awareness throughout the organization. By regularly updating these training programs to reflect new threats and best practices, organizations can ensure that their workforce remains agile and informed. Promoting a culture that prioritizes security encourages employees to actively contribute to the organization's defensive measures, recognizing that their actions play a pivotal role in upholding the overall security and integrity of the organization's digital assets.

Social Engineering Countermeasures

In addressing the nuanced threats posed by social engineering, as discussed in Section 13.2.5, the focus shifts toward deploying countermeasures and creating a heightened awareness among users. Social engineering exploits the natural tendency of individuals to trust, using deceptive interactions to manipulate users into granting unauthorized access to sensitive information or systems. Examples include attackers masquerading as new employees or external technicians, ingeniously piecing together information from various sources within an organization to breach its defenses. This underscores the importance of a vigilant and educated workforce trained to scrutinize seemingly innocuous requests for information or access. Organizations must emphasize the critical need for users to verify the identity of individuals making such requests, ensuring compliance is not based on the interaction alone but backed by a vigorous verification process.

To fortify defenses against these cunning tactics, specific practices such as requiring additional verification details during help desk interactions can be invaluable. For instance, rather than resetting a password based on a phone request alone, employees should be prompted to provide further identifying information, like their employee ID or company email address. Another social engineering technique, **pretexting**, involves acquiring personal information through false pretenses, often leading to **identity theft**. As shown in Figure 13-11, awareness and education are paramount in combating these threats. Regular training sessions can enhance users' ability to detect and respond to social engineering attempts. By embedding a culture of security that encourages critical thinking and skepticism in daily operations, organizations empower their employees to act as a crucial line of defense, minimizing the risk of social engineering breaches.

Figure 13-11 The U.S. Federal Trade Commission's website IdentityTheft.gov is the federal government's one-stop resource to help you report and recover from identity theft.
Source: U.S. Federal Trade Commission

New Technologies

New technologies represent a dual-edged sword in user security. On the one hand, these advancements offer promising avenues to bolster security measures beyond traditional protocols, introducing tools like biometric authentication and security tokens. For instance, the **security tokens** shown in Figure 13-12 are physical devices that users carry. The tokens generate a unique numeric code that must be entered alongside the standard password, adding an extra layer of authentication that enhances security considerably. This method exemplifies how innovation can fortify defenses against unauthorized access, showcasing the positive impact of technological evolution on security practices.

However, introducing new technologies invariably brings a host of new vulnerabilities and risks. Powerful search applications, capable of indexing and retrieving extensive personal data from computers, exemplify this double-edged nature. While they provide immense convenience in accessing stored information, they also allow intruders to quickly locate sensitive data, including password reminders, across a system. This potential risk highlights

Figure 13-12 Security tokens come in various forms and can provide additional security.
Source: AMJonik.pl/Shutterstock.com

the importance of segregating user accounts on multiple-user systems to safeguard individual privacy. Moreover, the rapid pace of technological advancements in cloud-based services necessitates a cautious approach, steering businesses and personal users to established vendors known for their robust security measures and expertise in cloud security. This careful navigation between embracing innovation and mitigating associated risks is crucial for maintaining the integrity and security of user data.

User Resistance

Overcoming user resistance is pivotal to integrating security policies and technologies within an organization. This challenge often stems from the perception that stringent security measures introduce inconvenience and lead to increased workload, a sentiment that can be prevalent even among senior management. Addressing this resistance is essential and benefits from building on training, awareness initiatives, and understanding the impact of emerging technologies. Organizations can navigate the implementation hurdles by cultivating an environment where the necessity and benefits of security measures are communicated and understood. It's about balancing security needs and user convenience, ensuring the user community enforces and embraces the security protocols.

Highlighting the importance of security to users plays a crucial role in changing attitudes toward security practices. Systems analysts and IT professionals can emphasize that security is not just a corporate mandate but a commitment to protecting the personal information of customers, employees, and shareholders. Such a perspective underscores the collective responsibility for safeguarding company assets and sensitive data. When individuals understand their role within the broader context of the organization's security efforts, they are more inclined to adopt stronger passwords, remain vigilant against potential security threats, and actively contribute to the security program's success. This shift in mindset is instrumental in building a culture where security is viewed as a shared goal, leading to better protection against cyber threats.

Resilience and Response to Threats

The concept of resilience and the capacity for threat response play pivotal roles in safeguarding organizational integrity. A focused approach to enhancing user resilience involves equipping individuals with the knowledge and tools to recognize, react to, and recover from security incidents. This entails in-depth training on identifying signs of potential breaches, such as unusual account activity or unauthorized access attempts, and understanding the immediate steps to take in response. Crucially, this also includes familiarizing users with the organization's incident reporting protocols and ensuring that any suspicion of a breach is promptly communicated to the relevant security personnel. An organizational culture that prioritizes swift and informed reactions to security threats empowers its users to act as the first line of defense, reducing the potential impact of cyber incidents.

Beyond the initial response, supporting users in the aftermath of security incidents is equally important for maintaining organizational resilience. This support can manifest in several ways, from providing psychological support to employees affected by a breach to ensuring they have access to the necessary resources to resume their duties with minimal disruption. Organizations should also focus on learning from security incidents and analyzing them to identify weaknesses in their current security posture and opportunities for improvement. By incorporating these insights into future training and awareness programs, organizations can continuously refine their approach to user security, creating a more resilient user base and, by extension, a more secure organizational environment. This approach to resilience and response mitigates the immediate effects of security incidents and strengthens the organization's overall security culture, making it better prepared to face future threats.

Case in Point 13.1

Pandemic Virtual University

Pandemic Virtual University plans to upgrade its digital infrastructure to support remote learning. This includes enhancing its existing network, applications, and data storage capabilities. Given the diverse user base of students, faculty, and administrative staff, how should the university implement comprehensive security levels to protect its digital assets and ensure the privacy and integrity of academic data?

13.5 Procedural and Organizational Security

Procedural and organizational security measures lay the groundwork for an all-encompassing cybersecurity strategy, emphasizing the critical role of structured policies, practices, and systems in defending against internal and external threats. Organizations ensure a disciplined approach toward protecting sensitive data and information systems by establishing clear security guidelines and adopting methodical practices. This framework integrates role-based access control systems to finely tune information access based on individual roles, supported by regular security audits to identify and address vulnerabilities. Deploying advanced **security information and event management (SIEM)** systems enhances the organization's capability for real-time threat detection and response, navigating the complexities of cybersecurity with agility.

The concept of **procedural security** extends into **operational security** practices, underlining the significance of managerial policies and controls over merely technological solutions. It nurtures a corporate culture that prioritizes security, delineating responsibilities for large-scale operations like data backups and routine activities such as email storage. This approach advocates for a forward-looking stance toward security, involving all employees in the collective effort to safeguard against attacks, including physical security measures like document shredding to prevent **dumpster diving** attacks. Implementing "need-to-know"

access principles and classifying company documents according to sensitivity levels refine the organization's control of information dissemination.

When endorsed by upper management and communicated to all organization members, procedural security is pivotal in cultivating a security-conscious corporate culture. This culture emphasizes the importance of individual responsibility in maintaining security, encompassing everything from logging out of systems to securing sensitive documents. Continuous training and periodic reminders reinforce security as a paramount concern, ensuring that procedural and organizational security measures are carefully designed and deeply ingrained in the organizational ethos. This guarantees a resilient infrastructure poised to adapt to and withstand the evolving digital threats of the future, securing a safe operational environment conducive to the organization's growth and success.

13.5.1 Developing and Implementing Security Policies

Developing and implementing security policies is a critical first step in establishing an organizational security framework. These policies provide a formal set of guidelines that dictate how data and information systems should be protected. They cover various security aspects, from password management and data encryption to acceptable use and incident response protocols. Developing these policies involves assessing the organization's risk landscape, defining security objectives, and aligning policies with regulatory requirements and industry best practices.

Once developed, implementing these security policies requires communication and training to ensure all employees understand their roles and responsibilities in safeguarding the organization's assets. This often includes regular security awareness training sessions, distribution of policy documents, and periodic reminders about security best practices. These policies mustn't be static; they should be regularly reviewed and updated in response to new threats, technological changes, and shifts in the regulatory environment.

The effectiveness of security policies hinges on enforcement. Organizations must establish mechanisms to monitor compliance and address violations. This could involve automated systems to track adherence to technical guidelines, such as password complexity requirements, and administrative processes to handle incidents of noncompliance. The goal is to create a culture of security where policies are integrated into daily operations and contribute to the organization's overall resilience.

13.5.2 Role-Based Access Control Systems

Role-based access control (RBAC) systems are pivotal in managing and restricting access to sensitive information and critical systems based on the roles of individual users within an organization. This approach minimizes the risk of unauthorized access by ensuring that employees have only the access necessary to perform their duties. Implementing RBAC involves defining roles based on job functions, assigning permissions to these roles, and then associating users with the appropriate roles.

The benefits of RBAC are manifold. It simplifies user permissions management, makes onboarding and offboarding employees more efficient, and enhances security by adhering to the principle of least privilege. RBAC facilitates compliance with regulatory requirements that mandate strict control over access to sensitive data.

For RBAC to be helpful, roles and permissions must be regularly reviewed and updated to reflect organizational changes, role evolutions, and shifts in the security landscape. This dynamic approach ensures that access controls remain relevant and that users are not granted more permissions than necessary, reducing the potential attack surface for cyber threats.

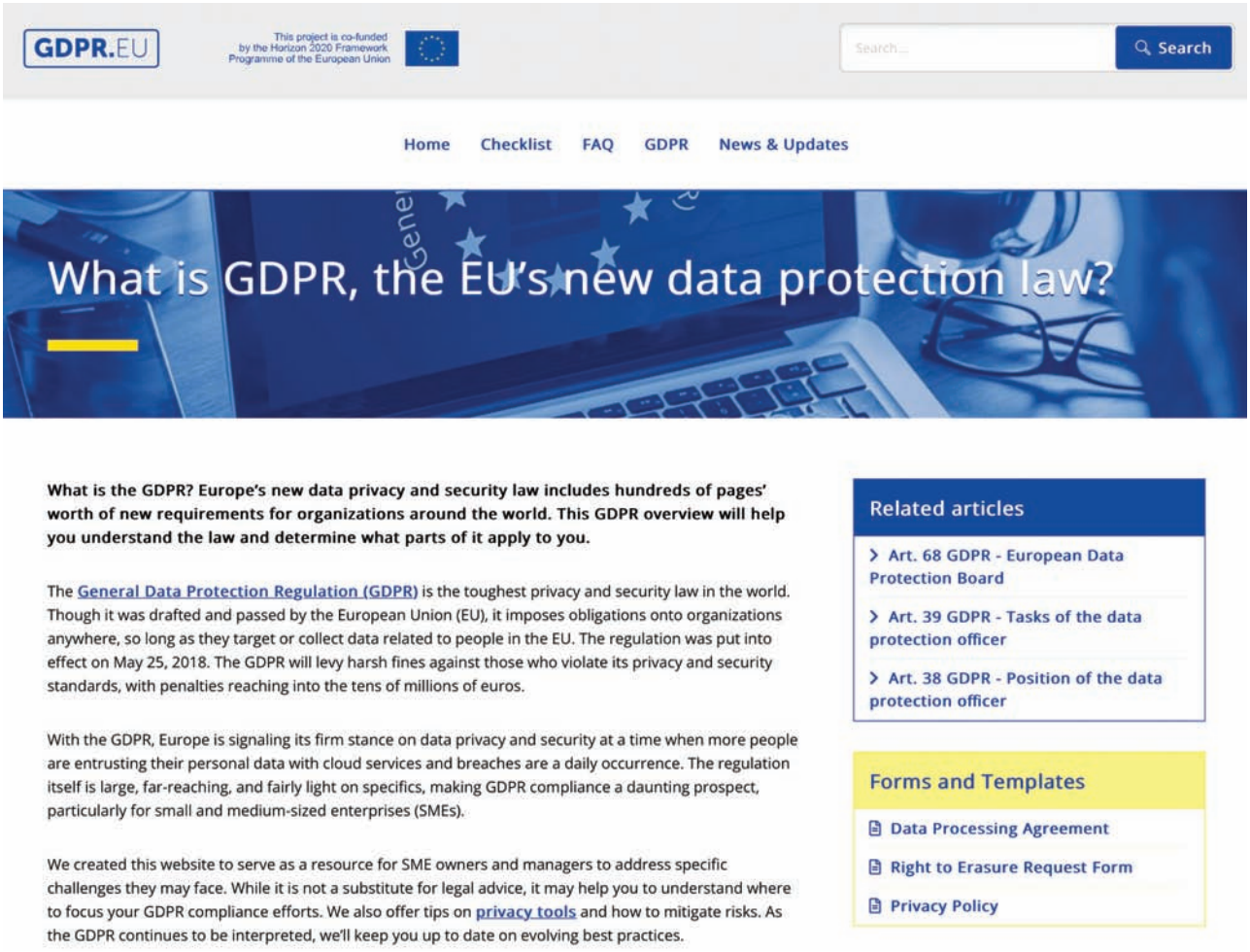
13.5.3 Security Auditing and Compliance

Security auditing and compliance involve regularly reviewing and assessing an organization's security practices and infrastructure to ensure they meet established standards and regulatory requirements. Audits can be conducted internally or by external entities and can cover many

areas, including access controls, data protection measures, and incident response capabilities. The goal is to identify gaps in the security posture and areas for improvement.

Compliance with industry standards and regulations, such as GDPR (Figure 13-13), HIPAA, or **SOC 2**, is critical to security auditing. These frameworks provide benchmarks for security practices and offer guidance on protecting sensitive data. Achieving and maintaining compliance enhances security, builds trust with customers and partners, and avoids potential legal and financial penalties.

Figure 13-13 GDPR is Europe's data privacy and security law.
Source: Proton AG



Implementing continuous auditing and compliance monitoring allows organizations to adapt to changes in the regulatory landscape and evolving threat scenarios. This ensures that security measures align with best practices and compliance requirements, bolstering the organization's defense against cyber threats.

13.5.4 Security Information and Event Management

SIEM systems enhance an organization's security posture by providing real-time analysis of security alerts generated by applications and network hardware. SIEM solutions aggregate and analyze log data from various sources, identifying patterns that may indicate a security incident. By centralizing security data collection, SIEM tools enable security analysts to detect, investigate, and respond to potential threats more efficiently.

Implementing a SIEM system involves configuring it to collect relevant data and setting up rules and analytics to identify anomalous activities that could signify an attack. This requires a deep understanding of the organization's IT environment and the potential threats it faces. A SIEM system's usefulness largely depends on the quality of its configuration and the skill of the analysts who monitor it.

Case in Point 13.2

Chain Link Consulting, Inc.

Chain Link Consulting is an IT consulting firm specializing in system security issues. The company's president has asked you to help make a presentation to potential clients at next month's trade show meeting. First, considering all security levels, the president wants you to review system security issues. Then, you should devise a list of ways Chain Link could test a client's security practices, getting a real-world vulnerability assessment.

To make matters more interesting, you have been told it is okay to be creative in your recommendations but not to propose any action that would be illegal or unethical. For example, it would be acceptable to pose as a job applicant with false references to see if they were being checked, but it would not be appropriate to pick a lock and enter the computer room.

Your report is due tomorrow. What will you suggest?

Beyond threat detection, SIEM systems also support compliance reporting and incident response efforts. They can generate reports demonstrating compliance with various security standards and assist in forensic analysis following a security incident. As such, SIEM is an indispensable tool for organizations looking to strengthen their security operations and comply with regulatory requirements.

13.5.5 Business Continuity Planning

Business continuity planning (BCP) is a strategic and operational framework that prepares organizations to maintain essential functions during and after a disruption, whether caused by a cyberattack, a natural disaster, or any other event that could impact operations. The essence of BCP lies in its considered approach to identifying essential business functions and crafting methodologies that ensure these functions can continue or be rapidly restored, regardless of the nature of the disruption. This planning process encompasses an analysis of potential threats that could impact operations, ranging from cyberattacks and technological failures to natural disasters and pandemics. A key aspect of BCP is the prioritization of business functions, which guides the allocation of resources and the development of responses to ensure minimal impact to operations.

BCP requires a collaborative effort across departments to map out critical processes and determine the necessary steps to maintain or swiftly resume business activities. This involves establishing alternative procedures, such as remote work arrangements or utilizing secondary locations, to maintain business continuity. Communication plays a pivotal role in BCP, necessitating clear channels and protocols for informing employees, customers, and stakeholders about the status of operations and recovery efforts. Regular training and exercises are essential to BCP, ensuring all personnel are prepared to execute the plan under real-world conditions.

BCP focuses on the immediate response to disruptions and the longer-term recovery and resilience-building policies. This includes integrating lessons learned from exercises and actual incidents into the planning process and continuously updating and refining the plan to address

new threats and vulnerabilities. By embedding business continuity into the organizational culture, companies can move toward operational resilience, ensuring they are well-equipped to handle disruptions and minimize their impact on critical business functions and overall organizational health.

13.6 Backup and Recovery Strategies

Building on the fundamental principles of data protection outlined earlier in the book, **backup** and recovery strategies are essential for maintaining organizational resilience, providing a safety net against data loss, and ensuring that operations can quickly resume following any disruption. Establishing rigorous **backup policies** is the first step, focusing on the critical aspects of data preservation, such as the frequency of backups, the types of data to be secured, and the methodologies for ensuring data integrity and availability. These policies form the backbone of a data protection plan, ensuring that valuable digital assets are systematically safeguarded.

The selection of **backup media** formats and storage solutions is critical for implementing the established backup policies. It highlights the need to choose the right combination of technologies—from on-site physical storage to off-site cloud-based services—to match the organization's specific requirements for data recovery, scalability, and security. Special attention is given to enhancing resilience against emerging threats, particularly ransomware attacks, underscoring the importance of protective data measures.

When restoring operations and data after an incident, the emphasis shifts to practical recovery techniques and disaster recovery planning. This involves the technical recovery of IT systems and aligning these efforts with the organization's overall business continuity plans. It's about creating a responsive and flexible approach that enables businesses to minimize downtime and operational impact during unforeseen events. Regular testing and refinement of these plans ensure that when disruptions occur, the path to recovery is clear, minimizing the effect on business operations and maintaining the trust of stakeholders.

13.6.1 Designing Effective Backup Policies

Designing effective backup policies is pivotal in establishing solid backup and recovery strategies within an organization. These policies delineate data handling, storage, and protection protocols, encompassing data categorization based on its importance and sensitivity. This aids in allocating resources efficiently to safeguard critical data assets while ensuring adaptability to counter emerging threats and incorporate new technological advancements. Moreover, understanding and adhering to regulatory compliance and industry-specific mandates are integral to these policies, necessitating their periodic review and updates in response to changing legal landscapes and organizational risk assessments.

Backup methodologies vary, including full, differential, incremental, and continuous backups, each with advantages and application scenarios. **Full backups** offer a complete snapshot of system data, though they can be resource-intensive. **Differential backups** provide a more efficient alternative by only capturing changes since the last full backup, facilitating quicker restoration processes. **Incremental backups**, while being the most expedient, necessitate a multistep restoration process. On the other hand, **continuous backups**, employing real-time data streaming, ensure immediate data recovery by leveraging **RAID (redundant array of independent disks)** systems for enhanced reliability and performance. RAID systems are called **fault tolerant** because a failure of any one disk does not disable the system. A RAID design offers better performance, greater capacity, and more reliability than one large drive. When installed on a server, a RAID array of multiple drives appears to the computer as a single logical drive. Table 13-5 shows a comparison of various backup methods. This diversity in backup tactics necessitates a tailored approach, balancing data protection needs with operational efficiency and technological capabilities.

Table 13-5 Comparison of full, differential, incremental, and continuous backup methods.

Comparison of Backup Methods			
Backup Type	**Characteristics**	**Pros and Cons**	**Typical Frequency**
Full	Backs up all files	Slowest backup time and requires the most storage space. Rapid recovery because all files are restored in a single step.	Monthly or weekly
Differential	Only backs up files that are new or changed since the last full backup	Faster than a full backup and requires less storage space. All data can be restored in just two steps by using the last full backup and the last differential backup.	Weekly or daily
Incremental	Only backs up files that are new or changed since the last backup of any kind	Fastest backup and requires the least storage space because it only saves files that have never been backed up. However, requires many restore steps.	Daily or more often
Continuous	Real-time, streaming method that records all system activity	Very expensive hardware, software, and network capacity. Recovery is very fast because system can be restored to just before an interruption.	Typically only used by large firms and network-based systems

The retention and security of backup data are also paramount, with policies specifying the duration for which backups are stored and outlining measures for their protection and eventual disposal or reuse. Adherence to defined **retention periods** is guided by legal obligations and organizational policies, underscoring the importance of secure storage practices and regular inventory checks to ensure data integrity. This approach to backup and recovery, spanning policy formulation to the practical aspects of data storage and security, underscores the critical role of well-conceived backup strategies in maintaining organizational resilience and data availability.

13.6.2 Backup Media Formats and Storage Solutions

The selection of backup media formats and storage solutions is critical in shaping the efficacy and resilience of an organization's backup strategy. From traditional tape systems, known for their durability and cost-effectiveness for long-term storage, to modern cloud-based solutions that offer scalability and remote accessibility, the choice of backup medium must align with the organization's recovery objectives and operational needs. Disk-based backups, for instance, provide rapid data access and restoration capabilities, making them suitable for environments where quick recovery is paramount.

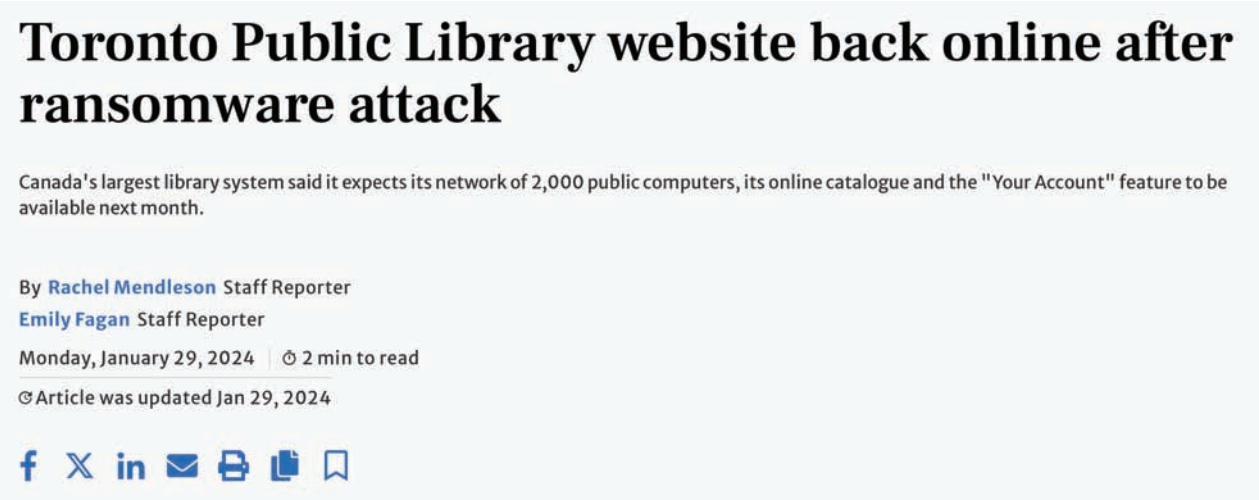
Integrating diverse storage solutions, including onsite and offsite options, is pivotal in safeguarding data against various threats, including natural disasters and cyberattacks. Offsite or cloud storage can offer added layers of protection by geographically separating backup data from the primary data center, reducing the risk of simultaneous compromise. However, this approach necessitates careful consideration of data security, encryption, and the potential implications of data sovereignty laws, mainly when using cloud services that may store data across international borders.

Leveraging a mix of backup media and storage solutions requires a nuanced understanding of their benefits and limitations. Organizations must consider factors such as the criticality of data, required speed of recovery, regulatory compliance, and cost constraints when designing their backup infrastructure. Regular testing and validation of backup solutions are also essential to ensure that data can be recovered, allowing organizations to refine their strategies based on performance and changing requirements.

13.6.3 Enhancing Resilience Against Ransomware Attacks

In the face of escalating ransomware threats, such as the attack on the Toronto Public Library described in Figure 13-14, enhancing the resilience of backup and recovery systems has become a critical concern for organizations worldwide. Ransomware attacks, which encrypt an organization's data and demand payment for its release, underscore the importance of maintaining secure and inaccessible backup copies of critical data. To combat this, organizations are adopting techniques such as air-gapped backups, which are physically or logically isolated from the network, making them inaccessible to attackers.

Figure 13-14 Three months after a debilitating ransomware attack hit the Toronto Public Library, its website was up and running again.
Source: Toronto Star



Toronto Public Library website back online after ransomware attack

Canada's largest library system said it expects its network of 2,000 public computers, its online catalogue and the "Your Account" feature to be available next month.

By Rachel Mendleson Staff Reporter
Emily Fagan Staff Reporter
Monday, January 29, 2024 | 2 min to read
Article was updated Jan 29, 2024

Implementing immutable backups, which cannot be altered or deleted for a set period, provides an additional defense against ransomware. These backups ensure that pristine copies of data remain available for recovery even in the event of an attack. Regularly testing backup and recovery processes enables organizations to verify the integrity and efficacy of their ransomware defense methods, ensuring they can swiftly restore operations without acceding to ransom demands.

Educating employees about the risks of ransomware and the common vectors for its delivery, such as phishing emails, is also a vital component of a thorough defense strategy. Organizations can reduce the likelihood of ransomware infiltration through a culture of awareness and vigilance. Coupled with proper technical defenses, this approach to ransomware mitigation is essential for maintaining the integrity and availability of critical data in an increasingly hostile cyber landscape.

13.6.4 Disaster Recovery Planning

Disaster recovery planning (DRP) focuses on restoring IT systems, data access, and infrastructure following a catastrophic event. It is a specialized component of organizational resilience focusing on technical recovery, distinguishing itself from broader business continuity planning

by emphasizing IT assets. DRP outlines the steps and technologies needed to recover lost data and restore IT operations, which is crucial for resuming business activities after a disruption. This involves detailed planning around data backup strategies, disaster recovery sites, and system failover processes to maintain data integrity and availability.

Implementing DRP requires a thorough inventory and valuation of IT assets, alongside assessing potential risks to these assets. This assessment informs the development of a recovery strategy that specifies backup methods, data storage solutions, and recovery time objectives that align with the organization's requirements. The plan must also detail the procedures for data restoration and system recovery, including the roles and responsibilities of the recovery team. Regular testing of DRP is imperative to validate the recovery procedures and identify areas for improvement.

Integration with the broader BCP ensures that DRP is not siloed but operates in concert with overall organizational resilience efforts. This alignment guarantees that IT recovery processes support the continuation of essential business functions, mitigating the impact of disruptions. Continuous refinement of DRP, informed by testing outcomes and evolving IT landscapes, enhances an organization's agility and capacity to respond to and recover from disasters. By prioritizing the recovery of critical IT systems and data, DRP plays a vital role in minimizing downtime, preserving business continuity, and safeguarding the organization's reputation in the aftermath of a disaster.

13.6.5 Restarting Operations from Backups

Restarting operations from backups is critical for any organization aiming to minimize downtime and maintain business continuity after data loss incidents. This process involves more than just data restoration; it requires a coordinated effort to ensure that systems are brought back online in a secure, orderly, and efficient way. Prioritization plays a key role here, with critical systems and data being restored first to resume essential operations, followed by less critical resources dictated by the organization's business continuity plans.

The challenge of restarting operations also includes ensuring that restored data and systems have not been compromised. This necessitates thorough scanning and validation to confirm the integrity of the data and the absence of malware or other threats that could lead to further disruptions. The restart process must also comply with any regulatory requirements governing data integrity and security, adding another layer of complexity to the recovery effort.

Training and rehearsing the restart process are indispensable for a smooth recovery. This approach enables IT teams to familiarize themselves with procedures and reduce the likelihood of errors. Simulated disaster exercises and drills can help identify potential bottlenecks or issues in the recovery plan, allowing for refinements and improvements. This preparedness is crucial for reducing recovery times, mitigating the impact of disruptions on operations, and ensuring a swift return to normalcy.

13.7 Legal and Ethical Considerations in System Security

The intertwining of legal compliance and ethical practices is essential for organizations and security professionals. The constant evolution of cyber threats and the growing value of digital data underscores the critical need for understanding and adhering to established legal frameworks. At the same time, maintaining high ethical standards is paramount. These dual pillars support the cultivation of trust and ensure technology is used responsibly and conscientiously. Adhering to legal regulations helps safeguard sensitive information and privacy, while ethical guidelines encourage a culture of integrity and respect toward users' rights. This dual focus forms the bedrock of a complete system security plan, emphasizing protection against digital threats alongside a commitment to individual rights and ethical norms.

The legal landscape dictates that organizations implement measures to comply with laws and regulations focused on data protection, privacy, and cybersecurity. This compliance

is about adhering to legal mandates, protecting the organization from potential penalties, and safeguarding its reputation. On the ethical front, the emphasis shifts to the moral responsibilities that govern the actions and decisions of individuals and corporations in the digital realm. Ethical considerations in cybersecurity involve ensuring fairness, privacy, and respect for intellectual property, guiding professionals to act with integrity even beyond the reach of legal requirements.

Legal compliance and ethical behavior in system security are the cornerstones of trust and reliability in the digital age. This ensures that while organizations strive to protect their assets from cyber threats, they also respect and uphold the rights and dignity of all stakeholders. By balancing these critical aspects, businesses can maintain a secure, respectful, and trustworthy digital environment crucial for thriving in today's technology-driven world.

13.7.1 Cyberethics

Cyberethics encompasses the study and evaluation of ethical issues and standards related to the use of IT. It involves analyzing the impact of digital technologies on society, individuals, and organizations and establishing principles for responsible behavior in the digital realm. Cyberethics touches on privacy, intellectual property, and access rights, providing a framework for ethical decision making in the face of emerging technological challenges.

The importance of cyberethics in system security lies in its role in guiding the development and implementation of technologies and practices that respect user rights and data integrity. For instance, ethical considerations influence how organizations collect, store, and process personal data, ensuring that such activities are conducted transparently and with respect for user privacy.

As digital technologies become more embedded in daily life, the role of cyberethics in shaping policies, practices, and user awareness continues to grow. Organizations must prioritize the ethical implications of their technological choices, promoting a secure and respectful digital environment for all stakeholders.

13.7.2 Compliance with Data Protection Laws

Compliance with data protection laws is critical to system security, requiring organizations to adhere to legal standards for collecting, using, and safeguarding personal information. These laws, such as the GDPR in the European Union and the **California Consumer Privacy Act (CCPA)** illustrated in Figure 13-15, aim to protect individuals' privacy rights and ensure that organizations handle personal data responsibly.

Navigating the requirements of these laws involves implementing detailed data protection strategies, including data minimization, consent management, and establishing clear processes for responding to data breaches. Organizations must also provide transparency for data processing activities and offer individuals control over their personal information.

The consequences of noncompliance can be substantial, including hefty fines, reputational damage, and loss of consumer trust. Therefore, organizations must continuously monitor and update their data protection practices to align with evolving legal standards, ensuring they remain compliant and maintain the trust of their users and stakeholders.

Case in Point 13.3

Continental Coders, LLC

Continental Coders is a software development company expanding into the European market. The company must comply with the GDPR for its products and services as part of this expansion. How should the company integrate legal and ethical considerations into its system security strategy to comply with GDPR and protect user data?

Figure 13-15 The CCPA protects individuals' privacy rights and ensures that organizations handle personal data responsibly.
Source: State of California Department of Justice

13.7.3 Ethical Hacking and Penetration Testing

Ethical hacking and penetration testing are proactive security practices in which skilled professionals simulate cyberattacks on a system to identify vulnerabilities before malicious hackers can exploit them. Unlike malicious hacking, ethical hacking is conducted with permission from the organization and aims to strengthen security by uncovering and addressing weaknesses. Numerous resources are available to learn about ethical hacking, such as the courses offered through LinkedIn Learning (Figure 13-16).

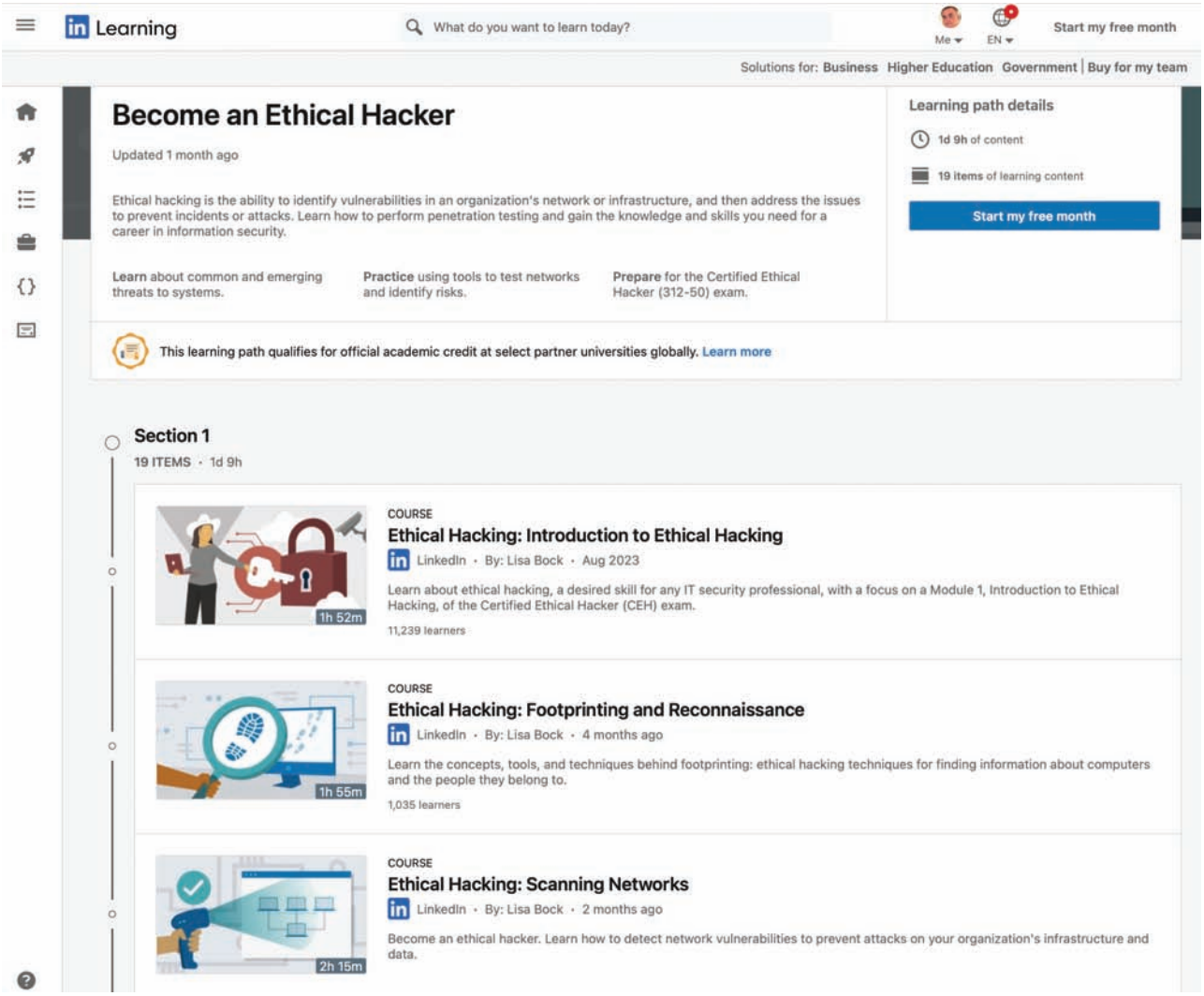
The ethical dimension of these practices is underscored by the adherence to a code of conduct that respects privacy, confidentiality, and the integrity of systems and data. Ethical hackers must obtain explicit authorization, define the scope of their activities, and ensure their actions do not harm the system or its users.

The value of ethical hacking and penetration testing lies in their ability to provide real-world insights into an organization's security posture. These practices allow for mitigating vulnerabilities in a controlled and ethical manner. These practices are essential components of a complete security scheme, contributing to the resilience of systems against cyber threats.

13.7.4 Privacy Issues and Solutions

Privacy issues in system security revolve around protecting individuals' personal information in the digital age. Widespread data collection and analysis and the increasing sophistication of cyber threats pose profound privacy risks. Addressing these issues requires a multifaceted approach, including implementing strong data encryption, access controls, and privacy-by-design principles in developing new technologies and systems.

Figure 13-16 LinkedIn Learning offers courses on ethical hacking.
Source: LinkedIn Corporation



Solutions to privacy issues also involve regulatory frameworks that set standards for data protection and give individuals rights over their personal information. Technologies such as **anonymization** and **pseudonymization** can further enhance privacy by ensuring that data cannot be easily linked to specific individuals.

Organizations play a crucial role in addressing privacy issues by adopting transparent data practices, educating users about their privacy rights, and providing tools to control their personal information. By prioritizing privacy, organizations can build trust with users and comply with legal and ethical standards in system security.

13.7.5 Intellectual Property Rights and Security

Intellectual property rights (IPR) and security intersect in the digital domain, where protecting creative works and innovations from unauthorized use or theft is a significant concern. System security measures are critical in safeguarding intellectual property, including proprietary information, software, and digital media, from cyber threats such as piracy, infringement, and espionage.

Protecting IPR requires a combination of legal, technical, and organizational measures. Technological solutions such as **digital rights management (DRM)** systems, encryption, and access

controls can prevent unauthorized access and distribution of protected works. At the same time, legal mechanisms offer recourse for rights holders in cases of infringement.

Beyond technical and legal measures, demonstrating an organizational culture that values and respects intellectual property is essential. Education and awareness efforts can help employees understand the importance of IPR and the role of security practices in protecting these assets. Organizations can defend their innovations and creative works by integrating IPR considerations into system security, ensuring continued growth and competitiveness in the digital marketplace.

13.8 Emerging Technologies and Security

The incorporation of cutting-edge technologies into the professional sphere and everyday life is altering the domain of cybersecurity. As these advancements progress, they present novel security challenges and avenues for enhancing protective measures. Technologies such as cloud computing, IoT, blockchain, and artificial intelligence (AI) are at the heart of this shift, each contributing to the introduction of new security vulnerabilities while providing innovative methods for strengthening system security. For organizations focused on protecting their digital assets, comprehending the effects of these technologies on system security is imperative, enabling them to leverage technological benefits while safeguarding against potential threats.

Organizations must adopt a forceful stance toward security to navigate the evolving cybersecurity landscape. This involves early identification and mitigation of risks associated with emerging technologies before they become exploitable by adversaries. This approach is essential, as the swift pace of technological development demands that cybersecurity professionals engage in ongoing education and adapt their techniques to address new challenges. Staying informed about the latest technological trends and understanding their potential security implications are crucial in developing resilient defenses against cyber threats.

As emerging technologies continue to advance, they will inevitably shape the future of cybersecurity. Organizations and security experts must remain vigilant, anticipating the security needs that these technologies may require and adapting their practices accordingly. This dynamic process of learning, adaptation, and innovation is key to mitigating the risks posed by new technologies and harnessing their potential to enhance system security. Through such continuous efforts, businesses can achieve a balanced approach to leveraging technological advancements while maintaining security measures to protect their critical information assets.

13.8.1 The Impact of Cloud Computing on System Security

Cloud computing has fundamentally changed how businesses store, process, and manage data, offering scalability, flexibility, and cost efficiency. However, shifting from on-premises to cloud-based infrastructure introduces new security considerations. Shared-responsibility models in cloud computing delineate the security obligations of cloud service providers and their clients, but misunderstandings about these responsibilities can leave security gaps. Ensuring data security in the cloud involves implementing access controls, encryption methods, and secure data transfer protocols.

Cloud service providers typically offer a range of built-in security features designed to protect their infrastructure and services. However, the users are ultimately responsible for securing cloud-hosted data and applications. This includes configuring privacy settings, managing user access, and monitoring for suspicious activities. Adopting a far-reaching cloud security strategy, such as using **cloud access security brokers (CASBs)**, as shown in Figure 13-17, can enhance visibility and control over cloud services.

Regular security assessments and adherence to best practices are essential for maintaining security in cloud environments. Organizations must stay informed about the latest cloud security trends and threats and be prepared to adapt their security measures rapidly in response to evolving risks. Collaboration with cloud providers to understand and leverage their security offerings can also bolster an organization's cloud security posture.

Figure 13-17 According to Gartner, CASBs are on-premises or cloud-based security policy enforcement points.
Source: Gartner, Inc.

13.8.2 Securing IoT Devices and Networks

The proliferation of IoT devices introduces a multitude of endpoints into networks, expanding the attack surface for cyber threats. IoT devices, ranging from consumer gadgets to industrial sensors, often lack strong security features, making them vulnerable to attacks. Ensuring the security of IoT devices and networks requires a multipronged approach, including secure device design, regular software updates, and the implementation of strong authentication and encryption protocols.

The heterogeneity and sheer volume of IoT devices pose unique challenges for security management. Traditional security solutions may not directly apply to the IoT ecosystem, necessitating the development of specialized tools and practices. Network segmentation can help isolate IoT devices from critical network resources, reducing the risk of attackers' lateral movement within networks.

Manufacturers play a critical role in the security of IoT devices by integrating security features at the design stage and providing regular firmware updates to address vulnerabilities. For end users, adopting a security-first mindset when deploying IoT devices, including changing default passwords and disabling unnecessary features, is crucial for mitigating potential risks.

13.8.3 Blockchain for Enhancing Security

Blockchain technology is renowned for providing secure, transparent, and tamper-proof record-keeping mechanisms. Its decentralized nature offers a novel approach to enhancing security, particularly in applications requiring secure transactions and data integrity. By leveraging cryptographic principles, blockchain can secure data exchanges, authenticate transactions, and prevent unauthorized alterations to data records.

The adoption of blockchain in supply chain management, identity verification, and secure voting systems illustrates its potential to revolutionize security practices. However, implementing blockchain-based solutions requires careful consideration of their scalability, efficiency, and integration with existing systems.

Despite its inherent security features, blockchain technology is not immune to risks. Smart contract vulnerabilities, for example, can be exploited if the code is not correctly audited. Therefore, organizations considering blockchain technology must adopt a balanced view, recognizing its security benefits and limitations, and remain vigilant against emerging threats.

13.8.4 Artificial Intelligence in Cybersecurity

AI is increasingly being leveraged in cybersecurity to automate threat detection and response. AI's ability to analyze vast amounts of data in real time allows for identifying complex patterns and anomalies that may indicate cyber threats. Machine learning algorithms, a subset of AI, can be trained on historical security data to predict and respond to potential attacks more efficiently than traditional methods.

The use of AI in cybersecurity enhances the speed and accuracy of threat detection, enabling organizations to respond to attacks more rapidly. However, reliance on AI also introduces new challenges, such as the risk of false positives and the need for continuous algorithm training to adapt to evolving threats.

Cyber attackers also utilize AI to develop more sophisticated attack methods, leading to an arms race between attackers and defenders. As such, cybersecurity professionals must stay abreast of advances in AI technology and consider its potential benefits and vulnerabilities in their security strategies.

13.8.5 Future Technologies and Potential Security Implications

As emerging technologies advance, they will undoubtedly introduce new security implications. Innovations such as quantum computing, augmented reality, and virtual reality promise to transform various sectors but pose potential risks that must be anticipated and addressed. For example, the power of quantum computing could render current encryption methods obsolete, challenging the central tenets of digital security.

Staying informed about these technological advancements and their potential impact on security is essential for organizations aiming to protect their assets in the future. This requires a forward-looking approach to cybersecurity, where potential threats are analyzed and mitigated before widespread technology adoption.

Collaboration among industry, academia, and government will be vital in developing security measures for future technologies. Society can harness emerging technologies through innovation and security consciousness while minimizing risks.

13.9 Building a Resilient Cybersecurity Culture

Cybersecurity has emerged as a pillar of organizational resilience and operational integrity. The acceleration of technological adoption—from AI and blockchain to IoT—has expanded the horizon of opportunities and introduced complex cybersecurity challenges. Against this backdrop, building a resilient cybersecurity culture transcends the realm of IT departments, demanding engagement across all levels of an organization. This culture is not static; it requires continuous nurturing, adaptation, and alignment with the rapid pace of technological innovation and the shifting landscape of cyber threats. The essence of a resilient cybersecurity culture is its ability to embed security consciousness in every employee, decision, and process, making cybersecurity an inseparable aspect of the organization.

A healthy cybersecurity culture is the cornerstone for safeguarding sensitive data and assets, ensuring business continuity, and maintaining customer trust. It encompasses more

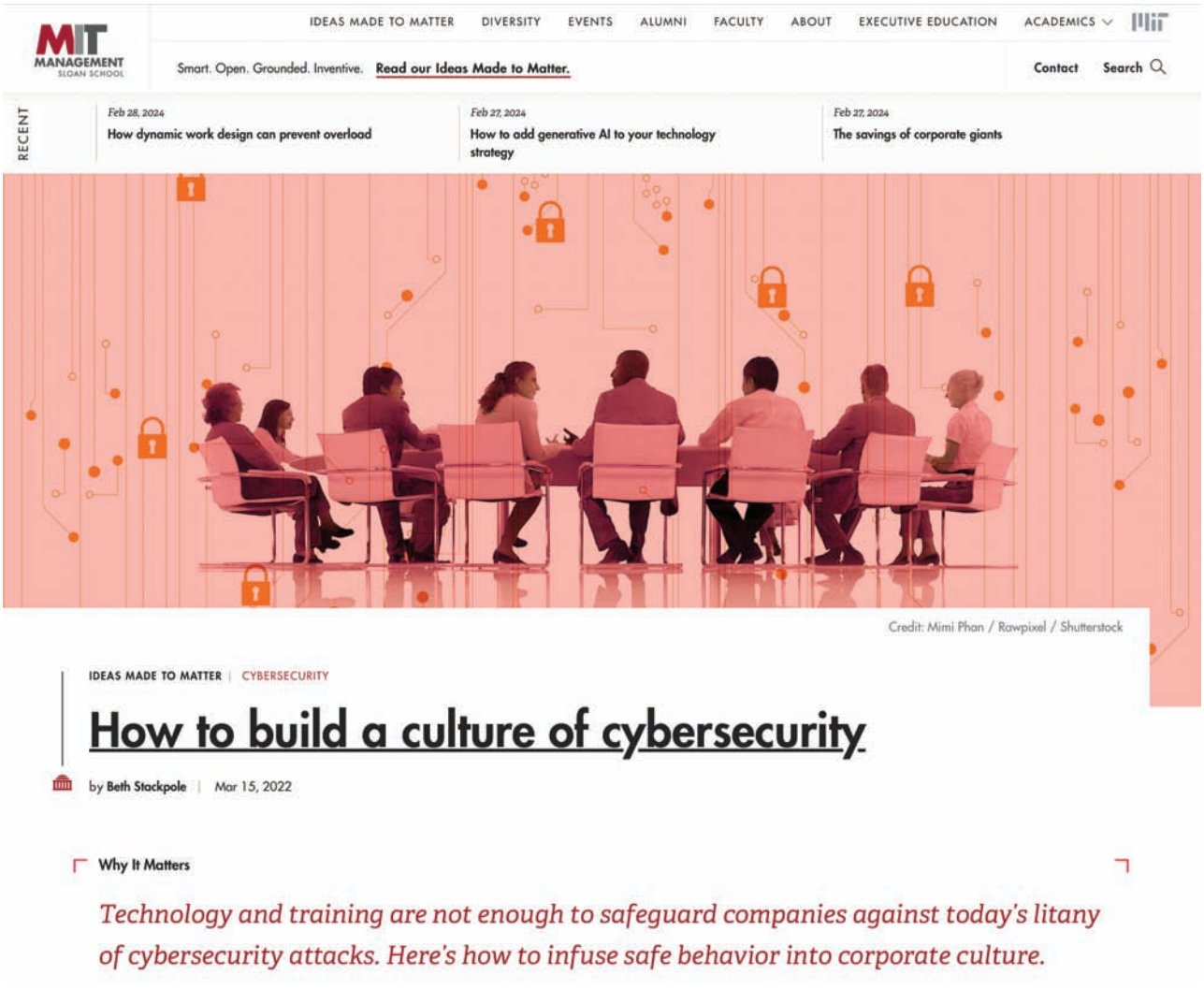
than just policies and procedures; it creates a shared sense of responsibility and vigilance among all stakeholders. This cultural shift toward prioritizing security can mitigate risks, deter threat actors, and enhance an organization's response to incidents. However, achieving such a culture has its challenges. It requires a clear vision, steadfast leadership, ongoing education, and a commitment to integrating security into the very DNA of the organization.

The following sections delve into the critical components of building and sustaining a resilient cybersecurity culture. From understanding the role of organizational culture in cybersecurity to integrating emerging technologies into security strategies, these insights offer a blueprint for a security-first mindset. Through leadership, governance, continuous innovation, and employee engagement, organizations can fortify their defenses and confidently navigate the complexities of the digital age.

13.9.1 The Role of Organizational Culture in Cybersecurity

Organizational culture is critical to any company's security posture. Figure 13-18 illustrates how a company embeds cybersecurity into its core values and everyday practices that directly impact its resilience against digital threats. The success of security policies and practices hinges on this cultural alignment, as it influences employee engagement and adherence to

Figure 13-18 According to MIT's Sloan School of Management, the weak link in cybersecurity "is typically people and behavior—a problem that is only resolved through a combination of technology investment and culture change."
Source: MIT Sloan School of Management



these measures. Examining examples of successful cybersecurity cultures can provide valuable insights, showcasing how a well-integrated cybersecurity ethos enhances security and supports business objectives via an environment of trust and vigilance.

Organizational Culture and Cybersecurity

Integrating emerging technologies into organizational workflows demands a paradigm shift toward prioritizing cybersecurity. As technology evolves at an unprecedented rate, it becomes imperative for organizations to adapt their cultures accordingly to mitigate emerging security threats. This adaptation involves a deep comprehension of what embodies a cybersecurity culture. In this environment, the collective mindset, behaviors, and practices are geared toward protecting digital resources and data.

Such a culture is defined by a unified approach to cybersecurity, where the importance of safeguarding digital assets is universally acknowledged and embedded into the organization's fabric. It is a culture where security principles are not just policy-driven but are part of the everyday ethos, influencing decisions and actions across all levels. This holistic embrace of cybersecurity practices strengthens an organization's defense mechanisms, making it resilient against the constantly evolving landscape of digital threats.

Impact on Security Policies and Practices

Embedding a cybersecurity culture within an organization cultivates a workforce that is acutely aware of and responsive to security policies and practices. This culture shift ensures that cybersecurity transcends being merely a set of guidelines to become an integral part of the organizational ethos. Such a transformation leads to heightened vigilance among employees, who safeguard the organization's digital assets to reduce the risk of security breaches.

In this environment, employees are followers of security policies and active participants in the organization's cybersecurity framework. They are more likely to recognize potential security threats and are better prepared to employ best practices for data protection. This collective commitment to cybersecurity creates a safer digital workspace, where security principles are embedded in every action and decision, minimizing vulnerabilities and enhancing the organization's resilience against cyber threats.

The Value of Cybersecurity Culture Examples

Real-world examples underscore the benefits of integrating cybersecurity into organizational culture, leading to fewer security incidents, reduced risk, and improved regulatory compliance. These examples can be found in numerous sources, including academic journals, industry reports, government publications, technology news outlets, conferences, and books. Professional social networks like LinkedIn are excellent online platforms for sharing experiences and lessons learned.

13.9.2 Developing a Security-First Mindset

Cultivating a security-first mindset is pivotal in fortifying an organization's defenses against cyber threats. This begins with awareness and education initiatives illuminating the intricacies of cybersecurity threats and each employee's critical role in safeguarding the organization. Employing behavioral change techniques further reinforces this mindset, leveraging psychological insights to encourage secure habits and practices among staff. Equally important is the commitment to continuous learning, ensuring that as new threats emerge and technologies advance, the organization's collective cybersecurity knowledge remains current in addressing potential security challenges.

Awareness and Education

Raising employees' awareness of the risks and benefits associated with emerging technologies plays a crucial role in cultivating a security-first mindset within an organization. By implementing awareness programs, companies can enlighten their workforce about the perils and how judicious use of these technologies can bolster personal and organizational security

frameworks. Such initiatives are instrumental in creating a culture of curiosity and openness, motivating employees to embrace and securely navigate the technological landscape.

This educational approach encourages staff to use new digital tools and solutions, furthering a workplace environment that values continuous learning and adaptation. By understanding the implications of their digital actions, employees are more likely to adopt secure practices and contribute to the overall resilience of their organization's cybersecurity defenses. This emphasis on education and adaptation is critical to developing a security-first mindset across all organizational levels.

Behavioral Change Techniques

Leveraging psychological strategies, including **gamification** and **incentivization**, inspires employees to adopt safer cybersecurity behaviors. Implementing these techniques transforms learning into an engaging and memorable experience, enhancing the probability of favorable behavioral modification. Such interactive and reward-based learning methods educate and entertain, making adopting cybersecurity best practices more appealing to employees.

By acknowledging and rewarding secure behavior, companies underscore the critical role of cybersecurity within their operational framework. This recognition is a powerful motivator, encouraging employees to take a more active stance in preventing security threats. Through these innovative approaches, organizations can embrace a culture where security measures are valued and practiced, strengthening their overall cybersecurity posture.

Continuous Learning

The threat landscape and technology ecosystem constantly evolve, making ongoing education and training essential. Such continuous learning initiatives inform the workforce about cybersecurity threats and the necessary measures to mitigate them. This strengthens the organization's security defenses and encourages employees to assume personal responsibility for maintaining cybersecurity standards.

Regular training and educational updates equip employees with the knowledge and skills required to navigate the complexities of modern cybersecurity challenges. By staying informed on emerging threats and evolving security practices, employees become more competent and confident as active participants in the organization's cybersecurity efforts. This investment in continuous learning promotes a culture of vigilance and preparedness, substantially enhancing the organization's overall security posture.

13.9.3 Leadership and Governance in Promoting Cybersecurity

The pivotal role of leadership and governance in an effective cybersecurity environment cannot be overstated. It hinges on the involvement of leaders who champion cybersecurity initiatives and integrate these efforts into corporate governance. This leadership is instrumental in developing and enforcing security policies aligning with the organization's strategic goals and establishing a security-conscious culture. Moreover, measuring the impact of these initiatives on the organizational culture is critical, providing valuable feedback that informs continuous improvement and adaptation in the face of evolving cyber threats. This synergy between leadership, policy, and culture change is essential for creating a resilient cybersecurity posture.

Role of Leadership

Leadership is crucial in advocating for an organization's cybersecurity culture. Executives and senior management must actively support cybersecurity initiatives, ensuring they are woven into corporate governance and operational strategies. Through their actions and leadership, these leaders set a precedent, demonstrating the significance of cybersecurity and encouraging a collective effort to protect the organization's assets and information.

By exemplifying a solid commitment to cybersecurity principles and practices, leaders can influence the organizational culture, instilling a sense of responsibility and shared commitment

among all employees. This approach enhances the organization's security posture and builds a resilient and aware workforce capable of responding to cybersecurity challenges.

Policy Development and Enforcement

Creating clear and complete security policies is essential to aligning an organization's cybersecurity policies with its business goals and regulatory obligations. Such policies must be enforceable, with unwavering support from management across all levels. Leadership must guarantee that these policies are communicated clearly and fully understood by every employee. This ensures that the policies are not merely formalities but are actively integrated into the organization's daily operations, forming a governance structure that underpins a strong and resilient cybersecurity stance.

Leadership's role extends beyond policy creation to encouraging an environment where the policies are lived values within the organization. Leaders can cultivate a culture of security awareness and compliance by emphasizing the importance of adhering to these policies and demonstrating their relevance to the organization's success. This approach mitigates risks and strengthens the organization's ability to respond to and recover from cybersecurity incidents, ensuring an adaptable cybersecurity posture.

Measuring Culture Change

Organizations must employ specific metrics and indicators to evaluate leadership's influence on cybersecurity culture. By conducting surveys, soliciting feedback, and monitoring security incidents, leaders can gain insights into the culture's current state and evolution over time. These measurements are pivotal in pinpointing areas that need enhancement and assessing the impact of cybersecurity awareness and training initiatives. Such data-driven insights allow leaders to identify strengths and weaknesses within their cybersecurity culture, guiding the development of targeted methods to bolster security awareness across the organization.

These metrics enable leaders to make informed decisions about future cybersecurity initiatives. By understanding the effectiveness of existing programs and the organization's responsiveness to cybersecurity education, leaders can tailor their methods better to meet the needs of their employees and the organization. This continuous evaluation and adaptation ensure that cybersecurity culture remains dynamic and responsive to evolving threats, reinforcing the organization's defensive posture and commitment to maintaining a secure and resilient digital environment.

13.9.4 Integrating Emerging Technologies into Security Strategies

Integrating emerging technologies into security strategies is a pivotal aspect of modern cybersecurity efforts, requiring a careful balance between innovation and risk management. A thorough assessment of new technologies lays the groundwork, ensuring that potential security implications are fully understood and addressed. Adopting a "security by design" approach is crucial, embedding security features early in the development and adoption phases to mitigate risks. Additionally, learning from industry examples offers valuable insights into the practical aspects of technology integration, demonstrating successful strategies and highlighting common pitfalls.

Assessing New Technologies

Conducting a thorough security assessment before adopting new technologies is imperative for organizations aiming to maintain a robust security posture. This evaluation process involves analyzing the risks and vulnerabilities that could be introduced with the latest technology, alongside its potential effects on current security frameworks. Such thorough scrutiny enables organizations to gauge the compatibility of new technologies with their existing security measures, ensuring a harmonious integration that bolsters their defense mechanisms.

By thoroughly assessing the implications of new technologies, organizations can make informed decisions on integrating them into operations. This safeguards against

unforeseen security challenges and ensures that technological advancements contribute to the organization's security. It underscores the importance of aligning technological innovation with security objectives, facilitating a secure and seamless transition to new technology.

Innovation and Security by Design

Incorporating security considerations at the initial stages of adopting and developing new technologies is essential for creating a proper security framework. By integrating security from the initial phases, organizations ensure that protective measures are an intrinsic part of the technology, not merely tacked on later. This "security by design" methodology drives innovation and prioritizes protecting information and assets, establishing a solid basis for security that evolves with the technology.

Adhering to this approach means that security is a fundamental aspect of technological innovation, ensuring that new developments are resilient against threats from their inception. It allows for the seamless incorporation of security into new technologies, aligning innovation efforts with stringent security standards. This enhances an organization's security posture and reinforces the importance of security in the rapidly advancing technological landscape.

The Importance of Learning from Industry Examples

Exploring case studies of organizations that have adeptly incorporated emerging technologies into their security frameworks offers critical insights for others aiming to follow suit. Such examples illuminate the path to reinforcing security measures through innovation, highlighting the hurdles these organizations faced and their strategies to surmount them. These real-world experiences are invaluable lessons for businesses seeking to understand the intricacies of integrating advanced technologies into their security defenses and anticipating future threats.

It's beneficial for companies to actively seek knowledge from industry forerunners who have pioneered integrating cutting-edge technologies with security practices. By drawing on the successes and lessons learned from these leaders, organizations can better formulate their approaches to adopting new technologies. This learning process is crucial for developing a security scheme that is both resilient and adaptable, enabling businesses to maintain security postures in the face of evolving technological landscapes and emerging digital threats.

13.9.5 Sustaining and Evolving a Cybersecurity Culture

Sustaining and evolving a cybersecurity culture is paramount for organizations aiming to navigate the complexities of digital threats. For organizations, adaptability is critical, ensuring that cybersecurity measures are not just reactive but also anticipatory of future risks. This focus on adaptability keeps security agile and aligned with the rapidly changing threat environment. Involving all employees in cybersecurity and incorporating their feedback emphasizes a collective responsibility for security and ongoing improvement. Additionally, planning for emerging threats is vital for a robust cybersecurity posture. This underscores the need for a dynamic, engaged, and forward-looking cybersecurity culture, enabling organizations to remain resilient and secure in a landscape of continuous digital evolution and new challenges.

Adapting to Change

A dynamic cybersecurity culture is indispensable for organizations striving to keep pace with the rapidly evolving landscape of digital threats, emerging technologies, and changing business practices. This culture prioritizes agility and adaptability, enabling organizations to reassess and refine their security tactics continually. Doing so ensures that cybersecurity efforts are responsive to current threats and anticipate future vulnerabilities. This helps maintain the potency of security measures, ensuring they are in harmony with the broader organizational goals and values.

This kind of adaptive cybersecurity culture requires a commitment to ongoing education, open communication, and employee engagement. Organizations must invest in regular

training programs to update staff on the latest security protocols and threat landscapes. Open communication between IT security teams and other departments facilitates a more responsive and cohesive approach to managing cybersecurity risks. By encouraging a culture of vigilance and shared responsibility, organizations can create a resilient framework that adapts to changes and challenges, safeguarding their assets and data.

Engagement and Feedback Loops

Establishing channels for employees to share feedback on security policies and practices promotes a participatory cybersecurity environment. Such feedback mechanisms are instrumental in gauging the efficacy of existing security measures and identifying opportunities for enhancement. They empower employees by giving them a voice in the cybersecurity discourse, nurturing a culture where every team member feels responsible for the organization's digital safety.

Incorporating employee feedback into cybersecurity improves security protocols and boosts employee engagement and morale. When staff members see their suggestions being considered and implemented, it provides a more profound sense of belonging and commitment to the organization's cybersecurity efforts. This collective engagement ensures a more inclusive approach to tackling cybersecurity challenges, making the organization more resilient against threats.

Looking Ahead

Organizations must anticipate and prepare for future cybersecurity challenges to maintain a competitive edge and ensure digital safety. They must also cultivate an environment of innovation, curiosity, and resilience within their cybersecurity practices. Anticipating potential threats and actively exploring new technological advancements are critical steps in staying ahead of potential adversaries. This enhances an organization's defense mechanisms and culture of continuous improvement and foresight in cybersecurity efforts.

Embedding a forward-looking perspective into an organization's cybersecurity culture is essential for secure navigation through the rapidly evolving digital landscape. By equipping themselves with the latest tools and adopting a mindset geared toward adaptability and anticipation, organizations can ensure they are well-prepared to face and overcome future cybersecurity challenges. This underscores the significance of being vigilant and adaptable as the digital world advances, ensuring organizations remain resilient in the face of new threats and opportunities.

Summary

On the front line of digital defense against cyber threats, managing systems security combines principles and strategies essential for protecting infrastructures. This chapter addresses the core of system security, including defense mechanisms, risk handling, and the critical role of a cybersecurity-aware culture. It highlights the importance of procedural and organizational security measures, ethical considerations, and the impact of new technologies on security practices.

System security is about building a robust foundation to counteract threats and vulnerabilities. It focuses on crafting strong security policies, adhering to international standards, and understanding the consequences of security breaches. This approach aims to safeguard organizational assets and data.

Recognizing and mitigating attacks involves understanding attacker profiles and the diverse cyberattacks they might use. From social engineering to advanced persistent threats, the goal is to develop proactive defense measures to prevent potential security breaches.

Risk management is a systematic approach to identifying, assessing, and handling security risks, and is crucial for maintaining vigilance and preparedness against incidents. It outlines a structured method for managing risks to ensure organizations respond quickly to security incidents.

Exploring the concept of security levels reveals the need for layered defense mechanisms. This encompasses physical, network, application, data, and user security measures, advocating for an approach that addresses vulnerabilities across all layers of an organization's infrastructure.

Procedural and organizational security emphasizes developing security policies, implementing access control, and the importance of security audits and compliance. It also covers the role of security information and event management systems in enhancing threat detection and the significance of business continuity planning.

Backup and recovery strategies are essential for ensuring resilience against attacks such as ransomware. They highlight the importance of effective backup policies, choosing the proper storage solutions, and the role of disaster recovery plans in quickly restoring systems.

Addressing legal and ethical considerations in system security sheds light on navigating cyberethics, compliance, privacy concerns, and intellectual property rights protection. It aims to foster an ethical framework that guides an organization's security plans and practices.

The impact of emerging technologies like cloud computing, IoT, blockchain, and AI on security practices underscores the need for evolving security measures. This ensures that security frameworks can adapt to technological advancements, enhancing defense mechanisms.

Cultivating a resilient cybersecurity culture within an organization is paramount for a security-first mindset. This emphasizes the role of culture, leadership, and governance in promoting cybersecurity, urging the integration of new technologies into security practices to face the evolving threat landscape.

Key Terms

acceptance
administrative privileges
advanced persistent threats (APTs)
anonymization
attack
automatic update service
availability
avoidance
backup
backup media
backup policy
BIOS-level password
boot-level password
business continuity planning (BCP)
California Consumer Privacy Act (CCPA)
CIA triangle
cloud access security brokers (CASBs)
confidentiality
continuous backup
criticality
data loss prevention (DLP)
denial of service (DOS)
DevSecOps
differential backup
digital rights management (DRM)
disaster recovery planning (DRP)
distributed denial of service (DDOS)
dumpster diving
exploit
fault tolerant
firewall
full backup
gamification
hardening
HIPAA
identity theft
identity management
IEEE 802.11i
incentivization
incremental backup
integrity
intellectual property rights (IPR)
intrusion detection system (IDS)
ISO/IEC 27000
keystroke logger
log
malware
mitigation
network
network interface
network intrusion detection system (NIDS)
NIST Cybersecurity Framework
Open Web Application Security Project (OWASP)
operational security
PCI DSS
plaintext
port
port scan
power-on password
pretexting
private key encryption
private network
privilege escalation attack
procedural security
pseudonymization
public key encryption (PKE)
RAID (redundant array of independent disks)
retention period
risk assessment
risk control
secure coding
security hole

security information and event management (SIEM)
security policy
security token
services
SOC 2
social engineering
superuser
tamper-evident case
third-party software
threat
transference
tunnel
unencrypted
uninterruptible power supply (UPS)
universal security slot (USS)
user rights
virtual private network (VPN)
vulnerability
Wi-Fi Protected Access (WPA)
Wired Equivalent Privacy (WEP)
WPA2
WPA3
zero-day exploits

Review Questions

1. What are the three foundational principles of system security?
2. Name two types of attacker profiles and describe their primary motivations.
3. What are the first two steps in the risk identification process?
4. What are two levels of system security?
5. What is a key component of developing and implementing security policies?
6. What role does SIEM play in organizational security?
7. Name an essential element of effective backup policies.
8. What is one primary principle of cyberethics?
9. How does cloud computing impact system security?
10. What is one component of a resilient cybersecurity culture within an organization?

Discussion Topics

1. How do attacker profiles and motivations influence the steps involved in risk identification?
2. Discuss the relationship between developing and implementing security policies and the essential elements of effective backup policies.
3. How do the different levels of systems security impact the legal and ethical considerations in system security?
4. How do emerging technologies affect the formulation and enforcement of security policies?
5. How does building a resilient cybersecurity culture enhance the effectiveness of risk management strategies in system security?

Hands-On Projects

1. Create a simulated cyberattack scenario based on a chosen attacker profile. Conduct a risk assessment for the scenario, identifying potential vulnerabilities and proposing mitigation strategies.
2. Design a comprehensive security policy for an organization that includes specific sections on backup strategies. Consider aspects like data criticality, backup frequency, and recovery objectives.

3. Analyze an organization's security measures across different levels (physical, network, application) and evaluate their compliance with specific data protection laws.
4. Research an emerging technology (e.g., IoT, blockchain) and draft a section of a security policy that addresses the unique challenges and security considerations introduced by this technology.
5. Develop a program that fosters a security-first mindset among employees and assesses its impact on reducing identified risks within the organization.

Ethical Issues

As a systems analyst at a dynamic and competitive company, you recently uncovered a significant data breach that threatens the personal information of thousands of customers. Despite the company's proclaimed commitment to transparency, the leadership delayed informing the affected individuals, motivated by fears of reputational harm and financial implications. This decision places you in an ethical dilemma, torn between your loyalty to the company and your moral obligation to protect the customers' rights to privacy and data security.

Faced with this situation, you are now at a moral crossroads, pondering the appropriate path forward. How can you navigate the complex interplay between the company's interests and the imperative to uphold customer trust and privacy? What advice would you seek, considering the ethical ramifications of postponing the breach disclosure to the impacted customers?

Appendix

A

Communications

Always pay attention to the importance of effective communication. A systems analyst must be adept at using tools like email, video conferencing, and collaboration platforms to ensure clear, timely, and effective communication, especially in distributed or remote teams. Practicing these skills increases your ability to communicate better.

A systems analyst needs strong written communication skills to document system designs, technical specifications, and instructions effectively. Developers, project managers, and non-technical stakeholders must understand the system's architecture and how it functions. Good oral communication skills are also essential for systems analysts, as they frequently interact with various stakeholders, from developers to executives, to gather requirements, provide updates, and explain complex technical concepts in an understandable manner. Excellent online communication skills are necessary for the digital age, where collaboration occurs remotely.

Appendix A discusses communication tools to help the systems analyst write clearly, speak effectively, and deliver powerful presentations—in person or online. In addition, the appendix provides more detailed information introduced earlier in this book. The resources, tools, and techniques described in the following sections will help you communicate effectively.

A.1 Successful Communication Strategies

Successful communication does not just happen. Usually, it results from following a specific agenda that includes careful preparation, hard work, and continuous practice. Five key communication strategies for a project manager are developing a communication plan, practicing active listening, simplifying technical language, providing regular updates, and promoting diversity, equity, and inclusion (DEI).

A.1.1 Develop a Communication Plan

In any systems project, a clear and concise communication plan is paramount. It ensures that all stakeholders understand their roles, responsibilities, and the project's overall progress. A communication plan outlines stakeholders' communication needs and expectations and clarifies communication channels' frequency and form, ranging from emails and meetings to status reports.

A well-structured communication plan can improve the efficiency of information exchange, reduce misunderstandings, and keep everyone aligned with the project goals. The project manager must ensure this plan is adaptable and can be revised as the project progresses, considering changes in project scope, resources, or unexpected challenges.

Creating a communication plan is not a one-size-fits-all process. Instead, it needs to be tailored according to the unique needs of the project and the stakeholders involved. This requires the project manager to have a deep understanding of the project and the stakeholders' diverse needs and communication preferences.

A.1.2 Practice Active Listening

Active listening is a fundamental skill for effective communication. It involves paying full attention to the speaker, understanding their message, reflecting, and responding. This skill allows project managers to grasp technical details, spot issues early, and understand the needs and concerns of the team and stakeholders.

As project managers often deal with complex and technical information in a systems project, active listening becomes even more critical. It ensures they correctly understand technical details and their implications on the project. Active listening fosters a culture of openness and respect, as team members feel heard and valued when their ideas and concerns are taken seriously.

Active listening also extends to nonverbal communication cues. For example, the project manager should understand body language, tone of voice, and other cues that might indicate unspoken concerns or issues. This sensitivity to the underlying message can help resolve issues before they escalate.

A.1.3 Simplify Technical Language

In a systems project, complex technical jargon is often unavoidable. However, not everyone involved in the project may be familiar with these terms. Hence, knowing how to break down complex concepts into more straightforward and understandable language is valuable for project managers.

Simplifying technical language helps ensure that everyone, regardless of their technical proficiency, understands the project goals, progress, and challenges. This is particularly important when communicating with stakeholders who may not have a technical background, such as clients, executives, or team members from different departments.

However, simplifying technical language doesn't mean oversimplifying it. Instead, it involves explaining the information in a way that is accessible and meaningful to the audience. This might include using analogies, examples, visual aids, or other strategies to convey complex ideas effectively.

A.1.4 Provide Regular Updates

Regular status updates are vital to keep everyone on the same page and ensure the project is on track. In addition, they allow team members and stakeholders to ask questions, share updates, discuss challenges, and make collective decisions.

In a systems project, these updates are especially crucial due to the technical and complex nature of the work. Regular updates help identify any technical issues early, ensure that the team's efforts align with the project goals, and help avoid costly or time-consuming mistakes down the line.

To make these updates effective, the project manager should ensure they are well-structured and efficient. For example, scheduling a meeting to report project status should involve setting a clear agenda, inviting the right people, focusing on key topics, and facilitating open and productive discussions. It's also important to keep these updates regular and consistent to maintain a steady flow of communication.

A.1.5 Promote Diversity, Equity, and Inclusion (DEI)

DEI stands for diversity, equity, and inclusion. These related concepts are crucial for a respectful, productive, and harmonious modern workplace. DEI has become an essential component of professional communications across most industries.

Diversity refers to the variety of differences between people in an organization. This includes race, gender, ethnic group, age, personality, cognitive style, tenure, organizational function, education, and background. Diversity not only involves how people perceive themselves but how they perceive others. Those perceptions affect their interactions.

Equity involves ensuring fairness within your organization. This doesn't mean treating every employee the same way but considering their unique characteristics, strengths, and challenges. It's about recognizing that advantages and barriers exist and that, to ensure fairness, we need to address them.

Inclusion is a state of being valued, respected, and supported. It's about focusing on the needs of every individual and ensuring the right conditions are in place for each person to achieve their full potential. Inclusion should be reflected in an organization's culture, practices, and relationships that are in place to support a diverse workforce.

DEI ensures that everyone feels respected, is treated fairly, and has opportunities to excel in their chosen careers. It is the responsibility of every individual in an organization, but leaders and managers need to set the right tone and lead by example. DEI plays a vital role in communication for systems projects because they involve complex technical challenges that benefit from various perspectives.

A.2 Essential Questions to Consider

No matter how well you plan your communication, you must know your subject. Your credibility and effectiveness depend on whether others believe and support your views. No one knows everything, so a specific preparation strategy is essential.

To help you prepare as a project manager or leader, remember the five key communication strategies described in Section A.1. As a project member, it's equally important to be a successful communicator yourself, which means you must consider five essential questions about yourself, your audience, and your objectives: why, who, what, when, and how.

A.2.1 Why

Know *why* you are communicating and what you want to accomplish. Then ask yourself, "Is this communication necessary, and what specific results am I seeking?" Your entire communication strategy depends on the results that you need.

A.2.2 Who

Know *who* your audience is. How users' information needs depend on their organizational and knowledge levels was described earlier in the book. When communicating with management, for example, sometimes a fine line exists between saying enough and saying too much. Each situation is different, so you must use good judgment and be alert for input from your audience.

A.2.3 What

Know *what* is expected of you and when to go into detail. This is directly related to knowing who your audience is and the organizational and knowledge levels of the organization. For example, a vice president might expect less detail and more focus on how a project supports the company's strategic business goals. You must design your communications just as carefully as your systems project. For instance, will the target audience expect you to address a specific issue or topic? Will they expect cost estimates or charts? Design your communications based on the answers to those questions.

A.2.4 When

Know *when* to speak and *when* to remain silent and let others continue the discussion. To be an effective speaker, you must be a good listener (see Section A.1.2) and use audience feedback to adjust your presentation. Good timing is an essential part of every presentation. Your delivery must be adequately paced: too fast, and you will lose your audience; too slow, and they might become bored.

A.2.5 How

Know *how* to communicate effectively. You can strengthen your communication skills by using suggestions in this appendix, reflecting on your own experiences, and observing successful and unsuccessful techniques used by others.

A.3 Written Communications

Good writing is essential because others often judge you by your writing. If you make a mistake while speaking, your audience probably will forget it. Your written errors, however, might stay around for a long time. Grammatical, typographical, and spelling errors distract readers from your message. Your written communications will include email messages, memos, letters, workgroup communications, and formal reports.

A.3.1 Writing Skills

Good writing skills are the backbone of effective written communication. For example, they are essential for providing clarity and accuracy in communication. These skills enable individuals to explain complex ideas, data, or technical details in a manner that is easy to understand, thereby minimizing the chance of misunderstandings or misinterpretations. This clarity is crucial in specialized fields such as systems analysis and design, where clear communication of requirements, specifications, and processes can significantly impact project success.

Writing involves more than just conveying information—it also involves crafting compelling messages that capture the reader's attention, maintain their interest, and sway them where necessary. This involves the strategic use of language, tone, and structure and the ability to effectively tell a story or present an argument. Whether it's a business proposal that needs to convince stakeholders, marketing material aimed at potential customers, or an opinion piece intended to sway readers, engaging and persuasive writing can significantly enhance the impact of written communication.

Powerful writing can tailor the tone, style, and language to different audiences, contexts, and purposes. For example, the language in a user manual will likely differ from that in an executive summary or a project proposal. By understanding the needs and expectations of the intended audience and adapting the message accordingly, writers can ensure that their communication is well-received and achieves its intended purpose. This adaptability in writing is critical for effectiveness in diverse communication scenarios.

A.3.2 Writing Style

If you need to take a writing course, consider doing so. If you have a choice of courses, select one focusing on business or technical writing. Any writing class, however, is worth the effort. Most bookstores and libraries have excellent books on effective communication, and many Internet sites offer writing guidelines, tips, and grammar rules. As you prepare written documents, keep in mind the following suggestions:

- Know your audience. If you are writing for nontechnical readers, use terms that readers will understand.
- Use the active voice whenever appropriate. For example, the active-voice sentence "Tom designed the system" is more engaging than "The system was designed by Tom," which is an example of the passive voice. However, passive voice is sometimes used in academic writing and formal publications.
- Keep your writing clear, concise, and well-organized. Each paragraph should present a single topic or idea.
- Use an appropriate style. For example, use a conversational tone in informal documents and a business tone in formal documents.
- Use lists. If a topic has many subtopics, a list can organize the material and make it easier to understand.
- Use short, easy-to-understand words. Your objective is not to impress your audience with the size of your vocabulary.
- Avoid repeating the same word too often. Instead, use a thesaurus to locate synonyms for frequently repeated words. Many word processing programs include a thesaurus and other tools to help you write better.
- Check your spelling. You can use the spell checker in your word processing program but remember that a spell checker identifies only words that do not appear in the program's dictionary. So, for example, a spell checker may not identify instances when you use the word "their" instead of "there."
- Check your grammar. Most word processing programs include a grammar checker, a tool that can detect usage problems and offer suggestions. When you use a grammar checker, you can set various options to match the level and style of the writing and to highlight or ignore certain types of usage. For example, you can set the grammar checker in Microsoft Word to check grammar rules only or configure it to check writing style, including gender-specific words, sentence fragments, and passive sentences. In addition, special-purpose tools, such as Grammarly (Figure A-1), can be used for more sophisticated grammar analysis.
- Review your work carefully. Then, double-check it for spelling, grammatical, and typographical mistakes. Finally, ask a colleague to proofread your work and suggest improvements.

A.3.3 Readability

All writers must consider readability, which analyzes the ease of comprehension by measuring specific characteristics of syllables, words, and sentences. Two popular readability measures are the Flesch Reading Ease score and the Flesch-Kincaid Grade Level score. Both scores can be calculated automatically using Microsoft Word, as shown in Figure A-2.

The Flesch Reading Ease score measures the average sentence length and the average number of syllables per word and rates the text on a 100-point scale. With this tool, the higher the score, the easier the text is to understand. Microsoft suggests that for most standard documents, you should aim for a score of 60 to 70. For example, this paragraph has a Flesch Reading Ease score of 65.

Figure A-1 Grammarly is a sophisticated tool for checking grammar that works with Microsoft Office or within a browser. This example uses text from https://en.wikipedia.org/wiki/Lloyd_Alexander.
Source: Grammarly Inc.

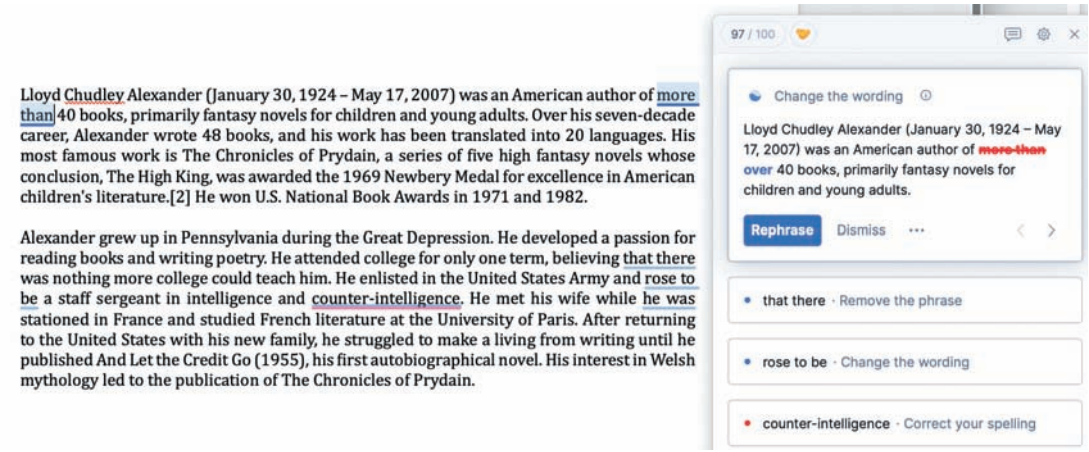


Figure A-2 Two popular readability measurement tools are the Flesch Reading Ease score and the Flesch-Kincaid Grade Level score.
Source: Microsoft Word

Readability Statistics

Counts	
Words	68
Characters	314
Paragraphs	1
Sentences	4
Averages	
Sentences per Paragraph	4
Words per Sentence	17
Characters per Word	4.5
Readability	
Flesch Reading Ease	65.1
Flesch-Kincaid Grade Level	8.3
Passive Sentences	0%

Readability statistics are based on your text selection.

The Flesch-Kincaid Grade Level score uses the same variables but in a different formula, producing a rating keyed to a U.S. grade-school level. For example, a score of 8.0 would indicate material easily understood by a person at an eighth-grade reading level. With this tool, Microsoft suggests that for most standard documents you should aim for a score of 7.0 to 8.0.

A.3.4 Email, Memos, and Letters

Because email will be one of your primary tools for written communication, it is vital to use it properly and effectively. Email usually is less formal than other written correspondence, but you still must follow the rules of good grammar, correct spelling, and clear writing.

Although many authors use a more conversational style for email, you should remember that email messages often are forwarded to other recipients or groups. Therefore, you must consider the users to whom your email might be distributed. If you regularly exchange messages with a specific group of users, most email programs allow you to create a distribution list that includes the members and their email addresses. Now that email has become the standard method of business communication, all users must know how to use it properly, professionally, and courteously. This topic is discussed in the following section.

Although email is the main form of internal communication, internal memos and announcements still are important, and external communications often require letters printed on company letterhead. Most companies use a standard format, or template, for internal memos and external letters. You can download and use the templates if your company stores them on a network. You can also use a word processor to create your own templates with specific layouts, fonts, and margin settings. A template gives your work a consistent look and makes your job easier. Most word processing programs also provide a feature that allows you to design your memos as forms and fill in the blanks as you work.

A.3.5 Social Media at Work

Social media are seeing explosive growth, which is sure to continue. This section focuses on just one aspect of this trend: social media in the office. It would be difficult to cite a communication method with greater potential benefits—and dangers—than the volatile mix of social networking and modern business settings. Many employees use social networking to advance their careers. Unfortunately, others have been reprimanded (or worse) for saying too much, or to the wrong people, or at the wrong time.

The Backdrop

Corporations are racing to embrace social media in their marketing plans. They have learned they can create excitement, call attention to their products, and reach a young, active, socially aware market. As a result, in the corporate world, social media-driven event marketing has become a specialty. From an IT viewpoint, these trends are healthy because they demonstrate that technology and creativity are essential in corporate strategy, which is good news for systems analysts.

Advantages

IT professionals can use social media to network with others, learn about new technology, meet colleagues, discuss career issues, and maintain an online presence that would have been impossible just a few years ago. In addition, someone can gain credibility by sharing information and technical tips, posting how-to videos, or documenting an especially interesting assignment or challenge. However, any one of these activities could also have disastrous consequences.

Risks

Drivers stopped for speeding often say, “I didn’t know I was going that fast.” Similarly, employees who misuse social media usually plead ignorance. But there is a better way: know the rules, observe the rules, and when in doubt—don’t!

Most firms have social media policies that govern workplace behavior. For example, some prohibit personal communications on company time or with company resources. Others use a case-by-case approach. Rather than guess your company’s policy, you should find out precisely what the policy is and make sure you understand it clearly.

Three Ways to Get Fired

You must be sure that your message content is appropriate and in no way violates your company's confidentiality policy. One way to get fired quickly is to brag about a new marketing strategy before it has been publicly announced. Another way is to be somewhere or do something that might degrade your company's image, especially if your nametag, uniform, or location can identify your employer. Finally, perhaps the quickest way to get fired is to launch an angry verbal attack on your fellow employees or managers.

Other Issues

You should use a "need-to-know" approach to direct your messages to those with a legitimate interest and people you trust. Avoid blanket messaging and broadcasting. Also, think about whether the content is appropriate for the site you are using. For example, a funny joke might be out of place on LinkedIn but okay on Twitter. Your minute-to-minute experience at a rock concert might interest some people, but you might not want to send it to your entire contact list or followers.

The bottom line is that social networking, instant messaging, and cell phone texting are popular because they allow informal, interactive, and immediate communication. While these can be valuable collaboration tools, users should exercise good judgment and common sense, just as they would in any form of interpersonal contact. No one will remember all your excellent messages. They will, however, always remember the inappropriate ones.

A.3.6 Netiquette

Netiquette combines the words *Internet* and *etiquette*. With the explosive growth of social networking, netiquette is more crucial than ever. If you follow the guidelines in the preceding section, you can avoid significant problems. But you should set your sights higher than that. An IT professional should have strong communication skills, good manners, proper style, and plenty of common sense. Dale Carnegie, a famous educator and authority on interpersonal relations, said many years ago that others evaluate us based on "what we do, how we look, what we say, and how we say it."

All email users should know some simple rules, including good manners and common sense. For example, an excellent starting point is to avoid sending personal or confidential material because your messages might be forwarded by others and distributed more widely than you intended. Another important rule is to refrain from sending or replying to an email when you are tired or upset. Instead, you can write a draft if you want, but save the unsent message so you can review it later.

Here are some common rules and tips:

- Always fill in the subject field with a brief description of the contents.
- Be concise—in most cases, less is more.
- Be professional. Remember, if it has your name on it, it reflects on you personally.
- Be sure to check your spelling.
- Don't forward jokes or chain letters without the permission of the recipient.
- Don't overuse humor or sarcasm that might work in a face-to-face situation but not in an email context.
- Don't type in all caps—it is like YELLING! It is also hard to read.
- Don't use colored fonts, backgrounds, or images in business email messages.
- Don't use the return receipt request feature unless there is a valid business reason.
- If you have large attachment files, try to zip or compress them before sending them, or use a file delivery service that uses cloud storage.
- If you send a message to a group of people, especially if they don't know each other, use a blind copy (Bcc) for all of the recipients to shield their addresses from the entire group.

- Only give out the personal contact information of others with their specific permission to do so.
- Only include personal information if you are 100% sure of your recipient and no other means of communication would provide better privacy and security.
- Remember that there are copyright laws. You do not have an unrestricted right to do whatever you please with someone else's email message. Laws against discrimination and defamation can also apply to email messages.
- Don't include all the earlier messages when replying unless there is a compelling reason to do so.

In addition to these guidelines, following company policy regarding communications at work is essential. For example, many firms restrict personal communications that involve company time or equipment, and courts have upheld an employer's right to limit or monitor such communications.

A.3.7 Reports

You must prepare many reports during systems development, including the preliminary investigation report, the system requirements document at the end of the systems analysis phase, the system design specification at the end of the system design phase, and the final report to management when the system goes into operation. You also might submit other reports, such as status reports, activity reports, proposals, and departmental business plans. You will create your reports as electronic documents so that you can attach them to emails.

Adobe PDF

When you create a report that will be delivered electronically, you should use the same high standards that apply to traditional printed materials. Notice that Microsoft Word provides many premade templates that you can use for your report. Once you create a Word document, it is simple to transform the document into an Adobe PDF.

A PDF offers several advantages over Microsoft Word for professional reports, including accessibility, security, consistency, compactness, and interactivity.

Universal Accessibility. PDF is a universal file format that can be opened and viewed consistently on any device (PC, Mac, smartphone, or tablet) and across different operating systems using free software, such as Adobe Reader. This ensures that the recipient can view the document exactly as the sender intended, regardless of the software used to create the original document.

Document Security. PDF files support robust security features. You can protect your document with a password, restrict editing and printing rights, or even include a digital signature. This makes it suitable for sharing sensitive information while maintaining confidentiality and integrity.

Consistent Formatting. One of the significant advantages of PDFs is that they preserve the document's formatting. This includes fonts, images, graphics, and layout, regardless of the application and system used to read the PDF. This is particularly useful for professional reports where consistent presentation and design are essential.

Compact and Optimized. PDF files can be compressed to a relatively small file size, making them easier to share and store, especially for large reports with numerous graphics and images. This makes it an ideal format for emailing and downloading from websites.

Interactive Features. PDF files can contain interactive features such as hyperlinks, fillable forms, buttons, video, audio, and other multimedia elements. This capability can make a professional report more engaging, interactive, and user-friendly, enhancing the reader's experience and comprehension.

Report Format

Whether you create the report using Microsoft Word, Adobe Acrobat, or some other program, your work must be professional, well-organized, and well-written. Experienced systems analysts know that when you put your name on a document, readers will see your work as a reflection of your skills and abilities. Also, for better or worse, the document will probably be around for a long time. With those thoughts in mind, consider reviewing the written communication guidelines that appear elsewhere in the book.

Although most reports are delivered electronically, you must sometimes prepare printed versions. For example, suppose you must submit a written system requirements document. Create various sections, including an introduction, an executive summary, findings, recommendations, time and cost estimates, expected benefits, and an appendix containing relevant data.

A professional report follows a structured format, allowing readers to understand and navigate the content quickly. This format may vary based on the report's nature, the assignment's or organization's specific requirements, and the audience's expectations. Here's a typical report structure:

Cover Page. The cover is the first thing readers see when receiving the report. An attractive, well-designed cover page can create a positive first impression, reflecting the professionalism and attention to detail of the author or the organization. It should include the report's title, the author's name, the submission date, and possibly the organization's name and logo.

Executive Summary or Abstract. This is a brief summary of the report's contents. It should provide an overview of the report's purpose, main points, and key findings or recommendations. It should be succinct and clear enough that a reader can understand the report's gist without reading the rest. Generally, the executive summary should be at most 250 words or one page.

Table of Contents. This section lists the primary headings and subheadings in the report, usually with corresponding page numbers for easy navigation.

Introduction. This section introduces the topic, outlines the report's purpose, and provides the background or context for the report. It should also state the report's objectives and define any specific terms.

Body. This is the central part of the report where the subject matter is thoroughly discussed. It often includes sections such as methodology (how the research or work was conducted), findings/results (what was discovered), and analysis/discussion (what the findings mean).

Conclusions. This section summarizes the main points made in the report and the conclusions drawn from the findings and analysis.

Recommendations. If applicable, the recommendations section presents the best system alternative, with a brief explanation that should mention economic, technical, operational, and schedule feasibility. This section could also include a costs and benefits analysis, which should list each primary system alternative's advantages, disadvantages, costs, and benefits. You should have a clear description of the financial analysis techniques you used. You should apply one or more financial analysis tools described in Appendix B. You can use tables or graphs to support and clarify alternatives when necessary.

References or Bibliography. This section lists the sources of information used in the report, giving credit to original authors and allowing readers to follow up on the sources if they wish.

Appendices. If additional supporting information is too detailed or lengthy to include in the body of the report, it can be added in an appendix. This may include data, tables, charts, interview transcripts, and so on.

Advantages of Written Reports

While both written reports and oral presentations (described in Section A.4) have their place in a systems project, there are several advantages to using a written report:

Clarity and Comprehensiveness. Writing can enforce a level of clarity and accuracy that may take more work to achieve in oral communication. Written reports usually allow for more detail and complexity. They can contain extensive data, analysis, references, and appendices, providing a more comprehensive project view.

Accessibility. Written reports can be accessed and reviewed anytime, allowing stakeholders to digest the information at their own pace and refer to it as needed. In contrast, oral presentations are time-bound unless archived for later video viewing and may require scheduling to accommodate everyone's availability.

Permanence and Referability. Written reports create a permanent record of information, findings, decisions, or recommendations that can be referred to at any time. This can be particularly helpful for complex systems projects, where it's crucial to have accurate historical data for reference and for tracking progress over time. Permanence and referability can also be helpful for audit purposes, demonstrating compliance with regulations or standards, and transitioning the project to new team members.

Review and Revision. Before a written report is finalized, it can be reviewed and revised multiple times to ensure accuracy and quality. This process also allows for input from various stakeholders, leading to a more collaborative and comprehensive report.

Allows Time for Consideration. Readers can carefully consider the content in a written report, leading to more thoughtful responses or decisions.

A.4 Oral Communications

An oral presentation is required at the end of the preliminary investigation and again after the systems analysis phase. Compared with written reports, oral presentations can be more engaging, allow immediate feedback and discussion, and be more suited to specific information or audiences. For example, you might need to give more than one presentation in some situations to present technical material to members of the IT department or to present an overview for top managers.

When preparing an oral presentation, you should perform five important tasks: define the audience, define the objectives, organize the presentation, prepare your presentation aids, and practice your delivery.

A.4.1 Define the Audience

Before developing a detailed management presentation plan, you must define the audience. For example, senior managers often prefer an executive summary to a detailed presentation. However, that is only sometimes the case, especially in smaller companies where top management is more involved in day-to-day activities. If you consider the expectations of your audience and design your presentation accordingly, you will improve your chances of success.

A.4.2 Define the Objectives

When you communicate, you should focus on your objectives. For example, in the management presentation for the systems analysis phase, your goals are the following:

- Inform management of the status of the current system.
- Describe your findings concerning the current system problems.

- Explain the alternative solutions that you developed.
- Provide detailed cost and time estimates for alternative solutions.
- Recommend the best alternative and explain the reasons for your selection.

A.4.3 Organize the Presentation

Plan your presentation in three stages: the introduction, the information, and the summary. First, you should introduce yourself and describe your objectives. During the presentation, make sure that you discuss topics in a logical order. It would be best if you were as specific as possible when presenting facts—your listeners want to hear your views about what is wrong, how it can be fixed, how much it will cost, and when the objectives can be accomplished. Finally, briefly review the main points in your summary and then ask for questions.

A.4.4 Prepare Presentation Aids

Much of what people learn is acquired visually, so you should use appropriate visual aids to help the audience follow the logic of your presentation and hold their attention. Visual aids also can direct the audience's attention away from you, which is helpful if you are nervous when you give a presentation. You can use a visual aid with an outline of topics to help you stay on track. You can enhance the effectiveness of your presentation with visual aids that use various media and software, as explained in the following sections.

Visual Aids

Visual aids can help you display a graphical summary of performance trends, cost-benefit examples, or a bulleted list of important points. You can use whiteboards, flip charts, overhead transparencies, slides, films, and videotapes to enhance your presentation. When preparing your visual aids, ensure the content is clear, readable, and easy to understand. Verify ahead of time that the audience can see the graphic material from anywhere in the room. Remember that equipment can fail unexpectedly, so be prepared with an alternate plan (e.g., speaking directly to the audience without visual aids).

Presentation Software

Some experienced speakers talk without slides, but most people are more comfortable using visual aids to support their presentations. With a computer and a projection system, you can use presentation software such as Apple Keynote, Google Slides, and Microsoft PowerPoint to create an engaging multimedia slide show.

Apple Keynote's ease of use, design capabilities, and compatibility with other Apple products make it an excellent tool for creating and delivering effective presentations. As shown in Figure A-3, Keynote offers many tips and techniques that can help create engaging and visually appealing presentations. It can assist in the following ways:

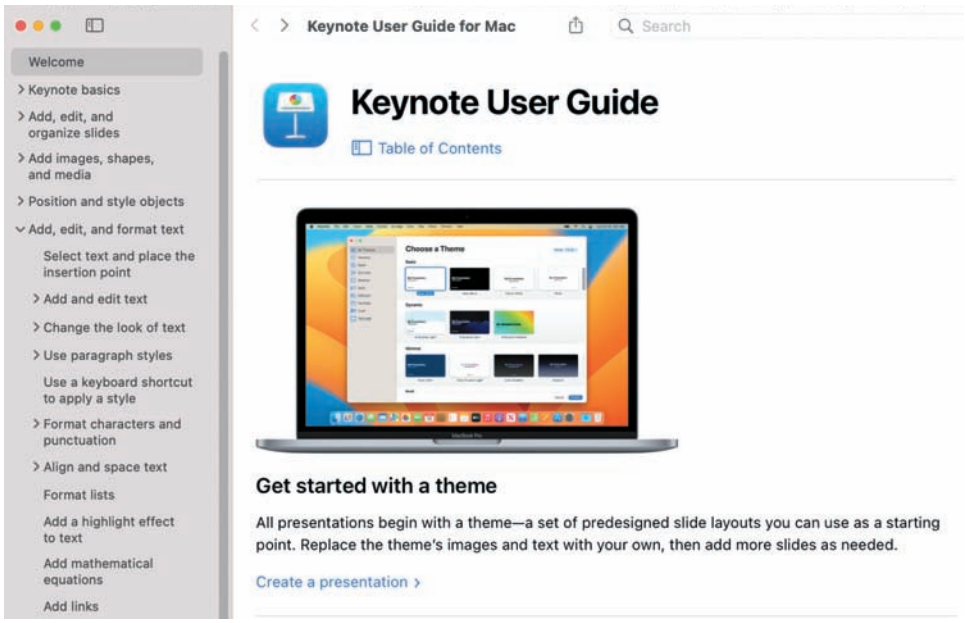
Templates. Keynote comes with a wide array of professionally designed templates. These can help you create a visually compelling presentation without requiring design skills.

Animation and Transition Effects. Keynote offers various slide transition effects and object animations that can make your presentation more dynamic and engaging.

Interactive Charts and Graphs. With Keynote, you can incorporate interactive charts and graphs into your presentation, making complex data easier to understand. This is particularly useful for presentations with numeric information.

Collaboration Features. Keynote supports real-time collaboration, allowing multiple team members to work on the presentation simultaneously. This can be extremely helpful in preparing for group presentations.

Figure A-3 Apple Keynote is an example of powerful presentation software.
Source: Apple Inc.



Compatibility with the Apple Ecosystem. If you already use other Apple products, Keynote integrates seamlessly. You can control your presentation from an iPhone or use an Apple Pencil on an iPad to draw or write on your slides during the presentation.

Magic Move. This feature creates smooth, animated transitions between slides when identical objects are present. It can add a professional touch to your presentations.

Media Integration. Keynote allows you to easily embed videos, images, and even audio files into your presentation, helping you to illustrate your points better.

Presenter Notes. You can add notes only you can see during the presentation, helping you remember key points without cluttering your slides.

Export Options. Keynote presentations can be exported in various formats, including PowerPoint and PDF, allowing you to share your work with those who don't use Keynote.

Rehearse Slideshow. This feature lets you practice your presentation with a timer, helping you prepare better.

Presentation Guidelines

Preparing a compelling slide presentation requires time and effort; personal experience is the best teacher. Unfortunately, there is no universal agreement about how to prepare a slide show, and many sources of information exist. However, some general guidelines include the following:

- Your first step (and perhaps the most important) is to prepare an overall outline that will be the foundation of your presentation. Focus on your presentation's content and structure before you consider visual issues.
- There's a fine line between providing too little information and too much.
- Display one topic per slide and try to follow the "7 by 7" rule: at most seven items per slide and at most seven words per item.
- When displaying a list of items, consider using a series of slides to add each point sequentially, especially if you want to focus on the item being discussed.

- Use bullets rather than numbers unless you show a specific sequence or order.
- Choose easily readable fonts. Use sans serif styles, such as Arial, for all body text. If you use a serif style (such as Times Roman), apply it only to titles.
- Use appropriate font sizes for titles and body text. Your goal is to prepare slides that are readable and visually attractive. Although font size preference depends on individual judgment and experience, for titles, try either 40- or 36-point fonts; for body text, 32- or 24-point fonts usually work well.
- Use special effects and animations judiciously. Too many graphics, colors, sounds, or special effects will distract your audience.
- You can include tables or graphics but keep them simple and easy to understand. Also, you can use a special effect, such as boldface, italic, underlining, or a different color, to highlight a meaningful word or phrase.
- Strive for a consistent look and feel among your slides and position visual elements in the same place on each slide. You should use a master template to ensure uniformity and conform to company-wide standards that might apply, such as a copyright notice, a confidentiality statement, or placement of the company name and logo. Choose colors carefully and keep them consistent. Usually, light letters on a dark background are the easiest to read. Presentation software normally has predefined color palettes that provide background and text colors that ensure readability. Use these palettes as a guideline for selecting colors when possible.
- Be sure to check spelling and grammar.
- During the presentation, do *not* read your slides to the audience; they can read the slides on their own. Your slide presentation is an outline that provides structure and emphasis—it is not the presentation itself.
- It is important to deliver a presentation that can be viewed easily anywhere in the room. When setting up, consider the size of the room, the number of people attending, the size and location of your visual aids, and the characteristics of any projection equipment you will use.

A.4.5 Practice

The most important part of your preparation is practice. It would be best if you rehearsed several times to ensure that the presentation flows smoothly and the timing is correct. In addition, practicing will make you more comfortable and build your confidence.

Do not be tempted to write a script. If you read your presentation verbatim, you cannot interact with your audience and adjust your content based on their reactions. Instead, prepare an outline of your presentation and practice from the outline. Then, when you deliver the actual presentation, you will not have to struggle to remember the exact words you planned to say, and you will be able to establish a good rapport with your audience.

A.4.6 The Presentation

When you deliver your presentation, remember the following pointers:

Sell Yourself and Your Credibility. As a presenter, you must sell yourself and your credibility. A brilliant presentation will only convince top managers to approve the system if they are sold on the person who gave the presentation. On the other hand, projects often are approved based on the presenter's knowledge, commitment, and enthusiasm.

Your presentation must show confidence about the subject and your recommendations. It would be best to avoid conflicts with the people attending the presentation. If you encounter

criticism or hostility, remain calm and focus on the issues—not the person commenting. You will have a successful presentation only if you know the material thoroughly, prepare properly, and sell yourself and your credibility.

Control the Presentation. During the presentation, you must control the discussion, maintain the pace of the presentation, and stay focused on the agenda—especially when answering questions. Although you might be more familiar with the subject material, you should not display a superior attitude toward your listeners. Maintain eye contact with the audience and use some humor, but do not make a joke at someone else's expense.

Define Technical Terms. You should avoid specialized or technical terminology whenever possible. If your audience is unfamiliar with a term you plan to use, define it or find another way to say it so they will understand.

Answer Questions Appropriately. Let your audience know whether you would prefer to take questions as you go along or have a question-and-answer session at the end. Sometimes the questions can be quite tricky. Therefore, you must listen carefully and respond with a straightforward answer. Try to anticipate your audience's questions to prepare your responses beforehand.

When answering a problematic or confusing question, repeat it in your own words to ensure you understand it. For example, you can say, "If I understand your question, you are asking . . ." This will help avoid confusion and give you a moment to think on your feet. To ensure you provide a clear answer, you can ask, "Have I answered your question?" Allow follow-up questions when necessary.

Use Effective Speaking Techniques. The delivery of your presentation is just as important as its content. You can strengthen your delivery by speaking clearly and confidently and projecting a relaxed approach. However, it would be best if you also controlled the pace of your delivery. If you talk too fast, you will lose the audience; if the pace is too slow, people will lose their concentration and the presentation will not be effective.

A.4.7 Overcoming Nervousness

Many speakers are nervous when facing an audience. This feeling often abates with experience over time. However, if this is a problem for you, keep the following suggestions in mind:

- Control your environment. If you are most nervous when the audience is looking at you, use visual aids to direct their attention away from you. If your hands are shaking, do not hold your notes. If you deliver a computer-based presentation, using a handheld wireless device to control the slides is a good idea. Concentrate on using a strong, clear voice. If your nervousness distracts you, take a deep breath and remind yourself that you know your subject.
- Turn your nervousness to your advantage. Many people do their best work when they are under a bit of stress. Think of your nervousness as normal pressure.
- Avoid meaningless filler words and phrases. Using words and phrases such as *okay, all right, you know, like, um*, and *ah* are distracting and serve no purpose.
- Practice! Practice! Practice! Some people are naturally gifted speakers, but most need lots of practice. You must work hard at practicing your presentation and building your confidence. Many schools offer speech or public speaking courses that are an excellent way of practicing your skills. It also can be advantageous to preview your presentation with one or more people and ask for input.
- Above all else, remember that most audience members want you to succeed!

A.5 Online Communications

The COVID-19 pandemic changed the world in numerous ways. Communication was more important than ever during the lockdown, including written (Section A.3) and oral (Section A.4) communications. But lockdowns also forced many people to work from home (WFH), leading to most forms of communication moving online. No workgroup software (groupware) exemplifies this shift more than Zoom, Slack, and Microsoft Teams.

A.5.1 Zoom

Zoom is an online, cloud-based video-conferencing platform that allows users to connect via video, audio, and chat. Zoom is available on various devices, including computers, smartphones, and tablets. It is used by businesses of all sizes and individuals for personal and educational purposes. Other online communications systems have been available for some time, such as GoTo Meeting by GoTo (previously LogMeIn) and Webex by Cisco, but Zoom (Figure A-4) is the undisputed market leader.

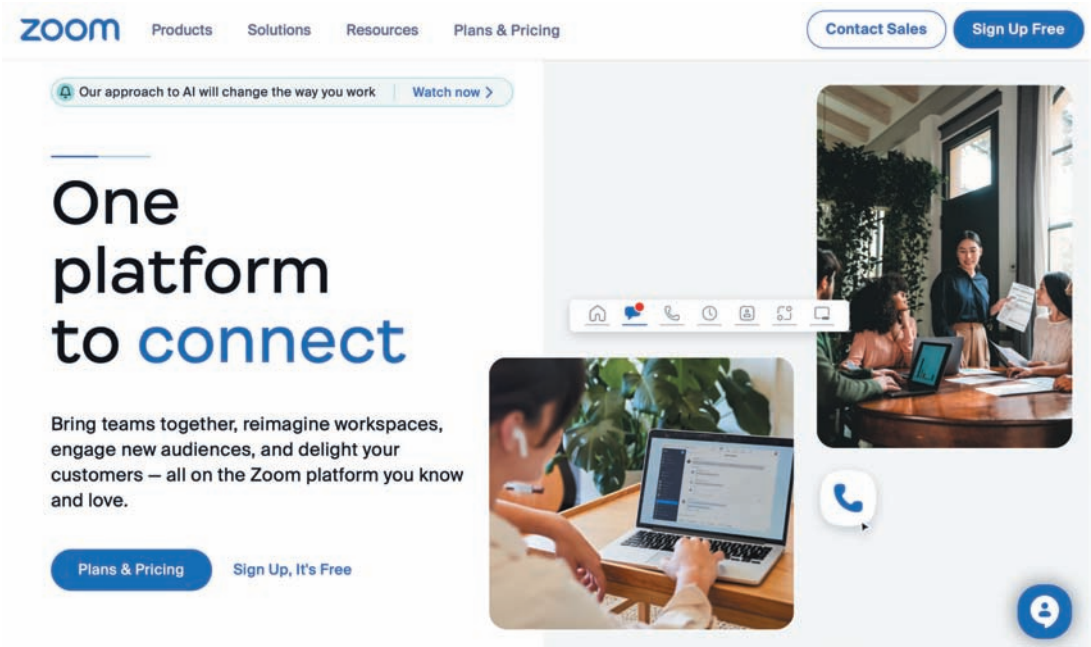
Features

Zoom offers a variety of features that make it a powerful tool for online communication and collaboration. It is easy to use and provides various features, making it a valuable tool for businesses and individuals. These features include:

Video Conferencing. Zoom users can connect in real time via video. This allows for face-to-face interaction, which can help build relationships and collaborate on projects.

Webinars and Large-Scale Virtual Events. Zoom has also made it easier to host webinars and large-scale virtual events, which have become a significant aspect of professional communication. Webinars are online presentations delivered much like a traditional in-person lecture.

Figure A-4 Zoom is a video-conferencing platform that provides online meeting and collaboration services. It is widely used for remote work, online learning, and virtual events.
Source: Zoom Video Communications, Inc.



Audio Conferencing. Zoom users can also connect via audio only. This is a good option for situations where the video is optional, such as phone calls or group chats, or when meeting participants are in remote locations with poor Wi-Fi coverage.

Chat. Zoom users can also chat with each other in real time. This is a good option for sending quick messages or collaborating on documents.

Screen Sharing. Zoom users can share their screens with other participants. This is a good option for presentations or showing others how to use a particular software program.

Breakout Rooms. Zoom users can create breakout rooms, which allow participants to break off into smaller groups for more focused discussions.

Live Transcription. Zoom offers live transcription, which provides a real-time meeting transcript. This is a good option for people who are hard of hearing or who want to be able to refer to the meeting later.

Using Zoom Effectively

Using Zoom properly is now an essential communication skill for project professionals. Here are 10 things Zoom users need to know to use the platform effectively:

Plan Your Meetings. Before you start a Zoom meeting, take some time to plan what you want to discuss and who you want to invite. This will help you maximize your time and ensure the meeting is productive.

Check Your Audio and Video Settings. Before starting a meeting, ensure your audio and video settings are working correctly. This will help you avoid any technical difficulties during the meeting.

Test Your Internet Connection. Ensure you have a strong connection before starting a meeting. This will help ensure that the session runs smoothly.

Set Ground Rules. At the beginning of the meeting, take some time to set ground rules for the discussion. This might include respecting each other's time, keeping the conversation on track, and avoiding distractions.

Dress Professionally. Even though you are not meeting in person, it is still important to dress professionally for Zoom meetings. This will help you make a good impression on your colleagues and clients.

Be Aware of Your Surroundings. When on a Zoom call, be mindful of your surroundings. For example, ensure you are in a quiet place where you will not be interrupted.

Pay Attention. When on a Zoom call, pay attention to the discussion. This will help you stay engaged and avoid any misunderstandings. Also, if you are running the Zoom meeting, be aware of uninvited "guests" disrupting the meeting and acting inappropriately (also called "Zoombombing").

Ask Questions. If you need help understanding something, feel free to ask questions. This will help you get the most out of the meeting.

Take Notes. Take notes if you need to remember anything discussed during the meeting. This will help you stay on track and follow up on action items.

Use Features. Zoom offers a variety of features that can help you make your meetings more effective. These features include breakout rooms, screen sharing, whiteboarding, and chat. They have made Zoom a more versatile and powerful tool for professional communications. Use these features to engage participants and make the most of your time.

Overall, Zoom has had a significant impact on the way businesses communicate. It has made video conferencing more accessible, affordable, and easy to use. This has led to more widespread adoption of video conferencing, which has, in turn, improved communication and collaboration within businesses. As a result, Zoom has become an essential communications tool for businesses of all sizes.

A.5.2 Slack

Slack is a popular cloud-based team collaboration tool that provides a platform for messaging, file sharing, and integration with many other productivity and business apps. Created by the company of the same name (Slack Technologies), it's designed to improve communication, collaboration, and productivity within teams and organizations. It's proven particularly useful for projects with remote team members.

Features

Slack (Figure A-5) is easy to use, affordable, and offers a variety of features that can help improve project communications. These features include:

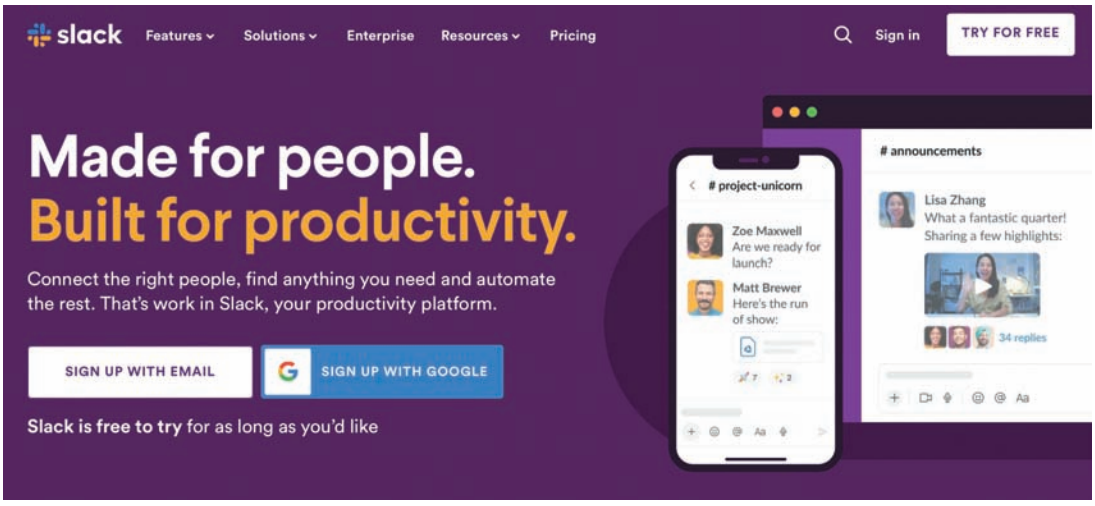
Channels. Slack organizes conversations through channels, which can be created for different topics, teams, or projects. Channels can be public (accessible to everyone in the workspace) or private (invite-only).

Messaging. Slack allows users to communicate with each other in real time, both one-on-one and in groups. Messages (direct messages or DM) can be sent as text, images, or files.

Audio and Video Calls. While its primary focus is text-based communication, Slack also supports audio ("huddles") and video calls. This functionality can be helpful for quick discussions that are easier to handle face-to-face.

File Sharing. Slack allows users to share files by uploading them directly into a channel or a DM. Slack supports various types of files and integrates with services like Google Drive and Dropbox.

Figure A-5 Slack is a digital communication tool that provides channels for team collaboration, private messages, file sharing, and integrations with other workplace tools. It is often used in professional and organizational environments.
Source: Slack Technologies, LLC



Integration with Other Tools. One of Slack's notable features is its ability to integrate with many other tools and services, such as Google Workspace, Office 365, Trello, Asana, and GitHub. These integrations allow users to work with information from other applications within Slack.

Search. Slack offers powerful search capabilities, making it easy for users to find specific conversations or files.

Security. Slack is a secure platform with end-to-end encryption to protect user data. Slack also offers a variety of security features, such as two-factor authentication and data loss prevention.

Using Slack Effectively

By using Slack effectively, communications professionals can improve communication, collaboration, productivity, and customer service. This can lead to a more successful and efficient organization. Here are 10 things that communications professionals should know:

Set Ground Rules. At the beginning of your team's Slack usage, take some time to set ground rules for how the tool will be used. This might include respecting each other's time, keeping the discussion on track, and avoiding distractions.

Be Mindful of Your Audience. When communicating on Slack, be cognizant of your audience. This means using the right tone and language for the situation. It also means being respectful of people's time and attention.

Use Features. Slack offers a variety of features that can help you make your communications more effective. These features include channels, threads, and @mentions. Use these features to engage your audience and maximize your time.

Use Channels for Different Topics. Channels organize your conversations and keep your team on track. Create channels for different projects, teams, or interests. Don't try to place all discussions under a generic channel (e.g., "Misc.").

Use Threads to Keep Conversations Organized. Threads can help keep discussions on track. When you reply to a message, it creates a new thread. This helps to keep the conversation focused and prevents it from getting too long.

Use @Mentions to Get People's Attention. @mentions are a great way to get someone's attention. When you @mention someone, they will receive a notification. This is an effective approach for getting someone's input on a conversation or asking them a question.

Use Emojis to Add Personality to Your Messages. Emojis add character to your messages. They can express emotions, add humor, or make your messages more visually appealing.

Use the Search Bar to Find Information. The search bar is an efficient way to find information that has been shared on Slack. This can be helpful if you are looking for a specific file, message, or conversation.

Use the Help Center for More Information. The Slack help center is a valuable resource for learning how to use the platform. It includes articles, videos, and tutorials on a variety of topics.

Use Slack to Build Relationships. Slack is a good tool for building relationships with your team members, customers, and partners. Use it to get to know people better, share personal stories, and show appreciation for their work. Keep the following suggestions in mind when using Slack to build relationships:

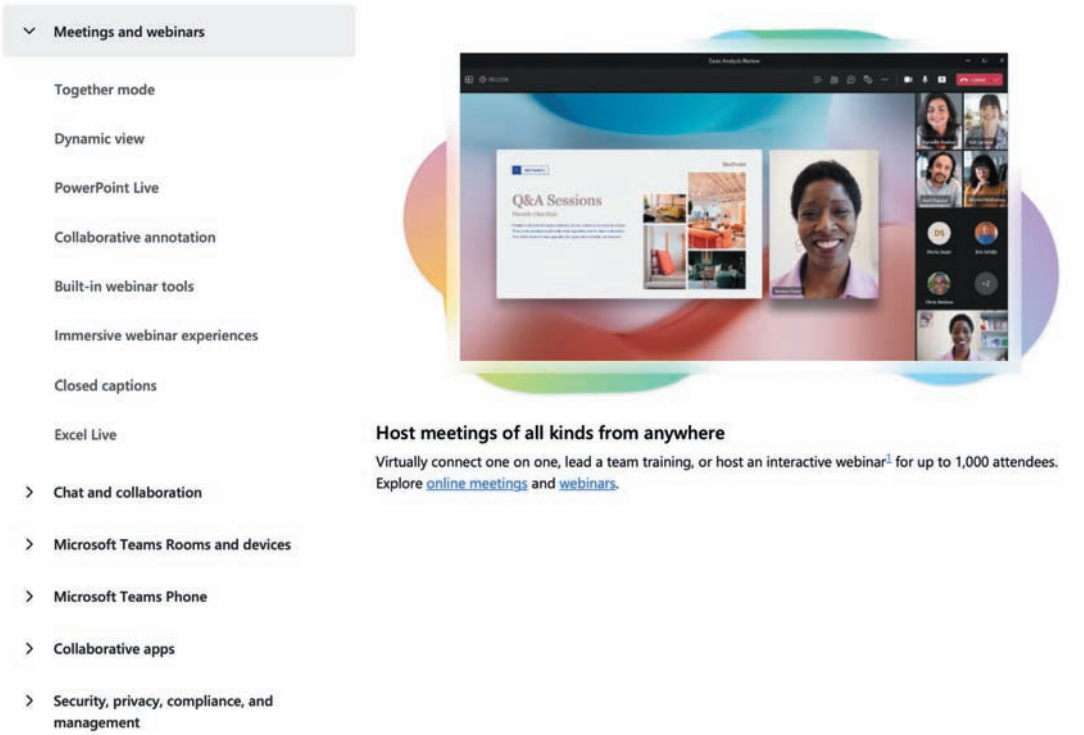
- **Be friendly, approachable, and positive:** When communicating on Slack, be friendly and approachable. This means using a warm and welcoming tone and being open to conversation. You should also be positive, which creates a more enjoyable and productive environment for everyone.
- **Be responsive:** When someone messages you on Slack, be responsive. This shows that you are interested in what they have to say and that you value their time.
- **Be helpful:** When someone asks you a question on Slack, be helpful. This shows that you are knowledgeable and willing to help others.

Following these tips, you can use Slack effectively to communicate with your team, customers, and other stakeholders.

A.5.3 Microsoft Teams

Microsoft Teams (Figure A-6) is a communication and collaboration platform that combines chat, video conferencing, file sharing, and productivity tools. It is built for hybrid work so distributed teams stay informed, organized, and connected online. Microsoft Teams is a popular tool for businesses of all sizes and individuals for personal and work-related purposes.

Figure A-6 Microsoft Teams is a collaboration platform that combines persistent chat rooms, video meetings, file storage, and application integration for businesses and educational institutions, all integrated with the Microsoft 365 suite of products.
Source: Microsoft Corporation



Features

Microsoft Teams is one of the core apps in an Office 365 subscription. It is available on all devices, including computers, laptops, tablets, and smartphones. Microsoft Teams is an excellent option for businesses of all sizes, offering a wide variety of features that include the following:

Chat. Microsoft Teams allows users to communicate with each other in real time, both one-on-one and in groups. Messages can be sent as text, images, or files.

Video Conferencing. Microsoft Teams allows users to have video conferences with each other. This can be helpful for meetings, presentations, or simply catching up with friends.

File Sharing. Microsoft Teams allows users to share files directly or through integration with other file-sharing services like Google Drive and Dropbox.

Tabs. Microsoft Teams allows users to create tabs for different types of content, such as files, apps, and conversations. This makes it easy to find the information you need and to stay organized.

Productivity Tools. Microsoft Teams integrates with other Microsoft Office applications like Word, Excel, and PowerPoint. This allows users to work on documents and presentations together in real time.

Bots. Microsoft Teams allows users to create and use bots. Bots are automated programs that can help users with tasks like scheduling meetings, finding information, and tracking progress.

Security. Microsoft Teams is a secure platform with end-to-end encryption to protect user data. Microsoft also offers a variety of security features, such as two-factor authentication and data loss prevention.

Using Microsoft Teams Effectively

To maximize the effectiveness of Microsoft Teams, all members must be familiar with its features and comfortable using it. Here are 10 tips for communications professionals to effectively use Microsoft Teams in systems projects:

Be Proactive with Communication. Use Microsoft Teams to regularly update your team members on project progress, upcoming deadlines, and any issues or changes. Make use of announcements in channels to highlight important news.

Use Channels Strategically. Create specific channels for different aspects of the system project. This will help to keep discussions organized, focused, and relevant.

Leverage Integrations. Make the most of Microsoft Teams' integrations with other tools, like Microsoft Planner for task management, SharePoint for document storage, or Power BI for data analysis.

Practice Consistent File Management. Use the Microsoft Teams file tab to share and co-edit documents. This keeps all relevant files in one place, accessible to all team members, and promotes collaboration.

Follow Video-Conferencing Etiquette. Encourage team members to use video during meetings to promote more engaged and interpersonal communication. Ensure everyone understands basic video-conferencing etiquette, like muting when not speaking.

Record Meetings. If team members can't attend a meeting, use the record feature in Teams. This way, those who missed the meeting can catch up, and attendees can revisit the discussions if needed.

Manage Tasks with Planner. Use Planner to assign tasks, set deadlines, and monitor progress. This helps to keep everyone accountable and provides a clear view of the project timeline.

Use Notifications Effectively. Use @mentions to direct attention to specific posts and encourage team members to set their notifications to stay updated.

Collaborate Online. Use Microsoft Teams' co-authoring and screen-sharing features to work together on tasks. This can improve efficiency and foster better collaboration.

Encourage Continuous Learning and Training. Encourage team members to familiarize themselves with Microsoft Teams' features and functionalities. Provide them with the necessary training. This ensures that everyone can make the most of the tool, enhancing overall productivity and collaboration.

These 10 tips enhance communications in Microsoft Teams by facilitating proactive and strategic communication, streamlining workflows through integrations and efficient file management, promoting collaborative work practices, ensuring all relevant information is captured and accessible, and enabling full platform utilization through continuous learning and training.

A.6 Soft Communication Skills

Project managers working on systems projects need strong technical communication skills. However, they must also address their soft communication skills to succeed and enjoy career advancement. These include negotiation and persuasion, facilitation, emotional intelligence, nonverbal communication, and feedback.

A.6.1 Negotiation and Persuasion

Negotiation and persuasion are crucial skills that help project managers achieve consensus and ensure the project advances in alignment with its objectives. This skill is particularly relevant when handling resource allocation, where managers must often negotiate to secure the necessary resources. For example, they may need to persuade senior management of the need for additional funding, personnel, or equipment, or negotiate with team members on task distribution or deadlines.

Moreover, conflicts are bound to arise in any project, and negotiation skills are vital in resolving them effectively. Project managers should be able to facilitate discussions between conflicting parties, guiding them toward a resolution that satisfies all parties involved. This also helps in maintaining a harmonious working environment conducive to productivity.

Persuasion is also instrumental when interacting with stakeholders. Not all stakeholders will initially agree with the project's direction or specific decisions. Thus, the project manager must be capable of articulating the benefits and reasoning clearly and persuasively to garner support and buy-in.

A.6.2 Facilitation Skills

Facilitation skills are vital for project managers as they frequently conduct meetings, brainstorming sessions, and discussions. This skill set ensures that these gatherings are productive, inclusive, and result oriented. Project managers need to guide the discussion, maintain the focus, and ensure that all participants have an opportunity to contribute their ideas and opinions.

During brainstorming or problem-solving sessions, facilitation skills help the project manager to encourage creative thinking and stimulate constructive discussions. In addition, project managers must harness their team's collective knowledge and expertise, leading them toward solutions and ideas that can benefit the project.

Finally, facilitation skills also play a significant role in decision-making processes. Project managers must gather and synthesize input from various team members and stakeholders, guiding the group toward a consensus. They must ensure everyone feels heard, their opinions are valued, and decisions are reached fairly and transparently.

A.6.3 Emotional Intelligence

Emotional intelligence (EI) refers to the ability to understand, interpret, and respond appropriately to one's own emotions and the emotions of others. For project managers, this is crucial as it plays a significant role in managing relationships, team dynamics, and conflicts. Therefore, they need to be aware of their emotional reactions and be able to manage them effectively to maintain professionalism and leadership effectiveness.

Empathy, a component of EI, is particularly valuable. When project managers empathize with their team members, they can better understand their team's perspectives, concerns, and motivations. This enables them to provide better support, motivate effectively, and foster a more positive, inclusive, and productive work environment.

Furthermore, EI is instrumental in navigating difficult conversations, such as providing constructive criticism or addressing issues. Being sensitive to others' emotional responses allows the project manager to communicate in a way that minimizes adverse reactions and promotes openness to feedback. Thus, EI contributes significantly to the overall success of a systems project.

A.6.4 Nonverbal Communication

Nonverbal communication includes facial expressions, body language, gestures, and tone of voice. These cues often convey more information than words themselves. For project managers, being aware of and accurately interpreting these signals can enhance understanding and communication with team members and stakeholders.

For example, suppose a project manager notices a team member needing clarification during a meeting. In that case, they can clarify or re-explain a point, even if the team member hasn't verbally expressed their confusion. This proactive response can prevent misunderstandings and ensure everyone is on the same page.

Project managers should also be aware of the nonverbal cues they are sending. For example, maintaining eye contact, using open body language, and modulating tone of voice can show engagement, confidence, and respect, fostering open and effective communication.

A.6.5 Feedback Skills

Giving and receiving feedback is an essential part of any project. Constructive feedback helps team members improve performance, correct mistakes, and grow professionally. As a project manager, delivering constructive, balanced, and respectful feedback can lead to more effective improvements and maintain a positive team atmosphere.

Receiving feedback is equally important. Project managers should be open to receiving feedback from their team members and stakeholders. This can provide valuable insights into their management style, the team's perception of the project, and areas for improvement.

Moreover, fostering a culture of feedback within the team can enhance communication, improve work processes, and drive the project's success. When everyone feels comfortable expressing their thoughts and suggestions, issues can be identified and addressed more promptly and innovative ideas can be discussed and implemented.

A.7 Resources to Help Manage Your Communication Skills

More than ever, systems analysts must rely on their skills and experience. In an uncertain world and a turbulent economy, individuals should think of themselves as profit-making companies with assets, liabilities, strengths, and areas for development. Earlier in the book, you learned that a company must have a strategic plan, which is also true for an individual. Armed with a plan to improve your communication skills, you will likely reach your full potential.

Communicating is like any other activity—the more you practice, the better you become. Many resources are available for students and IT professionals who want to improve their written and oral communication skills. For example, online learning websites such as Udemy (Figure A-7) offer a wide range of free and for-fee courses to help you become a better communicator. Writer's Digest (Figure A-8) offers several online resources to help you learn how to write better nonfiction. In addition, numerous writers' groups, such as the Florida Writers Association (FWA), shown in Figure A-9, provide a nurturing environment to help you hone your writing skills.

Regarding oral communication, some people find standing in front of a group and delivering a presentation or report difficult. For many years, membership in Toastmasters International has been a popular way to gain confidence, overcome stage fright, and develop public speaking skills. As shown in Figure A-10, Toastmasters offers a friendly environment where members positively critique each speech, note the strengths, and offer suggestions about what might be improved. As a result, the organization provides an excellent way to develop better public speaking skills.

Figure A-7 Udemy is an online learning platform offering courses on various topics, taught by experts and enthusiasts to individuals seeking to improve professional skills or pursue a personal interest.

Source: Udemy, Inc.

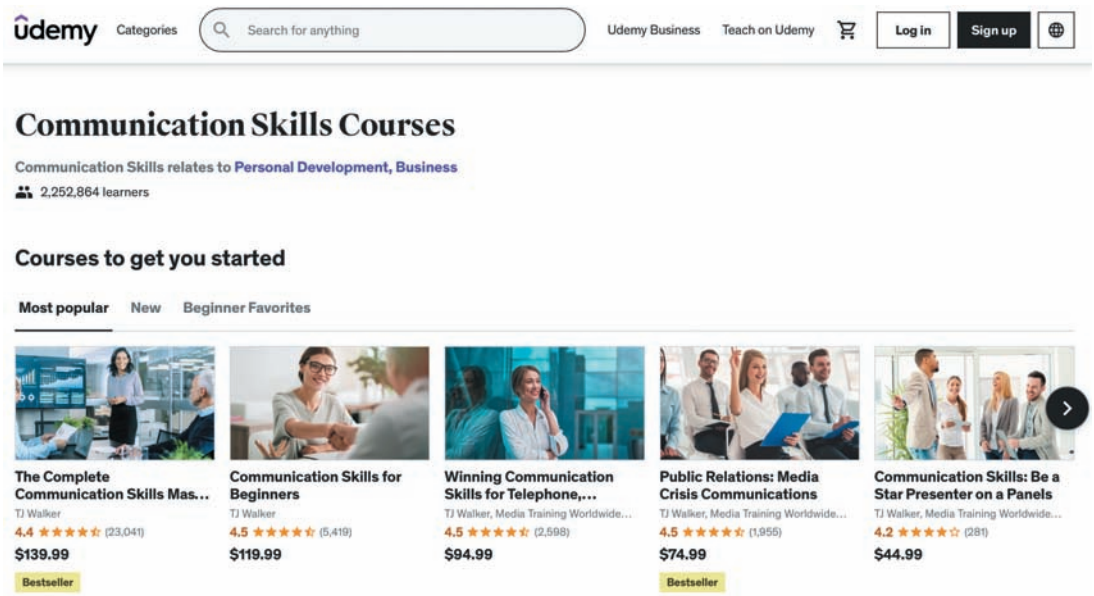
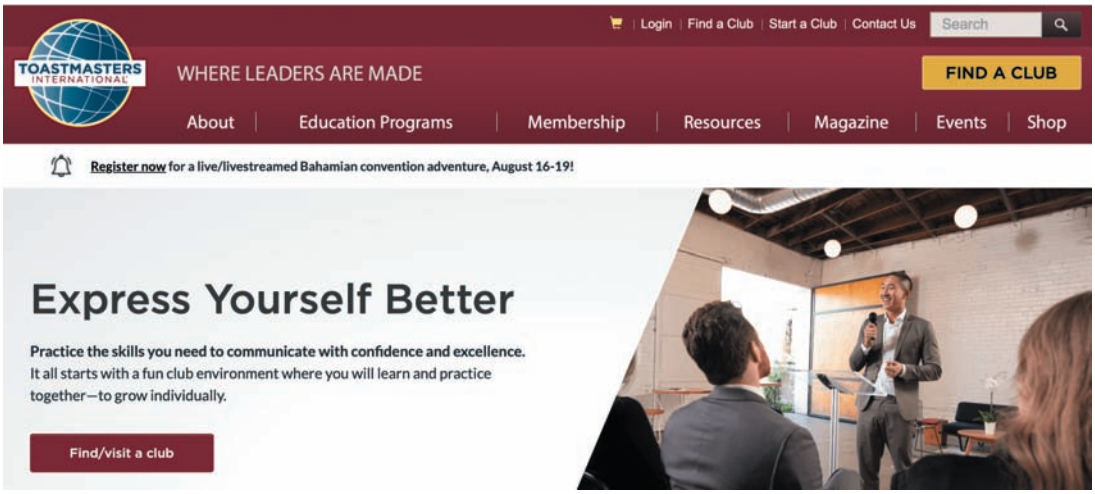


Figure A-8 Writer's Digest has been helping writers improve their craft, achieve their goals, and recognize their dreams since 1920. They offer a website, a magazine, and an online university.
Source: Active Interest Media

Figure A-9 Florida Writers Association (FWA) is a nonprofit organization serving writers of all genres and all levels. It provides a professional atmosphere, a vast networking system, and valuable tools to succeed in today's publishing industry. The FWA motto is "Writers Helping Writers."
Source: Florida Writers Association

Figure A-10 Toastmasters is one of the oldest organizations dedicated to helping individuals become more effective communicators.

Source: Toastmasters International



Appendix

B

Finance

Appendix B helps the systems analyst understand why the financial implications of a project are paramount. This begins with exploring costs and benefits, which provide a foundational overview of the expenses associated with a project and the potential gains it promises. Properly comprehending this balance allows stakeholders to set realistic expectations and budget effectively for both the short and long term.

Classifying costs offers insights into the various expenses involved in a systems project, ensuring that every financial aspect is accounted for. Learning how information systems costs and charges are managed emphasizes the importance of efficiently overseeing the monetary aspects of the system, ensuring optimal allocation and usage of resources. Conversely, classifying benefits delves into categorizing potential advantages a project may bring, from tangible to intangible and direct to indirect gains, highlighting the holistic value a system can offer.

Various financial analysis tools are employed to determine the viability and value of projects. Payback analysis helps calculate the time it takes for a project to recoup its initial investment. Return on investment (ROI) analysis evaluates the profitability of a project relative to its cost, giving stakeholders a clear picture of potential returns. Lastly, present value analysis provides an understanding of the current value of future cash flows, assisting decision makers in understanding the long-term financial implications of their choices.

B.1 Costs and Benefits

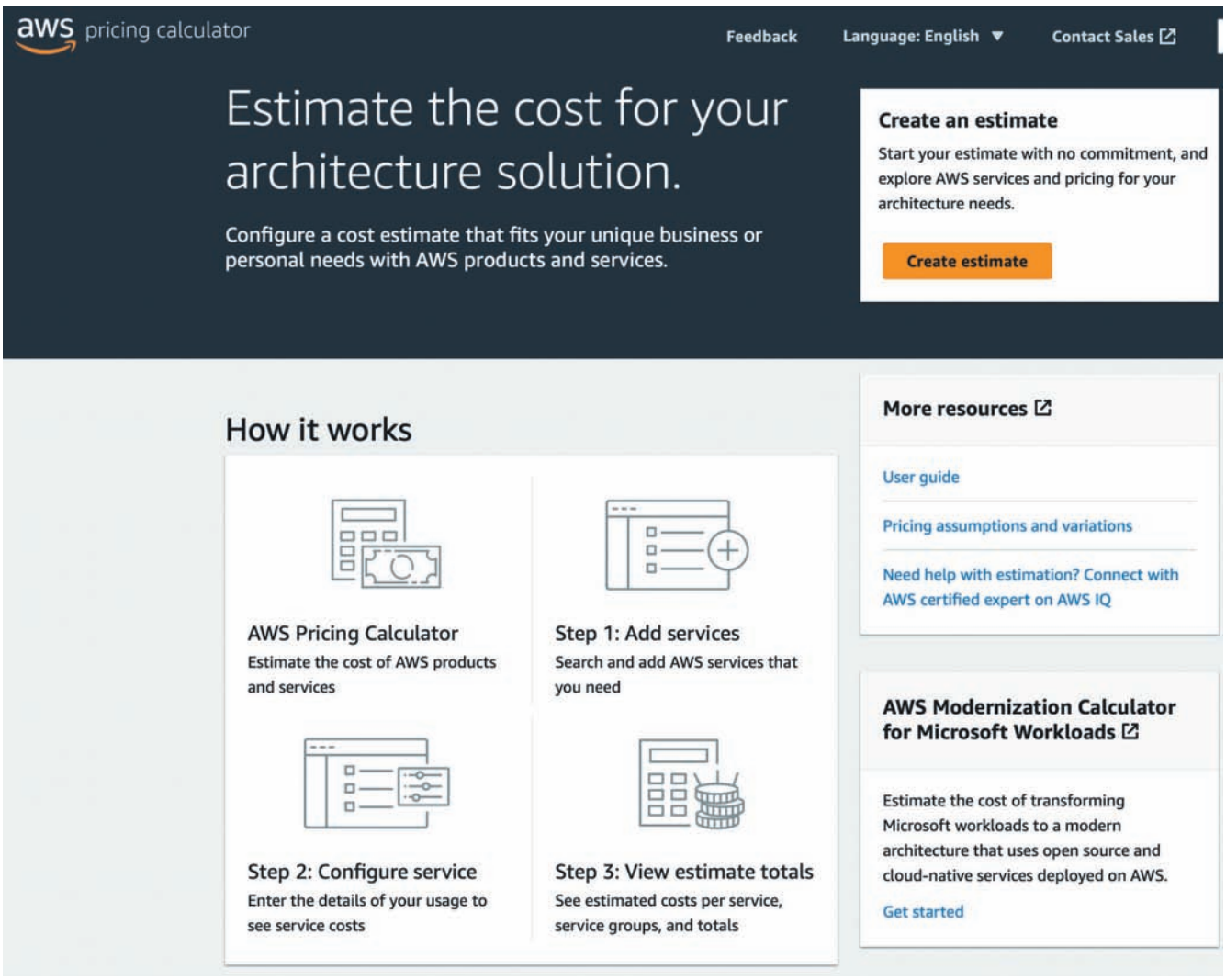
A systems analyst must review a project's costs and benefits at the end of each SDLC phase so management can decide whether to continue the project. Analysts must understand how to identify and classify all costs and benefits before the economic analysis tools described in this appendix can be used.

Cost–benefit analysis is the process of comparing the anticipated costs of an information system to the benefits expected. Cost–benefit analysis is performed throughout the SDLC to determine a project's economic feasibility and compare alternative solutions. Many cost–benefit analysis techniques exist. This appendix covers three standard methods: payback, ROI, and present value analysis. Each approach analyzes cost–benefit figures differently, but the objective is to provide reliable decision-making information.

As described earlier in the book, economic feasibility means that the projected benefits of the proposed system outweigh the projected costs. When economic feasibility has been determined, the project's advantages compared to the project's total cost of ownership (TCO) must be considered, including ongoing support, maintenance, and acquisition costs.

Figure B-1 shows a TCO calculator that compares the costs of running applications on-premises or in the cloud using the Amazon Web Services infrastructure. Because migrating to the cloud is complex, such calculators can help the systems analyst make informed decisions. Amazon.com, Google, IBM, Microsoft, and other vendors provide similar TCO guidance related to the cloud and other large-scale IT projects.

Figure B-1 Amazon.com offers a calculator to estimate the cost of running applications in the cloud using AWS products and services.
Source: Amazon Web Services, Inc.



B.2 Classifying Costs

Costs can be classified as tangible or intangible, direct or indirect, fixed or variable, and developmental or operational. There are also cross-category costs that apply to more than one area of expenses. Classifying costs in systems projects is not just a financial best practice but is crucial for several reasons:

- **Effective budgeting and financial planning.** Proper classification enables project managers to allocate funds accurately, ensuring that each aspect of the project is adequately financed. This helps prevent underfunding or overfunding specific areas, which can lead to inefficiencies or resource shortages.
- **Transparency and accountability.** Categorized costs provide stakeholders, including team members, sponsors, and clients, a transparent view of where funds are directed. This transparency fosters trust and ensures that funds are used responsibly and for their intended purposes.
- **Cost control and management.** Understanding different types of costs allows for better cost management strategies. For instance, while fixed costs may not be easily altered, variable costs can often be adjusted based on project needs, allowing for more flexible cost control measures.
- **Performance evaluation.** Classifying costs aids in evaluating the performance and profitability of a project. By comparing actual costs to budgeted amounts in each category, managers can identify areas where the project is overperforming or underperforming financially.
- **Informed decision making.** With a clear understanding of the various costs, project leaders can make more informed decisions, whether about scaling a project, determining client pricing, or deciding on the viability of continuing a project.
- **Risk management.** By recognizing and categorizing potential costs, especially unforeseen or hidden costs, project managers can better anticipate risks and set aside contingencies, ensuring the project remains financially stable.
- **Enhanced communication.** When costs are categorized, it's easier for team members across different departments or areas of expertise to communicate about the financial aspects of the project. This clarity can lead to better collaboration and more efficient problem solving.
- **Future project benchmarking.** Correctly classified cost structures from past projects can serve as valuable benchmarks for future endeavors. Historical data on categorized costs helps analysts make more accurate predictions and financial plans for upcoming projects.
- **Compliance and reporting.** In some organizations or industries, financial reporting standards or contractual obligations may require detailed breakdowns of costs. Proper classification ensures compliance with such requirements.
- **Holistic project understanding.** Classifying costs ensures a comprehensive understanding of the project's financial landscape. It allows stakeholders to grasp the total expenditure and the nuances of where and why funds are allocated in specific ways.

B.2.1 Tangible Costs

Tangible costs refer to the explicit and quantifiable financial expenditures incurred during a system's development, implementation, and operation. These costs have a direct monetary value and are readily measurable, leaving no room for ambiguity or interpretation. Tangible costs are concrete expenses that can be identified, measured, and recorded. Within a systems project, they form the foundation of the budgeting process and play a crucial role in financial analyses, projections, and decision making.

Some of the most prevalent examples of tangible costs in systems projects include the procurement of hardware and software, wages or salaries of the project team, expenses related to infrastructure such as servers or networking equipment, and licensing fees for proprietary software or tools. Other tangible costs may encompass training expenses for staff, costs of third-party services or consultants directly engaged in the project, and expenditures for materials or resources that are physically consumed during the project. Tangible costs also include the interest charges firms must pay when borrowing money for working capital or financing new projects. In times of high interest rates, these costs can be significant and must be considered. Each cost comes with a clear, associated price tag that can be accounted for in the project's financial records.

Accurately identifying and accounting for tangible costs is pivotal to the successful financial management of systems projects. A clear understanding of these costs ensures that budgets are realistic, financial forecasts are accurate, and there's a precise evaluation of the project's ROI. It helps stakeholders, including project managers and financiers, make informed decisions, manage resources efficiently, and measure the project's financial performance against its objectives. Misjudging or neglecting tangible costs can lead to financial miscalculations, potentially resulting in budget overruns or project failures. As such, meticulous tracking and management of these costs are essential throughout the project's life cycle.

B.2.2 Intangible Costs

Intangible costs refer to those expenses that, while real and impactful, are not immediately quantifiable in monetary terms. These costs lack a concrete or direct monetary value but can influence a project's overall success and value perception. Intangible costs can be abstract, often stemming from missed opportunities, reputational implications, or other nonquantifiable consequences. When assessing a systems project's total cost or impact, it's essential to consider both the tangible and intangible elements.

Several aspects of a systems project can lead to intangible costs. For instance, a decline in employee morale or satisfaction due to extensive overtime or challenging project conditions might lead to decreased productivity or higher turnover, both of which have indirect cost implications. Delays in project delivery might tarnish an organization's reputation, affecting its ability to secure future business. Similarly, a poorly executed project could lead to customer dissatisfaction, impacting loyalty and potential future sales. While these costs don't have a direct monetary value assigned, like purchasing software or hardware, they can influence a project's overall profitability and success.

Recognizing and accounting for intangible costs is crucial for a holistic understanding of a project's impact and potential risks. While these costs might not be evident on a balance sheet, they can profoundly influence the long-term success and viability of a project and an organization. Intangible costs can indicate underlying issues, such as inadequate training, poor project management, or misaligned project goals. Addressing these root causes early can mitigate potential adverse outcomes. Moreover, stakeholders can make more informed decisions by considering intangible costs in project evaluations, ensuring immediate and long-term implications are factored into project strategies and actions.

B.2.3 Direct Costs

Direct costs refer to the expenses specifically and directly associated with developing, implementing, and maintaining a systems project. These costs can be directly traced to a particular project without any ambiguity. Direct costs are a straightforward metric in budgeting and financial evaluation of a project's viability. In systems projects, these costs are immediately attributable to the project's life cycle and can be allocated accurately.

In the context of systems projects, several expenditures fall under direct costs. Examples include salaries of team members (such as systems analysts, designers, and developers) dedicated to the project, costs of specific hardware or software procured, and expenses related

to project-specific training or consultancy. Additionally, licensing fees for software tools used solely for the project, payments for third-party services explicitly contracted for the project, and costs of materials or resources used directly in the development or deployment of the system are considered direct costs.

Grasping the concept and components of direct costs is vital for system project stakeholders. A clear understanding aids in creating accurate budgets, forecasts, and financial models. It helps assess the project's financial feasibility and ensure the resources are appropriately allocated. By monitoring direct costs, project managers can control expenditures, make informed decisions about resource allocation, and gauge the project's financial health throughout its life cycle. Overlooking or underestimating direct costs can lead to budget overruns, making it essential to account for them meticulously in project planning and execution.

B.2.4 Indirect Costs

Indirect costs encompass those expenses that are not directly tied to a specific project but are spread across multiple projects or general operations of an organization. These costs support the execution of projects but cannot be attributed wholly or exclusively to one specific project. Their diffuse nature contrasts with direct costs, which are straightforwardly traceable to individual projects. In systems project budgeting and accounting, indirect costs pose a challenge due to their less tangible linkage to specific projects, necessitating allocation methods to distribute these costs among various initiatives.

Several expenditures qualify as indirect costs. The salaries of network administrators, copy machine rentals, and insurance expenses are examples of indirect costs. Other common examples include utilities such as electricity used in the office, general office supplies, or shared software licenses used by multiple projects or teams. Salaries or wages of personnel who play roles in various projects, like management or administrative staff, also come under indirect costs. Additionally, overhead such as rent or lease payments for office space, shared facilities or equipment maintenance, and organization-wide training programs can be counted as indirect costs. Even though they don't trace back to a single project, they are essential for the general operation and success of all projects within an organization.

Recognizing and accounting for indirect costs are vital for an organization's financial clarity and successful budgeting and management of systems projects. Properly allocated indirect costs ensure that projects bear their fair share of the overarching operational costs, providing a more comprehensive picture of a project's actual cost and profitability. Overlooking or improperly distributing indirect costs can result in misleading profitability metrics and distort the project's financial landscape. As such, accurate allocation methods are employed to spread these costs judiciously among various projects, ensuring fair financial assessment and strategic decision making.

B.2.5 Fixed Costs

Fixed costs refer to expenses that remain constant regardless of the volume of work or production. These costs do not vary with the project's intensity, duration, or scale. They are predetermined and are typically contracted or committed fees that an organization must pay, irrespective of any fluctuations in its operational activity. In the financial planning of systems projects, understanding fixed costs is essential to ensure that foundational expenses are covered, making it possible to assess the project's profitability and economic viability accurately.

Common examples of fixed costs in systems projects include the salaries of permanent staff, monthly or annual software licensing fees, and lease or rental payments for office spaces and hardware equipment. Costs such as regular maintenance contracts, recurring subscriptions to cloud services, and annual insurance premiums for professional indemnity also fall into this category. Even if the project scales up or down, these costs typically remain unchanged, at least for the contract's duration or predetermined period.

Mastering the concept of fixed costs is vital for efficient budgeting and financial planning in systems projects. A clear understanding ensures the project can cover its baseline expenses, even before factoring in variable or direct costs associated with specific project activities. Recognizing fixed costs aids in calculating break-even points, determining pricing strategies, and assessing the overall financial health of a project. Overlooking or underestimating fixed costs can jeopardize the financial stability of a project, leading to potential cash flow issues. Consequently, a meticulous account of these costs is imperative for strategic planning, risk assessment, and sustainable project management.

B.2.6 Variable Costs

Variable costs refer to those expenses that fluctuate directly with the volume or scale of project activity. These costs are directly proportional to the project's activity, meaning they increase as the workload or scale of the project grows and decrease when the activity levels drop. In financial planning and budgeting for systems projects, accounting for variable costs is critical for understanding the dynamic nature of project expenses.

The costs of printer paper, supplies, and phone charges are variable. Other expenditures can be classified as variable costs, including hourly wages for temporary or contract staff, cloud computing charges based on usage, or expenses for materials and resources consumed based on project activities. Costs for additional software licenses procured based on team size, expenses related to data storage that scales with project requirements, and fees for third-party services based on specific project phases are also variable. Any cost that changes with the project's operational volume or intensity falls into this category.

A thorough understanding of variable costs allows project managers and stakeholders to anticipate how project scale or scope changes will impact the overall budget. This insight is invaluable for making informed decisions about scaling up or down, allocating resources, and managing cash flows. Furthermore, understanding variable costs aids in setting pricing strategies for client projects, estimating ROI, and conducting cost–benefit analyses. Given their fluctuating nature, monitoring and managing variable costs are vital to ensure the project remains financially viable and within the allocated budget.

B.2.7 Developmental Costs

Developmental costs are incurred during a project's research, design, and development (or acquisition) phases. These costs are associated with creating and refining a new system or enhancing one. Before a system is operational or deployed to end users, a substantial amount of investment often goes into its development, and these expenditures make up the developmental costs. They are pivotal in project budgeting, capturing the foundational investments necessary to bring a system to fruition.

Some prominent examples of developmental costs include salaries and wages of the development team, which may comprise systems analysts, designers, programmers, and testers. Costs associated with prototyping, purchasing, or licensing developmental software tools, and acquiring hardware for testing and development, are also included. Additionally, expenditures related to initial research, feasibility studies, design consultations, and user testing sessions fall into this category. Training for the development team, costs associated with iterations based on feedback, and expenses for documentation during the development phase are also considered developmental costs.

Accurately identifying and budgeting for developmental costs is crucial for the successful financial management of systems projects. These costs give stakeholders a clear picture of the initial investment required to develop a system or solution. Understanding these costs aids in evaluating the financial feasibility of a project, securing adequate funding, and setting expectations for ROI. It also helps benchmark and compare similar projects, aiding decision-making processes about project continuation, alteration, or termination. Neglecting or underestimating developmental costs can lead to budget shortfalls, project delays, or

compromised quality, emphasizing the need for meticulous planning and accounting during the development phase.

B.2.8 Operational Costs

Operational costs pertain to the expenses incurred to maintain, support, and run a system once it has been developed and deployed. These are the ongoing costs associated with the system's day-to-day operations, ensuring its smooth functioning and addressing any issues that arise post-deployment. Unlike developmental costs, which are primarily front-loaded during the creation phase of a system, operational costs persist throughout the system's life cycle. For any organization or project manager, understanding these costs is essential to gauge the long-term financial implications of a system.

When it comes to systems projects, several expenses qualify as operational costs. These can include costs associated with hosting services, data storage, and backup services. Salaries and wages of support staff, maintenance technicians, and system administrators also fall under this category. Licensing fees for software that require periodic renewal, regular software updates and patches, and user training and retraining expenses are operational. Other operational costs encompass routine maintenance contracts, fees for troubleshooting and resolving issues, and payments linked to system enhancements or upgrades over time.

Recognizing and accurately budgeting for operational costs is foundational for the sustainable management of systems projects. A comprehensive understanding of these costs allows organizations to anticipate the long-term financial commitments of maintaining and supporting a system. Such foresight is invaluable for making informed decisions about end-user pricing, allocating annual budgets, and planning system upgrades or replacements. Moreover, understanding operational costs can highlight areas for potential cost savings or efficiencies, such as transitioning to a more cost-effective hosting solution or renegotiating maintenance contracts. Overlooking or misjudging these costs can strain financial resources and compromise system reliability, underscoring the importance of their thorough evaluation and management.

B.2.9 Cross-Category Costs

Cross-category costs encompass those expenses that may straddle multiple categories or phases of a project. Unlike costs strictly defined as developmental, operational, or another specific type, these costs have elements that can apply to more than one category. Their nature makes them versatile or multifaceted, requiring careful consideration during budgeting and financial analysis to ensure they're appropriately allocated across the project's life cycle.

Examples of cross-category costs in systems projects include the following:

- **Training costs.** While initial training for the development team might be considered a developmental cost, ongoing training sessions for users or refresher courses for the group can span developmental and operational phases.
- **Software licenses.** Some software licenses might have an initial acquisition cost (developmental) and recurring renewal fees (operational).
- **Consultation fees.** External consultants might be engaged during the development phase to provide expertise and then re-engaged during the operational phase for system evaluations or enhancements.
- **Infrastructure costs.** While the initial setup of a server might be a developmental cost, the ongoing maintenance, upgrade, and energy costs can span both developmental and operational phases. A monthly fee for maintaining the company's website would be regarded as operational, fixed, and indirect costs.
- **Quality assurance and testing.** Initial testing is developmental, but periodic testing and quality checks during the system's life cycle can fall into multiple categories.

Recognizing and adequately accounting for cross-category costs is crucial for the holistic financial management of systems projects. By understanding the multifaceted nature of these costs, organizations can more accurately allocate funds across different project phases and categories. This clarity ensures that budgets are well-balanced and financial projections are more accurate, avoiding potential pitfalls or shortfalls in any specific category. Properly addressing cross-category costs also facilitates more transparent communication with stakeholders about the nature of expenses and the ongoing financial commitments of a project. Given their overlap across categories, it's vital to continuously monitor and adjust allocations for these costs as the project evolves and progresses.

B.3 Managing Information Systems Costs and Charges

Managing information systems costs and charges is paramount for several reasons, many of which stem from the critical role that such systems play in modern organizations. It's a linchpin for operational success, financial health, strategic growth, and stakeholder confidence. Therefore, systems analysts need to understand direct costs, indirect costs, and methods of allocating IT charges within the company. Some of the reasons why managing these costs and expenses is essential include:

- **Ensuring ROI.** Information systems often require significant hardware, software, and human resources investment. Proper management ensures the organization achieves a favorable ROI by deriving maximum value from the system relative to its cost.
- **Budgetary control.** Costs related to information systems can quickly escalate if not adequately managed. Regular oversight of expenses ensures that expenditures align with fiscal constraints and prevent unexpected financial strains.
- **Optimal resource allocation.** Properly managed costs enable organizations to allocate resources effectively. This ensures that funds are directed to the most critical areas, leading to optimized system performance and utility.
- **Cost transparency.** When costs and charges are meticulously managed, it provides a clear view of where funds are allocated, promoting transparency. This benefits internal stakeholders, who may need to justify expenses, and external stakeholders, such as clients or partners, who seek clarity on billing.
- **Competitive advantage.** Efficiently managing information systems costs can lead to cost savings, which, when passed on to consumers or reinvested in innovation, can offer a competitive edge in the market.
- **Predictability.** Regularly monitoring and managing information systems costs makes financial outcomes more predictable, which is crucial for long-term planning and decision making.
- **Operational efficiency.** Overruns or unplanned expenditures can disrupt the normal functioning of information systems, potentially hindering operations. Proper cost management ensures continuous, efficient system operation.
- **Future planning.** Historical data on managed information systems costs is a valuable benchmark for future projects or system upgrades, aiding in more accurate budgeting and financial planning.
- **Stakeholder trust.** Properly managing and transparently reporting on information systems costs and charges builds trust among stakeholders, from board members and investors to employees and clients. Trust is especially crucial if the organization secures funding for future IT endeavors.
- **Compliance.** In some sectors, especially those heavily regulated, there may be standards or regulations governing the accounting and reporting of information systems costs. Proper management ensures compliance, minimizing legal or contractual risks.

B.3.1 Chargebacks

A chargeback method is a technique that uses accounting entries to allocate the indirect costs of running the IT department. It is a cost allocation approach in which internal departments or users are charged for the IT resources, services, or infrastructure they consume. Essentially, it's a way for an organization's IT department to bill other departments or business units for the specific services they use. The chargeback method provides benefits and challenges.

Benefits

- **Transparency.** Chargeback systems provide clear visibility into how IT resources are consumed across the organization. It allows departments to see precisely what they are being charged for, fostering a better understanding of IT costs.
- **Cost control and accountability.** When departments are directly billed for their IT consumption, they are more likely to use resources judiciously and minimize waste. It promotes a sense of responsibility and encourages efficient use.
- **Predictable budgeting.** The chargeback method can help budget forecasting as departments become more aware of their consumption patterns and can better predict future expenses.
- **Funding IT initiatives.** By charging for services, the IT department can ensure a steady stream of revenue that can be reinvested in infrastructure, services, or innovation.
- **Fair allocation.** Instead of a flat fee or an arbitrary division, the chargeback method ensures that departments pay for what they use, leading to a more equitable distribution of costs.

Challenges

- **Complexity.** Implementing a chargeback system can be complex, especially in large organizations with diverse IT needs. Determining accurate pricing for each service and tracking usage can be challenging.
- **Potential for conflict.** Introducing internal charges can sometimes lead to disputes or disagreements between the IT department and other business units over rates, usage metrics, or the value derived from certain services.
- **Administrative overhead.** Managing a chargeback system requires administrative effort, from billing to reconciliations to addressing discrepancies.
- **Potential for suboptimal resource use.** Departments might underuse IT resources to reduce charges or avoid leveraging beneficial but costly services.
- **Organizational resistance and politics.** Chargeback methods can amplify corporate politics as departments might feel unfairly burdened by allocated costs, leading to internal disputes. Without transparency, this system can foster mistrust and strain interdepartmental relationships.

Methods

Chargeback methods provide frameworks for allocating IT costs to internal departments or business units based on their usage of IT services. Organizations adopt various chargeback methods based on their needs, size, and organizational culture. Some might have a detailed and granular system that charges for every little service, whereas others might adopt a more aggregated approach. The key is to balance cost transparency and accountability without fostering a restrictive or adversarial environment.

Common chargeback methods include:

- **No charge.** Some organizations treat indirect expenses from the information systems department as a necessary cost of doing business, and IT services are seen as benefiting

the entire company. Thus, indirect IT department costs are treated as general organizational costs and are not charged to other departments. The information systems department is a cost center because it generates accounting charges without offsetting IT service credits.

- **Flat rate.** Each department or business unit pays a fixed amount, regardless of usage. This method is simple to administer but might not reflect the actual consumption of IT resources by each department.
- **Direct actual usage.** Departments are charged based on their direct consumption of IT resources. For instance, if a department uses 20% of storage space on a server, it would be billed for 20% of that server's cost.
- **User-based chargeback.** Like direct actual usage, this method involves charging departments based on the number of users accessing or utilizing the IT services. It's a straightforward approach in which the cost is allocated proportionally to the size of the user base in each department, making it particularly suitable for services like software licenses or application usage, where costs can be directly linked to the number of users.
- **Tiered or service level-based.** IT services are categorized into tiers based on service levels or quality. Departments select and pay for a specific tier or level of service. For instance, a higher tier might offer faster response times or more extensive support.
- **Allocated fixed and variable.** Fixed costs of IT (like salaries or infrastructure maintenance) are divided equally among departments. In contrast, variable costs (like server usage or cloud storage) are charged based on actual use.
- **Market price.** This method involves charging departments based on what similar IT services would cost in the open market. It helps departments understand the value of IT services in terms of market dynamics.
- **Negotiated or budgeted.** At the beginning of a fiscal period, the IT department and other business units negotiate and agree upon a fixed charge for the expected use of IT services. This method provides predictability in costs.
- **ROI-based.** Charges are determined based on the expected ROI from deploying specific IT resources or projects. It aligns IT expenses with desired business outcomes.
- **Hybrid.** Many organizations use a combination of the other methods mentioned in this list to capture the nuances of their operations and achieve a balanced approach to chargeback.

Each chargeback method has pros and cons and suits different organizational structures, cultures, and goals. Selecting the correct method requires understanding the organization's objectives, the nature of IT services rendered, the intricacies of interdepartmental relations, and the desired outcomes from the chargeback system.

B.4 Classifying Benefits

Classifying benefits in systems projects is as crucial as classifying costs. It provides a structured way to understand, measure, and communicate a project's value to an organization. Some of the reasons why classifying benefits is essential include:

- **Informed decision making.** Categorized benefits help stakeholders make informed decisions about project initiation, continuation, scaling, or termination based on tangible and intangible gains.
- **ROI calculation.** ROI is a critical metric in project evaluation. Classifying benefits ensures that all returns, whether direct financial gains, efficiency improvements, or qualitative enhancements, are accounted for in ROI calculations.

- **Stakeholder communication.** Different stakeholders might prioritize different types of benefits. For instance, a CFO might be most interested in direct financial gains, while a sales manager might value customer satisfaction improvements. Classifying benefits allows for tailored communication with various stakeholders.
- **Project prioritization.** Organizations often need more resources to juggle multiple potential projects. A precise classification of benefits helps prioritize projects that align best with organizational objectives.
- **Performance measurement.** Organizations can set specific performance indicators for each category by categorizing benefits, allowing for more granular and effective project performance measurement and management.
- **Risks and opportunity analysis.** Classified benefits can shed light on potential risks (if expected benefits are not realized) and additional opportunities (if certain benefits can be maximized further).
- **Enhanced motivation and morale.** It can boost motivation and morale when project teams see their efforts' varied benefits, especially in areas they care about (e.g., improving user experience or environmental impact).
- **Future benchmarking.** Historical data on classified benefits from past projects can serve as valuable benchmarks for future initiatives, aiding in more accurate planning and expectation setting.
- **Strategic alignment.** Classifying benefits ensures projects align with broader organizational goals and strategies. For example, if an organization's strategy emphasizes customer focus, projects that yield benefits in customer satisfaction and engagement might be prioritized.
- **Compliance and reporting.** Reporting on benefits may be mandatory for projects requiring external funding or under regulatory scrutiny. Having a transparent classification system aids in comprehensive and compliant reporting.

Classifying benefits ensures a structured, transparent, and comprehensive approach to understanding the value projects bring to an organization. It aids in strategic alignment, decision making, and effective communication, enhancing project management. In systems projects, benefits can be broadly classified based on different dimensions, depending on the project's context and the organization's priorities. Common benefit classifications are described in the following sections.

B.4.1 Tangible versus Intangible

Tangible benefits are directly measurable and quantifiable advantages that an organization reaps from a project or initiative. These benefits have a clear, identifiable impact on the organization's bottom line and are usually represented in monetary terms or specific metrics. Tangible benefits include cost savings from reduced manual processing, increased revenue from a new e-commerce platform, or decreased operational expenses due to automation. The immediacy of these benefits makes them particularly appealing when justifying a project, as they can provide a clear ROI in a relatively short time frame. They serve as concrete evidence of a project's success and are often used to gauge the initial effectiveness of an initiative.

On the other hand, intangible benefits are the nonquantifiable outcomes of a project, offering value that's often qualitative and subjective. While they might not be immediately reflected in the financial statements or performance metrics, they hold intrinsic worth for an organization. Intangible benefits include improved employee morale due to more user-friendly software, enhanced customer satisfaction from better service, or a more robust organizational culture with better collaboration tools. Although elusive in the short term, these benefits can lead to significant long-term advantages. Over time, the ripple effects of positive intangibles can even manifest as tangible benefits, like increased customer loyalty translating into repeated sales.

When evaluating systems projects, both tangible and intangible benefits deserve consideration. While tangible benefits offer a compelling snapshot of immediate value, intangibles paint a picture of the project's broader impact. Stakeholders might be naturally inclined toward tangible benefits due to their straightforward measurement. However, a holistic evaluation encompassing intangible benefits ensures the organization knows vital aspects of a project's value. This balanced approach provides a comprehensive understanding of a project's potential, influencing more informed decision making and strategic planning.

B.4.2 Direct versus Indirect

Direct benefits are immediate and attributable outcomes from a specific project or initiative. They are typically easy to measure, track, and quantify. Within the scope of systems projects, direct benefits include cost savings from eliminating a redundant process, increased efficiency due to an automated workflow, or revenue gains from a new feature that boosts sales. These benefits serve as primary justifications for the project, offering stakeholders a clear understanding of the immediate value delivered. Direct benefits can be readily observed in performance metrics, making them instrumental in ROI calculations and assessing a project's initial success.

Indirect benefits, by contrast, are secondary outcomes arising from the project instead of its primary objective. While their link to the project might be less overt, their impact can still be substantial. In the context of systems projects, indirect benefits could encompass improved brand reputation due to faster customer service, enhanced employee morale from a positive change in work culture, or even network effects where the value of a system increases as more users adopt it. Indirect benefits can sometimes be more challenging to quantify than direct ones, but they often represent the broader ripple-effect value a project brings to an organization.

In evaluating systems projects, both direct and indirect benefits warrant careful consideration. While direct benefits offer a straightforward lens into the project's immediate impact, indirect benefits showcase the initiative's broader, potentially transformative effects. Stakeholders might prioritize direct benefits for their clear measurability and immediate value, but understanding and valuing indirect benefits ensures a comprehensive appreciation of a project's holistic impact. Recognizing the interplay between the two types of benefits can lead to more informed decision making, ensuring that organizations capture the full spectrum of value from their systems projects.

B.4.3 Financial versus Nonfinancial

Financial benefits are outcomes from a project that directly impact the organization's financial statements, typically reflected in monetary gains or savings. These benefits are easily quantifiable and are most often used in calculating the ROI of a project. Within systems projects, financial benefits manifest as cost savings from reduced manual labor, increased revenue from a new sales platform, or decreased expenses due to more efficient software tools. These benefits are tangible evidence of a project's value and are crucial for justifying the initial investment. Stakeholders often look to financial benefits first when evaluating the success or potential of a project due to their immediate and clear influence on the bottom line.

Nonfinancial benefits are those advantages that don't directly translate into monetary terms but still offer significant value to the organization. These qualitative benefits might be more complicated to measure but can have profound long-term impacts. For instance, in systems projects, nonfinancial benefits include improved employee satisfaction due to intuitive software tools, stronger customer loyalty from enhanced user experiences, or an uplift in brand reputation due to innovative features. Additionally, strategic advantages like positioning the organization as a market leader or ensuring alignment with long-term goals fall under this category. Even though these benefits might not be immediately reflected in financial metrics, they can influence financial outcomes indirectly over time.

When appraising systems projects, giving weight to financial and nonfinancial benefits is essential. While financial benefits provide a clear snapshot of immediate and tangible value, nonfinancial benefits highlight the broader effects of a project. By focusing solely on the financial benefits, organizations risk missing out on understanding a project's full potential and impact. On the other hand, solely relying on nonfinancial benefits can obscure the concrete returns expected from a project. Therefore, an evaluation incorporating both benefits ensures that organizations can make well-rounded decisions, optimizing immediate returns and long-term value from their systems projects.

B.4.4 Operational versus Strategic

Operational benefits are immediate outcomes that enhance an organization's day-to-day functions and tasks. They typically lead to improvements in efficiency, productivity, or the quality of routine operations. Within systems projects, operational benefits can manifest as reduced processing times due to automation, decreased error rates due to improved data integrity, or less complex tasks through intuitive interfaces. These benefits can be tangible, such as cost savings from reduced manual labor, and intangible, like improved employee morale from a more user-friendly system. Operational benefits often produce quick returns and are crucial for teams looking to optimize and streamline existing processes.

Strategic benefits, on the other hand, align with the organization's broader long-term objectives and vision. These benefits position the company favorably in its market or industry and provide a competitive edge. In systems projects, strategic benefits include achieving a unique market position through exclusive system capabilities, the ability to enter new markets due to a novel digital solution, or alignment with industry trends and innovations. These benefits are more about setting the organization's direction and ensuring future growth and sustainability. Strategic benefits, though sometimes slower to materialize than operational ones, can be game changers, transforming an organization's trajectory.

In evaluating systems projects, both operational and strategic benefits deserve careful attention. While operational benefits offer a clear insight into the immediate efficiencies and improvements a project can bring, strategic benefits reveal the broader, longer-term potential of the initiative. It's not uncommon for stakeholders to prioritize operational benefits due to their direct and immediate impact. However, giving due weight to strategic benefits ensures that the organization is not just thinking about the present but is preparing for the future. A successful systems project often delivers on both fronts: optimizing the present-day operations while setting the stage for future growth and market leadership. Recognizing and valuing both benefits ensures comprehensive project evaluation and better-informed decision making.

B.4.5 Short Term versus Long Term

Short-term benefits are immediate advantages that an organization experiences within a relatively short period after implementing a project, typically within a year or less. These benefits can provide quick wins, demonstrating the project's efficacy and often serving as a justification for the initial investment. With systems projects, short-term benefits might encompass rapid efficiency gains, immediate cost savings due to process optimization, or a swift uptick in sales or user engagement because of a new system feature. These benefits are tangible, easily measurable, and crucial for stakeholders looking for quick returns on their investment or proof of concept in the early stages of a project's life cycle.

Long-term benefits are the sustained advantages that materialize over an extended period, often spanning several years after the project's implementation. These benefits contribute to the organization's growth, stability, and continuous improvement. Within systems projects, long-term benefits could include sustained competitive advantage, continuous innovation enabled by the system, or a gradual increase in customer loyalty due to enhanced user experiences. Such benefits, while sometimes subtle or less immediate than their short-term counterparts, can significantly influence an organization's strategic direction, market position, and overall sustainability.

When assessing systems projects, it's essential to consider both short-term and long-term benefits. While short-term benefits offer insights into the project's immediate impact and provide initial validation, long-term benefits paint a picture of its enduring value and potential to shape the organization's future. Stakeholders may be naturally inclined toward short-term benefits due to their direct and immediate nature. However, giving importance to long-term benefits ensures that the organization isn't merely reacting to the present but is strategically planning its future. A genuinely successful systems project should offer a blend of both, delivering immediate value while laying the foundation for sustained growth and continued success. Recognizing the significance of both benefits allows for a more comprehensive evaluation and forward-thinking decision making.

B.4.6 Measurable versus Nonmeasurable

Measurable benefits are those advantages that can be quantified using specific metrics or data points. They offer clear, tangible evidence of the value derived from a project, making them valuable for ROI calculations and performance evaluations. Examples of measurable benefits include a 10% increase in production efficiency or a 15% reduction in customer complaints. In a systems project, measurable benefits include reduced processing time, increased sales volumes, decreased system downtime, or savings in operational costs. These benefits provide a concrete way to track progress, set benchmarks, and evaluate success. They are often used to justify investments, monitor project milestones, and establish accountability. Given their quantifiable nature, stakeholders typically prefer measurable benefits for decision making, as they provide a clear picture of the project's direct impact.

Nonmeasurable benefits, in contrast, are crucial qualitative outcomes that cannot be easily quantified or represented by straightforward metrics. These benefits are often subjective, deriving their value from perceptions, sentiments, or strategic implications. For systems projects, nonmeasurable benefits could include improved user satisfaction, enhanced organizational reputation, better team collaboration, or alignment with future industry trends. While these benefits might take time to quantify, they hold significant intrinsic value for an organization. Their impact is often felt over extended periods and can sometimes indirectly influence measurable outcomes in the long run.

When evaluating systems projects, weighing both measurable and nonmeasurable benefits is prudent. While measurable benefits offer a direct, quantifiable view of the project's impact, nonmeasurable benefits capture the project's broader, intangible value. Focusing solely on measurable outcomes might lead to a narrow view of a project's success, overlooking crucial qualitative impacts. Conversely, only considering nonmeasurable benefits can make it challenging to gauge concrete returns and set tangible goals. Therefore, a balanced approach that appreciates both benefits ensures that organizations comprehensively understand a project's value.

B.4.7 Positive versus Cost Avoidance

Discerning positive and cost-avoidance benefits is essential for comprehensive project evaluation and decision making. Both categories play pivotal roles in assessing the value and impact of a project, but they offer different perspectives on the advantages a project can bring. While positive benefits offer a direct glimpse into the immediate value a project adds, cost-avoidance benefits provide insight into the potential future risks mitigated. Together, they comprehensively illustrate a project's ROI and multifaceted organizational impact.

Positive Benefits

Positive benefits encompass the direct, advantageous outcomes or gains achieved due to the system's implementation or enhancement. These are tangible or intangible benefits that an organization can experience shortly after the system's rollout. For example, a new CRM system might increase sales due to better customer relationship management, or a new automated human resources system might reduce manual processing times and errors.

Positive benefits can often be quantified and measured in monetary terms like increased revenue or nonmonetary terms such as improved employee morale or enhanced customer satisfaction. Some typical positive benefits associated with systems projects include:

- **Cost savings.** A new system might automate specific processes, reducing manual labor and associated costs. Additionally, better resource management and efficient operations can lead to significant cost savings.
- **Increased revenue.** Enhanced features, faster processing, better data management, and improved customer experiences can increase sales, subscription renewals, or other revenue streams.
- **Improved efficiency.** Streamlining processes, automating repetitive tasks, and integrating different systems can increase organizational efficiency, enabling teams to accomplish more in less time.
- **Better decision making.** Modern systems can provide comprehensive analytics, real-time data visualization, and predictive modeling, empowering managers and executives to make informed decisions.
- **Improved customer satisfaction.** Faster response times, personalized user experiences, and more reliable services can significantly enhance customer satisfaction and loyalty.
- **Competitive advantage.** Implementing advanced systems or features can provide an edge over competitors by offering superior services, reducing costs, or reaching the market faster.
- **Enhanced security.** Newer systems often have advanced security features that protect organizational data from breaches, unauthorized access, and other threats.
- **Regulatory compliance.** Some systems are designed to ensure that the organization meets regulatory requirements, reducing the risk of legal penalties and protecting the organization's reputation.
- **Improved collaboration.** Systems facilitating communication, document sharing, and collaboration can boost teamwork across departments or geographical locations.
- **Environmental benefits.** By digitizing processes, reducing paper usage, and optimizing resources, many systems can lead to a reduced carbon footprint and more sustainable operations.

It's worth noting that while these are potential positive benefits, the actual benefits realized will depend on various factors, including the nature of the project, the quality of implementation, the organization's readiness for change, and how well the system aligns with organizational objectives. Proper planning, stakeholder involvement, and thorough testing can help achieve maximum positive benefits.

Cost-Avoidance Benefits

Cost-avoidance benefits focus on the expenses an organization anticipates but manages to sidestep due to the new system. Rather than looking at what has been directly gained, cost avoidance evaluates what has been prevented. For instance, by adopting a modern, scalable IT infrastructure, a company might avoid the significant costs associated with frequent future upgrades. A company implementing a robust cybersecurity system might sidestep potential future costs from data breaches, including litigation and fines.

Cost-avoidance benefits can be more challenging to quantify because they're rooted in projecting future scenarios. Still, they are invaluable in understanding a project's long-term value and preventive capabilities. Examples of cost-avoidance benefits include:

- **Future hardware/software upgrades and maintenance costs.** By adopting a scalable system today, an organization might avoid the cost of frequent future upgrades or replacements as its needs grow. In addition, a system designed with modern, maintainable technologies can result in lower future maintenance costs than an outdated or legacy system.

- **Training expenses.** Implementing an intuitive system can reduce the need for extensive training in the future, leading to cost-avoidance benefits in training time and materials.
- **Downtime costs.** Investing in a more reliable system can prevent future outages or failures. This avoids the costs associated with downtime, such as lost sales, productivity loss, and potential penalties for not meeting service-level agreements.
- **Regulatory fines.** An organization can avoid fines or penalties by ensuring a system complies with industry regulations.
- **Potential litigation.** Implementing robust security features can avoid the costs of potential future data breaches, which could lead to litigation or compensation claims.
- **Staffing costs.** Automation and efficiency improvements can mean the organization avoids hiring additional staff in the future as the workload increases.
- **Vendor costs.** By adopting a system that includes multiple functionalities or is interoperable with other systems, an organization might avoid future costs associated with third-party vendors or additional integrations.
- **Opportunity costs.** A new system might prevent the loss of potential business opportunities. For instance, by enhancing the customer experience, an organization can avoid the cost of losing business to competitors.
- **Expansion costs.** A flexible and scalable system can accommodate organizational growth, potentially avoiding the costs of implementing entirely new systems when expanding to new markets or regions.
- **Redundancy and backup costs.** Investing in a system with built-in redundancy and backup features might avoid the future costs of implementing separate backup solutions or dealing with data loss.

Recognizing and quantifying cost-avoidance benefits can be challenging because they deal with hypothetical scenarios in the future. However, they are crucial for understanding a system's ROI. Decision makers must differentiate between direct cost savings and cost-avoidance benefits to make informed investment decisions.

B.5 Payback Analysis

Payback analysis is the process of determining how long it takes an information system to pay for itself. The time it takes to recover the system's cost is called the payback period. To perform a payback analysis, the following steps are executed:

1. Determine the initial development cost of the system.
2. Estimate annual benefits.
3. Determine yearly operating costs.
4. Find the payback period by comparing total development and operating costs to the accumulated value of the benefits produced by the system.

When system costs are plotted over the potential life of the system, there is typically a curve like the one shown in Figure B-2. After the system is operational, costs decrease rapidly and remain relatively low for some time. Eventually, as the system requires more maintenance, costs begin to increase. The period between the beginning of system operation and the point when operational costs are rapidly growing is called the economically useful life of the system.

When the benefits provided by an information system are plotted against time, the resulting curve usually resembles the one shown in the upper graph in Figure B-3. Benefits start appearing when the system becomes operational, might increase for a time, and then level off or decline.

Figure B-2 The costs of a typical system vary over time. In the beginning, system costs are high due to initial development expenses. Costs then drop during systems operation. Maintenance costs begin to increase until the system ends its economically useful life.

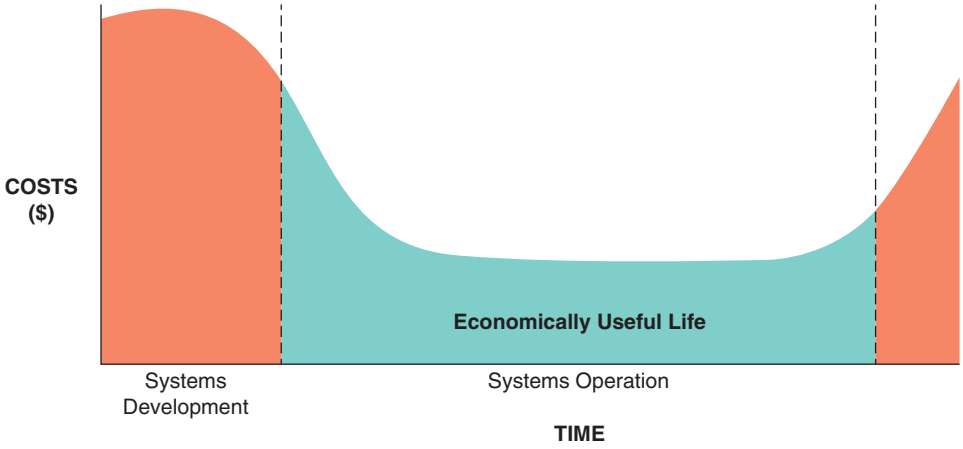
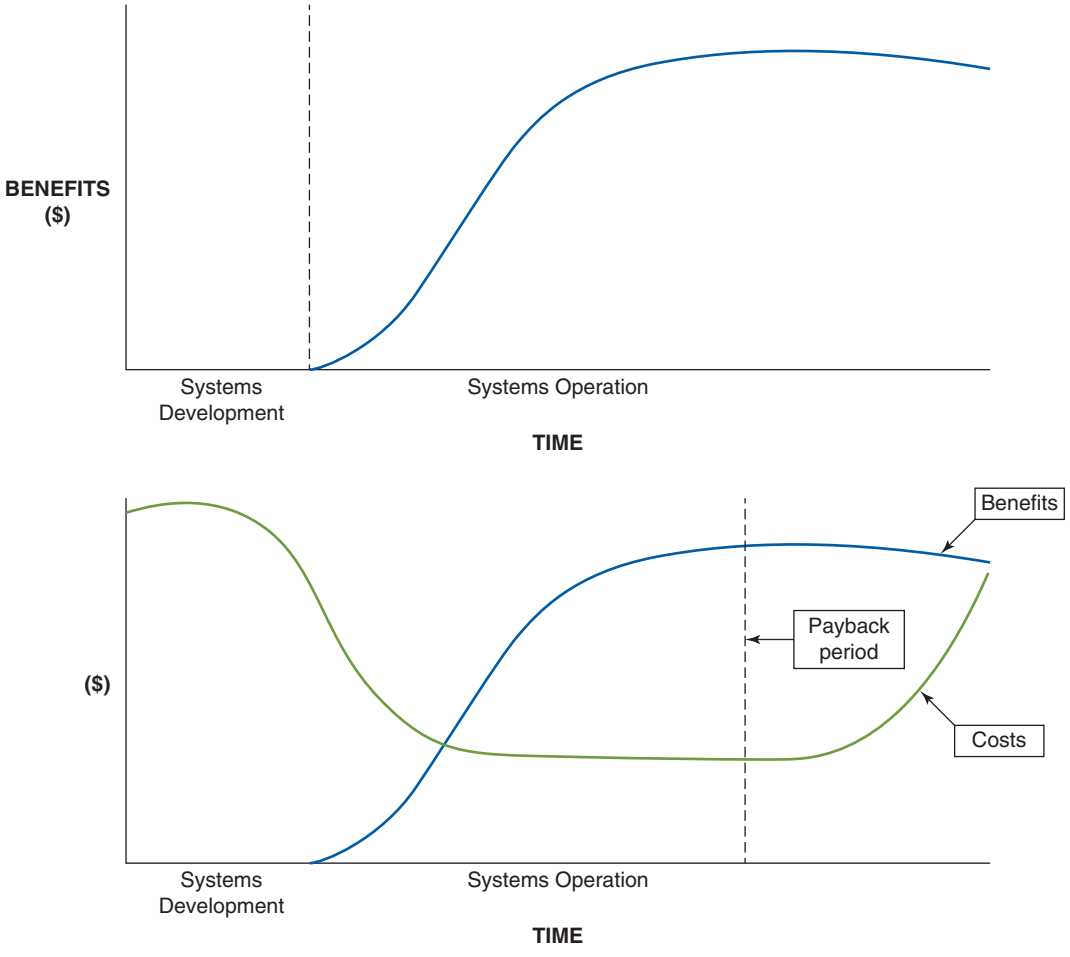


Figure B-3 Benefits of an information system change over time, as shown in the upper graph. The lower chart shows costs and benefits plotted on the same graph. The dashed line indicates the payback period, when accumulated benefits equal accumulated costs.



When conducting a payback analysis, a systems analyst calculates the time it takes for the accumulated benefits of an information system to equal the accrued costs of developing and operating the system.

The cost and benefit curves are plotted together in the lower graph in Figure B-3. The dashed line indicates the payback period. Notice that the payback period is not when current benefits equal current costs, where the two lines cross. Instead, the payback period compares accumulated costs and benefits. If current costs and benefits are graphed, the payback period corresponds to when the areas under the two curves are equal.

Consider the example in Table B-1, which contains two cost–benefit tables. The tables show the anticipated annual costs, cumulative costs, annual benefits, and cumulative benefits for two information systems projects. Year 0 (zero) corresponds to the year systems development begins. The development of Project A takes less than one year, so some benefits are realized in Year 0. Systems development for Project B requires more than one year, so the benefits do not begin until sometime in Year 1.

Table B-1 Payback analysis data for two information systems: Project A and Project B.

Payback Analysis Examples

Project A:				
Year	Costs	Cumulative Costs	Benefits	Cumulative Benefits
0	60,000	60,000	3,000	3,000
1	17,000	77,000	28,000	31,000
2	18,500	95,500	31,000	62,000
3	19,200	114,700	34,000	96,000
4	21,000	135,700	36,000	132,000
5	22,000	157,700	39,000	171,000
6	23,300	181,000	42,000	213,000

Project B:				
Year	Costs	Cumulative Costs	Benefits	Cumulative Benefits
0	80,000	80,000	____	____
1	40,000	120,000	6,000	6,000
2	25,000	145,000	26,000	32,000
3	22,000	167,000	54,000	86,000
4	24,000	191,000	70,000	156,000
5	26,500	217,500	82,000	238,000
6	30,000	247,500	92,000	330,000

In Project A, by the end of Year 4, the cumulative costs are $135,700, slightly exceeding the $132,000 in cumulative benefits. By the end of Year 5, however, the cumulative benefits of $171,000 far exceed the cumulative costs of $157,700. Therefore, at some point during Year 5, the accumulated costs and benefits are equal, and the payback period is established. In Project B, a similar situation exists. By the end of Year 4, Project B's cumulative costs

are $191,000, which is greater than the cumulative benefits of $156,000. At some point during Year 5, cumulative benefits will exceed cumulative costs, and the system will have paid for itself.

If more specific information is available for the timing of costs and benefits during a year, the payback period can be calculated more precisely. For example, a spreadsheet can be created to show when cumulative benefits exceed cumulative costs, as explained in Section B.5.1.

Some managers are critical of payback analysis because it emphasizes early costs and benefits and ignores the benefits received after the payback period. Even if the benefits for Project B in Year 6 soared as high as $500,000, the payback period for that project still occurs during the fifth year of operation. In defense of payback analysis, the earlier cost and benefit predictions usually are more confident. Generally, the further out in time projections extend, the more uncertain the forecast will be. Thus, payback analysis uses the most reliable cost and benefit estimates.

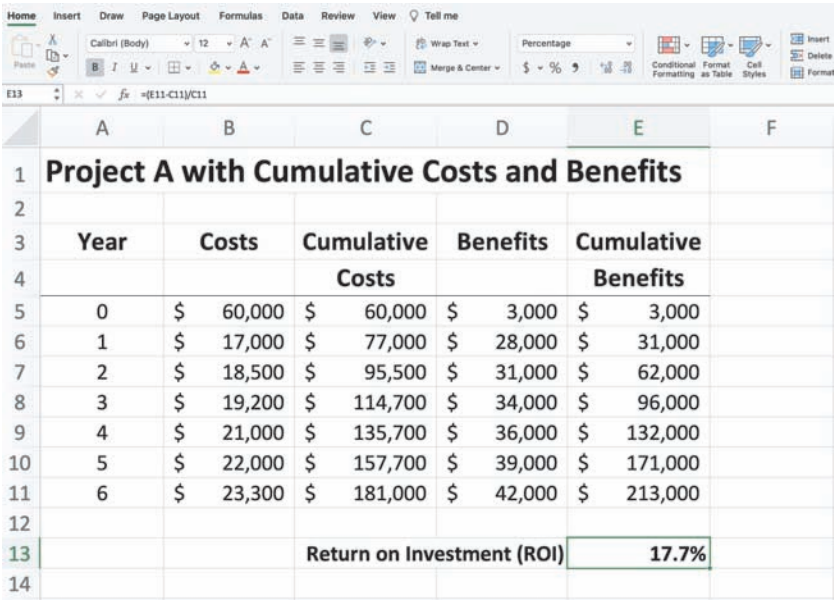
Payback analysis is rarely used to compare or rank projects because later benefits are ignored. It should not be concluded that Project A is better than Project B simply because A's payback period is less than B's; considering all costs and benefits when comparing projects makes more sense.

Even with its drawbacks, payback analysis is popular. Many business organizations establish a minimum payback period for approved projects. If company policy requires a project to begin paying for itself within three years, neither project in Table B-1 would be approved. However, both are economically feasible because total benefits exceed total costs.

B.5.1 Using a Spreadsheet to Compute Payback Analysis

A spreadsheet can record and calculate accumulated costs and benefits, as shown in Figure B-4. The first step is to design the worksheet and label the rows and columns. The formulas are entered after entering the cost and benefit data for each year. For payback analysis, a formula is needed to display cumulative totals year by year. For example, the first year in the cumulative costs column is the same as Year 0 costs, so the formula in cell

Figure B-4 A Microsoft Excel worksheet displays cost–benefit analysis data for Project A, including payback analysis and ROI calculations.
Source: Microsoft Corporation

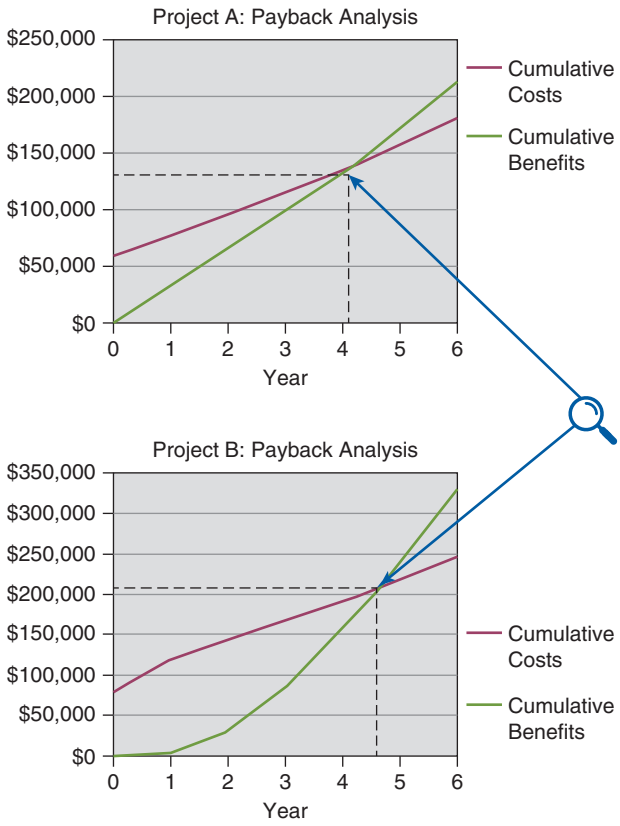


	A	B	C	D	E	F
1	**Project A with Cumulative Costs and Benefits**					
2						
3	**Year**	**Costs**	**Cumulative**	**Benefits**	**Cumulative**	
4			**Costs**		**Benefits**	
5	0	$ 60,000	$ 60,000	$ 3,000	$ 3,000	
6	1	$ 17,000	$ 77,000	$ 28,000	$ 31,000	
7	2	$ 18,500	$ 95,500	$ 31,000	$ 62,000	
8	3	$ 19,200	$ 114,700	$ 34,000	$ 96,000	
9	4	$ 21,000	$ 135,700	$ 36,000	$ 132,000	
10	5	$ 22,000	$ 157,700	$ 39,000	$ 171,000	
11	6	$ 23,300	$ 181,000	$ 42,000	$ 213,000	
12						
13			**Return on Investment (ROI)**		**17.7%**	
14						

C5 is =B5. The cumulative cost total for the second year is Year 0 cumulative total + Year 1 costs, so the formula for cell C6 is =C5 + B6, and so on.

After verifying that the spreadsheet operates properly, a line chart displays the cumulative costs, benefits, and payback period, identified by the intersection of the cost and benefit lines, as shown in Figure B-5.

Figure B-5 Microsoft Excel can show the payback period by creating a chart of cumulative costs and benefits. Note that Project A has a shorter payback period than Project B.



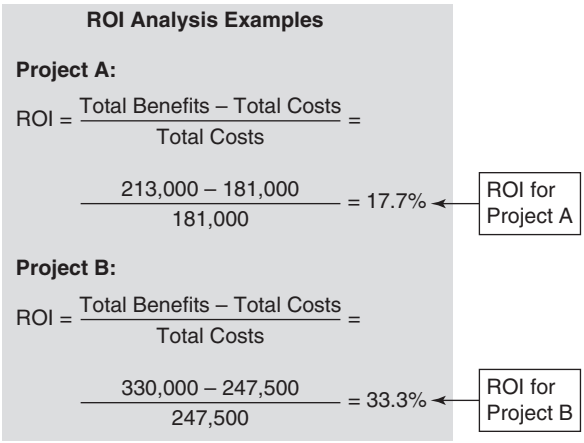
B.6 Return on Investment Analysis

ROI is a percentage rate that measures profitability by comparing the total net benefits (the return) received from a project to the total costs (the investment) of the project. ROI is calculated as follows:

$$\text{ROI} = (\text{total benefits} - \text{total costs})/\text{total costs}$$

ROI analysis considers costs and benefits over a more extended period than payback analysis. ROI calculations usually are based on total costs and benefits for five to seven years. For example, Figure B-6 shows ROI calculations for Project A and Project B. The ROI for Project A is 17.7%, and Project B's is 33.3%.

Figure B-6 ROI analysis for Projects A and B shown in Table B-1.



In many organizations, projects must meet or exceed a minimum ROI. This minimum ROI can estimate the return the organization would receive from investing its money in other opportunities, such as treasury bonds, or it can be a higher rate that the company requires for all new projects. For example, if a company needs a minimum ROI of 15%, then both Projects A and B would meet the criterion.

ROI can also be used for ranking projects. If Projects A and B represent two proposed solutions for a single information systems project, then the solution represented by Project B is better than the Project A solution. If Projects A and B represent two different information systems projects, and if the organization has sufficient resources to pursue only one of the two projects, then Project B is the better choice.

Critics of ROI analysis raise two points. First, ROI measures the overall rate of return for the total period, and annual return rates can vary considerably. Two projects with the same ROI might not be equally desirable if the benefits of one project occur significantly earlier than the benefits of the other project. The second criticism is that the ROI technique ignores the timing of costs and benefits. This concept is called the time value of money and is explained in Section B.7.

B.6.1 Using a Spreadsheet to Compute ROI

A spreadsheet can be used to calculate the ROI. To do so for Project A, first set up the worksheet and enter the cost and benefit data. Cumulative columns can be used (as was done for payback analysis), but two overall totals are also needed (one for costs and one for benefits), as shown in Figure B-4.

The last step is adding a formula to calculate the ROI percentage rate, displayed in cell E13 in Figure B-4. As stated, the ROI calculation is total benefits minus total costs divided by total costs. Therefore, the formula that displays the ROI percentage in cell E13 is =(E11−C11)/C11.

A significant advantage of using a spreadsheet is that if the data changes, the worksheet can be modified, and a new result can be calculated instantly.

B.7 Present Value Analysis

When inflation is present, a dollar today is worth more than a dollar received a year from today. A dollar in hand can be invested now and will grow in value. For example, most people would rather have $100 right now rather than the promise of $100 a year from now. If the $100 were invested today in a mutual fund with an annual return of 8%, there would be $108 instead of $100 one year from now.

ROI can also be approached from a different direction. For example, instead of asking, "How much will $100 be worth a year from now?" ask, "How much should be invested today, at 8%, to have $100 a year from now?" This concept is called the time value of money and is the basis of the present value analysis technique.

The present value of a future dollar is the amount of money that, when invested today at a specified interest rate, grows to precisely one dollar at a certain point in the future. The specified interest rate is called the discount rate. In present value analysis, a company uses a discount rate that represents the rate of return if the money is put into relatively risk-free investments, such as bonds, instead of being invested in the project.

Most companies require a higher return rate than the discount rate because of the degree of risk in any project compared with investing in a bond. Companies often reject attractive projects because the risk is not worth the potential reward.

To help the analyst perform present value analysis, adjustment factors for various interest rates and numbers of years are calculated and printed in tables called present value tables. Table B-2 shows a portion of a present value table, including values for 10 years at various discount rates.

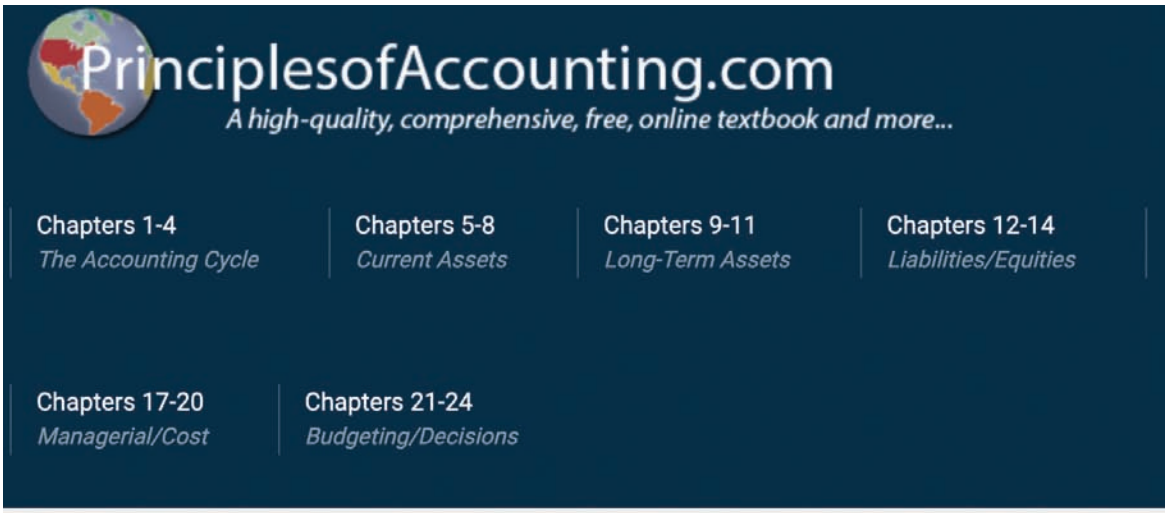
Table B-2 Portion of a present value table showing adjustment factors for various periods and discount rates. Values in the table are calculated using the formula shown in the text. Notice how the factors decrease as time and percentage increase.

Periods	6%	8%	10%	12%	14%
1	0.943	0.926	0.909	0.893	0.877
2	0.890	0.857	0.826	0.797	0.769
3	0.840	0.794	0.751	0.712	0.675
4	0.792	0.735	0.683	0.636	0.592
5	0.747	0.681	0.621	0.567	0.519
6	0.705	0.630	0.564	0.507	0.456
7	0.665	0.583	0.513	0.452	0.400
8	0.627	0.540	0.467	0.404	0.351
9	0.592	0.500	0.424	0.361	0.308
10	0.558	0.463	0.386	0.322	0.270

To use a present value table, locate the value in the column with the appropriate discount rate and the row for the proper number of years. For example, to calculate the present value of $1 at 12% for five years, navigate down the 12% column in Table B-2 until you reach the row representing five years. The table value is 0.567. To determine the present value of $3,000 in five years with a discount rate of 12%, multiply the present value factor from the table by the dollar amount; that is, PV = $3,000 × 0.567 = $1,701.

Many finance and accounting books contain comprehensive present value tables. This information can be obtained online, as shown in Figure B-7.

Figure B-7 Many online sources contain comprehensive present value tables.
Source: Principlesofaccounting.com



Present Value of Ordinary Annuity

To find the present value of ordinary annuity find the appropriate period and rate in the tables below.

PRESENT VALUE OF ORDINARY ANNUITY
(annuity in arrears -- end of period payments)

	RATE PER PERIOD							
Periods	0.25%	0.50%	0.75%	1.00%	1.50%	2.00%	2.50%	3.00%
1	0.99751	0.99502	0.99256	0.99010	0.98522	0.98039	0.97561	0.97087
2	1.99252	1.98510	1.97772	1.97040	1.95588	1.94156	1.92742	1.91347
3	2.98506	2.97025	2.95556	2.94099	2.91220	2.88388	2.85602	2.82861
4	3.97512	3.95050	3.92611	3.90197	3.85438	3.80773	3.76197	3.71710
5	4.96272	4.92587	4.88944	4.85343	4.78264	4.71346	4.64583	4.57971
6	5.94785	5.89638	5.84560	5.79548	5.69719	5.60143	5.50813	5.41719
7	6.93052	6.86207	6.79464	6.72819	6.59821	6.47199	6.34939	6.23028
8	7.91074	7.82296	7.73661	7.65168	7.48593	7.32548	7.17014	7.01969
9	8.88852	8.77906	8.67158	8.56602	8.36052	8.16224	7.97087	7.78611
10	9.86386	9.73041	9.59958	9.47130	9.22218	8.98259	8.75206	8.53020

The cost and benefit figures must be time-adjusted to perform present value analysis. First, multiply each projected benefit and cost by the proper present value factor, which depends on when the cost will be incurred or the benefit will be received. The second step is to sum all the time-adjusted benefits and time-adjusted costs. Then, calculate the project's net present value (NPV), which is the total present value of the benefits minus the total present value of the costs. Table B-3 shows the NPV calculation for two sample projects.

Table B-3 Net present value analysis for Project A and Project B. The tables use discount factors from external sources.

Net Present Value Examples

Project A:								
	Year 0	Year 1	Year 2	Year 3	Year 4	Year 5	Year 6	Total
Benefits:	3,000	28,000	31,000	34,000	36,000	39,000	42,000	
Present Value Factor (12%):	1.000	0.893	0.797	0.712	0.636	0.567	0.507	
Present Value:	3,000	25,004	24,707	24,208	22,896	22,113	21,294	143,222
Costs:	60,000	17,000	18,500	19,200	21,000	22,000	23,300	
Present Value Factor (12%):	1.000	0.893	0.797	0.712	0.636	0.567	0.507	
Present Value:	60,000	15,181	14,745	13,670	13,356	12,474	11,813	141,239
Net Present Value:								**1,983**

Project B:								
	Year 0	Year 1	Year 2	Year 3	Year 4	Year 5	Year 6	Total
Benefits:	____	6,000	26,000	54,000	70,000	82,000	92,000	
Present Value Factor (12%):	_____	0.893	0.797	0.712	0.636	0.567	0.507	
Present Value:	_____	5,358	20,722	38,448	44,520	46,494	46,644	202,186
Costs:	80,000	40,000	25,000	22,000	24,000	26,500	30,000	
Present Value Factor (12%):	1.000	0.893	0.797	0.712	0.636	0.567	0.507	
Present Value:	80,000	35,720	19,925	15,664	15,264	15,026	15,210	196,809
Net Present Value:								**5,377**

In theory, any project with a positive NPV is economically feasible because the project will produce a larger return than would be achieved by investing the same amount of money in a discount rate investment. However, risks are associated with any project, and management typically insists on a substantially higher return for high-risk projects. For example, both projects in Table B-3 have positive NPVs and appear economically worthwhile.

Suppose, however, that one of the projects had a known 90% probability of achieving its goals while the other had only a 70% chance. The project with the higher risk would have to offer a corresponding higher reward to be attractive. You learned how project managers evaluate risks earlier in this book.

NPV also can be used to compare and rank projects. All things being equal, the project with the highest NPV is the best investment. Table B-3 shows that Project B is a better investment than Project A because it has a higher NPV.

Present value analysis solves the shortcomings of payback and ROI analysis. Unlike payback analysis, present value analysis considers all costs and benefits, not just the earlier values. In addition, present value analysis considers the timing of costs and benefits, so their values can be adjusted by the discount rate that provides a common yardstick and recognizes the time value of money. Even so, companies often use all three methods to get more input for making decisions. Sometimes, a project will score higher on one analysis method and lower on another.

B.7.1 Using a Spreadsheet to Calculate Present Value

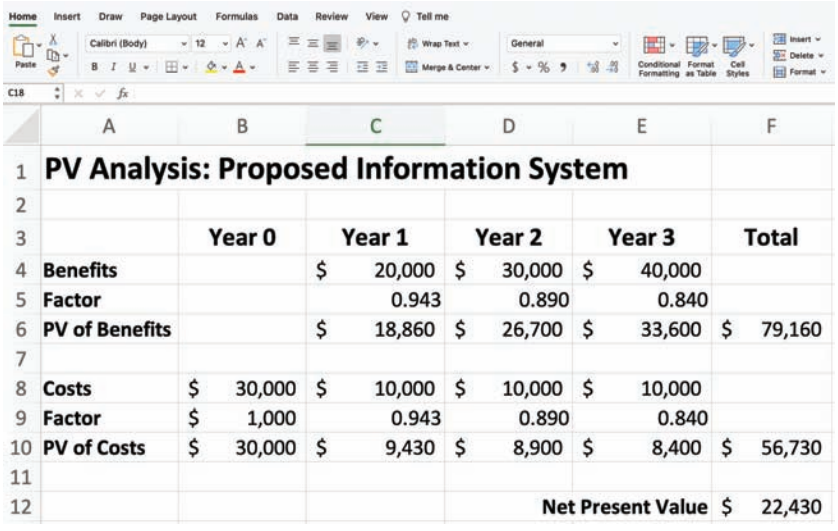
Using a spreadsheet program such as Microsoft Excel, there are two ways to calculate present value. The discount adjustment factors can be entered from an external table, and a simple formula can be used to apply the factors. However, many analysts prefer using a built-in NPV formula that handles the calculations. The two methods are described in the following sections. The example shows the costs and benefits of a proposed information system with a one-year development period, a three-year useful life, and a 6% discount rate.

Using External Factors

The first method requires a spreadsheet like the one in Figure B-8. Enter adjustment factors in cells C5, D5, and E5, starting with the estimated benefits. Then, create formulas in cells C6, D6, and E6 that multiply the three factors by the dollar amounts for each year, with a total shown in column F. The same thing is done for the estimated costs. Finally, when the total costs are subtracted from total benefits, the NPV for the proposed system is displayed in cell F12.

Figure B-8 This Microsoft Excel spreadsheet shows how to use discount factors to calculate present value. In this case, the present value of benefits is $79,160. The NPV function, a menu-driven formula built into Excel, can also calculate this amount.

Source: Microsoft Corporation



	A	B	C	D	E	F
1	**PV Analysis: Proposed Information System**					
2						
3		**Year 0**	**Year 1**	**Year 2**	**Year 3**	**Total**
4	**Benefits**		$ 20,000	$ 30,000	$ 40,000	
5	**Factor**		0.943	0.890	0.840	
6	**PV of Benefits**		$ 18,860	$ 26,700	$ 33,600	$ 79,160
7						
8	**Costs**	$ 30,000	$ 10,000	$ 10,000	$ 10,000	
9	**Factor**	$ 1,000	0.943	0.890	0.840	
10	**PV of Costs**	$ 30,000	$ 9,430	$ 8,900	$ 8,400	$ 56,730
11						
12					**Net Present Value**	$ 22,430

Using a Built-In Formula

The second method calculates the present value using the built-in spreadsheet function NPV. Enter the amounts, the discount rate, and the number of time periods, and the program does the rest. Using this approach, four steps are performed, starting with the benefits:

1. First, create a spreadsheet similar to the one in Figure B-8, but do not enter any factors or formulas—just the year-by-year cost and benefit amounts.
2. Next, select cell F6 and click the Function button on the formula bar. Select the NPV function; a dialog box appears. A discount rate of 6% is being used, so enter .06 in the Rate box.
3. Now enter the three benefit amounts: 20,000 for Value1, 30,000 for Value2, and 40,000 for Value3. Click the OK button.
4. At this point, a present value appears in cell F6. Now, follow the same method to enter the cost amounts. When the PV of costs is subtracted from the PV of benefits, the result is the NPV. Notice a slight difference in the two methods because Excel uses more decimal places. The difference is not significant.

Glossary

1:1 A type of entity relationship. A one-to-one relationship, abbreviated 1:1, exists when precisely one of the second entities occurs for each instance of the first entity.

1:M A type of entity relationship. A one-to-many relationship, abbreviated 1:M, exists when one occurrence of the first entity can be related to many occurrences of the second entity. Still, each occurrence of the second entity can be associated with only one occurrence of the first entity.

802.11 A family of wireless network specifications developed by the IEEE.

802.11ac An IEEE wireless network specification, approved in 2014, which uses expanded MIMO technology to achieve theoretical speeds of nearly 7 Gbps while increasing the wireless range. It is backward-compatible with 802.11 a, b, g, and n.

802.11ax An IEEE wireless network specification introduced in 2019 and 2020; also known as Wi-Fi 6 and Wi-Fi 6E. Replaces the 802.11ac, or Wi-Fi 5, standard.

802.11b An IEEE wireless network specification introduced in 1999, based on a frequency of 2.4 GHz and a maximum bandwidth of 11 Mbps, and replaced by 802.11g.

802.11be An IEEE wireless network specification introduced in 2024 and dubbed Extremely High Throughput; the next amendment of the IEEE 802.11 standard, designated Wi-Fi 7.

802.11g An IEEE wireless network specification introduced in 2003, based on a frequency of 2.4 GHz and a maximum bandwidth of 54 Mbps; compatible with and a replacement for 802.11b; superseded by the 802.11n standard.

802.11n An IEEE wireless network specification adopted in 2009 that uses MIMO technology to achieve speeds of 200+ Mbps while increasing the wireless range. It is backward-compatible with 802.11 a, b, and g.

abbreviation code An alphabetic abbreviation. For example, standard state postal codes include NY for New York, ME for Maine, and MN for Minnesota.

absolute date The total number of days from some specific base date. To calculate the number of days between two absolute dates, subtract one date from the other. For example, if you use a base date of January 1, 1900, then September 27, 2012, has an absolute date value of 41179, and July 13, 2011, has an absolute date of 40737. If the earlier date value is subtracted from the later one, the result is 442 days.

acceptance One of four risk control strategies. In acceptance, the risk is accepted, and nothing is done. Risk is usually accepted only if protection from risk is not worth the expense.

acceptance test A test in which users enter data, including actual or live data samples, and then perform queries and produce reports to simulate actual operating conditions. Users and the IT project development team verify all processing options and outputs to ensure the system functions correctly. An acceptance test is sometimes known as a system test. Testing involves the entire information system, including all typical processing situations.

access point A central wireless device that provides network services to wireless clients.

action code A code indicating what action is to be taken with an associated item. For example, a student records program might prompt a user to enter or click an action code such as D (to display the student's record), A (to add a record), and X (to exit the program).

activity Any work that has a beginning and an end and requires the use of company resources, including people, time, and money. Examples include conducting a series of interviews, designing a report, selecting software, waiting for the delivery of equipment, and training users. See also task.

activity diagram A diagram that resembles a horizontal flowchart that shows actions and events as they occur. Activity diagrams show the order in which actions occur and identify the outcome.

actor An external entity with a specific role. In a use case diagram, actors model interaction with the system.

adaptive maintenance Changes that add new capabilities and enhancements to an existing system.

administrative privileges The ability to execute policies, procedures, and practices related to the management and control of an organization or system.

advanced persistent threats (APT) Sophisticated, prolonged cyberattack campaigns targeting specific entities for espionage or data theft.

agile methods Systems development methods that attempt to develop a system incrementally by building a series of prototypes and constantly adjusting them to user requirements. Also called adaptive methods.

agile methods Systems development methods that attempt to develop a system incrementally by building a series of prototypes and constantly adjusting them to user requirements. Related to adaptive methods.

alias A term used in various data dictionaries to indicate an alternate name or a name other than the standard name used to describe the data element.

allocated baseline A reference point that documents the system at the end of the design phase and identifies any changes since the functional baseline. The allocated baseline includes testing and verification of all system requirements and features.

alphabetic code A code that uses alphabetic characters to distinguish one item from another based on a category, an abbreviation, or an easy-to-remember value, called a mnemonic code.

alternate key Any candidate key not chosen as the primary key.

anonymization The process of removing or altering personal information from data sets so individuals cannot be easily identified.

app A software application that runs on a mobile device, such as a smartphone or tablet.

application Part of an information system that handles input, manages processing logic, and provides required output.

application development The process of constructing the programs and code modules that are the building blocks of an information system. An application development group works within a traditional IT department composed of systems analysts and programmers who handle information system design, development, and implementation.

application lifecycle management (ALM) Activities that cover the entire SDLC, including requirements, design, development, testing, and deployment and management of software applications.

application logic The underlying business rules or logic for an application.

application server A computer that acts as a middleman between customers and an organization's databases and applications. It is often used to facilitate complex business transactions.

application service provider (ASP) A firm that delivers a software application or access to an application by charging a usage or subscription fee.

application software Software programs, such as email, word processors, spreadsheets, and graphics packages, used by employees in typical office scenarios.

applications programmer A person who works on new systems development and maintenance.

archive The storage of previous versions of a system when a new version is installed.

artificial intelligence (AI) The attempt to re-create natural intelligence through software in machines.

ASCII The abbreviation for American Standard Code for Information Interchange, a data storage coding method used on most personal computers and workstations.

asset Hardware, software, data, networks, people, or procedures that provide tangible or intangible benefits to an organization.

associative entity An entity that has its own set of attributes and characteristics. Associative entities are used to link between many-to-many (M:N) relationships.

attack Any attempt to expose, alter, disable, destroy, steal, or gain unauthorized access to an asset or make unauthorized use of an asset.

attribute A single characteristic or fact about an entity. An attribute, or field, is the smallest piece of data that has meaning within an information system. For example, a Social Security number or company name could be examples of an attribute. In object-oriented analysis, an attribute is part of a class diagram that describes the objects' characteristics. Also known as a data element.

audit fields Special fields within data records that provide additional control or security information. Typical audit fields include the date the record was created or modified, the name of the user who acted, and the number of times the record has been accessed.

audit log files Recorded details of all accesses and changes to a file or database; they can be used to recover modifications made since the last backup.

audit trail A record of data sources and when the data was entered into a system. In addition to recording the source, an audit trail must show how and when data is accessed or changed and by whom. These actions must be logged in an audit trail file and monitored carefully.

authorization zone Part of a form that contains any required signatures.

automated fax A system that allows a customer to request a fax using email, a company website, or a phone. The response is transmitted back to the user's fax machine in seconds. See faxback.

automatic update service A service that automatically downloads and applies software updates to ensure systems are protected against known vulnerabilities.

availability One of the three main elements of system security: confidentiality, integrity, and availability (CIA). Availability ensures that authorized users have timely and reliable access to necessary information.

avoidance One of four risk control strategies. In avoidance, adding protective safeguards eliminates the risk.

B2B (business-to-business) A commercial exchange (e.g., products or services) between businesses, typically enabled by the Internet or electronic means.

B2C (business-to-consumer) A commercial exchange (e.g., products or services) between businesses and consumers conducted over the Internet.

backup media Physical or virtual storage devices where data is copied and stored as a precaution against loss or damage.

backup policy A set of rules that define how data should be backed up, including what data to back up, how often, and the retention period.

backup The process of copying data to a secondary location for preservation in case of hardware failure or data loss.

backup The process of saving a series of files or data copies to be retained for a specified period. Data can be backed up continuously or at prescribed intervals.

balancing A process used to maintain consistency among an entire series of diagrams, including input and output data flows, data definition, and process descriptions.

bandwidth The amount of data that the system can handle in a fixed period. Bandwidth requirements are expressed in bits per second (bps).

baseline A formal reference point that measures system characteristics at a specific time. Systems analysts use baselines as yardsticks to document features and performance during the systems development process.

basic service set (BSS) A wireless network configuration in which a central wireless device called an access point serves all wireless clients. Also called infrastructure mode.

batch A group of data usually inputted into an information system simultaneously.

batch control A total used to verify batch input. Batch controls might check data items such as record counts and numeric field totals. For example, before entering a batch of orders, a user might calculate the total number of orders and the sum of the order quantities. When the batch of orders is entered, the order system calculates the same two totals. A data entry error occurs if the system totals do not match the input totals.

batch input A process in which data entry is performed on a specified schedule, such as daily, weekly, monthly, or longer. For example, batch input occurs when a payroll department collects time cards at the end of the week and enters the data as a batch.

benchmark A measure of the time a software package takes to process a certain number of transactions.

benchmark testing A form of testing used by companies to measure system performance.

best-case estimate The most optimistic outcome.

big data Extremely large data sets (e.g., petabytes) requiring nontraditional approaches to deal with them. Sometimes characterized by three terms: volume, variety, and velocity.

binary storage format A format that offers efficient storage of numeric data. For example, when numeric data types are specified using Microsoft Access, various storage format choices exist, including integer and long integer.

biometric devices Mechanisms used to uniquely identify a person by a retina scan or by mapping a facial pattern.

BIOS-level password A password set within the basic input/output system (BIOS) to prevent unauthorized access to the computer's firmware settings.

bit The smallest unit of data; one binary digit.

black box A metaphor for a process or action that produces results in a nontransparent or nonobservable manner. In data flow diagrams, a process appears as a black box in which the process's inputs, outputs, and general functions are known, but the underlying details are not shown.

black hole A process that has no output.

block sequence code A cipher that uses blocks of numbers for different classifications.

blockchain A distributed ledger system. The technology underlying Bitcoin.

blog An online journal. The term is a contraction of "web log."

Bluetooth A form of wireless transmission for short-distance wireless communication that does not require high power.

boot-level password See BIOS-level password.

bottom-up technique A method for analyzing a large, complex project as a series of individual tasks, called project tasks.

brainstorming A fact-finding technique for gaining information through a small group discussion of a specific problem, opportunity, or issue.

bring your own device (BYOD) An equipment management model where employees oversee their devices (e.g., computers, tablets, smartphones) at work, not the IT department. This includes device selection and setup, program installation and updating, and network connectivity (including security).

Brooks' Law Frederick Brooks, an IBM engineer, observed that adding more staffing to a late software project only makes it later.

bug tracking software Software used to document and track program defects, code changes, and replacement code, called patches. Also called defect tracking software.

build or buy A choice between developing in-house software and purchasing software. See also make or buy.

bus network A computer network in which a single communication path connects the mainframe computer, server, workstations, and peripheral devices. Information is transmitted in either direction from any workstation to another workstation, and any message can be directed to a specific device.

business case Refers to the reasons, or justification, for a proposal.

business continuity planning (BCP) A strategy to ensure critical business functions continue during and after a disaster.

business logic Rules to determine how a system handles data and produces useful information, reflecting the operational requirements of the business. Examples include adding the proper sales tax amount to invoices, calculating customer balances and finance charges, and determining whether a customer is eligible for a volume-based discount. Also called business rules.

business model A graphical representation of business functions that consist of business processes, such as sales, accounting, and purchasing.

business process A description of specific events, tasks, and desired results.

business process model (BPM) A graphical representation of one or more business processes.

business process modeling notation (BPMN) A standard set of shapes and symbols used to represent events, processes, and workflows in computer-based modeling tools.

business process outsourcing (BPO) The outsourcing of a basic business process. See also outsourcing.

business profile A definition of a company's overall functions, processes, organization, products, services, customers, suppliers, competitors, constraints, and future direction.

business rules How a system handles data and produces useful information. Business rules, also called business logic, reflect the business's operational requirements. Examples include adding the proper sales tax amount to invoices, calculating customer balances and finance charges, and determining whether a customer is eligible for a volume-based discount.

business support systems Provide job-related information support to users at all company levels.

byte A group of 8 bits; also called a character. A set of bytes forms a field representing an individual fact about a person, place, thing, or event.

C2C (consumer-to-consumer) A commercial exchange (e.g., products or services) between consumers conducted over the Internet.

calendar control A control that allows a user to select a date that the system will display and store as a field value.

California Consumer Privacy Act (CCPA) A state statute aimed at enhancing privacy rights and consumer protection by giving people more control over the personal information that businesses collect about them.

candidate key Any field that could serve as a primary key. Sometimes, it is possible to choose fields or field combinations as the primary key.

Capability Maturity Model (CMM) A model developed by SEI that integrates software and systems development into a process improvement framework.

Capability Maturity Model Integration (CMMI) An SEI-developed process to improve quality, reduce development time, and cut costs. A CMM tracks an organization's software development goals and practices using five maturity levels, from Level 1 (relatively unstable, ineffective software) to Level 5 (software that is refined, efficient, and reliable).

capacity planning A process that monitors current activity and performance levels, anticipates future activity, and forecasts the resources needed to provide desired service levels.

cardinality A concept that describes how instances of one entity relate to instances of another entity. Cardinality is described in class diagrams by a notation of zero or many, zero or one, one and only one, and one or many.

cardinality notation Code that shows relationships between entities.

case for action A part of the preliminary investigation report to management that summarizes project requests and makes specific recommendations.

CASE tools Powerful software used in computer-aided systems engineering (CASE) to help systems analysts develop and maintain information systems.

category codes Ciphers that identify a group of related items. For example, a local department store may use a two-character category code to identify the department where a product is sold.

certification A credential an individual earns by demonstrating a certain level of knowledge and skill on a standardized test.

change control (CC) A process for controlling changes in system requirements during software development. An important tool for managing system changes and costs after a system becomes operational.

character A group of 8 bits; also called a byte. A set of bytes forms a field, which is an individual fact about a person, place, thing, or event.

character-based report A report created using a single monospaced character set.

check box A control used to select one or more choices from a group. A checkmark or an X represents chosen options.

child In inheritance, an object that derives one or more attributes from a parent object.

child diagram The lower-level diagram in an exploded data flow diagram.

churn analysis The process of examining data to understand the reasons and rate at which customers stop doing business with a company. Often used to develop strategies to retain customers and reduce turnover.

CIA triangle A model designed to guide policies for information security within an organization, focusing on confidentiality, integrity, and availability.

cipher codes Keywords used to encode a number. A retail store, for example, may use a 10-letter word, such as CAMPGROUND, to code wholesale prices, where the letter C represents 1, A represents 2, and so on. Thus, the GRAND code would indicate that the store paid $562.90 for the item.

class A term used in object-oriented modeling to indicate a collection of similar objects.

class diagram A detailed view of a single use case, showing the classes that participate in the use case and documenting the relationship among the classes.

clicks to close The average number of page views needed to make a purchase or obtain desired information.

clickstream storage Recording web visitor behavior and traffic trends for later data mining use.

client A computer workstation that users interact with in a client/server design. These workstations are supplied with data, processing services, or other support from other computers called servers.

client/server architecture Generally, a system that divides processing between one or more networked clients and a central server. The client handles the entire user interface in a typical client/server system, including data entry, queries, and screen presentation logic. The server stores the data and provides data access and database management functions. Application logic is divided between the server and the clients.

closed-ended questions Queries that limit or restrict the range of responses. Used in the interview process when specific information or fact verification is desired.

cloud access security brokers (CASB) Security policy enforcement points between cloud service consumers and providers that consolidate security.

cloud computing An online software and data environment in which applications and services are accessed and used through an Internet connection rather than on a local computer; refers to the cloud symbol for the Internet.

code A set of letters or numbers that represents a data item. Codes can be used to simplify output, input, and data formats.

code review See structured walk-through.

coding The process of turning program logic into specific instructions that a computer system can execute.

cohesion A measure of a module's scope and processing characteristics. A module that performs a single function or task has a high degree of cohesion, which is desirable.

combination check A type of data validation check performed on two or more fields to ensure that they are consistent or reasonable when considered together. Even though all the fields involved in a combination check might pass their validation checks, the combination of the field values might be inconsistent or unreasonable.

combination key A type of data validation check performed on two or more fields to ensure that they are consistent or reasonable when considered together. Even though all the fields involved in a combination check might pass their validation checks, the combination of the field values might be inconsistent or unreasonable.

command button An onscreen button that initiates an action such as printing a form or requesting help.

common field An attribute that appears in more than one entity. Common fields can be used to link entities in various types of relationships.

composite key A primary key that consists of a combination of fields. In that case, the primary key is a combination, composite, concatenated, or multivalued key.

computer resources committee A group of key managers and users responsible for evaluating systems requests. The term "systems review committee" is also used.

computer-aided software engineering (CASE) A technique that uses powerful programs called CASE tools to provide an overall framework for systems development. The tools support a wide variety of design methodologies, including structured analysis and object-oriented analysis. Also referred to as computer-aided systems engineering.

computer-aided systems engineering (CASE) See computer-aided software engineering (CASE).

concatenated key See composite key.

concurrent task A task that can be completed simultaneously (in parallel) with another task.

condition A specified action or state in a structure chart.

confidentiality One of the three main elements of system security: confidentiality, integrity, and availability (CIA). Confidentiality protects information from unauthorized disclosure and safeguards privacy.

configuration management (CM) A process for controlling system requirement changes during the SDLC's development phases. Configuration management also is an important tool for managing system changes and costs after a system becomes operational.

constraint A requirement or a condition that the system must satisfy or an outcome that the system must achieve.

construction phase A phase focused on program and application development tasks similar to the SDLC.

context diagram A top-level view of an information system that shows its boundaries and scope.

context-sensitive A feature that is sensitive to current conditions when it is invoked. For example, context-sensitive help offers assistance for a task in progress.

continuous backup A backup method in which data is automatically backed up every time changes are made.

control break A character that causes specific actions, such as printing subtotals for a group of records.

control break report A detailed report that focuses on control breaks.

control couple In a structure chart, a message, also called a flag, that one module sends to another.

control field order In a control break report, the records are arranged or sorted in the same order as the control fields.

control module In a structure chart, a higher-level module that directs lower-level modules, called subordinate modules.

control structure The building blocks for a process. Control structures have one entry and one exit point. They may be completed in sequential order, as the result of a test or condition, or repeated until a specific condition changes. Also called logical structure.

corporate culture A set of beliefs, rules, traditions, values, and attitudes that define a company and influence its way of doing business.

corporate portal A website that provides various tools and features for an organization's customers, employees, suppliers, and the public.

corrective maintenance Changes to the system to fix errors.

coupling A measure of relationships and interdependence among modules. The opposite of cohesion.

critical path A series of events and activities with no slack time. If any activity along the critical path falls behind schedule, the entire project schedule is similarly delayed. As the name implies, a critical path includes all activities vital to the project schedule.

Critical Path Method (CPM) Shows a project as a network diagram. The activities are shown as vectors, and the events are displayed graphically as nodes. Although CPM was developed separately from the Program Evaluation Review Technique (PERT), the two methods are essentially identical. See also PERT/CPM.

critical success factors Vital objectives that must be completed for the enterprise to fulfill its mission.

critical thinking skills The ability to compare, classify, evaluate, recognize patterns, analyze cause and effect, and apply logic. Such skills are valued in the IT industry.

criticality The importance of a system component or process to an organization's overall operation or mission.

crow's foot notation A type of cardinality notation. It is called crow's foot notation because the shapes, including circles, bars, and symbols, indicate various possibilities. A single bar indicates one possibility, a double bar indicates one and only one, a circle indicates zero, and a crow's foot indicates many.

customer lifetime value (CLV) prediction A metric that represents the total amount of money a customer is expected to spend on your business or your products during their lifetime.

customer relationship management (CRM) A system implemented to integrate all customer-related events and transactions, including marketing, sales, and customer service activities.

customer segmentation The process of dividing a company's customer base into distinct groups based on shared characteristics, such as demographics, purchasing behavior, and preferences, to enable targeted marketing and personalized service.

customer The primary user of a system, service, or product.

cutover phase A phase that resembles the final tasks in the SDLC implementation phase, including data conversion, testing, changeover to the new system, and user training.

data The raw material or basic facts used by information systems.

data center A large concentration of networked computers working together.

data conversion A process in which existing data is loaded into a new system and transformed as needed. Depending on the system, data conversion can be done before, during, or after the operational environment is complete.

data couple In a structure chart, data that one module passes to another.

data dictionary A central storehouse of information about a system's data.

data element A single characteristic or fact about an entity. A data element, field, or attribute is the smallest piece of data that has meaning within an information system. For example, a Social Security number or company name could be examples of a data element. Also called data item.

data flow A path for data to move from one part of the information system to another.

data flow diagram (DFD) A graphical representation of a system, showing it stores, processes, and transforms data into useful information.

data frames Traffic on a computer network.

data item See data element.

data loss prevention (DLP) Strategies and tools used to prevent unauthorized access to, or disclosure of, sensitive information.

data manipulation language (DML) A language that controls database operations, including storing, retrieving, updating, and deleting data. Most commercial DBMSs, such as Oracle and IBM's DB2, use a DML.

data mart A specialized database designed to serve the needs of a specific department, such as sales, marketing, or finance. Each data mart includes only the data that users in that department require to perform their jobs.

data mining Looking for meaningful patterns and relationships among data. For example, data mining software could help a consumer products firm identify potential customers based on their prior purchases.

data processing center A central location where physical data is delivered or transmitted in some manner and entered into the system. Users in the organization have no input or output capability except for printed reports that a corporate IT department distributes.

data repository A location or system where data is stored, organized, and managed for an organization. It can include databases, data warehouses, or other data storage systems.

data science An interdisciplinary field that blends computer science, math and statistics, and business methods to analyze large data sets. It involves artificial intelligence, machine learning and predictive analytics, and visualization techniques.

data security A practice that protects data from loss or damage and recovers data when lost or damaged.

data store A repository where the system stores data while it is processed. In the context of data flow diagrams (DFDs), a data store represents a place where a system can store data for later retrieval, signifying that the data persists beyond the life of the process.

data structure A meaningful combination of related data elements that are included in a data flow or retained in a data store. A framework for organizing and storing data.

data type check A data validation check used to ensure that a data item fits the required data type. For example, a numeric field must have only numbers or numeric symbols, and an alphabetic field can contain only the characters A through Z or a through z.

data validation rule A mechanism to improve input quality by testing the data and rejecting any entry that fails to meet specified conditions.

data warehouse An integrated collection of data that can support management analysis and decision making.

database administrator (DBA) Someone who manages a DBMS. The DBA assesses overall requirements and maintains the database for the benefit of the entire organization rather than a single department or user.

database management system (DBMS) A collection of tools, features, and interfaces that enable users to add, update, manage, access, and analyze data in a database.

database programmer A person who focuses on creating and supporting large-scale database systems.

decision table A table that shows a logical structure with all possible combinations of conditions and resulting actions.

decision tree A graphical representation of the conditions, actions, and rules in a decision table.

decomposing Another way of conveying a process or system that has been broken down from a general, top-level view to more detail. The terms "exploded" and "partitioned" also can be used.

default value A value that a system displays automatically.

defect tracking software Software used to document and track program defects, code changes, and replacement code, called patches. Also called bug tracking software.

deliverable A polished, final product suitable for its intended use. End products or deliverables often coincide with the completion of each SDLC phase.

Delphi technique A structured communication method that uses a panel of experts who participate in multiple rounds of anonymous, independent forecasting and idea generation, typically followed by feedback, to reach a consensus on a specific topic or problem.

denial of service (DoS) An attack that prevents legitimate users from accessing specific computer systems, devices, or other network resources.

dependent task A task is said to be dependent when it has to be completed in a serial sequence.

derivation code A code built by combining data from different item attributes or characteristics. Most magazine subscription codes are derivation codes.

design patterns General reusable solutions to common design problems, serving as templates devised to help solve problems in a more efficient and maintainable manner. They are not blueprints for creating complete applications but are more like descriptions or templates for how to solve a problem that can be used in many different situations, promoting best practices, code reuse, and elegant, robust software and system design.

design prototyping Creating a prototype of user requirements, after which the prototype is discarded and implementation continues. Also called throwaway prototyping.

design review See structured walk-through.

design walkthrough A session with users to review an interface with a cross-section of people who will work with a new system. This continues the modeling and prototyping effort that began early in the systems development process.

desk checking The process of reviewing program code to spot logic errors that produce incorrect results.

detail report A report that produces one or more lines of output for each record processed.

DevSecOps An approach that integrates security practices within the DevOps process, aiming to build security into the development life cycle from the start.

diagram 0 A diagram depicting the first level of detail below the initial context diagram. Diagram 0 (zero) zooms in on the context diagram and shows major processes, data flows, and data stores, as well as repeating the external entities and data flows that appear in the context diagram.

dialog box A window that allows a user to enter information about a task a system will perform.

differential backup A type of backup that copies all the data that has changed since the last full backup.

digital process management (DPM) The optimization and automation of business processes using digital technologies to streamline operations and improve efficiency.

digital rights management (DRM) Technologies that control the use of digital content and devices after sale.

direct cutover An approach that causes the changeover from an old system to the new system to occur immediately when the new system becomes operational.

disaster recovery planning (DRP) The process of developing advance arrangements and procedures to enable an organization to recover from a disaster.

discretionary projects Where management has a choice in implementing a project, it is called discretionary. For example, creating a new report for a user is a discretionary project.

diskless workstation A network terminal that supports a full-featured UI but limits the printing or copying of data, except from specific network resources that can be monitored and controlled more easily.

distributed database management system (DDBMS) A system for managing data stored at multiple locations. Using a DDBMS offers several advantages: data stored closer to users can reduce network traffic; the system is scalable, so new data sites can be added without reworking the system design; and with data stored in various locations, the system is less likely to experience a catastrophic failure. A potential disadvantage of distributed data storage involves data security. Maintaining controls and standards can be more difficult when data is stored in various locations.

distributed denial of service (DDoS) A variant of the DoS attack that uses multiple compromised systems to target a single system.

distributed system A company-wide system connected by one or more LANs or WANs. The capabilities of a distributed system depend on the power and capacity of the underlying data communication network.

diverging data flow A data flow in which the same data travels to two or more different locations.

document review A review of baseline documentation. A useful fact-finding technique that helps an analyst understand how the current system is supposed to work.

documentation Material that explains a system and helps people interact with it. This material includes program documentation, system documentation, operations documentation, and user documentation.

domain The set of values permitted for a data element.

dumpster diving The practice of sifting through commercial or residential waste to find information that can be used in an attack.

duration The amount of time it will take to complete a task.

EBCDIC The abbreviation for Extended Binary Coded Decimal Interchange Code, a coding method used on mainframe computers and some high-capacity servers.

e-commerce (electronic commerce) Transactions (e.g., buying and selling of goods and information) that occur on the Internet. Includes both business-to-consumer and business-to-business.

economic feasibility Achieved if the projected benefits of the proposed system outweigh the estimated costs involved in acquiring, installing, and operating it.

economy of scale Cost advantages a business can achieve due to increased production, where the cost per unit of output decreases as the scale of output grows.

electronic data interchange (EDI) A process that involves the computer-to-computer transfer of data between companies.

electronic data interchange (EDI) The exchange of business documents between computers using a standard electronic format.

electronic health record (EHR) An electronic record of a patient's health information generated as the patient encounters various healthcare providers and shared among multiple facilities and agencies.

electronic product code (EPC) Technology that uses RFID tags to identify and monitor the movement of each product from the factory floor to the retail checkout counter.

electronic proof of delivery (EPOD) A supplier uses RFID tags on each crate, case, or shipping unit to create a digital shipping list to verify receipt of goods.

empowerment A business practice that places more responsibility and accountability throughout all levels of an organization.

encapsulation The idea that all data and methods are self-contained, as in a black box.

encryption A process where data is coded (converted into unreadable characters) so that only those with the required authorization can access the data.

engaged listening The ability to concentrate on what someone is saying and avoid the temptation to hear what is expected. It also includes noticing nonverbal communication.

enhancement A new feature or capability.

enterprise applications Company-wide applications, such as order processing systems, payroll systems, and company communications networks.

enterprise computing Information systems that support company-wide data management requirements, such as airline reservations or credit card billing systems.

enterprise resource planning (ERP) A process that establishes an enterprise-wide strategy for IT resources. ERP defines a specific architecture, including data, processing, network, and user interface design standards.

entity A person, place, thing, or event for which data is collected and maintained. For example, an online sales system may include CUSTOMER, ORDER, PRODUCT, and SUPPLIER entities.

entity-relationship diagram (ERD) A graphical information system model that depicts the relationships among system entities.

epic In an agile project, a simple, high-level statement of a requirement. See feature.

evaluation and selection team A group of people involved in selecting hardware and software. The group includes systems analysts and users. A team approach ensures that critical factors are noticed and a sound choice is made.

evaluation model A technique that uses a common yardstick to measure and compare vendor ratings.

event A reference point that marks a major occurrence. Used to monitor progress and manage a project. See also milestone.

exception report A document displaying only those records that meet a specific condition or conditions. Exception reports are helpful when the user wants information only on records that might require action but does not need to know the details.

existence check A type of data validation check that is used for mandatory data items. For example, if an employee record requires a Social Security number, an existence check only allows users to save the record once they enter a suitable value in the SSN field.

exploding A diagram is said to be exploded if it "drills down" to a more detailed or expanded view.

exploit A piece of software, a chunk of data, or a sequence of commands that takes advantage of a bug or vulnerability to cause unintended or unanticipated behavior on computer software or hardware.

extended service set (ESS) A wireless network configuration that comprises two or more BSS networks, allowing wireless clients to roam from BSS to BSS.

extensibility A system's ability to expand, change, or downsize quickly to meet the changing needs of a business enterprise. Also known as scalability.

fact-finding The process of gathering requirements. See requirements elicitation.

fat client A network design that locates all or most of the application processing logic at the client. Also called a thick client design.

fault management The timely detection and resolution of operational problems, including monitoring a system for signs of trouble, logging all system failures, diagnosing the problem, and applying corrective action.

fault tolerance The ability of a system to continue operating without interruption when one or more of its components fail.

faxback See automated fax.

feasibility study An initial investigation to identify the nature and scope of the business opportunity or problem. Also called a preliminary investigation.

feature In an agile project, a simple, high-level statement of a requirement. See epic.

field See data element.

file A document or table that contains data about people, places, things, or events that interact with an information system.

file-oriented system A system, also called a file-processing system, that stores and manages data in one or more separate files.

fill-in form A template used to collect data on the Internet or a company intranet.

finish day/date When a task is scheduled to be completed.

firewall A network security device that monitors and controls incoming and outgoing network traffic based on predetermined security rules.

first normal form (1NF) A record that does not contain a repeating group (a set of data items that can occur any number of times in a single record).

fishbone diagram An analysis tool that represents the possible causes of a problem as a graphical outline. Also called an Ishikawa diagram.

fixed fee model A service model that charges a set fee based on a specified level of service and user support.

flowchart A diagram used to describe program logic that represents logical rules and interaction graphically using a series of symbols connected by arrows. Flowcharts can be helpful in visualizing modular program designs.

focus In a sequence diagram, an indicator of when an object sends or receives a message. It is characterized by a narrow vertical rectangle that covers the lifeline.

foreign key A field in one table that matches a primary key value in another table to establish the relationship between the two tables.

form filling A very effective method of online data entry in which a blank form that duplicates or resembles the source document is completed on the screen. The user enters data and then moves to the following field.

form layout The physical appearance and placement of data on a form. Form layout makes the form easy to complete and provides enough vertical and horizontal space for users to enter the data.

forum An online discussion on a particular topic where people meet, offer support, and exchange ideas.

four-model approach An approach in which a physical model of the current system, a logical model of the current system, a logical model of the new system, and a physical model of the new system are all developed.

full backup A complete copy of all data in the system at a specific time.

functional baseline The system configuration as documented at the beginning of the project. It consists of all the necessary system requirements and design constraints.

functional decomposition diagram (FDD) A top-down representation of business functions and processes. Also called a structure chart.

functional primitive A single function that is not exploded further. The logic for functional primitives is documented in a data dictionary process description.

functional requirement A statement of the services a system provides.

functionally dependent An important concept for understanding the second normal form (2NF). The field X is said to be functionally dependent on the field Y if the value of X depends on the value of Y. For example, an order date depends on an order number; for a particular order number, there is only one value for the order date. In contrast, the product description is not dependent on the order number. There might be several product descriptions for a particular order number, one for each item ordered.

gamification Applying game-design elements and principles to improve user engagement, organizational productivity, learning, and participation.

Gane and Sarson symbols A popular symbol set used in data flow diagrams. Processes, data flows, data stores, and external entities have unique symbols.

Gantt chart A horizontal bar chart that illustrates a schedule. Developed many years ago by Henry L. Gantt as a production control technique but still in common use today.

garbage in, garbage out (GIGO) The concept that the quality of the output is only as good as the quality of the input.

gateway (1) In business processing modeling notation, a fork in the process that allows the flow to go one way or another. (2) A router or other network device used to connect to a larger, dissimilar type of network, such as the Internet.

Gbps (gigabits per second) A bandwidth or throughput measurement.

General Data Protection Regulation (GDPR) A comprehensive data protection law in the European Union that sets guidelines for the collection, processing, and storage of personal information, emphasizing privacy and data rights.

global outsourcing The practice of shifting IT development, support, and operations to other countries.

graphical user interface (GUI) The use of graphical objects and techniques that allow users to communicate with a system. A well-designed GUI can help users learn a new system rapidly and work with the system effectively.

gray hole A process with an input obviously insufficient to generate the shown output.

groupware Programs that run on a network enabling users to share data, collaborate on projects, and work in teams. Also called workgroup software.

hardening The process of securing a system by reducing its surface of vulnerability.

hardware The physical layer of the information system, including computers, networks, communications equipment, and other technology-based infrastructure.

hash totals Numbers that are not meaningful themselves but are useful for comparison purposes. Also known as batch control totals.

Hawthorne Effect A phenomenon in which employees who know they are being observed are more productive.

help desk A centralized resource staffed by IT professionals that provides users with the support they need to do their jobs. A help desk has three main objectives: to show people how to use system resources more effectively, provide answers to technical or operational questions, and make users more productive by teaching them how to meet their information needs.

hierarchical network A network design in which one computer (typically a mainframe) controls the entire network. Satellite computers or servers control lower levels of processing and network devices.

HIPAA The Health Insurance Portability and Accountability Act, a U.S. law designed to provide privacy standards to protect patients' medical records and other health information.

histogram A standard tool for showing the distribution of questionnaires or sampling results. It takes the form of a vertical bar chart.

horizontal application A software package that many different types of organizations can use.

horizontal system A basic system, such as an inventory or payroll package, commonly used by a variety of companies.

HTTP/2 The second major version of the network protocol used by the web. Released as a standard in 2015.

hub The center of a star network. Switches in modern networks have largely replaced hubs.

human–computer interaction (HCI) A description of the relationship between computers and those who use them to perform business-related tasks. HCI concepts apply to everything from a PC desktop to the main menu for a global network.

identity management The administrative process of identifying individuals in a system and controlling their access to system resources by associating user rights and restrictions with their established identity.

identity theft The fraudulent acquisition and use of a person's private identifying information, usually for financial gain.

IEEE 802.11i A standard for wireless network security that addresses authentication and encryption.

incentivization Using rewards or incentives to encourage specific actions or behaviors to achieve desired outcomes or improve performance.

incremental backup A backup method that only copies data that has changed since the last backup.

inference rules Instructions that direct a knowledge management system to identify data patterns and relationships.

informal structure An organization based on interpersonal relationships, which can develop from previous work assignments, physical proximity, unofficial procedures, or personal relationships.

information Data that has been changed into a useful form of output.

information system A combination of information technology, people, and data to support business requirements. The five key components are hardware, software, data, processes, and people.

information technology (IT) A combination of hardware, software, and telecommunications systems that support business operations, improve productivity, and help managers make decisions.

infrastructure mode A wireless network configuration in which a central wireless device called an access point serves all wireless clients. Also called basic service set (BSS).

inheritance A type of object relationship. Inheritance enables an object to derive one or more of its attributes from another object (e.g., an INSTRUCTOR object may inherit many traits from the EMPLOYEE object, such as hire date).

in-house software Custom software developed by an organization's own IT staff to meet unique business needs and requirements.

input control The necessary measures to ensure that input data is correct, complete, and secure. A systems analyst must focus on input control during every phase of input design, starting with source documents that promote data accuracy and quality.

input mask A template or pattern that makes it easier for users to enter data. It is often used in automated forms to guide an unfamiliar user.

instance A specific member of a class.

Institute of Electrical and Electronics Engineers (IEEE) A professional organization that establishes standards for telecommunications.

intangible benefits Positive outcomes that are difficult to measure in dollars. However, intangible benefits can be very important in the calculation of economic feasibility. An example of an intangible benefit is a new website that improves a company's image.

intangible costs Items that are difficult to measure in dollar terms, such as employee dissatisfaction.

integrated development environment (IDE) An application for building other software applications. It typically includes a visual code editor, an integrated compiler, a debugger, a configuration management system, and a test framework.

integration testing The testing of two or more programs that depend on each other.

integrity One of the three main elements of system security: confidentiality, integrity, and availability (CIA). Integrity prevents unauthorized users from creating, modifying, or deleting information.

intellectual property rights (IPR) Legal rights that grant creators control over their intellectual creations.

International Organization for Standardization (ISO) A network of national standards institutes from 140 countries working in partnership with international organizations, governments, industry, business, and consumer representatives. The ISO acts as a bridge between the public and private sectors.

Internet business services (IBS) Services that provide powerful web-based support for transactions such as order processing, billing, and customer relationship management.

Internet of Things (IoT) Devices connected over a computer network.

Internet operating system Part of the Web 2.0 model, an online computing environment created by online communities and services, based on layers of shared information that can contain text, sound bites, images, and video clips.

interview A planned meeting during which information is obtained from another person.

intrusion detection systems (IDS) Devices or software applications that monitor networks or systems for malicious activity or policy violations.

ISO 9001:2015 A global standard from the International Organization for Standardization (ISO) for quality management systems that outlines the criteria for a company to ensure consistent quality of services and products, enhancing customer satisfaction and meeting regulatory requirements through continual improvement.

ISO/IEC 27000 A series of standards for managing and securing information assets.

ISO/IEC/IEEE 90003:2018 Guidelines for applying ISO 9001:2015 to computer software.

iteration cycle An agile development cycle that includes planning, designing, coding, and testing one or more features based on user stories.

iteration planning meeting In agile development, a meeting held at the beginning of each iteration cycle to break down user stories into specific tasks assigned to team members.

iteration The completion of a process step that is repeated until a specific condition changes.

iterative Involving repetition.

JDBC (Java database connectivity) A standard that enables Java applications to exchange data with any database that uses SQL statements and is ODBC-compliant.

joint application development (JAD) A systems development technique that uses a task force of users, managers, and IT professionals who work together to gather information, discuss business needs, and define the new system requirements.

just-in-time (JIT) The exchange or delivery of information when and where it is needed. For example, JIT inventory systems rely on computer-to-computer data exchange to minimize unnecessary inventory.

Kbps (kilobits per second) A bandwidth or throughput measurement.

key fields Data fields used during the systems design phase to organize, access, and maintain data structures.

key process indicator (KPI) A measurable value demonstrating how effectively a company is achieving key business objectives, specifically related to its operational processes.

keystroke logger A type of surveillance technology used to monitor and record keystrokes typed on a keyboard.

knee of the curve The point at which response times increase dramatically in a client/server computing environment. Client/server response times tend to increase gradually and then rise dramatically as the system nears its capacity.

knowledge base A popular systems development technique that uses a group of users, managers, and IT professionals working together to gather information, discuss business needs, and define the new system requirements.

knowledge process outsourcing (KPO) A form of outsourcing in which high-value, knowledge-based tasks and analytical processes are delegated to external organizations or specialists.

leading questions Queries that suggest or favor a particular reply.

legacy data The data associated with an older, less technologically advanced legacy system.

legacy system An older system that is typically less technologically advanced than currently available systems.

legal process outsourcing (LPO) The practice of law firms or corporate legal departments outsourcing legal tasks, such as document review and legal research, to external, specialized service providers, often in different countries.

leveling The process of drawing a series of increasingly detailed diagrams to reach the desired level of detail.

library module In a structure chart, a module that is reusable and can be invoked from more than one point in the chart.

lifeline In a sequence diagram, a representation of the time during which the object above it can interact with the other objects in the use case. An X marks the end of a lifeline.

limit check Part of a validation check that involves a minimum or maximum value but not both. Checking that a payment amount exceeds zero but not specifying a maximum value is an example of a limit check.

list box An output mechanism that displays a list of choices the user can select.

local area network (LAN) A network design that allows the sharing of data and hardware, such as printers and scanners. Advances in data communication technology have made it possible to create robust networks that use satellite links, high-speed fiber-optic lines, and the Internet to share data.

log A record of events or processes in a computer or network.

logic errors Mistakes in the underlying logic that produce incorrect results.

logical model A model that shows what a system must do, regardless of how it will be implemented physically.

logical record A record that contains field values describing a person, place, thing, or event. Application programs see a logical record as a set of fields, regardless of how or where the data is stored physically.

logical storage A reference to information seen through a user's eyes, regardless of how or where that information is organized or stored.

logical structure See control structure.

logical topology A view of a network that describes how the components interact rather than the actual network cabling and connections.

loop In a structure chart, an indicator that one or more modules are repeated.

looping A process that is repeated until a specific condition changes. For example, looping is a process that continues to print paychecks until the payroll file ends. Also known as repetition.

loosely coupled Modules that are relatively independent. They are easier to maintain and modify because one module's logic does not affect others.

M:N A type of entity relationship. A many-to-many relationship, abbreviated M:N, exists when one instance of the first entity can be related to many instances of the second entity and one instance of the second can be related to many instances of the first entity.

machine learning (ML) A subfield of artificial intelligence that uses algorithms to automatically learn from data, adapt to new inputs, and improve from experience without being explicitly programmed.

machine learning An application of computer science and artificial intelligence that uses automated approaches to pattern recognition and predictive analytics based on large data sets.

mainframe architecture A system design in which the server performs all the processing.

maintenance activities Changes to programs, procedures, or documentation to ensure correct system performance and adapting the system to changing requirements to make it operate more efficiently. These activities include corrective, adaptive, perfective, and preventive maintenance.

maintenance agreement A specification of the conditions, charges, and time frame for users to contact the vendor for assistance when they have system problems or questions.

maintenance expenses Costs that vary significantly during the system's operational life and include spending to support maintenance activities.

maintenance release methodology A system of numbered releases used by organizations (especially software vendors) that helps organize maintenance changes and updates.

maintenance release A formal release of a new system version that contains several changes.

maintenance team One or more systems analysts and programmers working on product maintenance issues together.

make or buy A choice between developing in-house software and purchasing software. See also build or buy.

malware Malicious software designed to harm, exploit, or otherwise engage in illegal activity on a computer or network.

managed hosting An operation managed by an outside firm, or host. Another term for Internet business services (IBS).

management information system (MIS) A computer-based information system used in business planning, control, decision making, and problem solving.

many-to-many relationship See M:N.

market basket analysis A type of analysis that can detect patterns and trends in large amounts of data.

Mbps (megabits per second) A bandwidth or throughput measurement.

menu bar A set of user-selectable options in a software application, usually located across the top of the screen.

mesh network A network design in which each node connects to every other node. While this design is very reliable, it is also expensive to install and maintain.

message An object-oriented command that tells an object to perform a particular method.

method A definition of specific tasks that an object must perform. A method describes what an object does and how.

metrics Workload measurements that can include the number of lines printed, the number of records accessed, and the number of transactions processed in a given time period.

middleware Software that connects dissimilar applications and enables them to communicate and exchange data. For example, middleware can link a departmental database to a web server that client computers can access via the Internet or a company intranet.

milestone A reference point that marks a significant occurrence. Used to monitor progress and manage a project. See also event.

mission statement A document or statement that describes the company for its stakeholders and briefly states its overall purpose, products, services, and values.

mission-critical system An information system that is vital to a company's operations.

mitigation One of four risk control strategies. Mitigation reduces the impact of a risk by careful planning and preparation. For example, a company can prepare a disaster recovery plan to mitigate the effects of a natural disaster.

mnemonic code Ciphers using a specific combination of letters that are easy to remember. Many three-character airport codes are mnemonic codes. For example, LAX represents Los Angeles.

mobile devices Smartphones, tablets, and other computing devices that are not permanently tethered to a desk. They connect to the network wirelessly.

mock-up During report design, a sample report or prototype for users to review. The sample should include typical field values and contain enough records to show all the design features.

model-based systems engineering (MBSE) An approach to systems engineering that relies on domain models rather than traditional documents to design large-scale systems and convey information between engineers.

modeling An activity that produces a graphical representation of a concept or process that systems developers can analyze, test, and modify.

modular design A design that can be broken down into logical blocks. Also known as partitioning or top-down design.

modules Related program code organized into small, easy-to-understand, and maintainable units. A complex program could have hundreds or even thousands of modules.

Moore's Law A prediction that computing power would double every 18 to 24 months due to the increased miniaturization of electronic components.

multipath design A network design that relies on multiple data paths to increase bandwidth and range using MIMO technology.

multiple input/multiple output (MIMO) A wireless networking technology incorporated in the IEEE 802.11n and 802.11ac standards that uses multiple data streams and multiple antennas to achieve higher transmission speeds and substantially increase wireless range over earlier standards.

multivalued key A primary key that consists of a combination of fields. In that case, the primary key is called a combination key, composite key, concatenated key, or multivalued key.

natural language A software feature that allows users to type commands or requests in ordinary English (or other languages).

near-field communication (NFC) A technology that allows wireless data transmission over short distances using radio waves.

net present value (NPV) The total value of the benefits minus the total value of the costs, with both the costs and benefits adjusted to reflect the time they occur.

net-centric computing A distributed environment in which applications and data are downloaded from servers and exchanged with peers across a network on an as-needed basis.

network diagram Another name for a PERT chart.

network interface A hardware component or software interface that enables a computer to connect to a network.

network intrusion detection system (NIDS) A system that monitors network traffic for suspicious activity and issues alerts when such activity is detected.

network topology The way a network is configured. LAN and WAN networks typically are arranged in one of four common patterns: hierarchical, bus, star, and ring.

network A group of computers and other devices connected to share resources and information.

NIST Cybersecurity Framework A policy framework of computer security guidance for private-sector organizations in the United States to assess and improve their ability to prevent, detect, and respond to cyberattacks.

node A physical device, wired or wireless, that can send, receive, or manage network data.

nondiscretionary projects Where management has no choice in implementing a project, it is called nondiscretionary. For example, adding a report required by a new federal law is a nondiscretionary project.

non-functional requirements A statement of operational system constraints.

non-key field Any field that is not a primary or candidate key.

normalization A process by which analysts identify and correct inherent problems and complexities in their record designs.

NoSQL databases Database systems that use a flat, nontabular (nonrelational) structure to store and process large-scale data sets.

n-tier design A multilevel design or architecture. For example, three-tier designs are called n-tier designs, indicating that some designs use more than one intermediate layer.

object A representation of a real person, place, event, or transaction.

object In object-oriented analysis or programming, an object represents a real person, place, event, or transaction.

object model A description of objects that combine data and processes. Object models are the end product of object-oriented analysis.

object-oriented (O-O) analysis A description of an information system by identifying objects. An object represents a real person, place, event, or transaction. O-O analysis is a popular approach that sees a system from the viewpoint of the objects as they function and interact with the system.

object-oriented (O-O) analysis The act of understanding an information system by identifying objects that represent a real person, place, event, or transaction. Object-oriented analysis is a popular approach that sees a system from the viewpoint of the objects as they function and interact with the system.

object-oriented development (OOD) The process of translating an object model directly into an object-oriented programming language.

observation A fact-finding technique in which an analyst sees a system in action. Observation allows the verification of statements made in interviews.

ODBC (open database connectivity) An industry-standard protocol that allows software from different vendors to interact and exchange data.

offshore outsourcing The practice of shifting IT development, support, and operations to other countries.

offshoring See offshore outsourcing.

one-to-many relationship See 1:M.

one-to-one relationship See 1:1.

online data entry A data entry method used for most business activity. The online method offers significant advantages, including immediate validation and data availability.

online documentation Materials that provide immediate help when users have questions or encounter problems.

online system A system that handles transactions when and where they occur and provides output directly to users. Because it is interactive, online processing avoids delays and allows a constant dialogue between the user and the system.

onshoring See reshoring.

Open Web Application Security Project (OWASP) An online community that produces freely available articles, methodologies, documentation, tools, and technologies in web application security.

open-ended questions Queries that provide a range of answers. They encourage spontaneous and unstructured responses and are useful for understanding a larger process.

operational costs Expenses incurred after a system is implemented and that continue while the system is in use. Examples include system maintenance, supplies, equipment rental, and annual software license fees.

operational environment The environment for the actual system operation. It includes hardware and software configurations, system utilities, and communications resources. Also called the production environment.

operational feasibility A system that will be used effectively after it has been developed.

operational security The process of identifying critical information and analyzing friendly actions that are attendant to military operations and other activities.

operations documentation The information needed for processing and distributing online and printed output.

option buttons Radio buttons that represent groups of options. The user can select only one option at a time; a selected option contains a black dot. See also radio buttons.

orphan An unassociated or unrelated record or field. An orphan could be created if a customer order was entered in an order table when that customer did not already exist in the customer table. Referential integrity would prevent the creation of this orphan.

OSI (Open Systems Interconnection) model A model that describes how data moves from an application on one computer to another networked computer. The OSI model consists of seven layers, each performing a specific function.

output control Methods to maintain output integrity and security. For example, every report should include an appropriate title, report number or code, printing date, and period covered. Reports should have pages that are numbered consecutively and identified as "Page xx of xx," and the end of the report should be labeled clearly.

output security A practice that protects privacy rights and shields an organization's proprietary data from theft or unauthorized access.

outsourcing The transfer of information systems development, operation, or maintenance to an outside firm that provides the services for a fee on a temporary or long-term basis.

page footer Text in a report that appears at the bottom of each page and displays the name of the report and the page number.

page header Text in a report that appears at the top of each page and includes the column headings to identify the data.

pair programming A practice in Extreme Programming in which two programmers work on the same task on the same computer; one drives (programs) while the other navigates (watches).

parallel operation A changeover method that requires both the old and new information systems to operate fully for a specified period. Data is input into both systems and the output generated by the new system is compared with the equivalent output from the old system.

parent diagram The higher-level or more top-level diagram in an exploded data flow diagram.

parent In inheritance, an object from which the other object, the child, derives one or more attributes.

Pareto chart A vertical bar graph named for a 19th-century economist. The bars, which represent various causes of a problem, are arranged in descending order so the team can focus on the most important causes.

partitioning The breaking down of overall objectives into subsystems and modules.

patch Replacement code that is applied to fix bugs or security holes in software.

payback analysis A determination of how long it takes an information system to pay for itself through reduced costs and increased benefits.

PCI DSS The Payment Card Industry Data Security Standard, a set of security standards designed to ensure that all companies that accept, process, store, or transmit credit card information maintain a secure environment.

perfective maintenance Changes to a system to improve efficiency.

permissions User-specific privileges that determine the type of access a user has to a database, file, or directory. Also called user rights.

personal digital assistant A program that responds to user requests through a natural interface, such as regular speech, to aid general-purpose queries. Often embedded in devices such as Internet-connected speakers and smartphones.

personal information manager (PIM) A tool that helps manage tasks and schedules. Many handheld devices also include this function.

person-day The amount of work that one person can complete in one day.

PERT/CPM The Program Evaluation Review Technique (PERT) was developed by the U.S. Navy to manage complex projects, such as constructing nuclear submarines. At approximately the same time, the Critical Path Method (CPM) was developed by private industry to meet similar project management needs. The essential distinctions between the two methods have disappeared over time, and today the technique is called either PERT, CPM, or PERT/CPM.

PESTEL framework An analysis used in assessing external factors based on six environmental types: political, economic, social, technological, environmental, and legal.

phased operation A changeover method that allows a new system to be implemented in stages, or modules.

physical model A model that describes how a system will be constructed.

physical storage An information storage mechanism that is strictly hardware-related because it involves the process of reading and writing binary data to physical media, such as a hard drive or a USB thumb drive.

physical topology The connection structure of a network's cabling.

pilot operation A changeover method that involves implementing a complete new system at a selected company location.

pilot site In a pilot operation, the group that uses the new system first.

plaintext Unencrypted text that is readable without any decryption.

planning game An agile software development practice in which developers and customers collaborate to create a plan through iterative cycles, prioritizing features and estimating efforts to maximize value delivery within time and budget constraints.

platform A specific hardware and software configuration that supports IT business goals such as hardware connectivity and easy integration of future applications. Also called an environment.

podcast A web-based broadcast that allows a user to receive audio or multimedia files using media player software and listen to them on a computer or download them to a portable device or smartphone.

point-of-sale (POS) The part of an information system that handles daily sales transactions and maintains the online inventory file.

polymorphism The concept that a message gives different meanings to different objects (e.g., a GOOD NIGHT message might produce different results depending on whether a child or the family dog receives it).

pool The overall diagram in business process modeling notation (BPMN).

port scan The act of systematically scanning a computer's ports to find vulnerabilities.

port A virtual data connection that can be used by programs and services to exchange information.

portal An entrance to a multifunction website. After entering a portal, a user can navigate to a destination using various tools and features provided by the portal designer.

post-implementation evaluation An assessment of the overall quality of the information system. The evaluation verifies that the new system meets specified requirements, complies with user objectives, and achieves the anticipated benefits. In addition, by providing feedback to the development team, the evaluation helps improve IT development practices for future projects.

power-on password A security feature that prevents unauthorized access to a computer's contents by requiring a password upon startup.

predecessor task A single prior task upon which two or more concurrent tasks depend.

predictive analytics Historical data, statistical algorithms, and machine learning techniques used to identify the likelihood of future outcomes and help businesses make informed decisions.

preliminary investigation An initial analysis to identify the nature and scope of the business opportunity or problem. Also called a feasibility study.

pretexting A form of social engineering in which an attacker lies to obtain privileged data.

preventive maintenance Changes to a system to reduce the possibility of future failure.

primary key A field or combination of fields that uniquely and minimally identifies a particular member of an entity. For example, the customer number is a unique primary key in a customer table because no two customers can have the same customer number. That key is also minimal because it contains no information beyond what is needed to identify the customer.

private key encryption A type of encryption in which the key used to decrypt information is kept secret and not shared.

private network A network that is restricted to select users through security measures.

privilege escalation attack An attack that exploits a bug, design flaw, or configuration oversight in an operating system or software application to gain elevated access to resources that are usually protected.

probable-case estimate The most likely outcome.

procedural security Measures and policies that are implemented to protect data and manage risks through procedures and administrative actions.

process 0 In a data flow diagram, process 0 (zero) represents the entire information system but does not show the internal workings.

process A procedure or task that users, managers, and IT staff perform. Also, the logical rules of a system that are applied to transform data into meaningful information. In data flow diagrams, a process receives input data and produces output with a different content, form, or both.

process description Documentation of a functional primitive's details, representing a specific set of processing steps and business logic.

process improvement The framework used to integrate software and systems development by a new SEI model, CMMI.

product baseline A reference point that describes the system at the beginning of operation. The product baseline incorporates any changes made since the allocated baseline and includes the results of performance and acceptance tests for the operational system.

product lifecycle management (PLM) See application lifecycle management (ALM).

production environment The environment for the actual system operation. It includes hardware and software configurations, system utilities, and communications resources. Also called the operational environment.

productivity software Applications such as word processing, spreadsheet, database management, and presentation graphics programs.

product-oriented Companies that manufacture computers, routers, or microchips.

program documentation Documentation that starts in the systems analysis phase of a project and continues during systems implementation. Systems analysts prepare overall documentation, such as process descriptions and report layouts, early in the SDLC. Programmers provide documentation by constructing modules that are well supported by internal and external comments and descriptions that can be understood and maintained easily.

Program Evaluation Review Technique (PERT) See PERT/CPM.

programmer/analyst A designation for positions that require a combination of systems analysis and programming skills.

project coordinator The person who handles administrative responsibilities for the development team and negotiates with users who might have conflicting requirements or want changes that would require additional time or expense.

project creep The process by which projects with very general scope definitions expand gradually, without specific authorization.

project leader The person charged with leading a project from a technical perspective.

project management The process of planning, scheduling, monitoring, controlling, and reporting on the development of an information system.

project manager The person charged with managing a project from an administrative perspective.

project monitoring Guiding, supervising, and coordinating the project team's workload.

project planning Identifying project tasks and estimating completion time and costs.

project reporting Providing regular progress reports to management, users, and the project team.

project scheduling Creating a specific timetable to facilitate project completion. Also involves selecting and staffing the project team and assigning specific tasks to team members.

project scope A specific determination of a project's boundaries or extent.

project triangle Cost, scope, and time are the three major components of a project. A project manager tries to find the optimal balance among these factors.

properties In object-oriented (O-O) analysis, characteristics that objects inherit from their class or possess on their own.

prototype An early, rapidly constructed working version of the proposed information system.

prototyping The method by which a prototype is developed. It involves a repetitive sequence of analysis, design, modeling, and testing. It is a common technique that can be used to design anything from a new home to a computer network.

proxy server A networking device that provides Internet connectivity for internal LAN users.

pseudocode A technique for representing program logic.

pseudonymization The process of replacing private identifiers with fake identifiers or pseudonyms, allowing data to be matched with its source without revealing the source's identity.

public key encryption (PKE) An encryption system that uses two keys: a public key for encrypting messages and a private key for decrypting them.

qualitative risk analysis Evaluating risk by estimating the probability that an adverse event will occur and the degree of impact.

quality assurance (QA) A process or procedure for minimizing errors and ensuring product quality. Poor quality can result from inaccurate requirements, design problems, coding errors, faulty documentation, and ineffective testing. A QA team reviews and tests all applications and systems changes to verify specifications and software quality standards.

quality attributes See non-functional requirements.

quantitative risk analysis Evaluating risk in terms of the actual effect in terms of dollars, time, project scope, or quality.

query by example (QBE) A language that allows the user to provide an example of the data requested.

query language A language that enables the user to specify a task without specifying how the task will be accomplished. Some query languages use natural language commands that resemble ordinary English sentences.

questionnaire A document containing several standard questions that can be sent to many individuals. Also called a survey.

radio buttons Buttons that represent groups of options. The user can select only one option at a time; a selected option contains a black dot. See also option buttons.

radio frequency identification (RFID) Technology that uses high-frequency radio waves to track physical objects.

RAID (redundant array of independent disks) A data storage virtualization technology that combines multiple physical disk drive components into one or more logical units for data redundancy, performance improvement, or both.

random sample A selection taken in an arbitrary, unplanned manner—for example, a sample that selects any 20 customers.

range check A type of data validation check that tests data items to verify that they fall between a specified minimum and maximum value. The daily hours worked by an employee, for example, must fall within the range of 0 to 24.

range-of-response questions Closed-ended questions that ask a person to evaluate something by providing limited answers to specific responses or on a numeric scale.

rapid application development (RAD) A team-based technique that speeds up information systems development and produces a functioning information system. RAD is similar in concept to joint application development (JAD) but includes all phases of the systems development life cycle (SDLC).

reasonableness check A type of data validation check that identifies values that are questionable but not necessarily wrong. For example, input payment values of $0.05 and $5,000,000.00 pass a simple limit check for a payment value greater than zero, yet both could be errors.

record A set of related fields that describes one instance or member of an entity, such as one customer, one order, or one product. Depending on the information needed, a record might have one or dozens of fields.

records retention policy Rules designed to meet all legal requirements and business needs for keeping records.

recovery point objective (RPO) The maximum acceptable amount of data loss measured in time before a system failure or disaster occurs, indicating how frequently data backups should be performed.

recovery procedure A process for restoring data and restarting a system after an interruption. Recovery procedures can restore a file or database to its current state during the last backup.

recovery The process of restoring data and restarting a system after an interruption.

recovery time objective (RTO) The maximum acceptable duration of time that a service or system can be offline or unavailable after a failure or disaster before the impact becomes unacceptable to the business.

referential integrity A type of validity check. Referential integrity is a set of rules that avoid data inconsistency and quality problems.

relational database A database in which tables are related by common fields, creating a unified data structure that provides improved data quality and access.

relational model A model used in relational databases. The relational model was introduced during the 1970s and became popular because it was flexible and powerful.

relationships Ways that objects communicate and interact as they perform the business functions and transactions a system requires. Relationships describe what objects need to know about each other, how objects respond to changes in other objects, and the effects of membership in classes, superclasses, and subclasses.

release plan In agile development, a plan that specifies when user stories will be implemented and the timing of the releases. Releases are relatively frequent, and each release is treated as a system prototype that can be tested and modified as needed.

remote control software Applications that allow IT staff to take over a user's workstation and provide support and troubleshooting.

repeating group A set of one or more fields that can occur any number of times in a single record, with each occurrence having different values.

report footer Text that appears at the end of the report and can include grand totals for numeric fields and other end-of-report information.

report header Text that appears at the beginning of a report and identifies the report using the report title, date, and other necessary information.

request for proposal (RFP) A written list of features and specifications given to prospective vendors before a specific product or package has been selected.

request for quotation (RFQ) A document used to obtain a price quotation or bid on a particular product or package.

requirements definitions A description of the system requirements from the user's point of view.

requirements elicitation The process of gathering requirements. See fact-finding.

requirements engineering Used in the systems planning phase of the SDLC. It involves fact-finding to describe the current system and identify the requirements for the new system.

requirements modeling Used in the systems planning phase of the SDLC. It involves using various fact-finding techniques, such as interviews, surveys, observation, and sampling, to describe the current system and identify the requirements for the new system.

requirements planning phase A phase that combines elements of the systems planning and systems analysis phases of the SDLC.

requirements specifications A description of the system requirements from the analyst or engineering team's point of view.

research An important fact-finding technique that includes reviewing journals, periodicals, and books to obtain background information, technical material, and news about industry trends and developments.

reshoring Bringing back previously offshored operations to their home country.

response time The overall time between a request for system activity and the delivery of the response. In the typical online environment, response time is measured from the instant the user presses the Enter key or clicks a mouse button until the requested screen display appears or printed output is ready.

retention period The time that records or data should be kept before destruction or deletion.

return on investment (ROI) A percentage rate that measures profitability by comparing the total net benefits (the return) received from a project to the total costs (the investment) of the project. The equation for calculating ROI is: (total benefits – total costs) / total costs.

RFID (radio frequency identification) tag An input device used in source data automation.

ring network A network resembling a circle of computers communicating with each other. A ring network is often used when processing is performed at local sites rather than at a central location.

risk analysis The process of identifying, assessing, and prioritizing potential risks or uncertainties in a project or decision, with the aim of minimizing their impact on the project's objectives.

risk assessment The process of identifying, analyzing, and evaluating risks.

risk control Actions implemented to mitigate or eliminate identified risks.

risk identification Listing each risk and assessing the likelihood that it could affect a project.

risk management plan Includes a review of the project's scope, stakeholders, budget, schedule, and any other internal or external factors that might affect the project. The plan should define project roles and responsibilities, risk management methods and procedures, categories of risks, and contingency plans.

risk management Identifying, evaluating, tracking, and controlling risks to minimize their impact.

risk owner An individual or entity responsible for managing and mitigating a specific risk, including the development and execution of risk response strategies, in a project or business context.

risk response plan A proactive effort to anticipate risk and describe an action plan to deal with it. An effective risk response plan can reduce the impact by triggering timely and appropriate action.

risk The possibility of an adverse event occurring that could negatively affect the project.

roaming A process that allows wireless clients to move from one access point to another, automatically associating with the stronger access point and allowing for uninterrupted service.

router A device that connects network segments, determines the most efficient data path, and guides data flow.

sampling A process in which an analyst collects examples of actual documents, including records, reports, or various forms.

scalability A characteristic of a system implying that it can be expanded, modified, or downsized easily to meet the rapidly changing needs of a business enterprise.

scalable The ability of a system to expand to meet new business requirements and volumes.

scaling on demand The ability to match network resources to needs at any given time; a feature of cloud computing. For example, additional cloud servers might come online automatically during peak loads to support increased workloads.

scatter diagram A tool systems analysts use graphically to show the correlation between two variables. Also called an XY chart.

scenario In an agile project, a real-world example of how users will interact with the system.

schedule feasibility A project can be implemented in an acceptable time frame.

schema The complete definition of a database, including descriptions of all fields, records, and relationships.

scroll bar In UI design, a control that allows the user to move through the available choices for an input field.

Scrum A popular technique for agile project management. It is derived from a rugby term. In a scrum, team members play specific roles and interact in intense sessions.

second normal form (2NF) A record design in 1NF and all fields not part of the primary key depend on the entire primary key. If any field in a 1NF record depends on only one of the fields in a combination primary key, then the record is not in 2NF. A 1NF record with a primary key that is a single field is automatically in 2NF.

secondary key A field or combination of fields that can be used to access or retrieve records. Secondary key values are not unique.

secure coding Practices that aim to prevent the introduction of security vulnerabilities within software development.

security Hardware, software, and procedural controls that safeguard and protect a system and its data from internal or external threats.

security hole A vulnerability in a computer system that can be exploited to gain unauthorized access or perform unauthorized actions.

security information and event management (SIEM) A set of tools and services offering a holistic view of an organization's information security.

security policy A document outlining the rules, guidelines, and practices to manage and protect an organization's information assets.

security token A physical device used to access an electronically restricted resource.

selection A control structure in modular design. It is the completion of one of two or more process steps based on the results of a test or condition.

semantic web An evolution of the web in which documents shared on the Internet have semantics (meaning) and not just syntax (HTML markup). Sometimes called Web 3.0.

sentiment analysis The computational process of identifying and categorizing opinions expressed in digital text, primarily to determine whether the writer's attitude toward a particular topic, product, or service is positive, negative, or neutral.

sequence check A type of data validation check used when the data must be in some predetermined sequence. For example, if the user must enter work orders in numerical sequence, an out-of-sequence order number indicates an error. If the user must enter transactions chronologically, a transaction with an out-of-sequence date means an error.

sequence code Numbers or letters assigned in a specific order. Sequence codes contain no additional information other than an indication of the order of entry into a system.

sequence diagram A UML diagram that shows the timing of transactions between objects during system execution.

sequence The completion of steps in sequential order, one after another.

server A computer in a client/server design that supplies data, processing, and services to client workstations.

service desk A centralized resource staffed by IT professionals that provides users with the support they need to do their jobs. Also called a help desk.

service level agreement (SLA) A formal document that defines the level of service expected from a service provider, detailing the metrics by which service is measured and the remedies or penalties, if any, should agreed-upon service levels not be achieved.

service pack A maintenance release provided by commercial software suppliers.

service provider A firm that offers outsourcing solutions. Two popular outsourcing options involve application service providers and firms that provide Internet business services.

service-oriented A company that primarily offers information or services or sells goods produced by others.

services Functions or capabilities provided by one entity to another, often in the context of IT and support.

significant digit code A cipher that distinguishes items by using a series of subgroups of digits—for example, U.S. Postal Service zip codes.

simulation Using computer-based models to emulate a new system's operation, which allows managers and developers to test and evaluate its performance, identify potential issues, and make adjustments before full-scale deployment. Organizations typically include all procedures, such as those they execute at the end of a month, quarter, or year, in their simulations.

sink An external entity that receives data from an information system.

site visit A trip to a physical location to observe a system in use there.

slack time The amount of time by which an event can be late without delaying the project. The difference between the latest (LCT) and earliest (ECT) completion time.

SOC 2 A set of criteria for managing customer data based on five "trust service principles"—security, availability, processing integrity, confidentiality, and privacy.

social engineering The use of deception to manipulate individuals into divulging confidential or personal information that may be used for fraud.

software A program run by computers for a specific function or task.

software as a service (SaaS) A model of software delivery in which functionality is delivered on demand as a network-accessible service, rather than as a traditional software application that is downloaded and installed on the customer's computer.

software engineering A software development process that emphasizes solid design, effective structure, accurate documentation, and careful testing.

software license A legal agreement that gives users the right to use software under specific terms and conditions.

software package Software that is purchased or leased from another firm. A commercially produced software product or family of products.

software reengineering Analytical techniques that identify an information system's potential quality and performance improvements.

software vendor A company that develops software for sale.

source An external entity that supplies data to an information system.

source data automation A popular online input method that combines data entry and automated data capture using input devices such as magnetic data strips or swipe scanners.

source document A form used to request and collect input data, trigger or authorize an input action, and provide a record of the original transaction. During the

input design stage, you develop source documents that are easy to complete and inexpensive.

spiral model A development model with a series of iterations, or revisions, based on user feedback.

spontaneous generation An unexplained generation of data or information. With respect to data flow diagrams, processes cannot spontaneously generate data flows—they must have an input to have an output.

SQL (Structured Query Language) A query language that allows PC users to communicate with servers and mainframe computers.

stakeholder Anyone affected by a company's performance, such as customers, employees, suppliers, stockholders, and community members.

stand-alone A mode that allows users to run their own word processing, spreadsheet, and database applications without assistance from the IT group.

standard notation format A representation that makes designing tables easier, as it clearly shows a table's structure, fields, and primary key.

star network A network design with a central device and one or more workstations connected to it in a star pattern.

start day/date When a task is scheduled to begin.

state An adjective that describes an object's current status (e.g., a student could be a CURRENT, FUTURE, or PAST student).

state transition diagram A diagram that shows how an object changes from one state to another, depending on the events that affect the object.

status flag In structured application development, an indicator that allows one module to send a message to another module.

storyboard In an agile project, a simple graphic organizer that helps systems analysts visualize the status of a project.

strategic planning The process of identifying long-term organizational goals, strategies, and resources.

strategic plans The long-range plans that define the corporate mission and goals. Typically defined by top management, with input from all levels.

stratified sample A sample in which a set metric is collected across functional areas. For example, a certain percentage of transactions from every work shift, or five customers from each of four postal codes, could be a stratified sample.

structure chart A top-down representation of business functions and processes. Also called a functional decomposition diagram.

structured analysis A traditional systems development technique that uses phases to plan, analyze, design, implement, and support an information system. Processes and data are treated as separate components.

structured brainstorming A group discussion in which each participant speaks when it is their turn or passes.

structured English A subset of standard English that describes logical processes clearly and accurately.

structured walk-through A review of a project team member's work by other team members. Generally, systems analysts review the work of other systems analysts, and programmers review the work of other programmers as a form of peer review. Depending on the phase in which they occur, they should take place throughout the SDLC and are called design reviews, code reviews, or testing reviews.

stub testing A form of testing in which a programmer simulates each program outcome or result and displays a message to indicate whether the program executed successfully. Each stub represents an entry or exit point that will be linked later to another program or data file.

subclass A further division of objects in a class. Subclasses are more specific categories within a class.

subordinate module A lower-level module in a structure chart.

subschema A view of a database used by one or more systems or users. A subschema defines only those portions of the database that a particular system or user can access.

subscription model A service model that charges a variable fee for an application based on the number of users or workstations that have access to the application.

successor task Each of the concurrent tasks of a predecessor task.

summary report A report used by individuals at higher levels in an organization that includes less detail than reports used by lower-level employees.

super key A set of one or more columns that can uniquely identify a record in a table.

superclass A more generalized category to which objects may belong (e.g., a NOVEL class might belong to a superclass called BOOK).

superuser A user account with the highest level of access rights to manage a computer or network.

supply chain All the companies that provide materials, services, and functions needed to deliver products to a customer.

supply chain management (SCM) The coordination, integration, and management of materials, information, and finances as they move from suppliers to customers, both within and between companies. In an integrated supply chain, a customer order could cause a production planning system to schedule a work order, triggering a call for certain parts from one or more suppliers.

surrogate key An artificial key introduced in a table to uniquely identify a row. Typically used when a natural primary key is not present or suitable.

survey A document containing several standard questions that can be sent to many individuals. See questionnaire.

swim lanes In a business process diagram, the overall diagram is called a pool, and the designated customer areas are called swim lanes.

switch A central device in a star network that manages the network and acts as a conduit for all network traffic.

switchboard A set of command buttons in a UI that enables users to navigate a system and select from groups of related tasks.

SWOT analysis An examination of a company's strengths (S), weaknesses (W), opportunities (O), and threats (T).

syntax error A programming language grammar error.

SysML A dialect of UML used for representing requirements (and other things), primarily in Model-Based Systems Engineering (MBSE) applications.

system A set of related components that produces specific results.

system administrator A person responsible for the configuration management and maintenance of an organization's computer networks.

system architecture A translation of the logical design of an information system into a physical structure that includes hardware, software, network support, and processing methods.

system boundary A rectangle that shows what is included in and excluded from a system. The boundary is depicted by a shaded rectangle in use case diagrams.

system changeover The process of putting a new information system online and retiring the old system. Changeover can be rapid or slow, depending on the method.

system design specification A document that presents the complete design for a new information system, along with detailed costs, staffing, and scheduling for completing the next SDLC phase, systems implementation. Also called the technical design specification or the detailed design specification.

system documentation A description of a system's functions and how they are implemented. The analyst prepares most of this documentation during the systems analysis and systems design phases. System documentation includes data dictionary entries, data flow diagrams, object models, screen layouts, source documents, and the systems request that initiated the project.

system prototyping Producing a full-featured, working model of the information system being developed.

system requirement A characteristic or feature that must be included in an information system to satisfy business requirements and be acceptable to users.

system requirements document A document that contains the requirements for a new system, describes the alternatives that were considered, and makes a specific recommendation to management. It is the end product of the systems analysis phase.

system software Programs that control a computer, including the operating system, device drivers that communicate with hardware, and low-level utilities.

system testing A form of testing involving an entire information system that includes all typical processing situations. During a system test, users enter data, including samples of actual (or live) data, perform queries, and produce reports to simulate actual operating conditions. All processing options and outputs are verified by users and the IT project development team to ensure that the system functions correctly.

systematic sample A sample that occurs at a predetermined periodicity. For example, every tenth customer record might be selected as a systematic sample for review.

systems analysis and design The process of developing information systems that effectively use hardware, software, data, processes, and people to support a company's business objectives.

systems analysis phase The second SDLC phase. This phase aims to build a logical model of a new system.

systems analyst A person who plans, analyzes, and implements information systems. They may work internally within a company's IT department or be hired by a company as an independent consultant.

systems design phase The third SDLC phase. Systems design aims to create a blueprint for a new system that will satisfy all documented requirements, whether the system is being developed in-house or purchased as a package.

systems development life cycle (SDLC) Activities and functions that systems developers typically perform, regardless of how those activities and functions fit into a particular methodology. The SDLC model includes five phases: (1) systems planning, (2) systems analysis, (3) systems design, (4) systems implementation, and (5) systems operations.

systems implementation phase The fourth phase of the SDLC. During this phase, the new system is constructed; programs are written, tested, and documented; and the system is installed.

systems operations phase During this phase of the SDLC, the IT staff maintains, enhances, and secures the operational system.

systems planning phase The first phase of the SDLC. During this phase, the systems project gets started. The project proposal is evaluated to determine its feasibility, and the project management plan is formulated with the help of CASE tools where appropriate.

systems programmer A person who concentrates on operating system software and utilities.

systems request A formal appeal to the IT department that describes problems or desired changes in an information system or business process. For example, it might propose enhancements to an existing system, correcting problems, or developing an entirely new system.

systems request A formal request to the IT department that describes problems or desired changes in an information system or business process. It might propose enhancements for an existing system, correcting problems, or developing an entirely new system.

systems review committee A group of key managers and users responsible for evaluating systems requests. The term "computer resources committee" is sometimes also used.

table A file or document that contains data about people, places, things, or events that interact with the information system.

table design A design that specifies the fields and identifies the primary key in a particular table or file.

tamper-evident case A type of security device or packaging that shows visible signs of tampering.

tangible benefits Positive outcomes that can be measured in dollars. They can result from a decrease in expenses, an increase in revenues, or both.

tangible costs Expenses that have a specific dollar value. Examples include employee salaries and hardware purchases.

task Any work that has a beginning and an end and requires the use of company resources, including people, time, and money. Examples include conducting a series of interviews, designing a report, selecting software, waiting for the delivery of equipment, and training users. See also activity.

task box A component of a PERT/CPM chart containing important scheduling and duration information about a task. Each task in a project is represented by its task box in the PERT/CPM chart.

task group A task that represents several activities.

task ID A number or code that uniquely identifies a task.

task name A short descriptive name for a task, which does not have to be unique in the project. For example, a task named Conduct Interviews might appear in several project phases.

task pattern A logical sequence of tasks in a work breakdown structure. Can involve sequential tasks, multiple successor tasks, and multiple predecessor tasks.

technical feasibility When an organization has the resources to develop, purchase, install, or operate a system.

technical support Support necessary to help the wide variety of IT systems and users. It includes seven main functions: application development, systems support and security, user support, database administration, network administration, web support, and quality assurance. These functions overlap considerably and often have different names in different companies.

terminator A data flow diagram symbol indicating a data origin or final destination. Also called an external entity.

test data The data used in unit testing. It should contain both correct data and erroneous data and should test all possible situations that could occur.

test environment The environment that analysts and programmers use to develop and maintain programs.

test plan A plan designed by a systems analyst that includes test steps and test data for integration testing and system testing.

test-driven development (TDD) An Extreme Programming (XP) concept that unit tests are designed before code is written, focusing on initial results and preventing programmers from straying from their goals.

testing review See structured walk-through.

thick client A system design that locates most or all of the application processing logic at the client. Also called a fat client design.

thin client A system design that locates most or all of the processing logic at the server.

third normal form (3NF) A record design in 2NF in which no non-key field depends on another non-key field. A non-key field is a field that is not a candidate key for the primary key.

third-party risk management (TPRM) A strategic approach to identifying, assessing, and mitigating risks associated with outsourcing services and functions to external vendors or third-party providers.

third-party software Software produced by companies other than the hardware manufacturer or the operating system developer.

threat Any event that could harm an organization via unauthorized system actions.

three-tier design A network design in which the user interface runs on the client and the data is stored on the server, just as in a two-tier design. A three-tier

design also has a middle layer between the client and server that processes the client requests and translates them into data access commands that the server can understand and carry out.

throughput A measurement of actual system performance under specific circumstances; it is affected by network loads and hardware efficiency. Throughput, like bandwidth, is expressed as a data transfer rate, such as Kbps, Mbps, or Gbps.

throwaway prototyping See design prototyping.

tightly coupled A relationship between modules in which one module refers to internal logic contained in another.

toggle button A GUI element used to represent on or off status. Clicking the toggle button switches to the other status.

toolbar A GUI element that contains icons or buttons representing shortcuts for executing common commands.

top-down approach A design approach, also called modular design, in which the systems analyst defines the overall objectives of the system and then breaks them down into subsystems and modules. This process also is called partitioning.

total cost of ownership (TCO) A number used in assessing costs, including ongoing support, maintenance, and acquisition costs.

totals zone A section of a form that contains data totals.

traceability The ability to follow a requirement backward to its origins and forward through the SDLC to link design documents, code fragments, and test artifacts.

training plan A document that details requirements for training users, managers, and IT staff members in the use of a system. The success of the entire systems development effort can depend on whether people understand the system and know how to use it effectively.

train-the-trainer A strategy in which one group of users has been trained and can assist others. Users often learn more quickly from co-workers who share common experience and job responsibilities.

transaction model A service model that sets a variable fee for an application based on the volume of transactions or operations performed by the application. Also called a usage model.

transaction processing (TP) systems Operational systems used to process day-to-day recurring business transactions, such as customer billing.

transference The transfer of risk to a third party, typically through insurance or outsourcing.

transparent A network characteristic that makes data appear to be stored on users' workstations.

transparent interface A UI that users overlook; a user-friendly interface that does not distract the user and calls no attention to itself.

tunnel A secure, encrypted path through an untrusted network.

tuple A record or a set of related fields that describes one instance or member of an entity, such as one customer, one order, or one product. A tuple might have one or dozens of fields depending on the needed information.

turnaround document An output document that is later entered back into the same or another information system. For example, a phone or utility bill might be a turnaround document printed by the company's billing system. When the bill is returned with payment, it is scanned into the company's accounts receivable system to record the payment accurately.

turnaround time A measure applied to centralized batch processing operations, such as customer billing or credit card statement processing. Turnaround time measures the time between submitting a request for information and fulfilling the request. Turnaround time also can be used to measure the quality of IT support or services by measuring the time from a user's request for help to the resolution of the problem.

tutorial A series of online interactive lessons that present material and provide a dialogue with users.

two-tier design A network design in which the user interface resides on the client, all data resides on the server, and the application logic can run on the server or the client or be divided between them.

unencrypted The state of data or information that is not encrypted but is in a readable format.

Unicode A relatively recent coding method that represents characters as integers. Unlike EBCDIC and ASCII, which use 8 bits for each character, Unicode requires 16 bits per character, which allows it to represent more than 65,000 unique characters.

Unified Modeling Language (UML) A widely used method of visualizing and documenting software systems design. UML uses object-oriented design concepts, but it is independent of any specific programming language and can be used to describe business processes and requirements generally.

uninterruptible power supply (UPS) A device that provides battery backup when the electrical power fails or drops to an unacceptable voltage level.

unique key A constraint in a database table that ensures values in a specific column or a combination of columns are distinct across all records.

unit testing The testing of an individual program or module. The objective is to identify and eliminate execution errors that could cause the program to terminate abnormally and logic errors that could have been missed during desk checking.

universal security slot (USS) A standard device slot for attaching a security lock.

unnormalized A table design that contains a repeating group—a single record has multiple occurrences of a particular field, with each occurrence having different values.

unstructured brainstorming A group discussion in which any participant can speak at any time.

usability In UI design, a concept that includes user satisfaction, support for business functions, and system effectiveness.

usability metrics Data that interface designers can obtain by using software that can record and measure user interactions with the system.

usage model See transaction model.

use case A representation of the steps in a specific business function or process in UML.

use case description A description in UML that documents the name of the use case, the actor, a description of the use case, a step-by-step list of the tasks required for successful completion, and other key descriptions and assumptions.

use case diagram A visual representation of the interaction between users and the information system in UML.

user application A program that utilizes standard business software, such as Microsoft Office, and has been configured in a specific manner to enhance user productivity.

user design phase In this phase, users interact with systems analysts and develop models and prototypes that represent all system processes, outputs, and inputs.

user documentation Instructions and information provided to users who will interact with the system. It includes user manuals, help screens, and tutorials.

user interface (UI) The mechanism through which the user interacts with the system. The interface can be graphical, textual, aural, or a combination of modes.

user productivity systems Applications that provide employees of all levels a wide array of tools to improve job performance. Examples include email, word processing, graphics, and company intranets.

user rights The permissions granted to users, enabling them to access specific resources and perform certain tasks.

user stories In an agile project, a set of more refined requirements derived from features.

user story In agile development, a short, simple requirements definition provided by the customer. Programmers use user stories to determine a project's requirements, priorities, and scope.

user training package Materials that show users how the system can help them perform their jobs.

user-centered A term that indicates the primary focus is on the user. In a user-centered system, the distinction blurs between input, output, and the interface itself.

users Stakeholders inside and outside the company who will interact with the system.

validation and verification (V&V) The process of ensuring that system requirements are complete, accurate, consistent, and aligned with stakeholder needs and expectations.

validity check A type of data validation check used for data items that must have certain values. For example, if an inventory system has 20 valid item classes, any input item that does not match one of the valid classes will fail the check.

validity rules Checks that are applied to data elements when data is entered to ensure that the value entered is valid. For example, a validity rule might require an employee's salary number to be within the employer's predefined range for that position.

value-added reseller (VAR) A firm that enhances a commercial package by adding custom features and configuring it for a particular industry.

vendor risk assessments (VRAs) Evaluations conducted by organizations to analyze and mitigate potential risks posed by external suppliers and service providers, focusing on aspects such as security, compliance, and operational reliability.

version control The process of tracking system releases.

vertical application A software package developed to handle information requirements for a specific type of business.

vertical system A system designed to meet the unique requirements of a specific business or industry, such as a web-based retailer or auto-supply store.

virtual private network (VPN) A secure connection over the Internet from a device to a network, ensuring the confidentiality and integrity of data.

vulnerability A weakness in a system that a threat actor can exploit to perform unauthorized actions within a computer system.

waterfall model The traditional model of software development. A graph depicting the result of each SDLC phase flowing down into the next phase.

Web 2.0 A second generation of the web that enables people to collaborate, interact, and share information much more dynamically, based on continuously available user applications rather than static HTML

webpages. Interactive experience is a hallmark of Web 2.0.

Web 3.0 An evolution of the Internet into a more intelligent, interconnected, and decentralized network, where data is contextualized for intuitive user experiences and enhanced by technologies like artificial intelligence and blockchain.

webcast A one-way transmission of information or training materials, such as a webinar session, available on demand or for a specific period to online participants.

web-centric A strategy or approach that emphasizes a high degree of integration with other web-based components. A web-centric architecture follows Internet design protocols and enables a company to integrate a new application into its e-commerce strategy.

webinar An Internet-based training session that provides an interactive experience. The word combines "web" and "seminar."

weight An important multiplier that managers factor into estimates so they can be analyzed.

what-if analysis A feature of business support systems that allows analysis to define and account for a wide variety of issues (including issues not wholly defined).

wide area network (WAN) A network spanning long distances that can link users who are continents apart.

Wi-Fi (wireless fidelity) A family of popular IEEE local area network wireless networking standards, also known as 802.11.

Wi-Fi Alliance A nonprofit international association formed in 1999 to certify the interoperability of wireless network products based on IEEE 802.11 specifications.

Wi-Fi Protected Access (WPA) A security protocol developed to secure wireless computer networks.

wiki A web-based repository of information that anyone can access, contribute to, or modify.

Wired Equivalent Privacy (WEP) An outdated security protocol for wireless networks that provides data confidentiality comparable to that of a traditional wired network.

wireless access point (WAP) A central wireless device that provides network services to wireless clients. Also called an access point.

wireless local area network (WLAN) A wireless network that is relatively inexpensive to install and is well-suited to workgroups and users who are not anchored to a specific desk or location.

work breakdown structure (WBS) A project broken down into a series of smaller tasks. See also Gantt chart, PERT/CPM.

work from home (WFH) The practice of employees performing their job duties remotely from their homes, rather than in a traditional office setting, often facilitated by digital communication tools and technology.

worst-case estimate The most pessimistic outcome.

WPA2 An improved version of the Wi-Fi Protected Access security protocol that provides more robust data protection and network access control.

WPA3 The latest version of the Wi-Fi Protected Access security protocol that provides cutting-edge security features for wireless networks.

XY chart A tool systems analysts use to show the correlation between two variables graphically. Also called a scatter diagram.

Y2K issue A problem faced by many firms before the year 2000 because their computer systems used only two digits to represent the year; most dates now use a four-digit format for the year (YYYYMMDD).

Yourdon symbols A type of symbol set that is used in data flow diagrams. Processes, data flows, data stores, and external entities each have a unique symbol in the Yourdon symbol set.

zero-day exploits Attacks that take advantage of a security vulnerability on the same day that the vulnerability becomes generally known.

Index

1:1, 251
1:M, 251, 252
1NF. See first normal form (1NF)
2NF. See second normal form (2NF)
3NF. See third normal form (3NF)
802.11ac, 307
802.11ax, 307
802.11b, 307
802.11be, 307
802.11g, 307
802.11n, 307

A

abbreviation codes, 268
absolute date, 276
acceptance, 474
acceptance tests, 387
access control, 230–231
access point, 307
action codes, 269
active listening, 517
activity, 67
activity diagram, 193–194
Activity Monitor application, 430–431
actor, 138
adaptive maintenance, 414–417
administrative privileges, 486
Adobe PDF, 524
advanced persistent threats (APTs), 467, 471
agile development, 362
  extreme programming, 375
  iterations and releases, 376
  manifesto for, 374
  user stories, 375–376
agile methods, 18, 19, 22–24, 31, 118–120
agile mindset, 102
agile projects, requirements elicitation in, 133–135
AI. See artificial intelligence (AI)
Airbnb, 8
Alexa, 15
alias, 165
allocated baseline, 429
ALM. See application lifecycle management (ALM)
alphabetic codes, 268
alternate key, 247
Amazon.com, 543
Amazon Echo Dot, 15
Amazon Web Services (AWS), 543
ambiguous characters, in code design, 271
American Standard Code for Information Interchange (ASCII), 274
anonymization, 503
app, 8
Apple, 15, 199, 200, 328
  Face ID, 41
  Find My app, 478
Apple Keynote, 527–528
application and software security, 484–485
  application permissions, 486
  hardening, 486
  input validation, 486
  patches and updates, 486–487
  secure coding practices, 485
  services, 485
  software logs, 487
application development
  agile development methodology, 361, 362
  defined, 360
  design translation strategies, 360
  information technology, 26
  systems development tools, 362–365
  traditional methods, 361–362
application lifecycle management (ALM), 24
applications, 283
  logic, 290
  server, 290
  software, 6
  testing, 348
application service provider (ASP), 297, 331, 348
applications programmer, 420
APTs. See advanced persistent threats (APTs)
archived, 428
Articulate Storyline 360, 445
artificial intelligence (AI), 15, 316–317, 338, 469, 506
The Art of Deception: Controlling the Human Element of Security (Mitnick), 471
ASCII. See American Standard Code for Information Interchange (ASCII)
ASP. See application service provider (ASP)
asset, 424
associative entity, 252
Atlantic Systems Guild, 136
Atlassian's Confluence, 91, 92
attack, 468
  APTs, 471
  attacker profiles and motivations, 467–468
  insider threats, 471–472
  preventive measures against, 471
  social engineering, 471–472
  types of, 468–471
attributes, 182, 246
  ERD, 250
audit fields, 277
audit log files, 277
audit procedures, 443
audit trails, 218
augmented reality (AR), 201
authentication mechanisms, 230–231
authorization zone, 220
automated fax, 227
automatic update service, 487
availability, 464
avoidance, 474
AWS Database Migration Service, 454
Axure RP, 233

B

B2B (business-to-business), 8–9
B2C (business-to-consumer), 8
back door, 469
backup, 277, 497
  restarting operations from, 500
backup media, 497–499
backup policies, 497–498
balancing, 163
bandwidth, 294, 432
baselines, 429
basic service set (BSS), 307, 308
batch, 225
batch controls, 218
batch input, 225
batch processing, 299–300
benchmark, 348–349
benchmark testing, 431
benefits
  classification, 551–552
  direct vs. indirect, 553
  financial vs. nonfinancial, 553–554
  measurable vs. nonmeasurable, 555
  operational vs. strategic, 554
  positive vs. cost avoidance, 555–557
  short term vs. long term, 554–555
  tangible vs. intangible, 552–553
best-case estimate, 69
big data, 7
binary storage format, 275
biometric devices, 41
BIOS-level password, 477
bits (binary digits), 274
black box, 149–150, 183
black hole, 150
BlazeMeter, 437, 438
blockchain technology, 44
  for enhancing security, 505–506
block sequence codes, 268
blogs, 227
Bluetooth, 309
body zone, 220
Boehm, Barry, 23
Boeing, 12
boot-level password, 477
bottom-up technique, 66
BPM. See business process model (BPM)
BPMN. See business process modeling notation (BPMN)
BPO. See business process outsourcing (BPO)
brainstorming, 130
bring your own device (BYOD), 32
Brooks, Frederick, 99
Brooks' Law, 99
BSS. See basic service set (BSS)
budget issues, in systems project, 101
budgets, 82
bug tracking software, 388
Bugzilla, 91, 92
build/buy decision, 320
business case, 40
business continuity planning (BCP), 496–497
business information systems, 11–12
  business support systems, 12–13
  digital assistants, 15
  enterprise computing, 12
  knowledge management, 13–14
  systems integration, 15–16
  transaction processing systems, 12
  user productivity systems, 14–15
business issues, 99–100
business logic, 149
business model, 9
business process, 10
business process diagrams, 137
business process model (BPM), 10, 11, 137, 194

business process modeling notation (BPMN), 10, 11, 137
business process outsourcing (BPO), 329, 330
business profile, 9
business rules, 18
business skills, 29
business support systems, 12–13
bus network, 303
BYOD. See bring your own device (BYOD)
byte, 275

C

C2C (consumer-to-consumer), 9
calendar control, 213
calendar view, 97
California Consumer Privacy Act (CCPA), 501, 502
Cameo Systems Modeler, 25, 194
candidate key, 247, 248
Capability Maturity Model (CMM), 356, 357
Capability Maturity Model Integration (CMMI), 356
capacity planning, 432–435
cardinality, 190, 253–254
cardinality notation, 253, 254
career opportunities, systems analysts, 30–31
Carnegie Mellon University's Software Engineering Institute (CMU/SEI), 342
case for action, 59
CASE tools. See computer-aided systems engineering (CASE) tools
category codes, 268
CCPA. See California Consumer Privacy Act (CCPA)
centralized model, 421
certification, 29–30
change control (CC), 426
character-based reports, 221
characters, 274
chargeback method, 550–551
ChatGPT, 15, 16
check boxes, 213
child, 186
child diagram, 157
churn analysis, 273
CIA triangle, 464
cipher codes, 269
CISA. See Cybersecurity and Infrastructure Security Agency (CISA)
Cisco Support Community, 390, 391
Cisco Systems, 30, 31
class, 22, 184–186, 191
class diagram, 190–191
clicks to close, 274
clickstream storage, 274
clients, 287
client/server architecture
  client's role, 290
  cost-benefit issues, 292
  defined, 289
  mainframe systems vs., 289, 290
  middleware, 291–292
  performance issues, 292–293
  tiers, 290–291
closed-ended questions, 123
cloud access security brokers (CASBs), 504
cloud computing, 31, 293–294, 317, 338
  in information systems, 5, 6
  on system security, 504–505
CM. See configuration management (CM)
CMM. See Capability Maturity Model (CMM)
CMMI. See Capability Maturity Model Integration (CMMI)
Codecademy, 381–383
codes
  defined, 267
  designing, 269–271
  overview of, 267
  reviews, 98, 383
  types of, 268–269
coding, 355, 377
  integrated development environments, 378–382
  online learning resources, 381–385
  programming languages, 377–378
cohesion, 368
  object-oriented, 373–374
collaboration, 89, 91
  with maintenance team, 421–422
combination check, 218
combination key, 247
command button, 212, 213
command-line interfaces (CLIs), 201
common field, 246
communication, 89, 98, 516
  active listening, 517
  breakdown, 100
  DEI, 518
  management, 78–80
  online, 531–537
  oral, 28, 526–530
  plan, 517
  questions in, 518–519
  regular status updates, 517–518
  resources, 539–541
  soft skills, 537–538
  technical language, simplifying, 517
  written, 28, 519–526
community forums, 412–413
company finances, 43
company organization, 30
company size, 30
competitors, 45
compliance, 285
  and standards, security update, 441
  with data protection laws, 501
composite key, 247
comprehensive testing approach, 348
computer-aided software engineering, 24
computer-aided systems engineering (CASE) tools, 24, 39, 194, 205, 361–362, 430
computer resources committee, 45
computing devices security, 477–478
concatenated key, 247
concurrency, 242
concurrent task, 72
condition, 367
  structure charts, 367
confidentiality, 464
configuration management (CM), 426–427
  baselines, 429
  maintenance releases, 427–428
  version control, 428–429
consistent design, 223
constraints, 54, 55, 70
construction phase, 117
content management strategies, 449, 450
context diagram
  defined, 155
  for grading system, 156
  for order system, 156–157
context-sensitive, 210, 389
continuous backups, 497, 498
continuous improvement
  of data recovery processes, 445
  feedback loops for, 451–452
  quality assurance, 436
continuous learning
  in cybersecurity, 509
  in training program, 410
continuous risk monitoring, 474–475
control
  break, 221
  break report, 221
  couple, 366, 367
  field order, 221
  module, 365–366
  structures, 168
coordination, with maintenance team, 421–422
Co-Pilot, 337
corporate culture, 30–31, 282
corporate organization, 282
corporate portals, 285–286
corrective maintenance, 414–417
cost
  analysis, 58
  classification, 544
  cross-category, 548–549
  developmental, 547–548
  direct, 545–546
  fixed, 546–547
  indirect, 546
  intangible, 545
  managing information systems, 549–550
  operational, 548
  tangible, 544–545
  variable, 547
cost-avoidance benefits, 556–557
cost-benefit analysis, 339–340, 542–543
  checklist, 340–341
  performing, 349–350
cost estimation, 82
cost overruns, 101
couples, structure charts, 371
coupling, 368, 369
  object-oriented, 373–374
Coursera, 382, 385
COVID-19 pandemic, 9, 124, 330, 531
CPM. See Critical Path Method (CPM)
credential stuffing, 469
credit card validation
  sequence diagram, 139
  use case, 138, 139
criticality, 474
critical path, 74–76
Critical Path Method (CPM), 64, 66–67. See also PERT/CPM charts
critical success factor, 37
critical thinking skills, 29
CRM. See customer relationship management (CRM)
cross-category costs, 548–549
cross-site request forgery (CSRF), 469
cross-site scripting (XSS), 469
crow's foot notation, 253
cultural and language barriers, 80
custom-developed software, 320–321
customer lifetime value (CLV) prediction, 273
customer relationship management (CRM), 44–45
customers, 44–45, 374
  segmentation, 273
cutover phase, 117
cyberattacks, types of, 468–471
cyberethics, 501
cybersecurity
  artificial intelligence in, 506
  culture, sustaining and evolving, 511–512
  leadership and governance in, 509–510
  organizational culture in, 507–508
  resilient culture, 506–512
  security-first mindset, 508–509
Cybersecurity and Infrastructure Security Agency (CISA), 482, 483
cyclical information systems, 393–394
cyclic redundancy check (CRC), 271

D

data, 4, 6–7
  analysis, 58
  security, 487–488

data and process modeling
  data dictionary, 164–167
  DFD (See data flow diagram (DFD))
  logical vs. physical models, 148
  process description, 168–174
database administration, 27
database administrator (DBA), 242, 243
database management system (DBMS)
  advantages, 241–242
  components, 243–244
  defined, 241
database programmer, 420
data breach, 8
data center, 5
data coding
  ASCII, 274
  binary storage format, 275
  EBCDIC, 274–275
  store dates, 275–276
  Unicode, 275
data control, 276–277
data conversion, 395–396
data couple, 366
data design
  codes, 267–272
  data control, 276–277
  data normalization (See normalization)
  data storage and access, 272–276
  data structure, 239
  DBMS, 241–244
  entity-relationship diagrams, 250–255
  example, 239–241
  terminology, 246–250
  web-based design, 244–246
data dictionary, 164–165
  data elements documentation, 165–166
  data flows documentation, 166
  data stores documentation, 166
  entities documentation, 167
  process documentation, 166–167
  record documentation, 167
  reports, 167
data elements, 164, 274
  documenting, 165–166
data encryption, 230
data entry screen, user interfaces, 214–216
data field, 164
data flow, 150–151
  documenting, 166
data flow diagram (DFD), 18, 138, 148–149, 365
  context diagram, 155–157
  data flow symbols, 150–151
  data store symbols, 151–153
  diagram 0, 157–160
  drawing, 154–155
  entity symbols, 153, 154
  Gane and Sarson symbols, 149
  lower-level, 160–163
  objects, 180
  process symbols, 149–151
  rules for using symbols, 154
  structure charts, 369
  Yourdon symbols, 149
data frames, 304
data integrity, 231, 455
data item, 164, 274
data loss prevention (DLP), 479, 487
data management
  and analytics, 337–338
  user interface design, 218
data manipulation language (DML), 243
data mart, 272
data migration, 453–455
data mining, 273–274
data normalization. See normalization
data privacy, 101
data processing center, 287
data protection, 444
  backup strategies, 444
  data recovery processes, 445
  privacy considerations, 445–446
data recovery processes, 445
data repository, 165
data science, 31
data security, 101, 229
data storage and access
  data coding, 274–277
  data mining, 273–274
  data warehousing, 272–273
  logical vs. physical storage, 274
data store, 151
  documenting, 166
  symbols, 151–153
data structures, 164, 239
data type check, 217
data validation, 231
data validation rule, 216–217
data warehouse, 272–273
DBA. See database administrator (DBA)
DBMS. See database management system (DBMS)
DDBMS. See distributed database management system (DDBMS)
DDoS attack. See distributed denial of service (DDoS) attack
decentralized model, 421
decision making, 84
decision table, 170–174, 364–365
decision trees, 174, 364–365
decommissioning procedures, 455–457
decomposing, 160
deepfake attack, 469
default value, 209
defect tracking software, 388
deliverable, 19
Delphi technique, 85
denial-of-service (DoS) attacks, 468, 469, 482
dependent tasks, 72, 74
derivation codes, 268–269
design patterns, 179
design prototyping, 232
design reviews, 98
design translation strategies, 360
design walkthrough, 383
desk checking, 383
detail reports, 223
developmental costs, 547–548
development strategy
  AI and ML integration, 316–317
  cloud computing, 317
  data management and analytics, 337–338
  edge computing, 318
  in-house development options, 319–329
  Internet of Things, 317–318
  mobile devices, 318
  offshoring, 333–334
  outsourcing, 329–333
  selection, 338–341
  software acquisition, 342–350
  software as a service, 334–337
  traditional vs. web-based systems, 315–316
  Web 3.0, 319
DevSecOps, 485
DFD. See data flow diagram (DFD)
diagram 0
  defined, 157
  for grading system, 157–159
  for order system, 159–160
diagrams, 136
  business process, 137
  data flow diagrams, 138
  functional decomposition diagram, 137
dialog box, 213
differential backups, 497, 498
digital archiving methods, 459
digital assistants, 15
digital audio, images, and video, 227
digital media, computer output to, 227
digital pen transcription, 136
digital process management (DPM), 335
digital rights management (DRM) systems, 503
direct benefits, 553
direct costs, 545–546
direct cutover system changeover, 392–394
disaster recovery planning (DRP), 499–500
discretionary projects, 51
disk-based backups, 498
diskless workstation, 230
distributed database management system (DDBMS), 293
distributed denial of service (DDoS) attack, 469, 482
distributed systems, 288
diverging data flow, 157
diversity, equity, and inclusion (DEI), 518
Django, 380
DLP. See data loss prevention (DLP)
documentation, 89–90, 446
  archiving and, 458–460
  defined, 388
  feedback loops into, 451–452
  integrating with help desks, 451
  online, 390–391
  operations, 389
  program, 388
  self-service support through, 451
  standards, 446
  system, 388–389
  technical, 448
  user, 389–390
  user interface, 205–206
  user manuals and guides, 446–447
document management, 91
document review, 126–127
domain, 165
DoS attacks. See denial-of-service (DoS) attacks
DPM. See digital process management (DPM)
drive-by download attacks, 469
DRP. See disaster recovery planning (DRP)
Drupal, 449, 450
dumpster diving, 469, 493
dynamic priorities, 51

E

EBCDIC (Extended Binary Coded Decimal Interchange Code), 274–275
Eclipse, 381, 382
e-commerce (electronic commerce), 8
  application service provider, 297–298
  guidelines, 295
  in-house solutions, 295–296
  packaged solution, 296–297
economic activity, 45
economic feasibility, 49, 58, 339
economy of scale, 242
edge computing, 318
EDI. See electronic data interchange (EDI)
education, systems analysts, 29
edX, 382, 385
electronic data interchange (EDI), 9, 44
electronic health record (EHR) systems, 202
electronic product code (EPC), 43
electronic proof of delivery (EPOD), 45
email, 226–227, 522
emotional intelligence (EI), 538
empowerment, 18
encapsulation, 183
encryption, 41, 487–488
  network traffic, 480–481

engaged listening, 125
EngageOne Vault, 459
enhancements, 409
enterprise applications, 6
enterprise computing, 12
enterprise resource planning (ERP), 12, 77, 282–283
entity
  data design, 246
  documenting, 167
  ERD, 250
  symbols, 153, 154
entity-relationship diagram (ERD), 250–251, 363
  cardinality, 253–254
  drawing, 251
  library system, 254, 255
  many-to-many relationship, 252–253
  one-to-many relationship, 251, 252
  one-to-one relationship, 251
EPC. See electronic product code (EPC)
epic, 133
EPOD. See electronic proof of delivery (EPOD)
ERD. See entity-relationship diagram (ERD)
ERP. See enterprise resource planning (ERP)
error detection, in code design, 271
ESS. See extended service set (ESS)
ethical hacking, 502, 503
evaluation and selection team, 339
evaluation model, 344–346
Eventbrite, 214
events, 67
exception report, 223
existence check, 217
exploding, 160
exploit system vulnerabilities, 464
extended service set (ESS), 307, 308
extensibility, 283
extreme programming (XP), 23, 362, 375

F

fact-finding, 55–57, 120
  data analysis, 56–57
  documentation, 56
  interviewing process, 56
  operations, observing, 56
  organizational charts, 55
  user survey, conducting, 56
fat client, 290
fault management, 430–431
fault tolerant, 497
faxback system, 227
FDD. See functional decomposition diagram (FDD)
feasibility
  economic, 49, 58
  operational, 48–49, 58
  schedule, 50, 58
  study, 20, 48
  systems requests, 47–48
  technical, 50, 58
features, 133
feedback analysis, 439–440
feedback loops, for continuous improvement, 451–452
field, 246
file, 246
  processing systems, 240
  security, 487–488
file-oriented systems, 240
File Transfer Protocol (FTP), 485
fill-in form, 129
finance
  benefits classification, 551–557
  cost-benefit analysis, 542–543
  costs. See cost
  payback analysis, 557–561
  present value analysis, 562–567
  return on investment analysis, 561–562
financial analysis tools, 341
financial benefits, 553–554
financial forecasting, 101
financial management, 82
  budgets and cost estimation, 82
  return on investment analysis, 83
  skills and technology investment, 82–83
finish day/date, 72
firewall, 480, 481
first normal form (1NF), 257–258, 263, 264
fishbone diagram, 53
fixed costs, 546–547
fixed fee model, 332
Flesch-Kincaid Grade Level score, 520–521
Flesch Reading Ease score, 520, 521
Florida Writers Association (FWA), 539, 540
flowchart, 363
focus, 192
foreign key, 248–249
form filling, 214
form layout, 220
forums, 346
four-model approach, 148
FreeCodeCamp, 381–382, 384
full backups, 497, 498
full-scale test, 400
functional baseline, 429
functional decomposition diagram (FDD), 137, 202
functionally dependent, 258
functional primitive, 159
functional requirement, 109
functions and organizational levels, 16, 17

G

gamification, 509
Gane and Sarson symbols, 149, 152
Gantt charts, 64–66, 93, 95, 97
Gantter tool, 93
Gantt, Henry L., 65
GanttProject, 93
garbage in, garbage out (GIGO), 219
Gartner, 505
gateway, 306
Gbps (gigabits per second), 307, 432
GDPR. See General Data Protection Regulation (GDPR)
General Data Protection Regulation (GDPR), 333, 495
Generative Pre-trained Transformer (GPT), 15
gesture-based interfaces, 201
Git, 429
GitHub, 90
GitHub Copilot, 379
global market expansion, 330
global outsourcing, 333
Goal Seek, 433, 434
Google
  Assistant., 15
  mission statement, 37
Google Workspace, 91
GoToMyPC, 411
government regulations, 45
GPT. See Generative Pre-trained Transformer (GPT)
grading system
  context diagram for, 156
  diagram 0 DFD for, 157–159
graphical user interface (GUI), 199, 200, 203
gray hole, 151
group interviews, 122
groupware, 14
GUI. See graphical user interface (GUI)

H

handling maintenance requests
  flowchart, 422, 423
  prioritization criteria for, 422–423
  request intake process, 422
  tracking and reporting, 424
hardening, 486
hardware, 5, 6
  disposal, environmental considerations for, 456–457
hash totals, 218
Hawthorne Effect, 127, 128, 202
HCI. See human-computer interaction (HCI)
Health Insurance Portability and Accountability Act (HIPAA), 466
help desks, 27, 410–412
  integrating documentation with, 451
help system, user interface design, 209–210
hierarchical network, 302–303
high-fidelity prototypes, 232–233
Hilton Hotels reservation system, 12
histogram, 142
Hollerith, Herman, 3
horizontal application, 320
horizontal system, 6
HTTP/2 protocol, 293
hub, 304
human-computer interaction (HCI), 200–202, 206, 207
human element, people management, 76

I

IBM (International Business Machines), 3, 201, 381
  blockchain, 44
  client/server system, 292
  Engineering Requirements Management DOORS Family, 142, 143
  Rational product family, 194
  Token Ring network, 304
IBM Security QRadar SOAR, 475
IBS. See Internet business services (IBS)
IDE. See integrated development environments (IDE)
identity management, 489
identity theft, 490, 491
Idera's Uptime Infrastructure Monitor, 433, 435
IDEs. See integrated development environments (IDEs)
IEEE 802.11, 306, 307
IEEE 802.11i standard, 484
IEEE Standard 828-2012, 427
IEEE Systems Council, 358
incentivization, 509
incident response planning, 475
INCOSE. See International Council on Systems Engineering (INCOSE)
incremental backups, 497, 498
indirect benefits, 553
indirect costs, 546
ineffective change communication, 79–80
inference rules, 13
informal structures, 121
information
  defined, 4
  organizational models, 16–18
information system, 3–5
  components, 5
  data, 6–7
  data breach, 8
  hardware, 5, 6
  people, 7
  processes, 7
  software, 5–6

information technology (IT), 3–4
  changing nature of, 3
  department, 25–27
  maintenance activities, 412
  outsourcing in, 329, 330
  support, 417
  trends in, 31–32
InfoWorld, 131
infrastructure mode, 307
inheritance, 186
in-house software, 320, 321
in-house web development, 295–296
input control, 229–231
input mask, 215
input technology, 225–226, 228
input validation, 486
insider threats, 471–472
installation, 391
  data conversion, 395–396
  deployment, 400–403
  operational and test environments, 391–392
  system changeover, 392–395
  training, 396–400
instance, 184
instant messaging, 227
Institute of Electrical and Electronics Engineers (IEEE), 306
intangible benefits, 49, 552–553
intangible costs, 49, 545
integrated development environments (IDEs), 24, 377, 378
  Eclipse, 381, 382
  GitHub Copilot, 379
  PyCharm, 380, 381
  Visual Studio Code, 378–379
  Xcode, 380
integration testing, 386
integrity, 464
Intellectual property rights (IPR), 503–504
interactive training, 400
Interface Builder, 380
International Council on Systems Engineering (INCOSE), 28, 358
International Organization for Standardization (ISO), 275, 358–360
Internet-based architecture, 293
Internet-based information delivery, 226
Internet business services (IBS), 331
Internet business strategies, 8–9
Internet of Things (IoT), 44, 317–318, 338, 505
  physical security challenges, 476
  security, 479
Internet operating system, 295
interviews
  conducting, 125
  defined, 121
  determining people to, 121–122
  developing questions, 122–123
  documenting, 125–126
  establishing objectives, 122
  evaluating, 126
  preparing for, 123–125
  process, 121
  vs. questionnaire, 130
  variation of, 133–134
intrusion detection systems (IDSs), 480
InVision, 233
IoT. See Internet of Things (IoT)
ISBN numbering system, 358
ISO. See International Organization for Standardization (ISO)
ISO 9001:2015, 358
ISO/IEC 27000, 465
ISO/IEC/IEEE 90003:2018, 358
IT department
  role of, 38–39
  systems project, 43
iteration, 168
  cycles, 376
  planning meeting, 376
iterative development, 23

J

JAD. See joint application development (JAD)
James Webb Space Telescope, 4, 5
Java database connectivity (JDBC), 244
Jira, 24, 89, 90
job titles, 30
joint application development (JAD)
  benefits, 114–115
  defined, 112
  objective of, 112
  participants and roles, 113–114
  planning and preparation, 112
  sessions, conducting, 112–114
  user involvement, 113
just-in-time (JIT) inventory systems, 44

K

Kbps (kilobits per second), 432
key fields, 246–249
key process indicators (KPIs), 425
keystroke loggers, 477
knee of the curve, 292
knowledge base, 13
knowledge management, 13–14, 89–90, 446
  access control and security, 449
  content management strategies, 449, 450
  knowledge base, construction of, 448–449
knowledge process outsourcing (KPO), 330
knowledge workers, 17
KPO. See knowledge process outsourcing (KPO)

L

LAN. See local area network (LAN)
leading questions, 122
legacy data, 289
legacy systems, 6, 284
legal process outsourcing (LPO), 330
letters, 522
leveling, 160–162
library module, 366
licensing options, 341
lifeline, 192
limit check, 218
LinkedIn Learning, 502, 503
list boxes, 213
local area network (LAN), 288, 301, 304
logical model, 148
logical record, 274
logical storage, 274
logical structures, 168
logical topology, 301
logic errors, 383
long term benefits, 554–555
loop, 367
  structure charts, 371
looping, 168
loosely coupled modules, 368, 369
lower-level DFDs
  balancing, 163
  leveling, 160–162
low-fidelity prototypes, 232
LPO. See legal process outsourcing (LPO)
Lucidchart, 10, 11

M

machine learning (ML), 15, 82, 316–317, 338
mail bombing, 469
mainframe architecture, 287
mainframe systems, client/server architecture vs., 289, 290
maintenance
  activities, 412
  agreement, 349
  expenses, 414, 416
  process improvement, 425–426
  release methodology, 427
  team, 419–421
  testing, 436
maintenance and support
  configuration management, 426–429
  documentation, 446–452
  knowledge management, 446–452
  maintenance tasks, 414–418
  operations management, 419–426
  quality assurance, in system support, 433–440
  security management, 440–446
  system performance management, 430–433
  system retirement and transition, 452–460
  user support and training, 409–414
maintenance operations management
  defined, 419
  handling maintenance requests, 422–424
  maintenance process improvement, 425–426
  maintenance team, 419–421
  prioritization, 424–425
  roles and responsibilities, 421–422
make/buy decision, 320
malicious code, 469
malware, 468
managed hosting, 331
management, final report to, 403
management information systems (MIS), 12–13
managing for success
  agile mindset, 102
  issue resolution, 98–102
  project monitoring and control, 98
man-in-the-middle (MitM), 469
many-to-many (M:N) relationship, 252–253
market basket analysis, 273
Mars Science Laboratory landing system, 231, 232
Maxxum, 456
Mbps (megabits per second), 307, 432
MBSE. See model-based systems engineering (MBSE)
measurable benefits, 555
memos, 522
menu bars, 212
MERN (MongoDB, Express, React, and Node), 316
mesh network, 305
message-oriented middleware (MOM), 292
messages, 22, 182, 183–184, 192
methods, 22, 168, 182–183
metrics, 431
Microsoft, 201, 328
  Communities, 346, 347
  GitHub, 90
  service packs, 428
Microsoft 365, 336–337
Microsoft Access, 142
  easy-to-use tool, 222, 223
  input masks, 215
  report design tools, 221
  validation rule, 217
Microsoft Excel, 141
  absolute date, 276
  Goal Seek, 433, 434
Microsoft Office, 140
Microsoft OneNote, 126, 141
Microsoft Outlook, 140, 141
Microsoft PowerPoint, 142
Microsoft Project, 93, 96
Microsoft Teams, 535–537

Microsoft To Do, 95
Microsoft Visio, 10, 25, 142, 378
middle managers, 17
middleware, 244, 291–292
milestones, 67
MIMO. See multiple input/multiple output (MIMO)
MIS. See management information systems (MIS)
miscommunication, of requirements, 78
mission-critical system, 4
mission statement, 37
mitigation, 474
Mitnick, Kevin, 471
ML. See machine learning (ML)
M:N, 252–253
mnemonic codes, 268
mobile devices, 318
mock-up, 221
model-based systems engineering (MBSE), 9, 25, 138
modeling
  business operations, 9–10
  defined, 9
modular design, 168–169
modules, 207, 366
  structure charts, 370–371
monday.com, 89
monday work management tool, 93, 94
monitoring and control project, 90
Moore, Gordon, 5
Moore's Law, 5
multimedia, 398–399
multipath design, 307
multiple input/multiple output (MIMO), 307
multiple predecessor tasks, 72–74
multiple successor tasks, 72–74
multivalued key, 247

N

natural language, 110, 135–136, 208
near-field communication (NFC), 309
"need-to-know" approach, 523
.NET, 315, 316
net-centric computing, 290
netiquette, 523–524
net present value (NPV), 339, 340, 565
network, 464
  administration, 27
  devices, 305–306
  diagram, 66, 96–97
  evolution, 288
  infrastructure security, 479
  interface, 480
  topology, 301
network intrusion detection system (NIDS), 483, 487
network security, 480
  firewall, 480, 481
  network intrusion detection, 483
  network traffic encryption, 480–481
  ports and services, 481–483
  private networks, 484
  VPNs, 484
  wireless networks, 483–484
Newlight Technologies, 44
NFC. See near-field communication (NFC)
NIDS. See network intrusion detection system (NIDS)
NIST Cybersecurity Framework, 465, 466
node, 283
nondiscretionary projects, 51
nonfinancial benefits, 553–554
non-functional requirement, 109
non-key field, 247
nonmeasurable benefits, 555
nonverbal communication, 538
normalization, 218
  defined, 255
  first normal form, 257–258
  real-world examples, 262–267
  second normal form, 258–260
  stages, 255
  standard notation format, 256–257
  third normal form, 260–261
NoSQL database, 7, 77
NPV. See net present value (NPV)
n-tier designs, 290
nurturing growth, people management, 77

O

object model, 179
object modeling
  attributes, 182
  classes, 184–186
  messages, 183–184
  methods, 182–183
  object-oriented analysis, 179–180
  objects, 180–181
  relationships among objects and classes, 186–187
  tools, 194
  unified modeling language, 187–194
object-oriented development (OOD)
  characteristics of, 371–373
  cohesion and coupling, 373–374
  defined, 371
  implementation of, 372–373
  process description in, 168
object-oriented (O-O) analysis, 18, 19, 21–22, 179–180
objects, 21, 168, 179–181
observation, 127–128
ODBC. See open database connectivity (ODBC)
offshore outsourcing, 333, 334
offshoring, 333–334
off-the-shelf software, 320, 322, 323, 325, 326, 338
one-to-many (1:M) relationship, 251, 252
one-to-one (1:1) relationship, 251
online coding resources, 381–385
online communications
  Microsoft Teams, 535–537
  Slack, 533–535
  Zoom, 531–533
online data entry, 225
online diagramming tools, 10
online documentation, 390–391
online processing, 298–299
online support portals, 412
online system, 298–299
online training, 400
onshoring, 334
O-O analysis. See object-oriented (O-O) analysis
OOD. See object-oriented development (OOD)
OpenAI, 15, 16
OpenAI Codex, 379
open database connectivity (ODBC), 244, 395
open-ended questions, 123
open-source software, 321
Open Web Application Security Project (OWASP), 485
operational benefits, 554
operational costs, 414, 415, 548
operational employees, 18
operational environment, 391–392
operational feasibility, 48–49, 58
operational security, 493
operations center security, 477
operations documentation, 389
optimization, of marketing campaigns, 273–274
option button, 213
oral communications, 526
  audience, 526
  nervousness, 530
  objectives, 526–527
  practice, 529
  presentation, 527–530
order system
  context diagram for, 156–157
  diagram 0 DFD for, 159–160
organizational culture, in cybersecurity, 507–508
organizational information models, 16–18
organizational security, 493–497
orphan, 249
OSI (Open Systems Interconnection) model, 301
output control, 229–231
output security, 229–231
output technology, 226–228
outsourcing, 329–330
  fees, 331–332
  growth of, 330–331
  implications of, 333
  issues and concerns, 332–333
  management, 413–414
  options, 346–347

P

packaged solution, for Internet commerce, 296–297
page footer, 222, 223
page header, 222, 223
pair programming, 375
Panopto, 399
parallel operation system changeover, 392, 394
parent, 186
parent diagram, 157
Pareto chart, 56
partitioning, 160
password cracking, 470
password protection, 489–490
patches, 388
patch management, 441, 442
payback analysis, 339, 340, 557–561
payback period, 557
payroll system, 6, 7
PCI DSS, 465
penetration testing, 502
people, 7
  issues, in systems project, 100
  management, 76–77
perfective maintenance, 414–416, 418
performance and workload measurement, 431–433
performance testing, 437–438
permissions, 277, 488
personal computers, 287–288
personal digital assistant, 15
personal information manager (PIM), 140
person-day, 68
PERT. See Program Evaluation Review Technique (PERT)
PERT/CPM charts, 64, 66–67, 93
PESTEL framework, 43
phased operation system changeover, 392, 394–395
phishing attacks, 468, 470
physical data repository, 244
physical model, 148
physical security, 476–477
  computing devices security, 477–478
  IoT security, 479
  network infrastructure security, 479
  operations center security, 477
  storage device security, 479
physical storage, 274
physical topology, 301
pilot operation system changeover, 392, 394
pilot site, 394
PIM. See personal information manager (PIM)
plaintext, 480

planning and scheduling projects, 89
planning game, 376
platform, 283
podcast, 227, 397
point-of-sale (POS), 300–301, 320
polymorphism, 183
pool, 137
portal, 285
ports, 482
port scans, 482
PortSwigger's Burp Scanner, 443
Poshmark, 9, 10
positive benefits, 555–556
post-implementation evaluation, 401–403
potential losses, 84
potential vendors, 346–347
power-on password, 477
PPM. See project portfolio management (PPM)
predecessor tasks, 68
predictive analytics, 273
preliminary investigation, 20, 52
  planning, 52
  sample task summary for, 96
  steps, 52–58
  summarizing, 59
present value analysis, 562–567
pretexting, 490
preventive maintenance, 414–418
primary key, 246–248
printed output design, 220–224
priorities
  dynamic, 51
  factors affect, 51
  maintenance, 422–425
  setting, 50–51
private key encryption, 481
private networks, 484
privilege escalation attack, 470, 488
probable-case estimate, 69
procedural security, 493–497
process, 7
  documenting, 166–167
  improvement, 356
  symbols, 149–151
process 0, 156
process description
  decision table, 170–174
  decision trees, 174
  defined, 168
  modular design, 168–169
  in object-oriented development, 168
  structured English, 169
product baseline, 429
production environment, 391
productivity-optimized interface, 209
productivity software, 140
product lifecycle management (PLM), 24
product-oriented firms, 32
program documentation, 388
Program Evaluation Review Technique (PERT), 64, 66–67. See also PERT/CPM charts
programmer/analyst, 420–421
programming languages, 377–378
project coordinator, 65
project creep, 54, 99
project delays, 102
project leader, 65
project management, 18, 64–65
project management software, 88–89, 365
  collaboration, 91
  document management, 91
  issue and bug tracking, 91–92
  PPM, 92–94
  resource management, 93, 95
  supporting areas, 89–90
  task management, 93, 95
  time tracking, 95–96
  usage, 96–97
project manager, 65
project monitoring, 65, 98
project planning, 65, 84
project portfolio management (PPM), 92–94
project reporting, 65
project scheduling, 65
project scope, 54
project status meetings, 79
project status reports, 79
project triangle, 64–65
project updates, 78
project usability analysis, 58
properties, 22
prototyping, 24, 231–232
proxy server, 306
pseudocode, 169, 364
pseudonymization, 503
public key encryption (PKE), 481
PyCharm, 380, 381

Q

QA. See quality assurance (QA)
qualitative risk analysis, 85–86
quality assurance (QA), 27
  continuous improvement, 436
  defined, 355
  feedback and improvement, 438–440
  International Organization for Standardization, 358–360
  in maintenance, 434–435
  in outsourced services, 414
  software engineering, 355–357
  systems engineering, 356, 358
  in system support, 433–440
  testing practices, 436–438
  in user support, 435–436
quality attributes, 109
quantitative risk analysis, 86
query by example (QBE), 243
query language, 243
questionnaire, 128–129
  interviews vs., 130
quick response codes (QR codes), 43

R

Rackspace, 331
RAD. See rapid application development (RAD)
radio button, 213
radio frequency identification (RFID), 13
  tags, 43, 45, 225
RAID systems. See redundant array of independent disks (RAID) systems
random sample, 130
range check, 218
range-of-response questions, 123
ransomware attacks, 470
  enhancing resilience against, 499
rapid application development (RAD)
  applications, 115–116
  benefits, 117–118
  defined, 112, 115
  objectives, 117
  phases and activities, 116–117
readability, 520–521
reasonableness check, 218
records, 164, 246
  documentation, 167
records retention policy, 229
recovery, 444
recovery point objectives (RPOs), 445
recovery procedures, 277
recovery time objectives (RTOs), 445
redundant array of independent disks (RAID) systems, 497
referential integrity, 249–250
regression testing, 436–437
related information systems, DBMS, 243
relational database, 241
relational model, 241
relationships, 186
  among objects and classes, 186–187
  ERD, 251
  structure charts, 370–371
release plan, 376
remote control software, 411
repeating group, 256
report
  Adobe PDF, 524
  footer, 221, 222
  format, 525
  header, 221, 222
  written, advantages of, 526
report design, 221
  principles, 221–223
  types of, 223–224
Report Wizard, 221
request for proposal (RFP), 343–344
request for quotation (RFQ), 343, 344
request intake process, 422
requirements
  agreement, 110
  definitions, 109
  feature creep, 110
  functional, 109
  imprecision, 110
  modeling, 21
  non-functional, 109
  planning phase, 117
  representing, 135–139
  specifications, 109
  types of, 109–110
requirements elicitation, 120–121
  in agile projects, 133–135
requirements engineering
  defined, 109
  elicitation techniques, 126–132
  interview, 121–126
  representing requirements, 135–139
  requirements elicitation, 120–121, 133–135
  system requirements, 109–112
  team-based techniques, 112–120
  tools, 140–143
  validation and verification, 140
research, 131–132
reshoring, 334
resistance to change, 100
Resource Guru, 93, 95
resource management, 89, 93, 95
Resource Monitor, 431
response time, 431–432
responsive web design, 231
retention periods, 498
retirement. See system retirement and transition
retirement planning process, 453
return on investment (ROI), 83, 100, 339, 340, 549, 561–562
reuse-oriented development strategies, 321
RFID. See radio frequency identification (RFID)
RFP. See request for proposal (RFP)
RFQ. See request for quotation (RFQ)
Rigi research environment, 388
ring network, 304
risk
  defined, 83
  origins of, 84–85
risk analysis, 85
  qualitative, 85–86
  quantitative, 86

risk assessment, 472, 474
risk communication, 87–88
risk control, 472, 474
risk identification, 85, 472, 473
risk management, 90
  defined, 83
  importance of, 83–84
  issues, 88
  plan, 87–88
  in system security, 472–475
  tasks, 85–87
risk owner, 86–87
risk response plan, 86
Roadmunk, 39
roaming, 307
ROI. See return on investment (ROI)
role-based access control (RBAC) systems, 494
rootkit, 470
router, 305–306
RPOs. See recovery point objectives (RPOs)
RTOs. See recovery time objectives (RTOs)

S

SaaS. See Software as a Service (SaaS)
salary, 30
sample user evaluation form, 401, 402
sampling, 130–131
scalability, 81, 110, 111, 241, 283
scalable design, 21
scaling on demand, 294
scatter diagram, 57
scenarios, 133–135
schedule feasibility, 50, 58
schedule issues, in systems project, 101–102
schema, 244
SCM. See supply chain management (SCM)
screen generators, 205
scroll bar, 213
Scrum, 23, 118, 119
SDLC. See systems development life cycle (SDLC)
secondary key, 248, 249
second normal form (2NF), 258–260, 263–265
secure coding, 485
security, 110, 111, 413
  auditing and compliance, 494–495
  audits, 442–444
  by design approach, 510
  holes, 485
  policies, 464, 494
  tokens, 491, 492
  updates, 440–442
security-first mindset, 508–509
security information and event management (SIEM) systems, 493, 495–496
selection, 168
self-service support systems, 451
semantic web, 295
sentiment analysis, 273
sequence, 168
sequence check, 217
sequence codes, 268
sequence diagram, 139, 191–192
server, 287
service level agreements (SLAs), 413
service-oriented architecture (SOA), 289
service-oriented companies, 32
services, 482
  desk, 329
  packs, 428
  providers, 331
Sherwin-Williams, 9
Shneiderman, Ben, 206
short term benefits, 554–555
SIEM systems. See security information and event management (SIEM) systems
significant digit codes, 268, 269
simulation, 400
sink, 153
site visit, 131
skill development, maintenance team, 422
skill gaps, 100
skills and technology investment, 82–83
Slack, 14, 15, 533–535
slack time, 76
SmartDraw, 18, 20
Smartsheet, 93
sniffing, 470
SOA. See service-oriented architecture (SOA)
SOC 2, 495
social engineering, 470, 471–472
  countermeasures, 490–491
social media, 522–523
Society for Human Resource Management, 17
soft communication skills
  emotional intelligence, 538
  facilitation skills, 537–538
  feedback skills, 538
  negotiation and persuasion, 537
  nonverbal communication, 538
software, 5–6
  license, 349
  logs, 487
  reengineering, 418
  vendors, 320
software acquisition
  alternatives, 348–349
  cost-benefit analysis, 349–350
  defined, 342
  potential vendors/outsourcing providers identification, 346–347
  recommendation report, 350
  system requirements analysis, 342–346
Software Acquisition Capability Maturity Model (SA-CMM), 342
software as a service (SaaS), 294, 329, 331, 332, 334–335
  advantages, 335–336
  Microsoft 365, 336–337
  shortcomings, 336
software engineering, 355–357
Software Engineering Institute (SEI), 356, 357
software in-house development, 320–321
  business procedures and policies, changes in, 322
  business requirements, 321–322
  internal resources and capabilities, 323–324
  systems constraints, 322–323
  technology constraints, 323
software package, 320
  customizing, 327–328
  purchasing, 324–327
software security. See application and software security
SolarWinds Patch Manager, 441, 442
Sort Inventory Parts, 367
source, 153
source data automation, 225
source document, 220
spam, 470
spiral model, 23
spontaneous generation, 150
spoofing, 470
SQL (Structured Query Language), 243
  injection, 470
stakeholder communication, 425, 453
stakeholders, 7, 84
stand-alone PCs, 288
standardization, in designing codes, 270–271
standard notation format, 256–257
star network, 304–305
start day/date, 72
state, 182
state-of-the-art knowledge, 28
state transition diagrams, 193
status flag, 366
storage device security, 479
storyboards, 133, 134, 204
strategic benefits, 554
strategic plan, 42
strategic planning, 16–17
  defined, 37
  IT department, role of, 38–39
  overview, 37
  SWOT analysis, 37–38
  tools, 39
stratified sample, 130
structure charts
  condition, 367
  control couples, 366, 367
  data couples, 366
  defined, 365
  drawing, 369–371
  loops, 367
  modules, 366
structured analysis, 18–21
structured brainstorming, 130
structured development
  cohesion, 368
  coupling, 368, 369
  structure charts, 365–367, 369–371
  top-down approach, 365
structured English, 169
structured walk-through, 98, 383
stub testing, 384
subclasses, 185
subordinate modules, 366
subschema, 244
subscription model, 332
successor task, 72
summary report, 224
Sun Microsystems, 290
superclass, 185, 186
super key, 247
superuser privileges, 486
supervisors, 17–18
suppliers, 44
supply chain, 9, 470
supply chain management (SCM), 9, 283
support costs, for hardware and software, 341
support services, 410
  community forums, 412–413
  help desks, 410–412
  online support portals, 412
surrogate key, 247
survey, 128–129
swim lanes, 137
switch, 304
switchboard, 212
SWOT analysis, 37–38, 85
syntax errors, 383
SysML, 138
system administrator, 419–420
system architecture
  client/server architecture, 289–293
  cloud computing, 293–294
  compliance, 285
  corporate organization and culture, 282
  corporate portals, 285–286
  defined, 282
  e-commerce architecture, 295–298
  enterprise resource planning, 282–283
  Internet-based architecture, 293
  legacy systems, 284
  mainframe architecture, 287
  network devices, 305–306
  network evolution, 288
  network models, 301–305
  personal computers, 287–288
  processing methods, 298–301
  processing options, 284
  scalability, 283

security, 284–285
Web 2.0, 295
web integration, 283–284
wireless networks, 306–309
systematic sampling, 130
system boundary, 190
system changeover
defined, 392
direct cutover, 393–394
methods, 393
parallel operation, 394
phased operation, 394–395
pilot operation, 394
system, defined, 4
system deployment schedule, 403
system design specification, 21
system documentation, 388–389
system performance management, 430
capacity planning, 433–435
fault management, 430–431
performance and workload measurement, 431–433
system prototyping, 232–233
system requirement, 109–112
system requirements document, 21
system retirement and transition
archiving and documentation, 458–460
data migration, 453–455
decommissioning procedures, 455–457
planning for retirement, 452–453
transition to new systems, 457–458
systems analysis phase, 3–4, 21
systems analyst, 4, 420
career opportunities, 30–31
certification, 29–30
knowledge, skills, and education, 28–29
role, 27–28, 338–339
trends in information technology, 31–32
systems constraints, 322–323
systems design phase, 3–4, 21
systems development, 18
agile methods, 18, 19, 22–24
object-oriented analysis, 18, 19, 21–22
prototyping, 24
structured analysis, 18–21
tools, 24–25
systems development life cycle (SDLC), 18–21
systems development tools, 362–363
decision tables and decision trees, 364–365
ERDs, 363
flowchart, 363
pseudocode, 364
system security, 284–285, 464
application and software security, 484–487
APTs, 471
architecture, 476
artificial intelligence, in cybersecurity, 506
attacker profiles and motivations, 467–468
attacks, preventive measures against, 471
backup and recovery strategies, 497–500
blockchain technology, 505–506
cloud computing, 504–505
concepts, 464
cyberattacks, 468–471
cybersecurity, 506–512
data and file security, 487–488
frameworks and standards, 465–466
implications, 506
IoT devices, 505
legal and ethical considerations in, 500–504
legal implications and reputational damage, 466–467
network security, 480–484
physical security, 476–479
policies and procedures, 465
procedural and organizational security, 493–497
risk management in, 472–475
social engineering and insider threats, 471–472
threats and vulnerabilities, identifying, 464–465
user security, 488–493
systems engineering, 356, 358
systems implementation phase, 21
agile development, 374–376
application development, 360–365
coding, 377–385
documentation, 388–391
installation, 391–404
object-oriented development, 371–374
quality assurance, 355–360
structured development, 365–371
testing, 382–384, 386–387
systems integration, 15–16
system software, 5–6
systems operations phase, 21
systems planning phase, 20
systems programmer, 420
systems projects
external factors, 42–45
internal factors, 42–43
systems requests, 20, 40–41
forms, 45, 46
tools, 46
systems review committee, 45–47
systems support and security, 26–27
system testing, 386–387

T

table, 6, 246
table design, 255
tactics, techniques, and procedures (TTPs), 471
tamper-evident cases, 479
tangible benefits, 49, 552–553
tangible costs, 49, 544–545
task
box, 71–72
defined, 67
estimating duration, 68–70
group, 66
ID, 71
listing, 68
name, 71
overlapping, 102
in WBS, 68–70
task duration, 71–72
estimating, 68–70
underestimation of, 102
task management, 65, 93, 95
critical path, 74–76
task pattern, 71–74
work breakdown structure, 65–71
task patterns, 65
defined, 71
task boxes, 71–72
types, 72–73
working with complex, 73–74
TCO. See total cost of ownership (TCO)
TDD. See test-driven development (TDD)
team-based techniques
agile methods, 118–120
joint application development, 112–115
rapid application development, 115–118
team dynamics, 84
and collaboration, 76–77
team leaders, 17–18
team structure, 421
technical documentation, 448
technical feasibility, 50, 58
technical knowledge, 28
technical support, 25
technological change, 101
technological compatibility, 101
technological complexities, 84
technology, 43–44
constraints, 323
issues, in systems project, 100–101
technology management, 80–81
implementation phase, 81
review of, 81–82
selection, 81
terminators, 153
test data, 384
test-driven development (TDD), 375
test environment, 391–392
testing, 382–384
integration testing, 386
maintenance, 436
performance, 437–438
regression, 436–437
reviews, 98
system testing, 386–387
unit testing, 384, 386
test plan, 384
TestRail, 436, 437
thick client, 290
thin client, 290
third normal form (3NF), 255, 260–261, 264, 266, 267
third-party risk management (TPRM), 347
third-party software, 487
threats, 464–465, 472, 473
resilience and response to, 493
three-tier design, 290
throughput, 432
throwaway prototyping, 232
tightly coupled modules, 368, 369
time tracking, 95–96
Toastmasters, 539, 541
toggle button, 213
toolbar, 213
tool integration, 90
tools, 24–25
top-down approach, 365
top management, 42
top managers, 16–17
total cost of ownership (TCO), 49, 110–112, 290, 319, 324, 325, 339–340, 349, 543
totals zone, 220
TPMs. See transaction-processing monitors (TPMs)
TPRM. See third-party risk management (TPRM)
traceability, 142
trade-offs, 226, 233–234
traditional development, 315
training
interactive, 400
online, 400
outside resources, 398
plan, 396, 397
podcasts, 397
for power users, 410
programs, 409–410
tips, 398–400
tutorials, 397
vendor, 396–397
webinars, 397
train-the-trainer strategy, 400
transaction model, 332
transaction-processing monitors (TPMs), 292
transaction processing (TP) systems, 12, 16
transference, 474
transition planning, 457–458
transparent interface, 201
transparent network, 288
Trello, 93, 94
trust, in people management, 77
Tufte, Edward, 148
tunnel, 484
tuple, 246
TurboTax, 212
turnaround documents, 221

turnaround time, 432–433
tutorials, 397
two-tier design, 290

U

Udemy, 539
UML. See Unified Modeling Language (UML)
unencrypted form, 480
unexpected expenses, 101
Unicode, 275
Unified Modeling Language (UML), 138, 187
  activity diagram, 193–194
  business process modeling, 194
  class diagram, 190–191
  sequence diagram, 191–192
  state transition diagrams, 193
  use case diagram, 189–190
  use case modeling, 187–188
uninterruptible power supply (UPS), 477
unique key, 247
unit testing, 384, 386
universal security slot (USS), 478
unnormalized design, 256
unstructured brainstorming, 130
usability, 199
  focus on, 204–205
  metrics, 204
usage model, 332
use case, 187–188
use case description, 187–188
use case diagram, 138, 139, 189–190
user, 7
  application, 328–329
  DBMS, 243
  design phase, 117
  documentation, 389–390
  of existing software packages, 348
  feedback, collecting, 439
  groups, 488
  requests, 42–43
  resistance, 492
  rights, 486
  security, 488–493
  stories, 133, 134, 375–376
  support and training, 27, 409–414
  training package, 409
user-centered system, 200, 203–204
user interface (UI), 199–200
user interface design
  attractive layout and design, creating, 210–211
  business functions, 202
  control and security issues, 228–231
  creating interface, 207
  data entry, 214–216
  data management, 218
  documentation, 205–206
  emerging trends, 231–234
  enhancing interface, 211–214
  feedback, 205, 209–210
  flexibility, 209
  guidelines for, 206–219
  help system, 209–210
  human-computer interaction, 200–202
  input volume, reducing, 219
  maximizing graphical effectiveness, 203
  models and prototypes, 204
  printed output, 220–224
  source document and form design, 219–220
  technology issues, 224–228
  usability, focus on, 204–205
  user-centric mindset, 203–204
  user interface, 199–200
  user productivity, 208–209
  validation rules, 216–218
user involvement, 222
user productivity systems, 14–15, 208–209
user-selected help, 210
UserVoice, 439

V

validation and verification (V&V), 140
validation process, 455
validation rules, 216–218
validity check, 218
validity rules, 165
value-added reseller (VAR), 320
value-added services, 32
variable costs, 547
vendor management, 414
vendor risk assessments (VRAs), 347
vendor services, 348
vendor support, 324
vendor training, 396–397
version control, 428–429
vertical application, 320
vertical system, 6
virtual private networks (VPNs), 484
virtual reality (VR), 201
visual aids, 527
The Visual Display of Quantitative Information (Tufte), 148
Visual Studio Code (VS Code), 378–379
voice control, 207
voice-controlled systems, 228
voice user interfaces (VUIs), 201
Volere shell, 136
VRAs. See vendor risk assessments (VRAs)
VRBO, 8
vulnerabilities, 464–465, 472
  scanning, 443–444

W

Wall Street Journal's Technology, 131, 132
Walmart, 12
WAN. See wide area network (WAN)
WAP. See wireless access point (WAP)
waterfall model, 19, 20
WBS. See work breakdown structure (WBS)
wearable computing, 234
Web 2.0, 295, 319
Web 3.0, 295, 319
web-based design, 244–246
web-based systems development, 315–316
webcast, 226
web-centric architecture, 283
webinars, 397
web integration, 283–284
web support, 27
weight, 69
Western Electric Company, 127
what-if analysis, 433
wide area network (WAN), 288, 301
Wi-Fi Alliance, 307, 484
Wi-Fi Protected Access (WPA), 484
Wi-Fi (wireless fidelity), 307
wiki, 295
Wired Equivalent Privacy (WEP), 484
wireless access point (WAP), 307
wireless devices, 227
wireless local area network (WLAN)
  defined, 306
  standards, 306–307
  topologies, 307
  trends, 307–309
wireless networks, 483–484
WLAN. See wireless local area network (WLAN)
WolframAlpha, 13–14
work breakdown structure (WBS), 65, 96
  displaying, 70–71
  Gantt charts, 65–66
  identifying tasks in, 67–69
  PERT/CPM charts, 66–67
  task duration, factors affecting, 70
work from home (WFH), 330
worst-case estimate, 69
WPA2, 484
WPA3, 484
Writer's Digest, 539, 540
written communication
  email, memos, and letters, 522
  netiquette, 523–524
  readability, 520–521
  reports, 524–526
  social media, 522–523
  writing skills, 519
  writing style, 520, 521

X

Xcode, 380
XP. See extreme programming (XP)
XY chart, 57

Y

Y2K issue, 275
Yourdon symbols, 149

Z

zero-day exploits, 470, 471
zip code, 267
ZipRecruiter, 4
Zoho Projects, 89
Zoom, 531–533